*OS Internals

The truth is in there

Volume II

Kernel Mode

By Jonathan Levin

*OS Internals

(Mac OS X & iOS Internals, 2nd Edition)

Volume II: Kernel & Hardware

By Jonathan Levin

North Castle, NY

*OS Internals - Volume II - Kernel & Hardware

© 2019 by Jonathan Levin.
All rights reserved. No part of this work may be reproduced, transmitted in any form or any means, electronic or mechanical, including photocopying, recording, or by any information storage or retrieval system, without the prior written permission of the author.

Printed in Edison, New Jersey, USA

First Printing, 1.0.0 (10/16/2019)
ISBN-10: 0991055578
ISBN-13: 978-0-9910555-7-9

Publisher: Technologeeks.com
Production Editor: Jonathan Levin
Interior Design: Jonathan Levin
Cover Illustration: Jon "Hoops" Hooper

For information on distribution, translations, or bulk sales, please contact Jonathan Levin, at:

Jonathan Levin
Phone: (617)-3000-667
Email: inquiries@NewOSXBook.com
Web: www.NewOSXBook.com

Company and product names mentioned in this work may be trademarked by their respective owners. Any and every such name is used solely in an editorial fashion, to the benefit of the trademark owner, with no intention whatsoever of infringement.

Limit of Liability/Disclaimer of Warranty: The author neither makes or implies any representations or warranties with respect to the accuracy or completeness of the contents of this work. Further, this paragraph explicitly and specifically serves to disclaim all warranties, including without limitation warranties of fitness for a particular purpose. No warranty may be created or extended by sales or promotional materials. The advice and strategies contained herein may not be suitable for every situation. This work is sold with the understanding that the author is not engaged in rendering legal, accounting, or other professional services. If professional assistance is required, the services of a competent professional person should be sought. The author shall not be liable for damages arising herefrom. The fact that an organization or Web site is referred to in this work as a citation and/or a potential source of further information does not mean that the author endorses the information the organization or Web site may provide or recommendations it may make. Further, readers should be aware that Internet Web sites listed in this work may have changed or disappeared between when this work was written and when it is read.

Information in this book is distributed on an "As Is" basis, with no warranty. MacOS and the iOS variants are both evolving systems, with versions being released faster than any work can keep up with. Every effort and precaution has been taken to update this work to reflect up to and including MacOS Mojave (10.14) and iOS Peace (12) and the final betas of Catalina (10.15) and Yukon (13). Bear in mind, however, that different versions of MacOS, iOS or its variants may add, modify, or remove APIs and features with no notice. The author hereby disclaims any liability to any and every person or entity with respect to any loss or damage, whether caused or alleged to be caused directly or indirectly, by this work.

To report errors or omissions, please contact the author. You are also welcome to submit feedback and comments through http://NewOSXBook.com/forum.

About the [Author/Editor/Formatter/Publisher]

I started in my teens by hacking (mostly in the good sense of the word) and trying to figure out how things worked back in 1993, from an XT with a 2400 baud modem onto a shell I'm not even sure was mine. With no help per se, I had to `man man` and then `man *`..

A lot has happened in the two decades since, and from UNIX to Linux, I got to Windows and OS X. Over the years, I took up consulting and training - initially, in security. I then realized security is largely a projection of internals. Finally, I gathered a few good men and started Technologeeks.com - which is where I presently pass my time and function as CTO.

I'm still adjusting to authoring: I first took on Apple's OSes with "Mac OS X and iOS Internals" *(Wiley, 2012)*, which was well received. It was a painful process, but the dopamine rush was awesome. Now an addict, I took on Android, and that's when I also dared self-publishing for the first time, at the advice of my good friend Ronnie Federbush. That worked well - and gave me the chance to rewrite MOXiI the way I should have at first - free from publisher whims, free from censorship, and with no regard to page count or budget. Hence, you hold in your hand the second (but, technically, the final) volume of the new "*OS Internals" - a trilogy.

Contents at a glance

1. Welcome to the Machine: Hardware
2. Use the source, Luke: The XNU Codebase
3. E X T E N D: Kernel Extensions
4. Some Assembly Required: Kernel Architectures
5. Alone in the Dark: Boot, Panic, and Shutdown
6. BS"D - The BSD Layer
7. Fee, FI-FO, File: File Systems and the VFS
8. Space Oddity: APFS
9. Tempus Fugit: Mach Scheduling
10. Mixed Messages: Mach IPC
11. Mapped out: Mach Virtual Memory
12. Ceci n'est pas une "heap": Kernel Memory Management
13. All in the Family: I/O Kit
14. Stack up: Kernel-mode networking

Acknowledgments

MOXiI would never have been possible without the efforts of jailbreakers, which toil voluntarily and without compensation yet with incredulous efforts - all with the sole purpose of unrestricting Apple's amazing operating systems so that they mete out their full potential. I have had the honor of getting to know quite a few of them in person, and am deeply thankful to them all, including in particular:

- **@windknown** - Though unknown to most, surely all would recognize his group - Pangu! Hao and I forged a great friendship ever since we met for dinner in my favorite sushi joint in Sanlitun (Hatsune). He was also my first "surprise guest" in TechnoloGeeks' training, coming to NY to explain the inner workings of Pangu 9 for the very first time. Hao also helped me experiment - as with the other volumes - on a fully jailbroken iOS (12.1) for this book as well. 非常感谢, 我的朋友!
- **@key2fr** - a.k.a Ramtin Amin - known to the world as "the guy who made that lightning to serial cable". I had the honor of getting the very first cable he made! I later met him in Paris, and now have another reason to re-visit my favorite city every time. And that cable of his? That's just the tip of the iceberg. He has since demonstrated PCIe and NVMe wizardry which enabled, for the first time, the dumping of the SecureROM. What Ramtin can do to an iDevice over custom boards, soldering and JTAG is best left unspoken. Everything I know about the iDevice hardware, Ramtin taught me.
- **@_bazad** - a rising star in the world of *OS, this young researcher's first claim to fame was his "blanket" exploit, which demonstrated superior mastery of Mach ports providing a user-space jailbreak for iOS 11.3. I was truly honored when he presented this fantastic exploit in a world premiere as a mystery guest in @Technologeeks' training. This, however, was only a prelude to the "voucher_swap" exploit (now as a member of Google's Project Zero), which provided the core of the full iOS 12.1.2 jailbreak. Brandon didn't stop there - he also demonstrated an A12 PAC bypass, and provided important open source contributions, most notably `memctl`. Bran the Breaker reviewed the most difficult chapters of this work - those dealing with low level assembly, and the Mach primitives - and for that I am most grateful.
- **@s1guza** - already deserves my gratitude for being an admin of the NewOSXBook.com forum, and for taking my suggestions for his first public jailbreak, Phœnix. Since then, he has continued to break iOS time and time again, while producing unprecedented detail on Apple's most obscure protection mechanisms, notably APRR. This paragon hacker meticulously reviewed several chapters of this work.
- **@iH8sn0w** - a legendary jailbreaker who has made history on more than one occasion, thanks to his unparalleled mastery of iBoot. 'sn0w was willing to help me with helpful insights on all things iBoot and hardware.
- **@Xerub** - An anonymous but prolific hacker, who picked up where Sn0w left, crushing the last bits of iBoot and extending into SEP. Whatever details I have managed to glean of the latter would not have been remotely possible if not for his sharing of the SEP encrypted firmware keys. Xerub has also contributed to opensourcing jailbreaks (maybe overly so ;-), and will always be known as the one who brought all the gang together in 0x41 con.

Table of Contents

0. About this Book
1. Welcome to the Machine: Hardware 1
    - Devices
        - Mac Devices
        - iDevices
    - Ports
        - Serial
        - FireWire
        - ThunderBolt
        - USB
            - USB Restricted Mode
        - iDevice Connectors
            - 30-pin
            - Lightning
    - NVRAM
    - Configuration Data
    - The Device Tree
    - Dedicated Processors
        - Common Code: RTKit
        - The *OS Side: RTBuddy
        - The Always On Processor (AOP)
        - AGX
        - The Apple Video Encoder (AVE)
        - The Apple Neural Engine (ANE)
        - PMP
        - The Image Signal Processor (ISP)
    - External Processors
        - The Secure Element
        - The Baseband
        - The WiFi Chipset
        - BlueTooth
    - The Secure Enclave Processor (SEP)
        - Overall Architecture
        - SEPOS
        - SEPOS/*OS Communication Channels
        - User Mode Interface

2. Use the source, Luke: The XNU Codebase 41
- The XNU Sources
- Compiling the Kernel
- Debugging
- Don't Panic

3. E X T E N D: Kexts 61
- What's in a kext
  - The Info.plist
  - The Kext Mach-O
- The Kernel Programming Interface (KPI)
  - The Official KPIs
  - Unsupported
  - MacFramework
  - Private
- The KernelCache
  - Kernelcache structure
  - `__PRELINK_INFO.__info`
  - "1469" kernelcaches
- Kext Loading: The user mode persepctive
  - Kext Security Requirements
  - Kext Code Signing
  - BridgeOS kext_audit (MacOS 14, T2)
  - kextd(8) (MacOS)
  - OSKext* APIs
  - Multikexts
- Kext Loading: The kernel perspective
  - `OSKext::load()`
  - kxld
  - `OSKext::start()`
  - `OSKext::stop()`
  - Unloading a kext
- Kext metadata management
  - The `kmod_info`
  - `gLoadedKextSummaries`
  - `sLoadedKexts`, `sKextsById` and the `OSKext` class
  - Kext Accounting
- MacOS 15: System Extensions and the DriverKit.framework

4. Some Assembly Required: Kernel Architectures 91
    - Data structures
        - Linked lists & Queues (Mach)
        - Linked lists & Queues (BSD)
    - Concurrent Resource Access
        - Atmoic Operations
        - hwlocks
        - Spinlocks
        - Read-Write Locks
        - Mutex Locks
        - Lock Groups
        - Per-CPU data
    - Processor Execution Modes
        - Mode Traversal
        - Intel: SYSENTER
        - Intel: IDT
        - ARM exception vectors
    - Processor Level Exception Handling
    - Interrupt Handling
    - System Call Personalities
    - User Mode Considerations

5. Alone in the Dark: Boot, Panic, and Shutdown 127
    - Macs: EFI
    - iDevices: iBoot
    - T2 and later enabled Macs: iBoot + EFI
    - XNU boot flow
    - Kernel threads
    - Kernel Shutdown

6. BS"D: The BSD Layer 163
    - Startup
    - Processes
    - (U)Threads
    - Process Lifecycle
    - File Descriptors
    - The `struct uio`
    - `sysctl`
    - BSD MALLOC Zones
    - DTrace

7. Fee, FI-FO, File: File Systems and the VFS 209
- VFS Concepts
  - Filesystems
  - Mounts
  - Vnodes
    - The `ubc_info` (`V_REG` vnodes)
- Apple Extensions
  - Resource Forks
  - File Compression
  - Restricted (MacOS)
  - DataVault
  - Data Protection
  - FSEvents
  - Document IDs
  - Object IDs
  - Dist Mount Conditioning
  - Triggers (MacOS)
  - VFS Namespaces
- VFS KPIs
- VFS SPIs
- Case Study: devfs
  - Character & Block Devices
- Case Study: FUSE
- NFS

8. Space Oddity: APFS 245
- A Bird's Eye View
  - File System Features
- File System Internals
  - APFS Objects
  - APFS Object Structure
  - B-Trees
- Containers
- Volumes
- The Space Manager
- apfs.kext

9. Tempus Fugit: Mach Scheduling 269
- Mach Tasks
- Mach Threads
- Mach Scheduling Enhancements
  - Handoff
  - Continuations
  - Asynchronous Software Traps (ASTs)
- Mach Schedulers
  - The Run queue
  - Priorities
  - Load Average/Mach Factor
  - Scheduler Dispatch
  - Multicore Considerations
  - Darwin 17 additions
- Deferred Calls
  - Timer calls
  - Thread calls
- Wait Queues
- Turnstiles (Darwin 18)
- Ledgers
- Selective Forced Idle (SFI)

10. Mixed Messages: Mach IPC 311
- The High Level View
- The `ipc_space_t`
- The `ipc_port_t`
  - Port Lifecycle
    - Port Allocation
    - Rights and Names
    - Port Deallocation
    - Reference Counting
    - Handling Messages
- Revisiting `mach_msg`
  - Sending Mach Messages
  - Receiving Mach Messages
- Message Descriptors
  - Port Right Descriptors
  - Port Set Descriptors
  - OOL Memory Descriptors
- Vouchers
  - High Level View
  - API
  - Implementation
- Multinode

11. Mapped out: Mach Virtual Memory 339
- The Bird's Eye View
- The vm_map layer
  - The struct _vm_map
  - vm_objects
  - vm_pages
  - vm_map_enter and friends
  - Allocating memory
  - vm_map_copy
    - VM_MAP_COPY_ENTRY_LIST
    - VM_MAP_COPY_OBJECT
    - VM_MAP_COPY_KERNEL_BUFFER
  - Copying/Moving memory
- The pmap Layer
  - Page Tables
    - WIMG
    - Intel
    - ARM
    - ARM Page Protection Layer (iOS 12+, A12+)
- Pagers
  - The Apple protect pager
  - The compressor pager
  - The fourk pager (Darwin 18, ARM64)
  - The shared region pager (Darwin 18)
  - The Apple Protect Pager
- Universal Page Lists
  - Creating UPLs
  - Handling UPLs
- The PMAP Layer
  - Page Tables
  - WIMG
  - I/O Mappings
  - Intel PTEs
  - ARM PTEs

12. Ceci n'est pas une "heap": Kernel Memory Management 373
- Kernel Memory Allocation
  - kmem_alloc() and friends
    - kernel_memory_allocate()
    - kmem_suballoc()
    - kmem_realloc()
  - kalloc
    - kalloc.### Zones
    - The kalloc DLUT
    - The slow path
    - vm_allocation_sites

- The Zone Allocator
  - Zone Management
  - Zone Metadata Region
  - Zone Metadata
  - Garbage Collection
  - GC and UAF
  - Battling zone corruption
  - The Guard Mode Zone Allocator (MacOS)
  - The Zone Cache (Darwin 18+)
- Memorystatus (MacOS)/Jetsam (*OS)
  - Purgeable Memory
- Kernel Memory Layout
  - `kernel_map` regions
  - The kernel slide

13. All in the Family: I/O Kit 393
- A High Level View of IOKit
  - The IOKit.framework
  - IOKit error codes
- The IORegistry
  - User Mode APIs
  - Iterators
- The IOCatalog(ue)
  - Matching Dictionaries
  - Notifications
- Interlude: Libkern Base Classes
  - OSObject
  - OSMetaClass[Base]
    - APIs
    - DefaultStructors
    - Members, methods and the Fragile Base Class problem
  - Object Types
    - `OSCollections`
  - Serialization
    - XML Serialization
    - Binary Serialization
- The Class Menagerie
  - IOKit Built-in Classes
  - IOKit Families

- The Driver Lifecycle
  - Driver Matching
  - Case Study: VMWare Fusion VMIOPlug
  - Driver Activity and the `IOWorkLoop`
    - `IOEventSources` and `IOCommandGates`
  - Messaging
  - Matching Notifications
  - Interrupt Handling
- IOUserClients
  - IOUserClient lifecycle
  - Driver Properties
  - Notifications
  - Mapped Memory
  - External Traps
  - External Methods
  - `IOCFPlugInTypes`
- Darwin 19: The New DriverKit

14. Stacking Up: Kernel-Mode Networking 437
- The High Level View
- Layer V: Sockets
  - The `struct socket`
    - Socket Creation
    - `socreate_internal()`
    - `sockbufs`
  - `mbufs`
  - Sockets in Kernel Mode
- Layer IV: Domains & Protocols
  - Domains
  - Protocols
  - Case Study: `PF_SYSTEM` sockets
- Layer III: Network Protocols
  - Incoming packets
  - Outgoing packets
- Layer II: The Data Link Interface Layer
  - `struct ifnet`
  - Interface lifecycle
  - Case Study: The UTUN interface
- Network Data Processing
  - Receiving Data
  - Sending Data
- Firewalling & Filtering Mechanisms
  - Socket Filters
  - Content Filters (Darwin 14+)
  - IP Filters
  - PF
  - Interface Filters
- Network Extension Control Policies (NECP)

About this book

"He who controls the `kernel_task`, controls the universe!"

This is Volume II of what was originally destined to be "MacOS and iOS Internals" (a.k.a MOXiI), 2nd Edition. The entire trilogy was published out of order - Volume III came out first, then I, and now this one - but one way or another (and almost two years later), it is finally complete.

Unless you're starting with this volume, you know the story of why it was out of order - the second edition was supposed to be a two-parter, compensating for the first's partial coverage. As chapters progressed, I kept hitting the publisher's page count limit - which meant the more advanced chapters got less and less pages. I was determined to fix this in a new edition, free from all restrictions, and it also made sense to split into two volumes - one for user mode, the other for kernel. But as I started the security chapter, I found that, not only did it involve both realms, but it also expanded uncontrollably to fill a book on its own. And since Jailbreaks are the most exciting aspect of the entire trilogy, I found myself drawn to work on Volume III first.

The first edition of this work suffered significant criticism, and justly so. Far too many topics were left out of scope. Be it because of my publisher restricting the page count (and, at times, censoring me), or because I just didn't have the capability of taking on all aspects of such a complex ecosystem. Notable omissions were an in depth discussion of VFS, the IOKit families, and others. And as page count loomed, I ended up writing less and less about the advanced topics. People rightfully turned to other resources (primarily, Amit Singh's definitive but, alas, old bible) to find more information, particularly on Mach memory and IPC internals, which I was also forced to skim back then. But not so this time.

There is no point in even comparing between the first edition of MOXiI. The first edition's 800 small pages are dwarfed by a single volume of this work - and there are three of them. Free from page count restrictions, I could finally give the fine attention to the utmost detail, and I have. What you read here was uncovered by much research, perusing over the open sources (where available) and rigorous reverse engineering (where, more often, not).

This time, I aimed to leave no stone unturned. Even esoteric aspects of the operating system, such as MacOS-specific NFS support, have been detailed. A good measure I used was to keep the list of Darwin system calls in sight, and ensure that (eventually), I obtain full coverage of every single system call, no matter how rarely (if ever) it was used. The downside of this is that it makes it less likely this trilogy will ever be read "cover to cover". The upside is that, whatever the need is, it will be able to provide at least a solid introduction, if not more.

This book, like the entire trilogy, came to be in a non-linear fashion. Darwin's kernel space is so vast and uncharted, I worked associatively, dumping whatever fact came to mind, no matter how trivial, to ensure I don't leave anything important out. Then, when everything was laid out, I began the daunting task of stitching all those bits together. This took time. Literally, years - and every year passing meant another Darwin release, and more changes - forcing me at times to add more pages, yet at others to remove pages as some features I was writing about were made obsolete.

As Parkinson's Law states, the work expanded to fill the volume allotted for it. The more I invested into reading the kernel sources and disassembling the *OS kernelcaches, the more I found new code flows I was not aware of. I realized how much the first edition had left out, and sought to correct it. XNU can no longer be considered simply a hybrid of Mach and BSD - Apple has added, and is continuously adding so many exciting features and groundbreaking functionality.

If you've been following the saga, you also know this volume's release is late - exactly one year after schedule - due to my originally waiting on Apple to release sources (foolishly thinking they would in three days, like Darwin 17, and not three months, which is more than usual), and then due to other obligations and work load @Technologeeks. But - Now that the book is finally out, I can actually confess I'm happy it took so long - this got me to bring the book (and entire series) up to Darwin 19.

As you probably know from the other two books, I've tried to adopt a style closer to that of "Android Internals", which itself aims to approximate the legendary "Windows Internals", by Russinovich, Solomon and Ionescu. Unlike the first edition, in which many pages were consumed by code (especially from XNU), this edition shirks code snippets as much as possible, opting to demonstrate via illustrations and figures what can otherwise be gleaned through meticulous code. There are, however, exceptions in some cases, and more so in this volume:

1. When a particular code snippet is specifically relevant and important, or when it is actually documented so well, you might as well get the explanation straight from the horse's mouth. I'm happy to say some header comments (e.g. Corpses, kcdata and NECP) are so well explained I thought it would make sense to just cite them as they are, rather than explain the same thing.
2. When the code in question is disassembled or decompiled from an Apple binary or jailbreak with no open source to show. In the latter case, I provide annotations in the code to make it more readable.
3. When the code is part of a hands-on "Experiment", which requires typing specific sequences of instructions in order to achieve the exact output.

The numerous key structures of BSD (`proc`, `uthread`, VFS, etc) and those of Mach (`task`, `thread`, etc) keep mutating so much that, when discussing them, I chose to just point out their fields by name, rather than commit to the definition from XNU 4903's or other header file. In illustrations depicting these structures, however, you'll note that I either drew to scale, and/or mentioned offsets (0x..) besides key fields - especially those the avid kernel hacker will want to patch.

All in all, I think you'll find this new edition - in all three volumes - is worth reading from scratch, and entirely obsoletes its predecessor, and maybe its predecessor as well. As with "Android Internals", I aim to keep it "alive", and keep on updating it as MacOS and the iOS family continue evolving. Although print books get frozen in time, the relatively small batches printed at a time allow me to introduce point updates and changes just before I submit a batch. While working on Volume II I kept updating Volumes I and III - the latter having been updated more than six times with new chapters for jailbreaks over the years. All good things, however, must come to an end, and the version you're holding is pretty much the last update, barring errata or maybe integration of Darwin 19's XNU sources, if they come out by the time the hard cover color edition boxed set is out.

Contents, at a glance

If you've read the first edition of this book, you'll find the flow similar in some ways, but far, far more detailed - This book is what the second part of that edition was - detailing kernel mode, with more topics and more depth.

- Chapter 1 is a brand new discussion on **hardware** - a topic I had previously left untouched. Apple makes some of the most sophisticated devices - certainly when it comes to the mobile ones. Although hardware support isn't strictly the domain of XNU proper, and more the responsbility of specialized kexts and user mode daemons, there are some aspects - notably SEPOS and the baseband - that merit much more than a glance.
- Chapter 2 discusses the **XNU source**. It starts with a promenade of the source tree, briefly laying out the directory contents. It then looks at compiling the source, detailing the process for both Intel (MacOS) and ARM64 flavors. It then looks at some key command line arguments which may be passed to MacOS (or *OS variants, now that it's become possible again to break the iBoot chain). The chapter concludes by looking at kernel debugging over KDP, panics, and core dumps.
- With basic familiarity of the kernel sources achieved, Chapter 3 turns to **Kernel Extensions**. You might recall in the first edition this was deferred to the end, but this time around it's important to tackle this as early as possible, since they are used in experiments throughout the rest of the book. I also offer the first of two glances at MacOS15's new "user-mode" driver model, which Apple provides through System and Driver Extensions.
- Building a kext, however, is the easy part. The kernel programming environment is treacherous and deadly, and therefore Chapter 4 discusses **Kernel Primitives**, and serves as the informal kernel hacker's guide (Don't Panic :-) As its dual title implies - it is heavy on the assembly of both Intel and ARM64. This is one of the most detailed chapters, as it lays the foundations for primitives which are used ubiquitously throughout the kernel. These include synchronization, exceptions, interrupts and system calls, and accessing user mode memory.
- Chapter 5 takes a long and very detailed look at how the system boots. Unfortunately, MacOS and *OS bootloaders could not be more different, and so I first discuss the Intel EFI environment, and then go over the *OS boot chain - collectively known as iBoot - in detail. Just as I write this, the boot ROM has earned its 15 minutes of fame*, thanks to @Axi0mX's wonderfully executed "checkm8" exploit. This has dramatic effects both for boot-level researchers, forensics companies, and the jailbreaking community. I then take a detailed look at the future of MacOS boot (for as long as it's on Intel, anyway), examining how Apple synergizes EFI and iBoot in the T2 enabled devices. The chapter then concludes with a walk through of the kernel's own boot, painstakingly noting every one of the dozens of routines along the way, touching on subsystems which are encountered again later.
- Chapter 6 takes on the largest and most important subsystem of the kernel - BSD. Darwin still owes much of its code to FreeBSD, and in my humble opinion the best way to acquaint one's self with this huge codebase is to explore its startup - right where Chapter 5 leaves off. I then focus on the most important aspects: BSD's handling of processes, threads, and file descriptors. There are also two "odds and ends" in the discussion of the `sysctl` facility and of the amazing DTrace. If you've read I/15 you know how nigh omnipotent the latter is (Live on, beloved Solaris!) and its implementation is a true marvel.
- Chapter 7 corrects one of the first edition's grave omissions, in discussion the Virtual Filesystem Switch. As correctly noted by one of that edition's Amazon review, I failed then to give this important facility its just deserves. So here, after having covered files and descriptors in Chapter 6, I can dive in to this important foundation for all filesystems, and Apple's proprietary extensions to it. The chapter discusses both the Kernel Programming Interface (KPI), and the Service Provider Interface (SPI), bringing forth several case studies, and hopefully providing the reader everything he or she requires in order to create their very own custom file sytstem.

---

* - Take that, @iBootStage1 ;-)

- Chapter 8 covers Apple's Proprietary File System - APFS - which has become the de facto standard since iOS 10.3 and MacOS 12. HFS+ has faded to obsolescence after over 15 years of use, and since I believe its coverage was one of the first edition's finer chapter, I left it outside this book (the first edition remains freely available, should you want to read up on that). I hope that my coverage of APFS is on par, as Apple's file system specification, leaves far too much to be desired.
- Chapter 9 begins the long, dark descent into the reals of Mach, taking on scheduling. The task, thread and processor, first encountered in I/12, are detailed again, this time focusing on the objects behind the ports. Then, the focus shifts to how Mach's many schedulers make use of these objects, and further provide critical kernel functionality, such as deferred or timer calls, wait queues, resource accounting (via ledgers), and MacOS's Selective Forced Idle (SFI).
- Chapter 10 details Mach's most important aspect - IPC. I go over every single one of the kernel structures supporting ports - from the `ipc_space_t`, through the `ipc_entry`, down to the `ipc_object_t` and the `ipc_port_t`. Here, too, I believe I am filling some of the first edition's lacunae, because - while I did mention ports there - I didn't go into the nitty gritty detail, the exact detail which is the bread and butter of modern day Darwin exploitation. And, while on the subject, I discuss vouchers - a rather obscure (and arguably unnecessary) facility Apple added to Mach, providing dubious functionality yet unending exploit primitives. The chapter concludes with a glance of what I believe is Mach's biggest promise - multinode IPC/RPC - though only at a glance, since Apple has yet to implement this profound capability.
- Chapter 11 covers Mach's virtual memory management, through two important objects - the `vm_map` (virtual memory) and the `pmap` (physical memory) and the `vm_objects` and `vm_pages` linking these two planes. Mach's pager objects are discussed in detail, as are the Universal Page Lists (UPLs). BSD enthusiasts will feel comfortable here, as BSD shares some of these concepts to this very day.
- Chapter 12 still deals with memory management, this time that of the kernel proper, through `kernel_memory_allocate()` and the hugely important zone allocator. Yet again, I confess in shame that this was barely touched on in the first edition, and anyway it has changed so dramatically since so as to obsolete it anyway. Apple shaped and reshaped zones multiple times, though the implementation has remained fairly stable as of late (read: Darwin 16). Understanding the zone memory allocator is imperative for practicing "Feng Shui", itself an important tenet of successful exploitation.
- Chapter 13 fills the last gaping void in the first edition - IOKit. As the last chapter of that edition, I was above and beyond my page count, and so could not give this important environment and programming model the profound attention it merits. IOKit is definitely the world's most advanced driver architecture, but with great function come many avenues of exploitability. This time, I go into the internals of what makes the IORegistry, IOCatalogue, IOServices and IOUserClients actually work, circling back to other chapters (particularly, 10 and 4, for Mach ports and interrupt handling) to complete the picture painted by IOKit driver participation. Lastly, I mention Apple's new take on "DriverKit", once the foundation of modern day IOKit and now a new framework for user mode drivers, first presented in Chapter 3, and now detailed from the kernel's perspective.
- Chapter 14 concludes this work with another important subsystem - the networking stack. This was one of my favorite and most detailed chapters in the first edition (17), and I enjoyed revisiting this complex subsystem, noting how much it had evolved (via NECP and nexuses), and once again exploring its powerful filtering mechanisms (presented in I/16) from the perspective of their in-kernel implementation.

Reading Suggestions

This is the most detailed volume of the trilogy, and - just as it was the hardest to complete - it is also the hardest to read. Any operating system kernel is bound to be quite complex, and XNU's hybrid nature makes it much more so. Although I've tried to proceed in a step-by-step, as linear fashion as possible, there are unavoidable cases where key concepts (for example, kernel zones) are explained much later than when first encountered. In those cases, a first high-level read of the chapter is recommended, taking in the concept at a high level when encountered, then reading on to the low-level detail of said concept, with one other (or more) thorough read of the chapter so that the concept and its applications can "sink in".

The book can still be read in random access manner, but it is still recommended to stick to start at chapter boundaries, especially in a first read, and even more so with the later chapters. These each deal with a specific kernel subsystem, and to call them complex would be an understatement. Each chapter begins with a one page high-level view, containing a detailed illustration. The illustration is meant to be first seen at a glance, with tooltips highlighting the key components of the subsystem being detailed. The illustration can then be revisited as each component is explained in depth. I've tried to add as many illustrations as possible - pictures really are worth thousands of words. At one point I even thought that "XNU Illustrated" would have made a fine title (and homage to Stevens' bible of networking) :-)

There are two specific chapters - 5 and 6 - which contain walk throughs of two very complicated flows: The kernel startup and that of `bsd_init()`, respectively. Although I realize most readers would balk at tracing every one of the dozens of steps I walk through, I still chose to go that deep, for two reasons. The first, to serve as a snapshot in time of what these look like now, as of Darwin 19, as they keep evolving with every Darwin version. More importantly, they serve as a wonderful "checklist" of all the myriad subsystems and components XNU has, as each of them is initialized. In the case of BSD, I use the walk through of `bsd_init()` as a sweeping tour of the BSD layer. The two flows probably deserve an initial skim, and then revisited as their subsystems are encountered again, in greater depth, throughout the rest of this work.

A subtle but important note you might notice is that, whenever possible, I took care to note structure sizes, field offsets, and zone allocations. None of these are strictly necessary in order to understand the gist of things, but seeing as a substantial share of the trilogy's readers - and in particular this volume - are interested in exploitation, having these minutiae readily documented could save valuable time (and heartache). Let the reader beware, however, as none of these values are guaranteed to remain the same across Darwin versions.

> ⚠ Please note, that I've tried to be as accurate as I possibly could, but it's extremely hard to document a rapidly shifting landscape - especially that of the iOS variants. As such, specific cases - especially reversed ones - usually cite the exact version of iOS where they are taken from. Even then, bear in mind that kernel structures, IOUserClient calls and more often change without notice, sometimes in between minor versions.

As I started with Volume I, there are now "Review Questions" at the end of every chapter. These serve several purposes: First, as their name implies, to review and give the reader a chance to reflect and see if they got the salient points of each chapter. Second, to open up an avenue for this book's use as a textbook in academia, as I know some people have already started doing.

Online Resources

In addition to the many free tools accompanying this book, there are quite a few additional resources, which I keep on the book's companion website - http://NewOSXBook.com. The "new" there is because Amit Singh's pioneering http://OSXBook.com remains online, and almost as relevant eleven years later (although Apple introduced loads of new features since 10.5). The companion website holds:

- **Code samples**: Not only have I left most Darwin code snippets out of the book (unless annotated or exemplary), I left my own code out as well, aside from a very short few. This way, more pages can be devoted to explanation, and anyone - not just the readers - can easily download, compile and run the samples so as to learn from them.
- **Various articles**: which I started writing as unofficial updates to the first edition but continue with to the present day. Each article deals with a specific topic or tool, and some are in a series (notably, those dealing with Apple's OTA updates, which I had reversed and provided tools for)
- **The list of system calls and Mach traps**: which I started back in the first edition, due to a page count restriction, but am sticking with since it's a good idea to have an online reference for XNU's interfaces - the POSIX system calls and some of the Mach traps are actually documented, but Apple keeps adding more and more proprietary calls, without providing documentation even in the kernel sources.
- **The book's forum**: This took a *long* time to kick off, but (after five years..) has managed to become a mildly active* interchange of ideas and Q&A. I also use the forum for announcements, and it remains the best medium over which to ask me questions - although I am reachable via email or Twitter, if you ask me non-confidential questions over email I will likely politely redirect you to use the forum, since this way others can benefit from the answer.
- **The Entitlement database**: Which catalogs Apple's hundreds of entitlements, used as a declarative security mechanism. The database was created by `jtool --ent` over all binaries in MacOS and *OS, and shows entitlement holders (i.e. consumers), with plans to add producers (i.e. verifiers) at a later date. It is also one of the more popular landing pages among readers in 17.x.x.x.
- **The XNU Cross Reference:**: or XXR, which aims to approximate what "LXR" is for Linux: a hypertext linkable, searchable view of XNU's massive sources. I added this very recently (around v1.2 of this book) and it has already proven itself very useful. You can find representative sources of the XNU kernel from every major Darwin release, compare them and quickly find function and variable definitions.
- **The book's first edition**: is **freely available** as a PDF. Although it was pirated and torrented to death anyway (and you can easily find it by Googling "MacOS iOS Internals filetype:pdf"), it is now officially available through me as well. This is a gesture that I hope will help both expand the knowledge of MacOS/iOS for people, as well as help people with understanding this, second edition.

And, of course, let's not forget the powerful tools on the website! Read on...

---

* - That the occasional spammers get into the book's forum is unfortunate and swiftly dealt with, but at least it helps me think the forum is popular by some measures ..

Tools used in this work

I've used quite a few tools, all of my own devising, in order to gain the X-ray insight required for writing this book. Along with Apple's `lldb(1)`, for which Derek Selander's authoritative reference proved essential, I used the following:

- `jtool(j)`: is probably the oldest (and my most favorite) tool. I had written it back for the first edition of this work, and it began its life as a simple `otool -l` clone, as I found the `otool(1)` output dismal. One by one, I collapsed most of Apple's binutils - `nm(1)`, `segedit(1)`, `dyldinfo(1)` and others - and eventually wrote my own ARM64 disassembler, which I found much easier and more efficient to use than HexRays' .... IDA. `jtool(j)` is instrumental for the Mach-O and DYLD chapters of this book, as well as for the reversing of *OS binaries, which in many cases I continued all the way back to the source.
- `procexp(j)` (Process Explorer): is another tool from the first edition, whose original purpose was to be a demonstration of the unbelievably useful yet largely undocumented `proc_info` system call (which is covered in Chapter 15). Unhappy with `top(1)` as well, I eventually morphed `procexp(j)` into a replacement for it. Unlike `top(1)`, however, `procexp(j)` is also usable in pure CLI (no-curses) mode, allowing detailed process snapshots, core dumps and more.
- `jlutil(j)`: My clone of Apple's `plutil(1)`, which not only supports BPList16 and works on Linux, but also uses a far simpler format I couldn't help but call SimPLISTic. I use this all over now, presenting the omnipresent property lists in a simplified, compact form, rather than cumbersome XML.
- `Xn00p`: is probably the most powerful and important tool for *OS I've ever created. It is a kernel memory inspector/debugger, which is capable of working on the same machine. I started coding it as a tool to facilitate *OS kernel inspection, and realized it could work just as well on MacOS as well. The tool has since evolved in lockstep with this volume, as many of its features were driven by examples I needed for this book. Like all my tools, it is a CLI tool, designed with `grep(1)` in mind as the filter of choice. **Note, that this is the only tool which I have *NOT* made free**, since it is far too advanced and (unlike the other two here) was designed to be a commercial tool, licensed by my company, @Technologeeks.
- DTrace scripts: Since I'm not asking my readers to invest in any costly tools, those wanting to follow along on MacOS can make use of several DTrace scripts I've provided. DTrace has not only the built-in ability to read kernel memory, but also is aware (thanks to the kernel's `__CTF.__ctf` section) of most symbols and datatypes. Most of the "xnoop dump" output can be approximated with the simple `kmem.d` script I've provided, but for more advanced output you'll need to find the right `.d`, depending on context.

All the above (with the exception of Xn00p) are free to use, and some are even open source. Though as a personal request, if you find bugs, do let me know, as the tools are all built around my common use cases, and therefore it's quite likely that, when you use them differently, they might behave erratically or even crash. Also, modifying the sources is one thing, but GitHubbing them and claiming them as your own is quite another.

- filenames are specified like this. Apple's mile long path names have been abbreviated, where possible, so that /System/Library/Frameworks becomes /S/L/F, PrivateFrameworks becomes PF, Caches becomes C, etc. Likewise `com.apple.` is usually abbreviated as `c.a.` Where `#include` brackets are shown, the file they pertain to can be found in the appropriate SDK directory. GUIDs (which are useless to quote anyway and take up tons of space) are abbreviated to only a few characters, so their uniqueness can be followed. Files from projects (specifically in this volume, XNU) are usually mentioned in their context, and shown with a relative, not absolute path.

- `commands(1)`, `systemCalls(2)` or framework `classes` are specified thus. The numbers in parentheses refer to the manual section describing them (if one exists), using the MacOS `man(1)`

Additionally, this book is full of Figures, Listings and Outputs. Figures are illustrations of components or message flows. Listings are generally static files, as opposed to Outputs which are sequences of commands, often included as part of an experiment. In listings, I often add my own annotations over Apple's sources (particularly for structure fields), and those can be seen by the use of C++ style (//) comments, and a slightly different shade (or, in color, purple, rather than the comment green). In outputs, annotations appear brighter (or, in color, yellow). The idea in the outputs was to show the flow as well as usage of the commands, so what the user is supposed to type is in bright white, and the outputs are fully annotated, e.g.

```
# Comment, explaining what's being done
user@hostname (directory)$/# User_input
Output...
Output...  # Annotation, explaining output
```

Note, that the prompt user (and terminator, $ or #) will tell you if the command requires root privileges. The hostname shows you the device the command was tried on, but unlike other volumes in this series everything was demonstrated on MacOS 14-15 (Chimera or Bifröst, T1 and T2 devices), and all the others are the numerous iDevices.

There's another self-adopted convention I use quite a bit: Mile long constant and function names which appear in rapid succession are now "condensed" so I can fit them on a single line and not frustrate the reader. You can see lots of examples, but one of the more succinct one would be:

```
p_[/r/sv][ug]id
```

meaning p_uid, p_gid, p_ruid, p_rgid, p_svuid, p_svgid. (Compact, no? :-)

For property lists, Apple's favorite XML formats, I opted to use my own tool for presentation. I call it "simPLISTic", and it resembles MacOS's `plutil(1)`, in that it can handle all formats (binary1, json and xml1, as well the undocumented bplist16 used in NSXPC) - but it can also greatly simplify those annoying, cumbersome, and space hogging XML tags. Like most of my tools, I've made it open source and compilable for *OS and even Linux (intentionally not using the simpler but unportable `CF*` APIs) - and I hope you readers (and, who knows, maybe even Apple?) will find it useful.

Lastly, as you might have gathered, "*OS" refers (anywhere but the title of this work..) to iOS and its variants (i.e. iOS, WatchOS, TvOS, HomePod OS, and BridgeOS) but **NOT** MacOS. This was straightforward when MacOS was still "Mac OS X", but Craig Federighi ruined this nice distinction for me. When faced with a choice of correcting to "^[^M].*OS" or keeping it simple, I left it at that for readability.

Finally...

This second edition, like the first edition of MOXiI, owes its existence to two people. The first is Yoav Chernitz, my once and former taskmaster in China. My career as an author started with his inception that, in addition to delivering all the trainings I did, I could try to write a book. The second is my dear friend, Ronnie Federbush, who made me choose the self-publishing route - which forever freed me from the shackles of other publishers.

Once again, my thanks to Eddie Cornejo, who did a marathon read of the manuscript catching more than a few typos, and giving me important feedback. @axi0mx was super helpful - not only in providing the checkm8 exploit, but also meticulously reviewing the iBoot section of this book.

And to my One, my only Amy, thanks to whom goes without saying, yet cannot be understated - keeping me focused as my attention wandered and interest waned, and talking me out of times I wanted to `rm -fr ~/Documents/OSXBook` and make MOXiI into my own D.759

And to the other, Tg Amy - I thank you for making @Technologeeks the unique boutique training and consulting powerhouse it has become. Normally here I would be making a not-so-subtle pitch about our the awesome MacOS/iOS training at http://technologeeks.com, and finding me on Twitter (as the six underscored @Morpheus_____), but this era is at an end. Taking a cue from my great inspiration for this work, Amit Singh, I'm following in his footsteps and "just leaving this here", and ending my tenure as Darwinologist. If you, my faithful reader, want to fill in for me - let @Technologeeks know - we're always looking for a few good people :-)

And finally, finally - This book took way longer than it should. I'm the first to admit that. I cordially thank Apple for not only offering absolutely no help, but also taking their time in releasing the sources for Darwin 18, which I deemed absolutely critical to complete this work. This effectively delayed me by one whole year - until I slapped myself silly by realizing who needs them sources anyway, when you have DTrace and `ctfdump`! That said, delays have their benefits (כל עכבה לטובה) - and the one here is that this book is up to date with Darwin 19. I know some of you have been waiting for this a very long time, and I can only hope this proves worth the wait, but I will delay you no longer. Let's get to it!

"Once more unto the breach, dear friends!"

1

Welcome to the Machine: Hardware

Apple is uniquely positioned as a company which controls all levels of their devices, from the highest bits of software to the lowest depths of its hardware. This tight hold allows Apple to not only be an early adopter of emerging technologies and future standards, but also develop their own proprietary mechanisms, weaving together silicon-level protections with the software that drives them.

This chapter takes a stab at the low-level hardware that makes up Macs and iDevices. Though a full discussion would probably, on its own, fill a trilogy of books, our aim here is to touch on the components which sum up to Apple's hardware, and the kernel-level (read: Driver) support for them. Following a glance at the lineups of Macs, the main focus is on iDevices, wherein Apple designs the application processor and some of the dedicated coprocessors - notably, the Secure Enclave.

Devices

Apple started its life nearly half a century ago as a hardware company with the iconic Apple I created by Steve Wozniak, and steadily evolved it. 1984 brought the first "Mac", which has been used as the brand for an ever increasing line of desktop computers, servers, and laptops.

Macs

Apple has expanded its line of Mac products significantly, and it presently includes the MacBook, MacBook Air, iMac, Mac Mini, and Mac Pro. These are periodically refreshed, yet the brand name remains constant. It therefore follows that Apple needs some other type of nomenclature to figure out which version of a device is discussed.

Mac Model Numbers and code names

Mac users are probably familiar with Apple's customer-facing convention, of referring to their Mac Devices by year of manufacture, or hardware refresh. This produces such names as "MacBook Air (Late 2012)", etc. Internally, however, Apple uses a combination of a generational numbering scheme along with an Apple Part Number. The generational scheme doesn't always change when the hardware is refreshed – a good example of this is the Mac Pro, whose model was "MacPro5,1" throughout Mid 2010 to Mid 2012, and only changed to "MacPro6,1" with the introduction of the cylindrical Mac Pro in Late 2013. The device model is stored in the System Management BIOS.

Apple provides several technotes[1] for users to figure out the exact generational model and part numbers. The former are important when support is dropped for them in MacOS (e.g. the 4th generation (2012) no longer being supported as of Mojave, MacOS14). The latter are used when unfortunate manufacturing defects are discovered, leading to specific part number recalls.

Apple also uses code names for their devices. These are made up of a letter (usually 'J' or 'K') plus two digits. The exact algorithm used to deduce the code name for a given device is unknown (or, at least, eludes this author..), but seems to be loosely related to the year of manufacture. The codenames are generally hidden from the customer, and cannot be easily determined from a given device.

To find a Mac device's Internal Number, try `sysctl hw.model`, or filter it from the output of `ioreg -l -f`, where it it appears right under the Root node, as an instance of the `IOPlatformExpertDevice` class. As far as this author can tell, there is no known way to find the code name.

Processors

Macs were originally PowerPC based, but Apple made the surprise move to the Intel 32-bit Architecture in 2005 with 10.4.7, and 64-bit Intel gained kernel-mode support in 10.5. As legacy support, Apple provided "Rosetta", a system for binary emulation which lasted until PPC was officially dropped in 10.7

Intel 64-bit was supported fully by 10.6, and though it existed side by side with 32-bit (through universal binaries), as of 10.8 the kernel was 64-bit only, thus mandating a 64-bit processor. User mode 32-bit binaries are still supported through universal `dyld` and some of the frameworks, although as of 14 the OS warns of using 32-bit.

It has been long speculated that Apple might make another surprise move - this time from Intel to ARM64. This makes much sense, given ARM processors' superior power management, and even more so considering the architecture's advanced security features (which Apple is eager to adopt in its iDevices), and Apple's own custom architectural extensions, which simply cannot be made with Intel's generic chips. Maybe as soon as MacOS 16 or 17, Apple could launch a MacBook with an ARM chipset - and maybe even bring back Rosetta, to support "legacy" Intel binaries.

iDevices

Just as with the MacOS landscape, that of iOS has grown significantly since its introduction. From a single product – the iPhone - it has grown to encompass multiple distinct product lines (along with the iPads, Apple TV, the Watch, and most recently the HomePod), with multiple generations of models for each.

Model Numbers and code names

Internally, the iDevice models are referred to by a numbering scheme which is similar to Apple's Mac lines. The scheme likewise defines a major – the generational number – and a minor, used to differentiate GSM, CDMA, Wi-Fi only and other variants of the same iDevice. Unlike the Mac Device numbers discussed previously, those of the iDevices are clearly visible in the downloaded iOS software updates (.ipsw) files, which follow the convention:

```
iPhone6,2_8.1.2_12B440_Restore.ipsw

iDevice##    ,iOSver_Build#_Restore
```

The internal numbering scheme, however, is entirely different from the customer facing generational numbers – so that the iPhone 4 is actually 3,x, and the iPhone 6 is actually 7,x. In the figure above, iPhone6,2 actually means iPhone 5S. A nice side effect of the jump from iPhone 7 to iPhone 8/X (iphone10) was that the customer facing numbers partially caught up with the internal ones, but this was lost again with X[s/r] (iphone11) But, as if that wasn't confusing enough, the iDevice codes follow no known convention. Most iPhones code names (up to the iPhone 7) are all nXX, but the model numbers are apparently the result of some peculiar random number generator. The numbers and names for iPhones are shown in Table 1-1. A more comprehensive list is maintained in a dedicated Wikipedia Article [2].

Table 1-1: iPhone model numbers, marketing and code names

Model Number	Marketing name	Code Name
3,x	iPhone 4	n90
4,x	iPhone 4S	n94
5,1	iPod Touch 5G	n78
5,1 and 5,2	iPhone 5	n42
5,3 and 5,4	iPhone 5C	n48
6,1	iPhone 5S	n51
6,2		n53
7,2	iPhone 6	n61
7,1	iPhone 6+	n56
8,1	iPhone6S	n71
8,2	iPhone6S+	n66
8,4	iPhoneSE	n69
9,x	iPhone 7	d10/d20
9,x	iPhone 7+	d11/d21
10,x	iPhone 8	d20
10,x	iPhone 8+	d201?
10,x	iPhone X	d22
11,x	iPhone X_S , Max	d321, d331[p]
11,x	iPhone X_R	n841
12,x	iPhone 11	n104
12,x	iPhone 11 Pro (Max)	d42, d43

The platform code name (e.g. D10AP for the iPhone 7) can be found in the first child element of the IORegistry (as an instance of `IOPlatformExpertDevice`).

The iPads, along with the iPad Mini, start off with a similar internal numbering scheme, but lose sync with the iPad 4. The code names are even more confusing, starting off with "Kxx", then splitting off to "Jxx" and Pxxx" . The Model numbers accommodate for future releases, as can be seen in cases like the iPad Mini 4, which arrived well after the iPad Air 2 did, but still got the earlier 5,1 and 5,2 designation to the Air 2's 5,3 through 5,5. Likewise, the iPad Pro (6,7 and 6,8) leave room for the (later announced) iPad Air 3 (also an "iPad Pro") and an eventual Mini 5.

Table 1-2: iPad marketing names, internal numbers, and code names

Model Number	Marketing Name	Code Name
iPad 1,1	iPad	k48
iPad 2,1 - 2,4	iPad 2	k93, k94, k95
iPad 2,5 - 2,7	iPad Mini	p105, p107 (+3G)
iPad 3,1 - 3,3	iPad 3 ("New")	j1, j2 (+3G)
iPad 3,4 - 3,6	iPad 4	p101, p103
iPad 4,1 - 4,3	iPad Air	j71-73
iPad 4,4 - 4,8	iPad Mini 2,3	j85[m], j86[m]
iPad 5,1 - 5,2	iPad Mini 4	j96
iPad5,3 - 5,4	iPad Air 2	j82
iPad6,7 - 6,8	iPad Pro	j98a, j99a
iPad6,3 – 6,4	iPad Pro (9.7")	j127, j128
iPad8,1 - 8,4, 8,5 – 8,8	iPad Pro (11" / 12.9", 3rd)	j317, j318, j320, j321[x]
iPad11,3 - 11,4	iPad Air 3	j217, j218

The numbering and model scheme further applies to Apple's lesser devices and the other embedded iOS variants. Table 1-3 summarizes those:

Table 1-3: The lesser devices marketing names, internal numbers, and code names

Model Number	Marketing Name	Code Name
TV	4th Generation	j42d
iPod Nano (discontinued)	iPod Nano	n31
Watch (Series 1)	38mm/42mm	n27d/n28d
Watch (Series 2)	38mm/42mm	n74/n75
Watch (Series 3)	Wifi (38/42)	n111[b/s]
	Wifi	n121[b/s]
Watch (Series 4)	Wifi (40/44)	n131[b/s]
	Wifi	n141[b/s]
Watch (Series 5)	Wifi (40/44)	n144[b/s]
	Wifi	n146[b/s]
Touchbar (MacBook 2016)	T1	x619
Touchbar (MacBook 2018)	T2	j137
Haywire	Lightning to HDMI adapter	b137
AudioAccessory1,[1,2]	HomePod	b238a[a]

Unlike the Device code names, iDevice codes are clearly visible in the filenames contained in the iOS software updates. The code names gain greater visibility still on jailbroken devices, which reveal the existence of model-specific processes (e.g. fairplay), and even the output of the uname(1) command, as we show in a later experiment. Trying `sysctl hw.model` on an iDevice actually returns the code name, rather than the model number.

Processor Code Names

With each generation, the hardware of the iDevices improves significantly. Chief amongst the hardware components is the CPU itself, which – being a system-on-chip – integrates other components, including the graphics processor (GPU).

Apple develops its own processors, licensing the CPU intellectual property from ARM, and the GPU from Imagination Technologies (PowerVR). The processors are fabricated by Samsung (their frenemy), and by Taiwan Semiconductors (TSMC). As the iDevices evolve, their processors gain in frequency, transistor density (owing to a diminishing nm fabrication process, down to 7nm with the A12) and – gradually – cores. Table 1-4 shows the processors and their mappings to various devices. A far more comprehensive list can be found in the dedicated WikiPedia article [3].

The processors on the iDevices are referred to as "A"-series chips. Beginning with the A7, processors also have an "M"-series motion controller, with a matching number. Beginning with the A11, a specialized "H"-series Neural Engine.

Apple's processors are remarkable in several ways, not the least of which are performance and power efficiency. Although, as with Android devices, the underlying architecture is ARM (or ARM64), the processors exceed the specification with various enhancements, such as an L3 cache. Contrary to the Android paradigm of throwing more cores and increasingly higher clock rates in order to achieve better performance, Apple has resisted moving beyond a dual core CPU architecture up until the iPad Air 2 (its first triple core CPU), whereas Android processors were quick to adopt quad and octo-core processors. Nonetheless, despite having less cores and a greatly reduced clock rate, iOS devices were already considered blazing fast. In fact, the two cores in the A9 processor of the 6S don't trail far behind those the Exynos from S6 – which had eight cores.

This approach reaped dividends in power efficiency, since more cores and faster clock rates both have adverse effects on power consumption. For this reason, most Android devices power off cores whenever possible and try to dynamically govern the CPU frequency. iOS devices, by comparison, used to get by with significantly smaller batteries, though in recent years has caught up with Android, hoveringa bit under 3000mAh on average. Apple could probably do much better by increasing battery capacity. Sacrificing an additional ounce or two and thickening the (often too) thin devices could greatly extend battery life – yet Apple seems to be choosing form over function.

Apple eventually started expanding horizontally, as well. With the A10, Apple introduced a quad core configuration, but as a "n+n" configuration, wherein either the high performance (so called "p-cores") or the energy efficient ("e-cores") are active at any given time, similar to ARM's "big.LITTLE" implementation. The p-cores are codenamed after fierce winds ("Hurricane", "Monsoon" and "Vortex") whereas the e-cores use gentler winds ("Zephyr", "Mistral", "Tempest"). With the A10X and A11, quad (2p+2e) became hex (3p+3e), and the A12X is a monster octo-core (4+4) which far surpasses most desktop processors.

Another important consequence of Apple's constantly evolving processor architecture is the adoption of newer ARM architecture standards. All modern A-series processors as of the A7 are 64 bit ARMv8, but Apple is also quick to adopt ARM architectural improvements: A11 and later implement the ARMv8.1 instruction set - which enables better atomic instructions as well as the highly important Priveleged-Access-Never (PAN) protection against userland access from kernel. A12 processors further support ARMv8.3, which provides pointer authentication - mitigating the ROP/JOP attacks which are a staple of exploitation. It is only a matter of time (perhaps, in the A14?) before Apple adopts ARMv8.5, which further provides the protection of Memory Tagging Extensions (MTE), which should bring an end to UaF, type confusion, and other memory corruption attacks.

The ARM processor specifications leave much room for "`IMPLEMENTATION DEFINED`" features and registers, and Apple indeed implements their own custom architectural extensions. Three of those - KTRR and APRR, and the A13's SPRR - are instrumental in providing kernel physical memory protections no other player in the ARM processor space (certainly not any of the Android implementers) achieve. Indeed, these provide formidable defenses even in the face of a potentially compromised kernel, as is discussed later in this volume, and also in III/13.

Table 1-4: Processor marketing names, internal names, and specifications

	SOC name	Core name(s)	uname -p	Found in	Spec
ARMv8 (64-bit)					
A7	Alcatraz	Cyclone	S5L8960	5s, iPad Mini2 &3, ipad Air	Dual core GPU: 1.3Ghz Quad core PowerVR G6430
A8	Fiji	Typhoon	T7000	iPhone 6, 6+, TV 4 iPad Mini 4, iPod (6th)	Dual Core, 1.4Ghz ARMv8-A GPU: Quad core PowerVR GX6450
A8X	Capri		T7001	iPad Air 2	Triple core 1.5Ghz GPU: Octo-core GXA6850 PowerVM
A9	Maui	Twister	S8000 (Samsung)	iPhone 6S, 6S+, SE, iPad (5th)	Dual Core, 1.85Ghz GPU: Hex-core 7XT GT7600 PowerVR
	Malta		S8003 (TSMC)		
A9X	Elba		S8001	iPad Pro	Dual Core, 2.26Ghz GPU: 12 core 7XT GT7600 PowerVR
ARMv8.1 (+ atomic extensions, PAN)					
A10	Cayman	Hurricane Zephyr	T8010	iPhone 7, 7+, iPad (6th), iPod (7th)	Quad Core: 2 x 2.34Ghz + 2 x 1.2Ghz GPU: Hex-core GT7600 PowerVR
A10X	Myst		T8011	iPad Pro (2017) TV 4K	Hex Core: 3 x 2.34Ghz + 3 x 1.2Ghz GPU: 12 core GT7600 PowerVR
ARMv8.2					
A11	Skye	Monsoon Mistral	T8015	iPhone 8, 8+, X	Hex Core: 2 x 2.39Ghz + 4 x 1.2Ghz GPU: 3 core
ARMv8.3 (+ pointer authentication)					
A12	Cyprus	Vortex Tempest	T8020	iPhone XS[Max], XR, iPad Air (3), iPad Mini 5	CPU: Hex Core: 2 x 2.49Ghz + 4 x 1.4Ghz GPU: 4-core
A12X	Aruba		T8027	iPad Pro (2018)	CPU: Octo Core: 4 x 2.49Ghz + 4 x 1.4Ghz GPU: Septo Core
A13	Cebu	Lightning Thunder	T8030	iPhone 11 (2019)	CPU: Hex Core: 2 x 2.49Ghz + 4 x 1.4Ghz GPU: 6-core

The "lesser" devices, such as the Watch and AirPods, use custom chips which are given different code names and numbers. These include:

Table 1-5: Watch/Embedded models

	uname -p	Found in	Spec
S1	S7002	Watch	Single Core, 520Mhz 32-bit ARMv7k
S1P	T8002	Watch (Series 1)	Single Core, 520Mhz 32-bit ARMv7k
S2 T1	T8002	Watch (Series 2) TouchBar (MacBook 2016)	Dual Core, 520Mhz 32-bit ARMv7k
S3	T8004	Watch (Series 3)	Dual Core, ? Mhz 32-bit ARMv7k
S4	T8006	Watch (Series 4, 5?)	Dual Core, Vortex
T2	T8012	iMac Pro (2017) and all later Macs	Dual Core, Hurricane

Lastly, Apple's chip designs have extended into dedicated connectivity controllers. The trend started with the AirPods, which required truly miniaturized chips, and has since picked up, most recently with the A13's Ultra Wide Band processor (code named "Rose")

Table 1-6: Proprietary Connectivity modules

	Found in
H1	Air Pods (2nd)
W1	Air Pods, Beats headphones
W2	Watch (Series 3)
W3	Watch (Series 4)
U1	iPhone 11+ UWB

Apple used to cleverly phases out older models, by discontinuing support for them as newer versions of iOS come out. The primary reasoning behind this was likely to encourage (not to say, coerce) customers to upgrade, although in practice older devices could still very well handle the newer iOS versions. In this manner, Apple has obsoleted the iPhone 4 (the last of the A4 devices) with iOS 8, finally dropping support for the iPhone 5/5C (the last of the A6 devices) in iOS 11. With this, the move to 64-bit only iDevices (excluding the watch) was complete. One of iOS 12's big surprises was that Apple chose to provide it for all 64-bit devices - including the five year old 5S. Considering Apple's declining iPhone sales, however, it was no surprise if it resumed the phasing out trend with iOS 13, and iOS 14 will likely not support anything lower than the iPhone 7, or possibly even below 8/X.

Another important side effect of phasing out older generations is keeping iOS a moving target. The A4 devices were susceptible to a vulnerability in their bootrom, which effectively made iOS on those device permanently jailbreakable (albeit via tethered boot). Another vulnerability is suspected to exist somewhere in the boot sequence of 32-bit devices – yet no such vulnerabilities are known for 64-bit ones. This explains why the IPSW decryption keys (which rely on access to the iDevice GID keys) for 32-bit devices are available on the iPhone Wiki, whereas those of 64-bit devices are not. If Apple continues their trend and discontinue older model support in iOS 14, cutting off at the A10 (iPhone 7) mark would ensure all compatible devices have the hardware kernel integrity mechanism of KTRR going forward.

Part Code Names

Though the application processor is indubitably the heart of the iOS devices, it is in reality but one part of many, which must work together in harmony on a **System-on-Chip** (SoC). The wifi chipset, baseband, and various sensors are all self-contained parts, and have their own independent processors and firmware. Leaked schematics of the various iPhone models[4] reveal the unbelievable complexity of each component, as well as the herculean effort of putting them all together. These documents also give an idea to some of the internal code names used by Apple for these components, shown in Table 1-7:

Table 1-7: Code names

Code Name	Refers to
Pinot	LCD Panel
Stockholm	NFC/Secure Element
Graphite	Accelerometer & Gyro
Callan	Audio or Haptics Firmware
Acorn	PMU
Guiness	WiFi MLB
Magnesium	Compass
Phosphorus	Camera?
Romeo/Juliet/Rosaline	FaceID ("Pearl") infra-red scanner/module/Intermediary
Oscar	M7+ Motion Coprocessor
Mesa	Touch ID
Haywire	Lightning to HDMI converter
Icefall	Apple pay related
ICE	Intel Baseband (Intel Communication Equipment?)
Mav(erick)	Qualcomm Baseband (MDM96xx)
Nero/Octavia	Lightning to HDMI component
Trek	Qualcomm Baseband (MDM66xx)
Orion	iPad Smart Connector
Savage	FaceID
Maggie/Homer/Lisa	Unknown (FPGAs related to home button/haptics?)
(Borealis) Owl	Always On Processor
Petra/Zelus	Image Signaling Processor (A12/A13)
Quin/Metis	Apple Neural Engine (A12/A13)
Rose	"U1" Ultra Wideband Chip (A13)

Quite a few of the coprocessors have their own operating system images or, in other cases, configuration and calibration files. Those can be found in the .ipsw as well, as discussed in I/A. We discuss some of these in more detail later in this chapter.

NAND Storage

Like all mobile devices, iDevices use NAND flash as their persistent storage*. But the quoted storage size - 16, 32, 64, 128 or even 256GB in the newer devices - is not all accessible for user data . A significant portion - around 4GB - is taken up by iOS's system (/) partition, as can be expected. But little known is that the system and data (/private/var) partitions aren't the only ones on the NAND. In fact, both these partitions are in one physical partition, but there exist several others, which are not directly accessible.

Apple has traditionally formatted the NAND flash into about six physical partitions, using a custom partition table known as "GRDN", due to its magic. As of the iPhone 6, however, Apple began incorporating an NVMe controller. This enables the use of *namespaces*, rather than partitions, though the end result is largely the same: The NAND is divided into several namespaces, whose first (NS1) is the one containing / and /private/var partitions. Note, that these are **logical** partitions, using the Lightweight Volume Manager (LwVM) and HFS+ (prior to 10.3), or APFS (as a single container with two volumes, as of 10.3). As of iOS 11, the main namespace's APFS container is also used for baseband and other hardware support.

The namespaces outside the main mountable one contain "hidden" partitions. These were first uncovered thanks to the groundbreaking work by Ramtin Amin[5] are shown in Table 1-8:

Table 1-8: The layout of NAND partitions/Namespaces

Namespace	Size	Label	Contents
NS1	Variable	fsys	/ and /var. Accessible as /dev/disk0s1. GPT formatted.
NS2	8M	boot	Boot components (LLB, glyphs, icons, iBoot)
NS3	128K	scfg	System Configuration data (see later)
NS4	8K	wifi infos	Wifi Information
NS5	8K	nvrm	NVRAM variables (see later)
NS6	4K	plog (effaceable)	Device keybag data
NS7	1MB	?	Unknown/Reserved (empty)

As we'll see later in this chapter, the data in these namespaces remains accessible, albeit with limitations. The Effaceable partition is handled by `AppleEffaceableStorage.kext`. Likewise, NVRAM and SysCfg are accessible through dedicated kexts, although no direct raw block device interface for these namespaces is provided.

---

* - Pre iPhone 4 devices used NOR flash as well to store their configuration, but by now this is so obsolete it can be discounted entirely in the scope of this chapter's discussion.

Ports

Computer systems need a way to communicate with peripheral devices. Macs are no exception to this, and even the i-devices aren't, with Apple's proprietary lightning collapsing USB, serial, and other connection channels into one interface. Apple provides very little documentation [6] about ports.

Serial ports

Mac devices no longer have physical serial ports, but it is a simple matter to add a USB cable which provides a serial port (or define a virtual serial port in a Virtual Machine). Their significance stems from the kernel's built-in support, which allows for kernel debugging.

Table 1-9: Serial port related kernel extensions

Kernel Extension	Supports
AppleOnboardSerial	
Apple16X50Serial	
AppleUSBSerial	Serial port access over USB, for example for the ... adapters
IOSerialFamily	IOKit family for generic serial port operations

Serial devices have traditionally been represented in UN*X as character devices. `IOSerialFamily` provides the `IOSerialBSDClient` class to handle the automatic creation of two devices: The `IOCalloutDevice` property specifies a /dev/cu.*deviceName*, and the `IODialinDevice` property specifies a /dev/tty.*deviceName*. The former ("calling unit") is the "raw" device, and the latter provides a line discipline and terminal capabilities. The `tty` devices can be opened by capable software, such as `screen(1)`, `minicom`, and other terminal emulators. A third character device, /dev/uart.*devicename*, is also occasionally used to interface the UART chip, with /dev/uart.BLTH in MacOS, and multiple UART instances for the dedicated processors in the *OS variants.

Firewire

Firewire, also known as IEEE1394, has been supported by Apple for a time, since around MacOS 10.4 and the 2005 Macs. It came in two versions - 400 and 800 - named for their throughput in Mbit/sec. Although it was once heralded as the bus of the future, FireWire has since been extinguished, and Macs no longer have ports as of 2012, with Apple having chosen to adopt Thunderbolt instead, and supply Thunderbolt to Firewire adapters for a humble fee.

Although it has been largely discontinued, MacOS's XNU still provides support for FireWire by supplying the necessary kernel extensions, shown in Figure 1-10. As they are old and not as well maintained, the FireWire kernel extensions prove to be a reliable source of vulnerabilities.

Table 1-10: The IOFire kernel extensions

Kernel Extension	Supports
IOFireWireAVC	
IOFireWireFamily	
IOFireWireIP	FireWire as a network interface (`fw#`), supporting IPv4/IPv6
IOFireWireSBP2	
IOFireWireSerialBusProtocolTransport	

FireWire can be used as a medium to connect Macs for Target Disk Mode, or for kernel debugging. The `fwkdp(1)` tool proxies the Kernel Debugging Protocol over it, as is discussed in Chapter 2.

ThunderBolt

Apple is an early adopter of Intel's ThunderBolt standard.

ThunderBolt encompasses MiniDisplay Port and PCIe, and adapters exist for virtually any type of connection – FireWire, Ethernet, etc. The Mac pro sports no less than six ports(!) for peripherals, and most Mac models have two. Only the Macbook Air has been left with one port, which makes for a hard choice of external display (exclusive) or wired Ethernet.

Thunderbolt exists in three versions, each significantly increasing the capabilities of its predecessor. The first version supports up to 10Gb/sec per channel, with the second aggregating (i.e. allowing a logical channel of 20Gb/sec by utilizing both channels), and the third doubling the bandwidth to 40Gb/sec. Thunderbolt 1 and 2 use a mini displayport connection, and version 3 uses USB-C, which is also the standard port as of the 2016 MacBooks and onward.

Apple provides a partial Thunderbolt device driver programming guide[7] but (as with other useful documents) it is no longer maintained.

Table 1-11: The IOThunderbolt kernel extensions

Kernel Extension	Supports
AppleThunderboltDPAdapters	Thunderbolt-Display Port - contains `DPAdpaterFamily`, and `AppleThunderboltDP[In/Out]Adapter`
AppleThunderboltEDMService	
AppleThunderboltIP	
AppleThunderboltNHI	Native Host Interface
AppleThunderboltPCIAdapters	Thunderbolt PCI uplink/downlink
AppleThunderboltUTDM	
IOThunderboltFamily	Family driver used by all other kexts

USB

USB needs no introduction and - unlike other ports - is here to stay. Apple was an early adopter of the standard, and has kept ahead of the curve by being among the first to support USB 3.0, and the first to support USB-C (in the 2015 Macbook).

Apple has surprised many with its latest MacBook model (at of March 2016) offering a single USB-C port for all its connectivity needs. This continued with the refresh of the MacBook Pro line in late 2016, which abandoned all existing ports in favor of four USB-C ports (over which ThunderBolt 3 devices can be used as well). Apple even discontinued the extremely useful MagSafe, which undoubtedly saved many laptops from meeting their untimely end by disconnecting when gently tugged.

The USB Bus is used both externally (as ports) and internally. The Bluetooth host controller, for example, is a USB device. The 2016's MacBook Pro's "Touch Bar" appears as a USB device to its host, and communication with the integrated coprocessor is carried out over USB's Communication Device Class (CDC) Network Control Model (NCM) protocol. There are quite a few other CDC models used for networking and tethering, and MacOS supports them all.

A list of USB devices can be obtained through `ioreg -p IOUSB` or (for a more readable, albeit MacOS only output) `system_profiler SPUSBDataType`.

As with FireWire, USB can also be used as a medium for kernel debugging. The `usbkdp(1)` tool proxies the Kernel Debugging Protocol over USB, as is discussed in Chapter 2.

USB has evolved over the years, with the present version supported by Apple being USB 3.1, through XHCI. The AppleUSBHostPacketFilter.kext is responsible for bringing up a network interface (`XHC20`, `VHC128`, etc), which can be used to capture USB packets over the respective host controller.

Table 1-12: USB related kernel extensions

Kernel Extension	Supports
AppleUSBACM	MacOS: CDC Abstract Control Model
AppleUSBAudio	USB audio device support
AppleUSBCDC	MacOS: Communications Device Class (CDC) networking kext support
AppleUSBCardReader	*OS:
AppleUSBCommon	Common library functions used by most other extensions
AppleUSBDeviceNCM	
AppleUSBDeviceMux	
AppleUSBHSIC	USB High Speed InterChip (Celluar)
AppleUSBHostMergeProperties	
AppleUSBMike	Mikey (USB Audio hardware)
AppleUSBHost[...]	CPU specific (e.g. T8020)
AppleUSBDMM	CDC Modem model
AppleUSBDisplays	USB display driver support
AppleUSB[ECM/EEM]	MacOS: CDC Ethernet [Control/Emulation] Model
AppleUSBEthernet[Host]	USB Ethernet
AppleUSBFTDI	Future Technology Devices International chipset support
AppleUSBMultitouch	USB Multitouch devices
Apple[OSX]USBNCM	CDC Network Control Model
AppleUSBNetworking	General Networking services over USB
AppleUSBSerial	USB serial port support
AppleUSBTopCase	MacOS: Clamshell support?
AppleUSBTDM	Target Disk Mode
AppleUSBWCM	CDC WCM Device Management
AppleUSBiBridge	MacOS: Connectivity with BridgeOS (T1, T2)
IOUSBAttachedSCSI	
IODeviceFamily	Family driver for generic USB functions
IOUSBFamily	Family driver for generic USB functions
IOUSBHostFamily	Family driver for generic USB Host functionality
AppleUSBHostPacketFilter	Packet filtering via XHC* network interfaces
AppleUSB[EU]HCI[PCI]	Enhanced/Universal Host Controller Interface (USB 2.0/1.x)
AppleUSBVHCI	MacOS: Virtual Host Controller Interface
AppleUSBXHCI[PCI]	MacOS: Extensible Host Controller Interface (USB 3.x)
IOUSBMassStorageClass	USB Mass Storage (SD cards, hard drives, etc)
IOUSBMassStorageDriver	

USB Restricted Mode

An important enhancement to iDevice USB behavior in later iOS 11 builds is the introduction of **USB Restricted Mode**. In this mode, USB responsiveness of the iDevice is reduced - possibly entirely voided - when the iDevice is locked and a timeout occurs. This behavioral change was a necessity in the face of USB-based attacks devised by the likes of Cellebrite and Gray Key, who use vulnerabilities in the *OS USB stack and iBoot in order to gain code execution on the device, which can then facilitate brute forcing the user's pass code and recovering private data. The feature can be toggled from the UI, through the "USB Accessories" button under "FaceID & Passcode". It can also be disabled through MDM.

UserEventAgent's com.apple.accessoryd.matching.plugin handles the timout monitor which locks out access, set by default to one hour since the iDevice has been unlocked. The current state of USB Restricted Mode can be found from the IORegistry, through several properties, all under `IOResources`. The `AppleUSBRestrictedMode` boolean denotes the state. If restricted, `AppleUSBRestrictedModeWithReason` holds an integer specifying the cause of the restriction (timeout, policy, etc). The `AppleUSBRestrictedModeTimeout` holds the number of seconds after which restricted mode will be enforced.

The IOAccessoryManager.kext is responsible for enforcing USB Restricted Mode in kernel, and is helped by the private AppleCredentialManager.framework's AppleCredentialManagerDaemon in user mode. The daemon uses several files in /var/root/Library/USBRestricted, shown in Table 1-13:

Table 1-13: The files in /var/root/Library/USBRestricted

File	Purpose
acc-cache	Accessory cache
analytics	
policy	Policy file
splunk.db	Unknown - presumably used during internal testing or MDM

iDevice Connectors

Apple has adamantly refused to adopt either Mini or Micro-USB standards, and so iDevices, since their inception, have used proprietary connectors. At first, it was the 30-pin connector, followed by the lightning connector. Though proprietary, Apple provides complete details of both connector types through its MFi (Made-for-iPhone) hardware partner program. These confidential documents inevitably leak, so the design and implementation of both connector types are well understood.

30-pin

The legacy 30-pin connector can be found on models of the iPhone up to and including the 4S, and iPads before the iPad 2. The connector is obsolete, with the only traces of it (outside the ancient devices) being in hotel room stereo systems with a hard wired 30-pin male, to dock iDevices.

An important functionality of the 30-pin connector was providing a serial port interface to the iDevice. This, coupled with A4 devices' SecureROM vulnerability (which allows passing arguments to and booting any kernel image), proved paramount for iOS jailbreak and security research. The pin-out plans, along with details of how to create a "breakout" board, to expose the serial port pin of the 30-pin connector and create a USB serial to 30-pin cable. By further passing "`kdp_match_name=serial`" and "`debug=0x9`" in the kernel's `boot-args`, it was possible to use a kernel debugger on iOS. Apple countered by removing the option to pass any boot arguments to the kernel.

Lightning

Apple surprised many with the introduction of the Lightning connector, in the iPhone 5 (and later the iPad 3). Many were hoping that if Apple would abandon 30-pin, it would be to join the rest of the world in adopting the Micro-USB standard, and not move to another proprietary connector. But move they have, and to a novel and heavily patented one[8].

The name chosen – Lightning – may be an allusion to it being a companion of sorts to Thunderbolt, which is (currently) the interface of choice in Macs. The connector only has 8-pins, but all are used. Further, it is a fully reversible connector, meaning it can dynamically determine its orientation and re-assign the pins, which are "mirrored". Lightning is also unique in that its connector includes a built-in authentication chip. This is a first, and is meant to discourage 3rd party vendors from creating unapproved cables and accessories outside the MFi program (as well as net Apple royalties from each device or cable sold).

In the iDevice, a special proprietary chip referred to as the TriStar (with a kernel driver in Tristar.kext) acts as a multiplexer over multiple input/output channels, including:

Table 1-14:

Port	Purpose
Debug UART	The serial interface (discussed later)
Baseband UART	Interface with the baseband modem
Accessory UART	Serial targets: Used for accessories exposing a serial connection, or accessory authentication to iDevice (using MFi auth sequence)
SoC USB	USB Host or Target: Connects to USB host, used when charging, or to an accessory.
Baseband Debug USB	Varies with baseband. E.g. Qualcomm QPST
Mikey	USB Audio
JTAG	IEEE 1149.1-1190

When connecting the device to a host over lightning, the TIP gives a code, which allows the host to ask for up to two functions. Charging only cables ask for only one (SoC USB in target mode), but USB OTG (On-The-Go) cables or Apple's various adapters (e.g. Camera Connection Kit) use both SoC USB and Accessory UART, to authenticate and get the device to enter USB Host mode. iDevices contain some of the USB kexts shown in Table 1-12, to facilitate connectivity.

The iPhone 7 increased the importance of lightning, by getting rid of the venerable headphone jack, and routing audio through lightning. That this can be done is obvious (the device can be connected through lightning to external speakers and docks), and it would provide for higher audio quality (though arguably unnoticeable by many). Another advantage would be (further) thinning of the iPhone chassis, and better waterproofing. Though some have speculated that Apple may at some point move to USB-C (the way it adopted it in the 2015 MacBook, and as some Android devices do), Apple seems too invested in Lightning to switch again (and rile accessory makers yet again).

As was with the 30-pin connector, here, too, the existence of a serial port was discovered. The hardware hacker Ramtin Amin was not only able to expose the UART Debug port, but also produce his own version of a "breakout" board, to create a USB serial to lightning cable[9]. Since Ramtin Amin made his cable design public, Apple has countered, and as of 9.x (iBoot 28xx and later), iBoot emits obfuscated debug output over serial, with all its messages (save the banner) replaced by their hash, and any pointers hidden by "`<ptr>`". The kernel can be made aware of the serial port, emitting `dmesg(8)` output, through the `serial=3` boot-args, although iBoot no longer passes arguments to it. Apple likely requires serial port connectivity for its own debugging, or else it would have long been fully disabled by now. It is not implausible that Apple "geniuses" or their developers have the capability of booting into alternate iOS images wherein the serial port is not disabled by software means, and iBoot still responds via the console.

Although Apple has long adhered to its own connectors, the 2018 iPad Pro is the first indication that this might change, with support for a USB-C for the first time ever on an iDevice. Going forward, it is becoming increasingly likely that USB-C will eventually make it into iPhones as well (possibly as soon as the A14 models of 2020), deprecating Apple's Lightning. It is understandable if Apple seeks to delay the inevitable - less so to avoid riling the customer's base (if anything, many would be happy to use generic cables) - but because Lightning earns Apple hefty commissions from its MFi program and licensing.

NVRAM

The notion of **Non Volatile RAM** has long been a staple of UN*X based systems. The NVRAM was commonly implemented as an Electronically Eraseable Programmable Read-Only Memory (EEPROM) chip, which allowed the system administrator to store arbitrary keys and values. "Non volatile" meant that the values thus stored would persist across power cycles, and be available to the system starting with the early stages of its boot process. EEPROM has since evolved to flash memory.

In both MacOS and the *OS variants, access to the NVRAM is abstracted through a dedicated kernel extension, which subclasses the `IODTNVRAM` family (from iokit/Kernel/IONVRAM.cpp). The NVRAM variables exposed are made visible through the IORegistry, wherein the kext creates IODeviceTree:/options, and populates its properties from the NVRAM variable keys. NVRam variables may be defined with null values. In order to delete a variable, therefore, the special value of `@"IONVRAM-DELETE-PROPERTY` is used. Two other special values, `IONVRAM-[FORCE]SYNCNOW-PROPERTY`, are used to flush the kext's object properties into the physical NVRAM backing. The `nvram(1)` utility is part of the open source system-cmds project, and its source provides a clear example of getting, setting and deleting variables.

Common Variables

Variables are used across all Darwin variants to persistently store data ranging from mundane to system critical. XNU's iokit/Kernel/IONVRAM.cpp maintains a list of variable names and their permissions in the `gOFVariables` global. Table 1-15 shows some of these variables and their use.

Table 1-15: NVRAM variables

Consumer	Variable	Use
XNU	`APL,OSXPanic`	Panic information (q.v. Chapter 2)
*OS	`auto-boot?`	Boolean - if false, holds at iBoot instead of booting kernel
	`backlight-level`	Initial backlight
XNU	`boot-args`	Boot arguments passed to kernel (*OS: not in `RELEASE` builds)
iBoot	`bootdelay`	Amount in seconds to delay boot, or 0
	`com.apple.System.boot-nonce`	iBoot APTicket nonce (q.v. I/A)
	`com.apple.System.tz0-size`	Monitor size
XNU (MacOS)	`csr-active-config`	Configurable Software Restrictions (a.k.a) SIP) bitmap
	`csr-data`	Property list containing `kext-allowed-teams` teamID array

NVRAM variables are potentially dangerous. The `IODTNVRAM` family enforces basic permissions in its `::setProperty`, and the implementing flavor-specific kext further subjects all access to MACF checks. The `mac_iokit_check_nvram.._get`, `..set` and `..delete` hooks are called out, and enforced by Sandbox.kext.

MacOS: GUID Namespaces

With the move from PPC to Intel, and from Open Firmware to EFI, MacOS kept its use of NVRAM variables, along with a similar store called the Parameter RAM (PRAM). Apple documents this use in HT204063[10], which describes the key combination (option-command-p-r) used to reset it. NVRAM variables in MacOS are stored in a two level namespace, consisting of a well known GUID, and the NVRAM variable name. There are several well known GUIDs used in MacOS, which are also hard-coded in AppleEFINVRAM.kext's __TEXT.__const.

Table 1-16: The well known EFI GUIDs

GUID	Notes
7C436110-AB2A-4BBB-A880-FE41995C9F82	__ZL18gAppleEFINVRAMGuid
F68DA75E-1B55-4E70-B41B-A7B7A558EA	__ZL35gAppleEFINVRAMTRBSecureVariableGuid
...	__ZL35gAppleEFINVRAMTRBStagingCommandGuid
8BE4DF61-93CA-11D2-AA0D-00E098032B8C	EFI_GLOBAL_VARIABLE_GUID

The AppleEFINVRAM.kext (a plugin kext of AppleEFIRuntime.kext) is used to access variables, and internally applies the GUID filter of `APPLE_BOOT_GUID` so that only NVRAM variables used by MacOS are accessible.

*OS: the `nvrm` namespace

In *OS variants, the NVRAM store is a hidden partition (now NVMe namespace) of 8K on the NAND. The partition is inaccessible as a block device, but the .....NVRAM.kext provides an interface to the store - in the same was it does on MacOS. Unlike MacOS, the store is without a namespace, and variable/values are simply serialized as NULL terminated strings:

Listing 1-17: The **nvrm** namespace of a *OS device

```
# 'sig' is a one byte magic ('0x70' = common, e.g. '0xa1' = panic)
# 'chk' is a one byte checksum of header
# 'len' is specified as uint16_t in units of 0x10 (e.g. 0x2 = 0x20)
#
         sig chk _len_    _______________name_______________
00000000 5a  82  02  00   6e  76  72  61    6d  00  00  00  00  00  00  00  |Z...nvram.......|
          ____adler____   _generation_
00000010 06  93  0b  33   60  01  00  00    00  00  00  00  00  00  00  00  |...3`...........|
         sig chk _len_    _______________name_______________
00000020 70  7c  80  00   63  6f  6d  6d    6f  6e  00  00  00  00  00  00  |p|..common......|
00000030 62  61  63  6b   6c  69  67  68    74  2d  6c  65  76  65  6c  3d  |backlight-level=|
00000040 31  35  32  33   00  63  6f  6d    2e  61  70  70  6c  65  2e  53  |1523.com.apple.S|
00000050 79  73  74  65   6d  2e  66  70    2d  73  74  61  74  65  3d  ff  |ystem.fp-state=.|
00000060 04  3f  ff  0f   00  63  6f  6d    2e  61  70  70  6c  65  2e  53  |.?...com.apple.S|
00000070 79  73  74  65   6d  2e  74  7a    30  2d  73  69  7a  65  3d  30  |ystem.tz0-size=0|
00000080 78  43  30  30   30  30  30  00    6f  62  6c  69  74  65  72  61  |xC00000.oblitera|
00000090 74  69  6f  6e   3d  68  61  6e    64  6c  65  5f  6d  65  73  73  |tion=handle_mess|
000000a0 61  67  65  3a   20  4f  62  6c    69  74  65  72  61  74  69  6f  |age: Obliteratio|
000000b0 6e  20  43  6f   6d  70  6c  65    74  65  0a  00  6f  62  6c  69  |n Complete..obli|
000000c0 74  2d  62  65   67  69  6e  73    3d  4f  62  6c  69  74  54  79  |t-begins=OblitTy|
000000d0 70  65  3a  20   4f  62  6c  69    74  65  72  61  74  65  44  61  |pe: ObliterateDa|
000000e0 74  61  50  61   72  74  69  74    69  6f  6e  2e  20  52  65  61  |taPartition. Rea|
000000f0 73  6f  6e  3a   20  75  6e  6b    6e  6f  77  6e  00  61  63  63  |son: unknown.acc|
00000100 2d  6d  62  2d   6c  64  2d  6c    69  66  65  74  69  6d  65  3d  |-mb-ld-lifetime=|
00000110 39  34  30  00   72  65  73  74    6f  72  65  64  2d  65  78  69  |940.restored-exi|
00000120 74  2d  73  74   61  74  75  73    3d  30  78  31  30  30  00  61  |t-status=0x100.a|
00000130 75  74  6f  2d   62  6f  6f  74    3d  66  61  6c  73  65  00  62  |uto-boot=false.b|
00000140 6f  6f  74  2d   61  72  67  73    3d  00  72  65  73  74  6f  72  |oot-args=.restor|
00000150 65  2d  6f  75   74  63  6f  6d    65  3d  72  65  63  6f  76  65  |e-outcome=recove|
00000160 72  79  5f  6d   6f  64  65  00    72  65  73  74  6f  72  65  2d  |ry_mode.restore-|
00000170 63  68  69  6c   64  2d  66  61    69  6c  75  72  65  73  3d  7b  |child-failures={|
00000180 69  6e  69  74   69  61  6c  3a    45  58  49  54  28  31  29  5b  |initial:EXIT(1)[|
00000190 52  45  42  4f   4f  54  5d  7d    00  00  00  00  00  00  00  00  |REBOOT]}........|
000001a0 00  00  00  00   00  00  00  00    00  00  00  00  00  00  00  00  |................|
*
         sig chk _len_    _______________name_______________
00000820 a1  15  81  00   41  50  4c  2c    4f  53  58  50  61  6e  69  63  |....APL,OSXPanic|
00000830  (0x800 bytes of compressed panic log, or NULLs)
*
         sig chk _len_    ____name ('wwwwwwwwwwww' indicates free)_____
00001030 7f  17  fd  00   77  77  77  77    77  77  77  77  77  77  77  77  |....wwwwwwwwwwww|
00001040 00  00  00  00   00  00  00  00    00  00  00  00  00  00  00  00  |................|
*
00002000
```

As of Darwin 17, *OS's `gOFVariables` are `const`, meaning they are KPP/KTRR protected. This creates a challenge for overwriting them even after a jailbreak - especially important for APTicket nonce setters (q.v. I/A). Clever solutions like Victor Oreshkin's[11], however, work around this.

Configuration Data

Modern machines are full of integrated components, each with its own attributes. The operating system occasionally needs to make use of these attributes when providing diagnostics.

MacOS: The System Management BIOS

Intel architectures generally use the System Management BIOS (SMBIOS) to obtain information about motherboard and peripheral devices. The SMBIOS is an open standard[12], currently in version 3.2.

The SMBIOS data is obtained at boot time from the EFI configuration table (as explained in Chapter 5) by looking for a specific GUID (either that of version 2 or version 3). The SMBIOS data is an array of typed, variable length structures. Each structure derives from a common header, specifying a one byte type, one byte length, and two byte unique (though not necessarily contiguous) handle value. Types 0 through 127 are documented in the specification.

MacOS's AppleSMBIOS.kext is in charge of interfacing with the SMBIOS. The sources of the kernel extension can be found on on Apple's open source repository[13], though the latest version there (42, at the time of writing), seems partial and only references version 2.5 of the specification. Another notable omission are the Apple extensions to the specification, in the form of information elements from 128 onwards - Apple has continued to extend typed data past 132, though the sources do not account for these items.

The kernel extension makes the SMBIOS data readily available in user mode through a property in its IORegistry class. The raw data can thus be obtain with an application of `ioreg`:

Output 1-18: The SMBIOS data from MacOS

```
morpheus@Chimera (~) % ioreg -l -w 0 -f | grep -A .... "o AppleSMBIOS "
    | +-o AppleSMBIOS
    |     {
    |       "IOProbeScore" = 0
    |       "CFBundleIdentifier" = "com.apple.driver.AppleSMBIOS"
    |       "IOMatchCategory" = "AppleSMBIOS"
    |       "IOClass" = "AppleSMBIOS"
    |       "IOProviderClass" = "IOPlatformDevice"
    |       "SMBIOS" =
    |         00000000: 07 13 00 ....13 01 ........... . .....L1 Cache.....
    |         00000020: 00 01 80 ....01 81 ........ . .....L1 Cache........
    |         00000040: 01 00 04 ....00 20 ..... . .....L2 Cache..........
    |       "IONameMatch" = "bios"
    |       "IONameMatched" = "bios"
    |       "SMBIOS-EPS" = <5f534d5f461f03003901000000000005f444d495f7f5b0b00e0e87a2a0030>
    |     }
```

MacOS's system_profiler(8) can be used to obtain the SMBIOS data in a more readable form. As explained in I/4, system_profiler(8) and its GUI use .sbreporter plugins to get and process their data. The SPDiagnosticsReporter uses the `SmbiosParser` objective-C class to harvest the data from the IORegistry entry. The `smut(j)` tool, available on the book's companion web site, provides an open sourced example of parsing this data programmatically. A sample of this tool (from a T2 device, with an emulated SMBIOS) can be found in 5-21.

Though rare, SMBios data might be corrupted, leading to various problems, documented by Apple in HT201295[14]. T2-enabled devices (2017 and later Macs) have their SMBios emulated by the T2 chip. This is discussed later in Chapter 5.

*OS: SysCfg

Apple's iDevices store their system configuration data in a dedicated flash partition (nowadays NVMe namespace) called 'scfg'. As with other system reserved partitions, the syscfg namespace also isn't mountable or readily accessible. The *OS kernelcache contains the `AppleDiagnosticDataAccessReadOnly` kernel extension, which conveniently exports the raw data through its IORegistry entry. The `AppleDiagnosticDataSysCfg` property contains the partition's entire 0x20000 bytes, which can be dumped with `ioreg(8)`'s `-l` and `-f` switches. As the name implies, the kernel extension does not allow making any modifications in the partition, but there is no reason any code in kernel mode (or iBoot) couldn't do so.

The SCfg header format is straightforward to figure out (what more, it has been described in the leaked iBoot sources, as well), as shown in Table 1-19:

Table 1-19: Decoding the SCfg header

Offset	field	Purpose
0x0	magic	'scfg', in little endian form
0x4	size	Size of blob (below, 0x504)
0x8	maxSize	Maximum size (i.e. partition size - below, 0x20000)
0xC	version	SCfg version (below, 0x20002, denoting 2.2)
0x10	bigEndian	1 if little endian (below, 0, denoting little endian)
0x14	keyCount	Count of following containers (above, 0x3f)
Container (out-of-line key) format		
0x0	magic	'CTNB' in little endian form
0x4	tag	key, in little endian form
0x8	dataSize	Overall data size
0xc	dataOffset	Offset (from header) of data
0x10	reserved	-1 (0xffffffff)
Inlined data key format		
0x0	tag	key, in little endian form
0x4	data	16-bytes of inlined data

Listing 1-20 demonstrates a partial dump of the SCfg namespace from an iDevice. By applying the fields from Table 1-19 to the listing you can see the 'MLB#' container (pointing to 17 bytes at offset 0x01ffec), followed by the inlined keys of 'WMac', and 'BMac', containers of 'CBAT' (49 bytes at 0x1ffb8), 'NFCI' (32 bytes at 0x1f754), and an inlined 'SrNm'.

Listing 1-20: A sample dump of SCfg from an iDevice

```
00000000  67 66 43 53  04 05 00 00   00 00 02 00  02 00 02 00  |gfCS ....  .... ....|
00000010  00 00 00 00  3f 00 00 00   42 54 4e 43  23 42 4c 4d  |.... ?...  BTNC #BLM|
00000020  11 00 00 00  ec ff 01 00   ff ff ff ff  63 61 4d 57  |.... ....  .... caMW|
00000030  cc 08 8d b7  48 94 00 00   00 00 00 00  00 00 00 00  |.... H...  .... ....|
00000040  63 61 4d 42  cc 08 8d b7   48 95 00 00  00 00 00 00  |caMB  .... H... ....|
...
00000090  42 54 4e 43  6c 43 46 4e   20 00 00 00  54 f7 01 00  |BTNC lCFN   ... T...|
000000a0  ff ff ff ff  6d 4e 72 53   46 34 47 53  45 55 31 32  |.... mNrS  F4GS EU12|
000000b0  48 47 31 58  00 00 00 00   42 54 4e 43  74 74 61 42  |HG1X ....  BTNC ttaB|
000000c0  11 00 00 00  40 f7 01 00   ff ff ff ff  .. .. .. ..  |.... @....  |
...
0001ffb0  3d 67 16 80  00 01 ad 00   9c ff 63 00  70 66 64 00  |=g.. ....  ..c. pfd.|
0001ffc0  16 fe e9 01  60 7d ea 01   0c fe f3 01  67 54 fb 01  |.... `}..  .... gT..|
0001ffd0  22 fc dd 03  fe e3 e5 03   18 fc e7 03  d5 e6 fe 03  |"... ....  .... ....|
0001ffe0  30 f8 cf 07  9f a2 02 08   03 00 00 00  43 32 48 36  |0... ....  .... C2H6|
0001fff0  31 38 36 31  35 48 34 48   36 54 46 41  32 00 00 00  |1861 5H4H  6TFA 2...|
```

Much of the data is made available in the far simpler form of keys and values through the private `libMobileGestalt` (described in I/4). The `SysCfg` key provides the same raw data of the partition and the IORegistry property, and the `SysCfgDict` further breaks it up into a `CFDictionary`. None of the methods, however, bother to provide an explanation of the 32-bit key container keys. Table 1-21 shows a few of those:

Table 1-21: Some of the *OS SysCfg keys

Key	MobileGestalt Key	Value
DClr	DeviceRGBColor	Device Color (RGB value)
Batt	?	Battery serial number
BMac	BluetoothAddress	Bluetooth module MAC Address (6 bytes)
CFG#		Configuration Number (inlined string)
MLB#	MLBSerialNumber	Main Logic Board Number (string)
[B/F]CMS	[Back/Front]CameraModuleSerialNumString	Back/Front camera serial number.
NvSn	MesaSerialNumber	Mesa (TouchID) sensor serial number
SrNm	SerialNumber	Phone Serial Number (13-byte inlined string)
SwBh	SoftwareBehavior	Software Behavior
WMac	WifiAddress	WiFi module MAC Address (6 bytes)
EMac	EthernetAddress	Ethernet (tethering) MAC Address (6 bytes)
[B/W]Sn	[Baseband/Wireless]BoardSnum	Baseband/Wireless Board Serial Number

The Device Tree

The Device Tree is a standardized data structure which enables providing the kernel with a detailed description of a system's hardware components. The idea can be traced back to OpenFirmware, which was first used in MacOS during the PPC age. Device Trees survived the move to Intel, and it was only natural to use them in iOS - especially considering other ARM-based operating systems (notably, Android) use them as well.

Serialized format

Whereas Android vendors have standardized on the .dtb (Device Tree Binary) format for specifying the device tree, Apple uses a custom format. On *OS devices the device tree is embedded in an IM4P DER container like other IPSW files (as discussed in Chapter I/A), and immediately after the IMG4 header starts the tree data, as an opaque octet string. The tree nodes are internally serialized in a simple and unaligned format. Each node is specified (starting at the root) specifies a count of propertes, and of child nodes. Each property is specified by a 32-byte name, a 28-bit size, 4-bit flags (of which only the placeholder bit is defined), and a variable length (per specified size) property value.

A special case of property values are placeholders, indicated by the high order bit flag, and either prepopulated with NULL bytes, and/or loaded from SysCfgData. It is common to find values specified as `syscfg/container[/0x##[,zeroes/0x##]]`, with *container* a four letter SCfg entry, and optional byte count in hex. The optional `zeroes` which may follow indicates the placeholder needs to be initialized with the specified number of NULL bytes. Listing 1-22 shows a sample device tree:

Listing 1-22: The DeviceTree.d321ap.im4p file from an iOS 12.1.1 .ipsw

```
00000000  30 83 02 d5 d9 16 04 49  4d 34 50 16 04 64 74 72  |0......IM4P..dtr|
00000010  65 16 1e 45 6d 62 65 64  64 65 64 44 65 76 69 63  |e..EmbeddedDevic|
00000020  65 54 72 65 65 73 2d 33  38 37 31 2e 32 33 30 2e  |eTrees-3871.230.|
          octet ___len__
00000030  35 04 83 02 d5 a8
                           ___#props___ _#children_
                           12 00  00 00 11 00 00 00 74 61  |5.............ta|
00000040  72 67 65 74 2d 74 79 70  65 00 00 00 00 00 00 00  |rget-type.......|
00000050  00 00 00 00 00 00 00 00  00 00 00 00 00 05 00     |...............|
00000060  00 00 44 33 32 31 00 00  00 00 6d 6c 62 2d 73 65  |..D321....mlb-se|
00000070  72 69 61 6c 2d 6e 75 6d  62 65 72 00 00 00 00 00  |rial-number.....|
00000080  00 00 00 00 00 00 00 00  00 00 1d 00 00 80 73 79  |..............sy|
00000090  73 63 66 67 2f 4d 4c 42  23 2f 30 78 32 30 2c 7a  |scfg/MLB#/0x20,z|
000000a0  65 72 6f 65 73 2f 30 78  32 30 00 00 00 00 00 63  |eroes/0x20....co|
000000b0  6d 70 61 74 69 62 6c 65  00 00 00 00 00 00 00 00  |mpatible........|
000000c0  00 00 00 00 00 00 00 00  00 00 00 00 00 00 1b 00  |................|
000000d0  00 00 44 33 32 31 41 50  00 69 50 68 6f 6e 65 31  |..D321AP.iPhone1|
000000e0  31 2c 32 00 41 70 70 6c  65 41 52 4d 00 00 73 65  |1,2.AppleARM..se|
000000f0  63 75 72 65 2d 72 6f 6f  74 2d 70 72 65 66 69 78  |cure-root-prefix|
00000100  00 00 00 00 00 00 00 00  00 00 00 00 00 03 00     |................|
00000110  00 00 6d 64 00 00 41 41  50 4c 2c 70 68 61 6e 64  |..md..AAPL,phand|
00000120  6c 65 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |le..............|
00000130  00 00 00 00 00 00 04 00  00 00 01 ...
             ... properties/values follow .....
000003c0                                          6e 61  |              na|
000003d0  6d 65 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |me..............|
000003e0  00 00 00 00 00 00 00 00  00 00 00 00 00 00 0c 00  |................|
000003f0  00 00 64 65 76 69 63 65  2d 74 72 65 65 00        |..device-tree.

          Next node begins immediately after last property
           of current node is done (No alignment)
00000000                                          31 00  |              1.|
00000400  00 00 04 00 00 00 63 72  79 70 74 6f 2d 68 61 73  |......crypto-has|
00000410  68 2d 6d 65 74 68 6f 64  00 00 00 00 00 00 00 00  |h-method........|
00000420  00 00 00 00 00 00 09 00  00 00 73 68 61 32 2d 33  |..........sha2-3|
00000430  38 34 00 00 00 00 68 6f  75 73 69 6e 67 2d 63 6f  |84....housing-co|
00000440  6c 6f 72 00 00 00 00 00  00 00 00 00 00 00 00 00  |lor.............|
...
```

The `IODeviceTree` IORegistry plane

Once loaded by the kernel, the device tree spans a dedicated IORegistry plane called `IODeviceTree`. IORegistry planes are detailed in Chapter 13, but at this point suffice it to say that the two representations are just about isomorphic. The device tree can thus be seen on an active machine by using `ioreg(8)` and specifying the `IODeviceTree` plane, or by doing so programmatically. This is far simpler in most cases*, and can be done on any device via `sysdiagnose(1)`, which executes /usr/sbin/ioreg(8) as part of its diagnostics suite when the magic key/button sequence is pressed (q.v. I/15). The registry representation does "taint" the device tree output with `IO*` keys and values, but those stand out in Output 1-23:

Output 1-23: The output of `ioreg(8)` showing the device tree from the previous listing

```
....
 +-o device-tree  <class IORegistryEntry:IOService:IOPlatformExpertDevice, id 0x1000001e5,  ...
   | {
   |   "#address-cells" = <02000000>
   |   "AAPL,phandle" = <01000000>
   |   "serial-number" = <433339584145758424b5046570000000000000000000000000000000000000000>
   |   "IOBusyInterest" = "IOCommand is not serializable"
   |   "target-type" = <"D321">
   |   "platform-name" = <74383032300000000000000000000000000000000000000000000000000000000>
   |   "secure-root-prefix" = <"md">
   |   "region-info" = <4c4c2f41000000000000000000000000000000000000000000000000000000000>
   |   "name" = <"device-tree">
   |   "manufacturer" = <"Apple Inc.">
   |   "compatible" = <"D321AP","iPhone11,2","AppleARM">
   |   "config-number" = <2f2f2f2f2a2f2a000000000000000000000000000000000000000000000000000>
   |   "IOPlatformSerialNumber" = "C39XDWXBKPFW"
   |   "regulatory-model-number" = <413139323000000000000000000000000000000000000000000000000000000000>
   |   "clock-frequency" = <00366e01>
   |   "model" = <"iPhone11,2">
   |   "model-number" = <4d5439393200000000000000000000000000000000000000000000000000000000>
   |   "mlb-serial-number" = <473243383336373036355534a564e393400000000000000000000000000000000>
   |   "IONWInterrupts" = "IONWInterrupts"
   |   "device_type" = <"bootrom">
   |   "#size-cells" = <02000000>
   |   "IOPlatformUUID" = "61D1C054-4615-5E5B-9807-2D30163FCBEE"
   | }
   |
   +-o chosen
   | | {
   | |   "effective-production-status-ap" = <01000000>
   | |   ...
```

Device tree support is built into XNU, and the creation of the in-memory representation is handled by `IODeviceTreeAlloc()` (iokit/Kernel/IODeviceTreeSupport.cpp), which is called early during kernel initialization from `IOPlatformExpertDevice::initWithArgs()`, itself called by `StartIOKit()`. The serialized device tree is passed to the kernel as part of the `boot_args` structure (explained in Chapter 2), where it is hung on the Platform Expert's `PE_state` global (as `deviceTreeHead`). The Device Tree's "/chosen" node is the one containing the boot configuration and most other information which XNU later processes.

---

* - There is still a benefit in understanding the device tree in its serialized IM4P container, in cases where code execution cannot be obtained - on a non-jailbroken iDevice or devices which can't even run 3rd party code, like the HomePod

Dedicated Processors

As explained earlier, the Application Processor running *OS is only one of several processor cores working together on the logic board. Each of these runs its own operating system, aimed at performing the minimal set of functions governed by the processor. The Appendix of Volume I (v1.3) introduced these operating system images, and we now turn to inspecting them in detail.

Common Code: RTKit

RTKit is Apple's embedded RTOS framework, and unlike other "Kits" it is only used internally. Nearly all of Apple's dedicated processor images are Mach-O preloaded executables (`MH_PRELOAD`), and even those aren't (with the exception of the ISP and SEPOS firmware) show signs of being compiled with RTKit.

The RTKit Mach-O binaries are not unlike the common binaries of Darwin, with the same, well known `__TEXT` and `__DATA` segments. For lack of `dyld`, however, there is neither a `__LINKEDIT` nor an `LC_MAIN` load command, and the entry point (indicated by `LC_UNIXTHREAD`) is set to the beginning of the `__TEXT.__text` segment. Unlike regular Mach-Os (in which the `__TEXT` technically includes the Mach-O header), the `__TEXT.__text` address is aligned on a page boundary, is usually (outside the AOP) mapped to 0x0, and holds the AARCH64 Exception vector (refreshed later in Chapter 4, under "Processor excecution mode").

Output 1-24 demonstrates the above. Although taken from the AGX firmware (where some symbols remain), RTKit startup is the same in all images. Note a few instructions into the flow, wherein the loaded address is set to the value of `VBAR_EL1`. There we find a temporary vector, whose only active entry is the CurrentEL, SP0 Synchronous - and the rest halt. Later during the flow, however, other values for the exception vector will be set.

Output 1-24: Disassembling the initial `__TEXT` of the AGX firmware

```
morpheus@Bifröst (~)$ jtool2 -l ~/Documents/iOS/13/Firmware/agx/armfw_g11g.im4p | grep UNIX
LC 04: LC_UNIXTHREAD                Entry Point:              0x0
#
# Use NOPSUP=1 to suppress NOP and DCD 0x0 instructions (empty vector instructions)
#
morpheus@Bifröst (~)$ NOPSUP=1 jtool2 -d ~/Documents/iOS/13/Firmware/agx/armfw_g11g.im4p
0       0x14000081  B     0x204                     __rtk_armv8_reset
80      0xd538521c  MRS   X28, ESR_EL1
84      0xd538601d  MRS   X29, FAR_EL1
88      0xd538403e  MRS   X30, ELR_EL1
8c      0x14000000  B     0x0
100     0xd538521c  MRS   X28, ESR_EL1
104     0xd538601d  MRS   X29, FAR_EL1
108     0xd538403e  MRS   X30, ELR_EL1
10c     0x14000000  B     0x0
180     0xd538521c  MRS   X28, ESR_EL1
184     0xd538601d  MRS   X29, FAR_EL1
188     0xd538403e  MRS   X30, ELR_EL1
18c     0x14000000  B     0x0
200     0x14000000  B     0x0
#
# Make sure our current EL is NOT 3 - if it is, drop by ERET
#
204     0xd5384241  MRS   X1, CurrentEL
208     0xd3420c21  UBFX  X1, X1#3
20c     0xf1000c3f  CMP   X1, #3
210     0x540000c1  B.NE  0x228
214     0x100000a1  ADR   X1, #20                 R1 = 0x228
218     0xd51e4021  MSR   ELR_EL3, X1             X -  (1 0x228)
21c     0xd28078a1  MOVZ  X1, 0x3c5                ; R1 = 0x3c5
220     0xd51e4001  MSR   SPSR_EL3, X1            X - A,I,F,BigE (1 0x3c5)
224     0xd69f03e0  ERET
#
# EL1 startup continues - VBAR_EL1 set to vector at beginning of image (0x0)
#
228     0xd50343df  MSR   DAIFSet, #3             X -  (0 0x0)
22c     0xaa0003e7  MOV   X7, X0                  R7 = R0 (0x0)
230     0x10ffee80  ADR   X0, #-560               R0 = 0x0
234     0xd518c000  MSR   VBAR_EL1, X0   Vector Base Address Register  set to 0x0
...
4a4     0xd518c000  MSR   VBAR_EL1, X0   Vector Base Address Register  set to 0x4000
..
26b2c   0xd518c008  MSR   VBAR_EL1, X8   Vector Base Address Register set to 0x4800 - __rtk_arch_vectors
```

An `LC_SYMTAB` exists, and (with the notable exception of the AOP) most other RTKit binaries have not been stripped. Because so much common code is reused in between the binaries, it is possible to partially resymbolicate the AOP image, as well. RTKit symbols are C, but additional symbols in the various binaries show heavy use of C++ all throughout.

When generating Mach-O binaries, RTKit defines additional sections within the segments, which include:

Table 1-25: The RTKit specific sections

Section	Contents
`__TEXT.__rtk_mtab`	RTKit device dispatch tables
`__DATA.__rtk_init_stack`	The initial, IRQ and exception stack areas. These are each one page (0x1000), and initialized with "RTKSTACK" patterns, repeated
`__DATA.__rtk_irq_stack`	
`__DATA.__rtk_exc_stack`	
`__DATA.__rtk_ext_stack`	Extra stack space (e.g.for the AOP firmware).
`__DATA.__rtk_boot`	Usually one page, aligned, zeroed out
`__DATA.__page_tables`	Variable sized (zeroed out) area for page table maintenance
`__DATA.__patchbay`	4-byte ASCII "containers" followed by 32-bit length specification and that number of bytes, filled during runtime with values.
`__DATA.__power`	Regular and floating point register contexts, used when hibernating/resuming
`__DATA.__threads`	empty
`__DATA.__rtk_heap`	empty

As the table shows, most of these areas are either empty or initially zeroed out, as they are, in effect, reserving spaces for various data structures which initialize during runtime. The few sections which aren't zeroed out are:

- The `__TEXT.__rtk_mtab` section stores multiple function pointers. The AGX firmware, which remains unstripped, reveals those to be belonging to devices, such as `_RTK_dev_mailbox128_dispatch`, `_RTK_dev_timesync_dispatch`, `_RTK_dev_syslog_mbi_dispatch`, and others. Each device's dispatch table may be of variable size, but the first three entries - one for device control and the other two unimplemented (as `__rtk_dev_bad_method`) - imply that they all inherit from the same vtable superclass or parent structure. The device definition structures are in `__DATA.__data`.
- The `__DATA.__rtk_patchbay` section stores values which are patched during runtime (by the *OS kernel, when loading the firmware). These are variable length entries in "containers" (similar to those of SysCfg), wherein each value is specified by a 4 byte (7-bit ASCII) "tag", a 4-byte length, and a variable length value, initially zeroed. Table 1-26 shows the known values. The AOP contains quite a few more, but there is little information about them:

Table 1-26: Containers in `__DATA.__rtkpatchbay`

Tag	Contents	Tag	Contents
`CpAd`	CPU Physical address	`DSZS`	RTKit zero fill start
`WrAd`	CPU wrapper physical address	`DSZL`	RTKit zero fill length
`MCRA`	MC remote alignment	`IGNT`	ARMv8 ignore NT hint
`SOC_`	SoC	`..Fq`	ARMv8 timer frequency
`SOCR`	SoC Revision	`T1Ps`	MMU TTBR1 private space size
`DSCL`	Data segment copy length	`PTPB`	Page table base
`DSRL`	Data segment relocate	`PTPS`	Page table size
`DSVS`	Data segment virtual source	`PTVB`	Page table virtual base
`RTSZ`	RTKit runtime size	`GptB`	UAT gbtbat base
`STKG`	RTKit stack guard value	`PplB`	UAT PPL shared base

- The `__DATA.__power` section is used to store hibernation context and data. The section contents are consistent (and similarly ordered) in all RTKit firmwares, shown in Table 1-27:

Table 1-27: The symbols in `__DATA.__power`

Symbol	Description
`rtk_crt_mmu_addr_space_size_log2`	MMU address space (AOP:0x1e, AGX/AVE: 0x1f, ANE:0x22)
`rtk_crt_mmu_select_bitmap`	AOP/AGX/ANE/AVE: MMU bitmap selection (0x3)
`rtk_crt_mmu_select`	AOP/AGX/ANE/AVE: Uninitialized (16 bytes)
`__hb_context`	General purpose (X0-X31) register spill (0x128 bytes)
`__fp_context`	Floating point (Q0..) register spill (0x200 bytes)
`_arm_[hb/fp]_context`	Offsets of `_arm[hb/fp]_context`
`_arm_[hb/fp]_context_addr`	pointers to `_arm_[hb/fp]_context`
`_arm_hb_[wake/sleep]_count`	Wake and sleep counters, 4 bytes (each)
`_arm_resume_flag`	0xfeed1b00
`_arm_resume_address`	Address of power resume function

When the system hibernates, `_arm_arch_hibernate()` increments the `_arm_hb_sleep_count` and spills the registers into the `__[hb/fp]_context` areas, and when the system resumes, `_arm_arch_resume()` is called from the `_arm_resume_address` pointer, and reloads the registers.

The *OS Side: RTBuddy

The com.apple.driver.RTBuddy kernel extension is responsible from the *OS side for interfacing with the additional processors.

The RTBuddy kext traditionally provided two user clients:

- **`RTBuddyLoaderUserClient`**: provides three methods - `LoadFirmware`, `LoadImage4Bundle` and `ReportError`. This has been removed as of *OS 12.
- **`RTBuddyUserClient`**: provides four methods - `[Set/Get]SyslogVerbosity`, `ReloadFirmware`, and `NMICoprocessor`.

A dedicated daemon, /usr/libexec/rtbuddyd, was used on and off over several *OS versions. It was removed around iOS 9, only to appear again as of 10. The daemon not only possesses the usual LaunchDaemon property list, but is also started by `launchd`'s `__TEXT.__config` with the `--early-boot` flag. It is set to look for a property list file, through which it loads the coprocessor firmwares (with the help of the `RTBuddy.kext`'s `IOUserClient`).

As of *OS 12, rtbuddyd seems to have been deprecated again (and, as of *OS 13, apparently removed once more), and the `RTBuddyLoaderUserClient` is no more. The firmware loading functionality has apparently been put into MobileAccessoryUpdater.framework's fud, which is now started from launchd's `__TEXT.__config` with `--early-boot`. During the OS install process, iBoot takes the .im4p images from the .ipsw and copies them over to /usr/standalone/firmware/FUD as their three letter acronyms (ANE,AOP, GFX, ISP and SIO) with an .img4 suffix. `AudioCodecFirmware.img4` and `Multitouch.img4` are placed there as well. The `fud` daemon then locates the images, and loads the `FUDGenericKextUpdater.bundle` (plugin) from /System/Library/AccessoryUpdaterBundles. The plugin enables `fud` to communicate with the AppleFirmwareUpdateKext extension, and load the firmwares to their processors.

Even with reduced `IOUserClient` functionality, the RTBuddy.kext has another important aspect to it - it uses the `IOReportFamily` in order to provide detailed coprocessor information, including CPU utilization and thread status. The `RTBuddy` class provides the `IOP State` report, and the `IOReportingEndpoint` class provides `Performance, [Used/Total] stack` and other processor-specific runtime information, such as thread states.

AOP

The Always On Processor (codenamed "Borealis Owl", but better known by Apple's Marketing Name, "M#", with a number matching that of the application processor) is a low power coprocessor which, as the name implies, stays on even when the main processor is sleeping. This, the oldest of the dedicated processors, was originally a 32-bit processor (apparently codenamed "Oscar") in the M7 (accompanying the A7), but as of later devices it is a 64-bit processor.

As it evolved, the AOP took more and more responsibilities from the main AP, managing most of the other dedicated processors. Thus, after taking over the compass, gyro, accelerometer and pressure, Siri functionality was added in the M9, touch to wake in the M10, pearl (FaceID) support in A11, and rose (UWB) support in the A13.

The AOP exposes numerous (by the A13, about 20) **endpoints**, through which its services can be accessed from iOS. The endpoints are clearly visible in the output of `ioreg(8)` as shown below. They can be accessed by user-mode clients through the `AppleSPUHIDInterfaces`. They can also be found in the AOP firmware binary in the `__DATA._spu_service` and `__DATA._spu_endpoint` sections.

Output 1-28: AOP Endpoints, as shown in `ioreg`

```
root@LuckyA13# ioreg -l -w 0 -f      | # Dump IOServices
pipe>          grep AppleSPUInterf   | # Filter AppleSPUInterface instances
pipe>          grep '+'              | # just actual instances
pipe>          cut -d'+' -f2         | # trim leading spaces, etc
pipe>          cut -d' ' -f1,2       # Just name, please
-o SPUApp
-o AOP_spherecontrol
-o smc-control
-o i2c
-o AOP_baseband
-o accel
-o gyro
-o compass
-o pressure
-o wakehint
-o gesture
-o orientation
-o cma
-o devmotion
-o aop-audio
-o AOP_profile
-o prox
-o als
-o aop-voicetrigger   # "Hey, Siri"
-o romeo              # FaceID camera
-o juliet             # FaceID module
-o ttw                # Touch to wake(?)
-o rose               # U1 UWB control
-o devmotion6
-o devmotion10
-o rose-supervisor    # U1 USB control
```

The AOP image is, by far, the most complex of all the firmware images, with over a dozen threads (for the various endpoints), drivers and more. Unfortunately (and maybe because of its complexity), it is also stripped. The other firmware images, however, are not - so some of its symbols can be recovered through analysis. `jtool2`'s `--analyze` will do exactly just that.

AGX

AGX provides three classes - `AGXDevice`, `AGXSharedUserClient` and `AGXCommandQueue`. These are functionally the same as the trio provided by the MacOS `IntelAccelerator` instance (`com.apple.driver.AppleIntelKBLGraphics` or other, depending on the specific hardware present).

The Apple Video Encoder (AVE)

The AVE processor - presumably, Apple Video Encoder - is a dedicated processor charged with handling the HEVC (High Efficiency Video Codec) compression.

ISP

The **Image Signal Processor**, codenamed Petra in the A12 and Zelus in the A13, is responsible for real-time image manipulation (Bokeh and other effects).

The iOS kernel uses the `com.apple.driver.AppleAVE` kernel extension to communicate with the AVE.

ANE

The Apple Neural Engine (ANE) was introduced in the A11 as a dual-core processor to support on-device AI and ML tasks. Initially, it was able to provide 300 billion FLOPS. With the A12 the processor (codenamed " ") expanded to eight cores, and boasted 5 trillion FOPS (and, of course, "Memoji" support). A13 ("metis") further improves performance by 20%, while reducing power consumption by 15%.

The iOS kernel uses the `com.apple.driver.AppleH11ANEInterface` kernel extension to communicate with the ANE. Applications can draw on the ANE's formidable power through the `CoreML.framework` APIs.

PMP

The PMP is a special RTKit image - rather than the usual Mach-O, it is embedded as a raw image in the `__DATA.__fwFirmwareImage` section of A12. Unlike the other images (which used to be loaded by `rtbuddyd` and are now loaded by `fud`), the PMP image is handled by the `AppleT80xxPMPFirmware.kext`. The kext hard codes the address of the section containing the firmware, and creates a (rather large) `OSString` out of it. This is then subjected to patching, and then loaded onto the PMP. In the A13 iOS13 image the image is again a separate img4-encapsulated 64-bit Mach-O, in `Firmware/t8030pmp.im4p`.

Rose

The Ultra Wideband Chip Apple calls "U1" was introduced as part of the A13 board, with no formal announcement by Apple. Its use appears to be (at the time of writing) limited to enhancing AirDrop, by providing directional support. It is more than likely, however, that this chip's purpose is for far more precise iDevice location - up to a few feet, inter-device communication, and possibly supporting the newly announced "Find My", which could be extended to Apple provided tags.

External Processors

There are other processors still on the iDevice boards, but those are not ones manufactured by Apple.

The Secure Element

The Secure Element is a key component of the Near Field Connectivity (NFC) architecture. Apple adopted NFC to support Apple Pay, and uses an NXP industries chip for this purpose. The chip is referred to internally as "Stockholm", and the current version of the firmware is 5. The firmware file is an .sefw suffixed file in DER format, and (like IMG4 containers) can be unpackaged with `openssl asn1parse`. Doing so reveals multiple sections, starting with a Gilbert & Sullivan easter egg, and followed by image components.

The `com.apple.driver.AppleStockholmControl` is responsible for interfacing with the SE. The extension provides a user client, called on by nfcd, and known to expose the following methods:

Table 1-29: The methods exposed by the `AppleStockholmControlUserClient`

0	`setEnable`
1	`setDownloadRequest`
2	`setPMUStandbyPowerEnable`
3	`setDeviceWake`
4	`setIcefallDownloadRequest`
5	`SetVBATEnable`

The private NearField.framework of *OS variants allows Apple's own clients to interface with the driver, via the `nfcd` and `seld` daemons (both in /usr/libexec). Another binary of interest is /usr/libexec/nfrestore_service, which uses /usr/lib/libnfrestore.dylib to restore Stockholm firmware from /usr/standalone/firmware/nfrestore/firmware/fw

The Baseband

Apple has shifted several times between Intel and Qualcomm as the baseband providers for the iPhones and cellular iPads. The older iPhone models used Infineon (later to be acquired by Intel), and around iOS 5 Apple shifted to Qualcomm. With the iPhone 7 Intel basebands made a comeback, following a patent dispute with Qualcomm. Just as the case was brought to court, the two sides surprisingly settled, Apple announced Qualcomm's reinstatement as a provider till 2025 (perhaps as early as the A13). This, however, might prove short lived - Apple followed to announce the acquisition of Intel's 5G division in late June 2019, so it is likely that future generations after that will use a home grown solution.

This means that iPhone models from iPhone4,x through iPhone8x (and their corresponding iPads) contain a Qualcomm baseband (an MDM96xx, referred to as "Maverick" or "MAV")*. iPhone 9,x and iPhone 10,x may contain either a Qualcomm baseband or an Intel one (PMB99xx, known as "Intel Communication Equipment", or simply "ICE"). The end-user can tell which baseband is in a given device through the device model number (e.g. A1863/A1864 for the Qualcomm 8/8+, and A1905/A1897 for the Intel 8/8+). iPhone 11,x and iPhone12,x use Intel.

Owing to the two different basebands used, the property list contents are also different. Both variants are in binary form, and contain hashes for all the various subcomponents, which naturally differ as well. This is shown in Output 1-30.

* - The QCOM baseband is essentially the same as the one in SnapDragon Android devices. This has interesting security ramifications, as a baseband-level exploit would affect both devices (via the initial payload) in the same way.

Output 1-30: The ICE and MAV baseband firmwares

```
$ unzip -l ICE18-1.02.18.Release.bbfw
Archive:  ICE18-1.02.18.Release.bbfw
  Length      Date    Time    Name
---------  ---------- -----   ----
   242660  11-07-2018 01:07   ebl.bin
  5745372  11-07-2018 01:07   LTEFW.elf
   405484  11-07-2018 01:07   psi_ram.bin
   405484  11-07-2018 01:08   restorepsi.bin
 50481462  11-07-2018 01:07   SYS_SW.elf
  1687692  11-07-2018 01:07   TDSFW.elf
   495513  11-08-2018 00:21   bbcfg.bin
      996  11-07-2018 01:08   Info.plist
   118850  11-07-2018 01:07   2GFW.elf
  2057660  11-07-2018 01:07   3GFW.elf
   737596  11-07-2018 01:07   AudioFW.elf
     1104  11-07-2018 01:07   Debug_info.elf
   107028  11-07-2018 01:07   CDMA2FW.elf
  1586594  11-07-2018 01:07   RPCU.elf
 35490807  11-07-2018 01:07   RFFW.elf
   831589  11-07-2018 01:07   DPC.elf
    17255  11-07-2018 01:07   custpack.elf
        0  05-14-2018 20:34   Options.plist
$ unzip -l Mav16-3.21.01.Release.bbfw
Archive:  Mav16-3.21.01.Release.bbfw
  Length      Date    Time    Name
---------  ---------- -----   ----
   131112  09-26-2017 03:38   acdb.mbn
  3023560  09-26-2017 03:38   apps.mbn
   197312  09-26-2017 04:21   mba.mbn
 57659684  09-26-2017 04:21   qdsp6sw.mbn
   157424  09-26-2017 04:22   rpm.mbn            # Resource/Power Mgmt
   231360  09-26-2017 04:24   sbl1.mbn           # Secondary boot loader
   390176  09-26-2017 04:23   tz.mbn             # TrustZone image
   231360  09-26-2017 04:24   restoresbl1.mbn
     1174  09-26-2017 04:24   Info.plist
  1052753  10-03-2017 01:01   bbcfg.mbn
        0  09-29-2017 16:21   vinyl/
        0  09-29-2017 16:21   vinyl/recovery/
        0  09-29-2017 16:21   vinyl/update/
        0  09-29-2017 16:21   vinyl/update/gold/
   472187  09-19-2016 12:57   vinyl/update/gold/firmware.der
      538  09-29-2017 16:21   vinyl/update/gold/info.plist
      515  09-19-2016 12:57   vinyl/update/gold/signedprofile.der
        0  09-29-2017 16:21   vinyl/update/main/
   592403  09-19-2016 12:57   vinyl/update/main/firmware.der
      538  09-29-2017 16:21   vinyl/update/main/info.plist
      515  09-19-2016 12:57   vinyl/update/main/signedprofile.der
        0  02-13-2017 20:07   Options.plist
```

`libMobileGestalt` provides an excellent interface to many baseband details. The `BasebandChipset` key is a quick way to determine Intel (e.g. `ice18`) or Qualcomm (e.g. `mav16`). The `BasebandFirmwareVersion`, `..ManifestData`, and `..UpdateInfo` can be used for firmware details. The `Baseband[UniqueId/SerialNumber]` both return the unique baseband serial.

User mode management: `CommCenter` and friends

The user mode management of the baseband is performed by CoreTelephony.framework's `CommCenter`. As its name implies, this daemon is a hub for all network connectivity, and exists also in MacOS, albeit sans baseband functionality.

`CommCenter` uses several preferences property lists in /var/wireless/Library/Preferences. The main one of these (`com.apple.commcenter.plist`) is the most important, caching the `PhoneNumber`, IMEI, ICCID and more. These details (and more) can also be obtained through `libMobileGestalt`'s `BasebandPostponementStatusBlob` key. The daemon also maintains two databases in /private/var/wireless/Library/Databases: a `DataUsage.sqlite`, and `CellularUsage.db`.

The daemon opens up several XPC services to its own clients (e.g. @TODO), and those, in turn, must hold the `com.apple.CommCenter.fine-grained` entitlement in order to be provided service. As its name implies, this is one of the few non-boolean entitlements, whose `array` value can specify multiple keys - e.g. `phone`, `sms`, `identity`, `cellular-plan`, `spi`, `data-[usage/allowed[-write]]`), for pedantic enforcement of access restrictions.

Listing 1-31: The com.apple.CommCenter.plist LaunchDaemon property list

```
EnablePressuredExit: false
EnableTransactions: true
ExitTimeOut: 20
KeepAlive: true
Label: com.apple.CommCenter
MachServices:
   com.apple.CellularPlanDaemon.xpc: true
   com.apple.aps.CommCenter: true
   com.apple.basebandd.xpc: true
   com.apple.commcenter.ari.rt.xpc: true
   com.apple.commcenter.atcs.xpc: true
   com.apple.commcenter.carrierspace.xpc: true
   com.apple.commcenter.coretelephony.xpc: true
   com.apple.commcenter.cupolicy.xpc: true
   com.apple.commcenter.xpc: true
   com.apple.ipTelephony: true
   com.apple.usernotifications.delegate.com.apple.SIMSetupUIService: true
POSIXSpawnType: Interactive
ProgramArguments[0]: /System/Library/Frameworks/CoreTelephony.framework/Supp
UserName: _wireless
```

Of these various services, an entirely undocumented but powerful interface provide by CommCenter is **Apple Remote Invocation**, called ARI. This interface allows an entitled client to fully control the baseband and perform a multitude of other functions through the client `libari.dylib` over XPC.

`CommCenter` interfaces with the baseband through several instances of `AppleBasebandUserClient` and `AppleBasebandPCIUserClient`, provided by the `AppleBaseBand*` kernel extension. The daemon possesses both `com.apple.driver.AppleBasebandPCI[Control].user-access` entitlements, without which the extension refuses `IOUserClient` creation. Other holders of these important entitlements are `mediaserverd` and `abm-helper`.

The `AppleBasebandUserclient` through which `CommCenter` (and `abmlite`) can control high level features of the baseband, such as power, reset, frequency lists, diagnostics, and more. Thanks to iOS 12 β1, the user client has been enumerated, and can be found via `jtool2`'s `--analyze` in every kernel. Table 1-32 shows the User Client methods:

Table 1-32: The `AppleBasebandUserClient` methods

#	Method
0	extResetModem
1	extRadioOn
2	extEnableTrace
3	extGetModemReset
4	extOpen
5	extClose
6	extPowerCycle
7	extNotifyBasebandPoweringDown
8	extRadioUsbOn
9	extQuiesceBackpoweringPins
10	extSetHSICReady
11	extGetPmuExtOn
12	extClearFreqList
13	extAddFreqList
14	extRemoveFreqList
15	extSelfIdentify
16	extGetModemDiagReady
17	extGetEraseEFSDone
18	extGetResetRequest
19	extCoredumpModem
20	extTimeSync
21	extPmuExtOnConfig
22	extSetManagerState
23	extGetManagerState
24	extPPMEmergency
25	extChargerDesense

The baseband data streams are accessible via five interfaces - `pdp_ip[0-4]` - provided by the `AppleBasebandPCI[ICE/MAV]PDPManager`, and offering Skywalk support. The `ApplePDPHelperInterface.framework` is used by `CommCenter` to handle the PDP interfaces. The framework is C++, so thanks to XCode's handy symbolication it is easy to piece together the full API of this small library, which serves to abstract the `AppleIPAppender IOUserClient` methods (guarded by the `com.apple.driver.AppleIPAppender.user-access` entitlement), shown in Table 1-33:

Table 1-33: The `ApplePDPHelperInterface` class methods, and their `IOUserClient` methods

###	Wrapper	`ApplePDPHelperInterface` method
0x0	struct	`::addFlow`
0x1	scalar	`::removeFlow`
0x2	scalar	`::getFlow`
0x3	scalar	`::enable`
0x4	struct	`::setDormancyInterval`
0x5	scalar	`::getDormancyTime`
0x6	struct	`::getTimeAdjustment`
0x7	struct	`::getStatus`
0x8	scalar	`::setLinkQualityMetric`
0x9	struct	`::restartDormancyCheck`
0xa	---	`::copyWakeBuffer`
0xb	struct	`::reportLinkThroughput`
0xc	---	`::getOpenPortsExt`
0xd	scalar	`::enableLowPowerPDPThrottle`
0xe	scalar	`::activateLowPowerPDPThrottle`
0xf	struct	`::getNumLowPowerPDPThrottleStats`
0x10	scalar	`::blockPDPTraffic`
0x11	scalar	`::enableMetadataTrailer`
0x12	struct	`::getKeepaliveOffloadFrames`

An additional class, `AppleBasebandPCIRTIDevice`, provides a number of `AppleBasebandPCIRTIInterfaces`, through which specific aspects of baseband functionality can be accessed. These are:

Table 1-34: The `AppleBasebandPCIRTIInterfaces`

1	`data`	Data transport
2	`ari`	Apple Remote Invocation (libARI.dylib)
3	`vcom`	Voice transport
4	`trace`	Tracing
5	`at`	AT commands
6	`loopback`	Loopback
7	`test`	Internal testing?
8	`gnss`	Global Navigation Satellite System

The baseband control plane was a textual AT-command style one in older (Infineon/Trek) phone models. With Qualcomm's move to Maverick, however, the implementation shifted to using Qualcomm's proprietary (and binary) protocol, QMI (as has also occurred in Android).

The WiFi Chipset

The WiFi chipset is probably the most important connectivity mechanism, as it is available on all of the iDevices, including those without a baseband. Mac devices also have an embedded WiFi chipset, and the code - as well as firmware - appears to be nearly identical across Darwin flavors.

The chipset used is a WiFi and BlueTooth "combo", performing both functions. The specific model indicate the manufacturer can be either the Japanese Murata or the Shanghaiese USI, but the chipsets in both cases appear to be by Broadcom. The specific models are Broadcom's 43x7, (e.g, 4357 in the iPhone10,* and 4377 in the iPhone11,*), although the variants aren't listed as one of the official models in Broadcom's website, indicating it might be specific variants of the 43x5 series, produced for Apple.

The WiFi firmware is not part of the IPSW. Instead, it can be found at /usr/share/firmware - also on MacOS. The WiFi component is in firmware/, with the main firmware as a .trx file, and configuration file (which may be country specific) in accompanying regulatory (.clmb) Tx Cap (.txcb) and NVRAM (as .txt) files, all in a directory pertaining to the chipset (e.g. /usr/share/firmware/wifi/C-4377__s-B2).

There are multiple files in /usr/share/firmware, and the right one is chosen according to the `PlatformConfigFileName` property of the `AppleBCMWLANCore` class, which is in sync with the `module-instance` property of the AirPort device (MacOS: `ARPT@0`, *OS: `wlan`) from the Device Tree. The `RequestedFiles` property is a dictionary which holds the files which were loaded. A similar property, `IOImageLoader`'s `images`, contains the same information. Either property is unique, and thus easily `grep(1)`able in the output of `ioreg(8)`:

Output 1-35: The `RequestedFiled` property of the `AppleBCMWLANCore` class

```
#
# On an iPhone11,2: (module-name = "aladdink")
#
"RequestedFiles" = ({"Firmware"="C-4377__s-B2/aladdink.trx",
                     "TxCap"="C-4377__s-B2/aladdink-NA.txcb",
                     "Regulatory"="C-4377__s-B2/aladdink-NA.clmb",
                     "NVRAM"="C-4377__s-B2/P-aladdink-NA_M-DTCK_V-m__m-5.3.txt"})

#
# On a T2 MacBook Air, 2018: (module name = "hawaii")
#
RequestedFiles" = ({"Firmware"="C-4355__s-C1/hawaii.trx",
                    "TxCap"="C-4355__s-C1/hawaii-X0.txcb",
                    "Regulatory"="C-4355__s-C1/hawaii-X0.clmb",
                    "NVRAM"="C-4355__s-C1/P-hawaii-X0_M-YSBC_V-u__m-4.3.txt"})
```

The WiFi firmware is loaded (on both MacOS and *OS) during system startup, when launchd(8) starts /usr/libexec/wifiFirmwareLoader from the com.apple.wifiFirmwareLoader.plist LaunchDaemon property list. The daemon communicates with AppleBCMWLANCore.kext through its IOUserClient, using four of its five exported methods - `openUserClient`, `prepareMapping`, `completeMapping` and `closeUserClient`, leaving the fifth (`reportMissingFirmwareFiles`) unused.

The WiFi chipset gained notoriety when Gal Beniamini highlighted the process of exploiting it. Through three detailed articles for Project Zero[15], Beniamini walked through dumping, reverse engineering, security auditing and - finally - exploiting the chipset, culminating in adopting the sources of a host Access Point Daemon (`hostapd`) to serve as a malicious access point to attack all iDevices in range and gain kernel memory access. Apple fixed this horrendous vulnerability in iOS 10.3, which resulted in the *OS driver being too trusting of the chipset's DMA requests.

The WiFi software stack is provided by `Apple80211Family[V2]` and several private frameworks - `CoreWiFi.framework` and *OS's `MobileWiFi.framework`. A detailed discussion of the stack is presented as a case study in Chapter 14.

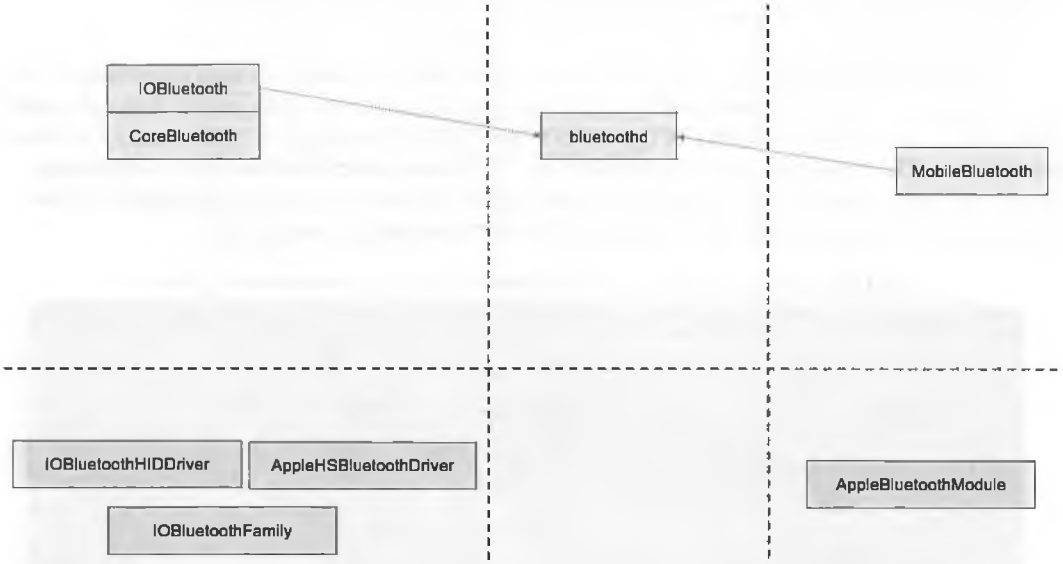
BlueTooth

Apple has been an avid adopter of Bluetooth technologies, with a fully capable stack built into all its Macs and iDevices, including support for the Bluetooth Low energy (BTLE) standard, for which Apple supplies iBeacons. Adoption of Bluetooth has accelerated - first with Apple's Air protocols, then with the introduction of the watch, which uses Bluetooth as its chief transport for pairing with the iPhone. Another incentive was the removal of the headphone jack from modern iDevices, forcing the use of Bluetooth headphones (preferably, AirPods).

Bluetooth allows for a wide range of devices, each of which expose a **service** and a **profile**. Both are abstracted by the **Generic ATTribute Profile** (GATT), maintained and well documented by the Bluetooth consortium[16], which Darwin's stack can readily work wit.

Firmware wise, the BlueTooth component (in *OS) is in /usr/share/firmware/bluetooth/...[MUR/USI]...bin files (along with corresponding .ptb). XNU maintains a stack of kernel extensions to communicate with the firmware, each targetting a specific subset of functions:

Figure 1-36: The Bluetooth stack



Apple provides the public CoreBluetooth.framework for developers. The framework exports an Objective-C only interface, exposing `CBPeripheral` (the Bluetooth device, acting as a server), and the `CBCentral` (the host, acting as a client). The `CBPeripheral` exposes one or more `CBCharacteristics`. Both Central and Peripherals have associated `CB..Manager` objects, which provide scanning and advertising services (respectively). Apple documents the public APIs in the (now archived) Core Bluetooth Programming Guide[17]. MacOS also provides the IOBluetooth[UI].frameworks, both of which are similarly documented by Apple[18]. The *OS variants have the private MobileBluetooth.framework, which provides simpler C-based `BT*` APIs to access the manager.

The functionality offered by all these frameworks still falls short of the stack's full potential. The kernel extension allows advanced features, some of which are accessible from user mode, but are deemed private APIs. Xcode's "Additional Tools" package includes two highly useful apps, utilizing these APIs. The first, "Bluetooth Explorer", lists details and notifications from paired devices. The second, Bluetooth Packet Logger, tracks packets by using the `IOBluetoothHCIPacketLogUserClient`.

The bluetoothd(8) daemon manages the stack from user mode. Client functions such as toggling Bluetooth on/off and performing `IOBluetoothDeviceInquiry`s are carried out over an XPC request to this daemon, which then calls on the Kext's `IOBluetoothHCIUserClient`. Notifications for detected devices are relayed system-wide through `distnoted`, posted as `com.apple.bluetooth.deviceUpdated` notifications, with the MAC address of devices detected in the `object`, and an XML plist in the `userinfo`.

The MacOS and *OS implementations of bluetoothd(8), however, are radically different, as evident from their `LC_SOURCE_VERSION`s and their library dependencies. In *OS, the dameon uses the `com.apple.uart.sk.bluetooth` `PF_SYSTEM/SYSPROTO_CONTROL` socket. It also maintains databases of devices (com.apple.MobileBluetooth.ledevices.[paired/other].db in a SystemGroup container. Both databases contain a single table (`[Paired/Other]Devices`, respectively, holding the `Uuid`, `NameOrigin`, MAC `Address`, `ResolvedAddress`, `LastSeenTime`, `LastConnectionTime`, `GATTServiceChangeConfig`, `Tags` and an `iCloudIdentifier`. Lastly, the *OS daemon provides MIG subsystem 1024768 with 153 messages (0xfa300-0xfa399), which are clearly visible in its `__DATA.__const`, and whose names can be worked out from the framework client code.

Output 1-37: The MIG subsystem exposed by iOS 13's bluetoothd

```
root@LuckyA13# jtool2 -d   DATA CONST.__const bluetoothd | grep MIG  | head -5
Dumping 115744 bytes from 0x100524558 (Offset 0x524558, __DATA_CONST.__const):
_autodetected_MIG_subsystem_1024768:     // 153 messages
0x100524558:      0x1000040a0           MIG Subsystem 1024768: Dispatcher
0x100524560:    0xfa300       0xfa399   MIG Subsystem 1024768: 153 messages
0x100524568:    0xe5c         0x0       MIG Subsystem 1024768: Msg size 3676 bytes
0x100524580:      0x8028000000040d8     _func_1000040d8 (MIG msg 1024768 unmarshall)
..
```

MacOS uses another daemon, `bnepd(8)`, which is started when tethering over bluetooth is selected. The `bnepd(8)` calls `IOEthernetControllerCreate()` to set up an ethernet interface over which frames can be forwarded to the tethering device. The startup is automatic, through an IOKit notification of `[PAN|PTP]ConnectionPresent`, received by `UserEventAgent` from `IOBluetoothL2CAPChannel`.

*OS uses several daemons, all in /usr/sbin. These include `BTAvrcp`, `BTMap` and `BTPbap` (implementing Audio/Video Remote Control, Message Access and Phone Book Access Profiles, respectively), and the `BTLEServer` (Bluetooth Low-Energy). Another binary used in `BlueTool`, which is a scriptable debugging tool, capable of sending raw ACI and HCI commands, as well as Broadcom specific ones.

The Secure Enclave Processor

The Secure Enclave Processor, commonly referred to as SEP, is an integrated coprocessor available on all of Apple's 64-bit chips (A7 and later), as well as some ARMv7k chips, notably the T1. Apple has patented SEP, and the main patent document (US Patent 8832465[19]) details some of the hardware aspects and considerations. As of the A12, the SEP is also paired with secure storage integrated circuit (apparently referred to as Lynx).

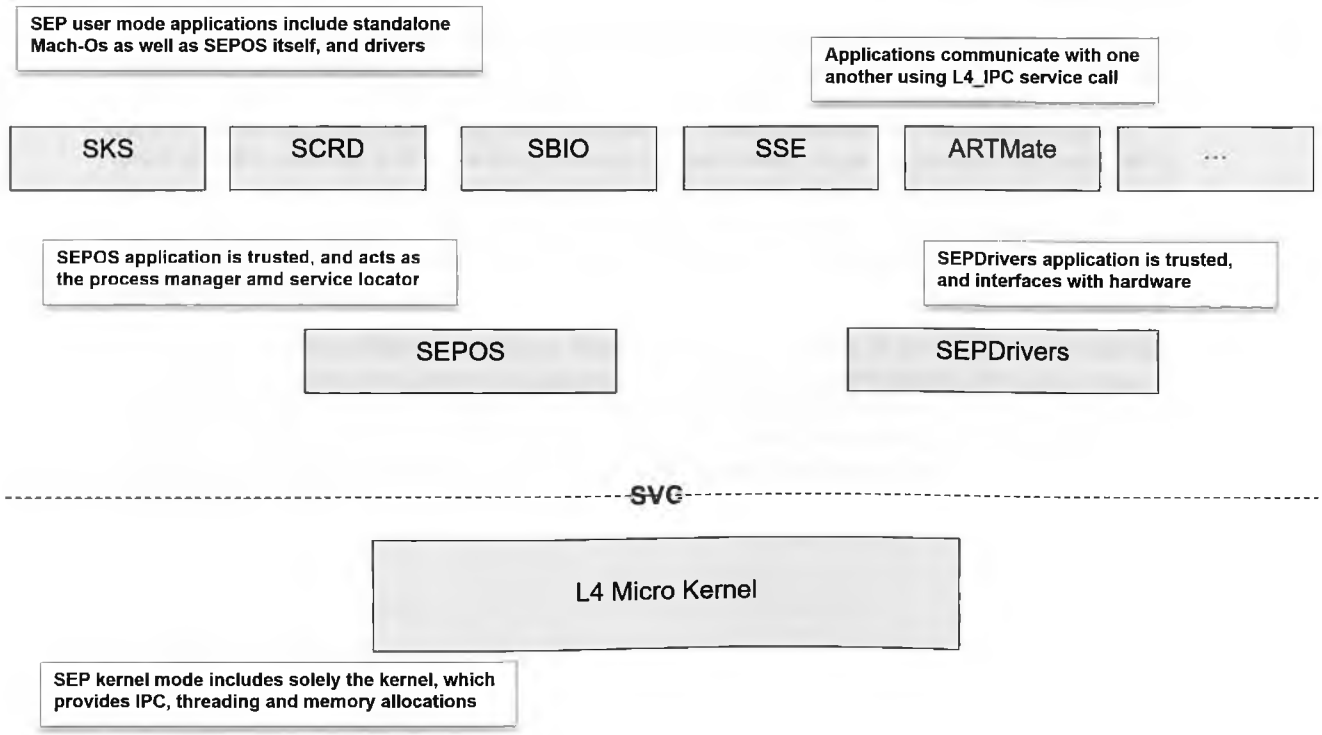
Unlike the main application processor, SEP runs a non-Darwin operating system, referred to as SEPOS. Up until the A11 series, SEP was an ARMv7k (presumably, KingFisher) processor, but as of the A11 SEP is also a 64-bit image. Its software image remains (along with iBoot 64 images) one of the only encrypted files in *OS images. This has not stopped researchers from reverse engineering this important component. Noted security researchers Tajei Mandt, Mathew Solnik and David Wang were the first to bring SEPOS to the limelight in a 2016 presentation given in BlackHat and Syscan[20], which provided elaborate details on the workings of the system, and helped many others (including this author) in bootstrapping research. This was followed two years later by another uncannily detailed presentation by Wang & Wade in TenSec 2018[21]. The details in this section naturally build on these valuable references.

Overall architecture

Inspection of the SEPOS kernel component reveals the name string of "AAPLDarwin L4" and several other hard-coded strings, which imply that it is directly derived from the L4 microkernel architecture. L4 was created by Jochen Liedtke in the late 90s as an improvement over his already efficient L3 microkernel. L4 provides much lighter processing of IPC than Mach (and is thus, faster). The L4 kernel is also a true microkernel, unlike Darwin's take on Mach, which eventually resulted in the monolithic (yet hybrid) XNU. This means the kernel is kept as small as possible, and responsible solely for the minimal task switching, basic memory management and IPC.

All other components run in separate processes, and there are a few of them. The components are loaded as Mach-O files into their own address spaces. These consists of SEPOS itself (as an init-style process), drivers (for hardware access), an ERT layer (presumably, embedded runtime), and the SEP applications, which serve iOS client requests through workloops. Figure 1-38 shows a high level view of the SEPOS architecture:

Figure 1-38: A high level view of the SEPOS architecture



Like all kernels, SEP's L4 exports a set of system calls - SVCs - and (unlike XNU, which uses a single SVC 0), they are spaced 4 apart (e.g. SVC 0x00 for IPC, SVC 0x08 for thread creation, 0x14 for allocation/free, 0x18 for address space creation, etc). Some of these system calls appear to be privileged, such that only the SEPOS master process can execute them. It, in turn, exports them to applications over IPC, and enforces "entitlements", thus restricting the calls to specific callers. In this way, process/thread creation and allocating new memory all has to be funneled through the SEPOS process.

The SEPDrivers process is the one handling all hardware access. This includes I2C, AES, TRNG (True Random Number Generator), Cons(ole), KEY, PKA, PMGR and more. Of these, a critical component is AKF (presumably, Apple KingFisher?), which handles the communication channel with the *OS operating system on the AP.

SEPOS

Once decrypted (with Xerub's `img4tool`), the sep-firmware....RELEASE.im4p file reveals a self extracting archive comprised of nine 32-bit Mach-O components, aligned on a 4k page boundary. The file starts with an ARMv7 exception vector, whose reset handler jumps through a small trampoline to the `LC_UNIXTHREAD` of the main Mach-O, as shown in Listing 1-41 in the next experiment. Once in the Mach-O, the code performs the standard proessor initialization, before switching to Thumb instructions and continuing to assume its function as the kernel of SEPOS.

One page before the SEPOS Mach-O binary is a metadata page (pointed to from the firmware header metadata, at offset 0x810). This page contains the application list to load - including the L4 microkernel and the SEPOS main application. Table 1-39 shows the contents of the SEP image, noting some applications which have only been spotted in the 64-bit images:

Table 1-39: The contents of the SEP image

Label	Contents
(None)	L4 Kernel
SEPOS	SEPOS "Kernel" (main app)
SEPDrivers	Drivers used by the SEPOS main app
sepServices	SEPOS service layer
ARTMate	Anti Replay Token
sks	AppleSEPKeyStore
sse	AppleSSE
sbio-triton	Biometric kit
scrd	AppleSEPCredentialManager
Hilo	? (SEPOS64 only)
EispApp	? (SEPOS64 only)
sprl	Secure Pearl (FaceID, SEPOS64 only)

The `SEPDrivers` image is particularly interesting - especially as it has support for an actual console, complete with command prompt and even a nifty ASCII-curses banner, which can be seen clearly when dumping strings from the image Mach-O:

Listing 1-40: The BBS-Style banner of the SEPOS console (from the SEPDrivers image)

```
morpheus@Bifrost (~) $ jtool2 -d __TEXT.__cstring ~/Documents/iOS/SEP/64/sep.part03 | grep -A8 0x242a1
Dumping 19687 bytes from 0x2323c (Offset 0x1b23c, __TEXT.__cstring):
0x242a1: %s
0x242a5:  ___ ___ ___ ___ ___ ___ ___ ___ \n
0x24401: |___|___|___|___|___|___|___|___|\n
0x2455d: |___|___|___|___|___|___|___|___|\n
0x246b9: |___|___|___|___|___|___|___|___|\n
0x24815: |___|___|___|___|___|___|___|___|\n
0x24971: |___|___|___|___|___|___|___|___|\n
0x24acd: |___|___|___|___|___|___|___|___|\n
0x24c29: \nSEP/OS AppleSEPOS-598\n
```

Experiment: First steps in deconstructing the SEP firmware image

Indubitably, the greatest contribution to SEP research came when the enigmatic Xerub, who specializes in the iOS boot chain, proved he could somehow figure out the decryption keys, one of which (for 5S 10.3.3) he made publicly available[22], enabling anyone to reverse engineer the binaries. The avid reader is encouraged to grab the key from the iPhone Wiki, obtain Xerub's img4tool[23], and follow this experiment, which shows the process of reversing the SEP image. The outputs in this experiment are from the SepOS image of iOS 12 (version 806.200.118), but the structures have not changed much.

Listing 1-41: The 32-bit SEPOS firmware image

```
            B 0x30       B 0x4          B 0x8       B 0xC
0000000   ea00000a     eafffffe       eafffffe    eafffffe
0000010   eafffffe     eafffffe       eafffffe    eafffffe
0000020   00000000     00000000       00000000    00000000
            +--------------------------+
          LDR R1,[PC]  B 0x87B4           ↓
0000030   e59f1000     ea0021de       00000800    00000000
0000040   00000000     00000000       00000000    00000000
*
0000400   01 00 00 00 00 08 00 00   42 75 69 6c 74 20 62 79  |........Built by|
0000410   20 6c 65 67 69 6f 6e 32   00 00 00 00 00 00 00 00  | legion2........|
0000420   00 00 00 00 00 00 00 00   00 00 00 00 00 00 00 00  |................|
*             ?            ?                        ?
0000800    00000002     f0000000       00000000    0007d000
           root-obj   SEPOSentry        LC_UUID of Kernel
0000810    00091000     0000b238       54c649bc    4b376fd5
           LC_UUID of kernel
0000820    2f2022bd     26870041       00000000    00000000
0000830    00000000     00000000       00000000    00000000
*
0001000    feedface     0000000c       00000009    00000002        (L4 Kernel)

00091000  01 00 11 00 00 00 00 00   00 00 00 00 00 00 00 00  |................|
00091010  bf 02 00 00 00 00 00 00   00 00 00 00 00 00 00 00  |................|
00091020  00 00 00 00 00 00 00 00   00 00 00 00 b2 48 ba 13  |.............H..|
00091030  00 00 00 00 00 00 00 00   00 00 00 00 00 00 00 00  |................|
*
00091130  00 00 00 00 00 00 00 00   00 10 09 00 00 00 00 00  |................|
          __vaddr___  ___size____   __entry____
00091140  00 70 00 00 00 60 02 00   38 b2 00 00 00 00 00 00  |.p...`..8.......|
           minus_1     ...........  name_____________
0091150   ff ff ff ff 00 00 00 00   53 45 50 4f 53 20 20 20  |........SEPOS   |
          ___________ ___________  LC_UUID_of_component__
0091160   20 20 20 20 43 79 c3 2c   a4 6a 3c ed 87 f1 b0 46  |    Cy.,.j<....F|
          ___________
0091170   36 51 a5 19 00 70 0b 00   00 00 00 00 00 80 00 00  |6Q...p..........|
0091180   00 70 02 00 dc de 00 00   00 00 00 00 ff ff ff ff  |.p..............|
0091190   00 00 00 00 53 45 50 44   72 69 76 65 72 73 20 20  |....SEPDrivers  |
          _____LC_UUID_of_component_____________________
00911a0   86 ee e9 dc 6e db 3e 9a   ae b5 4c ad 3e ab 59 16  |....n.>...L.>.Y.|
 ....
0092000    feedface     0000000c       00000009    00000002 SEPOS
00b7000    feedface     0000000c       00000009    00000002 SEPDrivers
00de000    feedface     0000000c       00000009    00000002 SEPServices
00fb000    feedface     0000000c       00000009    00000002 ARTMate
0104000    feedface     0000000c       00000009    00000002 sks
015f000    feedface     0000000c       00000009    00000002 sse
018d000    feedface     0000000c       00000009    00000002 sbio-triton
029a000    feedface     0000000c       00000009    00000002 scrd
```

That the individual parts are Mach-O proves useful in their reversing: Although `jtool2` can't handle ARM32, a multitude of disassemblers, including the newly available (and free!) GHIDRA, are able to dissect and even fairly reasonably decompile the images.

SEP/*OS communication channels

*OS variants need to be able to periodically communicate with SEPOS in order to tap on its services. The means to do so is through mailboxes. Messages to SEPOS are written to a special "inbox", and the process of doing so generates an interrupt which SEPOS's interrupt service routine can handle. Depending on the contents of the message, the right SEPOS service can be invoked, to pick up the request. When the service request has completed, results are written to an "outbox", this time generating an interrupt for the Application Processor, which the *OS kernel can handle.

XNU, of course, knows nothing of SEP and its intricacies. Which is why the AppleSEPManager.kext handles the kernel side work. Client code can use `AppleSEPManager` to request **endpoints** in order to communicate with specific services in SEPOS. Each Endpoint is identified by an 8-bit index. The `AppleSEPEndpoint::endpointWithIndex(uint8_t)` call can be used to obtain an endpoint, whose index is also embedded into messages sent to SEPOS. Table 1-42 lists the known endpoints:

Table 1-42: The SEP endpoints

#	enum/#define	Provides
0x0	EP_CONTROL	Direct access to SEPOS itself
0x1	EP_LOG	Logging interface
0x2	EP_ART_SAVE	Anti Replay Token save
0x3	(SEP ART requests)	
0x4	(SEP ART tracer)	
0x5	EP_DEBUG	Debug interface
0x7	(sks)	AppleSEPKeyStore
0x8	(Mesa)	Touch ID
0x9	(BiometricSensor)	Biometric sensor interface
0xa	(SCRED)	SEPCredentialManager
0xb	EP_PAIRING	(likely) SEP/AP Pairing
0xc	(SSE)	Interfaces the sse application
0xf	EP_TESTING	Test interface?
0x10	EP_XART_SEP_SLAVE	XART Storage: found on BridgeOS
0x11	EP_HILO	Unknown; likely only on 64-bit SEP
0x13	EP_XART_SEP_MASTER	XART storage: Found on BridgeOS
0xfe	EP_L4INFO	L4 Kernel information
0xff	EP_BOOTSTRAP	SEP ROM

Solnik et al. reversed the mailbox message format, and determined it is similar to the one shown in Listing 1-43:

Listing 1-43: The SEP message structure

```
struct {
        uint8_t         endpoint;
        uint8_t         messageTag;
        uint8_t         opCode;
        uint8_t         parameter;
        uint32_t        data;
} sep_msg;
```

The message `opCode` fields are endpoint dependent. Those of the `EP_CONTROL` endpoint are fairly easy to reverse, thanks to `AppleSEPControl::_cmsgSend` and its multiple wrappers for complex transactions. Solnik et al. constructed a partial table of the messages, shown in the next table with some additions.

Table 1-44: The `EP_CONTROL` opCode values, reversed from AppleSepManager.kext

opCode	Purpose
0x00	No operation
0x02	Set OOL in address
0x03	Set OOL out address
0x04	Set OOL in size
0x05	Set OOL out size
0x0a	TTYIN (write to console)
0x0c	Sleep the SEP
0x12	Nap the SEP
0x13	
0x14	Security mode request

opCode	Purpose
0x15	Generate nonce
0x16	Read nonce
0x17	Invalidate nonce
0x18	Self test
0x22	Start versioned apps
0x23	Commit hash
0x24	?? Event ??
0x25	Erase install
0x26	L4 Panic
0x27	SEPOS Panic

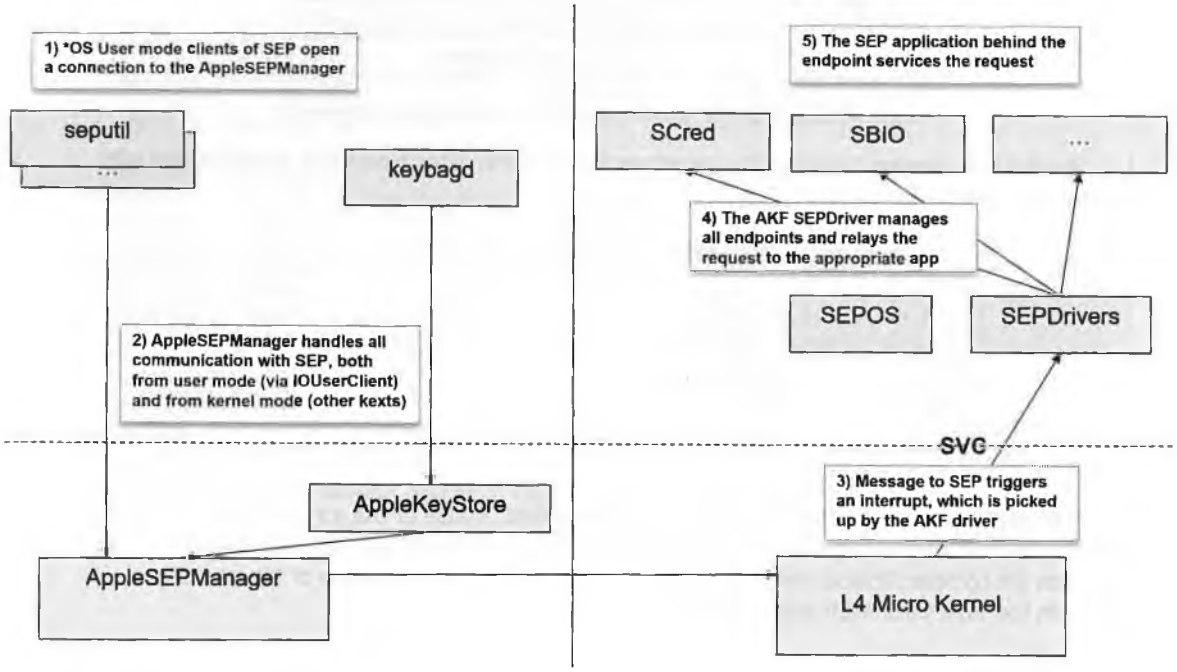
User mode interface

SEP is only accessible from kernel mode, yet user mode clients need to be able to interoperate with it, as well. The AppleSEPManager kernel extension provides an `IOUserClient` with a rich set of methods. *OS devices apparently only have two SEP-aware binaries:

- /usr/libexec/xartstoraged - on BridgeOS, this daemon registers with the `AppleSEPManager`, setting a notification port and responding on MIG Subsystem 4200, which handles xART lockers on the filesystem (an educated guess is that these are tamper proof data containers).
- /usr/libexec/seputil - This is symlinked from /usr/libexec/init-data-protection, which is started by `launchd(8)` through its `__TEXT.__bs_plist/__TEXT.__config`. When launched in this way, `seputil` attempts to automatically load the sep-firmware.img4. Otherwise, it serves as a debugging tool with numerous options.

Other binaries, however, may use SEP indirectly. A good example of this is `keybagd`, which interfaces with `AppleKeyStore.kext`. The 32-bit keystore functions were implemented directly by that kext, but on 64-bit devices `AppleSEPKeyStore.kext` takes over, and communicates internally with `AppleSEPManager`, leaving clients unaware of the hardware change. Figure 1-45 depicts the flow of messages between *OS and SEPOS.

Figure 1-45: The message flow between *OS and SEPOS



Using `seputil` and the `AppleSEPManager.kext` (especially in the iOS 12 β 1 kernels), it is possible to reconstruct the `AppleSEPUserClient` dispatch table, as shown in Table 1-46 (next page). Most are self explanatory, but a few remain cryptic, at least to this author.

Table 1-46: The AppleSEPUserClient methods

#	Method	
0	ping	Wake up SEP
1/2	sleep/nap	Sleep/Nap SEP
3	ROMCommand	Request service from SEPROM
4	securityMode	Get Security mode
5	ttyIn	
6	loadFirmware	Load SEPOS firmware image
7-8	loadRestoreFirmware[WithART]	
9	commitHash	
10	eraseInstallNotification	
(11)	L4Panic	Trigger panic at SEP's L4 microkernel level
(12)	SEPOSPanic	Trigger panic at SEPOS (application) level
11(13)	getLogSize	Get message log size
12(14)	getMailboxLog	Get message log (as per `seputil --rawlog`)
13(15)	generateNonce	Request new nonce from SEP
14(16)	invalidateNonce	Invalidate previously generated nonce
15(18)	startVersionedApps	
16-17-18	[get/set/clear]ART	Retrieve an Anti-Replay Token
19	pairKey	
20-21	testQuery[Cat/Test]	Various internal tests, through `EP_TESTING`. These can be invoked via `seputil`'s `--test*` options, but most fail - presumably they are only avilable on debug SEPOS builds.
22	runIndex	
23	runHandle	
24	testSelectCat	
25-26	testData[Set/Get]	
27	testErrorDetails	
28	testSetTimeout	
29	objectCreate	Create an object, possibly sharing it with service, and/or reading/writing data from/to it
30	objectShare	
31-32	object[Read/Write]	
33	rngDump	
34	CounterSelfTest	
35-37	**Unsupported (0xe00002c2)**	
38	xART OS UUID setup	
39	Marshall xART to SEP	
40	Icrt verification	
41-42	LynxGet[CPSN/PubKey]	Get SEP crypto coprocessor Serial/Public Key
43	Hilo accumulate test pattern	Various requesting targeting `EP_HILO` (64-bit SEP only)
44	Hilo gain calculation	
45-46	Hilo get/load LTH	
48	HiloUpdateTimer	
49	Hilo load Raw LTH	
52	HiloGetParams	
58-59	Hilo gain restriction [advance/set]	
..	ProcInfo	

Experiment: Communicating with SEP via `seputil(1)`

The `seputil` binary is a particularly powerful one, as it has many debugging features. One of its interesting (and safe) command line switches is `--art get`, which gets the Anti-Replay Token, as shown in the following Output:

Output 1-47: Using `seputil` to obtain the Anti Replay Token

```
root@hodgepodge (/usr/libexec) # ./seputil --art get
raw ART: 3063020100303c
    0202[redacted]                                                  # counter
    0414c76313eb19241adf668a2fb7ed5e2e317f49f840                    # manifest hash
    04149e43f0d9772ab78bc9f7639ec500f088362bb29a                    # sleep hash
    04003108 c2020278 c302023e                                      # subcounters
    04204a2110810ae010abfed51e0c4f971822098fe12712e6964ba292683a895cb490   # Unknown SHA-256
Successfully parsed ART:
counter: 7893
manifest hash (20 bytes): c76313eb19241adf668a2fb7ed5e2e317f49f840
sleep hash (20 bytes): 9e43f0d9772ab78bc9f7639ec500f088362bb29a
restore nonce is absent
subcounters[3]: 574
subcounters[2]: 632
```

An interesting function of `seputil` is its `--rawLog` (pre iOS 12) or `--rawlog` (as of iOS 12) argument, which obtains the SEP mailbox log and displays it in a raw format, consisting of three `uint64_t`s: An address mapped to `seputil`'s space (irrelevant), a timestamp, and the message (i.e. the structure from Listing 1-43, flattened). Due to endianness, the field ordering appears flipped, so that the endpoint specifier is actually the least significant byte.

The book's companion website offers a simple script to parse the SEP messages in the log to a human readable format.

The `seputil` tool should be used with extreme caution! It has many functions, and not all of them are necessarily safe. Arbitrarily trying command line arguments could potentially leave the SEP in a bricked state, which would impact iOS's security functions or worse.

Review Questions

1. Why would the contents of SCfg be considered sensitive?
2. Why would Apple choose the Mach-O (`MH_PRELOAD`) file format for the firmware images?
3. Compare and contrast the Secure Enclave model with that of ARM's TrustZone/EL3 Secure Monitor

References

1. Apple - "Identify your MacBook Air model" (HT201862) - https://support.apple.com/en-us/HT201862
2. The iPhone Wiki - "List of iPhones" - https://www.theiphonewiki.com/wiki/List_of_iPhones
3. WikiPedia - "Apple Designed Processors" - https://en.wikipedia.org/wiki/Apple-designed_processors
4. iPhone and iPad schematics - Gadget Manual - https://www.gadget-manual.com/apple/
5. ranvme - Ramtin Amin - "Demystifying the iDevice NVMe NAND" http://www.ramtin-amin.fr/#nvmepcie
6. Apple - HT201736 - https://support.apple.com/en-us/HT201736
7. Apple Developer - "Thunderbolt Programming Guide" - https://developer.apple.com/library/archive/documentation/HardwareDrivers/Conceptual/ThunderboltDevGuide/Basics/Basics.html
8. Apple - Patent #20130115821 - "DUAL ORIENTATION CONNECTOR WITH EXTERNAL CONTACTS AND CONDUCTIVE FRAME"
9. Ramtin Amin - "Apple Lightning" - https://ramtin-amin.fr/#tristar
10. Apple - HT204063 - HT204063
11. @stek29 - "What does NVRAM lock/unlock actually mean?" - https://stek29.rocks/2018/06/26/nvram.html
12. DMTF - SMBIOS 3.2 Reference Specification - https://www.dmtf.org/sites/default/files/standards/documents/DSP0134_3.2.0.pdf
13. OpenSource.Apple.com - AppleSMBIOS - https://opensource.apple.com/source/AppleSMBIOS/
14. Apple - HT201295 "How to reset the SMC on your MAC" - https://support.apple.com/en-il/HT201295
15. Gal Beniamini (Project Zero) - "Over the Air: Exploiting Broadcom's Wi-Fi Stack" - https://googleprojectzero.blogspot.com/2017/04/over-air-exploiting-broadcoms-wi-fi_4.html
16. Bluetooth SIG - "Bluetooth GATT specifications" - https://www.bluetooth.com/specifications/gatt/
17. Apple Developer - "Core Bluetooth Programming Guide" - https://developer.apple.com/library/archive/documentation/NetworkingInternetWeb/Conceptual/CoreBluetooth_concepts
18. Apple Developer - "Bluetooth device drivers" - https://developer.apple.com/library/archive/documentation/DeviceDrivers/Conceptual/Bluetooth
19. Gulati et Al. (Apple) - "SEP for a SoC" (US Patent 8,832,465 B2) https://patents.justia.com/patent/8832465
20. Mandt, Solnik & Wang - "Demystifying the Secure Enclave Processor" - https://www.syscan360.org/slides/2016_SH_Mathew_Solnik_Demystifying_the_Secure_Enclave_Processor.pdf
21. Wang & Wade - "SEPOS: A Guided Tour" - https://share.weiyun.com/5IJZPBp
22. @Xerub - The iPhone Wiki Firmware Keys - https://www.theiphonewiki.com/wiki/Greensburg_14G60_%28iPhone6,1%29
23. @Xerub - Img4tool https://github.com/xerub/img4tool

2

Use the source, Luke: The XNU Codebase

XNU's sources, like those of Linux, are (largely) open. This is a huge boon for researchers striving to glean and explore its internals and security. Although they have always been open, it is not so much by Apple's good graces, but owing to their third-party components (BSD and CMU). The source tree layout, however, has allowed Apple to keep the ARM and ARM64 branches private. Researchers and enthusiasts therefore had to resort to reverse engineering to uncover the idiosyncrasies of *OS kernels. What more, *OS kernels had traditionally been encrypted with the device GID key. This meant that, as a prerequisite, the unencrypted kernel image had to be obtained - whether by using an iBoot exploit, or by dumping from memory using a kernel vulnerability.

This chapter explores the XNU sources in detail. Briefly looking at the version history, we then explore every branch of the source tree, as familiarity with its many files is cardinal for any kernel enthusiast or researcher. We then look at the process of compiling the sources, which might be beneficial for specific research aspects (although the Apple Kernel Debug Kit usually suffices). Kernel debugging (over KDP) in introduced next, and - finally - the dread `panic()`s, and how to make sense of them.

If you are serious about kernel research, sifting through the source tree will be avoidable and become second nature. The reader is encouraged to download the latest XNU tarball (or follow along through the source trees) at http://OpenSource.apple.com/tarballs/xnu, or from the OpenSource Apple GitHub page.

To search the sources, the author usually uses the pattern:

```
grep -R$(PATH_TO_XNU_SRC_ROOT) _string_
```

Functions are almost always defined with the function name separated from its return value to it can be at the beginning of the line, and thus easily `grep(1)`able with a ^ marker. A good tip is to put the above line into a simple script and call it by a catchy name (`xrep`, etc), so you can quickly look for strings and/or functions this way.

The book's companion website provides an HTML-enabled cross reference, at http://NewOSXBook.com/xxr. Using it, you can easily navigate, search through the source code and compare various source versions up to the present day.

The XNU Source

XNU is one of the oldest kernels in modern use. Its Mach core predates Linux by several years, and is about as old as the Windows NT kernel, which still drives the latest versions of Windows today. As with the other kernels, it has evolved dramatically over the years, from the first available version (123, from MacOS X 10.0) to the present day (4903), though its underlying core and operating principles remain true to the original design.

Source Versions

The XNU source version can be obtained in one of several ways. From the binary, one way is to use `jtool -l` and isolate the `LC_SOURCE_VERSION` command, which has been included in the Mach-O header since around Darwin 12. This should also match the kernel's `kern.version` string, which can be found by `strings(1)`. The latter is also reported at runtime by the appropriate `sysctl(2/8)` or `uname(1/3)` commands and APIs.

XNU kernel versions tend to remain stable across patch (0.0.1) releases, providing only bug and vulnerability fixes, and leaving the kernel major version unchanged.A notable exception to that was in Darwin 10 (MacOS 10.6 through 10.6.8) and the corresponding versions of iOS, which evolved rapidly at that time. iOS XNU versions were also somewhat ahead of MacOS, but as of Darwin 12 the flavors have all aligned.

Table 2-1 Mapping Darwin and XNU versions

Darwin	MacOS	iOS	TvOS	WatchOS	XNU
1	10.0				123
5	10.1				201
6	10.2				344
7	10.3				517
8	10.4				792
9	10.5				1228
10	10.6	-4			1456-1504
11	10.7	5			1699/1750
12	10.8	6			2050
13	10.9	7			2422
14	10.10	8		1	2782
15	11	9		2	3248
16	12	10		3	3789
17	13	11		4	4570
18	14	12		5	4903
19	15	13		6	6153+

Apple eventually open sources XNU after Darwin releases are out. There is a delay window, however, which in the past has ranged from a few weeks to several months. During the beta period and this window, a good way to gain insights as to XNU kernel changes is by looking over Xcode's Kernel.framework headers (part of the MacOS.sdk). Although not `#defines` as `PRIVATE` (and thus stripped of some important internal changes), `diff(1)`ing these headers with those of the previous SDK often reveals many changes. The book's companion `joker` tool (which is now built-in to `jtool2 --analyze`) also comes in very handy.

As of XNU 4570 (Darwin 17), Apple includes the full ARM32 and ARM64 branches of the source - after originally going to great lengths to strip the `CONFIG_EMBEDDED` and `ARM` `#defines` before public releases. Whether this was due to a hasty mistake (the sources appeared less than a week after the release of iOS 11) or planned (as Apple would state) is irrelevant - as one should not look a gift horse in the mouth. The embedded branches provide unprecedented visibility into the inner workings of *OS variants, and potentially allows running iOS on non-Apple or emulated hardware. The practice continues with Darwin 18, though A11/A12 code portions (notably, APRR and PPL) have clearly been stripped.

The source tree

The XNU source code is arranged in a clean-cut tree, whose various branches roughly correspond to the subsystems.

Table 2-2: The subtrees of the XNU sources (as of XNU 4570)

Subtree	Contents
APPLE_LICENSE	Copy of Apple Public Source License (APSL)
BUILD/	Output directory for build products
EXTERNAL_HEADERS/	Miscellaneous Darwin headers (to avoid dependency cycles)
Makefile	File used by `make(1)` to build XNU
README.md	Installation instructions
SETUP/	Setup utilities
bsd/	The BSD subsystem, partial port of BSD with many Apple additions/modifications
config/	.exports for creating the pseudokexts of System.kext
iokit/	The IOKit Subsystem
libkdd/	Kernel Data Descriptors & Kernel Chunked Data (Darwin 15)
libkern/	Runtime APIs
libsa/	"Standalone" bootstrap code
libsyscall/	User-mode system call and mach trap wrappers (for libsystem_kernel.dylib)
makedefs/	Makefile defaults
osfmk/	Open Source Foundation Mach Kernel
pexpert/	Platform Expert
san/	Address Sanitizer (Darwin 17)
security/	MACF subsystem (q.v. III/4)
tools/	Miscellaneous scripts and test suites

Note, that some components of XNU still remain closed source, even in the MacOS kernel. Notable examples are XNU's CPU Power Management support (in osfmk/x86_64/xcpm) and SkyWalk (bsd/skywalk). Although not in the public source tree, the compiled contents of these directories are readily visible in the compiled kernel. This does not, however, prevent the public sources from correctly compiling, as discussed later in this chapter.

The build directories (SETUP/, config and makedefs/)

XNU's SETUP/ config/ and makedefs/ contain no kernel code, but are vital to the kernel build process. SETUP/ contains the code for several helper binaries, notably config, which is responsible for generating several structures (e.g. pseudo-devices).

The config/ (not to be confused with SETUP/config) directory is especially important, as it governs the Kernel's version string (config/version.c) as well as its exported symbols (multiple .exports files, broken by architecture and KPI bucket, as discussed in Chapter 3). The MASTER file serves as the machine independent kernel configuration file, `#define`ing important compilation directives which affect the build. Additional MASTER.*arch* provide machine specific customizations for the specific Darwin flavor being built*.

* - An interesting observation is tht XNU 4903 contains a config/MASTER.arm64.bcm2837, as well as other mentions to the BCM2837 platform - The Broadcom chip used in the Raspberry Pi 3. Whether this is an unstripped residue of Apple's internal builds or a hint at opening up the platform is unknown, but it certainly means that *OS can be adapted to run on generic (non-Apple) hardware.

bsd/

The bsd/ directory started as a direct port of FreeBSD 4.x code into XNU. Over time, this was updated to FreeBSD 6.0, but since then the source has greatly diverged. FreeBSD has gone on to present day (11.x), while Apple took the source in a different direction, deprecating some features and adding numerous other functions and system calls. It is no longer a simple matter to tell apart which code is Darwin specific.

Table 2-3: The subdirectories of the bsd/ sources (as of XNU 4570)

Subdirectory	Contents
arm/	ARM and ARM64 constants and configuration parameters
bsm/	Headers from Solaris's Basic Security Module providing audit subsystem
crypto/	Headers for /osfmk/crypto's algorithms, and RC4 implementation
dev/	Kernel-provided character devices, notably `[u]random` and DTrace
i386/	x86 and x86_64 constants and configuration parameters
kern/	BSD Kernel core
libkern/	Basic runtime support (CRC, memory and string handling)
machine/	Header files
man/	Collection of semi-maintained BSD manual pages
miscfs/	Implementation of pseudo-filesystems (deadfs, devfs, fifofs, mockfs, nullfs, routefs, specfs and union)
net/	General socket support and network stack components
netinet/	IPv4 socket and protocol support
netinet6/	IPv6 socket and protocol support
netkey/	IPSec socket and protocol support
nfs/	NFSv3/NFSv4 support
pgo/	Profiling data support (`#if PROFILE`)
security/	Implementation of Solaris BSM audit
skywalk/	Skywalk implementation (remains closed source)
sys/	Miscellaneous headers, some of which are exported to <sys/*.>
uuid/	Headers for UUID support (as provided by /libkern/)
uxkern/	Mach exception to UN*X signal bridging
vfs/	Virtual Filesystem Switch support
vm/	Virtual memory management

The BSD layer is largely hardware-independent: Although there are arm/ and i386/ subdirectories, these contain the same files, with slightly differing parameters in vmparams.h , different thread contexts in _mcontext.h, and different DTrace definitions in fasttrap_isa.h.

The magnificent DTrace facility is one of several vestiges of Solaris (also in bsd/dev/dtrace, discussed in Chapter 6), with another being the comprehensive auditing of the Basic Security Module (bsd/security and bsd/audit, discussed in III/3), both owing to their adoption by FreeBSD.

True BSD subsystems include the network stack (with sockets implemented in bsd/net, contingent on `CONFIG_SOCKETS`), its various protocols (IPv4 in bsd/netinet, IPv6 in bsd/netinet6 and IPSec in bsd/netkey), and the Virtual Filesystem Switch.

With the exception of NFS, note the absence of the core filesystem implementations. Earlier versions of XNU also included the sources for the HFS+ implementation, but this has rightfully been moved into a separate project, as a kernel extension. The present filesystem, APFS, has yet to be open sourced. The miscfs/ subfolder does include a few filesystems, but these are not true filesystems: The `deadfs` serves for vnode access revocation. `devfs` implements the /dev filesystem for device nodes. The `specfs` and `fifofs` are adapter layers for specific named pipes and device files outside of devfs. Other filesystems are not used by release kernels.

osfmk/

The osfmk/ subdirectory owes its name to the Open Source Foundation Mach Kernel, which accounts for the bulk of its contents to the present day. But as with the bsd/ directory, Apple has made significant modifications to the original sources.

Table 2-4: The subtrees of the osfmk/ sources (as of XNU 4570)

Subtree	Contents
arm/	ARMv6/7 (32-bit) support
arm64/	ARMv8[.1] (64-bit) support
atm/	Activity Trace Manager
bank/	Bank vouchers
chud/	Computer Hardware Understanding & Diagnostics (Removed in Darwin 18)
console/	Serial and video console support
corecrypto/	In-kernel implementations of AES, SHA[1/256] and base cryptographic/hash algorithms
corpses/	Task corpse support (Darwin 16+)
default_pager/	Implementation of default memory pager (Removed in Darwin 16)
device/	The "device service" (IOKit/Mach interface)
gssd/	MIG definitions shared with the user mode Generic Security Services Daemon
i386/	x86 (32-bit) support
ipc/	The Mach IPC ports, spaces, and object implementations
kdp/	Kernel Debug Protocol
kern/	Mach kernel core
kextd/	MIG definitions shared with the user mode Kernel Extension Daemon
kperf/	Kernel performance counters
libsa/	Headers used by /libsa
lockd/	MIG definitions shared with the user mode lockd (deprecated)
mach/	User-mode header files (which end up in <mach/>)
mach_debug/	Headers for Mach debugging APIs (`#if MACH_DEBUG_*`)
machine/	High-level machine interface headers
man/	The outdated (but pretty much the only) HTML manual pages for original Mach APIs
prng/	Pseudo Random Number Generation
profiling/	Interfaces to common profiling
UserNotification/	MIG definitions and generated code shared with Kernel/User Notification Daemon (kuncd)
vm/	Virtual Memory Management
voucher/	Voucher Support (Darwin 14+)
x86_64/	x86 (64-bit) support
x86_64/xcpm	XNU CPU Power Management

Unlike the BSD subdirectories, the architecture-specific subdirectories of osfmk/ contain both headers and code. This is similar to the model of the Linux kernel, in which these subdirectories provide the low-level, machine specific parameters, structures and implementations of similarly named functions. The MacOS (for now, Intel) sources are in i386/ and x86_64, whereas those of ARM are in arm/ and arm64/, but the 64-bit builds still use significant portions from the code of the 32-bit architectures, in cases where it is common to both.

Over time, osfmk/ has evolved above and beyond the original Mach kernel sources, as NeXTSTEP and, later, Apple, added and extended the core of Mach. It has become virtually impossible to see where the original OSF sources end and the extensions begin. Notable Apple additions are atm/, bank/ and voucher (all discussed in Chapter 10). Notable Apple removals are the default_pager (backing store pager for anonymous memory), and the recently (Darwin 18) removed chud/, used for profiling in the PowerPC and early Intel days.

libkern/

The libkern/ subtree functions as the kernel's "runtime library", similar in some respects to Windows' kernel `rtl` APIs. Various functional aspects are grouped into this subdirectory, as shown in Table 2-5:

Table 2-5: The contents of the libkern/ sources (as of XNU 4570)

File/dir	Contents
OSKextLib.cpp	The kernel-side implementation of IOKit.framework's OSKext APIs
OSKextVersion.c	Kernel extension version string handling
c++/	The core `OSObject` and base derived types
crypto/	Kernel corecrypto supported algorithms
firehose/	`os_log` kernel side support (As of Darwin 16)
gen/	Support for atomic operations and stack backtracing
kernel_mach_header.c	Mach-O header parsing functions (for OSKextLib.cpp)
kmod/	Kernel extension (module) support
kxld/	The kernel extension loader
libkern/	Header files
mkext.c	Multikext/LZSS support
net/	`inet_XtoY()` conversion functions
os/	libplatform and liblog kernel-side support (As of Darwin 15)
stack_protector.c	`__stack_chk_fail()`
stdio/	`[v]sscanf()` implementation
uuid/	UUID parsing support
zlib/	zlib 1.2.3 built-in to XNU

A key function of libkern/ is its support for kernel extensions. The `OSKext*` APIs of the IOKit.framework (discussed in detail in Chapter 3) are provided by OSKextLib.cpp, with help from Mach-O parsing functions in kernel_mach_header.c. Another important component is the kernel extension loader (kxld/), which has its own logic for parsing Mach-O kernel extensions and linking them in kernel mode (again, discussed in Chapter 3).

Another thing which stands out in libkern/ is that a large part of its code is implemented in C++, and not C. The choice of C++ in kernel mode is quite unusual, but brings with it immense benefits. Object orientation and inheritance, to name but two, enable a powerful runtime which can easily reuse and extend code bases. The C++ object hierarchy is declared in libkern/libkern/OS*.h and other headers in libkern/c++/. The libkern/ objects and C++ features are detailed in Chapter 13.

iokit/

The main client of libkern/'s C++ runtime is IOKit, the modern day evolution of NeXTSTEP's "DriverKit" framework. Also implemented in C++, IOKit relies heavily on libkern/ to support the base objects and APIs to manipulate them. IOKit uses libkern/'s object orientation - and in particular, inheritance, to provide "families" which are superclasses from which specific drivers and functions can be derived. Another key concept is the placement of drivers ("IOService"s) into a multiplanar data structure referred to as the "IORegistry", which enables the location of specific drivers .

IOKit is, at least by its original design, a self-contained subsystem in the kernel. Many of the functions implemented in iokit/Kernel/IOLib.cpp are, thus, wrappers over Mach (osfmk/) and BSD implementations. The IOKit environment is detailed in Chapter 13, but Table 2-6 shows the contents of iokit/:

Table 2-6: The contents of the iokit/ sources (as of XNU 4570)

File/dir	Contents
bsddev/	IOKit bindings to "old-style" BSD devices and interfaces
Examples/	A single example for a generic interrupt controller
Families/	The open sources of the IONVRAM and IOSystemManagement families
IOKit/	Headers and class definitions
Kernel/	Class implementations
KernelConfigTables.cpp	Deprecated pseudo-kext declarations used by the IOCatalogue
Tests/	Several tests of IOKit mechanisms

pexpert/

Kernels designed for multiple architectures must make use of some hardware abstraction layer. Windows provides this functionality with its standalone HAL.sys. Linux opts to compile architecture support in-kernel from its arch/ subdirectories, as we have seen is also done in XNU with the architecture specific subdirectories of osfmk/.The **platform expert** is somewhere in between these two approaches: This important component of the kernel has to be compiled in, but is fairly self-contained (and tightly integrated with IOKit).

The platform expert provides a set of APIs for accessing low level kernel functions. These include the boot arguments, device tree, system state, and the console. The APIs are conveniently prefixed by "PE_", which makes them easy to identify in the kernel's export table (although some of these are stripped in the ARM case). The common exports are shown in Table 2-7:

Table 2-7: The exported functions of the Platform Expert

Export	Function
`_PE_cpu_[start/halt]`	Start/halt processor (using `IOCPU::[start|stop]CPU`)
`_PE_cpu_machine_[init/quiesce]`	Take CPU in/out of quiescence (using `IOCPU::[init|quiesce]CPU`)
`_PE_cpu_signal`	Send a processor interrupt (using `IOCPU::signalCPU`)
`_PE_i_can_has_debugger`	Determine if kernel debugging is enabled
`_PE_i_can_has_kernel_configuration`	Determine if kernel configuration is enabled
`_PE_parse_boot_argn`	Parse a numeric boot argument by name
`_PE_get_default`	Parse a boot argument by name, or return default value if unset

The subdirectory structure of pexpert/ contains a gen/ subdirectory for the generic (and thus, cross platform) components, which include the BootCLUT (8-bit Boot Color Lookup Table), debugging support, boot arguments and device tree parsing. Architecture specific components can be found in arm/ or i386. Header files are in pexpert/pexpert/, with nested per-architecture subdirectories.

security/

The security/ subdirectory is the last one to be compiled into the kernel proper, but by no means is it the least. The directory provides the implementation of the Mandatory Access Control Framework (MACF). Originally implemented as part of TrustedBSD by Robert Watson (whose name adorns most of the file header comments), MACF was adopted in Darwin 9. Through a set of callback hooks which can be registered as a **MACF policy**, a kernel extension can intercept most of the operations performed on behalf of user mode - specific Mach APIs and a vast number of BSD-layer system calls.

MACF provides the substrate for virtually all of Apple's security features, from code signing enforcement (using `AppleMobileFileIntegrity.kext`), through MacOS's GateKeeper (`Quarantine.kext`), and the `Sandbox.kext`. MACF is detailed in III/4, and the above mechanisms are discussed in III/5 through III/8.

san/

The san/ subdirectory is the latest addition to XNU, having been added in Darwin 17. It provides the implementation of the **Kernel Adress SANitizer** (KASAN). KASAN is a kernel mode implementation of the Address Sanitizer project, an open source project (started by Google) with the aim of detecting memory corruption errors, thereby making the kernel more secure. The Linux kernel was an early adopter of this as well, as of around version 4.0.

Using the sanitizer implies a significant performance hit, owing to the use of shadow memory (used in verifying copy operations and detecting overflows) and other instrumentation directives emitted during compilaation. It is therefore contingent on a `CONFIG_KASAN #define` directive. Later builds of MacOS 14 use this feature, as shown by the `com.apple.kpi.kasan` pseudo-kext. KASAN is especially useful when combined with an automated testing or fuzzing harness, as detecting corruptions using KASAN is far easier than without it. That said, KASAN is not a panacea, and some corruptions may still elude it.

libsyscall/

Although it is part of the kernel sources, libsyscall is actually a user-mode component. The code in this subdirectory contains the various system call stubs, and gets compiled into libsystem_kernel.dylib. There are several subdirectories, including XCode support files (Libsyscall.xcconfig, Libsyscall.xcodeproj, xcodescripts), and the following:

- **Platforms/** - contains syscall.map files for MacOS and iPhoneOS, in which are alias mappings of symbols to double underscore and `$NOCANCEL` variants.
- **custom/** - contains several thin assembly wrappers.
- **mach/** - contains wrappers for `mach_msg()`, .sub files with subsystem error strings, and some trap wrappers.
- **os/** - contains support for libplatform APIs, like `os_alloc_once_table` and the `_os_tsd` features.
- **wrappers/** - contains C wrappers for some syscalls which require user-mode prolog or epilog handling (for example, `ioctl(2)` and its varargs handling), as well as support functions (e.g. `stack_snapshot_with_config`).

Note, however, that not all system calls included in libsyscall. Those for whom bsd/kern/syscalls.master specifies `NO_SYSCALL_STUBS`, are ignored by scripts in xcodescripts.

libkdd/

The libkdd/ subdirectory was introduced in Darwin 15. It provides the user mode parsing support of "Kernel Data Descriptors", described (in the Readme.md) as "dynamic data to be passed from the kernel to userspace tools without binding them to particular version of struct definition". Also known as "Kernel Chunked Data" (or KCData), this is a self-describing data format wherein every datum is preceded by a "chunk header". User-mode clients can then read the chunks through objective-C APIs in kdd.h, and present them to the user even if not "aware" of the actual field semantics.

The kernel mode support for Kernel Chunked Data is in osfmk/kern/kcdata.h and osfmk/kern/kcdata.c, with the latter file providing the "KCDATA MANIFESTO", explaining the use of these features in great detail. The README.md in the subdirectory root also reiterates some of the key points. The main user of these KPIs is the `stackshot` facility, which has been rewritten so as to package its myriad data into `kcdata` types.

The directory also contains an .xcodeproj subdirectory, so its products (kdd.framework and libkdd.a) can be built using `xcodebuild(1)`. This does require modifying the kdd.xcodeproj/project.pbxproj and commenting out `SDKROOT=macosx.internal` mentions.

tests/

The tests/ directory is newly added in XNU 4903 and later, with short test programs (formerly in tools/tests/darwintests) which Apple's own XNU developers use as unit tests for various system calls and traps, mostly experimental ones. Although oft-overlooked, these tests often provide the only open source examples of using these facilities.

Sadly, none of the tests will readily compile, due to their dependency on <darwintest.h> and <darwintest_utils.h>, which Apple does not make available. These likely hold the definitions of various `T_*` macros, which are used in the unit testing, providing wrappers for assertions and other debug printouts.

Adapting some of the tests to compile is fairly simple, and can be achieved by removing the missing header dependencies (or `touch(1)`ing them), and/or implenting the various `T_*` macros. An example of this was shown in Listing I/15-45. Another hurdle to compiling the tests is that some depend on internal and/or private headers. These can often be reconstructed, whether by `#include`ing the kernel private headers (with the required `#define`s of `PRIVATE`, `KERNEL_PRIVATE`, etc), or reversing the function prototypes from their use. And example of that is shown in Listing I/15-33, which recreates a portion of <ktrace.h>, a header file required for libktrace.dylib related tests.

tools/

With the tests/ subdirectory moved out as of XNU 4903, there is fairly little of use left in this directory. One notable exception, however, is the lldbmacros/ subdirectory, which contains useful Python language macros for use by `lldb` when in kernel debugging mode. The various macros are packaged into the .dSym companion files during the build process, and can then be used in `lldb` when the .dSym is loaded.

Each .py file in lldbmacros/ handles one or another domain in the kernel. The macros are commented along with their usage, which is also commonly displayed when the `help` command is used. As with the __CTF section, these macros can elucidate some of the redacted portions of XNU - skywalk.py provides clues as to the structure of Nexuses, channels and NECP details.

Kernel Headers

For developers to interface with the kernel, they need to be able to use kernel headers. Those headers sections not explicity marked with `PRIVATE` or `KERNEL_PRIVATE` are visible through the public Kernel.framework. Despite the misleading name, this is not an actual framework, just a collection of headers which may be `#included` when compiling a kernel extension in Xcode. Unfortunately, a fair amount of them is largely empty, due to the `#ifdefs` automatically compiling out their definitions.

The headers are not available unless the SDK has been installed (as is commonly done during Xcode's installation). As is discussed next chapter, the visible headers form the **Kernel Programming Interfaces**, or KPIs.

Output 2-8: The kernel headers (Kernel.framework)

```
#
# If XCode installation is complete, the headers will be in /System/Library/Frameworks/Kernel.framework.
# Otherwise (or if looking into different SDKs) check the directory path under XCode:
#
morpheus@Chimera (.../Headers)$ pwd
/Applications/Xcode.app/Contents/Developer/Platforms/MacOSX.platform/Developer/SDKs/MacOSX.sdk/
 System/Library/Frameworks/Kernel.framework/Headers
morpheus@Chimera (.../Headers)$ ls -F
AppleDSP/               UserNotification/       ipc/                    net/                    stdint.h
AssertMacros.h@         architecture/           kdp/                    netinet/                string.h
Availability.h@         atm/                    kern/                   netinet6/               sys/
AvailabilityInternal.h@ bank/                   kextd/                  netkey/                 uuid/
AvailabilityMacros.h@   bsm/                    libkern/                os/                     vecLib/
AvailabilityVersions.h@ console/                lockd/                  pexpert/                vfs/
DriverKit/              corpses/                mach/                   ptrauth.h               vm/
HIDDriverKit/           default_pager/          mach-o/                 security/               voucher/
IOKit/                  device/                 mach_debug/             stdarg.h                x86_64/
IOKitUser/              gssd/                   machine/                stdatomic.h
TargetConditionals.h@   hfs/                    math.h                  stdbool.h
USBDriverKit/           i386/                   miscfs/                 stddef.h
```

Compiling the kernel

Just because a code base is open source doesn't necessarily mean it is straightforward to compile. This is also true of XNU, which requires working through its many dependencies. Fortunately, the process has been well documented in several references[1] (including the first edition of this work), and even made into a script[2]. If followed carefully should not only work on existing kernel versions, but be applicable on future ones.

- **AvailabilityVersions**: A perl script which generates all supported versions of the Darwin flavor (`--macosx|--osx|--ios|--appletvos|--watchos|--bridgeos`)
- **DTrace**: Providing support for the powerful debugging facility. This is required for optional `CONFIG_DTRACE` dependencies, as well as the Compact Type Form (CTF) tools (`ctfmerge`, etc).
- **libplatform** (discussed in I/8) is required for its headers.
- **libdispatch** (discussed in I/8) is required in order to build its subproject, libfirehose. Specifically, the static archive libfirehose_kernel.a is required so that the kernel can utilize the `os_log` facility (q.v. I/15).

> For most use cases, it might not be necessary to compile the kernel. Apple is generally consistent in releasing the "Kernel Debug Kit" (commonly referred to as the KDK) as an "additional download" at http://developer.apple.com. This DMG includes a .pkg which expands to install both a `DEVELOPMENT` and `DEBUG` build of the kernel, as well as symbols of most IOKit families, along with handy `lldb` macros. Unless advanced kernel tweaking or source code modification is required, these kernels are often more than sufficient.

Compiling for x86_64

XNU's source tree is configured to compile correctly as an x86_64 build. This is despite the fact that Apple does not include the source code for the Nexus subsystem or for XCPM. As mentioned, however, the trick is in getting the dependencies right.

AvailabilityVersions

The `AvailabilityVersion` is a simple Perl script which, although shipped with a Makefile, can just be manually copied to /usr/local/libexec, where it needs to be so that the build process can find it. Doing so is a simple matter:

Listing 2-9: Installing the AvailabilityVersions dependency

```
# Extract the AvailabilityVersions project tar (assumes any version is present)
tar zxf Availability*.tar.gz ; rm Availability*tar.gz
# Enter extract directory to build
cd Availability*
sudo cp availability.pl /usr/local/libexec
```

DTrace CTF tools

The x86_64 supports DTrace, for which a special kernel segment, `__CTF.__ctf` must be present. The CTF segment contains the Compact Type Form encodings of the various kernel symbols and structures, which in turn allow DTrace to run scripts which address them. During the build process, the Makefile will look for three tools - `ctfconvert`, `ctfdump` and `ctfmerge`, which are not present by default. Obtaining the DTrace project will enable building them. For historical reasons, these require compatibility with (the dearly departed) Solaris, whose headers are included in the package. XCode's build environment, however, has to be directed accordingly:

Listing 2-10: Installing the DTrace CTF dependency

```
# Extract the DTrace project tar (whichever version, assumes one is present)
tar zxf dtrace-*.tar.gz ; rm dtrace-*.tar.gz
# Enter extract directory to build
cd dtrace-*
mkdir obj sym
export DSTROOT=/  # Install directly to XCode toolchain:
xcodebuild  install -sdk macosx \
 -target ctfconvert -target ctfdump -target ctfmerge \
 ARCHS=x86_64 \
 SRCROOT=$PWD OBJROOT=$PWD/obj SYMROOT=$PWD/sym  \
 HEADER_SEARCH_PATHS="$PWD/compat/opensolaris/** $PWD/lib/**"
```

Aside from being required for kernel compilation, the `dtrace` projects holds the valuable yet overlooked tool that is `ctfdump`. This utility can be used on the kernel binary or any other containing a Compact Type Form (CTF) section, to display structures, typedefs and functions. This not only helps piece together redacted components of XNU, but instantly reveals offsets of any structure field!

libplatform headers

As discussed in I/8, libsystem_platform is one of Darwin's libSystem re-exports, containing low level CPU operations, such as `os_lock*` and `OSAtomic*` operations, and the `os_alloc_once` facility. Its headers are not included in the standard SDK, but (fortunately) this library is open source and so the headers can be reconstructed fairly easily:

Listing 2-11: Installing the libplatform header dependency

```
# Extract the libplatform project tar (whichever version, assumes one is present)
tar zxf libplatform-*.tar.gz ; rm libplat-*.tar.gz
# Enter extract directory to build
cd libplatform-*
# Install headers onto SDK's /usr/include/os/internal, which needs to be created..
sudo mkdir -p `xcrun -show-sdk-path`/usr/include/os/internal
sudo cp -pR $PWD/private/os/internal `xcrun -show-sdk-path`/usr/include/os/internal
```

libdispatch (firehose)

Starting with Darwin 16, XNU outputs a plethora of messages into the very aptly named "firehose" facility. This enables log messages to be collected with the `log(1)` facility, discussed in I/15. This has resulted in a new dependency of XNU's - on os/firehose_buffer_private.h. The file can be found as part of libdispatch's private headers. Unfortunately, this step is not as easy as libplatform's headers were, owing to a fix the headers require, and their own dependency on those of XNU's:

Listing 2-12: Installing the libdispatch header dependency

```
# First, install XNU's headers - you need those for libdispatch to build..
cd XNU source directory
make SDKROOT=macosx ARCH_CONFIGS=X86_64 installhdrs
sudo cp -pR $PWD/BUILD/dst $(xcrun -sdk macosx -show-sdk-path)
# Extract the libdispatch project tar (whichever version, assumes one is present)
tar zxf libdispatch-*.tar.gz ; rm libdisp-*.tar.gz
# Enter extract directory to build
cd libdisp-*
mkdir dst sym obj
# Modify the xcconfig file (XCode's build settings) so as to build for kernel
awk '/include "<DEVELOPER/ {next;} /SDKROOT =/  \
     {print "SDKROOT = macosx"; next;} {print $0}' \
xcodeconfig/libdispatch.xcconfig > /tmp/foo &&
    mv -f /tmp/foo xcodeconfig/libdispatch.xcconfig
awk '/#include / { next; } { print $0 }' \xcodeconfig/libfirehose_kernel.xcconfig > /tmp/foo &
    mv -f /tmp/foo xcodeconfig/libfirehose_kernel.xcconfig
# Build
xcodebuild install -target libfirehose_kernel SRCROOT=$PWD OBJROOT=$PWD/obj \
  SYMROOT=$PWD/sym DSTROOT=$PWD/dst
# Install headers
sudo cp -pR $PWD/dst/usr/local  `xcrun -show-sdk-path`/usr/local
```

At this point, all dependencies are hopefully satisfied, and you can enter the XNU source directory, and follow the `README.md`, and build any or all subversions of the kernel - `RELEASE`, `DEVELOPMENT` or `DEBUG`.

Compiling for ARM64

With the advent of the ARM source branch, included in XNU's source tree as of XNU 4570, compiling XNU for ARM or ARM64 has become a viable option.The process generally builds on the prerequisites for x86_64, but requires overcoming a few more hurdles. The first to discuss the steps to do so was ProteasWang[3], and the process has since been fully documented[4].

Note, that one hurdle remains, and it is presently insurmountable: Apple does not provide the code to many of its kernel extensions (outside some IOKit families). This means that an ARM64 XNU kernel can minimally boot, but with no actual support for any hardware. There is also another issue of adapting some non-iBoot boot loader to boot XNU.

Recreating the full set of kernel extensions is arduous and likely unfeasible. Still, XNU built in this way should prove useful to run command line experiments. Another approach is to try and get the stock iOS kernel (from an .ipsw) to run in an emulator. Two posts by Zhuo Wei[5, 6] provide a detailed account on getting the kernelcache to boot into userspace on a heavily modified QEMU. This has been corroborated and expanded upon by Aleph Security.

Conditional compilation directives

XNU is abundant with conditional compilation directives, and most (but not all) are prefixed by `CONFIG_`. These directives have far reaching implications, as they directly affect critical code flows in the kernel, and the size of key `structs`. If one goes to the trouble of recompiling the kernel (rather than defaulting in MacOS to the KernelDebugKit.dmg, it is usually to tweak some of those configuration options.

The config/MASTER file specifies the various directives, and most are also well commented. The file also sets those values which are machine independent. Table 2-13 shows some of the key ones, indicating their settings in MacOS and the *OS variants.

Table 2-13: The key conditional compilation directives in XNU

#define directive	MacOS	*OS	Impact
CONFIG_ATM	+	+	Activity Trace Manager support
CONFIG_DTRACE	+	-	DTrace support
CONFIG_EMBEDDED	-	+	Defines *OS builds, as opposed to MacOS
CONFIG_GZALLOC	+	-	Enables the Guard Mode Zone Allocator (q.v. Chapter 12)
CONFIG_IOSCHED	+	+	Darwin 14: I/O Scheduling
CONFIG_JETSAM	-	+	Jetsam (low memory killer)
CONFIG_KASAN	-	-	Kernel Address Sanitizer and profiling
CONFIG_MACF	+	+	Mandatory Access Framework support (III/4)
CONFIG_PERSONAS	-	+	Personae support (I/8)
CONFIG_SECLUDED_MEMORY	-	+	Secluded memory set aside for Camera, mediaserverd
CONFIG_TRIGGERS	+	+	VNode Trigger (Chapter 7)
DIAGNOSTIC	-	-	Various diagnostic support, particularly for VFS
HIBERNATION	+	-	Hibernation support code
KPC	-	-	Kernel Performance Counters
MONOTONIC	-	+	Monotonic counter support
NEXUS	+	+	Nexus support (I/16). Stripped, though [IPSEC/UTUN]_NEXUS remain
NO_KEXTD	-	+	Disables kextd(8) upcalls and IOCatalog downcalls
NVNDEVICE	-	-	Provides /dev/vn# (up to NVNDEVICE) nodes for loop mounts
SECURE_KERNEL	-	+	Disables certain syscalls and MIG subsystems (e.g. kas_info)
__SMP__	+	+	Symmetric MultiProcessing (or multicore)
VM_MAPPED_KEXTS	+	-	Standalone, loaded kexts

Kernel Command Line

The kernel, like most programs, can accept arguments. Unlike user mode programs, however, there is no `argc` and `argv`: Instead, there is a command line string, which is prepared by the boot loader (EFI or iBoot), from any pre-existing arguments in the device tree, NVRAM or (if applicable) user input. In many cases, choosing the right value for a boot argument can alter the kernel behavior more efficiently than recompiling.

*OS variants no longer allows passing boot arguments to the kernel from `RELEASE` builds of iBoot as of around iOS 5.0 - a wise decision considering the then prevalent "limera1n" boot chain exploit which allowed users to pass any argument they chose, including those disabling security mechanisms. In MacOS arguments may still be passed through the `boot-args` NVRAM variable, or through `/Library/Preferences/SystemConfiguration/com.apple.Boot.plist`, in the value of the `Kernel Flags` key. Due to its extreme importance, this file is SIP restricted.

When booting, XNU does not take the command line directly. Rather, it is provided as one of the fields `CommandLine`) of the `boot_args` structure, which is passed to XNU by the boot loader as its sole parameter. Early during startup, this structure is saved by the Platform Expert, and PE* APIs - specifically, `PE_parse_boot_argn` or `PE_parse_boot_arg_str` - can be used to query the string and retrieve numeric or string arguments.

There are over 200 boot arguments in XNU proper, the vast majority of which are common across all Darwin flavors. They are parsed by the kernel at different points of execution, and can be collected by `grep(1)`ing the source. Table 2-14 shows some of the important ones:

Table 2-14: Some of the more important kernel command line arguments

Boot Argument	Parsed by	Controls
`-l`	`kernel_bootstrap()`	Leak logging
`-s`, `-b`, `-x`	`bsd_init()` (`parse_bsd_args()`) and passed to `launchd(8)`	Single user mode, skip boot.rc and safe boot
`-minimalboot`		Minimal userspace startup
`debug`	(various)	Debug flag bitmask (`DB_*` constants in osfmk/kern/debug.h)
`rd`	`IORamDiskBSDRootIOFindBSDRoot`	Ramdisk image
`rootdev`		Root filesystem device
`serial`	`[i386/arm]_init()`	Serial port to use as console
`-keepsyms`	(various)	Keep kernel and kext symbols - useful in `panic` dumps
`-show_pointers`	`kernel_bootstrap()`	show pointers in `dmesg(8)` output (if `i_can_has_debugger()`)
`amfi_get_out_of_my_way`	`AppleMobileFileIntegrity.kext`	Disable AMFI's functions (if `i_can_has_debugger()`)

The `PE_parse_boot_arg[n/_str]` APIs is also used by kernel extensions, with many more hundreds of arguments. The source is not available for extensions outside families, but the arguments can be easily collected by first analyzing the kernelcache and then disassembling the kernel using `jtool2`: The disassembler automatically tracks parameters in function calls and a simple `grep(1)` can then isolate the string names, as well as the target kernel variable into which the value is stored.

Output 2-15: Harvesting boot arguments directly from the kernelcache using `jtool2`

```
morpheus@Chimera (~)$ jtool2 --analyze ~/Downloads/kernelcache.release.iphone11
...
Symbolicated 4914 symbols to ./kernelcache.release.iphone11.ARM64.557D0BCE-5CB7-351A-88BA-65A0A68390A3
#
# Decompile, isolating all PE_parse* API calls whose parameters were identified
#
morpheus@Chimera (~)$ jtool2 -D ~/Downloads/kernelcache.release.iphone11 | grep PE_parse_boot |
                      grep \" > /tmp/bootargs.iphone11
#
# Produce sorted, unique list
#
morpheus@Chimera (~)$ cat /tmp/bootargs.iphone11 | cut -d\" -f2 | sort -u
-aes_spew
-b
-disable_atm
-- (1000+ boot args)
zrecs
zsize
```

Kernel Debugging

Programming mistakes in kernel mode are often fatal, but not always readily detectable. Debuggers can therefore prove useful in kernel mode. MacOS `RELEASE` kernels support debugging, and Apple's Kernel Debug Kit (freely available from http://developer.apple.com/) provides `DEBUG` and `DEVELOPMENT` compiled kernels, with companion .dSym bundles for kernels and key extensions. Kernel debugging is generally not available in *OS devices, although iOS 12 betas specifically provided full symbols and enabled the Kernel Debug Protocol.

Kernel debugging is far more complicated than user mode debugging. In user-mode, debugging only affects the debuggee process - whereas kernel mode debugging affects the entire system. The quickest human response time is still on order of years for the CPU, which adversely affects system critical behavior (e.g. interrupt handling and timers) when the debugger pauses the kernel for breakpoint processing. Kernel mode debugging might stumble upon bugs which may or may not have been encountered during normal system operation.

Kernel Debug Protocol (kdp)

It is extremely hard to debug the kernel from a user mode process, since existing in the operating system will affect its behavior. It is common, therefore, to use a two-system setup, wherein the debugged system communicates with the debugger system over one of several media. KDP was originally designed to operates over a serial port, if specified by the `serial=` boot arguments. Macs no longer have a direct serial port, but KDP can still be used when one is configured in a virtual machine, over USB, FireWire, or the network. The `usbkdp(1)` and `fwkdp(1)` commands are used to proxy the protocol, and their well documented man pages detail the process, along with the required boot args (namely, `debug=` for the debug flags, and `kdp_match_name` to select which interface KDP will operate on).

The flexibility in transport selection is achieved by using a set of `kdp_*` exports (all in the Private kpi), so that Kernel extensions can register their own KDP transports - links, interfaces, and send/receive packet handlers. `IONetworkingFamily` is one such kext to do so, and it re-exports those functions to other kexts through the `IONetworkController` object's `[attach/detach]DebuggerClient` and `enable/disable` methods.

Virtual machines are perfectly suited to kernel debugging. Not only do they alleviate the need for two physical machines, but their snapshots can prove to be immensely useful, as besides being able to go back in time to the circumstances leading to a panic, they also freeze the kernel memory layout - enabling you to inspect memory over and over again without the hindrance that is KASLR. There are quite a few well detailed explanations on the Internet on how to set up a virtual machine for debugging, and the interested reader is highly encouraged to peruse them[7, 8, 9].

XNU's kdp implementation is self contained in osfmk/kdp, and is enabled `#if MACH_KDP`, which is set by default on all Darwin flavors. There are several important files in this directory, shown in Table 2-16:

Table 2-16: Files in osfmk/kdp

File	Contains
kdp_protocol.h	KDP protocol definition
kdp_core.[ch]	Kernel core dumping routines and file format
kdp.c	KDP protocol implementation
kdp_serial.[ch]	KDP packet serialization
kdp_internal.h	Prototypes for machine specific functions
ml/*arch*/kdp_[machdep/vm].c	Machine dependent (VM, exception & register specific) routines

The KDP protocol is understandably a simple one, operating over UDP port 41139 (arbitrarily chosen and hard-coded as `KDP_REMOTE_PORT`), with subsequent ports used for exception messages. The packet format uses a fixed header followed by any payload. The header format and command codes are all listed in osfmk/kdp/kdp_protocol.h and shown in Listing 2-17. The implementations can be found through the `dispatch_table` in osfmk/kdp/kdp.c.

Listing 2-17: The protocol definitions from osfmk/kdp/kdp_protocol.h

```
typedef enum {
    /* connection oriented requests */
    KDP_CONNECT = 0,    KDP_DISCONNECT = 1,

    /* obtaining client info */
    KDP_HOSTINFO = 2,   KDP_VERSION = 3,    KDP_MAXBYTES = 4,

    /* memory access */
    KDP_READMEM = 5,    KDP_WRITEMEM = 6,

    /* register access */
    KDP_READREGS = 7,   KDP_WRITEREGS = 8,

    /* executable image info */
    KDP_LOAD,       KDP_IMAGEPATH,
    /* execution control */
    KDP_SUSPEND = 11,    KDP_RESUMECPUS = 12,

    /* exception and termination notification, NOT true requests */
    KDP_EXCEPTION, KDP_TERMINATION,

    /* breakpoint control */
    KDP_BREAKPOINT_SET = 15, KDP_BREAKPOINT_REMOVE = 16,

    /* vm regions */
    KDP_REGIONS = 17,

    /* reattach to a connected host */
    KDP_REATTACH = 18,

    /* remote reboot request */
    KDP_HOSTREBOOT = 19,

    /* memory access (64-bit wide addresses). Version 11 protocol */
    KDP_READMEM64 = 20,  KDP_WRITEMEM64 = 21,

    /* breakpoint control (64-bit wide addresses). Version 11 protocol */
    KDP_BREAKPOINT64_SET = 22, KDP_BREAKPOINT64_REMOVE = 23,

    /* kernel version string, like "xnu-1234.5~6". Version 11 protocol */
    KDP_KERNELVERSION = 24,

    /* physical memory access (64-bit wide addresses). Version 12 protocol */
    KDP_READPHYSMEM64 = 25,      KDP_WRITEPHYSMEM64 = 26,

    /* ioport access (8-, 16-, and 32-bit) */
    KDP_READIOPORT = 27, KDP_WRITEIOPORT = 28,

    /* msr access (64-bit) */
    KDP_READMSR64 = 29,  KDP_WRITEMSR64 = 30,

    /* get/dump panic/corefile info */
    KDP_DUMPINFO = 31,

    /* keep this last */
    KDP_INVALID_REQUEST = 32,
} kdp_req_t;
...
typedef struct {
        kdp_req_t       request:7;      /* kdp_req_t, request type */
        unsigned        is_reply:1;     /* 0 => request, 1 => reply */
        unsigned        seq:8;          /* sequence number within session */
        unsigned        len:16;         /* length of entire pkt including hdr */
        unsigned        key;            /* session key */
} KDP_PACKED kdp_hdr_t;
```

Paragon security researcher Ian Beer demonstrated the creation of a KDP-compatible iOS debugger for iOS 11.1.2. Building on his reliable exploit for that iOS version ("async_wake"), Beer implemented his debugger by locating a gadget (reusable snippet of machine code) which sets the processor's Monitor Debug System Control Register (MDSCR_EL1) KDE bit (#13) and reenables hardware level breakpoints. Beer presented his groundbreaking work at MOSEC 2016[10]. Brandon Azad followed up by presenting a full KTRR bypass, enabling unrestricted patching of kernel read-only memory, and a full fledged debugger as this book goes to print.

Don't Panic

Though (hopefully) rare, there are cases where internal errors occur in kernel mode. Unlike faults, traps, and exceptions, which the kernel is prepared to handle, these internal errors could arise from unhandled conditions, sanity checks or assertions which somehow fail, or actual bugs and vulnerabilities. The kernel chooses death before dishonor - and calls `panic()`.

The `panic()` function is a vararg (`printf()`-style) function, which internally calls `panic_trap_to_debugger()`. As of Darwin 17, this function first checks for a kernel debugger, by calling `ml_wants_panic_trap_to_debugger()`. On all architectures, however, this function returns `FALSE`, and so panic processing continues. *OS variants contain code to call `PE_arm_debug_panic_hook()`, but this callback is NULL outside `DEVELOPMENT || DEBUG`.

If the kdebug facility is active and `write_trace_on_panic` is set (by the `trace_panic` boot argument), the current kdebug buffer is written (complete with the thread map) to /var/[tmp/log]/panic.trace for MacOS/*OS (respectively), where it can persist after reboot for analysis. Since that is not the normal case, however, the panic data must be written to another medium which will persist. `SavePanicInfo()` (from osfmk/[arm|i386/AT386]/model_dep.c), which prints the backtrace, and then calls on the Platform Expert's `PESavePanicInfo()` routines to commit the log.

MacOS allows the setting of hooks, which an interested thread can install using `panic_hook()`. When the panic log is printed (by `panic_i386_backtrace()`), `panic_check_hook()` will iterate over the queue of up to 1,024 hooks, calling the ones installed by the thread causing the panic.

The Panic report

When a panic occurs, the integrity of the kernel can no longer be trusted. It is therefore out of the question to write the report to a file, as the filesystem logic may have been corrupted, which would only exacerbate the problem. UN*X has traditionally used the swap partition to record panic dumps, since its contents are otherwise meaningless across reboots anyway. Darwin systems, however, use an NVRAM partition - "APL, OSXPanic" - to store the panic report*. The partition contains a single key - `"aapl,panic-info"`, as can be seen back in Listing 1-17.

The `"APL, OSXPanic"` partition is limited to a maximum of 2048 (0x800) bytes, which forces the report to be compressed. The compression is a simple one - `packA()` (in osfmk/kern/debug.c), which collapses 7-bit ASCII into 8-bit characters, so that every 8 characters of text take up 7 bytes. The compression is admittedly inefficient, and a comment (in `SavePanicInfo()`, osfmk/i386/AT386/model_dep.c) suggests using the WKdm compressor, which is used in XNU's memory compression, though that would require modifying user mode binaries as well.

The Panic report is collected from the NVRAM when the system report by CoreServices' DumpPanic. The daemon reads the NVRAM property, uncompresses its data, and writes it (in MacOS) to /Library/Logs/DiagnosticReports, using the naming convention Kernel_*YYYY-MM-DD-HHMMSS_Hostname*.panic. With the advent of BridgeOS, this daemon is also responsible for proxying panic reports which occur on the embedded coprocessor system, since those cannot otherwise be read outside BridgeOS. MacOS panic logs are textual, whereas those of *OS variants are JSON formatted.

MacOS's UI usually displays the panic log when the system reboots, reporting a "shutdown due to an unexpected problem", and popping up a dialog suggesting to "view the report" or "Send to Apple". In *OS there is no UI, but panic logs can be viewed with other crash reports through Settings → Privacy → Analytics), or (if filesystem access is available) in /var/mobile/Library/Logs/CrashReporter with a naming convention of panic-full-*YYYY-MM-DD-HHMMSS.mmm*.ips.

Apple's TN2063[11] (though no longer maintained) describes panic reports in detail, including working with the (deprecated) `gdb` and macros of the Kernel Debug Kit. The following experiment illustrates the manual process of symbolicating panic dumps.

* - The partition name is `#defined` in iokit/IOKit/IONVRAM.h as `kIODTNVRAMPanicInfoPartitonName`. This is (yet another) typo in XNU's code, not this book. Also, note that *OS maps the first page of VRAM for panics (q.v. `check_for_panic_log()` in pexpert/arm/pe_init.c).

Experiment: Deciphering a panic log (x86_64)

Most `panic` messages are quite verbose, with a string denoting the reason (usually some assertion failure), and the file and line where the decision to panic was made.

Listing 2-18: A sample panic log

```
Anonymous UUID:       EFC6BFA7-ABE5-69AD-EDC9-76902633BDDD

Fri Apr  5 14:04:05 2019  # Date

*** Panic Report ***
panic(cpu 1 caller 0xffffff8008ccef2e): "copyio: kernel buffer 0xffffff800f7
  has size 1472 < nbytes 4096@/BuildRoot/Library/.../osfmk/x86_64/copyio.c:1
Backtrace (CPU 1), Frame : Return Address
0xffffff8064053900 : 0xffffff8008baeb0d
0xffffff8064053950 : 0xffffff8008ce8653
...
0xffffff8064053c70 : 0xffffff7f8b346db6
0xffffff8064053cc0 : 0xffffff7f8b346c02
...
  Kernel Extensions in backtrace:
     ....bundle identifier...(version) [UUID]@start-end
 com.newosxbook.xnoop(1.0)[4FCDC1E8-B495-38FC-A4FD-13EFF1980946]@0xffffff7f8
       ..
BSD process name corresponding to current thread: xnoop
Boot args: debug=0x400 -v

Mac OS version: 18D109
Kernel version: ... uname -v/kern.version string
Kernel UUID:    ... LC_UUID o ernel
Kernel slide:     0x0000000008800000
Kernel text base: 0xffffff8008a00000
__HIB  text base: 0xffffff8008900000
System model name: VMware7,1 (Mac-7DF21CB3ED6977E5)

System uptime in nanoseconds: 98641689520
last loaded kext at 81667554201: com.newosxbook.xnoop   1.0.0 (addr 0xffffff
loaded kexts:
com.vmware.kext.vmhgfs  0521.43.29
...
EOF
Model: VMware7,1, SecureROM VMW71.00V.6997262.B64.1710270607, 2 processors,
Graphics: Display, 128 MB
...
```

The inclusion of the kernel UUID enables Apple to locate the kernel and its `.dSym` companion file easily when processing the report, and auto-symbolicate it. Otherwise, you would need to work with the kernel's exported symbols (if any), and symbolicate manually. Fortunately, the process is fairly simple.

The addresses in the backtrace are all slid. The first step, therefore, is to locate the "Kernel Slide:" value, then subtract it from each address. In the example above, the slide value is 0x**88**00000, so applying this on the first address of the stack trace - 0xffffff800**8b**aeb0d, will yield ffffff80003aeb0d. This value will always be in the kernel image, or in some kext. In the former case, it can be located with `jtool2`. Note that addresses of kernel extensions easily stand out (e.g. `0xffffff7f8b346db6`, and the panic log automatically deduces their owning kext.

A good indication you are doing this correctly is to disassemble the kernel binary one instruction before the value of the return address (or, in ARM64, `lr`). The intel instruction should be a `CALL`, and the ARM64 one a `BL` (Branch and Link). If that is not the case, you might have a taken a wrong turn somewhere, as these are the instructions which store the return address. Otherwise, working back to the containing function will give you its name (if any), and the offset into the call.

Using `jtool2`'s `--symbolicate` option on a recovered `.ips` panic file will perform the entire symbolication process automatically, so long as the original kernel (or, in *OS, kernelcache) image or its companion file can be found. This feature is especially useful for *OS panics, along with `--analyze`, which recovers thousands of symbols from the otherwise symbol-stripped kernelcaches. In MacOS, setting `keepsyms=1` in `boot-args` will include symbols in the panic dump.

Kernel Core Dumps

There are cases where a panic report might be insufficient to determine the root cause of a problem. XNU can be configured to dump its full kernel memory image, in the same format as a user mode core file.

Kernel core dumping is performed by `kern_dump()` (in osfmk/kdp/kdp_core.c), which takes a `kd_variant` parameter of `KERN_DUMP_DISK` (to a local Mach-O `MH_CORE` file), `KERN_DUMP_SHMEM_DBG` (*OS, to shared memory, but only in `DEVELOPMENT` kernels) or `KERN_DUMP_NET` (over a network connection). With the first two being unreachable code paths in release kernels, it leaves the third as the only viable option for MacOS. The core dump is then sent over UDP/IP port 1069 to a remote `kdumpd(8)` server.

On the receiving end of the dump, `kdumpd(8)` must be explicitly enabled, and its default directory, /PanicDumps, created with permissions 01777, as explained in Apple's TN2118[12]. On the sending end, the following `boot-args` control kernel core generation:

Table 2-19: The command line arguments controlling kernel panic core dump generation

Argument	Defines
debug	• 0x400 (`DB_KERN_DUMP_ON_PANIC`) - kernel dump when `panic()` is called • 0x800 (`DB_KERN_DUMP_ON_NMI`) - kernel dump, rather than drop to debugger • 0x1000 (`DB_DBG_POST_CORE`) - wait in debugger post core • 0x4000 (`DB_REBOOT_POST_CORE`) - reboot post core • 0x2000 (`DB_PANICLOG_DUMP`) - send panic log, instead of a core dump • 0x40000 (`DB_DISABLE_GZIP_CORE`) - do not gzip kernel core
_panicd_ip	IPv4 with active kdumpd(8)
_panicd_port	UDP port. Usually omitted, so that default port (1069) is used
_router_ip	Required if `_panicd_ip` is not on same subnet
_panicd_corename	Filename to dump core as. Usually omitted, so that default is used

Core dumps reflect the entire kernel `vm_map` (also known as the `kernel_map`), and can grow to several GB. The default behavior is to compress them using the kernel's built-in `gzip` support. The filename created will be `core-xnu-`*`version-IPAddress-hexTimeStamp`*`.gz`. When `gunzip(1)`ed, the file is a Mach-O core, with the same format as user mode cores: First, a large number of `LC_SEGMENTS`, corresponding to each `vm_region` (`vm_map_entry` of the `kernel_map`). These are followed by a single `LC_THREAD` command, capturing the state of all threads at the time of the panic. Lastly, the otherwise obsolete `LC_IDENT` contains the kernel's version string, with an appended `stext` value to deduce the kernel slide.

Output 2-20: Demonstrating a kernel core dump

```
root@Zephyr (/PanicDumps)# ls -l
total 230864
-rw-rw----  1 nobody  wheel  109868783 Apr  5 16:10 core-xnu-4903.241.1-192.168.0.117-a442aef1.gz
root@Zephyr (/PanicDumps)# gunzip core-xnu-4903.241.1-192.168.0.117-a442aef1.gz
root@Zephyr (/PanicDumps)# ls -l
total 640896
-rw-rw----  1 nobody  wheel  324460544 Apr  5 16:10 core-xnu-4903.241.1-192.168.0.117-a442aef1
root@Zephyr (/PanicDumps)# file core-xnu-4903.241.1-192.168.0.117-a442aef1
core-xnu-4903.241.1-192.168.0.117-a442aef1: Mach-O 64-bit core x86_64
root@Zephyr (/PanicDumps)# itool2 -l core-xnu-4903.241.1-192.168.0.117-a442aef1
LC 00:   LC_SEGMENT_64 Mem: 0xffffff7f9e800000-0xffffff7f9e877000
...
LC 1091: LC_SEGMENT_64 Mem: 0xffffff888aa45000-0xffffff888b206000
LC 1092: LC_THREAD     Containing 1 thread states in 184 bytes
         x86_THREAD_STATE64:
         rax 0xffffff8024354000 rbx 0x00007ffeea5ddf60 rcx 0xffffff8024354000 rdx 0x0000000000000060
         rdi 0x0000000000000020 rsi 0x0000000000000000 r8  0x00007ffeea5ddf60 r9  0x0000000000000000
         r10 0xffffff80291203e0 r11 0x000000044b8941de r12 0x0000000000000018 r13 0x0000000000000001
         r14 0x0000000000000000 r15 0xffffff8024374010 rflags: 0x0000000000010202
         rbp 0xffffff807937bc20 rsp 0xffffff807937bbb8 rip 0xffffff801dffe866
LC 1093: LC_IDENT      Darwin Kernel Version 18.2.0: Thu Dec 20 20:46:53 PST 2018; root:xnu-4903.241.1-1/
                       RELEASE_X86_64;UUID=1970B070-E53F-3170-83F3-1B95FA340695; stext=0xffffff801de06000
```

If `lldb` can locate the binary of the core-generating kernel, it will automatically symbolicate the stack trace, by loading the symbols for the kernel proper and all of its kexts. This makes it far easier to diagnose the reason for the panic, as shown in Output 2-21:

Output 2-21: Diagnosing a kernel core dump with `lldb`

```
root#sinclacrum (/tmp) # lldb -c core-xnu-4903.241.1-192.168.0.117-a442aef1
(lldb) target create --core "core-xnu-4903.241.1-192.168.0.117-a442aef1"
Kernel UUID: 1970B070-E53F-3178-83F3-1B95FA340695
Load Address: 0xffffff801de00000
Kernel slid 0x1dc00000 in memory.
Loaded kernel file //System/Library/Kernels/kernel
Loading 115 kext modules ..................................................... done.
Core file '/Users/morpheus/core-xnu-4903.241.1-192.168.0.117-a442aef1' (x86_64) was loaded.
(lldb) bt
* thread #1, stop reason = signal SIGSTOP
  * frame #0: 0xffffff801dffe566 kernel`zone_element_size + 54
    frame #1: 0xffffff801e0ceb9e kernel`___lldb_unnamed_symbol983$$kernel + 142
    frame #2: 0xffffff801e4d4099 kernel`uiomove64 + 441
    frame #3: 0xffffff7fa0758db6
    frame #4: 0xffffff7fa0758c02
    frame #5: 0xffffff801e238920 kernel`spec_read + 656
    frame #6: 0xffffff801e21ea03 kernel`___lldb_unnamed_symbol1331$$kernel + 627
    frame #7: 0xffffff801e4edb57 kernel`___lldb_unnamed_symbol3461$$kernel + 279
    frame #8: 0xffffff801e4ed8c3 kernel`read_nocancel + 115
    frame #9: 0xffffff801e5b62bb kernel`unix_syscall64 + 619
    frame #10: 0xffffff801df5c466 kernel`hndl_unix_scall64 + 22
```

If the Kernel Debug Kit is installed and one of its kernels is used, then `lldb` backtraces (and the entire debugging session) will be more verbose. In fact, `lldb` can be set to source-level debugging. The KDK files hardcode Apple's internal source path (/Library/Caches/com.apple.xbs/Sources/xnu/xnu-*version*), but this can easily be adjusted using the `settings set target.source-map` to redirect the path to an extracted XNU source tarball.

Coredump helpers

MacOS XNU (and presumably *OS, in DEBUG builds) provides **coredump helpers**. These helpers can be installed By calling `kern_register_coredump_helper()`, one of whose arguments is a `kern_coredump_callback_config` structure, specifying callbacks for various points of creating the kernel core dump. The structure is defined in osfmk/kdp/processor_core.h):

Listing 2-22: The coredump helper API (from osfmk/kdp/processor_core.h)

```
typedef struct {
 kern_return_t (*kcc_coredump_init) (void *refcon, void *context); /* OPTIONAL -- ..*/
 kern_return_t (*kcc_coredump_get_summary)
    (void *refcon, core_save_summary_cb callback, void *context);
 kern_return_t (*kcc_coredump_save_segment_descriptions)
    (void *refcon, core_save_segment_descriptions_cb callback, void *context);
 kern_return_t (*kcc_coredump_save_thread_state)
    (void *refcon, void *buf, core_save_thread_state_cb callback, void *context);
 kern_return_t (*kcc_coredump_save_sw_vers)
    (void *refcon, core_save_sw_vers_cb callback, void *context);
 kern_return_t (*kcc_coredump_save_segment_data)
    (void *refcon, core_save_segment_data_cb callback, void *context);
 kern_return_t (*kcc_coredump_save_misc_data)
    (void *refcon, core_save_misc_data_cb callback, void *context); /* OPTIONAL */
 /* End of version 1 */
} kern_coredump_callback_config;

/*
 * kern_register_coredump_helper is called to register a core with the kernel
 * coredump infrastructure. In addition to the callback config and version of the config
 * structure, a description of the core should be provided -- i.e.: AP
 */
kern_return_t kern_register_coredump_helper(int kern_coredump_config_vers,
        kern_coredump_callback_config *kc_callbacks, void *refcon,
        const char *core_description, boolean_t is64bit, uint32_t mh_magic,
        cpu_type_t cpu_type, cpu_subtype_t cpu_subtype);
```

The callbacks are self-explanatory, and further documented in the header file. The actual registration is performed by The only user of this facility is KDP, which registers callbacks through the private `kern_register_xnu_coredump_helper()` in (osfmk/kdp/kdp_core.c).

Review Questions

1. What case can be made in favor of obfuscating panic dumps?

References

1. Jeremy Andrus - "Apple Kernel Development" - https://kernelshaman.blogspot.com
2. Brandon Azad - "build-xnu-4570.1.46.sh" - https://gist.github.com/bazad/654959120a423b226dc564073b435453
3. ProteasWang - XNU ARM64 Gist - https://gist.github.com/Proteas/fe7bbb4c1b35a50de5e44d7c121d9601
4. "FergoFrog" - Compiling XNU for ARM64 - https://blog.fergofrog.com/xnu/arm64/2018/07/07/compile-xnu-for-arm64.html
5. Zhuo Wei - "Almost booting an iOS kernel in QEMU" - https://worthdoingbadly.com/xnuqemu
6. Zhuo Wei - "emulate an iOS kernel in QEMU up to launchd and userspace" - https://worthdoingbadly.com/xnuqemu2
7. Klue.hithub - Debugging MacOS Kernel using VirtualBox - https://klue.github.io/blog/2017/04/macos_kernel_debugging_vbox/
8. Scott Knight - "MacOS Kernel Debugging" - https://knight.sc/debugging/2018/08/15/macos-kernel-debugging.html
9. QuarksLab - "MacOS Kernel Debugging" - https://blog.quarkslab.com/an-overview-of-macos-kernel-debugging.html
10. Ian Beer - "Build your own iOS Kernel Debugger" - https://bugs.chromium.org/p/project-zero/issues/attachment?aid=346425&signed_aid=drSMyPfPWvCZgYKtiwI2iA==
11. Apple - TN2063 - "Understanding and Debugging Kernel Panics" - https://developer.apple.com/library/archive/technotes/tn2063/_index.html
12. Apple - TN2118 - "Kernel Core Dumps" - https://developer.apple.com/library/archive/technotes/tn2004/tn2118.html

3

E X T E N D: Kernel Extensions

As is true with most kernels, the kernel proper has very minimal support, if any, for hardware. With the wide gamut of device types and configurations, it is simply unfeasible to compile support for all possible cases into the kernel, which must remain as compact and as efficient as possible. For this reason, the kernel must support its own version of loadable modules, to supply device driver functionality and link in other code into kernel space.

XNU is no different in this regard: What in Windows are *drivers* and in Linux *kernel modules* are in Darwin **kernel extensions**. But similarities end very quickly, as the architectural support and design of the extensions is quite different.

This chapter details kernel extensions. First, it focuses on the code of a simple kernel extension. It then talks about the process of loading extensions - taking a look behind the scenes at both the APIs required from user mode to enable this functionality, as well as the dedicated daemon (on MacOS) which is charged with this functionality.

There is little doubt in this author's mind that Apple is trying to get developers out of the MacOS kernel. The *OS flavors have demonstrated that a user-space only API for developers goes a long way, and in MacOS 15 Apple has announced System and Driver Extensions, with which this chapter concludes. Even if at some point the kernel is declared entirely off-limits to developers, as is the case with *OS, kernel extensions and their supporting architecture will persist, even if only for Apple's internal use.

What's in a kext

Kernel extensions in MacOS are always bundled, in that they all sport a well-defined directory structure. Under the Contents/ folder can be found the files and folders shown in Table 3-1:

Table 3-1: The common file and directory structure of a kext bundle

File/Directory	Purpose
Info.plist	The mandatory manifest property list
version.plist	Build version information (`BuildVersion`, `SourceVersion`, `ProjectName`, etc)
MacOS/	Contains the Mach-O binary of the kext, matching the Info.plist's `CFBundleExecutable`
_CodeSignature/CodeResources	Plist with hashes for Resources/*. Hash embedded in kext code signature
Resources/	Kext resources, such i18n files, firmware images, etc. Kext-dependent
PlugIns/	Dependent kexts, or other bundles. Optional, kext-dependent

In *OS variants there are no free floating kexts, as they have all been prelinked into the kernelcache. /System/Library/Extensions still exists, however, for those kexts which do have Resources/ or Plugins.

The Info.plist

As with most other bundle types, a kext is required to have an Info.plist. The Info.plist has the usual set of `CF*` keys, and the Xcode generated keys (`DT*` keys, capturing the developer tools metadata). The `CFBundleIdentifier` and `CFBundleVersion` serve as the kext's identifier and version number, and `CFBundlePackageType` is always set to `KEXT`. Extensions also use special `OS*` keys (attributed to `OSKext` APIs):

- **`OSBundleLibraries`**: is a required array of dependencies. Each dependency is listed as a `CFBundleIdenfifier` key, and a `string` value providing the version. These are generated automatically by running `kextlibs(8)` on the kernel extension's binary.
- **`OSBundleRequired`**: is an optional `string` specifying startup configurations for which loading this kernel extension is a prerequisite. Possible values (as defined in libkern/libkern/OSKextLib.h) are:

Table 3-2: The possible values of the `OSBundleRequired` Info.plist key

Value	Indicates kext...	Examples
`Console`	.. should be loaded for single-user mode	AppleTopCase.kext AppleMultitouchDriver.kext AppleHIDKeyboard.kext
`Root`	.. needed to mount root filesystem or always required	sandbox.kext AppleEFIRuntime.kext
`Local-Root`	.. needed to mount the root filesystem locally	hfs.kext apfs.kext AppleEffaceableStorage.kext
`Network-Root`	.. needed to mount the root filesystem remotely	IONetworkingFamily IO80211FamilyV2.kext
`Safe Boot`	.. safe to load when booting into safe mode	IOGraphicsFamily.kext IOSurface.kext

- **`OSBundleCompatibleVersion`**: is an optional version `string` declaring the minimum (oldest) version other kernel extensions can consider this kext to be, if linking with it.
- **`OSBundleAllowUserLoad`**: is an optional `boolean` (which defaults to `false` if not specified), indicating that non-root users can load this extension as well.
- **`OSBundleEnableKextLogging`**: is an optional `integer` providing the logging level set by default for this kext

Most of these keys get processed by `OSKext::setInfoDictionaryAndPath` as part of the "bare minimum validation" it provides, which aims to detect version incompatibilities and minor plist value malformations. `OSBundleAllowUserLoad` is enforced by `kextd(8)`, and `OSBundleRequired` is currently processed by `OSKext::isLoadableInSafeBoot`, which returns TRUE for any of the values in Table 3-2.

IOKit kernel extensions also contain `IO*` keys, grouped under the `IOKitPersonalities` dictionary, which allows drivers to expose one or more "personalities", which are cataloged by IOKit and then used to locate and call on the kernel extension, in a process known as driver matching, of which much will be told in Chapter 13.

Both `OS*` and `IO*` keys may be architecture variant (suffixed with *`_arch`*). If an architecture variant key exists alongside the regular one, it takes precedence, as `OSKext::getPropertyForHostArch` (libkern/c++/OSKext.cpp) handle attempts it first. An example of this can be found in the AppleKeyStore.kext , whose MacOS version specifies an `OSBundleLibraries_x86_64` with different dependencies than those of the *OS variants.

Special Info.plist keys

Most Info.plist consider the standard set of keys described above. A few keys, however, are less common and have special significance:

- **`OSKernelResource`**: a `boolean` set in `com.apple.kpi.*` pseudo-kexts (discussed next). Marks the kext as referring to a kernel component.
- **`AppleKernelExternalComponent`**: This `boolean` marks the extension as refactored XNU code which is required for normal kernel operations. XNU provides function shims and a registration function KPI, allowing the kexts to "plug in" when registered. The special key is detected early during kernel initialization (specifically, in `StartIOKit`), ensuring their functionality is available as early as possible.

  The family of kexts, easily recognized by their `com.apple.kec` bundle identifier prefix, presently includes three extensions - CoreCrypto, Libm, and pthread. Although not part of XNU, these are still open source in their own respective projects.

- **`AppleSecurityExtension`**: is another boolean marking the extension as providing security services for the kernel. These, too, are loaded as early as possible: from `mac_policy_initmach()`, during the MAC framework initialization, even before BSD.

  Unlike the `kec` extensions, these kexts have no recognizable prefix. On all flavors, they include `AppleMobileFileIntegrity`, and the `Sandbox`. iOS variants (as of 12) also include `CoreTrust` and `AppleImage4` (on both of which AMFI depends). MacOS security extensions also include `ALF` (the Application Layer Firewall), `Quarantine` (GateKeeper) and `TMSafetyNet` (the Time Machine auto-backup support).

- **`OSBundleForceDTraceInit`**: Indicates kext has DTrace probes, and should therefore be registered with DTrace after it is loaded and right before it is started (by a callout from `OSKext::load()`). This key is rare, but has been spotted in *OS's `AppleMCA2_T801*`.

The Kext Mach-O

Although they are kernel-mode objects, the semantics of the Mach-O are the same as those of user mode. There are three segments (`__TEXT`, `__DATA` and `__LINKEDIT`), and a few other load commands, as shown in Output 3-3:

Output 3-3: Comparing the kext Mach-O structure of a MacOS and iOS kext

```
#
# MacOS 14 - from System/Library/Extensions/AppleMobileFileIntegrity.kext:
#
LC 00: LC_SEGMENT_64          Mem: 0x000000000-0xe000    __TEXT
   Mem: 0x000006fc-0x000009210 __TEXT.__text    (Normal)
   Mem: 0x000009210-0x00000da3b __TEXT.__cstring (C-String Literals)
   Mem: 0x00000da40-0x00000ddaf __TEXT.__const
   Mem: 0x00000ddb0-0x00000dffa __TEXT.__os_log
LC 01: LC_SEGMENT_64          Mem: 0x00000e000-0x11000   __DATA
   Mem: 0x00000e000-0x00000e098 __DATA.__got
   Mem: 0x00000e098-0x00000e0b8 __DATA.__mod_init_func (Module Init Function Ptrs)
   Mem: 0x00000e0b8-0x00000e0c8 __DATA.__mod_term_func (Module Termination Function Ptrs)
   Mem: 0x00000e0d0-0x00000f838 __DATA.__const
   Mem: 0x00000f840-0x00000fb60 __DATA.__data
   Mem: 0x00000fb60-0x00000fbf0 __DATA.__common (Zero Fill)
   Mem: 0x00000fbf0-0x00001074O __DATA.__bss    (Zero Fill)
LC 02: LC_SEGMENT_64          Mem: 0x000011000-0x26000   __LINKEDIT
LC 03: LC_SYMTAB
   Symbol table is at offset 0x102e8 (66280), 1427 entries
   String table is at offset 0x18500 (99584), 40616 bytes
LC 04: LC_DYSYMTAB
         931 local symbols at index     0
          13 external symbols at index  931
         483 undefined symbols at index 944
          No TOC
          No modtab
          No Indirect symbols
LC 05: LC_UUID                UUID: A69BF990-ABB3-3731-A7A2-9C1FB76BCF6F
LC 06: LC_SOURCE_VERSION      Source Version:          283.231.1.0.0
LC 07: LC_CODE_SIGNATURE      Offset: 140208, Size: 10480 (0x223b0-0x24ca0
#
# iOS - kextracted from kernelcache.release.iphone10
#
LC 00: LC_SEGMENT_64              Mem: 0xffffff00585ee80-0xffffff0058850a0      __TEXT
   Mem: 0xffffff00585f488-0xffffff005861cc7    __TEXT.__cstring (C-String Literals)
   Mem: 0xffffff005861cc8-0xffffff005884f47    __TEXT.__const
   Mem: 0xffffff005884f47-0xffffff0058850a0    __TEXT.__os_log
LC 01: LC_SEGMENT_64              Mem: 0xffffff00602b000-0xffffff006031580      __TEXT_EXEC
   Mem: 0xffffff00602b000-0xffffff006030fc8    __TEXT_EXEC.__text (Normal)
   Mem: 0xffffff006030fc8-0xffffff006031580    __TEXT_EXEC.__stubs (Symbol Stubs)
LC 02: LC_SEGMENT_64              Mem: 0xffffff00775ee80-0xffffff00775f00c      __DATA
   Mem: 0xffffff00775ee80-0xffffff00775ef50    __DATA.__data
   Mem: 0xffffff00775ef50-0xffffff00775efd8    __DATA.__common (Zero Fill)
   Mem: 0xffffff00775efd8-0xffffff00775f00c    __DATA.__bss    (Zero Fill)
LC 03: LC_SEGMENT_64              Mem: 0xffffff006df03c0-0xffffff006df2210      __DATA_CONST
   Mem: 0xffffff006df03c0-0xffffff006df0820    __DATA_CONST.__got
   Mem: 0xffffff006df0820-0xffffff006df0838    __DATA_CONST.__mod_init_func (Module Init Function Ptrs)
   Mem: 0xffffff006df0838-0xffffff006df0848    __DATA_CONST.__mod_term_func (Module Termination Function Ptrs)
   Mem: 0xffffff006df0848-0xffffff006df2210    __DATA_CONST.__const
LC 04: LC_SEGMENT_64              Mem: 0xffffff007a38000-0xffffff007a38000      __LINKEDIT
LC 05: LC_SYMTAB
LC 06: LC_DYSYMTAB                          No local symbols
          No external symbols
          No undefined symbols
          No TOC
          No modtab
          No Indirect symbols
LC 07: LC_UUID                UUID: 672A8CC4-33C2-38EB-9AC8-1C885BA87CA1
LC 08: LC_SOURCE_VERSION      Source Version:          283.200.44.0.0
LC 09: LC_SEGMENT_SPLIT_INFO
```

The `__TEXT` segment is used for read only and/or executable data: Kext code is in `__TEXT.__text`, with ASCII strings and constant data in their respective sections. As of MacOS 12, `__TEXT.__os_log` contains format strings for log messages the kext reports with `__os_log_internal`. Similarly, `__DATA` is used for read/write data, with the Global Offset Table (`__DATA.__got`), "constant" data (`__DATA.__const`), etc. `__DATA._mod_init_func` contains pointers to static initializers, and `__DATA.__mod_term_func` likewise for destructors. MacOS does not enforce these critical sections be read-only, but prelinked kexts in *OS kernels correct this potential vulnerability, and use `__DATA_CONST` instead.

Extensions have a `__LINKEDIT` segment, but there is no `dyld` to link them. Instead, the kernel contains its own loader for extensions - `kxld`. Dependencies are specified in `LC_SYMTAB`/`LC_DYSYMTAB`, and they link with kernel symbols. The `LC_UUID` and `LC_SOURCE_VERSION` have no linkage significance. The `LC_CODE_SIGNATURE` is a code signing blob. Note, however, that the signature is assumed to have been *validated in user space* by `kextd(8)`, as discussed later in this chapter (under "kext code signing").

Experiment: Creating a kernel extension

Apple provides a reasonably detailed documentation on kernel extension programming in the "Kernel Extension Programming Topics" Guide. After a short sermon trying to disuade and dishearten the avid kernel developer, the document provides a clear walk through of how to create a skeleton kext. Apple has since pulled the PDF version of this valuable document, but mirrors of it can still be found on the internet[1], and in this book's resources pages. Since kernel extension programming is crucial for testing code in kernel, it's a good idea to go through the steps briefly:

- XCode can generate most of the boilerplate code automatically, when the project selected is a "Generic Kernel Extension" or "IOKit Device Driver". The latter is a subset of the former, and not only creates the extension skeleton, but also modifies the Info.plist accordingly, and adds C++ support.
- The kext's entry and exit pointers (`__realmain` and `__antimain`) will be generated to point to `..._start` and `..._stop`, but those two will require manual implementation. Both functions take the kext's `kmod_info` structure (by reference), and a void pointer (`kmodStartData`, which is nowadays just NULL). The return code should be `KERN_SUCCESS` in both cases, unless errors occurred which would prevent loading/unloading, respectively. Note, however, that some kexts - especially malicious ones - are fine with returning a failure from their `..._start`, after achieving whatever kernel-related goals (e.g. patching) they require.
- The new kext's dependencies (`OSBundleLibraries`) will be empty, so these, too, need to be added. The `kextlibs(8)` command can be used to resolve dependencies, by iterating through the kext binary's unresolved symbols and locating them in /System/Library/Extensions. Using `kextlibs(8)` with `-xml` will produce XML compatible output, which is then easy to cut and paste into the Info.plist.
- The resulting product (in build/[Debug|Release]) will also need to be `chmod(1)`ed and `chown(1)`ed approriately (i.e. recursively, on the entire directory structure), so as to meet the kext security requirements discussed later in this chapter.

Once created, a kext can be loaded by `kextload(8)` or `kextutil(8)`. Both binaries can be instructed to crank up the logging to the maximum possible, specifying a level from 0 to 6, or (preferably) with a bitmask, as specified in `kext_logging(8)` man page. Of the two tools, `kextutil(8)` presents the full picture, because it operates directly, rather than use `kextd(8)`.

The Kernel Programming Interface

Developers who are not intimidated by Apple's Kernel Development Guide warning find a rich programming interface which XNU provides for extensions. The KPI presently comprises about 4,500 functions. A good way to get the full list of functions is too look at the *OS kernel exports, which are restricted to KPI functions (and a few structures) only (as opposed to those of MacOS, which have on order of 22,000 symbols, for debugging and backtrace purposes).

Apple, however, doesn't want developers using this interface as they see fit. The KPI is therefore divided into seven "buckets", implemented by "pseudokexts", which can be found in System.kext as "plugins". The Mach-O objects are all codeless, and do not actually export any functions, instead linking with those of the kernel proper. These pseudokexts are identifiable by their `com.apple.kpi` prefix, and visible as the first seven extensions in the output of `kextstat(8)`, as in Output 3-4. Table 3-5 (next page) further details the KPIs.

Output 3-4: The pseudokexts

```
#
# Pseudokexts all identify as com.apple.kpi.*, and are always first in the kextstat output:
#
morpheus@Bifröst (/)$ kextstat | grep com.apple.kpi
  1  117 0xffffff7f80c2e000 0x78f0   0x78f0   com.apple.kpi.bsd (18.5.0) 99F78B01-0923-4BC8-A89A-32B588291E23
  2   10 0xffffff7f80c36000 0x1460   0x1460   com.apple.kpi.dsep (18.5.0) F349726D-1D7C-4F3B-8799-245129C4D03C
  3  145 0xffffff7f80c02000 0x1f378  0x1f378  com.apple.kpi.iokit (18.5.0) 15DF89A5-FEF7-48EF-B1C3-8090E5BBAF91
  4    0 0xffffff7f828f5000 0x1004   0x1004   com.apple.kpi.kasan (18.5.0) 46CE5FFD-D352-49D2-BFBE-76F2605A7CEE
  5  151 0xffffff7f80c22000 0xb208   0xb208   com.apple.kpi.libkern (18.5.0) 5DC791CE-0A86-4CCA-968B-F8BDDD1F4D97
  6  133 0xffffff7f80c00000 0x1ac0   0x1ac0   com.apple.kpi.mach (18.5.0) F7066957-8392-4D2B-859B-741848F0C75B
  7   82 0xffffff7f80c3c000 0xaeac   0xaeac   com.apple.kpi.private (18.5.0) F8500ADE-FFC9-41F6-AC6D-A44D304C8BA5
  8   90 0xffffff7f80c38000 0x39d8   0x39d8   com.apple.kpi.unsupported (18.5.0) 7302336C-D807-4423-B6C8-20FB975104AE
#
# Pseudokexts reside in System.kext as plugins
#
morpheus@Bifröst (/System/Library/Extensions)$ cd System.kext/PlugIns ; ls -F
AppleMMI.kext/                  IONVRAMFamily.kext/             MACFramework.kext/
ApplePlatformFamily.kext/       IOSystemManagement.kext/        Mach.kext/
BSDKernel.kext/                 Kasan.kext/                     Private.kext/
IOKit.kext/                     Libkern.kext/                   Unsupported.kext/
#
# The pseudokexts are codeless Mach-O with the exported functions in the symbol table
#
morpheus@Bifröst (/System/Library/.../PlugIns)$ jtool -l BSDKernel.kext/BSDKernel
LC 00: LC_SEGMENT_64            Mem: 0x000000000-0x9000    __LINKEDIT
LC 01: LC_SYMTAB
        Symbol table is at offset 0x1000 (4096), 833 entries
        String table is at offset 0x4410 (17424), 13536 bytes
LC 02: LC_UUID                  UUID: 99F78B01-0923-4BC8-A89A-32B588291E23
LC 03: LC_CODE_SIGNATURE        Offset: 30960, Size: 9568 (0x78f0-0x9e50)
```

The official KPIs

The Kernel.framework's Resources/ folder contains a list of supported KPIs in two text files: SupportedKPIs-x86_64 and SupportedKPIs-all-archs.txt. The former is naturally MacOS specific, whereas the latter - originally meant to provide the KPIs common to x86 and PPC - provides those also "supported" on *OS variants. The files break down the exports of BSD, IOKit, LibKern, and Mach. These roughly correspond to XNU's source directories, although not all the functions and structures are exported. Apple maintains some documentation on the supported KPIs[2], tracking changes but not providing much beyond the declaration of the symbol. The `SupportedKPI` lists are generated by the list_supported.sh shell script in XNU's config/ directory, which also contains the .exports files used to generate all the pseudokexts. The Kernel.framework's Headers/ (requiring the SDK) provide the prototypes for all supported KPIs, structures, and types.

Unsupported

Certain KPIs are deemed "Unsupported", indicating that while at one time they may have been allowed, they are now on the deprecation list. Performing a `diff(1)` on the SupportedKPIs* list will reveal recently deprecated KPIs, and the full list can be found (as with other KPI kexts) by running `jtool -S` on the Unsupported.kext/Unsupported binary.

Apple turns a blind eye towards the use of Unsupported KPIs, seeing as a kext could require them for legacy reasons. It's not uncommon to find reliance on Unsupported KPIs - especially in Apple's own kexts. In practice, KPIs stay unsupported for a very long, if not indefinite time.

Table 3-5: The Pseudo Kexts of XNU

Pseudo-kext	com.apple.kpi	XNU source	Exports
BSDKernel.kext	bsd	bsd/	830+ symbols: VFS, Kauth, etc.
Mach.kext	kpi.mach	osfmk/	69 symbols: absolutetime, clock, semaphore, thread_call, etc
Libkern.kext	libkern	libkern/	890+ symbols: OSKext, OS* objects, and the kernel C and C++ runtimes
IOKit.kext	iokit	iokit/	2100+ symbols: IO* objects, gIO* globals, PE_* OSKext, OS* objects, and the kernel C and C++ runtimes
Not really KPIs			
kasan.kext	kasan	N/A	Darwin 18: Indicates KASAN kernel build. Nothing exported
Legacy (deprecated but still supported) KPIs			
Unsupported	unsupported	N/A	260+ deprecated functions
Mostly disallowed			
MACFramework	dsep	security/	37+ symbols: mac_* and sbuf_* APIs.
Absolutely disallowed			
Private	private	N/A	916+ symbols: IOPlatformExpert, IOCPU, ifnet, decmpfs, nexus, etc - reserved for com.apple.* kexts only

MACFramework

The Mandatory Access Control Framework's KPIs are indubitably amongst the most useful, and a(n anti)-Malware developer's dream come true: With these KPIs - particularly, `mac_policy_register()`* - a kernel extension can register callbacks to audit, block, or intercept almost every system call made by user mode clients. Apple's official stance has long disavowed the existence of malware on MacOS, however, and these KPIs are kept private. According to QA1574[3], the headers were mistakenly included in MacOS 10.5, but they persisted since despite a comment claiming they are subject to removal. Only years later in MacOS 10.14 were these headers finally removed from the Kernel.framework.

The eventual removal of the headers, however, does not remove the KPI. Apple relies on the MAC framework (discussed in detail in III/4) for its most important security mechanisms - AppleMobileFileIntegrity (III/7) and the Sandbox (III/8) - as well as (in MacOS) the less effective Gatekeeper (III/6), Managed Configuration Extensions (III/6), and Time Machine's features. Security software developers (e.g. Jonathan Zdiarsky's LittleFlocker and Patrick Wardle's BlockBlock) have long ignored the warning, and built much-needed products to augment MacOS's defenses in face of new and emerging malware. It's not unlikely that companies who carry sufficient clout with Apple be informally allowed to use the KPIs. With the enforcement of special code signatures for kernel extensions, however (as discussed later in this chapter), the use of these powerful KPI features is likely to be more tightly controlled by Apple.

Private

The Private KPIs are strictly reserved for Apple's own use. There are many useful KPIs here, such as machine layer (`ml_*`) functions, kernel level socket interfaces, network interface functions (`ifnet*`, some vnode and VFS functions, `proc` struct accessors, and Mach `vm_map` KPIs. Registration-based KPIs, such as XNU's multinode support `mnl_*` and MacOS's Hypervisor (`hv_*`) are also defined as private, since they are exported but reserved solely for Apple.

In order to guard against "misuse" by third parties, the kernel linker even goes so far as to check that any kext trying to link it will sport a `com.apple.*` bundle identifier, and an `NSHumanReadableCopyright` indicating Apple. This could have been bypassed by faking these values, but doing so would have legal ramifications. It is also more of a challenge with the advent of kernel extension code signing enforcement (unless SIP is disabled or bypassed).

---

* - Not all dependents of com.apple.kpi.dsep are policy modules requiring `mac_policy_register()`: Some kexts, like `AppleFSCompression*` require other `mac_*` functions (e.g. `mac_vnop_removexattr`)

The kernelcache

The kernel cannot successfully boot without using kernel extensions, and it commonly uses a closed set. Prelinking that set into the kernel is therefore a good idea, as it not only reduces the risk of a critical extension going missing, but also speeds up boot time. The prelinked kernel can be found on MacOS in /System/Library/PrelinkedKernels, which contains an immutablekernel and a `prelinkedkernel`. As of MacOS 14, APTickets for the immutablekernels of BridgeOS (as .im4m files) can be found there as well. The chosen prelinked kernel is symlinked from /System/Library/Caches/com.apple.kext.caches/Startup/ kernelcache, and the term kernelcache is often used instead. The original standalone (i.e. non-prelinked) kernel still resides in /System/Library/Kernels, but isn't used for booting.

*OS variants find another important advantage in prelinking the kernel: The kernelcache may be digitally signed by Apple (and further authenticated by an APTicket). This way, only prelinked extensions may be loaded, thus mitigating the risk of third party extensions. The *OS kernelcache is in - /System/Library/Caches/com.apple.kernelcaches/kernelcache.

When on disk, kernelcaches are compressed. *OS variants use Lempel–Ziv–Storer–Szymanski (LZSS), whereas MacOS (as of 10.10) use the more efficient Lempel-Ziv (LZVN) algorithm. Both cache types use the same compression header, but there are subtle differences. The MacOS `prelinkedkernel` is a universal (FAT) binary, though it only contains a single slice (`x86_64`). The *OS variants are thin, but prefix with an IMG4 (DER) header. The compression header (once thinned or out of the IMG4) is described in kext_tools's kernelcache.h:

Listing 3-6:

```
// prelinkVersion value >= 1 means KASLR supported
typedef struct prelinked_kernel_header {
        uint32_t  signature;
        uint32_t  compressType;
        uint32_t  adler32;
        uint32_t  uncompressedSize;
        uint32_t  compressedSize;
        uint32_t  prelinkVersion;
        uint32_t  reserved[10];
        char      platformName[PLATFORM_NAME_LEN]; // unused
        char      rootPath[ROOT_PATH_LEN];         // unused
        char      data[0];
} PrelinkedKernelHeader;
```

Decompressing LZSS can be achieved with the open source `lzssdec` utility (from the offset of the Mach-O magic, usually around offset 441-448). The LZVN decompression can be handled using /usr/lib/libcompression.dylib, but its `lzvnDecode` symbol remains local. Apple released the algorithm to the open source domain as LZFSE[4], and an open source implementation recreated from the libcompression's assembly[5] has been made available. `jtool2` can operate directly on either type of kernelcache.

Kernelcache structure

The kernelcache is still very much a Mach-O, but with additional segments used to combine the data of all the prelinked kexts. Table 3-7 (next page) shows this segment structure as of Darwin 16.

Kernel extensions are now split so as to separate their `__TEXT` and `__DATA` (which would otherwise be contiguous), and further differentiate read-only data, so as to facilitate kernel patch protection on these segments. There are still minor differences in kernelcache formats between MacOS and the *OS variants, owing to the latter's stringent memory protections. In MacOS, the kernel `__TEXT` segment also includes `__TEXT.__text`, whereas on the variants it includes only the constant data (`__cstring`, `__os_log`, `__const`, etc), and the `__text` section is instead in a separate `__TEXT_EXEC` section. Also, whereas the MacOS XNU uses a `__CONST.__constdata` section, the *OS variants are more particular in defining a `__DATA_CONST` segment, wherein the `__const`, and constructors/destructors of `__mod_[init/term]_func` are placed.

Table 3-7: Additional segments found in a kernelcache (as of Darwin 16)

Segment	Contents
__PRELINK_TEXT	Read-only (but not executable) sections
__PLK_TEXT_EXEC	Read-only executable (__text) of kexts
__PRELINK_DATA	Combined __DATA for all kexts
__PLK_DATA_CONST	Constant (read only) data sections
__PLK_LLVM_COV	LLVM code coverage map (-fprofile-instr-generate)
__PLK_LINKEDIT	Combined __LINKEDIT for all kexts
__PRELINK_INFO	Combined Info.plist dictionaries for all kexts
__LAST	Last address of kernel proper (__last), IOKit post constructor (__mod_init_func). KTRR kernels also have protected instructions (__pinst)
__KLD	Kernel Loader
__VECTORS	MacOS: Fault recovery section (__recover)
__HIB	MacOS: Hibernation
__BOOTDATA	*OS: presently empty

__PRELINK_INFO.__info

Recall, that the kext's Info.plist is a required component, as it is used to resolve OSBundleLibraries dependencies and declare important IOKitPersonalities. When prelinking the kext Mach-Os into the kernelcache, the information must be retained, so it is available during kext initialization. KLDBootstrap::readPrelinkedExtensions (from libsa/bootstrap.cpp) is tasked with passing the massive amalgam of all the Info.plists to OSUnserializeXML(), and iterating over the _PrelinkInfoDictionary array, and for each entry calling OSKext::withPrelinkedInfoDict, unless the kext is marked as a developer kext (relevant only for internal builds) or its dictionary contains the OSBundleRamDiskOnly when a RAM disk is not used.

The __PRELINK_INFO.__info dictionary is tightly compacted with no white space, making it hard to read. After extracting it from the kernelcache with jtool -e, a good way to view it on the command line is by using xmllint --pretty 1 or using jlutil to see it in SimPLISTic format. Safari is also a pretty good option for viewing the XML in a collapsible GUI.

The _PrelinkInfoDictionary is raw XML, and so is quite massive, taking up close to one megabyte. In an effort to remain as small as possible, in addition to eliminating white space it also uses a simple technique to reduce size: When repeating values are encountered for the first time they are given an ID attribute. From the second time onward, rather than repeating the value, they are marked with an IDREF attribute, and their element content remains empty. This only extends to value elements, not keys, and some values seem to escape this method and repeat anyway.

As of Darwin 15 (XNU-3789) the dictionary contains a _PrelinkLinkKASLROffsets key, which defines the offsets to which the kernel slide value needs to be applied. Another key, _PrelinkKCID, holds the base64 encoded data of the kernelcache's UUID.

Once all kexts are loaded and slid, OSKext::setAllVMAttributes() is called to set memory protections. The number of prelinked kexts thus loaded is stored in the IORegistry's OSPrelinkKextCount key, and the __PRELINK_INFO segment is then freed.

"1469" hard-fused kernelcaches

Another class of kernelcaches are "1469" kernelcaches, so-named by the Author due to a apparent mistake in their build process, leaving the `RC_ProjectSourceVersion` environment setting from the KernelCacheBuilder, so that it is incorrectly embedded in XNU's own `LC_SOURCE_VERSION`. These kernels have first appeared in iPhone 12 betas for the iPhone7,x (i.e. 6S) and iPad4,x series, but have since been adopted by the iPhone11,x as well. It is reasonable to believe that this format will be the preferred one by Apple going forward. Indeed, iOS 13 betas continue this pattern, with the `LC_SOURCE_VERSION` set to the 1700s, in line with the KernelCacheBuilder string.

XNU's compilation process and the kernelcache creation is now substantially different:

- The `__PRELINK_[TEXT/EXEC/DATA]` segments and various `PLK_*` segments (`PLK_[LINKEDIT/LLVM_COV/DATA[_CONST]`) are defined, but are not mapped to the file, meaning they might as well be removed in some later version. The kext segments are now all statically and monolithically linked into those of the kernel.
- The `__PRELINK_INFO.__info` section size is significantly reduced, with the `_PrelinkLinkKASLROffsets` and `_PrelinkLinkKCID` keys removed. Further, the `PrelinkExecutableLoadAddress` of all the kexts has been removed as well, making this dictionary far less useful for reversing the kernel structure.
- In place of the missing extension load address keys, `__PRELINK_INFO` now contains two additional sections - `__PRELINK_INFO.__kmod_info` and `...__kmod_start`. The first provides a list of `struct kmod_info` structures (discussed later in this chapter), and the second an address list of the extensions' Mach-O Headers. The headers, however, now only contain a single segment, `__TEXT_EXEC.__text`, which is merged with that of the kernel. Strings and other kext data is likewise now intermingled with those of other kexts, and that of the kernel proper, by respective sections.
- Other existing segments are resectioned: `__TEXT` now contains `__fips_hmacs`, `__ustring` and an embedded `__info_plist`, and the `__TEXT_EXEC` contains an `initcode` section. The various `__llvm*` sections (which weren't mapped anyway) are removed from `__DATA`.
- Most importantly, whereas *OS kernels traditionally exported some 4,800 symbols (of the various KPIs), the 1469 kernels export none at all. This is no longer necessary, since all kernel extensions are now monolithically prelinked.

Those readers lamenting the loss of symbols can draw solace from that the `joker` tool has been revised to work with 1469 kernelcaches. As a parting gift for developers Apple left in some 12β1 kernels not just the regular set of some 4,800 or so exported symbols - but over 85,000(!) of them - including static, unexported, and kext internal symbols. Unlike the ARM64 branch inclusion in XNU 4570's sources, claiming that this, too, was intentional sounds even more far fetched - as Apple further made the unusual move of removing all iOS 12 Beta 1 versions (across all devices) from their download servers. By whatever reason, the over-symbolication proved a true boon, thanks to which `jtool2` now recovers thousands of symbols.

A12 and later kernelcaches

With the A12, Apple introduced additional important modifications to the kernelcache layout:

- All pointers have been converted to a special tagged notation (in their top and otherwise unnecessary 24-bits), and in some cases (e.g. the `sysent` and `mach_trap_table` entries) replaced by tagged offsets relative to the kernel base load address. Brandon Azad was the first to document this publicly[6]. This compression scheme is similar to the one used by the `dyld_shared_cache` (discussed in I/7).
- The kernelcache segmentation is altered to add `__PPL*` segments, containing PMAP management code which is protected by the Page Protection Layer, which provides physical memory protections, aimed at preventing kernel memory overwriting, even in the case of kernel compromise (e.g. mitigating the TrustCache injection method exposed (and, effectively, all but killed) by @Xerub (III/25)). These segments include `__PPLDATA[_CONST]` and `__PPLTEXT` (similar to their regular counterparts) and a `__PPLTRAMP` containing trampolines pointers initialized at runtime.

Figure 3-8:

- The A12 has a new section, `__DATA.fwFirmwareImage`, containing the A12 PMP firmware. The A13 no longer has this, instead loading the firmware from a separate IM4P file.
- `__auth_ptr` sections have been added in `__DATA_CONST` and `__KLD`, to enable ARMv8.3 Pointer Authentication Codes (PAC).

Kext Loading: The user mode perspective

Loading kext requires collaboration between user mode (which usually initiates the request and has access to the files required) and kernel mode (which is responsible for actually loading and linking the kext in kernel space). It is user mode where kext security requirements are enforced.

Kext Security Requirements

Loading a kernel extension effectively injects code from a user space supplied binary direct into the kernel. If obtained from an untrusted source, extensions could harbor the worst kind of malware - kernel level rootkits. At a minimum, MacOS has restricted extensions by ensuring:

- Ownership by `root:wheel`: `root` is the superuser, and the `wheel` group represents the "inner circle" who are allowed to `su(1)` at will. The ownership of kernel extensions must therefore be by no others.
- Permissions of at most `rw-r--r--` (0644): less restrictivce is fine, but anything allowing writability for anyone but the `root` user is perceived as insecure, because of the potential of a malicious actor overwriting the kernel extension while on disk. Permissions on folders must be defined similarly, with `rwxr-xr-x` (0755) at maximum.

Kernel extensions have traditionally resided in /System/Library/Extensions (for Apple supplied ones) or /Library/Extensions (for third parties). Starting with MacOS 11, System Integrity Protection restricts /System/Library/Extensions. Note, however, the extensions may be loaded from anywhere in the filesystem - provided the above permission restrictions are met, though locations outside of /System/Library/Extensions cannot be used during the boot process.

Kext code signing

Restricting kernel extensions to root ownership is by no means sufficient - permissions say nothing about the origin of the extension being loaded. It is therefore necessary to ensure the integrity of extensions before allowing them to load into the kernel. Apple started doing exactly that with MacOS 10.9 and the introduction of Kext code signatures, announced to developers in WWDC2013 session 707[7].

As with other Mach-O types, kexts can be signed by an `LC_CODE_SIGNATURE` (Volume I, Chapter 6), with the code signature format deeply detailed in the next volume of this work (Volume III, Chapter 5). Validation, however, is different. Rather than use the `mac_kext_check_load` hook to perform the signature validation, Apple chose to leave it in user mode, and use the hook in order to perform an entitlement check for `com.apple.rootless.kext-management`. The entitlement holders - kextd(8), and the other `kext_tools` of kextcache/kextload/kextutil - are trusted to be the ones performing the code signature validation.

The `kext_tools` project is open source, and so it offers a view as to the validation process. The `checkKextSignature()` function in security.c (common to several of the tools) contains a hard-coded requirement string, shown in Listing 3-9 (next page).

Kexts whose bundle identifier is `com.apple.` must be signed by an Apple root certificate (the "anchor apple"), whereas all others (as of 10.11 with SIP) must be signed by a specific kernel extension enabled signing certificate. Looking at Volume III's Figure 5-22 reveals the certificate must contain the Developer ID CA intermediate marker (6.2.6), Developer ID Applications (6.1.13) AND Developer ID Kexts (6.1.18). Apple will only provide certificates containing the latter to registered developers if they specifically request one at https://developer.apple.com/contact/kext/. GPU bundles are subject to an even tighter requirement, hardcoded in kextd's staging.m to add `certificate leaf[subject.OU] = "6KR3T733EC"`, which is NVIDIA's team ID.

Listing 3-9: The hard-coded requirement string for Developer ID kexts

```
/* set up correct requirement string.  Apple kexts are signed by B&I while
 * 3rd party kexts are signed through a special developer kext devid
 * program */
myCFString = OSKextGetIdentifier(aKext);
if (CFStringHasPrefix(myCFString, __kOSKextApplePrefix)) {
    requirementsString = CFSTR("anchor apple");
}
else {
   /* DevID for kexts cert
    */
   requirementsString =
       CFSTR("anchor apple generic "
             "and certificate 1[field.1.2.840.113635.100.6.2.6] "
             "and certificate leaf[field.1.2.840.113635.100.6.1.13] "
             "and certificate leaf[field.1.2.840.113635.100.6.1.18]");
}
```

This, however, is not where our story ends: In order to prevent kernel extensions from malicious developers who could somehow connivingly obtain a certificate, Apple further hardended the kext loading process in 10.13. As detailed in tn2459[8], initial loading requests for all kexts but Apple's own are denied, and the user has to explicitly bring up System Preferences.app (or use the command line `spctl kext-consent`, also mentioned in III/6) in order to allow future load attempts to succeed.

Behind the scenes, the validation of such third party kernel extensions is performed by the `syspolicyd(8)`, which maintains the list of approved kernel extensions in /var/db/SystemPolicyConfiguration/KextPolicy, a SQLite3 file. The current list can easily be displayed using the `sqlite3` command line:

Output 3-10: Displaying the user-approved kext list from /var/db/SystemPolicyConfiguration/KextPolicy database

```
morpheus@Chimera(~)$ sqlite3 /var/db/SystemPolicyConfiguration/KextPolicy -header \
                                                "select * from kext_policy"
team_id  |bundle_id                          |allowed|developer_name|flags
EG7KH642X6|com.vmware.kext.vmioplug.15.2.1|   1   |VMware, Inc.  |12
EG7KH642X6|com.vmware.kext.vmx86            |   1   |VMware, Inc.  |12
EG7KH642X6|com.vmware.kext.vmnet            |   1   |VMware, Inc.  |12
EG7KH642X6|com.vmware.kext.vmci             |   1   |VMware, Inc.  |12
EG7KH642X6|com.vmware.kext.vmioplug.17.1.4  |   1   |VMware, Inc.  |1
```

A special key, `kext-allowed-teams` in the `csr-data` NVRAM variable serves as a whitelist of Team IDs for kernel extensions used during system boot with no prompts. Direct NVRAM access is not allowed when SIP is enabled, which which is why `spctl(8)` has been entitled with `com.apple.private.iokit.nvram-csr`.

Output 3-11: Displaying the `kext-allowed-teams` from the `csr-data` NVRAM variable

```
morpheus@Chimera(~)$ nvram -p | grep kext-allowed-teams
csr-data    <dict><key>kext-allowed-teams</key><array><string>FMP5JFMYUP</string></array></dict>%00
morpheus@Chimera(~)$ spctl kext-consent status
Kernel Extension User Consent: ENABLED
morpheus@Chimera(~)$ spctl kext-consent list
Allowed Team Identifiers:
FMP5JFMYUP
```

The `syspolicyd` also checks for kexts whose SHA-1 code signature hashes are found as keys in the `OSKextSigExceptionHashList` dictionary of AppleKextExcludeList.kext's Info.plist. This codeless kext defines a kernel "blacklist" of kexts, whose loading is denied.

> ⚠ Validation of kext code signatures in user mode is precarious! Up until MacOS 11, it was a simple matter to bypass the validations imposed by Apple's kext_tools binaries (`kextload(8)`, `kextutil(8)` and `kextd(8)`) and just call the `OSKextLib` APIs directly from any root owned process - an example of which can be found on the book's companion forum[9]. With the advent of SIP, Apple has fixed this the way it does most vulnerabilities - by slapping the `com.apple.rootless.kext-management` entitlement. This does go a long way to mitigate, but not elimitate the threat, which is viable if SIP is bypassed - or worked around. Another issue to consider are vulnerabilities in the `kext_tools` themselves - exploited as recently as MacOS 10.14.4.[10]

Kextd(8)

MacOS uses a dedicated "kernel extension daemon" in user mode, /usr/libexec/kextd. This daemon is started from /usr/libexec by the `com.apple.kextd.plist` LaunchDaemon plist, which ensures it is `RunAtLoad` and kept alive if crashed. The property list also assigns it host special port #15 (`HOST_KEXTD_PORT`), which is registered in the system domain as `com.apple.KernelExtensionServer`, for `kextload(8)` (but not `kextutil(8)`, which loads extensions directly) to connect to.

The kernel extension daemon thus serves as the focal point for kext management. `kexutil(8)` is generally used only in debugging and troubleshooting scenarios, so the usual flow is `kextload(8)` to contact `kext(8)`, so it issue the load request on its behalf. This also allows `kextload(8)` to be run by any user, and have `kextd(8)` perform the validation of the `OSBundleAllowUserLoad` key (by calling `checkNonrootLoadAllowed()`) when servicing the request (from `kextdProcessUserLoadRequest()`).

Thus, `kextd(8)` operates in a `CFRunLoop` over its Mach port and a signal source, as well as several notifications. Two MIG subsystems are served:

- **MIG subsystem 77000**: (from XNU's /osfmk/kextd/kextd_mach.defs) with 1 message, `kextd_ping`, which serves to start `kext(8)` if not already running, and provide the kernel→user requests (shown later in Table 3-14).
- **MIG subsystem 70000**: (from `kextmanager_mig.defs` in the kext.subproj of the IOKitUser project) with messages intended for user mode clients (specifically, `kextcache(8)` and `kextload(8)`), as shown in Table 3-12:

Table 3-12: MIG subsystem #70000, exposed through `com.apple.KernelExtensionServer`

#	kext_manager..	Notes
70000	..path_for_bundle_id	
70001	..create_property_value_array	Used by loginwindow
70002-5,11	skip	
70006	.._lock_reboot	Used by kextcache(8)
70007-8	.._[lock/unlock]_volume	Used by kextcache(8)
70009-10	.._[lock/unlock]_kextload	Suspend/Unsuspend kernel requests
70012	.._load_kext	kextd(8) assisted kext load request
MacOS 15 (System Extension Manager)		
70013	KextdExtensionManagerStop	
70014	KextdExtensionManagerUpdate	
70015	KextdExtensionManagerValidate	

As of MacOS 13, `kextd(8)` is also responsible for enforcing the system policy, as described in the previous page. The daemon calls `SPAllowKextLoad()`, which uses the private SystemPolicy.framework's `SPKernelExtensionPolicy` class to hide the requests to `syspolicyd(8)`. This is also used by `kextutil(8)` (*but only as of MacOS 13*). Another enhancement is **kext staging**, in which `kextd(8)` calls `stageBundle()` (from kext_tools's `staging.m` to move the kernel extension to a secure location - /Library/StagedExtensions, which is SIP-restricted with a label of `KernelExtensionManagement`. `kextd(8)` can write to that directory as it possesses the appropriate entitlement (as can `kextcache(8)`, for `-[clear/prune]-staging`), and the secure location prevents any race attacks.

Looking at the sources of the `kext_tools` project shows some references to `HAVE_DANGERZONE`, which is the internal name of `libdz.dylib` APIs. When servicing kext load requests `kextd(8)` calls `dz_notify_kext_load_v2`, which invokes an XPC simpleroutine to /usr/libexec/dznd. Seeing as the library also checks for the presence of `AppleConnect.app`, and the daemon is not present on any public builds on MacOS, the "dangerzone" code is very likely left over from Apple Internal builds.

As of MacOS 15, kextd also checks the kextallow property list (in /System/Library/Caches/com.apple.kext.caches/). The plist, created during system boot, contains the current `BootSessionUUID` (matching the `kern.bootsessionuuid sysctl(2)` MIB) and a `CDHashArray` key, wherein the code directory hashes of all user approved kernel extensions are maintained.

MacOS 13: logkextloadsd

MacOS 13 adds another daemon to the kext_tools project, /usr/libexec/logkextloadsd, though for some reason its main is not open sourced. The daemon uses `asl_*` APIs to log messages (to `os_log`), and maintains a(n unrestricted) property list at /var/db/loadedkextmt.plist, with three keys: `Alerts sent`, `All requests` and `User requests`. The last two are `arrays` of kext `CFBundleIdentifiers`, and the first is an `array` of `dicts`, containing `com.apple.message.*` keys corresponding to the `User requests`.

MacOS 14: BridgeOS kext_audit

T2 enabled devices support a new feature, called **Kext auditing**. This allows BridgeOS to keep track of its MacOS host's kernel, by receiving notifications whenever a kernel extension is loaded or unloaded. As of MacOS 14 (kext_tools version 528 and upwards), kextd(8), kextload(8) and kextutil(8) call `OSKextSetLoadAuditFunction` to register a callback function. The callback is implemented in common code (kextaudit.c), and it calls `KextAuditNotifyBridgeWithReplySync()` to send a message to BridgeOS, recording the kext loading. A call to `KextAuditMakeKALNFromInfo()` constructs the notification, and includes the Kext's `CFBundleIdentifier`, CD Hash, and Team ID.

The notification is sent to the kernel via an IOUserClient created by the KextAudit.kext kernel extension (and requires the `com.apple.private.KextAudit.user-access` entitlement). This kext links with the AppleSMC.kext, through which it can access BridgeOS. The notification is then sent to BridgeOS, wherein the `com.apple.internal.xpc.remote.kext_audit` service picks it up. This remote XPC service is provided by the kext_audit.plugin, in /System/Library/MultiversePlugins. If the host SIP is enabled, the plugin logs the kext loading.

The OSKext* APIs

The `OSKext*` functions are a family of APIs used for user-mode kernel extension management. They comprise the `OSKext.subproj` of the open source IOKitLib project, and provide rich functionality to interface with kernel side component - implemented in libkern's OSKextLib.cpp.

kext_request

Kernel extensions commonly have to be loaded from user mode, and XNU traditionally provided three special MIG routines for this purpose. The `kmod_create` (#409) `kmod_destroy` (#411) and `kmod_control` (#411) routines of the `host_priv` subsystem allowed the loading, unloading and setting of kernel "modules" (as extensions were known by their Mach name) on older versions of XNU.

Apple introduced `kext_request` (#425) into `host_priv` around Darwin 10, and have since retired the `kmod_*` variants. Rather than using three separate MIG routines, it is now possible to use a single routine with more parameters:

Listing 3-13: `kext_request`'s definition (host_priv.defs) and its MIG generated header (host_priv.h)

```
/*
 * Perform an operation with a kernel extension, on the kext loading system,
 * or request information about loaded kexts or the state of the kext loading
 * system.
 * Active operations (load, unload, disable/enable) require host_priv/root access.
 * Info retrieval does not.
 *
 * WARNING: THIS ROUTINE IS PRIVATE TO THE KEXT-MANAGEMENT STACK AND IS
 * SUBJECT TO CHANGE AT ANY TIME.
 */
routine kext_request(                          //  kern_return_t kext_request
        host_priv       : host_priv_t;         //   (host_priv_t host_priv,
    in  user_log_flags  : uint32_t;            //    uint32_t user_log_flags,
    in  request_data    : pointer_t;           //    vm_offset_t request_data,
                                               //    mach_msg_type_number_t request_dataCnt,
    out response_data   : pointer_t;           //    vm_offset_t *response_data,
                                               //    mach_msg_type_number_t *response_dataCnt,
    out log_data        : pointer_t;           //    vm_offset_t *log_data,
                                               //    mach_msg_type_number_t *log_dataCnt,
    out op_result       : kern_return_t);      //    kern_return_t *op_result);
```

As is somewhat the manner with `WARNINGS`, `kext_request` has remained rock solid stable to this day. It is a rather unique API, preferring to get the `request_data` as an XML dictionary, and returning the same (thanks to XNU's OS[un]Serialize support). Apple's entire `kext_tools` project revolves around the various dictionary requests this MIG routine can process, as wrapped by the `OSKext*` APIs.

Normally, requests are sent from user mode to kernel. In MacOS, however, there are certain requests, which are upcalls from the kernel to the `HOST_KEXTD_PORT` (hopefully held by kextd(8)). These serve as notifications or load requests, notifying kextd of a required kext dependency or resource which needs to be obtained from user mode. Another request made by the kernel is to rebuild the kernel cache. Table 3-14 shows the set of kext requests. The highlighted lines (from user mode to kernel) also require root privileges.

Originally, any client with the `host_priv` `SEND` right could make a user mode request, and the two informational requests could be made by any user. Apple has since hardened the requests with `mac_kext_check_*` checks. The MAC framework calls (hooked by Sandbox.kext as of Darwin 15) ensure that the requestor holds `com.apple.private.kernel.get-kext-info` (for informational requests) and/or `com.apple.rootless.kext-management` for all others. These entitlements are held by `kextload(8)` (though not `kextunload(8)`) and `kextutil(8)`, as well as kextd(8).

Table 3-14: The `Predicate` strings used in `kext_request` (from libkern/libkern/kext_request_keys.h)

Direction	Predicate	Purpose
User → Kernel	Get Loaded Kext Info	`kextstat(8)`. Optional: "Kext Request Info Keys" filters
User → Kernel	Get All Load Requests	`kextstat(8)`, with history
User → Kernel	Get Kernel Requests	Get pending requests by kernel
User → Kernel	[Load/Unload]	Load or Unload a kernel extension
User → Kernel	[Start/Stop]	Start or Stop a (previously loaded) kernel extension
User → Kernel	Send Resource	Reply to a previous kernel "Kext Resource Request"
Kernel → User	Kext Load Request	ask kextd to initiate loading of extension with identifier
Kernel → User	Kext [Load/Unload] Notification	Notification said kernel extension was [un]loaded
Kernel → User	Kext Prelinked Kernel Request	Ask kextd to create a new prelinked kernel (kernelcache)
Kernel → User	Kext Resource Request	Ask kextd to provide a specific kext resource to kernel
Kernel → User	Kextd Exit	Instruct kextd to exit (when rebooting)
Kernel → User	Dext Daemon Launch	MacOS 15: Launch a Driver Kit extension

Multikexts

XNU supports not only kexts, but a packaged format called **multikext**. The idea behind multikext is to speed up kext loading in cases where the extensions has dependencies which have not been loaded yet. Rather than go back and forth and load the dependencies one at a time, the dependencies can be prepackaged into an `.mkext` file, which can then be loaded into the kernel with one opeation.

Multikexts aren't normally found anywhere in the file system, but when `kextload(8)` or `kextutil(8)` load kernel extensions, the format is used. They query the kernel for the loaded kernel extension information, and according to it (and the kext-to-be-loaded's `OSBundleLibraries`) retrieve all unloaded dependencies, and create an MKext archive on the fly to submit to the kernel (this is shown later in this chapter through an experiment).

There are two MKext formats, version 1 and version 2, both start with the `MKEXT_HEADER_CODE`, which expands the fields of the `mkext_basic_header` struct, but version 2 adds an additional property list (at the end of the MKext archive) and uses zlib, rather than lzss. Following the header in version 2 are one or more `mkext2_file_entry` structures, holding the actual Mach-O binaries of the kext(s) loaded. As a legacy of XNU's PPC heritage, the fields are big endian. Listing 3-15 shows the version 1 header:

Listing 3-15: The MKext version 1 header (from libkern/libkern/mkext.h)

```
/*********************************************************************
* Core Header
*
* All versions of mkext files have this basic header:
*
* - magic & signature - always 'MKXT' and 'MOSX' as defined above.
* - length - the length of the whole file
* - adler32 - checksum from & version to end of file
* - version - a 'vers' style value
* - numkexts - how many kexts are in the archive (only needed in v.1)
* - cputype & cpusubtype - in version 1 could be CPU_TYPE_ANY
*     and CPU_SUBTYPE_MULTIPLE if the archive contained fat kexts;
*     version 2 does not allow this and all kexts must be of a single
*     arch. For either version, mkexts of specific arches can be
*     embedded in a fat Mach-O file to combine them.
*********************************************************************/
#define MKEXT_HEADER_CORE   \
uint32_t      magic;        \
uint32_t      signature;    \
uint32_t      length;       \
uint32_t      adler32;      \
uint32_t      version;      \
uint32_t      numkexts;     \
cpu_type_t    cputype;      \
cpu_subtype_t cpusubtype;

typedef struct mkext_basic_header {
MKEXT_HEADER_CORE
} mkext_basic_header;
```

Experiment: Behind the scenes of `kextstat(8)`

Using `lldb` you can easily trace not just the `kext_request` calls, but also their underlying Mach messages. Just breaking on all `mach_msg` calls, however, can be excruciating becuase of many unrelated messages.

In the case of `kextstat(8)`, though, the Mach messages are wrapped by `OSKext*` APIs. You can start off by setting a breakpoint on `OSKextCopyLoadedKextInfo`, which is the main call used in the binary. Once that breakpoint is trigged, break on `mach_msg`:

Output 3-16-a: Debugging `kextstat(8)`

```
morpheus@Chimera(~)$ lldb kextstat
Current executable set to 'kextstat' (x86_64).
(lldb) b OSKextCopyLoadedKextInfo
Breakpoint 1: where = IOKit`OSKextCopyLoadedKextInfo, address = 0x00000000000632fa
(lldb) r
Process 6715 launched: '/usr/sbin/kextstat' (x86_64)
Process 6715 stopped
* thread #1, queue = 'com.apple.main-thread', stop reason = breakpoint 1.1
    frame #0: 0x00007fff4d7042fa IOKit`OSKextCopyLoadedKextInfo
IOKit`OSKextCopyLoadedKextInfo:
->  0x7fff4d7042fa <+0>: pushq  %rbp
Target 0: (kextstat) stopped.
(lldb) b mach_msg
Breakpoint 2: where = libsystem_kernel.dylib`mach_msg, address = 0x00007fff728b7ca0
(lldb) c
Process 6715 stopped
* thread #1, queue = 'com.apple.main-thread', stop reason = breakpoint 2.1
    frame #0: 0x00007fff728b7ca0 libsystem_kernel.dylib`mach_msg
libsystem_kernel.dylib`mach_msg:
->  0x7fff728b7ca0 <+0>: pushq  %rbp
Target 0: (kextstat) stopped.
#
# Read the Mach message from the first argument
#
(lldb) mem read $rdi
0x7ffeefbff5d0: 13 15 00 80 | 01 00 00 00 | 03 09 00 00 | 03 06 00 00  ................
0x7ffeefbff5e0: 00 00 00 00 | a9 01 00 00 | ___count___ | _addr(1/2)_  ........
                _addr(2/2)_               | 01 00 00 00 | 90 07 10 00          ........
0x7ffeefbff5f0: 01 00 00 00 | 00 01 00 01 | ae 00 00 00 | 00 00 00 00  ....................
0x7ffeefbff600: 01 00 00 00 | f2 0f 00 00 | ae 00 00 00 | 01 00 00 00  ................
```

Note the Mach message. In the output above, it is sent from port 0x0603 to port 0x0903. You can resolve the ports using `procexp` *`pid`* `ports`, which will reveal the former to be an ephemeral port, and the latter to be the Host port. The message is indeed #425 (0x1a9), and it is a complex message, as indicated by the most significant bit in 0x80001513. We therefore proceed to inspect its descriptor portion, which immediately follows the header and indicates one descriptor. In the output above, the descriptor is an OOL memory buffer at 0x0100100790 spanning 0xae bytes. Inspecting the memory there will reveal the request dictionary, as raw XML:

Output 3-16-b: Dumping the complex message descriptor of a `kext_request()`

```
#
# Message is complex (0x80001315) with one out of line descriptor
# at 0x100100790 - read that memory
#
(lldb) mem read 0x100100790
0x100100790: 3c 64 69 63 74 3e 3c 6b 65 79 3e 4b 65 78 74 20  <dict><key<Kext
0x1001007a0: 52 65 71 75 65 73 74 20 50 72 65 64 69 63 61 74  Request Predicat
0x1001007b0: 65 3c 2f 6b 65 79 3e 3c 73 74 72 69 6e 67 3e 47  e</key><string>G
0x1001007c0: 65 74 20 4c 6f 61 64 65 64 20 4b 65 78 74 20 49  et Loaded Kext I
0x1001007d0: 6e 66 6f 3c 2f 73 74 72 69 6e 67 3e 3c 6b 65 79  nfo</string><key
0x1001007e0: 3e 4b 65 78 74 20 52 65 71 75 65 73 74 20 41 72  >Kext Request Ar
0x1001007f0: 67 75 6d 65 6e 74 73 3c 2f 6b 65 79 3e 3c 64 69  guments</key><di
0x100100800: 63 74 3e 3c 6b 65 79 3e 43 46 42 75 6e 64 6c 65  ct><key>CFBundle
0x100100810: 49 64 65 6e 74 69 66 69 65 72 3c 2f 6b 65 79 3e  Identifier</key>
0x100100820: 3c 61 72 72 61 79 3e 3c 2f 61 72 72 61 79 3e 3c  <array></array><
0x100100830: 2f 64 69 63 74 3e 3c 2f 64 69 63 74 3e 00 00 00  /dict></dict>...
```

The `kextstat(8)` utility normally doesn't apply any filters, which is why the `Kext Request Arguments` dictionary in the output is an empty `CFBundleIdentifier` array. The array will be populated if `kextstat(8)` is used with `-b`.

Experiment: Intercepting an MKext in the wild using `lldb`

Although MKexts aren't used on disk, they are created on-the-fly when loading kexts. You can inspect the structure format by tracing `kextd`, breaking on `kext_request` while issuing a `kextload(8)` command in another terminal session.

Setting a breakpoint on `kext_request` will cause multiple hits, because (as was shown earlier) `kext_request` is the catch-all MIG message for kext maintenance. Inspecting $rdx will reveal XML dictionaries in some requests, but the one of interest (which can also be identified from its backtrace of `OSKextLoadWithOptions`) will sport an MKXT header, as shown in the following output, which breaks the hex dump into the respective fields of the structure, shown in Figure 3-17-a:

Ouput 3-17-a Tracing the MKext (version 2) loading

```
morpheus@Zephyr (~)$ sudo lldb -p `pgrep kextd`
...
* thread #1, queue = 'com.apple.main-thread', stop reason = signal SIGSTOP
...
(lldb) b kext_request
Breakpoint 1: where = libsystem_kernel.dylib`kext_request, address = 0x00007fff583a1b8f
.... continuing over two breakpoints...
(lldb) bt
* thread #1, queue = 'com.apple.main-thread', stop reason = breakpoint 1.1
* frame #0: 0x00007fff583a1b8f libsystem_kernel.dylib`kext_request
frame #1: 0x00007fff331f15bc IOKit`OSKextLoadWithOptions + 1776
(lldb) mem read $rdx
                magic      signature      length       adler32
0x7f8eda847600: 4d 4b 58 54 | 4d 4f 53 58 | 00 00 dc 52 | 44 69 ea 67   MKXTMOSX..?RDi?g
               version       numkexts       cputype     cpusubtype
0x7f8eda847610: 02 00 20 01 | 00 00 00 01 | 01 00 00 07 | 00 00 00 03  .. ............

                plist_off    plist_comp    plist_full
0x7f8eda847620: 00 00 d4 d4 | 00 00 00 00 | 00 00 07 7e |
                                                        comp_size
              full_size         data                    00 00 00 00  ..??....
0x7f8eda847630: 00 00 d4 a0 | cf fa ed fe   07 00 00 01   03 00 00 00  ..ϱ????........
...
```

In the case of a single kext, it doesn't make sense to use compression, so both the kext itself and the property list are uncompressed, as indicated by zeroes in their compressed_size entries. You can view the property list at its specified offset (in the above example, 0xd4d4) by casting the memory to a character pointer, or dumping the memory to a file, as shown in Output 3-17-b:

Ouput 3-17-b: Inspecting the property list of an MKext2

```
#
# In the example above, the plist_offset was 0xd4d4, so that relative to $rdx we have:
#
(lldb)  p (char *) 0x7f8eda854ad4
(char *) $7 = 0x00007f8eda854ad4
"<dict><key>Kext Request Predicate</key><string>Load</string>"
"<key>Kext Request Arguments</key>"
#
# If dumping to file, we specify the size (0x77e) which is 1918, and pretty print:
#
(lldb) mem read -b  -c 1918 --force -o /tmp/mkext.plist  0x7f8eda854ad4
1918 bytes written to '/tmp/mkext.plist'
morpheus@Zephyr(~)$ cat /tmp/mkext.plist | xmllint --pretty 1 -
<?xml version="1.0"?>
<dict>
<key>Kext Request Predicate</key>
<string>Load</string>
<key>Kext Request Arguments</key>
<dict>
<key>Start Exclude Level</key>
<integer ID="0" size="8">0x0</integer>
<key>Start Matching Exclude Level</key>
<integer IDREF="0"/>
<key>CFBundleIdentifier</key>
<string ID="1">com.apple.filesystems.webdav</string>
</dict>
<key>_MKEXTInfoDictionaries</key>
<array>
...
```

Kext Loading: The kernel perspective

So far, we've considered the process of kext loading from the user mode perspective, but - once validated - the actual loading of the MKext must be performed in kernel space.

The kernel-mode side of `kext_request` first makes sure there is a request to handle. It then calls `vm_map_copyout()` to retrieve the request body into kernel space. Once the request body is in, it is first checked if it an MKext, and if so loading can commence. Else, it is assumed to be an XML dictionary, and passed to `OSKext::handleRequest` (a self-proclaimed "big ugly mess"), which calls `OSUnserializeXML()` to convert it to an `OSDictionary`, and then handles all the other User to Kernel predicates shown in Table 3-14.

Focusing on the case of kext loading, a call to `OSKext::loadFromMkext()` follows, which creates an `OSData::withBytesNoCopy` from the `mkextBuffer`, and then calls `readMkext2Archive`. Recall, that an MKext2 may use compression on its kexts or plist - for which a call to `OSKext::extractMkext2FileData` is made. This function wraps the kernel's built in `zlib`, and optionally calls its inflater if the data is compressed. Following inflation (or if uncompressed), the property list is converted (via `OSUnserializeXML()`) to an `OSDictionary`, and new `OSKext` class instances are instantiated for each of the archived kexts, using the `::withMkext2Info` constructor, which also calls the `registerIdentifier()` method to add the object to the global `sKextsByID` array. The `OSKext` class defined in libkern/libkern/c++/OSKext.h and implemented in libkern/c++/OSKext.cpp, represents an instance of a kext known to the kernel, and is also used to load the kext, as will be demonstrated shortly.

The property list is expected to include the `Load` predicate, and a `Kext Request Arguments` dictionary. The keys may include any of the `KextRequestArgument[...]Key` constants in libkern/libkern/kext_request_keys.h but a mandatory one is `CFBundleIdentifier`, which is then provided to `OSKext::loadKextWithIdentifier()`. This function records the identifier (for possible inclusion in a prelinked kernel), and checks if the kext is already in the `sKextsByID` `OSDictionary`. If it is (as it will be if bundled with the Mkext), loading may proceed using the object's `load()`.

OSKext::load

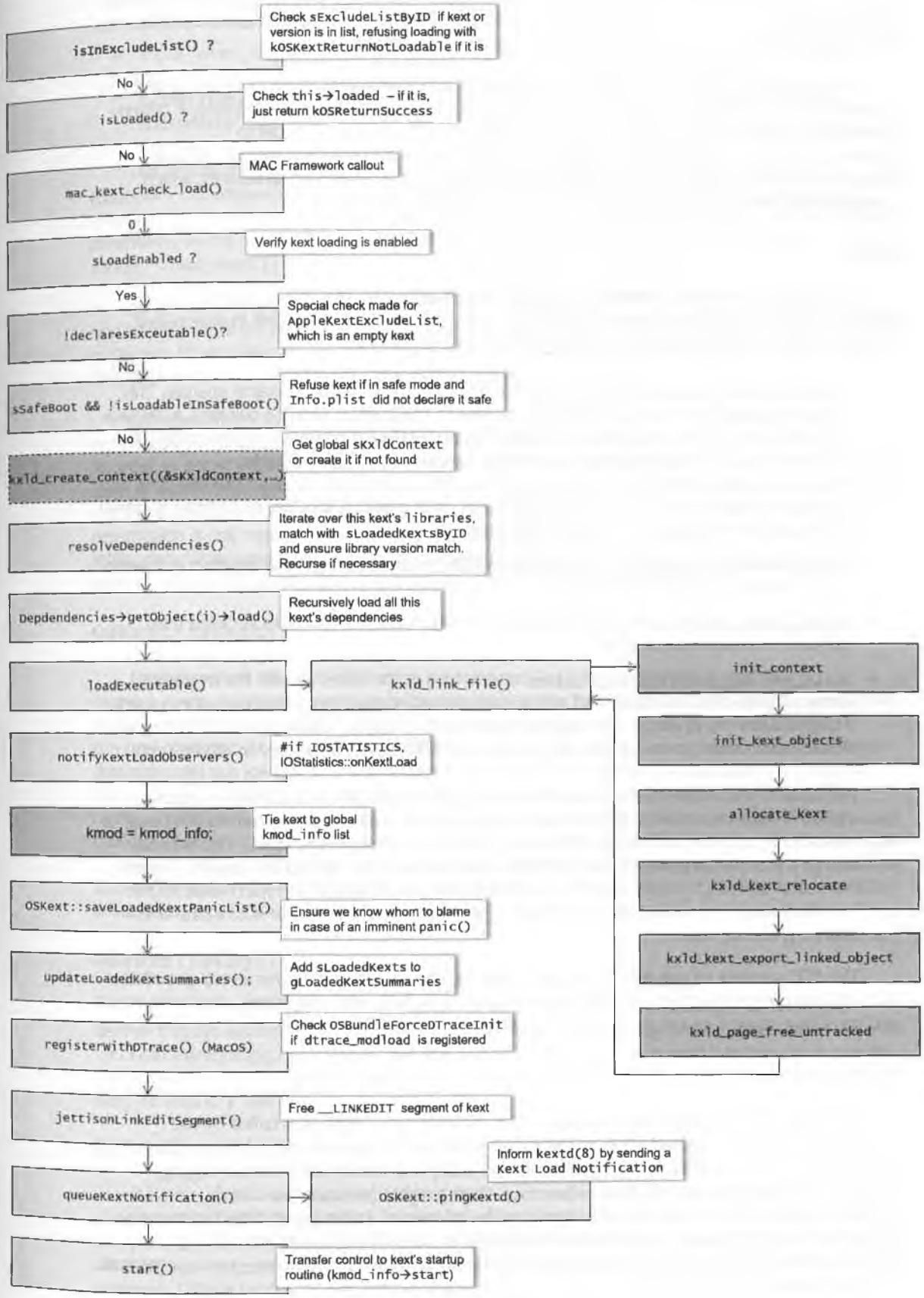
The loading of the kext is performed by the `load` method of the `OSKext` object. This is a lengthy function, whose many steps are illustrated in Figure 3-18. The first step is ensure the kext in question hasn't been blacklisted by the `sExcludeListByID` dictionary. `OSKext::updateExcludeList()` is called to populate the dictionary, from `KLDBootstrap::readStartupExtensions` (in libsa/bootstrap.cpp), through either `OSKext::createExcludeListFromBooterData()` (in MacOS) or `OSKext::createExcludeListFromPrelinkInfo()` (in *OS variants).

In the *OS case, where on-demand loading is disabled anyway, there is no use for an exclusion list. In the case of MacOS, the list is found in AppleKextExcludeList.kext, and is loaded by special case handling in `OSKext::load()`, as it does have an associated `CFBundleExecutable`. The Info.plist contains an `OSKextExcludeList` dictionary, specifying the excluded kexts' `CFBundleIdentifiers` as keys, and version number (with optional GT/GE/LT/LE) as string values. The plist also contains a `OSKextSigExceptionHashList` with SHA-1 hashes as keys, but these are enforced by `syspolicyd`, rather than the kernel.

Beginning with Darwin 13, Apple added MAC framework hooks for kext operations. One of those - `mac_kext_check_load` is called out to if the calling task is not the `kernel_task` (which is the case when load opertions are started from user mode). The Sandbox.kext enforces the `com.apple.rootless.kext-management` entitlement. This (hopefully) ensures that the caller is `kextd(8)` or `kextutil(8)`.

Next, a check is made if kext loading is enabled. Kext loading (and unloading) may be controlled by toggling `s[Load/Unload]Enabled`. These are two private static variable which are both normally true, but can be accessed and controlled through `OSKext::[get/set][Load/Unload]Enabled`. Since kext loading is enabled by default, the flow continues with one last check - whether the system is in safe boot, but the kext's Info.plist didn't specify an `OSBundleRequired` key with one of the values from Table 3-2.

Figure 3-18: The flow of `OSKext::load()`, visualized



At this point, all is ready for the actual load operation. `OSKext::load()` first acquires a context to the kernel extension loader, `kxld` (described next), and then proceeds to `resolveDependencies()` into `OSKext` objects. Once resolved, the dependencies are loaded, in turn, by recursively calling each object's `load()` method. With all dependencies satisfied, the kext may be loaded, by calling the `loadExecutable()` method.

The `loadExecutable()` method short circuits if in a prelinked cache (jumping straight to the epilog). Otherwise, it checks for the `com.apple.rootless.kext-management` directly (rather than a MACF callout), by calling `IOTaskHasEntitlement()`. A call to `getExecutable()` obtains (and possibly extracts) the Mach-O from the MKext. The link dependencies are obtained, and populate an array of `KXLDDependency` structures, which completes all the information required to call on `kxld_link_file()`.

kxld

XNU contains a **kernel extension loader** known as `kxld`. This is a complete implementation of a Mach-O parser and loader, coded in C and self-contained in libkern/kxld, which exposes an implementation of but four functions to `OSKext`:

- **`kxld_create_context()`:** constructs a `KXLDContext`, which remains opaque. This function expects an `out` (by reference) pointer to the context (itself a pointer), a callback for allocation, another callback for logging, flags, CPUType/SubType, and the page size. `OSKext` passes 0 for the integer arguments, and provides both callbacks - `kern_allocate` and `kxld_log_callback`. This constructor is called the first time a context is needed (from `OSKext::load`) and, once created, the context pointer is stored in the global `sKxldContext`, until destroyed.
- **`kxld_destroy_context()`:** takes a previously created context and destroys it, freeing any allocated resources along the way.
- **`kxld_link_file()`:** performs the actual linking of the Mach-O. This is called from `OSKext::loadExecutable`.
- **`kxld_verify_copyright_string`:** which, true to its name, checks the Info.plist's `CFBundleGetInfoString` and `NSHumanReadableCopyright` to ensure they match "`Copyright © {year[,]}+ Apple Inc. All rights reserved.`". This is how Apple ensures that `com.apple.kpi.private` KPIs can only be used by Apple kexts.

The code in kxld makes heavy use of `require_action` statements. This is a macro (`#defined` in AssertMacros.h) which ensures its `assertion` returns a non-zero value (true), or else executes an `action` and then performs a `goto` to an `exceptionLabel`. This is used primarily to check that arguments are non-NULL (with an `action` setting the function return value to `KERN_INVALID_ARGUMENT` it any of them are), and testing the return value to the result of allocations (returning `KERN_RESOURCE_SHORTAGE`), branching to a `finish` label if the assertions do not hold.

The *OS variants all use a prelinked kernelcache, and so kxld code is removed, and XNU is instead linked instead with stubs from libkern/kxld/kxld_stubs.c, returning `KERN_SUCCESS/TRUE` and causing a panic if `kxld_link_file` is called.

`kxld_link_file()`

The heart of kxld is its linker function, `kxld_link_file`, which was called by the `loadExecutable()` method. The function counts among its arguments the kxld `context`, `link_info` (which is NULL, unless split kexts are used), the linked kext's name, logging callback data, the array of kxld `dependencies`, their number (`ndependencies`), `linked_object_out`, which is an out pointer (by reference) to the linked object in memory, and a pointer to the `kmod_info` structure, whose load address will be set after linking. Following the validation of all its arguments (by the aforementioned `require_action` macros, linking proceeds as follows:

- `init_context` is called on the supplied context with `ndependencies`. The number of dependencies is used to allocate arrays of objects and dependencies, various symbols and vtables.
- `init_kext_objects` creates a `KXLDObject kext_object` for each of the linked file's dependencies, exporting their symbols and vtables.

- `allocate_kext` (from libkern/kxld/kxld_kext.c) calls the context's allocation callback to allocate memory in the `linked_object` pointer.
- `kxld_kext_relocate()` handles relocation.
- `kxld_kext_export_linked_object()` obtains the file's `kmod_info` symbol, and calls `kxld_object_export_linked_object()` (in libkern/kxld/kxld_object.c) to further populate the corresponding `kxld_object` with data from the Mach-O load commands.

The resulting `linked_object` and the `kmod_info` are returned back to the caller (`OSKext::loadExecutable()`), which begins its epilog to register the `kmod_info`, and connect its dependencies to it. The exeuctable's memory protections are also set, unless already prelinked). In the case of KASAN enabled kernels (`#if KASAN`), there is another call to `kasan_load_kext()`. `loadExecutable()` then cleans up and returns.

As of XNU 3789, the sources show another function, `kxld_link_split_file()`, apparently meant for use in cases where the kexts are "split", the way they are on kernelcaches in Darwin 16 and later. There is no actual call to this file in the MacOS sources as of yet (Darwin 18), and kexts remain in the old segmentation style (i.e. not split).

On return from `loadExecutable()`, the `load()` method's work isn't done. The flow continues (on MacOS, where `IOKITSTATS` is true) with a call to `IOStatistics`'s `onKextLoad` in order to notify interested parties that the kext has loaded. The panic list is updated, so that in case of a panic the newly loaded kext is in the list of probable suspects. In MacOS, kexts are also registered with DTrace (through `dtrace_module_loaded()`, in bsd/dev/dtrace/dtrace.c). If registration is not required (or following registration), the kext's `__LINKEDIT` segment can be jetissoned. Unless the load request specifically requested exclusions, the loaded kext can be started, and any IOPersonalities (discussed in Chapter 13) are registered.

`OSKext::start()`

Loading a kext places it into kernel memory and links its dependencies, but it is `OSKext::start()` which is responsible for running the kext supplied code. Before starting the kext, a call to `validateKextMapping` ensures that the kext's virtual pages are backed by physical, and that its `kmod_info` metadata is valid. `OSKext::start()` also makes sure any dependencies are loaded, and starts them (recursively) if necessary. A tracepoint to `logd(8)` (via `OSKextLogKextInfo`, with the kext's address and size) notifies of the load operation. Lastly, it calls `OSRuntimeInitializeCPP` (from libkern/c++/OSRuntime.cpp), which uses `OSRuntimeCallStructorsInSection` (ibid.) to process the loaded kext's `__DATA.__mod_init_func`. This section contents are the same in extensions as they are in user mode dylibs - an array of static initiazer pointers. Once all initializers have run, the kext's actual entry point (from its `kmod_info->start`) may finally be called.

`OSKext::stop()`

Stopping a kext is similar. First, a check is made that the kext in question is not in use - neither through instantiated C++ objects, nor through references by clients. Once more, a call to `validateKextMapping` ensures that the `kmod_info->stop` is a valid function, so it can be called. After the `stop` operation succeeds, a tracepoint to `logd(8)` (again, with address and size) informs it of the stop.

Unloading a kext

A kext may also be unloaded, subject to the same constraints as stopping (i.e. that it is not already in use), and the process is similar to loading, with a few variations. Started kexts will be stopped first, and the panic string will be saved just in case. `OSRuntimeFinalizeCPP` calls any destructors, the `kmod_info_t` removed from the linked lists of kexts. The kext's references are removed. DTrace probes, if any, are unregistered. Kext unload observers are notified. Kext accounting is freed. The kext segments are unloaded. Once the kext has been unloaded, the panic string of last loaded kexts is saved (in case a panic occurs indirectly due to the kext loading, and kext summaries are updated.

kext metadata management

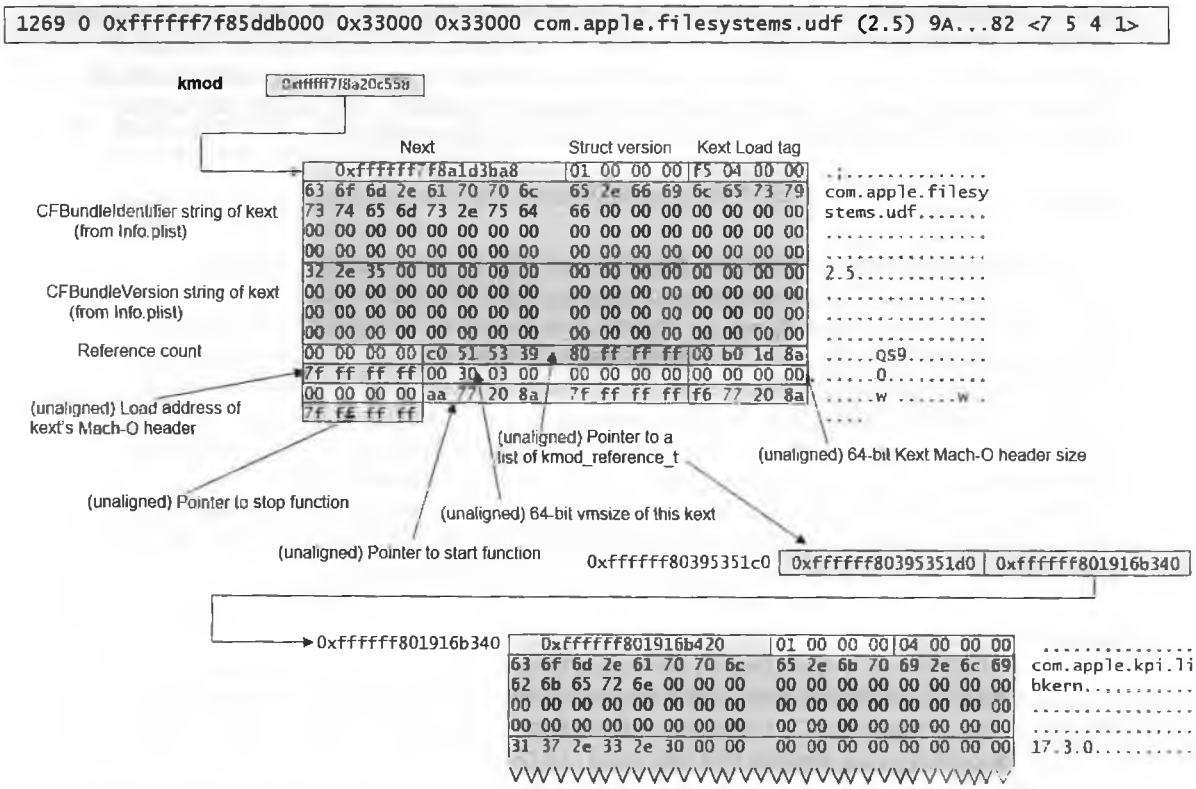
The kernel has to keep accurate track of loaded extensions for its own purposes, as well as to provide details to user-mode tools (notably, `kextstat(8)`). There are several structures in the kernel for this purpose.

the `kmod_info` list

Back in the days before "kernel extensions" were formalized, they were referred to as `kmod` objects, and the Mach subsystem maintained the list of loaded modules as a `kmod_info_t` chain pointed to by the `kmod` global. The symbol is exported in MacOS, but not in *OS. It can nonetheless be reliably detected by `joker`, and is pointed to by the `lowGlo` structure.

Every kernel extension exports its own `kmod_info` structure (in its `__DATA.__data`, or via the `_PrelinkKmodInfo` integer key in the kernelcache's `__PRELINK_INFO`). That symbol is picked up (by `populate_kmod_info` (from libkern/kxld/kxld_object.c) or `OSKext::initWithPrelinkedInfoDict` for a kernelcache) and then added to the linked list during `OSKext::load`. XNU maintains its own `kmod_t` in the `g_kernel_kmod_info` global, using "`__kernel__`" for its `name` and the Darwin version number for its `version`. The structure, however, is fake, and unused outside of `OSRuntimeInitializeCPP()`.

Figure 3-19: The `kmod_info` structure, visualized



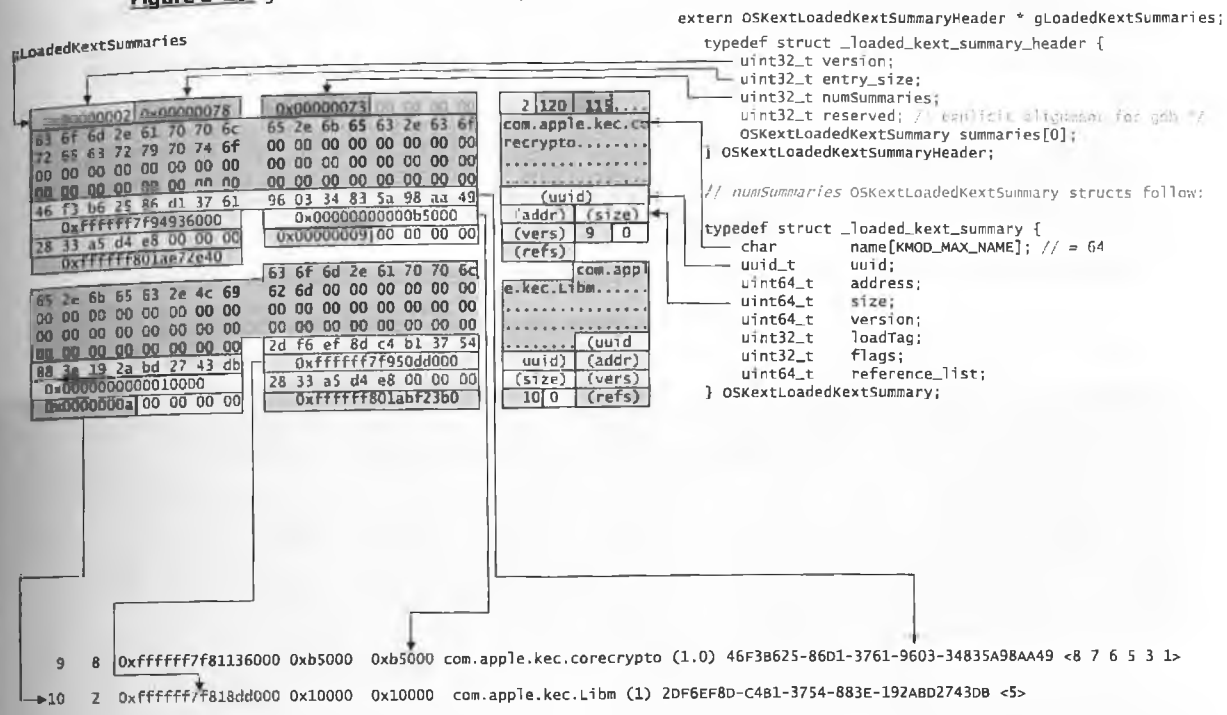
Although presently deprecated, the `kmod_info` structure is still in use, especially during early stages of boot initializtion. As mentioned earlier, the iPhone11,x models use a "1469" kernelcache in which the `__PRELINK_INFO.__kmod_info` section contains an array of addresses for the `kmod_info` structures of all kernel extensions which have been prelinked. The load addresses of the Mach-O headers (which would normally be in the `address` field of the structure) are stored in a separate `__PRELINK_INFO.__kmod_start` section.

`gLoadedKextSummaries`

Apple introduced `gLoadedKextSummaries` as a way to kext information, deprecating the `kmod` list. This is an exported global pointer to a `OSKextLoadedKextSummaryHeader` followed by an array of `OSKextLoadedKextSummary` structures (both defined in libkern/libkern/OSKextLibPrivate.h). Calling `OSReportWithBacktrace()` (from libkern/gen/OSDebug.cpp) is a useful way of walking through the kernel stack, correlating addresses with those in the summary structure, and printing out relevant kext information.

During a panic, `OSKext::printKextsInBacktrace()` uses this method when creating a panic dump. Figure 3-20 shows this important structure, which is easy to find in memory due to its unique header.

Figure 3-20: `gLoadedKextSummaries` (from XNU 4903's libkern/libkern/OSKextLibPrivate.h), visualized



`sLoadedKexts`, and `sKextsById`

The `kmod_info` lists are deprecated, and the `gLoadedKextSummaries` is only used during a panic. XNU maintains the real time data of loaded kexts in a third location, `sLoadedKexts`. This is a static `OSArray` pointer (defined in libkern/c++/OSKext.cpp), and it is further protected by the top level `sKextLock`. The array elements are pointers to `OSKext` classes.

As we've seen, `OSKext` represents an instance of a kext known to the kernel - though not necessarily loaded ones. Those kexts which are in the `sKextsById` array are added by the `registerIdentifier()` method, called from one of the `OSKext` constructors - commonly `withMkext2Info` for MacOS kext loading, but also `withPrelinkedInfoDict` when kernelcaches (in all variants) are involved. The extensions can then be looked up, using `OSKext::lookupKextWithIdentifier()`. The `OSKexts` in the `sLoadedKexts` are inserted into the `OSArray` only after a successful load, and their `flags.loaded` attribute is set to `true` as well. These kexts are commonly looked up by `OSKext::lookupKextWithLoadTag()`, `OSKext::lookupKextWithAddress()` or `OSKext::lookupKextWithUUID()`

Kext Accounting

The kernel needs to keep track of memory used by kernel extensions. For this, XNU provides the `OSKextAccount` structure. This is a minimal structure (defined in libkern/libkern/c++/OSKext.h), containing the `vm_allocation_site_t` associated with kext allocations, the kext's `loadTag`, and a pointer to the `OSKext` object representing the kext itself in kernel memory. A second structure, `OSKextActiveAccount`, contains the address range associated with the kext, and a pointer to the kext's `OSKextAccount`.

Memory allocations made by kexts can be flagged with `VM_TAG_KMOD`. This exploits the fact that the `OSKextAccount` effectively extends the `vm_allocation_site` (discussed later in Chapter 12), and use it instead of the standard `vm_allocation_site`. This way, `OSKextGetKmodIDForSite` can retrieve the responsible kext's load tag, and from it - the bundle identifier. The `zprint(1)` utility displays per-kext wired information by default, or with the `-l` switch.

MacOS 15: System Extensions and DriverKit

With the gradual but systematic approach of locking down MacOS as much as possible, it is clear that kernel extensions must be dealt with. Although XNU's design is such that they are a necessary evil, MacOS seems destined to eventually follow *OS, wherein all the kernel extensions are prelinked at boot into the single kernelcache, which is signed and personalized by Apple.

Unlike *OS, however, quite a few developers rely on kernel extensions for their functionality. Be it for third party drivers, security products, networking, virtualization, or some other purpose, kernel extensions provide vital functionality which third parties need to tap into. The risk, however, is that a poorly coded kernel extension can panic the entire system, or - worse - a malicious one undermine its entire security.

As we've seen, Apple has been tightening kernel extensions, starting with code signatures (10.9), through requirements (10.13) and GateKeeper (`syspolicyd`) interaction. Darwin 19, however, provides an alternative - allowing developers to create, what are in effect, user-mode extensions and drivers, through two new frameworks - SystemExtensions and DriverKit . Apple introduced the concept in WWDC '19 session #702[11], though with little details on its implementation.

The idea is not unlike that of Windows' User Mode Driver Framework (UMDF), in which kernel code calls out to some user space process, in order to perform some operations. In other cases, user mode code can call onto special kernel functions (via some specialized syscall or device handle), to perform the privileged operations. An example of this can be found in MacOS 10.10's Hypervisor.framework and its use of machine-dependent (`machdep`) system calls, as discussed in the next chapter. Darwin 17's Network Extensions model (NECP, discussed in I/16), enables functions previously implemented by Network Kernel Extensions (NKE). Another, less official example can be seen in Darwin's port of FUSE (Filesystem in USEr mode), discussed in Chapter 7.

Apple classifies **Driver Extensions** as those extensions which seek to replace (now legacy) IOKit drivers, and **System Extensions** for all other traditional in-kernel functionality, such as Network Extensions (for packet filtering, tunneling, etc), and Endpoint Security Extensions. The latter are implemented through the new EndpointSecurity.kext, which performs in-kernel KAuth hooks, relaying to the user mode endpointsecurityd(8), to work on behalf of EndpointSecurity.framework clients. For two good examples of using this framework, refer to the Objective See Blog[12]

System Extensions

A system extension is a user-mode Mach-O, but not allowed to exist as a standalone object. It must be deployed as part of an app, housed in the app's Contents/Library/SystemExtensions folder, as its own separate bundle, with an Info.plist specifying a `CFBundlePackageType` of `SYSX`. The extension's Info.plist should also provide a descriptive `CFBundleIdentifier` and must provide a `NSSystemExtensionUsageDescription` string. Network extensions have slightly different requirements, and the `NetworkExtension` key provides a dictionary with the `NEMachServiceName` string and `NEProviderClasses` dictionary. This enables `nesessionmanager(8)` to start the extensions on demand.

The sysextd, a daemon residing in the SystemExtensions.framework. The daemon registers the `com.apple.sysextd` XPC service, overwhich it responds to client requests. These are abstracted as framework `activationRequests`, sent by the owning app when the extension is needed. The XPC service operates over both standard XPC and NSXPC, with the latter allowing remoting through `OSSystemExtensionActivationRequest` Objective-C object.

When a request is issued, the daemon handles the required user approval, which is provided through an authorization dialog using the `NSSystemExtensionUsageDescription` key from the extension's Info.plist. When an application terminates, is uninstalled, or removed to the trash sysextd is notified, so that any contained system extensions can be removed as well.

Driver Extensions

With Darwin 19, Apple provides the standardized DriverKit.framework*. This provides a framework for user mode drivers, aiming to replace commonly used IOKit families (discussed in Chapter 13). As with IOKit, developers can use C++, but unlike it, this is a full C++17 compatible runtime, rather than IOKit's restricted C++.

DriverKit defines a new bundle type, a **Driver Extension**, or .dext. Unlike .kexts, .dexts are flat (i.e. no `Contents/`) user-mode bundles, containing the mandatory `Info.plist`, a `version.plist`, the Mach-O specified by the `CFBundleExecutable` key (commonly with the same name as the bundle), and a `_CodeSignature/` with `CodeResources`.

DriverKit executables are essentially ordinary Mach-O `MH_EXECUTE` types, albeit with slight differences, shown in Figure 3-21:

Figure 3-21: Displaying a .dext Mach-O, using `jtool2(j)`

The DriverKit.framework contains contains about two dozen built-in classes, which are provided as its object-oriented APIs. It also contains two initializers (in its `__DATA_CONST.__mod_init_func`):

- `__GLOBAL__sub_I_IOLib.cpp`, sets `IOVMPageSize` to be the machine's page size.
- `_DriverKit_init` calls `__dyld_register_driverkit_main`, with another symbol, `DriverExecutableMain`. The dyld function is a special addition as of `dyld-732`, which calls `dyld:setMainEntry(void *)()` to set `dyld::sEntryOverride`. This is why DriverKit executables require no `LC_MAIN` (and, even if they had a defined entry point, it would be ignored).

An important aspect of DriverKit is a new code generation tool introduced in XCode 11 - `iig`, the IOKit Interface Generator. Similar in some respects to the venerable `mig`, `iig` is more advanced, in that it can autogenerate C++ code, and is GCD aware. `iig` generates class code from new .iig templates. It is `iig` which is responsible for generating the class structures in `__DATA.__osclassinfo`, as well as serializing and deserializing the `IORPC` messaging format within Mach messages. This is detailed in Chapter 13, which continues the discussion of DriverKit from the kernel's perspective.

---

* - Coincidentally, "DriverKit" is also the name of the original NeXTSTEP kernel framework which eventually became IOKit. It is unclear whether or not Apple chose the name to knowingly collide, but the name is apt, since the original vision of NeXTSTEP's DriverKit was also to provide drivers in user mode.
** - There is also a copy of DriverKit.framework in /System/Library/Frameworks but its purpose is unclear.

Extension Loading

Unlike other System Extensions, which the kernel knows little about, Driver Extensions do have an in-kernel representation. Just like normal IOKit kexts, the .dext's Info.plist specifies an `IOKitPersonalities` dictionary, as is required of "true" IOKit extensions. A new method, `OSKext::isDriverKit()` (`__ZN6OSKext11isDriverKitEv`) identifies Driver Extensions through their `CFBundlePackageType` of `DEXT` (instead of the regular `KEXT`), and the kernel relays the load request to `kextd` in an upcall with a `Kext Request Predicate` of `Dext Daemon Launch`, with the requested `CFBundleIdentifier`, `Driver Extension Server Name` and `Driver Extension Server Tag`.

Before satisfying the load request, `kextd(8)` verifies that the requested .dext is signed with the `com.apple.developer.driverkit` entitlement. As of MacOS 15, `kextd(8)` makes use of SystemExtensions.framework's `OSSystemExtensionClient`, to handle user authorization requests. If the approval is successful, `kextd(8)` adds an `IOUserServerCDHash` key to the personalities dictionary. This is then used in the instantiation of an `IOUserServer` in-kernel object. This class (which we detail in Chapter 13) is similar to `IOUserClient`, but is conceptually its inverse, relaying kernel-supplied method calls to the user mode server - the .dext. The actual work of loading and unloading .dexts is performed by `launchd`, as instructed by kextd(8) over a generated LaunchDaemon plist, submitted over `launch_msg` APIs.

Listing 3-22: A sample `launch_msg` for starting a kext, as caught by `xPoCe(j)`

```
subsystem: 7
handle: 0
request:
  SubmitJob:
    ProcessType: "Driver"
    EnvironmentVariables":
        "LIBTRACE_DRIVERKIT": ""
    UserName: "_driverkit"
    Label : "com.apple.DriverKit.AppleUSBFTDI-(0x10000241f)"
    LaunchOnlyOnce: true
    RunAtLoad: true
    _NullBootstrapPort: true
    SandboxProfile: "com.apple.dext"
    ProgramArguments[0]: "/System/Library/DriverExtensions/DriverKit.AppleUSBFTDI.dext/DriverK
    ProgramArguments[1]: "com.apple.DriverKit.AppleUSBFTDI"
    ProgramArguments[2]: "0x10000241f" # Driver Extension Server Tag from kext request
routine: 100
type: 7
```

Review Questions

1. In Output 3-4, what is another indication that the `com.apple.kpi.*` kexts are in fact pseudo kexts and not real kernel extensions?
2. `OSKext`'s `s[Load/Unload]` is private, as are the accessors, `[get/set][Load/Unload]Enabled`. When are they actually used? What would be the benefit of making them public KPI? Suggest a way to provide them safely in user mode, and note the advantages of doing so.
3. In Output 3-19, why are fragments of the Mach-O header still visible if the data is compressed?
4. Why would Apple leave the MacOS kernelcache as a FAT file, if it only contains a single architecture?
5. How is `kextstat(8)` able to call `kext_request`, a `host_priv` MIG message, if it runs as a normal user?
6. Why is maintaining the kext list in an unexported static `sLoadedKexts` advantageous? What are the potential disadvantages?
7. An IMPORTANT note in `kext_request` (libkern/OSKextLib.cpp) mentions that if `KERN_SUCCESS` is not returned the kernel map "gets messed up (reason as yet unknown)" and therefore uses `op_result` to return success instead. Apple engineers apparently never bothered to figure out the reason. What is it?
8. Additional "xxx" comments in `kext_request` wonder if the name of the calling task can be determined to be `kextd`, or can be found out at all. Can it?
9. What were to happen if `kextutil(8)` from before MacOS 13 (specifically, MacOS 11 or 12) would be "ported" to a later version of MacOS? Why? How is this indicative of a far greater, systemic security issue?

References

1. Apple Developer - Documentation on the supported KPIs - https://developer.apple.com/documentation/kernel
2. Apple - QA1574 - https://developer.apple.com/library/archive/qa/qa1574/_index.html
3. Apple - LZFSE sources - https://github.com/lzfse/lzfse
4. PikerAlpha - LZVN (GitHub)- https://github.com/Piker-Alpha/LZVN
5. Brandon Azad- "Analyzing the iOS 12 kernelcache's tagged pointers" - https://bazad.github.io/2018/06/ios-12-kernelcache-tagged-pointers
6. WWDC 2013 Session 707 - "Sign Here Please" - https://developer.apple.com/videos/play/wwdc2013/707/
7. Apple - Technical Note 2459 - "User-Approved Kernel Extension Loading" - https://developer.apple.com/library/archive/technotes/tn2459/_index.html
8. NewOSXBook forum - "Kext loading in iOS"
9. Zhi Zhou - "ModJack - Hijacking the MacOS Kernel" - HITB 2019 D2T2
10. WWDC 2019 Session 702 - "System Extensions and DriverKit" - https://developer.apple.com/videos/play/wwdc2019/702/
11. Patrick Wardle - "Writing a Process Monitor with Apple's Endpoint Security Framework" - https://objective-see.com/blog/blog_0x47.html

4

Some Assembly Required: Kernel Primitives & Paradigms

Programming in kernel mode demands attention to fine detail. Every allocation counts, and every mistake is lethal. The kernel provides a small set of primitives which are used extensively throughout it, in order to manipulate data structures. These primitives are critical, as they must be both correct and efficient.

This chapter deals with the low level data structures used to manipulate collections of objects - queues, which are built on top of linked lists. It then takes a look at concurrency considerations - ensuring safe access to data shared between cores, using various types of locking primitives - mutexes and spinlocks. These require low level assembly and specialized processor instructions, and so the discussion splits at times to handle both Intel and ARM architectures. This schism continues when discussing processor execution modes - Both Intel and ARM support privilege separation, but the specifics vary greatly, especially when handling traps and exceptions.

Finally, the architectures converge as the chapter discusses the kernel interfaces with user mode - the multiple personalities exposed by XNU - which are largely the same across all Darwin flavors.

Data Structures

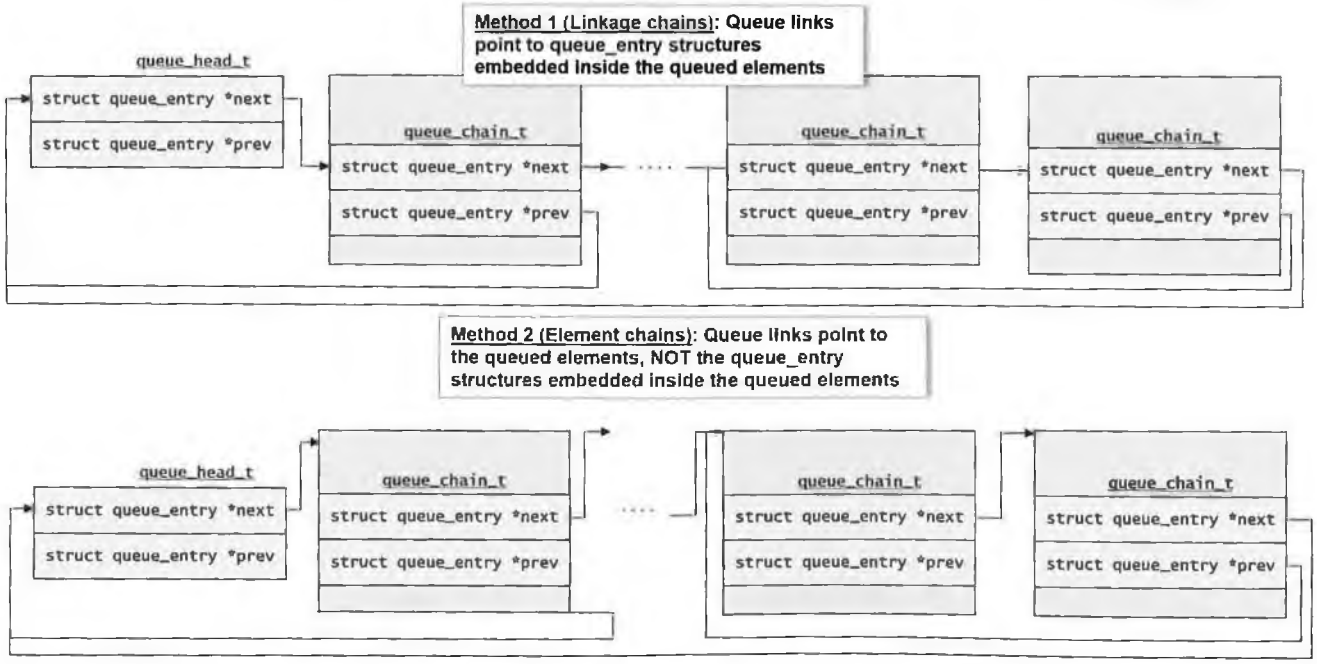
In a perfect world, the kernel could keep track of all objects in contiguous memory arrays, with constant time access. But in the ever-changing, dynamic world of an operating system, objects spring in and out of existence all the time, sometimes quite unpredictably. It follows, therefore, that dynamic data structures must be used to track and hold references to objects, which must be as efficient as possible - memory and CPU-wise.

Queues (Mach)*

At the Mach layer, a `struct queue_entry` is a doubly linked list entry, specifying pointers to the `next` and `prev` elements. This allows the representation of empty queues by having the head element point to itself (i.e. `head->next = head->prev = head`), as well as indicates when the queue has been fully traversed (which can be detected by testing `current->next == head`). There are, however, subtleties with how queues are used. As explained by the comment in osfmk/kern/queue.h, one of two (incompatible) methods may be used:

- **Linkage chains:** Wherein the pointers of the `queue_entry` element point to the corresponding (next or previous) `queue_entry` elements. This makes for fast traversal of queue elements using `qe_foreach*` variants, which simply dereference `next`. The `queue_entry`s themselves are embedded inside their objects as fields. The very first queue element is a "detached" entry, which may exist on its own outside any element. To get to the actual element, pointer arithmetic (specifically, casting the `queue_entry` to a byte pointer and subtracting the `offsetof`) is used.
- **Element chains:** Wherein the pointers point to the object itself, (i.e. the first field in the object structure). With this method, element access is immediate, but iterating through the queue elements requires the pointer arithmetic. Still, this method proves useful when the `queue_entry` is placed at the first field of the object.

Figure 4-1: The queue types offered by Mach



A given element may be used in different queue contexts and methods concurrently. A good example of that are the Mach `thread` structures (osfmk/kern/thread.h), which simultaneously belong to scheduler run queue lists, wait queues, their own task thread groups, as well as the global list of all threads.

Initialization and basic checks can be performed with common methods. Insertion, removal, and iteration, however, will differ between the two methods. Table 4-2 summarizes the queue operations, in line with the verbose header comment of osfmk/kern/queue.h:

* - As of Darwin 19, there is a new queue type - `mpsc_queue_chain_t` - for a Multi-Producer Single-Consumer queue, detailed in kern/mpsc_queue.h. This is an interrupt safe, highly efficient queueing mechanism, but still appears to be of limited use.

Table 4-2: Queue operation styles on Mach queues

Operation	Linkage Chains	Element Chains
Initialize queue	`queue_init`	
Get next/prev element	`queue_[next/prev]`	
Get head/tail	`queue_[first/last]`	
Check if empty	`queue_empty`	
Check if at end	`queue_end`	
Insert (at tail)	`enqueue[_tail]`	`queue_enter`
Insert (at head)	`enqueue_head`	`queue_enter_first`
Remove (head)	`dequeue_head`	`queue_remove_first`
Remove (tail)	`dequeue_tail`	`queue_remove_last`
Insert (after)	`insqueue`	`queue_insert_after`
Remove (current)	`remqueue`	`queue_remove`
Iterate	`qe_foreach`	`queue_iterate`

Linked Lists & Queues (BSD)

The BSD layer also defines its own linked lists and queues, in bsd/sys/queue.h. As detailed in the comments, the file defines four types of structures*:

- **SLISTs**: are singly linked - forward only - lists. Elements contain exactly one pointer, with the `SLIST_HEAD` holding referring to it `slh_first`, and all `SLIST_ENTRY` elements as `sle_next`, with the tail's `sle_next` NULL. These are the simplest and lightest data structures, and elements are inserted at the head, so as to provide LIFO (stack) ordering. Examples of `SLIST`s are the `sysctl_oid_list` (q.v. Chapter 6), and several interface structures used by the network stack.
- **STAILQs**: are singly linked tail queues. They are implemented on top of singly linked lists, with a subtle difference: The `STAILQ_HEAD` holds **two** pointers, an `stqh_first`, and an `stqh_last` - so that at any point the last element of the queue may be found. `STAILQ_ENTRY` elements still hold a single pointer (`stqe_next`). Fairly few kernel objects use `STAILQ`s. Examples include `oslog_stream_[free/buf]_head` and the `nstat_src` list.
- **LISTs**: are doubly linked lists, started by a `LIST_HEAD` element with an `lh_first` pointer. Elements have an `le_next` and `le_prev`, with the tail element's `le_next` and the head element's `le_prev` pointing back to the head. Examples include PID and process group hash tables, persona members, IPv6 firewall chains, and more.
- **TAILQs**: are doubly linked tail queues. TAILQs are used extensively, and examples include VFS mount and vnode lists, process thread lists, and Jetsam's bucket lists.

Unlike Mach queues, BSD queues solely use element chains. Additionally, the last element points to a NULL, rather than back to the queue head. Traversal is performed through *xxx*`_FOREACH` macros in bsd/sys/queue.h, which take in `var`, `head` and `field`, start at *xxx*`_FIRST(head)`, and call *xxx*`_NEXT((var),field)` (which evaluates to `(var)->field.`*xxx*`_next`) until hitting a NULL.

Figure 4-3: The definition of the BSD `SLIST` data structure

```
#define SLIST_HEAD(name, type)                                          \
__MISMATCH_TAGS_PUSH
struct name {  struct type *slh_first; /* first element */  }
__MISMATCH_TAGS_POP

#define SLIST_ENTRY(type)                                               \
__MISMATCH_TAGS_PUSH                                                    \
struct { struct type *sle_next;  /* next element */          }
__MISMATCH_TAGS_POP

#define SLIST_FIRST(head)       ((head)->slh_first)
#define SLIST_NEXT(elm, field)  ((elm)->field.sle_next)
#define SLIST_FOREACH(var, head, field)                                 \
        for ((var) = SLIST_FIRST((head));                               \
            (var);                                                      \
            (var) = SLIST_NEXT((var), field))
```

* - There are also definitions in bsd/sys/queue.h for a `CIRCLEQ`, though marked deprecated and "disgraceful".

The reader is encouraged to familiarize with the BSD structures, (also documented in the `queue(3)` manual page), as they are ubiquitously used throughout XNU. An extensive explanation in bsd/sys/queue.h's header comment summarizes the APIs in a way similar to as was shown in Table 4-2:

Figure 4-4: The queue types offered by BSD, documented in bsd/sys/queue.h

```
/*
 * This file defines five types of data structures: singly-linked lists,
 * singly-linked tail queues, lists, tail queues, and circular queues.
 *
 * A singly-linked list is headed by a single forward pointer. ...
 *                                        ... Singly-linked lists are ideal
 * for applications with large datasets and few or no removals or for
 * implementing a LIFO queue.
 ...
 * A singly-linked tail queue is headed by a pair of pointers, one to the
 * head of the list and the other to the tail of the list ...
 * ...
 * Singly-linked tail queues are ideal for applications with large datasets
 * and few or no removals or for implementing a FIFO queue.
 *
 * A list is headed by a single forward pointer (or an array of forward
 * pointers for a hash table header). ...
 *                                                              ... A list
 * may only be traversed in the forward direction.
 *
 * A tail queue is headed by a pair of pointers, one to the head of the
 * list and the other to the tail of the list.  ...
 * ..... A tail queue may be traversed in either direction.
 *
 * For details on the use of these macros, see the queue(3) manual page.
 *
 *                           SLIST   LIST    STAILQ  TAILQ   CIRCLEQ
 * _HEAD                     +       +       +       +       +
 * _HEAD_INITIALIZER         +       +       +       +       -
 * _ENTRY                    +       +       +       +       +
 * _INIT                     +       +       +       +       +
 * _EMPTY                    +       +       +       +       +
 * _FIRST                    +       +       +       +       +
 * _NEXT                     +       +       +       +       +
 * _PREV                     -       -       -       +       +
 * _LAST                     -       -       +       +       +
 * _FOREACH                  +       +       +       +       +
 * _FOREACH_SAFE             +       +       +       +       -
 * _FOREACH_REVERSE          -       -       -       +       -
 * _FOREACH_REVERSE_SAFE     -       -       -       +       -
 * _INSERT_HEAD              +       +       +       +       +
 * _INSERT_BEFORE            -       +       -       +       +
 * _INSERT_AFTER             +       +       +       +       +
 * _INSERT_TAIL              -       -       +       +       +
 * _CONCAT                   -       -       +       +       -
 * _REMOVE_AFTER             +       -       +       -       -
 * _REMOVE_HEAD              +       -       +       -       -
 * _REMOVE_HEAD_UNTIL        -       -       +       -       -
 * _REMOVE                   +       +       +       +       +
 * _SWAP                     -       +       +       +       -
 */
```

Tree data structures

XNU contains support not just for lists, but also trees. Use of trees, however, is very limited. Up until Darwin 11, splay trees (self-adjusting binary search) were used to maintain Mach's IPC spaces. XNU 2050 did away with this, and now the only trees that are used are Red-Black trees. Support is provided through libkern/libkern/tree.h (which also supports the unused splay trees) and `RB_*` macros, which generate the tree handling functions. `RB_HEAD` expands to a structure containing the `rbh_root`, and `RB_ENTRY` expands to a structure of three pointers, `rbe_left`, `rbe_right` and `rbe_parent`.

The `IOStatistics` class, as well as several networking stack components (e.g. necp, PF, flow control, and (as of Darwin 19) SkyWalk's protons) use RB trees. One main subsystem using these structures is the VM map layer, as we discuss in Chapter 11.
A good way to find RB using components is:

```
jtool2 -S /System/Library/Kernels/kernel |grep _RB_FIND
```

Note, that kernel code is not required to use the built-in data structure support. HFS+ contains its own B-Tree implementation, and APFS (discussed in Chapter 8) uses B+ Trees.

Concurrent resource access

Most threads may be preempted at any time. Such preemptions are unpredictable, and could lead to dangerous race conditions if a thread is preempted during access to memory used by other threads as well. The problem is further exacerbated in multicore architectures, wherein two threads may concurrently access the same memory.

Atomic operations

The simplest way to prevent race conditions is by providing hardware support for **atomic operations**. These are non-preemptible operations, which are also guaranteed to be synchronized by the hardware. Intel uses the `LOCK` opcode, followed by regular access to the memory location, whereas ARM64 uses Load-Store Exclusive opcodes (`[LD/ST]XR`). To provide hardware abstraction, the atomic operations are wrapped by `__c11_atomic_compare_exchange_strong` and others, for which `clang` can automatically emit the right assembly.

hwlocks

The most basic form of locking supported by Mach is **hardware locks**. The comment in osfmk/i386/hw_lock_types.h describes the requirements from such machine dependent locks perfectly:

Listing 4-5: Mach's expectations of a hardware lock, from osfmk/i386/hw_lock_types.h

```
/*
 *      The "hardware lock".  Low-level locking primitives that
 *      MUST be exported by machine-dependent code; this abstraction
 *      must provide atomic, non-blocking mutual exclusion that
 *      is invulnerable to uniprocessor or SMP races, interrupts,
 *      traps or any other events.
 *
 *       hw_lock_data_t          machine-specific lock data structure
 *       hw_lock_t               pointer to hw_lock_data_t
 *
 *      An implementation must export these data types and must
 *      also provide routines to manipulate them (see prototypes,
 *      below).  These routines may be external, inlined, optimized,
 *      or whatever, based on the kernel configuration.  In the event
 *      that the implementation wishes to define its own prototypes,
 *      macros, or inline functions, it may define LOCK_HW_PROTOS
 *      to disable the definitions below.
 *
 *      Mach does not expect these locks to support statistics,
 *      debugging, tracing or any other complexity.  In certain
 *      configurations, Mach will build other locking constructs
 *      on top of this one.  A correctly functioning Mach port need
 *      only implement these locks to be successful.  However,
 *      greater efficiency may be gained with additional machine-
 *      dependent optimizations for the locking constructs defined
 *      later in kern/lock.h..
 */

struct hslock {
        uintptr_t       lock_data;
};
```

Indeed, hwlocks are as minimal as they get, and the `struct hslock` is defined in the same manner in osfmk/arm/hw_lock_types.h. A set of `hw_lock_*` functions are declared in osfmk/kern/simple_lock.h, and shown in Table 4-6:

Table 4-6: The `hw_lock_*` functions, from osfmk/kern/locks.h

Function	Purpose
`hw_lock_[un]lock(`*`lock`*`)`	Lock or unlock a hardware lock
`hw_lock_[un]lock_nopreempt(`*`lock`*`)`	Variants of above, called only from non-preemptible context
`hw_lock_try[_nopreempt](`*`lock`*`)`	Try to acquire lock (possibly in non-preemptible context)
`hw_lock_held(`*`lock`*`)`	Inquire if lock is currently being held
`hw_lock_to(`*`lock,timeout`*`)`	Attempt to acquire *lock, until successful or timeout occurs*

Note, that all the `hw_lock_*` KPIs are kernel internal (`MACH_KERNEL_PRIVATE`), which is why the rows in Table 4-6 appear shaded. Further, these functions, though exported by XNU in the MacOS build, are not part of any official KPI, not even the Private.kext. They are rarely used directly (save for during `panic` or early serial console initialization). Acquiring hardware locks involves the aforementioned atomic operations as well as repeated attempts ("spinning"), so the higher level spinlock abstractions are commonly used instead.

Hardware locks are as optimized as the architecture can allow. Intel's `PAUSE` instruction is used after a failed lock, so as to indicate to the CPU that the calling thread is in a "spin-wait" loop[1], and potentially optimize power consumption. Similarly, in ARM64 failing to acquire a lock will call a `WFE` instruction[2], which puts the CPU into low power mode, while waiting for an event to be generated. The event can be explicitly generated (using a `SEV` instruction), but this functionality is already built into `STREX` so that an event is automatically generated when a lock is unlocked by this instruction.

Spinlocks

There are cases where locks are only held for few cycles, whereas the cost of the context switching resulting from waiting from them might be orders of magnitude more. In those cases, it makes more sense for the lock contender to do so in a **busy wait**, constantly checking in a loop if the lock is available.

Spinlocks are usually directly declared using the `lck_spin_t typedef`, but the `decl_lck_spin_data(class,name)` macro (from osfmk/kern/locks.h) may also be used. The `class` specifies a modifier (e.g. `extern`, `static`..), and can be omitted. XNU also offers **simple locks**, which are closely related to spinlocks and are either part of the spinlock implementation (in Intel) or aliased over it (in ARM). These are similarly declared with a `decl_simple_lock_lock_data` macro, and IOKit further wraps them as `IOSimpleLocks`.

By whatever name, spinlocks are functionally the same. The implementation, however, is hardware specific. The `lck_spin_t` is a type normally defined (in osfmk/[arm|i386]/locks.h) over an opaque `struct __lck_spin_t__`. If `KERNEL_PRIVATE` is `#defined`, the structure is expanded to either 10 `longs` (on Intel) or two `uintptr_t` (ARM), which are still dubbed opaque, but at least reveal the structure's `sizeof()`, which is required for storage considerations. It is only when `MACH_KERNEL_PRIVATE` is `#defined` that the spinlock object's fields become fully visible.

Listing 4-7: The `lck_spin_t` definitions, from osfmk/[arm|i386]/locks.h

```
// i386:

#if     defined(MACH_KERNEL_PRIVATE)
typedef struct {
        volatile uintptr_t      interlock;
#if     MACH_LDEBUG
        unsigned long   lck_spin_pad[9]; /* XXX - usimple_lock_data_t */
#endif
} lck_spin_t;

// arm:
typedef struct {
        struct hslock   hwlock;
        uintptr_t       type;
} lck_spin_t;
#endif
```

Spinlocks have a grave drawback - they burn 100% CPU during a busy wait - but such spans would hopefully not extend beyond a few dozens of cycles, assuming the lock holder is concurrently executing on a different core. In cases where this is not true (for example, if the lock holder dies and fails to release the lock), the spinning thread cannot be preempted and will spin forever on the core it was dispatched to.

Outside of osfmk/[arm|i386]/locks[_arm|].c spinlocks remain opaque, so a standardized KPI must be used to access them. This KPI, defined in osfmk/kern/locks.h is machine-independent, and shown in Table 4-8. Shaded rows are available only `#if KERNEL_PRIVATE`:

Table 4-8: The `lck_spin_*` KPIs, from osfmk/kern/locks.h

Function	Purpose
`lck_spin_alloc_init(grp,attr)`	Allocate (and return) a new spinlock object, belonging to lock group *grp*, with attributes *attr*
`lck_spin_init(lck,grp,attr)`	(re)-Initialize an already allocated spinlock object, belonging to lock group *grp*. The attributes remain unused.
`lck_spin_[un]lock(lck)`	Acquire or release a spinlock
`lck_spin_[destroy|free](lck,grp)`	Destroy (deallocate) or free a spinlock
`lck_spin_sleep[_deadline](lck,action,event,interruptible[,deadline])`	Sleep until *lck* becomes available (or until deadline)
`lck_spin_{un}lock_nopreempt(lck)`	Acquire or release a spinlock, non-preemptible context only
`lck_spin_try_lock[_nopreempt](lck)`	Try once to acquire (without spinning)

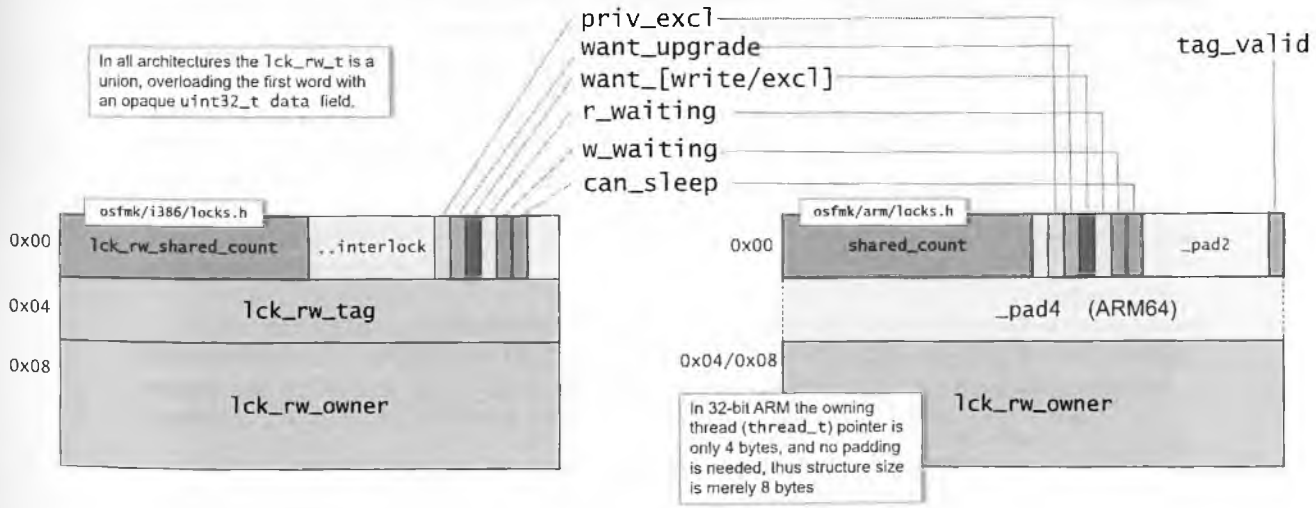
Read-Write Locks

There are specific scenarious where drawing a distinction between different access requirements proves beneficial. The classic case is differentiating between read and write access: Multiple readers might be concurrently accessing an item, yet not interfere with one another. Only writers interfere with each other - and with other readers.

It follows, therefore, that readers may ask for the lock to be shared, whereas writers would require exclusive access. So long as at least one reader exists, writers would have to wait, and when exactly one writer acquires the lock (exclusively), both readers and writers must wait. Further optimizations are to order the lock queue so that writers get priority over readers when in the queue (lest they be starved).

The `lck_rw_t` meets all these requirements. As with spinlocks, the implementation is `#defined MACH_KERNEL_PRIVATE`, and machine specific. Figure 4-9 shows the MacOS and *OS implementations, which are nearly identical, though the bit field names are slightly different.

Figure 4-9: The Intel and ARM implementations of a `lck_rw_t`



The `lck_rw_ts`, like their spinlock relatives, are often declared with `decl_lck_rw_data`. KPI-wise, they naturally offer a richer set of functions:

Table 4-10: The `lck_rw_*` KPIs, from osfmk/kern/locks.h

Function	Purpose
`lck_rw_alloc_init(grp,attr)`	Allocate (and return) a new rwlock object, belonging to lock group *grp*, with attributes *attr*
`lck_rw_init(lck,grp,attr)`	(re)-Initialize an already allocated rwlock object, belonging to lock group *grp*. The attributes remain unused.
`lck_rw_[un]lock_exclusive(lck)`	Acquire or release a rwlock exclusively (i.e. for write)
`lck_rw_[un]lock_shared(lck)`	Acquire or release a rwlock shared (i.e. for read)
`lck_rw_lock_yield_shared(lck,force)`	Acquire a rwlock shared, possibly forcing yield
`lck_rw_[un]lock(lck, type)`	Acquire or release a rwlock
`lck_rw_[destroy|free](lck,grp)`	Destroy (deallocate) or free a rwlock
`lck_rw_sleep[_deadline](lck,action,event,interruptible[,deadline])`	Sleep until *lck* becomes available
`lck_rw_lock_shared_to_exclusive(lck)` `lck_rw_lock_exclusive_to_shared(lck)`	Upgrade or downgrade a rw lock
`lck_rw_try_lock_[shared|exclusive](lck)`	Try once to acquire (without spinning)

Mutex locks

There are cases wherein memory access involves a non-primitive object, which cannot be accessed atomically, such as a structure object. Further, a spinlock often proves inadequate in some cases, due to the amount of time (CPU cycles) for which the object must be held. This calls for a new type of synchronization object - the `lck_mtx_t`. Unlike spinlocks, the scheduler is aware of mutex lock objects, and waiting threads can block, and relinquish the CPU to other threads in the interim. Mutex locks also offer another advantage, of priority boosts. Much in the way read-write lock holders enjoy priority boosts, so do those of mutex locks. This way, waiters can be compensated for their idle time, and the lock can hopefully be dropped as soon as possible.

Mutexes may be defined with `decl_lck_mtx_data(class,name)`, from osfmk/kern/locks.h. The macro use is preferred, especially when mutex locks are embedded in more complex structures. And, as with the other lock types we've seen so far, their implementation is greatly machine dependendent. Listing 4-11 compares the Intel and ARM mutex locks:

Listing 4-11: The ARM `lck_mtx_t`, from osfmk/arm/locks.h

```
typedef struct _lck_mtx_ {
    union {
        uintptr_t    lck_mtx_data;     /* Thread pointer plus lock bits */
        uintptr_t    lck_mtx_tag;      /* Tag for type */
        };                             /* arm: 4   arm64: 8 */
    union {
        struct {
          uint16_t   lck_mtx_waiters; /* Number of waiters */
          uint8_t    lck_mtx_pri;     /* Priority to inherit */
          uint8_t    lck_mtx_type;    /* Type */
                };
          struct {
              struct _lck_mtx_ext_    *lck_mtx_ptr; /* Indirect pointer */
                };
          };                           /* arm: 4   arm64: 8 */
} lck_mtx_t;                           /* arm: 8  arm64: 16 */
```

Listing 4-12: The Intel `lck_mtx_t`, from osfmk/i386/locks.h

```
/* The definition of this structure, including the layout of the
 * state bitfield, is tailored to the asm implementation in i386_lock.s
 */
typedef struct _lck_mtx_ {
    union {
            struct {
                    volatile uintptr_t              lck_mtx_owner;
                    union {
                            struct {
                                    volatile uint32_t
                                            lck_mtx_waiters:16,
                                            lck_mtx_pri:8,
                                            lck_mtx_ilocked:1,
                                            lck_mtx_mlocked:1,
                                            lck_mtx_promoted:1,
                                            lck_mtx_spin:1,
                                            lck_mtx_is_ext:1,
                                            lck_mtx_pad3:3;
                                    };
                            uint32_t        lck_mtx_state;
                    };
                    /* Pad field used as a canary, initialized to ~0 */
                    uint32_t                        lck_mtx_pad32;
            };
            struct {
                    struct _lck_mtx_ext_            *lck_mtx_ptr;
                    uint32_t                        lck_mtx_tag;
                    uint32_t                        lck_mtx_pad32_2;
            };
    };
} lck_mtx_t;
```

The `lck_mtx_*` KPIs (also in osfmk/kern/locks.h) are identical to those of spinlocks (shown in Table 4-8), with `spin` changed to `mtx`, and without the `_nonpreempt`, since a mutex lock by definition is preemptible. `KERNEL_PRIVATE` interfaces include `lck_mtx_try_lock[_spin[_always]]` and `lck_mtx_yield`. The private interfaces also handle lock contention and the potential need for backoff.

Lock Groups

Comparing the `hw_lock_*` functions (Table 4-6) and those of `lck_spin_*` (Table 4-8), one notable difference is the reference to **lock groups** and attributes. Lock groups can also be seen in the `rw_lock_*` functions, as well as those of `lck_mtx_*`.

Mach's lock groups are used to loosely lump together locks with related function, and maintain very high-level statistics on the locks used in them. The groups are created before any of their locks are (using a call to `lck_grp_alloc_init`), allocated, and tail enqueued in the `lck_grp_queue`. When the individual locks are first initialized, a call to `lck_grp_lckcnt_incr(`*group*`,`*type*`)` increments the *type* counter in the group. Similarly, when a lock is destroyed, its group's *type* counter is decremented. The *type* in question may be `LCK_TYPE_[SPIN|MTX|RW]`.

If the lock group is created with the `LCK_GRP_ATTR_STAT` attribute (which is the only one presently defined), statistics are kept for each lock type, including counts of locks held, waited, missed, etc, in the `lck_grp_[spin/mtx/rw]_stat_t` structures, which are all part of the combined `lck_grp_stat` field of the group. Listing 4-13 shows the `lck_grp_t` type:

Listing 4-13: The `lck_grp_t` structure, from osfmk/kern/locks.h

```
typedef struct _lck_grp_ {
/* 0x00 */ queue_chain_t    lck_grp_link;   /* lck_grp_queue chain */
/* 0x10 */ uint32_t         lck_grp_refcnt;
           uint32_t         lck_grp_spincnt;
/* 0x18 */ uint32_t         lck_grp_mtxcnt;
           uint32_t         lck_grp_rwcnt;
/* 0x20 */ uint32_t         lck_grp_attr;
/* 0x24 */ char             lck_grp_name[LCK_GRP_MAX_NAME];
/* 0x64 */ lck_grp_stat_t   lck_grp_stat;   /* union: lck_grp_[spin/mtx/rw]_stat */
```

The `lck_grp_alloc_init()` is a simple wrapper over `kalloc()`, and `lck_grp_init`. The latter, however, is `MACH_KERNEL_PRIVATE`, as are most of the other lock group methods. The public lock lifecycle methods call these (`lck_grp_lckcnt_[incr|decr]`, etc). Virtually every subsystem of the kernel initializes its own lock group. Calls to `lck_grp_alloc_init()` are abundant, and thanks to the human readable names, it is easy to locate them in the kernel disassembly. This is one of the techniques used by the `joker` module to symbolicate *OS kernels.

The lock group statistics can be retrieved using `host_lockgroup_info`, which is MIG routine #218 in the mach_host subsystem.

Lock Debugging/Tracing

Locks protect critical code paths in kernel, and are critical in and of themselves: Using locks incurs overhead of spinning or sleeping, and using them incorrectly can severely impact performance - or hang the entire system. It's therefore important to be able to debug and trace locks.

XNU's primary tracing facility, `kdebug`, is well equipped to trace read-write and mutex locks. The `DBG_MACH_LOCKS` subclass (0x60) traces these locks' lifecycle functions. When DTrace is enabled, it can go even deeper, through the `lockstat` provider (the legacy /dev/lockstat). The `lockstat(1M)` command is well documented and is part of the open source `dtrace` project.

Summary

Table 4-14 summarizes the key attributes of all the lock mechanisms offered by Mach. Note, that the table is not entirely accurate, in the sense that Mach's spinlocks do have a sleep-friendly interface, and mutex locks have a spinlock oriented one. Being the non-standard and less used interfaces, these can generally be discounted for comparison purposes.

Table 4-14: Comparison of Mach locking mechanisms

Attribute	SpinLocks	Read/Write Locks	Mutex Locks
Recursive	NO		
Wait	Busy	Idle	Idle
Preemptible when held	NO	YES	YES
Sleep/Block safe	NO	YES	YES
Priority boosts (promotions)	NO	YES	YES
Lock queue ordering	None	Write before read	None

Another important observation is that none of the Mach locking primitives allow for recursive locking. IOKit's `IORecursiveLock` object (from iokit/Kernel/IOLocks.cpp) wraps a `lck_mtx_t` in a structure, along with a lock group, `thread` pointer, and `count`. If the owning `thread` `IORecursiveLock[Try]Lock()`s it, the `mutex` is left locked, and the `count` is incremented, and likewise decremented on `IORecursiveLockUnlock()`. Any other thread accessing an owned lock has to wait on the `mutex` until it is freed. This will occur when the owner calls `IORecursiveLockUnlock()` the same number of times as it did `..Lock`, which will decrement `count` to zero and release the mutex.

As we will see in Chapter 10, Darwin 18 Adds a significant enhancement to locking with turnstiles, (and Darwin 19, with gates), which enable the scheduler to wake up waiters in a prioritized, rather than FIFO order.

Per-CPU data

Multiprocessor and multicore environments provide a useful feature of **per-CPU data**. This is an addressable area which resides in the core's own cache, such that every core has its own private copy of the data. The `current_cpu_datap()` macro provides access to this important data structure - in Intel, through the %GS register, and in ARM64, through the special `TPIDR_EL1` register. The kernel also maintains the per-cpu data of all CPUs in the `cpu_data_ptr[]` array. Though uncommon, a thread running on one core might need access to data on another. For this, the `cpu_datap(cpunum)` may be used. This evaluates to the array entry.

The cpu data is initialized during machine specific startup code. For *OS, this means `arm_init()`. The `cpu_data_init` is called with a pointer to the `BootCpuData` structure. Slave processors call `cpu_data_init` on their own data pointer during `ml_processor_register()`. The `CpuDatap` is loaded onto the current Mach thread's `machine.CpuDatap` field every time `machine_switch_context()` is called. The per-CPU data is for all CPUs is stored at a fixed physical address (`CPU_DATA_ENTRIES`) as a hard-coded offset from the `LowResetVectorBase`, so it can be quickly located when a core is started or resumed.

The per-CPU data is naturally machine-specific, and ARM's is much larger than Intel's. There are fields common to both the Intel and ARM architectures (at different offsets). Table 4-15 shows some of these:

Table 4-15: common CPU data fields, in MacOS and *OS

Intel	ARM64	Description
`cpu_number`		Core number of this CPU
`cpu_processor`		Mach `processor_t` object
`cpu_running`		Running or suspended state
`cpu_int_state`		Interrupt state (pointer)
`cpu_[type/subtype]`		`CPU_[SUB]TYPE_*` constants
`quantum_timer_deadline`		Expiration of current scheduler quantum
`rtclock_timer`		An `rtclock_timer_t` handle
`cpu_pending_ast`		An `ast_t` bitmask of `AST_*` flags (q.v. Chapter 10)
`cpu_active_thread`		Mach `thread_t` object

Intel's `cpu_data` (from osfmk/i386/cpu_data.h) also includes `cpu_task_map`, `cpu_task_cr3`, `cpu_kernel_cr3` and `cpu_ucr3` and a `cd_shadow` address of double mapped data. Intel code makes use of the `CPU_DATA_GET` macro (ibid), which takes the member and type.

ARM's `cpu_data` (from osfmk/arm/cpu_data_internal.h) includes L2 and L3 cache id and size, a `cpu_stat` field with various exceptions and interrupt counts, and the `interrupt_handler`, `..nub`, `target` and `..refCon` (detailed later in this chapter, under "Interrupt Handling"). ARM code makes frequent use of `getCpuDatap()`, which returns a pointer to the entire data structure, then dereferencing the specific member.

Low level assembly code in both architectures (e.g. osfmk/x86_64/idt64.s or osfmk/arm64/locore.s), makes use of `CPU_*` macros, generated by `DECLARE` statements in osfmk/[i386/arm/arm64]/genassym.c, which compiles to a tool, which emits a assyms.s file, in which the values (e.g. `CPU_DATA_ENTRIES` above) are initialized using `offsetof()`.

Processor execution modes

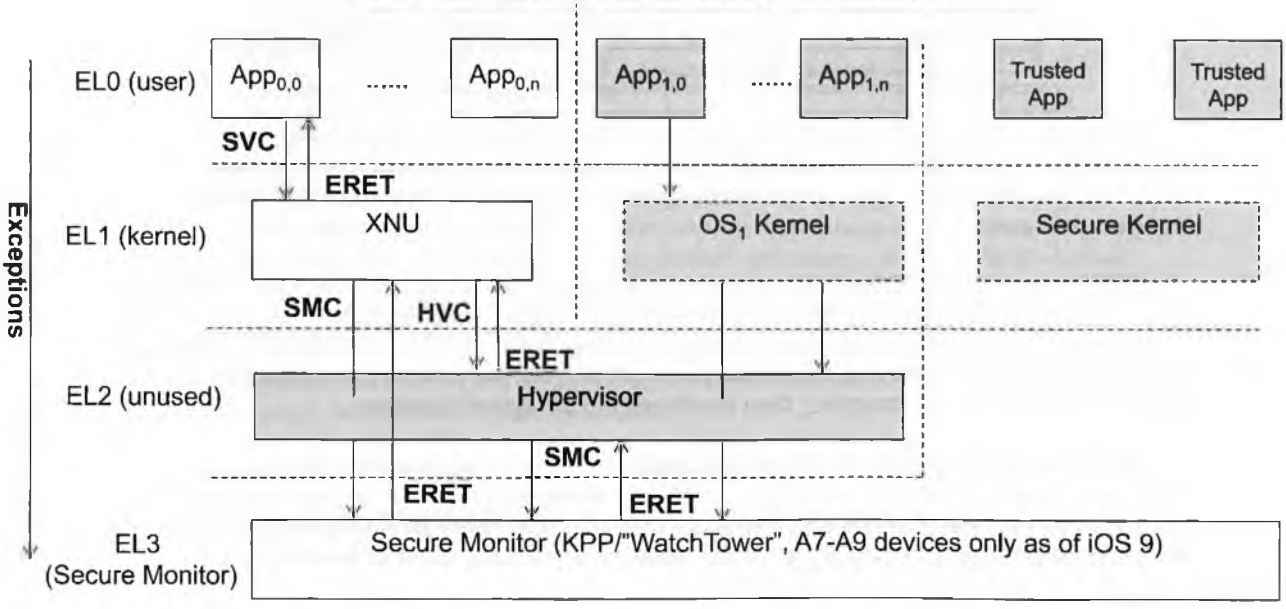
Modern CPU architectures enforce different modes of execution, which are sometimes referred to as **protection rings**, due to their common visualization as concentric rings. Unlike Venn diagrams, though, the innermost rings are deemed more privileged: Certain instructions or special register access may only be permitted in them. The MMU may further impose restrictions, such that certain page mappings may only be accessible in higher modes as well. It is only natural, then, to enforce privilege separation between user mode and kernel mode using these modes of execution.

In Intel architectures, the execution modes are actually referred to as rings. The outermost ring - Ring 3 - is for user mode. Ring 2 and 1 remain unused, and the kernel occupies Ring 0. The choice of ring is made by setting the **I/O P**rivilege **L**evel bits (12-13) of the `EFLAGS` register. Every process also has a **C**urrent **P**rivilege **L**evel. With the advent of virtualization, a new protection ring was required to enable hypervisor operation, which is even more privileged than the kernel. To solve this, a "Ring -1" was added.

32-bit ARM architectures traditionally recognized only two basic modes - **USeR**, and Supervisor (`SVC`), so the ring analogy was not used. These modes were implemented as bits in the **C**urrent **P**rogram **S**tatus **R**egister. ARMv7 introduced TrustZone (the Secure Monitor), which required an additional mode, implemented by an additional register, the **S**ecure **C**onfiguration **R**egister, whose least significant bit (`NS`) indicates if the mode is not secure or not-not secure (i.e. secure). There were also additional "sub" modes for special case handling - interrupts (IRQ), fast interrupts (FIQ), data/instruction abort, and undefined instructions, maintained as additional bits in the CPSR.

The move to 64-bit enabled ARM to redesign things more neatly, as well as build in hypervisor support. Aarch64 therefore recognizes four **Exception Levels**: EL0 is user mode, EL1 is kernel mode, EL2 is reserved for the hypervisor (if any), and EL3 for the secure monitor (if any). Note, "if any", because Apple has opted to leave out the hypervisor (thus opting for a single EL0/EL1), and uses EL3 as just a monitor layer, and only on A7 through A9. Figure 4-16 shows the ARMv8 exception model, emphasizing those exception levels which Apple actively makes use of:

Figure 4-16: The ARMv8 exception level model, and Apple's usage



In AArch 64 the general purpose registers (X0-X29), Link Register (X30, a.k.a LR), Stack Pointer (X31, a.k.a SP) and the program counter (PC) are shared across all exception levels, along with 32 128-bit registers (for SIMD vector/FP). In addition to those there are quite a few system registers, most of which are suffixed by "_ELx", denoting the minimal exception level they can be accessed in. This allows for privilege separation, by placing the kernel's address space mappings in `TTBR[0/1]_EL1`, whereas those of user mode are in `TTBR[0/1]_EL0`.

Different execution modes define different privileges, so it follows that mode traversals be subject to specific, well-defined constraints. Both Intel and ARM architectures do so, and distingish between two possible mode traversals.*

Mode Traversal

- **Voluntary Traversals** are the result of a specific instruction in a less-privileged mode, which will result in the CPU switching to the other mode. The instruction may be called freely, but the caller has no control as to where in the more privileged mode execution will continue. That choice is made by the more privileged code, before it yields execution to the less privileged one. In Darwin x86_64, the special instruction to get to ring 0 is `syscall` and to get back - `sysret`. In the past other instructions - 32 bit architectures traditionally used `INT` (which, in effect simulates an interrupt) or `sysenter`.
- **Involuntary traversals** are due to an interrupt, exception, fault, or abort. There are some differences between the above (pertaining mostly to whether or not the instruction they occurred during may be retried), but the common denominator of these being that they (commonly) cannot be intercepted in the same mode, and force a mode traversal to a more privileged handler. Once again, the handler needs to be installed by the more privileged code before "dropping" to the lower privilege level. In the Aarch64 case, a handler may be installed in the same mode from EL1 and higher.

Intel: SYSENTER (Voluntary Mode Traversal)

When `syscall` is called, execution is transferred to Ring 0 at a predefined point indicated by the `MSR_IA32_LSTAR` model specific register. XNU sets these registers (along with `MSR_IA32_SYSENTER_*` for the legacy `sysenter` method of mode traversal) to the `hi64_syscall` handler. The `syscall` instruction also loads the CS and SS selectors from `MSR_IA32_STAR`, and masks the `RFLAGS` using `MSR_IA32_FMASK`. All of these Model Specific Registers are set by XNU in osfmk/i386/mp_desc.c, shown in Listing 4-17:

Listing 4-17: Setting the Model Specific Registers for Syscall/sysret (in osfmk/i386/mp_desc.c)

```
/*
 * Set MSRs for sysenter/sysexit and syscall/sysret for 64-bit.
 */
void cpu_syscall_init(cpu_data_t *cdp)
{
#if MONOTONIC
        mt_cpu_up(cdp);
#else /* MONOTONIC */
#pragma unused(cdp)
#endif /* !MONOTONIC */

    wrmsr64(MSR_IA32_SYSENTER_CS, SYSENTER_CS);
    wrmsr64(MSR_IA32_SYSENTER_EIP, DBLMAP((uintptr_t) hi64_sysenter));
    wrmsr64(MSR_IA32_SYSENTER_ESP, current_cpu_datap()->cpu_desc_index.cdi_sstku);
    /* Enable syscall/sysret */
    wrmsr64(MSR_IA32_EFER, rdmsr64 (MSR_IA32_EFER | MSR_IA32_EFER_SCE);
    /*
     * MSRs for 64-bit syscall/sysret
     * Note USER_CS because sysret uses this + 16 when returning to
     * 64-bit code.
     */
    wrmsr64(MSR_IA32_LSTAR, DBLMAP((uintptr_t) hi64_syscall));
    wrmsr64(MSR_IA32_STAR, (((uint64_t)USER_CS) << 48) | (((uint64_t)KERNEL64_CS) << 32));
    /*
     * Emulate eflags cleared by sysenter but note that
     * we also clear the trace trap to avoid the complications
     * of single-stepping into a syscall. The nested task bit
     * is also cleared to avoid a spurious "task switch"
     * should we choose to return via an IRET.
     */
    wrmsr64(MSR_IA32_FMASK, EFL_DF|EFL_IF|EFL_TF|EFL_NT);
}
```

* - A somewhat nitpicky observation is that the ARMv8 documentation states that the only way to move between exception levels is by taking or returning from an exception. In this sense, the `SVC`, `HVC` and `SMC` special instructions can be thought of as manually and deterministically triggering a synchronous exception.

Intel: IDT (Involutary Traversals)

Intel architectures handle involuntary traversals through the **Interrupt Dispatch Table** (IDT). XNU prepopulates the table with values from osfmk/x86_64/idt_table.h, visible in the kernel's `__HIB.__desc` section `_master_idt64` (which is readily exported). XNU has limited freedom here - since the IDT is defined by Intel (and is thus largely the same in Linux and Windows as well). The IDT is a 256 entry vector used for traps, exceptions and interrupts, specifying the address of an Interrupt Service Routine (ISR) for each. The first 32 are reserved for traps and exceptions, which are defined by the processor, as shown in Figure 4-20 (next page). XNU also uses entries 0x7f through 0x82 (for legacy 32-bit system call gates), leaving all the rest to interrupt handling (discussed later).

The IDT is defined in osfmk/i386/mp_desc.c, which includes osfmk/x86_64/idt_table.h twice: First, with the `TRAP*` and `INTERRUPT` macros `#defined` as mere `externs` (so the handler symbols are brought to scope). The second time to declare the table itself, by redefining the macros as `fake_descriptor64` structures (from osfmk/i386/seg.h, shown in Table 4-18).

Table 4-18: The IDT macro redefinitions, from osfmk/i386/mp_desc.c

Macro	limit	size_or_IST	access
TRAP	KERNEL64_CS (0x8)	0	K_INTERRUPT_GATE
TRAP_IST1		1	
TRAP_IST2		2	
INTERRUPT		0	
USER_TRAP[_SPC]		0	U_INTERRUPT_GATE

The `K_..` and `U_INTERRUPT_GATE` macros are `#defined` further as `(ACC_P|ACC_PL_[KU]|ACC_INTR_GATE)` macros (respectively), which themselves are `#defined` in osfmk/i386/seg.h. The resulting table is page aligned and placed into a special section, `__HIB.__desc`.

Table 4-19-a: The fake descriptor constructs, from a MacOS 14 kernel

```
morpheus@Bifröst (~)$ itool2 -d _master_idt564 /System/Library/Kernels/kernel
_master_idt64:
0xffffff8000107000: 0xffffff8000193580          _idt64_zero_div
0xffffff8000107008: 08 00 00 8E 00 00 00 00
0xffffff8000107010: 0xffffff80001944e0          _idt64_debug
0xffffff8000107018: 08 00 10 8E 00 00 00 00
0xffffff8000107020: 0xffffff8000194630          _idt64_nmi
0xffffff8000107028: 08 00 20 8E 00 00 00 00
0xffffff8000107030: 0xffffff8000193590          _idt64_int3
0xffffff8000107038: 08 00 00 EE 00 00 00 00
0xffffff8000107040: 0xffffff80001935a0          _idt64_into
0xffffff8000107048: 08 00 00 EE 00 00 00 00
..
0xffffff8000107ff0: 0xffffff80001944c0          _idt64_preempt
0xffffff8000107ff8: 08 00 00 8E 00 00 00 00
```

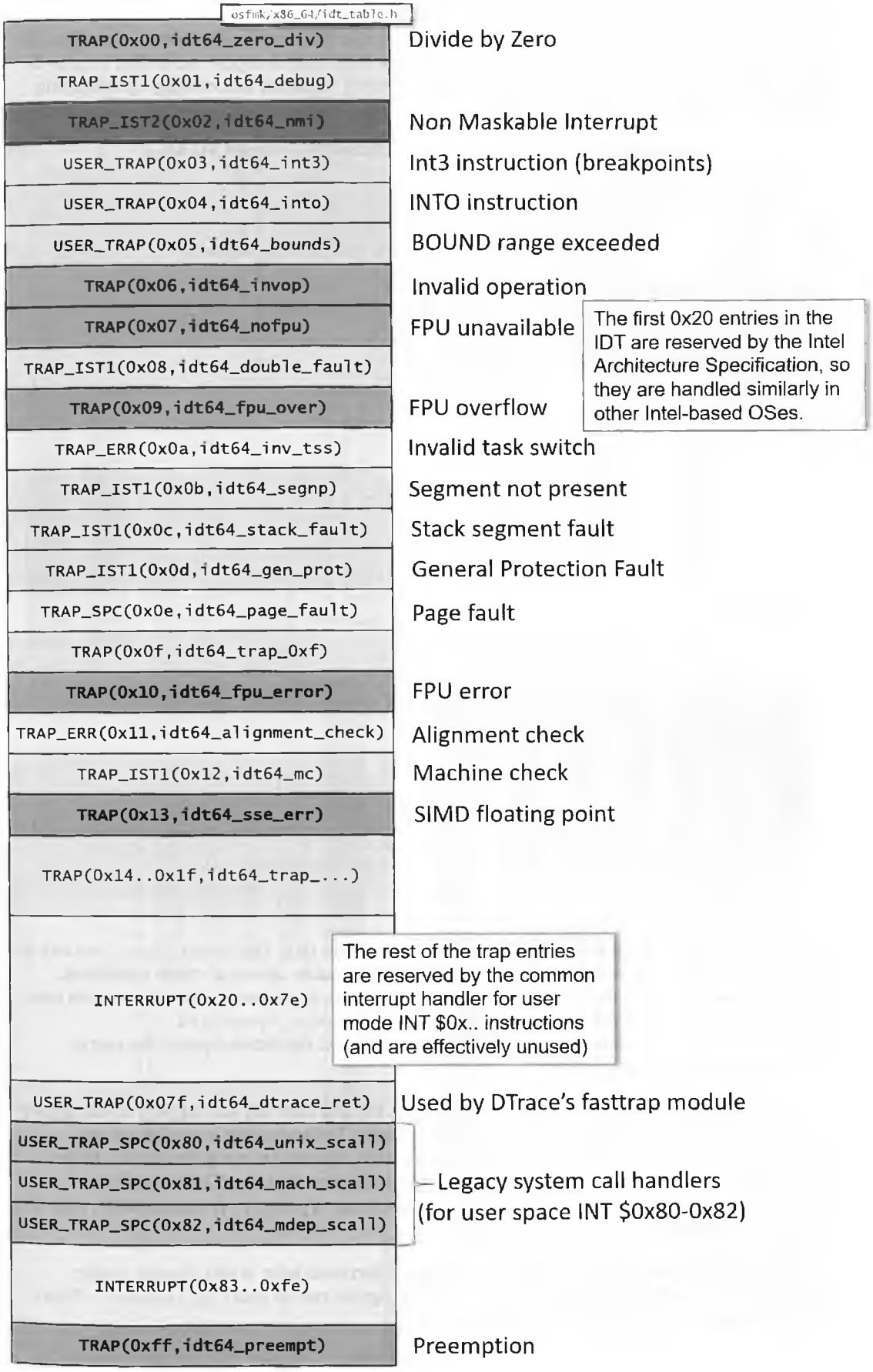
During runtime, the `fake_descriptor64` structures are loaded and fixed up by calls to `fix_desc64()` in `cpu_desc_init()`. The resulting table in memory is more compact, and uses offsets into the kernel segment, rather than the absolute addresses above:

Output 4-19-b: The actual descriptor constructs, from a MacOS 14 kernel in-memory image

```
morpheus@Bifröst (~)$ xnoop dump _master_idt64
0xffffff8007307000 80 35 08 00 00 8e 09 00   a5 f6 ff ff 00 00 00 00   .5..............
0xffffff8007307010 e0 44 08 00 01 8e 09 00   a5 f6 ff ff 00 00 00 00   .D..............
0xffffff8007307020 30 46 08 00 02 8e 09 00   a5 f6 ff ff 00 00 00 00   0F..............
0xffffff8007307030 90 35 08 00 00 ee 09 00   a5 f6 ff ff 00 00 00 00   .5..............
0xffffff8007307040 a0 35 08 00 00 ee 09 00   a5 f6 ff ff 00 00 00 00   .5..............
..
```

The descriptors point to handlers in `__HIB.__text` code. These are generated using the same macros of osfmk/x86_64/idt_table.h, only this time the `TRAP` and `INTERRUPT` macros are used, redefined so as to generate the assembly of the actual functions, using two additional macros: `Entry`, which creates the symbol, and `IDT_ENTRY_WRAPPER`, which pushes the trap/interrupt number (`T_...` constant, from osfmk/i386/trap.h) on the stack, and jumps to the common `L_dispatch`.

Figure 4-20: The Intel architecture IDT



The `L_dispatch` code checks if the flow came from user or kernel mode, swaps the value of GS accordingly, and (as of mid Darwin 18) jumps to the first entry of the `idt64_hndl_table0`. The `idt64_hndl_table0` is another vector of pointers, to variants of `ks_dispatch()` and `ks_[64bit]_return()`. The flow then diverges - according to user or kernel mode origin and 32- or 64-bitness (saving different registers, accordingly, in each), but eventually merges at `L_common_dispatch`.

Listing 4-21: The `L_dispatch` code (from XNU 4903's osfmk/x86_64/idt64.s)

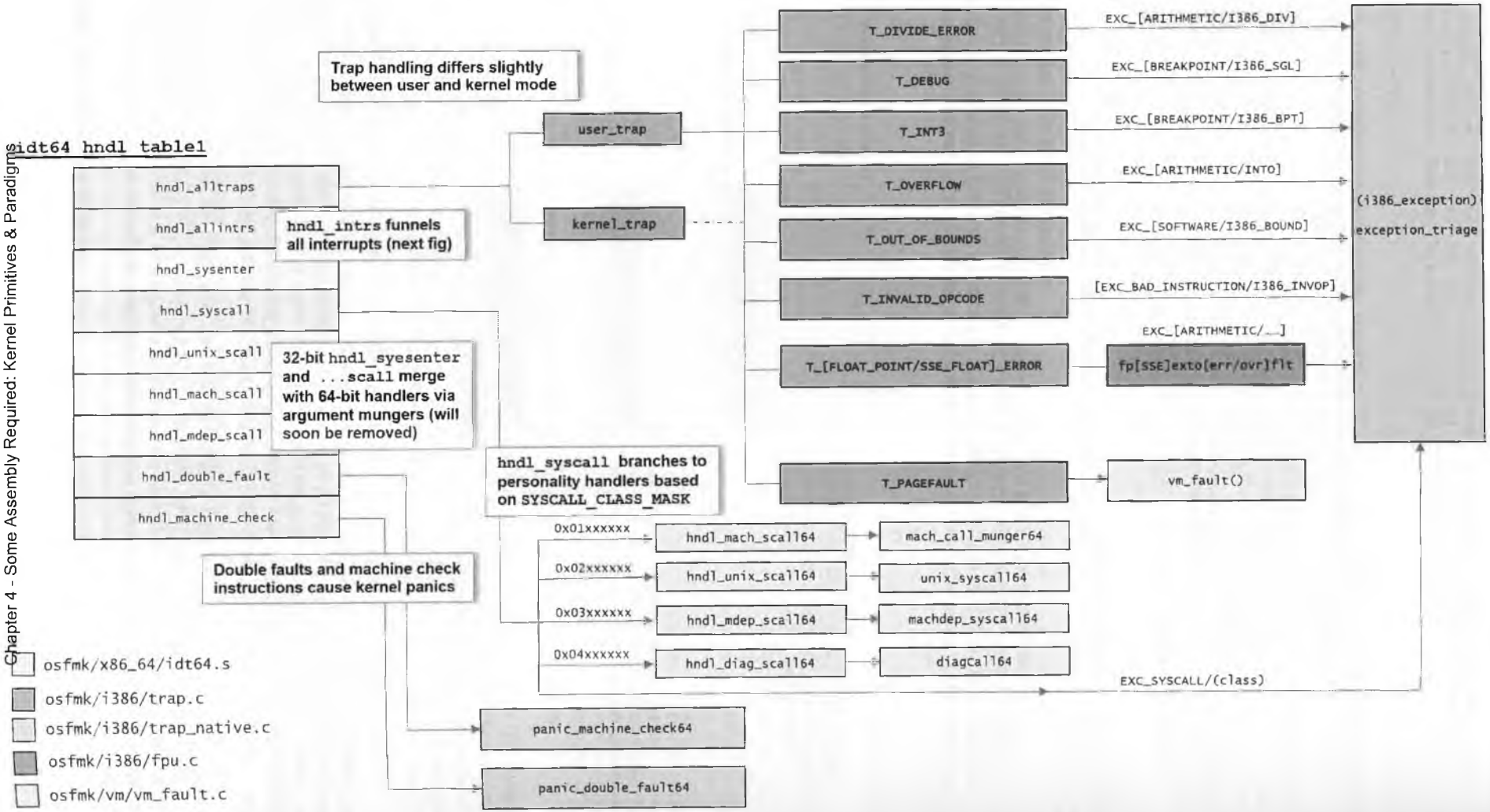
```
.globl EXT(idt64_hndl_table0)
EXT(idt64_hndl_table0):
/* 0x00 */      .quad EXT(ks_dispatch)
/* 0x08 */      .quad EXT(ks_64bit_return)
/* 0x10 */      .quad 0 /* Populated with CPU shadow displacement*/
/* 0x18 */      .quad EXT(ks_return)
#define TBL0_OFF_DISP_USER_WITH_POPRAX  0x20
/* 0x20 */      .quad EXT(ks_dispatch_user_with_pop_rax)
#define TBL0_OFF_DISP_KERN_WITH_POPRAX  0x28
/* 0x28 */      .quad EXT(ks_dispatch_kernel_with_pop_rax)
#define TBL0_OFF_PTR_KERNEL_STACK_MASK  0x30
/* 0x30 */      .quad 0 /* &kernel_stack_mask */

EXT(idt64_hndl_table1):
        .quad   EXT(hndl_allintrs)
        .quad   EXT(hndl_alltraps)
        .quad   EXT(hndl_sysenter)
        .quad   EXT(hndl_syscall)
        .quad   EXT(hndl_unix_scall)
        .quad   EXT(hndl_mach_scall)
        .quad   EXT(hndl_mdep_scall)
        .quad   EXT(hndl_double_fault)
        .quad   EXT(hndl_machine_check)
..
L_dispatch:
        pushq   %rax
        testb   $3, 8+ISF64_CS(%rsp)
        jz      1f
        swapgs
        leaq    EXT(idt64_hndl_table0)(%rip), %rax
        mov     16(%rax), %rax
 L_dispatch_kgsb:
        mov     %gs:CPU_TASK_CR3(%rax), %rax
        mov     %rax, %cr3
1:
        leaq    EXT(idt64_hndl_table0)(%rip), %rax
        /* The text/data relationship here must be preserved in the doublemap,
         * and the contents must be remapped */
        /* Indirect branch to non-doublemapped trampolines */
        jmp *(%rax)
```

The `L_common_dispatch` made to `idt64_hndl_table1`. The `idt64_hndl_table1` is the "real" handler table, which has entries to handle all possible causes of mode transitions. Voluntary transitions (i.e. system calls) are carried out by `hndl_syscall`, which assumes one of the system call personalities - `mach_call_munger64`, `unix_syscall64`, `machdep_syscall64` or `diagCall64` - based on the most significant byte of the syscall number (`SYSCALL_CLASS_MASK`).

Although Apple has announced the Catalina (10.15) and later will not support 32-bit, legacy (32-bit) system calls are still supported in the final betas. Their handling is carried out by `hndl_sysenter`, or (in the case of the even older `INT $0x80-$0x82` instructions), by the `hndl_[mach/unix/mdep]_scal` labels, which call the corresponding 32-bit handlers (`mach_call_munger()`, `unix_syscall()`, or `machdep_syscall()`, respectively), from the architecture agnostic C portions of the kernel.

Interrupts are carried out by `hndl_allintrs` (discussed later in this chapter, under "Interrupts"). All other non-voluntary transitions are carried out by `hndl_alltraps()`. Figure 4-22 (next page) shows a slightly simplified flow.

Figure 4-22: Intel trap (exception) handling

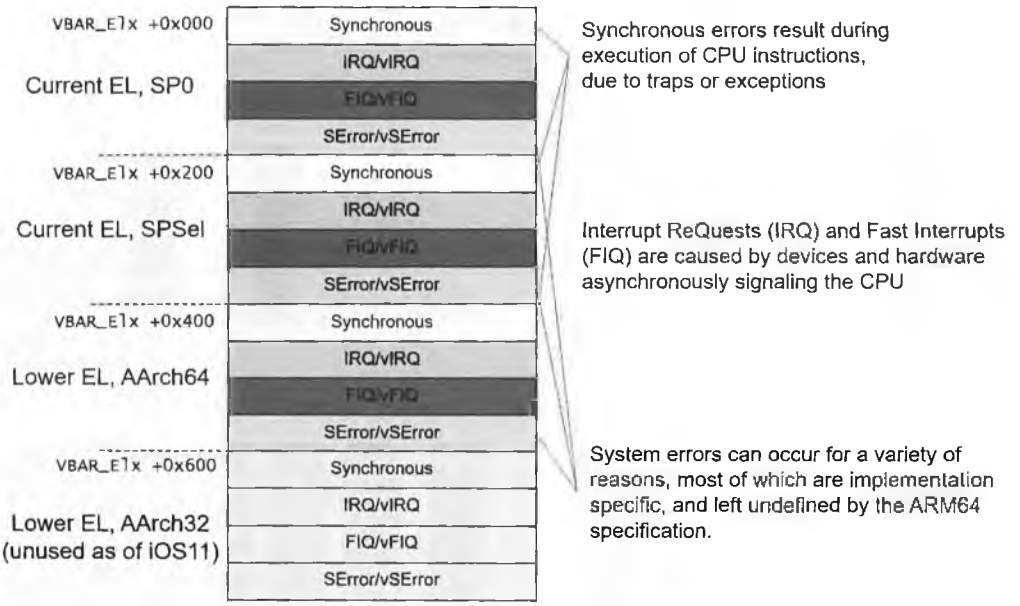


ARM: Exception Vectors

ARM doesn't really distinguish between voluntary and non-voluntary mode traversals: As with the Intel case, there is a specialized mode traversal instruction for the voluntary case (SVC, for Aarch32 supervisor mode or Aarch64's EL1)*. In either case, however, ARM architectures treat all mode traversals the same way, through **exception vectors**.

Whereas the slots of the ARM32 exception vectors are merely 4 bytes, each, those of Aarch64 are 128 bytes each. Unlike the ARM32 case, wherein the code has to branch eslewhere, AArch64's 32 possible instructions often allow the exception to be handled inline rather than branching elsewhere. AArch64 also makes a distinction as to which EL (current or lower) and architecture the exception occurred in. The slots are thus used to provide different exception handlers, with the same semantics duplicated for possible levels, as shown in Figure 4-23:

Figure 4-23: The aarch64 Exception Vector



Outside A10 and A11, the exception vector is set by initializing the value of the VBAR_EL1 register to the area of memory holding the vector contents. Due to the special instruction and the vector structure - entries 0x80 apart - it's easy to find the vector when inspecting the kernelcache (or the firmware binaries). Listing 4-24 shows the vector setting on an A12 kernelcache, with NOPs suppressed.

Listing 4-24: The Exception vector as set by XNU 4903 on an iPhone11

```
_ExceptionVectorsBase:
fffffff0079e2000  B     0xfffffff0079e3000 ; Synchronous      |
fffffff0079e2080  B     0xfffffff0079e3074 ; IRQ/vIRQ         | CurrentEL,
fffffff0079e2100  B     0xfffffff0079e30f4 ; FIQ/vFIQ         |    SP0
fffffff0079e2180  B     0xfffffff0079e3174 ; SError/vSError   |_____________
fffffff0079e2200  B     0xfffffff0079e31e4 ; Synchronous      |
fffffff0079e2280  B     0xfffffff0079e3228 ; IRQ/vIRQ         | CurrentEL,
fffffff0079e2300  B     0xfffffff0079e3268 ; FIQ/vFIQ         |    SPSEL
fffffff0079e2380  HALT (self-ref. branch) ; SError/vSError   |_____________
fffffff0079e2400  B     0xfffffff0079e32e8 ; Synchronous      |
fffffff0079e2480  B     0xfffffff0079e336c ; IRQ/vIRQ         | LowerEL,
fffffff0079e2500  B     0xfffffff0079e33f4 ; FIQ/vFIQ         |    Aarch64
fffffff0079e2580  B     0xfffffff0079e347c ; SError/vSError   |_____________
....  (No LowerEL, AArch32, as of iOS11)
..
_common_start:
..
fffffff0079ec31c  ADRP X0, 2097142        ; R0 = 0xfffffff0079e2000
fffffff0079ec320  ADD  X0, X0, #0         ; R0 = R0 + 0x0 = 0xfffffff0079e2000
fffffff0079ec324  ADD  X0, X0, X22        ; Convert exception vector address to KVA
fffffff0079ec328  SUB  X0, X0, X23        ;
fffffff0079ec32c  MSR  VBAR_EL1, X0       ; Exception Vector set to 0xfffffff0079e2000
```

* - Aarch64 also supports HVC for hypervisor (EL2), and both flavors support SMC (Secure Monitor/EL3), as shown in Figure 4-16. The hypervisor is not used by Apple, and EL3 is only used from A7 through A9.

The exception vector is set from `common_start`. A secondary exception vector, `LowExceptionVectorBase` (from osfmk/arm64/start.s), is used during early-boot (`start_first_cpu`), but this is only temporary as all of its entries spin (`B 0x0`).

Setting `VBAR_EL1` is a security-critical operation, and so as in the A10 and A11 processors it is protected in KTRR-enabled kernels by the `__LAST.__pinst` section. This section (generated from osfmk/arm64/pinst.s) contains thin trampolines for "protected instructions", which first call the `check_instruction` macro, and only then branch to perform the instructions. The `check_instruction` macro (ibid.) compares the protected instruction and the following `RET` so as to ensure it has not been modified in any way. The `__LAST.__pinst` is initially executable, but `arm_vm_prot_finalize()` (from osfmk/arm64/arm_vm_init.c) removes the permission when they are no longer expected to be needed, after the kernel has initialized.

When `__ARM_KERNEL_PROTECT__` is #defined, the vector entries are populated by a `BRANCH_TO_KVA_VECTOR` macro, loading the "real" vector entry from an `TPIDR_EL1` offset address into X18, and then branching to it. As of the iPhone11 (A12+), however, `VBAR_EL1` (along with `TTBR1_EL1` and `TCR_EL1`), is again set through normal calls without either of the abovementioned macros, as Apple instead relies on special lockdown registers (described in Volume III, v1.7+) to protect against malicious tampering.

Listing 4-25: The setting of `VBAR_EL1` in XNU 4903, from iPhone9 and iPhone10 kernels

```
// .macro BRANCH_TO_KVA_VECTOR expansion:
fffffff0070dc000    0xd538d092  MRS     X18, TPIDR_EL1      ;
fffffff0070dc004    0xf9422e52  LDR     X18, [X18, #1112]   ; R18 = *(R18 + 1112)
fffffff0070dc008    0xf9407252  LDR     X18, [X18, #224]    ; R18 = *(R18 + 224)
fffffff0070dc00c    0xf9400252  LDR     X18, [X18, #0]      ; R18 = *(R18 + 0)
fffffff0070dc010    0xd61f0240  BR      X18                 ;
...
// Setting the exception vector is performed through a call to _pinst_set_vbar
fffffff0070e835c    0x90ffffa0  ADRP    X0, 2097140         ; R0 = 0xfffffff0070dc000
fffffff0070e8360    0x91000000  ADD     X0, X0, #0          ; R0 += 0x0 = 0xfffffff0070dc000
fffffff0070e8364    0x8b160000  ADD     X0, X0, X22         ;
fffffff0070e8368    0xcb170000  SUB     X0, X0, X23         ;
fffffff0070e836c    0xaa1e03e1  MOV     X1, X30             ; R1 = R30
fffffff0070e8370    0x9400005a  BL      0xfffffff0070e84d8  ;
        _pinst_set_vbar (0xfffffff0070dc000,ARG1);
...
_pinst_set_vbar:
// The .macro check_instruction expansion
fffffff0070e84d8    ----------  MOVZKKK  X2, 0xd65f03c0d518c000
fffffff0070e84e8    0x90002a03  ADRP    X3, 1344            ; R3 = 0xfffffff007628000
fffffff0070e84ec    0x91002063  ADD     X3, X3, #8          ; R3 += 0x8 = 0xfffffff007628008
fffffff0070e84f0    0xf9400063  LDR     X3, [X3, #0]        ; R3 = *R3 = *0xfffffff007628008
fffffff0070e84f4    0xeb03005f  CMP     X2, X3              ;
fffffff0070e84f8    0x54000001  B.NE    0xfffffff0070e84f8 ; HALT (Self-referential branch)
fffffff0070e84fc    0x1414fec3  B       __pinst_set_vbar    ;  0xfffffff007628008       ;
// and, in __LAST.__pinst, we find:
__pinst_set_vbar:
fffffff007628008    0xd518c000  MSR     VBAR_EL1, X0            ; VBAR_EL1 set to ARG0
fffffff00762800c    0xd65f03c0  RET
```

All ARM exception vector entries funnel to the First Level Exception Handlers (`fleh_*`), implemented directly in assembly (also in osfmk/arm64/locore.s), and then by the Second Level Exception Handler (`sleh_*`) implemented in C (osfmk/arm64/sleh.c). In between the two levels is a common `fleh_dispatch64`, responsible for "spilling" the register state (that is, saving it to stack as an `arm_context_t` structure), and optionally nuking all register values if the exception is from userspace, before dispatching to the second level handler.

The Interrupt-driven mode transitions (IRQ and FIQ) are discussed in the next section, so this leaves us with the two remaining types - Synchronous Exceptions and System Errors.

Synchronous Exceptions

Most mode traversals will be of the synchronous exception type. Handling will get to the second level handler, `sleh_synchronous`, which will take three arguments:

- **The ARM64 register state:** This is an `arm_context_t` structure, defined in osfmk/mach/arm/thread_status.h as a struct containing the `arm_saved_state` and `arm_neon_saved_state` fields. The former is the familiar state encoded in Mach exception messages and used by the `thread_act` MIG's `thread_[get/set]_state` - an `arm_state_hdr_t` indicating bitness, and a union of 32-bit or 64-bit registers (As of iOS 11, 64-bit only). The latter holds the NEON (Q,D and S) registers.
- **The Exception Syndrome Register (ESR_EL1):** A 32-bit value whose six most significant bits (26:31) hold the exception code. The `ESR_EC` macro is used to retrieve the value, which is then switched over, according to exception codes per the AARCH64 specification[3] and defined by `ESR_EC_*` codes (in osfmk/arm64/proc_reg.h). The 25 least significant bits (0:24) hold an Instruction Specific Syndrome, retrieved by the `ESR_ISS` macro. As the name implies, the internal encoding varies with exception code.
- **The Fault Address Register (FAR_EL1):** A 64-bit value specifying the virtual address which resulted in the fault or exception. This is important when handling page faults (so as to page in the address, if valid), and detecting kernel memory manipulation (faults occuring in the kernel text).

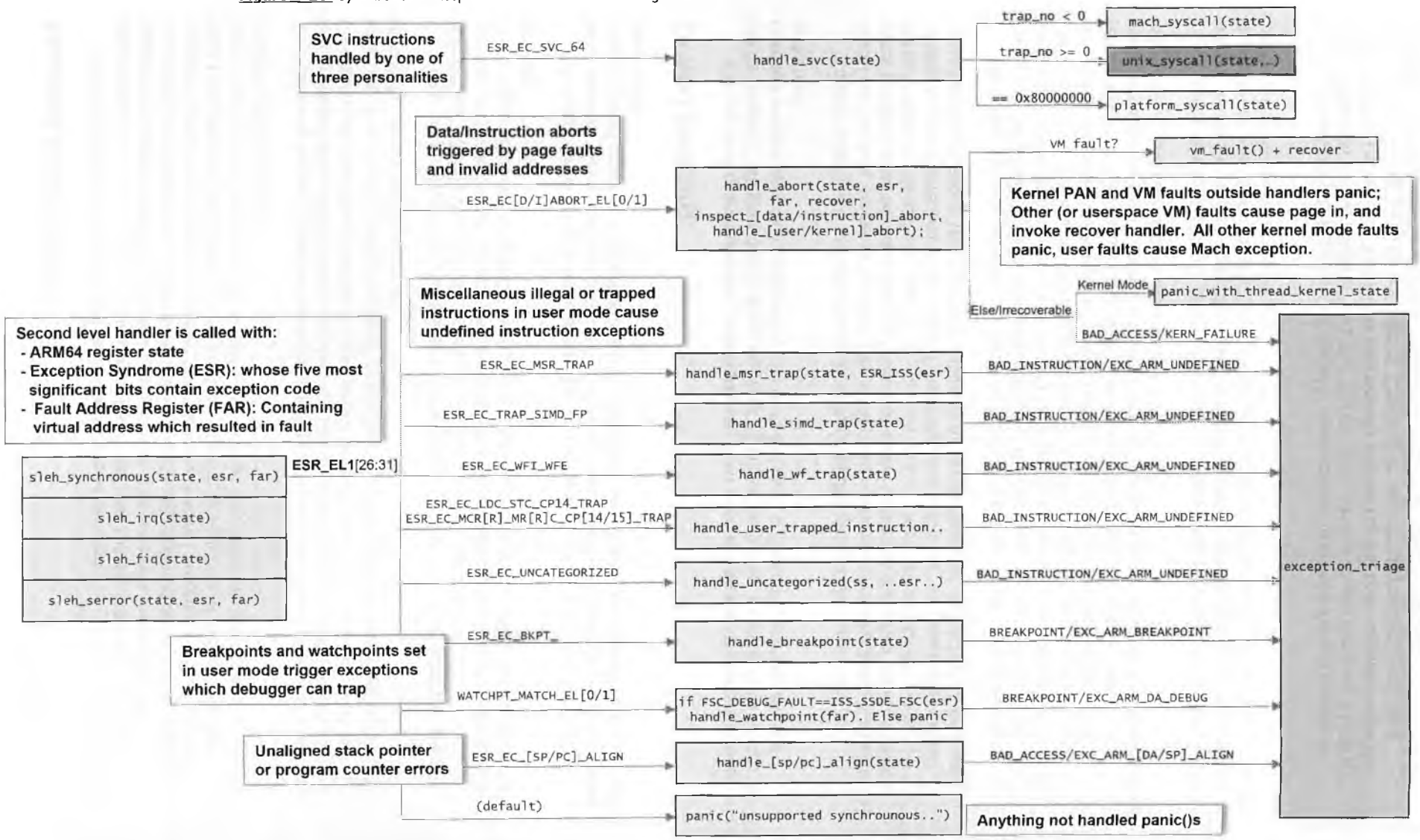
The handling of the various codes by sleh_synchronous is depicted in Figure 4-26 (next page). As shown, exceptions can generally be classified into three categories:

- **The SVC instruction:** providing the means for voluntary traversal from user mode. Code 0x15 indicates the SVC instruction was called in user mode. The ISS value (holding the SVC's immediate) is ignored, as the only SVC value used is #128. Control is transferred to `handle_svc()` (also in sleh.c), which then provides one of the three personalities exposed by *OS variants (i.e. one of `unix_syscall()`, `mach_syscall`, or `platform_syscall()`, from bsd/dev/arm/systemcalls.c, osfmk/arm64/bsd_arm64.c or osfmk/arm64/machine_routines.c), calling `get_saved_state_svc_number()` (from osfmk/mach/arm/thread_status.h) to get the value from the ABI defined register (X16).
- **Instruction/Data aborts:** these are commonly due to page faults, and as such may be recoverable. Exceptions to the rule are when the abort was from kernel mode, during an interrupt, is a kernel text violation, Priveleged-Access-Never (PAN), or is otherwise unexpected. Page faults are otherwise handled as explained in Chapter 11. If the fault results in a successful page in, execution resumes normally. User mode unserviceable faults result in a Mach exception (via `exception_triage()`), and kernel-mode ones panic. This behavior is also the same for aborts which do not arise from page faults.
- **All other anticipated exceptions:** which include miscellaneous instruction trapping, breakpoints and watchpoints. When caught from EL0, most these all result in Mach exceptions, variants of `EXC_BAD_INSTRUCTION`, `EXC_BAD_ACCESS`, or `EXC_BREAKPOINT`, via `exception_triage()`.

System Errors

Handling of system errors is performed by `sleh_serror()`, which takes the same arguments of `sleh_synchronous()`. After a quick sanity check on the context, handling is deferred to `arm64_platform_error()`. This function varies according to the exact type of A-chip used. In A7-A9 with Kernel Patch Protection (`KERNEL_INTEGRITY_WT`), control is first given to the `kernel_integrity_error_handler`, which kicks in and panics with the all-too familiar `SError` panic. In other cases, if a platform handler is registered with the current CPU data, that handler will be invoked. `arm64_implementation_specific_error()` is called in cases where no suitable handler can be found, and calls `panic_plain()` to dump the ESR, FAR, and a slew of specific registers.

Figure 4-26: Synchronous exception second level handler logic



Returning to user mode

Kernel code servicing user mode callers calls `thread_exception_return()` to return to user space. Although the function is implemented in machine specific assembly, the flow is basically the same in both Intel and ARM architectures. The code first checks for any asynchronous software traps which need servicing. ASTs are explained in detail throughout Chapter 10, but for now we can consider them as mere deferred procedure calls. If no ASTs need to be serviced, the kernel can return to user by simply restoring all the register state (`ks_64bit_return` in osfmk/x86_64/idt64.s or `exception_return_restore_registers` in osfmk/arm64/locore.s).

Returning to user mode requires switching the processor back from a privileged mode to a non privileged one, and this always involves the use of special instructions. On Intel, the transition back from Ring 0 to Ring 3 is performed through `IRET` when returning from an interrupt, or `sysretq` (as the inverse of `sysenter`*). In ARM, the transition from SVC/EL1 to USR/EL0 is performed by an `ERET`.

There are two noteworthy additions in the ARM64 case: The first, is on systems wherein WatchTower (KPP) is used (A7-A9, for which `CONFIG_KERNEL_INTEGRITY` and `KERNEL_INTEGRITY_WT` are both `#defined`). On these systems, the secure monitor portion of KPP is triggered by enabling NEON instructions. This is shown in III/13-16, which discusses KPP in depth.

The second addition is a more recent one, and applies in *OS 11.3 (XNU 4570.31.3) and later, wherein `__ARM_KERNEL_PROTECT__` is `#defined`. In order to avoid Spectre-style cache attacks, kernel space EL1 mappings must be unmapped, and `TTBR0_EL1` is switched to the user space ASID.

Context Switching

It is not at all guaranteed that, upon returning from a mode traversal, the original thread will be resumed. Whether due to some interrupt, quantum expiration, or some other reason, a **Context switch** operation, suspending one thread in favor of executing another, may be performed.

Context switching is understandably a highly processor idiosyncratic operation, so `machine_switch_context()`, from the osmfk/*[i386/arm/arm64]*/pcb.c is called. If an address space switch is required, `machine_switch_context()` switches the pmap (Intel's CR3 or ARM's TTBR0). In MacOS `act_machine_switch_pcb()` (from osfmk/i386/pcb_native.c) handles segment, interrupt stack and LDT switching, and calling `ml_hv_cswitch()` (from osfmk/i386/pcb.c) allows the hypervisor callbacks for preemption and dispatching to be invoked.

`machine_switch_context()` can then finally perform the actual context switching - i.e. saving the old thread registers and loading those of the new one, using machine-specific assembly in osfmk/*arch*/cswitch.s. Listing 4-27 (next page) shows the machine specific code of MacOS and *OS, juxtaposed, with the ARM64 macros (which elegantly shrink the source) deliberately expanded. Note that A12 adds code to restore the per-process ARMv8.3 PAC registers of `AP[ID]BKey[Lo/Hi]_EL1`, and store/validate the JOP Hash (a G-key authenticated value) in the thread context. The code has so far been redacted, but is pointed out in the Listing, and can be seen post-analysis with `jtool2`.

A similar symbol `machine_load_context()`, merely loads the register state into the CPU without saving the original state. Because this has the effect of destroying the current context, it is used only once - from `kernel_bootstrap` when loading the `kernel_bootstrap_thread` function (both in osfmk/kern/startup.c).

Apple's iOS ABI Function Call Guide[4] mentions that in ARM64 the x18 register is "reserved for the platform" and that "Conforming software should not make use of it.". Note in Listing 4-27 that X18 is intentionally trashed (by the code expanded from the `set_thread_registers` macro). Checking its value thus provides a neat (and possibly, the only) way for a thread to detect if it has been context switched.

* - `otool(1)` disassembles instruction as a `sysexitl`

Listing 4-27: The `Switch_context` implementations of x86_64 and arm64, juxtaposed

```
/*
 * thread_t Switch_context(thread_t        old,
 * /*thread_continue_t */  void            (*cont)(void),
 *                         thread_t        new)
 *
 */

ENTRY(Switch_context)                   LEXT(Switch_context)
popq  %rax
/* Test for a continuation and skip all state saving if so... */
cmpq  $0, %rsi                          cbnz    x1, Lswitch_threads
jne   5f
                        /* get old kernel stack top */
movq  %gs:CPU_KERNEL_STACK,%rcx         ldr     x3, [x0, TH_KSTACKPTR]

                                        // save_general_registers  x3, x4 (expanded) //
movq  %rbx,KSS_RBX(%rcx)                stp             x16, x17, [x3, SS64_X16]
movq  %rbp,KSS_RBP(%rcx)                stp             x19, x20, [x3, SS64_X16]
movq  %r12,KSS_R12(%rcx)                stp             x21, x22, [x3, SS64_X19]
movq  %r13,KSS_R13(%rcx)                stp             x23, x24, [x3, SS64_X21]
movq  %r14,KSS_R14(%rcx)                stp             x25, x26, [x3, SS64_X23]
movq  %r15,KSS_R15(%rcx)                stp             x27, x28, [x3, SS64_X25]
movq  %rax,KSS_RIP(%rcx)                stp             fp,  lr,  [x3, SS64_FP]
                                        // A12+ code differs and calls out to PACGA signer func here //
                                        mov             x3, sp
movq  %rsp,KSS_RSP(%rcx)                str             x3, [x2, SS64_SP]

                                        str             d8,  [x3, NS64_D8]
                                        str             d9,  [x3, NS64_D9]
                                        str             d10, [x3, NS64_D10]
                                        str             d11, [x3, NS64_D11]
                                        str             d12, [x3, NS64_D12]
                                        str             d13, [x3, NS64_D13]
                                        str             d14, [x3, NS64_D14]
                                        str             d15, [x3, NS64_D15]

5:                                        Lswitch_threads:
/* return old thread */                 // set_thread_registers x2, x3, x4 (expanded) //
movq  %rdi,%rax                         msr  TPIDR_EL1, x2
/* new thread in %rdx, set active */    ldr  x3, [$0, TH_CTH_SELF] // Get cthread pointer
movq  %rdx,%gs:CPU_ACTIVE_THREAD        mrs  x4, TPIDRRO_EL0       // Extract cpu number from TPIDRRO_EL0
                                        and  x4, x4, #(MACHDEP_CPUNUM_MASK)
                                        orr  x4, x3, x4            // Save new cthread/cpu to TPIDRRO_EL0
                                        msr  TPIDRRO_EL0, x4
                                        ldr  x3, [x2, TH_CTH_DATA] // Get new cthread data pointer
                                        msr  TPIDR_EL0, x3         // Save data pointer to TPIDRRW_EL0
/* get kernel stack */                   /* ARM64_TODO Reserve x18 until we decide what to do with it */
movq  TH_KERNEL_STACK(%rdx),%rdx        mov  x18, x3    // ... and trash reserved x18
lea   -IKS_SIZE(%rdx),%rcx              // A12+ code validates JOP hash here or panics //
add   EXT(kernel_stack_size)(%rip),%rcx
/* set current stack */
movq  %rdx,%gs:CPU_ACTIVE_STACK
/* set stack top */
movq  %rcx,%gs:CPU_KERNEL_STACK         ldr         x3, [x2, TH_KSTACKPTR]

/* switch stacks, restore regs */       //  load_general_registers  x3, x4 (expanded) //
movq  KSS_RSP(%rcx),%rsp                ldp    x16, x17, [x3, SS64_X16]
movq  KSS_RBX(%rcx),%rbx                ldp    x19, x20, [x3, SS64_X19]
movq  KSS_RBP(%rcx),%rbp                ldp    x21, x22, [x3, SS64_X21]
movq  KSS_R12(%rcx),%r12                ldp    x23, x24, [x3, SS64_X23]
movq  KSS_R13(%rcx),%r13                ldp    x25, x26, [x3, SS64_X25]
movq  KSS_R14(%rcx),%r14                ldp    x27, x28, [x3, SS64_X27]
movq  KSS_R15(%rcx),%r15                ldp    fp,  lr,  [x3, SS64_FP]
                                        ldr    x4, [x3, SS64_SP]
                                        mov    sp, x4

                                        ldr    d8,  [x3, NS64_D8]
                                        ldr    d9,  [x3, NS64_D9]
                                        ldr    d10, [x3, NS64_D10]
                                        ldr    d11, [x3, NS64_D11]
                                        ldr    d12, [x3, NS64_D12]
                                        ldr    d13, [x3, NS64_D13]
                                        ldr    d14, [x3, NS64_D14]
                                        ldr    d15, [x3, NS64_D15]
/* return old thread */                 // A12+ code restores AP[ID]B keys for diff. processes //
jmp   *KSS_RIP(%rcx)                    ret   // old thread still in x0 //
```

* - The header comment for x86_64's `Switch_context` incorrectly flips between `%rdi` and `%rsi`

Accessing user mode memory

User-mode must never be allowed to access kernel mode memory, but not only is the opposite possible, it is fairly a common occurrence. The kernel often operates on behalf of user mode callers (when performing system calls and traps), so it is inevitable input need to be obtained from user-mode memory (e.g. the buffer supplied for `write(2)`), or output be written to it (e.g. in `read(2)`).

There are two considerations, however. First, it is not guaranteed that user-space memory will be paged in at that point. Unlike kernel memory, which is normally wired (resident), user-space memory may be swapped. If that is the case, access will trigger a page fault. A fault, however, is recoverable - so kernel code can explicitly brace for it, and recover. This is handled by machine level `bcopy` variants, which set up recovery handlers from the fault, so that it can be worked around.

Access to user-mode memory, therefore, is always performed through a single function - `copyio()` - with an enum specifying copy type (WORD, STRing) and direction (IN or OUT). The values are slightly different in between ARM and Intel architectures, but thin `copyin*` and a `copyout()` wrapper standardize the copy operations for use elsewhere in the kernel. Once user mode memory access is possible, the copy operation proceeds using a `bcopy()` variant, implemented in machine assembly to set the page fault recovery. As Figure 4-29 (on the next page) shows, the process differs between the Intel and ARM architectures.

Intel's page fault causes a jump to the Interrupt Dispatch Table's General Protection Fault entry. Code anticipating a fault - such as that of the `bcopy()` variants - sets entries in the system fault recovery table. This is done at compile time using `RECOVERY` macros, which embed the faulting address and the recovery address in the table. The table is set out in a separate section of the kernel - `__VECTORS.__recovery`. The IDT handler follows the process illustrated earlier (in @TODO), to eventually end up in `kernel_trap()`. This function first tries to service the page fault (by calling on `vm_fault()`, whose operation is described in Chapter 11). If the fault can be serviced, execution can resume normally, and the faulting instruction can be retried. If the paging fails, however, the code falls through to general protection fault handling. This code scans the recovery table for the faulting address, and sets the instruction pointer to the recovery address - ensuring a graceful exit.

ARM[64]'s `bcopy*()` also uses macros - `[SET/CLEAR]_RECOVERY_HANDLER`. These, however, do not set up a table, but instead use registers. The page fault similarly cause a jump - albeit to different locations. On ARM32, this is to the data prefetch abort handler. On ARM64, this is a synchronous error, which (eventually) gets to `sleh_synchronous` (as discussed earlier in this chapter), with ESR indicating a data abort at EL0 or EL1 (that is, `ESR_EC_DABORT_EL[0/1]`), which will result in `sleh_synchronous()` calling `handle_[user/kernel]_abort()` (respectively).

The `SET_RECOVERY_HANDLER` macro uses the X10, X11, and X3: X10 reads the thread pointer from `TPIDR_EL1`, X11 reads the previous recovery handler, and X3 is used to hold the recovery handler address as it is written to the recovery handler address. X10 and X11 continue to hold these values until the call to `CLEAR_RECOVERY_HANDLER`, which stores the saved value (in X11) to the thread structure (in X10).

Thus, when `sleh_synchronous()` executes, it will call `handle_user_abort`. This first tries to perform the recovery handler code, which in this case unsets the handler and returns an `-EFAULT`. The page fault handling code is using the same `vm_fault()`, which is hardware independent at this level. Once again, if the fault is serviced, normal execution resumes. For kernel faults, an attempt is first made to resolve the fault, and then the recover handler is called.

Memory Access Protections

A second consideration is that underlying architecture may (or may not) enforce access protections. Intel architectures define **Secure Mode Access Prevention** (SMAP), and ARM (v8.1 and later) architectures similary have **Privileged Access Never** (PAN). When these mechanisms are enforced, access to user space (Intel: Ring 3, ARM64: EL0) memory from kernel mode triggers a fault.

Intel's SMAP support is detected by a simple test of the `cpuid_leaf7_features()` (which are also exported through the `machdep.cpu.leaf7_features` sysctl(8) MIB). The test is a bitwise AND with `CPUID_LEAF7_FEATURE_SMAP` (bit #20), made in `pmap_cpu_init()` (in osfmk/x86_64/pmap.c), and cached in `pmap_smap_enabled`. It is enforced through a setting of CR4's `CR4_SMAP` bit (0x00200000). SMAP uses two dedicated instructions - `stac` and `clac` - which toggle the AC bit of the EFLAGS register, and those are abstracted by `user_access_[en/dis]able` (in osfmk/x86_64/copyio.c). Code in osfmk/i386/trap.c's `kernel_trap()` explicitly checks for these, jumping to `debugger_entry`, which starts KDP (if `MACH_KDP`), or just fires `panic_trap()`.

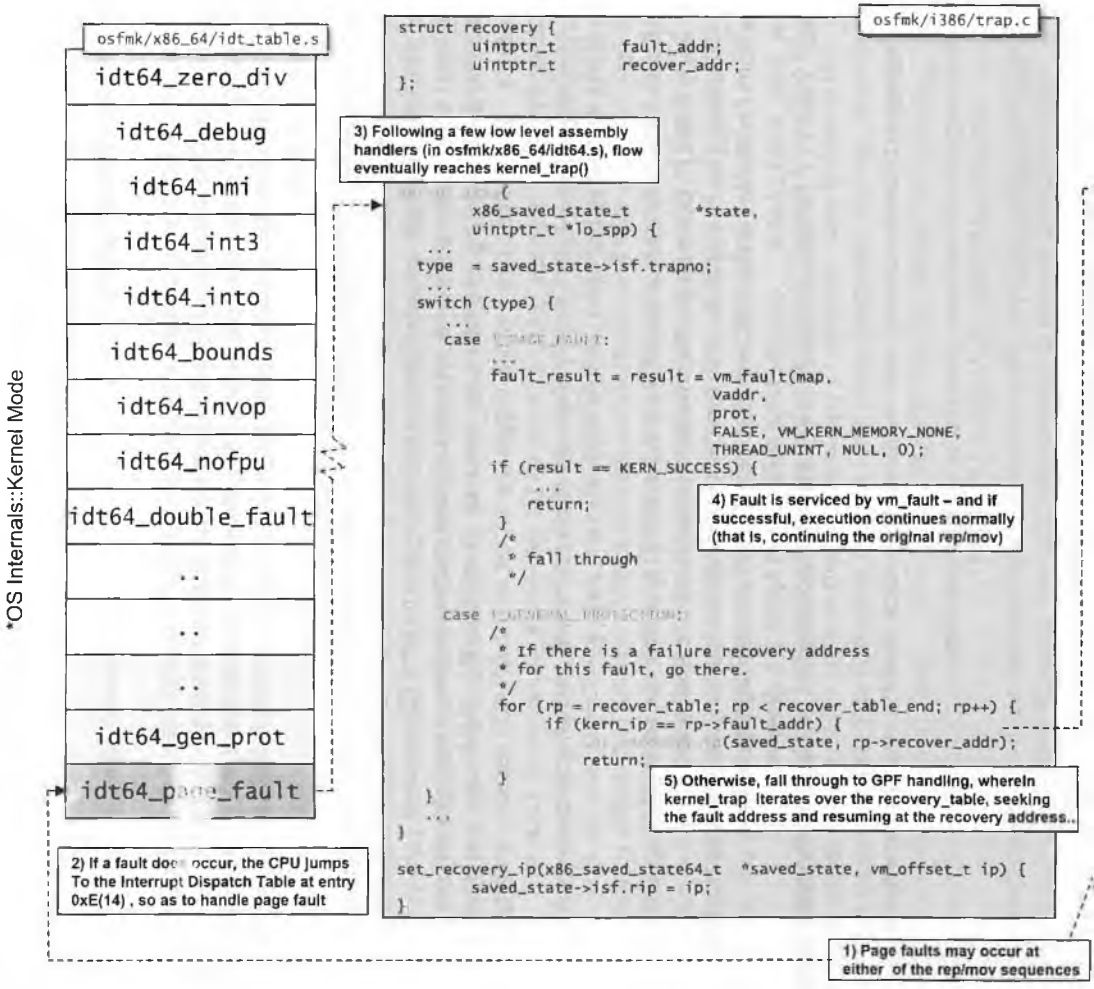
ARMv8.1 similarly adds a the PAN bit in the PSTATE, which can be toggled accordingly using "MSR PAN, #[0/1]". Once again, the low level instructions are abstracted by `user_access_[en/dis]able`, this time in ./osfmk/arm64/copyio.c. In this case the PAN instructions (available on the A10 and later processors) are contingent on the `__ARM_PAN_AVAILABLE__` compile time define. PAN faults are determined by `is_pan_fault()` (in osfmk/arm64/sleh.c), and - if encountered - cause an unforgiving kernel panic.

Listing 4-28: `user_access_[enable/disable]()` calls in ARM64's `copyio()` implementation

```
static int
copyio(copyio_type_t copytype, const char *src, char *dst,
           vm_size_t nbytes, vm_size_t *lencopied)
{
        ...
        user_access_enable();

        /* Select copy routines based on direction:
         *   COPYIO_IN - Use unprivileged loads to read from user address
         *   COPYIO_OUT - Use unprivleged stores to write to user address
         */

        switch (copytype) {
        case COPYIO_IN:
                result = _bcopyin(src, dst, nbytes);
                break;
        case COPYIO_INSTR:
                result = _bcopyinstr(src, dst, nbytes, &bytes_copied);
                if (result != EFAULT) {
                        *lencopied = bytes_copied;
                }
                break;
        case COPYIO_IN_WORD:
                result = _copyin_word(src, (uint64_t *)(uintptr_t)dst, nbytes);
                break;
        case COPYIO_OUT:
                result = _bcopyout(src, dst, nbytes);
                break;
        default:
                result = EINVAL;
        }

        user_access_disable();
        return result;
```

There are similar protections on executing memory: Intel's **Secure Mode Execution Prevention** (SMEP) and ARM's `Priveleged eXecute Never` (PXN) operate similarly to SMAP/PAN, but error handling in those cases is much less forgiving, and ends up in a `panic()`.

Figure 4-29-a: Recovering from a page fault on the x86 architecture



XNU's 4903.201.2~1
__VECTOR.__recover section

```
_recover_table:
0xffffff8000e95000: 0xffffff8000358d75    fault in _rdmsr_carefully
0xffffff8000e95008: 0xffffff8000358d7e    _rdmsr_fail
0xffffff8000e95010: 0xffffff8000358d92    fault in _rdmsr64_carefully
0xffffff8000e95018: 0xffffff8000358d9c    _rdmsr64_carefully_fail
0xffffff8000e95020: 0xffffff8000358dba    fault in _wrmsr_carefully
0xffffff8000e95028: 0xffffff8000358dbf    _wrmsr_fail

0xffffff8000e95030: 0xffffff8000358deb    fault 1 in _bcopy (movsb)
0xffffff8000e95038: 0xffffff8000358df8    _bcopy_fail
0xffffff8000e95040: 0xffffff8000358df3    fault 2 in _bcopy (movsq)
0xffffff8000e95048: 0xffffff8000358df8    _bcopy_fail
...
```

osfmk/x86_64/locore.s

```
#define RECOVERY_SECTION(addr)          \
        .align  3;                      \
        .quad   9f                      ;\
        .quad   addr                    ;\
        .text                           ;\
9:
...

/*
 * Copyin/out from user/kernel address space.
 * rdi: source address
 * rsi: destination address
 * rdx: byte count (in fact, always < 64MB -- see copyio) */
Entry(_bcopy)
        xchg    %rdi, %rsi              /* source %rsi, dest %rdi */
        cld                             /* count up */
        mov     %rdx, %rcx              /* move by longwords first */
        shr     $3, %rcx
        RECOVERY_SECTION
        RECOVERY_LABEL(_bcopy_fail)

        rep
        movsq                           /* move longwords */

        movl    %edx, %ecx              /* now move remaining bytes */
        andl    $7, %ecx
        RECOVERY_SECTION
        RECOVERY_LABEL(_bcopy_fail)

        rep
        movsb

        xorl    %eax,%eax               /* return 0 for success */
        ret                             /* and return */

_bcopy_fail:
        movl    $(EFAULT),%eax          /* return error for failure */
        ret
```

0) RECOVERY macros insert the recovery address and fault address pairs into the recovery table (at compile time)

6) .. And on recovery, execution resumes with graceful failure

Figure 4-29-b: Recovering from a page fault on the arm64 architecture

osfmk/arm64/machine_routines_asm.s

```
/*
 * SET_RECOVERY_HANDLER
 *
 *      Sets up a page fault recovery handler
 *
 *      arg0 - persisted thread pointer
 *      arg1 - persisted recovery handler
 *      arg2 - scratch reg
 *      arg3 - recovery label
 */
.macro SET_RECOVERY_HANDLER
        mrs             $0, TPIDR_EL1           // Load thread pointer
        ldr             $1, [$0, TH_RECOVER] // Save previous recovery handler
        adrp    $2, $3@page                     // Load the recovery handler address
        add             $2, $2, $3@pageoff
        str             $2, [$0, TH_RECOVER] // Set new recovery handler
.endmacro
..
/*
 * int _bcopyin(const char *src, char *dst, vm_size_t len)
 */
        .text
        .align 2
        .globl EXT(_bcopyin)
LEXT(_bcopyin)
        ARM64_STACK_PROLOG
        PUSH_FRAME
        SET_RECOVERY_HANDLER x10, x11, x3, copyio_error
        /* If len is less than 16 bytes, just do a bytewise copy */
        cmp             x2, #16
        b.lt    2f
        sub             x2, x2, #16
1:
        /* 16 bytes at a time */
        ldp             x3, x4, [x0], #16
        stp             x3, x4, [x1], #16
        subs    x2, x2, #16
        b.ge    1b
        /* Fixup the len and test for completion */
        adds    x2, x2, #16
        b.eq    3f
2:      /* Bytewise */
        subs    x2, x2, #1
        ldrb    w3, [x0], #1
        strb    w3, [x1], #1
        b.hi    2b
3:
        CLEAR_RECOVERY_HANDLER x10, x11 //  str  x11, [x10, TH_RECOVER]
        mov             x0, #0
        POP_FRAME
        ARM64_STACK_EPILOG
```

0) SET_RECOVERY_HANDLER macro sets address and retrieves thread pointer and any existing handler

1) Page faults may occur at either of the ld[rb|p]/st[rb|p]

7) .. And on successful exit, handler is cleared

2) If a page fault does occur, sleh_synchronous checks the exception code. Which will be ESR_EC_IABORT_ELO

osfmk/arm64/sleh.c

```
void sleh_synchronous(arm_context_t *context, uint32_t esr, vm_offset_t far)
{
 esr_exception_class_t   class = ESR_EC(esr);
 ..
 switch (class)
 case ESR_EC_IABORT_ELO:
      handle_abort(state, esr, far, recover, inspect_instruction_abort, handle_user_abort);
      assert(0); /* Unreachable */
```

```
handle_user_abort(arm_saved_state_t *state, uint32_t esr, vm_offset_t fault_addr,
                  fault_status_t fault_code, vm_prot_t fault_type, vm_offset_t recover)
{
 if (is_vm_fault(fault_code)) {
     kern_return_t   result = KERN_FAILURE;
     vm_map_t        map = thread->map;
     vm_offset_t     vm_fault_addr = fault_addr;

     assert(map != kernel_map);

     if (!(fault_type & VM_PROT_EXECUTE) && user_tbi_enabled())
                     vm_fault_addr = tbi_clear(fault_addr);
..
     (void)recover;

 /* check to see if it is just a pmap ref/modify fault */

 if ((result != KERN_SUCCESS) && !is_translation_fault(fault_code)) {
          result = arm_fast_fault(map->pmap, trunc_page(vm_fault_addr), fault_type, TRUE);
      }
 if (result != KERN_SUCCESS) {
        {
                /* we have to fault the page in */
                result = vm_fault(map, vm_fault_addr, fault_type,
              /* change_wiring */ FALSE, VM_KERN_MEMORY_NONE, THREAD_ABORTSAFE,
                /* caller_pmap */ NULL, /* caller_pmap_addr */ 0);
                }
        }
  if (result == KERN_SUCCESS || result == KERN_ABORTED) {
          thread_exception_return();
          /* NOTREACHED */
   }/
```

4) Handler clears itself, returns EFAULT

```
copyio_error:
 CLEAR_RECOVERY_HANDLER x10, x11
 mov     x0, #EFAULT  // Return an EFAULT error
 POP_FRAME
 ARM64_STACK_EPILOG
```

3) Execution goes to recovery handler

5) arm_fast_fault() handles software induced faults, possibly reenabling mappings

6) Otherwise, page explicitly gets faulted in, so at this point accessing it would be OK

Interrupt Handling

Of the various mode traversals seen so far, interrupts are a special case. They are unpredictable, all too frequent, and their handling is critical to dealing with hardware devices and keeping the scheduler ticking. Interrupts may occur virtually any time (unless explictly disabled or masked), and their handling must be performed transparently, so as not to interfere with the thread state their handler effectively takes over.

Different architectures use different controller architectures. x86_64 uses the Advanced Programmer Interrupt Controller (APIC), which (as of Nehalem) is known as **x2APIC**. MacOS supports either, through two `lapic_ops_table_t` structures in osfmk/i386/lapic_native.c. During startup, `lapic_init()` reads the APIC state, chooses the correct structure (on Core i5/i7, the `x2apic_ops`), and calls the `init` function. Slave processors (secondary cores) also reinitialize their interrupt controller if `do_init_slave` is called in slow restart.

ARM recognizes two types of interrupts - the regular interrupt requests (IRQ) and "fast interrupts requests" (FIQ). The former type is generated by external devices, and therefore requires a corresponding driver's handler to be called. The latter is used by the system real time clock, and (contingent on `#if MONOTONIC`) for the performance monitor.

Enabling/disabling interrupts

Interrupts may arrive at inopportune times, in which the kernel is not to be disturbed. There is therefore a need to enable and disable interrupts - which is exactly the purpose of the `ml_set_interrupts_enabled` function. The function accepts a boolean `enable`, and proceeds accordingly:

- The x86_64 architecture (from osfmk/i386/machine_routines.c) reads the interrupt state from `EFLAGS`. It uses the `sti/cli` special instructions to enable/disable interrupts.
- The ARM architectures (from osfmk/arm/machine_routines_common.c) read the interrupt states using `DAIF` (or, in ARM32, the `CPSR`). The `DAIFClr/DAIFSet` enable/disable interrupts, either standard (IRQ, #2), fast (FIQ, #1), or both (#3).

BSD systems have traditionally controlled interrupts through priority levels, with `spl*` routines used to set priority levels to one of several predefined. XNU's BSD layer recognizes 5 such routines, but in practice they are all macros in osfmk/kern/spl.h, which are `#defined` over `ml_set_interrupts_enabled`. `splhigh()`, `splsched()`, `splclock()` all disable interrupts, `spllo()` enables them, and `splx()` takes a boolean argument to either disable or enable. The macros, therefore, return the previous interrupt state as a boolean (because that's what `ml_set_interrupts_enabled()` does), and it is common practice to use `splx` to restore the previous state when interrupts do not need to be disabled anymore.

Re-enabling interrupts in both architectures triggers a check for urgent Asynchronous Software Traps, which (as explained later, in Chapter 10) are deferred actions which must be performed as soon as possible.

Machine Level handling of interrupts

When an interrupt occurs, the core which receives it saves the current program state, and immediately executes the designated low level handler, which was set by the kernel during initialization. In x86_64, these low level handlers are set in the Interrupt Descriptor Table (IDT), that is the 256 entry vector used for interrupts as well as exceptions and traps, as discussed previously. With the first 32 entries reserved for traps and exceptions, XNU sets the common interrupt handler routine - `hndl_allintrs` (from osfmk/x86_64/idt64.s) for all other entries (except 0x7F-0x82, which are reserved for DTrace and legacy 32-bit system calls).

In ARM architectures, interrupts are treated like other exceptions, in that the core immediately jumps to the corresponding exception vector slot. As discussed earlier, the exception vectors funnel to the `fleh_*` and `sleh_*` handlers, in these cases `[fleh/sleh]_[irq/fiq]`.

Darwin redirects all interrupts to the main core (CPU0), save for Timer and Inter-Processor-Interrupts (IPI). Their vector entries can be found (on MacOS) through the `machdep.vectors.timer` and `machdep.vectors.IPI` `sysctl(2)` MIBs.

The little known `powermetrics(1)` utility can be used (among other things) to display which IRQ lines on the system are in use. Individual interrupt firings can be detected using the power `kdebug` facility.

Output 4-30: Using the `powermetrics(1)` command to show interrupts statistics

```
morpheus@Chimera(~)$ sudo powermetrics --samplers interrupts
***

***  Interrupt distribution ***

CPU 0:
        Vector 0x46(SMC): 3.36 interrupts/sec
        Vector 0x72(IGPU): 80.75 interrupts/sec     # Graphics
        Vector 0x73(XHC1): 0.79 interrupts/sec      # USB
        Vector 0x79(XHC2): 0.40 interrupts/sec      # USB
        Vector 0x7c(ARPT): 6.73 interrupts/sec      # WiFi
        Vector 0x8c(IOBC): 13.46 interrupts/sec
        Vector 0xdd(TMR): 695.26 interrupts/sec
        Vector 0xde(IPI): 64.52 interrupts/sec
CPU 1:  # CPU 1+ only get TMR and IPI
        Vector 0xdd(TMR): 30.28 interrupts/sec
        Vector 0xde(IPI): 128.44 interrupts/sec
CPU 2:
        Vector 0xdd(TMR): 199.10 interrupts/sec
        Vector 0xde(IPI): 108.85 interrupts/sec
CPU 3:
        Vector 0xdd(TMR): 18.80 interrupts/sec
        Vector 0xde(IPI): 55.22 interrupts/sec
```

XNU's Handling of Interrupts

In all architectures, XNU pivots to a dedicated interrupt stack, by loading its address (from the per-CPU data structure) into the stack pointer. Although the mechanics of interrupts are different across architectures, XNU's interrupt handling logic is largely the same:

- Update interrupt counts in the per-cpu data structure.
- Emit a KDebug code for interrupt entry (`..._MACH_EXCP_INTR| ..._FUNC_START`, 0x01050001) with the saved PC value (slid, if from kernel mode)
- Optionally mark the current thread for telemetry (`#if CONFIG_TELEMETRY`, which is set by default)
- Service the interrupt: in an architecture specific, and driver dependent manner
- Gather entropy: Interrupts are generically unpredictable, and therefore their timestamps of their occurrence can make for a good entropy source, for /dev/random and other uses.
- Emit a KDebug code for interrupt exit (`..._MACH_EXCP_INTR | ..._FUNC_END`, 0x01050002), with no further arguments. The two KDebug codes are what enables utilities such as `latency(1)` to deduce time spent in interrupt handlers.

The `ml_*` layer provides abstraction over the low level interrupt handling. The `ml_[get/set]_interrupt_enabled` and `_ml_at_interrupt_context` routines (part of the libkern KPI) provide a platform agnostic way of setting and determining interrupt state. Listing 4-31 shows the implementations across the different architectures. Intel's (from osfmk/i386/cpu_data.h) gets the `cpu_interrupt_level` from the local CPU data (in %gs), whereas ARM's (from osfmk/arm/machine_routines_common.c) checks `DAIF` (for `ml_get_interrupts_enabled()` and then checks the `cpu_int_state` from local CPU data.

Listing 4-31: Comparing `ml_at_interrupt_context` between Intel (left) and *OS (right)

```
_ml_at_interrupt_context:
// CPU_DATA_GET(cpu_int_level,int)
 pushq   %rbp
 movq    %rsp, %rbp
 xorl    %eax, %eax
 cmpl    $0x0, %gs:0x40
 setne   %al
 popq    %rbp
 retq
```

```
// ml_get_interrupts_enabled()
MRS         X8, DAIF
TBNZ        W8, #7, ints_enabled
MOVZ        W0, 0x0
RET
ints_enabled: // getCpuDatap()->cpu_int_state
MRS         X8, TPIDR_EL1
LDR         X8, [X8, #1152]
LDR         X8, [X8, #256]
CMP         X8, #0
CSINC       W0, W31, W31, EQ
RET
```

Intel

In MacOS (Intel) all interrupt requests are handled by a single handler - `interrupt()`, from osfmk/i386/trap.c - which (as discussed earlier) is invoked from the "true" interrupt entry point, `hndl_allintrs`, from osfmk/x86_64/idt64.s. The handler differentiates between two types of interrupts:

- LAPIC interrupts, which it handles internally (with `lapic_interrupt()`, from osfmk/i386/lapic_native.c). The local interrupt dispatch table is maintained in `lapic_intr_func`, a static (unexported) table (also in osfmk/i386/lapic_native.c). There is a `lapic_set_intr_func()` exported symbol, but it is used internally (primarily in `i386_smp_init()`, to set the NMI and IPI handlers). The symbol is wrapped by `lapic_set_pm_func()`, which AppleIntelCPUPowerManagement.kext calls (through `pmKextRegister()`) to set the power management handler.
- Device interrupts, deferred to the Platform Expert (`PE_incoming_interrupt()`, from pexpert/i386/pe_interrupt.c). This calls the `handler` function pointer in the `PE_interrupt_handler` structure (ibid). The pointer may be set by a call to `PE_install_interrupt_handler()`. `ml_install_interrupt_handler()` (from osfmk/i386/machine_routines.c) wraps this function, and it is IOKit's `IOCPUInterruptController`'s `::enableCPUInterrupt()` method which registers its `::handler` method. The common `::handler` multiplexes IRQs through the use of a private `vectors` array. IOKit drivers have indirect access to this array through `IOCPUInterruptController::registerInterrupt()`. There is a detailed discussion of IOKit interrupt handling in Chapter 13.

After servicing the interrupt, `ml_entropy_collect()` extract entropy bits from the timestamp, mixing them into the `EntropyData` buffer in osfmk/prng/prng_random.c.

ARM

ARM architectures recognize two types of interrupts - a regular interrupt request (IRQ) and a fast interrupt request (FIQ), during which all other interrupts are masked. ARM64 allows to differentiate further, based on whether the interrupt occurred at the same or different EL. As with other exceptions, XNU's ARM flavor defines two levels of interrupt handling: The First Level Exception Handlers (`fleh_[irq/fiq]`, in osfmk/arm[64]/locore.s) which are virtually identical and handle the interrupt statistics and KDebug, and the Second Level Exception Handlers (`sleh_[irq/fiq]`, in osfmk/arm64/sleh.c) which service the interrupt from the register handler. Here, `ml_install_interrupt_handler()` (from osfmk/arm/machine_routines.c registers the handler directly in the per-CPU data, bypassing the platform expert. XNU maintains a preemption level, incrementing in the FLEH prolog code (`BEGIN_INTERRUPT_HANDLER`) and decrementing in its epilog (`END_INTERRUPT_HANDLER`), and asserting no underflow (else resulting in a panic).

The FIQ triggers `rtclock_intr(TRUE)`, serving as a timer tick, incrementic monotonic counts, and possibly performing telemetry. The IRQ is the interrupt used by devices, and so the second level `sleh_irq` calls the specific handler - which is again in the domain of IOKit. Here, too, entropy is collected, albeit directly, rather than calling `ml_collect_entropy()`, as was in the Intel case.

System call personalities

XNU is unique in that, once in kernel mode, it exposes not one but up to four personalities. As explained in Volume I, the choice of personality is made by modifying the call or trap number: Either flagging it with high order bits (x86_64) or using a negative number for traps (elsewhere).

The BSD Personality

XNU's BSD layer syscalls are generated in a similar manners to those of the original BSD. The bsd/kern/syscalls.master file, a direct rip from BSD, defines the system call numbers, names and prototypes. The master file is processed by a shell script, makesyscalls.sh (in the same directory), and produces the headers and - most importantly - the system call table, `sysent`.

Listing 4-32: The syscalls.master from XNU 4903

```
;       derived from: FreeBSD @(#)syscalls.master       8.2 (Berkeley) 1/13/94
;
; System call name/number master file.
; This is file processed by .../xnu/bsd/kern/makesyscalls.sh and creates:
;       .../xnu/bsd/kern/init_sysent.c
;       .../xnu/bsd/kern/syscalls.c
;       .../xnu/bsd/sys/syscall.h
;       .../xnu/bsd/sys/sysproto.h
;       .../xnu/bsd/security/audit_syscalls.c

; Columns -> | Number Audit Files | { Name and Args } | { Comments }
;       Number:         system call number, must be in order
;       Audit:          the audit event associated with the system call
;                       A value of AUE_NULL means no auditing, but it also means that
;                       there is no audit event for the call at this time. For the
;                       case where the event exists, but we don't want auditing, the
;                       event should be #defined to AUE_NULL in audit_kevents.h.
;       Files:          with files to generate - "ALL" or any combo of:
;                                       "T" for syscall table (in init_sysent.c)
;                                       "N" for syscall names (in syscalls.c)
;                                       "H" for syscall headers (in syscall.h)
;                                       "P" for syscall prototypes (in sysproto.h)
;       Name and Args:  function prototype, optionally followed by
;                       NO_SYSCALL_STUB (which mean no system call stub will
;                       be generated in libSystem) and ending with a semicolon.
;                       (Note: functions prefixed by double-underbar are
;                       automatically given the NO_SYSCALL_STUB attribute.)
;       Comments:       additional comments about the sys call copied to output files

...

0    AUE_NULL  ALL { int nosys(void); }   { indirect syscall }
1    AUE_EXIT  ALL { void exit(int rval) NO_SYSCALL_STUB; }
2    AUE_FORK  ALL { int fork(void) NO_SYSCALL_STUB; }
...
// Apple chose to strip mach_bridge_remote_time from syscalls.master
531     AUE_NULL        ALL     { int enosys(void); }
```

There are quite a few "holes" with unused or retired system call numbers, and `nosys` or `enosys` are used in those cases. Both (in bsd/kern/subr_xxx.c) return `-ENOSYS` (-78) to user mode, and the former also sends a `SIGSYS` (#12). The `sysent` table itself is generated in two files: bsd/sys/sysent.h, which defines the table entry, and bsd/kern/init_sysent.c, which defines the table itself. Depending on 32-bit kernel support (`#if CONFIG_REQUIRES_U32_MUNGING`), the table may or may not be defined with an additional `sy_munge_t`, which is a pointer to a function meant to "munge" (process) the arguments of a 32-bit caller, and convert them to 64-bits. These are multiple functions named `munge_[w|l]*` (defined in bsd/dev[/arm]/munge.c , with "w" and "l" standing for word or long arguments. As of Darwin 17, in all *OS variants but WatchOS this is no longer an issue, due to the move to 64-bit only. With both WatchOS (in Series 4) and MacOS (as of 15) moving to 64-bit as well, munging will likely be entirely `#undefed` sometime in the near future.

Listing 4-33: The sysent definitions from XNU 4903

```
struct sysent {          /* system call table */
        sy_call_t       *sy_call;       /* implementing function */
#if CONFIG_REQUIRE      NGING || (__arm__ && (__BIGGEST_ALIGNMENT__ > 4))
        sy_munge_t      *s      munge32; /* system call arguments munger for 32-bit process */
#endif
        int32_t         sy_return_type; /* system call return types */
        int16_t         sy_narg;        /* number of args */
        uint16_t        sy_arg_bytes;   /* Total size of arguments in bytes for
                                         * 32-bit system calls
                                         */
};

__private_extern__ const struct sysent sysent[] = {
#if CONFIG_REQUIRES_U32_MUNGING || (__arm__ && (__BIGGEST_ALIGNMENT__ > 4))
        { (sy_call_t *)nosys, NULL, _SYSCALL_RET_INT_T, 0, 0}, /* 0 = nosys indirect syscall */
#else
        { (sy_call_t *)nosys, _SYSCALL_RET_INT_T, 0, 0},
#endif
#if CONFIG_REQUIRES_U32_MUNGING || (__arm__ && (__BIGGEST_ALIGNMENT__ > 4))
        { (sy_call_t *)exit, munge_w, _SYSCALL_RET_NONE, AC(exit_args), 4},     /* 1 = exit */
#else
        { (sy_call_t *)exit, _SYSCALL_RET_NONE, AC(exit_args), 4},
#endif
..
};
unsigned int    nsysent = sizeof(sysent) / sizeof(sysent[0]);
```

The distinct structure of `sysent` allows it to be trivially found across architectures, even though it is not exported. This is important in newer kernels, which contain tagged pointers:

Output 4-34: Analyzing and viewing the `sysent` of an iOS kernelcache

```
morpheus@Chimera (~) % jtool2 --analyze kernelcache.release.iphone11
Analyzing kernelcache...
This is Darwin Kernel Version 18.2.0: Tue Oct 16 21:02:38 PDT 2018; root:xnu-4903.222.5~1/RELEASE_ARM64_T8020
...
Analyzing __DATA_CONST...
...
- Found sysent (0xfffffff0077e81f0)
...
morpheus@Chimera (~) % jtool2 -d sysent kernelcache.release.iphone11 | head
opened companion file ./kernelcache.release.iphone11.6C3F6D35-4DC5-3B5D-8579-9D452C0F0901
Dumping 2191232 bytes from 0xfffffff0077e81f0 (Offset 0x7e41f0, __DATA_CONST.__const):
sysent:
0xfffffff0077e81f0: A8 A7 F2 00 00 00 30 80  _nosys
0xfffffff0077e81f8: 00 00 00 00 00 00 00 00  NULL
0xfffffff0077e8200: 01 00 00 00 00 00 00 00  _SYSCALL_RET_INT_T, 0 , 0

0xfffffff0077e8208: A0 5E ED 00 00 00 10 80  _exit
0xfffffff0077e8210: EC D5 BB 00 00 00 20 80  _munge_w
0xfffffff0077e8218: 00 00 00 00 01 00 04 00  _SYSCALL_RET_NONE, AC(exit_args), 4
..
```

All BSD level system calls share common handling, in `unix_syscall[64]` (from bsd/*arch*/dev/systemcalls.c), namely:

- **Auditing**: The call to the implementing function from `sysent` is preceded by `AUDIT_SYSCALL_ENTER` and followed by the `AUDIT_SYSCALL_EXIT` macros (q.v. III/Fig 2-10). This means that, irrespective of which system call, all generate an audit event. The individual system call implementations often add specific auditing of their arguments (using `AUDIT_ARG`). Auditing is described in detail in III/2.
- **KDebug:** All system calls (save for the `kdebug_*` variants themselves) emit corresponding KDebug codes. On entry, this is (0x040c0### + 1), with the first four arguments of the system call (even it if has less). On exit, this is (0x040c0### + 2), with the return code, `uu_rval` (spread over two 32-bit arguments) and the pid.
- **Arguments**: When servicing a system call, its number and arguments are stored on the BSD `uthread`'s `syscall_code` and `uu_arg[]` array.
- **Noreturn**: System calls don't return on exit from `unix_syscall()` - but by `thread_exception_return()`.

The Mach Personality

The largely undocumented Mach personality is the "true core" of XNU, offering a legacy and proprietary interfacy which Darwin is strongly dependent on*. Unlike the system call table, the Mach trap table is fixed at 128 entries. This has proven more than enough as presently only about half of those are actually in use, although Apple does add traps occasionally, and as recently as Darwin 19 three traps were added.The `mach_trap_table` table is defined in osfmk/kern/syscall_sw.c, contingent on munging, just as in the BSD case.

Listing 4-35: The mach_trap_table definitions (from XNU 4903's osfmk/kern/syscall_sw.c)

```
// From osfmk/kern/syscall_sw.h:
typedef struct {
  int             mach_trap_arg_count;    /* Number of trap arguments (Arch independent) */
  kern_return_t   (*mach_trap_function)(void *);
#if CONFIG_REQUIRES_U32_MUNGING || (__arm__ && (__BIGGEST_ALIGNMENT__ > 4))
   mach_munge_t *mach_trap_arg_munge32; /* system call argument munger routine for 32-bit */
#endif
   int            mach_trap_u32_words;    /* number of 32-bit words to copyin for U32 */
#if  MACH_ASSERT
   const char*    mach_trap_name;
#endif /* MACH_ASSERT */
} mach_trap_t;

#define MACH_TRAP_TABLE_COUNT   128
// From osfmk/kern/syscall_sw.c:
const mach_trap_t       mach_trap_table[MACH_TRAP_TABLE_COUNT] = {
/* 0 */         MACH_TRAP(kern_invalid, 0, 0, NULL),
..
/* 31 */        MACH_TRAP(mach_msg_trap, 7, 7, munge_wwwwwww),
/* 32 */        MACH_TRAP(mach_msg_overwrite_trap, 8, 8, munge_wwwwwwww),
..
/* 99 */        MACH_TRAP(kern_invalid, 0, 0, NULL),
/* traps 100-107 reserved for iokit (esb) */
/* 100 */       MACH_TRAP(iokit_user_client_trap, 8, 8, munge_wwwwwwww),
..
/* 126 */       MACH_TRAP(kern_invalid, 0, 0, NULL),
/* 127 */       MACH_TRAP(kern_invalid, 0, 0, NULL),
};
```

Similar to BSD syscalls in `sysent`, unimplemented Mach traps use the `kern_invalid` function (in osfmk/kern/syscall_sw.c), which returns `KERN_INVALID_ARGUMENT` (4), and may optionally drop to a debugger if `kern_invalid_debug` (ibid.) is set to 1. Due to the large number of `kern_invalid`s, the `mach_trap_table` has a structure as distinct as that of `sysent`, and can also be found in the kernelcache trivially:

Output 4-36: Viewing the `mach_trap_table` of an iOS kernelcache analyzed with `jtool2`

```
morpheus@Chimera (~) % jtool2 -d mach_trap_table kernelcache.release.iphone11
mach_trap_table:
0xfffffff0077bc9f0: 00 00 00 00 00 00 00 00    ........
0xfffffff0077bc9f8: 0xfffffff007a5d6e8         _kern_invalid
0xfffffff0077bca00: 00 00 00 00 00 00 00 00    ........
0xfffffff0077bca08: 00 00 00 00 00 00 00 00    ........
0xfffffff0077bca10: 00 00 00 00 00 00 00 00    ........
0xfffffff0077bca18: 0xfffffff007a5d6e8         _kern_invalid
...
0xfffffff0077bcb38: 0xfffffff007a17a08         __kernelrpc_mach_vm_allocate_trap
0xfffffff0077bcb40: 0xfffffff007b9f324         _munger(ignored)
0xfffffff0077bcb48: 05 00 00 00 00 00 00 00    ........
0xfffffff0077bcb50: 04 00 00 00 00 00 00 00    ........
0xfffffff0077bcb58: 0xfffffff007a17ef8         _kernelrpc_mach_vm_purgable_control_trap
0xfffffff0077bcb60: 0xfffffff007b9f3c4         _munger(ignored)
0xfffffff0077bcb68: 05 00 00 00 00 00 00 00    ........
...
0xfffffff0077bd9d0: 00 00 00 00 00 00 00 00    ........
0xfffffff0077bd9d8: 0xfffffff007a5d6e8         _kern_invalid
0xfffffff0077bd9e0: 00 00 00 00 00 00 00 00    ........
0xfffffff0077bd9e8: 00 00 00 00 00 00 00 00    ........
0xfffffff0077bd9f0: FF FF FF FF FF FF FF FF    ........
```

* - The first edition of this work foolishly dared to make some predictions as to the future of Darwin based operating systems. While some (e.g. the gradual convergence of the various Darwin flavors) were spot on, a notable "miss" was stipulating that Apple might want to move away from Mach. As time since has shown, this is an actual impossibility. Though technically doable, it would require such a massive rearchitecting of the kernel, that it simply won't happen unless Apple ends up adopting a brand new operating system.

Machine Specific Syscalls

As explained in I/2 (Listing I/2-27), machine specific syscalls - as their name implies - vary in flavor and implementation between architectures. On Intel, machine specific system calls are encoded by 0x30000000 and the syscall number, whereas in ARM the indicator is 0x80000000 (`PLATFORM_SYSCALL_TRAP_NO`) as the system call number (r12/X16), with an index in r3/X3. Both MacOS and *OS use this to support `set_cthread_ self` (although with differing indexes).

`platform_syscall (*OS)`

Machine specific system calls in *OS variants are handled by `platform_syscall()` (in osfmk/arm64/machine_routines.c). Four such syscalls are supported: #0 and #1 handle instruction and data cache flushes (respectively), whereas #2 and #3 handle `get/set_cthread_self()`. Inspecting the DYLD shared cache, however, it doesn't seem that any of those are actually in use, and libplatform.dylib's cache invalidation functions merely use `DSB ISH`.

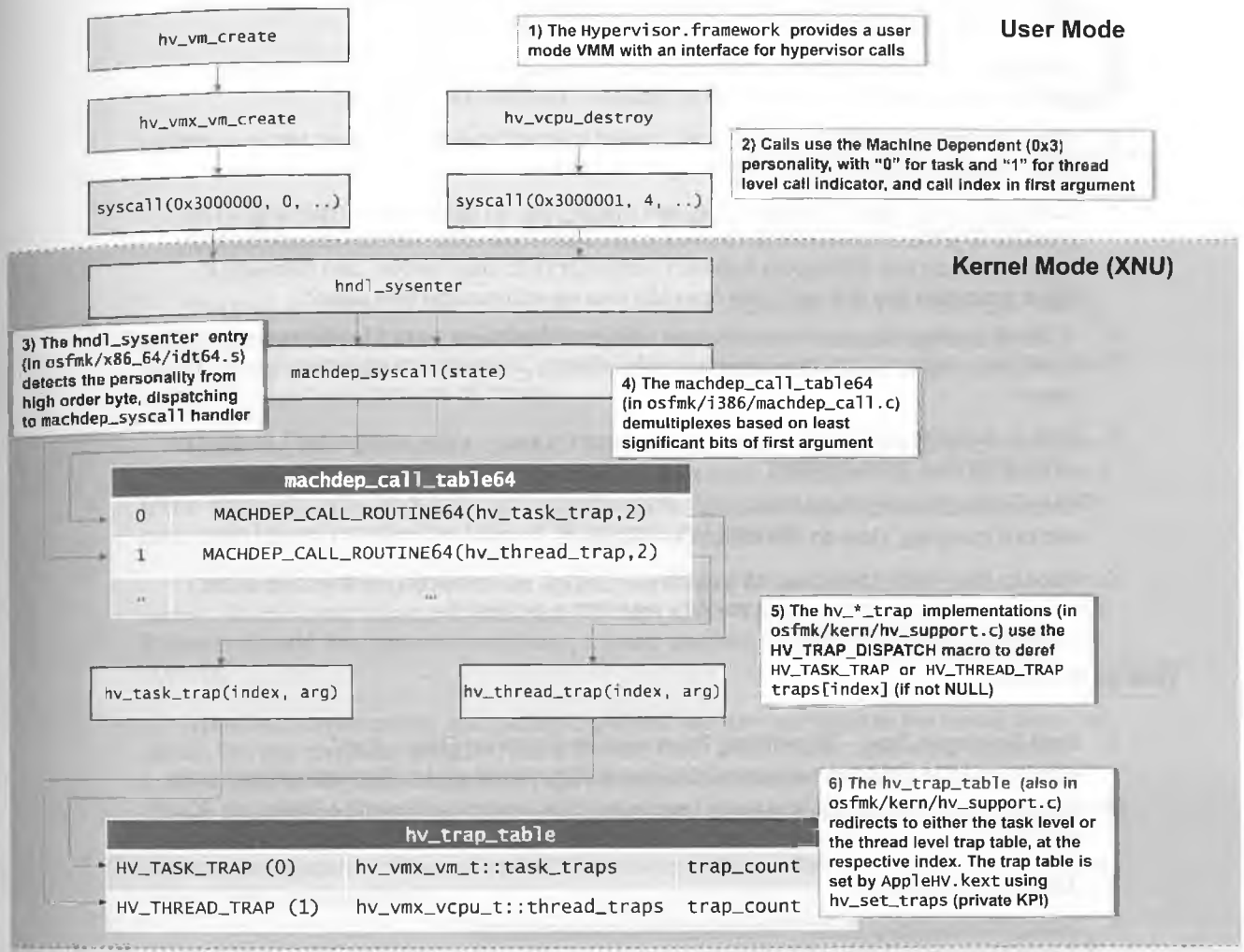
Hypervisor support (MacOS)

The main function of MacOS's machine dependent syscalls is kernel support for hypervisors (virtual machine monitors), as of 10.10. As discussed in I/2-28, the HyperVisor.framework handles the user-mode APIs, which in turn map to machine specific syscalls in the x86_64 kernel. Doing so allows implementing the hypervisor logic (e.g. VirtualBox, VMWare, etc.) in user mode, without the need for a kernel extension: The machine specific syscalls bridge to Intel virtualization extension instructions (Intel-VT), which are privileged and therefore can only be invoked from kernel mode.

MacOS XNU provides an interface for a kernel extension to provide the necessary kernel-side support, using `hv_set_traps` and `hv_set_callbacks`. These KPIs, however, are deemed private, and are only used by AppleHV.kext, which is the default (and presently, only) allowed Hypervisor kernel extension.

Apple provides ample documentation for the Hypervisor.framework[5]. Several examples of its usage can be found in hvdos[6] and similar open source projects, and some commercial products, (e.g. Parallels), provide their virutalization through the framework. Note, that as of 10.11 use of the framework APIs (specifically, though which open the kext's `AppleHV IOUserClient`) is subject to the `com.apple.vm.hypervisor` entitlement, enforced through /System/Library/Sandbox/Profiles/application.sb.

Figure 4-37: Illustrating the flow of a machine dependent hypervisor call on MacOS



Kernel Mode (AppleHV.kext)

```
hv_vmx_vm_t::task_traps:
0x72d0:  0x156e  hv_vmx_vm_t::TASK_TRAP_vm_create(hv_vmx_vm_t*, unsigned long long)
0x72d8:  0x1636  hv_vmx_vm_t::TASK_TRAP_vm_destroy(hv_vmx_vm_t*) z
0x72e0:  0x16fa  hv_vmx_vm_t::TASK_TRAP_vm_map(hv_vmx_vm_t*, hv_vm_map_item_t*)
0x72e8:  0x178a  hv_vmx_vm_t::TASK_TRAP_vm_unmap(hv_vmx_vm_t*, hv_vm_map_item_t*)
0x72f0:  0x1812  hv_vmx_vm_t::TASK_TRAP_vm_protect(hv_vmx_vm_t*, hv_vm_map_item_t*)
0x72f8:  0x189e  hv_vmx_vm_t::TASK_TRAP_vm_sync_tsc(hv_vmx_vm_t*, unsigned long long)
0x7300:  0x1918  hv_vmx_vm_t::TASK_TRAP_vcpu_create(hv_vmx_vm_t*, unsigned int*)
0x7308:  0x0a46  hv_vmx_vm_t::TASK_TRAP_vcpu_interrupt(hv_vmx_vm_t*)

hv_vmx_vcpu_t::thread_traps:
0x7ea0:   0x3174  hv_vmx_vcpu_t::THREAD_TRAP_vcpu_run(hv_vmx_vcpu_t*)
0x7ea8:   0x373a  hv_vmx_vcpu_t::THREAD_TRAP_vcpu_invalidate_tlb(hv_vmx_vcpu_t*)
0x7eb0:   0x376c  hv_vmx_vcpu_t::THREAD_TRAP_vcpu_enable_native_msr(hv_vmx_vcpu_t*, unsigned int)
0x7eb8:   0x3788  hv_vmx_vcpu_t::THREAD_TRAP_vcpu_disable_native_msr(hv_vmx_vcpu_t*, unsigned int)
0x7ec0:   0x37a0  hv_vmx_vcpu_t::THREAD_TRAP_vcpu_destroy(hv_vmx_vcpu_t*)
0x7ec8:   0x38e0  hv_vmx_vcpu_t::THREAD_TRAP_vmx_vcpu_read_vmcs(hv_vmx_vcpu_t*, unsigned int)
0x7ed0:   0x38e9  hv_vmx_vcpu_t::THREAD_TRAP_vmx_vcpu_set_apic_address(hv_vmx_vcpu_t*, unsigned long long)
```

7) AppleHV.kext's callbacks can then call on privileged Intel VMX instructions to provide hardware assisted virtualization functions

Review Questions

1. What is the substantial risk which is incurred in systems without hardware SMAP/SMEP and or PAN/PXN? Why is execution prevention alone insufficient?
2. Although machine-dependent, the implementation of the `hw_lock_*` KPIs is in the generic osfmk/kern/locks.c. How is this apparent contradiction resolved?
3. Why is it important to use `splx()` on a saved interrupt state, rather than just re-enabling interrupts when done with the other `spl*` variants?
4. Looking through older sources of XNU will reveal that, up until around XNU 4570.21.40 (MacOS 13), the Intel architecture's mode traversal code in the kernel was substantially lighter. Analyze the differences between version prior to that version, and following it. What prompted the change? How does the change address the new need?
5. A similar change occurred in the ARM64 branch of the kernel, which is well documented in osfmk/arm64/proc_reg.h. How does the `ARM_KERNEL_PROTECT` feature help mitigate the issue?
6. Why is it a good idea to leave `sysent` and `mach_trap_table` unexported? In practice, what difference does it make?
7. Write expressions to match the `sysent` and `mach_trap_table`, both for with and without munging. How do the structures change?
8. Prior to XNU 4903, there was no explicit `mov x18, x3` instruction in the code of the `set_thread_registers` macro. Why was that a problem?

References

1. Intel Developer Zone - "Benefitting Power and Performance Sleep Loops" - https://software.intel.com/en-us/articles/benefitting-power-and-performance-sleep-loops
2. ARM Architecture Reference Manual - "B2.9.6 - Use of WFE and SEV instructions by spin-locks"
3. AARCH64 specification
4. https://developer.apple.com/library/archive/documentation/Xcode/Conceptual/iPhoneOSABIReference/
5. Apple Developer - "Hypervisor" - https://developer.apple.com/documentation/hypervisor
6. PageTable.com - "Using the OS X 10.10 Hypervisor.framework" - https://www.pagetable.com/?p=764

5

Alone in the Dark: The Boot Process

The boot processes of MacOS and the *OS variants, however, are quite different. MacOS uses the **Extensible Firmware Interface** (EFI), so we first turn our attention to it. EFI provides a full operating environment, capable of supporting files, and binaries - and the MacOS boot loader, `boot.efi` is but one of those.

Unlike the Intel architecture, in which Apple had to conform to standards, *OS variants (wherein Apple has full control) use a proprietary boot chain, collectively known as iBoot. As a home-grown solution, iBoot has been designed from the ground up for security, and the *OS boot chain indeed provides the basis of all integrity and security.

Contemporary Macs, which utilize Apple's proprietary T2 (or later) chips, synergize both worlds. iBoot is used to securely boot the T2 chip (which runs BridgeOS, essentially a reduced iOS variant), and then provide a verifiable, trusted environment for "legacy" EFI to continue booting.

With Boot environments and loaders explained, we turn our focus to the kernel boot. The kernel, like any computer program, has a defined entry point. Unlike user mode programs, however, the kernel boot process is a highly complex, architecture dependent process. Both MacOS and ARM64 flows are explored and juxtaposed when possible, until they converge in the (relatively) architecture independent portion that is `kernel_bootstrap()`. As more cores initialize, once again architecture-specific constraints and considerations apply, as is discussed under "SMP Considerations".

> The kernel boot process is remarkably long and complex. It also changes regularly between Darwin releases as more subsystems are added and others are removed. Nonetheless, it is detailed in full. A concerted effort has been made to illustrate the flows as much as possible, but some unavoidable (and lengthy) detail must be presented in list form.

Even after boot is complete, the kernel still maintains numerous threads. The key ones are detailed along with their creation points. Finally, we conclude our chapter's tasks as does the kernel itself - upon system shutdown.

MacOS: EFI

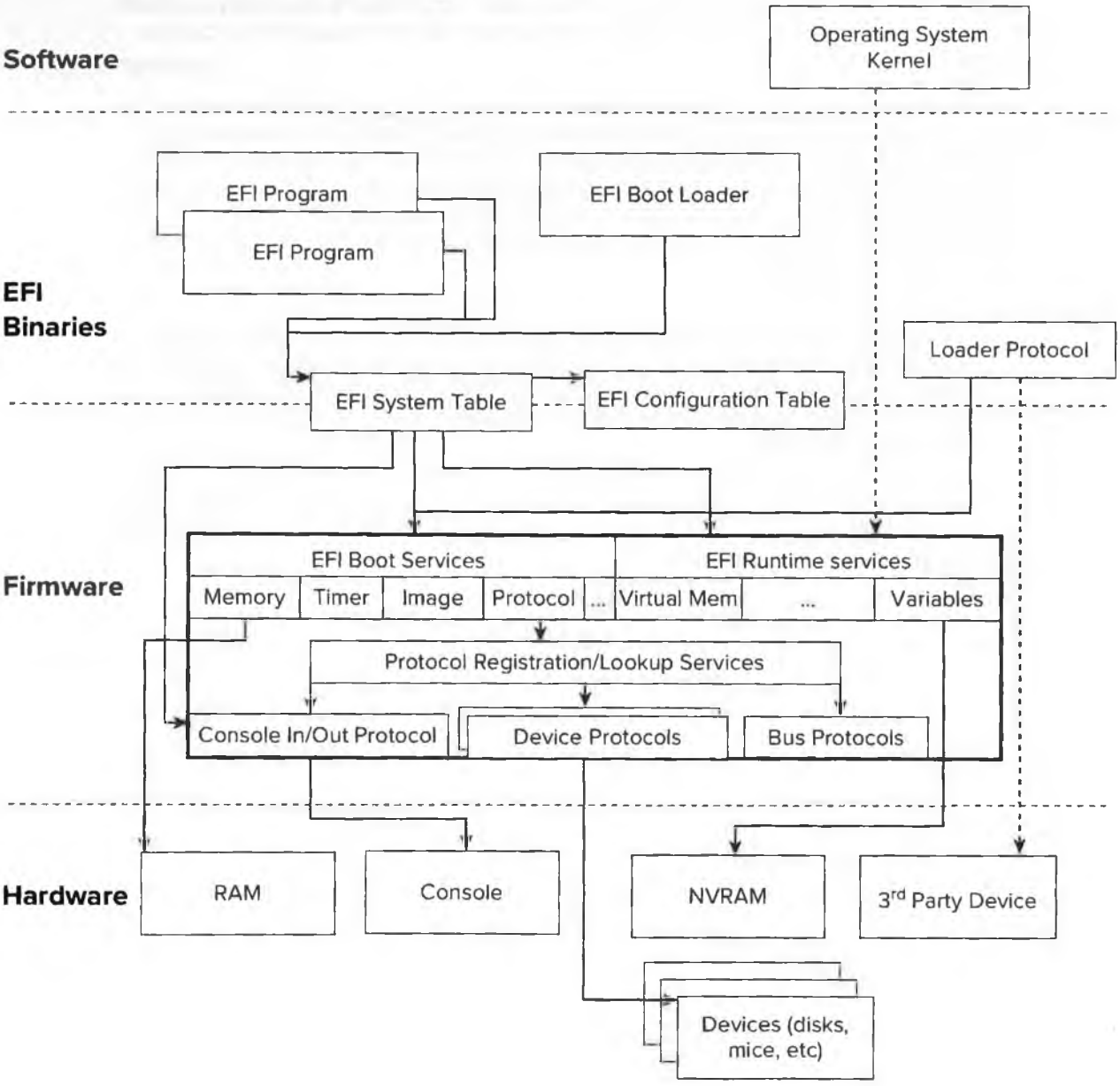
During the PowerPC age, Mac computers used OpenFirmware as the boot environment. With the migration to the Intel architecture, it was only natural for Apple to adopt the **Enhanced Firmware Interface** (EFI), as it offered many similarities. The contemporary BIOS interface used by other Intel/PC manufacturers simply could not offer the rich set of APIs tht EFI provided. An added plus of choosing EFI over BIOS was that MacOS could not boot on non-Apple hardware without it, though that was rectified by the EFI emulation layer known as "Chameleon", which was used in Hackintosh projects.

Apple seems to have implemented the EFI 1.10 specification[1], and stopping there rather than standardizing on the later Unified EFI (UEFI) specification, despite being a promoter in the latter forum. UEFI is nowadays ubiquitous, with Qualcomm devices also using it.

Basic Concepts

Unlike BIOS, EFI is in some respects a mini operating system. It provides a full runtime environment through a standardized interface to the firmware services, which EFI binaries can utilize. It is capable of supporting multiple device driver types (referred to as "protocols"), including networking and a full GUI environment - the initial lock screen encountered on Macs with encrypted volumes is, in fact, EFI's, although it has been made to look exactly like loginwindow.app .

Figure 5-1: The EFI Architecture



As the figure shows, the interface provided by EFI is comprised by two sets of services:

- **The Boot Services:** Which are available to boot loaders only. The boot loaders are expected to use these services to access system resources to facilitate the process of finding and loading the chosen operating system kernel image. The services are maintained in a fairly large table of function pointers, grouped by service classes. Table 5-2 shows this table, with a slightly different set of classes from the EFI specification.

Table 5-2: The EFI Boot Services

Category	Offset	Routine(s)	Purpose
Task Priority	0x18/0x20	`[Raise/Restore]TPL`	Raise/lower Task Priority Level
Memory Allocation	0x28/0x30	`[Allocate/Free]Pages`	Allocate pages for use with pools
	0x38	`GetMemoryMap`	Get virtual memory layout
	0x40/0x48	`[Allocate/Free]Pool`	`malloc()/free()` equivalents
Event	0x50/0x70	`[Create/Close]Event`	Create or destroy an event object
	0x60/0x68/0x78	`[WaitFor/Signal/Check]Event`	Event object lifecycle functions
Timer	0x58/0x100	`Set[Watchdog]Timer`	Set one-time or recurring timer
Protocol Handlers	0x80/0x88/0x90	`[I/Rei/Uni]nstallProtocolInterface`	Attach/detach protocol interface to device
	0x98	`HandleProtocol`	Queries handle for protocol support
	0xa8	`RegisterProtocolNotify`	Register callback for protocol installation
	0xb0	`LocateHandle`	Retrieve handles supporting a given protocol
	0xb8	`LocateDevicePath`	Locate device supporting a given protocol
	0xc0	`InstallConfigurationTable`	Add/update/remove configuration table
	0x118/120	`[Open/Close]Protocol`	Extended `HandleProtocol()`
	0x128	`OpenProtocolInformation`	Return list of protocol users
	0x130	`ProtocolsPerHandle`	Retrieve protocol list for a given handle
	0x138	`LocateHandleBuffer`	Retrieve handles matching criteria
	0x140	`LocateProtocol`	Returns instance of matching protocol GUID
	0x148/0x150	`[I/Uni]nstallMultipleProtocolInterfaces`	Attach/detach 1+ protocol interfaces
Image Handling	0xc8/0xe0	`[Un]LoadImage`	Un/Load an EFI binary from/into memory
	0xd0	`StartImage`	Start an EFI binary
	0xd8	`Exit`	Return to boot services
	0xe8	`ExitBootServices`	Signal EFI Boot Services can be discontinued
Memory Operations	0x160/0x168	`[Copy/Set]Mem`	`memcpy()/memset()` equivalents
	0x158	`CalculateCrc32`	Calculate CRC32 of a given buffer
Driver Support	0x108/0x110	`[C/Disc]onnectController`	Connect/Disconnect drivers to controller
Miscellaneous	0xf0	`GetNextMonotonicCount`	Return monotonically increasing counter
	0xf8	`Stall`	Stalls processor

Of the above services, one is particularly important - `ExitBootServices()`. When called, this effectively terminates the availability of of the boot services, reducing the role of EFI to that of the Runtime Services.

- **The Runtime Services:**

Whereas the boot services support the boot environment, the runtime services support the loaded operating system. The OS kernel is able to assume responsibilities for most functions, but the runtime services remain available to support a minimal set of functions:

Table 5-3: The EFI Runtime Services

Category	Offset	Routine(s)	Purpose
Time Services	0x18/0x20	[Get/Set]Time	Get/Set system time
	0x28/0x30	[Get/Set]WakeupTime	Get/Set alarm wakeup time
Memory Allocation	0x38	SetVirtualAddressMap	Change from physical to virtual addressing
	0x40	ConvertPointer	Physical to virtual address translation
Variable	0x48	GetVariable	Get value of an NVRAM variable
	0x50	GetNextVariableName	Enumerate NVRAM variables
	0x58	SetVariable	Set value of an NVRAM variable
Variable	0x60	GetNextHighMonotonicCount	Return high 32-bit of monotonic counter
	0x68	ResetSystem	Restart the system

The Runtime Services remain accessible throughout the kernel, and are defined in pexpert/pexpert/i386/efi.h as the `EFI_RUNTIME_SERVICES_64` structure. They are also provided through the AppleEFIRuntime.kext. The symbols exported by this kext correspond almost 1:1 with those of the runtime services:

Listing 5-4: The EFI runtime services, made accessible through the AppleEFIRuntime.kext

```
morpheus@Chimera(~)$ jtool2 -S /System/Library/Extensions/AppleEFIRuntime.kext/Contents/MacOS/AppleEFIRuntime |
                     grep "T _" | cut -d' ' -f3 | c++filt
_ReadSpiFlash
AppleEFIRuntime::resetSystem(EFI_RESET_TYPE, unsigned int, unsigned int, short*)
AppleEFIRuntime::setVariable(short*, EFI_GUID*, unsigned int, unsigned int, void*)
AppleEFIRuntime::readSpiFlash(unsigned long long, unsigned int, unsigned int, unsigned char*)
AppleEFIRuntime::setWakeupTime(unsigned char, EFI_TIME*)
AppleEFIRuntime::__OSFinalClass()
AppleEFIRuntime::getConfigurationTable(EFI_GUID*, void**)
AppleEFIRuntime::stringWithUTF16String(short*)
AppleEFIRuntime::UTF16StringWithCString(char const*)
AppleEFIRuntime::getNextHighMonotonicCount(unsigned int*)
AppleEFIRuntime::start(IOService*)
AppleEFIRuntime::setTime(EFI_TIME*)
 .. /* OSMetaClass stuff omitted */ ...
AppleEFIRuntime::getVariable(short*, EFI_GUID*, unsigned int*, unsigned int*, void*) const
AppleEFIRuntime::getMetaClass() const
AppleEFIRuntime::getWakeupTime(unsigned char*, unsigned char*, EFI_TIME*) const
AppleEFIRuntime::callEFIRuntime(unsigned int (*)(void*, void*, void*, void*, void*, void*), ..
AppleEFIRuntime::getNextVariableName(unsigned int*, short*, EFI_GUID*) const
AppleEFIRuntime::getTime(EFI_TIME*, EFI_TIME_CAPABILITIES*) const
AppleEFIRuntime::MetaClass::alloc() const
_callEFI64FromKernel64
```

EFI Protocols

EFI components communicate with eachother by means of **protocols**. A protocol is an object exposing a set of API functions, which is identified by a Globally Unique IDentifier (GUID). The GUID is used with the Boot Services protocol handler functions (from table 5-2) by a provider to register with the environment, and (by a client) to look up and create an endpoint object. The endpoint can then be type cast into the specific object instance, and its methods can be invoked.

There is an inordinate number of protocol GUIDs in Apple's EFI environment. Most are well known GUIDs, appearing in the EFI/UEFI specifications, and provided by the firmware itself so as to provide console, disk or other basic and common I/O services. There are also several Apple specific GUIDs, which are undocumented, but have been reversed. A partial list of the GUIDs can be found in the open sources of the Clover bootloader[2], and Table 1-16 already showed few. Table 5-5 shows a few more:

Table 5-5: Some of the EFI protocol GUIDs used in Apple's EFI environment

GUID	Name
8FFEEB3A-4C98-4630-803F740F9567091D	Apple firmware password protocol
0DFCE9F6-C4E3-45EE-A06AA8613B98A507	Apple image codec protocol

The EFI System Partition

The first partition of the boot device - `/dev/disk0s1` - is designated as the EFI system partition. It is formatted as FAT, and contains a standard directory structure. Over the years, the contents of the partition has changed somewhat, with the following subdirectories, all in /EFI/APPLE:

Table 5-6: The directory structure of the Mac EFI system partition

Subdirectory	Contents
EMBEDDEDOS	T1 devices: combined.memboot (a `bvx2` compressed kernelcache of BridgeOS) and `FDRData` (a personalization property list)
FIRMWARE	Pre Tx devices: Base firmware for the model, `MB[PA][##].scap`
EXTENSIONS	Firmware.scap
UPDATERS/	Accessory firmware updaters (e.g. `USBCVA`)
MULTIUPDATERS/	MultiUpdater.efi and Thorutil.efi binaries (T2: with coupled .im4m personalization files). Also: Mac-*board-id*-R#.bin files. These are specified in IOKit matching dictionaries, in `efi-apple-payload#` NVRAM variables.

The EFI system partition can easily be mounted using `mount_msdos`, but it is recommended to mount it as read-only, lest Spotlight, FSEvents or extended attributes litter it with indexing, UUID and other such files. Output 5-7 shows the mounting of a T2-device's EFI system partition:

Output 5-7: Mounting the EFI system partition (Example from a T2 device)

```
root@Bifröst (~)# mkdir /tmp/mnt; mount -o ro -t msdos /dev/disk0s1 /tmp/mnt
root@Bifröst (/tmp/mnt)# ls -lRF
./EFI:
total 8
drwxrwxrwx  1 _unknown  _unknown  4096 Jun  3 20:43 APPLE/

./EFI/APPLE:
total 16
drwxrwxrwx  1 _unknown  _unknown  4096 Nov  7  2018 EXTENSIONS/
drwxrwxrwx  1 _unknown  _unknown  4096 Jun  3 20:43 UPDATERS/

./EFI/APPLE/EXTENSIONS:
total 30728
-rwxrwxrwx  1 _unknown  _unknown  15729264 Jul 28 11:33 Firmware.scap*

./EFI/APPLE/UPDATERS:
total 8
drwxrwxrwx  1 _unknown  _unknown  4096 Jun  3 20:43 MULTIUPDATER/

./EFI/APPLE/UPDATERS/MULTIUPDATER:
total 1664
-rwxrwxrwx  1 _unknown  _unknown  516096 Jun  3 20:43 Mac-827FAC58A8FDFA22-C0_6.84.0-C1_41.2-R0.bin*
-rwxrwxrwx  1 _unknown  _unknown   83768 Jun  3 20:43 MultiUpdater.efi*
-rwxrwxrwx@ 1 _unknown  _unknown    3044 May 20 21:51 MultiUpdater.efi.j140kap.22DE0018A0026.im4m*
-rwxrwxrwx  1 _unknown  _unknown    3012 May  4 06:57 MultiUpdater.efi.j140kap.im4m*
-rwxrwxrwx  1 _unknown  _unknown  223128 Jun  3 20:43 ThorUtil.efi*
-rwxrwxrwx@ 1 _unknown  _unknown    3044 May 20 21:51 ThorUtil.efi.j140kap.22DE0018A0026.im4m*
-rwxrwxrwx  1 _unknown  _unknown    3012 May  4 06:57 ThorUtil.efi.j140kap.im4m*
```

Software capsules

EFI uses a ROM component to bootstrap the environment, and software images to provide it. The image is a .scap file, formatted as a **software capsule**, which can normally be found in /usr/standalone/i386, or in MacBook "Firmware updates". The capsules are pushed to the EFI system partition by the `firmwaresyncd(8)`, which starts on system boot, and moves them to /EFI/APPLE/EXTENSIONS.

The software capsule format isn't properly documented, but open sources like TianoCore and others offer views into its structure. The capsule begins with a well known (v0.9) `EFI_CAPSULE_GUID` (3B6686BD-0D76-4030-B70E-B5519E2FC5A0), and tools such as the author's `imgtool(j)` can both inspect and extract the file contents.

EFI Binaries

Developing EFI required the standardization of a binary format and bytecode. Intel, being the developer of EFI, naturally chose its own assembly for the bytecode. As Microsoft owned the dominant platform at the time, it made sense to choose Windows Portable Executable (PE) as the binary format.

EFI binaries' entry points accept two arguments: A handle to themselves, and a pointer to the `EFI_SYSTEM_TABLE`. The table can be used to access the `Console[In/Out]` and `StandardErrorHandles`, as well as the `RuntimeServices` and the `BootServices`. Up until the call to `ExitBootServices()`, both can be used by the binaries. Most commonly, the `BootServices`'s `LocateProtocol()` method is called with GUIDs of requested protocols, and (if successful) its return value cast into a protocol object pointer. Depending on the specific protocol, this pointer exposes even more function pointers, through which specific operations can be carried out. The definition of the `EFI_SYSTEM_TABLE` can be found in any EFI toolkit, or in XNU's platform expert (specifically, `pexpert/pexpert/i386/efi.h`), which contains the header files required for the kernel to interface with EFI. Output 5-8 shows the table, which remains mapped in kernel memory:

Output 5-8: The `EFI_SYSTEM_TABLE`, in memory

```
Xn00p> dump _gPEEFISystemTable,8
0xffffff801801a2b0    0xffffff801b00ae10
Xn00p> dump 0xffffff801b00ae10
                EFI_SYSTEM_TABLE_SIGNATURE   __Revision__   HeaderSize
0xffffff801b00ae10 49 42 49 20  53 59 53 54   0a 00 01 00  78 00 00 00 IBI SYST....x...
                    ___CRC32___   __Reserved__       FirmwareVendor
0xffffff801b00ae20 e2 56 e2 59  00 00 00 00     0xffffff801afb5b90      .V.Y........[.....
              FirmwareRevision
0xffffff801b00ae30 0a 00 01 00  00 00 00 00           NULL              ................
                         ConIn                  ConsoleOutHandler
0xffffff801b00ae40         NULL                      NULL              ................
                        ConOut                StandardErrorHandler
0xffffff801b00ae50         NULL                      NULL              ................
                        StdErr                   RuntimeServices
0xffffff801b00ae60         NULL                   0xffffff801b00ab90   ................
                    BootServices              NumberOfTableEntries
0xffffff801b00ae70         NULL                      0x0a              ................
                  ConfigurationTable
0xffffff801b00ae80    0xffffff801b009c10      70 74 61 6c 10 02 00 00   ........ptal....
```

MacOS's boot.efi

MacOS's `boot.efi` is a rare bird - a PE32+ binary* among all the other Mach-Os. `/usr/standalone/i386` contains another copy of this important file, along with `apfs.efi`, which (as of MacOS 13) implements a protocol for APFS (to allow filesystem readability).

Although stripped, the multitude of progress messages makes it easy enough to reverse, and thus illustrate the process of bootloading. When the `boot-args` specify `-v` (for XNU's verbose boot), the Mac will remain in text mode for longer, leaving the EFI boot messages visible, albeit briefly before the copious kernel output will flood the screen.

Output 5-9: Verbose messages for MacOS 14's `boot.efi`

```
SetConsoleMode
OpenKernelRootVolume
ReadKernelCache
UncompressKernelCache
CalculateAdler32
LoadKernelFromStream
InitBootStruct
        root device uuid is '........'
Load RamDisk
FinalizeBootStruct
RandomSeed
(boots kernel)
```

Table 5-10: Parameters in `com.apple.Boot.plist(5)`

Key	Value
`Kernel Flags`	a `String` containing boot arguments to be passed to kernel cache
`Background Color`	
`Boot Logo[ Scale]`	
`RAM Disk`	

* - Complete with the familiar "This program cannot be run in DOS mode." at offset 0x4E

Blessed Art Thou

Apple's firmware locates the EFI bootloader through the `efi-boot-device` NVRAM variable. Up until MacOS 13, this directed to the /System/Library/CoreServices/boot.efi on the root volume (which also functioned as the boot volume). With the advent of APFS, however, this variable now redirects to the boot.efi of the Preboot volume. Another copy of the file can be found in /usr/standalone/i386.

The common way of modifying the boot file is through the `bless(8)` utility. This is a very powerful tool, which can also be used to enable MacOS's netboot feature. Using it with extra verbosity (`sudo bless --info --verbose`) shows the process of boot device selection. Another method is to look through `nvram -p` or the output of `ioreg(8)` on the `IODeviceTree` plane, looking for the `efi-boot-device` key:

Output 5-11: Displaying the `efi-boot-device`

```
morpheus@Bifrost(~)$ sudo bless --info --verbose
EFI found at IODeviceTree:/efi
Current EFI boot device string is: '<array><dict><key>IOMatch</key><dict><key>IOProviderClass</key>
<string>IOMedia</string><key>IOPropertyMatch</key><dict>
<key>UUID</key><string>35915419-0117-401B-AA73-FF173357C272</string></dict></dict>
<key>BLLastBSDName</key><string>disk1s2</string></dict><dict><key>IOEFIDevicePathType</key>
<string>MediaFilePath</string><key>Path</key><string>\33A34D4A-54D0-4FF6-9A21-A40BD946DE1F\
 System\Library\CoreServices\boot.efi</string></dict></array>'
Boot option is 8BE4DF61-93CA-11D2-AA0D-00E098032B8C:Boot0080
Processing boot option 'Mac OS X'
Boot option matches XML representation
...
  919 => Blessed System File is /33A34D4A-54D0-4FF6-9A21-A40BD946DE1F/System/Library/CoreServices/boot.efi
   43 => Blessed System Folder is /33A34D4A-54D0-4FF6-9A21-A40BD946DE1F/System/Library/CoreServices
The blessed volume in this APFS container is "/".
#
# Alternatively, just display efi-boot-device (no entitlements or root required, but can't set)
#
morpheus@Bifrost(~)$ nvram efi-boot-device
efi-boot-device <array><dict><key>IOMatch</key><dict><key>IOProviderClass</key><string>IOMedia</string>
<key>IOPropertyMatch</key><dict><key>UUID</key><string>35915419-0117-401B-AA73-FF173357C272</string>
</dict></dict><key>BLLastBSDName</key><string>disk1s2</string></dict><dict><key>IOEFIDevicePathType</key>
<string>MediaFilePath</string><key>Path</key><string>\33A34D4A-54D0-4FF6-9A21-A40BD946DE1F\
 System\Library\CoreServices\boot.efi</string></dict></array>%00
```

The EFI boot sequence is notoriously insecure, and every couple of years since 2007[3] (e.g. BlackHat 2012[4]) this makes headlines as security researchers rediscover just how bad the situation is. EFI related vulnerabilities (specifically ThunderStrike [1/2][5]) made headlines, and Apple's EFI implementation, in particular, has been subjected to detailed reversing[6]. As of the 2017 iMac Pro, however , all Macs have an embedded security chip - T2 - which greatly enhances the boot proecss, with the help of BridgeOS. To better understand this, however, we must first take a look at the *OS boot process.

MacOS systems have a /usr/libexec/firmwarecheckers, with binaries for checking the EFI firmware and that of the NIC (BCM57xx only). The `eficheck(8)` binary proves useful for dumping the EFI firmware (on pre-T2 devices only), generating hashes and performing integrity checks against known hashes in .../eficheck//EFIAllowListShipping.bundle/allowlists .ealf files. The check is performed by opening an `IOUserClient` instance to `eficheck.kext`, and mapping the firmware memory into the task.

Another binary, /usr/libexec/efiupdater is used when installing firmwares. By default, it looks for files in /AppleInternal/Packages/EFIPayloads/, which does not exist in release systems. The actual updating is performed by a call to `bless(8)` with the undocumented `--firmware` argument.

The open source `rEFInd` boot manager provides a great way to explore EFI environments in general, and Apple's EFI in particular. The previous edition of this book also contains a simple example of using the GNU EFI toolkit to create a simple EFI program, which can be run under `rEFInd`'s shell.

*OS: iBoot

iBoot is the collective name given to Apple's boot loader components, and sometimes to the particular second stage boot loader. These components form the boot chain of iOS, starting with the SecureROM, and (during normal boot flow) ending at iBoot, which loads the kernelcache, unless any part of the process fails, in which case the device is put into recovery mode ("Connect to iTunes") or USB-DFU (discussed later).

Although Apple claimed to have stopped encrypting iOS images "to help researchers" and the iBoot 32-bit images were indeed plaintext as of iOS 10, those of iBoot 64-bit remain encrypted with the device-specific (GID) keys to this day. 64-bit images have several important differences than those of 32 (notably, coprocessor and KPP support), and 32-bit images are no longer relevant as of iOS 11 anyway. This has not stopped researchers, who find ways to publish the keys and IVs (initialization vectors) required to decrypt the iBoot 64 images. These keys were scarce for the longest time, until multiple ones for all 12.x versions started appearing on the iPhone Wiki. Although of dubious origin (an iBoot-level exploit or development-fused devices), they are nonetheless welcome, as they provide visibility onto this important codebase, which forms the fulcrum for all of *OS security.

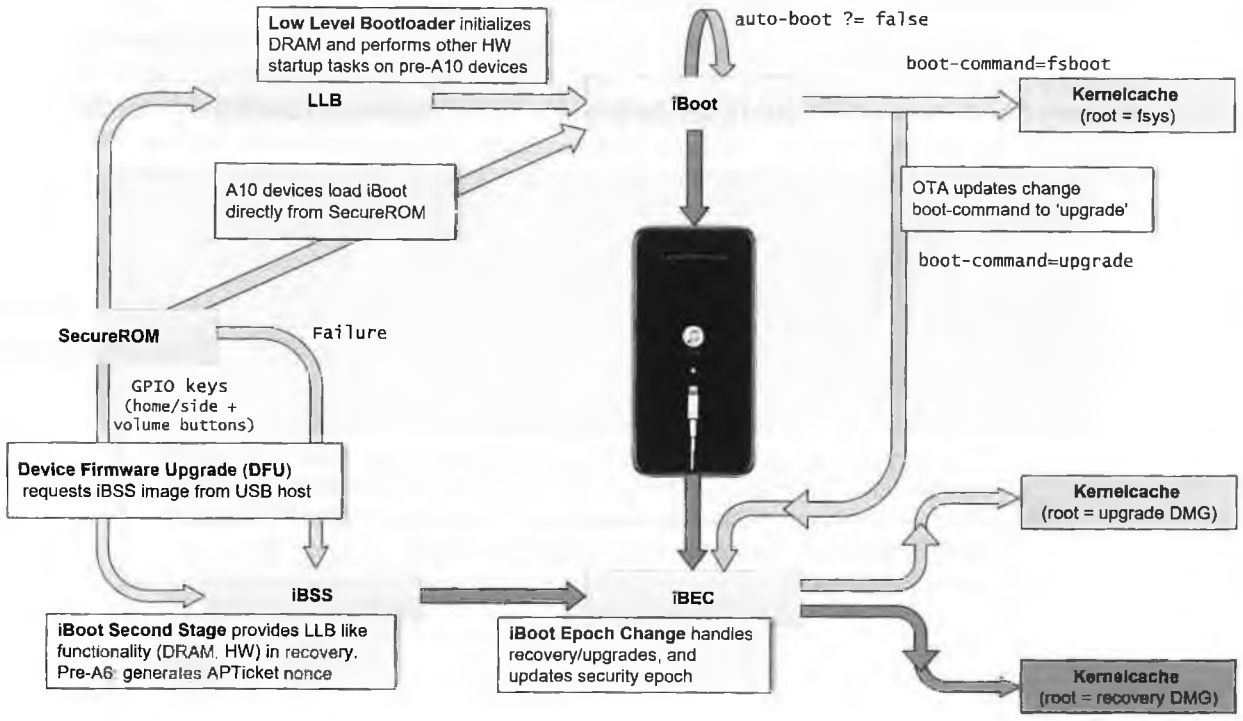
The iBoot sources were also leaked from Apple - making a public appearance on GitHub in mid February 2018 (with apparent links posted even earlier on Reddit). Although the exact iBoot version (which is provided as an external `#define`) is hard to determine, the latest board support - t8010 as a simulator/FPGA - indicates iPhone 6S and along with the copyright (2015) is thus likely no later than iBoot-2817, i.e. early iOS 9.

This work cannot show code snippets from the iBoot sources, as those were leaked illegally and are copyrighted by Apple. Apple quickly invoked the DMCA in an effort to contain the leaked sources, but this proved futile, as countless mirrors can still be found. These pages, instead, demonstrate disassembly from the publicly available decrypted image of iBoot, thanks to iOS 13 beta keys posted anonymously to the NewOSXBook.com Forum[7].

The Boot Chain

The boot chain of *OS refers to the sequence of steps carried out by the application processor, starting with the loading of the SecureROM image, and ending with the loading of the XNU kernelcache. This is considered a chain, since it is comprised of distinct stages, wherein each stage is charged with its own various initialization tasks, as well as locating and loading the next stage. Figure 5-12 shows the traditional boot chain of *OS.

Figure 5-12: The traditional boot chain of *OS



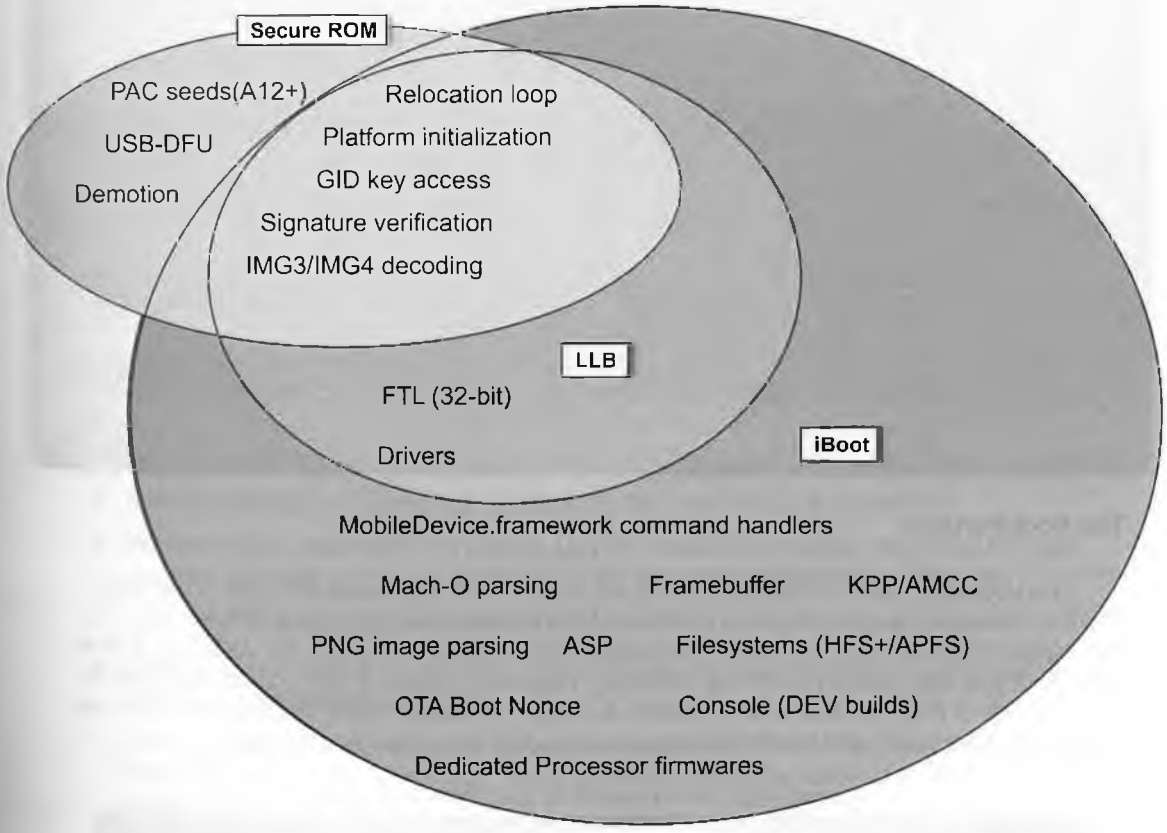
Common Code

Dumping the boot components reveals a very large amount of common code, and the eventual leak of the sources corroborated this, by showing that the components are all built from the same source base. All components are known to contain the following:

- Rebase loop: which checks if the code has been loaded at a predefined address (e.g. 0x100000000 for the A12 SecureROM, or 0x19c030000 for iBoot-5540). If not, the code is moved (see Listing 5-17 for an example).
- Platform bootstrapping: CPU initialization, Setting up the Vector Base Address Register, the MMU and other ARM MSRs.
- Task management, allowing multithreading. This is primarily used by the the second stage iBoot, but even the SecureROM has support for spinning a USB task, when in DFU mode.
- IMG3/IMG4 code: 32-bit iDevices (excluding watches) used the proprietary IMG3, and 64-bit images (and watches) use IMG4 (a form of DER) to encode components - binaries and graphic images.
- Certificate handling: including both an X.509v3 hard-coded certificate, and the DER-code to parse and verify images signed with its public key.

The later components in the chain have increasing functionality, culminating with iBoot's ability of reading file systems, Mach-O objects, and console support. Conceptually, this could be visualized as follows:

Figure 5-13: The functional relationships of the various components



The reason for using four different components (LLB, iBoot, iBEC and iBSS) with so much common code was the limitations of SRAM, and possibly security considerations (e.g. due to DMG loading in iBEC). A10 and later devices (and thus, later SecureROM versions) still have four separate images, but inspecting them (past decryption) reveal they are all identical. In other words, one single iBoot image now handles all phases. LLB is no longer required, and thus the SecureROM (which still shares its code with iBoot) directly loads the iBoot image, which is still tagged as 'illb', rather than 'ibot'.

The Chain of Trust

An important aspect of the boot chain is its resulting **chain of trust**. The trust begins with a hard-coded root certificate, whose bytes are burnt into the ROM. Normally, root certificates pose a problem to implicitly trust, but because the certificates are in a read-only memory component, they cannot be modified in any way, and can thus kick off the chain. In this way, the ROM certificate can vouch for the integrity of the iBoot components, and it is iBoot, in turn, which verifies the digital signature (and APTicket entry) and integrity of the kernelcache with the certificate embedded in its image.

The hard-coded and embedded certificates are easy to extract, given a SecureROM dump or decrypted image. They are DER encoded, and can thus be found by looking for the "30 82 05 5a" encoding a SEQUENCE of 1,370 bytes. Output 5-14 shows the certificate from a SecureROM dump, but could be applied on any iBoot image:

Output 5-14: Locating the Apple Secure Boot CA in a SecureROM image

```
morpheus@Bifröst(~) % hexdump -C SecureROM.t8020si.3865 | grep "30 82 05 5a"
0001ff10  30 82 05 5a 30 82 03 42  a0 03 02 01 02 02 08 52  |0..Z0..B.......R|
morpheus@Bifröst(~) % dd if=SecureROM.t8020si.3865 of=t8020cert bs=0x1ff10 skip=1
morpheus@Bifröst(~) % openssl asn1parse -inform der -in t8020cert
    0:d=0  hl=4 l=1370 cons: SEQUENCE
    4:d=1  hl=4 l= 834 cons: SEQUENCE
    8:d=2  hl=2 l=   3 cons: cont [ 0 ]
   10:d=3  hl=2 l=   1 prim: INTEGER           :02
   13:d=2  hl=2 l=   8 prim: INTEGER           :521563F9FDF7D0C6
   23:d=2  hl=2 l=  13 cons: SEQUENCE
   25:d=3  hl=2 l=   9 prim: OBJECT            :sha384WithRSAEncryption
      ...
   44:d=5  hl=2 l=   3 prim: OBJECT            :commonName
   49:d=5  hl=2 l=  30 prim: UTF8STRING        :Apple Secure Boot Root CA - G2
      .....
   85:d=5  hl=2 l=   3 prim: OBJECT            :organizationName
   90:d=5  hl=2 l=  10 prim: UTF8STRING        :Apple Inc.
      ..
  117:d=3  hl=2 l=  13 prim: UTCTIME           :1412192013102
  132:d=3  hl=2 l=  13 prim: UTCTIME           :3412142013102
      ..
  153:d=5  hl=2 l=   3 prim: OBJECT            :commonName
  158:d=5  hl=2 l=  30 prim: UTF8STRING        :Apple Secure Boot Root CA - G2
      ..
  194:d=5  hl=2 l=   3 prim: OBJECT            :organizationName
  199:d=5  hl=2 l=  10 prim: UTF8STRING        :Apple Inc.
      ..
  780:d=5  hl=2 l=   3 prim: OBJECT            :X509v3 Subject Key Identifier
  785:d=5  hl=2 l=  22 prim: OCTET STRING      [HEX DUMP]:[illegible]
  809:d=4  hl=2 l=  15 cons: SEQUENCE
  811:d=5  hl=2 l=   3 prim: OBJECT            :X509v3 Basic Constraints
  816:d=5  hl=2 l=   1 prim: BOOLEAN           :255
  819:d=5  hl=2 l=   5 prim: OCTET STRING      [HEX DUMP]:[illegible]
  826:d=4  hl=2 l=  14 cons: SEQUENCE
  828:d=5  hl=2 l=   3 prim: OBJECT            :X509v3 Key Usage
  833:d=5  hl=2 l=   1 prim: BOOLEAN           :255
  836:d=5  hl=2 l=   4 prim: OCTET STRING      [HEX DUMP]:[illegible]
      ..
```

The Boot Partition

The updatable boot components of *OS are copied from their respective files in the .ipsw (from firmware/all_flash) to the boot partition of the iDevice (now the second NVMe namespace), wherein they are stored in a large IMG4 container along with the APTicket. In this way, iBoot can load them by their four letter tag. Images are stored in LZSS compressed ARGB form (and as of A11, illb itself is compressed, as well). The device tree is stored in raw form (as shown in Listing 1-22), and iBoot/LLB is stored encrypted by the device's GID key.

Output 5-15: The contents of the boot NVMe namespace

```
# Using openssl, parse the img4 containers in the boot NVMe namespace,
# looking for the tags following the IM4P magic, but removing the magic
#
openssl asn1parse -inform der -in nvme0n2p1 |grep -A 1 IM4P | grep -v IM4P
     22:d=2  hl=2 l=   4 prim: IA5STRING         :illb   # iBoot
 456046:d=2  hl=2 l=   4 prim: IA5STRING         :batl   # Battery
 464518:d=2  hl=2 l=   4 prim: IA5STRING         :liqd   # Liquid detected
 958023:d=2  hl=2 l=   4 prim: IA5STRING         :dtre   # Device Tree
1110215:d=2  hl=2 l=   4 prim: IA5STRING         :glyP   # Glyph
1121423:d=2  hl=2 l=   4 prim: IA5STRING         :chg0   # Charge lightning bolt
1137201:d=2  hl=2 l=   4 prim: IA5STRING         :bat0   # Battery (empty)
1187055:d=2  hl=2 l=   4 prim: IA5STRING         :batF   # Battery (full)
1275655:d=2  hl=2 l=   4 prim: IA5STRING         :chg1   # Charge lightning bolt
1316715:d=2  hl=2 l=   4 prim: IA5STRING         :logo   # Apple Logo
1329574:d=2  hl=2 l=   4 prim: IA5STRING         :recm   # "connect to iTunes"
```

The SecureROM

When power is first supplied to the system, the first code to execute on the application processor is that of the boot ROM, referred to by Apple as "SecureROM". The code of the SecureROM is a subset of the iBoot source code, but unlike it cannot be updated. As its name implies, the SecureROM is a **Read Only Memory** component, which - for better or for worse - cannot be modified after fabrication by anyone, not even Apple. The version of the SecureROM therefore matches the iBoot version at the time when device was sent to manufacturing (e.g. iBoot-2696.0.0.1.33 for A10, or iBoot-4479.0.0.100.4 for A13). This version is reported to the host as part of the serial number when the device is in DFU mode, and is visible in the IORegistry.

Other than that identification, the SecureROM remains largely invisible, as access to it is disabled when loading the next stages. Nonetheless, the first public demonstration of SecureROM dumping was demonstrated by Ramtin Amin, who ingeniously devised a method to perform a man-in-the-middle attack against the system's PCIe. This landmark achievement, however, faded away once the ROM source code leaked along with the rest of the iBoot sources. JTaggable ("dev-fused") iDevices can also be made to dump the SecureROM, as can a SecureROM exploit such as @axi0mX's Checkm8, discussed later.

The SecureROM is very small - about 150,320 or so bytes in the A12 - and is therefore limited in the amount of code it contains. It presently consists of the following routines:

- Platform startup: The code starts with a call to a board specific platform startup, which sets board specific registers.
- Relocation Loop: A check to see if it is already based at its desired address in memory. If not, it relocates itself to that address using the relocation loop described earlier.
- continue start: which sets up the address of the main as its return (i.e. in `LR`), then proceeds to low level initializations, which include setting the `VBAR_EL1`, exception, interrupt and bootstrap stacks, page tables, and a heap guard page, before returning into the main, which proceeds according to the following stages:
- CPU initialization: Initializes the MMU, and sets the value of the System Control Register, `SCTLR`. The bits set are `M` (#0, enabling MMU translation), `SA` (#3, SP alignment check), `C` and `I` (#2 and #12, for data and instruction cache), and `WXN` (#19, for Write-eXecute-Never).
- Initialization of clocks, the SRAM bus, and GPIO pins.
- On A12, the SecureROM sets the ARMv8.3 Pointer Authentication Code (PAC) random seeds.
- Miscellaneous initializations: Consisting of clock, internal memory and GPIO pins.
- System initialization: Consisting of setting up the heap and task subsystem.
- Platform early initialization: An iDevice specific initialization routine, branching to other routines for power management, board and chip specific initialization, UART, etc.
- Checking the force DFU pin: to determine if Device Firmware Update is forced.
- Platform late initialization: Another iDevice specific routine, branching to other board component initialization routines.
- Obtain the boot device and selected boot configuration. This will normally load the next stage (marked by the '`illb`' tag), but if the stage cannot be loaded (or if DFU is forced), it will drop to DFU.

Once loaded in memory, booting an image is performed by preparing and jumping, using a function which takes the boot image type, load address, and argument pointer. The function quiesces the hardware and CPU, disables interrupts, and transfers control by calling the load address as a function through a trampoline. This is expected to never return.

DFU mode

If the boot process cannot continue past the SecureROM (for example, if the next stage cannot be loaded), the boot chain cannot be trusted, and the device is effectively bricked. To allow recovery, the SecureROM contains code to implement a fallback to **Device Firmware Update** (DFU) mode, as a failsafe for recovery when the boot chain fails. This mode can also be requested by the GPIO keys (i.e. iDevice buttons) as documented in the iPhone Wiki[8].

DFU is a well documented USB standard[9]. The SecureROM starts a `usb` task, which emits the iDevice identifier "Apple Mobile Device (DFU Mode)" a long string as a serial number, consisting of entries as shown in Figure 5-16. When the phone is booted normally, the CPID and the ECID concatenated together make up the actual serial number.

Figure 5-16: A DFU emitted serial number string

The values (along with another USB descriptor holding the SEP nonce value as `SNON`) are picked up by iTunes and used in an APTicket request. The SecureROM then awaits a DFU download request, through which it expects an iBSS (recovery mode LLB/iBoot) image. The image is subject to signature and APTicket verification. If verified, control is transferred and the image is booted.

Demotion

Retail devices are hard fused with a CPFM of 03 (i.e. 00000011b), indicating secure and production, as shown in Figure 5-16. The highly sought after (and often illegally obtained) "dev-fused" devices have CPFM of 01 (secure, development), or 00 (insecure, development). The fusing is performed by hardware, so once the device is fabricated, it cannot be changed. The SecureROM is responsible for loading the fuse values during early startup, and enforcing security.

Code execution at the SecureROM level allows **demotion**, which enables overriding the CPFM bits from the fused value. Although the fuses are immutable, they are loaded into a memory mapped register, whose value *can* be changed. Overriding the CPFM is thus simply a matter of writing to that memory mapped address. Note, that the demotion does not actually change the state of the fuses, and therefore is only temporary, persisting until the next reboot.

Demoting a device can enable **JTAG**. The **J**oint **T**est **A**ccess **G**roup standard for testing devices (more formally known as IEEE Standard 1149.1-1990), is a powerful testing mechanism which consists of a test access port (TAP), and a set of debug registers. JTAG's capabilities are nigh omnipotent. Using JTAG, one can directly debug the Application Processor at any time. While this will probably still not allow retrieval of the UID/GID keys (as those are maintained by an AES coprocessor), it would still mean that execution could be controlled at any time, allowing usage of the keys for encryption and decryption (chosen ciphertext/plaintext attacks). Execution could be stopped and inspected at any stage of the boot chain – from as early as the ROM to the kernel – and the bootchain trust could easily be compromised, allowing any iOS configuration – jailbroken or other – one would see fit.

A special proprietary Apple cable with a symian code name (e.g. "Kong" or "Kanzi") is required to use JTAG on an iDevice. Additionally, proprietry software called "Astris" (traces of which can be seen in the *OS filesystem) handles the transport to allow a debugger interface. Nonpareil hardware hacker Ramtin Amin has managed to reproduce its function with a "Bonobo" cable, available through his company's website[10], and compatible with OpenOCD.

iBoot (the Second Stage Bootloader)

The second stage boot loader, referred to internally as `iBootStage2`, is also the main one. It starts with a relocation loop to move the image to a specific virtual address, if not already there. The address changes between releases - in iOS 12 it is 0x1800b0000, and in iOS 13 is `0x19c030000`. The address is clearly visible in the relocation loop, as shown in Figure 5-17:

Listing 5-17: The iBoot relocation loop

```
#
# Get loaded address (here, 0 because disarm assumes image was mapped there)
# and compare to fixed address (stored at 0x318, here 0x19c030000).
#
_start:
0x00000000      ADRP X0, 0          ; X0 = 0x0..
0x00000004      ADD X0, X0, #0      ; X0 = X0 = actual load address
0x00000008      LDR X1, #784        ; X1 = *(0x318) = 0x19c030000 = desired load address
0x0000000c      BL 0x1b5e8          ; = platform_start(0x0, 0x19c030000)..
0x00000010      CMP X1, X0, ...     ; ..
0x00000014      B.EQ 0x60           ; if equal, _continue_start
0x00000018      LDR LR, #48         ; X30 = *(0x48) = 0x19c030050 else, set desired address
0x0000001c      LDR X2, #772        ; X2 = *(0x320) = 0x19c1a33c0 (end of image)
0x00000020      SUB X2, X2, X1      ; X2 = end - beginning = image len
0x00000024      MOV_R X5, X0        ; X5 = X0 (0x0)..
0x00000028      MOV_R X6, X2        ; X6 = X2 (0x19c1a33c0)..
0x0000002c      MOV_R X7, X1        ; X7 = X1 (0x19c030000)..
copy_loop:
0x00000030      LDP X3, X4, [X0], #0x10          ; [X0, X0] = *[X0]..
0x00000034      LDP X3, X4, [X1], #0x10          ; [X0, X0] = *[X0]..
0x00000038      SUBS X2, X2, #16 ;
0x0000003c      B.NE 0x30           ;  copy_loop
0x00000040      RET                 ;  Goto LR (= 0x19c030050)
0x00000044      NOP                 ;
0x00000048      DCQ 0x19c030050     ;
0x00000050      LDP XZR, XZR, [X5], #0x10        ; [X0, X0] = *[X0]..
0x00000054      SUBS X6, X6, #16,; ..
0x00000058      B.NE 0x50           ;
0x0000005c      BR X7               ; jump to 0x19c030000 (relocated)
_continue_start:
0x00000060      MSR DAIFSet, #15         ; D=1 A=1 I=1 F=1 ..
```

Following the relocation, startup continues with miscellaneous initializations (`VBAR_EL1` to 0x19c031000, etc), before setting the `LR` value to the generic main function (`0x19c03340c` in the 13β1 iBoot image), and then returning to it. This function, in turn, performs much of the initialization already discussed in the SecureROM section (notably, setting up the default environment), and starts up the `main` task (`0x19c0337d4`, using a thread starting function (`0x19c06b584`).

The `main` is one of several tasks (= threads) running in parallel. The various tasks are responsible for handling asynchronous events, such as USB current detection, idle timers, etc. Isolating calls to the thread string function will reveal these tasks, as shown in Output 5-18:

Output 5-18: iBoot tasks (Addresses from iBoot-5540.0.15.0.2 d321, iOS 13β1)

```
morpheus@chimera (~)$ disarm -s 0x19c0a9d00-0x19c0ae7a0 -opcodes -base 0x19c030000 iBoot-5540.0.15.2 |
pipe>                 grep "func.*6b584"    # 0x6b584 is task_create()
0x19c0335e4   BL 0x19c06b584 ; func_06b584("main",0x19c0337d4)              # Main thread
0x19c033d50   BL 0x19c06b584 ; func_06b584("poweroff",0x19c033ef4)          # Button power off
0x19c033d7c   BL 0x19c06b584 ; func_06b584("command",0x19c09a298)          # command console (output only)
0x19c033dc0   BL 0x19c06b584 ; func_06b584("idlepwr",0x19c034098)          # Idle power off
0x19c06ddb4   BL 0x19c06b584 ; func_06b584("usb_req",0x19c06de68)          # USB "vendor" requests (iTunes)
0x19c06ddfc   BL 0x19c06b584 ; func_06b584("usb-high-current",0x19c06deb0) # Detects iDevice plugging in
0x19c06de50   BL 0x19c06b584 ; func_06b584("usb-no-current",0x19c06dee0)   # Detects iDevice disconnection
0x19c07fe0c   BL 0x19c06b584 ; func_06b584("usb",0x19c07fe2c)              # Controller (Synopsis) OTG task
0x19c083138   BL 0x19c06b584 ; func_06b584("usb_serial",0x19c083478)       # USB serial console
```

The task structure has changed size over the years, but as recently as iBoot-5540 it is 432 bytes, with two magic values ('task' and 'tsk2') at their ends, further containing the task name (@408), its function pointer (@376) and argument (@384), and a pointer to a stack (@392) of a given length (@400), preinitialized to a magic of "stak". The tasks are linked through doubly linked list entries to the global task list (@8) and to the run queue (@24, with the queue @0x19c1a2b90)

Environment Variables

The RELEASE builds of iBoot allow a very limited subset of environment variables to be set or read from the NVRAM, using two whitelist tables - one for `setenv` and the other for `getenv`. The lists are shown in Output 5-19:

Output 5-19: The whitelisted environment variables, retrieved using `disarm(j)`

```
morpheus@Bifröst (....) % disarm -base 0x19c030000 iBoot-5540.0.15.2 |
pipe>                     grep -A26 0x19c0ae910:
0x19c0ae910:    DCQ     0x19c0a9ecb ; "auto-boot"
0x19c0ae918:    DCQ     0x19c0a9ed5 ; "boot-args"                   # Boot arguments (ignored)
0x19c0ae920:    DCQ     0x19c0a9edf ; "debug-uarts"                 # Debug flags (3 = Serial UART)
0x19c0ae928:    DCQ     0x19c0a9d7b ; "filesize"                    # USB DFU filesize
0x19c0ae930:    DCQ     0x19c0a9eeb ; "pwr-path"
0x19c0ae938:    DCQ     0x0
0x19c0ae940:    DCQ     0x19c0a9ecb ; "auto-boot"                   # Boolean: Boot boot-command, or stop
0x19c0ae948:    DCQ     0x19c0a9ef4 ; "backlight-level"             # Integer specifying display brightness
0x19c0ae950:    DCQ     0x19c0a9f04 ; "boot-command"                # 'fsboot', 'upgrade', etc.
0x19c0ae958:    DCQ     0x19c0a9f11 ; "com.apple.System.boot-nonce" # Seed for APTicket nonce
0x19c0ae960:    DCQ     0x19c0a9edf ; "debug-uarts"
0x19c0ae968:    DCQ     0x19c0a9f2d ; "device-material"
0x19c0ae970:    DCQ     0x19c0a9f3d ; "display-rotation"            # Rotate display (on watches)
0x19c0ae978:    DCQ     0x19c0a9f4e ; "display-vsh-comp"
0x19c0ae980:    DCQ     0x19c0a9f5f ; "idle-off"                    # Turn off after # of seconds idle
0x19c0ae988:    DCQ     0x19c0a9f68 ; "is-tethered"                 # Connected to USB?
0x19c0ae990:    DCQ     0x19c0a9f74 ; "darkboot"
0x19c0ae998:    DCQ     0x19c0a9f7d ; "ota-breadcrumbs"             # OTA update progress
0x19c0ae9a0:    DCQ     0x19c0a9d64 ; "boot-breadcrumbs"            # Boot breadcrumbs
0x19c0ae9a8:    DCQ     0x19c0a9f8d ; "recovery-breadcrumbs"        # Recovery update progress
0x19c0ae9b0:    DCQ     0x19c0a9fa2 ; "com.apple.System.tz0-size"   # SEP memory
0x19c0ae9b8:    DCQ     0x19c0a9fbc ; "com.apple.System.rtc-offset" # RTC Offset
0x19c0ae9c0:    DCQ     0x19c0a9eeb ; "pwr-path"
0x19c0ae9c8:    DCQ     0x19c0a9fd8 ; "upgrade-retry"
0x19c0ae9d0:    DCQ     0x19c0a9fe6 ; "preserve-debuggability"      # ?
0x19c0ae9d8:    DCQ     0x0
```

RELEASE iBoot recognizes several possible values for the `boot-command` NVRAM variable:

- **`diags`**: Boot a diagnostics image. The `diags` image is not present on release devices, but can be found on Switchboard devices, complete with an EFI runtime environment and a rich command line full of diagnostics functions and utilities.
- **`fsboot`**: The default for release builds, this finds and loads the iOS kernelcache, subject to APTicket verification.
- **`upgrade`**: Start an upgrade of the system. Used during OTA installation.
- **`recover`**: Fall to recovery mode, possibly using a recovery partition (unused?).

iOS 13 has several other recognized commands - the table of commands and their handlers is easy to locate, and shown here in the output of `disarm(j)`.

Output 5-20: The recognized `boot-commands` in iBoot-5540

```
morpheus@Bifröst (...)$ disarm -s 0x19c0a9d00-0x19c0ae7a0 -opcodes \
>     -base 0x19c030000 iBoot-5540.0.15.2 | grep -A15 ^0x19c0f4c0
0x19c0f84c0:    DCQ     0x19c0a9e21     ; "alamo"
0x19c0f84c8:    DCQ     0x19c03214c
0x19c0f84d0:    DCQ     0x19c0a9e27     ; "rtos"
0x19c0f84d8:    DCQ     0x19c032280
0x19c0f84e0:    DCQ     0x19c0a9e2c     ; "rbm"
0x19c0f84e8:    DCQ     0x19c0322ac
0x19c0f84f0:    DCQ     0x19c0aa128     ; "diags"
0x19c0f84f8:    DCQ     0x19c0322d8
0x19c0f8500:    DCQ     0x19c0aa12e     ; "fsboot"
0x19c0f8508:    DCQ     0x19c032934
0x19c0f8510:    DCQ     0x19c0aa135     ; "upgrade"
0x19c0f8518:    DCQ     0x19c034aa8
0x19c0f8520:    DCQ     0x19c0aa13d     ; "recover"
0x19c0f8528:    DCQ     0x19c034f0c
0x19c0f8530:    DCQ     0x19c0aa145     ; "recover-once"
0x19c0f8538:    DCQ     0x19c0350fc
```

iBoot is also responsible for initializing the SEP firmware (as discussed in Chapter 1). As of iOS 12, iBoot also handles loading the various coprocessor firmwares. The iBoot image holds an array of structures which, for each coprocessor image, keeps the coprocessor name, device tree paths (e.g. arm-io/aop/iop-aop[-nub]), path to the image (an .img4 file, in /usr/standalone/firmware/FUD), and 32-bit tag.

Booting the kernelcache

iBoot's raison d'être is to locate and boot the iOS kernelcache. The default `boot-command` environment variable value is, thus, `fsboot`. Upon executing the `fsboot` command, iBoot checks the environment to locate the `boot-path`, `boot-device` and `boot-partition` (set to 0). These parameters provide the necessary parameters for mounting the root filesystem. iBoot thus needs a filesystem driver - either HFS+ (up to iOS 10.3) or APFS (thereafter). The default `boot-path` is /System/Library/Caches/com.apple.kernelcaches/kernelcache, though the default `boot-device` varies by device type (`nand0`, `asp_nand0` or `nvme_nand0`). Because neither of these values is in the environment variable white list, the defaults are also the only options for `RELEASE` builds.

iBoot next checks the `dt-path` and `boot-ramdisk` variables, which determine the Device Tree to pass to the kernelcache, as well as an optional RAM disk image (for iBEC mode). The `struct boot_args` (see Table 5-27, later in this chapter), is constructed, and passed to the kernelcache (uncompressed in memory) as an argument.

Threat modeling iBoot

The boot process is the linchpin of iOS security. For all of Apple's formidable hardware security mechanisms, they must be explicitly enabled by software. But if the boot sequence is compromised the kernelcache can be patched a priori of loading, thus removing the instructions vital for starting KPP, KTRR and/or APRR, or neutering AMFI and the Sandbox.kext. From the jailbreaking perspective, a boot sequence vulnerability could enable a tethered jailbreak or dual booting, and (if in the SecureROM) potentially be unpatchable for the afflicted iDevice models.

Security researchers and jailbreakers alike thus invest a great deal of effort in seeking potential vulnerabilities. At a high level, three attack surfaces can be identified:

- **USB**: All boot components have some form of USB support, with functionality increasing from the bare minimum of DFU (in the SecureROM) to full iTunes protocol support (in iBoot). The A4 "limera1n" exploit was, for almost a decade, the last effective example of a SecureROM vulnerability, and was reliably exploited to provide tethered, unpatchable jailbreaks up until the iPhone 4. This changed just as this book was going to print, when @axi0mX released his "checkm8" tool[11] to provide a working SecureROM level exploit for all devices up to and including the A11 (iPhone 8/X)*.
- **NAND**: Each component of the boot chain has to locate the next stage, which involves reading either the partitions (NVMe namespace) or the filesystem (iBoot). Targeted malformations could potentially cause memory corruption at one of the boot stages, yielding code execution. Another deliberate malformation could be in the digital signature of each component (specific X.509/DER fields). A mitigating factor is that such attacks require initial low level access to the NAND in order to mount (commonly, through a pre-existing jailbreak, or NAND MiTM). An iBoot vulnerability in HFS+ handling was demonstrated and eventually fully documented by @Xerub for 32-bit iBoot, though no vulnerabilities are publicly known for iBoot 64-bit.
- **Digital Signatures**: aside from signature malformation as a vector for memory corruption, there is also the potential for digital signature forgery, which would allow loading a compromised stage. This is highly improbable (an adversary capable of reliably breaking RSA or performing a second pre-image for SHA-256 likely has even more lucrative targets than iPhones), but cannot be entirely discounted. Another risk is that Apple's root certificate private keys could be stolen or otherwise selectively leak.

Considering that companies such as Cellebrite and GrayShift make their business off of breaking into iPhones, one can only deduce vulnerabilities do exist. Their extremely high value in the exploit market, coupled with Apple's stingy bounty, makes it likely they will remain hidden for as long as possible, maybe even indefinitely. USB based vulnerabilities would be of great value, since they allow for "evil chambermaid" attacks, wherein an iDevice left unattended could be rebooted and compromised by an attacker with momentary access. Said attacker could compromise the kernelcache, and wait for the user to unlock their device - and thus capture their passcode. Though powerful, this attack would not survive a hard reset of the iDevice, but when coupled with NAND vulnerabilities, persistence could be achieved.

* - axi0mX was also most helpful as he personally and meticulously reviewed this section - and for that, my gratitude

MacBook Pro (2018) and later: iBoot + EFI

Apple first introduced embedded processors with BridgeOS to the 2016 MacBook Pro and the T1 chip. It wasn't until shortly after, with the iMac Pro and the T2 chip, that BridgeOS was also harnessed to address the gaping insecurities of EFI boot. The T2 is now standard in all 2018 mac models, including the Mac Mini*. Apple documents the secure boot features in the T2 Security Overview document[12], but a series of articles by Mikhail Davidov[13] and his BlackHat 2019[14] provides more technical detail. Also from BlackHat 2019, Apple's Ivan Krstić officially detailed T2's secure boot[15].

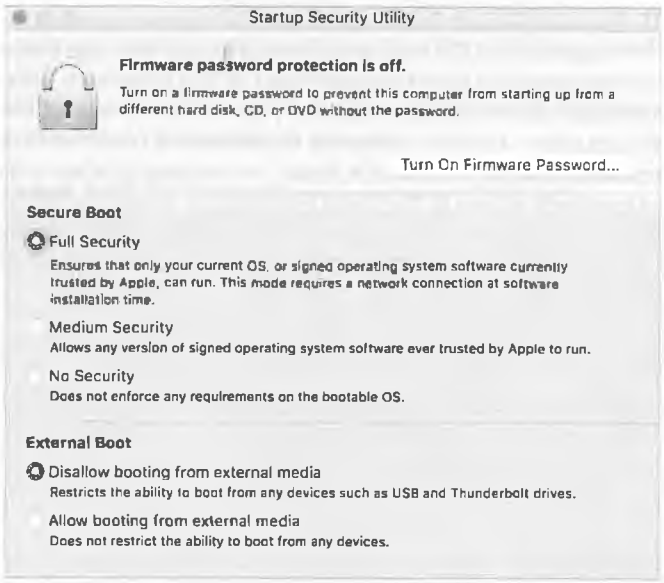
Secure Boot

BridgeOS dramatically improves the security of the boot process in MacOS by leveraging the previously discussed features found in the *OS boot sequence:

- **Tamperproof bootchain:** BridgeOS runs iBoot, and its attack surface is even smaller than that of the other *OS variants, since there is no direct USB access**. The BridgeOS boot chain is extended, so that it (through `MacEFIChecker`) verifies the signature of the UEFI firmware. Following said verification, UEFI is allowed to boot on the Intel processor, and it proceeds to verify `boot.efi`, and - finally - the MacOS immutable kernel image.
- **Secure Enclave:** The T2 also has a SEP component, which means that even if iBoot and/or BridgeOS are compromised, there is an extra hurdle to cross before the data decryption keys can be obtained.
- **APTickets:** are used for what Apple describes as "Personalization" of the boot process, enforcing the same Draconian restrictions known from *OS (and loathed by jailbreakers), to prevent downgrading or installation of any OS but ones currently deemed secure, latest and greatest by Apple.

Unlike *OS, the choice of security is given to the administrator (and settable for Managed Clients, as discussed in III/6). The "Startup Security Utility" (shown in Figure 5-21), available in Recovery Mode, augments the previously available Firmware Password utility, and adds control over the secure boot sequence, as well.

Figure 5-21: The startup security utility



"Full Security" means using an APTicket - which must be generated with the help of Apple's TSS server, and thus "requires a network connection at software installation time". "Medium" allows EFI launchers which are signed by Apple (i.e., standard `boot.efi` Boot Camp).

---

* - Lamentably, the 2016 MacBook Pro is not in any way upgradeable and cannot support secure boot, leaving the T1 useful only for controlling the Touch Bar, and rendering that year's models effectively obsolete.
** - Note, that "smaller" does not equate "nil". iBoot and SEPOS are both software systems, and still process complex inputs, such as filesystem structures, and DER encoded certificates. BridgeOS itself is also accessible over USB-NCM.

SMC BIOS emulation

The SMC BIOS of T2 devices is no longer a static component, but is now provided by the T2 chip. Using the `smut(j)` tool on a T2 enabled device shows that the BIOS vendor and version report those of iBridge:

Output 5-22: The output of `smut(j)` showing T2/iBridge emulated SMC BIOS

```
..
0x294:BIOS Information:
        Vendor: Apple Inc.
        Version: 220.240.2.0.0 (iBridge: 16,16,3131.0.0,0)
        Release Date: 01/02/2019
..
0x457:OEM Strings: (1 Strings)
        String 0: Apple ROM Version\r
                  BIOS ID:      AAPL3140F.88Z.F600.B00.1901022043\r
                  Mode:         AAPL3140F\r
                  EFI Version:  220.240.2.0.0\r
                  Built by:     root@db177\r
                  Date:         Wed Jan  2 20:43:24 2019\r
                  Revision:     220.240.2 (B&I)\r
                  ROM Version:  F600_B00\r
                  Build Type:   Official Build, RELEASE\r
                  Compiler:     Apple LLVM version 10.0.0 (clang-1000.2.42)\r
```

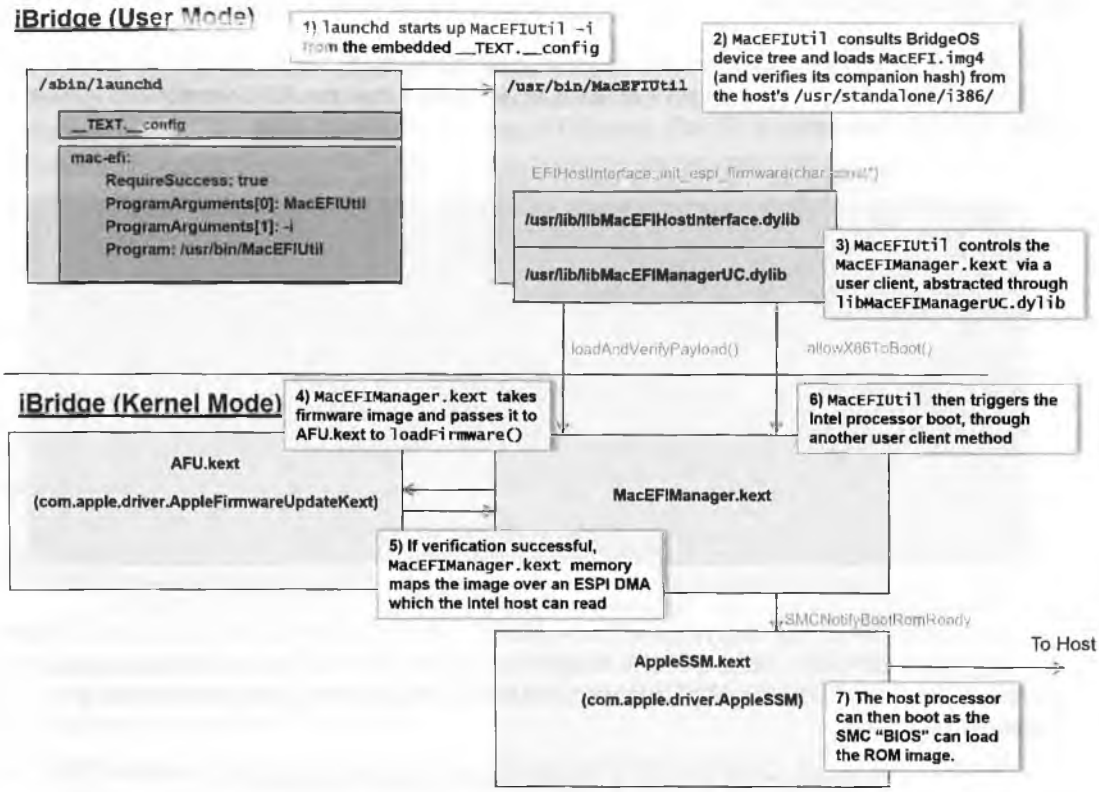
The very same ROM strings are found in the BridgeOS .ipsw, in BridgeOS's MacEFI/*boardid*.RELEASE.im4p files, implying the entire BIOS is, in fact, a software image and not a ROM, meaning Apple can now update the SMC image freely when the need arises. Using `imgtool(j)` on one of the many board-specific variants corroborates this, showing that they contain a complete EFI image:

Listing 5-23: Analyzing a BridgeOS MacEFI image using `imgtool(j)`

```
morpheus@Chimera (~)$ imgtool ~/Downloads/BridgeOS4.0/Firmware/MacEFI/J680.RELEASE.im4p
4 entries:
        pdr_: @0x1000   (0x1000 bytes)
        IFWI: @0x3000   (0x47000 bytes)
        nvar: @0x15b050 (0x2efb0 bytes)
        mbox: @0x18a000 (0x1000 bytes)
Size: 212000, tag: 4856465f, attr: 47eff, checksum:443c, version: 2, blockSize: 0x1000, blockCount:0x212
Next GUID@0x18b163 FFFFFFFF-FFFF-FFFF-FFFF-FFFFFFFFFFFF (34 bytes, type Padding, attr 0)
Next GUID@0x18b19b 5E7BE016-33CF-2D42-8758-C69FA5CDBB2F (3778 bytes, type Freeform, attr 0)
        Section @0x18b1b3 Type: Raw, Size: 0x3760
Next GUID@0x18e913 D6A2CB7F-6A18-4E2F-B43B-9920A733700A (fd46 bytes, type Driver eXecution Environment Core)
        Section @0x18e92b Type: Freeform, Size: 0xfd2e
Next GUID@0x19e65b 2AD3DE79-63E9-9B4F-B64F-E7C6181B0CEC (138bla bytes, type Firmware Volume Image, attr 0)
        Section @0x19e673 Type: Pre-EFI Initialization Dependencies, Size: 0x6
        Section @0x19e67b Type: Freeform, Size: 0x138afa
Next GUID@0x2d717b AAPL ROM Info (17f bytes, type Freeform, attr 0)
        Section @0x2d7193 Type: Raw, Size: 0x167
Next GUID@0x2d72fb EFI BIOS (9e bytes, type Freeform, attr 0)
        Section @0x2d7313 Type: Raw, Size: 0x86
Size: 2a7000, tag: 4856465f, attr: 47eff, checksum:4515, version: 2, blockSize: 0x1000, blockCount:0x2a7
Next GUID@0x39d163 11A8DD1E-4F83-7344-8F4F-B8722D541E26 (18ef95 bytes, type Firmware Volume Image, attr 0)
        Section @0x39d17b Type: Freeform, Size: 0x18ef7d
Next GUID@0x52c0fb 32F8943E-76FF-5083-9658-6E5E2C5AFEC4 (cc184 bytes, type Firmware Volume Image, attr 0)
        Section @0x52c113 Type: Freeform, Size: 0xcc16c
Next GUID@0x5f8283 AAPL ROM Info (17f bytes, type Freeform, attr 0)
        Section @0x5f829b Type: Raw, Size: 0x167
Next GUID@0x5f8403 EFI BIOS (9e bytes, type Freeform, attr 0)
        Section @0x5f841b Type: Raw, Size: 0x86
Size: 1bc000, tag: 4856465f, attr: 77eff, checksum:a495, version: 2, blockSize: 0x1000, blockCount:0x1bc
Next GUID@0x644163 FFFFFFFF-FFFF-FFFF-FFFF-FFFFFFFFFFFF (34 bytes, type Padding, attr 0)
Next GUID@0x64419b A Priori Pre-EFI Initialization (PEI) (7c bytes, type Freeform, attr 0)
        Section @0x6441b3 Type: Raw, Size: 0x64
..
```

Reversing BridgeOS's relevant components reveals the full process, shown in Figure 5-24, which follows the flow from the BridgeOS side:

Figure 5-24: The process of loading the MacEFI firmware image onto the Intel processor



Note the use of .img4 file for the EFI image - The boot image is now "personalized" per device, with the stock .img4 further paired to an accompanying .im4m file. This, like the *OS APTicket, is obtained from Apple's servers and stored in /usr/standalone/i386. Apple seems happy to provide .im4m for all T2-enabled device versions, not just the variant in the particular Mac, though only a specific one (to which the T2's ECID is appended) is personalized:

Output 5-25: The EFI file in /usr/standalone/i386

```
morpheus@Bifrost% ls -sb /usr/standalone/i386/
total 33128
    0 EfiLoginUI
30728 Firmware.scap
    0 SecureBoot.bundle
 1176 apfs.efi
 1160 boot.efi
#
# Personalization files: Actual device is the j140kap
#
    8 boot.efi,j132ap.im4m
    8 boot.efi,j137ap.im4m
    8 boot.efi,j140kap.22DE0018A0026.im4m
    8 boot.efi,j140kap.im4m
    8 boot.efi,j174ap.im4m
    8 boot.efi,j680ap.im4m
    8 boot.efi,j780ap.im4m
    8 boot.efi,x589amluhev.im4m
```

Another important implication of using T2 to provide SMC services is Apple's newfound capability to enforce platform specific code encryption. Presently, MacOS's code encryption is rather feeble, relying on Dont Steal MacOS.kext using a hard-coded (and visible) plaintext secret in the SMC BIOS (as detailed in Chapter 11 under the "Apple Protect Pager" discussion). With the SMC BIOS now emulated, Apple can stash the MacOS Mach-O segment protection key in the T2's secure enclave, which would make it irretrievable by Hackintosh enthusiasts who dare defile MacOS on generic Intel hardware. In fact, this could be extended even further than platform level, down to machine-specific encryption.

Storage Encryption

Another important feature provided by the T2 is support for full disk ("FileVault") and file (class)-based ("`NSFileProtectionClass`"). Apple achieves this by positioning the T2 in between the Intel CPU and the NAND, so that the former cannot access the latter without going through the T2 chip. The T2, in turn, will refuse decryption of the media without the user's passcode. This is specifically aimed to prevent "cold boot" attacks, in which an adversary gains possession of the device or desolders the NAND, so as to launch brute force and possibly parallelized attacks. The T2 enforces a hard limit of 30 incorrect password attempts per mechanism (password, FileVault recovery, iCloud recovery, or institutional key), with an induced delay. If all of these are exhausted, the enclave effaces the keys permanently.

The beauty of this mechanism is that the T2 enforced encryption is effectively transparent when the device is unlocked. MacOS sees nothing of it, and access to the raw block device retrieves unencrypted data. All encryption and decryption occurs on the T2's secure enclave, which means that even a MacOS or BridgeOS kernel compromise will not yield access to the keys. There is, however, a possible Achilles' Heel - malicious code running on the Mac (or iDevice, for that matter) can access the plaintext data - even higher class keys, when the device is unlocked. That said, there is no reconciling this without impacting usability. MacOS's datavaulting (discussed in III/9) provides the final layer of protection, restricting access to data only to those processes which possess specific entitlements.

Other T2 operational aspects

The T2 chip provides other important security features, outside the realm of boot and storage. The FaceTime Camera and microphone are both under its control, and not of the Mac's own Intel processor. This means that any access to these important devices must be routed through the T2, and its executing BridgeOS. The BridgeOS image contains a few specific daemons which use RemoteXPC with the host MacOS to transparently provide access to the devices:

Table 5-26: BridgeOS XPC services

com.apple..	Service
`.corespeech.xpc.remote.[control/record]`	Speech/audio (Siri)
`.powerchime.remote`	Responsible for boot and power-cable connect audio effects
`.aveservice`	Video Encoding
`.avvc.xpc.remote`	Used by FaceTime

From the MacOS side, special kernel extensions are employed as well:

- AppleUSBiBridge.kext: Used for older iBridge 1,1 devices
- AppleBridgeAudioController and AppleBridgeAudioCommunication: Handles submittin audio buffers to BridgeOS, over shared memory buffers. The `IOBufferCopyEngine` kernel extension provides low level memory support

Kernel Boot Process

When XNU is compiled, the end result is the rather large Mach-O file. which ends up in MacOS's /System/Library/Kernels, and gets further prelinked with kernel extensions to create the kernelcache (discussed in Chapter 3). The `LC_UNIXTHREAD` load command of this Mach-O provides the designated entry point to the kernel, to which the boot loader - EFI or iBoot - will transfer control.

The `struct boot_args`

The kernel receives a single argument, the `struct boot_args`, an architecture dependent structure (defined in pexpert/pexpert/[i386/arm]/boot.h), through which it will figure out the device tree, command line and more, assuming control of all resources, and bringing up the system. The `struct boot_args` is not to be confused with the `boot-args` string (parsed by `PE_parse_boot_argn()`, etc), although the string pointer is the `CommandLine` member of the larger structure. Table 5-27 compares the structure in between the two Darwin 19 flavors. Note the fields appear in the MacOS order, and the ones also present in ARM are ordered differently in the structure.

Table 5-27: Comparing the Intel and ARM `boot_args` structures

Intel	ARM[64]	Holds
`Revision`		Currently, 2
`Version`		Currently, 0 in MacOS, 2 in *OS
`efiMode`		A `uint8_t` holding EFI-bitness (64)
`debugMode`		A `uint8_t` holding debug flags (0)
`flags`	`bootFlags`	`uint16_t` (MacOS)/`uint64_t` (*OS) holding boot flags
`CommandLine`		`BOOT_LINE_LENGTH` (1024/256 byte) kernel command line
`MemoryMap[/Size]`		Physical address of memory map descriptors
`MemoryMapDescriptor[Size/Version]`		EFI memory descriptor size and version
`VideoV1`		Legacy video information
`deviceTree[P/Length]`		Flattened device tree pointer and length
`kaddr`	`physBase`	Physical address of kernel text
	`virtBase`	Virtual address of kernel text
`ksize`		Size of combined kernel text+data+efi
	`topOfKernelData`	Highest physical address of kernel data area
`kslide`		KASLR slide
`efi*`		EFI runtime services physical/virtual, `efiSystemTable`
`performanceData[Start/Size]`		Used by `IOHibernate`
`keyStoreData[Start/Size]`		Used by `IOKeyStoreHelper`
`bootMem[Start/Size]`		
`PhysicalMemorySize`	`memSize[Actual]`	Physical RAM available
`FSBFrequency`		Front Side Bus frequency
`pciConfigSpace*`		PCI configuration space Base address, start and end bus
`csr[ActiveConfig/Capabilities]`		MacOS 10.10: SIP Settings (III/9)
`boot_SMC_plimit`		
`bootProgressMeter[Start/End]`		MacOS 11: `uint16_t`s holding progress bar
	`machineType`	iDevice type
`Video`		A `Boot_Video` structure holding video information
`apfsData[Start/Size]`		MacOS 14: APFS volume structure (q.v. Chapter 8)
`__reserved4`		710 `uint32_t` padding the structure to a 4k page

Experiment: Displaying the `struct boot_args` from memory Exper/

MacOS's DTrace facility can not only read kernel memory, but (thanks to the kernel's `__CTF.__ctf` compact type form section) is also symbol aware. The `boot_args` structure pointer can be found from the Platform Expert's `_PE_state` symbol. The simple D program shown in Listing 5-28 can be used to print out the boot argument structure on systems wherein SIP has been disabled:

Listing 5-28: A D program to display the `struct boot_args` of MacOS

```
#!/usr/sbin/dtrace -s
#pragma D option quiet
BEGIN {
   self->boot_args = ((struct boot_args*)(`PE_state).bootArgs);
   self->deviceTreeHead = ((struct boot_args*)(`PE_state).deviceTreeHead);
   self->video = ((PE_Video ) (`PE_state).video);

   printf("EFI: %d-bit\n", self>boot_args->efiMode);
   printf("Video: Base Addr: %p\n", self->video.v_baseAddr);
   printf("Video: %s\n", (self->video.v_display == 1 ? "Graphics":"Text"));

   /** Complete this with fields from pexpert/pexpert/i386/boot.h **/
   ....
   exit(0);
}
```

Output 5-29 shows a raw dump of the `boot_args` structure, partially annotated:

Output 5-29: An annotated dump of the MacOS `boot-args`

```
                  Rev   Ver  efi   flags          CommandLine
0xffffff80114ca000 00 00 02 00 40 00 49 01   00 00 00 00 00 00 00 00   ....@.I.........
...          (CommandLine spans 1024 bytes)  ....
0xffffff80114ca400 00 00 00 00 00 00 00 00    0x11595000    0x25b0      .........PY..%..
0xffffff80114ca410 30 00 00 00 01 00 00 00   00 00 00 a0 01 00 00 00   0...............
0xffffff80114ca420 00 18 00 00 a0 05 00 00   84 03 00 00 20 00 00 00   ................
                  DeviceTreeP   DTLength       kaddr        ksize
0xffffff80114ca430  0x114cf000    0x15d4       0xe7000    0x02e98000   ..L.........p...
0xffffff80114ca440 d1 14 01 00 c4 00 00 00       0x000fffffff80114d1   ................
                efiSystemTable   kslide
0xffffff80114ca450  0x11526e10  0x0e600000   00 00 00 00 00 00 00 00   .nR.`...........
0xffffff80114ca460 00 e0 4c 11 48 00 00 00   00 00 00 00 00 00 00 00   ..L.H...........
0xffffff80114ca470 00 00 00 00 00 00 00 00   00 00 00 00 02 00 00 00   ................
0xffffff80114ca480 00 e1 f5 05 00 00 00 00   00 00 00 e0 00 00 00 00   ................
0xffffff80114ca490 00 00 00 00 9b 00 00 00   67 00 00 00 00 00 00 00   ........g.......
0xffffff80114ca4a0 00 00 00 00 dc ba a8 c7   01 00 00 00 00 18 00 00   ................
0xffffff80114ca4b0 a0 05 00 00 84 03 00 00   20 00 00 00 00 00 00 00   ........ .......
0xffffff80114ca4c0 00 00 00 00 00 00 00 00   00 00 00 00 00 00 00 00   ................
0xffffff80114ca4d0 00 00 00 00 00 00 00 00   00 00 00 a0 00 00 00 00   ................
```

DTrace has been unfortunately (but understandably) compiled out of *OS variants, but `jtool2`'s `joker` module can easily locate the address of the structure in kernel memory. Assuming a jailbreak like LiberiOS enables you to read kernel memory, you can apply the kernel slide value onto this address and dump the structure in its raw form:

Output 5-30: Dumping the `struct boot_args` of iOS

```
Xn00p> dump _PE_state
Initialized: 1
Video: 640x1136x32
Device Tree Head: 0xffffffff00fd01000
BootArgs: 0xffffffff00fd00000
        Revision: 2, Version: 2
        Virtual Base: 0xffffffff00de00000 Physical Base: 0x800800000
        MemSize: 0x3e400000 (0x0 actual)
        DeviceTree: 0xffffffff00fd01000, 104508 bytes
        Command Line:
        Boot Flags: 0x0
Xn00p> dump 0xffffffff00fd00000 raw
                  Revision/Version                    virtBase
0xffffffff00fd00000   02 00 02 00 00 00 00 00      0xffffffff00de00000
                        physBase                      memSize
0xffffffff00fd00010   00 00 80 00 08 00 00 00   00 00 40 3e 00 00 00 00   ..........@>....
                     topOfKernelData                  Video
0xffffffff00fd00020   00 c0 71 02 08 00 00 00   00 00 c0 3e 08 00 00 00   ..q............>....
0xffffffff00fd00030   01 00 00 00 00 00 00 00   00 0a 00 00 00 00 00 00   ................
                       640                         1136
0xffffffff00fd00040   80 02 00 00 00 00 00 00   70 04 00 00 00 00 00 00   ........p.......
                       32
0xffffffff00fd00050   20 00 01 00 00 00 00 00   00 00 00 00 00 00 00 00    ...............
                        DeviceTreeP                 deviceTreeLength
0xffffffff00fd00060      0xffffffff00fd01000      3c 98 01 00 20 00 00 00   ........<... ...
                        CommandLine [BOOT_LINE_LENGTH]
0xffffffff00fd00080   00 00 00 00 00 00 00 00   00 00 00 00 00 00 00 00   ................
                      ..
```

x86_64: _start → _vstart

Although the 32-bit architecture is a relic of the past millenium, Intel processors still boot into 32-bit mode, with only one processor (or core) active, referred to as the Master, or the BootStrapping Processor (BSP). The kernel's entry point (`_start`, in osfmk/x86_64/start.s) therefore must therefore switch to 64-bit mode, and start up the other cores (or "slave" processors, discussed later). The `_start` entry point is in the `__HIB.__text`, which is also used when resuming from hibernation.

The very first thing `_start` does is save the `struct boot_args` it receives in `%rax` to `%rdi`. It then sets up segmentation using the `LGDTL` opcode, and rebases the boot page tables to point to the kernel's base address. With this complete, it is safe to switch to 64-bit mode, which is performed by calling the `SWITCH_TO_64BIT_MODE` macro at the `L_pstart_common` label. The macro (`#defined` in start.s) toggles Physical Address Extensions in CR4, loads the kernel page tables into CR3, and enables paging and protected mode (`CR0_PG|CR0_WP`) in CR0.

The `SWITCH_TO_64_BITMODE` is also where slave processor startup funnels to (hence the `L_pstart_common` label). The master processor can be distinguished from the slaves because its `%rdi` is non-zero (holding the `struct boot_args` pointer). Ivy Bridge and later processors support the `RDRAND` instruction, which (if determined to be available by the 30th bit of `%ecx` following `CPUID` with `%eax = 1`) is used to populate the stack canary value. All processors then end up executing `vstartshim`, which aligns the stack and calls `_vstart`.

The `_vstart` function (implemented in osfmk/i386/i386_init.c) is a C function, taking the `struct boot_args` (or NULL, on slave processors) as its only argument. In the initial code path (of the boot CPU) the boot args are provided to `PE_init_platform()`, which load them into the Platform Expert's `PE_state`, initializes the device tree and also parses debug boot-args (`debug` and `assertions`). Initialization then continues with the machine specific `i386_init()` through the `x86_init_wrapper`, which uses the per-CPU interrupt stack.

ARM64: _start → start_first_cpu

The entry point of the kernel is a symbol called `start`, which immediately jumps to `start_first_cpu`. This assembly code is responsible for performing the operations of cold booting the primary core. The code is easily recognizable due to its first instruction, which zeroes `OSLAR_EL1`. It then proceeds to disable interrupts, save the `struct boot_args`, and the low reset exception vector to `LowExceptionVectorBase`, which halts on every exception.

The next step is to determine the kernel's virtual and physical address mapping. Looking back at Table 5-26 note that the `boot_args` structure contains the kernel virtual base, physical base, memory size, and top of kernel data. These get loaded (into X22, X23, X24 and X25, respectively), so they can be used to easily convert from physical addresses (which are what the CPU initially uses) to virtual ones. The conversion is simple: Add the virtual base (X22) and subtract the physical (X23). Using this method, the start routine sets up the pointers to the exception stack, interrupt stack, initialization function (`arm_init()`, loaded into the Link Register), and set up the bootstrap page table mappings.

Although `start_first_cpu` open sourced (implemented in osfmk/arm64/start.s), Apple redacts A12 and later portions pertaining to the use of APRR, which provides low-level hardware-level physical memory protection. Listing 5-31 shows most of the disassembly of `start_first_cpu`, interleaved with the start.s comments and macro expansions, with the A12 and later portions emphasized.

The flow of `start_first_cpu` then moves on to `common_start`, thus called because this is where the flow of additional cores ("slave processors", started at `start_cpu`) merges. The `common_start` sets up the Translation Control Register (TCR_EL1) and the kernel page tables in the Translation Table Base Register (`TTBR_EL1`). As of the A10, setting the registers using KTRR "protected instructions" in `__LAST.__pinst`). In between the registers, a call to `set_mmu_ttb` (from osfmk/arm64/machine_routines_asm.s) sets the kernel space ASID in `TTBR0_EL1`, and `common_start` then sets the Memory Attribute Index Register (MAIR_EL1). The permanent exception vector may then be set up (as was shown in Listing 4-24). The rest of the code is processor specific tunables and workarounds, and the flow then returns into the initialization function, which was loaded earlier into `LR`.

Listing 5-31: An annotated listing of `start_first_cpu` from an iPhone11 13β8 kernelcache

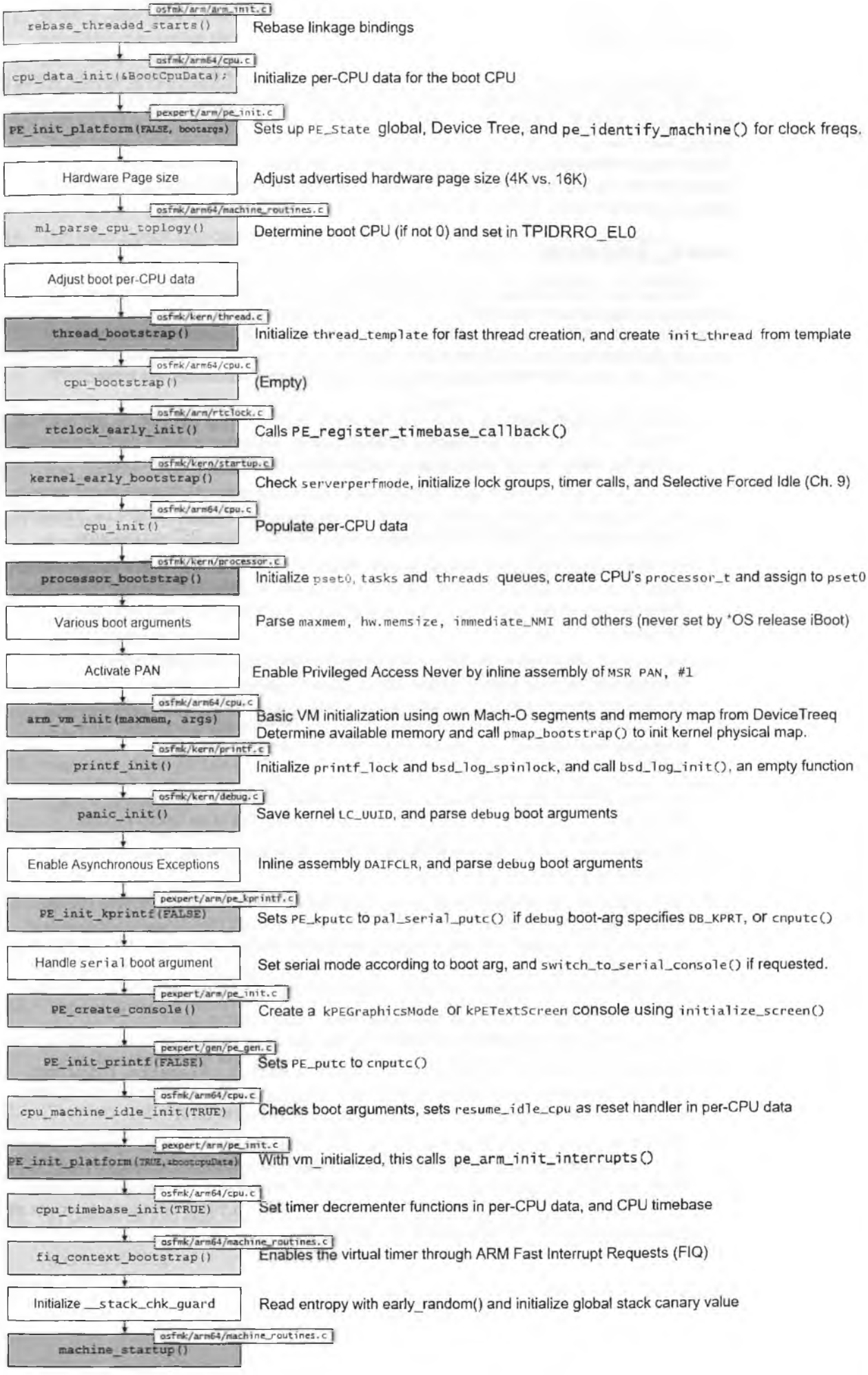
```
_start_first_cpu:
   // Unlock the core for debugging  /* yes, you read right. Debugging. */
   0xd510109f  MSR         OSLAR_EL1, XZR          X - (31 0x0)
   0xd5034fdf  MSR         DAIFSet, #15            X - (0 0x0)
   0xaa0003f4  _MOV_R      X20, X0                 R20 = R0 (0x0)
   0xd2800015  MOVZ        X21, 0x0                ; R21 = 0x0
   // Set low reset vector before attempting any loads
   0xb0ffffe0  ADRP        X0, 2097149             R0 = 0xfffffff008155000
   0x91000000  ADD         X0, X0, #0              R0 = R0 + 0x0 = 0xfffffff008155000
   0xd518c000  MSR         VBAR_EL1, X0            Exception Vector set to 0xfffffff008155000
#if defined(APPLEVORTEX)
   0xaa1e03e1  _MOV_R      X1, X30                 R1 = R30 (0x0)
   0xf2e88aa0  MOVKKKK     X0, 0x4455445564666677, LSL 48
   0x94000df7  BL          0xfffffff00815b878      _func_fffffff00815b878
        _func_fffffff00815b878(0x4455445564666677, LR);
   0xf2e8a8a0  MOVKKKK     X0, 0x4545010065670001
   0x94000de8  BL          0xfffffff00815b850      _func_fffffff00815b850
        _func_fffffff00815b850(0x4455010065670001);
   0xf2e00000  MOVKKKK     X0, 0x0, 0          R0 = 0
   0x94000df7  BL          0xfffffff00815b8a0      _func_fffffff00815b8a0
        _func_fffffff00815b8a0(0);
   0xf2e64641  MOVKKKK     X1, 0x3232767712107776
   0xd51cf2e1  MSR         S3_4_C15_C2_7, X1       S3_4_C15_C2_7 = 0x3232767712107776)
   0xaa0103fe  MOV         LR, X1                  LR = R1 (0x3232767712107776)
#endif
   // Get the kernel memory parameters from the boot args
   0xf9400696  _LDR        X22, [X20, #8]          ...R22 = *(R20 + 8) =
   0xf9400a97  _LDR        X23, [X20, #16]         ...R23 = *(R20 + 16) =
   0xf9400e98  _LDR        X24, [X20, #24]         ...R24 = *(R20 + 24) =
   0xf9401299  _LDR        X25, [X20, #32]         ...R25 = *(R20 + 32) =
   0xf9416a9a  _LDR        X26, [X20, #720]        ...R26 = *(R20 + 720) = *0x2d0
   // Clear the register that will be used to store the userspace thread pointer and CPU num
   0xd51bd075  MSR         TPIDRRO_EL0, X21        X - Thread Pointer/ID Register (21 0x0)
   // Set up exception stack pointer
   0x90009220  ADRP        X0, 4676                R0 = 0xfffffff00939c000
   0x91000000  ADD         X0, X0, #0              R0 = R0 + 0x0 = 0xfffffff00939c000
   0x8b160000  ADD         X0, X0, X22             R0 = R0 + 0x0 = 0xfffffff00939c000
   0xcb170000  SUB         X0, X0, X23             R0 = R0 + 0x0 = 0xfffffff00939c000
   0x94000dc8  BL          0xfffffff00815b828      _pinst_spsel_1
        __pinst_spsel_1(0xfffffff00939c000);
...
   /* Set up the bootstrap page tables with a single block entry for the V=P
    * mapping, a single block entry for the trampolined kernel address (KVA) ... */
   0xd2800000  MOVZ        X0, 0x0                  ; R0 = 0x0
   0xaa1903e1  _MOV_R      X1, X25                 R1 = R25 (0x1121d3620)
   0xd2810002  MOVZ        X2, 0x800                ; R2 = 0x800
   0xd37ef442  UBFX        X2, X2#61
Linvalidate_bootstrap:
   0xf8008420  STR         X0, [X1] #64            ;  #(1 << TTE_SHIFT)
   0xf1000442  SUBS        X2, X2, #1, {, shift}   ;  __ARM64_TWO_LEVEL_PMAP__
   0x54ffffc1  B.NE        0xfffffff008158148      ; Linvalidate_bootstrap
   // TZ0 stuff..
   0x90ff7560  ADRP        X0, 2092716             R0 = EXT(_mh_execute_header)@page
...
   0xaa1703ef  _MOV_R      X15, X23                R15 = R23 (0x10)
   // L1 Tables
   0xaa1903e1  _MOV_R      X1, X25                 R1 = R25 (0x0)
   0x91401023  ADD         X3, X1, #16384!!        R3 = R1 + 0x4000 = 0x4000 PGBYTES
   0xaa0303e2  _MOV_R      X2, X3                  R2 = R3 (0x4000)
   /* Setup the V=P bootstrap mapping: x5 - total number of L2 entries to allocate */
   0xd359ff05  LSR         X5, X24 #63                ; #(ARM_TT_L2_SHIFT)
   /* create_bootstrap_mapping(vbase, pbase, num_ents, L1 table, freeptr) */
   create_bootstrap_mapping x0,  x4,  x5, x1, x2, x6, x10, x11, x12, x13
   0x92672806  AND         X6, X0,  #0xffe000000 ;(ARM_TT_L2_INDEX_MASK)
..
   0x91401042  ADD         X2, X2, #16384!!        R2 = R2 + 0x4000 = PGBYTES
..
   0xd2810006  MOVZ        X6, 0x800              ; R6 = #(TTE_PGENTRIES)
   0x17ffffe3  B           0xfffffff008158198
        /* Setup the KVA bootstrap mapping */
   0xd359ff05  LSR         X5, X24#63     ; #(ARM_TT_L2_SHIFT)
        /* create_bootstrap_mapping x14, x15, x5, x3, x2, x9, x10, x11, x12, x13 */
   0x926729c9  AND         X9, X14, #0xffe000000 ; (ARM_TT_L2_INDEX_MASK)
..
   0xd2810009  MOVZ        X9, 0x800              ; R9 = #(TTE_PGENTRIES)
   0x17ffffe3  B           0xfffffff008158224
        /* Ensure TTEs are visible */
   0xd5033b9f  DSB         ISH
   0x14000001  B           0xfffffff0081582a4      common_start
```

i386_init() and arm_init()

At this point across all variants, the processor has moved to using virtual memory (albeit with no memory management from the kernel's side, yet). The architecture-dependent functions `[arm/i386]_init()` (from osfmk/[arm/i386]/[arm/i386]_init.c) continue the initialization process. Although their names imply 32-bitness, they are used in the 64-bit flavors as well.

As C functions, `[arm/i386]_init()` are easy to follow, and that is further helped by most of the initializtion being performed by other functions (though some are inlined in the assembly). Most of these have machine specific implementations, yet some of them provide the same prototypes, and others still (e.g. `kernel_early_bootstrap()`, from osfmk/kern/startup.c) are common code, though called in different order. Figure 5-32 shows `i386_init()` (left) contrasted with `arm_init()`, with different colors indicating machine specific (cyan or orange), same API (blue), or common code (green).

Figure 5-32: The flows of `i386_init()` (left) and `arm_init()` (right)


osfmk/arm/arm_init.c
rebase_threaded_starts()
Rebase linkage bindings
osfmk/arm64/cpu.c
cpu_data_init(&BootCpuData);
Initialize per-CPU data for the boot CPU
pexpert/arm/pe_init.c
PE_init_platform(FALSE, bootargs)
Sets up PE_State global, Device Tree, and pe_identify_machine() for clock freqs.
Hardware Page size
Adjust advertised hardware page size (4K vs. 16K)
osfmk/arm64/machine_routines.c
ml_parse_cpu_topology()
Determine boot CPU (if not 0) and set in TPIDRRO_EL0
Adjust boot per-CPU data
osfmk/kern/thread.c
thread_bootstrap()
Initialize thread_template for fast thread creation, and create init_thread from template
osfmk/arm64/cpu.c
cpu_bootstrap()
(Empty)
osfmk/arm/rtclock.c
rtclock_early_init()
Calls PE_register_timebase_callback()
osfmk/kern/startup.c
kernel_early_bootstrap()
Check serverperfmode, initialize lock groups, timer calls, and Selective Forced Idle (Ch. 9)
osfmk/arm64/cpu.c
cpu_init()
Populate per-CPU data
osfmk/kern/processor.c
processor_bootstrap()
Initialize pset0, tasks and threads queues, create CPU's processor_t and assign to pset0
Various boot arguments
Parse maxmem, hw.memsize, immediate_NMI and others (never set by *OS release iBoot)
Activate PAN
Enable Privileged Access Never by inline assembly of MSR PAN, #1
osfmk/arm64/cpu.c
arm_vm_init(maxmem, args)
Basic VM initialization using own Mach-O segments and memory map from DeviceTreeq
Determine available memory and call pmap_bootstrap() to init kernel physical map.
osfmk/kern/printf.c
printf_init()
Initialize printf_lock and bsd_log_spinlock, and call bsd_log_init(), an empty function
osfmk/kern/debug.c
panic_init()
Save kernel LC_UUID, and parse debug boot arguments
Enable Asynchronous Exceptions
Inline assembly DAIFCLR, and parse debug boot arguments
pexpert/arm/pe_kprintf.c
PE_init_kprintf(FALSE)
Sets PE_kputc to pal_serial_putc() if debug boot-arg specifies DB_KPRT, or cnputc()
Handle serial boot argument
Set serial mode according to boot arg, and switch_to_serial_console() if requested.
pexpert/arm/pe_init.c
PE_create_console()
Create a kPEGraphicsMode or kPETextScreen console using initialize_screen()
pexpert/gen/pe_gen.c
PE_init_printf(FALSE)
Sets PE_putc to cnputc()
osfmk/arm64/cpu.c
cpu_machine_idle_init(TRUE)
Checks boot arguments, sets resume_idle_cpu as reset handler in per-CPU data
pexpert/arm/pe_init.c
PE_init_platform(TRUE,&BootCpuData)
With vm_initialized, this calls pe_arm_init_interrupts()
osfmk/arm64/cpu.c
cpu_timebase_init(TRUE)
Set timer decrementer functions in per-CPU data, and CPU timebase
osfmk/arm64/machine_routines.c
fiq_context_bootstrap()
Enables the virtual timer through ARM Fast Interrupt Requests (FIQ)
Initialize __stack_chk_guard
Read entropy with early_random() and initialize global stack canary value
osfmk/arm64/machine_routines.c
machine_startup()

machine_startup

The machine_startup() routine handles the last portion of the machine-specific phase of boot. Its implementation is largely identical across all platorms, wherein it checks the assert and preempt boot args (and, in *OS, bg_preempt), then calls machine_conf() (a one liner setting machine_info.memory_size on all architectures). The x86_64 implementation checks several other boot arguments (unsafe, poll and yield) and also calls panic_hooks_init(), discussed earlier. All implementations end by transferring control to kernel_bootstrap(), which never returns.

kernel_bootstrap

The kernel_bootstrap() routine handles the final aspects of kernel low-level and architecture-independent initializations. The function (implemented in osfmk/kern/startup.c) is neatly designed, performing most of its work in subroutines, interleaved with calls to kernel_bootstrap_log() (implemented over kernel_debug_string_early() logging each state. On non-1469 *OS kernels, however, most of these subroutines are inlined.

1. The routine starts with a printf() of the kernel version string (on MacOS, but not *OS, in which printf() is a null macro). A check is then made over several boot arguments: -l (to log leaks), and several trace_* arguments, for early KDebug tracing.
2. scale_setup() is called next, to determine the scaling factor for various dynamic limits, as a function of available system memory (sane_size),and calls bsd_scale_setup() (from bsd/dev/unix_startup.c), to propagate the limits to various BSD subsystems.
3. vm_mem_bootstrap (from osfmk/vm/vm_init.c) is called to set up Mach's virtual memory subsystem. As explained later in Chapter 11, vm_mem_bootstrap() initializes the Mach pmap and vm_map layers, the zone allocator, kalloc zones (for arbitrary allocations), and pagers.
4. cs_init() (from bsd/kern/kern_cs.c) is called to determine the code signing enforcement settings. MacOS checks the cs_enforcement_disable, cs_enforcement_panic, cs_debug, cs_relax_platform_task_ports and cs_library_val_enable arguments, to set the corresponding variables (also accessible over sysctl(2)) accordingly. *OS compiles it all out, due to SECURE_KERNEL.
5. vm_mem_init() (from osfmk/vm/vm_init.c) is called. This falls through to vm_object_init() (from osfmk/vm/vm_object.c), but this is an empty routine.
6. The machine_info global is initialized. This is a structure holding memory_size, max_mem and version_[major/minor].
7. oslog_init() (from bsd/kern/subr_log.c) is called to initialize XNU's logging to Darwin's unified logging. This allocates the firehose_buffer, with firehose_chunk_count chunks and firehose_io_pages pages, as passed from boot arguments or initialized to default values. The user mode daemons logd and diagnosticd (both in /usr/libexec) can later connect to the buffer so as to process kernel messages into the unified log stream.
8. On CONFIG_KASAN enabled kernels (apparently, Apple Internal only), kasan_late_init is called to initialize its fake stack and globals.
9. If CONFIG_TELEMETRY is set (which it is, across all flavors), telemetry_init (from osfmk/kern/telemetry.c) to set up the telemetry subsystem. Introduced in Darwin 13, this enables task sampling and microstackshotting, as explained in I/15.
10. If CONFIG_CSR is set (on MacOS), csr_init() (from bsd/kern/kern_csr.c) determines the Configurable Software Restrictions, better known to the masses as System Integrity Protection (SIP). Specifically, if the Platform Expert's PE_state.bootArgs flags stated kBootArgsFlagCSRBoot, then csr_allow_all is set to TRUE, which short circuits all csr_check()s to effectively disable all SIP checks.

11. If `PE_i_can_has_debugger`, then the boot arguments of `-show_pointers` and `-no_slto_panic` are consulted, so as to not hide pointers in kernel message output, nor panic on spinlock timeouts (respectively).
12. `console_init()` (from osfmk/console/serial_console.c) sets up the console ring buffer. On Macs with a serial port, this buffer later gets flushed to the serial console, but not so on release versions of *OS.
13. `stackshot_init` (from osfmk/kern/kern_stackshot.c) is called to set up the `stackshot_subsys_mutex`, and possibly set the `max_tracebuf_size` according to the `stackshot_maxsz` boot argument.
14. The Mach thread scheduler is set up with a call to `sched_init()`. As explained in Chapter 10, this installs the hard-coded scheduler, and enables more threads to be created. Note that the scheduler is merely initialized, not actively started - the kernel is still in a single threaded state at this point.
15. `ltable_bootstrap()` (from osfmk/kern/ltable.c) bootstraps link tables, introduced in Darwin 16 to support wait queues. This is followed by a call to `waitq_bootstrap()` (from osfmk/kern/waitq.c). Wait queues are discussed in Chapter 10.
16. `ipc_bootstrap()` (from osfmk/ipc/ipc_init.c) creates the `ipc spaces`, `ipc ports`, `ipc port spaces`, and `ipc kmsgs`, which (as discussed in Chapter 11) support Mach IPC. It also creates two special spaces - the `ipc_space_kernel` and `ipc_space_reply`, and performs additional initializations of subsystem components needed to support the `kernel_task`.
17. If `CONFIG_MACF` (which is universally TRUE), `mac_policy_init()` is called to initialize the Mandatory Access Control Framework, detailed in III/4. This is a critical security infrastructure, so it is imperative to enable its function as early as possible, before any user affected code can be invoked.
18. `ipc_init()`: creates the `ipc_kernel_map` and `ipc_kernel_copy_map`, both submaps of the `kernel_map`. It also calls `ipc_host_init()` and `ux_handler_init()` to allocate the host special and exception ports.
19. The `PMAP_ACTIVATE_KERNEL()` macro (from osfmk/vm/pmap.h) is called with the `master_cpu`. This macro expands using another, `PMAP_ACTIVATE`, but since it is not `#define`, this is just ignored.
20. `mapping_free_prime()` is called to setup physical to virtual entry objects for the kernel free list. The implementation is machine specific (osfmk/i386/pmap_common.c and osfmk/arm/pmap.c).
21. `machine_init()` branches to more machine specific initializations residing in osfmk/i386/AT386 (for MacOS), and osfmk/arm[64]/machine_routines.c (for *OS). The *OS implementations call `debug_log_init()`, `clock_config` and (if debug is enabled) `pmap_map_globals()`. The MacOS one does more work, reallocating the per-CPU data, displaying the CPU identifcation, initializing EFI, SMP and FPU, as well as MTRR and PAT for the boot processor, before calling `pmap_lowmem_finalize()`.
22. `clock_init()` (from osfmk/kern/clock.c) sets up the Mach clock abstraction.
23. `ledger_init()` (from osfmk/kern/ledger.c) merely initializes the `ledger` lock group.
24. if `CONFIG_COALITIONS` is set (as it is, from Darwin 14) `coalitions_init()` (from osfmk/kern/coalition.c) sets the `coalitions` zone, lock group, and the two initial system coalitions (init and corpse).
25. The Mach task and thread subsystems are initialized, by calls to `task_init()` and `thread_init` (in osfmk/kern/task.c and ..thread.c).
26. The work queue subsystem is initialized through `workq_init()` (from bsd/pthread/pthread_workqueue.c)
27. In Darwin 18 and later, `turnstiles_init()` (from osfmk/kern/turnstile.c) is called to set up the turnstile subsystem. As explained in Chapter 10, this is a mechanism which helps optimize locking.
28. If `CONFIG_ATM` is set (which it is, across flavors), `atm_init()` (from osfmk/atm/atm.c) is called to set up the Activity Trace (resource) Manager.

29. `mach_init_activity_id()` (from osfmk/ipc/ipc_voucher.c) simply initializes the global `voucher_activity_id` to 1, to tag the kernel's activity. As explained in Chapter 10, this value monotonically increments as more activities are created.
30. `bank_init()` (from osfmk/bank/bank.c) initializes the bank subsystem, by creating two zones - `bank_task` and `bank_account`, their respective queues, and registering the bank manager with the voucher subsystem (explained in Chapter 10).
31. `ipc_pthread_priority_init()` (from osfmk/voucher/ipc_pthread_priority.c registers the `ipc_pthread_priority` with the voucher subsystem (explained ibid.)
32. `corpses_init()` (from osfmk/corpses/corpse.c) sets the paramters for the task corpse subsystem. As explained in I/15, corpses were introduced into Darwin 16 to improve crash reporting by maintaining the task's "cadaver" in-kernel (rather than as a core dump) for post mortem analysis. Parameters can be tweaked for the kernel's boot arguments, and include `-no_corpses` (disabling the feature), `corpse_for_fatal_memkill` (on Jetsam) and `exc_via_corpse_forking`.
33. `host_statistics_init()` (from osfmk/kern/host.c) sets the lock group and time window used to maintain the Mach host object statistics.
34. Finally, the rest of the bootstrap process is delegated to a newly created thread, created with `kernel_thread_create()` and set to run the `kernel_bootstrap_thread` routine. A call to `load_context()` overwrites the current register state, so `kernel_bootstrap` never returns.

`kernel_bootstrap_thread()`

Although the `kernel_bootstrap()` has ended at this point, the bootstrapping work is far from over. The `kernel_bootstrap_thread()` (also in osfmk/kern/startup.c) picks up at this point, and continues the (generally) subroutine based initialization of Mach subsystems, laced with `kernel_bootstrap_thread_log()` calls (aliased over `kprintf()`) as follows:

1. `idle_thread_create()` (from osfmk/kern/sched_prim.c) constructs the main processor's idle thread. This runs `idle_thread()` (ibid.), calling `process_idle()` to quiesce the CPU if possible. When `processor_idle()` returns, it will either select a new thread, or restart via a continuation.
2. `sched_startup()` (also in osfmk/kern/sched_prim.c) starts the scheduler services. As discussed in Chapter 9, this starts the `sched_maintenance_thread`, the scheduler routine book-keeping several times a second. Multi-threading is now enabled.
3. `thread_daemon_init()` (from osfmk/kern/thread.c) initializes various thread (and, as of Darwin 18, turnstile) queues, and spins the `thread_terminate_daemon`, `thread_stack_daemon` and `thread_exception_daemon` threads (see Table 5-35).
4. `vm_kernel_reserved_entry_init()` (from osfmk/vm/vm_map.c) the VM map reserved and holes zones (discussed in Chapter 12).
5. `thread_call_initialize()` (from osfmk/kern/thread_call.c sets up the lock groups and queues of the thread call subsystem, sets up the `thread_call_groups`, and starts the `thread_call_daemon`. As discussed in Chapter 10, thread calls provide a means for deferring work throughout the kernel.
6. A call to `thread_bind()` ensures that the bootstrap thread remains affined to the current (= master) processor/core, as additional ones are started.
7. On 64-bit *OS, if booting from a ramdisk (in the case of recovery/update) a call to `cpm_preallocate_early()` ensures enough contiguous physical memory for the ramdisk.
8. `ipc_thread_call_init()` (from osfmk/ipc/ipc_init.c) calls through to `ipc_importance_thread_call_init` (from osfmk/ipc/ipc_importance.c)
9. `mapping_adjust()` (from osfmk/i386/pmap_common.c or osfmk/arm/pmap.c) starts the `mapping_replenish` thread. The Intel implementation also sets up a pagetable corruption log.

10. `clock_service_create()` (from osfmk/kern/cloc.c starts the Mach clock service (which was initialized earlier in `kernel_bootstrap()`, step #21).
11. `device_service_create()` (from osfmk/device/device_init.c) allocates the `HOST_IO_MASTER_PORT`, which is used as the interface with the IORegistry.
12. On MacOS, `cpu_physwindow_init()` (from osfmk/i386/mp_desc.c) creates and initializes the physical copy window for the master CPU.
13. If `MACH_KDP` is true (as it is on MacOS), `kdp_init()` (from osfmk/kdp/kdp_udp.c) initialized KDP on the serial port, if required. There is a call to `alternate_debugger_init()`, contingent on `ALTERNATE_DEBUGGER`, which is false.
14. If Kernel Performance counting support has been compiled (`KPC`), `kpc_init()` (from bsd/kern/kern_kpc.c) sets up the lock group, and components of the subsystem through calls to `kpc_arch_init()`, `kpc_common_init()` and `kpc_thread_init()`.
15. If `CONFIG_ECC_LOGGING` is set (on MacOS), `ecc_log_init()` (from osfmk/kern/ecc_logging.c) sets the `ecc-data` lock group.
16. MacOS calls `hv_support_init()` (from osfmk/kern/hv_support.c) sets up the `hv_support` lock group, required by the hypervisor.
17. If `CONFIG_TELEMETRY` (true), `bootprofile_init()` (from osfmk/kern/telemetry.c) allocates a `bootprofile_buffer`, and registers a `bootprofile_timer_call_entry` according to defaults, which may be overridden by `bootprofile*` boot arguments.
18. MacOS kernels call `vmx_init()` (from osfmk/i386/vmx/vmx_cpu.c) sets up the lock groups for the Intel VMX facility.
19. The KTrace and KDebug subsystems are initialized, through calls to `[ktrace/kdebug]_init`. The latter is set up with the `trace_*` variables.
20. If `MACH_BSD` (which is, of course, always `TRUE`), `bsd_early_init()` (from bsd/kern/bsd_init.c), which directly calls `sysctl_early_init()` to start up the `sysctl(2)` facility (discussed in the next chapter).
21. At this point, ARM64 *OS calls `ml_lockdown_init()`. This merely sets up the `lockdown_handler` lock group, which protects the registration (from AppleT80xx.ext) and execution (from machine_lockdown()) of the handler.
22. If `IOKIT` (again, TRUE), `PE_init_iokit()` initializes IOKit through the Platform Expert. This initializes the GUI progress bar, and calls `StartIOKit()` (from iokit/Kernel/IOStartIOKit.cpp), which instantiates the `IOPlatformExpertDevice` and loads any kexts marked as `AppleKernelExternalComponent` (the `com.apple.kec.*` kexts). At this point, `early_boot_complete` can be set, which is checked on by the `os_log` support.
23. Finally, with the scheduler enabled and critical subsystems initialized, interrupts may be enabled. A call to `spllo()` does that.
24. MacOS: `cpu_userwindow_init(0)` (from osfmk/i386/mp_desc.c) sets up the CPU user map windows, which are saved in the per-CPU data (`cpu_copywindow_[base/pdp]`).
25. The shared region module is initialized. Through calls to `vm_shared_region_init()`, `vm_commpage_init()`, and `vm_commpage_text_init()` (in this order).
26. `mac_policy_initmach()` (security/mac_base.c) is called to preload kexts designated as `AppleSecurityExtensions`.
27. If the Vnode Guard is enabled (`CONFIG_VNGUARD`, which is, as of yet, FALSE), `vnguard_policy_init()` initializes the subsystem.
28. On MacOS, where `CONFIG_DTRACE` is set, `dtrace_early_init()` and `sdt_early_init()` are called (DTrace is discussed in Chapter 6).
29. `OSKextRemoveKextBootstrap` (from libkern/OSKextLib.cpp) jetissons segments used to bootstrap kext loading.

30. *OS devices, for which `CONFIG_KERNEL_INTEGRITY` is `#defined`, call `machine_lockdown_preflight()` (from osfmk/arm64/machine_routines.c) at this point. This is only implemented for `KERNEL_INTEGRITY_KTRR`, wherein it calls `rorgn_stash_range()` to set up the KTRR range.
31. All *OS devices call `arm_vm_prot_finalize()` (from osfmk/arm64/arm_vm_init.c) tightens permissions on kernel memory, and revokes the execution permissions of `__LAST.__pinst`.
32. If `CONFIG_SCHED_SFI`, `sfi_init()` initializes the Selective Forced Idle subsystem (explained in Chapter 10).
33. Multiple calls to `read_random()` set up the kernel address obfuscation values.
34. `vm_set_restrictions()` (osfmk/vm/vm_pageout.c) binds the `vm_pageout_scan`, compressor and memorystatus threads to a single core when less than four cores are available.
35. A conditional of `CONFIG_POST` compiles in XNU's Power On Self Tests, but this is apparently unused outside Apple's internal builds.
36. In MacOS, XNU CPU Power Management (XPCM) is initialized, through a call to `xcpm_early_init()`. This can't be found in the open sources, however, since `CONFIG_XCPM` is redacted.
37. The most important of all subsystems - BSD - can now be initialized at last. `MACH_BSD` is true, and therefore the call to `bsd_init()` is made here. This is a huge function, explained in the next chapter.
38. As of Darwin 19, `_x86_64_protect_data_const()` is called to prevent modification of the kernel's `__DATA_CONST` segment.
39. `xcpm_init_complete()` is called to finalize XCPM (again, this is redacted).
40. `kdebug_free_early_buf()` can be called to free the buffers used to trace kernel startup. This inlines to a call to `ml_static_mfree()`.
41. The serial keyboard, if any, can be started with a call to `serial_keyboard_init()` (from osfmk/console/serial_general.c).
42. The `vm_page` queue (discussed in Chapter 11) is initialized with a call to `vm_page_init_local_q()`.
43. Finally, the bootstrap thread is unbound from the master processor, and "becomes" the page out daemon, through a call to `vm_pageout()` (from osfmk/vm/vm_pageout.c), which never returns.

SMP Considerations

It is rare to find true SMP Apple computers, as even the 2019 (cheese grater) Mac Pro uses a single processor with multiple cores. From XNU's perspective, however, cores and are equivalent to physical processors, and hyperthreading (SMT) is treated as logical ones (as can be seen in the output of `hostinfo(1)`). Additional cores (or processors) are considered **slave processors** - with the primary core (the one which was activated when the CPU was powered on) assuming the role of master, and referred to as the Boot CPU.

Recall (from I/12) that XNU provides the Mach processor abstraction. Thus, processors can be started and stopped using the object's routines, which are also accessible from user mode (via the <mach/processor.defs> subsystem). The object is machine agnostic, but its routines call down to machine dependent implementations: `processor_start()` calls `cpu_start()` (from osfmk/[i386/arm[64]]/cpu.c), and `processor_exit()` calls `processor_exit()`, which - despite the misleading name - is not part of the subsystem, and machine specific as well (osfmk/i386/pcb.c or osfmk/arm64/machine_routines.c).

When a slave core is activated for the first time (or woken up from reset) its startup differs from that of the master. After all, the master core is responsible for fully booting the kernel (or resuming, if waking up from hibernation or sleep) - but when a slave core starts up, the kernel is already active. The sequence also differs somewhat between x86_64 and ARM, as explained below. Both architectures eventually end up in `slave_main()` (from osfmk/kern/startup.c), which takes the `next_thread` from the `current_processor()` (or the `idle_thread`, if not thread is set), and calls `load_context()` to run it. At that point, the slave core is up and running some thread, and from here on it is up to the Mach scheduler (discussed in Chapter 10) to dispatch other threads to it.

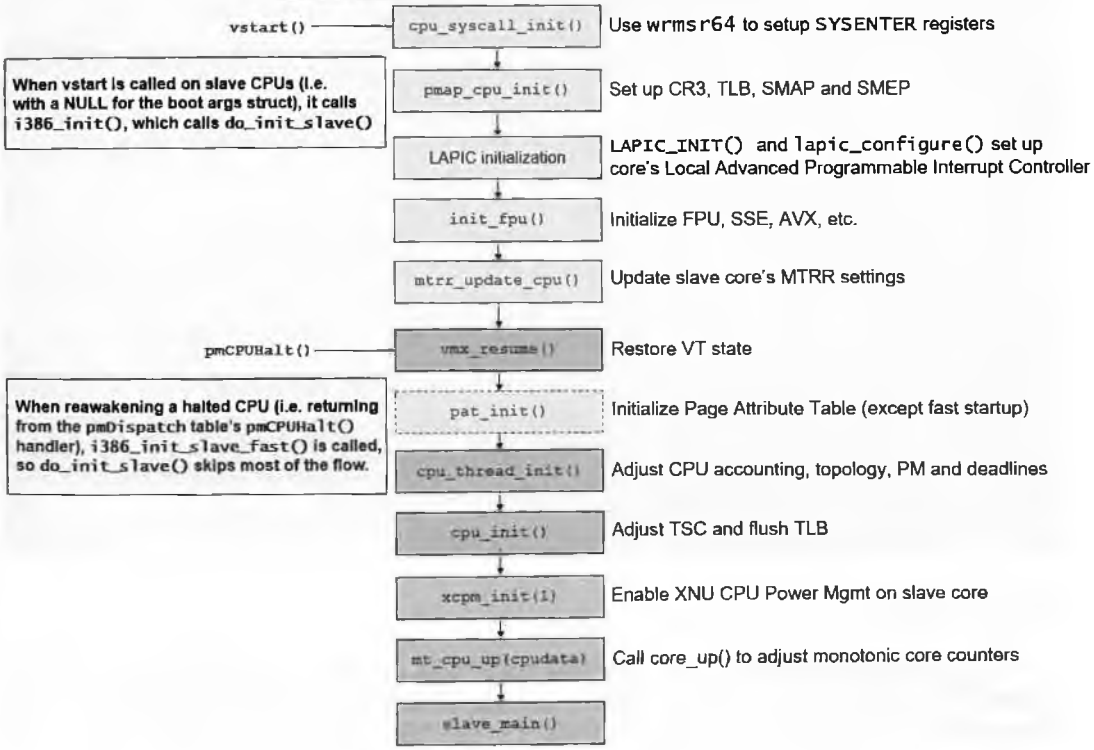
x86_64

Slave processor are also started through `vstart()`, dicussed earlier, but all they need do is switch from the boot page tables to those of the kernel (which the master processor will have set up by the time they started). The next stage of the flow on slaves is `i386_init_slave()`.

`i386_init_slave[_fast]()`

Initializing a slave is significantly simpler than a master CPU, since page tables and vital kernel subsystems are already up and running at this point. The flow of `i386_init_slave()` (actually, `do_init_slave()`, which isn't exported, but is immediately after in the assembly) shares many similarities with `i386_init()`, but is significantly simpler.

Figure 5-33: Slave processor startup on Intel platforms



The flow can be made even simpler if this is a `fast_restart`, as is the case when `i386_init_slave_fast()` is called from `pmCPUHalt()` when it returns, as shown above. In both flows, the end state is the call to `slave_main()`.

ARM

If `cpu_start()` (from osfmk/arm64/cpu.c) detects it is running on a different processor than the started one, it loads the physical address of `start_cpu` onto the `cpu_reset_handler` function pointer, in the per-CPU data, and ensures that the CPUData is associated with the `cpu_active_thread` which will be started. Oddly enough, it calls `configure_coresight_registers()` (ibid.), which can potentially enable ARMv8's CoreSight[16], a powerful hardware level feature enabling the debugging of one core from another (and is also performed when `cpu_start` runs on the same CPU).

In order for a slave processor to start, it must be enabled, but this is a board-specific operation. If `cpu_start()` needs to start another core, it thus calls on the Platform Expert's `PE_cpu_start()` (from iokit/Kernel/IOCPU.cpp) with the `cpu_id` from the per-CPU data. This, however, is not a numeric ID but an `IOCPU` instance, which is provided by a specialized kext - AppleARMPlatform.

Slave processors in *OS start through the `LowResetVectorBase` (in osfmk/arm64/start.s). Following a quick reset of the KTRR register (on A10+ devices), the local CPU number is obtained from `MPIDR_EL1`, and the CPU's local data (`CPU_DATA_PADDR`) is located in `CPU_DATA_ENTRIES`, which are at a fixed offset from the `LowResetVectorBase`.

The CPU local data contains the address of the `CPU_RESET_HANDLER` - but, as a sanity check, it is verified to be one of only two allowed values:

- `resume_idle_cpu()` - is called when an active, but idle core is reawakened. The address was placed in the CPU data when `cpu_machine_idle_init()` is called. This sets `arm_init_idle_cpu()` in LR.
- `start_cpu()` - starts a sleeping core. The address was placed in the CPU data when `cpu_sleep()` is called, from Mach's `processor_offline()` (osfmk/kern/machine.c). This sets `arm_init_cpu()` in LR.

Both cases now converge into `start_cpu`. When started, `X20` holds the `struct boot_args` physical address, from which the kernel physical and virtual addresses can be obtained - a necessary step for the target handler's physical address is to be converted to virtual. `X21` holds the per-CPU data, from which that CPU's exception and interrupt stacks can be set.

The converged path continues in `common_start`, which is also common to `start_first_cpu`, discussed earlier. Since `X21` holds per-CPU data set by `start_cpu`, however, the path may diverge again. The flow for slave CPUs thus branches to a trampoline, which converts the per-CPU data to virtual, loads it in the first argument (`X0`) and then returns to the original handler (still in `LR`) through a `RET` instruction*, which will hold either `arm_init_idle_cpu()` or `arm_init_cpu()` .

`arm_init_cpu()`

If a processor core came out of reset, `arm_init_cpu()` (from osfmk/arm/arm_init.c) is responsible or bringing the core up to speed, so it can service kernel threads. Somewhat analogous to `i386_init_slave()` and `i386_init()`, this flow is simpler than `arm_init()`'s, calling only a few of the initialization functions. Here, too, the flow can be further simplified when the processor is resumed back to an idle state: `arm_init_idle_cpu()` (also from osfmk/arm/arm_init.c) is called. It sets up PAN, the active thread, PMAP and MMU TLB flushing, `fiq_context_init()` and then a call to `cpu_idle_exit()`, through which the idle thread context will be loaded.

---

* - The comment for the `RET` at the end of the `arm_init_tramp` reads " `Return to arm_init()` ", but this is likely a cut/paste error from the non trampoline case (from `start_first_cpu`).

Kernel Threads

Once the init task has begun spawning its lineage, much of the action takes place in user mode, through the various processes and their threads. These occasionally cross the divide - through system calls - and operate in kernel mode. In addition, the `kernel_bootstrap_thread` continues to execute in kernel mode, along side numerous other worker threads spun by subsystems (using `kernel_thread_create` or `kernel_thread_start_[priority]`) and drivers (using the now deprecated `IOCreateThread` wrapper). The kernel threads are normally invisible from user mode, but can nonetheless be revealed using the proprietary system call `stack_snapshot_with_config` (#491), or through the `procexp(j)` tool, as shown in Output 5-34:

Output 5-34: The output of `procexp 0 threads`

```
root@Chimera (~)# procexp 0 threads
   PID : 0 (0), comm: kernel_task Flags: Shared Reg Unavail
   Size: 136M Max Res: 7126M
   IOStats: Disk Reads: 188064 reads (10440994816 bytes), 10202360 writes (92918127616 bytes)

        TID: 101 State: Waiting Uninterruptible  PRI: 91/91 Flags: (none)
        Continuation: 0xffffff8000464630 (no kernel stack)
        Thread Name: VM_pageout_scan
        CPU Times: User: 0.000000 secs, System: 127.917976 secs
        IOStats: Disk Reads: 508 reads (2080768 bytes), 0 writes (0 bytes)

        TID: 102 State: Running Idle  PRI: 0/0 Flags: 0x2
        Thread Name: idle #0
        CPU Times: User: 0.000000 secs, System: 891266.674814 secs
        Backtrace:
                Frame 0: 0xffffff80004da58a (machine_idle + 0x30a)
                Frame 1: 0xffffff80003cc077 (processor_idle + 0xf7)
                Frame 2: 0xffffff80003cc700 (idle_thread + 0x20)
                Frame 3: 0xffffff80003590ce (call_continuation + 0x2e)

        TID: 103 State: Waiting Uninterruptible  PRI: 95/95 Flags: (none)
        Continuation: 0xffffff80003c9af0 (no kernel stack)
        Thread Name: sched_maintenance_thread
        CPU Times: User: 0.000000 secs, System: 107.314860 secs

        TID: 104 State: Waiting Uninterruptible  PRI: 80/80 Flags: (none)
        Continuation: 0xffffff80003e2770 (no kernel stack)
        CPU Times: User: 0.000000 secs, System: 145.330449 secs

        TID: 105 State: Waiting Uninterruptible  PRI: 81/81 Flags: (none)
        Continuation: 0xffffff8000a41300 (__ZN10IOWorkLoop10threadMainEV)
        Thread Name: CoreAnalyticsHub

        ..
        TID: 116 State: Waiting Uninterruptible  PRI: 81/81 Flags: (none)
        Continuation: 0xffffff8000a41300 (__ZN10IOWorkLoop10threadMainEV)
        Thread Name: IOPMrootDomain

        ..

        TID: 132467 State: Waiting Uninterruptible  PRI: 81/81 Flags: (none)
        Continuation: 0xffffff8000a41300 (__ZN10IOWorkLoop10threadMainEV)
        Thread Name: AppleUSB20HubPort
```

As you can see in the output, there are two operational modes for kernel threads - those which have a traditional kernel stack, for which a backtrace can be provided, and those with **continuations**. The latter mode is a unique feature of Mach's, which allows kernel threads to quickly resume without the need to maintain a dedicated kernel stack, and is discussed in Chapter 10.

In between the subsystem threads and those of drivers, the kernel's thread count can easily exceed 300 or more. Table 5-35 details the threads started by the kernel proper, and you will encounter some of them in more detail later in these pages. Note, also, that not all threads have names - most do, and are named via `thread_set_thread_name()`. This useful KPI names the corresponding BSD `uthread`'s `pth_name`, through `bsd_setthreadname()`. Unfortunately, dozens have yet to be named (as indicated by parentheses in the table). Using `procexp`, however, the built-in symbolication will usually give you a good idea of what these anonymous threads are - the continuation address can be found in the kernelcache binary, and with the help of `jtool2 --analyze` you can deduce the symbol name. In the output above, for example, we have:

Table 5-35: The kernel threads in XNU 4903 (Darwin 18)

Thread name	Created by	Used for
(async_work_continue)	vntblinit()	VFS layer asynchronous worker
(thread_exception_daemon)	thread_daemon_init()	Deliver EXC_[GUARD|RESOURCE] exceptions
(thread_stack_daemon)		Allocate thread stacks
(thread_terminate_daemon)		Clean up terminating threads
(vm_object_reaper_thread)	vm_object_reaper_init()	Mach VM Object garbage collection
(io_reprioritize_thread)	vm_io_reprioritize_init()	IO Reprioritization (#if IOSCHED)
VM_compressor	vm_pageout_internal_start()	Handles swap to compressed RAM
VM_cswap_trigger	vm_compressor_init()	Handle memory compression triggers
VM_memorystatus	memorystatus_init()	Handles MemoryStatus/jetsam
VM_memorystatus_[1-3]	memorystatus_init()	Additional threads, as of Darwin 18
VM_pageout_scan	vm_pageout()	Page out queue
VM_pressure		Handles VM pressure triggers
VM_swapfile_create	vm_compressor_swap_init()	(Additional) Swap file creator
VM_swapfile_gc		Swap file garbage collector
VM_swapout		Pushes memory out to swapfile
dlil_input_[...]	dlil_create_input_thread()	One thread per every network interface
idle #[0-9]	idle_thread_create()	One thread per core, for CPU idle
ifnet_start_[...]	ifnet_attach()	One thread per every network interface
kperf sampling	pet_init()	Samples all tasks periodically
sched_maintenance_thread	sched_startup()	Thread scheduler periodic (q.v. Ch. 10)
thread_call_daemon	thread_call_initialize()	Handle Mach thread calls (q.v. Ch. 10)
Darwin 17		
thread call high #...	thread_call_thread_create()	Thread calls of different priority levels specified in the tcq_thread_pri member of the thread_call_groups's respective array entry (see Chapter 9)
thread call kernel #...		
thread call kernel-high #...		
thread call user #...		

Kernel Shutdown

Some server systems may maintain an indefinite uptime, but sooner or later most systems - especially desktops and mobile devices - are shutdown by the user. In those cases, the kernel has to perform an orderly process, in order to minimize the risk of data loss/corruption and ensure the system can be properly rebooted. As with the boot flow, there is therefore a shutdown flow.

In UN*X, shutdown is initiated from user mode, via the `reboot(2)` system call (usually invoked from the `reboot(8)`, `halt(8)` or `shutdown(8)` command line). The system call first checks the caller's credentials imply `root`, and defer to the MAC framework hook (`mac_system_check_reboot`). Both calls must agree for the reboot process to begin.

Recall, however, that XNU is a UN*X and Mach hybrid - and Mach has its own API. The `host_reboot()` MIG routine (from the host_priv subsystem) is a fast path to rebooting the system. The `reboot(2)` system call implementation calls on `host_reboot` as well, but invoking the latter directly from user mode completely bypasses the `reboot(2)` logic. The Mach `host_reboot` (osfmk/kern/machine.c) handles the machine level rebooting, calling `halt_all_cpus()` (from osfmk/*arch*/machine_routines.c) which calls on the Platform Expert (`PEHaltRestart()`) to either halt or restart the CPU.

The BSD layer chores of rebooting is performed by `reboot_kernel()` (bsd/kern/kern_shutdown.c)). This function proceeds according to the `RB_*` flags specified by the kernel. The `RB_QUICK` offers a fast path, otherwise the slow path first posts an `kIOSystemShutdownNotificationStageProcessExit`. Unless `RB_NOSYNC` was specified, the slow path also shuts down the process and audit subsystems, and interally `sync()`s all filesystems, and then unmounts them. All interfaces are downed, and the flow can then merge with Mach's `host_reboot()` for the processor-level shutdown.

Review Questions

1. Note in Output 5-15 that iBSS/iBEC are not part of the firmware boot partition. Why is that so? Where can they be found?
2. What simple, yet important security feature is missing from the components of the iOS boot chain?
3. A comment in `kernel_bootstrap_thread()` warns "not to stick anything else before" the call to `sched_startup()`. Why?
4. Why is the code for supporting and using the ARM CoreSight Registers in the ARM64 port of XNU unwise?

References

1. Intel - The EFI 1.10 Specification - https://www.intel.com/content/dam/doc/product-specification/efi-v1-10-specification.pdf"
2. Clover BootLoader
3. John Heasman - "Hacking the Extensible Firmware Interface" (BlackHat 2007) - https://blackhat.com/presentations/bh-usa-07/Heasman/Presentation/bh-usa-07-heasman.pdf
4. Snare - "De Mysteriis Dom Jobsivs: Mac EFI Rootkits" (BlackHat 2012) - https://www.youtube.com/watch?v=W21ZIaKf5HA
5. "ThunderStrike" https://trmm.net/Thunderstrike_31c3 and a video at https://www.youtube.com/watch?v=KFJn6cGKJQw
6. OSXReverser - "Apple EFI Firmware Passwords and the SCBO Myth" - https://reverse.put.as/2016/06/25/apple-efi-firmware-passwords-and-the-scbo-myth/
7. NewOSXBook.com - iBoot 13β keys - http://newosxbook.com/forum/viewtopic.php?f=7&t=19686
8. The iPhone Wiki - "DFU Mode" - https://www.theiphonewiki.com/wiki/DFU_Mode
9. USB Consortium - The USB DFU 1.1 standard - https://usb.org/sites/default/files/DFU_1.1.pdf
10. Lambda Concept - Bonobo Cable - http://lambdaconcept.com
11. axi0mX - "iPwnDFU" - https://github.com/axi0mX/ipwndfu/blob/master/checkm8.py
12. Apple - "T2 Security Overview" https://www.apple.com/mac/docs/Apple_T2_Security_Chip_Overview.pdf
13. Mikhail Davidov - "Secure Boot in the era of the T2" - https://duo.com/labs/research/secure-boot-in-the-era-of-the-t2
14. Mikhail Davidov - "Inside the Apple T2" (BlackHat 2019) - https://i.blackhat.com/USA-19/Thursday/us-19-Davidov-Inside-The-Apple-T2.pdf
15. Ivan Krstić - "Behind the Scenes of iOS Security" - BlackHat 2019
16. ARM CoreSight Specification - http://infocenter.arm.com/help/topic/com.arm.doc.ihi0029e/coresight_v3_0_architecture_specification_IHI0029E.pdf

6

BS"D: The BSD Layer

This chapter discusses the BSD layer, emphasizing the features common to Darwin and FreeBSD, at times pointing out their different implementations. We begin by walking through the startup process - the flow of `bsd_init()`, thereby getting acquainted with the numerous subsystems. Special attention is given to `launchd(8)`, owing to its special status as PID 1.

XNU depends on the BSD layer to supply all the objects which Mach, as a micro kernel, cannot - processes on top of tasks, richer threads on top of the Mach primitives, signals on top of exceptions, and - of course, the many types of files. A large part of our exploration of the BSD subsystem is thus dedicated to these objects, their implementation, and supporting KPIs. We begin with processes, looking through the `struct proc`. We next continue to `uthreads`, and discuss Apple's powerful (but relatively undocumented) Pthread support in kernel. This primarily revolves around **work queues**, which were introduced to support GCD.

After covering what we can of processes and uthreads, we turn to their lifecycle, in particular, taking behind the scenes look at `fork()`ing, `execve()`ing and/or `posix_spawn()`ing a file. This introduces XNU's `execsw`s and the inevitability of Mach-O. For terminating a process, we consider both the voluntary `exit(2)` and the involuntary death by signal.

From there, we move on to discuss file descriptors, and the underlying file objects. The Virtual Filesystem Switch discussion is left for its own chapter, but the basic tenets - `filedesc`s, `fileproc`s, `fileglob`s and the `uio` driven operations - are discussed here. The file discussion continues by exploring some of the special file types - Posix shared memory, kqueues, and pipes.

Last are some "odds and ends". Specifically, BSD memory allocation, and the rich `sysctl(2)` interface - KPIs which no doubt no kernel extension developer should be without. Finally - last but far, far from least - is the long overdue kernel-side discussion of the fabulous DTrace facility (discussed in I/15), with all its power (on MacOS, when SIP is disabled) to run probes into any process in the system, or the very kernel itself.

A Tour of BSD

The Mach microkernel at the heart of XNU is undoubtedly powerful and capable, and what functionality it does not offer directly through its traps, it provides through its own MIG subsystems, or those of user mode daemons. All this, however, still does not a full fledged operating system make. An operating system must support filesystems, networking, and various other objects and subsystems, preferably with an easy to use API.

Mach was designed as a proprietary microkernel in an age when many operating systems were converging around the emerging POSIX standard of UNIX-based operating systems. NeXT wanted to conform to it as well, which required adding another layer, on top of Mach, for the POSIX compatible APIs. Rather than implement something from scratch, the choice was made to adopt FreeBSD implementation. The code base at the time was more mature than that of the budding Linux operating system, and its completely free licensing was preferable to the somewhat limiting GNU Public License (GPL).

This BSD layer exists in modern-day XNU to this day. The bsd/ subdirectory of the XNU source tree, still closely resembles FreeBSD 6.0. Some of those shared responsibilities include all POSIX syscalls (primarily those of process management, pthread support, file management and I/O), as well as some BSD specific extensions such as `kqueues` (I/8), and Solaris inspired code, namely the BSM auditing subsystem (discussed in III/2) and the fabulous DTrace (I/15 and this chapter).

In the meantime, however, FreeBSD has evolved from to the present-day 11.0, and Apple has continuously added new functionality and proprietary system calls as well. As a result, the BSD layer has expanded in size and complexity, and diverged from its sibling. Although the differences are notable, the commonalities are significant enough that the interested reader will benefit from reading the definitive BSD bible, by McKusick, Neville-Neil and Watson[1], the legendary figures who wrote parts of BSD's core, and whose code still lives in XNU.

Darwin, too, kept evolving with more features. Consequentially, a lot of Apple's proprietary extensions, when implemented through system calls, are technically part of the BSD layer. These include features such as Memorystatus/Jetsam, properietary diagnostic system calls (`kdebug*`, `proc_info()`, `stack_snapshot*`, etc) and others, which are discussed elsewhere in this work.

As one can expect, initializing the BSD layer requires substantial work, due to its multitide of subsystems and their interdependencies. The previous chapter, which annotated the kernel startup sequence, deliberately left out the flow of `bsd_init()` (from bsd/kern/bsd_init.c), when called from `kernel_bootstrap_thread()` (Step #37). The kernel startup was already quite lengthy, and it was convenient, at the time, to consider BSD as a black box.

bsd_init()

Although it is quite long, `bsd_init()` is well structured, and consists mostly of calls to each of the subsystem initialization functions. It is further laced with calls to `bsd_init_kprintf()` describing the functions to be called, but unfortunately these are strippd out of `RELEASE` kernels. With or without the helpful `kprintf()`s, `bsd_init()`'s call pattern is easy to find in the kernel binary. Walking through its flow provides a gentle introduction to the numerous subsystems, as they are initialized in turn, and before we cover most of them in more detail throughout the rest of this work.

> `jtool2 --analyze` will usually detect `bsd_init()` along with thousands of other important symbols. An annoyance in reversing old-style *OS kernels, however, is that `bsd_init()` is inlined into the other startup code. If inlined, a good way to find `bsd_init` is to look for the parsing of the `"-s"` boot argument, early into its flow.

1. **`throttle_init()`** (from bsd/miscfs/specfs/spec_vnops.c) sets up I/O throttling, which limits the bandwidth of I/O operations through `throttle_timer()` thread calls. There are three defined tiers, and their window sizes are hard-coded, but may be overridden through the Device Tree's `kern.io_throttle_window_tier[123]` or the corresponding `boot-args`. After boot is complete, the window sizes may be retrieved or modified using `sysctl(8)`.

2. **`printf(copyright);`** spits the BSD copyright string to the console. This is compiled out of *OS kernels

3. **`kmeminit()`** (from bsd/kern/kern_malloc.c) initializes BSD memory zones. These, like their underlying Mach zones, are used when allocating objects of fixed sizes, and by the BSD `_MALLOC*` variants.

4. **`parse_bsd_args()`** (inlined) is called to parse select boot arguments. These include `-s`, `-b`, `-x` and `-minimalboot` (shown in Table 2-14), which are passed on to `launchd(8)`, along with a few others we will encounter later, and several more available only in `DEBUG || DEVELOPMENT`.

5. **`dev_kmem_init()`** (from bsd/dev/mem.c) can set up the /dev/[k]mem character devices. These allow user-mode (root) access to kernel and physical memory - a veritable deathwish - so this code is contingent of `CONFIG_DEV_KMEM`, which is unset by default.

6. **`kauth_init()`** (from bsd/kern/kern_authorization.c) sets up the KAuth subsystem, which (as discussed in III/2) handles credentials and access control throughout the kernel.

7. **`procinit()`** (from bsd/kern/kern_proc.c) sets up the hash tables required for PIDs, process groups, sessions and UI. When `CONFIG_PERSONAS` is set (as it now is across flavors), this also calls `personas_bootstrap()`.

8. **`tty_init()`** (from bsd/kern/tty.c) initializes the lock group used by the tty subsystem, providing physical and logical terminal support.

9. **`set_bsdtask_info()`** (from osfmk/kern/bsd_kern.c) is called to tie the `kernproc` to the `kernel_task`. This important instance of `struct proc` provides a representation of the kernel itself as PID 0, as discussed later. At this stage, however, the structure is empty, and a call to `process_name()`a(inlined) dubs it `kernel_task` by `strlcpy()` it over the `p_name` and `p_comm` fields.

10. **Process-related lock groups are set up:** These include `proc-kqhashlock` and `proc-knhashlock` for the kqueue and knote subsystems. If `CONFIG_FINE_LOCK_GROUPS` is set (which it is, across flavors), groups are set for `proc-slock`, `proc-ucred-mlock`, `proc-mlock` and `proc-fdmlock`.

11. **The `execargs_cache` is set up:** The cache is served to store process command line arguments during `exec()`. Each entry is a pointer (`vm_offset_t`), and the cache is allocated according to `bsd_simul_execs` (set up from `scale_setup()`, earlier in `kernel_bootstrap()`).

12. **`mac_policy_initbsd()`** (from security/mac_base.c) initializes the Mandatory Access Control (MAC) Framework - specifically, setting up the `mac_policy_list` containing policy modules, the MAC lock groups, and the `MAC Labels` zone. This extremely important subsystem provides the substrate for all system security, as discussed in III/4.

13. **ulock_initialize()** (from bsd/kern/sys_ulock.c) initializes the lock group, hash buckets and [illegible] used by ulocks. This facility, introduced in Darwin 17, provides efficient locks to user mode, through the `ulock_wait()` (#515) and `ulock_wake()` (#516) proprietary system calls.

14. **The kernproc can now be set up:** The initialization code is quite lengthy, as it requires initializing its complex fields - `pgrp0`, `session0`, `filedesc0`, and the highly important kernel KAuth credentials - discussed under "kernproc", later in this chapter.

15. **fpxlog_init()** (from bsd/vfs/vfs_fslog.c) initializes the lock group used by the FPX facility, which (as of Darwin 17) logs floating point exceptions.

16. **bsd_bufferinit()** (from bsd/dev/unix_startup.c) sets up buffers used by VFS (discussed in Chapter 7), and the `mbufs` of the networking subsystem (Chapter 14). Limits are set by calling `bsd_startupearly()` (ibid), which also `kmem_suballoc()`s the `bufferhdr_map`. `bsd_bufferinit()` then calls `kmem_suballoc()` directly, to allocate the `mb_map` from which `mbufs` are allocated , followed by `bufinit()` (from bsd/vfs/vfs_bio.c) to initialize VFS buffers and their hash links.

17. **IOKitInitializeTime()** (from iokit/Kernel/IOStartIOKit.cpp) initializes the "calendar", or "wall clock". It waits until the `IORTC` (and, in MacOS, `IONVRAM`) has been published, and then calls `clock_initialize_calendar()` (from osfmk/kern/clock.c), which calls `PEGetUTCTimeOfDay()` to get the calendar time, initialize the monotonic clock (on *OS), record the boot time, and update the commpage.

18. **ubc_init()** Initializes the Unified Buffer Cache, discussed in chapter 7.

19. **vfsinit()** initializes the Virtual Filesystem Switch (VFS). As discussed throughout Chapter 7, the VFS is the adapter layer allowing multiple file system implementations (mostly, extensions) to "plug-in" into the kernel's file system APIs.

20. **proc_uuid_policy_init()** is called to.. @TODO. This is contingent on `CONFIG_PROC_UUID_POLICY`, which is `TRUE` across all Darwin variants.

21. **The prerequisites for the socket subsystem:** are initialized. This includes calls to `mcache_init()` and `mbinit()`, to set up the `mbuf` structures and their per-CPU cache allocator, and `net_str_id_init()` for tags.

22. **audit_init()** is called to set up the Auditing subsystem. This important vestige of Solaris, described in III/2, is unfortunately contingent on `CONFIG_AUDIT`, so it is only available in MacOS.

23. **knote_init()** (from bsd/kern/kern_event.c) initializes the four zones required to support the kqueue subsystem (discussed later in this chapter) - `knote zone`, `kqueue file zone`, `kqueue workqzone` and `kqueue workloop zone` - along with the `kqueue` lock group. It also initializes the memorystatus list lock (`CONFIG_MEMORYSTATUS`).

24. **eventhandler_init()** (from bsd/kern/subr_eventhandler.c) sets the lock group and two memory caches used by the event handler facility (introduced in of Darwin 17), used exclusively by the networking subsystem.

25. **aio_init()** (from bsd/kern/kern_aio.c) sets up the lock groups, work queues, `aio_work_threads` and `aiowq` zone which support asynchronous I/O.

26. **pipeinit()** (from bsd/kern/sys_pipe.c) sets up the lock group and `pipe zone` required to support the pipe facility, discussed later in this chapter.

27. **System V IPC locks:** are initialized `#if SYSV_SHM`, `SYSV_SEM` and `SYSV_MSG` are `#defined`, for shared memory, semaphores and message queues.

28. **POSIX shared memory and semaphore locks:** are initialized thereafter. Unlike their System V counterparts, support for these is not conditional.

29. **pthread_init()**: (from bsd/pthread/pthread_shims.c) is indeed a shim, which calls the implementation from the `pthread_functions` global structure of function pointers. The structure will have been initialized at this point through a call to `pthread_kext_register()` (ibid), which pthread.kext will have called on load, owing to its `AppleKernelExternalComponent` property.

30. **POSIX shared memory and semaphore caches:** are set up.

31. **time_zone_slock_init()**: (from bsd/kern/kern_time.c) initializes the time zone lock group and `tz_slock` spin lock, which protects `settimeofday()`.

32. **select_waitq_init()**: (from bsd/kern/sys_generic.c) sets up `select_conflict_queue`, maintained for those cases wherein multiple waiters are stuck `select(2)`ing on the same file object.
33. **Networking support is initialized** (`#if NETWORKING`, which is of course always set). This is performed through calls to `nwk_wq_init()` (to start work queues), `dlil_init()` (initialized the Data Link Interface Layer) and `proto_kpi_init()` (setting up protocol lock groups and the protocol hash), as discussed in Chapter 14.
34. **The main socket initialization:** ensues, through calls to `socketinit()` and `domaininit()` (for address families). `iptap_init()` (inlined) sets up the IPTap facility (for packet capture, as discussed in I/16). Apple's proprietary Flow Divert mechanism is set up here as well, `#if FLOW_DIVERT` (which is always set). Another important initalization here is of SkyWalk, through `skywalk_init()`, which is redacted from the open sources.
35. **Memorystatus**, the mechanism for low-memory handling (and, in *OS, the violent Jetsam) is (are) set up through `memorystatus_init()` (inlined).
36. **`kmstartup()`** is called, but only if `GPROF` is `#define`, to enable kernel profiling - which it is not by default.
37. **`acct_init()`** (from bsd/kern/kern_acct.c) initializes the `acct` lock group, which supports the system call of the same name (#51), a legacy BSD 4.3 syscall.
38. **`sysctl_mib_init()`** (from bsd/kern/kern_mib.c) populates some of the `hw.*` `sysctl(2)` MIBs.
39. **`bsd_autoconf()`** (inlined) calls `kminit()`, the initialization of any pseudodevices (virtual character devices) using the `pseudo_init` structure. The structure is created during build time by the `config` helper binary (built from SETUP/config/*.c), so it cannot be seen in the open sources prior to building. It is easy to spot in the MacOS kernel (wherein it is exported), or in *OS kernelcaches, following analysis. Table 5-35 shows the `pseudo_inits` found in XNU, in order of appearance (and thus, initialization):

Table 6-1: The `pseudo_inits` of XNU

Function	Device
`pty_init`	/dev/pty* pseudo-terminals
`ptmx_init`	/dev/ptmx, Pseudoterminal master device
`mdevinit`	/dev/[k]mem (if allowed), /dev/null and /dev/zero
`bpf_init`	The /dev/bpf* devices, used for the Berkeley Packet Filter (I/16)
`fsevents_init`	/dev/fsevents, described in I/4 and Chapter 7
`random_init`	/dev/random and /dev/urandom
`dtrace_init`	/dev/dtrace (and /dev/fasttrap, via `fasttrap_init()`)
`helper_init`	/dev/dtracehelper
`lockstat_init`	/dev/lockstat, used to obtain lock statistics
`lockprof_init`	DTrace lock profiling
`sdt_init`	/dev/sdt (DTrace Statically Defined Tracing)
`systrace_init`	/dev/systrace
`fbt_init`	/dev/fbt, DTrace's function boundary tracer
`profile_init`	/dev/profile
`mt_dev_init`	/dev/monotonic and `kern.monotonic sysctl` MIBs

Lastly, `IOKitBSDInit()` (from iokit/bsddev/IOKitBSDInit.cpp) publishes `IOBSD`.

40. **`os_reason_init()`** (from bsd/kern/sys_reason.c, added in Darwin 16) sets up the lock group and initializes the `os reasons` zone. The zone is prefilled, to ensure that when a failure reason needs to be allocated (through `os_reason_create()`), one can be quickly obtained without blocking.
41. **`dtrace_postinit()`** finishes DTrae initalization, started when `bsd_autoconf()` called `dtrace_init()` (two steps ago) as one of the pseudo-device initialization functions.
42. **Network interfaces are set up**: `loopattach()` (for `lo0`, `gif_init()` (`#if NGIF`), `pfloginit()` (for packet filtering and /dev/pflog, `#if PFLOG`), and the Ethernet interface support of DLIL, through `ether_family_init()` (`#if NETHER < 0`).

43. **Network initialization continues** by calling `net_init_run()`. This enables any networking kernel extensions to have their code, which they have `net_init_add()`ed, to be executed at this point. The only caller of this is MacOS's ALF.kext.

44. **Proprietary networking extensions** subject to `#if..` blocks, which are all set:

Table 6-2: The Apple networking extensions

Routine	#if	Facility
`cfil_init()`	`CONTENT_FILTER`	`com.apple.content-filter` system socket (I/16 (v1.3.5+), 14)
`pkt_mnglr_init()`	`PACKET_MANGLER`	
`necp_init()`	`NECP`	Network Extension Control Policies (I/16, 14)
`netagent_init()`		for net agents (I/16), as of Darwin 15.
`utun_register_control()`		for user-mode tunneling (I/16)
`ipsec_register_control()`	`IPSEC` (D13)	for `com.apple.net.ipsec_control`
`netsrc_init()`		`com.apple.netsrc` system socket (network/route policies/statistics)
`nstat_init()`		`com.apple.network.statistics` system socket (I/16)
`tcp_cc_init()`		TCP congestion control, as of Darwin 14.
`mptcp_control_register()`	`MPTCP`	Multipath TCP (Darwin 13+).

45. **`vnode_pager_bootstrap()`** initializes the Mach pager for vnodes, and also all the other Mach pagers, as discussed in Chapter 11.

46. **`inittodr(0)`** (from bsd/kern/kern_time.c) verifies that the calendar clock value is positive, or else resets it to the UNIX epoch, like it's 1970 all over again.

47. **The root filesystem is mounted:** `setconf()` (inlined) calls on `IOFindBSDRoot()` (from iokit/bsddev/IOKitBSDInit.cpp) to locate the block device designated as the root, which may be a RAMDisk or network interface, as specified by the `rd/rootdev` boot-args. The root device can also be passed through the Device Tree, with a RAMdisk specified in /chosen/memory-map.RAMDisk or an IOKit `boot-uuid-media` specified in /chosen/root-matching.boot-uuid. RAMDisks are used in *OS during OTA and/or recovery. `vfs_mountroot()` then attempts to mount the root device. BSD would normally prompt on the console for a root device if this fails, but XNU `panic()`s, or (for MacOS NFS Clients) hangs indefinitely while displaying a noroot icon.

48. **`IOSecureBSDRoot()`** (from iokit/bsddev/IOKitBSDInit.cpp) does nothing on MacOS, but in *OS it calls the AppleARMPlatform.kext's `AppleARMPE::callPlatformFunction()`, which resolves `SecureRootName` to a function.

49. **In MacOS `netboot_setup()`** is called to find the root device, else again boot hangs with the same `noroot` icon.

50. **XNU can also boot from a disk image**, though this is rare. `CONFIG_IMAGEBOOT` is set across all Darwin flavors, so `imageboot_needed()` (bsd/kern/imageboot.c) checks the `rp0` and `rp1` boot arguments. If detected, `imageboot_setup[_new()]` parses several others, including `-rootdmg-ramdisk`, `root-dmg` and `auth-root-dmg`.

51. **With the root filesystem healthy and mounted, `/dev` is next**, handled by a call to `devfs_kernel_mount()`.

52. **siginit()** (from bsd/kern/kern_sig.c) initializes the `kernproc`'s signal handlers (even though the kernel cannot actually be signalled).

53. **bsd_utaskbootstrap()** (also in bsd/kern/bsd_init.c) gives birth to `launchd(8)`, in a process detailed shortly.

54. **pal_kernel_announce()** allows for platform specific behaviors now that the kernel is nearly done initializing. Both the implementations, however (osfmk/i386/pal_native.h and osfmk/arm/pal_routines.h) are empty macros.

55. **mountroot_post_hook()** is a function pointer which may be set by extensions which have been loaded at this point. If it has been, it is called here.

56. **init_system_override()** (from bsd/kern/kern_overrides.c) sets up the lock group and mutex required to protect the Darwin proprietary `system_override()` (#454) system call. These enable the caller (the newborn `launchd(8)`) to override I/O and CPU throttling or (in *OS) force a fast jetsam run (discussed in @MEM).

Launching `launchd(8)`

The `launchd(8)` daemon likely needs no introduction at this point, after having been discussed in depth throughout I/13. `bsd_utaskbootstrap()` (from `bsd_init()`, step #53) which is responsible for bringing up `launchd(8)`, but the process is somewhat unorthodox.

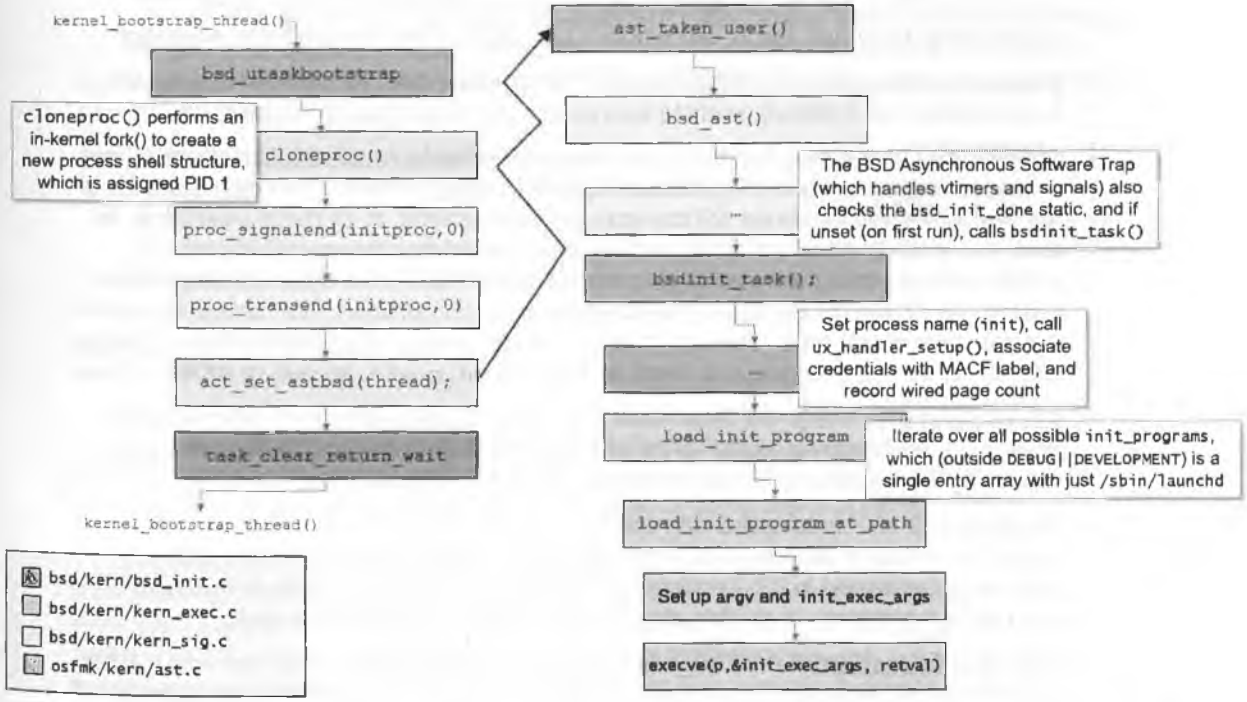
First, `cloneproc()` is called to create a new process, which is given PID 1. The actual initialization of the process, however, is performed asynchronously, by calling `act_set_astbsd()`. This fires the `AST_BSD` Asynchronous Software Trap. ASTs (discussed in Chapter 10) are a form of deferred event handling, and are to be "taken" (i.e. handled) at the earliest possible time. When `ast_taken_user()` is invoked, it detects the `AST_BSD` and calls its handler (`bsd_ast()`, in bsd/kern/kern_sig.c), which wraps `bsdinit_task()` (back in bsd/kern/bsd_init.c).

The `bsdinit_task()` routine sets the process name to "init" (a legacy of its UN*X/BSD origins), calls `ux_handler_setup()` (to enable signals), and notifies the MAC Framework of the new credentials through `mac_cred_label_associate_user()`. The count of wired pages at this point is saved by `vm_init_before_launchd()` (from osfmk/vm/vm_resident.c).

The `load_init_program()` routine (from bsd/kern/kern_exec.c) is then charged with the work of bringing up the daemon. It first calls `mach_vm_allocate_kernel()` to allocate a page of memory into `scratch_addr`. The routine then iterates over a static array of `init_programs`, but outside of `DEBUG || DEVELOPMENT` kernels this contains only a single hardcoded entry - /sbin/launchd.

The pathname and `scratch_addr` are provided to `load_init_program_at_path()` (ibid.), which uses the `scratch_addr` page to set up `launchd(8)` initial argument vector (`argv`), populated with the pathname (`argv[0]`) and possibly `-s` (as `argv[1]`, if `RB_SINGLE`). The process shell is given a `root:root` token through `set_security_token`, and the rest of the initialization is handled just as any other process - by calling `execve()`.

Figure 6-3: The birth of `launchd(8)`



Note, that `launchd(8)` must never return to the kernel! This means that it is expected to never `exit(2)`, or be `kill(2)`ed in any way. To help ensure immortality, `launchd(8)` enjoys immunity from all user mode signals - even a trigger happy administrator's `kill -9`. One abnormal scenario you can test is by faking a code signature error, using the `csflags` system call (#169) as `root` to remove the `CS_VALID` bit - slaying the daemon with no saving throw - and causing a `launchd_crashed_panic()` ("initproc exited").

Processes

Mach defines tasks, but the BSD layer provides the higher level constructs that are processes. As explained in I/8, the POSIX system calls dealing with processes provide lifecycle control, signaling, and basic resource management. These all operate on the BSD process control block - `struct proc`.

The `struct proc`

The `struct proc` is a mammoth structure, defined in bsd/sys/proc_internal.h, which provides the Process Control Block (PCB) containing all the metadata pertaining to the process which it represents. Several of the `struct proc` fields still match the semantics of BSD's, but the two structures have by now greatly diverged. Apple has deprecated some and added others across Darwin versions. This naturally affects the structure size, as well as field offsets.

Field also differ in between Darwin variants (due to `#ifdef` blocks). While this is normally inconsequential to kernel developers (who would use accessor functions anyway), it is of the utmost importance for hackers and jailbreakers, which rely on precise offsets for patching purposes. The size of the struct had been 1,168/1,040 bytes in Darwin 17, but actually shrunk to 1,136/1,016 bytes in Darwin 18, before increasing to 1,200 in Darwin 19.

If all that isn't enough, field ordering changes in between the versions as well, with important fields such as the `p_pid` migrating from one location to the other between version. Rather than illustrating the structure, then, it makes sense to look over the `struct proc`'s many fields by loosely classifying them, as follows:

- **Identifiers:** of the process (`p_pid`), parent (`p_ppid`) and group (`p_pgid`) are stored in the beginning of the process structure. Shortly after are also stored the unique `p_uniqueid` and `p_ppuniqueid`, which are 64-bit extensions of the respective counterparts. The high order 32-bit contain a generation number, ensuring uniqueness.
- **Family ties:** A process retains links to its `p_sibling` (a `LIST_ENTRY`) and `p_children` (a `LIST_HEAD`. Additionally, `p_pgrp` points to a `struct pgrp` representing the group, and `p_pglist` (a `LIST_ENTRY`) links to additional members.
- **Pointer to the underlying Mach task:** in the `p_task` field. To maintain opacity, this is a void pointer, but it always points to a `struct task` (discussed in Chapter 10).
- **Credentials:** These include the `p_[/r/sv][ug]id` fields, which hold the effective, real and saved uids and gids of the process in question. These, however, are merely caches of the true values, which are part of the `struct cred` pointed to by the `p_cred` field. As discussed in III/2, this important structure is the one holding the credentials for authorization by the KAuth mechanism, as well as the MACF label slots, wherein policies (namely, AMFI and the Sandbox) may store per-policy information. The cached values are populated using the `PROC_UPDATE_CREDS_ONPROC` macro, often immediately or shortly after the `p_ucred` changes, as a result of `set[ug]id`, `posix_spawn`, or similar actions.
- **File descriptors:** The `p_fd` fields holds a `struct filedesc` pointer, from which the array of file descriptors available to the process, and their underlying file objects can be accessed. This is discussed in greater detail later in this chapter.
- **Personae:**: If XNU is compiled with `CONFIG_PERSONA` (as it is in *OS), this holds a pointer to the active persona, and a pointer to a list of processes sharing the same persona. As explained in I/8, personae aren extension introduced by Apple in XNU-3789 which allows processes to quickly adopt credential sets in a single operation.
- **DTrace information:** if `CONFIG_DTRACE` is set (on MacOS) several `p_dtrace_*` fields hold the lists of pages and helpers, and counts of probes and tracepoints. This substantial set of fields accounts for most of the difference in structure size between Darwin flavors.
- **MemoryStatus information:** Continent on `CONFIG_MEMORYSTATUS` (which is true across variants), several `p_memstat_*` fields hold important state information which MemoryStatus/Jetsam can take into consideration when choosing candidates for idle exit or cold blooded murder.
- **knote list:** Per-process knote list for process and signal kevents.
- **aio lists:** Per-process asynchronous I/O queues - `p_aioactiveq` and `p_aiodoneq` holding aio requests, and their counts.

- **Pthread and GCD support:** holding the user mode addresses of the `p_threadstart` function (to start ordinary pthreads) and `p_wqthread` (to start work queue threads). There are also `p_dispatchqueue[_serialno]_offset` fields, used for telemetry and stackshotting, and `p_mach_thread_self_offset/p_return_to_kernel_offset`, which are used by the Pthread SPI (pthread.kext).

There is an additional group of fields explictly marked as being `bcopy(9)`ed on `fork()`, conveniently `#defined` by the `p_[start/end]copy` markers. These include some of the most important fields of the `struct proc`, namely:

- **Process name:** with not one, but two instances of 16-character strings - `p_comm`, and `p_name`. The latter can be changed by the process using `sysctl(2)` on `kern.procname`, which can actually be used, despite the disparaging comment in bsd/kern/kern_sysctl.c.
- **The process group pointer:** (`p_pgrp`), holding the group membership and ensuring new children are automatically group members.
- **Signal sets:**: including `p_sigcatch`, for signals caught by the process (i.e. those with a `setsigaction(2)` or `signal(2)` installed handler), and `p_sigignore`, for those whose behavior was set to `SIG_IGN`. A third `sigset_t` is `p_sigmask`, marked as deprecated.
- **Vnode pointer:** indicating which vnode this process was loaded (i.e. `execve(2)`d) from. The offset marking the beginning of the `__text` segment is also saved, in `p_textoff`.
- **Mach-O data:** `p_cpu[sub]type` are set from the Mach-O of the above vnode, as is UUID (from `LC_UUID`), which is useful for debugging and crash reports.
- **Code signing flags:** as discussed in III/5.
- **Priority and nice values:** in `p_priority` and `p_nice`. There are also `p_resv[0/1]` fields in between those fields, but they are unused (reserved).
- **Stack pointer:** The `user_stack` address holds the virtual (user mode) address of the main thread's stack, `p_argc` holds the argument count, and `p_arglen` records the total length of the argument vector. With these values, it's possible to pinpoint the start of `argv[]`, and return its contents for the `KERN_PROCARGS* sysctl(2)`s, as is used by `ps(1)` and `procexp(j)`.
- **Resource starvation behavior:** is maintained in the `pc_pcaction` field, which is accessed through the `PROC_CONTROL_STATE` macro. This holds one of the values of `P_PCTHROTTLE` (1), `P_PCSUSP` (2) or `P_PCKILL` (3), which controls what action will be taken - throttle, suspend or kill the process. The value can be set by a process (on itself) using the `proc_info` system call with the `PROC_INFO_CALL_SETCONTROL` flavor.

The `kernproc`

Although the kernel itself is technically not a process, it nonetheless has a corresponding `struct proc`. This structure is accessible by dereferencing the `kernproc` global (which is exported across architectures), and represents the kernel as "PID 0". As the very first "process" to be created, the `kernproc` also serves as the head of the global process list, `allproc`.

The `kernproc` (exported in MacOS kernel) is set to the address of `proc0`, a `struct proc` instance defined in bsd/kern/bsd_init.c. Several substructures are defined along with it: `session0`, `pgrp0`, `filedesc0`, `limit0`, `pstats0` and `sigacts0`. The kernel credentials (`kernproc`'s `p_ucred`) are initialized by initializing a temporary `struct ucred` and calling `kauth_cred_create()`. A call to `mac_cred_label_associate_kernel()` enables static policy modules (all registered at this point) to note the credentials as belonging to the kernel.

Although early versions of Darwin made PID 0 visible in user mode, this was quickly corrected by Apple. The `kernproc` still proves highly useful for jailbreaking and post-exploitation, however, because it serves as head of the process list. Additionally, its various fields hold kernel data which can be read or copied over by an attacker. This is particularly the case with the `kernproc`'s credential structures, possession of which automatically bypasses sandbox restrictions, incuding those of the platform profile. It's also helpful that `kernproc` is still an exported symbol (in non-1469 kernels), or can be found by `joker`.

Process lists

All processes are maintained in the aptly named `allproc` global process list. This process list is a critical structure, and one that is quite volatile as new processes are created and others destroyed. The list is therefore protected by the `proc_list_mlock` mutex, and accessed through `proc_list_[un]lock()`. A second list, `zombproc`, maintains zombies (processes which have exited but their exit code has yet to be reaped by a call to `wait4()` variants).

When operating in a process context (servicing a system call), it is possible to get the current process by calling `proc_self()`. Walking the process list to find another process by PID, however, can get quite long and prove costly to traverse. Instead, `proc_find(pid)` can locate the process through the `pidhashtbl` global table, allocated in the `M_PROC` BSD zone to contain `maxproc /4` entries, and use the `PIDHASH` over the pid. Since hash collisions are to be expected, each `struct proc` maintains a `p_hash LIST_ENTRY`.

Obtaining a `struct proc` pointer will result in incrementing the process reference count (the `p_refcount` field), so care must be taken to call `proc_rele()` when done. In the case of the current process, `current_proc()` can be used as a faster alternative to `proc_self()`, as it does not involve modifying the reference count, and thus needs no lock on the process list. This is implemented (in bsd/kern/bsd_stubs.c) by getting the bsd info of the `current_thread()`, a machine routine.

In addition to the list and hashes, processes maintain complex genalogical relationships between parents, children, siblings, and groups. Process groups may be looked up by `pgid` (e.g. when `kill(2)`ing), through the `pgrphashtbl`. The following table and the illustration on the next page summarize all the process list memberships:

Table 6-4: The many interrelations of processes

Symbol/Field	Scope	Iterator	List
`allproc/p_list`	Global	`proc_iterate`	All process list
`zombproc/p_list`	Global		Zombie process list
`pidhashtbl/p_hash`	Global	---	hash buckets for locating processes by PID
`p_pglist`	Group	`pgrp_iterate(p->p_pgrp)`	Members of the same process group
`p_sibling`	Family	`proc_childrenwalk(p->p_pptr)`	Children of the same `p_pptr` parent
`p_children`	Family	`proc_childrenwalk(p)`	Children of this process
`p_persona_list`	Personae	---	Processes adopting the same persona
`p_memstat_list`	MemoryStatus	---	Processes with same memorystatus/Jetsam priority

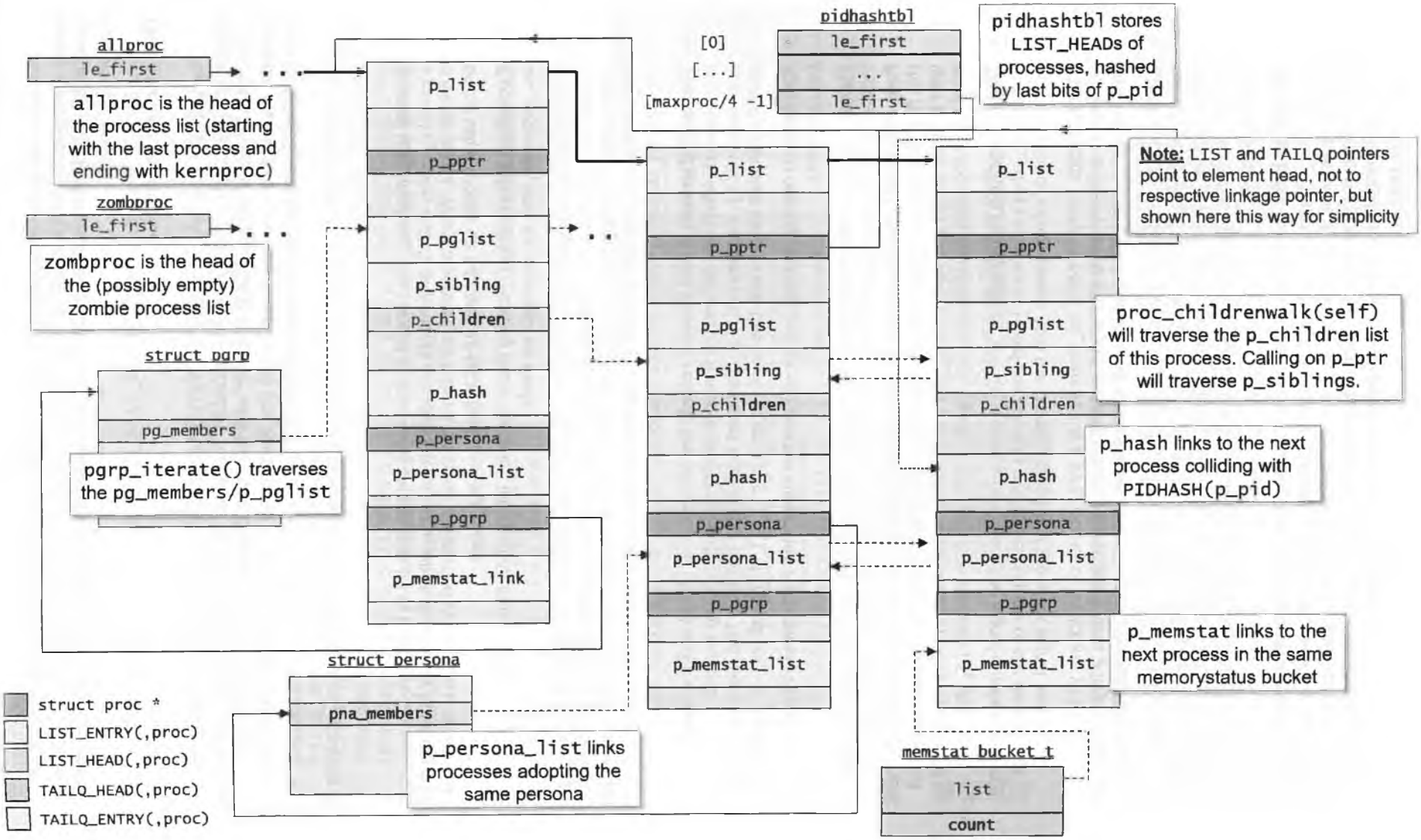
There exist also macros for walking family (`PCHILDREN_FOREACH`) and group members (`PGMEMBERS_FOREACH`) members, as well as the global lists (`[ALL/ZOMB]PROC_FOREACH`). The global scope macros require their respective list symbols and their associated locks to be visible, which is not so the case with the functions. Another advantage of the functions is that they can be passed two `proc_iterate_fn_t`, a filter function and a callout function (which is invoked if the filter function returns non-zero).

Process data in user mode

The fields of the `struct proc` are of immense importance to user mode diagnostic tools, notably `ps(1)`, `top(1)` and the author's `procexp(j)`. The structure, however, cannot be made accessible as is, not even as read-only data. Some of its fields contain pointers (which must be unslid so as to not disclose KASLR), and others should be redacted. Process data is readable in user mode through two primary interfaces:

- **The standard `sysctl(2)` interface:** provides the `kern.proc` MIB, whose data items are `struct kinfo_proc` entries. The structure is itself comprised of two other structures - the `extern_proc` and the `eproc`, both of which are populated by `fill_user[32/64]_e[xtern]proc` (respectively), in bsd/sys/kern_sysctl.c.
- **The properietary `proc_info` system call (#336):** retrieves a plethora of information through its numerous flavors, as discussed in I/15. Each flavor has its own associated structure, and the `proc_pid[short]bsdinfo` functions populate them.

Figure 6-5: Visualizing process interrelations



(U)Threads

Processes are mere containers for threads. The BSD layer therefore provides support for thread management, which is further augmented by POSIX threads (pthread) support. The user mode aspects and APIs were discussed in I/8, so we can turn our attention to the kernel-side support of threads.

The `struct uthread`

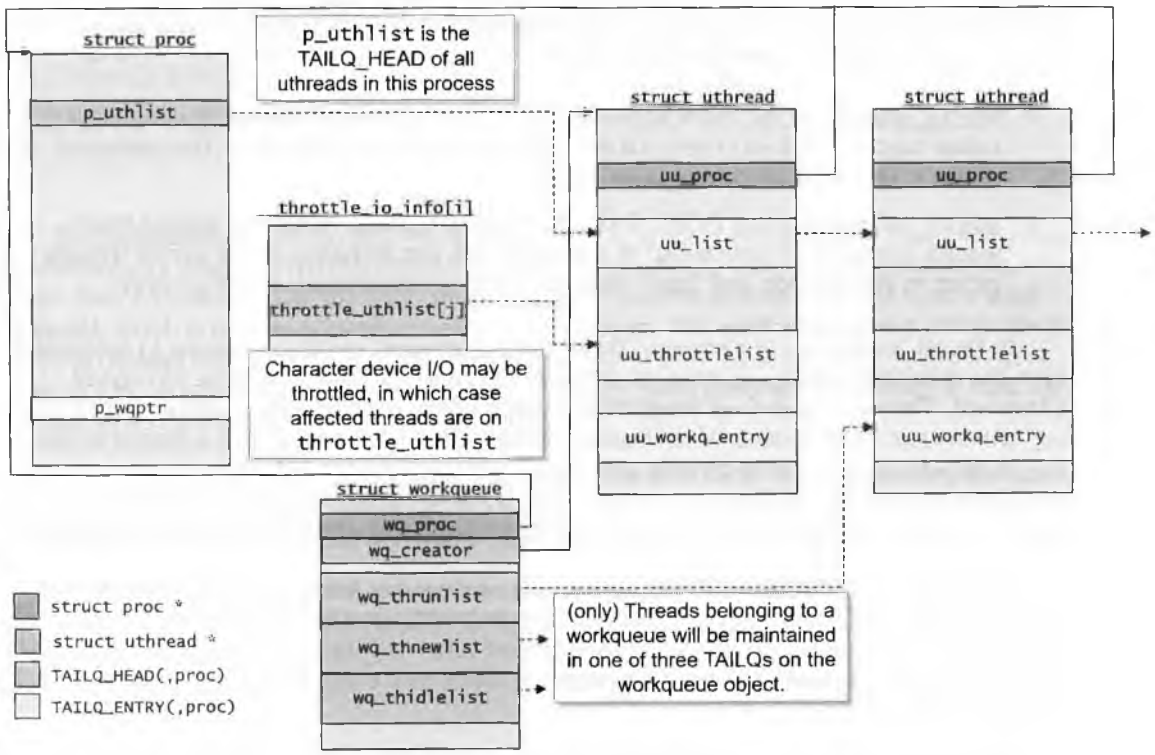
The `struct proc`'s `p_uthlist` is a `TAILQ_HEAD` of `uthread` structures. These are defined in bsd/sys/user.h and hold all the necessary metadata on the thread. As with the `struct proc`, the `uthreads` are constantly updated, with their size between flavors and Darwin versions: Darwin 17's `uthread` was 824/568 (for MacOS and *OS, respectively), Darwin 18's grew to 840/592, and Darwin 19's strink to 808/560.

With so many fields to consider, we examine the `uthread` as we did the `proc`, grouping its fields into several classes:

- **Syscall information:** The current system call a thread may be in is held in `syscall_code`, and its arguments are saved in `uu_arg`, which is an array holding up to 8 64-bit values. When a system call returns, its value is recorded in `uu_rval`, which is a 64-bit value broken up into two 32-bit values.
- **Exception information:** Holding Mach exception information in `uu_code`, `uu_subcode`, and `uu_exception` fields.
- **Continuation support:** Certain system calls use continuations. These include `select(2)`, `kevent(2)`, and the `wait(2)` variants. The data required for the continuation is stored in the `uu_kevent` union. The continuation pointer itself is stored in `uu_continuation`. The `select(2)` system call also makes use of a dedicated `uu_select` structure, which stored the selection bits of the file descriptor set.
- **The wait channel:** A wait channel records the address (`uu_chan`) and message (`uu_wmesg`), which are set when a thread is blocking using `_sleep` variants (`msleep[/0/1]`, `tsleep[/0/1]`, discussed later in this section). The message can be picked up in user mode using the `kern.proc sysctl(2)` MIBs and `ps -o wchan`.
- **Pointers:** to owning process (`uu_proc`) and the supporting Mach thread object (`uu_thread`).
- **Flags:** `uu_flag` holds a bitmask of `UT_*` flags (also defined in bsd/sys/user.h) holding indicators for throttled/passive I/O, `set[u/g]id`, and `vfork()` state. There are also flags dealing with thread cancelation, notably `UT_CANCEL`. Various I/O system calls check this flag (through `pthread_testcancel()`), and (if set) either return from the syscall with `EINTR`, or call `thread_abort_safely()`. Otherwise, those syscalls merge with their non-cancelable counterparts (e.g. `read()` calls `read_nocancel()`, etc).
- **Signal handling information:** The `uu_cursig` field holds the signal number sent to a thread. `uu_sig[list/wait]` hold the pending and waited signals of the thread, and `uu_sig[/old/vfork]mask` hold the current, saved and pre-vfork signal masks.
- **VFS context:** Every thread's `uu_context` is a `vfs_context` structure, which is comprised of the Mach thread object and the thread credentials. This is a highly important structure for authorization checks, and is elaborated on in Chapter 7 (under "VFS KPIs").
- **Audit record:** In MacOS, threads are subject to auditing (see III/2). Each thread's current audit record is held in the `uu_ar` field, and it is to there that audited arguments and codes are written, betfore being collected by the audit subsystem and committed to the log. Note that this field is still defined in the *OS variants, but is unused for lack of auditing support.
- **Throttling info:** Thread I/O operations may be throttled for performance reasons. Each thread's `uu_throttle_info` is a pointer to its throttle information, which is kept opaque (though is actually a `_throttle_io_info_t`, defined in bsd/miscfs/specfs/spec_vnops.c). Several booleans also maintain quick indicators about the current throttling state, and a list of all throttled threads can be obtained from the `TAILQ_ENTRY` of `uu_throttlelist`.

- **DTrace information:** Plentiful data, held in various `t_dtrace_*` fields and a `_tdu` union, supports DTrace probes. The total data size spans a few dozen bytes, but is contingent on `CONFIG_DTRACE`, which (as with the `proc`) contributes to the size difference of the `uthread` structure between MacOS and the *OS variants.
- **Document tombstone information:** As discussed in Chapter 7, tombstones provide support for Document ID (`UF_TRACKED`) files.
- **Exit reason:** As of XNU-3789, the `uu_exit_reason` field is a `struct os_reason`, used for `exit_with_reason()`.
- **Thread name:** Settable by the `kern.threadname sysctl(2)` call, is stored in `pth_name` field. The thread name is limited to `MAXTHREADNAMESIZE` (=64), but the name itself is not stored inline - rather, `pth_name` is a pointer to a `kalloc`'ed region of memory.
- **Thread list connectors:** Each `uthread` is a member of several lists. The `uu_list` is a `TAILQ_ENTRY` of all `uthreads` belonging to the same `struct proc`. The aforementioned `uu_throttlelist` points to throttled `uthreads`. Up until Darwin 18 there was also a `uu_threadlist` void pointer, reserved for the pthread SPI provider (`pthread.kext`). As of 18, there is a `uu_workq_entry`, which links threads of the same work queue (implemented in bsd/pthread/pthread_workqueue.c). Figure 6-6 shows an example of several threads from the same process:

Figure 6-6: Visualizing thread interrelations



Note that BSD level threads have no identifier which can be globally visible in user mode. There is the underlying Mach thread's ID, but there is no BSD style API to retrieve it. There is also the `pthread_t`, but that is internally maintained by `libpthread.dylib` and not visible outside the process. The threads can, however, be given names of up to `MAXTHREADNAMESIZE` (64) bytes, through the `kern.threadname sysctl(2)` MIB or `proc_info()` with the `PROC_INFO_CALL_SETCONTROL` op and `PROC_SELFSET_THREADNAME` argument. The `uthread`'s name is stored in the `pth_name` field, and may be retrieved by `bsd_getthreadname()`, or in user mdoe via the same MIB, or `proc_info()` with the `PIDTHREAD*INFO` flavors.

Pthread shims and callbacks

Starting with Darwin 13, Apple chose to refactor pthread support into a separate kernel extension, pthread.kext, otherwise known as `com.apple.kec.pthread`. The extension's source is part of the open source libpthread project.

The kernel, therefore, provides only shims, to which the kernel extensions can "plug in". The `pthread_functions` structure allows the kext to supply its own implementations for functions required by the kernel. The structure pointer is initially `NULL`, and pthread.kext is expected to claim them using `pthread_kext_register()` by the time the kernel calls `pthread_init()` during `bsd_init()`.

The `pthread.kext` is an `AppleKernelExternalComponent` and is thus guaranteed to load a priori, when `StartIOKit()` calls `KLDBootstrap::readStartupExtensions` (via the `record_startup_extensions_function` pointer) to load such marked extensions.

The `pthread_callbacks` structure works in reverse, by allowing XNU to provide the kext with callbacks to get/set task and thread fields (rather than rely on offsets) as well supply handlers for thread related events.

Work Queue threads (the kernel-side portion of GCD)

Work queues were introduced in I/8 as an important building block of GCD, as they provide an in-kernel thread-pool. Recall, that the implementation is entirely opaque to user mode, with only two system calls provided, as of Darwin 10:

- **`workq_open()`** (#367, from pthread/pthread_workqueue.c) prepares a process for work queue support. This `zalloc()`s a work queue object and initializes it. The new work queue is set as the process `p_wqptr`.
- **`workq_kernreturn()`** (#368, ibid.) allowing the manager thread to request more worker threads or a "narrowing" of the queue (i.e. less threads), or the worker threads to return to kernel mode and "park" there until they are awakened with more work.

In kernel, a `struct workqueue` (defined in bsd/pthread/workqueue_internal.h) is allocated from the dedicated `workq.wq` zone. As shown in Figure 6-6, it maintains three `TAILQ`s of `uthreads`. The `wq_thrunlist` holds those threads which are presently running. `wq_thnewlist` holds threads which have been added, but have not yet had a chance to run. `wq_thidlelist` holds those threads which have previously performed work, but have since finished it and `workq_kernreturn()`ed to park in kernel space. Parked thread block on `workq_unpark_continue()`, a continuation (q.v. Chapter 9) which allows quick resumption.

In addition (and not shown in the Figure), the `workqueue` holds counts and statistics, its own lock and (as of Darwin 18) a turnstile, and three priority queues - overcommit, constrained, and special. The `workqueue` also holds three thread calls - `wq_delayed_call` (to lazily create new threads), `wq_immediate_call` (to urgently create new threads) and `wq_death_call`, to dispose of old threads.

The user mode libdispatch manager thread issues a `workq_kernreturn()` with a `WQOPS_QUEUE_REQTHREADS` code, when work is available, specifying the number of worker threads required. This results in the `workq_reqthreads()` routine creating a new `workq_threadreq`, specifying the number of threads requested (possibly constrained by the system's global `wq_max_parallelism` QoS bucket, or the work queue's own settings), and the QoS level.

When serving thread requests, the preference is to reuse idle threads, by calling `workq_pop_idle_thread()`, which moves a thread from the workqueue's `wq_thidlelist` to the `wq_thrunlist`. If new threads are required, they are added to the workqueue by `workq_add_new_idle_thread()`. This routine calls the `pthread_functions`'s `workq_create_threadstack()` (from Pthread's kern/kern_support.c) and Mach's `thread_create_workq_waiting()` (from osfmk/kern/thread.c) to create a Mach thread "parked". The new Mach thread's `uthread` is then inserted to the work queue's `wq_thnewlist`.

Thread requests are executed when `workq_unpark_continue()` resumes and calls `workq_select_threadreq_or_park_and_unlock()`, which (as its name implies) will either dequeue a thread request and call `workq_setup_and_run()`, or `workq_park_and_unlock()`. Another case handled in the continuation is `workq_unpark_for_death_and_unlock()`, which is called on idle threads.

Assuming a thread is to be setup and run, `workq_setup_and_run()` calls Pthread.kext's `workq_setup_thread()` (from Pthread's kern/kern_support.c), which page faults the thread's user mode stack, and returns to user mode through either `thread_bootstrap_return()` (on first return) or `unix_syscall_return()`, so the effect in user mode is that `workq_kernreturn()` just returned, and userspace resumes in `pthread_wqthread()`. If the thread was unparked for death, `workq_setup_thread()` is called with the `WQ_SETUP_EXIT_THREAD`, to allow the thread to exit gracefully - the special code of `WORKQ_EXIT_THREAD_NKEVENT` is used as a fake value to `pthread_wqthread()`'s `nkevents` parameter, causing it to short circuit and `pthread_wqthread_exit()`.

BSD `*sleep` and `wakeup*`

The BSD KPIs provide the `[m/t]sleep*` and `wakeup*` family of functions to allow kernel code to wait for triggerable events, similar to POSIX condition varaibles. These functions are defined in bsd/sys/proc.h as shown in Listing 6-7:

Listing 6-7: The BSD layer `msleep()` and `wakeup[_one]()` functions

```
extern int  msleep(void *chan, lck_mtx_t *mtx, int pri, const char *wmesg, struct timespec *ts);
extern int  tsleep(void *chan, int pri, const char *wmesg, int timo);
extern int  msleep1(void *chan, lck_mtx_t *mtx, int pri, const char *wmesg, u_int64_t timo);

extern void wakeup(void *chan);
extern void wakeup_one(caddr_t chan);
```

The implementation of the above is in bsd/kern/kern_synch.c, along with `tsleep[0/1]` and an `msleep[0]` wrappers (omitted from the headers), which take a timeout in nanoseconds, rather than a `timespec`. All the sleep variants require a kernel address, referred to as a **wait channel**, which is used as a token to wake up the sleepers. The **wait message** is stored along with the channel in the `uuthread`, and can be retrieved in user mode through the `kern.proc` `sysctl(2)`, as is sometimes shown by `ps -l`. The *`pri`* value specified in the sleep is used as the priority to reschedule the thread when woken.

The value of `PZERO` (22) was traditionally used to block or allow signal processing during sleep, but there is no longer need to do so. Priorities max out at 127, so higher order bits are used as flags to indicate wakeup and signal behavior, shown in Listing 6-8:

Listing 6-8: Priority high order bits used as sleep flags (from bsd/sys/param.h

```
#define PRIMASK   0x0ff  // Masks out up to 255, priorities max at 127

#define PCATCH    0x100  /* OR'd with pri for tsleep to check signals */
#define PTTYBLOCK 0x200  /* for tty SIGTTOU and SIGTTIN blocking */
#define PDROP     0x400  /* OR'd with pri to stop re-aquistion of mutex upon wakeup */
#define PSPIN     0x800  /* OR'd with pri to require mutex in spin mode upon wakeup */
```

All the sleep variants funnel to `_sleep()` (ibid.). The *`chan`* and *`wmesg`* arguments are stored on the `uuthread`'s `uu_wchan` and `uu_wmesg`. If a mutex is specified (as it is in the `msleep*` case), the wait is performed by a call to `lck_mtx_sleep[_deadline]`. Else the wait is performed by a Mach level `thread_block`, with or without a Mach thread continuation (as may be provided by `msleep0` or `tsleep[0/1]`).

If a timeout was specified and has elapsed, the blocking thread will wake up. Otherwise, the wake up functions - `wakeup[_one]()` - behave the same way as their POSIX layer cousins, waking up either all or a single thread waiting on the same wait channel using the Mach layer `thread_wakeup[_one]()` routines. The Mach layer APIs and continuations are discussed in more detail throughout Chapter 10.

Process Lifecycle

Throughout the system lifespan, processes are created and terminated at sometimes dizzying rates. This This section takes a look at those two crucial stages of the lifecycle.

fork(2)/vfork(2)/posix_spawn(2)

UN*X processes are traditionally created when a parent `fork(2)`s or `vfork(2)`s, both of which create a clone of the parent. The newer `posix_spawn()` also creates a new process, combining it with some of the `execve(2)` logic, to start a different executable. Internally, all paths lead to `forkl()` which takes the *`kind`* of forking requested, as documented in the header comment:

Listing 6-9: The description of `forkl()` forking kinds

```
 *
 *              PROC_CREATE_FORK        Create a complete process which will
 *                                      return actively running in both the
 *                                      parent and the child; the child copies
 *                                      the parent address space.
 *              PROC_CREATE_SPAWN       Create a complete process which will
 *                                      return actively running in the parent
 *                                      only after returning actively running
 *                                      in the child; the child address space
 *                                      is newly created by an image activator,
 *                                      after which the child is run.
 *              PROC_CREATE_VFORK       Creates a partial process which will
 *                                      borrow the parent task, thread, and
 *                                      uthread to return running in the child;
 *                                      the child address space and other parts
 *                                      are lazily created at execve() time, or
 *                                      the child is terminated, and the parent
 *                                      does not actively run until that
 *                                      happens.
 *
 *              At first it may seem strange that we return the child thread
 *              address rather than process structure, since the process is
 *              the only part guaranteed to be "new"; however, since we do
 *              not actualy adjust other references between Mach and BSD (see
 *              the block diagram above the implementation of vfork()), this
 *              is the only method which guarantees us the ability to get
 *              back to the other information.
 */
int forkl(proc_t parent_proc, thread_t *child_threadp, int kind, coalition_t *coalitions) {
...
```

The `PROC_CREATE_FORK` and `PROC_CREATE_SPAWN` kinds both call `cloneproc()`, which then calls `forkproc()` to duplicate the BSD layer `struct proc`, by `bzero()`ing out statistics, `bcopy()`ing the fields between `p_start` and `p_end`, `fdcopy()`ing the open file descriptors, manually copying some other parent fields, and assigning a new `p_pid`. The new PID is usually monotonically increasing, but may wrap, as long as it does not belong to any groups or sessions. An additional, 64-bit identifier is the `p_uniqueid`, incremented on assignment with no danger of wrapping.

The Mach layer is handled when `cloneproc()` calls `fork_create_child()`, which creates a new Mach `task` (with `task_create_internal()`) and a child Mach `thread` (with `thread_create_waiting()`). The task joins the parent's coalitions, and it the target `thread` which is returned to the caller.

The `PROC_CREATE_VFORK` kind calls `forkproc()` but not `fork_create_child()` thus only handling the BSD layer, but not Mach. The new process "piggybacks" on the parent's `task` and `thread` objects, as well as the BSD layer `uthread`. As the `vfork(2)` man page explains, this is much more efficient, but does incur some user mode constraints, and leaves the parent suspended. There also other considerations when a `vfork()`ed child terminates. The man page promises the system call will be eliminated" when proper system sharing mechanisms are implemented" - which apparently has yet to happen.

[__mac_]execve()/posix_spawn()

Cloning a process with [v]fork() isn't that useful, and so the child usually proceeds to [__mac_]execve() a binary. Both system calls are implemented in bsd/kern/kern_exec.c, and the latter (more well known) calls the former, which is a MACF-aware variant, also discussed in III/4. XNU also supports the more efficient posix_spawn() system call (also in bsd/kern/kern_exec), which merges the two operations into one system call, and also allows plenty of attributes and flags. From there, the flow merges into the flow shown in Figure 6-10:

Figure 6-10: A high level flow of BSD process execution

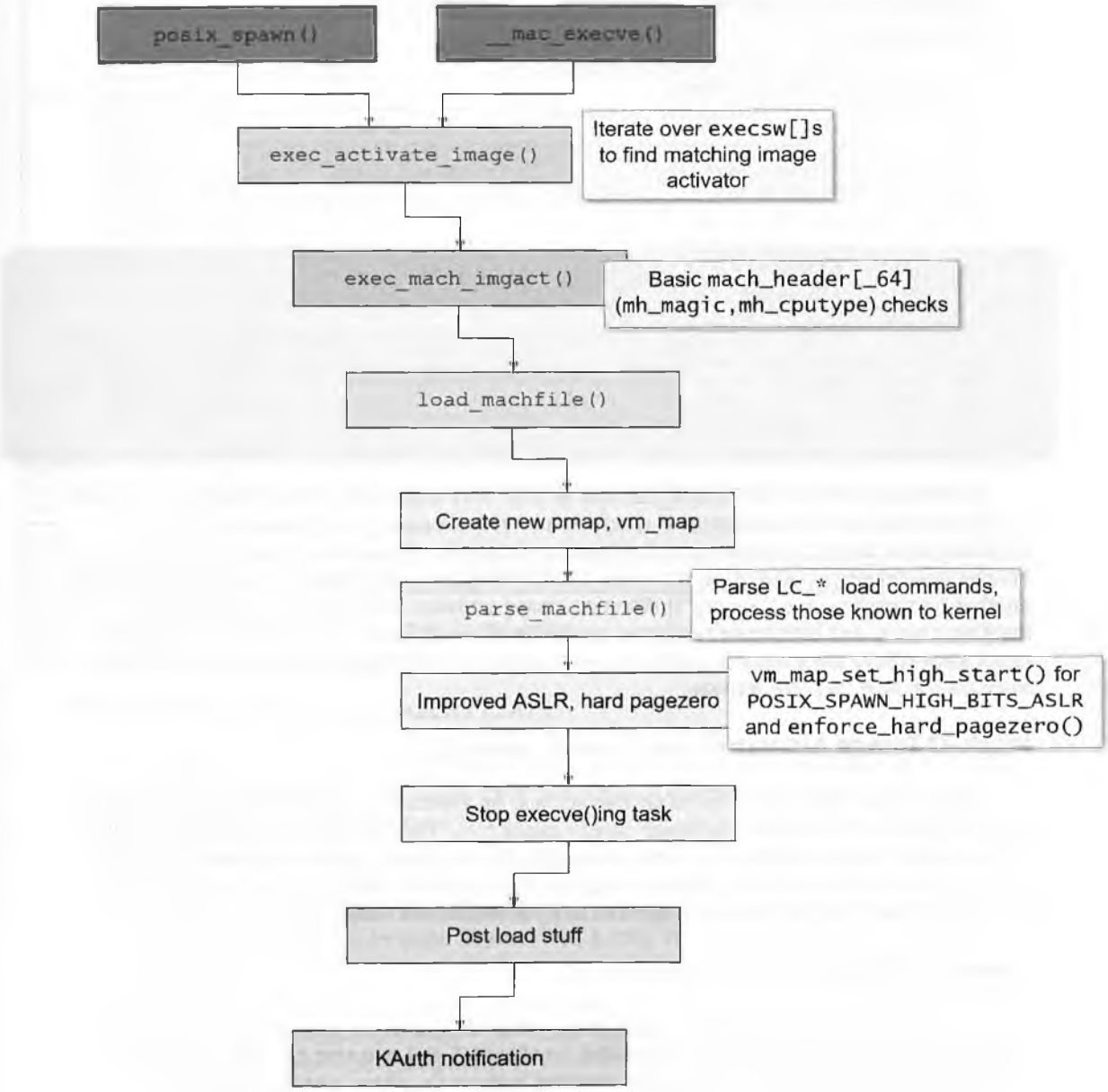


Image Activation: exec_activate_image()

Whichever system call path was taken, the calls allocate an image_params structure (from bsd/sys/imgact.h). The structure is also partially filled by those syscalls (with the image filename, arguments, vnode pointer, and more), and passed to exec_activate_image(), which assumes most of the work. exec_activate_image() (from bsd/kern/kern_exec.c) uses the filename supplied in the image_params's (ip_user_fname) to instantiate a vnode (ip_vp) and load its contents. The routine then iterates over the **image activators**.

The image activators are hard-coded in the `execsws` array, as shown in Listing 6-11. Unfortunately, the `execsw` array cannot be extended. Unlike Linux, wherein kernel-level code can use `register_binfmt()` to dynamically add format interpreters, not only is there is no interface to add entries, but the array is placed by the compiler into `__DATA[_CONST].__const`.

Listing/Output 6-11: The `execsw` array of image activators, in code

```
/*
 * Our image activator table; this is the table of the image types we are
 * capable of loading.  We list them in order of preference to ensure the
 * fastest image load speed.
 *
 * XXX hardcoded, for now; should use linker sets
 */
struct execsw {
        int (*ex_imgact)(struct image_params *);
        const char *ex_name;
} execsw[] = {
        { exec_mach_imgact,             "Mach-o Binary" },
        { exec_fat_imgact,              "Fat Binary" },
        { exec_shell_imgact,            "Interpreter Script" },
        { NULL, NULL}
};
```

```
#
# MacOS's _execsw is exported but *OS's is not. Either way, grep(1)ing around "Mach-o" works well:
#
morpheus@Zephyr (~)$ jtool2 -d __DATA_CONST.__const /System/Library/Kernels/kernel | grep -B2 -A 4 Mach-o

0xffffff8000e7e2e0: 0xffffff8000877980  __func__0xffffff8000877980   # exec_mach_imgact
0xffffff8000e7e2e8: 0xffffff8000bb7009  "Mach-O Binary"
0xffffff8000e7e2f0: 0xffffff800087ac80  __func__0xffffff800087ac80   # exec_fat_imgact
0xffffff8000e7e2f8: 0xffffff8000bb7017  "Fat Binary"
0xffffff8000e7e300: 0xffffff800087afc0  __func__0xffffff800087afc0   # exec_shell_imgact
0xffffff8000e7e308: 0xffffff8000bb7022  "Interpreter Script"
```

All activators take in the `image_params` as their only argument, and are expected to return a 0 if successful, or -1 if unclaimed. A special return value is `exec_fat_imgact()`'s -2, indicating a fat binary, and restarting the iteration via a `goto` ("`encpasulated_binary`"). Another special return value is -3, for `exec_shell_imgact()` to indicate a script, necessitating another backwards `goto` ("`again`"). When an image activator does claim the image, the iteration breaks, and temporary resources can be freed. `exec_imgact()` also fires a KAuth notification (III/2) using `kauth_authorize_fileop()`, not bothering with the return code since it's too little, too late anyway.

Mach-O Image Activator: `exec_mach_imgact()`

Interpreter scripts end up as fat or Mach-O, and fat inevitably contain Mach-O slices. Thus, all roads leads to Mach-O, and to `exec_mach_imgact()`. The Mach-O image activator starts with a header check, rejecting big endian binaries with `EBADARCH`, and all other non Mach-O `MH_EXECUTE`. Following a quick header magic and architecture check, this function resets the task and thread, and calls on `load_machfile()` to handle the dirty work.

Loading Mach-O: `load_machfile()`

`load_machfile()` begins by calling `pmap_create()` and `vm_map_create()` to create new maps for the image to be executed. After handling NX and ensuring no heap execution, it initializes ASLR for the process by randomizing the both `aslr_page_offset` (for the executable) and `dyld_aslr_page_offset`. The routine then initializes the `load_result` structure, which will be used by subsequent calls to populate the Mach-O details. The structure is shown in Table 6-12:

Table 6-12: The Mach-O `struct _load_result` returned by `load_machfile()`

Offset	Field	Set by	Value
0x0	`mach_header`	`load_segment()`	`LC_SEGMENT` (`vmaddr` for fileoff 0), + slide
0x8	`entry_point`	`load_unixthread()`	`LC_UNIXTHREAD` + slide
0x10	`user_stack`	`load_main/unixthread()`	`LC_..` or default
0x18	`user_stack_size`	`load_main()`	`LC_MAIN`
0x20	`user_stack_alloc`	`create_unix_stack()`	---

Table 6-12: The Mach-O `struct load_result` returned by `load_machfile()` (cont.)

Offset	Field	Set by	Value
0x28	`user_stack_alloc_size`	`load_main()`	`user_stack_size()` + guard page
0x30	`all_image_info_addr`	`load_dylinker()`	/usr/lib/dyld load address + slide
0x38	`all_image_info_size`		
0x40	`thread_count`	`load_unixthread()`/`load_main()`	per `LC_UNIXTHREAD` or 1 (`load_main()`)
0x44	`unixproc`	`load_main()`/`load_unixthread()`	1, for all binaries with load commands
	`needs_dylinker`	`*OS:parse_machfile()`/`load_main()`	1 on *OS, else if `LC_MAIN`
	`dynlinker`	`load_dylinker()`	1, if dynamic linker is being loaded
	`validentry`	`load_segment()`	1, if `entry_point` is properly mapped
	`has_pagezero`	`parse_machfile()`	1, if `__PAGEZERO` segment found
	`using_lcmain`	`load_main()`	1, if `LC_MAIN`
	`legacy_footprint`	`parse_machfile()`	If `LC_VERSION_MIN_IPHONEOS` pre iOS 12
	`is_64bit_addr`	`load_machfile()`	`imgp->ip_flags`, set from Mach-O header `cputype` in `exec_mach_imgact()`
	`is_64_bit_data`		
0x48	`csflags`	`load_code_signature()`, others	`LC_CODE_SIGNATURE` blob, others
0x4c	`uuid`	`load_uuid()`	`LC_UUID`. Field is unaligned
0x60	`min_vm_addr`	`load_segment()`	Min/Max values of `LC_SEGMENT[_64]`s loaded
0x68	`max_vm_addr`		
0x70	`platform_binary`	`load_code_signature()`	`LC_CODE_SIGNATURE` blob
0x78	`cs_end_offset`		
0x80	`threadstate`	`load_threadstate()`	`LC_UNIXTHREAD` (else zero)
0x88	`threadstate_sz`		
Darwin 19			
0x90	`ip_platform`	*unknown at time of writing*	`LC_BUILD_VERSION`
0x94	`lr_sdk`		

Parsing Mach-O: `parse_machfile()`

As explained in I/6, The Mach-O header contains the architecture information, a few flags, and `ncmds` load commands. Only a few command types (specifically, those in I/6-13) are of interest to XNU. These commands are processed by `parse_machfile()` in four passes, counting from zero*:

- The zeroth pass (added in XNU-3248) applies to `LC_SEGMENT[_64]` commands, determining if they can be page aligned. This is only the case for ARM64 binaries, however, so in MacOS this is skipped.
- The first pass parses `LC_UNIXTHREAD`/`LC_MAIN`, `LC_VERSION_MIN_IPHONEOS`, `LC_UUID` and - most importantly - `LC_CODE_SIGNATURE`. Code signatures (III/5) are loaded using `load_code_signature()` (which first checks the UBC cache, and (if not found), adds the CS blob to it, subject to MACF validation). They are also subsequently validated using `cs_validate_range()`.
- The penultimate pass processes the `LC_SEGMENT[_64]` and loads the segments into memory using `load_segment()`.
- The last pass handles the `LC_ENCRYPTION_INFO[_64]` and `LC_LOAD_DYLINKER` initializes the thread state (if the entry point is valid).

If the passes were successful, a check is made to ensure code signatures were successfully obtained. If `LC_LOAD_DYLINKER` was detected in the third pass, `load_dylinker()` handles dyld's loading into the new process, and adjusts the entry point accordingly. Special consideration is given to Mach-O binaries containing an `__XHDR` segment, which is a form of self extracting compressed executables. If such a segment is detected, `parse_machfile()` returns `LOAD_BADMACHO_UPX`. As of XNU 3789, these executables are disallowed.

Post Load: `exec_mach_imgact()`

After `load_machfile()` returns to `exec_mach_imgact()`, there is still much work to be done by the latter. Assuming the load was successful, the `load_result` passed to it by reference will have been populated as shown in Table 6-12. `exec_mach_imgact()` can then perform "post loading" operations, which include the following:

- **Code signing flags**: (III/5) are set up at this point, by combining those in the load result and any defined in the parent process as `CS_EXEC`.
- **`vm_map_exec`**: (osfmk/vm/vm_map.c) sets up the child's commpage and shared region.
- **fdexec()** (from bsd/kern/kern_descrip.c) closes any file descriptors which are marked `O_CLOEXEC`, and thus should not be inherited by a child process.
- **exec_handle_sugid()**: Explicitly checks for and deals with setuid/setgid programs, which require issuing new process credentials for the child.
- **swap_task_map()**: Switches the task `vm_map` to the new clean map of `execve()`.
- **activate_exec_state():** Calls `task_set_dyld_info()`, `task_set_64bit()` and sets up the main thread state (through `thread_state_initialize()`, `thread_setstatus()` and `thread_setentrypoint()`) from values in the load result.
- **`thread_set_mach_voucher`**: Removes any voucher from the execve() caller.
- **psignal()**: sends a `SIGTRAP` if the process is marked `P_LTRACED`, to preempt any interruptions during the exec operation.
- **Create the process stack**: If the load result indicates `unixproc` is true (which it always is, due to `LC_UNIXTHREAD` or `LC_MAIN`), `create_unix_stack()` calls `mach_vm_allocate_kernel()` to allocate `user_stack_alloc_size` bytes at the `user_stack` address (both indicated in the `load_result`). If the allocation is unsuccessful, the allocation is retried with `VM_FLAGS_ANYWHERE`. Since the stack grows downwards, its highest memory address (which is technically its bottom) is set up with a guard page, `mach_vm_protect()`ed to disallow all access.
- **Copy `apple[]` array onto stack:**: With the stack added, the `apple[]` array argument is injected onto the new stack by `exec_add_apple_strings()`. As discussed in I/7, this fourth argument (after `envp`) to the program's `main()`, and supplies the string arguments (shown in I/7-3) consumed by libsystem_c.dylib and others.
- **`vm_map_switch()`:** to the executed process. Then, `exec_copyout_strings()`, `thread_setuserstack()`, and (if the dylinker) `task_set_dyld_info()`. Also while on the child map, `exec_prefault_data()`. Then `vm_map_switch()` back.
- **`execsigs()`:** (from bsd/kern/kern_sig.c) resets the signal state.
- **Set secluded memory:** Explictly allow `Camerea` and `mediaserverd` to use secluded memory `#if CONFIG_SECLUDED_MEMORY`, on *OS.
- **Handle platform binary indication:** calling `task_set_platform_binary()` (osfmk/kern/task.c) to set the corresponding task's `TF_PLATFORM`, required for `task_conversion_eval()` checks when messing with the `SEND` right to task ports.
- **Copy UUID:** from `LC_UUID` value (in load result) to the process `p_uuid`.
- **Emit KDebug event:** of `TRACE_DATA_EXEC`, with path name in extra arguments.
- **Suspend the process:** if started via `posix_spawn()` with `POSIX_SPAWN_START_SUSPENDED`.
- **Wake up parent:** if this process' `P_LPPWAIT` flag was set.
- **`psignal_vfork()`** is called to deliver delayed signals if the process is `P_LTRACED`.

Process Termination

A process dies when its usefulness is at an end. This could be of natural causes (i.e. a voluntary `exit(2)`) or untimely termination via some lethal signal, like the dreaded `SIGKILL`. Either way, this involves cleaning up resources, freeing memory, and more.

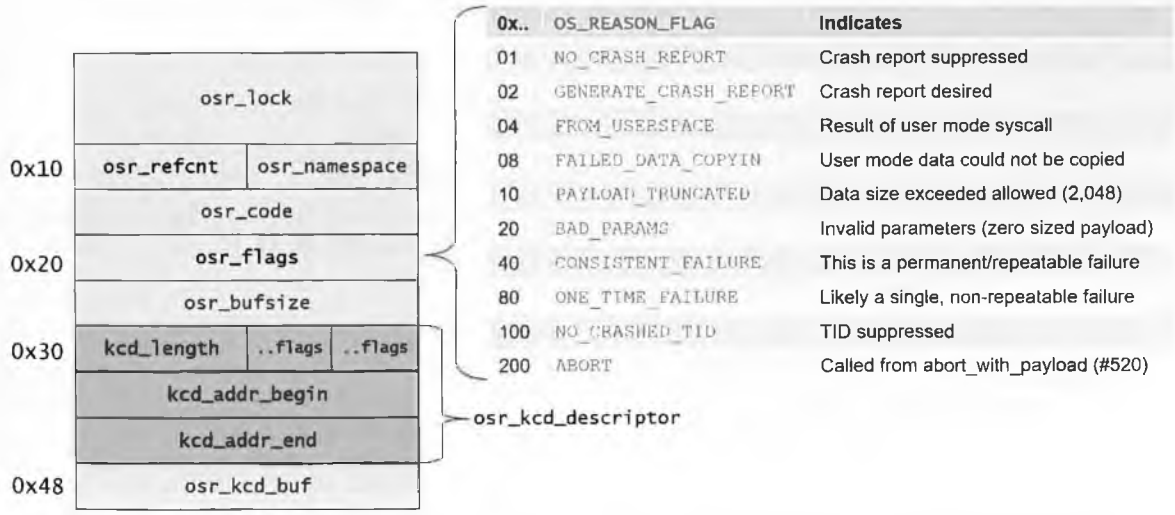
The most important aspect of the process' entire lifecycle - literally, its ultimate goal - is to return a 32-bit **exit status** (as the argument of `exit(2)`), which will be picked up by its parent using `wait4()`. The `exit(2)` system call implementation takes the exit status and passes it to `exit1()` using the `W_EXITCODE` macro. The macro shifts the exit code by eight bits, so as to make room for any signal number. Thus, even though `exit(2)` passes 0 for the signal, it must also save the 8 most significant bits of the exit status, and park them in the process structure's `p_xhighbits` to avoid their truncation.

Exit reasons

`exit1()` calls `exit1_internal()`, which has been refactored in Darwin 16 to call `exit_with_reason()`. The voluntary termination through `exit1()` specifies no reason (`OS_REASON_NULL`), but signal-induced exit (from `postsig_locked()`, in bsd/kern/kern_sig.c) calls `build_signal_reason()`, an `OS_REASON_SIGNAL` object recording the signal number, process name and more.

An `os_reason` is a `kcdata` structured object allocated from the `os reasons` dedicated zone. It is defined in bsd/sys/reason.h, which also lists some two dozen `OS_REASON_*` constants. A few of these are used by kernel (e.g. `OS_REASON_[JETSAM/CODESIGNING]`, and most of the others used by user mode enforcer daemons (when killing the process via `terminate_with_payload()`, system call #520), or when process itself opts for assisted suicide (`[abort/os_fault]_with_payload()`, #521 and #529, as discussed in I/15). These all eventually merge with the signal death path (and thus, also with `exit_with_reason()`).

Figure 6-13: The `os_reason` structure (from bsd/sys/reason.h)



A main stage of exiting at the BSD plane is performed by `proc_prepareexit()` (also in bsd/kern/kern_exit.c). This handles the work of possibly generating a user mode crash report and/or a core dump, creating a corpse (as of Darwin 15), removing it from the `allproc` list, and inserting it at the head of the `zombproc`.

After `proc_prepareexit()` has returned, `exit_with_reason()` calls `task_terminate_internal()` (from osfmk/kern/task.c). This performs the exit at the Mach plane, destroying all task related objects (e.g. the IPC space, ledgers, VM Map, etc), and - most importantly - terminate each thread. The last thread to be terminate triggers `proc_exit()`, which performs the BSD cleanup - Asynchronous I/O, VFS namespaces, file descriptors, and more. The process is then also officially marked as a zombie, and its parent is signaled with `SIGCHLD`. The child will be granted eternal rest when its parent (or `launchd`, if adopting it) call `wait4()` to reap the exit status. At this point, however, a zombie process no longer holds any resources, aside from its process list entry.

Core dumps

Core generation is contingent on `CONFIG_COREDUMP` being `#defined`, and the `kern.coredump` MIB set to 1, which is the case in MacOS, but not on *OS where `SECURE_KERNEL` is set. S[UG]ID processes also must have the `kern.sugid_coredump` agree as well.

When the signal behavior is flagged `SA_CORE`, then `postsig_locked()` calls `coredump()` (from bsd/kern/kern_core.c) to generate the core. This checks the above MIBs, and additionally prevents SIP-restricted processes from generateing core dumps unless unrestricted debugging and/or Apple Internal settings are allowed. A final consideration is the process level `p_rlimit` at `RLIMIT_CORE` (settable by `setrlimit(2)` or the `ulimit(1)` command), which inhibits core size (or creation at all, if set to 0).

If all above prerequisite conditions are satisfied, then the task is suspended, and the core file is opened directly in kernel (via `vnode_open()`), using the name returned by `proc_core_name()`, taking into account the `kern.corefile` MIB. The rest of the code walks the process `vm_map` using `mach_vm_region_recurse()`, converting the regions into `LC_SEGMENT[_64]`, and calling `task_act_iterate_wth_args()` to apply `collectth_state()` for every task thread, generating a corresponding `LC_THREAD`.

Crash Reports

A crash report (generated be the system `CrashReporter`) is generated by throwing an `EXC_CRASH` Mach exception, through `task_exception_notify()` (from osfmk/kern/exception.c). As the comment in `proc_prepareexit()` shows, this is performed under very specific conditions:

Listing 6-14: Generating a crash report (via a Mach exception)

```
/*
 * Generate a corefile/crashlog if:
 *      The process doesn't have an exit reason that indicates no crash report should be created
 *      AND any of the following are true:
 *      - The process was terminated due to a fatal signal that generates a core
 *      - The process was killed due to a code signing violation
 *      - The process has an exit reason that indicates we should generate a crash report
 *
 * The first condition is necessary because abort_with_reason()/payload() use SIGABRT
 * (which normally triggers a core) but may indicate that no crash report should be created.
 */
if (!(PROC_HAS_EXITREASON(p) && (PROC_EXITREASON_FLAGS(p) & OS_REASON_FLAG_NO_CRASH_REPORT)) &&
     (hassigprop(WTERMSIG(rv), SA_CORE) || ((p->p_csflags & CS_KILLED) != 0) ||
     (PROC_HAS_EXITREASON(p) && (PROC_EXITREASON_FLAGS(p) &
                OS_REASON_FLAG_GENERATE_CRASH_REPORT)))) {
                        ...
      kr = task_exception_notify(EXC_CRASH, code, subcode);
     /* Nobody handled EXC_CRASH?? remember to make corpse */
      if (kr != 0) {
            create_corpse = TRUE;
       }

}
```

`task_exception_notify()` calls `exception_triage()` which performs the Mach exception handling already discussed in I/12 and depicted in Figure I/12-11. When the exception message gets to the host exception port, `launchd` transfers it to the `HostExceptionServer` (i.e. `ReportCrash`) for processing. Whether or not the `EXC_CRASH` was generated, `sys_perf_notify()` (also in osfmk/kern/exception.c) delivers an `EXC_RPC_ALERT` with a special code of `0xFF000001/`*`pid`* to the host port's handler.

Corpses

Another new concept added in Darwin 15 are **corpses**, discussed in Volume I/15. The reasoning and rationale for corpses has been well detailed in the header comment of osfmk/corpses/corpse.c:

Listing 6-15: The overview and functionality of corpses, as detailed in osfmk/corpses/corpse.c

```
/*
 * Corpses Overview
 * ================
 *
 * A corpse is a state of process that is past the point of its death. This means that process has
 * completed all its termination operations like releasing file descriptors, mach ports, sockets and
 * other constructs used to identify a process. For all the processes this mimics the behavior as if
 * the process has died and no longer available by any means.
 *
 * Why do we need Corpses?
 * -----------------------
 * For crash inspection we need to inspect the state and data that is associated with process so that
 * crash reporting infrastructure can build backtraces, find leaks etc. For example a crash
 *
 * Corpses functionality in kernel
 * ===============================
 * The corpse functionality is an extension of existing exception reporting mechanisms we have. The
 * exception_triage calls will try to deliver the first round of exceptions allowing
 * task/debugger/ReportCrash/launchd level exception handlers to  respond to exception. If even after
 * notification the exception is not handled, then the process begins the death operations and during
 * proc_prepareexit, we decide to create a corpse for inspection. Following is a sample run through
 * of events and data shuffling that happens when corpses is enabled.
 *
 *    * a process causes an exception during normal execution of threads.
 *    * The exception generated by either mach(e.g GUARDED_MARCHPORT) or bsd(eg SIGABORT, GUARDED_FD
 *      etc) side is passed through the exception_triage() function to follow the thread -> task -> hos
 *      level exception handling system. This set of steps are same as before and allow for existing
 *      crash reporting systems (both internal and 3rd party) to catch and create reports as required.
 *    * If above exception handling returns failed (when nobody handles the notification), then the
 *      proc_prepareexit path has logic to decide to create corpse.
 *    * The task_mark_corpse function allocates userspace vm memory and attaches the information
 *      kcdata_descriptor_t to task->corpse_info field of task.
 *      - All the task's threads are marked with the "inspection" flag which signals the termination
 *        daemon to not reap them but hold until they are being inspected.
 *      - task flags t_flags reflect the corpse bit and also a PENDING_CORPSE bit. PENDING_CORPSE
 *        prevents task_terminate from stripping important data from task.
 *      - It marks all the threads to terminate and return to AST for termination.
 *      - The allocation logic takes into account the rate limiting policy of allowing only
 *        TOTAL_CORPSES_ALLOWED in flight.
 *    * The proc exit threads continues and collects required information in the allocated vm region.
 *      Once complete it marks itself for termination.
 *    * In the thread_terminate_self(), the last thread to enter will do a call to proc_exit().
 *      Following this is a check to see if task is marked for corpse notification and will
 *      invoke the the task_deliver_crash_notification().
 *    * Once EXC_CORPSE_NOTIFY is delivered, it removes the PENDING_CORPSE flag from task (and
 *      inspection flag from all its threads) and allows task_terminate to go ahead and continue
 *      the mach task termination process.
 *    * ASIDE: The rest of the threads that are reaching the thread_terminate_daemon() with the
 *      inspection flag set are just bounced to another holding queue (crashed_threads_queue).
 *      Only after the corpse notification these are pulled out from holding queue and enqueued
 *      back to termination queue
 *
```

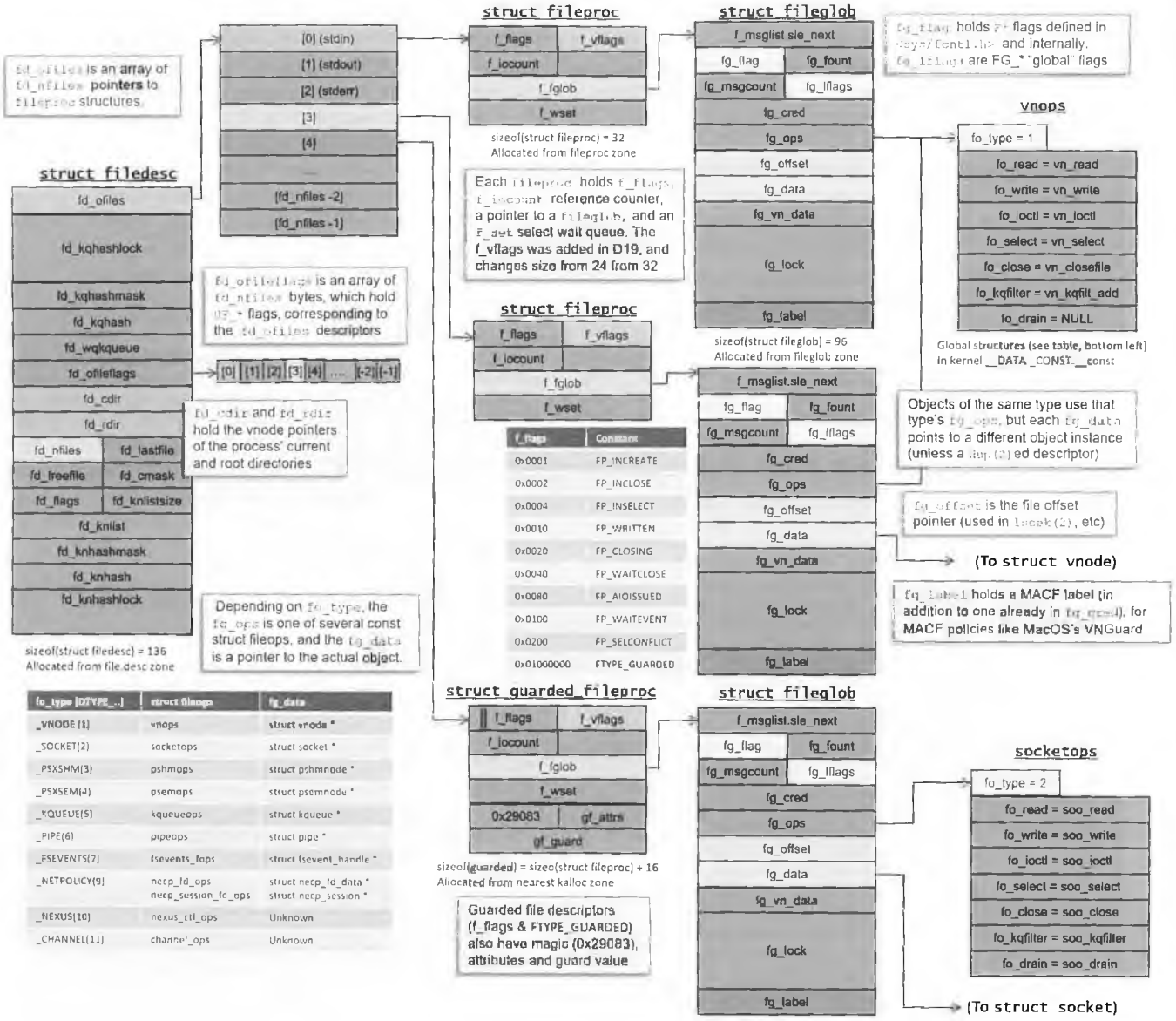
A corpse is created when `proc_prepareexit()` sends the `EXC_CRASH` exception, but none of the exception triage handlers claim it (as shown in Listing 6-14). A call to `task_mark_corpse()` (from osfmk/kern/task.c) sets up the corpse by calling `task_collect_crash_info()`, and sets the `TF_CORPSE` and `TF_PENDING_CORPSE` flags.

If memory for a corpse can be allocated, `proc_update_corpse_exception_codes()` and `populate_corpse_crashinfo()` (both in bsd/kern/kern_exit.c) fill the corpse informatio. This includes a slew of `TASK_CRASHINFO_*` `kcdata` items, `#defined` in osfmk/kern/kcdata.h. When the last thread in the task exits, `task_deliver_crash_notification()` delivers the `EXC_CORPSE_NOTIFY`. Whoever catches this exception can map the course information, and the corpse can be removed.

File Descriptors

File descriptors, which appear in user mode as simple integers, are in fact quite complicated beasts. A menagerie consisting of several object types - `filedesc`, `fileproc` and `fileglob`. These are connected through the relationships shown in Figure 6-16. Let us now investigate each one of these objects, in turn.

Figure 6-16: A visualization of a process file descriptors



The `struct filedesc`

Our exploration begins with the `proc` structure's `p_fd` field, which is a pointer to a `struct filedesc`. This structure holds metadata common to all file descriptors, including the process working directory (`fd_cdir`), root directory (`fd_rdir`).

Recall, that user mode file descriptors are merely integers. In kernel, mode these integers serve as indices into the `p_fd`'s `fd_ofiles`, which is an array of `fd_nfiles` pointers. Access to some file descriptor *i* of a given process, therefore, is achieved through the `struct proc`'s `p_fd->fd_ofiles[i]`. The array entry is a pointer to a `struct fileproc`, and `fp_lookup()` (from bsd/kern/kern_descrip.c) can be used to retrieve it by finding the array entry, and ensuring it is not NULL or flagged in the corresponding `fd_ofileflags` entry by the `UF_RESERVED` bit. The two arrays are closely connected, and the `fdfile(fd)` and `fdflags(fd)` macros (from bsd/sys/filedesc.h) are frequently used to shorten the two pointer dereferences (from the process to its `p_fd`, and from there to the respective `fd` array entry). Access to descriptors is protected by the `p_fdmlock` of the `proc` structure, a `mtx_lck_t` often accessed through `proc_fdlock[_spin]()` and `proc_fdunlock()`.

The `struct fileproc`

The `struct fileproc` is a small structure with just a few fields. The first two provide a 32-bit reference count (`f_iocount`), and 32-bit flags (`f_flags`). The reference count is increased by `fp_lookup()`, and decreased by `fp_drop()` or one of its variants (`fp_drop_written()` or `fp_drop_event()`), which also set corresponding flags.

The flags hold various state indications (e.g. `FP_IN[CREATE/CLOSE/SELECT]`..) in their lower 16 bits, and a type value in the upper 3 (conveniently accessible through the `FILEPROC_TYPE` macro). Although this would allow for up to 8 types, only two types - `FTYPE_SIMPLE` or `FTYPE_GUARDED` - are presently defined. In this way, a guarded descriptor is marked in the kernel by an `FTYPE_GUARDED` indication in the corresponding `struct fileproc`'s `f_flags`.

The `struct fileprocs` marked guarded can be "converted" to `guarded_fileprocs`, using the `FP_TO_GFP` macro. In practice, no actual conversion need be made, since the `guarded_fileproc` is implemented by adding a few fields immediately after the `fileproc`: The `gf_magic` (a 32-bit `GUARDED_FILEPROC_MAGIC`, 0x29083), the `gf_attrs` bitmask (specifying the type of `GUARD_[CLOSE/DUP/SOCKET_IPC/FILEPORT/WRITE]`), and the `gf_guard`, the 64-bit guard value needed to access the descriptor for the operation. Guarded descriptors are retrieved by `fp_lookup_guarded()` (from bsd/kern/kern_guarded.c), which calls `fp_lookup()` internally, but also verifies the `GUARDED_FILEPROC_MAGIC` and a match between the `gf_guard` and its argument.

Darwin 19 adds an additional `f_vflags`, bringing the structure size up from 24 bytes to 32 (due to an additional 4 bytes of padding). The last two fields of the `fileproc` are the `f_fglob` and `f_wset`. The `f_wset` points to a wait queue when the descriptor is part of a `select(2)` operation (`FP_INSELECT`). The `f_fglob` field, holds a pointer to a `struct fileglob`.

The `struct fileglob`

The `struct fileglob` holds the details of the open object. Contrary to the name, this structure is actually unique to every descriptor: If two processes end up opening the same objet - or even two descriptors in the same process do - a different backing `fileglob` is used in each case. The object backing the descriptor is stored in the `fileglob`'s `fg_data` - a void pointer, which is commonly type cast to a `struct vnode` (for ordinary files), but can point to any of the structures representing descriptor object types (e.g. `struct socket`, etc).

In order to know which cast is approriate, a different field, the `fg_ops` is consulted. This is a `struct fileops` structure, which is defined inline as part of the `struct fileglob`. The `fileops`'s `fo_type` is an integer `DTYPE_*` value, describing what the underlying file object is. As its name implies, the `fileops` structure also contains the file operations - function pointers to implementations of common descriptor-based system calls, as shown in Table 6-17.

Table 6-17:

Function	Operation
`fo_read`(*`fp, uio, flags, ctx`*)	Read from a file descriptor
`fo_write`(*`fp, uio, flags, ctx`*)	Write to a file descriptor
`fo_ioctl`(*`fp, cmd,data, ctx`*)	Perform an ioctl *cmd* with *data* buffer
`fo_select`(*`fp,which, wql, ctx`*)	Select on file descriptor (as part of an `FD_SET`)
`fo_close`(*`fg, ctx`*)	Close file pointed to by `fileglob` *fg*
`fo_kqfilter`(*`fp,kn,kev,ctx`*)	Called on kqueue filter attachment
`fo_drain`(*`fp, ctx`*)	Drain pending I/O operations.

As the table shows, the `fo_*` operations (with the exception of `fo_drain`) are very similar to the syscalls which call them. The `fp` is a `struct fileproc`, as looked up from the file descriptor. The `ctx` is a VFS context (described in the next chapter), which VFS maintains for kernel and subsystem interopreability. Note, that not all operations necessarily be defined for a particular file type.

File types

As shown in Figure 6-16, the `struct fileglob` entries point to an `fg_ops` structure, wherein the file type is maintained, holding one of (presently) 10 defined types. Vnodes, FSEvents and the networking types (sockets, netpolicies, nexuses and channels) are described in their own chapters (7 and 14, respectively), so in this section we consider the remaining types.

POSIX Shared Memory

POSIX shared memory allows two or more processes, not necessarily a parent and a child, to establish a shared memory handle. The handle, created by `shm_open(2)`, is given a name, but unlike a normal `open(2)` it need not be the name of a file. Nonetheless, the returned descriptor can then be used with `mmap(2)` just as any vnode descriptor would be. When the shared memory needs to be disposed of, a call to `shm_ulink()` with the same name reduces the reference count by one, and may - if the count drops to zero - destroy the mapping. The caller is expected to `munmap(2)` the memory as well.

In kernel mode, the `fg_ops` points to the global `pshmops` structure, but of its functions save the `fo_close` handler (`pshm_closefile`) return `ENOTSUP`. The `fg_data` points to a `pshmnode` structure, which holds the information required for maintaining the shared memory mapping. The structure is minimal, consisting of the `mapp_addr` and a `pinfo` pointer to a `struct pshminfo`*. The `pinfo` is the one holding the important details, in an 80 byte structure allocated from the nearest kalloc zone (`kalloc.128`). Output 6-18 shows the `(com.apple.C)FNetwork.defaultStorageSession` shmem, used in MacOS:

Output 6-18: The `(com.apple.C)FNetwork.defaultStorageSession` POSIX shared memory object

```
#
# Find some shmem holder using procexp over all file descriptors:
#
root@Zephyr (~) # procexp all fds | grep shmem | head -1
familycircled      541 FD  7r  shmem FNetwork.defaultStorageSession@0x10c99a000
#
# Get address of fg_data (here shown with xnoop - can also be obtained with DTrace)
#
root@Zephyr (~) # xnoop dumppid 541 | grep -i Shmem
   7: Shmem (0xffffff802835bfa0)
#
# Dump pshmnode:
#
root@Zephyr (~) # xnoop object 0xffffff802835bfa0
Object at address 0xffffff802835bfa0 is kalloc.16
root@Zephyr (~) # xnoop dump 0xffffff802835bfa0,16 smart
                                                  kalloc.128
0xffffff802835bfa0 00 a0 99 0c 01 00 00 00      0xffffff803c3b7080
#
# Dump pshminfo:
#
root@Zephyr (~) # xnoop dump 0xffffff803c3b7080,128  smart
                  pshm_flags  ...usecount          pshm_length
0xffffff803c3b7080 1c 00 00 00 0b 00 00 00   00 10 00 00 00 00 00 00
                   pshm_mode    pshm_uid      pshm_gid    pshm_name
0xffffff803c3b7090 80 01 00 00 f5 01 00 00   14 00 00 00 46 4e 65 74   ............FNet
0xffffff803c3b70a0 77 6f 72 6b 2e 64 65 66   61 75 6c 74 53 74 6f 72   work.defaultStor
0xffffff803c3b70b0 61 67 65 53 65 73 73 69   6f 6e 00 00 00 00 00 00   ageSession......
                                                  MAC Labels
0xffffff803c3b70c0 00 00 00 00 00 00 00 00      0xffffff803c5c0540
                  ... uninitialized data up to zone element size ...
#
# As an exercise, the reader is encouraged to create an equivalent pshminfo dumping D script
#
root@Zephyr (~) # xnoop dump 0xffffff803c3b7080 pshminfo
pshm_flags:      0x1c      (ALLOCATED, MAPPED, INUSE)
pshm_use_count:  11
pshm_length:     0x100
pshm_mode:       0x180
pshm_name:       FNetwork.defaultStorageSession
pshm_memobjects: (NULL)
pshm_label:      0xffffff803c5c0540
```

Although NULL in the above output, the `pshm_memobjects` field may point to a `struct pshmobj`. This is a 24 byte structure holding the `pshmo_size`, a `pshmo_memobject`, and a `pshmo_next`, allowing multiple objects to be chained. The `pshmo_memobject` is declared as a void pointer, but in fact holds a `mem_entry_name_port_t`, which is a low level Mach memory object, created with `mach_make_memory_entry_64` (discussed in @MACHMEM).

* - Up until mid Darwin 18 the structure also held a `map_size` and `pshm_usecount`, and was thus 32 bytes. The `map_size` "`XXX unused ?`", and `pshm_usecount` was redundant since the `pshminfo` also holds a use count.

KQueues

KQueues are a kernel event notification mechanism which Darwin adopted from BSD. As described in the Interlude of Chapter 8, these important descriptor types (fundamental to GCD) are special file descriptors created by a call to `kqueue()`, over which event filters can be set with the `kevent[64]` and `kevent_qos()` system calls and the `EV_SET[64]` macros. The same system calls can then be used to block on the descriptors, and read event records as they are generated by various kernel subsystems.

What appears in user mode as a kqueue file descriptor can be in kernel one of several objects: A `kqfileq`, `kqworkq`, or `kqworkloop`. These (all in bsd/sys/eventvar.h) can be considered "subclasses" of the `struct kqueue`, which they all have as their first member. They are collapsed into a transparent union as a `kqueue_t`, but do note that the structures are different sizes, and are thus allocated from different, dedicated Mach zones.

User mode can only alloc a `kqfile`, but internally the `flags` argument of `kqueue_alloc()` can contain `KEVENT_FLAG_WORKQ` or `KEVENT_FLAG_WORKLOOP`. Since our interest is in descriptors, let us restrict ourselves to the `kqfile`. The sources are not yet available at the time of writing, using the existing definition in bsd/sys/event.h and aided by DTrace's `ctfdump(1)` we can piece together the object definition:

Listing 6-19: The `struct kqueue` (from XNU 4903's bsd/sys/eventvar.h), with Darwin 19 notes

```
struct kqueue {
    struct {
    struct waitq_set    kq_wqs;       /* private waitq set */
    lck_spin_t          kq_lock;      /* kqueue lock */
    uint16_t            kq_state;     /* state of the kq */
    /* Darwin 19 overloads this with kq_waitq_hook, in a union: */
    uint16_t            kq_level;     /* nesting level of the kq */
    uint32_t            kq_count;     /* number of queued events */
    struct proc        *kq_p;         /* process containing kqueue */
    struct knote_locks  kq_knlocks;   /* list of knote locks held */
    /* Darwin 19 does away with kq_reqlock, reducing structure (by 80) to 168 bytes */
    lck_spin_t          kq_reqlock;   /* kqueue request lock */
    }; /* make sure struct padding is put before kq_queue */
    struct kqtailq      kq_queue[0];      /* variable array of queues */
};

// A kqtailq is defined in bsd/sys/event.h to be a TAILQ_HEAD(kqtailq, knote);

struct kqfile {
        struct kqueue       kqf_kqueue;     /* common kqueue core */
        struct kqtailq      kqf_queue;      /* queue of woken up knotes */
        struct kqtailq      kqf_suppressed; /* suppression queue */
        struct selinfo      kqf_sel;        /* parent select/kqueue info */
};

typedef union {
        struct kqueue       *kq;
        struct kqworkq      *kqwq;
        struct kqfile       *kqf;
        struct kqworkloop   *kqwl;
} __attribute__((transparent_union)) kqueue_t;
```

XNU supports **dynamic kqueues**, which are maintained at the `filedesc` level in the `fd_kqhash` table. This is an array of `struct kqlist` (i.e. lists of `kqueues`), hashed by the Kqueue ID (read: file descriptor number) supplied by the caller with the `fd_kqhashmask` and protected by `fd_kqhashlock`. The `kqueue_hash_lookup()` function can retrieve the `struct kqueue` associated with a given *id* for a calling process.

The `knotes` at the end of the `struct kqueue` are what user mode sees as a `kevent[/64_s/qos_s]` structures (q.v. I/8-33). The `struct knotes` are allocated from the dedicated `knote zone`. Although the structure has remained stable over several Darwin versions, Darwin 19 reduces it from 144 bytes to 128, somewhat simplifying it. The important component in it is the `kevent_internal_s` (extended in Darwin 19 to 72 bytes). Once again, `ctfdump(1)` comes to our aid:

Listing 6-20: The `struct knote`, reconstructed in Darwin 19 using `ctfdump`

```
struct knot
/* 0x00 */ TAILQ_ENTRY(knote)  kn_tqe;       /* linkage for tail queue */
/* 0x10 */ SLIST_ENTRY(knote)  kn_link;      /* linkage for search list */
/* 0x18 */ SLIST_ENTRY(knote)  kn_selnext;   /* klist element chain */
/* 0x2? */ int kn_status:12;    /* KN_[ACTIVE/QUEUED/../SUPPRESSED], etc. */
           :long  kn_qos_index : 4,
/* 0x22 */ ulong  kn_qos_override:3,
                  kn_is_fd:1,
                  kn_vnode_kqok:1,
                  kn_vnode_use_ofst:1,
                  kn_packed_type:42;
/* 0x28 */ union {
        void    *kn_hook;
        uint32_t kn_hook32;
        uint64_t kn_hook64; }
/* 0x30 */  union {
    struct fileproc *kn_fp;
    struct proc *kn_proc;
    struct ipc_mqueue *kn_mqueue;
    struct thread_call *kn_thcall;
    struct thread *kn_thread; }
/* 0x38 */ struct kevent_internal_s  {  /* 72 bytes */
             uint64_t kei_ident;
/* 0x40 */  int8_t  kei_filter;    /* filter for event */
/* 0x41 */  uint8_t  kei_filtid;   /* filter id  */
/* 0x42 */  uint16_t kei_flags;    /* general flags */
/* 0x44 */  int32_t  kei_qos;      /* quality of service */
/* 0x48 */  uint64_t kei_udata;    /* opaque user data identifier */
/* 0x50 */  uint32_t kei_fflags;   /* filter specific data */
/* 0x54 */  uint32_t kei_sfflags;
/* 0x58 */  int64_t  kei_sdata;
/* 0x60 */  uint64_t kei_ext[4];   /* filter-specific extensions */
   } kn_kevent;
};
```

Each `knote` maintains three linkage chains. The first is the `kn_tqe`, a `TAILQ_ENTRY` linking it to its `kqueue`'s `TAILQ`. The second is `kn_link`, a `SLIST_ENTRY` linking it to a search list, from the `filedesc`'s `fd_knhash` and `fd_knlist`. The third is `kn_selnext`, another `SLIST_ENTRY` linking it to the process's `p_klist` element chain.

The "conversion" from the kernel structure to the user visible one is performed by `kevent_copyout()`, called from `kevent_internal()` (the common implementation of `kevent[/64/qos/id]()` (#363., #369, #374 and #375, respectively), and from `kevent_callback()` invoked from `kqueue_process()` (itself called from `kqueue_scan()`, which is where `kevent_internal()` blocks, as it awaits more events. There are also `kevent_copyout_dynkq[ext]info()` routines, which handle the `proc_info()` (#336) syscall's `PROC_INFO_CALL_PIDDYNKQUEUEINFO` callnum.

Pipes

UN*X pipes are unidirectional IPC mechanisms, appearing in user mode as descriptor pairs (0,1) created by the `pipe(2)` system call. Data written to one end of the pipe (`fd[1]`) can be read from the read end of the pipe (`fd[0]`).

The implementation of pipes is contained in two files - bsd/sys/pipe.h, and bsd/kern/sys_pipe.c. The `struct pipe` provides the kernel representation of pipes, with one structure for each end of the pipe, allocated from the `pipe.zone`. Although commonly allocated in sequence, there was no guarantee they would be adjacent in kernel memory. Darwin 19 instead makes the `pipe.zone` 400 bytes (though the structure size remains 192 bytes), which is enough for two pipe structures to be allocated together, thus ensuring peer adjacency.

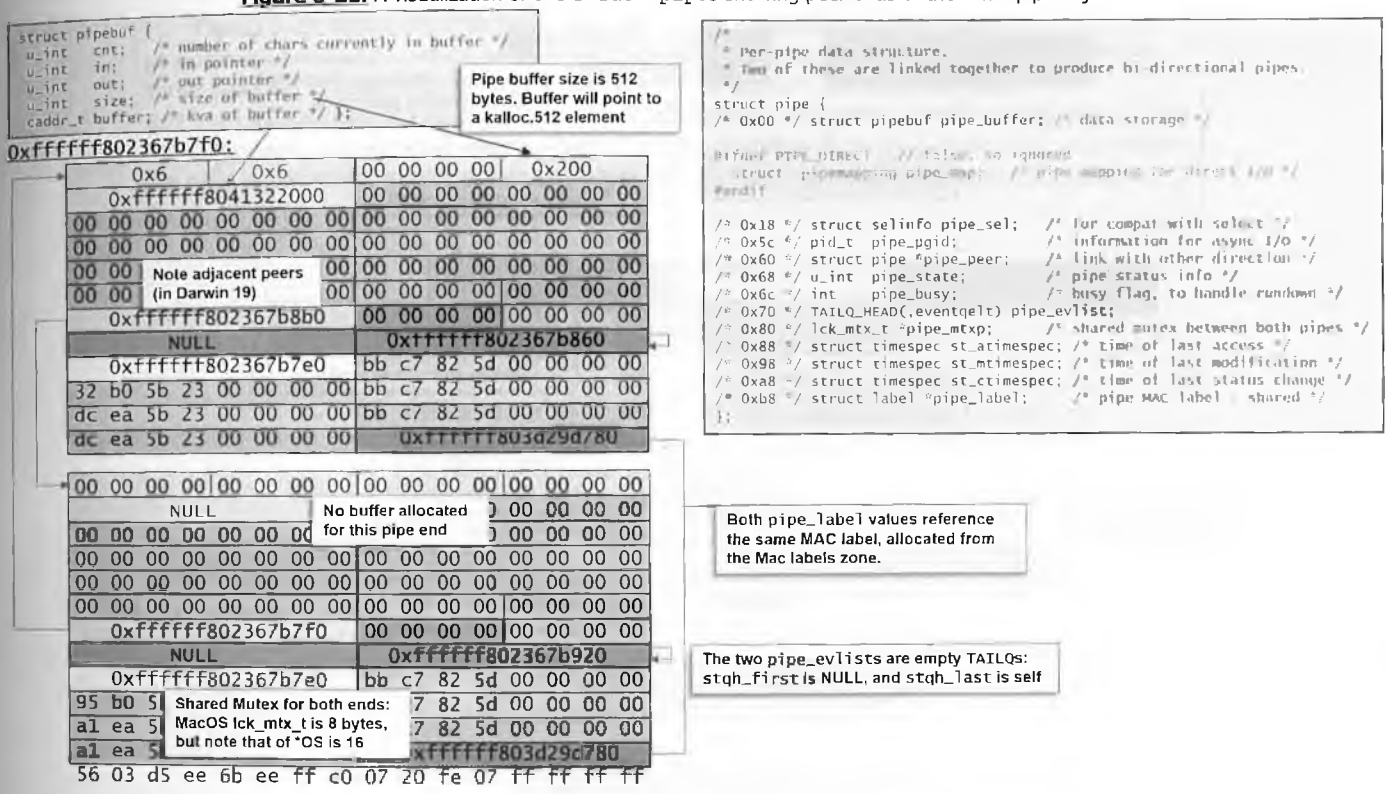
The `struct pipe` holds several fields*, including the `timespecs` for access, modification and change, a `lck_mtx_t`, a link to the peer end, and an embedded `struct pipebuf` used to maintain the pipe buffer, which is a circular buffer. The `struct pipebuf` holds the `size`, `cnt` (count of bytes used), `in` (write) and `out` (read) offsets, as well as a pointer to the actual `buffer`. The header comment in bsd/kern/sys_pipe.c depicts various cases for the `in` and `out` pointer with respect to the `cnt` of bytes, well explained and need not be repeated here.

* - There is also a `struct pipemapping` embedded in the pipe, allowing for direct transfers between processes for pipes, but this is only `#if PIPE_DIRECT`, which is `false` across Darwin flavors.

The pipe buffer is `kalloc()`ed in the first call to `pipe_write()`. This routine calls `choose_pipespace()`, which considers an array of `pipesize_blocks`, ranging in powers of two from 512 through 8,192, then `PIPE_SIZE` (#defined to be 16,384 in bsd/sys/pipe.h), then double that (32,768), to find the most adequate allocation size. The buffer is then allocated using `pipespace()`, or its read-safe variant `expand_space()` - both of which may potentially resize the buffer, if it has been filled. If the buffer cannot be resized, the write operation returns `-ENOMEM`. As the buffer is circular, if the reader consumes buffer data it can be overwritten, thus alleviating the need for multiple reallocations.

Note, however, that the pipe's design is such that data is expected to be unidirectional. Thus, only the first descriptor is associated with a buffer, and the other's `struct pipebuf` remains `bzero()`ed. Both structs share the same mutex and MAC label, and often have the same timestamps. Figure 6-21 visualizes a `pipe(2)` descriptor pair:

Figure 6-21: A visualization of two `struct pipes` showing peer ends of the same pipe object



A pipe dies when its read end is closed, in which case the writer gets a `SIGPIPE` when attempting a write (unless suppressed). When the write end is closed, data will be kept until the reader closes its descriptor as well. Dead pipes can normally be immediately disposed of by the system, save for a specific case wherein they are marked for write select. As the comment in bsd/kern/sys_pipe.c explains, the way to avoid a panic is to defer the pipe free operation, by marking its `pipe_state` as `PIPE_DEAD`, and enqueueing it for future garbage collection.

The `pipe_garbage_collect()` routine provides this functionality by allocating a `pipe_garbage` structure. This minimalistic 24-byte structure, consisting of a pointer to the dead pipe, a pointer to the next dead entry, and a 64-bit timestamp, is allocated from a dedicated `pipe garbage zone`. Garbage collection is triggered on every `pipeclose()` operation (not just write selected ones), as `pipe_garbage_collect()` first attempts to iterate over the queue and free any entries found to be over `PIPE_GARBAGE_AGE_LIMIT` (5 seconds) old. If a pipe was specified for garbage collection, it is then added to the queue.

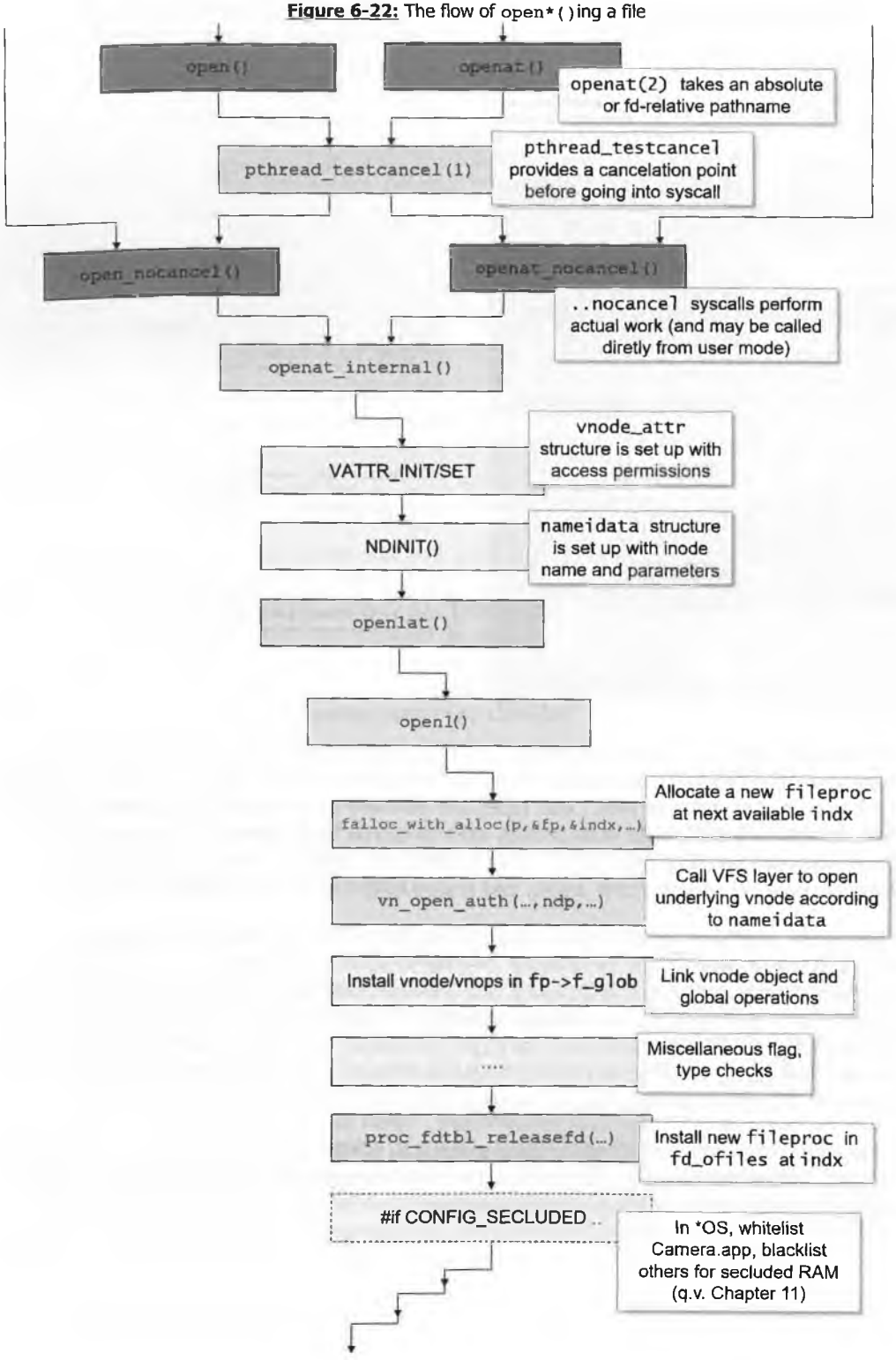
Pipe usage statistics can be set through `sysctl(2)` MIBs, but only if XNU is compiled with `PIPE_SYSCTLS` (which it is not, by default). The same settings also once allowed for imposing the pipe KVA wired limit, but this is presently unused.

File I/O

Files exists as sources of input and targets of output, and the largest category of system calls in UN*X kernels is for I/O related calls. XNU is no exception to this, and Apple has further added proprietary system calls to handle I/O as well. Since the POSIX standard ones are well described elsewhere, this section assumes familiarity with them, and focuses on their implementation.

Starting with the basic operation of opening a file - whether through the traditional `open(2)` (#5), its newer variant, `openat(2)` (#464), or the `_nocancel` variants (#398 and #464, respectively), all of which funnel to `openat_internal()` (from bsd/vfs/vfs_syscalls.c). This routine calls on `NDINIT` to initialize a `vnode` object, and then call `openlat()`, which eventually gets to `openl()`.

Figure 6-22: The flow of `open*()`ing a file

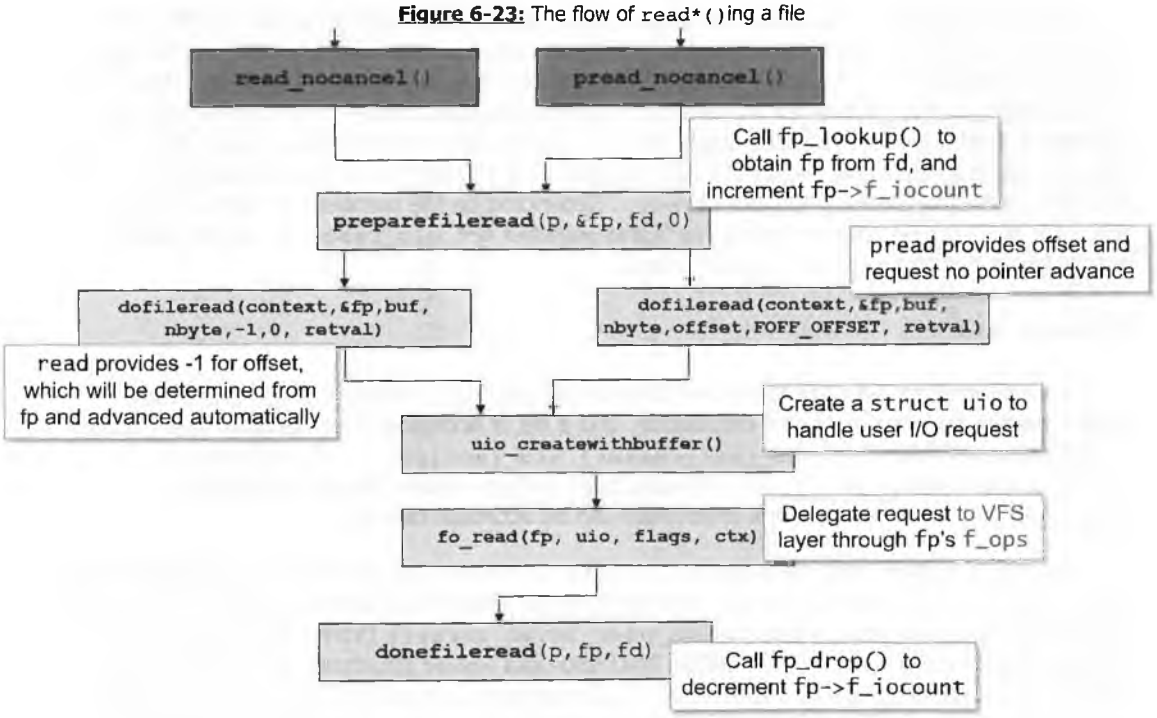


`openl()` calls `falloc_withalloc()`, providing the `fileproc_alloc_init()` function as the allocator in the `fp_zalloc` argument. This allocates a fileproc from the BSD `M_FILEPROC`, and its underlying `fileproc` zone. `falloc_withalloc()` will return the descriptor number in the `indx` (out) argument, and the new `fileproc` in the `fp` (out) argument. A call to `procfdtbl_releasefd()` installs the `fp` in the process' `fd_ofiles` table. A call to `vn_open_auth()` then delegates the operation to the VFS layer (discussed in the next chapter). If the VFS layer open operation is successful, the `f_glob`'s `fg_data` can be set to the (opaque) vnode pointer.

With the file open, other operations can be performed on the descriptor. Taking the read family as an example, we find traditional `read(2)` (#3) and its `..._nocancel` variant (#396), and also `readv(2)` (#120), `pread(2)` (#153) and their `..nocancel` variants (#411 and #414). These syscalls (in bsd/kern/sys_generic.c) all follow similar patterns, with the calls merging with their respective `..nocancels`.

Figure 6-23 shows the flow from this point: The `..nocancels` then call `preparefileread()`, which wraps `fp_lookup()` to retrieve the `fileproc` and perform some type/flag checks, and then `dofileread()` to perform the read operation. The `dofileread` routine initializes a `uio` object (discussed imminently), which is then passed (via the `fo_read()` wrapper) to the `fo_read` handler specified in the `fileproc`'s `f_ops`. When the read is complete, `donefileread()` calls `fp_drop()` to release the reference taken by `fp_lookup()`, under the process `p_fdlock`.

Figure 6-23: The flow of `read*()`ing a file



The devil, of course, is in the details - specifically, how the `fileproc`'s `f_ops` handler (an alias for the `fileglob`'s `fg_ops`, installed by `openl()`, as shown in Figure 6-22) performs the read operation. This is the responsibility of VFS, and discussed in the next chapter.

The struct uio

User mode I/O requests are standardized into a struct uio, which represents the metadata detailing an I/O request. This includes indicators (uio_segflg, set to UIO_[SYS/USER]SPACE variants) and the direction of the request (uio_rw, set to UIO_[READ/WRITE]). The nature of I/O requests is such that they may be satisfied completely or partially (or not at all), for which the uio_resid() retrieves the residual (remaining) count of bytes. Additional uio_flags (all private) define UIO_FLAGS_INITED, .._WE_ALLOCED (indicating kfree() required) and .._IS_COMPRESSED_FILE (used internally by VFS).

To allow I/O requests to span more than one buffer, uio make use of one or more I/O vector structures, allocated immediately following it in memory. An I/O vector is either a struct kern_iovec or struct user_iovec (from bsd/sys/uio_internal.h), both of which define a buffer, pointed to by iov_base, spanning iov_len bytes. The uio also specifies how many I/O vectors were allocated (uio_max_iovs), how many are currently in use (uio_iovcnt) are currently in use, and which is the current active iovec (pointed to from uio_iovs).

When uio_create() is called to instantiate a new uio, all of its iovecs are empty (iov_base=NULL, and iov_len=0). Vectors are added one at a time, by calling uio_addiov(*baseaddr*, *len*). Doing so increments the uio_iovcnt, and adds the *len* to the uio_resid_64 field. Figure 6-24 (next page) details the process of uio_create() initializing a new uio, and then uio_addiov().

Because a struct uio is allocated along with the iovecs, its size will be a function of how many uio_max_iovs it has. The UIO_SIZEOF macro can be used to determine the size required for various struct uios based on the number of iovecs. UIOs may be created with a pre-existing (usually, stack) buffer, using uio_createwithbuffer() - but this is a private (unexported) KPI. When created through the uio_create() KPI (from com.apple.kpi.bsd) the struct uio are always kalloc()ed, and the UIO_FLAGS_WE_ALLOCED flag is set, indicating that uio_free() will call kfree(). Depending on the number of iovecs, they will end up in kalloc.64 (at a minimum) or higher. Another KPI, uio_reset(), can be used to reinitialize an existing struct uio for reuse.

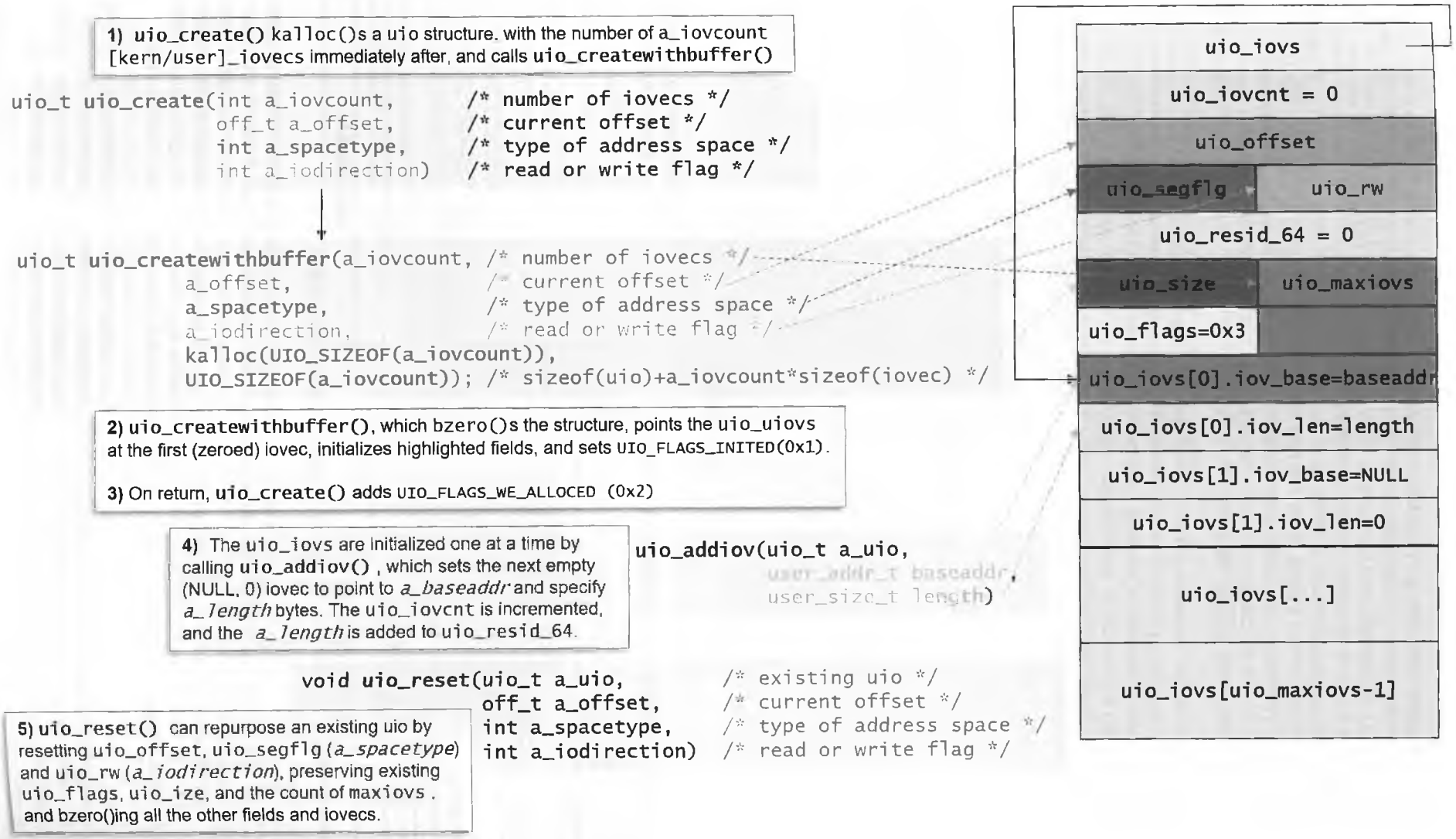
Handling uios

In an attempt to keep this important structure as opaque as possible, the bsd/sys/uio.h header merely provides a forward declaration, and a set of accessors (from bsd/kern/kern_subr.c) to manipulate its fields (e.g. uio_[set]resid(), uid_[set]rw(), uio_iovcnt(), uio_curriov[base/len](), etc). The structure implementation remains hidden in bsd/sys/uio_internal.h, so that none of its fields can be accessed directly.

The POSIX system calls of readv(2)/writev(2) allow working with iovecs directly from user mode, by uio_create()ing a uio large enough to hold the specified number of iovecs provided by the caller, and calling copyin_user_iovec_array() (from bsd/kern/kern_subr.c) to populate the uio's iovecs accordingly. XNU also uses iovec structures even in the single buffer read(2) and write(2) system call implementations of dofile[read/write] (from bsd/kern/sys_generic.c. These prepare a static buffer with one iovec (i.e. of UIO_SIZEOF(1) bytes), and then the user input arguments of buf and nbytes are put into the iovec's fields through uio_addiov().

When servicing a uio_t, calling uiomove[64]() is the preferred way to move a buffer in or out of the iovecs without breaking the abstraction. uiomove[64]() iterates over all of the iovecs in a uio, using one of copyin(), copyout(), copywithin() or copypv() (depending on uio_segflg's indication of the address space type) to move the data. When some or all of the request has been satisfied, uiomove calls uio_update() with the count of bytes, as it will advance the uio_offset, decrement uio_resid_64 and advance uio_iovs to the next iovec, if necessary. An example of this can be found in Listing 7-34, as part of an experiment dealing with implementing a simple character device driver.

Figure 6-24: Creating and populating a `struct uio`



Asynchronous I/O

Asynchronous I/O (aio) is the facility by means of which a thread can issue an I/O request, yet not have to block for its result. Instead, the requester can instruct that the I/O completion (or error) be delivered by means of a notification (e.g. a signal), or simply be picked up by the thread at some later, more adequate time.

XNU inherited its aio BSD, which implemented it as far back as FreeBSD 3.0 in order to support the POSIX `aio*` interfaces. The implementation is contained in bsd/kern/kern_aio.c, with private kernel definitions in bsd/sys/aio_kern.h, and the structures and system calls shared with user mode in bsd/sys/aio.h. The key structure is the `struct aiocb`, which contains the file descriptor, buffer, priority and requested signal notification on completion/error. The `aiocb` is used as the argument in all the system calls, shown in Table 6-25:

Table 6-25: The asynchronous I/O calls of XNU

#	system calls	Description
313	**aio_fsync**(*op, aiocbp*)	Perform asynchronous `fsync(2)`
314	**aio_return**(*aiocbp*)	MUST be called for *aiocbp* at end of lifecycle, to return status (count of bytes)
315	**aio_suspend**(*aiocblist, nent, timeoutp*)	Block until at least one of the *nent* requests in *aiocblist* complete, process is singaled, or *timeoutp* expires. `aio_suspend_nocancel` is Apple extension
421	**aio_suspend_nocancel**(...)	
316	**aio_cancel**(*fd, aiocbp*)	Cancel present or specific*aiocbp* for descriptor *fd*
317	**aio_error**(*aiocbp*)	Return status/error of *aiocbp* (0, error code, `EINPROGRESS`, or -1 if own error)
318, 319	**aio_[read/write]**(*aiocbp*)	Perform asynchronous read/write request
320	**lio_listio**(*mode, aiocblist, nent, sigp*)	Initiate the *aiocblist*, containing *nent* entries, with a *mode* indicating `LIO_[NO]WAIT` and `aio_sigevent` signal behavior in *sigp*

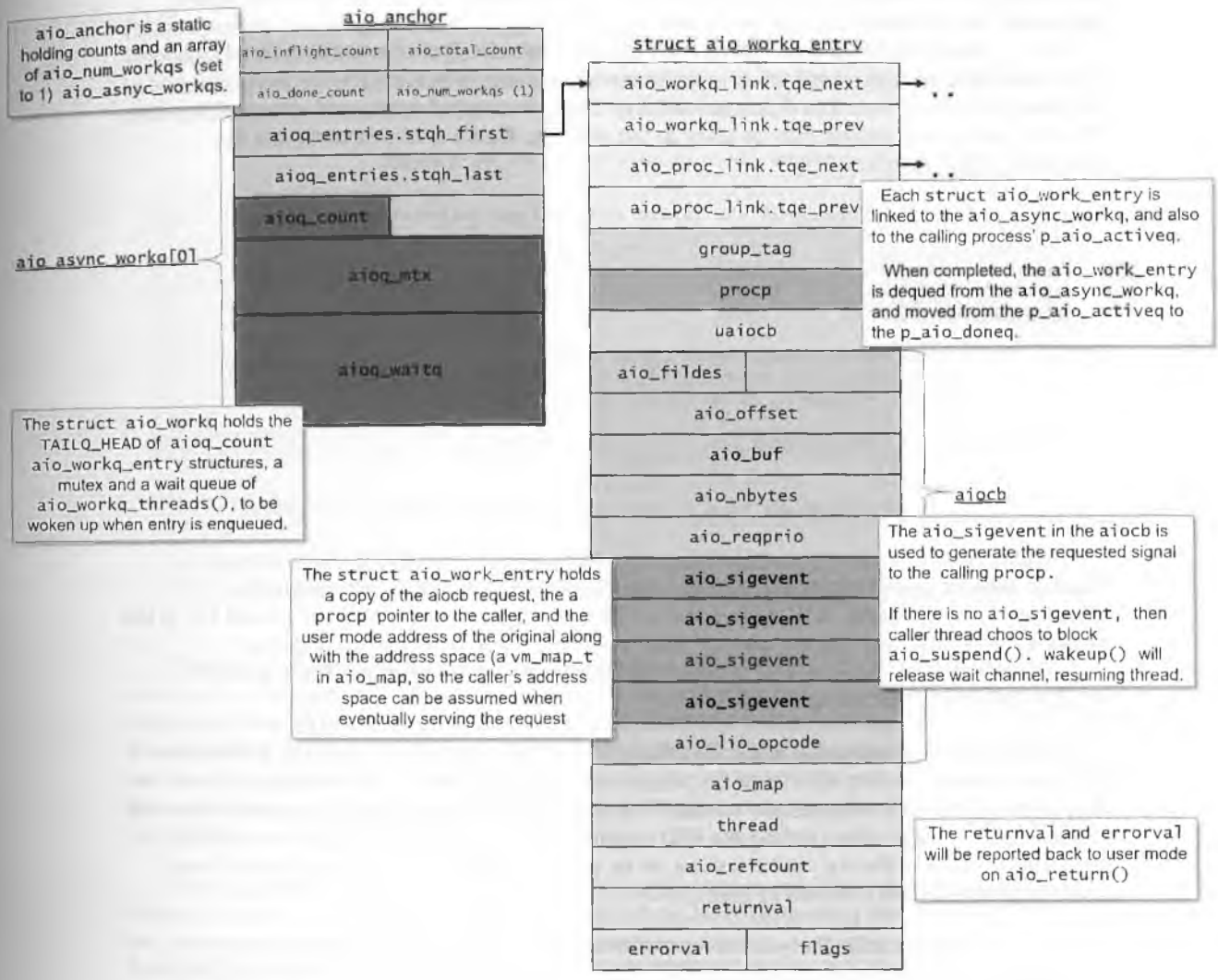
Asynchronous I/O is fairly uncommon, so Listing 6-26 demonstrates the use pattern, using some of the above system calls:

Listing 6-26:

```
// From <sys/aio.h>
struct aiocb {
        int             aio_fildes;         /* File descriptor */
        off_t           aio_offset;         /* File offset */
        volatile void   *aio_buf;           /* Location of buffer */
        size_t          aio_nbytes;         /* Length of transfer */
        int             aio_reqprio;        /* Request priority offset */
        struct sigevent aio_sigevent;       /* Signal number and value */
        int             aio_lio_opcode;     /* Operation to be performed */
};

void sigio_handler(int signo, siginfo_t *info, void *context)
        { printf("read %d bytes\n", aio_return(aio)); }

int main(int argc, char **argv) {

  /* Simple example: Read up to BUFSIZE bytes asynchronously, requesting SIGIO */
  int fd = open(argv[1], O_RDONLY);
  aio = calloc(sizeof(struct aiocb), 1);
  char *buf = malloc(BUFSIZE);
  aio->aio_fildes = fd;
  aio->aio_buf = buf;
  aio->aio_nbytes = BUFSIZE;
  aio->aio_sigevent.sigev_notify = SIGEV_SIGNAL;
  aio->aio_sigevent.sigev_signo = SIGIO;
  aio->aio_sigevent.sigev_value.sival_ptr = &aio;

  // Can use signal(2) or the newer sigaction(2) to install handler:
  struct sigaction sig_act = {0};
  sigemptyset(&sig_act.sa_mask);
  sig_act.sa_flags = SA_SIGINFO;
  sig_act.sa_sigaction = sigio_handler;
  sigaction(SIGIO, &sig_act, NULL);

  int ret = aio_read(aio);
  if (ret < 0) { perror("aio_read"); }

  /* Do something else while I/O completes, e.g. sleep... */
  ...
}
```

When a process submits an `aiocb` through `aio_[read/write]()` or `aio_fsync()`, the system calls all call `aio_queue_async_request()`, with three arguments - the calling process, the `aiocb` and the type of I/O. `aio_create_queue_entry()` puts these into an `aio_workq_entry` structure allocated from the dedicated `aiowq` zone, and checks that the entry has not been previous enqueued on either of the two `TAILQ`s maintained by the process - the `p_aio_activeq` and the `p_aio_doneq`. If not, and the process has not exceeded `aio_max_requests_per_process` (set to `CONFIG_AIO_PROCESS_MAX`), then request can be enqueued.

Figure 6-27: Asynchronous I/O requests



Enqueuing is performed by `aio_enqueue_work()`, which puts the entry onto the process `p_aio_activeq`, and the global workqueue, the first entry of the `aio_async_workqs` array in the (static) `aio_anchor`. Although the design allows for multiple workqueues and hashing, `AIO_NUM_WORK_QUEUES` is presently hardcoded for a single work queue. The `aio_workq` maintains both a list of entries (`aioq_entries`) and idle work queue threads (`aioq_waitq`), and a call to `waitq_wakeup64_one()` picks the first available one.

The work queue threads will have been spun by `aio_init()`, which starts the number of `aio_work_threads` configured (commonly, @TODO). These work threads threads run `aio_work_thread()`, blocking in a continuation until woken up by the act of enqueueing. `aio_get_some_work()` takes the queue lock and dequeues an entry. The address space (`vm_map`) of the entry's owner is switched to, and `do_aio_[read/write/fsync]()` performs the request in the synchronous manner described earlier. The address space is switched back, and the entry is removed from the process' `p_aio_activeq` to `p_aio_doneq`. `do_aio_completion()` then either wakes up any threads blocking on `aio_suspend()`, or sends the signal requested in `aio_sigevent`.

BSD Memory Zones

BSD provides the notion of **memory zones**: Zones are preallocated arrays of objects of an identical size. Similarities, however, end there. BSD presently uses `uma_zcreate`, but Darwin's implementation passes the request through to the underlying Mach zone allocator (which is described later in Chapter 12). Although the Mach layer can directly call its own APIs, the BSD layer generally adheres to using higher level macros and functions.

The main macros used are `MALLOC` and `FREE`, as well as (rarely) `REALLOC`. The `MALLOC` macro returns its first argument as a cast pointer in the memory zone indicated in its `type` argument*. As of Darwin 15, the macro also sets up a `vm_allocation_site` in the `__DATA.__data` section before calling the double-underscore implementations. The allocation site is a 24 byte structure (defined in osfmk/mach/vm_types.h) which tracks allocation usage at the lower level Mach layer (as discussed shortly). Sites are created empty and untagged, but the `MALLOC_external` variant may be used to tag and flag. Figure 6-28 demonstrates the operation of the macros:

Figure 6-28: The operation of the BSD layer `MALLOC` macro

```
#define M_WAITOK        0x0000    // kalloc_canblock(TRUE)
#define M_NOWAIT        0x0001    // kalloc_canblock(FALSE)
#define M_ZERO          0x0004    /* bzero the allocation */
#define M_NULL          0x0008    /* return NULL if space is unavailable*/

#define MALLOC(space, cast, size, type, flags)                  \
        ({ VM_ALLOC_SITE_STATIC(0, 0);                                  \
        (space) = (cast)__MALLOC(size, type, flags, &site); })

extern void  *__MALLOC(size_t                 size,
              int                    type,
              int                    flags,
              vm_allocation_site_t *site)  __attribute__((alloc_size(1)));
```

Another pair of macros, `[MALLOC/FREE]_ZONE` provides a similar KPI, but whereas `MALLOC` uses an underlying Mach layer `kalloc_canblock`, `MALLOC_ZONE` performs allocations in a BSD **zone**. XNU defines some 129 memory zones (as of Darwin 18 and 19) in the `kmzones` array (bsd/kern/kern_malloc.c). Each array entry is a structure, holding the `kz_elemsize`, a `kz_noencrypt` boolean, and a pointer to the underlying Mach zone (to actually satisfy the allocation), if any.

A corresponding `memname` array of string entries (from bsd/kern/kern_malloc.c) holds the BSD zone names, starting with "free" (`M_FREE`) and "mbuf" (`M_MBUF`). Some indexes are not in use, however, and not all zones are named. The zones are initialized early in bsd_init() with a call to `kmeminit()` right after printing the BSD copyright string. `kmeminit()` uses special `KMZ_*` constants in place of the `kz_zalloczone` an be used, to instruct it how to link to the lower level Mach zones. This is shown in Listing 6-29:

Figure 6-29: The `kmzones` structure (from XNU 4903's bsd/kern/kern_malloc.c)

```
/* for use with kmzones.kz_zalloczone */
#define KMZ_CREATEZONE_ACCT     ((void *)-3)
#define KMZ_CREATEZONE          ((void *)-2) // zinit() the underlying Mach zone
#define KMZ_LOOKUPZONE          ((void *)-1)
#define KMZ_MALLOC              ((void *)0)
#define KMZ_SHAREZONE           ((void *)1)  // Use size as zone index, share
struct kmzones {
        size_t          kz_elemsize;
        void            *kz_zalloczone;
        boolean_t       kz_noencrypt;
} kmzones[M_LAST] = {
#define SOS(sname)      sizeof (struct sname)
#define SOX(sname)      -1
        { -1,           0, FALSE },                     /* 0 M_FREE */
..
        { SOX(lockf),   KMZ_CREATEZONE_ACCT, TRUE },    /* 40 M_LOCKF */
        { SOS(proc),    KMZ_CREATEZONE, FALSE },        /* 41 M_PROC */
..
        { 0,            KMZ_MALLOC, >FALSE },           /* 128 M_CFIL */
#undef  SOS
#undef  SOX
};
```

* - Prior to iOS 10, `__MALLOC` would store the allocation size before the returned buffer. This was removed, since the Mach Zone Allocator can keep track of element sizes.

When zone allocations are called for, `MALLOC_ZONE` uses the `type` argument as an index into the `kmzones` array, always of the form `M_` as `#defined` in bsd/sys/malloc.h. The BSD zone's `kz_zalloc_zone` is obtained, and `zalloc[_noblock]` is then called to get an element directly from the backing Mach zone (if the size matches). Calls to `MALLOC_ZONE` must be matched by calls to `FREE_ZONE`, which likewise calls `zfree()` or `kfree()` accordingly. Here, too, a `MALLOC_ZONE_external` variant exists, but it provides NULL as the allocation site.

vm_allocation_sites

Darwin 15 introduces the `vm_allocation_site` as a structure containing metadata on virtual memory allocations. This structure allows tracking totals for allocations with the same tag. The sites themselves are statically allocated by the `VM_ALLOC_SITE_STATIC` macro (osfmk/mach/vm_types.h), which places the site structure in the `__DATA.__data` section, setting the specified tags and flags, as well as the reference count to 2.

Figure 6-30: The `vm_allocation_sites`, visualized

```
// From osfmk/mach/vm_types.h
struct vm_allocation_site
{
    uint64_t  total;
#if DEBUG || DEVELOPMENT
    uint64_t  peak;
#endif /* DEBUG || DEVELOPMENT */
    uint64_t  mapped;
    int16_t   refcount;
    vm_tag_t  tag;
    uint16_t  flags;
    uint16_t  subtotalscount;
    struct vm_allocation_total subtotals[0];
    char      name[0];
};

#define VM_ALLOC_SITE_STATIC(iflags, itag)
 static vm_allocation_site_t site __attribute__((section("__DATA, __data"))) \
  = { .refcount = 2, .tag = (itag), .flags = (iflags) };
```

The `vm_allocation_site` structure also allows for subtotals, as maintained by the `subtotals` field, an array of `subtotalscount` `vm_allocation_total` structures, as well as a `name` field. In practice, however, the `name` is as of yet unused, and the `subtotalscount` is commonly 0, leaving the array unused as well. Nearly all `vm_allocation_site` structures are thus 24 bytes, with the only exception being the the allocation sites of `IOMemoryDescriptors`, whose subtotals are updated on `::dmaMapRecord` and `dmaUnmap` operations.

The `tag` field of the `vm_allocation_site` is an integer value specifying the type of memory, to help in later determining the identity of the code performing the allocation. The tags are maintained in osfmk/mach/vm_statistics.h, under a `#KERNEL_PRIVATE` which strips them from the user mode header (mach/vm_statistics.h). The tags also serve as indices to the `vm_allocation_sites` array, which is defined in osfmk/vm/vm_resident.c. Each array entry (up to `VM_MAX_TAG_VALUE`, commonly 1536) is a pointer to the allocation site. The array is guarded by the `vm_allocation_sites_lock`.

Note, that the `vm_allocation_site` pointer is used differently for kext accounting (Chapter 3) - if the VM tag indicates `VM_TAG_KMOD` then `OSKextGetKmodIDForSite` is used to get the name of the extension responsible for the allocation. In user mode, `mach_memory_info` (`mach_host` MIG routine #227) provides the site data. For a good example of using the API, consult the open source `zprint(1)` (from the system-cmds project).

* This is somewhat similar in idea to Windows' `ExAllocatePoolWithTag()`, only tags there are 4 byte ASCII

sysctl

The `sysctl(2)` API and its command-line utility (`sysctl(8)`) provide a standardized way across UNIX variants to view and (in some cases) set certain kernel variables*. The parameters are arranged in an SNMP-like "Management Information Base" (MIB), commonly an array of two to five integers. The MIBs can also be mapped to human-readable with the array entries mapping to human readable namespaced variables, which can be accessed programmatically through `sysctlbyname(3)`. Commonly the `sysctl(8)` command is used to inspect and/or modify the values. An example of the command line usage was shown in I/4 ("System Information").

XNU uses several well-known namespaces, which are exposed to user-mode through <sys/sysctl.h>, and kernel extensions can dynamically register their own namespaces through a call to `sysctl_register_oid`, which is a supported `com.apple.kpi.bsd` KPI (as is done, for example, by VMWare Fusion's extensions). The interested reader is encouraged to try the `sysctl(8)` command with one of the namespaces from Table 6-31. Another useful method (especially on new iDevices or iOS versions) is using `sysctl(8)` with the `-X` switch, which dumps all namespaces, including opaque variables (as a hexdump).

Table 6-31: The well-known `sysctl` namespaces

#	#define	Namespace	Contains
0		sysctl	Metadata: `debug` (0), `name` (1), `next` (2), `name2oid` (3), `oidfmt` (4)
1	CTL_KERN	kern	Miscellaneous kernel globals, process structs and more
2	CTL_VM	vm	Virtual memory subsystem related variables and statistics
3	CTL_VFS	vfs	Virtual Filesystem Switch, providing global and specific FS-level variables
4	CTL_NET	net	All things network and socket related
5	CTL_DEBUG	debug	Debugging parameters
6	CTL_HW	hw	Generic hardware parameters
7	CTL_MACHDEP	machdep	Specific (machine-dependent) hardware parameters
8	CTL_USER	user	User-level and POSIX-option support
9		security	MACF settings

The `sysctl(8)` utility "knows" how to "walk" the MIB namespaces and format the variable values through a hidden namespace, `sysctl`, which is not displayed in its output but is nonetheless reachable as MIB 0. Calling on its `oidfmt` child (i.e. 0.4), other OIDs can be queried and their suggested formatting obtained. The namespace is also useful in providing `name`/`name2oid` (0.1 and 0.3, respectively) to translate between the integer array and textual names, as well as `next` (0.2) used to walk MIBs. Using `sysctl.debug` (0.0) dumps the entire namespace `sysctl__children`, recursively calling `sysctl_sysctl_debug_dump_node` to output each node's oid, name, Read/Write/Locked status and type (if any). The copious output is visible by `dmesg(8)`, although it often completely fills the log buffer.

The `sysctl(2)` facility should be treated with caution: Writable MIBs present a vector for writing into kernel memory from user mode. Readable MIBs may potentially leak out kernel data, if not properly implemented. Some APTs found a clever use for `sysctl(2)` by writing custom values to certain MIBs post exploitation, so that the values could be used as indicator of previous exploitation.

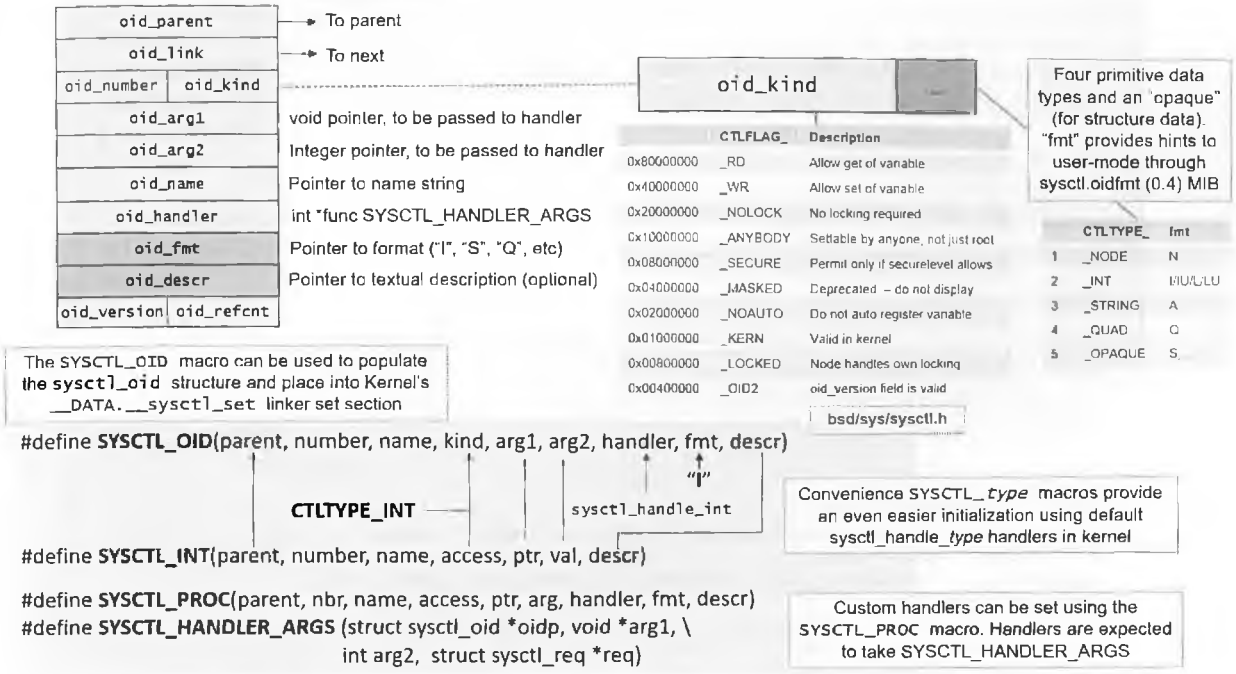
Implementation

The sysctl facility provides the `sysctl[byname](3)` system calls (#202 and #274, respectively). Both are implemented in bsd/sys/kern_newsysctl.c**. An older bsd/sys/kern_sysctl.c still compiles, but is marked deprecated. The difference between the two syscalls is mostly for convenience: `sysctl(3)` takes the MIB *`name`* integer array containing *`namelen`* entries, whereas `sysctlbyname(3)` takes a string *`name`* instead. Both calls funnel to `sysctl_root()`, which may resolve the string into an array form by calling `name2oid()`.

* - Linux users are probably familiar with the pseudo-files in /proc/sys, which are merely a filesystem-based way of presenting the very same variables with namespaces as subdirectories and the variables as the files.
** - An older file, bsd/sys/kern_sysctl.c has been deprecated for quite some time, but has yet to be removed.

The namespaces from Table 6-31 are implemented in the `sysctl__children`, which is an `SLIST_HEAD` of `sysctl_oid` structures. Figure 6-32 shows the layout of this structure, defined in bsd/sys/sysctl.h along with its initializing macros:

Figure 6-32: The `sysctl_oid` structure (from bsd/sys/sysctl.h)



The structure can be initialized at compile time using the `SYSCTL_OID` macro. When created, OIDs can be classified as one of several kinds: `CTLTYPE_[INT/QUAD]` are used for numeric values, `CTLTYPE_STRING` for C-strings, and `CTLTYPE_OPAQUE` for structured data. containers can be created by specifying `CTLTYPE_NODE`. The `oid_kind` may further be flagged to control access.

For fixed data types, the facility provides default handlers - `sysctl_handle_type`, where *type* is `int`, `long`, `quad`, `int2quad`, `string` or `opaque` for the matching `SYSCTL_TYPE` macros. These can also be used inside custom `SYSCTL_PROC` handlers in order to provide data access, though the newer interface of `sysctl_io_[number/string/opaque]` (which the sysctl_handle_*type*s use internally) is recommended. Low level I/O of the values can also be performed using `SYSCTL_[IN/OUT]` macros on the `sysctl_req` obtained from user mode.

The sysctl tree is protected by two locks, both in the `sysctl` lock group:

- The `sysctl_geometry_lock`: Used to protect the tree layout (`sysctl__children`) from concurrent access, when removing or adding any nodes or namespaces
- The `sysctl_unlocked_node_lock`: Used for node access unless the node is flagged with `CTLFLAG_LOCKED`, in which case the node's handler is assumed to handle locking.

__DATA.__sysctl_set

Poking around the kernel binary and looking for the `__sysctl_children` symbol will reveal it is populated in runtime. Declared MIBs are stored in a dedicated Mach-O section of the kernel, called `__DATA.__sysctl_set`. The section is created and populated through a novel use of a compile-time construct called **linker sets**. A linker set is a collection of related global objects, which - through macros in bsd/sys/linker_set.h - are grouped by the linker into their own section. Although this functionality could no doubt put to good use elsewhere, in practice it isn't - and is used only in the case of the sysctl facility, whose multitude of variables are thus grouped together. The section is processed by `sysctl_register_set`, which `bsd_early_init()` calls during the `kernel_bootstrap_thread` flow (step #20). This technique only applies to sysctls in the kernel proper, and not to those of extensions, which must be explicitly registered with `sysctl_register_oid()`.

The `SYSCTL_OID` macro thus expands to contain the `LINKER_SET_ENTRY` macro, which itself expands to a linker definition of the section, as explained in the header file:

Listing 6-33: The linker set macros, from bsd/sys/linker_set.h

```
/*
* The following macros are used to declare global sets of objects, which
* are collected by the linker into a `linker set' as defined below.
* For Mach-O, this is done by constructing a separate segment inside the
* __DATA section for each set.  The contents of this segment are an array
* of pointers to the objects in the set.
*
* Note that due to limitations of the Mach-O format, there cannot
* be more than 255 sections in a segment, so linker set usage should be
* conserved.  Set names may not exceed 16 characters.
*/
..
/*
* Private macros, not to be used outside this header file.
*
* The objective of this macro stack is to produce the following output,
* given SET and SYM as arguments:
*
*  void const * __set_SET_sym_SYM __attribute__((section("__DATA,SET"))) = & SYM
*/
..

#define __LINKER_MAKE_SET(_set, _sym)                                   \
/*__unused*/ /*static*/ const struct linker_set_entry /*const*/      \
__set_##_set##_sym_##_sym                                       \
__attribute__ ((section(__LS_VA_STRCONCAT(__DATA,_set)),used)) = \
{ (void *)&_sym }
/* the line above is very fragile - if your compiler breaks linker sets,
just play around with "static", "const", "used" etc. :-) */

/*  Public macros.  */
#define LINKER_SET_ENTRY(_set, _sym)    __LINKER_MAKE_SET(_set, _sym)
```

The section ends up containing pointers to `sysctl_oid` structures, which are defined elsewhere in the binary. The `joker` tool, available on the book's companion website, can dump the `sysctl_oid` structures from a kernel or kernelcache when used with the `-S` switch. This proves useful when symbolicating the iOS kernelcaches, or when analyzing new releases of XNU during the beta window, when the sources are not available. Note that not all `sysctl` OIDs contain a textual description, and that addresses are subject to sliding.

Output 6-34: Sample output of `joker -S` on an iOS kernelcache

```
root@iPhonoClaXt (/)# joker -S /S*/L*/C*/com.apple.kernelcaches/kernelcache
This is a 64-bit kernel from iOS 12.0, or later
Dumping sysctl_set from 0xffffff00766389c (offset in file: 0x65f89c), 496 sysctls follow:
0xffffff00763c328: kern.nbuf    Description:
        Handler: 0xffffff00748e2d8
        Format: I
        Parent: 0xffffff007666148
        Arg1: ffffff0076b1d8c
        Arg2: 0x0
0xffffff00763c378: kern.maxnbuf         Description:
        Handler: 0xffffff00748e2d8
        Format: I
        Parent: 0xffffff007666148
        Arg1: ffffff0076b1d80
        Arg2: 0x0
0xffffff00763c418: kern.monotonic.supported     Description: whether monotonic is supported
        Handler: 0xffffff007210238
        Format: I
        Parent: 0xffffff0076b1db8
        Arg1: 0
        Arg2: 0x4
..
root@iPhonoClaXt (/)# jtool2 -d __DATA.__sysctl_set /S*/L*/C*/com.apple.kernelcaches/kernelcache
Dumping 9392 bytes from 0xffffff00766389c (Offset 0x65f89c, __DATA.__sysctl_set):
0xffffff0076638698:    0xffffff00763c328    __sysctl_kern_nbuf
0xffffff00766386a0:    0xffffff00763c378    __sysctl_kern_maxnbuf
0xffffff00766386a8:    0xffffff00763c3c8    __
0xffffff00766386b0:    0xffffff00763c418    __sysctl_kern_monotonic_supported
0xffffff00766386b8:    0xffffff00763c468    __
..
```

Experiment: Creating a custom `sysctl(2)`

It is not uncommon for kernel developers to choose the `sysctl(2)` facility as a way to interface with their extensions from user mode. Apple provides documentation on how to do so in the Kernel Programming Guide[2]. Using most `SYSCTL_*` typed macros usually suffices if the goal is to just export some variable or struct (`CTLTYPE_OPAQUE`) to user mode. A more advanced use of the facility, however, is to invoke a kernel callback in response to a user-mode get or set operation. For this, one would use the `SYSCTL_PROC` macro, shown in 6-32. Note, that one may still set a variable despite setting a handler callback (e.g. `net.inet.tcp.rexmt_thresh` in bsd/netinet/tcp_input.c).

A custom handler requires a specific prototype, so the `SYSCTL_HANDLER_ARGS` (shown in Figure 6-32) can be used. When expanded, this would make the handler look something like this:

Listing 6-35: A basic skeleton for a `sysctl` custom handler

```
int my_sysctl_handler (struct sysctl_oid *oidp, // oid for this sysctl
                       void *arg1, // argument from SYSCTL_* declaration
                       int arg2,
        struct sysctl_req *req) {
        int error = 0;

        // Do something
        ...
        // Call to default handlers, if desired, e.g.
        // error = sysctl_handle_int(oidp, arg1, arg2, req);
        //       or (new interface)
        // error = sysctl_io_number(req, arg1, sizeof(int), arg1, NULL);
        return error;
}
```

It is fine to use the legacy `sysctl_handle_type()` routines to provide access to a variable or structure exported by the `sysctl(2)`, if any. These require using `arg1` and `arg2` The newer interface use three `sysctl_io_*` functions which work similarly.

Listing 6-36: The newer `sysctl` handlers, from bsd/sys/sysctl.h

```
int sysctl_io_number(struct sysctl_req *req,
                     long long bigValue, size_t valueSize,
                     void *pValue, int *changed);
int sysctl_io_string(struct sysctl_req *req,
                     char *pValue, size_t valueSize,
                     int trunc, int *changed);
int sysctl_io_opaque(struct sysctl_req *req,
                     void *pValue, size_t valueSize,
                     int *changed);
```

The `struct sysctl_req *` (in the last argument of the registered handler), is shown in Listing 6-37. It can and should remain opaque, however, and passed to the `handle_*` or `sysctl_io_*()` handlers, which use it internally to perform the data copying operations. Thus, the the user mode caller can get the existing value into the `oldptr` user buffer (of `oldlen` bytes) using the `oldfunc` function, invoked by `SYSCTL_OUT`. Likewise, the `newptr` user buffer (of `newlen` bytes) specifying a value to set is read using `SYSCTL_IN`, which uses the `newfunc` from the request.

Listing 6-37: The `sysctl_req` structure, from bsd/sys/sysctl.h

```
struct sysctl_req {
  struct proc   *p;
  int           lock;
  user_addr_t   oldptr;         /* pointer to user supplied buffer */
  size_t        oldlen;         /* user buffer length (also returned) */
  size_t        oldidx;         /* total data iteratively copied out */
  int           (*oldfunc)(struct sysctl_req *, const void *, size_t);
  user_addr_t   newptr;         /* buffer containing new value */
  size_t        newlen;         /* length of new value */
  size_t        newidx;         /* total data iteratively copied in */
  int           (*newfunc)(struct sysctl_req *, void *, size_t);
};
```

DTrace

The DTrace facility has been described in I/15, and some usage examples were presented there and throughout this volume as well. This all-powerful mechanism for debugging and tracing has long been one of MacOS's hidden gems, as of 10.5, when it was introduced to take over CHUD. SIP neuters most DTrace capabilities, owing to a `dtrace_is_restricted()` check (in bsd/dev/dtrace/dtrace_subr.c), unless `CSR_ALLOW_UNRESTRICTED_DTRACE` (0x20) is set. DTrace is also compiled out of *OS variants, though the ARM64 sources clearly show it is supported (`#if CONFIG_DTRACE`, which is unset). Presumably, Apple's development builds make use of it, as its tracing capabilities are far more versatile that KDebug's.

The implementation of DTrace can be though of as three-tiered. The top tier consists of the user mode interface, via several device nodes exposed. The /dev/dtrace character device is the one used to control DTrace, enabling/disabling probes when `dtrace(1)` is executed. The /dev/dtracehelper character device is used by user-mode providers to load their DTrace Object Formats (DOFs) into the kernel, as is commonly done by `dyld` on process load by locating specially marked Mach-O sections.

The middle tier is an architecture indepedent layer, implemented mostly in bsd/dev/dtrace.c with objects defined in bsd/sys/dtrace_*.h. This layer consists of the DTrace provider, module and probe kernel representations, as well as the built-in (kernel-based) providers. The code here is almost self-contained, faithfully inspired by the Open Solaris (now IllumOS) implementation. Darwin/BSD's DTrace also ports some of Solaris' implementation features directly, e.g. the `vmem` allocatorand `kmem_cache`. The Solaris kernel module structure implementation (`modctl`, in bsd/sys/dtrace_glue.h) has been adapted so as to mirror XNU's modules (`kmod_info_t`).

The bottom layer is machine specific (in bsd/dev/*arch*), and contains the machine specific for stack and register handling (for probes to inspect), as well as the function boundary tracing (`fbt`) and statically defined tracing (`sdt`) providers. DTrace even has a machine-specific partial disassembler implementation(!) which is required due to the dynamic code-patching these probes necessitate.

Apple provides no formal documentation of any kind for DTrace, but the various files (in particular, dtrace. and the headers in bsd/sys/dtrace_*.h) are superbly commented and highly verbose. There are multiple references to the Solaris Dynamic Tracing Guide[3], which - although focused on user mode - is still a comprehensive resource for DTrace usage and syntax. Another excellent guide on D syntax can be found at dtrace.org[4].

Initialization

DTrace is initialized from `dtrace_init()`, which is called when `bsd_autoconf()` calls other `pseudo_init` functions. The initialization creates the /dev/dtrace device node, and a dedicated `dtrace.dtrace_probe_t` zone, initially set to hold up to 1024 probes. It also creates and initializes the various locks, all in the `dtrace_lck_grp`. The /dev/dtracehelper is initialized from `helper_init()`, which follows `dtrace_init()` in the `pseudo_init` array, and right after it are the initializers of the `lockstat`, `sdt`, `systrace` and `fbt` providers.

An important function also carried out during `dtrace_init()` is setting the `dtrace_cpu_state_changed` hook function (from dtrace_glue.c), which is called (from osfmk/kern/machine.c) on `processor_[up|doshutdown]`. This is important because DTrace makes use of timer calls, which need to be canceled or reentered when the processor is put to sleep or brought back online. The initialization also takes care of setting up DTrace structures per CPU, and calls `dtrace_isa_init()`, which is supposed to take care of the instruction set architecture specific initialization, which is ignored on MacOS and merely sets a lock on ARM architectures.

Initialization continues when `bsd_init()` explicitly calls `dtrace_postinit()` (just before network interface setup). This calls `dtrace_attach()`, which performs the work of initializing DTrace's memory, creating probe hash tables, and registering the `dtrace` provider and its `BEGIN/END/ERROR` probes. `dtrace_postinit()` then sets up the `mach_kernel` provider, and calls `OSKextRegisterKextsWithDTrace`, so as to register any kernel extension based providers.

Providers

DTrace providers* are installed with a call to `dtrace_register`, but this is not a public KPI. The DTrace subsystem internally registers several built-in providers, shown in Table 6-1: `dtrace`, `profile`, `syscall`, `mach_trap`, `lockstat`, `sdt` and `fbt`. External providers are registered when their DOFs are supplied from user mode (i.e. through the /dev/dtracehelper's `DTRACEHIOC_ADDDOF*`), or found in loaded kernel extensions (via OSKext::registerWithDTrace).

Providers are `dtrace_provider_t`s, defined in bsd/sys/dtrace_impl.h. They are `kalloc()`ed (via `kmem_zalloc()`) from `kalloc.160`, and maintained in the unexported `dtrace_provider` linked list, which is guarded by the `dtrace_provider_lock` mutex (in addition to the global `dtrace_lock`, ownership of which is required for any DTrace state modification). Each `dtrace_provider_t` holds the provider name, argument, details about its attributes and privleges, and a `dtpv_pops` pointer to its lifecycle operation callbacks.

Probes

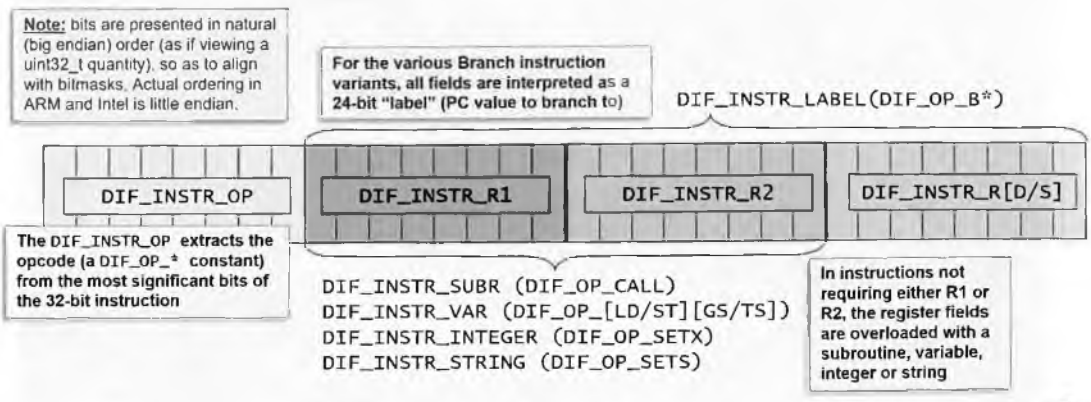
DTrace providers are expected to provide one or more probes. Probes are created with `dtrace_probe_create()`, and allocated from their dedicated `dtrace.dtrace_probe_t` zone**, which makes it easy to find the providers, as each probe references its provider through its `dtpr_provider` (at offset 0x28). The probe also holds pointers to the names of its module, function and name.

DTrace allows probes to be specified with wildcards and omissions, to it is important to maintain quick lookups for all cases. DTrace therefore internally maintains four `dtrace_hash_t` tables - `dtrace_byprov`(ider), `dtrace_bymod`(ule), `dtrace_byfunc`(tion) and `dtrace_byname`. `dtrace_match()` first attempts to find the optimal hash table, and if one can be found it avoids iterating over the `dtrace_probes` array. Each `dtrace_probe_t` contains pointers to the previous and next in each hash table.

When a provider "fires" a probe, it does so through `dtrace_probe()` (the "epicenter of DTrace"). This locates the probe and, in it, a list of **Enabling Control Blocks** (ECBs). Each ECB holds the state, a predicate (in compiled D), and a list of `dtrace_action_t`s, which also hold compiled D expressions, to be executed in kernel by the D emulator.

The **DTrace Intermediate Format Objects** (DIFOs, defined in bsd/sys/dtrace.h) are used to represent the compiled D statements, which are held in a `dtdo_buf` of `dtdo_len` 32-bit instruction. Each instruction holds an 8-bit `DIF_OP_*` (ibid.) in its most significant byte, which is emulated by `dtrace_dif_emulate()`. The other bytes hold three registers - R1, R2 and Rd (in the other bytes, respectively), or special arguments (as shown in Figure 6-38, below). These can be variables (represented by `DIF_VAR_*` constants and looked up by `dtrace_dif_variable()`), built-in D subroutines (`DIF_SUBR_*` constants, emulated by `dtrace_dif_subr()`), integers, strings or branch targets.

Figure 6-38: The structure of a compiled D instruction



* - DTrace also allows for **meta providers**, which are provider of providers, i.e. ones which can create other providers on the fly. The only one supported is `fasttrap` (bsd/dev/dtrace/fasttrap.c), which enables user-land trap-based tracing by replacing instructions in the process with traps. The meta-provider creates a /dev/fasttrap, which allows a `FASTTRAPIOC_MAKEPROBE` `ioctl(2)`. Though not in MacOS 14, `fasttrap` is in the 10.15 betas.

** - An `APPLE NOTE` comment in bsd/dev/dtrace/fasttrap.c states the zone element size is 72 bytes (the 32-bit case), but it has grown over time to the present-day (64-bit) 144 bytes.

Case Study: The `fbt` provider

Of all the formidable DTrace providers, the **function boundary tracer** (fbt) and **statically defined tracing** (sdt) are by far the most powerful and complex*. Similar to Linux's `ftrace` facility (specifically, the `function` and `function_graph` tracers), these providers are able to trace function calls in kernel space by dynamically patching instructions at the function boundary. The `sdt` provider allows tracing pre-defined points (built into the kernel binary and exported as `__dtrace_probe$num__provider__probe`). The `fbt` provider automatically provides probes for all exported kernel functions (save for specifically excluded ones), by scanning the kernel Mach-O image: `fbt_provide_module_kernel_section()` iterates over all kernel global text symbols, and calls `fbt_provide_probe()` for each. The two are fairly similar in their operation from that point, so we will consider `fbt`.

Dynamic patching is naturally architecture-specific, so `fbt_provide_probe()` has three different implementations, for arm/, arm64/ and i386/. The function inspects the instruction stream at the address, trying to find the familiar `PUSH RBP` in Intel, and an `STP FP, LR, ..` (the frame pointer and link register) in ARM. If found, a probe is created on the fly. When a probe is made active, a call to `fbt_enable()` (back in the architecture agnostic bsd/dev/dtrace/fbt.c), performs the memory patching (through `ml_nofault_copy()`) required - in Intel, to an 0xf0 opcode (LOCK), and in ARM64 to 0xe7eeee7e (invalid), and flushes the data/instruction caches, if applicable.

Special care must be taken to avoid placing probes in known critical paths, and those functions are forbidden by a callto `fbt_excluded()`. A hard-coded `critical_blacklist` holds those routines which cannot be probed due to resulting instability. These include `cpu_*` routines, memory copying functions, as well as DTrace and fbt's own paths. Other excluded probes are maintained in the `probe_ctx_closure` list, singling out routines which the `fbt` probe code path uses, and therefore must be avoided (to prevent infinite recursion). Both lists are maintained in alphabetical increasing order, so it can be efficiently `bsearch()`ed. As an extra measure, literal string comparisons (using the `LIT_STRNSTART` macro) exclude known additional prefixes, which may be hit on the exception handling or panic paths. Another (shorter) list is hard coded into `fbt_module_excluded()`, to prevent power management and platform kexts from being traced, due to similar concerns.

There are on order of 150,000 `fbt` probes in Darwin 19, so accessing them requires using a hash table. The `fbt_probetab` is a table of `FBT_PROBETAB_SIZE` (32k) entries, `kalloc()`ed from `fbt_attach()` (in bsd/dev/dtrace/fbt.c). The hash used is the `FBT_ADDR2NDX` macro (ibid.), which shifts the address by 4 bits, and then extract the 15 least significant bits to obtain an index into the table. The probes are `struct fbt_probe` objects, defined in bsd/sys/dtrace.h, as shown in Listing 6-39:

Listing 6-39: The `struct fbt_probe` (from bsd/sys/dtrace.h)

```
typedef struct fbt_probe {
   struct fbt_probe *fbtp_hashnext;     // Next probe, for hash collisions
   machine_inst_t   *fbtp_patchpoint;   // Address of patched instruction
   int8_t            fbtp_rval;
   machine_inst_t    fbtp_patchval;     // fbt patched instruction opcode
   machine_inst_t    fbtp_savedval;     // Original instruction opcode
   machine_inst_t    fbtp_currentval;   // = fbtp_patchval? active : inactive
   uintptr_t         fbtp_roffset;
   dtrace_id_t       fbtp_id;
   /* FIXME!
    * This field appears to only be used in error messages.
    * It puts this structure into the next size bucket in kmem_alloc
    * wasting 32 bytes per probe. (in i386 only)
    */
   char              fbtp_name[MAX_FBTP_NAME_CHARS];          // = 16
   struct modctl    *fbtp_ctl;            // Owning module
   int              fbtp_loadcnt;         // Module load count
   struct fbt_probe *fbtp_next;
} fbt_probe_t;
```

* - As powerful as they are, these providers are unfortunately somewhat unstable. Every so often probes can cause a panic due to timing out a non maskable interrupt (NMI) acknowledgment, or other users. It's not likely that Apple will address this - with SIP already neutering it (and other providers), additional signs point to Apple deprecating DTrace support in the future, sticking with the kdebug facility for passive (and much safer) monitoring and tracing.

Experiment: Watching the internals of the `fbt` provider

We can easily examine FBT probes in kernel memory since the `fbt_probetab` is exported. Looking at bsd/dev/dtrace/fbt.c we find that the symbol is a double pointer to an `fbt_probe_t` - that is, a pointer to an array of pointers. Xnooping around, we find:

Output 6-40-a: Displaying the `fbt_probetab`

```
# Locate the fbt_probetab.
#
root@Zephyr (~) # jtool2 -S /System/Library/Kernels/kernel| grep fbt_probetab
ffffff8000e01940 S _fbt_probetab
ffffff8000e01948 S _fbt_probetab_mask
#
# Dump memory: If using kmem.sh, don't forget to account for the slide
#
root@Zephyr (~) # xnoop dump _fbt_probetab smart | head -4
                                          fbt_probetab_mask
                          *fbt_probetab            |          ignore_fbt_blacklist
0xffffff8011a01940    0xffffff80e104a000      0x07fff      0x0          .......
                          *sdt_probetab       sdt_probetab_size
0xffffff8011a01950    0xffffff80e1042000      0x100        0xfff        ........
#
# Dump several probes from the fbt_probetab (memory address here already slid)
#
root@Zephyr (~) # xnoop dump   0xffffff80e104a000 | head -4
0xffffff80e104a000 00 00 00 00 00 00 00 00      0xffffff803a0eedc0
0xffffff80e104a010    0xffffff803cd4e960        0xffffff803cd4e780
0xffffff80e104a020    0xffffff803c63c3c0        0xffffff803c2defa0
...
```

Looking at some arbitrary probe, let us randomly consider `ip_input`. `jtool2` reports the address of this function is 0xffffff800073f390. Although the address will be slid, the slide is greater than 20 bits, so it does not affect our hash key calculation. Taking the last five hex digits of the address and shifting by four, we get 0x3f39. This will be our `fbt_probetab` index. The table entries are pointers, so:

Output 6-40-b: The hex math required to find the hash bucket for `ip_input`

```
root@Zephyr (~)# jtool2 -S /System/Library/Kernels/kernel | grep ip_input
ffffff800073f390 T _ip_input
root@Zephyr (~)# printf "%x\n" $[0x3f39 * 8]
1f9c8
root@Zephyr (~) # xnoop dump 0xffffff80e10699c8,8
0xffffff80e10699c8    0xffffff803c661a50
```

Indeed, xnooping at the address, we find it points to an object from `kalloc.80` - i.e. a structure of (up to) 80 bytes, in line with the `struct fbt` (from Listing 6-39):

Output 6-40-c: Dumping the `fbt_probe_t` for `ip_input`

```
# Check type of object based on zone allocation
#
root@Zephyr (~) # xnoop object 0xffffff803c661a50
Object at address 0xffffff803c661a50 is kalloc.80
#
# Raw dump (like kmem.sh)
#
root@Zephyr (~) # xnoop dump 0xffffff803c661a50,80
0xffffff803c661a50    0xffffff803c525aa0        0xffffff801133f391
0xffffff803c661a60 0b f0 48 48 00 00 00 00   00 00 00 00 00 00 00 00    ..HH............
0xffffff803c661a70 09 9c 04 00 69 70 5f 69   6e 70 75 74 00 00 00 00    ....ip_input....
0xffffff803c661a80 00 00 00 00 00 00 00 00      0xffffff8022c14140
0xffffff803c661a90 01 00 00 00 00 00 00 00   00 00 00 00 00 00 00 00
#
# Smart dump (correlate with 6-39). savedval = current, so probe inactive.
#
root@Zephyr (~) # xnoop dump 0xffffff803c661a50 fbtprobe_t
fbtp_hashnext:                  0xffffff803c525aa0 (nfs_vnop_remove)
fbtp_patchpoint:                0xffffff801133f391 (ip_input +1)
fbt_rval:                       0x0b
fbt_[patch/saved/current]val:   0xf0/0x48/0x48
fbtp_roffset:                   0
fbtp_id:                        0x9c09
fbtp_ctl:                       0xffffff8022c14140 (mach_kernel)
fbtp_name:                      ip_input
fbtp_loadcnt:                   1
fbtp_next:                      (null)
```

The `fbtp_name` indicates `ip_input`, which clearly indicates the math was correct, but if your results indicate another symbol, remember that different functions collide in different kernel versions, and so you might hit a collision before getting to `ip_input`. In the example here, `fbtp_hashnext` shows `nfs_vnop_remove`. This, too, makes sense. The two functions indeed collide in this kernel, with `nfs_vnop_remove` at 0xffffff800053ea90.

Review Questions

1. Looking over `parse_bsd_args()` (from bsd/kern/bsd_init.c) and elsewhere in the kernel sources, you might notice the absence of `-v` argument parsing. How is it, then, that specifying `-v` causes a verbose boot?
2. What is a requirement to allow a kernel extension to register its code using `net_init_add()`? How does MacOS's ALF.kext satisfy this requirement?
3. Linux uses identifiers from the same namespace as PIDs for thread identifiers. What capability does that enable from user mode, which Darwin does not (by design) support?
4. In Output 6-18, why is the POSIX shared memory object name truncated this way?
5. Pipes can be transformed into vehicles for constructing fake objects, if a kernel object pointer can be made to point to a pipe buffer (e.g. as the result of a Use-after-Free vulnerability), as first demonstrated by Ian Beer in his iOS 11.3 multipath exploit. How would that work? Explain.
6. What are the challenges associated with implementing the `select(2)` and `poll(2)` system calls? What would the pseudocode for implementing them in-kernel look like? Compare your answer with XNU's implementation

References

1. McKusick, Neville-Neil & Watson - "The Design and Implementation of the FreeBSD Operating System" (2nd Edition) - ISBN -978-0321968975
2. Apple Developer - "Kernel Programming Guide"/"Boundary Crossings" https://developer.apple.com/library/archive/documentation/Darwin/Conceptual/KernelProgramming/boundaries/boundaries.html
3. Sun Microsystems (now Oracle) - Solaris Dynamic Tracing Guide - https://docs.oracle.com/cd/E23824_01/pdf/E22973.pdf
4. dtrace.org - The illumos Dynamic Tracing Guide - http://dtrace.org/guide/dtrace-ebook.pdf

7

Fee, Fi-fo, File - the Virtual Filesystem Switch

Ever since Solaris introduced the **Virtual Filesystem Switch**, it has revolutionized the way filesystems are implemented in UN*X systems. Prior to its inception, individual filesystem drivers often had to re-implement custom kernel interfaces for their underlying implementations. VFS changed all that by providing what, in design pattern parlance, is an *adapter*: A layer providing a standardized top interface for the kernel, while forcing its bottom implementations to conform through a set of specific callbacks. In this way, kernel access to filesystems could be abstracted, allowing compatibility and full interoperability with virtually any filesystem, so long as its driver could adhere to the standard specification.

Although specific implementations vary, the various flavors have all adopted one or another "interpretation" of VFS as the substrate for their filesystem stacks. Darwin's, unsurprisingly, closely matches that of FreeBSD's. Unlike BSD VFS, however, for which there is no shortage of documentation, Darwin's VFS remains virtually undocumented, at least publicly. The first edition of this work took it for granted, but no longer.

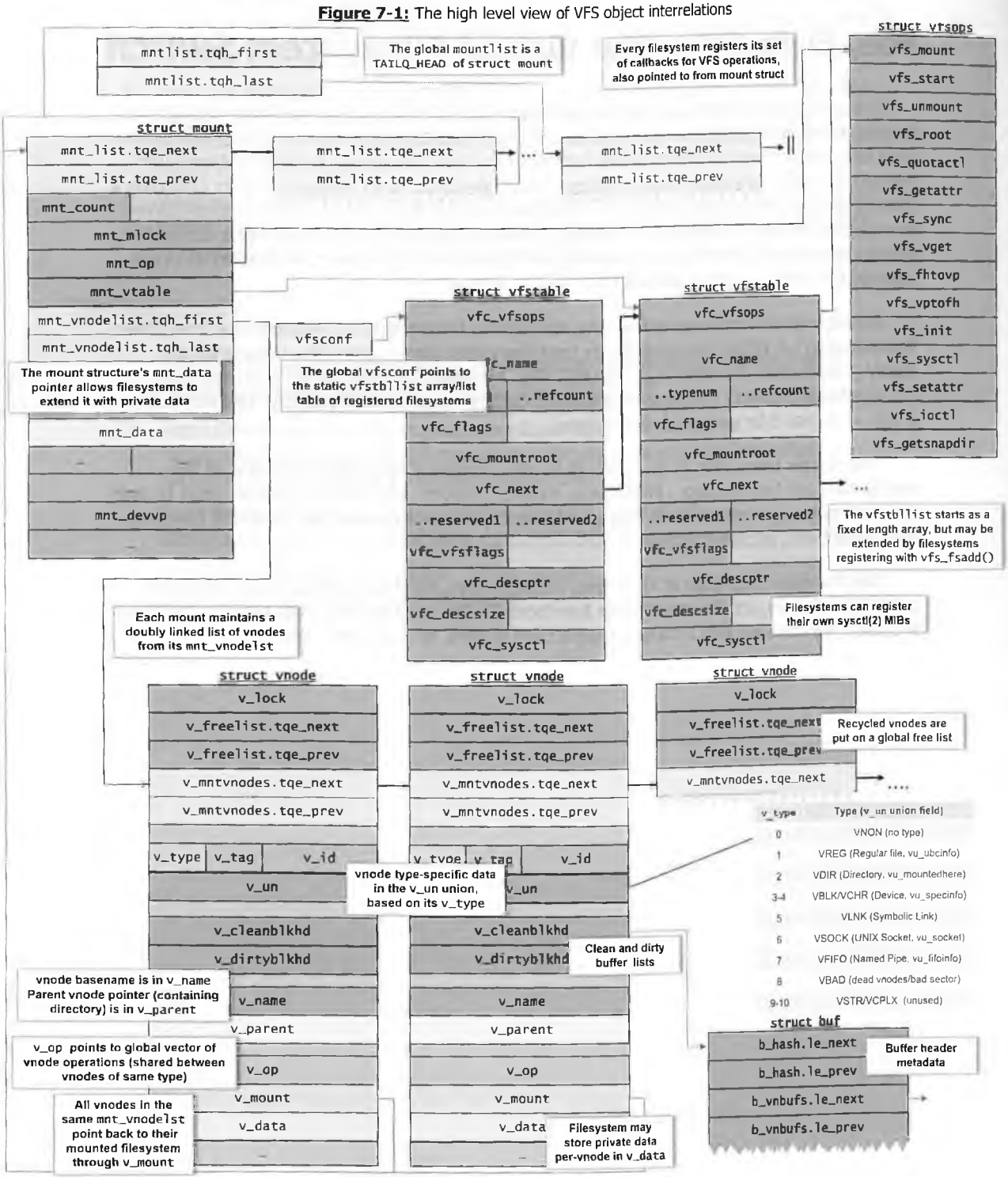
This chapter takes a top down approach to VFS, starting with a high level view of the architecture, and key concepts - Filesystems, mounts, vnodes, and buffers. Special detail is given to file attributes, and extended attributes, as those in particular provide the substrate for many Apple extensions, discussed as well.

The discussion then looks at VFS Kernel Programming Interfaces (KPIs), used by kernel code to access files, and Service Provider Interfaces (SPIs), used by drivers to provide the files to VFS itself. We conclude with a couple of quick case studies, of devfs, NFS and others.

VFS Concepts

As an adapter layer, the Virtual Filesystem Switch defines several key objects, which it expects its providers to support. The internal implementations of these objects at the filesystem level is irrelevant, so long as the provider can populate the object fields as expected. These objects provide the abstractions of registered filesystems, active mounts, and the vnode, and are interconnected to one another in various ways. Figure 7-1 depicts the high level view of the objects and their interrelations, before we move to discuss each of them, in more detail.

Figure 7-1: The high level view of VFS object interrelations



Filesystems

The list of registered filesystems is maintained in the `vfstbllist`, which is accessible through the global `vfsconf`. This is an array `vfstable` entries, which is prepopulated statically in bsd/vfs/vfs_conf.c. Depending on compilation options, this array initially holds the following filesystems:

Table 7-2: The built-in filesystems in the `vfstbllist`

Filesystem	Condition	MacOS	*OS	Description
nfs	#if NFSCLIENT	Yes	No	NFS mounts
devfs	Always	Yes	Yes	The device node file system (/dev)
nullfs	#if NULLFS	Yes	No	Loopback file system subtree
mockfs	#if MOCKFS	No	No	fake memory mapped filesystem
routefs	#If ROUTEFS	No	Yes	(Apparenly unreferenced, though defined in iOS)

At the end of the `vfstbllist` array are two empty slots (labeled "<unassigned>" with other values all zeros or NULLs), but those are quickly taken up by kernel extensions registering their filesystems through `vfs_fsadd()`, as discussed later. Thus, the `vfstbllist` holds in total five entries on MacOS and four on *OS variants (which do not support NFS). Should more filesystems be added past that limit, additional `struct vfstable` entries are created as a linked list.

Each `vfstable` entry contains the filesystem name (as a 15 character string, which should match the first argument supplied to the `[f]mount` system call(s). It contains a reference count, and two sets of flags - the `vfc_flags` and `vfc_vfsflags`. Most importantly, it contains several callbacks, which are installed by the filesystem provider. These include the `vfc_mountroot` (called when if the filesystem is used as the root), and the `vfc_vfsops`, a view of which can be seen in Figure 7-1. This structure contains (at present) some 15 callbacks, corresponding to filesystem lifecycle events. Note, that a filesystem implementation may opt to leave any of these as NULL, or choose a generic implementation. More discussion can be found later in this chapter, under "VFS KPIs".

The `vfsconf` information is exported to user mode via `CTL_VFS/VFS_GENERIC/###` `sysctl(2)` MIBs, or by the wrapped `getvfsbyname(3)`*. This prunes the pointer, descriptor and `vfc_vfsflags` values out of the `struct vfstable` and provides the user mode called with a `struct vfsconf`, defined in <sys/mount.h>. The `lsvfs(1)` command (part of the adv-cmds project) demonstrates use of these interfaces.

Figure 7-3 (next page) shows the `vfsconf` from a MacOS 14.3, which is also the first version of MacOS to introduce the AppleVirtIO.kext. When the table is full, it overflows to accommodate `autofs` (the MacOS automounter) and `apfs`. Note, that not all `vfstable` entries must necessarily be used - even if the filesystem is registered, its reference count could still be zero (as is the case with HFS, nullfs, and autofs) since there are no mounted instances of it. The same reasoning also allows a filesystem's reference count to be greater than 1, for example apfs - due to two separate volumes being mounted.

When inspecting the Figure, you might want to correlate it with Figure 7-1, which showed the higher level view. Note, also, that it is easy to tell those filesystems in XNU which are not built-in, since their `vfc_vfsops` and (optionally) `vfc_mountroot` point to kext addresses, rather those in XNU proper. The figure expands particularly the `apfs` filesystem entry, showing both kext owned pointers, as well as the dump of the apfs's `vfsops` structure.

* - This MIB has to be accessed programmatically, and not from `sysctl(8)` command. It is not to be confused with `vfs.generic.fsname`, which is command-line accessible, but retrieves different, file-system specific information.

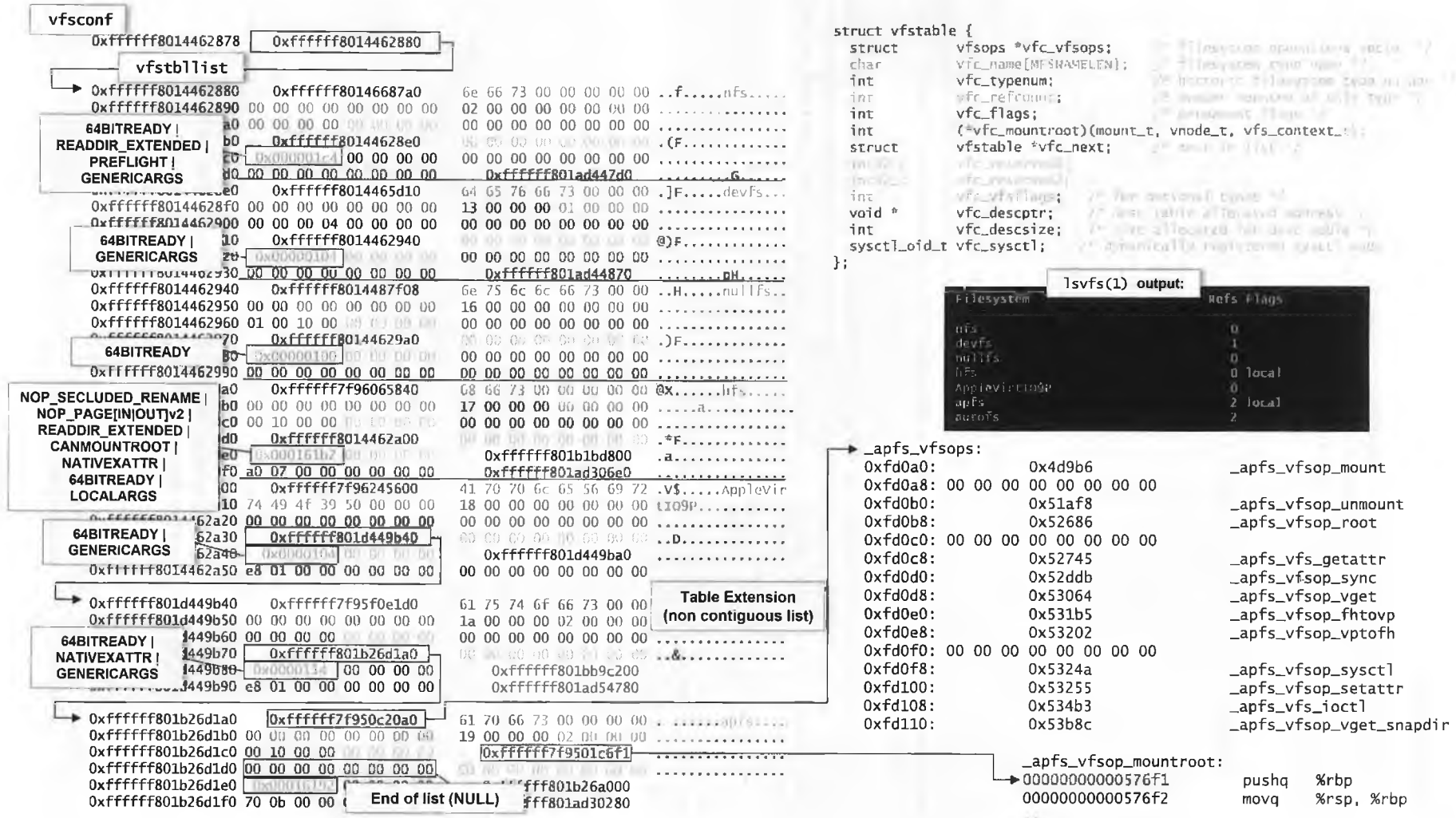


Figure 7-3: The VFS mount table, visualized

Mounts

In order to be accessible, a filesystem must be mounted. The `mount(8)` command line can be used for this purpose, and it internally executes the filesystem specific mount_*XXX*fs variant from /System/Library/FileSystems. For filesystems not internally supported by the kernel, this triggers an automatic kext loading operation, as was discussed in I/3.

Whichever filesystem ends up being mounted, the `mount(2)` system call (#167) is responsible for performing the mounts. Its implementation is common with `__mac_mount` (#424), which also allows specifying a MAC Framework label, which can later be retrieved from files in the mount through `__mac_get_mount` (#425). Darwin 17 adds an additional call, `fmount` (#526) (in bsd/vfs/vfs_syscalls.c), which can be used to mount over an open file descriptor. This system call cannot be used to mount the root filesystem, union mounts or mounting by index. An additional method of mounting is through `kernel_mount`, also in bsd/vfs/vfs_syscalls.c, but reserved for in-kernel mounts (such as devfs and routefs) and not through a system call. Unmounting is performed through the standard `unmount(2)` call (#159).

Mounting a filesystem involves selecting a directory as a mount point, whose vnode will be covered. Every mount retains a pointer to the vnode in its `mnt_vnodecovered` field. Union mounts (`MNT_UNION`) allow partial coverage - that is, accessing files on the mounted file system but still allowing access to those of the underlying filesystem. Due to multiple counts of abuse by jailbreakers, however, the option is denied on `SECURE_KERNEL`s as of XNU-3248.

The system maintains all its mounts in the `mountlist`. This is a `TAILQ_HEAD` which is not exported, but can nonetheless be easily found by using `joker`. Each of its `struct mount` elements is as defined in bsd/sys/mount_internal.h, and allocated from the dedicated `mount` zone. The structure is quite massive, weighing in at 2,592/2,560 bytes as of MacOS 14-15/iOS 12/13. A large part of this is due to the `mnt_vfsstat`, which caches a `vfsstatfs` structure (defined in bsd/sys/mount.h). This structure alone (inlined, rather than pointed to) spans hundreds of bytes, owing to its `f_mntonname` (name of mount point) and `f_mntfromname` (mounted filesystem), both of which are `MAXPATHLEN` sized strings. To preserve opacity (which is important, given the size differences of MacOS and the *OS variants), some of its fields can be accessed through `vfs_*` KPIs (declared in bsd/sys/mount.h and implemented in bsd/vfs/vfs_kpi.vfs.c). The `[f]statfs[64](2)` system call variants (#157, #158, #345 and #346) can be used to retrieve some of the visible fields in user mode.

The `struct mount` holds three types of flags:

- **`mnt_flag`:** holds the `MNT_*` flags, which are user visible (through <sys/mount.h>). The lower eight bit flags are settable through `mount(2)` options, and are well documented in the man page, though the others are unmentioned outside of the header file.
- **`mnt_kern_flag`:** holds `MNTK_*` some two dozen flags, settable only by the kernel. These include `MNTK_PERMIT_UNMOUNT` (permitting non-root `umount(2)`, `MNTK_EXTENDED_SECURITY` and `MNTK_TYPENAME_OVERRIDE`. The latter two can be set through an `FSIOC_SET_FSTYPENAME_OVERRIDE` `fsctl(2)` operation setting the filesystem name to `mtmfs`.
- **`mnt_lflag`**: holds the `MNT_L*` lifecycle flags, which include `MNT_LUNMOUNT` (being unmounted), `MNT_LFORCE` (forced unmount), `MNT_LWAIT` (waiting for unmount) and `MNT_LDEAD` (already unmounted). More temporary events are `MNT_LITER` (set during vnode iteration), and `MNT_LNOTRESP` (set from `vfs_event_signal()` on `VQ_[NOTRESP/DEAD]`).

Mounts are MACF aware, and can be assigned an `mnt_mntlabel` and an `mnt_fslabel`. The former serves as a mount label, and can be retrieved with `vfs_mntlabel` (as is done by the Sandbox.kext), though the latter does not have a KPI and seems unused. The `mnt_data` field is a pointer to filesystem specific data, which can be retrieved with `vfs_fsprivate()`. HFS+ and APFS both use this to hang their own structures on their mounts.

vnodes

The chief construct in VFS is that of a **vnode**. A vnode is a representation of a file or special object, independent of the underlying file system. Commonly, a vnode would map to the underlying filesystem's index node (inode) object, though filesystem drivers are free to use the vnode's unique identifier in whatever method suits them. For example, table based filesystems (e.g. FAT) which do not support inodes can use that value as a table index. HFS+ and APFS use the number as a B-Tree node identifier.

The vnode representation is defined as a `struct vnode`. This is a 248/240-byte structure defined in bsd/sys/vnode_internal.h and allocated from the dedicated `vnodes` zone. The structure tracks everything the kernel needs to know about the vnode - from its `v_type`, through various `v_flags`, `v_lflags` and `v_listflags`, the `v_owner` (owning thread), and its `v_name` (the file's `basename(3)`). The 16-bit `v_type` field holds an `enum` specifying if this is a regular file or directory (`VREG`/`VDIR`) or some special case (`VLNK`, `VSOCK`, `VBLK`/`VCHR`, etc.), which affects the interpretation and usage of other fields.

The structure, however, is meant to remain opaque, and accessed through public KPIs, all in bsd/sys/vnode.h. These are some 120 or so functions, all well documented, and providing getters/setters to the vnode's private fields, as well as miscellaneous operations.

Vnodes are closely linked to each other. All vnodes belonging to the same mounted filesystem can be accessed through the `struct mount`'s `mnt_vnodelist`, and walked through the vnode's `v_mntvnodes`. The mounted filesystem can also be quickly accessed through the `v_mount` field, and is free to hold private data (as it does at the mount level's `mnt_data`), in an opaque `v_data` pointer. Each vnode also holds a `v_freelist` `TAILQ_ENTRY` for easy access to the vnode freelist, and name cache entry links to child vnodes and links. Further down the structure each vnode also holds a `v_parent` pointer which, along with the `v_name` pointer (pointing to its component name), allows for quick full pathname reconstruction.

A key field in the structure is the `v_ops`, a pointer to a vnode operations vector. Not to be confused with the `vfstable`'s `vfc_vfsops` (which operate at a file system level), the `v_ops` provide the implementations of the common vnode lifecycle methods. The implementations, are commonly derived from the filesystem the vnode belongs to, but there are a few quasi-filesystems defining operations as well. These are "quasi", in the sense that they are not mountable, yet define their own vnode operations - even if their vnodes are found in another file system.

- **`fifo_vnodeop_entries` (bsd/miscfs/fifofs/fifo_vnops.c):** Used for named pipes (FIFOs), as created by mkfifo(2).
- **`dead_vnodeop_entries` (bsd/miscfs/deadfs/dead_vnops.c):** Used when access to the vnode is `revoke(2)`d.
- **`spec_vnodeop_entries` (bsd/miscfs/specfs/spec_vnops.c):** Used for "special" files (devices)

Thus, the `v_op` may conveniently change according to vnode type or lifecycle stage. Not all vnode operations are necessarily supported. More detail on this can be found later in this chapter, under "VFS SPIs", and in the NFS case study. Another common occurrence during the vnode lifecycle is that its buffered data changes state - as some of it gets "dirtied" (i.e. modified). Each vnode's buffered data is maintained in two `struct buflists` - `v_cleanblkhd` and `v_dirtyblkhd`.

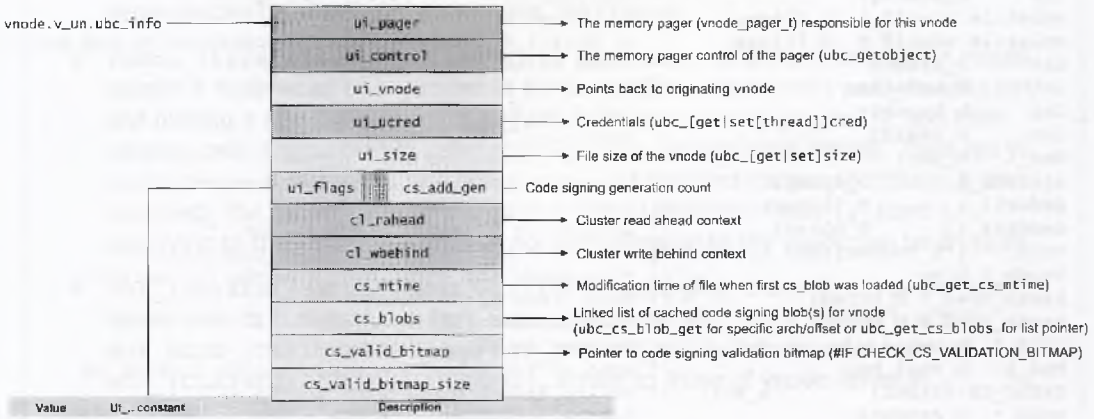
The underlying type data is maintained in the `v_un` union, which holds one of several pointers. For directory vnodes (i.e. when `v_type` is `VDIR`), this points to a `struct mount`, which is either the containing filesystem or (when the directory is a mountpoint) another `struct mount`. For UNIX domain sockets (`VSOCK`), to a `struct socket`, discussed in Chapter 14. For device files (`VCHR`/`VBLK`), to a `struct specfs` (as discussed in Chapter 6). For most vnodes (`VREG`), this points to a `ubc_info`, discussed next.

The `ubc_info` (`V_REG` vnodes)

The Unified Buffer Cache (UBC) is a concept first introduced into NetBSD[1]. Its aim is to unify the caching mechanisms of VFS (named mappings) and the VM subsystems (used for anonymous memory), thereby using one cache which can benefit from being central and common to both, reducing duplicate caching. UBC was also adopted by Apple in XNU, although the implementation varies from that of *BSD.

A key structure in UBC is the `ubc_info`. This is a structure pointed to from the vnode's `ubc_info` field (in the `v_un` union field, which applies for a `vtype` of `VREG`, that is regular files). `ubc_info` structures are allocated from their own dedicated zone (the `ubc_info zone`). Each `ubc_info` is created in the context of its vnode (by a call to `ubc_info_init_with_size()`, from `vnode_create_internal()`), and - if the vnode in question already has one due to vnode reuse, it is reused as well. The `ubc_info` also points back to the `struct vnode` which refers to it. Figure 7-4 visualizes the `ubc_info` structure:

Figure 7-4: The `struct ubc_info` (from bsd/sys/ubc_internal.h)



Value	UI_* constant	Description
0x00000000	_NONE	No flags
0x00000001	_HASPAGER	Has a pager associated (ui_pager, ui_control)
0x00000002	_INITED	Newly created vnode
0x00000004	_HASOBJREF	Holds a reference on object
0x00000008	_WASMAPPED	Vnode was mapped, but is not at this time (ubc_is_mapped[_writable])
0x00000010	_ISMAPPED	Vnode is currently mmapped (ubc_is_mapped)
0x00000020	_MAPBUSY	Vnode in process of being mapped, or unmapped
0x00000040	_MAPWAITING	Waiting for a UI_MAPBUSY
0x00000080	_MAPPEDWRITE	Vnode is mapped with PROT_WRITE

Most of the UBC information deals with maintaining the vnode content data in memory. The pager and pager control elements point to the Mach memory pager (in particular, a vnode pager, as discussed in Chapter 11). The cluster read-ahead and write-behind handle the fetching and syncing of the vnode contents to disk through maintaining Universal Page Lists (UPLs, also discussed in Chapter 11).

The rest of the elements are used for the code signing subsystem. The most important of which are the cached `cs_blobs`, which (as discussed in III/5) are used by XNU to enforce code signatures on individual pages, store entitlements, and report code singing information back to user mode via the `csops[_auditttoken]` system calls (#169, #170). The blob information is added to the `ubc_info` structure by `ubc_cs_blob_add()`, from `load_code_signature()` (bsd/kern/mach_loader.c), unless a blob already exists for the vnode, as may be retrieved by `ubc_cs_blob_get()`. More details on code signing, including the specifics of blob validation can be found in III/5.

In the interest of preserving opacity, access to the `ubc_info` fields is performed using `ubc_[get/set]*` functions (all in bsd/sys/ubc_subr.c), which internally call `UBCINFOEXISTS`, a macro checking the vnode's `ubc_info` pointer, and then dereferencing it to get the specific field. The `ubc_info_t`'s getters and setters are just one part of the KPI exported by the UBC layer. There are quite a few code signing related functions (`ubc_cs_*`, discussed in III/5), and the remaining functions deal with Universal Page Lists (UPLs), discussed in more detail throughout Chapter 11.

Buffers

Vnodes maintain **buffers**, which are used to hold the data of their various I/O requests. The vnode maintains two `struct buflists` pointers - one in `v_dirtyblkhd` (for dirty buffers, i.e. those which may need flushing) and another in `v_cleanblkhd`.

The `buflists` are `LIST_HEAD`s of `struct bufs`. These (from bsd/sys/buf_internal.h) serve as buffer headers, holding the metadata for individual buffers, which include their linkage, state, and more. The `struct buf` however, is meant to remain opaque, and accessible through the KPIs in the well documented bsd/sys/buf.h.

Listing 7-5: The `struct buf` (from bsd/sys/buf_internal.h)

```
struct buf {
    LIST_ENTRY(buf) b_hash;          /* Hash chain. */
    LIST_ENTRY(buf) b_vnbufs;        /* Buffer's associated vnode. */
    TAILQ_ENTRY(buf) b_freelist;     /* Free list position if not active. */
    int     b_timestamp;             /* timestamp for queuing operation */
    struct timeval b_timestamp_tv; /* microuptime for disk conditioner */
    int     b_whichq;                /* the free list the buffer belongs to */
    volatile uint32_t  b_flags;      /* B_* flags. */
    volatile uint32_t  b_lflags;     /* BL_BUSY | BL_WANTED flags... protected by buf_mtx */
    int     b_error;                 /* errno value. */
    int     b_bufsize;               /* Allocated buffer size. */
    int     b_bcount;                /* Valid bytes in buffer. */
    int     b_resid;                 /* Remaining I/O. */
    dev_t   b_dev;                   /* Device associated with buffer. */
    uintptr_t       b_datap;         /* Memory, superblocks, indirect etc.*/
    daddr64_t       b_lblkno;        /* Logical block number. */
    daddr64_t       b_blkno;         /* Underlying physical block number. */
    void    (*b_iodone)(buf_t, void *);     /* Function to call upon completion. */
    vnode_t b_vp;                    /* File vnode for data, device vnode for metadata. */
    kauth_cred_t b_rcred;            /* Read credentials reference. */
    kauth_cred_t b_wcred;            /* Write credentials reference. */
    void *  b_upl;                   /* Pointer to UPL */
    buf_t   b_real_bp;               /* used to track bp generated through cluster_bp */
    TAILQ_ENTRY(buf)        b_act;   /* Device driver queue when active */
    void *  b_drvdata;               /* Device driver private use */
    void *  b_fsprivate;             /* filesystem private use */
    void *  b_transaction;           /* journal private use */
    int     b_dirtyoff;              /* Offset in buffer of dirty region. */
    int     b_dirtyend;              /* Offset of end of dirty region. */
    int     b_validoff;              /* Offset in buffer of valid region. */
    int     b_validend;              /* Offset of end of valid region. */

    /* store extra information related to redundancy of data, such as
     * which redundancy copy to use, etc
     */
    uint32_t b_redundancy_flags;
    proc_t  b_proc;                  /* Associated proc; NULL if kernel. */
#ifdef BUF_MAKE_PRIVATE
    buf_t   b_data_store;
#endif
    struct bufattr b_attr;
};
```

As rich as the `buf` structure is, it does not hold the actual buffer data, which is maintained separately, through the `buf`'s `b_datap` (accessible via `buf_[set]dataptr()`), and/or the `b_upl` (accessible through `buf_[set]upl()`. UPLs (Universal Page Lists) are explained in chapter 11, but for now they can just be thought of as their names - lists of memory pages, reserved to hold the buffer contents, and populated by a call to `buf_map()`. The `b_datap` may also contain buffer metadata, `zalloc()`ed from respective dedicated zones, with element ranging in powers of two from 512 through 16,348.

Buffer headers are allocated from `bsd_startupearly()`, through a call to `kmem_suballoc()` which attempts to allocate `max_nbuf_headers` and `niobuf_headers` `buf` structures. The value of `max_nbuf_headers` can be obtained via the `nbuf` boot argument, or (otherwise) set to 2% of ram (`sane_size / 50`), but clipped so it is no less than `CONFIG_MIN_NBUF` but no more than 16,384. Additionally, `niobuf_headers` is set to match `max_nbuf_headers` (if less than 4,096), or to half of it plus 2,048.

Reading or writing a buffer from/to disk is performed by `buf_b[read/write]()`. The actual operation is then carried out by `VNOP_STRATEGY`, by default asynchronously. This may be made blocking, by calling `buf_biowait()` on the buffer, as is performed by `buf_bread`, and by `buf_bwrite()` for buffers marked as synchronous (i.e. in which the `B_ASYNC` is not set).

Experiment: Iterating over VFS structures using KPI iterators

Given the usefulness of mounted filesystems and their vnodes, it's often required to iterate over these structures. This can be done using `TAILQ_FOREACH` over the structures, as is done for example by `panic_print_vnodes()` (in bsd/vfs/vfs_subr.c) during a panic. The problem, however, is the structures are not readily visible outside the VFS files.

Fortunately, a series of `_iterate` functions are all supported KPIs and allow their caller to iterate over the structures, specifying a callback action to perform:

- `vfs_iterate()` applies a callback for each mounted filesystem (struct mp *). Internally, it calls `mount_fillfsids` to an `fsid_list` array, and then iterates on it, calling `mount_list_lookupby_fsid` to get back the `struct mount`, skipping dead mounts or unmounts in progress. Iteration can be performed sequentially or in reverse order, if `VFS_ITERATE_TAIL_FIRST` is specified as a flag. Another optional flag is `VFS_ITERATE_CB_DROPREF` which is used (along with the former) by `vfs_unmountall()`, traversing the mount list in reverse order (to avoid dependencies) and calling the `unmount_callback`.
- `vnode_iterate()` applies a callback for each `vnode` in the `struct mp *` mount pointer it receives as an argument. It works by taking the mount's `mnt_vnodelist` and moving it into the `mnt_workerqueue` (in vnode_iterate_prepare()), and then iterating over it according to caller specified flags. Several such `VNODE_` flags are in bsd/sys/vnode.h, but in practice only `VNODE_RELOAD` and `VNODE_WRITEABLE` are observed. The callback can return `VNODE_[CLAIMED|RETURNED][_DONE]`, specifying to the iterator whether or not the reference to the vnode can be dropped.
- `buf_iterate()` (in bsd/vfs/vfs_bio.c) can be used given a `struct vnode *` to iterate over its buffers. Flags from bsd/sys/buf.h are `BUF_SKIP_[NON]LOCKED`, `BUF_SCAN_[CLEAN/DIRTY]` and `BUF_NOTIFY_BUSY`. Return codes here are `BUF_[CLAIMED/RETURNED][_DONE]`, similar to those of vnode iteration.

A good example of this can be seen in the implementation of `vfs_purge`, a non-POSIX system call (#455) added in Darwin 13 to allow purging (flushing) of all vnodes on all mounted filesystems, also available as the `purge(1)` utility. The system call implementation (in bsd/vfs/vfs_syscalls.c) calls `vfs_iterate()` to loop over all mounted filesystems, with `vfs_purge_callback` as the action to perform to each. This, in turn, calls `vnode_iterate` on its mount point argument, specifying the `vnode_purge_callback` as the action to perform for each vnode.

Listing 7-6: `vfs_purge` demonstrating vnode iteration

```
static int vfs_purge_callback(mount_t mp, __unused void * arg) {
 vnode_iterate(mp, VNODE_WAIT|VNODE_ITERATE_ALL, vnode_purge_callback, NULL);
 return VFS_RETURNED;
}

int vfs_purge(__unused struct proc *p,
              __unused struct vfs_purge_args *uap, __unused int32_t *retval) {
   if (!kauth_cred_issuser(kauth_cred_get())) return EPERM;
   vfs_iterate(0 /* flags */, vfs_purge_callback, NULL);
   return 0;
}
```

Using the example, create your own vnode iterator over all active mounts in the system in a kernel extension. You may want to experience with various callbacks, as well as assign different triggers (for example, on extension load, by `sysctl(2)` or device `ioctl(2)`).

One specific real-world application of this experiment is in the iOS 11.3 jailbreak, which required remounting the root filesystem read-write by overwriting the initial mountpoint data. Previously opened vnodes had their backpointer to the mount data incorrectly set - but the `vnode_iterate` could be used to overcome that.

File System Attributes

VFS maintains various attributes for file system objects. Attributes are broken into five groups: common attributes (`ATTR_CMN_*`) apply to all types of objects. Volume attributes (`ATTR_VOL_*`) apply to the mounted filesystem itself. Directory attributes (`ATTR_DIR_*` are defined for folders. Files have both file and fork attributes (`ATTR_FILE_*` and `ATTR_FORK_*`, respectively). Fork attributes, however, are deprecated and when the `FSOPT_ATTR_CMN_EXTENDED` flag is specified they are repurposed to provide additional, non-standard common attributes.

The attributes are all hardcoded in four corresponding tables, defined in bsd/vfs/vfs_attrlist.c - `getattrlist_common_tab`, `getattrlist_dir_tab`, `getattrlist_file_tab` and `getattrlist_common_extended_tab`. The tables are all `getattrlist_attrtab` structures, also defined (ibid.) as shown in Listing 7-7. The unique structure of the tables (shown in the annotation comment, above) makes it easy to locate the table in the decompressed kernelcache's `__TEXT.__const`, using the `joker` module.

Listing 7-7: The `struct getattrlist_attrtab` attribute tables (from bsd/vfs/vfs_attrlist.c)

```
/*
 * Table-driven setup for all valid common/dir/file/fork attributes against files.
 */
struct getattrlist_attrtab {
        attrgroup_t     attr;
        uint64_t        bits;
#define VATTR_BIT(b)    (VNODE_ATTR_ ## b)
        ssize_t         size;
        kauth_action_t  action;
};

/*
 * A zero after the ATTR_ bit indicates that we don't expect the underlying FS to report
 * back with this information, and we will synthesize it at the VFS level.
 */
static struct getattrlist_attrtab getattrlist_common_tab[] = {
//   0x00000001   0000000002000000    sizeof(int32_t + uint32_t)              (1<<7)
 {ATTR_CMN_NAME, VATTR_BIT(va_name), sizeof(struct attrreference), KAUTH_VNODE_READ_ATTRIBUTES},
 {ATTR_CMN_DEVID, 0,                 sizeof(dev_t),                KAUTH_VNODE_READ_ATTRIBUTES},
 ..
 {ATTR_CMN_DATA_PROTECT_FLAGS,
    VATTR_BIT(va_dataprotect_class), sizeof(uint32_t),        KAUTH_VNODE_READ_ATTRIBUTES},
 {0, 0, 0, 0}
};
```

Thus, each attribute is represented by a single bit, using the `VNODE_ATTR_...` macros (in bsd/sys/vnode.h). When attributes are queried or set, a `struct vnode_attr` is used (ibid.) is used, along with the vnode pointer and the vfs context, as an argument to `vnode_[get/set]attr()`. Although the structure is not kernel private, `VATTR_*` macros are the preferred way of dealing with the attribute bitmaps in this structure, which also contains the classic UN*X attributes (ownership and mode), timestamps (access/creation/modification) and the fields returned by `[f]stat(2)` on the file.

There are a variety of ways to query attributes from user mode. On a path name, `getattrlist(2)` (#220), or (as of Darwin 14) `getattrlistat(2)` (#476) may be used. Alternatively, `fgetattrlist(2)` (#228) can be used on a file descriptor. To handle so many attributes, the system calls use a `struct attrlist`, which breaks the attributes into five 32-bit bitmaps. In this way, a caller can ask for multiple attributes at once. Darwin 14 introduces another system call, `getattrlistbulk(2)` (#461), specifically intended for retrieving attributes for multiple objects in the same directory.

Perusing the respective manual pages of all the above provides good examples as to the system call usage. Note, also, that certain attributes may be defined as writable, in which case `setattrlist(2)` or `fsetattrlist(2)` (system calls #221 and #229, respectively) can be used to modify them. Darwin 17 adds `setattrlistat(2)` (system call #524), which provides the basis for `utimensat(2)` and other calls.

Finally, note that in Listing 7-7 that attribute table entries also have a `kauth_action_t` associated with them, commonly `KAUTH_VNODE_READ_ATTRIBUTES`. As explained in III/3, The KAuth facility is a precursor (circa 10.4) to the Mandatory Access Control Framework (as of 10.5). KAuth is called out from `vnode_authorize` and `..authattr[_new]`, after MACF's `mac_vnode_check_[get/set]*` callouts are called. As discussed in III/4, the MAC Framework delegates the decision to a policy extension, commonly Sandbox.kext.

`fsctl(2)`

The `fsctl(2)` system call (#242), along with the file-descriptor based `ffsctl` (#245) are proprietary system calls meant for high level filesystem control operations*. Using any one of `ioctl(2)`-style predefined control codes, a user mode caller

The `fsctl(2)` codes known to XNU proper are defined in bsd/sys/fsctl.h, which is also exported to the user mode`<sys/fsctl.h>`. The codes are `#define` both as `FSIOC_*` and as corresponding `FSCTL_*`, with the latter being an application of the `IOCBASECMD` macro over the former. The codes are shown in Table 7-8:

Table 7-8: Filesystem independent `[f]fsctl(2)` codes

FSIOC_../FSCTL_.. code	Purpose	
.._SYNC_VOLUME	Force a `sync(2)` on mounted volume	
.._SET_PACKAGE_EXTS	LaunchServices: set package extensions to ignore	
.._NAMESPACE_HANDLER_GET	VFS Namespace handlers	
.._NAMESPACE_HANDLER_UPDATE		
.._NAMESPACE_HANDLER_UNBLOCK		
.._NAMESPACE_HANDLER_CANCEL		
.._NAMESPACE_HANDLER_SET_SNAPSHOT_TIME		
.._OLD_SNAPSHOT_HANDLER_GET		
.._SNAPSHOT_HANDLER_GET_EXT		
.._SET_FSTYPENAME_OVERRIDE	0x80044a3a	Override fs name, or mount with `MNTK_EXTENDED_SECURITY`
.._NAMESPACE_ALLOW_DMG_SNAPSHOT_EVENTS	[Dis]Allow snapshot events on disk images	
.._ROUTEFS_SETROUTEID	(`#if ROUTEFS`) Set new route ID	
.._FIOSEEKHOLE	Deprecated: now in `fcntl(2)`	
.._FIOSEEKDATA		
(DISK_CONDITIONER_)IOC_GET	Disk conditioner	
(DISK_CONDITIONER_)IOC_SET		
(SPOTLIGHT_IOC_)GET_MOUNT_TIME	Used by Spotlight (I/4)	
(SPOTLIGHT_IOC_)GET_LAST_MTIME		

The implementation of both syscalls, `fsctl_internal()` (in bsd/vfs/vfs_syscalls.c) first converts the `FSCTL*` codes to their `FSIOC_*` counterparts, and then `switches` over the generic, filesystem independent commands shown in Table 7-8. Any other commands are passed directly to the underlying filesystem's handler, via the `VNOP_IOCTL` wrapper.

* - The man page for both these calls is still found in XNU's bsd/man/man2/fsctl.2, but not installed to the MANPATH. Perhaps this is for the best, seeing as the man page is terribly outdated, and the one code it lists (`FSGETMOUNTINFOSIZE`) is not even present in the XNU sources anymore.

Extended Attributes

Apple makes heavy use of extended attributes, or **xattrs**. As explained in Volume I/3, extended attributes provide the implementations of important filesystem features, such as compression, data protection, and resource forks. Thus, any filesystem which can support xattrs (as both HFS+ and APFS do), can also provide these features.

As with standard attributes, there are several system calls to handle extended attributes: `[f]getxattr` (#234,235), `[f]setxattr` (#236,237) and `[f]removexattr` (#238,239) can be used to manipulate known attributes by name (all in bsd/vfs/vfs_syscalls.c). As is the common convention, the `f...` variants work on an already open file descriptor. `[f]listxattr` (#240,241) can be used to list the extended attributes of a pathname or descriptor, although `com.apple.system.*` xattrs get filtered through `xattr_protected()`.

The system calls funnel to the kernel internal functions, `vn_[get|set|remove]xattr` (in bsd/vfs/vfs_xattr.c). Similar to standard attributes, both the MAC Framework's callout (`mac_vnode_check_[get|set|delete]extattr`, hooked by Sandbox.kext and MacOS's Quarantine.kext) and that of Kauth's (i.e., calling `vnode_authorize()` with `KAUTH_VNODE_[READ|WRITE]_EXTATTRIBUTES`) must agree to allow the operation.

Some filesystems natively support extended attributes, whereas others do not. Those which do, advertise this capability with the `VFS_TBLNATIVEXATTR` flag (q.v. Listing 7-27). In those which do not (e.g. FAT-derivatives), extended attributes may be emulated by use of hidden "Apple Double" dot-underscore (._) files, `#defined` as `ATTR_FILE_PREFIX`. The emulation is also used when archiving files into formats which do not support extended attributes, e.g. `tar(1)`. Thus, when `CONFIG_APPLEDOUBLE` is set (as it is, by default), the implementations of `default_[get/set/list]xattr()` (in bsd/vfs/vfs_xattr.c) call `open_xattrfile()` to open the Apple Double file in kernel, and then `get_xattrinfo()` to populate the `attr_info_t`. The AppleDouble file format is documented with ASCII art in bsd/vfs/vfs_xattr.c) as shown in Listing 7-9:

Listing 7-9: The AppleDouble Header File layout (from bsd/vfs/vfs_xattr.c)

```
/*
 Typical "._" AppleDouble Header File layout:
 ------------------------------------------------------------
        MAGIC           0x00051607
        VERSION         0x00020000
        FILLER          0
        COUNT           2
    .-- AD ENTRY[0]     Finder Info Entry (must be first)
.---+-- AD ENTRY[1]     Resource Fork Entry (must be last)
|   '-> FINDER INFO
|       /////////////   Fixed Size Data (32 bytes)
|       EXT ATTR HDR
|       /////////////
|       ATTR ENTRY[0] --.
|       ATTR ENTRY[1] --+--.
|       ATTR ENTRY[2] --+--+--.
|          ...          |  |  |
|       ATTR ENTRY[N] --+--+--+--.
|       ATTR DATA 0   <-'  |  |  |
|       /////////////      |  |  |
|       ATTR DATA 1   <----'  |  |
|       /////////////         |  |
|       ATTR DATA 2   <-------'  |
|       /////////////            |
|          ...                   |
|       ATTR DATA N   <----------'
|       /////////////
|                     Attribute Free Space
|
'-----> RESOURCE FORK
        /////////////   Variable Sized Data
        /////////////
        /////////////
        /////////////
           ...
        /////////////
 ------------------------------------------------------------
 NOTE: The EXT ATTR HDR, ATTR ENTRY's and ATTR DATA's are
 stored as part of the Finder Info.  The length in the Finder
 Info AppleDouble entry includes the length of the extended
 attribute header, attribute entries, and attribute data.
*/
```

Experiment: Examining AppleDouble attribute files

The easiest way to create AppleDouble files is to insert a removable drive formatted with FAT32 or vFAT, both of which do not support extended attributes natively. Using `xattr(1)` to create any arbitrary attribute will result in the creation of an AppleDouble. Removing the AppleDouble will remove the attribute, as shown below:

Output 7-10-a: Demonstrating extended attributes on FAT32, which does not support them natively

```
morpheus@Zephyr (/Volumes/NO NAME) % touch x; xattr -w test value x
morpheus@Zephyr (/Volumes/NO NAME) % ls -la@ x
-rwxrwxrwx@ 1 morpheus  staff  0 Apr 24 20:30 x
        test    5
#
# Removing the file will remove the attribute
#
morpheus@Zephyr (/Volumes/NO NAME) % rm ._x
morpheus@Zephyr (/Volumes/NO NAME) % ls -la@ x
-rwxrwxrwx  1 morpheus  staff  0 Apr 24 20:30 x
```

When the attribute and file exist, hexdumping the file will show the structure presented in Listing 7-9. Output 7-10-b shown the file created by the above xattr addition, annotated. Note that entries are in big endian format, and 16-bit aligned.

Output 7-10-b: Viewing an AppleDouble file

```
#
# The extended attribute is implemented by a hidden ._ file:
#
morpheus@Zephyr (/Volumes/NO NAME) % ls -la@ ._x
-rwxrwxrwx  1 morpheus  staff  4096 Apr 24 20:32 ._x
morpheus@Zephyr (/Volumes/NO NAME) % hexdump -C ._x
           MAGIC        VERSION          Filler (ADH_MACOSX)
00000000  00 05 16 07   00 02 00 00      4d 61 63 20  4f 53 20 58  |........Mac OS X|
                                    numEntries AD_FINDERINFO
00000010  20 20 20 20   20 20 20 20      00 02 00 00  00 09 00 00  |        ........|
                    |          AD_RESOURCE    offset
00000020  00 32 00 00   0e b0 00 00      00 02|00 00  0e e2|00 00  |.2..............|
          length
00000030  01 1e|00 00   00 00 00 00      00 00 00 00  00 00 00 00  |................|
00000040  00 00 00 00   00 00 00 00      00 00 00 00  00 00 00 00  |................|
                                                      total_size
00000050  00 00 00 00   41 54 54 52   |  3b 9a c9 ff  00 00 0e e2  |....ATTR;.......|
          data_start    data_length
00000060  00 00 00 88   00 00 00 05      00 00 00 00  00 00 00 00  |................|
                                         offset       length
00000070  00 00 00 00   00 00 00 01      00 00 00 88  00 00 00 05  |................|
          flags nl   name[6]
00000080  00 00 05 74   65 73 74 00      76 61 6c 75  65 00 00 00  |...test.value...|
00000090  00 00 00 00   00 00 00 00      00 00 00 00  00 00 00 00  |................|
*
00000ee0  00 00 00 00   01 00 00 00      01 00 00 00  00 00 00 00  |................|
00000ef0  00 1e 54 68   69 73 20 72      65 73 6f 75  72 63 65 20  |..This resource |
00000f00  66 6f 72 6b   20 69 6e 74      65 6e 74 69  6f 6e 61 6c  |fork intentional|
00000f10  6c 79 20 6c   65 66 74 20      62 6c 61 6e  6b 20 20 20  |ly left blank   |
00000f20  00 00 00 00   00 00 00 00      00 00 00 00  00 00 00 00  |................|
*
00000fe0  00 00 00 00   01 00 00 00      01 00 00 00  00 00 00 00  |................|
00000ff0  00 1e 00 00   00 00 00 00      00 00 00 1c  00 1e ff ff  |................|
00001000
```

The two mandatory attributes, `AD_ATTRIBUTES` (0x09, at 0ffset 0x32 and spanning 0xeb0 bytes) and `AD_RESOURCE` (0x02, at offset 0xee2 spanning 0x11e bytes, for the resource fork), are created automatically, and highlighted. The `AD_ATTRIBUTES` contain one attribute, identified by the `ATTR_HDR_MAGIC` ('ATTR'), and conforming to the `struct attr_header` (also in bsd/vfs/vfs_xattr.c), with the attribute defined as an `attr_entry`:

Listing 7-10-c: The `struct attr_entry` (from bsd/vfs/vfs_xattr.c)

```
typedef struct attr_entry {
    u_int32_t   offset;  /* file offset to data */
    u_int32_t   length;  /* size of attribute data */
    u_int16_t   flags;
    u_int8_t    namelen;
    u_int8_t    name[1]; /* NULL-terminated UTF-8 name (up to 128 bytes max) */
} __attribute__((aligned(2), packed)) attr_entry_t;
```

Apple Extensions

Apple makes heavy use of VFS features - specifically, extended attributes - in order to provide additional, non-standard and mostly private functionality.

Table 7-11: Non-standard VFS extensions, and the mechanisms providing them

Extension	Support	Provides
Resource Forks	Extended Attributes	Alternate Data Streams
Compression		Transparent file compression
Restricted		Darwin 15: Prevent modification to file, sans entitlement
Data Vaulting		Darwin 17: Prevent read access to file, sans entitlement
Data Protection		`NSFileProtectionClass` encryption for sensitive files
FSEvents	Char device	Filesystem Notifications via /dev/fsevents character device
Document IDs	Proprietary	32-bit Identifiers tagging files & directories to track their lifecycle
Object IDs	Proprietary	64-bit identifiers uniquely identifying an object for direct open
Disk Conditioning	Proprietary	Intentional I/O degradation/throttle for specific mount points
Triggers	Proprietary	Trigger vnodes used for automounting filesystems in MacOS
`EVFILT_VNODE`	kqueues	Trigger vnodes used for automounting filesystems in MacOS
`/dev/vn##`	device	Loop mount device nodes, `#if NVNDEVICE`
File Providers	Host port	Designated processes serving as VFS namespace resolvers

Resource Forks

Resource forks are an antiquated legacy of the MacOS Classic days. The Macintosh File System (MFS) could support a number of "forks", which enabled storing multiple related data elements in the same file*. The main fork used was the resource fork, in which application resources (icons, images and other media) could be stored. The NeXTSTEP bundle format provides a far better method of storing resources, but resource forks are nonetheless supported to this day. This support is enabled by `#define`ing `NAMEDRSRCFORK`, as is done by default across all Darwin flavors.

As discussed in Volume I (Output 3-22), the resource fork may be accessed by requesting the file's `com.apple.ResourceFork` extended attribute, or by simply appending "`..namedfork/rsrc`" to any file. Special handling in `cache_lookup_path()` (in bsd/vfs/vfs_cache.c) checks if a filename component requested starts with two dots and followed by the `_PATH_RSRCFORKSPEC`, and the filesystem supports forks (the `mount` structure's `mnt_kern_flag` of `MNTK_NAMED_STREAMS` is set). If so, then the cached vnode's `cn_flags` `CN_WANTSRSRCFORK` is set, and VFS syscalls operate on the fork instead of the actual vnode.

Operating on the fork involves a call to `vnode_getnamedstream` (in bsd/vfs/vfs_xattr.c). If the filesystem supports named streams, it is expected to provide a vnop callback for this operation. If not the `default_getnamedstream` implementation is called. The HFS+, APFS, and NFSv4 filesystems all provide callbacks for getting, making and removing named streams.

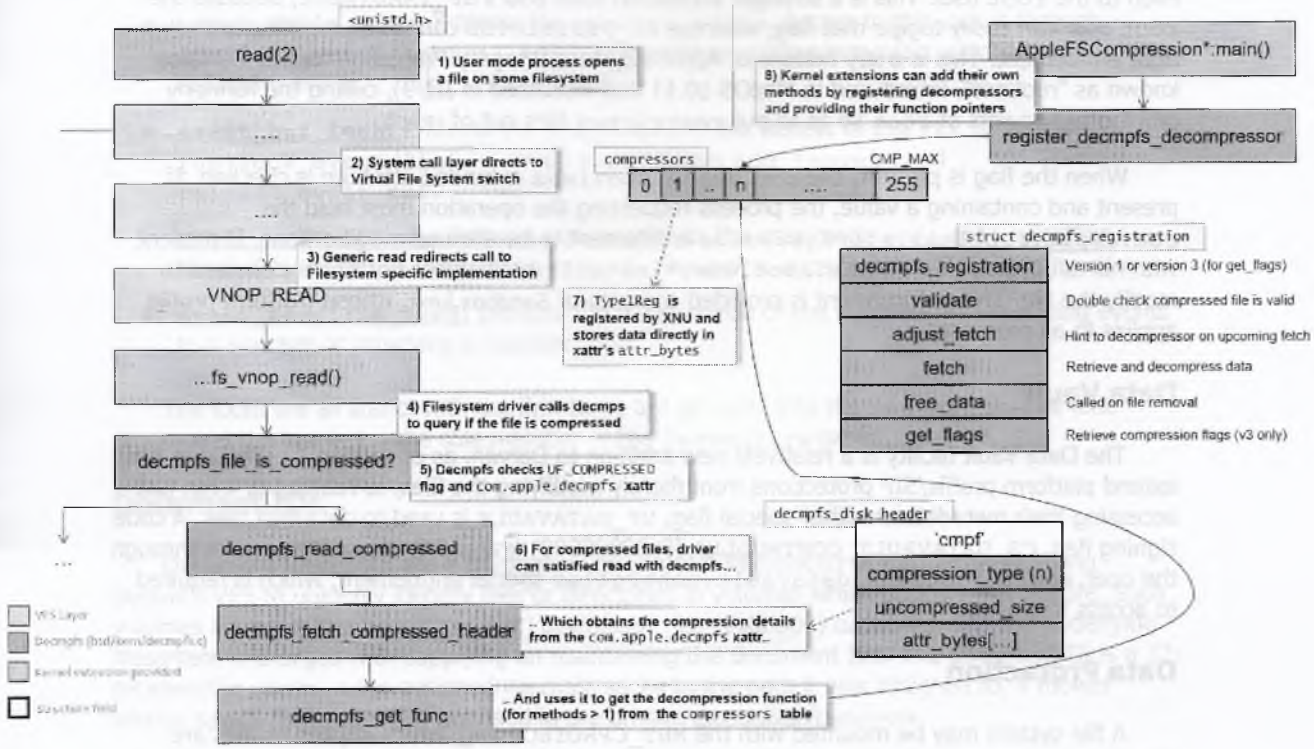
File compression

The `com.apple.decmpfs` xattr implements the transparent filesystem compression. "Transparent", in that the calls manipulating a compressed file with VFS have no idea if a file is compressed or not. The filesystem calls `decmpfs_file_is_compressed()` on vnode access, (i.e. when implementing `..._vnop_open()`), which calls `decmpfs_cnode_get_vnode_state` to check a cached result. The slower path checks for the `UF_COMPRESSED` flag, which must always be accompanied by a `com.apple.decmpfs` extended attribte. The extended attribute is expected to hold, at a minimum, a `decmpfs_header` (from bsd/sys/decmpfs.h), which will indicate the `compression_type` and the `uncompressed_size` (which is reported as the file size for `ls(1)` and similar tools when the file is flagged as `UF_COMPRESSED`). Files which are small enough may have their contents compressed into the extended attribute's value. In other cases the compressed data may be held in the resource fork.

* - Windows users may be familiar with the NT equivalent of "Alternate Data Streams", e.g. `::$DATA` and the like.

When the file data is requested, the driver can call `decmpfs_pagein_compressed` and `decmpfs_read_compressed` to handle the decompression, while remaining entirely oblivious to the decompression algorithm used. This is shown in Figure 7-12:

Figure 7-12: The `decmpfs` mechanism architecture, visualized



Kexts supplying decompression functions register with the kernel by calling `register_decmpfs_decompressor`*. Although the `compression_type` is a 32-bit field, the `decompressors` table is limited to `CMP_MAX` (255) methods. In practice, far fewer are used. `register_decmpfs_decompressor` also publishes the decompression methods as `IOResource` objects, so they are visible in the IORegistry:

Output 7-13: Displaying the registered AFSC types, using `ioreg(8)`

```
#
# Get either com_apple_AppleFSCompression_* kext names, or com.apple.AppleFSCompression.providesType* properties.
# This has the caveat that it might miss on AppleFSCompression providers not following the naming convention,
# but that hasn't happened yet
#
morpheus@Chimera (~)$ ioreg -l -w 0 -f |
                      egrep -- "(-o,.*FSCompressionType|AppleFSCompression.providesTy)"
      +-o com_apple_AppleFSCompression_AppleFSCompressionTypeZlib
          <class com_apple_AppleFSCompression_AppleFSCompressionTypeZlib, ..>
          "com.apple.AppleFSCompression.providesType7" = Yes
          "com.apple.AppleFSCompression.providesType9" = Yes
          "com.apple.AppleFSCompression.providesType4" = Yes
          "com.apple.AppleFSCompression.providesType10" = Yes
          "com.apple.AppleFSCompression.providesType8" = Yes
          "com.apple.AppleFSCompression.providesType11" = Yes
          "com.apple.AppleFSCompression.providesType12" = Yes
          "com.apple.AppleFSCompression.providesType3" = Yes
      +-o com_apple_AppleFSCompression_AppleFSCompressionTypeDataless
          "com.apple.AppleFSCompression.providesType5" = Yes
```

The flow in the above diagram can (somewhat) be traced thanks to KDebug codes, which are emitted at specific points as of Darwin 18. Compression is transparent, but might pose a challenge for third party raw filesystem tools, which access the filesystem data from outside XNU, and therefore need to implement their own decompression logic. `fsleuth` handles most common compression types known at the time of writing.

---

* - MacOS's type 5 compression also registers /dev/afsc_type5

Restricted (MacOS)

One of Apple's most notable extensions is the `com.apple.rootless` extended attribute. When coupled with the `SF_RESTRICTED` `chflags(2)` flag, it marks the file as immutable, even to the `root` user. This is a stronger protection than BSD's `SF_IMMUTABLE`, because the `root` user can easily toggle that flag, whereas `SF_RESTRICTED` cannot be modified with the right entitlement. This is a key feature of Apple's System Integrity Protection for MacOS (also known as "rootless", introduced in MacOS 10.11 and discussed in III/9), culling the formerly omnipotent powers of `root` so as to put `restricted` files out of reach.

When the flag is present, the `com.apple.rootless` extended attribute is checked. If present and containing a value, the process requesting the operation must hold the `com.apple.rootless.storage.`*`value`* entitlement to be allowed modifications. If present with no value, only `com.apple.rootless.*install*` entitlement holders are allowed to modify the file. This enforcement is provided courtesy of `Sandbox.kext`, whose platform profile applies to all processes.

Data Vault

The Data Vault facility is a relatively new addition to Darwin, as of version 17. The idea is to extend platform profile/SIP protections from merely modifying the files, to reading or even just accessing their metadata. Another special flag, `UF_DATAVAULT` is used to datavault files. A code signing flag, `CS_DATAVAULT_CONTROLLER` (0x80000000) granted to blessed processes through the `com.apple.rootless.datavault.controller` special entitlement, which is required to access these files.

Data Protection

A file system may be mounted with the `MNT_CPROTECT` flag, which implies its files are protected through `NSFileProtectionClass`. As described in Volume III (Chapter 11, specifically 11-6 through 11-9), the `com.apple.system.cprotect` extended attribute holds the wrapped per-file key, which is unwrapped by `Apple[SEP]KeyStore.kext` callbacks. Calling `getattrlist(2)` with the `ATTR_CMN_DATA_PROTECT_FLAGS` will retrieve the file protection class for a given file system object. Refer to III/11 for more details about the extended attribute format, protection classes, and `AppleKeyStore` callbacks.

FSEvents

When XNU is compiled with `CONFIG_FSE` (as is the case by default), filesystem events also get directed to the **FSEvents** facility. As described in I/4 (under "FSEvents"), this facility (entirely self-contained in `bsd/vfs/vfs_events.c`) presents itself to user mode as the `/dev/fsevents` character device. Clients can then use the device to listen on global filesystem notifications, reading in a stream of `kfs_event` structures (q.v. Figure 4-1 in Volume I). When in kernel mode, the `kfs_event` structures are buffered into their own dedicated zone, `fs-event-buf`. The size of the zone is set at `MAX_FSEVENTS` (4096) entries, though may be overridden by the `kern.maxkfsevents` boot argument.

The FSEvents clients are referred to as **watchers**. recall (from I/4) that watchers are expected to use the `FSEVENTS_CLONE` `ioctl(2)`, and supply a `clone_args` structure, containing the event reporting array and a queue depth. The kernel mode handler `fseventsioctl` takes these arguments and calls `add_watcher()` to populate an `fs_event_watcher` entry in the `watcher_table` array. Then, when an fsevents record is generated (in numerous locations throughout VFS, by calling `add_fsevent`), the watcher table is consulted, and - if the specified event type is marked `FSE_REPORT` and the device node (=volume) it is from was not on the `devices_not_to_watch` list, the watcher (which is presumably blocking on `read(2)` from the cloned descriptor) is woken up. The cloned descriptor is of `DTYPE_FSEVENTS`, and its `read(2)` is serviced by `fmod_watch()`, which populates the `kfs_event` record .

There is a hard-coded limit of `MAX_WATCHERS` (8). Apple therefore discourages direct use of the character device (in fact warning that it is "unsupported"), and offers the user-mode `FSEvents.framework`, which uses `fseventsd`. The daemon, along with other Apple processes (namely `coreservicesd`, `revisiond` and Spotlight's `mds` get flagged as `WATCHER_APPLE_SYSTEM_SERVICE` (0x0010). This flag prevents events from being dropped

when the watcher queue is over 75% full. This also allows watchers to set directories to ignore, as per some internal radar.

To handle concurrent thread access, FSEvents uses four `lck_mtx_t` locks:

- **`watch_table_lock`:** Protects the `watcher_table`. Access to this lock is through `[un]lock_watch_table()`, which is used when adding/removing watchers or delivering events.
- **`event_buf_lock`:** Protects the `kfs_fsevent` list. Access to this lock is through `[un]lock_event_list()`, which is called from `add_fsevent` and `release_event_ref`.
- **`event_writer_lock`:** Protects concurrency to the user mode `write(2)` operation, handled by the `fseventswrite` callback. The lock is accessed directly in said function.
- **`event_handling_lock`:** protects the event queue of the watchers, when adding events to a watcher or removing a watcher.

The locks are all static, with the first three are grouped into the `fsevent-mutex` lock group, and the last being the sole member of the `fsevent-rw` group.

Document IDs

Document Identifiers are a proprietary mechanism introduced in XNU-2422, enabling Darwin's VFS to uniquely identify files or directories in volumes which support this feature. Such volumes advertise the `VOL_CAP_FMT_DOCUMENT_ID` (0x80000) capability, and the underlying filesystem is charged with supplying an maintaining the document IDs. The document ID is a 32-bit identifier which - once assigned - remain sticky to the path it was assigned to, if moved and/or saved. Document IDs are used in the private CloudDocs.framework.

The undocumented `UF_TRACKED` flag of `chflags(2)` is used to assign a document ID to a file, and also remove it (when the flag is removed). Note, that the flag *does not* appear in the output of `ls -O` (which normally prints other flags), nor is it recognized by the `chflags(1)` utility. The ID of a given filesystem object can be retrieved as the `ATTR_CMN_DOCUMENT_ID` (0x100000) attribute along with the `FSOPT_ATTR_CMN_EXTENDED` flag (so that fork attributes get reinterpreted). ID lifecycle changes can be tracked through the FSEvents mechanism: `FSE_DOCID_[CREATED/CHANGED]` (#12, #13) events track their usage. The `filemon` tool is able to monitor these events.

Document tombstones

Files marked with a document ID are closely monitored for lifecycle changes. When such files are created, edited, renamed or removed the VFS layer offers "document tombstones" as a way to store the metadata about the last operation on the particular Document ID.

Tombstones are `doc_tombstone` structures, defined in bsd/sys/tombstone.h along with their KPIs, as shown in Listing 7-14 (next page). The KPIs are all private, and their main users are filesystem drivers. APFS remains closed source, but some examples of these KPIs can be found in HFS.kext, whose sources are available in the hfs project. The tombstone is saved in the BSD `uthread`'s t_tombstone field.

Object IDs

Another undocumented feature is the ability to open a file by specifying the filesystem and object ID. The undocumented `openbyid_np` system call (#479). The operation requires a MACF privilege (`PRIV_VFS_OPEN_BY_ID`), which the Sandbox enforces with the `com.apple.private.vfs.open-by-id` entitlement. Among the holders of the entitlement are `backupd`, `searchd`, `revisiond` and the iCloud components (`bird(8)/brctl(1)`, `cloudd(8)` and others), which utilize the syscall through the private `CloudDocsDaemon` framework's `BRCOpenByID` wrapper.

Listing 7-14: Document tombstone structures and KPIs, from bsd/sys/tombstone.h

```
/*
 * struct representing a document "tombstone" that's recorded
 * when a thread manipulates files marked with a document-id.
 * If the thread recreates the same item, this tombstone is
 * used to preserve the document_id on the new file.
 *
 * It is a separate structure because of its size -- we want to
 * allocate it on demand instead of just stuffing it into the
 * uthread structure.
 */
struct doc_tombstone {
        struct vnode     *t_lastop_parent;
        struct vnode     *t_lastop_item;
        uint32_t          t_lastop_parent_vid;
        uint32_t          t_lastop_item_vid;
        uint64_t          t_lastop_fileid;
        uint64_t          t_lastop_document_id;
        unsigned char     t_lastop_filename[NAME_MAX+1];
};
struct doc_tombstone *doc_tombstone_get(void);
// remove a tombstone
void doc_tombstone_clear(struct doc_tombstone *ut, struct vnode **old_vpp);

// Save parent/child name so that doc id can be reapplied
void doc_tombstone_save(struct vnode *dvp, struct vnode *vp,
                        struct componentname *cnp, uint64_t doc_id,
                        ino64_t file_id);

// ignore if nameptr ends with .bak or .tmp
bool doc_tombstone_should_ignore_name(const char *nameptr, int len);
// true unless should_ignore_name or name is NULL
bool doc_tombstone_should_save(struct doc_tombstone *ut, struct vnode *vp,
                               struct componentname *cnp);
```

Disk Conditioning (Darwin 17)

Disk Conditioning is a facility for intentional degradation of I/O performance from specific mount points. It allows delaying access time as well as restricting read/write throughput of devices, through callouts made throughout VFS. As with the Network Link Conditioner, it cannot be used to improve times - only to introduce artificial latency and delay. The `dmc(1)` utility controls the facility when applied over a mount point. The utility works its magic by two specific `[f]fsctl(2)` (#242/245) codes - `DISK_CONDITIONER_IOC_[GET/SET]`, both of which accept a `disk_conditioner_info` structure. The structure is defined in bsd/sys/fsctl.h, as shown in Listing 7-15. The entire file is marked `XNU_KERNEL_PRIVATE`, so it is not exported to the user mode headers.

Listing 7-15: The `disk_conditioner_info` structure (from XNU 4570's bsd/sys/fsctl.h)

```
/* Disk conditioner configuration */
typedef struct disk_conditioner_info {
  int enabled;
  uint64_t access_time_usec;     // maximum latency until transfer begins
  uint64_t read_throughput_mbps; // maximum throughput for reads
  uint64_t write_throughput_mbps; // maximum throughput for writes
  int is_ssd; // behave like an SSD - accessed by disk_conditioner_mount_is_ssd
} disk_conditioner_info;

..
/* Disk conditioner */
#define DISK_CONDITIONER_IOC_GET    _IOR('A', 18, disk_conditioner_info)
#define DISK_CONDITIONER_FSCTL_GET  IOCBASECMD(DISK_CONDITIONER_IOC_GET)
#define DISK_CONDITIONER_IOC_SET    _IOW('A', 19, disk_conditioner_info)
#define DISK_CONDITIONER_FSCTL_SET  IOCBASECMD(DISK_CONDITIONER_IOC_SET)
```

The in-kernel functionality is provided by bsd/vfs/vfs_disk_conditioner.c, through a set of `disk_conditioner_*` functions. The `disk_container_delay` is the main one, called from `buf_biodone` (in bsd/vfs/vfs_bio.c) to introduce the artificial delay on a conditioned mount device. `disk_conditioner_[get/set]_info` are used to support the corresponding `ioctl(2)` codes from Listing 7-15, and `disk_conditioner_unmount` is used when unmounting a conditioned mount point, so as to free the `disk_conditioner_info` structure.

Triggers (MacOS)

Darwin's VFS implementation provides support for **vnode triggers**. These enable an interested kernel extension to set callbacks which will be invoked when the vnode is accessed, particularly for mounting. Triggers are dependent on the `CONFIG_TRIGGERS` compile-time option, which is set for MacOS but not elsewhere. This makes sense, as the chief use of triggers is in autofs.kext, which performs NFS-automounting on MacOS - a feature which the *OS variants have no need of. As a consequence, the `sizeof(struct vnode)` is larger by eight bytes on MacOS, owing to the need to place a `vnode_resolve_t` at the end of the structure.

Triggers are registered through a call to `vfs_addtrigger()` (defined in bsd/sys/mount.h) to accept a mount point, relative path, trigger info and the VFS context. The trigger info is a `struct vnode_trigger_info`, defined in bsd/sys/vnode.h as follows:

Listing 7-16: The `vnode_trigger_info` (from XNU 4903's bsd/sys/vnode.h)

```
struct vnode_trigger_info {
  trigger_vnode_resolve_callback_t    vti_resolve_func;
  trigger_vnode_unresolve_callback_t  vti_unresolve_func;
  trigger_vnode_rearm_callback_t      vti_rearm_func;
  trigger_vnode_reclaim_callback_t    vti_reclaim_func;
  void *                              vti_data;   /* auxiliary data (optional)  */
  uint32_t                            vti_flags;  /* optional flags (see below) */
};
..
#define VNT_AUTO_REARM          (1 << 0)
#define VNT_NO_DIRECT_MOUNT     (1 << 1)
```

When the vnode is accessed a call to `vnode_trigger_resolve()` checks if its `v_resolve` field holds a `vr_resolve_func` and it is not already mounted or already flagged with `VNT_RESOLVED`. As of Darwin 18, triggers are also subject to a new MACF hook, `mac_vnode_check_trigger_resolve()`.

Assuming all checks allow, the callback is called, and its implementation is entirely up to the kext. In the case of `autofs.kext`, it works with `triggers.kext` to propagate the event up to a user mode daemon - `automountd`, on host special port #11, implementing MIG subsystem #666 with 8 messages, and configurable through /etc/autofs.conf.

The `autofs` project, consisting of both kexts (autofs and triggers), daemons (autofsd and automountd), mount_autofs, mount_url and several interesting test utilities is open source[2], and the interested reader is encouraged to peruse it, to find multiple examples of working with triggers and mounting.

`EVFILT_VNODE kevent(2)` notifications

In addition the formidable FSEvents mechanism, Apple extended their VFS implementation so as to integrate it the `kevent(2)` facility. As explained in I/8, `kqueue(2)`s and their `kevent(2)` provide a substrate for GCD. Unlike FSEvents, which is global, `EVFILT_VNODE` requires arming with a specific file descriptor, which will be watched for write-oriented lifecycle events (`NOTE_[WRITE/LINK/EXTEND/ATTRIB/LINK/RENAME/REVOKE]` and `[..FUNLOCK]`). This makes it closer in operation style to Linux's `inotify` than FSEvents, though directory descriptors cannot be used here.

/dev/vn## (conditional)

Darwin adopts from BSD support for a special device called /dev/vn##, which can be used to provide a device interface (character, but most commonly block) to a file. This can be used for the common practice of loop mounting, wherein a file containing a raw filesystem image can be mounted like a device. This is contingent on XNU being compiled with `NVNDEVICE #defined` as an integer, which will cause the creation of /dev/vn## nodes from 0 to that number minus one.

Loop mounting is a common practice, but the `-o loop` option of `mount(8)`, which might be familiar to some from Linux, is not supported. Instead, Darwin requires `open(2)`ing the file to be used in this way, and issuing a `VNIOCATTACH` `ioctl(2)`, supplying the file name, file size, and read/write in a `struct vn_ioctl` (from <sys/vnioctl.h>). This causes the creation of a `/dev/vn###` with `###` taking the next available number. Once the `/dev/vn###` device is initialized, it can be treated as any other device, and can be `mount(8)`ed, etc.

The `/usr/libexec/vndevice` binary on MacOS and BridgeOS (which still identifies internally as `vncontrol`) provides a CLI which performs the VN `ioctl(2)`s. The kernel-side implementation is in bsd/dev/vn/vn.c. This feature, which was supported by default in earlier versions of Darwin, is no longer supported, with `NVNDEVICE` undefined since the HFS+ legacy volume name exploit[3] which relied on this mechanism was used to jailbreak iOS 4.x.

File Providers

Apple introduced file providers back in iOS10, modifying the API before making it public in Darwin 19 for MacOS as well. These are application extensions, allowing developers to provide a filesystem-like interface for documents on remote servers, using the Fileprovider.framework and the `NSFileProviderExtension` class. The framework is somewhat documented by Apple[4], though all details of implementation are hidden.

Internally, file providers are an application of VFS namespaces ("nspaces"). The nspace subsystem is initialized by a call to `nspace_resolver_init()`, during `vfsinit()`. Apple originally used this to support filesystem snapshots. As of Darwin 19, is seems* that this has been extended to support a new concept of "dataless" files. Not much is known about these, but they are marked with `SF_DATALESS` and appear to be placeholders for remote files. Such files can be materialized (fetched) or evicted. The evictability of a file can be retrieved through `ffsctl(2)` with the (presently) undocumented code of 0x40084a47, and toggled through `0xc0084a44`.

When a file is materialized, `vnode_materialize_dataless_file` calls `resolve_nspace_item[_ext]()`. This checks if a resolver is registered, and ensures (through a call to `vfs_context_is_dataless_manipulator()`, that the holds the `com.apple.private.vfs.dataless-manipulation` or `..dataless-resolver` entitlements. Then, an upcall to it is made using `send_nspace_resolve_path()`. The upcall is performed through a MIG message to host special port #30, which presently contains the following messages:

Table 7-17: The `nspace*` MIG subsystems

#	Routine	Description
867800	`nspace_handle`	Presently (β 1) unimplemented
867801	`nspace_resolve_cancel`	Cancel an existing path resolution request
867802	`nspace_resolve_path`	Issue a path resolving request for specified path

The nspace resolver, as of Darwin 19, is the filecoordinationd(8) daemon. The daemon was introduced as far back as Darwin 17, but in 19 it also claims HSP #30. The daemon registers itself by setting the `vfs.nspace.resolver` `sysctl(2)` MIB. When a resolver process exits, `proc_exit()` calls `nspace_resolver_exited()` to automatically deregister.

The FileProvider.framework's fileproviderd, acts as an intermediary between file provider extensions (for which there is some documentation) and the `filecoordinationd`.

---

* - At the time of writing the XNU sources are not available (and, from experience, may not be for a while). When they are, one could hope to find the code behind this facility in bsd/vfs/vfs_syscalls.c.

Experiment: Using the `fileproviderctl(1)` utility

The powerful `fileproviderctl(1)` utility, which Apple thankfully includes both on MacOS and *OS (for `sysdiagnose(1)`) as Darwin 19, can be used to test file eviction and materialization. This fascinating utility also supports unicode emojis in the Terminal environment, which makes it surprising to use.

For this experiment, iCloud Drive must be enabled - which is easy to check with `fileproviderctl listproviders`. If it is, go to one of the drive-enabled folders. It's easy to check those too through the utility:

Output 7-18: The `fileproviderctl(1)` utility in action

```
morpheus@Bifröst (~)$ cd ~/Desktop
morpheus@Bifröst (~/Desktop)$ fileproviderctl listproviders
com.apple.CloudDocs.MobileDocumentsFileProvider
morpheus@Bifröst (~/Desktop)$ fileproviderctl enumerate `!!`
    0: Desktop id:i6a9d2  10/14/19, 9:37 PM
    1: Documents id:i6a9c2  Today, 10:58 AM
   collection  gathering completed in 0.224s
   observed item: iCloud Drive id:NSFileProviderRootContainerItemIdentifier
   Today, 10:57 AM 4.61 GB available on iCloud
```

If iCloud Drive is enabled, placing a file on the desktop and calling `fileproviderctl evict` *`file`* will make the file vanish, leaving in its place a hidden file, *.file*.icloud:

Output 7-19: File eviction

```
morpheus@Bifröst (~/Desktop)$ ls -l ./passwd
-rw-r--r--  1 morpheus  staff  6946 Oct 12 13:59 ./passwd
#
# Perform eviction
#
morpheus@Bifröst (~/Desktop)$ fileproviderctl evict !$
morpheus@Bifröst (~/Desktop)$ ls -l ./passwd
ls: ./passwd: No such file or directory
#
#
#
morpheus@Bifröst (~/Desktop)$ ls -l
total 48
drwx------@  8 morpheus  staff   256 Oct 14  2019 .
drwxr-xr-x+ 28 morpheus  staff   896 Oct 12 13:57 ..
-rw-r--r--@  1 morpheus  staff  6148 Oct 12 13:59 .DS_Store
-rw-r--r--   1 morpheus  staff     0 Apr 25 20:24 .localized
-rw-r--r--@  1 morpheus  staff   154 Oct 12 13:59 .passwd.icloud
#
# Examine this new .passwd.icloud file (a bplist) using jlutil(j)
#
morpheus@Bifröst (~/Desktop)$ jlutil .passwd.icloud
   NSURLNameKey: passwd
   NSURLFileSizeKey: 6946
   NSURLFileResourceTypeKey: NSURLFileResourceTypeRegular
```

The hidden file holds the basic metadata required to pull this file back from iCloud. Doing so requires using `fileproviderctl(j)` again:

Output 7-20: File materialization

```
#
# Materialize the file
#
morpheus@Bifröst (~/Desktop)$ fileproviderctl materialize passwd
Attempting to materialize item at ~/D{5}p/p{4}d
file ~/D{5}p/p{4}d:
ASCII text
morpheus@Bifröst (~/Desktop)$ ls -l ./passwd
-rw-r--r--  1 morpheus  staff  6946 Oct 12 13:59 ./passwd
```

Trying this experiment while either debugging `filecoordinationd` or `XPOcE(j)`ing `fileproviderd` will reveal a spate of messages and interactions between the daemons.

VFS KPIs

The main use of VFS is through its rich set of Kernel Programming Interfaces, all neatly defined and commented in bsd/vfs/vfs_kpi.vfs. Through the KPIs, kernel level code requiring VFS client functionality - accessing vnode member data - can do so through accessors, leaving the `vnode_t` as an opaque pointer. This is important, considering the `vnode_t` structure tends to change every now and then between releases. Over four dozen such accessors exist, and a good way to find them is to try `grep ^vnode_ kpi_vfs.c | grep "vp)"`. Note, however, that not all accessors are defined as "approved" KPIs. `vnode_tag` is presently Unsupported, and a few are Private.

Additional VFS functionality is provided by bsd/vfs/vfs_vnops.c, which provides `vn_[open/read/write/rdwr/access/select/close]*` - i.e. calls that mimic (and, in some cases, support) the standard file descriptor lifecycle. Additional, less common calls (such as `vn_[get/set/list/remove]xattr`) can be found in respective files. Lastly, bsd/vfs/vfs_subr.c, providing advanced functionality, such as authorization, triggers and resolvers.

The `vfs_context_t`

An important structure used throughout VFS KPIs is the `vfs_context_t`. This is a pointer to a `struct vfs_context`, defined in bsd/sys/user.h as a structure of two fields - the `vc_thread` (the Mach `thread_t`) and the `vc_ucred` (the thread's `kauth_cred_t` credential structure). Normally, the `vfs_context_current()` can be used to retrieve the current thread context, which is equivalent to accessing the `uu_context` field of the `uthread` returned by `get_bsdthread_info(current_thread())`, or `vfs_context_kernel()` if no other context could be found. `vfs_context_kernel()` itself is (as noted in its comment) a **very** dangerous hack, but one which has been in use for a long time and Apple shows no signs of addressing. Calling `vfs_context_create()` on a given context can clone it, or create a new context (with the `current_thread` and `kauth_cred_get`) if called with NULL. Context thus created are `kalloc()`ed from `kalloc.16`, and should be freed with `vfs_context_rele()`.

The `vfs_context_t` is used extensively by VFS to determine the attributes of the current operation. Table shows all the KPIs which can be used:

Table 7-21: KPIs used when working with `vfs_context_t`

vfs_context_..	Purpose
.._proc	Get the context's `vc_thread`'s owning `proc_t`, or `current_proc()`
.._pid	Convenience to get the PID from `vfs_context_proc()`
.._suser	Check if `kauth_cred_t` is that of the superuser (uid 0)
.._is64bit	Check if `kauth_cred_t` is that of the superuser (uid 0)
.._issignal	Convenience to get `proc_pendingsignals` from `vfs_context_proc()`
.._cwd	Get the `uu_cdir` from the `vc_thread`'s matching `uuthread`, or the `vfs_context_proc()`'s `p_fd->fd_cdir`
.._[get/set]_special_port	`task_[get/set]_special_port` on the `vc_thread`'s task.
.._issuser	Convenience returning true if context is superuser
.._iskernel	Convenience returning true if context is `vfs_context_kernel()`
..ucred	Return `kauth_cred_t` of process in context, or NULL

Manipulating files in kernel mode

It is inevitable that, sooner or later, kernel code will need to access a file directly. This is the case when, for example, a file is provided to `execve(2)`, or other system calls which accept a pathname as an argument - such as `open(2)`, `getfh(2)` and the like. Another is when the file needs to be created from kernel mode - such as when dumping a core or kdebug tracing. In these cases, there must be a (relatively) simple way to convert a pathname to a `struct vnode`. Such functionality is provided by `namei()`, from bsd/vfs/vfs_lookup.c.

The `namei()` function takes a single argument - a pointer to a `struct nameidata` to encapsulate the lookup arguments, defined in bsd/sys/namei.h as follows:

Listing 7-22: The `struct nameidata` (from bsd/sys/namei.h)

```
struct nameidata {
        /*
         * Arguments to namei/lookup.
         */
        user_addr_t ni_dirp;            /* pathname pointer */
        enum    uio_seg ni_segflg;      /* location of pathname */
#if CONFIG_TRIGGERS
        enum    path_operation ni_op;   /* intended operation, enum path_operation in vnode.h */
#endif /* CONFIG_TRIGGERS */
        /*
         * Arguments to lookup.
         */
        struct  vnode *ni_startdir;     /* starting directory */
        struct  vnode *ni_rootdir;      /* logical root directory */
        struct  vnode *ni_usedvp;       /* directory passed in via USEDVP */
        /*
         * Results: returned from/manipulated by lookup
         */
        struct  vnode *ni_vp;           /* vnode of result */
        struct  vnode *ni_dvp;          /* vnode of intermediate directory */
        /*
         * Shared between namei and lookup/commit routines.
         */
        u_int   ni_pathlen;             /* remaining chars in path */
        char    *ni_next;               /* next location in pathname */
        char    ni_pathbuf[PATHBUFLEN];
        u_long  ni_loopcnt;             /* count of symlinks encountered */

        struct componentname ni_cnd;
        int32_t ni_flag;
        int ni_ncgeneration;            /* For a batched vnop, grab generation beforehand */
}
```

As the listing shows, the `nameidata` is a rather complex structure. File access therefore begins with a call to `NDINIT`. This is a macro `#define` in bsd/sys/namei.h as accepting several arguments:

Table 7-23: The arguments of the `NDINIT` macro

Arg	Purpose
nd	is a `struct nameidata` to be initialized by the call, passed by reference.
op	An operation which is one of `LOOKUP` (0), `CREATE` (1), `DELETE` (2) or `RENAME` (3). This is passed to `VNOP_LOOKUP`.
pop	is a more precise operation, which is used only if the kernel is compiled with `CONFIG_TRIGGERS`. This is an `OP_*` `path_operation` value, passed to resolvers
flags	Theser set the `cn_flags` (component name flags of the `ni_cnd` field of the node data. Common values are `[NO]FOLLOW` (symbolic links) ,`LOCKLEAF` (to auto-lock vnode on return) and `AUDITVNPATH[1/2]`, to request auditing of pathname.
segflg	A `UIO_*` value specifying the origin of the `namep` argument (`UIO_USERSPACE`/`UIO_SYSSPACE`)
namep	The pathname of the vnode to be opened. When taken from userspace, it is used with the the `CAST_USER_ADDR_T` macro.
ctx	is the VFS context, commonly `vfs_current_context()`

With the `struct nameidata` initialized, the next step is to call `namei()` on it. This performs the vnode lookup, by taking the `namep` component, calling `copyinstr(9)` (if obtained from `UIO_USERSPACE`) or `copystr(9)`, and then perform the step by step directory traversal (by processing pathname components in between '/' separators), resolving symbolic links if the `FOLLOW` flag is set. The operation continues until the name can be resolved, and MACF is consulted (using `mac_vnode_check_lookup_preflight` before the actual `lookup()` operation or if a symbolic link is encountered.

The `lookup()` function is, as its comment states, "a very central and rather complicated routine". Its complexity arises from the many special cases it needs to consider: double dot (..) directory specifiers, union mounts, resource forks, and other idiosyncrasies. The heart of the lookup operation, however, is in a call to `VNOP_LOOKUP`, through which VFS finds the underlying filesystem driver's `...vnop_lookup` handler. This way the filesystem specific logic can be entirely decoupled from the pathname and link processing. The return code of the lookup can be any of the common `errno` codes, such as `ENOENT`, `EACCES`, `EPERM`, etc.

Remaining optimistic and assuming the lookup was successful, `namei()` will likewise be successful, and the vnode will be ready for use in the `ni_vp` member of the `struct nameidata`. When the `nameidata` contains a buffer (as indicated by the `HASBUF` flag of `ni_cnd.cn_flags` and a non NULL `ni_cnd.cn_pnbuf`), care must be taken to free it. This is handled by a call to `nameidone()`, which resets the flag, resets the pointer to NULL and uses `FREE_ZONE` to free the buffer from the dedicated `M_NAMEI` BSD zone. When the `vnode` is no longer needed, it **must** be released with a call to `vnode_put()`, which will decrease its `v_iocount` and possibly put it on the vnode list for recycling (discussed later).

Kernel code can also take a different route over the `NDINIT/namei` approach, and call `vnode_open` (from bsd/vfs/vfs_subr.c). This simplifies the process into a single line of code, hiding the eventual use of both the macro and `namei` (by the internal `vn_open_auth()`), but the route is more scenic (and laced with more authentication checks).

The main use of a `struct vnode` is for I/O operations, through `vn_rdwr()`. This function, declared and well commented in bsd/sys/vnode.h, can be used to either `UIO_READ` or `UIO_WRITE` any `len` bytes from/to `offset` to/from a specified buffer `base` address. Additional arguments to this function are the `seglg` indicating whether `base` is a `UIO_USERSPACE` or `UIO_SYSSPACE`, the credentials of the requestor, and the `struct proc p` of the process on behalf of which the I/O request is done. Additionally, `ioflg` specifies a bitmask of `IO_*` options - from bsd/sys/vnode.h, and `aresid`, a pointer to an integer storing the number of bytes which remain in the I/O request after `vn_rdwr` completes.

The `vn_rdwr()` implementation is a wrapper of `vn_rdwr_64()`. The caller supplied buffer is used to create a `uio_t`, and an `iov_t` is added to it, created from the caller specified `base` and `len`. Unless the caller specified `IO_NOAUTH` as an `ioflg`, A MACF check is performed here - calling out to `mac_vnode_check_[read/write]`. Assuming the check succeeds (or the caller bypassed by specifying `IO_NOAUTH`), the operation is performed by `VNOP_[READ/WRITE]`, with the only exception being if the vnode provided is the swap vnode (in which case reads are services by `vn_read_swapfile()`). On return, the `uio` is checked for any unsatisfied request bytes using `uio_resid`, and the value is written to the `aresid` argument (or the call returns `-EIO`, if `aresid` was not specified).

This pattern of access can be seen in several locations around the kernel. The Mach-O loading process of `get_macho_vnode()` (from bsd/kern/mach_loader.c, discussed in Chapter 6) is one such example. Another good one is `exec_activate_image`, which supports the `execve(2)` implementation:

Listing 7-24: The code of `exec_activate_image` pertaining to vnode handling (from bsd/kern/kern_exec.c)

```
exec_activate_image(struct image_params *imgp)
{
    struct nameidata *ndp = NULL;
    ...
    MALLOC(ndp, struct nameidata *, sizeof(*ndp), M_TEMP, M_WAITOK | M_ZERO);
    ....     NDINIT(ndp, LOOKUP, OP_LOOKUP, FOLLOW | LOCKLEAF | AUDITVNPATH1,
        UIO_SYSSPACE, CAST_USER_ADDR_T(excpath), imgp->ip_vfs_context);

again:
    error = namei(ndp);
    if (error)
            goto bad_notrans;

    imgp->ip_ndp = ndp;       /* successful namei(); call nameidone() later */
    imgp->ip_vp = ndp->ni_vp; /* if set, need to vnode_put() at some point  */

    ...
    error = vn_rdwr(UIO_READ, imgp->ip_vp, imgp->ip_vdata, PAGE_SIZE, 0,
                    UIO_SYSSPACE, IO_NODELOCKED,
                    vfs_context_ucred(imgp->ip_vfs_context),
                    &resid, vfs_context_proc(imgp->ip_vfs_context));
...
bad_notrans:
...
    if (imgp->ip_ndp)
            nameidone(imgp->ip_ndp);
    if (ndp)
            FREE(ndp, M_TEMP);

}
```

Direct File I/O

There is a special case in kernel mode for `kern_open_file_for_direct_io()`. This function is used exclusively by IOKit's `IOPolledInterface` class. The class is used during hibernation (from iokit/Kernel/IOHibernateIO.cpp) and certain cases of crash dumping. The class also demonstrates a use of the kernel's built-in AES support (aes_[encrypt/decrypt]_cbc, from libkern/crypto/corecrypto_aes.c).

`IOPolledInterface` provides `IOPolledFile[Open/Close/Read/Write]` methods, and the `..Open..` method uses `kern_open_file_for_direct_io()` (from bsd/kern/kern_symfile.c). The pattern is initially not unlike that of `exec_activate_image()`, shown in Listing 7-24, but also involves requesting specific file attributes, and then calling `vn_open_auth()`. It thus makes a great example of how to perform various file handling tasks in kernel.

Listing 7-25-a: the `open_file_for_direct_io()` routine (from bsd/kern/kern_symfile.c)

```
...
    // Demonstrating vn_open_auth() in kernel
    ref->ctx = vfs_context_kernel();

    fmode  = (kIOPolledFileCreate & iflags) ? (O_CREAT | FWRITE) : FWRITE;
    cmode =  S_IRUSR | S_IWUSR;
    ndflags = NOFOLLOW;
    NDINIT(&nd, LOOKUP, OP_OPEN, ndflags,
    UIO_SYSSPACE, CAST_USER_ADDR_T(name), ref->ctx);
    VATTR_INIT(&va);
    VATTR_SET(&va, va_mode, cmode);
    VATTR_SET(&va, va_dataprotect_flags, VA_DP_RAWENCRYPTED);
    VATTR_SET(&va, va_dataprotect_class, PROTECTION_CLASS_D);
    if ((error = vn_open_auth(&nd, &fmode, &va))) {
        kprintf("vn_open_auth(fmode: %d, cmode: %d) failed with error: %d\n",
            fmode, cmode, error);
        goto out; }
...
    // Demonstrating vnode_getattr() in kernel
    VATTR_INIT(&va);
    VATTR_WANTED(&va, va_rdev);
    VATTR_WANTED(&va, va_fsid);
    VATTR_WANTED(&va, va_devid);
    VATTR_WANTED(&va, va_data_size);
    VATTR_WANTED(&va, va_data_alloc);
    VATTR_WANTED(&va, va_nlink);
    error = EFAULT;
    if (vnode_getattr(ref->vp, &va, ref->ctx)) goto out;
```

The file opened in this way may also be a partition, which would require getting device geometry, etc - hence necessitating an `ioctl` in kernel. This is achieved by using a function pointer, linking it to either a device-based `ioctl()`, or a file-based one. Through the in-kernel `ioctl` functionality the device geometry or file block map/extent layout can be obtained. Blocks can be mapped into memory, and then read/write operations are as simple as an in-memory copy. When the file is closed, `kern_writefile()` (a wrapper over `vn_rdwr()`) is called.

Listing 7-25-b In kernel `ioctl` (from `open_file_for_direct_io()`, in bsd/kern/kern_symfile.c)

```
static int file_ioctl(void * p1, void * p2, u_long theIoctl, caddr_t result)
{
    dev_t device = *(dev_t*) p1;
    return ((*bdevsw[major(device)].d_ioctl)
                    (device, theIoctl, result, S_IFBLK, p2));
}
...
   do_ioctl = &file_ioctl;
...
   // get partition base
   if (partitionbase_result)
    {
       error = do_ioctl(p1, p2, DKIOCGETBASE, (caddr_t) partitionbase_result);
       if (error) goto out;
    }
    // get block size & constraints
    error = do_ioctl(p1, p2, DKIOCGETBLOCKSIZE, (caddr_t) &blksize);
    if (error) goto out;
```

Vnode lifecycle

Vnodes are allocated from the BSD `M_VNODE` zone (#25), which is backed by the dedicated `vnodes` zone. The zone grows dynamically as vnodes are allocated, but there is an upper cap on vnodes. The maximum number of vnodes is determined during `bsd_startupearly()`, set to the kernel's `sane_size` divided by 64k + 1,024. The value is further capped by the compile type `CONFIG_VNODES` macro, which is commonly 263,168. This can be overridden by the `kern.maxvnodes` argument.

File I/O, however, is very frequent. So sooner or later any limit will be hit, but vnodes never get freed - instead, they are recycled. The `struct vnode` maintains two counts - `v_usecount` (a reference count, modified by `vnode_ref_ext/vnode_rele_internal`) and `v_iocount` (for I/O operations, modified by `vnode_get()/vnode_put()` and other operations). When both these counts are zero the vnode may be put on one of three vnode freelists, by `vnode_list_add()`, depending on the vnode flags:

- Vnodes marked `VL_DEAD`: are added to the `vnode_dead_list`. This list is tried first when obtaining a new vnode.
- Vnodes marked `VRAGE`: are put on the `vnode_rage_list`, which holds "rapidly aging" vnodes. Aged vnodes are put in the front of free lists, rather than their end.
- Other vnodes: are put on the `vnode_free_list`. It is easy to determine if a given vnode is already on the freelist, through the `VONLIST` macro, which expands to a check the vnode's `v_freelist` member against the magic value of `0xdeadb`.

Vnodes have quite a few structures associated with them, so it's not a simple matter to just put them on a free list. They must be properly laundered - which is the task of `vclean()`. This routine is responsible for `fsync(2)`ing the vnode contents, cleaning the associated memory pages in the Unified Buffer Cache, calling `VNOP_INACTIVE` if the last reference to the vnode has been dropped, to advise the filesystem of this. It additionally calls `VNOP_RECLAIM()`, giving the filesystem a chance to remove the vnode from any cached structures or hash lookups, as well as deallocate filesystem private structures.

VFS SPIs

After considering all the objects and KPIs defined for use by clients VFS, let us turn our attention to those interfaces required of the service providers of VFS, and the process of developing a VFS filesystem.

Registering Filesystems

A filesystem provider can register its filesystem with VFS by calling on `vfs_fsadd`. This function (in bsd/vfs/kpi_vfs.c) takes in a `struct vfs_fsentry` by reference. If registration is successful, its second argument, a `vfstable_t` is populated with an opaque handle, which can be used when deregistering. The magic of VFS is that it handles filesystems of multiple types and varieties. For this, the very notion of a filesystem needs to be abstracted, and VFS's `struct vfs_fsentry`, shown in Listing 7-26, aims to achieve exactly that:

Listing 7-26: The `struct vfs_fsentry` (from bsd/sys/mount.h)

```
struct vfs_fsentry {
  struct vfsops * vfe_vfsops;     /* vfs operations */
  int             vfe_vopcnt;     /* # of vnodeopv_desc being registered (reg, 
  struct vnodeopv_desc ** vfe_opvdescs; /* null terminated; */
  int             vfe_fstypenum;  /* historic filesystem type number */
  char            vfe_fsname[MFSNAMELEN]; /* filesystem type name */
  uint32_t        vfe_flags;              /* defines the FS capabilities */
  void *          vfe_reserv[2];  /* reserved for future use; set this to zero*/
};
```

The `vfe_fsname` is used to locate the filesystem, when matched against the filesystem `type` specified by the `mount(2)` system call. Every filesystem should also declare the `vfe_flags`, a bitmap of `VFS_TBL*` constants (also from bsd/sys/mount.h), which inform the kernel of the filesystem capabilities. The most important fields in the `vfs_fsentry` are the `vfe_vfsops` and `vfs_opvdescs`, which specify the filesystem level operations and individual vnode level operations (respectively) that the filesystem supports. In this way, the higher level VFS operations are really just higher level shims, with the kext-supplied filesystem specific logic performing all the actual work.

Listing 7-27: The VFS filesystem flags in XNU 4903

```
/*
 * Filesystem Registration information
 */
#define VFS_TBLTHREADSAFE             0x0001  /* Only threadsafe filesystems are supported */
#define VFS_TBLFSNODELOCK             0x0002  /* Only threadsafe filesystems are supported */
#define VFS_TBLNOTYPENUM              0x0008
#define VFS_TBLLOCALVOL               0x0010
#define VFS_TBL64BITREADY             0x0020  // File system is 64-bit compatible
#define VFS_TBLNATIVEXATTR            0x0040  // File system natively supports extended attributes
#define VFS_TBLDIRLINKS               0x0080
#define VFS_TBLUNMOUNT_PREFLIGHT      0x0100  /* does a preflight check before unmounting */
#define VFS_TBLGENERICMNTARGS         0x0200  /* force generic mount args for local fs */
#define VFS_TBLREADDIR_EXTENDED       0x0400  /* fs supports VNODE_READDIR_EXTENDED */
#define VFS_TBLNOMACLABEL             0x1000
#define VFS_TBLVNOP_PAGEINV2          0x2000  // v2 APIs: Filesystem will handle UPL creation, //
#define VFS_TBLVNOP_PAGEOUTV2         0x4000  // population, locking, and release
#define VFS_TBLVNOP_NOUPDATEID_RENAME 0x8000  /* should not call vnode_update_ident on rename */
#define VFS_TBLVNOP_SECLUDE_RENAME    0x10000
#define VFS_TBLCANMOUNTROOT           0x20000
```

The `vfsent` supplied by the registrant as a parameter to `vfs_fsadd()` leads to the creation of a new `struct vfstable` entry. The `VFS_TBL*` flags (from Listing 7-27) are converted by `vfs_fsadd()` to the `VFC_VFS*` flags of the `vfstable`'s `vfc_flags`, and the `vfe_vfsops` are moved to the `vfc_vfsops`. The vnode operations (`vfe_opvdescs`) are similarly copied to a `MALLOC`'d buffer of `vfe_descsize` bytes, pointed to by `vfe_descptr`. The newly created `struct vfstable` is then either copied over an unassigned slot, or extends the table through the `vfc_next` pointer of the last entry. If the registered callback functions in the `vfc_vfsops` include a `vfs_init()` handler, it is invoked.

VFS operations

Any filesystem registered using `vfs_fsadd` may opt to install a number of callback functions, as a `struct vfsops` pointer which is provided as the first member (`vfe_vfsops`) of the `struct vfs_entry`. The structure presently defines some 15 callbacks, though not all are required. The callbacks (all pointers to functions returning an integer) are well documented in bsd/sys/mount.h, and shown in Table 7-28:

Table 7-28: The VFS operations defined in the `struct vfsops` (from bsd/sys/mount.h)

Operation	Purpose
vfs_mount(*mp, devvp, data, context*);	Mount the fs from *devvp* on *mp*
vfs_start(*mp, flags, context*);	Start the mounted fs at *mp*. *flags* unused
vfs_unmount(*mp, mntflags, context*);	Unmount fs at *mp* with *mntflags* (e.g. `MNT_FORCE`)
vfs_root(*mp, out vpp, context*);	Return root vnode *vpp* of fs mounted on *mp*
vfs_quotactl(*mp, cmds, uid, arg, context*);	Perform `quotactl(2)` *cmds* with *arg* for *uid*
vfs_getattr(*mp, out attr, context*);	Get VFS attributes *attr* of fs mounted on *mp*
vfs_sync(*mp, waitfor, context*);	Sync fs cache with device, optionally waiting.
vfs_vget(*mp, ino, out vpp, context*);	Get the vnode pointer (vpp) by inode number (ino)
vfs_fhtovp(*mp, fhlen, fhp, out vpp, context*);	Convert NFS file handle *fhp* to vnode *vpp*
vfs_vptofh(*in vp, out fhlen, out fhp, context*);	Convert vnode *vp* to file handle *fhp*
vfs_init(*vfsconf*);	Prepare filesystem for having instances mounted.
vfs_sysctl(*mib, mibLen, oldp, in/out oldlenp, new, newlen, context*);	Perform vfs `sysctl(3)` on filesystem
vfs_setattr(*mp, in attr, context*);	Set attributes *attr* of filesystem mounted on *mp*
vfs_ioctl(*mp, cmd, data, flags, context*);	Perform `ioctl(2)` *cmd* on fs mounted on *mp*
Darwin 16	
vfs_vget_snapdir(*mp, out vpp, context*);	

Examples of using this KPI can be found in the open source FUSE (discussed later), or by disassembling Apple's own filesystem kexts.

Vnode operations

The `vfe_opvdescs` field of the `vfs_fsentry` defines the operations which populate the `v_ops` vector of every vnode in the registered filesystem, unless otherwise stated (through a quasi filesystem). The operations are defined as an array of `vnodeopv_entry_desc` (defined in bsd/sys/vnode.h) structures, each with two fields - a pointer to the `vnodeop_desc` and another to the function implementing the operation. The structure is shown in Listing 7-29 (next page).

The `vnodeop_desc` structures are kept opaque (in osfmk/vnode/vnode_internal.h), but there is no need to expose them. There is a limited set of operations, and XNU exports pre-initialized structures corresponding to each of them. Filesystems can thus prepare their implementations, link to the corresponding descriptors, and pass the structure to be registered. The `..._desc` structure is commonly found in `__DATA.__data`, and is easily recognizeable thanks to its `vdesc_name` field, which discloses the operation. Working back from the structure to the containing kext (or, in 1469, the kernel's `__DATA_CONST.__const`) can help symbolicate the operations structure provided by individual filesystems.

Listing 7-29: The VFS operation entry and descriptor structures, from XNU 4903's bsd/sys/vnode.h

```
struct vnodeopv_entry_desc {
        struct vnodeop_desc *opve_op;   /* which operation this is */
        int (*opve_impl)(void *);       /* code implementing this operation */
};

struct vnodeopv_desc {

  int (***opv_desc_vector_p)(void *);   /* ptr to the ptr to the vector where op should go
  struct vnodeopv_entry_desc *opv_desc_ops; /* null terminated list */
};

struct vnodeop_desc {
    int     vdesc_offset;           /* offset in vector--first for speed */
    const char *vdesc_name;         /* a readable name for debugging */
    int     vdesc_flags;            /* VDESC_* flags */

  /*
    * These ops are used by bypass routines to map and locate arguments.
    * Creds and procs are not needed in bypass routines, but sometimes
    * they are useful to (for example) transport layers.
    * Nameidata is useful because it has a cred in it.
    */
    int     *vdesc_vp_offsets;          /* list ended by VDESC_NO_OFFSET */
    int     vdesc_vpp_offset;           /* return vpp location */
    int     vdesc_cred_offset;          /* cred location, if any */
    int     vdesc_proc_offset;          /* proc location, if any */
    int     vdesc_componentname_offset; /* if any */
    int     vdesc_context_offset;       /* context location, if any */
     /*
    * Finally, we've got a list of private data (about each operation)
    * for each transport layer.  (Support to manage this list is not
    * yet part of BSD.)
    */
   caddr_t *vdesc_transports;
};
```

Once the filesystem is registered, execution moves to a callback model, through `VNOP_*` wrappers over common vnode operations. VFS fulfills its role as an adapter layer, performing common logic for the defined operations before dispatching them to the filesystem-specific implementations, found in the vnode's `v_op` member. Most wrappers are similar, loading an operation-specific argument structure passing it to the operation pointer (provided by the filesystem). The `VNOP_READ` wrapper serves as a typical example:

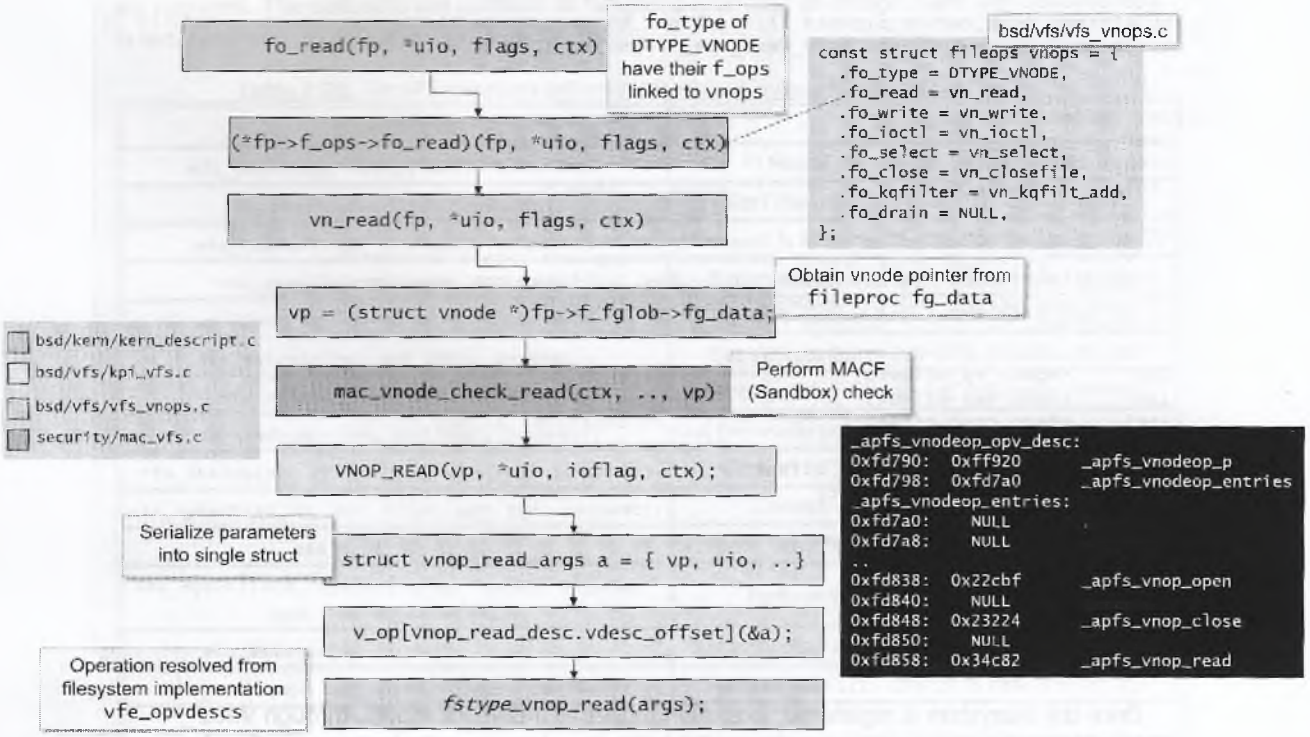
Listing 7-30: Example of a `VNOP` wrapper in `VNOP_READ` (from bsd/vfs/kpi_vfs.c)

```
errno_t
VNOP_READ(vnode_t vp, struct uio * uio, int ioflag, vfs_context_t ctx)
{
        int _err;
        struct vnop_read_args a;
#if CONFIG_DTRACE
        user_ssize_t resid = uio_resid(uio);
#endif
        if (ctx == NULL) { return EINVAL; }

        a.a_desc = &vnop_read_desc;
        a.a_vp = vp;
        a.a_uio = uio;
        a.a_ioflag = ioflag;
        a.a_context = ctx;

        _err = (*vp->v_op[vnop_read_desc.vdesc_offset])(&a);
        DTRACE_FSINFO_IO(read,
            vnode_t, vp, user_ssize_t, (resid - uio_resid(uio)));

        return (_err);
}
```

Case Studies

Putting together all we've seen so far we end up at the flow presented in Figure 7-31, which connects with Figure 6-23:

Figure 7-31: The flow of `fo_read()`



A good way of gaining familiarity with VFS APIs and KPIs is to look at them in context - by examining the implementations of some of the file systems used in XNU. The three case studies picked are quite different - devfs, MacOS's NFS support and FUSE, but they are thankfully all open source, and through them some common implementation patterns can be observed.

/dev (devfs)

For devices to be usable by user mode callers, they must have some filesystem representation, in the form of **device nodes** (which appear in `ls -l` as 'b'(lock) or 'c'haracter. Device nodes traditionally had to be created (by the `mknod(2)` system call) or removed manually following the driver addition or removal - a cumbersome requirement which could lead to unnecessary complications. Modern day UN*X systems (notably, Linux/Android) solved this by installing a user mode daemon to automatically maintain the nodes. Darwin and FreeBSD, however, adopt a different approach.

The /dev directory is itself a a mount point, for the `devfs` special filesystem. This is a virtual filesystem (somewhat like Linux's /proc), where nodes can be created directly from kernel code. Only node pathnames can be created this way, but this proves sufficient. Kernel code can call on `devfs_make_node()` (from bsd/miscfs/devfs/devfs_tree.c) to create the node, and obtain an opaque handle as it magically appears in /dev. The handle can be used with `devfs_remove()` (ibid.) to just as magically make it disappear. Once added, the device ready for use: User mode operations will be redirected by the VFS layer to the implementing callback. Both operations take the `devfs_mutex` (bsd/miscfs/devfs/devfs_tree.c), through the `DEVFS_[UN]LOCK` macros (`#defined` in bsd/miscfs/devfs/devfsdefs.h)

Darwin's `devfs` implementation closely resembles that of BSD's, with the original author comments and a few Apple modifications. Device nodes are created in the `M_DEVFSNODE` BSD Zone. The node names are allocated from `M_DEVFSNAME`. The device nodes are maintained as `struct devnodes`, with their `dn_typeinfo` (a `devnode_type` union) holding either their `dev_t`, directory entry, or symbolic link name. The root node is `dev_root`, a `devdirent_t`, from which all files are linked.

The [b|c]devsw entries

Creating a device in kernel requires initializing an appropriate structure - a `bdevsw` or `cdevsw`, both defined in bsd/sys/conf.h - optionally setting the `d_type` (`D_TTY`, `D_DISK`, or the nostalgic `D_TAPE`), and specifying callback functions corresponding to the allowed operations on the device (shown in Table-xxbcdevsw, next page). The structure can then be registered with the corresponding `[b/c]devsw_add()` (from bsd/kern/bsd_stubs.c), which adds it at its major *index* entry in the global `[b/c]devsws` array. Should the device ever need to be removed, a call to `[b/c]devsw_remove` with the major and structure will do the trick.

Table 7-32: The callbacks of the `bdevsw` and `cdevsw` structures

Operation	Block	Char
`int` **`open`**`(dev_t dev, int flags, int devtype, proc_t p)`	Yes	Yes
`int` **`close`**`(dev_t dev, int flags, int devtype, struct proc *p)`	Yes	Yes
`void` **`strategy`**`(struct buf *bp)`	Yes	Yes
`int` **`ioctl`**`(dev_t dev, u_long cmd, caddr_t data, int fflag, proc_t p);`	Yes	Yes
`int` **`read`**`(dev_t dev, struct uio *uio, int ioflag);`	No	Yes
`int` **`write`**`(dev_t dev, struct uio *uio, int ioflag);`	No	Yes
`int` **`stop`**`(struct tty *tp, int rw);`	No	Yes
`int` **`reset`**`(int uban)`	No	Yes
`int` **`select`**`(dev_t dev, int which, void * wql, struct proc *p);`	No	Yes
`int` **`mmap`**`(void)`	No	Yes
`int` **`dump`**`(void)`	Yes	No
`int` **`psize`**`(dev_t dev)`	Yes	No

Block devices are commonly created in conjunction with more complicated, IOKit-enabled logic. In these cases, the `IOMediaBSDClient` IOKit class (discussed in Chapter 13) can be used to handle the block device creation automatically, without the need to call the `bdevsw*` functions at all (or the devfs registration, as discussed next). Similar IOKit handling can be found in IOKit's `IOSerialBSDClient` which handles character devices for serial port devices, but in most cases creating a character device is best done manually.

It is possible to manifest a single hardware device as both block and character. This is, in fact, quite common, with disk devices, whose block representation is used for mounting filesystems, and the character representation as a "raw" device, for purposes of `fsck(8)` and the like. Calling `cdevsw_add_with_bdev()` will use the same major index for both node types (as in the case, for example, with /dev/[r]disk* nodes).

> ⚠ Raw access to block devices entirely bypasses the filesystem, and thus any file permissions, or extended attributes and flags like those used in SIP are rendered irrelevant. Apple thus enforces the `com.apple.rootless.restricted-block-devices` (MacOS) and `com.apple.private.security.disk-device-access` (*OS) master entitlements, which are bestowed upon the OS's own low-level tools (notably, the `fsck*` family). On a jailbroken *OS device the entitlement can easily be faked, but in MacOS bypassing it requires disabling SIP.

specfs nodes

Device nodes are still represented as vnodes, but with a `v_type` of `VBLK` or `VCHR`. In addition, when the vnode is created (by `devfs`, `mknod(2)`, `vnode_create_internal()`, or otherwise), its `vnfs_vops` are set to `[devfs_]spec_vnodeop_p`. This puts such nodes, sooner or later, within the realm of the **`specfs`** filesystem.

The `spec_vnodeop_p` (in bsd/miscfs/specfs/spec_vnops.c) use the vnode's `v_rdev` to obtain the major, which gives them an index into the `bdevsw` or `cdevsw` arrays. What happens next can be generalized into three cases:

- When an implementation exists for the operation in both character or block device switches, (`open`, `close` and `ioctl`) it is called upon, in order to perform the operation in a manner determined by the driver. There may still be some specific device specific tweaks or hacks - for example, preventing opening of mounted block devices, or handling the closing of a controlling tty.
- When dealing with read or write operations, can directly invoke the callbacks for a character device driver. For block devices, however, these callbacks do not exist, and thus one of the `buf_bread[n]` or `buf_b[/a/d]write` are used.
- Other callbacks in Table 7-32 not called from `specfs` either have different code paths to call them, or were initially put due to compatibility with BSD, but were quickly phased out or left unsupported.

The `fdesc` quasi-filesystem

Hidden in /dev is the rather peculiar /dev/fd quasi-filesystem, called `fdesc`. First - unlike other filesystems, it is not an actual mounted filesystem (though it used to be in older versions of MacOS). Second, the filesystem appears different to each process which uses it. Every process sees in `fdesc` numbered entries, corresponding to its open file descriptors*. A good way to see that is to list the directory with two different process - one, such as `ls(1)`, and the other one a shell (through autocomplete functionality in /dev/fd. `fdesc` also creates symbolic links to descriptors 0, 1 and 2 from /dev/stdin, stdout and stderr (respectively).

`fdesc`'s implementation is contained in bsd/miscfs/devfs/devfs_fdesc_support.c and bsd/miscfs/fdesc.h, requiring `CONFIG_FDESC`, which is set in MacOS. There are two sets of operations, beginning with those of at the directory entry level, implemented using the callbacks in `devfs_devfd_vnodeop_entries` (bsd/miscfs/devfs/devfs_vnops.c). The key operations are:

- `devfs_devfd_readdir()`: called from `VNOP_READDIR()` when the user requests a directory listing, through `getdirentries[64]`. The callback obtains its position in the directory listing, dividing the `uio_offset` by `UIO_MX` (16), the record size. It then checks if that position is a valid file descriptor in the `current_proc()`'s space - i.e. non NULL, and not flagged by `UF_RESERVED`, using the `fdfile` and `fdflags` macros.

  Valid descriptor indices result in a creation of a shortened `dirent` record of `UIO_MX` bytes, in which the descriptor `sprintf()`ed into the `d_name`. This continues for as long as the `uio` has room (i.e. `uio_resid(uio) ≥ UIO_MX`), and the index has not exceeded the number of files in the caller.
- `devfs_devfd_lookup()`, which obtains the calling process from the VFS context pointer, and then checks if the looked up name (actually the descriptor number, in string form) is valid, in the same way `devfs_devfd_readdir()` does. If the name is indeed valid, it calls to `fdesc_allocvp()` to create a vnode for that descriptor on the fly, and returns it in the `lookup`'s `vpp`.

  The created vnode is tagged as `VT_FDESC`, and its `vnode_fsparam` is set such that the `vnfs_vtype` is `VNON`, and the vnode level operations are `fdesc_vnodeop_p`. The `vnode_fsparam`'s `vnfs_fsnode` (which ends up in `v_data`) points to a `struct fdescnode` (from bsd/miscfs/devfs/fdesc.h), which holds the descriptor number in fd_fd.

Moving to the vnode level operations (in `devfs_fdesc_vnodeop_entries`, from bsd/miscfs/devfs/devfs_fdesc_support.c), we see that the only operations actually supported are:

- `fdesc_[get/set]attr`: accesses the vnode's `v_data`, where it finds the `fdescnode` structure, from which it retrieve the descriptor, and uses `fp_lookup()` to obtain it.
- `fdesc_open()`: implemented in an admitted "XXX Kludge", storing the descriptor number in the `uthread`'s `uu_dupfd`, and deliberately returning `ENODEV`. This forces a release of the vnode by `vn_open_auth()`, and code back in `open1()` calls `dupfdopen()` (from bsd/kern/kern_descrip.c) on the descriptor number. The actual vnode opened is thus the real vnode pointed to by the descriptor, which explains why all the other operations return `ENOTSUP`.

* - Linux's /dev/fd is a symbolic link to /proc/self/fd, wherein pseudofiles are managed by the proc filesystem.

Experiment: Creating a simple character device

As we have seen, character devices make up a large part of the nodes in devfs. It is common practice to implement anything outside of mass storage devices as character devices - and therefore useful to be able to build the a simple character device driver from scratch. Such a driver can then be used as a template for more complex devices, real or virtual, which communicate via the POSIX model.

Using an empty kernel extension as a starting point, we can put in the code to create the device node. First, we need to populate a `struct cdevsw` with callbacks. These can initially all be NULL, but better practice is to link them to `enodev` (from bsd/kern/subr_xxx.c), which returns `-ENODEV` to user mode. In the entry point, we can then create the device with `cdevsw_add`. Unless there is a penchant for a specific major, `-1` specifies that the caller is requesting dynamic allocation of a major index for the added device. If successfully added, the return code will indicate the major assigned. The devices managed then need to be published to user mode, using `devfs_make_node`. This is shown in Listing 7-33:

Listing 7-33: Creating a new character device node in /dev

```
struct cdevsw  devsw = { 0 };

int major = cdevsw_add(-1, &devsw);
if (major == -1) { /* fail */ }

void *devNode = devfs_make_node(makedev(major, minor),
                                DEVFS_CHAR,
                                MY_DEV_USER,  // e.g. UID_ROOT
                                MY_DEV_GROUP, // e.g. GID_WHEEL
                                MY_REV_PERMS, // e.g. 0644
                                MY_DEV_NAME); // e.g. "test"

if (!devNode) { /* fail, cdevsw_remove() */ }
```

At this point, building and `kextload(8)`ing your module should make a new device node appear in /dev, thanks to the magic of `devfs`. Trying any operation on the node will result in an error message, because no callbacks have been implemented.

The next step is to implement a few callbacks. To make the device "functional", the implemented callbacks usually include `read` and `write`. Keeping the example simple, we can have our device act as a clipboard of sorts, holding data provided by the user using `write(2)`, and supplying it back to the user through `read(2)`. A partial implementation of a read function is shown below (the write function can be implemented similarly):

Listing 7-34: A sample reader function for a memory buffer backed character device

```
char buf[BUFSIZE];
int writePos;

int my_read(dev_t Dev, struct uio *Uio, int IoFlag) {
   int error = 0;

// TODO: SAMPLE ONLY! Don't forget sanity/bounds checks on kernel memory here..
// read() only uses one iovec in the uio, but good code should handle multiple
// When moving to/from a single buffer, max copy size can be set to uio_resid()
// but scatter/gather needs to consider multiple iovec sizes..

    int available = MIN(BUFSIZE - Uio->offset, uio_resid());

    error = uiomove64(buf + Uio->uio_offset, available, Uio);

    return error;
}
```

As optional enhancements, the ardent reader is encouraged to implement `open(2)` (with access control based on the process credentials or entitlement) and `ioctl(2)` with some proprietary code (e.g. to clear the kernel buffer).

NFS (MacOS)

Most UN*X flavors have adopted the Network File System (NFS) standard to provide file sharing services. MacOS does so as well, supporting both NFSv3 (RFC1813) and NFSv4 (RFC3530).

NFS is a legacy mechanism, and is best discussed elsewhere (in the RFCs specified or a good reference like Callaghan's excellent reference[5] or the BSD Implementation[6]. The aim of this section is to detail the Darwin implementation specifics, and not get bogged down with the protocol or component explanation.

The user mode portions of NFS are handled in MacOS similar to other operating systems, by several daemons:

- **/sbin/nfsd**: providing support for remote client requests using the NFS and/or mount protocols (formerly provided by the now obsolete `mountd(8)`). This LaunchDaemon starts from com.apple.nfsd.plist, contingent on the presence of /etc/exports (which contains the list of filesystems to export).
- **/sbin/rpc.statd**: providing the host status service, as a way for local daemons to probe their remote counterparts.
- **/sbin/rpc.lockd**: providing the locking service, which is required when a remote client requests a local file lock.
- **/usr/libexec/automountd**: managing the `autofs` mechanism, which transparently mounts remote filesystems when access to them is attempted. This LaunchDaemon starts from com.apple.automountd.plist, and claims Host Special Port #11.
- **/sbin/nfsiod**: sets the maximum number of asynchronous I/O threads. This is a deprecated daemon, because control of the number of threads can be done by merely setting the `vfs.generic.nfs.client.nfsiod_thread_max` `sysctl(2)` value - which is exactly what this binary does, and exits.

Darwin's NFS support is contignent on `#defineing` `NFSSERVER` and `NFSCLIENT`, which is done on MacOS, but not the *OS variants. The flags enable the inclusion of file contents from bsd/nfs/, which provides several system calls as well as the kernel implementation of the NFS server logic, and NFS client VFS layer code.

NFS server operations

The `NFSSERVER` `#define` enables several system calls:

- **`nfssvc` (#155)**: This is a "pseudo system call", in that most of the NFS service handling is done in kernel mode, and so this system call is not expected to return. The nfsd(8) merely provides a user-mode process shell, spawning any number of server threads, all of which invoke this call with the `NFSSVC_NFSD` argument, and remain in it until the daemon exits or is killed. Another use of the system call is with the `NFSSVC_ADDSOCK` argument, which registers the server sockets with the kernel. Lastly, the `NFSSVC_EXPORT` flag, to maintain the server's map of exported filesystems.
- **`getfh` (#161)**: Enabling the translation of any pathname to an NFS handle. Fortunately, only on filesystems which are exported.
- **`fhopen` (#248)**: Enabling the translation of an NFS file handle to an open file descriptor with the O_ flags from fcntl.h. This is required by /sbin/rpc.lockd, so as to enable locking when handling NFS requests.

NFS Client operations

NFS Client services are started automatically when the `mount(8)` command is given a mount point and remote file system specified with `-t nfs`. This, in turn, calls `mount_nfs(8)`, and mounts a remote server specification of a filesystem on a local directory mount point.

As a filesystem, NFS provides operations for NFSv2 and NFSv4 implementations (`nfsv[24]_vnodeop_entries`, in bsd/nfs/nfs_vnops.c), which vary somewhat with the protocol version. NFS also provides operations for the `spec_..` and `fifo_..` cases. The vnode I/O is performed by the NFS BIO layer, which manages the data buffers sent from and to the remote server. This is integrated with the local UBC.

The `NFSCLIENT #define` also enables the `nfsclnt` system call (#247). This call, used by rpc.lockd(8), supports a flag, which may be `NFSCLNT_LOCKDNOTIFY` or `..LOCKDANS` (for rpc.lockd(8) notification or answers) and the `NFSCLNT_TESTIDMAP`, used by nfs4mapid(8).

The `nfsstat(1)` utility can be used to display client and server statistics, by polling various `sysctl(8)` MIBs in the `vfs.nfs` namespace. The utility has also been spotted in iOS 13 beta 2, indicating that Apple could be testing NFS client functionality in *OS internal builds.

Filesystems in USEr mode (FUSE)

The **F**ilesystems in **USE**r mode architecture challenges the tradtional implementation of filesystems as kernel drivers. Rather than implement the complex filesystem logic in kernel, FUSE deploys only a lightweight kernel extension, which serves as a proxy for VFS callbacks. The actual work behind them, however, can be carried out by a user-mode process (commonly, a daemon).

The mechanism behind the kernel to daemon interaction is a reverse system call. In this implementation, the user mode daemon performs a system call (commonly, `read(2)`) on a device node supplied by the kernel-level VFS driver code. The system call is left to block until the kernel-level code requires some service from the daemon. It encodes the request in the "read" data, which is then processed by the daemon and acted upon. The daemon can then `write(2)` the reply back to the device node, supplying it back to the VFS driver.

FUSE is by no means unique to Darwin systems. It was started in other UNIX flavors, and is in fact not officially supported - the Darwin implementation was introduced by Amit Singh (author of the seminal precursor to this work) called MacFUSE[7]. The project was later picked up by the open source community, and the present implementation - OSXFUSE[8] - is maintained to this day. Because FUSE does require a kernel component, it is not applicable in the *OS variants, wherein Apple uses DMG mounts (by registering loop block devices) instead.

Apple uses their own version of filesystems in user mode, in the private UserFS.framework, as of iOS 11. The project is naturally closed source and does not share any design ideas with FUSE - It does not rely on a character device, nor does it implement the reverse syscall mechanism. The private framework uses XPC to communicate with its `userfsd` daemon and `userfs_helper`, over `com.apple.filesystems.[userfs[d/fs_helper]` ports. The master daemon is entitled for raw device access, and loads filesystem support rom the framework's PlugIns/ directory (though these are prelinked into the shared cache). Present plugins are msdos.dylib and exfat.dylib, obviating the need of the corresponding kernel extensions, which were indeed removed from *OS kernelcaches. To support iOS 13's "liveFS" feature, additional livefile_*xxx*.dylib plugins were introduced, for APFS, exfat, msdos and HFS.

Review Questions

1. Look through the manual pages of BSD's `vnode(9)`, `vget(9)` and `vput(9)`, comparing these with Darwin's implementation.
2. Why are filesystems in user mode a good idea? What would the disadvantage be?
3. Why is Apple using their home grown implementation, rather than something like FUSE?

References

1. Silvers - "UBC: An Efficient Unified I/O and Memory Caching Subsystem for NetBSD" https://www.usenix.org/legacy/publications/library/proceedings/usenix2000/freenix/full_papers/silvers/silvers_html/
2. Apple Open Source - autofs project - http://opensource.apple.com/tarballs/autofs
3. The iPhone Wiki - "HFS Legacy Volume Name Exploit" - https://www.theiphonewiki.com/wiki/HFS_Legacy_Volume_Name_Stack_Buffer_Overflow
4. Apple Developer - File Provider Documentation - https://developer.apple.com/documentation/fileprovider
5. McKusick, Neville-Neil & Watson - "The Design and Implementation of the FreeBSD Operating System" (2nd Edition) - ISBN -978-0321968975
6. MacFUSE project page on Google Code: http://code.google.com/p/macfuse/
7. OSXFUSE project page on github:http://osxfuse.github.com/
8. https://www.amazon.com/NFS-Illustrated-Brent-Callaghan/dp/0201325705

8

Space Oddity: APFS

Apple first introduced its newest filesystem, APFS, as a special preview in MacOS 12, announcing plans to finally retire the venerable (18+ years old) HFS+. Though still not a full fledged and bootable filesystem, APFS showed great promise by providing 64-bit compatibility and plenty of new features.

It was only almost a year later, however, that APFS was deemed stable enough to be used as a default filesystem. Over this time, Apple kept working and reworking the filesystem internals, breaking compatibility with previous implementations. The filesystem finally stabilized with the first out-of-box implementation in iOS 10.3, probably chosen due to the relative safety of *OS, wherein users are not given free rein to the filesystem. It was then enabled in MacOS 10.13, and has pushed HFS+ to the sidelines.

Although Apple promised the specification of APFS was to be available "by the end of the year" (2016), it failed to deliver it, providing a paltry and partial placeholder document extolling APFS's features, but disclosing virtually no detail on the implementation. In the meantime, it took extensive reverse engineering to figure out how the filesystem really worked. Preliminary analysis by Jonas Plum[1] provided detail on the data structures. This was followed by extensive research detailing APFS internals, performed by Hansen et al. In a detailed article[2], they provide a forensic view of the data structures used, which proved invaluable for future work, including the author's implementation of his filesystem tool.

Finally, two and half years after its initial release and coinciding with that of Darwin 18, the APFS specification showed up with no announcement on developer.apple.com[3]. The document is fairly detailed in documenting the data structures and constants, but seems at times to be minimalistic and created automatically from the source code comments of the header files - certainly not on par with the HFS+ specification of TN1150. This chapter, along with the reference provided by Apple, should hopefully provide a clear view of APFS' intricate structures and logic.

This book is filled with hands-on experiments, but this chapter, in particular, is where the reader is encouraged to follow along with each and every one. Filesystem implementations make very specific use of very particular data structures - and the best way to understand them is through careful step-by-step tracing of filesystem operations, and dumping raw blocks. The `fsleuth` tool, which is freely available from the book's website, was especially designed with verbose debugging output to allow the avid APFS (and HFS+) enthusiast to inspect the filesystem internals.

A Bird's Eye View of APFS

APFS is a relatively neatly designed filesystem, but before we get bogged down in detail it's wise to consider the high level view of APFS. Figure 8-1 (next page) depicts such a view:

Partitions are defined in the GUID Partition Table (GPT), which is at the second block of the disk (and another backup stored towards the end of the disk). The APFS partition type is identified by a well-known GUID. In MacOS, another well-known GUID is used for APFS recovery volumes.

An APFS partition consists of a single **container**, which provides the superblock functionality and metadata for the entire partitioned space. The container manages an Object Map (**omap**), which is a B-tree used to manage various object types - the most important of which is the **volume**. Within the confines of the container can be up to 100 volumes, where every such volume is a mountable filesystem, which can be mounted independently of its siblings. All volumes, however, share the same space of the container with each other, and therefore the total size of all volumes cannot exceed that of the container.

Being a filesystem, each volume usually maintains its own object map (though in some cases may use that of its container) which is again a B-Tree. Two specific objects make up the filesystem itself:

- **The RootFS Tree:** is a B-Tree wherein file metadata is maintained. This includes the file's inode attributes (`stat(1)`, and the like), extended attributes (`xattr(1)`), and extent records.
- **The Extent Tree:** maps logical extents to physical blocks, where the file data is actually stored.

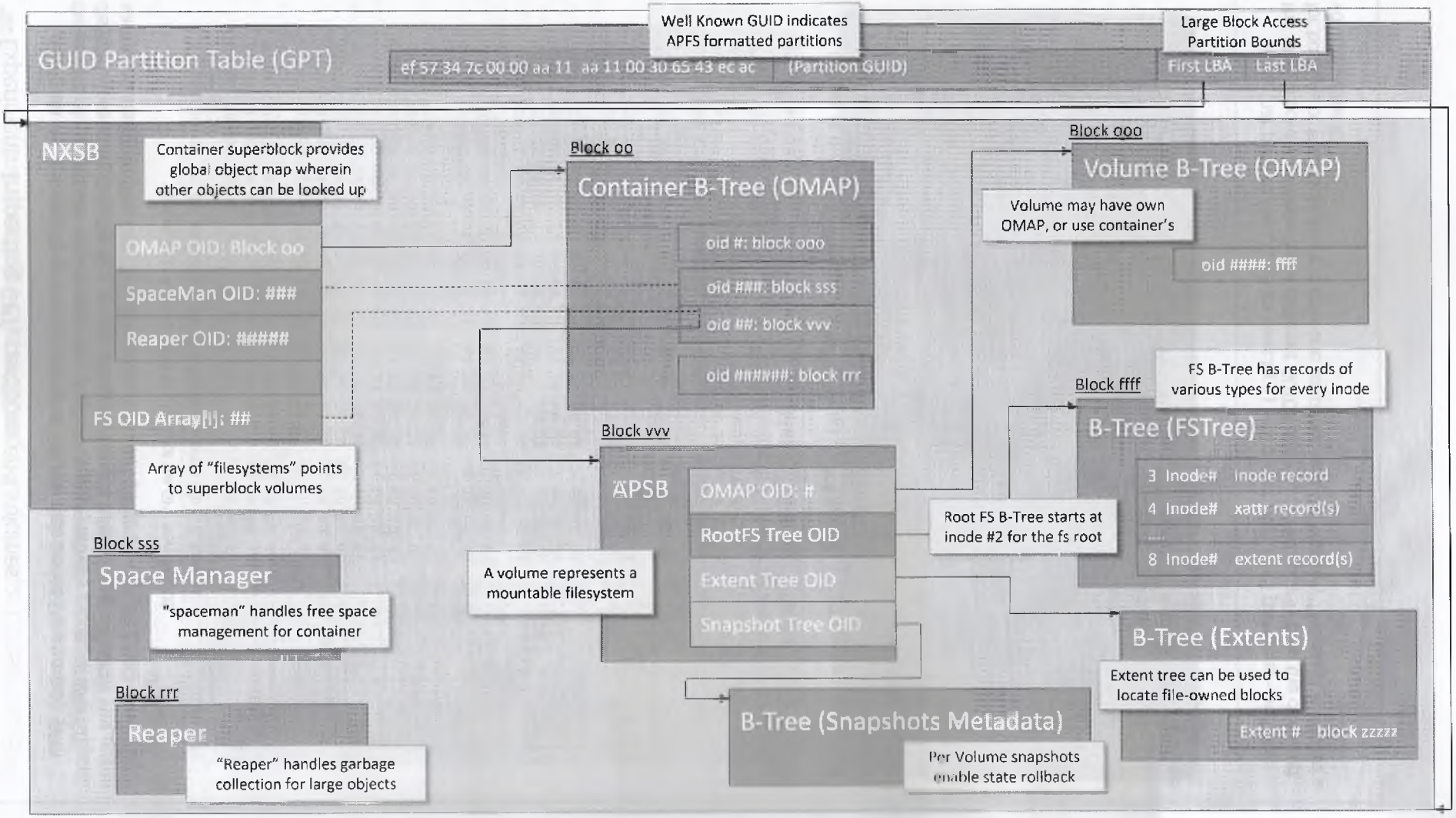
In addition to the volumes and their filesystems, the container needs to maintain state for all of its blocks. This is the role of the **Space Manager** object. The Space Manager maintains a logical bitmap, wherein '0' indicates the corresponding block is free, and '1' indicates it is in use. Although every block is 4K, the number of blocks in a given container can be huge, and so the Space Manager groups contiguous blocks into **chunks**, and makes use of Chunk Info Blocks (CIBs) to maintain the bitmaps at a chunk level, and **CIB Allocation Blocks** (CABs) to group together continguous CIBs.

Our last object in the APFS bestiary is the **Reaper**. Reapers track the state of large objects, so that they can be safely deleted and their space reclaimed. An example of that is on snapshot deletion, which requires destroying all deleted objects whose state was preserved for the snapshot, but is no longer needed if the snapshot is destroyed. The objects to be reaped are maintained in **Reaper List** blocks, which, as their name implies, may span multiple blocks and list entries.

There are additional objects, although less commonly encountered. Fusion drives, which enable containers to span traditional (magnetic platter) hard drives and solid state disks, maintain write back caches and "middle trees" to track hard drive blocks cached on the solid state disks. APFS also contains built-in support for encryption, supporting an intermediate state, as the drive is in the process of being encrypted (when enabling FileVault) , through an "encryption rolling state" object. Finally, in order to provide EFI support in the face of APFS's frequent changes, the "EFI jumpstart" object which is an encapsulated EFI driver.

As we continue our exploration, `fsleuth(j)` will be used to unravel the structure of APFS, one object at a time, in a series of experiments - starting with inpecting the GUID Partition Table itself.

Figure 8-1: A very high level view of APFS



Experiment: Inspecting GPT, partitions and volumes

Using `dd(1)` it is easy to grab a raw dump of the raw disk device (`/dev/rdisk0`). With the advent of SIP this will require you to try the operation in recovery mode, or with SIP disabled. Inspecting only the first couple of blocks through a hex dump, will show you something similar to this:

Figure 8-2: An annotated hexdump of the GPT from MacOS

```
00000  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
*
001b0  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 fe  |................|
001c0  ff ff ee fe ff ff 01 00  00 00 a3 70 3d 3a 00 00  |...........p=:..|
001d0  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
*
001f0  00 00 00 00 00 00 00 00  00 00 00 00 00 00 55 aa  |..............U.|
00200  45 46 49 20 50 41 52 54  00 00 01 00 5c 00 00 00  |EFI PART....\...|
00210  5e 23 a6 27 00 00 00 00  01 00 00 00 00 00 00 00  |^#.'............|
00220  a3 70 3d 3a 00 00 00 00  22 00 00 00 00 00 00 00  |.p=:....".......|
00230  82 70 3d 3a 00 00 00 00  02 34 84 e2 fd b4 c8 43  |.p=:.....4.....C|
00240  a4 df 3c 80 53 bc f1 00  02 00 00 00 00 00 00 00  |..<.S...........|
00250  80 00 00 00 80 00 00 00  88 6d cd 75 00 00 00 00  |.........m.u....|
00260  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
*
00400  28 73 2a c1 1f f8 d2 11  ba 4b 00 a0 c9 3e c9 3b  |(s*......K...>.;|
00410  da f6 98 ff 49 ec 8d 4e  b3 55 75 11 1c 98 c6 ec  |....I..N.Uu.....|
00420           0x28                    0x64027         |(.......'@......|
00430  00 00 00 00 00 00 00 00  45 00 46 00 49 00 20 00  |........E.F.I. .|
00440  53 00 79 00 73 00 74 00  65 00 6d 00 20 00 50 00  |S.y.s.t.e.m. .P.|
00450  61 00 72 00 74 00 69 00  74 00 69 00 6f 00 6e 00  |a.r.t.i.t.i.o.n.|
00460  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
*
00480  ef 57 34 7c 00 00 aa 11  aa 11 00 30 65 43 ec ac  |.W4|.......0eC..|
00490  b8 63 2d e3 a3 3b 5f 44  83 a0 3d fc 1a 30 b9 19  |.c-..;_D..=..0..|
004a0           0x64028                 0x3a29e87f      |(@........):....|
004b0  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
*
00500  4d 64 61 53 63 61 aa 11  aa 11 00 30 65 43 ec ac  |MdaSca.....0eC..|
00510  b5 c7 3c 3a 70 f6 3e 4a  a7 e2 6e 3d 7b 74 d1 f9  |..<:p.>J..n={t..|
00520        0x3a29e880                 0x3a3d707f       |..):.....p=:....|
00530  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
*       0x28 * 0x200 = 0x5000
05000  eb 58 90 42 53 44 20 20  34 2e 34 00 02 01 20 00  |.X.BSD  4.4... .|
```

Looking at the hexdump can be a bit daunting - but fortunately GPT recognition is built-in to `fsleuth(j)`. Try the tool on the raw disk device will show:

Output 8-2: GPT parsing with `fsleuth(j)`

```
root@Zephyr (~)# fsleuth /dev/rdisk0
...
Autoselected first APFS partition (change with "partition")
...
Autoselected first volume - 'Preboot' (change with "volume")
...
Encrypted Container spanning 465.11 GB (121926923 blocks) with 4/100 volumes
...
FSleuth:Preboot:/)> gpt
GPT Header found at LBA 1 with backup @977105059
   Revision: 0x10000    Size: 0x5c
   Spanning 34-977105026
   UUID: 023484E2-FDB4-C843-A4DF-3C8053BCF100
   # Entries: 128, each 128 bytes, Starting at LBA 2
   Entry 0: Type: EFI System     Name: EFI System Partition GUID: DA..8C6EC @LBA 40-409639
   Entry 1: Type: APFS           Name: (none) GUID: B8632DE3-....30B919 @LBA 409640-975825023
   Entry 2: Type: APFS Recovery Name: (none) GUID: B5C73C...74D1F9 @LBA 975825024-977105023
```

Note, that prior to MacOS 14 `fsleuth(j)` will detect both APFS partitions - and that the APFS recovery partition has a different GUID (`B5C7...-7B74D1F9`) than the one used for boot. As of MacOS 14 there is only one container, and the recovery filesystem is instead a volume within it.

Experiment: Inspecting GPT, partitions and volumes (cont.)

The *OS case is considerably simpler, since there is are no EFI or recovery partitions. There's no SIP, but there is the Platform Profile to get around - for which you will need to entitle `dd(1)` with the `com.apple.private.security.disk-device-access` entitlement. Doing so, and repeating the same command, will yield something quite similar to Listing 8-3:

Listing 8-3: An annotated hexdump of the GPT from iOS

```
00000   00 00 00 00 00 00 00 00   00 00 00 00 00 00 00 00  |................|
*
001b0   00 00 00 00 00 00 00 00   00 00 00 00 00 00 00 fe  |................|
001c0   ff ff ee fe ff ff 01 00   00 00 c9 9a 3b 00 00 00  |............;...|
001d0   00 00 00 00 00 00 00 00   00 00 00 00 00 00 00 00  |................|
*
001f0   00 00 00 00 00 00 00 00   00 00 00 00 00 00 55 aa  |..............U.|
00200   00 00 00 00 00 00 00 00   00 00 00 00 00 00 00 00  |................|
*
01000   45 46 49 20 50 41 52 54   00 00 01 00 5c 00 00 00  |EFI PART....\...|
01010   35 d5 86 19 00 00 00 00   01 00 00 00 00 00 00 00  |5...............|
01020   c9 9a 3b 00 00 00 00 00   06 00 00 00 00 00 00 00  |..;.............|
01030   c4 9a 3b 00 00 00 00 00   ee 82 c8 4a 59 f2 48 47  |..;........JY.HG|
01040   be 0e 90 a0 b1 a7 6f 7c   02 00 00 00 00 00 00 00  |......o|........|
01050   80 00 00 00 80 00 00 00   c8 7b 71 3c 00 00 00 00  |.........{q<....|
01060   00 00 00 00 00 00 00 00   00 00 00 00 00 00 00 00  |................|
*
02000   ef 57 34 7c 00 00 aa 11   aa 11 00 30 65 43 ec ac  |.W4|.......0eC..|
02010   35 89 fb 7a f9 4b 1c 42   9b 15 c4 00 34 2c 73 99  |5..z.K.B....4,s.|
02020            0x6                      0x3b9ac4        |.........;.....|
02030   00 00 00 00 00 00 00 00   43 00 6f 00 6e 00 74 00  |........C.o.n.t.|
02040   61 00 69 00 6e 00 65 00   72 00 00 00 00 00 00 00  |a.i.n.e.r.......|
02050   00 00 00 00 00 00 00 00   00 00 00 00 00 00 00 00  |................|
*        0x6 * 0x1000 = 0x6000
06000   fc d9 eb 95 7c 5c 87 2d   01 00 00 00 00 00 00 00  |....|\.-........|
06010   3c 2e 00 00 00 00 00 00   01 00 00 80 00 00 00 00  |<...............|
06020   4e 58 53 42 00 10 00 00   bf 9a 3b 00 00 00 00 00  |NXSB......;.....|
```

Note, that the blocksize in *OS is set to 0x1000 (4,096) rather than MacOS's 512. Inspecting this through the eyes of `fsleuth(j)` again, we see:

Output 8-4: A hexdump of the GPT from iOS

```
root@hontifex(~)# fsleuth /dev/rdisk0
...
Autoselected APFS partition

Volume
Volume
Autosele ted first volume (chage with "volume")
...
FSleuth:(System:/)> gpt
GPT Header found at LBA 1 with backup @3906249
        Revision: 0x10000       Size: 0x5c
        Spanning 6-3906244
        UUID: EE82CB4A-59F2-4847-BE0E-90A0B1A76F7C
        # Entries: 128, each 128 bytes, Starting at LBA 2
        Entry 0: Type: APFS Name: Container GUID: 3589FB7A-...2C7399 @LBA 6-3906244
```

The APFS partition is made visible as `/dev/disk0s1`. The two volumes in it are assigned `/dev/disk0s1s1` and `/dev/disk0s1s2`. The container and volume structures will naturally be demonstrated in a later experiment.

Filesystem Features

Apple had solid rationale in deciding to abandon HFS+, which has served it well for almost 20 years. The move to a new filesystem is no mere feat, and could only be justified if the new filesystem had plenty of new features and improvements to make it worth while.

Full 64-bit filesystem

Although HFS+ can support file-sizes of up to 2^{63} bytes (i.e. 8EB), the filesystem itself is still 32-bit. APFS, by contrast, allows 2^{63} allocation blocks, and uses 64-bits for the file IDs, as well. This makes sense with enormous filesystems (i.e. those with over 4 billion objects), as the limits now increase into the quintillion (10^{18}), which should hold for the foreseeable future.

The timestamp on files has also been extended to 64 bits, and now counts in nanonseconds, rather than seconds, from the well known epoch (January 1st, 1970). This pushes the dreaded maximum date from January 2038 to the year 2554.

Volume Management

Logical volume management has long been used with filesystem to provide greater flexibility than standard partitioning, by allowing the dynamic expansion, shrinking and striping of partitions to fit storage needs. Traditionally, however, it was always performed by a separate mechanism. Apple introduced CoreStorage for MacOS 10.7, and the Lightweight Volume Manager (LwVM) for *OS variants as of around iOS 6. Neither is necessary anymore as APFS takes volume management into its own hands.

Whereas other filesystems commonly fill up the entire partition space with a single volume, APFS utilizes a different approach. APFS uses a **container** to span the partition, or the entire disk. One or more **volumes** can then be created (using `newfs_apfs -A`) within the container. The free space is managed at the container level, so any volumes therein can grow (claiming space from it) or shrink (releasing space to it). On MacOS, the root filesystem and the /private/var/vm partition (mounted for swapfile usage) are two volumes in the same container, which (as of MacOS 15) also contains the user data. This can be seen from the output of `df(1)`:

Output 8-5: The output from `df(1)` on a standard MacOS system with APFS formatting

```
#
# Note the Size/Avail are identical for volumes of the same container.  #
morpheus@Zephyr (~)$ df -H
Filesystem      Size   Used  Avail Capacity  iused      ifree %iused  Mounted on
/dev/disk1s5    499G    10G    52G    17%  474158 4876602762    0%   /
/dev/disk1s1    499G   435G    52G    90% 1636489 4875440431    0%   /System/Volumes/Data
/dev/disk1s4    499G   1.1G    52G     3%       4 4877076916    0%   /private/var/vm
..
```

Encryption

MacOS was one of the first operating systems to provide full disk encryption, when Apple introduced FileVault in MacOS 10.7. FileVault evolved over the years and was an integral part of CoreStorage. With APFS volume management, however, CoreStorage is no longer necessary, and encryption can now be selectively applied on APFS volumes. The APFS container contains an `nx_keylocker` field (providing the block range where cryptographic information is stored), and every volume contains the `apfs_meta_crypto` field.

Fast Directory Sizing

Summing up directory contents is often a lengthy process, due to the recursive nature of the operation. Depending on depth and count on dentries, it can take many seconds and involve much I/O, as anyone who has used `du(1)` or `Finder`'s "Get Info" on a folder can likely relate.

APFS provides a significant speed up, by storing the directory usage statistics as additional metadata (an `APFS_TYPE_DIR_STATS` record) for the directory object.

Sparse File support

It is not uncommon to find large files containing vast yet empty swaths. Such files would normally claim blocks for their empty space, but not so in APFS: Here, the file metadata specifically marks empty ranges, so that they don't actually take up space. Sparse inodes are marked with a special flag (`INODE_IS_SPARSE`), and contain an extended field indicating the number of sparse bytes. Sparse files are usually created when the creator deliberatly `lseek(2)`s past the end of the file (e.g. `dd if=/dev/zero of=sparsefile bs=1 count=1 seek=0x1000000`), when created through `fcntl(..F_PUNCHHOLE)`, or when a file size is adjusted through `[f]truncate(2)`.

Atomic safe-save

Renaming a file, as for example is done with temporary files, commonly involves several operations at the filesystem level. Darwin 14 adds the `renameatx_np` system call (#488), which provides renaming functionality as an atomic operation, when used with the `RENAME_SWAP` flag (another flag, `RENAME_EXCL`, also allows protection from accidentally renaming to an existing destination).

The `renameatx_np` system call can be called through `rename[at]x_np(2)` wrappers in libSystem, which are documented in the same man page. This functionality may also be provided by other filesystems, which may report this through `getattrlist(2)` for `ATTR_VOL_CAPABILITIES`, with the `VOL_CAP_INT_RENAME_*` flags.

File/Directory Cloning

File **cloning** refers to creating an identical copy of a file. Similar to a hard link, the clone and its origin use the original file blocks. Unlike a hard link, however, when either a clone or its origin are changed, APFS records the changes in additional blocks, which are maintained for the changed instance only.

Since the original blocks at the time of cloning are guaranteed to remain unmodified, the cloning operation is much faster than a normal file copy. Cloning functionality is provided by a proprietary system call, `clonefileat(2)` (#462), whose number was reserved as far back as Darwin 14, but was introduced only in Darwin 16, when the preview version of APFS was made available. Darwin 16 also provided the `fclonefileat(2)` variant (#517), and both system calls are documented in the same manual page. Another way to clone is by using Darwin's `clonefile(2)` (#227) with the `COPYFILE_CLONE` flag, or by specifying a full copy operation with the other `COPYFILE_*` flags (as well documented in the manual page). If the filesystem supports clones, the implementation of `copyfile(2)` chooses to perform cloning, rather than full copying. The higher level `FileManager` Foundation.framework object operates similarly through its `copyItem*` methods.

Although in theory applicable to other filesystems, cloning is presently provided only by APFS. Similar to the atomic safe-save functionality, a caller can determine if cloning is supported by a volume through a call to `getattrlist(2)`, checking the `ATTR_VOL_CAPABILITIES` for `VOL_CAP_INT_CLONE`.

Copy-on-Write

File cloning is merely an application of a more far reaching principle, of **Copy-on-Write**. Rather than edit file data in place, most APFS operations use new blocks only to record changes - both in file content and filesystem metadata. Changes are marked with a **Transaction ID** (referred to as an 'XID').

Copy-on-Write has far reaching implications on the filesystem performance and behavior. Performance-wise, this greatly speeds up I/O, since it's easier to record just the delta of an operation. This also makes APFS a "flash friendly" filesystem (deferring Program/Erase cycles of the same blocks), and provides built-in crash protection, as when a volume is mounted, any inconsistencies can be automatically rolled back to the last coherent XID.

Disk-wise, however, the effect is inconclusive. On the one hand, recording the delta is not only faster, but takes significantly less disk space. On the other hand, over time this leaves a lot of past changes on the disk. Thanks to the XIDs, and the space manager, it's easy to keep track of disk space consumed by previous transactions, and over time APFS reclaims blocks of antiquated XIDs.

There are two other highly important aspects of Copy-on-Write, which Apple doesn't address. APFS is a forensic analyst's dream, as nothing really gets deleted (at least not while there is ample disk space). Forensic tools can perform low-level block I/O to retrieve blocks. Though this can be done with any filesystem, in the case of APFS such tools can easily determine a given block's content by navigating through existing XID data. The other aspect is that APFS can provide undelete functionality, since deleting a file involves a transaction on its directory metadata. Suprisingly, however, Apple chose not to provide an undelete tool, instead offering a different model, of snapshots.

Snapshots

Snapshots are a concept which was popularized by virtual machines. Snapshotting allows to freeze the state of a disk, so that it can be rolled back to at will. This is a highly useful feature, which allows specifying designated "checkpoints", usually after software installation or system upgrades.

APFS Copy-on-Write naturally lends itself to snapshots. An APFS snapshot is, in essence, recording additional metadata pertaining to a specific XID, and providing a human-readable name (in practice, a GUID). The `fs_snapshot` system call (#518) provides this functionality, through libsystem_kernel wrappers documented in `fs_snapshot_create(2)`.

Although snapshots provide a means to undelete files, they do so in a more limited fashion. A snapshot can be mounted, but only as a read-only mount. Thus, "forking" into different configurations from the same snapshot is not supported. A snapshot can be reverted to, but doing so destroys all changes after the snapshot was taken, including any other snapshots taken thereafter. More importantly, unlike XIDs, which are maintained automatically, snapshots must be actively taken.

Starting with iOS 11.3, the *OS variants mount their root filesystem as a read-only snapshot. Although this gave some jailbreakers a headache (as discussed in Volume 3, v1.5 and later), this was likely not done as a security measure, but rather as a way to enhance the resilience of the update process.

Defragmentation

As of Darwin 18, APFS supports defragmentation. In iOS 13 variants, apfs.fs's apfs_defragd is started as a `Maintenance` priority XPC activity every 3 days,

Volume Groups and Firm Links (Darwin 19+)

Darwin 19 offers enhancements to volume mangement through **Volume Groups**, grouping several volumes into a set with its own GUID (usually, that of the leading volume). Volume groups allow cross volume, directory level links, called **firm links**. These are conceptually similar to hard links, but allow linking a directory in one volume to another. The powerful functionality (implemented through a hidden extened attribute, `com.apple.fs.firmlink` and the `SF_FIRMLINK` file flag) is presently not provided for developers, and there is little documentation about it save for WWDC 19 session 710[4].

Purgeable Files (Darwin 19+)

Another enhancement in Darwin 19 is the notion of **purgeable files**. These are files which can be flagged as such during creation, and then automatically collected by the filesystem, rather than having to be explicitly deleted.

File System Internals

APFS was designed to support blocks ranging from 4k to 64k, and opts for the smaller value by default. The minimum container size is 1MB.

Any given block in an APFS filesystem may be one of three classes:

- **Unallocated:** Blocks that haven't been used for any purpose since the initial formatting of the container are in the unallocated state. The container's space manager maintains all these blocks in a bitmap and associated structures, as is discussed later in this chapter.
- **Used by a file object:** Inevitably, most blocks fit into this class, as filesystems naturally serve to store file data. Once such blocks are allocated, they are no longer available for the filesystem, until all generations of the corresponding object has been removed.
- **Used by APFS itself:** Like any filesystem, APFS requires its own private space, in which it can maintain the copious metadata about its inodes. The constant creation of new objects, along with object generations, means that this metadata expands over time.

Generational data - both file objects' as well as APFS's own - is considered available for purposes of `df(1)`-type reporting. Once a system runs out of Unallocated space, however, previous generational data can be scavenged, and its blocks are recycled. This does not apply to generations which have been frozen using snapshots.

APFS Objects

The blocks used by APFS itself contain APFS objects. Every block contains exactly one object. Such objects are identified by a unique 64-bit object identifier (referred to as an **OID**, and stored using one of three methods:

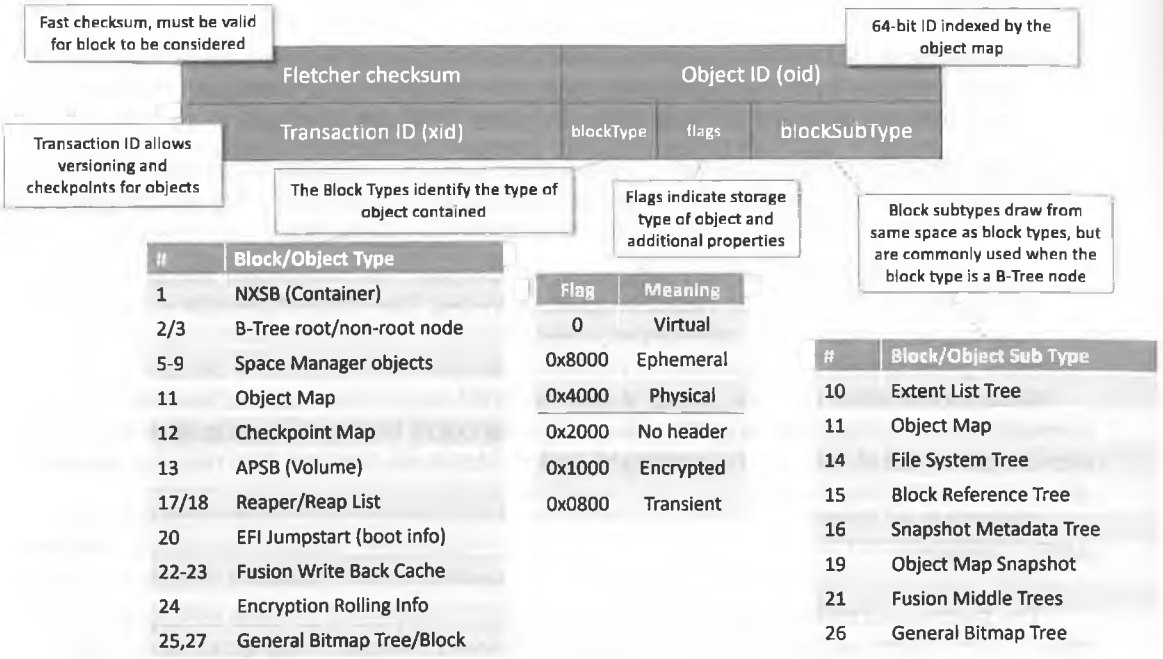
- **Physical:** objects are stored at fixed block addresses, and their oid is the same as their block address. In cases where physical objects are copied, their OID will change accordingly to match their block address. There is therefore exactly one instance of a given physical object.
- **Virtual:** objects may exist in multiple instances, corresponding to different versions or **generations** of the same object. All generations of a virtual object therefore have the same object ID, but each exists in a different block address.
- **Ephemeral:** objects are the same as virtual objects, but are maintained in memory for the duration of the filesystem mount. When unmounted, these objects are committed to storage as part of a checkpoint.

Aside from physical objects, there is no guarantee that the oid matches the block number. By definition, this cannot be the case with virtual objects, wherein the OID and block number will quickly disagree. It follows, therefore, that for virtual objects, there must be a mechanism to map oids to block numbers. Indeed, such a mechanism exists in the Object Map (**omap**). The omap is a B-tree (technically, B+), whose leaf nodes provide the mapping between an oid key and a 64-bit block number. B-trees are discussed later in this chapter.

APFS object structure

Blocks used by APFS always start with a fixed header of 32 bytes, as shown in Figure 8-6:

Figure 8-6: The APFS Block header



#	Block/Object Type
1	NXSB (Container)
2/3	B-Tree root/non-root node
5-9	Space Manager objects
11	Object Map
12	Checkpoint Map
13	APSB (Volume)
17/18	Reaper/Reap List
20	EFI Jumpstart (boot info)
22-23	Fusion Write Back Cache
24	Encryption Rolling Info
25,27	General Bitmap Tree/Block

Flag	Meaning
0	Virtual
0x8000	Ephemeral
0x4000	Physical
0x2000	No header
0x1000	Encrypted
0x0800	Transient

#	Block/Object Sub Type
10	Extent List Tree
11	Object Map
14	File System Tree
15	Block Reference Tree
16	Snapshot Metadata Tree
19	Object Map Snapshot
21	Fusion Middle Trees
26	General Bitmap Tree

The first field in every block header is a Fletcher 64-bit Checksum. This enables quick verification of block contents before any further processing, and also enables quick distinction between the block classes, as the checksum won't match for an arbitrary data block. A notable exception to this are blocks of all zeroes or all ones, but those are not a concern because neither 0 nor 0xffff account for valid block types. The checksum is easy to calculate with the following simple code, although the kernel extension can tap into SSE/AVX instructions for even faster SIMD computation:

Figure 8-7: The Fletcher checksum algorithm

```
pen
uint64_t fletcher64(const uint32_t *data,
                    size_t cnt,
                    uint64_t init) {
        size_t i;

        uint64_t sum1 = init & 0xFFFFFFFFU;
        uint64_t sum2 = (init >> 32);

        for (k = 0; k < cnt; k++) {
                sum1 += data[i];
                sum2 += sum1;
        };

        sum1 = sum1 % 0xFFFFFFFF;
        sum2 = sum2 % 0xFFFFFFFF;
        return ((sum2) << 32) | (sum1);
}
```

The next field is the Object ID (**oid**), which refers to the object contained inside the block. Because of the Copy-on-Write paradigm, an object may exist in multiple versions. The Transaction ID denotes that version. An object is thus uniquely identified by an <oid,xid> tuple, which captures its generation. Following that are the object (=block) type and subtype, though the latter is only used for B-tree nodes, which hold different content according to the tree subtype. In between the type and subtype (technically, in the high-order bits of the type, if considered as a 32-bit identifier) are flags which hold the object storage class, as well as indicate if the object is encrypted, non-persistent, or stored without a header.

Experiment: Using `fsleuth` to display container block maps

Apple's `hdiutil(1)` provides a safe environment to explore APFS intricacies without fear of data loss. The tool can create and attach disk images on the fly - which can then be inspected without an entitlement. A simple way to create an APFS filesystem with this tool is shown in Output 8-8:

Listing 8-8: Creating and/or repartitioning a DMG with APFS

```
morpheus@Zephyr (~)$ hdiutil create -size 10m -type UDIF -attach ~/apfsTest.dmg  -fs apfs
/dev/disk3              GUID_partition_scheme
/dev/disk3s1            Apple_APFS
/dev/disk4              EF57347C-0000-11AA-AA11-0030654
/dev/disk4s1            41504653-0000-11AA-AA11-0030654 /Volumes/untitled
created: /Users/morpheus/apfsTest.dmg
#
# If you change your mind you can always repartition an existing DMG, like so:
#
morpheus@Zephyr (~)$ diskutil partitionDisk disk3 apfs "TEST APFS" 100%
Started partitioning on disk3
Unmounting disk
Creating the partition map
Waiting for partitions to activate
Formatting disk3s1 as APFS with name TEST APFS
Mounting disk
Finished partitioning on disk3
/dev/disk3 (disk image):
   #:                       TYPE NAME                    SIZE       IDENTIFIER
   0:      GUID_partition_scheme                        +10.5 MB    disk3
   1:                Apple_APFS Container disk4          10.4 MB    disk3s1
```

Opening a raw block device would require an entitlement, but since the device is backed by the DMG you can use `fsleuth(j)` directly on the image. The tool will automatically detect the APFS partition, and pseudo-mount the first available volume it sees, with the latest possible checkpoint. On our test DMG, this would look something like this:

Output 8-9: Opening an APFS dmg with `fsleuth(j)`

```
morpheus@Zephyr (~)$ fsleuth ~/apfsTest.dmg
...
GPT detected, blocksize 0x200...
Container (GUID: 7B866D22-...569422FF9B30) spanning 9.96 MB (2550 blocks) with 1/1 volumes
Chose NXSB copy @0x6 with XID 0x3
Volume omap OID: 0x64
Volume 1: (Block 0x62)  (XID: 0x3)
  Label: 'TEST APFS'   Role: none
  with 1 files, 1 dirs, 0 symlinks     Size: 24.0 KB (6 blocks)
Autoselected volume 'TEST APFS' (change with 'volume')
FSleuth(TEST APFS[0x3]:/)> ls
```

Note, that the XID is already 3! This is because after the DMG was attached, the fseventsd created its directory, which involved committing transactions. Using the `blockmap` command will reveal that, although the filesystem itself is nearly empty, there are quite a few metadata blocks in use by APFS, including several XID versions:

Output 8-10: The `fsleuth(j) blockmap` command

```
FSleuth(TEST APFS[0x3]:/)> blockmap
Block 0x0 (OID: 0x1, XID: 0x2) is container
Block 0x1 (OID: 0x1, XID: 0x1) is checkpoint
Block 0x2 (OID: 0x1, XID: 0x1) is container
Block 0x3 (OID: 0x3, XID: 0x2) is checkpoint
Block 0x4 (OID: 0x1, XID: 0x2) is container
Block 0x5 (OID: 0x5, XID: 0x3) is checkpoint
Block 0x6 (OID: 0x1, XID: 0x3) is container
Block 0x9 (OID: 0x400, XID: 0x1) is space manager
Block 0xa (OID: 0x401, XID: 0x1) is reaper
Block 0xb (OID: 0x400, XID: 0x2) is space manager
Block 0xc (OID: 0x401, XID: 0x2) is reaper
Block 0xd (OID: 0x403, XID: 0x2) is B-Tree
Block 0xe (OID: 0x405, XID: 0x2) is B-Tree
..
Block 0x51 (OID: 0x51, XID: 0x3) is CIB
Block 0x53 (OID: 0x53, XID: 0x1) is object map
Block 0x54 (OID: 0x54, XID: 0x1) is B-Tree
..
```

The DMG in this case is intentionally small - but when using `blockmap` on large files iterating over all blocks might take a few minutes or up to a few hours. Results are cached, so subsequent operations will be immediate. `blockmap` can also take as an argument the block type (e.g. '`container`', '`volume`', etc) to display only blocks of that type.

B-Trees

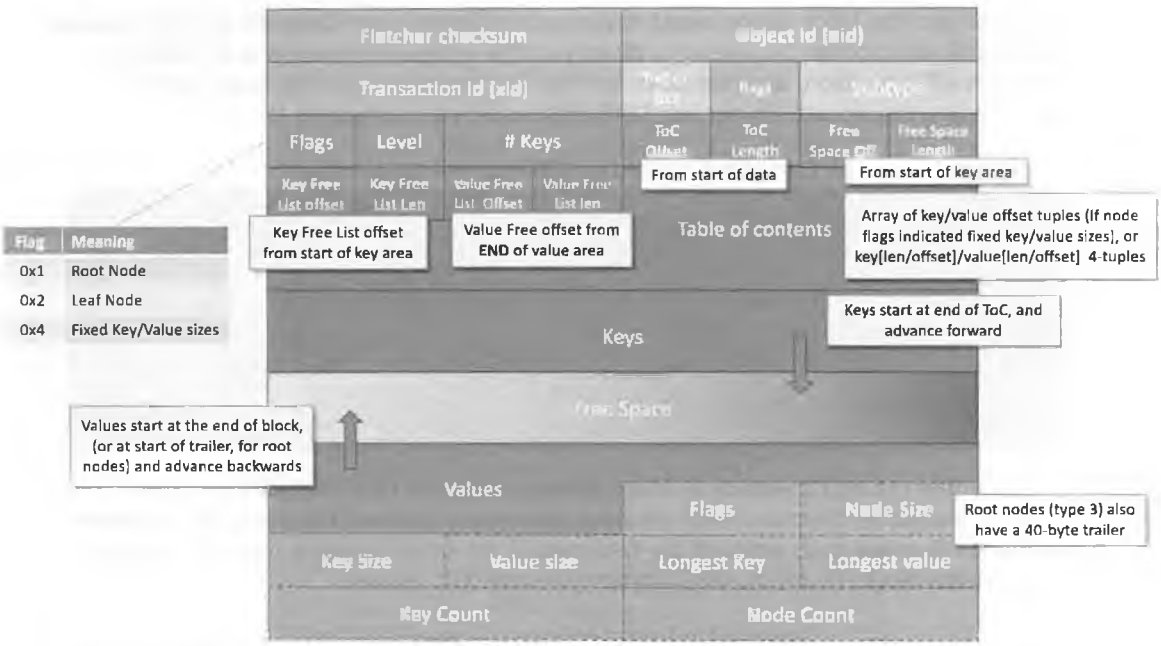
B-trees are important data structures which prove exceptionally useful in filesystems. Unlike binary trees, B-tree nodes can have more than two child nodes, while still maintaning the balanced properties. Increasing the child capacity of nodes delivers an improvement by improving the base of the logarithmic time (i.e. $O(\log_b(n))$ for $b > 2$). B-trees are well documented in the literature, and the interested reader is encouraged to familiarize herself with their insertion, search and deletion algorithms at this point.

The B-tree used by APFS are actually B+ trees - a refinement on classic B-trees, by restricting values to leaf nodes only. Thus, non-leaf nodes (the root and deeper levels) hold only keys and identifiers of child nodes. APFS nodes further have no sibling pointers, which further compacts space needed, but impacts sequential reading of values: When the end of the node is reached, the next value in its sibling record must be located by starting the search at the root.

The B-Tree Node Format

APFS B-tree nodes are block objects of either type 2 (for root nodes) or type 3 (non-root nodes). Because B-trees are used for various data management, the block sub-type field is non-zero, with one of the values from Figure apfs-blkheader indicating the B-tree value data. Figure apfs-genbtreenode builds on the previous figure, to depict the general structure of a node:

Figure apfs-genbtreenode: A visualization of a generalized B-Tree node



Every tree naturally has a single root node, but if this is the only node in the tree then it would also qualify as a leaf node. The combination of `BTNODE_ROOT` (0x1) and `BTNODE_LEAF` (0x2) can therefore identify these tree types. The distinction between type 2 (root) nodes and type 3 (other) nodes is important, because the former have a special trailer, containing tree-wide metadata.

Experiment: Using `fsleuth` to follow B-Tree navigation

Finding anything in an APFS filesystem requires navigating its B-Trees. `fsleuth` routinely does so when looking up OIDs in the container B-Tree, and files in a volume's FSTree.

Using the `debug btree` command will enable you to follow the B-Tree operations. When looking up object IDs, for example, copious output will be displayed to enable you to follow node traversal, key comparison operations, and recursions. Output 8-11 shows an example when looking up an object identifer using the "oid" command. The object, 0x404, is the root volume itself:

Output 8-11: The `fsleuth(j) oid` command

```
FSleuth(TEST APFS[0x4]:/)> oid 0x404
Starting from BTOM Block 0x6a
B-Tree flags: 0x12, node size: 0x1000, keySize: 0x10 valueSize: 0x10. 1 Key, 1 Node
Comparing desired (0x404) vs. key 0 (0x404) and block 0x68...
OID 0x404 found in block 0x68
```

Note the lookup started at block 0x6a, which was recognized to be a single node B-Tree. This is unsurprising, since we are dealing with an empty filesystem - exactly why the search was for 0x404, which exists even when no other objects have been created. We can now look into the block:

Output 8-12: The `fsleuth(j) block` command

```
FSleuth(TEST APFS[0x4]:/)> block 0x6a
Block 0x6a (OID: 0x6a, XID: 0x4) is B-Tree (subtype object map)
Flags: 0x7 (root, leaf, fixed key), Level: 0, with 1 keys. TOC@0x38-0x1f8
Key Free List at 0x10 Value Free List at 0x20
B-Tree flags: 0x12, node size: 0x1000, keySize: 0x10 valueSize: 0x10. 1 Key, 1 Node
Key 0 @01f8: 0x404, XID 0x4       Value @0xfc8: 0x100000000000,0x68
```

We can get right down to the bytes by using the `hexdump` command on the block. You'll want color here, as it makes for a much more readable output, shown here with additional annotations:

Output 8-13: The `fsleuth(j) hexdump` command

```
FSleuth(TEST APFS[0x4]:/)> hexdump 0x6a
                    Checksum                        OID
0x0000:     0x91e3f0141e59d9f4          0x000000000000006a
                     XID                BTree                OID
0x0010:     0x0000000000000004      0x2       0x4000      0xb
                flags    #keys        TOC   TLen   FSO FSL
0x0020:          0x7       0x1         0x00 0x1C0  0x20 0xDA0    ..........  ..
          KFLO  KFLL  VFLO  VFLL
0x0030:   0x10  0x10  0x20  0x10    00 00 10 00 00 00 10 00    ................
...
0x01F0:   00 00 00 00 00 00 00 00   04 04 00 00 00 00 00 00    ................
0x0200:   04 00 00 00 00 00 00 00   FF FF 10 00 00 00 00 00    ................
0x0210:   03 00 00 00 00 00 00 00   00 00 00 00 00 00 00 00    ................
...
0x0FB0:   00 00 00 00 00 00 00 00   FF FF 10 00 00 10 00 00    ................
0x0FC0:   5F 00 00 00 00 00 00 00   00 00 00 00 00 10 00 00    _...............
                                    B-Tree footer
0x0FD0:   68 00 00 00 00 00 00 00   02 00 00 00 00 10 00 00    h...............
0x0FE0:   10 00 00 00 10 00 00 00   10 00 00 00 10 00 00 00    ................
0x0FF0:   01 00 00 00 00 00 00 00   01 00 00 00 00 00 00 00    ................
```

The ToC starts at 0x00 - meaning 0x38, and spans 0x1c0 - i.e. till 0x1f8. This exactly as we see in the dump: Most of the ToC outside the (first eight bytes) is empty - which also makes sense, as we see some keys do start at 0x1f8.

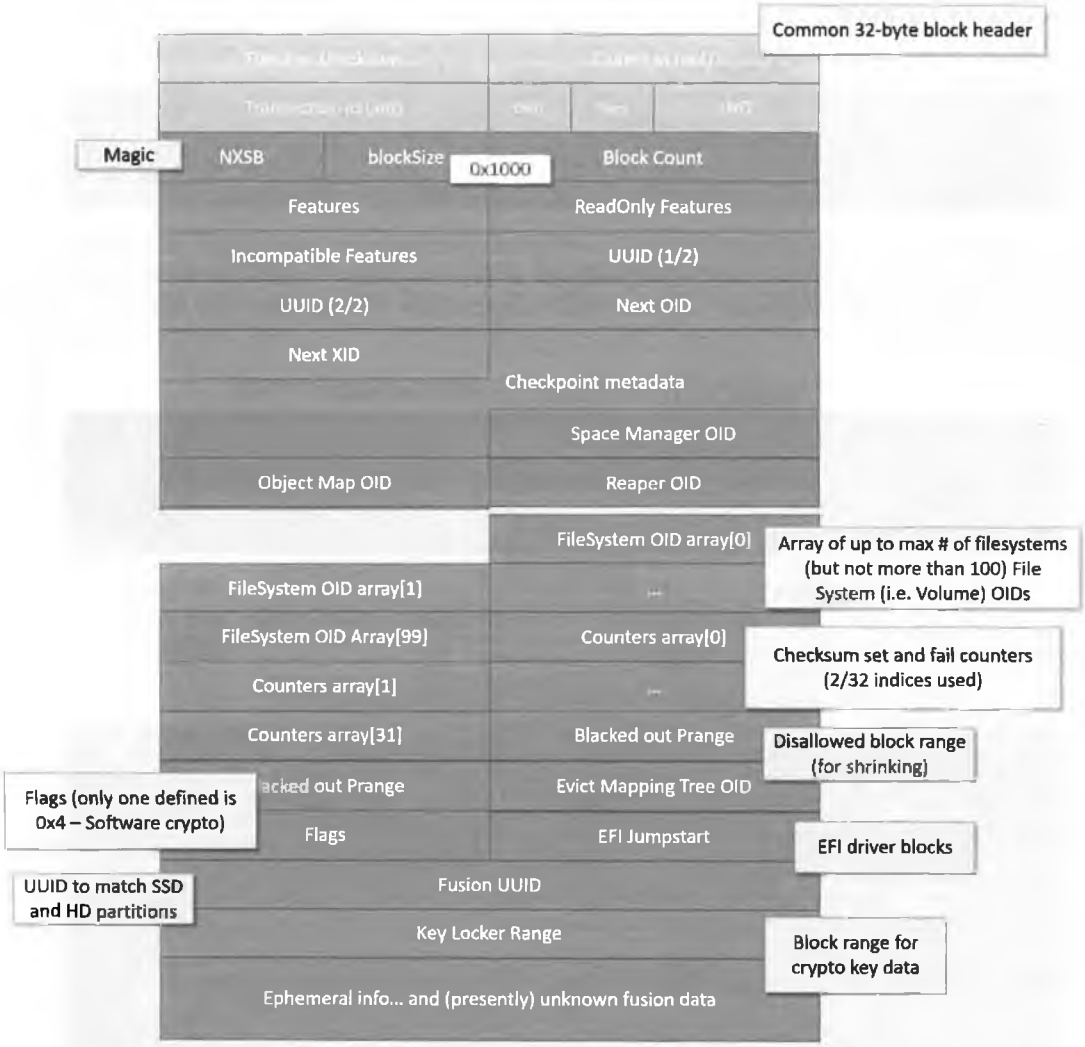
Looking further, we see that the Free space offset is reported at 0x20 from the key area - i.e. from 0x218. The free space is reported as spanning 0xda0, meaning it ranges until 0xfb8. This means the "FF FF 10 .." we see there are values. As this is a B-Tree root node, we have to account for the footer (at 0xfd8), and that indeed is corroborated by subtracting the Value Free List Offset (VFLO), i.e. 0xfd8 - 0x20, bringing us exactly to 0xfb8.

Using this feature on a few OID lookups will hopefully contribute to a better understanding of the B-Tree structure. Bear in mind that, depending on filesystem parameters and other circumstances, the B-Tree you encounter when repeating the experiment might be different.

Containers & Volumes

The APFS **Container** is at the base of the partition. A multiple partition may contain more than one mountable volume, but all volumes coexist in the same container, which serves as the logical volume manager and manages the available space. The container superblock can always be found at block address 0x0 of the partition, is a block of type 1, and is easily recognizable by the 'NXSB' magic, which immediately follows the standard APFS block header. Figure 8-14 shows the layout of the superblock structure:

Figure 8-14: The layout of the APFS container superblock structure

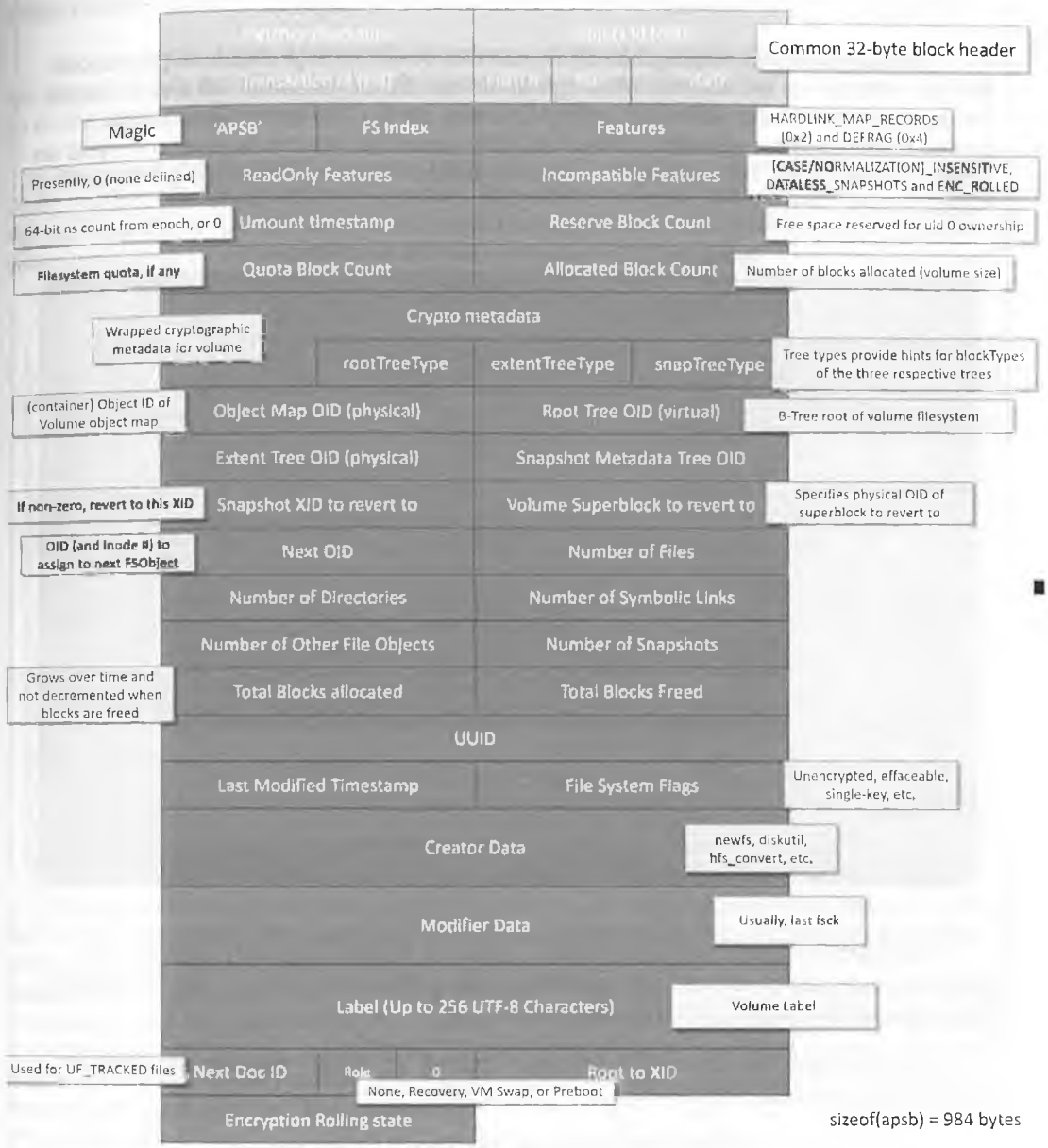


Note, that the superblock residing at address 0x0 is not necessarily the active container superblock. Thanks to APFS's built-in transaction mechanism, multiple copies of the container superblock may exist, each with a different XID. Mounting the volumes in an APFS container thus requires using block 0x0 as the point of departure, but finding the active superblock by looking through its checkpoint area, iterating through the various checkpoint descriptors, and locating the container superblock with the highest XID, whose objects are all consistent (that is, their respective checksums are valid). The respective volume OIDs can then be located in the container's OMAP, and mounted.

Volumes

The APFS volumes are the actual mountable objects of the APFS container. A container can hold up to the hard-coded limit of 100 volumes, though in practice most containers hold but one, or a few. Each volume has a superblock of its own, which is indicated to from the container's FSOID array. These superblocks are always of type 13, and recognizable by an 'APSB' magic. Each volume maintains three trees - filesystem, snapshot metadata and extent. The first is indicated by a virtual OID, and the latter two are physical. Figure 8-15 (next page) shows the layout of the superblock structure, with Table 8-16 showing the known volume roles.

Figure 8-15: The layout of the APFS volume superblock structure



sizeof(apsb) = 984 bytes

Table 8-16: Volume roles

#	Role	Use
0	(none)	Value used when no role is assigned
0x01	System	Root filesystem
0x2	User	User defined role
0x04	Recovery	MacOS: Recovery OS
0x08	VM	MacOS: Swap (/var/vm)
0x10	Preboot	MacOS: Pre-boot environment
0x20	Installer	MacOS: Set up during OS installation
0x40	Data	User data. (MacOS 15: /System/Volumes/Data, *OS: /private/var)
0x80	Baseband data	*OS: Baseband .nvm and mrc.dat (/private/var/wireless/baseband_data)
0xc0	Update	*OS: Used for system updates (/private/var/MobileSoftwareUpdate)
0x100	xART	BridgeOS: XART lockers (/private/xarts). Other *OS: empty
0x180	Backup	?
0x140	Hardware	?
0x1c0	SideCar	Somehow related to the multi-display feature?
0x200	Internal	Not seen in the wild - likely Apple Internal
0x240	Enterprise data	Presumably Apple's internal corporate data (Internal + Data)
0x280	iDiags	Presumably Apple Internal

Experiment: Using `fsleuth` to look at APFS volumes

Continuing with our investigation of the raw disk device through `fsleuth(j)`, we can turn our attention to the volumes. When given the raw device, `fsleuth` will enumerate all the volumes it finds, and select the largest volume in the list. The volume information printed is limited to the essentials, unless explicitly requested with the 'volume' command on a given volume block. The command can also be used to switch volumes, using the keyword 'select'.

Output 8-17: The volumes in a standard MacOS container

```
root@Bifrost (-)# fsleuth /dev/disk0
..
Container spanning 465.62 GB (122061322 blocks) with 4/100 volumes
Warning: Unable to verify block from FD 4:0x329ea000!
Chose NXSB copy #0x1fd67 with XID 0x2494
Volume 1: (Block 0xd4935)       (XID: 0x2494)
   Label: 'Macintosh HD' with 626028 files, 175055 dirs, 13466 symlinks (+ 50 others)
          Size: 27.5 GB (7092445 blocks)
Volume 2: (Block 0xd48d9)       (XID: 0x23f7)
   Label: 'Preboot' with 50 files, 26 dirs, 0 symlinks     Size: 42.27 MB (10823 blocks)
Volume 3: (Block 0xd5315)       (XID: 0x23f8)
   Label: 'Recovery' with 25 files, 4 dirs, 0 symlinks     Size: 501.22 MB (128313 blocks)
Volume 4: (Block 0xd5idc)       (XID: 0x1609)
   Label: 'VM' with 1 files, 0 dirs, 0 symlinks            Size: 1024.1 MB (262149 blocks)
FSleuth (Macintosh HD:/)> volume 0xd4935
Volume @0xd4935 (XID: 0x2494)  (No role)
        Label: 'Macintosh HD' with 626028 files, 175055 dirs, 13466 symlinks (+50 others)
        Size: 27.5 GB (7092445 blocks)
        Features: 0x2   # snapshots: 0  none
        Unmounted: Nov 01 03:35         OMAP: 0xd482b
        Next DocID: 3   Next OID: 0x15e6a7
        Reserve:   None Quota:   None
                GUID:    33A34D4A-54D9-47F6-9A21-A40BD9460E1F
                Formatted by: 'newfs_apfs (945.200.84)' on Oct 15 05:35 , last XID: 0x2
                Modified by:  'apfs_kext (945.220.25)'
                ACreator:  apfs_kext (945.220.25)
        Last modified: Nov 19 11:53
        FS Flags: One key (0x2)
        OMAP Oid: 0xd482b           Root Tree OID: 0x411     Extent Ref OID: 0xd46a3
        Snap Meta Tree OID: 0xd2e05  Revert To XID: 0         Sblock OID: 0x0
        Total Blocks: Alloced 13167050 (53932236800 bytes) Freed: 6147940
```

The MacOS container has four volumes - The root, Preboot, Recovery, and VM. The Recovery and Preboot volumes are accessible as /dev/disk1s2 and disk1s3. The VM volume (/dev/disk1s4) is mounted over /private/var/vm.

In *OS variants the single APFS container (labeled 'Container') contains two volumes: `System` (mounted as /) and `Data`.

Output 8-18: The volumes in a standard iOS container

```
Container (GUID: B8....AA25) spanning 59.60 GB (15624939 blocks) with 3/100 volumes
Chose NXSB copy #0xe4 with XID 0x1652
Volume omap OID: 0xead772
Volume 1: (Block 0xeb32d9)      (XID: 0x1628)
  Label: 'System'  Role: none, with 122302 files, 58346 dirs, 1933 symlinks
  Size: 4.2 GB (1055159 blocks)
Volume 2: (Block 0xeb3304)      (XID: 0x1477)
  Label: 'Baseband Data'  Role: none, with 1 files, 2 dirs, 0 symlinks
  Size: 1.1 MB (261 blocks)
Volume 3: (Block 0xeace2d)      (XID: 0x1652)
  Label: 'Data'  Role: none, with 19266 files, 7217 dirs, 20 symlinks (and 6 other objects)
  Size: 2.11 GB (553420 blocks)
```

Filesystem Trees

The actual file objects of the APFS filesystem are stored in a dedicated B-tree, referred to as the **fstree**. This is a B-Tree whose nodes' subtypes are marked as 14. The root node is pointed to by the Root FS OID field of the volume header (at offset @TODO).

The keys in the filesystem tree are commonly 64-bit identifiers, which also serve as the inode numbers of the objects they represent. In practice, however, only sixty bits are used, with the most significant four bits used to represent the object type. This allows for up to 16 types, which are referred to in the reference as `j_obj_types`.

Table apfs-fileobjtypes: APFS filesystem object types

#	Object type	Value contents
0	Reserved	N/A
1	Snapshot metadata	Name, timestamps, volume superblock OID, and extent information
2	Physical extent	Length, kind, owning OID and reference count
3	Inode	Inode data - timestamps, mode, ownership, etc. and optional extended fields
4	XAttr	Flags, value length and value data
5	Sibling link	Name of link, matching its directory record key
6	Data stream identifier	Reference count
7	Crypto state	Reference count and wrapped crypto state
8	File extent	Length, flags, physical block number, and encryption type
9	Directory record	inode number, timestamp, flags and optional extended fields
10	Directory statistics	# children, total size, parent dir OID and generational count
11	Snapshot name	Last XID in snapshot
12	Sibling map	The inode number of the target
15	Reserved (invalid)	N/A

There are a few special cases wherein the key may be longer than 64-bits, denoted in the above table by shaded cells. Sibling links contain an additional 64-bits uniquely identifying the sibling. Keys for named objects (extended attributes (type 4), direcrory entries (9) and snapshots(11)) are longer, and their 64-bit identifier is followed by a 16-bit length, and the name in question, as a NULL-terminated UTF-8 string (of length bytes). Directory entry keys also exist in another variant, `j_drec_hashed_key_t`, wherein the name length field is extended to 32-bits. Even in those cases, however, the maximum allowed length is 1023 bytes - as masked by the 10 least significant bits. The remaining 22 bits are used as a hash of the name, storing the 2's complement of the CRC-32c of the name, as normalized and represented as UTF-32.

Inodes commonly have multiple associated records. The first to appear is always the inode record itself (#3). If the file is uncompressed, a file extent record (#8) will follow. Compressed files will have the `com.apple.decmpfs` extended attribute record (#4), and possibly (if the file contents cannot be embedded in decmpfs) the `com.apple.ResourceFork` extended attribute record. Naturally, any other extended attributes may be present as well. In order to access a file, the filesystem first locates the inode, obtaining from it most of the basic `stat(2)` attributes. It then iterates over all records with the same object id (but different object types), to complete the full picture of the file, including its size, which is found in the extent or in the `com.apple.decmpfs` data.

Directory inodes have their additional dentry records (with the same inode number, but a key with the filename) following them, unless empty. They may also have the directory statistics record (#10). To list or search a directory, the filesystem first locates its inode, then iterates over all records with same object id and type #9, to get the associated file names and FileIDs (i.e. inode numbers). Long directory listings (e.g. `ls -l`) will require accessing each of the file inodes as well.

Experiment: Using `fsleuth` to inspect file access

The `fsleuth` tool is especially useful to show low level file access. Starting with any inode (for example, '2', for the filesystem root), you can use the `inode` command to lookup which FSTree block and record it is located in. Using 'block' will then show you all the block's records, as shown in Output 8-19:

Output 8-19: Using `fsleuth`'s `inode` command to locate an inode in the FSTree

```
FSleuth(PeaceB16B92.N102OS:/)> inode 2
Inode #2 located in block 0x6:2
FSleuth(PeaceB16B92.N102OS:/)> block 0x6
Block 0x6 (OID: 0x89b4, XID: 0x8) is B-Tree node
1-private-dir   Value: Oct 19 20:57  FileID 3,      flags:0x4 (dir)
1-root          Value: Oct 19 20:57  FileID 2,      flags:0x4 (dir)
2 Inode (mode 40755, #links: 18, Owner: 0, Group: 0)
2-.HFS+ Private Directory Data? Value: - FileID 19, flags:0x4
2-.Trashes      Value: Oct 19 20:57  FileID 162875, flags:0x8 (file)
2-.ba           Value: Oct 19 20:49  FileID 799,     flags:0x4 (dir)
2-.file         Value: Oct 19 20:49  FileID 1332,    flags:0x8 (file)
2-.mb           Value: Oct 19 20:49  FileID 1562,    flags:0x4 (dir)
2-Applications  Value: Oct 19 20:49  FileID 20,      flags:0x4 (dir)
2-Developer     Value: Oct 19 20:50  FileID 11030,   flags:0x4 (dir)
2-Library       Value: Oct 19 20:50  FileID 10708,   flags:0x4 (dir)
2-System        Value: Oct 19 20:50  FileID 11021,   flags:0x4 (dir)
2-bin           Value: Oct 19 20:57  FileID 148979,  flags:0x4 (dir)
2-cores         Value: Oct 19 20:57  FileID 148414,  flags:0x4 (dir)
2-dev           Value: Oct 19 20:57  FileID 148421,  flags:0x4 (dir)
2-etc           Value: Never         FileID 148428,  flags:0xa (symlink)
2-private       Value: Oct 19 20:57  FileID 148435,  flags:0x4 (dir)
2-sbin          Value: Oct 19 20:57  FileID 161440,  flags:0x4 (dir)
2-tmp           Value: Never         FileID 161450,  flags:0xa (symlink)
2-usr           Value: Oct 19 20:57  FileID 161443,  flags:0x4 (dir)
2-var           Value: Never         FileID 162453,  flags:0xa (symlink)
3 Inode (mode 40644, #links: 0, Owner: 0, Group: 0)
..
```

But how did `inode` locate the record? Toggling debug output will show you the steps taken behind the scenes:

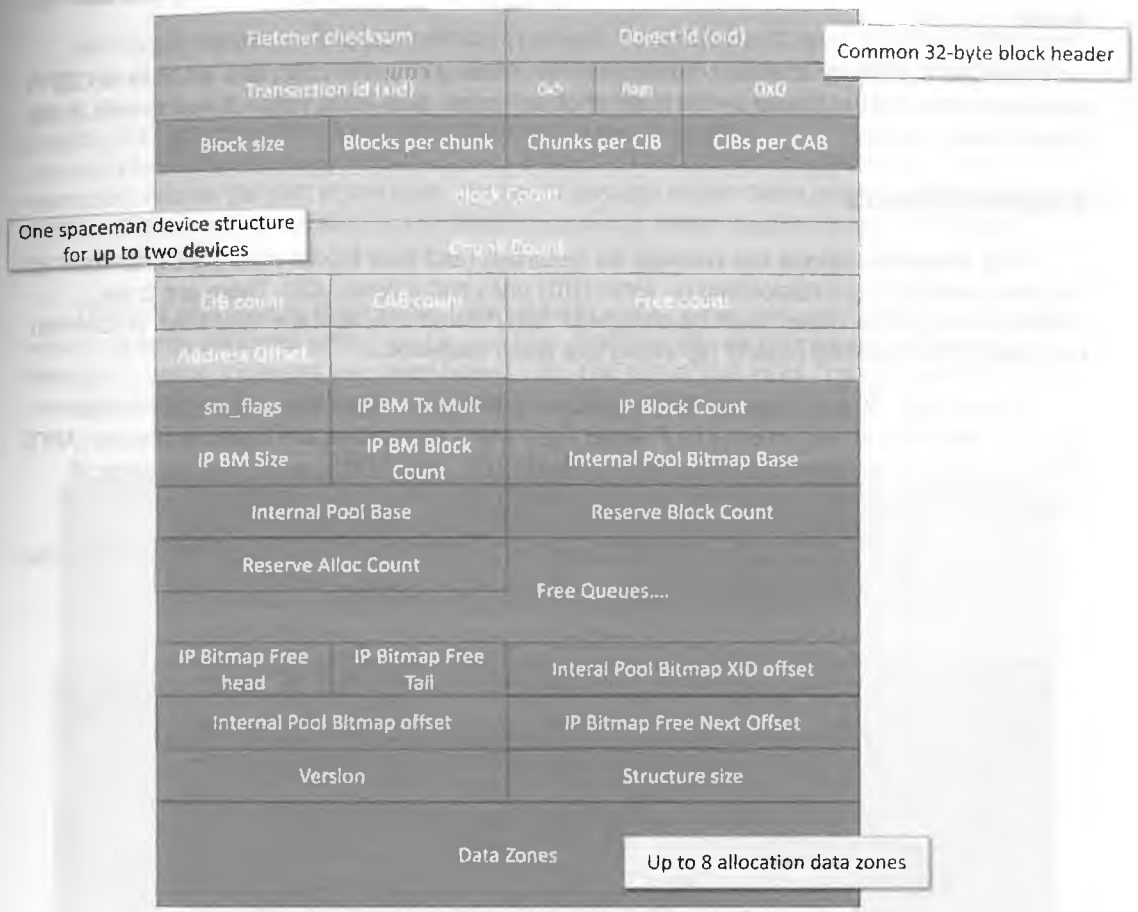
Output 8-20: Debugging `fsleuth`'s `inode` command

```
FSleuth(PeaceB16B92.N102OS:/)> debug
Debug tracing is on
FSleuth(PeaceB16B92.N102OS:/)> inode 2
#
# Level I: Start at the root of the FSTree, which we got from the volume block
#
Starting at Block 0x398a @OFFSET: 0x1018f000
0x398a:0: key (22 bytes) at 0x0  (0x9000000000000001:private-dir), value at 0xfd0 (0xb394)
0x398a:1: key (8 bytes)  at 0x16 (0x300000000001091b:), value at 0xfc8 (0xb393)
Looking for OID 0xb394  # We need 2 - greater than 1, less than 0x1091b - so recurse on 1
Starting from BTOM Block 0x38
... omap lookup omitted for brevity (refer to B-Tree navigation experiment)
FOUND OID 0xb394 in Block 0x323f!
#
# Level II: Narrowing down somewhat
#
0x323f:0: key (22 bytes) at 0x0  (0x9000000000000001:private-dir), value at 0xff8 (0x8a23)
0x323f:1: key (8 bytes)  at 0x16 (0x300000000000003f1:), value at 0xff0 (0x8a22)
Looking for OID 0x8a23 # Once again 2 - greater than 1, less than 0x0x3f1 - so recurse on 1
Starting from BTOM Block 0x38
... omap lookup omitted for brevity
FOUND OID 0x8a23 in Block 0x86f!
#
# Level III - almost there...
#
0x86f:0: key (22 bytes) at 0x0  (0x9000000000000001:private-dir), value at 0xff8 (0x89b4)
0x86f:1: key (26 bytes) at 0x16 (0x9000000000000014:SLYahooAuth.app), value at 0xff0 (0x89b3)
Looking for OID 0x89b4 # Getting closer, but still less than 0x14 - so recurse on 1
Starting from BTOM Block 0x38
... omap lookup omitted for brevity
FOUND OID 0x89b4 in Block 0x6!
#
# Level IV - got leaf node, have value
#
Block 0x6 @OFFSET: 0xc80b000
0x6:0: key (22 bytes) at 0x0  (0x9000000000000001:private-dir), value at 0xfee (0x3)
0x6:1: key (15 bytes) at 0x16 (0x9000000000000001:root), value at 0xfdc (0x2)
0x6:2: key (8 bytes) at  0x25 (0x3000000000000002:), value at 0xf70 (0x1)
Inode #2 located in block 0x6:2
```

The Space Manager

Recall, that the APFS container utilizes a **space manager** to maintain block allocation data. This is depicted in the following illustration:

Figure 8-21: An APFS space manager



The space manager oversees the entire container space, much like the HFS+ Volume Bitmap. As the container may span well into the GB and TB, however, the scheme is different, and requires two level management, of CIBs and CABs.

Chunk Info Blocks (CIBs)

Block-level allocation status is maintained in one or more **Chunk Info Blocks**, known as CIBs. The CIB are APFS blocks (type 7), consisting of a small header, denoting the CIB index and number of chunks it contains, followed by that number of chunk info structures. Each 32-byte chunk info structure contains the start block address to which it pertains, and address of another block, which serves as a bitmap tracking allocations of blocks from the start block onwards. The bitmap block has no header, and so all of its 32,768 bits can be used for tracking 0x8000 corresponding block allocations. The exact number of bits used is also maintained in the chunk info structure, along with the number of free bits.

Figure 8-22: APFS Chunk Info Blocks

```
FSleuth(PeaceF16F156.D321OS[0xb]:/)> block 0x77aa
Block 0x77aa (OID: 0x77aa, XID: 0xb) is CIB
CIB (index: 0) with 40 chunk infos:
Chunk   0: XID: 0xb  @0x0     - 32238/32768 blocks with bitmap @0x77ab
Chunk   1: XID: 0xb  @0x8000  -  9400/32768 blocks with bitmap @0x77ac
Chunk   2: XID: 0xb  @0x10000 - 25293/32768 blocks with bitmap @0x77ad
        ...
```

The space manager maintains counts of `sm_blocks_per_chunk` and `sm_chunks_per_cib`.

CIB Address Blocks (CABs)

A quick calculation will show that CIBs can hold up to 126 chunk info structures. If each corresponding bitmap block can cover up to 128MB of disk space, each CIB will span at most 16GB. This is hardly enough when containers often span hundreds of GB and into the TB. Multiple CIBs (pertaining to contiguous block spans) can thus be grouped into **CIB Address Blocks**, or CABs.

The CAB is a simple structure, holding only an index, a count of CIBs, and an array of CIB addresses. The value is bound by the number of `sm_cibs_per_cab`, which is maintained at the Space Manager level.

Reaping Objects

APFS filesystem objects can normally be discarded (and their blocks repurposed) as soon as they are replaced by corresponding (= same OID) ones and a newer XID. There are cases, however, wherein the object must be preserved, and deleted only at some later time. A common example is when purging objects belonging to a given snapshot.

To keep track of such objects, APFS maintains a **Reaper** object. The reaper (APFS block type 17) maintains an `nr_rlcount` of reaper lists, which themselves are separate objects (APFS block type 18). Each reaper list entry block contains `nrl_count` `nrl_entries`, an array of reaper list entries, which encode the object identifiers which are to be reaped.

As an example, consider Output 8-23, from the test DMG used in the experiments. Here, the reapers are empty, so the output is kept simple:

Output 8-23: APFS Reapers

```
#
# Note one reaper here for every XID
#
FSleuth(TEST APFS[0x4]:/)> blockmap reaper
Block 0xa (OID: 0x401, XID: 0x1) is reaper
Block 0xc (OID: 0x401, XID: 0x2) is reaper
Block 0x10 (OID: 0x401, XID: 0x3) is reaper
Block 0x14 (OID: 0x401, XID: 0x4) is reaper
#
# Empty reaper for this XID
#
FSleuth(TEST APFS[0x4]:/)> block 0x14
Block 0x14 (OID: 0x401, XID: 0x4) is reaper
Next reap id: 0x1       Completed: 0x0
Head: 0x0       Tail: 0x0       Flags: 0x1
0 reaper lists
```

A good way to see "live" reapers would be to take a snapshot, and then start removing files from it - and their blocks should show up on the reaper lists. If the snapshot is then removed, those files which have been removed will be reaped, and the reaper lists themselves will be cleared.

APFS.kext

As with HFS+ and other filesystems, APFS support in kernel is not built-in, but provided by the `com.apple.filesystems.apfs` kernel extension (a.k.a /System/Library/Extensions/apfs.kext). This kext is part of the same project as the userland APFS binaries, and (unlike HFS+) is still closed source at the time of writing.

APFS.kext registers the `AppleAPFSContainer IOProviderClass`. This way, when new APFS volumes are detected they are published through an IOKit matching notification. In user mode, `launchd` (or, more accurately, `UserEventAgent`'s `com.apple.iokit.matching` plugin) picks up the notifications and starts /usr/libexec/apfsd and /System/Library/CoreServices/APFSUserAgent. When the kext obtains the credentials of an encrypted volume (as part of the login process or through a user mode prompt), it creates an additional block device (in MacOS, this is the /dev/disk1), which represents the unencrypted container volumes.

The kext also creates three zones (from data in `apfs_zone_infos`) in which it maintains various objects, allocated with `__apfs_zalloc`. Arbitrary size allocations are performed through `__apfs_[c/m]alloc`, over `OSMalloc`. The size of the `APFS_INODES` elements has changed in Darwin 19, which also added the tree related zones.

Output 8-24: The `apfs_zone_infos` in apfs.kext

```
morpheus@Zephyr (~)$ itool2 -d _apfs_zone_infos /S*/L*/Ext*/apfs.kext/C*/MacOS/apfs
Dumping 31056 bytes from 0x1181e0 (Offset 0x1181e0, __DATA.__const):
_apfs_zone_infos:
0x1181e0: 00 00 00 00 00 00 00 00    index(0)
0x1181e8: 00 10 00 00 00 00 00 00    element size(4,096)
0x1181f0:          0xe1a39             "APFS_4K_OBJS"
0x1181f8: 01 00 00 00 00 00 00 00    flags(1)
0x118200: 01 00 00 00 00 00 00 00    index(1)
0x118208:           0x2000           element size(8,192)
0x118210:          0xe1a46             "APFS_8K_OBJS"
0x118218: 01 00 00 00 00 00 00 00    flags(1)
0x118220: 02 00 00 00 00 00 00 00    index(2)
0x118228: 70 01 00 00 00 00 00 00    element size(368 (Darwin 19) 304 (earlier))
0x118230:          0xe1a53             "APFS_INODES"
0x118238: 01 00 00 00 00 00 00 00    flags(1)
# Darwin 19 adds these:
0x118240: 03 00 00 00 00 00 00 00    index(3)
0x118248:           0x1220           element size(4,640)
0x118250:          0xe1a5f             "APFS_TREE_LOOKUP_ZONE"
0x118258: 01 00 00 00 00 00 00 00    flags(1)
0x118260: 04 00 00 00 00 00 00 00    index(4)
0x118268: 10 01 00 00 00 00 00 00    element size(272)
0x118270:          0xe1a75             "APFS_BTREE_NODES"
0x118278: 01 00 00 00 00 00 00 00    flags(1)
```

`fsctl(2)` codes

As with other filesystems, the APFS.kext provides sets of `fsctl(2)` codes, which are a mixture of the standard codes (supported by most other filesystems) and some specific ones.

Table 8-25: The APFS `fsctl(2)` code

##	Name/function
	`APFSIOC_GET_CRYPTO_KEY`
0x82fd4a42	Synthetic folder creation
0x80044a3[a/b]	Set/Get Atime setting
0x81304a06	`APFSIOC_SET_CONV_CTX` (used by `hfs_covert`)
0xc40843ac	`APFSIOC_FIRMLINK_CTL`
0x80084a02	`APFSIOC_MAINTAIN_DIR_STATS` (`apfs.util -M /....`)
0x40184a03	`APFSIOC_GET_DIR_STATS` (`apfs.util -S /...`)
0xc0204a45	`APFSIOC_PURGE_FILES` (`apfs.util -P -high /.. , deleted_helper`)
0x40b04a46	`APFSIOC_GET_PURGEABLE_STATS` (`apfs.util -G /.., deleted_helper`)
0x40084a47	get purgeable file stat (-Z /..)
0x40044a48	`APFSIOC_J_SET_SYNC_ROOT`
0x80084a2f	Called from apfs_defragd
0x40044a30	

A few of the codes above require privileges (III/4), which are enforced by the Sandbox.kext's `priv_check_hook`. The privileges require entitlements, all in `com.apple.private.apfs`: `..set-firmlink`, `..set-free-space-change-threshold`, `..trim-active-file` and `..revert-to-snapshot`.

UserClient Methods

In addition to the standard `fsctl(2)` interface, The APFS.kext provides an `IOUserClient` with a rich set of methods.

Table 8-26: The APFS UserClient methods

##	Name	##	Name
0	VolumeCreate	22	ContainerSetDefrag
1	VolumeDelete	23	VolumeGetDefrag
2	ResizeContainer	24	VolumeSetDefrag
3	DefragContainer	25	VolumeEscrowVEK
4	ContainerGetResizeProgress	26/0x1c	VolumeInfo
5	ContainerWipeKeys	27	GetContainerStats
6	ContainerSpacemanInfo	28	GetVolumeStats
7	VolumeSpace	29	GetContainerHistograms
8	ContainerExtendedInfo	30	VolumeGetUnlockRecordState
9	VolumeRole	31	CancelContainerResize
10	VolumePayload	32	GetMinContainerSize
11	ContainerEFIEmbed	33	ContainerStampUUIDs
12	ContainerEFIGetVersion	34	DeltaCreatePrepare
13	VolumeGetRecord	35	DeltaCreateChunk
14	VolumeListUUIDsOfRecords	36	DeltaRestorePrepare
15	VolumeUnlockRecord	37	DeltaRestoreChunk
14	VolumeAddUpdateRecords	38	DeltaRestoreFinalize
15	VolumeDeleteWipeRecords	39	ContainerVolumeGroupAdd
16	VolumeCryptoOperation	40	ContainerVolumeGroupRemove
17	VolumeGetVEKState	41	ContainerVolumeGroupGet
18	VolumeEnableUserProtection	42	ContainerVolumeGroupGetVolumes
19	VolumeEnableSystemProtection	43	ContainerVolumeGroupGetFirmlinks
20	VolumeBindNewKEKToVEK	44	GetFragmentationHistogram
21	ContainerGetDefrag	45	ContainerVolumeGroupSyncUnlockRecords

* - As of Darwin 19, `mount` recognizes "phases", passed through the `-P` command line argument. This gets passed to `apfs.util`

Review Questions

- Compare the APFS B-Tree implementation with that of HFS+ (discussed in the first edition). What are the similarities and what are the differences?
- What is one barrier preventing APFS from offering arbitrary undelete functionality on iDevices? How is that different in some desktops and servers?
- How many CIBs and CABs will be required to support the Space Manager of a 4GB container? 1TB? 16TB?

References

1. Jonas Plum - APFS - https://blog.cugu.eu/post/apfs/, also https://blog.cugu.eu/files/pub/2018_04_ernw_whitepaper_apfs.pdf
2. Hansen et Al - "Decoding the APFS file system" - http://cyberforensicator.com/wp-content/uploads/2017/11/DIIN_698_Revisedproof.1-min-ilovepdf-compressed.pdf
3. Apple Developer - "The Apple File System Reference" - apple-file-system/Apple-File-System-Reference.pdf
4. WWDC 2019 Session 710 - https://developer.apple.com/videos/play/wwdc2019/710/

9

Tempus Fugit: Mach Scheduling

Although the BSD `struct proc` and `uthread` (discussed in Chapter 6) hold a significant amount of information, it is Mach which is responsible for the low level process attributes, IPC and memory management, and the scheduling of its therads. The true "beef" of processes and threads is therefore in their Mach objects. Chapter 12 in Volume I of this work introduced the Mach task and thread objects - but through their user mode interface, of ports. In kernel mode, these objects are far more detailed, and the already significant functionality exposed by their MIG subsystems is greatly expanded.

This chapter begins with a closer look at the kernel representation of tasks. As we've looked at the BSD's `kernproc` in Chapter 6, it's only fair to examine the `kernel_task` as well. Threads are subjected to a similar investigation, with more detail on their lifecycle - creation, termination, and policy changes.

After acquainting with thread objects, we turn to investigate the most important aspect of their lifecycle: scheduling - the practice of keeping track of all threads in the system, and suggesting which one(s) to assign to CPU cores. It is assumed the reader has a good grasp of what scheduling entails (The main aspect, context switching, was already discussed in Chapter 4). We begin with a discussion of Mach scheduling enhancements - handoff, continuations, and Asynchronous Software Traps (ASTs) - all important to ensure efficient scheduling.

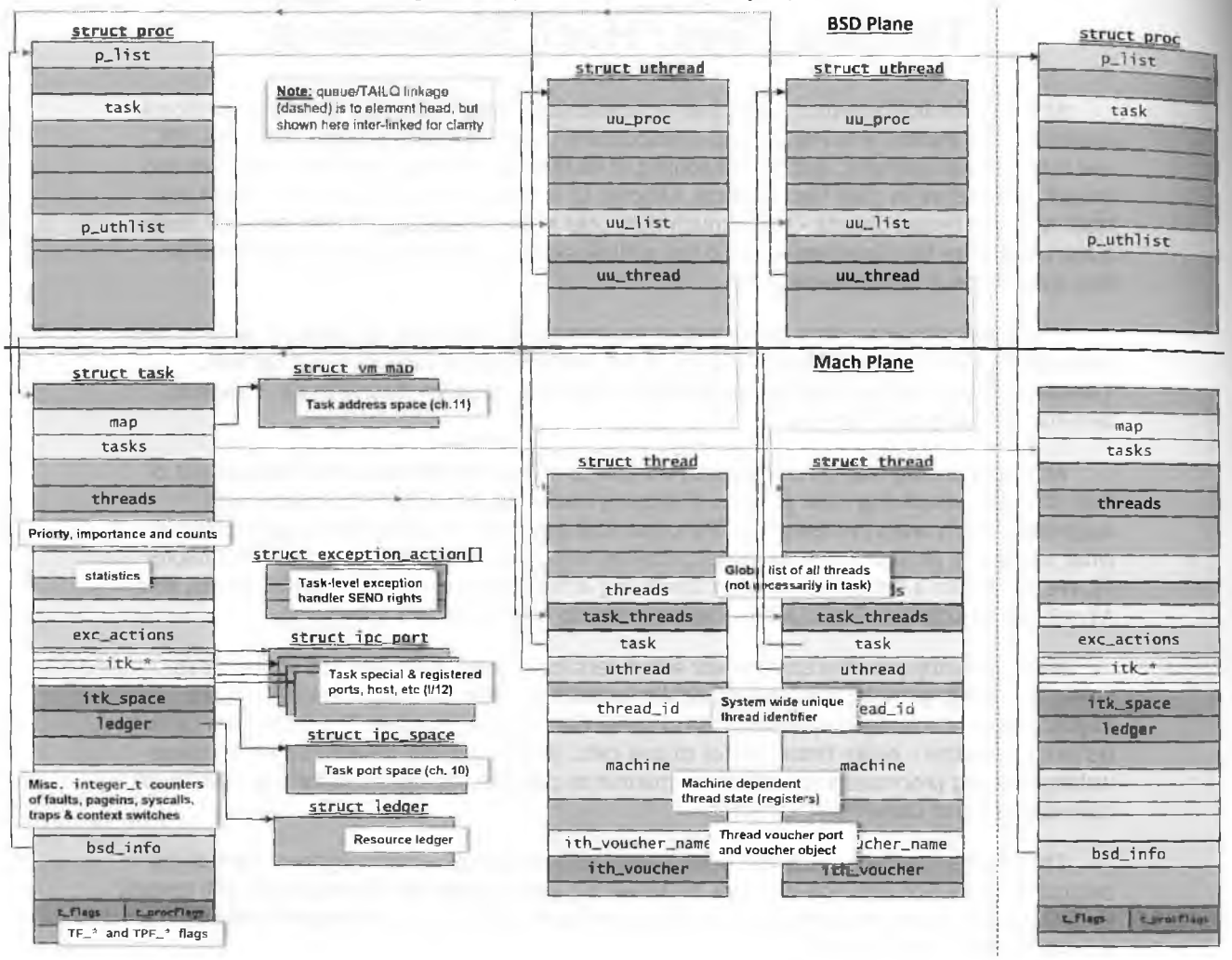
Mach presently provides six different schedulers (only one can be selected at any given time), but all interact with the same scheduling primitives - The run queue, priorities and a dispatch table of scheduler operations. All of these are explored in detail. The focus then turns to deferred execution - either timer calls or thread calls. Next, scheduler assisted synchronization - waking sleeping processes in as efficient a manner as possible - is discussed through XNU's wait queues and (as of Darwin 18) turnstiles.

The chapter concludes with two other Darwin enhancements - Mach's ledgers, part of the original design which was brought back in Darwin 13, and enables QoS like behavior with respect to CPU, memory and other resources. Lastly, Selective Forced Idle - a mechanism to force threads to idle even if runnable.

The High Level View

As explained in I/12, the Mach logic takes place in what can be conceived as a parallel plane of existence. BSD memory management syscalls delegate the logic to the `mach_vm/vm_map` subsystems. Signals are generated due to Mach exceptions. As we delve into the innards of these objects and scheduling, we can revisit their visualization, offering a bit more detail than was shown in I/12-6. Figure 9-1 illustrates the Mach objects, their relationships with the BSD objects, and with each other.

Figure 9-1: Visualizing the Mach plane `task` and `thread` objects, and their interconnections



Mach Tasks

The Mach task object, defined in bsd/kern/task.h, is another one of the large but critical structures of XNU. Its size often fluctuates among Darwin versions (e.g. 1,472 in MacOS 14, versus 1,560 in 15), and due to machine specific fields between flavors as well (1,440 and .... in iOS 12 and 13, respectively). Like the BSD `struct proc`, its fields - constantly modified in between Darwin versions - can be grouped by classes:

- **The task lock:** is a `lck_mtx_t` which synchronizes access to the entire task object.
- **Statistics:** include `total_user_time`, `total_system_time`, `total_ptime` and (as of Darwin 18) `total_runnable_time`.
- **Priority, maximum priority and importance:** The task is not a scheduleable object, but these values are inherit by its threads. The `importance` field - not be confused with task importance (`task_imp_base`) as described later - corresponds to BSD's `nice(1)`.
- **The `vm_map`:** providing the address space of the task, and accessed from user mode using the `vm_map` or `mach_vm` MIG subsystems.
- **Linkage:** through the queue chain link of `tasks` to the global task list.
- **Threads:** The task's `threads` is a `queue_head_t` of `thread_count` threads, of which `active_thread_count` are presently active. The `pset_hint` points to the default (de facto, only) processor set, and its `suspend_count` is used for the `task` subsystem's `task_suspend/resume`.
- **Task port space:** The task's `itk_space` field is a `struct ipc_space` pointer holding the mappings from the user mode `mach_port_name_t` integers to the underlying `struct ipc_object`. This mapping is discussed in greater detail in chapter 11.
- **Task special ports:** The task maintains quite a few `struct ipc_port itk_*` pointers, which correspond to the "task special port" `SEND` rights retrievable in user space, and inherited from parents. Rather than be kept in an array, these are specifically named. `itk_[sself/host/nself/bootstrap/seatbelt/task_access]` and `[debug_control]` correspond to task special ports numbers #1,2,3,4,7,9 and 10 (q.v. Table I/12-11). Note that `TASK_NAME_PORTS` (#3) cannot be set, `..SEATBELT` and `TASK_ACCESS` cannot be overwritten, and `itk_gssd` is still defined, although its user mode number (#8) is available no longer.
- **Task registered ports:** Up to `TASK_PORT_REGISTER_MAX` (=3) `SEND` rights stashed by `mach_ports_register()` are stored in the `itk_registered` array. These rights are inherited by child tasks, but only made available after a call to `mach_ports_lookup()`.
- **Task exception ports:** An array of `EXC_TYPES_COUNT` (=14) `exception_actions`, which hold the registered exception ports of the task for all possible types. These are `SEND` rights which may be obtained, set or swapped from user space by task MIG calls.
- **The Machine task:** This is a machine-dependent collection of fields grouped together by a single `MACHINE_TASK` macro. The macro is `#define` in osfmk/[i386|arm]/task.h, and expands in ARM architectures to a single `task_debug` void pointer, which points to an `arm_debug_state[32/64]_t`. In intel architectures a few other fields `task_debug` similarly points to an `x86_debug_state64_t`, and several other legacy i386 fields. The machine state values (referenced by the `task_debug` pointer) can be accessed from user mode using the `task` subsystem's `task_[get/set]_state`, which internally call `machine_task_[get/set]_state` (from osfmk/*arch*/machine_task.c).
- **Security and audit tokens:** The task's `sec_token` is a `security_token_t` whose 0th entry (the corresponding process `uid_t`) determines if the task is privileged. The security token is also used in Mach message trailers. The `audit_token` is an `audit_token_t` (described in III/2), and also used in Mach message trailers, and by XPC. Both tokens are inherited, can be retrieved using `task_info`'s `TASK_SECURITY..` and `..AUDIT_TOKEN` flavors. Both tokens can also be be set through a call to `host_security_set_task_token`.
- **Counts:** `faults/pageins/cow_faults`, `messages_[sent/received]`, `syscalls_[mach/unix]`, and `[c/p/ps]_switch` hold various monotonically increasing counts which can all be retrieved in user mode by `proc_info`'s `PROC_PIDTASKINFO` flavor, as is done by `top(1)` and `procexp(j)`.

- **Resource usage:** `rusage_cpu*` fields hold flags, and task-wide/per-thread CPU limit percentage and interval.
- **The corresponding `struct proc`:** is accessible through the `bsd_info` field. As was with the task to the proc, the `bsd_info` is a void pointer to maintain opacity. It is further conditionally defined `ifdef MACH_BSD`. This is, of course, true, but does show that in Mach the task could at one time exist independent of a BSD process. `get_bsdtask_info()` (from osfmk/kern/bsd_kern.c is used as an accessor.
- **Corpse information:** (q.v Ch. 6). The `corpse_info` is a `kcdata_descriptor_t`. There are also a `crashed_thread_id`, and a link to other `corpse_tasks`.
- **I/O statistics:** are in an `io_stat_info` pointed to from the `task_io_stats` member, along with a set of of `[immediate/deferred/invalidated/metadata]_writes`.
- **Flags:** The `TF_* t_flags` and `TPF_* t_procflags` are `#defined` right in the middle of the structure along with accessor macros.
- **Purgeable VM objects:** Counts are maintained for both volatile and non-volatile. The `task_objq` (protected by the `task_objq_lock` is a `queue_head_t` of the VM objects owned by the task. Chapter 11 contains a discussion of purgeable memory.
- **Coalitions:** belonged to are held in an array, and a `task_coalition` queue chain.
- **Associated hypervisor Virtual Machine (MacOS):** in the `hv_task_target` pointer.
- **Secluded memory:** in an enhancement Apple introduced in Darwin 16, (q.v. Chapter 11).
- **External Modification statistics:** The important `extmod_statistics` structure, retrievable from user mode by `task_info`'s `TASK_EXTMOD_INFO` flavor, holds the counts of `task_for_pid` calls, foreign `thread_create` and `thread_set_state` calls, both made on the task and by the task.
- **Effective and requested scheduling policies:** As described later in this chapter, under "Effectuating policy changes".
- **IOUserClients:** which are held by the task are kept in the `io_user_clients` `queue_head_t`. As explained in Chapter 13, `IOUserClients` are handles of IOKit kernel drivers the task holds, and can use to access driver properties, register for notifications, map memory, and/or invoke methods.
- **Task watching** (*OS). The `task_watchers` is a `queue_head_t` of `num_taskwatchers` `task_watch_t` (with an additional field, `watchapplying`), representing threads watching this stack for policy state changes. Threads can watch using `process_policy` (#323) with with the `PROC_POLICY_APP_LIFECYCLE` flavor and the `PROC_POLICY_APPLIFE_PIDBIND` subtype.

The `kernel_task`

One task in particular stands out from the rest, deserving special mention - and it is the `kernel_task`. Just as the `kernproc` represents the kernel as a process, so does the `kernel_task` represent it as a task, and it, too, is a global variable (not exported, but located by `jtool2`'s `joker` module). The importance of the `kernel_task` cannot be overstated. As with other task objects, it exposes the rich MIG API of the `task`, `mach_vm` and `[mach]_exc` subsystems - enabling user mode to entirely control the kernel.

Around Darwin 9 Apple has realized just how powerful these APIs are when targeting the kernel, and has taken steps to limit the visibility of `kernel_task` in user mode. First, by explicitly checking for PID 0 in calls to `task_for_pid`, then removing the `kernel_task` and the kernel threads from the task and thread sets returned by `processor_set_things`.

This has not, however, made things much more secure. The `kernel_task` still exists, and so does its port abstraction. Exploiters and jailbreakers go to extreme lengths to expose a `SEND` right to the `kernel_task` port - and examples of techniques to do so about in Volume III's later chapters. Apple caught on to that, as well, adding explicit checks in Darwin 17 for cases where the `kernel_task` is referenced. Unfortunately, these checks only compare the address of the referenced object with that of `kernel_task` - so exploiters instead clone or construct a fake (but semantically equivalent) kernel task object.

Mach Threads

For all their size, Mach tasks (like UN*X processes) are merely resource containers. It is their threads which are the scheduleable entities.

A Mach thread can conceptually be thought of as two separate entities - the shuttle, and the activation. The Mach APIs still reflect this, in that the MIG subsystem name for managing threads is in thread_act.h. Internally, there is just one object, a `struct thread`. The Mach thread object is, like the parent task, a large object - 1,264/1,296 in Darwin 17, and 1,296/1,328 in Darwin 18, and 1,296/... in Darwin 19. Continuing with our approach of classifying fields, the `struct thread` contains the following:

- **Execution State:** `TH_[WAIT/SUSP/RUN/UNINT/TERMINATE/IDLE]` flags are bitwise OR-ed in the `state` field.
- **Linkage:** Each thread is linked to its parent `task`, the `threads` global list, and its siblings (`task_threads`). The `thread`'s structure first element is a union linking it to either the run queue (if `state` is `TH_RUN`) or wait/priority queues (if `TH_WAIT`).
- **Wait data:** If the thread state is `TH_WAIT`, `wait_event` also holds the event ("wait channel"), and a `waitq` pointer references a wait queue, in which this thread is enqueued.
- **Ports:** Each thread holds a pointer to its port representation in `ith_self`, a `SEND` right to its port in `ith_sself`, and (Darwin 18+) an `ith_special_reply_port` reference.
- **Priority:** Five `int16_t` values are used to indicate the `sched_pri`, `base_priority`, `max_pri`, `task_priority` and `promotion_priority`.
- **Scheudling information:** The `sched_mode` and `saved_mode` hold thread scheudling modes, which are one of the `TH_MODE_[REALTIME/FIXED/TIMESHARE]` values.
- **Continuation:** Threads may either maintain a stack, or use continuations (discussed later) when resuming execution. In the latter case, the `continuation` function pointer and its `parameter` are stored in the eponymous fields.
- **Affinity values:** The `affinity_set` and `affinity_threads`
- **Page fault recovery handler:** As discussed in Chapter 4, for handling `copyin()/copyout()` errors.
- **Thread call state:** holding a thread call group and thread call pointer, for kernel threads.
- **Guard exception codes:** holding the code and subcode of an `EXC_GUARD` for user mode threads.
- **Turnstile:** As of Darwin 18, discussed later in this chapter.
- **The BSD `uuthread` object:** is linked from the `uthread` field, as a void pointer. This is similar to the way the `struct proc` is linked from the `task` - also contingent on `MACH_BSD`.
- **DTrace data:** `#if CONFIG_DTRACE` (on MacOS), the thread object size is increased by .... bytes which hold DTrace details.
- **Per-thread statistics:** Count of `syscalls_unix` and `syscalls_max`.
- **Ledger details:** Ledgers (introduced in Darwin 13 and discussed later in this chapter) allow resource allocation.
- **Associated voucher:** The `ith_voucher` is the associated voucher object (discussed later). The `ith_voucher_name` is the port name of the voucher.
- **Tag**: Mach threads have no names (`thread_set_thread_name()` actually sets the corresponding UThread's `pth_name`) , but they do have a 16-bit `thread_tag`. The individual tags are flags, and so far five out of the possible 16 are defined: `THREAD_TAG_MAINTHREAD`, `.._CALLOUT`, `..IOWORKLOOP`, `.._PTHREAD` and `.._WORKQUEUE`.
- **The machine dependent thread object:** holding the actual register state of the thread, for context switching.

Thread creation

Not all threads are created equal. User mode threads are created by a call to `thread_create[_running]` (in osfmk/kern/thread.c, like all other creation routines discussed in this section). Thread creation is normally trigged from user space using MIG routines #3411 and #3412 of the `task` subsystem. Threads are normally created suspended, but using the `..._running` variant allows the caller to set the initial register state of the process and immediately schedule it for execution. A good example of using the user mode MIG routines can be found in I/12-9, with a full example of remote thread injection on the book's website[1].

Kernel threads are created by `kernel_thread_create()`, or (similar to the `..._running` variant) `kernel_thread_start[_priority]`. Unlike user mode threads, a *continuation* function and its *parameter* must be specified. `kernel_thread_create()` does not start the thread, whereas the `_start` variants do (and possibly set its priority). IOKit provided the (deprecated) `IOCreateThread()` as a wrapper over `kkernel_thread_start`. There are a couple of other wrapper calls, shown in Figure 9-2 (next page).

All roads eventually lead to `create_thread_internal()`. With the exception of the first thread, all threads are allocated from the `threads` zone and are effectively cloned off of the `thread_template`. This is a `thread` structure with all its field prepopulated with default values by `thread_bootstrap()` (cleared counters and statistics, no vouchers/ledgers, etc), in order to optimize startup time.

BSD Tasks (i.e. user mode processes) also require the Mach thread to have a corresponding `uthread`, so `thread_create_internal()` takes care of that. At their lower end, the Mach thread also needs a machine thread as well, so `machine_thread_create()` (osfmk/i386/pcb_native.c or osfmk/arm[64]/pcb.c) is called for that. Assuming that is successful, thread initialization can continue, overwriting template values with the specific continuation and parameter supplied to `thread_create_internal()`. The thread's locks (thread_lock, wake_lock and mutex) are initialized, as are its IPC structures, and I/O statistics. The thread is connected to its task (incrementing its reference count by one), and a new ledger is constructed. Scheduling parameters (importance, priority and policy) are set. The thread can then be chained into the task's `threads` list, as well as the global list. If Kdebug is enabled, `TRACE_DATA_NEWTHREAD` is emitted, and a DTrace probe is fired. Darwin 18 and later also account for thread turnstiles (discussed later in this chapter). Thread creation is now complete.

On return from `thread_create_internal()`, the various callers add embellishments to the newly created thread. The `..from_user` variants increment the external modification statistics if this thread was created from a different task. The `...running` and `..start` variants also call `thread_start()`, and the `thread_create_waiting_internal` variant calls `thread_start_in_assert_wait()` (from osfmk/kern/thread_act.c), which adds the thread to the specified event's wait queue.

Thread termination

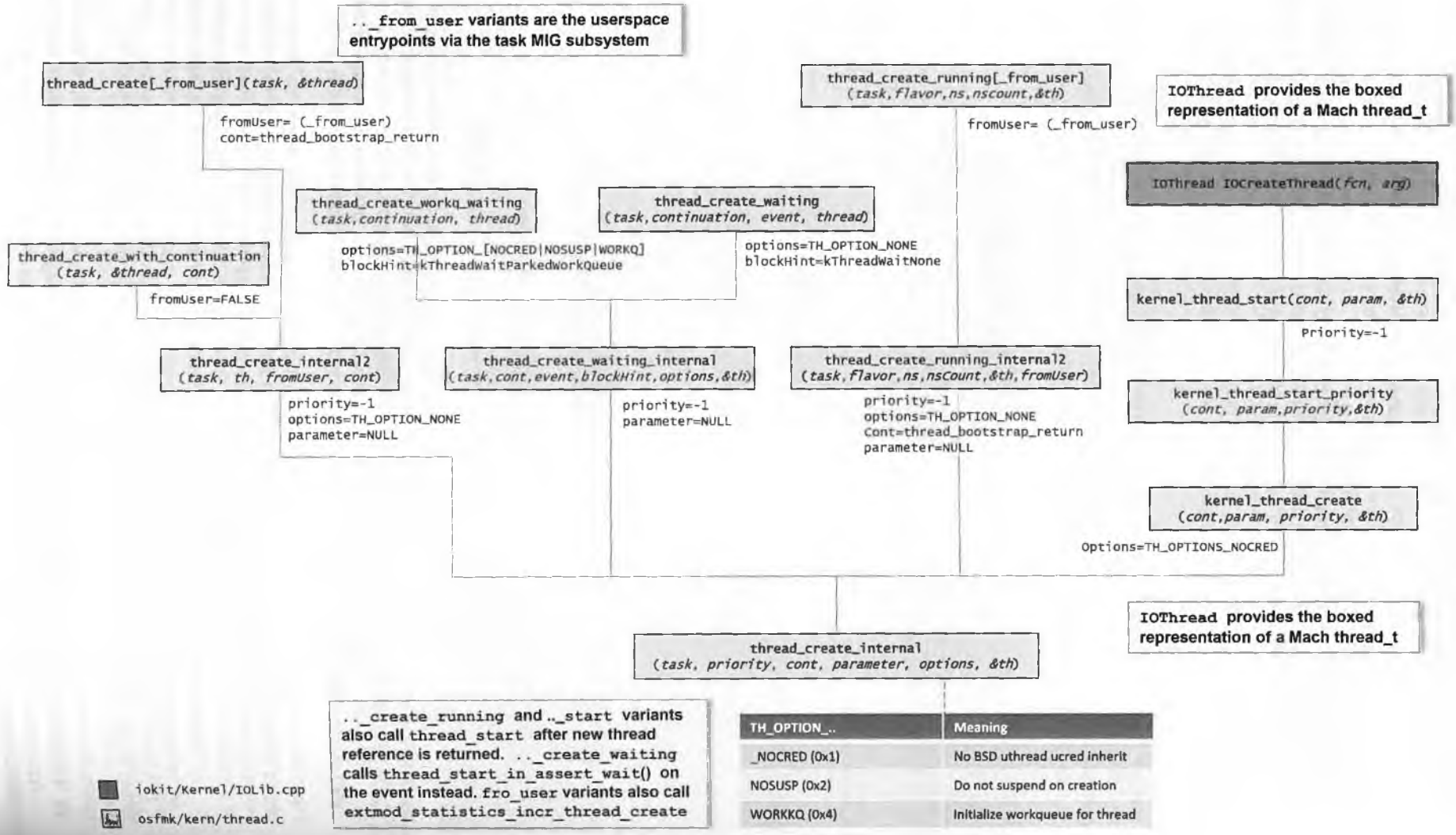
One would think that terminating a thread would be the inverse of its creation. While that is, at a high level, true, the flow required for it is significantly more complex. The `thread_terminate()` function (from osfmk/kern/thread_act.c) begins the flow. The function is called from numerous points in kernel space, and can also be triggered from user mode through the `thread_act` subsystem's MIG routine #3600. After ensuring it is dealing with either a user mode thread, or a kernel thread voluntarily terminating, `thread_terminate` calls `thread_terminate_internal` (ibid). If the thread is active, `act_abort()` is called, and signals the `AST_APC`.

Asynchronous Software Traps (ASTs) are described later in this chapter, but think of them as deferred procedure calls. On return to user mode, the AST's callback (`thread_apc_ast`, in osfmk/kern/thread_act.c) is invoked*, which ensures the thread is inactive before calling `thread_terminate_self()` (osfmk/kern/thread.c).

---

* - kernel threads never return to user mode, so they take a faster path and `thread_apc_ast` is called directly after `thread_terminate_internal()`.

Figure 9-2: The roads to `create_thread_internal()`



TH_OPTION_..	Meaning
_NOCRED (0x1)	No BSD uthread ucred inherit
NOSUSP (0x2)	Do not suspend on creation
WORKKQ (0x4)	Initialize workqueue for thread

`thread_terminate_self()` performs cleanups of data structures, triggers BSD process exit (when this is the last thread in the process), cleans up any timers and - most importantly - flags the thread object's `state` field as `TH_TERMINATE`. It then puts the thread into a block, to continue on `thread_terminate_continue`.

This continuation, however, will never be reached (if it were to be reached, the kernel would panic). Instead, when the scheduler next encounters the thread during `thread_dispatch()` (osfmk/kern/sched_prim.c) it notes the `TH_TERMINATE` flag and enqueues it for termination, by placing it in the `thread_terminate_queue`. The `thread_terminate_queue` queue (guarded by the simple `thread_terminate_lock`), is maintained by the `thread_terminate_daemon`, and the act of enqueueing wakes up the daemon from its continuation. If the thread is part of a corpse (`thread->inspection` is set), it is moved to the `crashed_threads_queue`. Otherwise, reaping begins.

First, the thread statistics (timers, syscall counters, etc.) are merged with those of the task. Then the thread is removed from the task's `threads` queue, and from the global `threads` queue. The thread object can then be deallocated. `thread_deallocate()` decrements the thread's reference count, and when it drops to zero, `thread_deallocate_complete()` kicks in and performs the actual release of the thread structure and any resources:

Listing 9-3: `thread_deallocate_complete()` (from osfmk/kern/thread.c)

```
void
thread_deallocate_complete( thread_t                            thread) {
      task_t      task;

      if (!(thread->state & TH_TERMINATE2))
            panic("thread_deallocate: thread not properly terminated\n");
#if KPC
      kpc_thread_destroy(thread);
#endif
      ipc_thread_terminate(thread);
      proc_thread_qos_deallocate(thread);
      task = thread->task;

#ifdef MACH_BSD
      {
            void *ut = thread->uthread;
            thread->uthread = NULL;
            uthread_zone_free(ut);
      }
#endif /* MACH_BSD */

      if (thread->t_ledger) ledger_dereference(thread->t_ledger);
      if (thread->t_threadledger) ledger_dereference(thread->t_threadledger);

      if (thread->turnstile)
            turnstile_deallocate(thread->turnstile);
      if (IPC_VOUCHER_NULL != thread->ith_voucher)
            ipc_voucher_release(thread->ith_voucher);

      if (thread->thread_io_stats)
            kfree(thread->thread_io_stats, sizeof(struct io_stat_info));

      if (thread->kernel_stack != 0) stack_free(thread);

      lck_mtx_destroy(&thread->mutex, &thread_lck_grp);
      machine_thread_destroy(thread);

      task_deallocate(task);
      zfree(thread_zone, thread);
}
```

The deallocation operation is quite lengthy, and so the `thread_deallocate_safe()` variant is used at the more critical points in the kernel wherein threads are dereferenced. This function merely drops the reference count, and - if the count drops to zero - enqueues the thread on the `thread_deallocate_queue`. The `thread_terminate_daemon()` inspects the queue (which is guarded by the same `thread_terminate_lock`), and calls `thread_deallocate_complete()` for each entry.

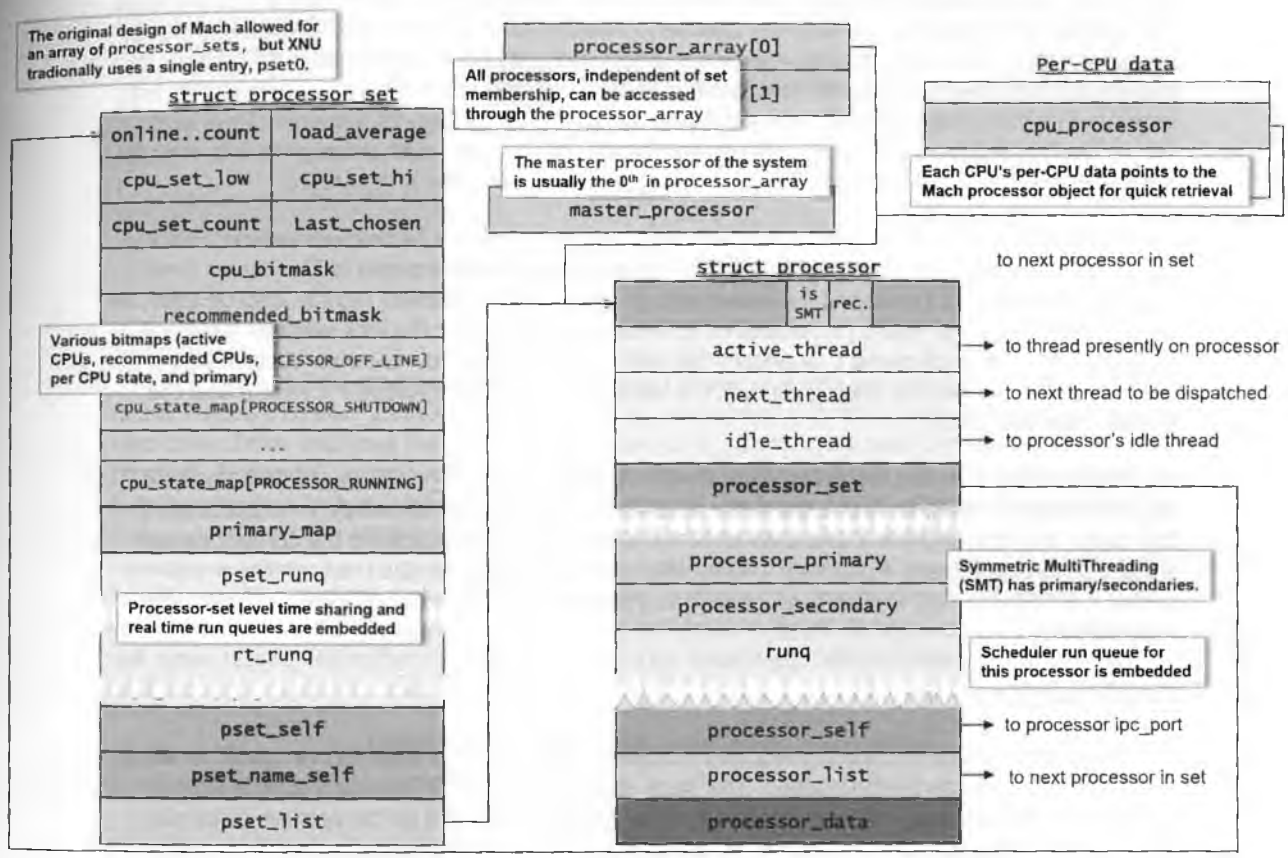
The `thread_deallocate_daemon` also handles work queue deallocation, and (as of Darwin 18), turnstiles which were allocated with `turnstile_deallocate_safe()`. Once all deallocation queues are empty, the `thread_deallocate_daemon` goes back to blocking in its continuation, to be restarted on the next enqueue by `thread_terminate()`, etc.

Processor Management

Recall (from I/12) that Mach provides the `processor` and `processor_set` abstractions. These are architecture independent representations of logical CPUs, which may map to physical CPUs, cores, or hyperthreading. These were previously considered through the eyes of their MIG subsystems, which leave them opaque, outside the provided routines. The scheduler, however, needs to involve itself with the objects' internal structure, which we consider here.

The two objects are amongst Mach's older primitives, but they are anything but static. Just as recently as Darwin 19 both have changed, with new fields added and others changed. The modification of the reduction of the `run_queues` embedded in both has brought the `processor_set` from 2,432 to 1,152 bytes (in all flavors) and the `processor` from 2,768 to 1,480 (in MacOS).

Figure 9-4: The Mach `processor_set` and `processor` object, from Darwin 19



As the figure shows, `processor` objects are readily accessible in a variety of ways: From their containing processor set, or (independent of set membership) through their position in the `processor_array`. The system's `master_processor` is also directly maintained in a global variable, and - most importantly - every `processor` object can be quickly accessed from the physical processor's per-CPU data.

Recall, that Mach allows for multiple processor sets (and processor nodes), which allows to quickly and easily scale up to NUMA and other distributed processing architectures. Apple, however, has neutered this in XNU, relying only on the single `pset0` - returned by the `mach_host` subsystem's `processor_set_default()` MIG routine (#213). There are, at the time of writing, hints that this may finally change - at least in iOS, wherein the A-series processor's cores are asymmetric ("P"erformance cores and "E"fficiency cores), and could thus greatly benefit from a multi-set configuration.

The processor lifecycle is a state machine, first documented in Darwin 13's osfmk/kern/processor.h. The state machine has been updated in Darwin 15 for a power management optimization (allowing a state transition from `DISPATCHING` to `IDLE`), but this is contingent on `CONFIG_SCHED_DEFERRED_AST`, which is FALSE.

Mach Scheduling Enhancements

Mach is quite unlike other operating system kernels, and one of the areas where this is evident (alongside IPC) is scheduling. The original design of Mach leaves in the hands of the micro kernel only the IPC implementation and low level CPU scheduling. For IPC to be efficient, the scheduler must be highly effective - as Mach strives to be.

Handoff

One of the problems faced by developers is that, even when voluntarily yielding execution to another thread, they have little to no control over which thread gets scheduled next. This makes sense, because such a capability could ostensibly be misused by a cadre of threads to take control of the CPU. Mach supports **handoff** in addition to the standard yield. Both calls result in giving up the CPU immediately, but handoff allows the yielding thread to supply the scheduler with a hint as to the next best thread to execute. Note, however, that this is only a hint and does not obligate the scheduler - which may choose to transfer control to some other, possibly higher priority thread. Commonly, though, the hint is acted upon, and the new thread is scheduled. To prevent threads from abusing this mechanism, there's one important condition: Thread handoff requires that the yielding thread not only hand over the CPU, but also its remaining time slice. When the donated time slice ends, regular preemption occurs, and both threads have to wait for whenever their turn arrives next. This protects against misuse of the mechanism.

Handoff in the kernel is performed by a call to `thread_run()` (osfmk/kern/syscall_subr.c), which calls `thread_invoke()` supplying the target thread as an argument. The higher level `thread_handoff_[internal/parameter]()` (as of around Darwin 16) can also be used, as can be seen in `ulock_wait()` (wherein the thread contending for the lock willingly hands off its quantum to the lock owner). Quantum calculations are handled by `thread_dispatch()` (discussed later), which is made aware of the handoff by the `AST_HANDOFF reason` flag the thread is tagged with.

Handoff can even be requested from user mode: The `thread_switch()` Mach trap (#61) supports handoff when its first argument - a SEND right to another thread port - is specified. In this case, the `SEND` right is interpreted as a hint to the scheduler indicating the context switch should be to that thread, if possible. The function pulls the target thread from whatever run queue it is presently on, and calls `thread_run()` (from osfmk/kern/sched_prim.c) to perform the handoff.

Continuations

The classic model of context switching requires every thread to have its own stack, in which the thread context can be maintained. The stack size is normally 16KB (and quadruple that in kernels with KASAN enabled), but this can incur significant overhead as the number of threads increases. Continuations, first devised by Rich Draves[2] (one of Mach's original authors), offer a simple but efficient alternative. A continuation is a function, along with an optional parameter, which is provided as an argument to `kernel_thread_create()`, or to `thread_block[_reason]`. When the thread in question is created or resumed, the continuation function is loaded onto its instruction pointer, along with the optional argument.

The stack space used by a continuation is undefined, and may "piggyback" on any other thread's stack. A continuation must therefore never return - and if it erroneously does the thread will be killed. It may call other functions, but must make sure its flow always ends with a call to `thread_block[_reason]`, and specify itself (or another continuation) as an argument. Whatever context variables are required for the continuation's operation must be supplied in the optional argument, and a continuation may also use any global variables (subject to locking).

As a typical example of a continuation, consider the `vm_pressure_thread` (from osfmk/vm/vm_pageout.c), which is started by `vm_pageout()`, and occasionally woken up by `vm_pressure_response()` to dispatch memory pressure events. The thread has minimal internal state, and so it uses the continuation model from its inception:

Listing 9-5: The `vm_pressure_thread`

```
#if VM_PRESSURE_EVENTS
void
vm_pressure_thread(void) {
        static boolean_t thread_initialized = FALSE;

        if (thread_initialized == TRUE) {
                vm_pressure_thread_running = TRUE;
                consider_vm_pressure_events();
                vm_pressure_thread_running = FALSE;
        }

        thread_initialized = TRUE;
        assert_wait((event_t) &vm_pressure_thread, THREAD_UNINT);
        thread_block((thread_continue_t)vm_pressure_thread);
}
#endif /* VM_PRESSURE_EVENTS */

void
vm_pageout(void) {
...
#if VM_PRESSURE_EVENTS
   result = kernel_thread_start_priority((thread_continue_t)vm_pressure_thread,
                NULL,
                BASEPRI_DEFAULT,
               &thread);

   if (result != KERN_SUCCESS)
            panic("vm_pressure_thread: create failed");

        thread_deallocate(thread);
#endif
```

The Mach thread structure holds any set `continuation` and `parameter` in the respective fields. Mach's `thread_invoke()` and `thread_continue()` (both in osfmk/kern/sched_prim.c) take these fields and pass them in arguments to `call_continuation`, a machine specific function. Figure 9-6 juxtaposes the continuation call in ARMv8 and x86_64. The minor difference between the two is that in the x86_64 case the kernel stack top can be obtained directly from %gs, whereas in the ARMv8 case it is passed as a a parameter (from `current_thread()->machine.kstackptr`, by a wrapper). Note in both cases the flow is to call the continuation function pointer, and the code after it is hopefully unreachable in both cases, and would terminate the thread if somehow reached. This will never happen, though, as the continuation will never return, but rather `call_continuation` recursively. The recursion never unwinds, and the stack pointer is reset every time, so there is no risk of a stack overflow.

Listing 9-6: Comparing continuations on x86_64 and ARM64

```
/*   void Call_continuation( void (*continuation)(void),
 *                           void *param,
 *                           wait_result_t wresult,
 *                           vm_offset_t stack_ptr)
 */
movq %rdi,%rcx  /* get continuation   */
movq %rsi,%rdi  /* continuation param */
movq %rdx,%rsi  /* wait result */

movq %gs:CPU_KERNEL_STACK,%rsp
xorq %rbp,%rbp  /* zero frame pointer */

call *%rcx    /* call continuation */
movq %gs:CPU_ACTIVE_THREAD,%rdi
call EXT(thread_terminate)
```

```
mrs  x4, TPIDR_EL1          // Get current thread pointer
ldr  x5, [x4,TH_KSTACKPTR] // Get top of kernel stack
mov  sp, x5                 // Set stack pointer
mov  fp, xzr                // Clear the frame pointer

mov  x4, x0                 // Load the continuation
mov  x0, x1                 // Set the first parameter
mov  x1, x2                 // Set the wait result arg

blr  x4                     // Branch to the continuation
mrs  x0, TPIDR_EL1          // Get current thread pointer
b    EXT(thread_terminate) // Kill thread (unreachable)
```

Most kernel threads (q.v. Table 5-34) use continuations regularly, and can easily be seen in the output of `procexp 0 threads` (which also symbolicates them). A non-zero value in the `thread_snapshot`'s `ths_continuation` field, along with no reported kernel frames, both indicate use of this model. Mach's message receiving functionality may at times use a continuation, as can its semaphores. These use different fields (`ith_continuation` and `sth_continuation`, respectively).

Continuations are also used in the BSD layer - as discussed in Chapter 6, the `struct uthread` holds a `uu_continuation`, which is used to hold the continuation on return from `_sleep` (in bsd/kern/kern_synch.c), which itself uses `_sleep_continue`). The `select(2)` and `sigsuspend(2)` system calls are examples of continuation usage (through the `tsleep[0/1]` and `msleep0` wrappers), though their continuations are used only once, to return to user space via `unix_syscall_return()`.

Asynchronous Software Traps (AST)

Asynchronous Software Traps, (also known as **Asynchronous System Traps**, and commonly referred to by their acronym, **ASTs**), are a software-based preemption mechanism whose origins can be traced to the late Digital VMS operating system. Though that OS is long gone, the mechanism is still in use in some BSD variants, and - of course - XNU. An AST is somewhat similar to a UNIX signal, in that it is an indication of one or more events which are used internally by the system. An AST is signalled (or "raised") when the event happens, and are to be handled (or "taken") by the system as early as possible.

The user mode return checks are hard-coded into the assembly level code, but other preemption codes are implemented by a call to `cause_ast_check()`. This machine-dependent call (implemented in osfmk/i386/mp.c or osfmk/arm64/machine_routines.c) takes the processor as an argument, and - if it is not the current processor - signals the CPU core that an AST is requested, and emits a kdebug code (`MACH_SCHED_REMOTE_AST`).

The CPU signaling is also machine dependent:

- **On x86_64**: `i386_signal_cpu` is called with `MP_AST` bit, and generates an Inter-Processor-Interrupt (IPI) using `i386_cpu_IPI`, which sets the `MP_AST` bit in the per-CPU `cpu_signals`, and sends the IPI using `lapic_send_ipi()` (from osfmk/i386/lapic_native.c) . This triggers `cpu_signal_handler` to be called, retrieving the `cpu_signals` value from the local `cpu_data_ptr`. On seeing the `MP_AST` bit, it calls `ast_check()`, and the AST can be handled.
- **On ARM (both v7 and v8)**: `cpu_signal` is called with `SIGPast`. The target processor's `cpu_signal` is atomically exchanged with its value, and the `SigPast` bit lit. A `DSB` instruction (Data Synchronization Barrier) ensures that the mask is available to other cores before the IPI is issued, using the Platform expert's `PE_cpu_signal` (actually in iokit/Kernel/IOCPU.cpp, implemented by the `AppleARMPlatform` kext).

Setting or retrieving the ASTs is similar: The `ast_pending(void)` function (in osfmk/i386/cpu.c or osfmk/arm/cpu_common.c) retrieves the address of the `cpu_pending_ast` from the per-processor data struture, after which the machine independent `ast_[on/off](ast_t)` (from osfmk/kern/ast.c) can be used to directly set or clear the AST reasons. The ASTs set at the thread level can be set at the processor level by a call to `ast_propagate(thread)` (ibid.). Normally ASTs are raised with `[thread/act]_set_ast`, which also causes a check, but the `act_set_ast_async` variant can skip the check, which defers the AST handling till the next quantum expiration or context switch.

Handling ASTs

Rather than rely on a hardware trap or interrupts, ASTs are checked at fixed preemption points throughout kernel code:

- On return from interrupt
- On return to user mode
- At thread blocking points

The actual handling of the AST is performed by either `ast_taken_kernel()` or `ast_taken_user()` (both in osfmk/kern/ast.c). The former only handles `AST_URGENT` or `AST_PREEMPT` ASTs, but the latter can handle all the others, with interrupts enabled. Taking an AST calls `ast_consume` (to return and reset the AST bits which were set), and then call the corresponding handler, dependent on the AST reason code.

AST reasons

The reasons for AST are defined as bits in osfmk/kern/ast.h. The number of reasons was fairly stable until Darwin 13, when Apple started adding more low level functions to the scheduler, and has been increasing steadily since. Listing 9-7 (next page) details the reasons defined in Darwin 18. Not all reasons are actual AST's - the bold reasons are ones used as indicators in the thread's `reason` field, rather than actual CPU signaled ASTs. The indicators are used by the scheduler to implement quantum expiration and handoff functionality.

Listing 9-7: AST reasons defined in osfmk/kern/ast.h, annotated

```
/*
 *      Bits for reasons
 *      TODO: Split the context switch and return-to-user AST namespaces
 *      NOTE: Some of these are exported as the 'reason' code in scheduler tracepoints
 */
define AST_PREEMPT              0x01    // Thread preemption in osfmk/kern/sched_prim.c
define AST_QUANTUM              0x02    // Quantum expiration from thread_dispatch (sched_prim.c)
define AST_URGENT               0x04    // Realtime/urgent thread dispatching in (sched_prim.c)
define AST_HANDOFF              0x08    // Thread handoff (thread_run) in (sched_prim.c)
define AST_YIELD                0x10    // Voluntary context switch from osfmk/kern/syscall_subr.c
define AST_APC                  0x20    /* migration APC hook */
define AST_LEDGER               0x40    // Ledger events (act_set_astledger)
define AST_BSD                  0x80    // Reserved to start the init_task
define AST_KPERF                0x100   /* kernel profiling */
define AST_MACF                 0x200   /* MACF user ret pending */
define AST_CHUD                 0x400   // Removed in Darwin 18
define AST_CHUD_URGENT          0x800   // Removed in Darwin 18
// XNU 2422
define AST_GUARD                0x1000 // thread_guard_violation in osfmk/kern/thread.c
define AST_TELEMETRY_USER       0x2000 /* telemetry sample requested on interrupt from userspace */
define AST_TELEMETRY_KERNEL     0x4000 /* telemetry sample requested on interrupt from kernel */
// XNU 2782
define AST_SFI                 0x10000 /* Evaluate if SFI wait is needed before return to userspace
// XNU 3789
define AST_DTRACE              0x20000 // Used by DTrace when probes are active
define AST_TELEMETRY_IO        0x40000 /* telemetry sample requested for I/O */
// XNU 4570
define AST_KEVENT              0x80000  // Used by kqueue workloops (bsd/kern/kern_event.c)
define AST_REBALANCE          0x100000  /* thread context switched due to rebalancing */
// XNU 4903
define AST_UNQUIESCE          0x200000 /* catch unquiesced processor before returning to userspace

define AST_NONE                    0x00
define AST_ALL                     (~AST_NONE)

define AST_PREEMPTION  (AST_PREEMPT | AST_QUANTUM | AST_URGENT)
define AST_SCHEDULING  (AST_PREEMPTION | AST_YIELD | AST_HANDOFF)

define AST_TELEMETRY_ALL (AST_TELEMETRY_USER | AST_TELEMETRY_KERNEL | \
                AST_TELEMETRY_PMI | AST_TELEMETRY_IO)

/* Per-thread ASTs follow the thread at context-switch time. */
define AST_PER_THREAD  (AST_APC | AST_BSD | AST_MACF | AST_LEDGER | AST_GUARD | \
                        AST_TELEMETRY_ALL | AST_KEVENT)
```

AST_PREEMPTION

The `AST_PREEMPTION` reason is a collective name for `AST_PREEMPT | AST_QUANTUM | AST_URGENT`. `AST_PREEMPT` code is used to indicate thread preemption. Handling this AST calls `csw_check`, and `thread_block_reason`, which suspends the currently executing thread. `AST_URGENT` may be used along with `AST_PREEMPT`, which indicates the thread needs to be preempted immediately. Preemptions flagged with `AST_URGENT` are handled before all other ASTs, immediately blocking the current thread. The `AST_QUANTUM` code indicates thread quantum expiration, as set when `thread_quantum_expire()` gets called due to `quantum_timer_expire()` being invoked (from the quantum timer thread call).

AST_SCHEDULING

The `AST_SCHEDULING` reason is a collective name for `AST_YIELD`, `AST_HANDOFF` and `AST_PREEMPTION`. `AST_YIELD` and `AST_HANDOFF` are two similar reasons, indicating the current thread has either yielded execution unconditionally, or asked to yield so that another specific thread be able to run in its stead, and yield its remaining quantum. When any of the `AST_SCHEDULING` reasons are active, `kperf_lazy_cpu_sample()` is called to take a kperf sample.

AST_LEDGER (Darwin 12)

Ledgers are discussed later in this chapter. The AST_LEDGER reason is signaled when a ledger resource has been depleted, or when the cpumon facility is enabled or suspended on a task (by iterating over all its threads), in which case the CPU limit may be imposed (or lifted). This AST is serviced by ledger_ast(thread) (in osfmk/kern/ledger.c). If a resource has been depleted, a refill is scheduled, and the thread is blocked until the resource becomes available.

AST_MACF

The AST_MACF reason is reserved to allow the MAC Framework to perform an operation upon a thread returning to user space. When the AST is claimed, mac_thread_userret() (from security/mac_mach.c) calls the MAC_PERFORM macro to allow interested MACF policies to hook the operation. In practice, this remains unused.

AST_BSD

The AST_BSD reason is signaled by a call to act_set_astbsd() at various process lifecycle points revolving around signal handling. Taking this AST results in a call to bsd_ast(thread) (from bsd/kern/kern_sig.c). This function handles various housekeeping tasks for the BSD layer, such as updating timers (TASK_VTIMER_USER/PROF/RLIM) and firing their corresponding signals (SIG[VTALRM/PROF/XCPU], respectively). In somewhat of a hack, bsd_ast is also responsible for starting the bsdinit_task() - PID 1 - the very first time it fires. Initially, AST_BSD was marked as temporary, but its author (JMM) must have eventually given up as the comment was removed and AST_BSD lingers on.

AST_GUARD (Darwin 13+)

The AST_GUARD is signaled to the thread when a thread_guard_violation() (in osfmk/kern/thread.c) is indicated on access to a guarded resource - by [mach_port/fp/vn]_guard_exception(). The thread_guard_violation() sets AST_GUARD and propagates it to the current core's cpu_pending_ast.

Taking this AST results in a call to guard_ast() (from osfmk/kern/thread.c), which retrieves the exception code from the thread's guard_exc_info field, and deduces the guard violation - GUARD_TYPE_MACH_PORT, ..FD or (#if CONFIG_VNGUARD) ...VN, and transfers through [mach_port/fd/vn]_guard_ast, all of which use task_exception_notify() to raise the EXC_GUARD exception and terminate the process with a SIGKILL.

AST_TELEMETRY[_USER/KERNEL/IO] (Darwin 13+)

The AST_TELEMETRY[_USER/KERNEL] types are signaled from telemetry_mark_curthread() on an interrupt if the global telemetry_needs_record is TRUE (and #if CONFIG_TELEMETRY, which is set by default). This global is toggled by compute_telemetry (from osfmk/kern/telemetry.c), which is called periodically by the scheduler as one of its sched_average computations. A call to telemetry_ast (also from telemetry.c) sets microstackshot flags and calls on telemetry_take_sample (ibid.) to take a sample into the microstackshot buffer, which can then be picked up from user space by a call to stack_microstackshot(..., STACKSHOT_GET_MICROSTACKSHOT,...).

AST_SFI (Darwin 14+)

AST_SFI was introduced to support Selective forced idle, as described later in this chapter.

AST_DTRACE (Darwin 16+)

The AST_DTRACE reason is serviced by dtrace_ast() (in bsd/dev/dtrace/dtrace.c). The AST is set when a probe readies its output buffer. When taken, dtrace_ast() disables preemption, and proceeds to wake up all the registered DTrace clients of this process.

AST_KEVENT (Darwin 17+)

The AST_KEVENT has associated kevent_ast_bits, which are stored as a field on the Mach thread. ast_taken_user() retrieves and clears these bits by an atomic_exchange(), and then passes them to kevent_ast(), in bsd/kern/kern_event.c. The bits may presently indicate only two values - AST_KEVENT_REDRIVE_THREADREQ and AST_KEVENT_RETURN_TO_KERNEL. The former results in a call to the PThread kext's implementation of workq_threadreq with WORKQ_THREADREQ_REDRIVE.

AST_REBALANCE (Darwin 17+)

As explained later, the scheduler's thread_select() evaluates if the current thread may continue to execute, or if it must be forced to context switch, due to one of the several criteria. If any of these hold (needs_smt_rebalance || affinity_mismatch || bound_elsewhere || avoid_processor), AST_REBALANCE is signaled. This causes thread_dispatch to set SCHED_REBALANCE (and possibly SCHED_PREEMPT, if AST_QUANTUM was also indicated), leading to a corresponding IPI to kick off the preemption and/or rebalancing.

AST_UNQUIESCE (Darwin 18+)

Darwin 18 adds a new CPU quiescent counter, to track CPU transitions in and out of idle state. When a CPU becomes idle or is woken from an interrupt, it either leaves or joins the counter mechanism. When the CPU is active, it is expected to check-in, and when all CPUs have checked in the counter value (in the commpage) is incremented. The code dealing with the AST and checkin bitmasks is self contained in osfmk/kern/cpu_quiesce.c.

Mach Schedulers

Scheduling in any operating system is one of its most critical functions, with a direct impact on performance. What complicates matters further is that there is no one right way to do so, which would fit all cases. As it turns out - there's a Mach abstraction for that - the Mach scheduler object.

Mach uses the scheduler object to maintain the logic behind thread scheduling. This allows multiple schedulers (with multiple scheduling paradigms) to exist, and indeed - XNU presently provides no less than 7 schedulers, each in its respective file (all in osfk/kern/sched_*), showin Table 9-8. In practice, however, only one chosen scheduler is hardcoded into the kernel. The choice could be overridden at boot by specifying the `sched` boot-argument, but as of XNU 4570 this is no longer possible outside of debug kernels. Further, the schedulers are dependent on `CONFIG_SCHED_*` `#defines`.

Table 9-8: Schedulers found in Darwin 19 (XNU 6153)

Scheduler	#define	Description
traditional	`..._TRADITIONAL`	....
traditional_with_pset_runqueue		
proto	`(unused)`	Global runqueue with alternating processors
grrr	`..._GRRR_CORE`	Group Ratio Round-Robin
Darwin 14 (XNU 2782)		
dualq	`..._MULTIQ`	Uses per-processor (bound) and processor-set run queues
multiq		As `dualq`, with scheduler group functionality
Darwin 19 (*OS only)		
amp	`__AMP__`	P/E core aware scheduling

Darwin versions before Darwin 17 use `multiq`, but Darwin 18 shifts to `dualq`. *OS 13 variants use a new scheduler called AMP, which takes into account the core type (Performance or efficiency) as well. The `kern.sched` `sysctl(8)` MIB will show the currently active scheduler.

All Mach schedulers "plug in" to the scheduler primitives defined in osfmk/kern/sched_prim.c. This effectively decouples the precise decision making logic from the general act of scheduling, and standardizes the flow of scheduling up to the points where decisions have to be made. At those points, the scheduler primitives (or other code in the kernel) calls on the chosen scheduler to make the decision via a well defined callback. The scheduling primitives neither know nor care about the decision making - but are nonetheless bound by it. Likewise the chosen scheduler interfaces with the Mach core's standard objects - first and foremost of which is the Run Queue.

The Run Queue

A **run queue** is a fundamental concept in scheduling, providing the representation of runnable threads with some ordering that takes into account the thread priority, ordering (usually, FIFO), and possibly other scheduling constraints. Mach's representation is the `struct run_queue` defined in osfmk/kern/sched.h as follows:

Listing 9-9: The Mach `struct run_queue` (from osfmk/kern/sched.h)

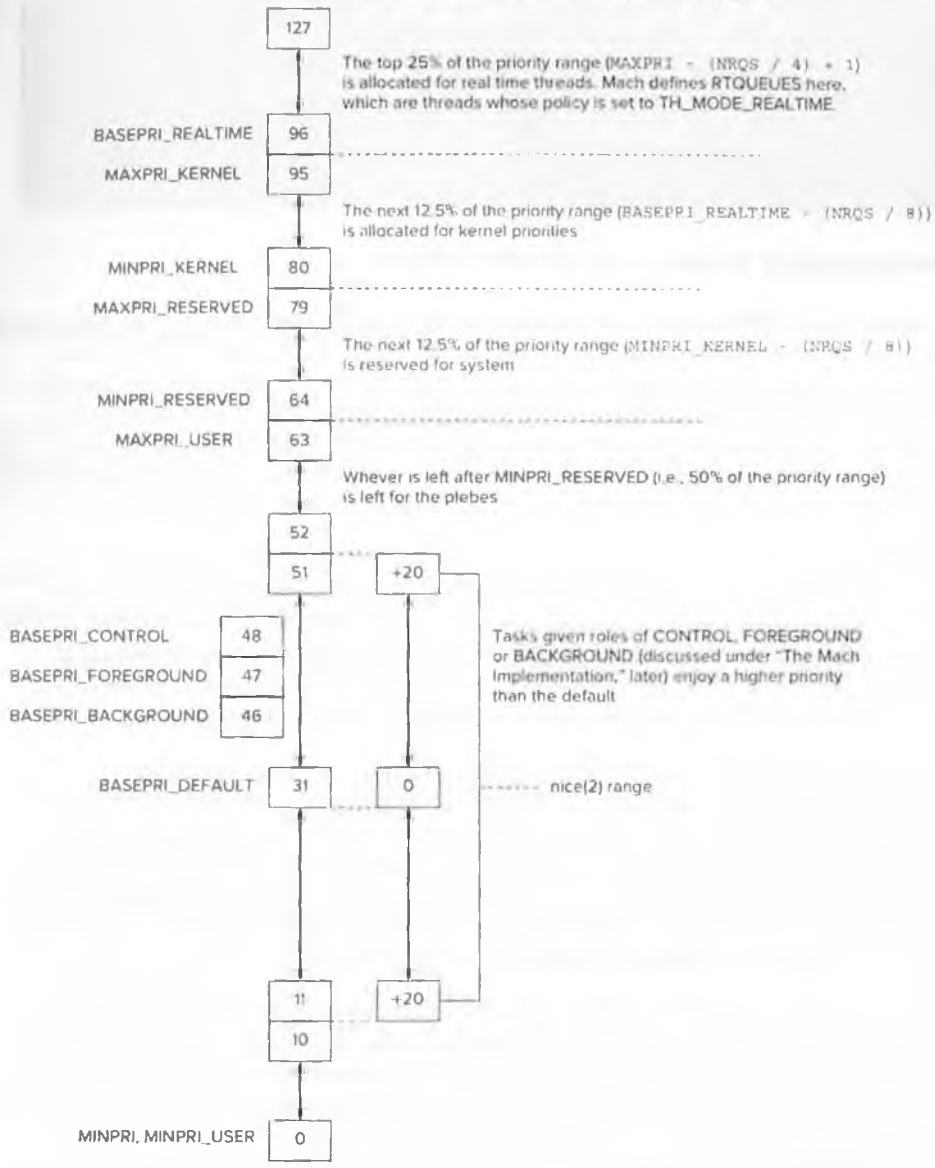
```
struct run_queue {
    int             highq;                  /* highest runnable queue */
    bitmap_t        bitmap[BITMAP_LEN(NRQS)]; /* run queue bitmap array */
    int             count;                  /* # of threads total */
    int             urgency;                /* level of preemption urgency */
    queue_head_t    queues[NRQS];           /* one for each priority */

    struct runq_stats  runq_stats;
};
```

If the runqueue array were the only component of the data structure, finding the next available thread would be O(`NRQS`). The operation can be sped up considerably by the other fields of the `struct run_queue`: The `count` holds the number of runnable threads in whichever of the queues. The highest priority non-empty queue index is maintained in `highq`. Finally, using the `bitmap` enables finding the next non-empty queue in under O(NRQS/64) operations - merely two operations in the worst case.

Priorities

Threads are queued in one of the `NRQS` `queues`, in FIFO ordering. `NRQS` is `#defined` (up until Darwin 18) to be 128, providing a wide gamut of priorities which is carved into priority ranges, as shown in Figure 9-10:

Figure 9-10: The Mach Multi-Level Feedback Run Queue



User-mode threads are given the bottom half of the range (i.e. 0-63). Those threads with the standard threading policy are granted `BASEPRI_DEFAULT` at the half-way point of that, i.e. the low quarter of the range, or 31. The traditional `nice(1)` gamut is mapped from the `BASEPRI_DEFAULT` - so as to span 11 through 51, with the negative (and thus, not-so) nice values end up increasing the priority, whereas normal nice values lower it. This can be programmatically achieved through the `setpriority(2)` system call, which is further extended with `PRIO_DARWIN_[THREAD/PROCESS]` to allow process background priorities, which are useful on *OS. These further depress the priority into the doldrums of the 0 through 10 range. User-mode threads can further request elevated priorities between 52 and 63.

In the top half of the range, the bottom quarter (i.e. 64 through 79) is marked as `[MIN/MAX]PRI_RESERVED` and is seldom used. The next quarter (80 through 95) is `[MIN/MAX]PRI_KERNEL`, and most kernel threads indeed live within that range. The remaining half (96 through 127) is reserved for realtime threads, which (as of Darwin 19) use a separate run queue mechanism, and therefore do not participate in the bitmap.

Priorities of processes (= main threads) can be viewed by supplying `-O pri` to `ps(1)`, or with the `-l` switch, which includes it along with other common options. To see into all threads of the processes in the output, use `-M` switch. A good combination of switches to try is `ps -Me`, to get the listings of all processes, their respective threads, with the priority information, as shown in Output 9-11:

Output 9-11: The output of `ps -Me` showing all user-mode threads on a system

```
morpheus@Chimera (~)$ ps -Me
USER               PID   TT   %CPU STAT PRI     STIME     UTIME COMMAND
root                 1   ??    0.0 S    31T   0:00.06   0:00.02 /sbin/launchd
                     1         0.0 S    37T   0:00.27   0:00.03
root                55   ??    0.0 S     4T   0:02.89   0:00.93 /usr/sbin/syslogd
                    55         0.0 S     4T   0:00.00   0:00.00
                    55         0.0 S     4T   0:00.00   0:00.00
                    55         0.0 S     4T   0:00.00   0:00.00
                    55         0.0 S     4T   0:00.00   0:00.00
...
```

Load Average/Mach Factor, and Priority Shifts

A key metric in any UN*X system is its **load average**, which is reported by commands such as `w(1)` and `uptime(1)`. The load average counts the number of runnable threads, which enables the user to determine how light or heavy the system's work load is - the smaller the better, and best when lower than the number of available (logical) CPUs. Mach computes the **Mach factor**, which subtracts the number of threads from the number of cores available, or (if more threads than cores) returns the ratio of cores to threads. The Mach factor can be retrieved using `hostinfo(1)` (which itself uses the `host_info` MIG routine, #200 in the `mach_host` subsystem).

There is more to the load average that just informing the user of system load. XNU also calculates a more fine grained scheduler load, which it uses to implement **priority shifts**. This enables tweaking the priorities of each thread, so as to accommodate for the thread's CPU usage, and overall system load. Threads can thus "drift" in their priority bands, decreasing in priority when using the CPU too much, and increasing in priority if not getting enough CPU.

The `update_priority()` scheduler function (in osfmk/kern/priority.c) and its `lightweight_..` variant are shared by most scheduler implementations, and used to dynamically update the priority of threads whose quantum expires. This toggles the thread priority by subtracting its calculated `sched_usage` (calculated by the function, accounting for CPU usage delta), shifted by a `pri_shift value`. The `pri_shift` value is derived from the global `sched_pri_shift`, which is updated by the scheduler regularly as part of the system load calculation in compute_averages (osfmk/kern/sched_average.c).

Subtracting the CPU usage delta effectively penalizes those threads with high CPU usage (positive usage delta detracts from priority) and rewards those of low CPU usage (negative usage delta adds to priority). To make sure the thread's CPU usage doesn't accrue to the point where the penalty is lethal, the update_priority function gradually ages CPU usage. It makes use of a sched_decay_shifts structure, to simulate the exponential decay of the CPU usage by a factor of $(^{5n}/_{8})$, defined (ibid.) as shown in Listing 9-12. By using the pre-computed shift values, the computation can be sped up, expressed in terms of bit shifts and additions, which take less time than multiplication. The tick decay can be artificially accelerated by setting the `sched_decay_usage_age_factor` boot argument, which causes the tick count to be multiplied before being evaluated against `SCHED_DECAY_TICKS`.

Listing 9-12: The `sched_decay_shifts` structure (from osfmk/kern/priority.c)

```
/*
 *      Define shifts for simulating (5/8) ** n
 *
 *      Shift structures for holding update shifts.  Actual computation
 *      is  usage = (usage >> shift1) +/- (usage >> abs(shift2))  where the
 *      +/- is determined by the sign of shift 2.
 */
struct shift_data {
        int     shift1;
        int     shift2;
};

#define SCHED_DECAY_TICKS       32
static struct shift_data        sched_decay_shifts[SCHED_DECAY_TICKS] = {
        {1,1},{1,3},{1,-3},{2,-7},{3,5},{3,-5},{4,-8},{5,7},
        {5,-7},{6,-10},{7,10},{7,-9},{8,-11},{9,12},{9,-11},{10,-13},
        {11,14},{11,-13},{12,-15},{13,17},{13,-15},{14,-17},{15,19},{16,18},
        {16,-19},{17,22},{18,20},{18,-20},{19,26},{20,22},{20,-22},{21,-27}
};
```

Scheduling buckets and the EWMA

As of Darwin 16 (XNU-3789), XNU uses `sched_bucket_ts` to calculate the scheduler load. There are presently six such buckets. The first two (`TH_BUCKET_RUN` and `..FIXPRI`) hold runnable and fixed-priority threads. The other four carve out the priority space by dividing it along the lines of `BASEPRI_DEFAULT`, `BASEPRI_UTILITY`, `MAXPRI_THROTTLE` and `MINPRI`.

XNU's averages was further tweaked in Darwin 18 to an **Exponentially Weighted Moving Average** algorithm (EWMA). The algorithm is well described in the header comment in sched_average.c:

Listing 9-13: The scheduler load calculation algorithm (from XNU 4903's osfmk/kern/sched_average.c)

```
/*
 * Scheduler load calculation algorithm
 *
 * The scheduler load values provide an estimate of the number of runnable
 * timeshare threads in the system at various priority bands. The load
 * ultimately affects the priority shifts applied to all threads in a band
 * causing them to timeshare with other threads in the system. The load is
 * maintained in buckets, with each bucket corresponding to a priority band.
 *
 * Each runnable thread on the system contributes its load to its priority
 * band and to the bands above it. The contribution of a thread to the bands
 * above it is not strictly 1:1 and is weighted based on the priority band
 * of the thread. The rules of thread load contribution to each of its higher
 * bands are as follows:
 *
 * - DF threads: Upto (2 * NCPUs) threads
 * - UT threads: Upto NCPUs threads
 * - BG threads: Upto 1 thread
 *
 * To calculate the load values, the various run buckets are sampled (every
 * sched_load_compute_interval_abs) and the weighted contributions of the the
 * lower bucket threads are added. The resultant value is plugged into an
 * exponentially weighted moving average formula:
 *      new-load = alpha * old-load + (1 - alpha) * run-bucket-sample-count
 *      (where, alpha < 1)
 * The calculations for the scheduler load are done using fixpoint math with
 * a scale factor of 16 to avoid expensive divides and floating point
 * operations. The final load values are a smooth curve representative of
 * the actual number of runnable threads in a priority band.
 */
```

Scheduler dispatch

Mach's generic scheduler abstraction is in osfmk/kern/sched_prim.c, which defines the various primitives and interfaces a scheduler is expected to implement. Chief among those is the **dispatch table**, which holds over two dozen function pointers (which can be thought of as methods) and fields (properties) that all schedulers are meant to provide. A clever macro, `SCHED(f)` is used as an accessor from the dispatch table. This allows for schedulers to be dynamically switched (on boot and, in theory, during runtime) but in XNU 4570 the macro has been changed to statically select the functions ('.') rather than having to dereference the pointers (->) during every context switch. It is this macro (in sched_prim.h) which sets the default (and thus, only) scheduler. The table is anything but constant, as Apple has both added and removed methods and properties, including a whole new set added in XNU 4570.

Listing 9-14: The `sched_dispatch_table` struct (from osfmk/kern/sched_prim.h)

```
struct sched_dispatch_table {
   const char *sched_name;
   void  (*init)(void);                                  /* Init global state */
   void  (*timebase_init)(void);                         /* Timebase-dependent initialization */
   void  (*processor_init)(processor_t processor);       /* Per-processor scheduler init */
   void  (*pset_init)(processor_set_t pset);             /* Per-processor set scheduler init */
   void  (*maintenance_continuation)(void);              /* Function called regularly */

   /** Choose a thread of greater/equal priority from per-processor runqueue
     * for timeshare/fixed threads
     */
   thread_t        (*choose_thread)(processor_t   processor,
                                    int           priority,
                                    ast_t         reason);

   /* True if scheduler supports stealing threads */
   boolean_t   steal_thread_enabled;

   /* Steal a thread from another processor in the pset so that it can run immediately */
   thread_t        (*steal_thread)(processor_set_t pset);
   /* Compute priority for a timeshare thread based on base priority.  */
   int (*compute_timeshare_priority)(thread_t thread);

   /* Pick the best processor for a thread (any kind of thread) to run on.  */
   processor_t     (*choose_processor)(processor_set_t  pset,
                                       processor_t      processor,
                                       thread_t         thread);
   /* Enqueue a timeshare or fixed priority thread onto the per-processor runqueue */
    boolean_t (*processor_enqueue)(processor_t           processor,
                                       thread_t             thread,
                                       integer_t            options);

   /* Migrate threads away in preparation for processor shutdown */
   void (*processor_queue_shutdown)(processor_t         processor);

   /* Remove the specific thread from the per-processor runqueue */
   boolean_t       (*processor_queue_remove)(processor_t  processor,
                                             thread_t     thread);
   /** Does the per-processor runqueue have any timeshare or fixed priority
     * threads on it? Called without pset lock held, so should
     * not assume immutability while executing.
     */
   boolean_t       (*processor_queue_empty)(processor_t   processor);

   /** Would this priority trigger an urgent preemption if it's sitting
     * on the per-processor runqueue?
     */
   boolean_t       (*priority_is_urgent)(int priority);

   /** Does the per-processor runqueue contain runnable threads that
     * should cause the currently-running thread to be preempted?
     */
   ast_t           (*processor_csw_check)(processor_t processor);

   /** Does the per-processor runqueue contain a runnable thread
         * of > or >= priority, as a preflight for choose_thread() or other
         * thread selection
         */
        boolean_t    (*processor_queue_has_priority)(processor_t  processor,
                                                     int          priority,
                                                     boolean_t    gte);
```

Listing 9-14 (cont): The `sched_dispatch_table` struct (from osfmk/kern/sched_prim.h)

```
        /* Quantum size for the specified non-realtime thread. */
        uint32_t        (*initial_quantum_size)(thread_t thread);

        /* Scheduler mode for a new thread */
        sched_mode_t    (*initial_thread_sched_mode)(task_t parent_task);

        /* Is it safe to call update_priority, which may change a thread's
         * runqueue or other state. This can be used to throttle changes
         * to dynamic priority.
         */
        boolean_t       (*can_update_priority)(thread_t thread);

        /* Update both scheduled priority and other persistent state.
         * Side effects may including migration to another processor's runqueue.
         */
        void            (*update_priority)(thread_t thread);

        /* Lower overhead update to scheduled priority and state. */
        void            (*lightweight_update_priority)(thread_t thread);

        /* Callback for non-realtime threads when the quantum timer fires */
        void            (*quantum_expire)(thread_t thread);

        /* Runnable threads on per-processor runqueue. Should only
         * be used for relative comparisons of load between processors.
         */
        int                     (*processor_runq_count)(processor_t     processor);

        /* Aggregate runcount statistics for per-processor runqueue */
        uint64_t    (*processor_runq_stats_count_sum)(processor_t   processor);

        boolean_t       (*processor_bound_count)(processor_t processor);

        void            (*thread_update_scan)(sched_update_scan_context_t scan_context);

        /* Use processor->next_thread to pin a thread to an idle
         * processor. If FALSE, threads are enqueued and can
         * be stolen by other processors.
         */
        boolean_t   direct_dispatch_to_idle_processors;

        boolean_t   multiple_psets_enabled;  /* Supports more than one pset */
        boolean_t   sched_groups_enabled;    /* Supports scheduler groups   */
        boolean_t   avoid_processor_enabled; /* Supports avoid-processor    */

        /* Returns true if this processor should avoid running this thread. */
        bool    (*thread_avoid_processor)(processor_t processor, thread_t thread);

// XNU 4570 additions

    /* Invoked when a processor is about to choose the idle thread
     * Used to send IPIs to a processor which would be preferred to be idle instead.
     * Called with pset lock held, returns pset lock unlocked.
     */
    void                (*processor_balance)(processor_t processor, processor_set_t pset);
    rt_queue_t          (*rt_runq)(processor_set_t pset);
    void                (*rt_init)(processor_set_t pset);
    void                (*rt_queue_shutdown)(processor_t processor);
    void                (*rt_runq_scan)(sched_update_scan_context_t scan_context);
    int64_t             (*rt_runq_count_sum)(void);

    uint32_t            (*qos_max_parallelism)(int qos, uint64_t options);
    void                (*check_spill)(processor_set_t pset, thread_t thread);
    sched_ipi_type_t    (*ipi_policy)(processor_t, thread_t, boolean_t dst_idle, sched_ipi_event_t);
    bool                (*thread_should_yield)(processor_t processor, thread_t thread);

// XNU 6153 additions

    uint32_t (*run_count_incr)(thread_t);
    uint32_t (*run_count_decr)(thread_t);
    void     (*update_thread_bucket) (thread_t);
    void     (*pset_made_schedulable) (processor_t, processor_set_t, boolean);
#if CONFIG_EMBEDDED
    void     (*unknown)(void *, uint32_t); // Wait for sources to see what this one is...
#endif
};
```

Thanks to the `sched_dispatch_table`, all scheduler implementation provide the same delegate methods and have exactly the same lifecycle, driven by the scheduling primitives in osfmk/kern/sched_prim.c. Schedulers are free to implement whatever callbacks they see fit, or call on the default implementations (also in osfmk/kern/sched_prim.c). Most of the `dispatch_table` callbacks are shown in their context in Figure 9-15 (next page). The Figure still leaves out some of the more complicated flows, discussed next.

`thread_select`

If the next thread to execute is known (for example, as a result of a handoff), it can be run directly through a call to the `thread_invoke()` primitive (which also supports `thread_[run/handoff]`). In cases where the next thread needs to be determined (or if a handoff has failed), the scheduler has to engage in the complex logic of finding out which will be the next thread to run. This is the role of the `thread_select()` scheduler primitive.

The first stage of thread selection is checking the processor we are managing. The processor object holds an `is_recommended` field, which is usually `true`, but may be set to `false` according to the containing `processor_set`'s `recommended_bitmask`, as discussed later in this chapter. If the `processor is_recommended` is indeed false, only bound threads will be allowed to execute on it, or else it will be assigned its idle thread.

Assuming that is not the case, there is another special case to consider - which is if the currently executing thread is still runnable. For this, several criteria must hold:

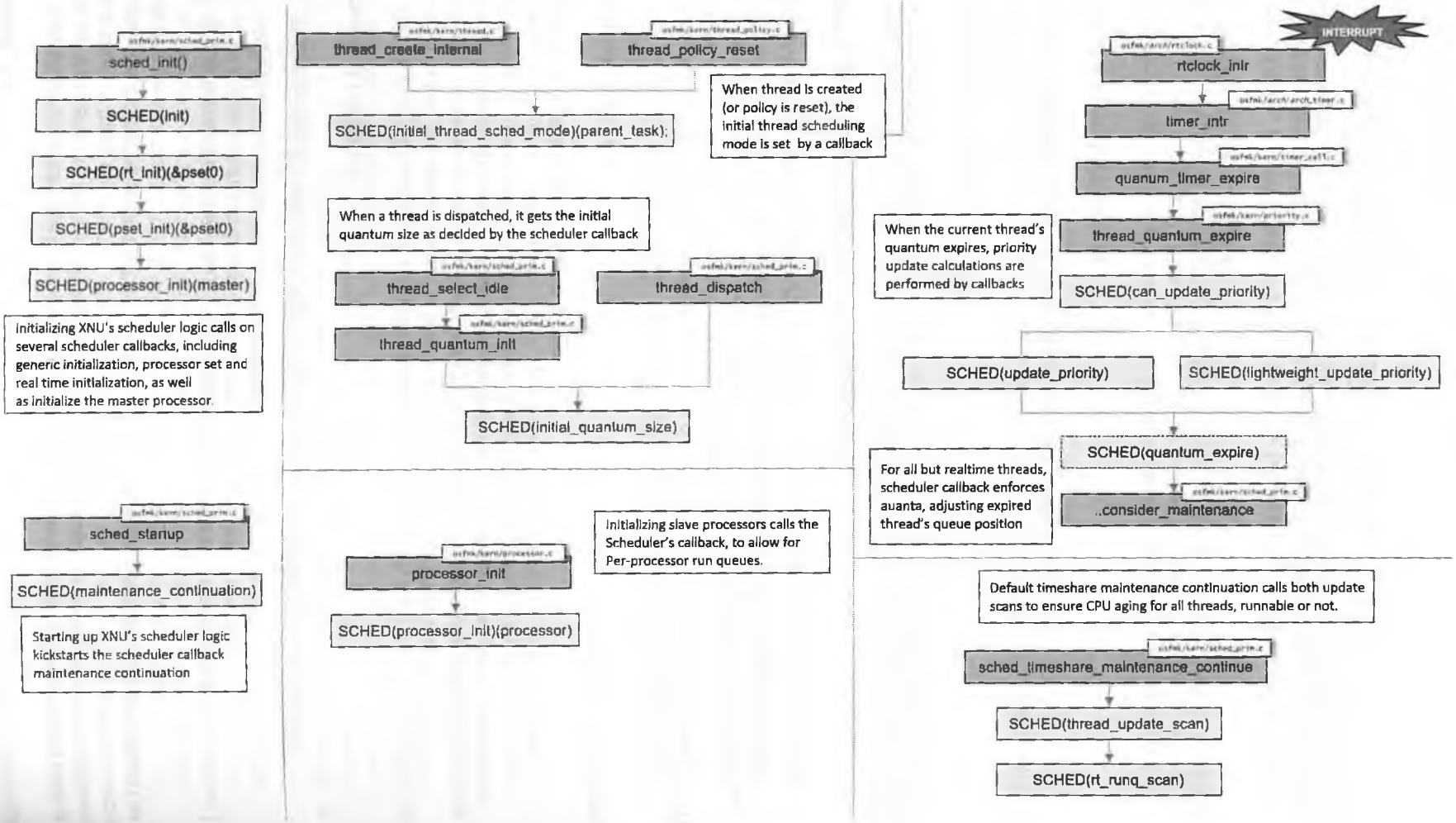
- The thread does not need SMT rebalancing: i.e. the present core is indeed the primary one of this processor, or the thread a realtime thread
- There is no affinity mismatch between the thread's `affinity_set` and that that of the current processor
- The thread isn't bound elsewhere, i.e. to another processor
- The thread isn't set to avoid this particular processor. This test is left to the chosen scheduler to decide, as `thread_select()` will call the scheduler's `avoid_processor_enabled` to see if avoidance is at all supported by it, and - if it is - `thread_avoid_processor()`.

If all the above hold, the thread will be allowed to continue run if it is a real-time (constrained) thread, and there is no other real-time thread with an earlier deadline. The runtime queue check is made by a call to the chosen scheduler's `rt_runq` accessor. If the thread is not real-time, but is still with the highest priority, it will also be allowed to run. Here, too, a call to the chosen scheduler must be made - specifically, to `processor_queue_has_priority`.

Assuming the current thread must be context switched, real-time threads get first dibs on the processor after a context switch. The first real-time thread on the chosen scheduler's `rt_runq` for the processor set will be dequeued and run. Otherwise, the decision must be delegated to the chosen scheduler with a call to `choose_thread()`, providing the current processor as one of the arguments.

If at this point there are no runnable threads on this processor and this is an SMP-built kernel (as all flavors of XNU are by now), the chosen scheduler may support "stealing" threads from other processors in the same processor set. This is checked by a call to its `steal_thread_enabled()` callback, and - if enabled - followed by an attempt to find a thread using the `steal_thread()` callback. Since these operations are lengthy, if no thread could be stolen, the scheduler is consulted again through the `processor_queue_empty` callback, to see if during this time another runnable thread hasn't popped up. If at the end of this process no other thread could be chosen, the default thread which ends up being returned is the processor's `idle_thread`.

Figure 9-15: Scheduler dispatch table callbacks and the scheduling lifecycle



thread_invoke

Selecting the next thread to execute is just one part of the process. Once selected, control has to be transferred to the new thread, as is performed through `thread_invoke()`. The routine takes in three arguments - the `current_thread()`, the new thread to execute, and an AST `reason`. Following several sanity checks and thread state updates, the context switch can ensue:

- **If the targert `thread` has a `continuation`**: It is loaded (along with its `parameter`). `stack_handoff()` is called, and then `thread_dispatch()`, and `call_continuation()` transfers control.
- **If the target thread has a stack, then a full context switch is performed.**

thread_dispatch

The `thread_dispatch()` routine handles the context switch from the old `thread` to the new `self`, when the active context is that of the new thread, yet it has not yet begun executing. This involves updating the quanta of both threads (donating the quantum from one to the other in case of a handoff), freeing the stack(in case continuations are used), performing ledger billing. To make sure the quantum expiration is observed and enforced, `therad_dispatch()` calls `timer_call_quantum_timer_enter()`, which sets the quantum expiry as a timer expiration trigger.

quantum_expire

The `quantum_expire` is called when, following a timer interrupt, the current's CPU quantum deadline is exceeded, indicating that the current thread has outgrown its welcome. The actual behavior in this case is dependent on the scheduler, but the `sched_default_quantum_expire()`, common to all the built-in schedulers, does nothing, and so the schedulers just queue the quantum expired-threads at the tail of the scheduler queue, giving other threads a chance to run first.

update_priority (osfmk/kern/priority.c)

The default `update_priority` implementation (in `osfmk/kern/priority.c`) is shared by the dualq, multiq and the traditional schedulers, as was discussed earlier (under "priority shifts").

sched_maintenance_thread

The `sched_maintenance_thread` is designed to be as lightweight as possible, using a continuation (described earlier in this chapter) to avoid having to maintain a stack.

Multicore considerations

Although Apple's Mac and iDevices are single processor systems, they are all multi-core and/or (in the case of Intel i7) hyperthreaded. As far as the scheduler is concerned, therefore, there are multiple logical processors onto which threads can be dispatched. Each such processor then maintains its own run queue.

On occasion, a given processor's run queue may become busy, while another processor's load might be very light, or even idle. This calls for **rebalancing**, by moving queued threads from busy processor(s) to the less busy one(s), based on the respective run queue lengths.

SMT (Symmetric MultiThreading) is determined by the value of the `kern.sched_enable_smt` MIB:

```
kern.sched_enable_smt: 0x00000001
kern.sched_allow_NO_SMT_threads: 0x00000001
```

Darwin 17 additions

XNU 4570 brought significant enhancements to the `sched_dispatch_table`, by extending the structure to accommodate additional functions:

- **Real time threading support:** has been extended by implementing five more functions. `rt_init` accepts a processor set, and initializes the the runtime thread queue, which may be retrieved by `rt_runq`. The queue scan can be performed by `rt_runq_scan`, and `rt_runq_count_sum` retrieves sum statistics. The queue can also be shutdown with `rt_queue_shutdown`. Real time threads were already supported to an extent in previous versions, but as global functions, and not on a per-scheduler basis. So far, however, all schedulers default to `sched_rtglobal_*` implementations.
- **Multi processor support:** has been extended with three functions. `processor_balance` to act as a load balancer, is currently a null function. `check_spill` (to handle cases where threads "spill" from one processor to another (another null function), and `ipi_policy` to handle Inter-Processor-Interrupts when rebalancing or spilling over threads.
- **Thread yield checks:** are now handled by a new callback, `thread_should_yield`, rather than be hard-coded at respective preemption points in osfmk/kern/syscall_subr.c, namely the `swtch[_pri]` Mach trap implementation and `thread_yield_internal()`.

Darwin 19 additions

XNU 6153 further extends the `sched_dispatch_table` with additional functions to support the following:

- **Run counts** - `run_count_[incr/decr]()`
- **Thread buckets** - `update_thread_bucket()`
- **Multiple processor set support:**: `pset_made_schedulable()`

Darwin 19 also brings a new scheduler to *OS. This scheduler, called AMP, can take advantage of the core type - performance or efficiency, when scheduling threads. This is explained well in the header comment of Kernel.framework's sched_amp_common.h:

Listing 9-16: The description of the AMP scheduler

```
/*
 * The AMP scheduler uses spill/steal/rebalance logic to make sure the most appropriate threads
 * are scheduled on the P/E clusters. Here are the definitions of those terms:
 *
 * - Spill:     Spill threads from an overcommited P-cluster onto the E-cluster.
 *              This is needed to make sure that high priority P-recommended threads experience
 *              low scheduling latency in the presence of lots of P-recommended threads.
 *
 * - Steal:     From an E-core, steal a thread from the P-cluster to provide low scheduling
 *              latency for P-recommended threads.
 *
 * - Rebalance: Once a P-core goes idle, check if the E-cores are running any P-recommended
 *              threads and bring it back to run on its recommended cluster type.
 */
```

Presumably, Apple may separate the Efficiency (E) cores from the Performance (P) cores by assigning them into multiple processor sets, reacting to cores coming on line through the new `pset_made_schedulable()` callback, but that has yet to be observed in iOS 13.0.

Effectuating policy changes

Thread policies may be internal (when the thread requests them for itself) or external (when requested by or for another thread).

After `task..` and `thread_policy_set()` switch over their possible flavors, most cases funnel to a call to `[task/thread]_policy_update_complete_unlocked()`, which take the (unlocked) `task` or `thread` and a `struct task_pend_token`. The token, defined in osfmk/kern/policy_internal.h, is a bit-field with nine bits presently defined. The token is passed around by various policy updaters in osfmk/kern/[thread/task]_policy.c to this function and to its task-level counterpart (`task_policy_update_complete_unlocked`), and according to its bits the updaters can communicate and know which updates are applicable, depending on thread and task lock status. Listing 9-17 shows the token and the updater functions called as a result of setting its bits.

Listing 9-17: The `struct task_pend_token` (osfmk/kern/policy_internal.h)

```
typedef struct task_pend_token {
 uint32_t tpt_update_sockets      :1,  // proc_apply_task_networkbg (thread->task->bsd_info)
          tpt_update_timers       :1,  // ml_timer_evaluate
          tpt_update_watchers     :1,  // apply_appstate_watchers
          tpt_update_live_donor   :1,  // task_importance_update_live_donor(task)
          tpt_update_coal_sfi     :1,  // coalition_for_each_task(task_sfi_reevaluate_cb)
          tpt_update_throttle     :1,  // rethrottle_thread(thread)
          tpt_update_thread_sfi   :1,  // sfi_reevaluate(thread)
          tpt_force_recompute_pri :1,  // thread_recompute_priority(thread);
          tpt_update_tg_ui_flag   :1;
} *task_pend_token_t;
```

At the thread level, the `requested_policy` and `effective_policy` may differ, so they are maintained in two separate fields in the Mach `thread` structure. The two fields are both bit-level structures overlaid on a `uint64_t`, but their sub-fields differ, with the effective policy only applying to some of them. Listing 9-18 shows the `thread_requested_policy` structure, and emphasizes in **bold** those fields also in the `thread_effective_policy` structure (prefixed there by `thep_` instead of `thrp_`).

Listing 9-18: The `thread_requested_policy` structure (from XNU 4903's osfmk/mach/thread_policy.h)

```
#define THREAD_POLICY_INTERNAL_STRUCT_VERSION   4
struct thread_requested_policy {
 uint64_t thrp_int_darwinbg       :1,   /* marked as darwinbg via setpriority */
          thrp_ext_darwinbg       :1,
          thrp_int_iotier         :2,   /* IO throttle tier */
          thrp_ext_iotier         :2,
          thrp_int_iopassive      :1,   /* should IOs cause lower tiers to be throttled */
          thrp_ext_iopassive      :1,
          thrp_latency_qos        :3,   /* Timer latency QoS */
          thrp_through_qos        :3,   /* Computation throughput QoS */

          thrp_pidbind_bg         :1,   /* task i'm bound to is marked 'watchbg' */
          thrp_qos                :3,   /* thread qos class */
          thrp_qos_relprio        :4,   /* thread qos relative priority (store as inverse) */
          thrp_qos_override       :3,   /* thread qos class override */
          thrp_qos_promote        :3,   /* thread qos class from promotion */
          thrp_qos_ipc_override   :3,   /* thread qos class from ipc override */
          thrp_terminated         :1,   /* heading for termination */
          thrp_qos_sync_ipc_override:3, /* thread qos class from sync ipc override */

          thrp_reserved           :29;
};
```

The `thread_effective_policy` contains three additional bit fields: `thep_qos_ui_is_urgent` - which allows "bumping up" QoS when UI is involved, and `thep_[all/new]_sockets_bg`, marking all or newly created sockets of this thread as background.

Experiment: Using `kdv` to inspect Mach scheduling

Scheduling operations are criticial and occur many times a second, so the best facily for tracing them is KDebug. Using the `kdv(j)` tool, you can see when the scheduler's trace points are hit.

Normally, processes may be preempted due to a variety of reasons, commonly blocking system calls or traps. To make sure a process only gets preempted by interrupts or quantum expirations, a simple tight loop process will do the trick. Using `kdv all` in another window while that process (below, "cpuburn1") is running, will show output similar to the following:

Output 9-19: KDebug messages output by the scheduler, as viewed with `kdv(j)`

```
#
# The CPU burning thread gets dispatched to an available core (#4)
#
69615947 4 0x01400008   MACH_STKHANDOFF          kernel_task/0/0x8f      -Args:0 688062 0 1f
69616847 4 0x01400080   MACH_DISPATCH            cpuburn1/83083/0x688062 -Args:8f 0 84 4
#
# Since it is a tight loop, the only kdebug events during its operation will be
# from power management (ignored in this output), timers or interrupts
#
69619541 4 0x01090018   TMR_set_tsc_deadline     cpuburn1/83083/0x688062 -Args:4103880a7b159 4
69619703 4 0x0109001d > TMR_TimerCallEnter       cpuburn1/83083/0x688062 -Args:8e4de4c0e7bd0ed
69619918 4 0x01090018   TMR_set_tsc_deadline     cpuburn1/83083/0x688062 -Args:4103880a7b31c 4
69619991 4 0x01090004   DecrSet                  cpuburn1/83083/0x688062 -Args:ace84 2 4103880
69620090 4 0x0109001e < TMR_TimerCallEnter       cpuburn1/83083/0x688062 -Args:8e4de4c0e7bd0ed
#
# Timer interrupt spontaneously occurs here; a scheduled timer has expired, and is serviced
#
70328732 4 0x01050001 > INTERRUPT                cpuburn1/83083/0x688062 -Args:dd 106c25f3b 1
70328978 4 0x01090028   TMR_TimerCallExpire      cpuburn1/83083/0x688062 -Args:8e4de4c0e7bd0ed
70329035 4 0x01090009 < TMR_TimerCallIntr        cpuburn1 83083/0x688062 -Args:8e4de4c0e7bd0ed
#
#     The timer is the thread quantum, which has been exhausted
#
70329127 4 0x014000bd > mach_sched_quantum_expired      cpuburn1/83083/0x688062 -Args:0 0 0 0
= mt_sched_update emits this:
70329518 4 0x0c010004   MT_InsCyc_CPU_CSwitch   cpuburn1/83083/0x688062 -Args:13ec12d5b31ae f
70329785 4 0x0109001d > TMR_TimerCallEnter       cpuburn1/83083/0x688062 -Args:8e4de4c0e7bd0ed
70329979 4 0x01090018   TMR_set_tsc_deadline     cpuburn1/83083/0x688062 -Args:4103880b288dd 4
70330024 4 0x01090004   DecrSet                  cpuburn1/83083/0x688062 -Args:98948a 2 410388
70330083 4 0x0109001e < TMR_TimerCallEnter       cpuburn1/83083/0x688062 -Args:8e4de4c0e7bd0ed
70330140 4 0x014000be < mach_sched_quantum_expired      cpuburn1/83083/0x688062 -Args:0 0 0 0
70330199 4 0x0109000a < TMR_TimerCallIntr        cpuburn1/83083/0x688062 -Args:8e4de4c0e7bd0ed
70330390 4 0x01090018   TMR_set_tsc_deadline     cpuburn1/83083/0x688062 -Args:4103880b28a78 4
70330438 4 0x01090004   DecrSet                  cpuburn1/83083/0x688062 -Args:9892ef 2 410388
70330636 4 0x01050002 < INTERRUPT                cpuburn1/83083/0x688062 -Args:dd 0 0 0
#
# another interrupt (of lower IRQ) spontaneously, but did not need to be serviced
#
78640104 4 0x01050001 > INTERRUPT                cpuburn1/83083/0x688062 -Args:de 106c25f3b 1
78640604 4 0x01050002 < INTERRUPT                cpuburn1/83083/0x688062 -Args:de 0 0 0
78641875 4 0x0c010004   MT_InsCyc_CPU_CSwitch   cpuburn1/83083/0x688062 -Args:13ec12eecee35 f
#
# Stack is handed off to another thread - chosen to be WindowServer
#
78642060 4 0x01400008   MACH_STKHANDOFF          cpuburn1/83083/0x688062 -Args:5 687feb 1f 4f
78643714 4 0x01400060   MACH_SCHED_CHOOSE_PROCESSOR     WindowServer/190/0x687feb       -Args
```

DTrace's function boundary tracer (`fbt:::`) can also be used to trace scheduling. The perspective, though, is different, as it will trace function calls and flow similar to Figure 9-19, but not show the interrupts. Remember that using `fbt` requires both that SIP be disabled, as well as caution - so as to not accidentally lock up the machine.

Deferred Calls

There is often a need for some kernel thread to do deferred work - not as critical as an AST, yet still asynchronously. Mach addresses this need with `code_entry` structures, defined in osfmk/kern/call_entry.h:

Listing 9-20: The `call_entry` structure (from osfmk/kern/call_entry.h)

```
typedef void            *call_entry_param_t;
typedef void            (*call_entry_func_t)(
                        call_entry_param_t      param0,
                        call_entry_param_t      param1);

typedef struct call_entry {
        queue_chain_t       q_link;
        queue_head_t        *queue;
        call_entry_func_t   func;
        call_entry_param_t  param0;
        call_entry_param_t  param1;
        uint64_t            deadline;
#if TIMER_TRACE
        uint64_t            entry_time;
#endif
} call_entry_data_t;

typedef struct call_entry       *call_entry_t;
```

A `call_entry` can be initialized with the `call_entry_setup` macro (ibid.), which takes the `call_entry` pointer, the function, and its argument (`param0`), and initializes the fields accordingly, leaving the `queue` NULL, the `q_link` empty, and no `deadline`. Should the call make use of both its parameters (i.e. `param1` as well), the caller must set up the second parameter manually. In practice, however, this is rarely done - as `call_entry` objects are always used in one of two contexts - timer calls, or thread calls.

Timer calls

A common use case of deferred calls is when the operation needs to be performed at some time in the future. For this, Mach provides the `timer_call` structure*. This can be thought of a subclass of the `call_entry`, as the structure extends it with timer metadata:

Listing 9-21: The `timer_call` structure (from osfmk/kern/timer_call.h)

```
/*
* NOTE: for now, bsd/dev/dtrace/dtrace_glue.c has its own definition
* of this data structure, and the two had better match.
*/
typedef struct timer_call {
struct call_entry       call_entry;
decl_simple_lock_data( ,lock);          /* protects call_entry queue */
uint64_t                soft_deadline;
uint32_t                flags;
boolean_t               async_dequeue;  /* this field is protected by
                                           call_entry queue's lock   */
uint64_t                ttd;          /* Time to deadline at creation */
} timer_call_data_t, *timer_call_t;

// Used to disable timers by setting an unreachable value**

#define EndOfAllTime            0xFFFFFFFFFFFFFFFFULL
```

The `timer_call` is initialized with `timer_call_setup()`, which internally calls `call_entry_setup()`, initializes the `lock` field, and sets `async_dequeue` to FALSE, so as to make the default handling synchronous. The `call_entry` already has a `deadline`, but `timer_call` extends it with the `soft_deadline`. There are also `flags`, which indicate urgency (`TIMER_CALL_[SYS/USER]_[NORMAL/CRITICAL/BACKGROUND]`. The `...USER..` flags differ from those of `..SYS..` in indicating that the flag should be taken in relation to the calling thread's QoS parameters as well as the urgency requested. An additional flag, `TIMER_CALL_LOCAL` specifies the timer should not "migrate" from the core it was set on.

---

* - Linux has a similar mechanism in `struct timer_list`. Windows has `KTIMER` and the `Ke*Timer*` calls.
** - The `EndOfAllTime` is, indeed, the end of all time. With nanosecond count, it's somewhere around 600-billion years in the future, after our galaxy, and possibly the entire universe will cease to be. Then again, humans might destroy one another or the Earth first.

Neither flags nor deadlines, however, are set until the timer call is actually submitted to the kernel. Submission is usually through `timer_call_enter`, specifying the `timer_call` objects, flags and the deadline. A timer call may be canceled (subject to pending execution) by a call to `timer_call_cancel()`. Should both parameters of the call be needed, the `timer_call_enter1` variant takes `param1` as its second argument (bringing its argument count to four). An additional variant (as of Darwin 13) is `..with_leeway`, which additionally takes a `leeway` in nanoseconds for its fourth argument, and a boolean `ratelimited` for its last (i.e. sixth) argument. Rate limiting uses the soft deadline, rather than the regular one, and the leeway enables timer coalescing.

Timer coalescing

Figure 9-22: Timer coalescing (source: Apple)

One of Darwin 13's most important "under the hood" changes was the introduction of **Timer Coalescing***. When timers start up too frequently, the CPU can enjoy less idle periods - and waking up the CPU can actually take more power than just leaving it on for a slightly longer period. Coalescing allows tweaking the timer deadlines so that more deadlines expire at the same time, thus elongating the wakeup period, but reducing the overall number of wakeups, thus enabling longer idle periods. The effect on battery life can be quite dramatic, and improves with higher CPU clock rates.

The system handles coalescing automatically, provided the timers are set with `timer_call_enter_with_leeway` with the maximum allowed leeway specified. The actual slop value is computed using `timer_compute_leeway`, which takes the system parameters from osfmk/*arch*/*arch*_timer.c. This would seem to imply that the parameters are different across the `i386` and `arm` branches, but they are in fact the same, as shown in Listing 9-23, which is annotated with the default values:

Listing 9-22: The `timer_coalescing_priority_params_t` (from osfmk/kern/timer_call.h) and its defaults

```
#define NUM_LATENCY_QOS_TIERS (6)

typedef struct {
uint32_t powergate_latency_abstime;
uint32_t idle_entry_timer_processing_hdeadline_threshold_abstime; // 5000usec
uint32_t interrupt_timer_coalescing_ilat_threshold_abstime; // 30 us
uint32_t timer_resort_threshold_abstime;  // 50ms

int32_t timer_coalesce_rt_shift;     // CRITICAL  (0)
int32_t timer_coalesce_bg_shift;     // [USER/SYS]_BACKGROUND (-5)
int32_t timer_coalesce_kt_shift;     // Caller priority gt;= MINPRI_KERNEL (3)
int32_t timer_coalesce_fp_shift;     // sched_mode == TH_MODE_FIXED (3)
int32_t timer_coalesce_ts_shift;     // sched_mode == TH_MODE_TIMESHARE (3)

uint64_t timer_coalesce_rt_abstime_max; // 0
uint64_t timer_coalesce_bg_abstime_max; // 100ns
uint64_t timer_coalesce_kt_abstime_max; // 1ns
uint64_t timer_coalesce_fp_abstime_max; // 1ns
uint64_t timer_coalesce_ts_abstime_max; // 1ns

uint32_t latency_qos_scale[NUM_LATENCY_QOS_TIERS];  // {3,2,1,-2,-15,-15}
uint64_t latency_qos_abstime_max[NUM_LATENCY_QOS_TIERS]; // {1ms,5ms,20ms,75ms,10s,10s}
boolean_t latency_tier_rate_limited[NUM_LATENCY_QOS_TIERS]; // 0,0,0,1,1
} timer_coalescing_priority_params_t;
```

* - Windows 8 and later have a similar mechanism in the `EX_TIMER`s and `Ex*Timer` routines, with "No-Wake timers". Linux 2.6.22 and later timer_lists offer `timer_set_deferrable()`.

Scheduling timers

Either with or without leeway, an absolute time deadline for a submitted timer will be calculated by the system. From there, the roads diverge. Timers which aren't rate limited, and whose deadline exceeds `timer_longterm.threshold.interval` will be enqueued on the `timer_longterm`. This is a global structure, whose `threshold.interval` is set by default to `TIMER_LONGTERM_THRESHOLD` (one second), but may be modified by the `longterm` boot-argument (given in milliseconds). On servers (i.e. when `serverperfmode` is true), this queue is disabled.

If the `timer_longterm` isn't used, a call to `timer_queue_assign` (in osfmk/*arch*/*arch*_timer.c) assigns either the per CPU (=core) queue (potentially advancing its deadline, if this call expires sooner) or that of the master CPU. At this point, interrupts must be disabled and the target queue (as well as the source, when migrating) must be unlocked. A call to `timer_call_enqueue_deadline_unlocked()` can then be made, and checks whether this `call_entry` is being queued for the first time, or is being migrated from another queue. Migration may occur as a result of `timer_queue_migrate()`, which may be called when the Power Management subsystem (in an external kext) lets a CPU idle, moving all of its timers to extend its sleep. The locking semantics of the individual entries (which protects the `timer_call`'s internal `call_entry`'s `queue`) are well described before `timer_call_enter_internal()` in osfmk/kern/timer_call.c.

Scheduling timer calls may take differing paths, based on the exact type of timer involved, but eventually it all boils down to `timer_resync_deadlines()`. This architecture-specific function looks through the deadlines stored in the per-CPU data: The `rtclock_timer` deadline, `idle_timer_deadline` (ARM) or power management deadline (Intel), and the `quantum_timer_deadline`, and selects the most imminent (i.e. smallest) one. It then calls the architecture specific implementation of `setPop()` (osfmk/[arm/i386]/rtclock.c). The Intel implementations requests an interrupt, whereas the ARM one loads the value onto the per-CPU data's `rtcPop` value.

Servicing Timer calls

Timer calls are serviced when a timer interrupt occurs. XNU's handler, `rtclock_intr()` (again, machine specific) is called. This triggers the "event timer" interrupt (artificial, through a call to `timer_intr()`, which checks if any of the per CPU deadlines expires. If they have, their corresponding expiration handlers are called:

Table 9-23: The various timer expiration behaviors

Architecture	Timer set	Expiration	Behavior
ARM	`cpu_idle[_exit]()`	`idle_timer_notify()`	per-CPU data's `idle_timer_notify()`
Intel/ARM	`timer_call_enter()`	`timer_queue_expire()`	Run all expired `timer_call`s
Intel	`pmCPUSetDeadline()`	`pmCPUDeadline()`	Defined by Power Management module
Intel/ARM	`timer_call_quantum_timer_enter()`	`quantum_timer_expire()`	`thread_quantum_expire()` (q.v. Figure 9-15)

Thread calls

For deferred work that isn't necessarily time constrained (but may be), Mach offers the `thread_call` construct*. As with the `timer_call`, this structure extends the base `call_entry`, but in a different direction:

Listing 9-24: The `thread_call` structure (from osfmk/kern/thread_call.h)

```
struct thread_call {
struct call_entry     tc_call;          /* Must be first for queue macros */
uint64_t              tc_submit_count;
uint64_t              tc_finish_count;
uint64_t              tc_ttd;           /* Time to deadline at creation */
uint64_t              tc_soft_deadline;
thread_call_index_t   tc_index;
uint32_t              tc_flags;
int32_t               tc_refs;
};
```

Thread calls may be allocated through normal means, but the common usage pattern is to use `thread_call_allocate[_[with_priority/options/qos]]()`. The base call is used in all cases to `zalloc a struct thread_call` from the eponymous zone, and then `thread_call_setup()` to initialize the fields of the `tc_call`, and assign a priority of `THREAD_CALL_PRIORITY_HIGH`. The variants take the allocated call, and may change the priority (to `THREAD_CALL_PRIORITY_[KERNEL/USER/LOW/KERNEL_HIGH]`), or set QoS tiers (from `THREAD_QOS_*` constants, as discussed elsewhere. Both priority and QoS tiers map to `THREAD_CALL_INDEX_..` values, which the call keeps in the `tc_index` member. If specifying options, the caller may request `THREAD_CALL_OPTIONS_ONCE` to invoke the call and handle freeing it (assuming it was likewise allocated). Another option is `THREAD_CALL_OPTIONS_SIGNAL`, which invokes the call on the next timer interrupt. These set corresponding `THREAD_CALL_[ONCE/SIGNAL]` in `tc_flags`.

Servicing thread calls

After a `thread_call` has been created, the caller may call `thread_call_enter[1][_delayed[with_leeway]]`. As with the `timer_call`, the '1' variant allows setting the call's `param1`, and the `..._with_leeway` allows a slop (as well as setting the `tc_flags` directly). Once enqueued, thread calls are served by the `thread_call_daemon`.

The `thread_call_daemon` is a dedicated kernel thread, which is started at `BASEPRI_PREEMPT_HIGH +1` by `thread_call_initialize()`, and immediately enters a continuation. The continuation first disables all interrupts and locks the thread call lock, calling `thread_call_group_should_add_thread` for each `thread_call_groups` from highest priority to lowest, If a new thread is necessary, interrupts are enabled and the lock is dropped, a `thread_call_thread` is created according to the group specified priority, and then interrupts are disabled again, and the lock is picked up again. Once the group scan is complete, the daemon puts itself on the `daemon_waitq`, and blocks on its continuation.

---

* - Linux provides these through `work_structs`, though unlike Mach it also allows custom workqueues.

The `thread_call_groups` are an array of `struct thread_call_groups`, each specifying the default priority for its threads, as well as a target thread count. The daemon's call to `thread_call_group_should_add_thread` will decide affirmatively if there are less threads than the target number, or the amount of work (`pending_count`) is greater than four times the number of threads. There are two safeguards, however: Threads will not be added if the number of threads exceeds the hard-coded `THREAD_CALL_GROUP_MAX_THREADS` (500), or the Mach factor is greater than `THREAD_CALL_MACH_FACTOR_CAP` (3). Table 9-25 lists the threads groups defined in Darwin 18. As of Darwin 17 threads created will be named by group name and thread number (as was shown in Table 5-34).

Table 9-25: The `thread_call_groups` defined in Darwin 18

tcg_name	tcg_thread_pri	target_thread_count	flags
high	BASEPRI_PREEMPT_HIGH	4	TCG_NONE
kernel	BASEPRI_KERNEL	1	TCG_PARALLEL
user	BASEPRI_DEFAULT	1	TCG_PARALLEL
low	MAXPRI_THROTTLE	1	TCG_PARALEL
kernel-high	BASEPRI_PREEMPT	2	TCG_NONE
qos-ui	BASEPRI_FOREGROUND	1	TCG_NONE
qos-in	BASEPRI_USER_INITIATED	1	TCG_NONE
qos-ut	BASEPRI_UTILITY	1	TCG_NONE

Each `thread_call_group` has its own `pending_queue`, for normal calls, and `TCF_COUNT delayed_queues`, for delayed execution. `TCF_COUNT` is two, allowing for two queues: one for `TCF_ABSOLUTE` (calls with a deadline specified by `mach_absolute_time`, and the other for `TCF_CONTINUOUS` (for `mach_continuous_time`). The choice between the two was made when the thread call is created, `tc_flags` is set appropriately (to `THREAD_CALL_CONTINUOUS` or `...ABSOLUTE`). The delayed queues have corresponding delayed timers, which are armed (using an underlying `timer_call_enter_with_leeway`) when a delayed call is entered.

When a `thread_call_thread` is created, it first marks itself as `THREAD_TAG_CALLOUT`, for purposes of differentiating its statistics from the rest. It then checks its group's `pending_queue` (with the lock, and interrupts disabled), calling `thread_call_invoke()` to execute the call. When the call returns, it optionally calls `thread_call_finish()` (if the call was `thread_call_allocated()`) to either dispose or reschedule the call. If the queue empties and the group's `idle_count` is less than the `target_thread_count`, the thread blocks (putting itself on the group's `idle_waitq`), using its own function as a continuation. Otherwise, the thread may exit. As of Darwin 16 groups may be marked `TCG_PARALLEL`, which allows idle threads to block and pick up work. In these cases all threads always block, but the group also has a `dealloc_timer` which runs a timer callasking idle threads to terminate by waking them up with `THREAD_INTERRUPTED` every `THREAD_CALL_DEALLOC_INTERVAL_NS` (5 ms).

Throughout their lifetime, `thread_calls` may be tracked by their respective callers, since they have the original pointer. The `thread_call`'s `tc_[submit/finish]_count` are used (through `thread_call_isactive()`) to check if a call is currently active. The call may also be waited for (until it finishes, using `thread_call_wait_once`, and canceled (subject to prior execution) through `thread_call_cancel[_wait]`.

Scheduler assisted synchronization

Chapter 4 already covered the basic synchronization primitives used at the low levels, but deferred the discussion of the sleepable ones (`rwlock_t`s and `lck_mtx_t`s) are woken, since this requires scheduler involvement. Let us now consider those aspects.

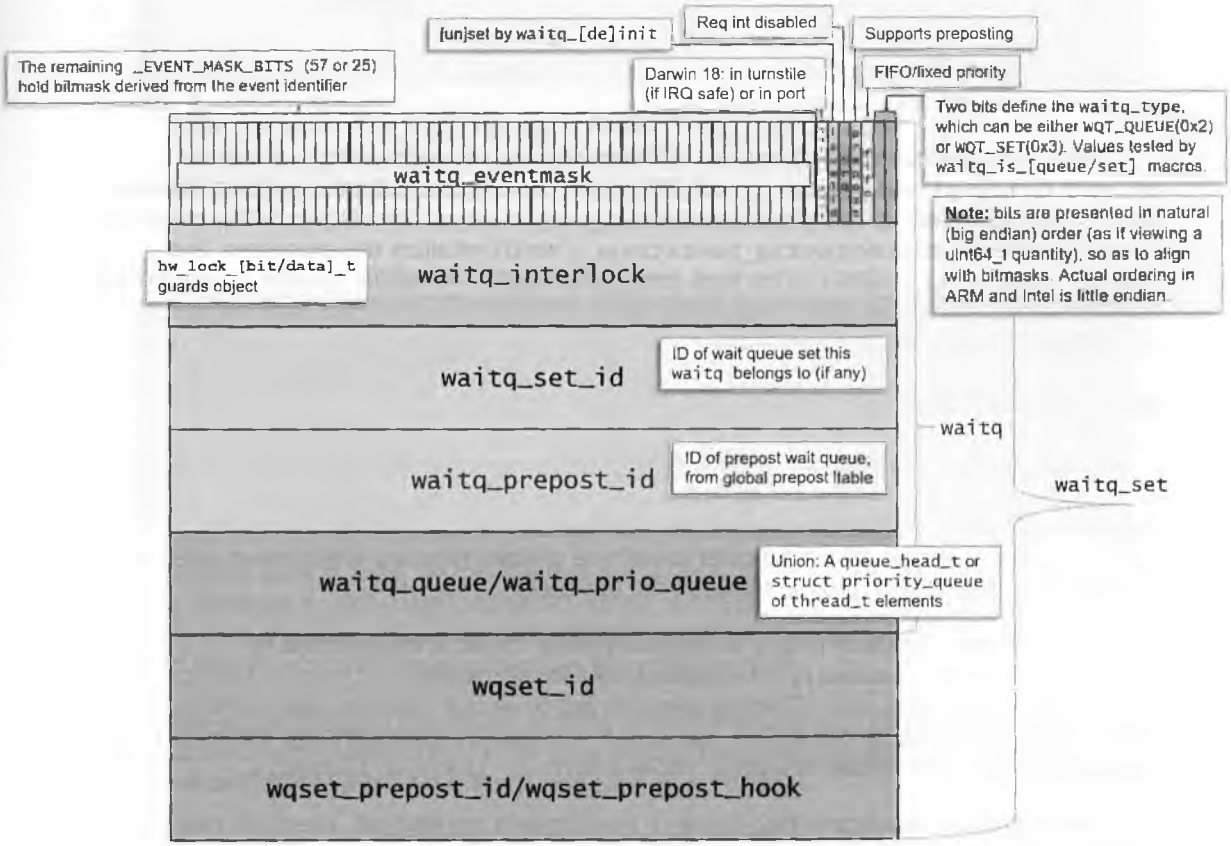
Wait Queues

Threads often block voluntarily, when they wait for some unavailable resource, such as I/O from a device, a mutex, etc. Since multiple threads may end up waiting for the same resource, a simple solution used across all operating systems is that of a **wait queue**, waking up sleeping threads who are on the queue, usually on a a first come, first served basis. Mach follows this pattern as well, with the `waitq` and `waitq_set` structures. The `waitq`s can be found embedded in Mach ports, semaphores and (as of Darwin 18) turnstiles, and they also support BSD's `select(2)` and AIO implementations. The `waitq_set`s back `select(2)` as well, along with kqueueues and Mach `ipc_mqueue`s.

Wait queues are initialized with `SYNC_POLICY_*` flags (from osfmk/mach/sync_policy.h). A common policy is `SYNC_POLICY_FIFO` (first come first served), but `.._FIXED_PRIORITY` is allowed as well (though unused), and even `..._REVERSED`, which turns `..FIFO` into `..LIFO`. A wait queue may be defined as requiring interrupts disabled (`SYNC_POLICY_DISABLE_IRQ`). Another important flag is `SYNC_POLICY_PREPOST`.

Both `waitq` and `waitq_set` are defined in osfmk/kern/waitq.h, with the latter being an extension of the former, as shown in Figure 9-26:

Figure 9-26: The `struct waitq` (from osfmk/kern/waitq.h), visualized



As the figure shows, Mach's `waitq` implementation takes the actual queue (`waitq_queue`, which (as of Darwin 18) is overloaded as `waitq_prio_queue`, for turnstile queues, discussed later) and adds important metadata on top of it.

The seven least significant bits are currently defined as holding the wait queue object type (presently, `WQT_SET` (0x3), or `WQT_QUEUE` (0x02) are defined), the queue type (FIFO or LIFO), whether the queue supports preposting (per `SYNC_POLICY_PREPOST`), must interrupts be disabled, has the queue been properly initialized, and (in Darwin 18), is the queue embedded inside a larger object (a turnstile, if `waitq_irq_` is set, or else a port). The remaining (most significant) bits encode the event, usually derived from some kernel object address.

Both the wait queue and wait queue sets remain opaque to the kernel (save for `waitq_type`), and can only be accessed using specific interfaces, none of which are meant for use outside the kernel proper, so they are defined as `MACH_KERNEL_PRIVATE`. These are shown in Table 9-27. For purposes of size calculation while preserving the structure's opacity, the `WQ[S]_OPAQUE_[SIZE/ALIGN]` can be used, e.g. `select(2)`'s `struct selinfo` (bsd/sys/select.h), whose first element is the `si_waitq`.

Table 9-27 The functions used for `struct waitq` handling

Function	Purpose
waitq_wakeup64_[one/all](*waitq, wake_event, result,priority*)	Wake up one or all threads waiting on *waitq* for particular *wake_event* with *result* and *priority*
waitq_wakeup64_[one/all]_locked(*waitq, wake_event, result,reserved_preposts,priority,lock_state*)	As above, when *waitq* is locked.
waitq_wakeup64_identify_locked(*waitq, wake_event, result,* out *spl, reserved_preposts,priority,lock_state*)	As above, returning the `thread_t` being woken, and its *spl*. Caller must `thread_unlock()` the returned thread, and restore with `splx` (as interrupts may have been disabled)
waitq_assert_wait64_locked(*waitq, wait_event, interruptible, urgency, deadline, leeway, thread*)	Declare *thread* is waiting in *waitq* for *wait_event*, with timer_call *urgency*, *deadline*, and *leeway*. `waitq_assert_wait64()` and `waitq_assert_wait64_leeway()` wrap this.
waitq_member(*waitq,wqset*)	Return `TRUE` if *waitq* is a member of *wqset*
waitq_in_set(*waitq*)	Return `TRUE` if *waitq* is a member of any set

BSD's `wakeup[_one]()` (from Listing 6-7) are implemented directly over `thread_wakeup[_one]()`, both macros from osfmk/kern/sched_prim.h which call `thread_wakeup_prim()` indicating a `result` of `THREAD_AWAKENED`. This then calls `waitq_wakeup64_[one/all]()`, using the BSD wait channel for the `event` argument. Wait queues are also used extensively by `assert_wait[/_timeout/_deadline]()` in the scheduler code, and are initialized with `SYNC_POLICY_FIFO|SYNC_POLICY_DISABLE_IRQ`.

The system maintains its wait queues in the static `global_waitqs` array, which is accessed through `global_eventq()` call. This takes the `event` argument, applies a Jenkins hash (osfmk/kern/waitq.c), and clips the result to `g_num_waitqs`. This value (i.e. the number of queues) is determined during `waitq_bootstrap()`, which initializes the subsystem, and `kernel_memory_allocate()`s the hash table dynamically. The default number is determined as a function of `waitq_hash_size()`, itself set by a fifth of `thread_max`, or overridden by the `wqsize` boot argument.

selection callbacks

Although a queue is traditionally a FIFO structure, threads are prioritized. The standard practice of dequeueing the first waiting thread, therefore, might not be suitable when there is some other, higher priority and/or possibly time-critical thread also waiting. Mach's implementation therefore allows the caller to define a callback function, which queue selection functions will consult when picking threads from the queue.

The common use case of finding the highest priority thread is implemented by `waitq_select_max_locked()`. The callback used in this case is `waitq_find_max_pri_cb()`. There is also a variant to `waitq_select_one_locked()` (i.e. with no callback), and both call `waitq_select_n_locked()`, which performs the selection by iterating the queue with either of `waitq_[queue/prioq]_iterate_locked()`.

An important observation is that, if one or more threads are selected, interrupts may be disabled. This is reported back to the caller through an `spl` argument (passed by reference, as an out parameter), and the caller must call `splx()` on this value to restore the interrupts to the previous state, if necessary. The thread, though woken, is still in a locked state, and will not be picked up by the scheduler until `thread_unlock()`ed.

Experiment: Examining `waitq` structures in memory

MacOS exports global_eventq(), but not the `global_waitqs` array, nor `g_num_waitqs`. It's a simple enough matter to determine the address in memory of all these symbols, by looking at the disassembly. Listing 9-28 shows the disassembly of `global_waitq()` from MacOS and *OS:

Listing 9-28-a: Disassembling `global_eventq()` to find `global_waitqs`

```
# MacOS 15 β9                                  | # iOS 13.1 iPhone12:
#                                              | #
_global_eventq:                                | _global_eventq:
..8000390330 pushq %rbp                        | ..f007bfb230 ADRP   X8, 5690
..8000390331 movq  %rsp, %rbp                  |              LDR    [X8, #3104]
..8000390334 movl  %edi, %eax                  |              ADRP   X9, 5690
..8000390336 xorl  %edx, %edx                  |              LDR    W9, [X9, #3112]
..8000390338 divl  0x87adfa(%rip)              |                     W10, W0, W9
..800039033e leaq  (%rdx,%rdx,2), %rax         |              MSUB   W9, W10, W9, W0
..8000390342 shlq  $0x4, %rax                  |
..8000390346 addq  0x87ade3(%rip), %rax        |              MOVZ   W10, 0x28
..800039034d popq  %rbp                        |              UMADDL W9, W10, W10
..800039034e retq                              | ..f007bfb250 RET
```

The *OS symbols will all be recovered by `jtool2`'s analysis module, but the MacOS ones require a little hex math. There are two RIP relative addresses (in *OS, loaded by `ADRP/LDR`):

Output 9-28-b: Locating `global_waitqs` from memory

```
root@Zephyr(~)# printf "%x\n" $[ 0x800039033e + 0x87adfa ]
8000c0b138
root@Zephyr(~)# printf "%x\n" $[ 0x800039034d + 0x87ade3 ]
8000c0b130
root@Zephyr(~)# xnoop sdump 0xffffff8000c0b130
dumping addr: ffffff801180b130
0xffffff801180b130    0xffffff80f6ab9000     00 08 00 00 00 00 00 00
```

Thus, the value of `g_num_waitqs` is 0x800 (2048), and the address of `global_waitqs` (accounting for kernel slide) is in our case 0xffffff80f6ab9000 (and will always start on a page boundary due to `kernel_memory_allocate()` being used in its allocation. Dumping the `global_waitq` would look something like the following:

Output 9-28-c: Dumping `global_waitqs` from memory

```
root@Zephyr (~)# xnoop dump 0xffffff80f6ab9000 smart
0xffffff80f6ab9000 36 00 00 00 24 5d df 48   00 00 00 00 00 00 00 00    6...$].H........
0xffffff80f6ab9010 00 00 00 00 00 00 00 00   00 00 00 00 00 00 00 00    ................
                                  ???                  ???
0xffffff80f6ab9020    0xffffff80f6ab9020        0xffffff80f6ab9020
0xffffff80f6ab9030 36 dc 1d fa 23 d0 00 1b   00 00 00 00 00 00 00 00    6...#...........
0xffffff80f6ab9040 00 00 00 00 00 00 00 00   00 00 00 00 00 00 00 00    ................
                               threads              threads
0xffffff80f6ab9050    0xffffff802a0b4530        0xffffff802a0b4530
0xffffff80f6ab9060 36 70 97 60 22 fe 21 7f   00 00 00 00 00 00 00 00    6p.`".!.........
0xffffff80f6ab9070 00 00 00 00 00 00 00 00   00 00 00 00 00 00 00 00    ................
```

Looking back at 9-26, we can see that the value of 0x36 (for the first queue entry's flags) correlates to IRQ (0x20), PREPOST (0x10), `FIFO` and `WQT_QUEUE`. The queue is empty, since the `queue_head_t` (at 0xffffff80f6ab9020, but note that on *OS the queue would start at +0x18 due to smaller size) refers back to itself.

The second waitq in the output (starting at 0xffffff80f6ab9030) indeed holds a thread, as indicated by the smart dump. If we go and dump that thread as well (assuming no context switch occurred for it in the interim) we would find:

Output 9-28-d: Dumping the Mach thread in a waitq

```
root@Zephyr (~)# xnoop dump 0xffffff802a0b4530 thread
Thread @0xffffff802a0b4530: State: 2
        ??..??..ffffff8034ad0920: (PID 602, devicecheckd)
        Continuation: 0xffffff801703b670 (_workq_thread_set_max_qos)    Parameter: 0x0
```

An even quicker way to find `global_waitqs` is to use `procexp(j) 0 regions` and `grep(1)` for "Wait Queues". This (at least up to Darwin 19.0) reveals the address, due to `kernel_memory_allocate()` tagging it with `VM_KERN_MEMORY_WAITQ` (q.v. 12-17).

Ulocks (Darwin 16+)

Darwin 16 introduces a new type of user-mode visible, scheduler-aware locks, called **ulocks**. These are accessible through two system calls - `__ulock_wait()` and `__ulock_wake()` (#515 and #516, respectively). As the double underscores imply, user mode is not intended to use these system calls directly, instead working with libplatform.dylib's higher level `os_unfair_lock_t`.

The `__ulock_wait` will block on a given ulock, or create a new one. The syscall first provides a cancelation point (via `__pthread_testcancel()`) if called with the `ULF_WAIT_CANCEL_POINT`. The user mode address supplied by the caller is paired with the caller's PID to generate a `ulk_t` key structure. The key is then passed to `ull_get()`, which looks through the ulocks in hash buckets, indexed using `ull_hash_index()` (a Jenkins hash) to return the ulock object in kernel, a `struct ull`. If a `ull` cannot be found, one is created, by allocating from the dedicated `ulocks` zone) and enqueued in the appropriate hash bucket. The zone and the hash buckets are created by `ulock_initialize()`, during `bsd_init()`. The number of hash buckets is set to $\lceil \log_{2(\texttt{thread_max})} \rceil - 2$, and each hash bucket is a `queue_head_t` of `ull_t` elements, guarded by a `lck_spin_t`.

The `struct ull` has undergone quite a few changes since its introduction, and the metamorphosis continues in Darwin 19. Figure 9-29 shows the object.

Figure 9-29: The implementation of `ulocks` (from bsd/kern/sys_ulock.c)

```
typedef struct ull_bucket {
        queue_head_t ulb_head;
        lck_spin_t   ulb_lock;
} ull_bucket_t;

static ull_bucket_t *ull_bucket;

typedef struct ull {
    /*
     * ull_owner is the most recent known value for the owner of this ulock
     * i.e. it may be out of date WRT the real value in userspace.
     */
    thread_t          ull_owner; /*/* holds +1 thread reference */
    ulk_t             ull_key;   /* ulk_addr + ulk_pid */
#ifdef PRE_19
    ulk_t             ull_saved_key;
#endif
    ull_lock_t        ull_lock;          /* interlock */
    uint              ull_bucket_index;  /* Jenkins hash index, from  ull_hash_index() */
    int32_t           ull_nwaiters;      /* incremented by ulock_wait()
#ifdef PRE_19
    int32_t           ull_max_nwaiters;
#endif
    int32_t           ull_refcount;      /* [inc/dec]remented by ull_get()/ull_put() */
    uint8_t           ull_opcode;        /* UL_UNFAIR_LOCK/UL_COMPARE_AND_WAIT */
    struct turnstile *ull_turnstile;     /* Associated turnstile (Darwin 18+) */
    queue_chain_t     ull_hash_link;     /* Hash bucket linkage */
} ull_t;
```

In the original (Darwin 16-17) model, the wait was performed by converting the ulock to a global wait queue, using `ULOCK_TO_EVENT` to obtain the wait queue, and then calling `assert_wait[_timeout]()`. Prior to waiting, the thread would donate its priority to the lock owner, with the hope that this will enable the owner to release the lock as soon as possible.

Releasing the lock is performed via `ulock_wake()`, which similarly uses a `ulk_t` key structure, and provides it to `ull_get()`, but this time with the `ULL_MUST_EXIST` flag. The lock is then converted with `ULOCK_TO_EVENT`. Depending on the arguments to `ulock_wake()`, `thread_wakeup ()` wakes up all waiters (if `ULF_WAKE_ALL`, `thread_wakeup_thread()` wakes up the caller supplied thread (if `ULF_WAKE_THREAD`) or `thread_wakeup_one_with_pri()` wakes the next waiter.

Darwin 18 changes ulocks substantially, with two notable improvements: First, the wait employs a continuation, and uses the BSD `uthread`'s `uu_save` union, storing in it the `uus_ulock_wait_data`, a small structure consisting of `thread_t` pointers to the present `owner_thread` and the `old_owner`, `flags`, and a pointer to the eventual `retval`. Also as of Darwin 18, the ulock promotion involves a turnstile, as we discuss imminently.

Turnstiles (Darwin 18+)

Darwin 18 brings an additional concept into the world of thread scheduling - that of **turnstiles**. The concept first appeared in Solaris[3], and was then adopted by FreeBSD[4], and well explained in the BSD bible[5]. Apple's developers implemented it differently from both systems, and the implementation is entirely internal to XNU (i.e. not exported through any KPI).

Theory

The basic idea behind turnstiles is to optimize the handling of short term locks and optimize the scheduling of waiters when the locks become available. Each thread has its own turnstile object. Just before sleeping on (waiting for) a lockable object (referred to as a **proprietor**), it checks if there is already a turnstile associated with it. If there isn't, the thread provides its turnstile to the lock. Subsequent waiters do so as well, and the turnstiles for this proprietor are maintained in a list. The proprietor's turnstile maintains a wait queue, kept ordered by priority.

When the lock becomes available, the highest priority thread in the wait queue is next to be woken up. The thread also gets back a turnstile object (but not necessarily its own, due to the priority queue ordering) from the turnstile list. When the very last thread is awakened, there are no more turnstiles save for the one associated with the lock. Since the lock will be freed by the last thread, the lock's associated turnstile can be returned to that thread.

Darwin implementation of Turnstiles

Darwin turnstiles are 128-byte objects allocated from their own dedicated `turnstiles` zone, set up from `turnstiles_init()`, which also initializes the `turnstile_htable`. This is the hash table used to lookup turnstiles by proprietors, wherein each table entry is a `turnstile_htable_bucket`, consisting of a lock and an `SLIST_HEAD` of `turnstiles`. The table presently contains `TURNSTILE_HTABLE_BUCKETS_DEFAULT` (32) entries. As noted in osfmk/kern/turnstile.c, the choice of this value is important (since it directly affects hash collisions), and may be overridden by the `ts_htable_buckets` boot-argument.

The hash key is a Jenkins hash (from osfmk/kern/waitq.h) using the in-memory address of the proprietor (which is also saved in the turnstile object, so as to detect collisions). There are different types of turnstiles, allowing them to be used with various interlocks: `mtx_lck_ts`, `ulock_ts`, other kernel objects (workloops, work queues, knotes) and even pthread mutexes*. Each type has an associated promotion policy, which is consulted when the turnstile's inheritor is updated, or its priority is recomputed, although as of Darwin 18 `TURNSTILE_KERNEL_PROMOTE` is unhandled, and `TURNSTILE_USER[_IPC]_PROMOTE` are effectively the same. Table 9-30 shows the `turnsyle_type_ts` defined in XNU 4903:

Table 9-30: Turnstile types (from XNU 4903's osfmk/kern/turnstile.h)

Type	Proprietor	Promotion	Inheritor	Lock Order
`TURNSTILE_NONE`	N/A	`..NONE`		
`TURNSTILE_KERNEL_MUTEX`	Unused?	`KERNEL`		
`TURNSTILE_ULOCK`	`ulock`	`USER`	thread	Ts, Th
`TURNSTILE_PTHREAD_MUTEX`	`pthread mtx`			
`TURNSTILE_SYNC_IPC`	`ipc_port_t->mqueue`	`USER_IPC`	turnstile	
`TURNSTILE_WORKLOOPS`	`wl lock`		thread, Workqueue turnstile	
`TURNSTILE_WORKQS`	`wq lock`		thread	
`TURNSTILE_KNOTE`	`knote lock`		Workloop Turnstile	

The turnstile type is stored in its structure, compacted into bits next to a generational count. The `turnstile_set_type_and_increment_gencount()` ensures that the two are always modified in tandem, indicating turnstile has been repurposed.

---

* - Both Solaris and BSD implementations support read-write locks, as well, in which case the turnstile has not one, but two wait queues associated with it. This might be supported by XNU as well in some future version of XNU.
** - Readers familiar with the Solaris implementation or reading [solts] can find parallels between Solaris' `turnstile_block()` to Darwin's `turnstile_prepare()`. Solaris's `turnstile_wakeup()` is likewise similar to Darwin's `turnstile_complete()`.

Listing 9-31 shows the Darwin turnstile implementation. Every turnstile begins with an embedded `waitq`, which is a priority queue of waiting threads, or other turnstiles.

Listing 9-31: The Darwin turnstile structure, from XNU 4903's osfmk/kern/turnstile.h

```
struct turnstile {
  struct waitq            ts_waitq;           /* waitq embedded in turnstile */
  turnstile_inheritor_t   ts_inheritor;       /* thread/turnstile inheriting the priority(IL, WL) */
  union {
    struct turnstile_list ts_free_turnstiles; /* turnstile free list (IL) */
    SLIST_ENTRY(turnstile) ts_free_elm;       /* turnstile free list element (IL) */
  };
  struct priority_queue    ts_inheritor_queue;/* Queue of turnstile with us as an inheritor (WL) */
  union {
    struct priority_queue_entry ts_inheritor_links; /* Inheritor queue links */
    queue_chain_t         ts_deallocate_link;       /* thread deallocate link */
  };
 SLIST_ENTRY(turnstile)   ts_htable_link;     /* linkage for turnstile in global hash table */
 uintptr_t                ts_proprietor;      /* hash key lookup turnstile (IL) */
 os_refcnt_t              ts_refcount;        /* reference count for turnstiles */
 _Atomic uint32_t         ts_type_gencount;   /* gen count for priority chaining (IL), type (IL) */
 uint32_t                 ts_port_ref;        /* # explicit refs from ports on send turnstile */
 turnstile_update_flags_t ts_inheritor_flags; /* flags for turnstile inheritor (IL, WL) */
 uint8_t                  ts_priority;        /* priority of turnstile (WL) */
```

Benefits of Turnstiles

Using turnstiles mitigates two problematic and unfortunately common locking scenarios. The first is the "thundering herd" problem, a contention scenario in which multiple threads vying for the same resource are all woken up together. The second is priority inversion, wherein a lower priority lock holder blocks a higher priority thread waiting for that lock): As a thread is about to block on a given lock, it may donate its priority to the lock holder, so that it will get a "boost" and be scheduled, thus hopefully releasing the lock sooner.

KDebug codes

As a new feature, Turnstiles have been laced with KDebug codes, which facilitate tracing their lifecycle. KDebug facility `DBG_TURNSTILE` (0x35) has been reserved, with subcodes 0x10, 0x20 and 0x30 for tracking thread/turnstile operations, priority and use.

```
0x35100004      TURNSTILE_thread_added_to_turnstile_waitq
0x35100008      TURNSTILE_thread_removed_from_turnstile_waitq
0x3510000c      TURNSTILE_thread_moved_in_turnstile_waitq
0x35100010      TURNSTILE_turnstile_added_to_turnstile_heap
0x35100014      TURNSTILE_turnstile_removed_from_turnstile_heap
0x35100018      TURNSTILE_turnstile_moved_in_turnstile_heap
0x3510001c      TURNSTILE_turnstile_added_to_thread_heap
0x35100020      TURNSTILE_turnstile_removed_from_thread_heap
0x35100024      TURNSTILE_turnstile_moved_in_thread_heap
0x35100028      TURNSTILE_update_stopped_by_limit
0x3510002c      TURNSTILE_thread_not_waiting_on_turnstile
0x35200004      TURNSTILE_turnstile_priority_change
0x35200008      TURNSTILE_thread_user_promotion_change
0x35300004      TURNSTILE_turnstile_prepare
0x35300008      TURNSTILE_turnstile_complete
```

Gates (Darwin 19)

Darwin 19's provides **gate**s, which build on turnstiles, but seem (as of yet) to be unused.

Ledgers

Mach has long defined **ledgers**, though Apple only brought them to XNU in Darwin 12. Just like its banking namesake, a ledger provides a means for resource accounting, with debits and credits. The ledger's balance is determined by simply deducting its `le_debit` from its `le_credit`, and may be negative. A ledger may also impose an `le_limit`, which the balance cannot exceed (in absolute value). Ledgers can also be set to periodically refill (with the bank analogy, think direct deposit). Not all ledgers necessarily have both credit and debit - a ledger flagged as `LF_TRACK_CREDIT_ONLY` does exactly that, leaving the debit set to zero. This flag may be set with a call to `ledger_track_credit_only`, as is the case with many task ledgers.

Should the ledger somehow end up exceeding its limit, a Mach exception occurs. Ledgers can also be configured with a threshold value (`le_warn_level`), which is lower than the limit. Doing so is useful in tandem with `ledger_set_callback()`, which sets a callback to be executed when the balance exceeds the threshold or falls back under it.

`ledger` (#373)

Ledgers are pure Mach layer construct, but XNU 2050 introduces the undocumented `ledger` system call (#373) which can be used to obtain information about them from user mode. This is an `ioctl(2)` style system call, taking in a `cmd` argument and three more buffers, with `cmd` being one of `LEDGER[/_ENTRY/_TEMPLATE]_INFO` (0, 1 or 2, respectively).

The `ltop` command is a perfect example of this system call in practive, and provides a great way to display ledger statistics for all processes in the system. Although not part of either MacOS or iOS's default binaries, its source code is part of the system_cmds project, and can thus be compiled for all Darwin variants - and is included in the Author's binpack. Output 9-32 shows a sample output of this command:

Output 9-32: The output of `ltop`

```
morpheus@Chimera (~)$ ltop -p 1
PID  COMMAND      RESOURCE          CREDITS      DEBITS      BALANCE      LIMIT      PERIOD
1    launchd      cpu_time            8538G           0        8538G       none           -
                  tkm_private          307K           0         307K       none           -
                  tkm_shared          2330M        1940M         389M       none           -
                  phys_mem            1468K        1449M          19M       none           -
                  wired_mem              0M           0            0       none           -
                  internal              18M           0           18M       none           -
                  iokit_mapped           0M           0            0       none           -
                  alternate_accounting   0M           0            0       none           -
                  alternate_accounting_compressed  0M  0            0       none           -
                  page_table             0M           0            0       none           -
                  phys_footprint        29M           0           29M       none           -
                  internal_compressed   10M           0           10M       none           -
                  purgeable_volatile     0M           0            0       none           -
                  purgeable_nonvolatile  0M           0            0       none           -
                  purgeable_volatile_compress  0M     0            0       none           -
                  purgeable_nonvolatile_compress  0M  0            0       none           -
                  network_volatile       0M           0            0       none           -
                  network_nonvolatile    0M           0            0       none           -
                  network_volatile_compressed  0M     0            0       none           -
                  network_nonvolatile_compressed  0M  0            0       none           -
                  platform_idle_wakeups  63184           0        63184       none           -
                  interrupt_wakeups      106K           0         106K       45000          0
                  SFI_CLASS_DARWIN_BG    0M           0            0       none           -
                  SFI_CLASS_APP_NAP      0M           0            0       none           -
                  SFI_CLASS_MANAGED      0M           0            0       none           -
                  SFI_CLASS_DEFAULT      0M           0            0       none           -
                  SFI_CLASS_OPTED_OUT    0M           0            0       none           -
                  SFI_CLASS_UTILITY      0M           0            0       none           -
                  SFI_CLASS_LEGACY       0M           0            0       none           -
                  SFI_CLASS_USER_INITIATED  0M        0            0       none           -
                  SFI_CLASS_USER_INTERACTIVE  0M      0            0       none           -
                  SFI_CLASS_MAINTENANCE  215G           0         215G       none           -
                  cpu_time_billed_to_me  0M           0            0       none           -
                  cpu_time_billed_to_others  1224G    0         1224G       none           -
                  physical_writes        152G           0         152G       none           -
                  logical_writes          13G           0          13G       none           -
                  energy_billed_to_me    0M           0            0       none           -
                  energy_billed_to_others  0M         0            0       none           -
```

Initialization

As part of the initialization of the task subsystem (`task_init()`), a call is made to `init_task_ledgers()` (in osfmk/kern/task.c), which sets up the `task_ledgers` global. This is a `_task_ledger_indices` (defined in osfmk/kern/task.h), whose indices point to entries in

the global `task_ledger_template` . Another `_thread_ledger_indices` structure, the `thread_ledgers` (both in osfmk/kern/thread.c) similarly points to entries in the `thread_ledger_template`. Indices are assigned by calls to `ledger_entry_add()` (osfmk/kern/ledger.c), which allows providing descriptive metadata - the key display name, group name, and units of measure.

When new tasks and new threads are created, their `ledgers` are created from the corresponding template by a call to `ledger_instantiate()`. Subsequent changes to the templates will not affect the ledgers after instantiation, which is why the templates are created well ahead of their use. Each template has its own zone (i.e. "Per-task.ledger", "Per-thread.ledger", and "Bank.ledger") from which its ledgers are instantiated.

Table 9-33 shows the ledgers found in Darwin 18.

Table 9-33: The ledgers used in Darwin 18

Group	Ledger name	Units	Updater	Notes
sched	cpu_time	ns	thread_quantum_expire	Per-Task, Per-Thread, Bank
	cpu_time_billed_to_[me/others]		bank_rollup_chit_to_tasks	Per-Task
physmem	tkm_shared	bytes	kalloc_canblock/kfree	Per-Task
	tkm_private		pmap_tt_[de]allocate	
	phys_mem		pmap_[enter/remove_range].. etc	
	wired_mem			
	internal[_compressed]			Per-Task, Credit only
	alternate_accounting[_compressed]			
	page_table			
	purgeable_[non]volatile[_compress]			
	iokit_mapped		vm_map_iokit_[un]mapped_region	
	phys_footprint		All other physmem updaters	Per Task, credit only, callback
	network_[non]volatile[_compressed]			
power	platform_idle_wakeups	count	thread_call_thread	Per-Task
	interrupt_wakeups			Per-Task, callback
	energy_billed_to_[me/others]	nj	bank_rollup_chit_to_tasks	Per-Task
	energy	nj		Bank
res	physical_writes	bytes	thread_update_io_stats	Per task, callback
	logical_writes		task_update_logical_writes	
sfi	SFI_CLASS_...	MATUs	sfi_wait_cleanup	MacOS only (`CONFIG_SCHED_SFI`)

Maintenance

For ledgers to be effective, two conditions need to apply. The first is that the system keep careful track of them. This is what the updaters in Table 9-33 are used for - as part of the resource consumption or free operation, the kernel finds the responsible task/thread and makes sure the ledger notes it appropriately. The update is performed by an `OSAddAtomic64` (called from `ledger_[debit/credit]_thread`, which relieves the need for any locking.

The second condition for effectuating ledgers is their enforcement. As previously mentioned, there are threshold callbacks on some ledgers (as noted in Table 9-33), and there is also the potential case of the thread limit being exceeded. For this, ledgers use an Asynchronous Software Trap (AST). In a few points in the thread lifecycle, but primarily when a ledger is updated (in `ledger_entry_check_new_balance()`), calls are made to `act_set_astledger[_async]` to raise the `AST_LEDGER` reason.

The ledger mechanism is used for the various monitoring facilities, such as `cpumon`, which oversees CPU utilization by threads, and the task I/O monitor. The `thread_cputime_callback` is used to output a message ("process...thread... caught burning CPU!") to the kernel log, and raise a Mach `EXC_LEDGER` exception. If Jetsam is present (`CONFIG_JETSAM`, on *OS variants), the process is killed. The `task_footprint_exceeded` callback calls `memorystatus_on_ledger_footprint_exceeded`, which either warns the process (through a knote) or kills is outright. The `task_io_rate_exceeded` callback also outpus a message ("process ..caught causing excessive I/O") and raises `EXC_LEDGER`, but is otherwise not lethal.

Selective Forced Idle (SFI)

Selective Forced Idle (SFI) is a new mechanism Apple introduced into XNU in Darwin 13. It allows forcing predetermined groups of threads into an idle state, even if they are otherwise normally runnable. SFI defines a window interval (the "off phase"), within which the SFI-enabled threads are put by the scheduler on a wait queue, rather than scheduled. If the window timer fires and an SFI-enabled thread is executing, it is immediately forced to idle (hence the name) through an AST. Outside the interval (the "on phase") threads may be run normally, subject to other scheduler constraints. SFI is an optional feature, determined by `CONFIG_SCHED_SFI`, which is `#defined` in MacOS but not (yet?) in the *OS variants.

The MacOS kernel contains a special section, `__DATA.__sfi_class_reg`, containing an array of `sfi_class_registration_t`. Each structure contains the `class_id`, `class_continuation`, a string `class_name` and a `class_ledger_name` (usually same). These get loaded by `sfi_get_registration_data()` (called from `sfi_early_init()` into the `sfi_classes` array. `sfi_init()` then sets up timer calls and queues for each class.

Output 9-34: SFI classes in `__DATA.__sfi_class_reg`

```
morpheus@Zephyr(~) itool2 -d    DATA,__sfi_class_reg /System/Library/Kernels/kernel
Dumping 512 bytes from 0xffffff8000c9aec0 (Offset 0xa9aec0, __DATA.__sfi_class_reg):
0xffffff8000c9aec0: 02 00 00 00 00 00 00 00
0xffffff8000c9aec8: 0xffffff800036a350          __func_0xffffff800036a350
0xffffff8000c9aed0: 0xffffff8000b16f72          "SFI_CLASS_APP_NAP"
...
0xffffff8000c9b0a0: 09 00 00 00 00 00 00 00
0xffffff8000c9b0a8: 0xffffff800036a190          __func_0xffffff800036a190
0xffffff8000c9b0b0: 0xffffff8000b16d8b          "SFI_CLASS_UTILITY"
0xffffff8000c9b0b8: 0xffffff8000b16d8b          "SFI_CLASS_UTILITY"
```

The main user-mode client of the SFI facility is the `thermald`. To allow management from user mode, SFI provides two new system calls in bsd/kern/kern_sfi.c `sfi_ctl` (#456) controls SFI's global settings, and `sfi_pidctl` (#457) controls SFI's local (i.e. per-process) settings. Neither of the system calls is exported to user mode headers (`NO_SYSCALL_STUB`). Several convenience functions over these system calls are also provided though these, too, aren't officially defined, with their prototypes only in XNU's bsd/sys/sfi.sys:

```
// __sfictl() with SFI_CTL_OPERATION...
int system_set_sfi_window(uint64_t sfi_window_usec); // sfictl with ..._SET_WINDOW
int system_get_sfi_window(uint64_t *sfi_window_usec); // sfictl with ..._SFI_GET_WINDOW
int sfi_set_class_offtime(sfi_class_id_t class_id, uint64_t offtime_usec);  // ..._SET_CLASS_OFFTIME
int sfi_get_class_offtime(sfi_class_id_t class_id, uint64_t *offtime_usec); // ..._GET_CLASS_OFFTIME
// __sfi_pidctl with SFI_PIDCTL_OPERATION...
int sfi_process_set_flags(pid_t pid, uint32_t flags);  // ..._PID_SET_FLAGS
int sfi_process_get_flags(pid_t pid, uint32_t *flags); // ..._PID_GET_FLAGS
```

The minimum SFI window size is `MIN_SFI_WINDOW_USEC` , set in osfmk/mach/sfi_class.h to be 500. The header also mentions another function, `coalition_set_sfi_class`, but this function apparently has yet to be implemented, presumably over the `coalition_info()`.

SFI uses several classes, which map to the thread QoS levels introduced in Darwin 13. Additional refinements are the `_FOCAL` and `_NONFOCAL` distinctions, which are determined by the QoS role and visibility status of the app.

Table 9-35: SFI classes

#	SFI_CLASS_ code	
1	_DARWIN_BG	setpriority(2) W/PRIO_DARWIN_[THREAD/PROCESS]
2	_APP_NAP	App Nap
3-4	_MANAGED[_FOCAL/_NONFOCAL]	Set by [process/coalition]_set_sfi_class()
5-6	_DEFAULT[_FOCAL/_NONFOCAL]	Otherwise unspecified
7	_KERNEL	Kernel threads; generally kept off
8	_OPTED_OUT	Real time and otherwise critical threads that are never subject to SFI
9	_UTILITY	Per corresponding QoS class
10-11	_LEGACY[_FOCAL/_NONFOCAL]	
12-13	_USER_INITIATED[_FOCAL/NONFOCAL]	
14-15	_USER_INTERACTIVE[_FOCAL/NONFOCAL]	
16	_MAINTENANCE	

Review Questions

- Why would re-enabling interrupts (using `ml_set_interrupts_enabled(TRUE)`) mandate checking for any urgent software traps?
- Why is it ok for a continuation to return through `unix_syscall_return`? What other (similar) function may be used instead?
- Why is it important for the kernel to know if a given `processor` object is SMP or SMT?
- How does the Mach scheduler `struct runqueue` resemble the Linux O(1) scheduler? What major drawback cause Linux to abandon O(1)? How could that be recitfied in the case of XNU?
- How could handoff be applied to Mach messaging? What would be the advantages of doing so?
- When the load average is reported (via `top(1)`, `procexp(j)`, `w(1)` and `uptime(2)`, or the `vm.loadavg sysctl(2)` MIB), three averages are provided. What are they, and what does it mean when the values are monotonically increasing? decreasing? Otherwise?
- Why are turnstiles allocated per-thread, rather than per-lock?

References

1. NewOSXBook.com - Thread injection sample - http://newosxbook.com/src.jl?tree=listings&file=inject.c
2. Draves, et al. - "Using Continuations..." - http://zoo.cs.yale.edu/classes/cs422/2010/bib/draves91continuations.pdf
3. solts Jim Mauro - Inside Solaris (SunWorld, August 1999) - Turnstiles and priority inheritance - http://sunsite.uakom.sk/sunworldonline/swol-08-1999/
4. FreeBSD sources - Turnstiles - https://github.com/freebsd/freebsd/blob/master/sys/sys/turnstile.h
5. McKusick, Neville-Neil & Watson - "The Design and Implementation of the FreeBSD Operating System" (2nd Edition) - ISBN -978-0321968975

10

Mixed Messages: Mach IPC

Our previous discussion of Mach ports (in I/11) left them at the abstract level of the `mach_port_t`, which was (to the process) nothing more than an opaque identifier, much like a file descriptor. In kernel mode, however, Mach ports are elaborate structures with far more to them than is exposed to user mode.

This chapter explores Mach ports and IPC objects at the lowest level possible - their kernel implementation. We begin with the `ipc_space_t`, which provides the namespace in which the task ports "live" in. We next look at what an `ipc_port` looks like in kernel space, and the many types of ports which exist.

The focus next is on messaging - looking behind the scenes of `mach_msg` - which, while efficient, is far from trivial. We pay special attention to message descriptors in kernel space, exploring them both for their power and for their potential to be abused.

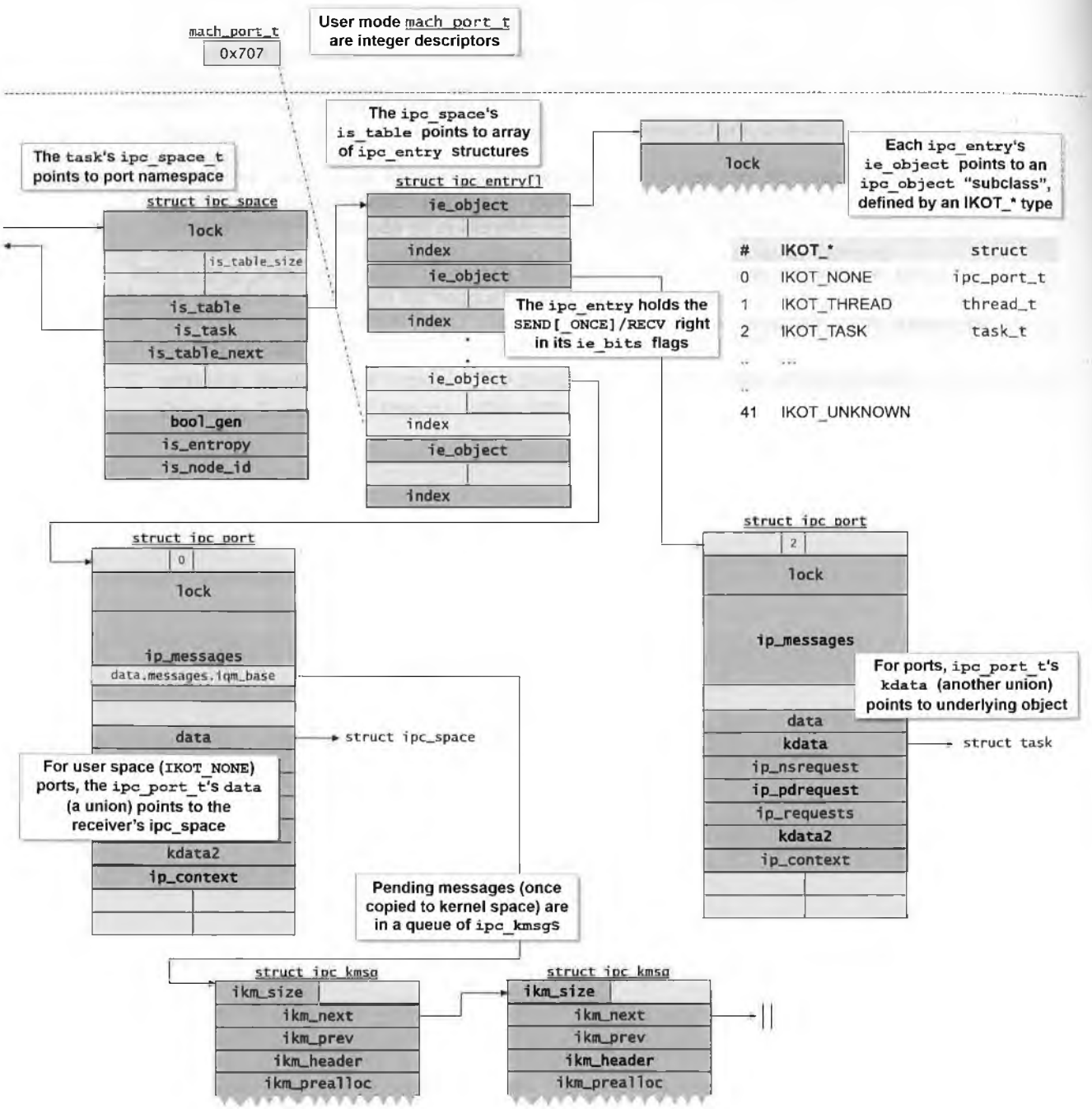
Lastly, we examine vouchers - marking the first change to Mach's messaging core in years - and not necessarily for the better. Lastly, we consider support for multinode, which may someday (re-)enable Mach messages between hosts.

The High Level View

Mach is, first and foremost, a kernel optimized for message passing. The beauty is that the user mode API's aspects previously discussed (in I/11 and I/12) are fairly simple to understand and use. Behind the scenes, however, is a rather complex infrastructure supporting and implementing the constructs that are rights, ports and messages.

Figure 10-1 depicts a high level view of the Mach IPC subsystem. Although undergoing a few changes over the years, the XNU implementation of Mach messaging still closely resembles the original one devised by Tevanian, Draves et al. Port descriptors in user space correlate with `ipc_entry_t`s in kernel space, which reside in the task's `ipc_space`. The `ipc_entry`s point to the actual port objects through their `ie_object` fields, and the ports refer to their underlying objects through their `kdata`, with messages copied to kernel space and queued on the ports' `ip_messages`, an `ipc_mqueue_t`.

Figure 10-1: The high level view of Mach IPC



Task ipc_space_t

Recall that, as a process is a container for file descriptors, so is a task a container for ports. User-mode tasks view their ports as 32-bit `mach_port_t` values. In kernel-mode, however, these values are interpreted as handles into a task's `itk_space`. Similarly to file descriptors (the `struct proc`'s `p_fd`), the `itk_space` provides a way to interpret integer numbers into the "true" kernel structures supporting ports - the `ipc_port_t`.

Unlike with process file descriptors, however, for which the descriptor value is merely an array index, obtaining the `ipc_port_t` is a bit more involved. The `itk_space` is not a simple array, but a pointer to a `struct ipc_space`, defined in osfmk/ipc/ipc_space.h as follows:

Listing 10-2: The `struct ipc_space` definition (from XNU 4570.41.2's osfmk/ipc/ipc_state.h)

```
/*
 *      Every task has a space of IPC capabilities.
 *      IPC operations like send and receive use this space.
 *      IPC kernel calls manipulate the space of the target task.
 *
 *      Every space has a non-NULL is_table with is_table_size entries.
 *
 *      Only one thread can be growing the space at a time.  Others
 *      that need it grown wait for the first.  We do almost all the
 *      work with the space unlocked, so lookups proceed pretty much
 *      unaffected while the grow operation is underway.
 */

typedef natural_t ipc_space_refs_t;
#define IS_REFS_MAX     0x0fffffff
#define IS_INACTIVE     0x40000000      /* space is inactive */
#define IS_GROWING      0x20000000      /* space is growing */

struct ipc_space {
        lck_spin_t      is_lock_data;
        ipc_space_refs_t is_bits;       /* holds refs, active, growing */
        ipc_entry_num_t is_table_size;  /* current size of table */
        ipc_entry_num_t is_table_free;  /* count of free elements */
        ipc_entry_t is_table;           /* an array of entries */
        task_t is_task;                 /* associated task */
        struct ipc_table_size *is_table_next; /* info for larger table */
        ipc_entry_num_t is_low_mod;     /* lowest modified entry during growth */
        ipc_entry_num_t is_high_mod;    /* highest modified entry during growth */
        // Added in Darwin 17.2 (XNU 4570.31)
struct bool_gen bool_gen;       /* state for boolean RNG */
        unsigned int is_entropy[IS_ENTROPY_CNT]; /* pool of entropy taken from RNG */
        // Added in Darwin 16
        int is_node_id;          /* HOST_LOCAL_NODE, or remote node if proxy space */

};
```

All `ipc_space` structures are allocated using a call to `ipc_space_create()`, save for two special spaces - that of the kernel (`ipc_space_kernel`) and another used for reply ports (`ipc_space_reply`). These (created with `ipc_space_create_special`) serve as placeholders for naked receive rights. In all cases, however, the spaces are allocated using an `is_alloc()` macro, which `zalloc()`s them from their own zone, `ipc spaces`. Note that, due to the `lck_spin_t` at the beginning of the space object, the size will differ between MacOS and *OS variants by 8 bytes.

The main element in the `ipc_space_t` is the `is_table`*. Although it is an array, the 32-bit port name is not a simple index into it. Rather, it is broken into the most significant 24 bits as an index, and the least significant 8 bits as a "generational" number:

Listing 10-3: The `mach_port_name` breakdown

```
#define MACH_PORT_INDEX(name)           ((name) >> 8)
#define MACH_PORT_GEN  (name)           (((name) & 0xff) << 24)
#define MACH_PORT_MAKE (index, gen)      \
        (((index) << 8) | (gen) >> 24)
```

* - XNU versions prior to 2050 (Darwin 12) also used an `is_tree` which was a splay tree (self adjusting binary tree which favors recently accessed elements), and `ie_index` was a union containing either a table or a tree.

Port names used to be quite predictable (monotonically increasing), but as Listing 10-4 shows, the structure has recently (Darwin 17.2) been expanded to provide entropy, in an effort to make port names (specifically, the generational count) more difficult to guess, in the face of name collision or prediction attacks. As of that version, the `sizeof(ipc_space_t)` is 96 on MacOS and 88 bytes on the *OS variants.

Each entry in the `is_table` represents a right to a port, in an `ipc_entry` structure. The `is_table` is created with an initial number of entries as specified in the argument to `ipc_space_create()`, and may dynamically grow. Unlike the `ipc_space` structure, the `is_table` is `kalloc`ed, and therefore does not fall into a dedicated zone.

Listing 10-4: The `struct ipc_entry` definition (from osfmk/ipc/ipc_entry.h)

```
/*
*      Spaces hold capabilities for ipc_object_t's.
*      Each ipc_entry_t records a capability.  Most capabilities have
*      small names, and the entries are elements of a table.
*
*      The ie_index field of entries in the table implements
*      a ordered hash table with open addressing and linear probing.
*      This hash table converts (space, object) -> name.
*      It is used independently of the other fields.
*
*      Free (unallocated) entries in the table have null ie_object
*      fields.  The ie_bits field is zero except for IE_BITS_GEN.
*      The ie_next (ie_request) field links free entries into a free list.
*
*      The first entry in the table (index 0) is always free.
*      It is used as the head of the free list.
*/
struct ipc_entry {
        struct ipc_object *ie_object;
        ipc_entry_bits_t ie_bits;
        mach_port_index_t ie_index;
        union {
                mach_port_index_t next;     /* next in freelist, or...  */
                ipc_table_index_t request; /* dead name request notify */
        } index;
};
```

The `ipc_entry_lookup(space, name)` function (in osfmk/kern/ipc_entry.c) is used to resolve the port `name` in the IPC `space` to its `struct ipc_entry`.

Listing 10-5: The `ipc_entry_lookup` implementation (from osfmk/ipc/ipc_entry.c)

```
ipc_entry_t
ipc_entry_lookup(
ipc_space_t             space,
mach_port_name_t        name)
{
        mach_port_index_t index;
        ipc_entry_t entry;

        assert(is_active(space));

        index = MACH_PORT_INDEX(name);
        if (index <  space->is_table_size) {
                entry = &space->is_table[index];
                if (IE_BITS_GEN(entry->ie_bits) != MACH_PORT_GEN(name) ||
                    IE_BITS_TYPE(entry-gt;ie_bits) == MACH_PORT_TYPE_NONE) {
                        entry = IE_NULL;
                }
        }
        else { entry = IE_NULL; }

        assert((entry == IE_NULL) || IE_BITS_TYPE(entry->ie_bits));
        return entry;
}
```

Output 10-6 (next page) demonstrates the IPC space from both the user mode and kernel mode perspectives. A simple port lister program using `mach_port_space_info()` (left as an exercise for the reader) outputs its ports and sleeps. While it does, `xnoop(j)` is used to inspect those same ports from kernel mode, and then dump the raw kernel memory.

Output 10-6: Demonstrating `ipc_space_t`, from user mode and kernel

```
root@Chimera (~)# /tmp/mp &
[1] 7578
Port 1: 0x103,  Type: 0x10000  Object 0x3cfae159  Refs: 2 mach_task_self
Port 2: 0x203,  Type: 0x20000  Object 0x3cfada21  Refs: 0
Port 3: 0x307,  Type: 0x10000  Object 0x3cfaf311  Refs: 1 mach_thread_self
Port 4: 0x403,  Type: 0x20000  Object 0x3cfac869  Refs: 0
Port 5: 0x503,  Type: 0x20000  Object 0x3cfacd01  Refs: 0
Port 6: 0x607,  Type: 0x20000  Object 0x3cfadf61  Refs: 0
Port 7: 0x707,  Type: 0x10000  Object 0x2f9f9671  Refs: 4 bootstrap port
Port 8: 0x803,  Type: 0x10000  Object 0x2c742829  Refs: 1
Port 9: 0x903,  Type: 0x10000  Object 0x3cfaedd1  Refs: 1
Port 10: 0xa03, Type: 0x30000  Object 0x3cfae549  Refs: 1
#
# view from inside the kernel, with xnoop:
#
bash-3.2# xnoop ports 7578
Task IPC space is @0xffffff802cec15c0, containing table @0xffffff802b7b8000 with 21 entries
0x103: SEND urefs: 2 to obj@0x..ff803a151c38: TASK:  PID 7578, mp (kdata: 0xffffff802d00eaf0)
0x203: RECV urefs: 0 to obj@0x..ff803a151500: NONE(0), Receiver: PID 7578, mp (0xffffff802cec15c0)
0x307: SEND urefs: 1 to obj@0x..ff803a152df0: THREAD (1) of PID, kdata: 0xffffff802e544ed0
0x403: RECV urefs: 0 to obj@0x..ff803a150348: NONE(0), Receiver: PID 7578, mp (0xffffff802cec15c0)
0x503: RECV urefs: 0 to obj@0x..ff803a1507e0: NONE(0), Receiver: PID 7578, mp (0xffffff802cec15c0)
0x607: RECV urefs: 0 to obj@0x..ff803a151a40: NONE(0), Receiver: PID 7578, mp (0xffffff802cec15c0)
0x707: SEND urefs: 4 to obj@0x..ff802cb9d150: NONE(0), Receiver: PID 1, launchd (0xffffff80298e02c0)
0x803: SEND urefs: 1 to obj@0x..ff80298e6308: CLOCK(25), kdata: 0xffffff8018251230, Dest: 0xffffff80298e0200
0x903: SEND urefs: 1 to obj@0x..ff803a1528b0: SEMAPHORE(23), kdata: 0xffffff80361e0ae0, Dest: 0xffffff80298e0200
0xa03: R/S  urefs: 1 to obj@0x..ff803a152028: NONE(0), Receiver: PID 7578, mp (0xffffff802cec15c0)
#
# The ipc_space_t, in kernel:
#
bash-3.2# xnoop dump 0xffffff802cec15c0,96 smart
0xffffff802cec15c0 00 00 00 00 00 00 00 00   02 00 00 00 15 00 00 00 ................
                                               kalloc.512
0xffffff802cec15d0 0a 00 00 00 ef be ad de      0xffffff802b7b8000   ..........{+....
                     (PID 7578, mp)              kalloc.1024
0xffffff802cec15e0    0xffffff802d00eaf0        0xffffff80298e1804   ..-.....*.......
0xffffff802cec15f0 01 00 00 00 0a 00 00 00   09 d4 91 17 af f5 b8 67 ...............g
0xffffff802cec1600 8e 86 50 6f 24 14 78 2d   00 00 00 00 ef be ad de ..Po$.x-........
0xffffff802cec1610 00 00 00 00 00 00 00 00   dd c2 ac 00 ff ff ff ff ................
#
# The first four ie_entry_t structures in kalloc.512 (0 is reserved)
#
bash-3.2# xnoop dump 0xffffff802b7b8000,110  smart
0xffffff802b7b8000 00 00 00 00 00 00 00 00   00 00 00 ff 00 00 00 00 ................
                                               Task (PID 7578, mp)
0xffffff802b7b8010 00 00 00 00 00 00 00 00    0xffffff803a151c38   ........8.......
0xffffff802b7b8020 02 00 81 03 00 00 00 00   00 00 00 00 00 00 00 00 ................
                     NONE (PID 7578, mp)
0xffffff802b7b8030    0xffffff803a151500      00 00 c2 03 00 00 00 00 ................
                                               THREAD (PID 7578, mp)
0xffffff802b7b8040 00 00 00 00 00 00 00 00    0xffffff803a152df0    ........-.......
0xffffff802b7b8050 16 00 81 07 00 00 00 00   00 00 00 00 00 00 00 00 ................
..
```

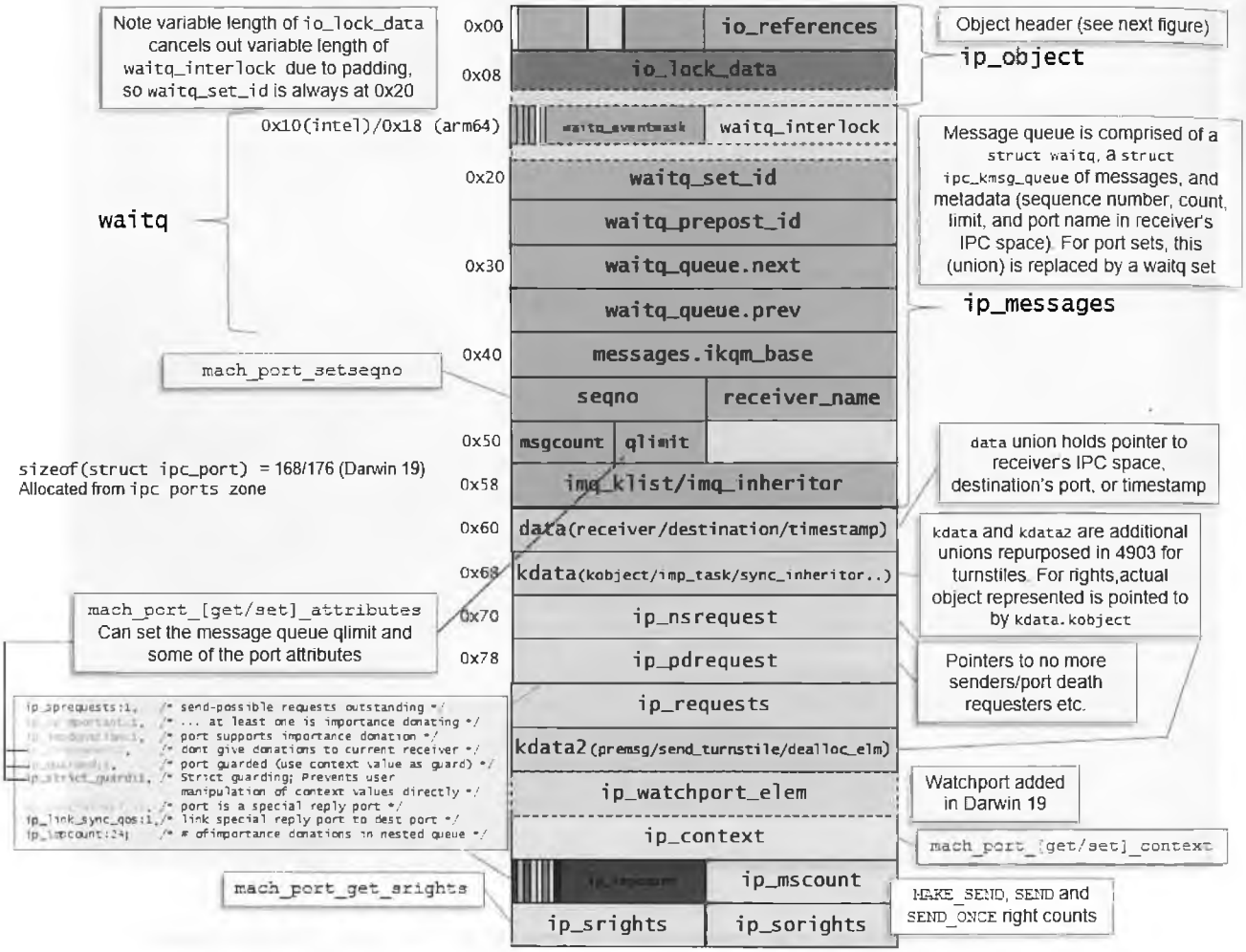
The `ipc_space_t` needs to be locked during the operation. If the space is already locked (as it is, for example in `ipc_kmsg_copyin_header()` or in `ipc_mqueue_copyin()`) then `ipc_entry_lookup` can be used directly. Otherwise, `ipc_entry_lookup` is often called through the `ipc_right_lookup_[read/write] (space, name, out *entryp)` wrappers. These lock the `ipc_space_t`, ensure it is active, and call `ipc_entry_lookup`, returning the `entry` through the supplied pointer. Although `ipc_right_lookup_read` and `...write` may imply two types of locks, the former is a macro aliasing to the latter. Another useful wrapper is `ipc_object_translate(space, name, right, out *objectp)`, which calls on `ipc_right_lookup_read`, and on success returns the `ipc_entry`'s `ie_object`, locked.

Because inevitably each IPC object has a unique representation in kernel space, All `ipc_entry` references to that object, irrespective of `ipc_space_t`, must have the same `ie_object` value. This value thus traditionally doubled as the unique identifier of the object, and was retrievable in user mode via the `mach_port_space_info` routine of the `mach_port` subsystem (#3223). This persisted for decades until the advent of KASLR, as of which leaking kernel addresses to user mode constitutes a vulnerability. `mach_port_space_info` routine was modified to return a permuted address, but this, too, had repeated issues. As of Darwin 15, `mach_port_space_info` only operates on release kernels if the MAC framework `expose_task` hook agrees (which it does on MacOS). This permuted value is the one displayed by `lsmp`, as shown in the above output, and the experiment in I/11.

The ipc_port

Mach ports are, unsurprisingly, implemented as `ipc_port` structures. These are considerably detailed structures, which are all allocated from the dedicated `ipc ports` zone. Figure 10-7 shows such a structure, as visualized from its definition in osfmk/ipc/ipc_port.h:

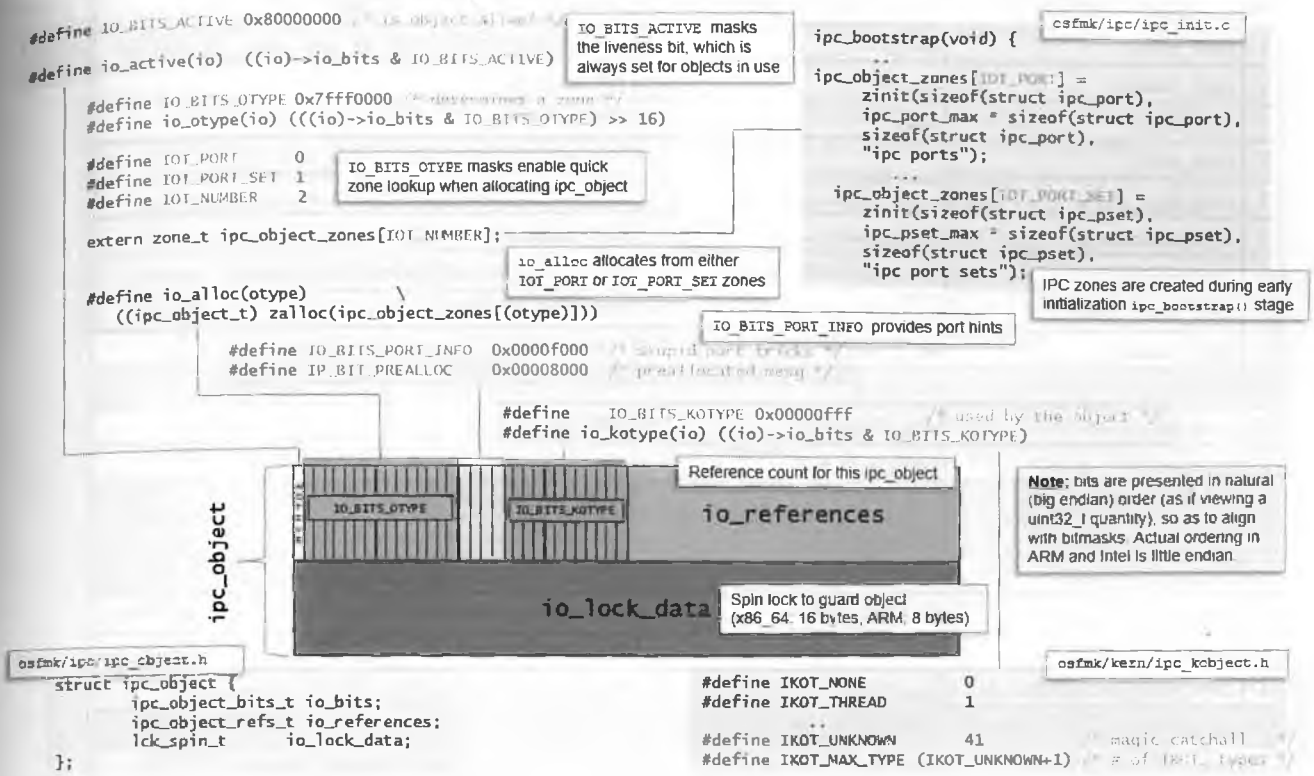
Figure 10-7: The `struct ipc_port` (from osfmk/ipc/ipc_port.h), visualized



The port structure has been revised and extended over time, most recently to support importance donations and Darwin 18's turnstiles (discussed later). Using `unions` helps keep the fields compacted by repurposing their space when not needed. To keep field references manageable, ipc_port.h defines the union fields as `ip_*` macros. Darwin 19 adds a new element (`ip_watchport_elem`, after the `kdata2`, which pushes some fields ahead and increases the structure size to 176, from 168 previously. Should `MACH_FLIPC` ever be enabled, this will add the `flipc_port` pointer to the port's `ipc_mqueue`, extending the entire port structure (at 0x58, if all else remains the same) by another 8 bytes.

If you look back at the `ipc_entry` structure, you will note that the `ie_object` field is a pointer to a `struct ipc_object`. This poses no problem, since the first field of the `ipc_port` structure is the `ip_object`, itself a `struct ipc_object`. Indeed, the `ipc_object` structure was designed in mind as a common header, so that specific object types could simply place their own data after it (akin to C++ subclassing). The common header, shown in Figure 10-8, is as minimalistic as it could be:

Figure 10-8: Visualizing the `struct ipc_object` definition (from osfmk/ipc/ipc_object.h)



The `io_bits` field, which is a `natural_t` (i.e. `uint32_t`) is split into subfields: its most significant bit, `IO_BITS_ACTIVE`, is used to mark the object as "alive". The next 15-bits function as the `IO_BITS_OTYPE`, providing a quick lookup of which zone the object is to be maintained in (though in practice only two zones, `IOT_PORT` and `IOT_PORT_SET`, are used). The next four bits are masked as `IO_BITS_PORT_INFO`, which despite a descriptive comment is unused. Lastly, the least significant 12 bits allow specifying an `IKOT_` constant, which provides the object type. Corresponding macros (`io_kotype` and `io_otype`) allow for quick isolation of the subfields of `io_bits`.

The various object types are `#defined` in osfmk/kern/ipc_kobject.h. Over the years Apple has added its own object types. Those types defined as of XNU 4570 are shown in Table 10-9 (next page), with apparently unused ones indicated by shaded rows. There are cases where the same underlying `ip_kobject` type is used, albeit with a different `IKOT_*` constant (for example, `IKOT_HOST[/_PRIV/_SECURITY]`, `IKOT_TASK[/_NAME/RESUME]` etc) in which cases the `io_kotype` is used to differentiate the object's usage methods or priveleges.

Going back to Listing 10-7, the actual object the port represents is in its `ip_kobject`. This was originally a member field, but with Apple's expansion of ports for importance and and QoS (Darwin 13), the field was recast as the `kdata` union. Nonetheless, accessing it remains essential the same, as `ip_kobject` is now a macro expanded to `kdata.ipc_kobject`. Further, the field is seldom set directly: It has a setter in `ipc_kobject_set`, which takes the `ipc_port_t`, the `ipc_kobject_t` and the `ipc_kobject_type_t` as arguments, and sets the `ip_bits` and `ip_kobject` fields of the `ipc_port_t`. This is done after taking the `io_lock_data` lock, but the lighter implementation (`ipc_kobject_set_atomically`) can be called for faster operation if the port object is already locked.

Getting the `ip_kobject` back out of a port is commonly performed by a call to one of the corresponding conversion functions (`convert_port_to_...`), spread around in various locations throughout the kernel (easy to locate using `jtool -S` on the kernel and piping to `grep convert_port`). These commonly verify the `IKOT_` type in the `io_bits` (using the `ip_kotype(port)` macro), handle any reference counting (through the object's reference counter function, e.g. `[task/thread]_reference_internal`, `is_reference()`, etc), and return the `ip_kobject` fields. Here, too, there exist some `_locked_` variants which return the object after taking a lock.

Table 10-9: Kernel object types (from osfmk/kern/ipc_kobject.h)

#	IKOT_* constant	ip_kobject	Use
0	IKOT_NONE	ipc_port_t	User-mode ports (created by mach_port_allocate)
1	IKOT_THREAD	thread_t	Thread ports
2	IKOT_TASK	task_t	Task ports
3	IKOT_HOST	realhost	The host port (from mach_host_self())
4	IKOT_HOST_PRIV		The host priveleged port (from mach_host_priv())
5	IKOT_PROCESSOR	processor	Host processor
6	IKOT_PSET	pset	Host processor set (presently only pset0)
7	IKOT_PSET_NAME		Processor set name
8	IKOT_TIMER	mk_timer_t	Timer (from mk_timer_[create/destroy]_trap
9	IKOT_PAGING_REQUEST	(unused)	Paging request to External Memory Manager
10	IKOT_MIG	mig_object_t	MIG objects (convert_mig_object_to_port())
11	IKOT_MEMORY_OBJECT	memory_object	Mach pagers
12	IKOT_XMM_PAGER	(Unused)	External Memory Manager pager objects
13	IKOT_XMM_KERNEL		External Memory Manager, kernel
14	IKOT_XMM_REPLY		External Memory Manager, reply objects
15	IKOT_UND_REPLY	UNDReplyRef	User Notification Daemon reply (from kuncd, HSP #10)
16	IKOT_HOST_NOTIFY	host_notify_t	Host notification port
17	IKOT_HOST_SECURITY	realhost	Host security port
18	IKOT_LEDGER	(unused)	Ledger objects
19	IKOT_MASTER_DEVICE	void *	The IO_MASTER port (IORegistry handle, HSP #3)
20	IKOT_TASK_NAME	task	Task names
21	IKOT_SUBSYSTEM	(unused)	
22	IKOT_IO_DONE_QUEUE	(unused)	
23	IKOT_SEMAPHORE	semaphore_t	Mach semaphore objects
24	IKOT_LOCK_SET	(unused)	Lock sets (removed as of Darwin @TODO)
25	IKOT_CLOCK	clock_t	The Host's clock port (from @TODO)
26	IKOT_CLOCK_CTRL		The Host's clock control port (from @TODO)
27	IKOT_IOKIT_SPARE	(unused)	
28	IKOT_NAMED_ENTRY	vm_named_entry_t	Named memory entry ([mach_]make_memory_entry())
29	IKOT_IOKIT_CONNECT	IOUserClient	IOServiceOpen()ed user clients (io_connect_t)
30	IKOT_IOKIT_OBJECT	(various)	IORegistry/driver entries, iterators, etc
31	IKOT_UPL	upl_t (unused)	Universal Page List
32	IKOT_MEM_OBJ_CONTROL	memory_object_control_t	Pager control port
33	IKOT_AU_SESSIONPORT	auditinfo_addr	Audit session port
34	IKOT_FILEPORT	fileglob	Darwin 11: File descriptors wrapped by a Mach port
35	IKOT_LABELH	Unused	Darwin 11: MACF Labels
36	IKOT_TASK_RESUME	task	Darwin 12: Task suspension/resumption token
37	IKOT_VOUCHER	ipc_voucher_t	Darwin 14: Voucher (ipc_voucher_t)
38	IKOT_VOUCHER_ATTR_CONTROL	ipc_voucher_attr_control_t	Darwin 14: Voucher attribute control
39	IKOT_WORK_INTERVAL	work_interval	Darwin 17: Work interval
40	IKOT_UX_HANDLER	ux_handler_kobject	Darwin 18: UNIX exception handler port
--	IKOT_UNKNOWN		Marks end of IKOT values (varies with version).

Conversion in the opposite direction - from an object type right to a port - is naturally achievable using inverse functions. These mostly wrap around `ipc_port_make_send()`, which increments the port's `ip_mscount` and `ip_srights` counts.

User mode code ends up primarily invoking conversion functions as it makes use of the task, thread and vm_map ports, whose significance was explained and stressed in I/12. Darwin 16 extended the task and (ipc) space types with `_inspect` counterparts, although at present these are identical to the raw types. Presumably, this is aimed to address the gaping flaws which result from unfettered access to the raw types, as the `task_inspect` object have begun replacing the `task_t` in the MIG definitions of the task.defs subsystem*. Starting in Darwin 17, XNU uses `task_coversion_eval()` (from osfmk/kern/ipc_tt.c) when converting a port to a task, to provide an extra measure of security by preventing access to platform applications (`TF_PLATFORM`) from non-platform ones.

The `mach_port_kernel_object` and `mach_port_kobject` routines of the `mach_port` subsystem (#3225 and #3230, respectively) could traditionally be used to leak the address of the `ip_kobject` (or its last 32-bits, for the former) to user mode. The was a well known disclosure in XNU, which was only supposed to be compiled in on `MACH_IPC_DEBUG`. Nonetheless, it was present in iOS kernels and so proved useful in jailbreaks from iOS 7 through 8.4. Apple at first tried to permute the value, but eventually gave up and just wrapped the leaking code by another `#ifdef DEVELOPMENT || DEBUG`, leaving it out of the kernel for good, and instead returning 0 in both cases.

The extensibility of the `ipc_object` structure has led to an oft-used exploitation method: Given a kernel memory read/write primitive, it can be applied to an existing port object, so as to polymorph it into any desired type. All it takes is changing the `io_bits` and the `kdata` fields. In this way, jailbreaks (such as 10.2's Yalu) start off with a generic port, then (by exploiting a Use-After-Free) change it to a clock port (to help bypass KASLR), or a task port (to smuggle the `kernel_task` to user mode). More uses of this technique are discussed in Volume III (specifically, Chapters 24 and 25).

Case Study: resolving a port name to the underlying object address

The kernel is often required to resolve a port right to the underlying port object in kernel mode. This is referred to as **translating**, and is handled by the `ipc_object_translate()` routine. Defined in osfmk/ipc/ipc_object.c, it calls `ipc_right_lookup_read()` on the target *space* and port *name*, returning the *entry* by reference. Assuming the entry is of the valid *right* type, it is resolved to its `ie_object`, which is returned to the caller. A common practice is to use the `ipc_port_translate_[receive/send]` wrapper macros, which call `ipc_object_translate()` with the corresponding *right* set to `MACH_PORT_RIGHT_[RECEIVE/SEND]`.

Another common scenario is when taking port names from user mode (commonly, from message headers or descriptors), and resolving them to the `ipc_port_t`s. The `ipc_object_copyin()` function (from osfmk/kern/ipc_object.c) is an important function used to do this. Given an `ipc_space_t` and a `mach_port_name_t` in it, it retrieves the underlying `ipc_object_t`, which it returns (as an out parameter), according to the disposition set by its `mach_msg_type_name_t` argument. More discussion on the exact steps can be found in Figure 10-18 and the discussion of message descriptors, later.

The `port_name_to_[thread/task/task_inpsect]` (in osfmk/kern/ipc_tt.c), which all operate on a `mach_port_name_t` argument, make use of `ipc_object_copyin()` (calling it on the `current_space()`), supplying a disposition of `MACH_MSG_TYPE_COPY_SEND`, and receive the object into a local `ipc_port_t`. The port can then undergo conversion to the target object (i.e. thread, task or task_inspect) by verifying its `ip_kotype()` is the correct `IKOT_*` type, and then dereferencing the `ipc_port_t`'s `ip_kobject` field. Once the actual kernel object is retrieved, the retrieved `SEND` right is dropped using `ipc_port_release_send()`. The `port_name_to_host` functions stands out from its siblings, in that it uses `ipc_port_translate_send` (a macro over `./osfmk/ipc/ipc_object.c`), and does not increase the reference count.

Port lifecycle

Now that we've seen what port and their rights actually look like in kernel mode, let's examine the lifecycle - creation, destruction, and usage.

Port allocation

Mach port rights are normally created when a user mode client requests them, using one of the `mach_port_allocate` variants or the `mach_port_construct` call of the `mach_port` MIG subsystem, or the `_kernelrpc_mach_port_[allocate/construct]` traps (#16 and #24). The choice doesn't matter much, as the Mach traps (in osfmk/ipc/mach_kernelrpc.c) and the MIG calls end up taking the same path in the kernel, implemented by functions in osfmk/ipc/mach_port.c.

As discussed in I/11, the the `mach_port_construct` variant provides a richer interface to the port creation operation, which includes setting various options (e.g. guard) and the context value. Using `...construct` is more of a convenience, as all the parameters can also be set later during the port's lifetime using `mach_port_set_options`, `mach_port_[un]guard` etc. Using the `mach_port_allocate_name` variant may in some cases be more useful, since it can request any given port name (as an *in* parameter) rather than receive a dynamically generated out (as the *out* parameter all the other calls use).

Restricting the discussion, then, to the `mach_port_allocate_*` variants, all the implementations in osfmk/ipc/mach_port.c end up funneling to `mach_port_allocate_full` (which is also accessible for the masochistic user mode caller as MIG message #3220). This function is thus called because it takes on the full functionality of the other variants, accepting optional pointers to a `mach_port_qos_t` pointer and `mach_port_name_t`.

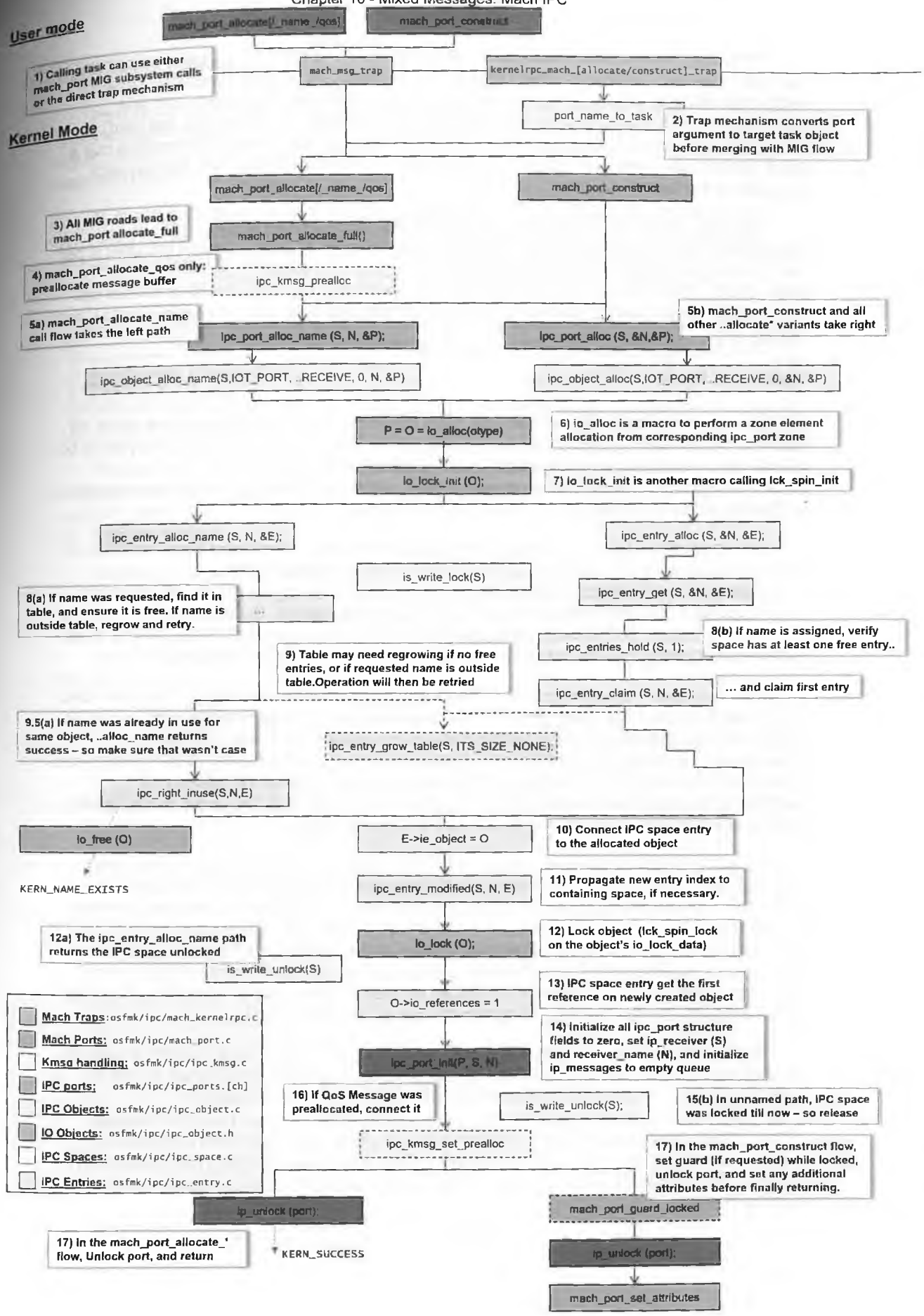
Following a few quick argument sanity checks, `mach_port_allocate_full()` inspects the fields of the `qosp` structure passed by reference by the caller (or other wrapper). If the `prealloc` flag is set, it specifies a need to preallocate a message buffer for the port, which results in a a call to `ipc_kmsg_prealloc()`. Another flag, `name`, may be true if set by the `mach_port_allocate_name` wrapper, and this is used in the subsequent allocation calls.

`mach_port_allocate_full()` will handle the allocation using the respective `ipc_...alloc()` variant, as dictated by the requested `right`. This is commonly `MACH_PORT_RIGHT_RIGHT_RECEIVE`, but may also be `MACH_PORT_RIGHT_PORT_SET`, or `MACH_PORT_RIGHT_DEAD_NAME`. The first is the most important one, so let's examine it first, along with Figure 10-9, on the opposite page.

The `MACH_PORT_RIGHT_RECEIVE` type requires the port to be created, so a call is made to `ipc_object_alloc()` or `..alloc_name()`, based on the boolean indication of the `mach_port_qos_t`'s `name` field (which was set by the `mach_port_allocate[_name]` wrapper). If successful, these will return the `ipc_port_t` (as an out parameter), and handle the stitching together of the `ipc_entry` in the IPC space to the connected `ie_object`.

If the `qos prealloc` flag was specified, recall that we had preallocated a buffer. If the port was successfully created, this buffer will be stashed in the `ipc_port`'s `ip_premsg` (in the `kdata2` union), and the `ipc_object` header's `io_bits` are flagged with `IP_BIT_PREALLOC` (0x8000, see Figure 10-7) to indicate the semantics of the union. If the allocation of the port is successful, the allocated object is returned locked, so `.._allocate_full()` unlocks it, and the port name is returned to the caller (in an out parameter), ready for use.

The interesting part of the flow is `ipc_object_alloc[_name]()`, in osfmk/kern/ipc_object.c. Allocating an IPC object involves both allocating the object itself (using `io_alloc`) and its IPC space entry (using `ipc_entry_alloc[_name]`, respective of the origin call). In the case of the `.._name` call, another check must be made to see if the name was not already occupied (using `ipc_right_inuse()`). A call to `ipc_entry_modified()` then follows in either case, to adjust the IPC space if necessary.


User mode
mach_port_allocate[/_name_/qos]
mach_port_construct
1) Calling task can use either mach_port MIG subsystem calls or the direct trap mechanism
mach_msg_trap
kernelrpc_mach_[allocate/construct]_trap
Kernel Mode
port_name_to_task
2) Trap mechanism converts port argument to target task object before merging with MIG flow
mach_port_allocate[/_name_/qos]
mach_port_construct
3) All MIG roads lead to mach_port allocate_full
mach_port_allocate_full()
4) mach_port_allocate_qos only: preallocate message buffer
ipc_kmsg_prealloc
5a) mach_port_allocate_name call flow takes the left path
ipc_port_alloc_name (S, N, &P);
ipc_port_alloc (S, &N,&P);
5b) mach_port_construct and all other ..allocate* variants take right
ipc_object_alloc_name(S,IOT_PORT, ..RECEIVE, 0, N, &P)
ipc_object_alloc(S,IOT_PORT, ..RECEIVE, 0, &N, &P)
P = O = io_alloc(otype)
6) io_alloc is a macro to perform a zone element allocation from corresponding ipc_port zone
io_lock_init (O);
7) io_lock_init is another macro calling lck_spin_init
ipc_entry_alloc_name (S, N, &E);
ipc_entry_alloc (S, &N, &E);
is_write_lock(S)
ipc_entry_get (S, &N, &E);
8(a) If name was requested, find it in table, and ensure it is free. If name is outside table, regrow and retry.
...
ipc_entries_hold (S, 1);
8(b) If name is assigned, verify space has at least one free entry..
9) Table may need regrowing if no free entries, or if requested name is outside table.Operation will then be retried
ipc_entry_claim (S, N, &E);
... and claim first entry
9.5(a) If name was already in use for same object, ..alloc_name returns success – so make sure that wasn't case
ipc_entry_grow_table(S, ITS_SIZE_NONE);
ipc_right_inuse(S,N,E)
io_free (O)
E->ie_object = O
10) Connect IPC space entry to the allocated object
KERN_NAME_EXISTS
ipc_entry_modified(S, N, E)
11) Propagate new entry index to containing space, if necessary.
12a) The ipc_entry_alloc_name path returns the IPC space unlocked
io_lock (O);
12) Lock object (lck_spin_lock on the object's io_lock_data)
is_write_unlock(S)
O->io_references = 1
13) IPC space entry get the first reference on newly created object
Mach Traps:osfmk/ipc/mach_kernelrpc.c
Mach Ports: osfmk/ipc/mach_port.c
Kmsg handling: osfmk/ipc/ipc_kmsg.c
IPC ports: osfmk/ipc/ipc_ports.[ch]
IPC Objects: osfmk/ipc/ipc_object.c
IO Objects: osfmk/ipc/ipc_object.h
IPC Spaces: osfmk/ipc/ipc_space.c
IPC Entries: osfmk/ipc/ipc_entry.c
ipc_port_init(P, S, N)
14) Initialize all ipc_port structure fields to zero, set ip_receiver (S) and receiver_name (N), and initialize ip_messages to empty queue
16) If QoS Message was preallocated, connect it
is_write_unlock(S);
15(b) In unnamed path, IPC space was locked till now – so release
ipc_kmsg_set_prealloc
17) In the mach_port_construct flow, set guard (if requested) while locked, unlock port, and set any additional attributes before finally returning.
ip_unlock (port);
mach_port_guard_locked
17) In the mach_port_allocate_' flow, Unlock port, and return
KERN_SUCCESS
ip_unlock (port);
mach_port_set_attributes

The MACH_PORT_RIGHT_PORT_SET flow is very similar to that of .._RIGHT_RECEIVE: ipc_pset_alloc[_name] is called (instead of ipc_port..). Similar to their counterparts, these both call ipc_object_alloc[_name] as well, though with IOT_PORT_SET and MACH_PORT_TYPE_PORT_SET for the otype and type arguments (the second and third). As a result, the pset object is allocated from ipc port set zone. One notable difference is that a call is made to ipc_mqueue_init(). When a port is a member of a port set, the port set's message queue is used.

The MACH_PORT_RIGHT_DEAD_NAME allocation process is much simpler - ipc_object_alloc_dead is called instead of ipc_object_alloc(), and all this does is call ipc_alloc_entry and sets the resulting entry's ie_bits to specify MACH_PORT_TYPE_DEAD_NAME. Since there is no underlying port here, it suffices to then call ipc_entry_modified(), unlock the space (which was locked by ipc_alloc_entry()), and return.

Rights and Names

When a port is created, its sender only holds the MACH_PORT_RIGHT_RECEIVE name, as was returned by mach_port_allocate[_name] or ..construct. In order for the port to be usable, there must be at least one MACH_PORT_RIGHT_SEND or MACH_PORT_RIGHT_SEND_ONCE - otherwise there is little use in waiting for messages on the port which will never arrive.

As discussed in I/11, this is normally the case when starting a Mach port based transaction. Alice will call mach_port_allocate() to obtain an ephemeral port with a .._RIGHT_RECEIVE name, and then call on mach_port_insert_right() (MIG message #3214) to add a SEND or SEND_ONCE right to the port. Alternatively, the richer API of mach_port_construct could be used, specifying a flag of MPO_INSERT_SEND_RIGHT so as to have mach_port_insert_right() be automatically performed by the kernel.

The mach_port_insert_right() has a somewhat confusing prototype, The caller starts off by specifying the target ipc_space_t and the mach_port_name_t of the right in that IPC space. The third argument ("poly") is the name of the inserted right, and the fourth ("polyPoly") is the type of right inserted. Note that the same name can be used in for both the second and third arguments, which is common practice since it avoids the nuisance of maintaining too many port names. This call is effectively equivalent to the target space receiving a right, as would happen when a right was sent to it in a message.

It is also possible to call mach_port_extract_right() (MIG message #3216). This powerful call enables not only to remove a right from an ipc_space, but to also translocate it to another. The caller can operate on any space they have a handle to (in user space, through the Mach task port), specifying any name which is valid in the target. Once again, there is a poly argument (the fourth), which specifies a target name (implicitly in the caller's ipc_space_t) which will assume the right, and polyPoly (the fifth) which indicates which right type was extracted. An example of this can be seen in Listing I/12-8. The call is effectively equivalent to the space sending the right to the caller, albeit involuntarily.

The kernel keeps count of all MACH_PORT_RIGHT_SEND[_ONCE] right holders. Should the number drop to zero, the kernel will check if the port's ip_nsrequest field is set. If it is, it holds a SEND_ONCE right to some interested party who has registered it using MACH_NOTIFY_NO_SENDERS using mach_port_request_notification (MIG message #3212). This is only allowed if the .._RECEIVE right is held at the time, so it is commonly owned by the same task. The notification informs the notified party that this right is effectively useless, unless the mach_port_insert_right() choreography described earlier can be performed anew.

Reference counting

Ports rights get created, passed around in messages and destroyed at a dazzling pace. It is therefore highly important to keep track at any given time of just how many `SEND[_ONCE]` rights of each type exist, and maintain a clear reference count on the underlying port and its associated object.

As shown in Output 10-5, a port begins its existence with one reference counted in its `io_references` - that of the `ipc_entry_t` which points to it. Calls to `ip_reference()` and `ip_release()` can be used to toggle the reference count. Manipulating rights will affect `io_references`, as well as the `ipc_port`'s own counters, as shown in Table 10-10:

Table 10-11: `io_references` counts

Call	io_references	ip_mscount	ip_srights	ip_sorights
ip_port_make_send[_locked]	++	++	++	
ip_port_make_sonce[_locked]	++			++
ip_port_copy_send	++		++	

The `ipc_right_copyin()` function will observe the same semantics based on the disposition it is given (i.e. `MACH_MSG_TYPE_MAKE_SEND`, `..MAKE_SEND_ONCE` and `..COPY_SEND`). The `...MOVE..` dispositions do not alter any references, because the object just transfers owners.

Deallocating a port will call `ipc_right_deallocate()`. For inactive ports, `ipc_right_check()` handles decrementing the `ip_srights` and `ip_sorights` counters according to the entry's `ie_bits`, but not the `ip_mscount`, which is never decremented.

Internally, `ip_reference` and `ip_release` are just macros to `io_` counterparts, which are inlined functions defined in osfmk/ipc/ipc_object.h.

Listing 10-12: The `io_reference()` function (from osfmk/ipc/ipc_object.h)

```
/* Sanity check the ref count.  If it is 0, we may be doubly zfreeing.
 * If it is larger than max int, it has been corrupted or leaked,
 * probably by being modified into an address (this is architecture
 * dependent, but it's safe to assume there cannot really be max int
 * references unless some code is leaking the io_reference without leaking
 * object).  Saturate the io_reference on release kernel if it reaches
 * max int to avoid use after free.
 *
 * NOTE: The 0 test alone will not catch double zfreeing of ipc_port
 * structs, because the io_references field is the first word of the struct,
 * and zfree modifies that to point to the next free zone element.
 */
#define IO_MAX_REFERENCES                       \
(unsigned)(~0 ^ (1 << (sizeof(int)*BYTE_SIZE - 1)))

static inline void
io_reference(ipc_object_t io) {
ipc_object_refs_t new_io_references;
ipc_object_refs_t old_io_references;

assert((io)->io_references > 0 &&
    (io)->io_references < IO_MAX_REFERENCES);

do {
        old_io_references = (io)->io_references;
        new_io_references = old_io_references + 1;
        if (old_io_references == IO_MAX_REFERENCES) {
                break;
        }
} while (OSCompareAndSwap(old_io_references, new_io_references,
                &((io)->io_references)) == FALSE);
}
```

Listing 10-13: The `io_release()` function (from osfmk/ipc/ipc_object.h)

```
static inline void
io_release(ipc_object_t io) {
        ipc_object_refs_t new_io_references;
        ipc_object_refs_t old_io_references;

        assert((io)->io_references > 0 &&
            (io)->io_references < IO_MAX_REFERENCES);

        do {
                old_io_references = (io)->io_references;
                new_io_references = old_io_references - 1;
                if (old_io_references == IO_MAX_REFERENCES) {
                        break;
                }
        } while (OSCompareAndSwap(old_io_references, new_io_references,
                        &((io)->io_references)) == FALSE);

        /* If we just removed the last reference count */
        if (1 == old_io_references) {
        /* Free the object */
        io_free(io_otype((io)), (io));
        }
}
```

Port reference counting has proved to be extremely tricky over the years, and in particular when coupled with MIG-generated code. MIG semantics mandate that a failed operation will result in deallocating associated ports. When the implementation itself (called by the MIG stub) deallocates a port and returns anything but `KERN_SUCCESS`, MIG takes it as an indication that the onus of deallocating the port lies with it, and this results in a double free. Darwin 18 adds a check in `ipc_port_finalize` (osfmk/ipc/ipc_port.c) which ensures that an active port cannot be destroyed - even if its reference count drops to zero.

Port deallocation

Port deallocation is performed similar to creation, by `mach_port_deallocate` or `...destroy` (#3206/#3205). Here too, the corresponding traps (#18 and #17) may be used, and eventually use the same implementations as those of the MIG stubs.

The `mach_port_deallocate()` might seem like the inverse to `mach_port_allocate()`, but it is not. `mach_port_allocate()` will create a new port (when called with `MACH_PORT_RIGHT_RECEIVE`) and return a name, which implies the `RECEIVE` right to it. `mach_port_deallocate()`, however, is called on a name, which corresponds to a `SEND`, `SEND_ONCE`, `DEAD_NAME` or a port set. Thus, it merely decrements a reference, but does not necessarily destroy the port.

The `mach_port_destroy()` call destroys **all rights in the same IPC space** as the one denoted by the name. There is an additional MIG call, `mach_port_destruct()` (#3232), which was added along with its neighboring `mach_port_construct()`, and performs the same function as `..destroy()`, but allows specifying a guard value and a delta value.

When a port is deallocated or destroyed, the kernel must honor any notifications set on it using `mach_port_request_notification` (MIG message #3212), using messages from <mach/notify.defs>. Table 10-14 shows the notifications sent, and the events generating them:

Table 10-14: The notifications sent on port deallocation/destruction

#	Notification	`ipc_port` Field	Event
65	`MACH_NOTIFY_PORT_DELETED`	`ip_request`	The right is destroyed or deallocated
69	`MACH_NOTIFY_PORT_DESTROYED`	`ip_pdrequest`	The `RECEIVE` right is deallocated/destroyed
70	`MACH_NOTIFY_NO_SENDERS`	`ip_nsrequest`	The last `SEND` or `SEND_ONCE` right is deallocated
71	`MACH_NOTIFY_SEND_ONCE`	--	A `SEND_ONCE` right died (or was consumed)
72	`MACH_NOTIFY_DEAD_NAME`	--	A `SEND` or `SEND_ONCE` right becomes `DEAD_NAME`

Handling messages

With all this detailed discussion of the port lifecycle, it is easy to forget that the purpose of ports is to serve as endpoints for message passing. The `ip_messages` member of the `ipc_port` structure provides the message queue implementation. This is a large struct, defined in osfmk/ipc/ipc_mqueue.h, and inline expanded in the `struct ipc_port` as shown in Figure 10-7.

Listing 10-15: The `struct ipc_mqueue` definition (from osfmk/ipc/ipc_mqueue.h)

```
typedef struct ipc_mqueue {
        union {
struct {
  struct  waitq           waitq;
  struct ipc_kmsg_queue   messages;       // Pending message queue
  mach_port_seqno_t       seqno;          // Sequence number
  mach_port_name_t        receiver_name;  // Receive right name in recipient's ipc_space
  uint16_t                msgcount;       // Count of pending messages on queue
  uint16_t                qlimit;         // Maximum number of pending messages possible
#if MACH_FLIPC
  struct flipc_port      *fport; // Null for local port, or ptr to flipc port
#endif
        } port;
        struct {
         struct waitq_set        setq;
        } pset;
      } data;
      union {
                struct klist imq_klist;
                uintptr_t imq_inheritor;
        };
} *ipc_mqueue_t;
```

The first observation which can be made for the `ipc_mqueue` is that it behaves differently for a port, than for a port set. For the former, it provides a full structure in `data.port`, whereas for the latter it provides only a `waitq_set` in `data.pset`, which is the set of individual `waitq` structures (q.v. Chapter 10) from the set's member ports. This enables the `select(2)`-like functionality offered by port sets. In the case of individual ports, the `data` union is interpreted according to the `port` structure, and the messages are enqueued in the `messages` queue, as `ipc_kmsg` structures (discussed shortly, and shown in Listing 10-16). All fields are aliased through `imq_*` macros to ease readability through the data union.

An `ipc_mqueue_t` will be considered full when its `imq_msgcount` (i.e. `data.msgcount`) meets or exceeds the `imq_qlimit`, as tested by the `imq_full()` macro. There is also a second limit, `MACH_PORT_QLIMIT_KERNEL` (set in osfmk/mach/port.h to 65,534) which is assessed for messages sent from kernel mode (using `imq_full_kernel()`) when `MACH_SEND_ALWAYS` is specified. The `ipc_mqueue` also maintains an `imq_klist` of knote waiters, overloaded in Darwin 18 with a turnstile inheritor.

The osfmk/ipc/ipc_mqueue.h header defines several APIs for dealing with queues, but the two most important ones are, naturally, for `ipc_mqueue_send()` and `ipc_mqueue_receive()`. They are not, however, half as intuitive as one could perceive them to be at this point. Mach message delivery is a highly optimized process, at times skipping the `ipc_mqueue` entirely. To further this discussion, we need to revisit the most commonly used Mach trap and one of the most critical code paths in XNU - `mach_msg`.

mach_msg revisited

As we've established (in Volume I), `mach_msg` and its sibling, `mach_msg_overwrite` are wrappers over the corresponding Mach traps (#30 and #31). Additional wrappers, like `mach_msg_[receive/send]`, fall through to `mach_msg` with the corresponding `MACH_[RECEIVE/SEND]_MSG` flags and other arguments populated by default values.

The source for the wrappers can be found in XNU's libsyscall/mach/mach_msg.h. Both wrappers are simple ones, handling the `MACH_SEND_INTERRUPT|MACH_RCV_INTERRUPT` flags, and performing the retrying in user mode. The flags are thus filtered, to enable the kernel implementation to take the fast path. Then the trap is performed. Discounting the user-kernel mode traversal (already discussed in Chapter 4), executing resumes in kernel mode, with `mach_msg_[overwrite_]trap`, from osfmk/ipc/mach_msg.c. As it turns out, `mach_msg_trap` just falls through to `mach_msg_overwrite_trap` with a NULL as the `rcv_msg` buffer.

The very first processing performed by `mach_msg_overwrite_trap` is filtering out kernel-only flags, as ensured by bitmasking `MACH_MSG_OPTION_USER` (a macro over `(MACH_SEND_USER | MACH_RCV_USER)`, themselves macros over `MACH_[SEND/RCV]_*` flags in <mach/message.h>). The flow then handles two distinct cases: First, sending - if the `MACH_SEND_MSG` option was specified - and then receiving - if `MACH_RCV_MSG`.

Sending Mach messages

Messages originating from user mode must first be copied to kernel space. `ipc_kmsg_get()` (from osfmk/ipc/kmsg.c) calls `copyinmsg()` to copy the message header, and then calls `ipc_kmsg_alloc()` to allocate an `ipc_kmsg` structure from the `ipc kmsgs` zone (if under 256 bytes), or the nearest `kalloc` zone (if more)*. Kernel clients can opt to call `ipc_kmsg_get_from_kernel()`, which checks if the message's target port's bits specify `IP_BIT_PREALLOC`, indicating the `ip_premsg` holds a preallocated `ipc_kmsg`, thus saving the potential blocking allocation.

The `ipc_kmsg` is defined in osfmk/ipc/ipc_kmsg.h as follows:

Listing 10-16: The `struct ipc_kmsg` (from XNU 4903's osfmk/ipc/ipc_kmsg.h)

```
/*
 *      This structure is only the header for a kmsg buffer;
 *      the actual buffer is normally larger.  The rest of the buffer
 *      holds the body of the message.
 *
 *      In a kmsg, the port fields hold pointers to ports instead
 *      of port names.  These pointers hold references.
 *
 *      The ikm_header.msgh_remote_port field is the destination
 *      of the message.
 *
 *      sync_qos and special_port_qos stores the qos for prealloced
 *      port, this fields could be deleted once we remove ip_prealloc.
 */
struct ipc_kmsg {
   mach_msg_size_t            ikm_size;
   struct ipc_kmsg            *ikm_next;        /* next message on port/discard queue */
   struct ipc_kmsg            *ikm_prev;        /* prev message on port/discard queue */
   mach_msg_header_t          *ikm_header;
   ipc_port_t                 ikm_prealloc;     /* port we were preallocated from */
   ipc_port_t                 ikm_voucher;      /* voucher port carried */
   mach_msg_priority_t        ikm_qos;          /* qos of this kmsg */
   mach_msg_priority_t        ikm_qos_override; /* qos override on this kmsg */
   struct ipc_importance_elem *ikm_importance;  /* inherited from */
   queue_chain_t              ikm_inheritance;  /* inherited from link */
   // Darwin 18 - replaces sync_qos & special_port_qos
   struct turnstile           *ikm_turnstile;   /* send turnstile for ikm_prealloc port */
#if MACH_FLIPC
        struct mach_node      *ikm_node;        /* Originating node - needed for ack */
#endif
};
```

* - Darwin versions prior to 18 first tried a per-processor cache, since it has less overhead and no locking requirements. The comment describing this still exists in XNU 4903, but the per-processor cache has been removed.

The ipc_kmsg_alloc() function calls the ikm_init macro to initialize the ipc_kmsg fields to their defaults, but the message itself still needs to be copied to kernel space. ipc_kmsg_get() further copies the message header (to kmsg->ikm_header), the body (right after the (sizeof(ikm_header)) bytes) and the msgh_audit and msgh_sender trailers, which are automatically inserted.

At this point, the message buffer resides in kernel space - but any ports the user has specified (the local, remote, voucher) and/or descriptors are just the mach_port_t names of user space. Back in mach_msg_overwrite_trap, then, ipc_kmsg_copyin() is called. This follows to ipc_kmsg_copyin_header(), and possibly (if the message is complex) ipc_kmsg_copyin_body(). Both of these functions deal with any embedded port names and dispositions specified in the header, or (respectively) descriptors in the body, to convert them to their kernel representation. Thus, the msgh_remote_port in the kmsg's ikm_header is actually an ipc_port_t, as are the port descriptor's names. Similarly, any OOL descriptor's address fields are replaced with pointers vm_copy_t objects.

Next, the actual sending of the ipc_kmsg is performed by ipc_kmsg_send(). This routine first checks if the remote port is active. If it isn't, the kmsg is silently destroyed, and MACH_MSG_SUCCESS is returned. If the remote port's ip_receiver points to the kernel's address space, this is taken to be a MIG message, and the kernel's ipc_kobject_server() (from osfmk/kern/ipc_kobject.c) is called to handle the message. The MIG_HASH macro is used over the message header's id (msgh_id, but the macro is NULL, and the hash "key" is simply taking the message id's remainder from MAX_MIG_ENTRIES (1031). Assuming a matching id can be found and it has a registered routine (i.e. it was not skipped), the routine is called, and its reply message is returned from ipc_kobject_server() as the new message to send (with the target now being the sender). This joins with the code path in ipc_kmsg_send(), which places the message to be sent on the receiver's message queue using ipc_mqueue_send(). If this completes with no error, the message is guaranteed to be safely in the receiver's queue, and will be read in the order received.

ipc_mqueue_send()

The ipc_mqueue_send() routine is called to send an ipc_kmsg to a target mqueue (resolved from the destination port's ip_messages) with an optional send_timeout and any MACH_SEND_* option. Sends normally happen immediately, but may block if the ipc_mqueue is full (unless the send operation uses a SEND_ONCE right, in which case it will be attempted anyway and consume the right). A call to waitq_assert_wait64_leeway() blocks for up to the caller specified send_timeout with no leeway, or indefinitely (if send_timeout was not specified). As of Darwin 18, this has been augmented by turnstiles (q.v. Chapter 10) - preparing a turnstile over the ipc_mqueue's port, and setting its inheritor to that port (or, if in transit, the destination port). The turnstile is completed upon the wait's return, but any wait result other than THREAD_AWAKENED will result in ipc_mqueue_send() returning an error.

The actual send operation is performed by ipc_mqueue_post(). This first calls imq_reserve_and_lock(), to lock the associated imq_wait_queue and waitq_prepost_reserve(). Assuming the port wasn't destroyed in the interim, the flow enters a loop over waitq_wakeup64_identify_locked(), which should wake up and return a thread waiting on receiving from the ipc_mqueue. Assuming it does, the message is handed directly to that thread, by linking it to the thread's ith_kmsg field, with the ith_state set accordingly, and the ith_seqno copied from the ipc_mqueue's (which is then incremented). The ipc_mqueue can then be unlocked, and the task's messages_sent counter incremented.

If a receiver thread cannot be located (i.e. waitq_wakeup64_identify_locked() returns (THREAD_NULL), it means there are no blocking receivers, and so the message is either added to the queue using ipc_kmsg_enqueue_qos() or (if sent to an inactive port) set to be destroyed. ipc_kmsg_enqueue_qos() (from osfmk/ipc/ipc_kmsg.c), is a variant of the regular ipc_kmsg_enqueue(), which (like it) inserts at the tail of the queue (thus maintaining FIFO ordering), but also allows propagating an enqueued message's ikm_qos_override field to previous messages in the queue (if higher than their existing QoS).

Receiving Mach messages

Incoming messages are enqueued on their corresponding port's `ipc_mqueue`. When `mach_msg[_overwrite]_trap` is called, it is provided a user mode port name, which must be translated to the kernel's mode `ipc_port`, and then resolved to the port's `ip_messages` - an `ipc_mqueue_t`. This is handled by the `ipc_mqueue_routine()`, which performs the following steps:

- Call `ipc_entry_lookup()` on the target *space* and *name*, resolving it to the `ipc_entry_t`.
- Check that the `ipc_entry_t` is indeed a port receive right or a port set, or else return `MACH_RCV_INVALID_NAME`.
- Increase the `ie_object`'s reference count, and return both the object reference and its corresponding mqueue reference.

With the message queue reference at hand, reception may be attempted. It is at this point that Darwin 17 and later kernels add an additional check, handling the special case wherein the options specify `MACH_RCV_SYNC_WAIT` but not `MACH_SEND_SYNC_OVERRIDE`, two options also added in XNU 4570. If such is the case, a special reply port is linked to the reply.

Receiving the message is a two step process. First, a call to `ipc_mqueue_receive()` with the `mqueue` and other arguments is performed. Then, a call `mach_msg_receive_results()` to perform message reception. Often, however, a message may not be available, and so message reception might block - indefinitely, if called with `MACH_TIMEOUT_NULL`. Thus, before calling `ipc_mqueue_receive()`, additional arguments are loaded onto the Mach `thread` structure. Those include the message buffer address, and the continuation - `thread_syscall_return()`, so that the operation can return asynchronously to user mode. In the case wherein `ipc_mqueue_receive()` does return (synchronously), `mach_msg_receive_results()` is called immediately afterwards.

At this point, the behavior may differ. If the caller did not specify `MACH_RCV_TIMEOUT`, the flow returns with `mach_msg_receive_results()`. The function does not accept any arguments, instead using state variables passed through the Mach `thread_t` object. In cases where `ipc_mqueue_receive()` reported some error status, it is handled and reported back to user mode. If the messages was successfully received, however, any importance and voucher attributes (as of Darwin 14) are automatically applied to the receiving (current) thread.

Destroying messages

Undeliverable messages (e.g. those with a `!IP_VALID` port) must be destroyed. This naturally involves `kfree()`ing the `ipc_kmsg` which was `kalloc()`ed by `ipc_kmsg_alloc()`, but the message may contain additional rights and descriptors (described shortly), which must be properly disposed of as well.

Message destruction is handled by `ipc_kmsg_destroy()`. The function, however, does not immediately destroy the message provided, instead calling `ipc_kmsg_delayed_destroy()`, to place the `ipc_kmsg` on the `current_thread`'s `ith_messages` queue. The actual destruction is performed by `ipc_kmsg_reap_delayed()`, which is called only if the `ipc_kmsg` is the first in the queue, or when the `RECEIVE` right is cleared. The reaping function iterates over the `ith_messages` queue, performing, for each:

- `ipc_kmsg_clean()`: which first takes on the importance chain, then `ipc_object_destroy()`s the remote, local and voucher ports in the message's `ikm_header`. Complex messages require handling of the additional descriptors, which is the role of `ipc_kmsg_clean_body()`.
- `ipc_kmsg_rmqueue()`: the inverse of `ipc_kmsg_enqueue()`, which is a simple unlinking of a queue element.
- `ipc_kmsg_free()`: Preallocated `ipc_kmsgs` (for which `ikm_prealloc_inuse_port()` will return the port, whose `ip_premsg` will be the `ipc_kmsg` in question) are recycled with a call to `ikm_prealloc_clear_inuse()`. Otherwise, the `ipc_kmsg` is freed - either `zfree()`d into the `ipc_kmsg_zone`, or `kfree()`d, depending on the `ikm_size`.

Experiment: DTracing the flow of `mach_msg_trap`

A simple DTrace script (on SIP-disabled MacOS) can trace all calls to `mach_msg_trap` across all callers. The `mach_msg_header_t` is known to DTrace, so a few lines do the trick:

Listing 10-17: Tracing all calls to `mach_msg_trap`

```
#!/usr/sbin/dtrace -s
#pragma D option flowindent

/* Start tracing on trap entry; */
mach_trap::mach_msg_trap:entry  {
  self->tracing = 1;
  this->msg = (mach_msg_header_t *) copyin (arg0, 24);

printf ("PID %d (%s) message #%d from port 0x%x to port 0x%x with voucher %x",
   pid, execname,
   this->msg->msgh_id,
          (uint32_t) this->msg->msgh_local_port,
          (uint32_t) this->msg->msgh_remote_port,
          (uint32_t) this->msg->msgh_voucher_port);
}
```

With a bit of work, the script could be modified to show the behind-scenes-flow of `mach_msg_trap`, illustrating the full process of sending and receiving messages in kernel. This requires using the powerful Function Boundary Tracer (`fbt`), between the `entry` and `return` of `mach_msg_trap`:

Listing 10-18: Tracing the flow generated by `mach_msg_trap`

```
......

// Trace inner flow after mach_msg_trap
fbt::::/self->tracing/ { printf("%x %x, %x", arg0, arg1, arg2); }

// Turn off tracing at the end of mach_msg and exit
mach_trap::mach_msg_trap:return  {
        self->tracing = 1;
        exit(0);
}
```

The flow revealed by the probes, however, will be wildly different every time you run the script - dependent on such factors as whether the tracked message will have vouchers, or what the underlying `IKOT_` port types are (IOKit ports, for example, forcing lengthy processing on `is_io_connect_method`, as described in Chapter 13).

Message Descriptors

Recall (from I/11), that messages may be marked as complex, in which case they contain one or more **descriptors**. Descriptors provide the sender with the ability to include additional port rights (or sets) in the message, as well as out of line (OOL) memory descriptors. The `MACH_MSGH_BITS_COMPLEX` (0x80000000) in the message header bits alerts the kernel that a complex message requires further processing of these descriptors, which is the role of `ipc_kmsg_copyin_body()`.

Port right descriptors

As was previously discussed, when a Mach message includes a reply port, it is automatically processed according to the disposition. The same processing is needed when a message contains port descriptors, denoting additional rights the sender wishes to provide to the recipient. A common case of this are the port rights included by `launchd(8)` in replies to successful `bootstrap_look_up()` requests. If such rights are included, they need to be processed in a manner similar to the reply port.

`ipc_kmsg_copyin_body()` calls `ipc_kmsg_copyin_port_descriptor`, which obtains the port `name` from the descriptor. The `name` and the `ipc_space_t` whence it came allow the kernel (which has access to all `ipc_space_ts`) to obtain the underlying object the port descriptor represents, by a call to `ipc_object_copyin()`, which locates the object and increases its reference count. The flow is shown in Figure 10-19 (next page), which shows the way the arguments flow from one function to another, along with an annotated DTrace script which can be used on a live system.

The actual retrieval of the port right is performed by `ipc_right_copyin()`, which as we've discussed before handles the reply port name in the message header. Once the right is retrieved, it is resolved to its underlying port. If the port is not dead (as verified by the call to `ipc_right_check()`, the port reference count is increased, and it is made available to the caller, subject to the disposition specified in the descriptor.

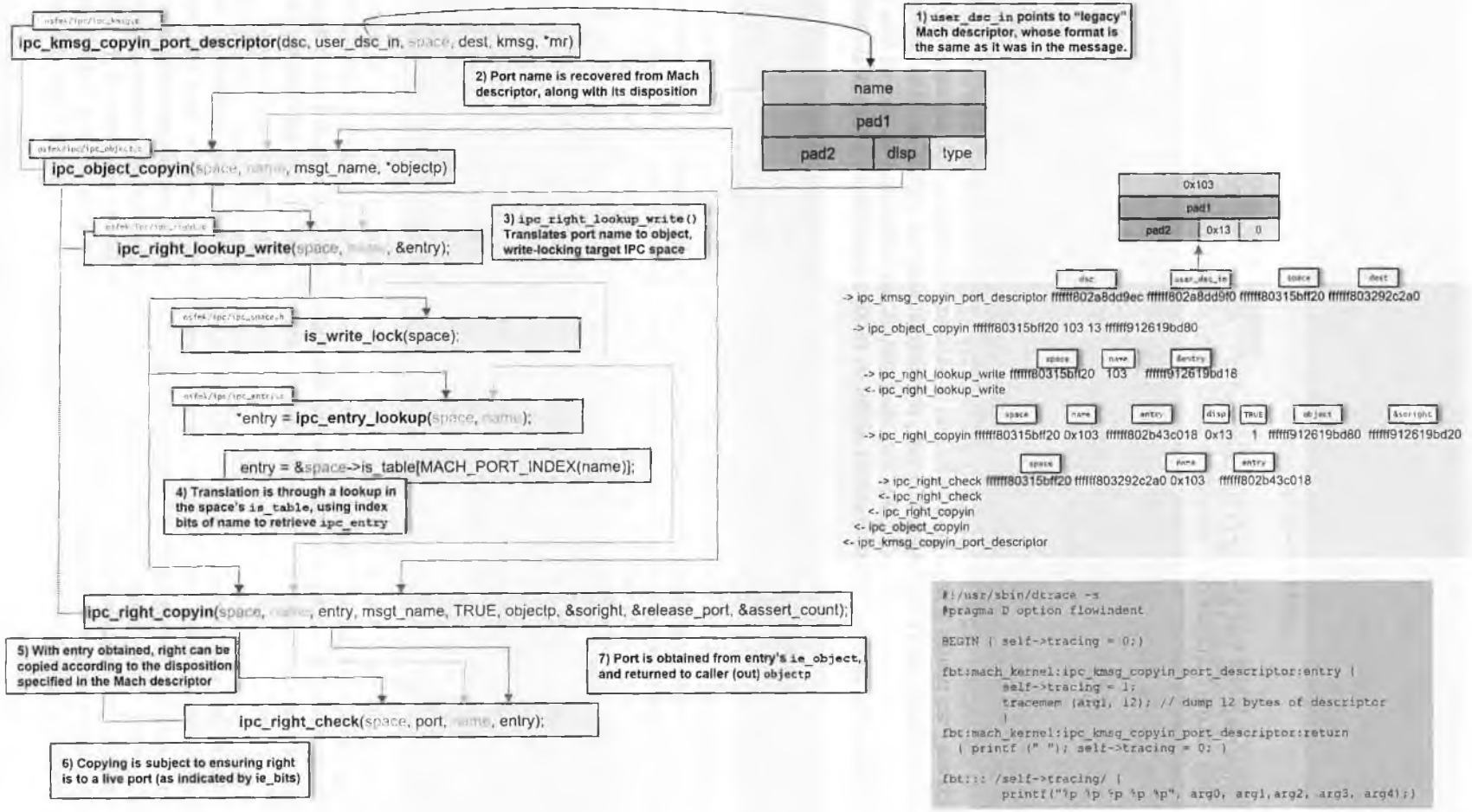
Port set (OOL ports) descriptors

Port set descriptors (referred to as "OOL ports descriptors") contain any number of ports which the caller wants to put in a message. `ipc_kmsg_copyin_body()` calls `ipc_kmsg_copyin_ool_ports_descriptor()` to handle these descriptors. Since an OOL ports descriptor is really just an array of individual port descriptors, the routine is in principle similar to `ipc_kmsg_copyin_ool_port_descriptor()`, with another step - calling `copyinmap()` to get the port names, and then calling `ipc_object_copyin()` in a loop over those names. Should any of the `..copyin()` operations fail, the objects copies created so far are destroyed.

OOL memory descriptors

OOL memory descriptors allow the sender to provide a chunk of its virtual address space, along with disposition fields specifying if the memory is to be copied, moved or shared. `ipc_kmsg_copyin_body()` calls `ipc_kmsg_copyin_ool_descriptor`, which converts the OOL descriptor into a `struct vm_map_copy`. This is a structure describing a pending copy operation, which points to an item in kernel space. The "item" can be one of three types: a `vm_map_header` (contiguous memory region), single `vm_object_t` (group of resident pages), or a `kdata` kernel buffer (inlined data, for small enough (< 8,192 bytes) data items). The `vm_map_copy_t` is described in Chapter 11.

Figure 10-19: The flow of `ipc_kmsg_copyin_port_descriptor()`




```
#!/usr/sbin/dtrace -s
#pragma D option flowindent

BEGIN { self->tracing = 0;}

fbt:mach_kernel:ipc_kmsg_copyin_port_descriptor:entry {
        self->tracing = 1;
        tracemem (arg1, 12); // dump 12 bytes of descriptor
        }
fbt:mach_kernel:ipc_kmsg_copyin_port_descriptor:return
  { printf (" "); self->tracing = 0; }

fbt::: /self->tracing/ {
        printf("%p %p %p %p %p", arg0, arg1,arg2, arg3, arg4);}
```

Descriptors as a vehicle for malicious attacks

An interesting effect of sending port right descriptors (or port sets) is that the kernel handles copying them into the target's address space automatically - without the target asking, or in fact being aware this occurred. Normally, this isn't a problem, since the ports would normally remain unused unless the target task is made aware of their existence (usually, by MIG generated code taking them from the message in installing them onto user variables). This, however, has been abused, causing the allocation of user space port names in the target and leading to intentional collisions. Ian Beer's `mach_portal` (III/23) provided a fine example of this technique.

OOL descriptors (`vm_map_copy` objects), in particular have been a long time favorite of jailbreakers. Several jailbreaks constructed starting with iOS 6's evasi0n (III/14) constructed fake `vm_map_copy_t` objects. Back then, the `vm_map_copy_t` allowed `kdata` not as an inlined buffer, but as an arbitrary pointer and `kalloc_size`. These were easy to fake into kernel memory read/write primitives, and this method was used until Apple had to fix the descriptor to its present state, in XNU-3248 (iOS 9).

This, however, hasn't lessened the usefulness of OOL descriptors. As with port and port set descriptors, OOL memory descriptors also get automatically handled by the kernel on message reception, according to the descriptor semantics specified by the caller. While this is normally the expected and intended behavior, here, too, this can be abused as a method of memory spraying in the target.

Yet another important aspect of the descriptor handling process is that, until the message is received, the descriptors have to be "parked" in kernel space. This has been repeatedly exploited by attackers as a method for kernel memory manipulation. Allocating an arbitrary port, and sending messages to it - but not yet receiving - provides a means to spray these descriptors into various kernel memory zones. Sending multiple OOL memory descriptors has the effect of spraying kernel memory with user controlled data. Sending multiple port descriptors has been similarly used for exploiting port UaF conditions. Examples of these techniques abound throughout the later chapters of Volume III.

Note, that XNU does explicitly impose an upper bound on the number of right descriptors which may be sent in a message (`MACH_PORT_UREFS_MAX/4`, in `ipc_kmsg_copyin_body()`), commenting that "Sending more than 16383 rights in one message seems crazy". This bound, however, is far higher than what it should be. One could argue that the entire approach taken by Apple to enforce "fair" use of this mechanism is potentially wrong, and so descriptor manipulation with crafted messages remains a staple of jailbreaks to this day.

Lastly, an unintended conequence of OOL memory descriptors is transforming them into an arbitrary kernel memory allocation primitives. An attacker with arbitrary kernel memory read/write capabilities can use memory descriptors (specifically, `VM_MAP_COPY_KERNEL_BUFFER` types, as discussed in the next chapter) to force a `kalloc` allocation of a buffer with controlled size and contents. This is similar to the spraying just discussed, but requires finding the descriptor memory address, disconnecting it from the `ipc_kmsg` (by simplifying the message - i.e. removing the COMPLEX bit), and then reusing the descriptor memory for some other purpose.

Experiment: Inspecting mach message passing in kernel memory

Building on the Mach messaging APIs introduced in I/11, we create a simple program to send (but not receive) a message to an ephemeral port. The message we craft is a complex one, containing an OOL descriptor with some arbitrary payload - say, 128 bytes of 'J'. The program is left for the reader as a (highly recommended) exercise. We then turn to inspect kernel memory, so we can observe the low level data structures previously discussed in this chapter.

Output 10-20 shows what the `ipc_port` structure looks like, before and after sending (but not receiving) a message. The two kernel addresses at 0xffffffe006d87948 are the empty wait queue (as both entries point to the queue itself). Immediately thereafter is a NULL value, but when we actually send the message, something happens: the NULL value - a pointer to the message list - is linked to an entry, and the msgcount increases to one:

Output 10-20-a: The `ipc_port`, in memory

```
Found port 0xd0f in PID 330: @0xffffffe006d87918
#                  Before (queue is empty, msgcount 0 )     |  (queue linked, msgcount 1)
0x..6d87918  0x80000000  0x00000001          0x0            | 0x80000000 0x00000002    ....
0x..6d87928 11 00 00 00 00 00 00 00 66 00 00 00 00 00 00 00 |       ........
0x..6d87938 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |       ........
0x..6d87948    0xffffffe006d87948       0xffffffe006d87948  |       ........
0x..6d87958 00 00 00 00 00 00 00 00 00 00 00 00    0x0d0f   |  0xffffffe0065ca500      ....
0x..6d87968 00 00 05 00 00 00 00 00 00 00 00 00 00 00 00 00 | 01 00 05 00 00 00 00 00  ....
0x..6d87978    0xffffffe004bd4fc0    00 00 00 00 00 00 00 00 |       ........
0x..6d87988 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 |       ........
...
```

Dumping the message queue (`ip_messages.data.port.messages.ikmq_base`) from the `ipc_port` structure shows that we indeed have an `ipc_kmsg` waiting:

Output 10-20-b: The `ipc_kmsg` object waiting in the message queue

```
0xffffffe0065ca500 a8 00 00 00 16 87 cf b9    0xffffffe0065ca500    ..........\.....
0xffffffe0065ca510    0xffffffe0065ca500      0xffffffe0065ca588    \.....\.....
0xffffffe0065ca520 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ................
0xffffffe0065ca530 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ................
```

Even though the `ipc_kmsg` does not contain a count, we can tell it's a single message because the `ikm_[next/prev]` fields both point to the message itself (above, `0xffffffe0065ca500`). Right after is the `ikm_header` field, which we can also dump:

Output 10-20-c: The `ikm_header`

```
0xffffffe0065ca588 11 00 00 80 34 00 00 00    0xffffffe006d87918    ....4.......y...
0xffffffe0065ca598 00 00 00 00 00 00 00 00 00 00 00 00 0b b0 00 00 ................
0xffffffe0065ca5a8 01 00 00 00 00 e7 5d 06 e0 ff ff ff 00 01 00 01 ......].........
0xffffffe0065ca5b8 a8 00 00 00 00 00 00 00 08 00 00 00 00 00 00 00 ................
0xffffffe0065ca5c8 f5 01 00 00 f5 01 00 00 ff ff ff ff f5 01 00 00 ................
0xffffffe0065ca5d8 f5 01 00 00 f5 01 00 00 f5 01 00 00 4a 01 00 00 ............J...
0xffffffe0065ca5e8 00 00 00 00 4b 01 00 00 00 00 00 00 00 00 00 00 ....K...........
0xffffffe0065ca5f8 2f c4 6f fe 00 00 00 00 a8 00 00 00 16 87 cf b9 /.o.............
0xffffffe0065ca608    0xffffffe0065ca600      0xffffffe0065ca600    \.....\.....
0xffffffe0065ca618    0xffffffe0065ca680    00 00 00 00 00 00 00 00 \.............
0xffffffe0065ca628 00 00 00 00 00 00 00 00
```

Note the unaligned value at `0xffffffe0065ca5ac` - This is the address of our specified OOL buffer - `0xffffffe0065de700`. Proceeding to dump this as well:

Output 10-20-d: Dumping an `ipc_kmsg` from memory

```
                  type
0xffffffe0065de700 03 00 00 00 [illegible] 00 00 00 00 00 00 00 00 ................
                  size
0xffffffe0065de710 a8 00 00 00 00 00 00 00 4a 4a 4a 4a 4a 4a 4a 4a ........JJJJJJJJ
0xffffffe0065de720 4a 4a 4a 4a 4a 4a 4a 4a 4a 4a 4a 4a 4a 4a 4a 4a JJJJJJJJJJJJJJJJ
                   4a 4a 4a 4a 4a 4a 4a 4a 4a 4a 4a 4a 4a 4a 4a 4a JJJJJJJJJJJJJJJJ
0xffffffe0065de790 4a 4a 4a 4a 4a 4a 4a 4a 4a 4a 4a 4a 4a 4a 4a 4a JJJJJJJJJJJJJJJJ
0xffffffe0065de7a0 4a 4a 4a 4a 4a 4a 4a 4a                         JJJJJJJJ
```

containing our 'J's, as expected. The small header before it indicates the data is embedded in a `vm_map_copy_t` type 3 (`VM_MAP_COPY_KERNEL_BUFFER`), of 0xa8 bytes. Factoring out 0x18 bytes for the header, this gives us our exact payload, of 0x80.

Vouchers

Vouchers are a fairly recent introduction into the Mach landscape. Appearing for the first time in Darwin 14 (XNU 2782), Vouchers are described (in a header comment from osfmk/ipc/ipc_voucher.n) as "reference counted immutable (once created) set of indexes to particular resource manager attribute values (which themselves are reference counted)". When created, the user mode caller supplies one or more "recipes" for creating them, which contain the attribute key, command, and content. In kernel, each attribute is handled by an attribute manager, which operates on the command. The main uses of vouchers appear to be in GCD (libdispatch) and the Activity Trace Manager (ATM) facility. There is no API for vouchers intended for third party developers, and vouchers remain undocumented in any official capacity, save for some comments in the source files.

Vouchers are a fundamental addition to Mach. It is the first time that Apple has added code directly into the Mach subsystem, changing low-level details such as the Mach message header, message handling, and MIG. This was not without some far reaching implications - from its inception, voucher code has been riddled with bugs - one upon which the 10.2 jailbreak was based (III/24), and another which provided a partial jailbreak for 12.1.2 and below[1].

User-mode API

Two new Mach traps have been added to support vouchers:

- `host_create_mach_voucher_trap` (#70) is used to create a voucher. The trap expects the recipes as an array of of `mach_voucher_attr_recipe_data_t` entries, defined in <mach/mach_voucher_types.h> thus:

Listing 10-21: The `mach_voucher_attr_recipe_data` (from <mach/mach_voucher_types.h>)

```
typedef struct mach_voucher_attr_recipe_data {
        mach_voucher_attr_key_t                 key;
        mach_voucher_attr_recipe_command_t      command;
        mach_voucher_name_t                     previous_voucher;
        mach_voucher_attr_content_size_t        content_size;
        uint8_t                                 content[];
} mach_voucher_attr_recipe_data_t;
```

- `mach_voucher_extract_attr_recipe_trap` (#72), extracts the creation *recipe* of a given attribute *key* from an existing voucher *name* as a `mach_voucher_attr_raw_recipe_t`, which is an array of *recipe_size* bytes.

The same header file also defines the following attributes:

Listing 10-22: Voucher attribute keys (from <mach/mach_voucher_types.h>)

```
/* other well-known-keys will be added here */
#define MACH_VOUCHER_ATTR_KEY_ATM          ((mach_voucher_attr_key_t)1)
#define MACH_VOUCHER_ATTR_KEY_IMPORTANCE   ((mach_voucher_attr_key_t)2)
#define MACH_VOUCHER_ATTR_KEY_BANK         ((mach_voucher_attr_key_t)3)
#define MACH_VOUCHER_ATTR_KEY_PTHPRIORITY ((mach_voucher_attr_key_t)4)

#define MACH_VOUCHER_ATTR_KEY_USER_DATA    ((mach_voucher_attr_key_t)7)
#define MACH_VOUCHER_ATTR_KEY_BITS         MACH_VOUCHER_ATTR_KEY_USER_DATA /* deprecated */
#define MACH_VOUCHER_ATTR_KEY_TEST         ((mach_voucher_attr_key_t)8)

#define MACH_VOUCHER_ATTR_KEY_NUM_WELL_KNOWN    MACH_VOUCHER_ATTR_KEY_TEST
#define IV_ENTRIES_INLINE MACH_VOUCHER_ATTR_KEY_NUM_WELL_KNOWN
```

Two MIG subsystems have also been added to support vouchers: The first, <mach/mach_voucher.defs> (#5400), is used to extract attribute content or creation recipes. The second, <mach/mach_voucher_attr_control.defs> (#5600), is used for attribute control.

---

* - It is tempting to think that another Darwin 14 addition, the ATM, is related to vouchers and banks, but in fact the acronym stands for the Activity Trace Manager.

Table 10-23: The `mach_voucher_*` MIG subsystems

#	`mach_voucher_...` routine	Description
5400	`.._extract_attr_content`	Extract content data for a <voucher, key> pair
5401	`.._extract_attr_recipe`	Get recipe for a <voucher, key> pair item in voucher
5402	`.._extract_all_attr_recipes`	Get recipe array for all key/values in voucher
5403	`.._attr_command`	Execute a command against a given voucher attribute
5404	`.._debug_info`	As `mach_voucher_extract_all_attr_recipes`, but on remote task space and voucher name
5600	`.._attr_control_get_values`	Extract the given voucher-control's value-handle from the supplied voucher
5601	`.._attr_control_create_mach_voucher`	Create a new voucher with the control's privilege (to directly assign value-handles)

Implementation

Vouchers appear in user mode as port references, but in kernel mode the ports are `ipc_objects` of type `IKOT_VOUCHER` (#37), whose `ip_kobject` points to a `struct ipc_voucher`, an 80-byte object allocated from the `ipc vouchers` zone, defined in osfmk/ipc/ipc_voucher.h as follows:

Listing 10-24: The `struct ipc_voucher` definition (from XNU 4903's osfmk/ipc/ipc_voucher.h)

```
/*
 * IPC Voucher
 *
 * Vouchers are a reference counted immutable (once-created) set of
 * indexes to particular resource manager attribute values
 * (which themselves are reference counted).
 */
struct ipc_voucher {
    iv_index_t      iv_hash;        /* checksum hash */
    iv_index_t      iv_sum;         /* checksum of values */
    iv_refs_t       iv_refs;        /* reference count */
    iv_index_t      iv_table_size;  /* size of the voucher table */
    iv_index_t      iv_inline_table[IV_ENTRIES_INLINE];
    iv_entry_t      iv_table;       /* table of voucher attr entries */
    ipc_port_t      iv_port;        /* port representing the voucher */
    queue_chain_t   iv_hash_link;   /* link on hash chain */
};
```

The `convert_port[_name]_to_voucher()` routines are used to get from the `ipc_port_t` (or the `mach_port_name_t`) to the `ipc_voucher_t`, and the voucher's `iv_port` points back to the port.

It is entirely possible that, when a voucher is added, its values would already exist in those of another voucher. The `iv_dedup()` is called to remove duplicates.

Voucher attribute managers are `ipc_voucher_attr_manager` structures, which provide the callbacks for the voucher value lifecycle methods, as well as some flags. Presently, though, only two flags are supported - `IVAM_FLAGS_SUPPORT_SEND_PREPROCESS` and `..RECEIVE_POSTPROCESS`. Each attribute manager thus has its own implementations and set of supported voucher commands.

Listing 10-25: The `ipc_voucher_attr_manager` definition

```
struct ipc_voucher_attr_manager {
 ipc_voucher_attr_manager_release_value_t        ivam_release_value;
 ipc_voucher_attr_manager_get_value_t            ivam_get_value;
 ipc_voucher_attr_manager_extract_content_t      ivam_extract_content;
 ipc_voucher_attr_manager_command_t              ivam_command;
 ipc_voucher_attr_manager_release_t              ivam_release;
 ipc_voucher_attr_manager_flags                  ivam_flags;

#define IVAM_FLAGS_SUPPORT_SEND_PREPROCESS          0x1
#define IVAM_FLAGS_SUPPORT_RECEIVE_POSTPROCESS      0x2

};
```

Multinode

The biggest and most exciting change coming to XNU is multinode support. At the time of writing, the compilation option (`MACH_FLIPC`) it is dependent on is not defined in any of the flavors. The code has been around in some form since the early days of XNU (as both NORMA and FLIPC), but NORMA was removed in Darwin 14, and FLIPC was dropped (temporarily) in Darwin 15. FLIPC was reintroduced with a revised implementation in Darwin 16, and reservation of a host special port in that Darwin release implies that it might some day become a reality.

This section discusses the present (Darwin 18) implementation of multi-node support, which is incomplete, and left out of `RELEASE` kernels. The implementation may very well change before it comes to fruition, or it may not ever make into release kernels, at all. That said, the functionality offered by it is so revolutionary that, for the sake of forward applicability (and the author's deep enthusiasm about its prospects), an attempt is made to detail it as much as is presently possible.

Multinode requirements

Starting with Darwin 17, Apple supports moving XPC messages from one host to another, providing the groundwork for BridgeOS/MacOS communication. But extending Mach IPC to a multinode environment is more than just about getting messages across. It requires making available the Mach primitives - host, tasks, threads, processor_sets and more - not just to the local machine, but also from a remote machine. Just as there is a `mach_host_self()` trap, there needs to be some `mach_host_other()` routine or API by means of which a handle to a remote node's host and host_priv objects can be obtained.

With a remote host port at hand, myriad possibilities open up. As shown in III/12-1, the host port allows access to the `clock`, `processor` and `processor_set` primitives. Through the last one, access to remote tasks and threads (via `processor_set_things()`) can be obtained. And through the host's I/O Master port the vast landscape of I/O Kit drivers emerges. This requires some mechanism of **proxying** remote ports, so that user mode remains oblivious to the remote nature of those objects, and can message them using the same age-old MIG routines and Mach messages.

All this is no doubt complicated to implement, and that's without even taking into account the paramount security considerations such remote access entails. Although the original design of Mach accounted for some considerations in mind, security was unhandled, so it is understandable why Apple chose to deliberately cripple this functionality. Yet, as mentioned earlier, it seems like this vast potential (and danger) might be realized in the foreseeable future.

FLIPC

Fast **L**ocal **I**nter**P**rocess **C**ommunication (FLIPC) was first introdcuced in 1996 by Black et al [2]. It has appeared in older XNU sources as far back as XNU-123 as headers, but was dropped in Darwin 15 (XNU-3248), until it reappeared in Darwin 16 - this time with a C implementation.

The present implementation is comprised of two layers:

- **Mach Node:** (osfmk/kern/mach_node.c): contains two sets of APIs. Those of `mach_node_*`, handle remote node lifecycle methods. These include `mach_node_register()`, to create a local representation of a remote node, and `mach_node_publish()` (called in the process of registration), which takes the node and makes it available to the local node's bootstrap server (i.e. `launchd(8)`).

  The second set of APIs are the `mnl_*` routines, which are reserved for the "link driver" to interface with the lower `mach_node` layer. Thus, for example, `mnl_register()` wraps `mach_node_register()`. The other `mnl_*` routines handle messaging, and so calls such as `mnl_msg_to_node()` and `mnl_msg_from_node()` are used for outgoing and incoming messages, respectively.

- **FLIPC**: (osfmk/ipc/flipc.c) handles the (present) implementation. It handles low level operations, such as translation the in-kernel `ipc_kmsg` structures to FLIPC-layer `mnl_msg` structures, and converting local Mach ports to FLIPC ports (and vice versa).

There are two major problems in assessing this implementation. The first is that there is no "link driver" implementation to be found. The link layer logic plays a crucial part in enabling multinode functionality, yet it has been delegated to an external kext, and no such kext has been provided by Apple. Worse, the entire implementation rests of `MACH_FLIPC` being `TRUE` - which (at least at the time of writing) - it is not in any of the Darwin flavors. So while the various functions exist, they are all empty stubs returning, at best, an error code.

Lack of link driver aside, here is what the present code base reveals:

There is a new host special port, `HOST_NODE_PORT` (#26). When claimed in user mode (via `host_set_special_port()`), `#if MACH_FLIPC` (which, as mentioned, it is not) a call to `mach_node_port_changed()` informs the system. The code in mach_node.c obtains a `SEND` right to the port, and installs it as a "bootstrap_port" (not to be confused with THE bootstrap port, i.e. task special port #4, held by `launchd(8)`). The local kernel publishes its presence by a special Mach message to the `HOST_NODE_PORT` owner, referred to as the "noded".

Neighboring nodes known to the kernel are added to a local `mach_node_table` by their node id when they send a registration message, which is picked up by `mnl_register()` (called by the missing link driver). When the `HOST_NODE_PORT` is claimed, these nodes are also published to the user mode daemon, which presumably makes them known to other user mode clients requiring a remote connection. Should a remote node become unavailable, the missing link driver is expected to call `mnl_terminate()`.

The link driver is also responsible for inter-node messaging. Incoming messages call `mnl_msg_from_node()`, supplying an `mnl_msg_t`. The only supported type is `MACH_NODE_SUB_FLIPC`, and it is handed to the FLIPC layer. Outgoing messages call mnl_msg_to_node(), which again defers handling to FLIPC. When the link driver is confident the messages have been delivered, `mnl_msg_complete()` FLIPC layer to free the message.

The `mach_node_t` representing nodes in kernel also holds an `ipc_space_t`, which functions as a proxy to the any remote `ipc_port` references held at the remote node by local user. These are grouped in a `ipc_pset_t`. Another `ipc_port_t` in the node serves for signaling or control. The `mach_node` object is defined in osfmk/kern/mach_node.h like so:

Listing 10-26: The `mach_node` object

```
/*** Mach Node Section
 *
 *  An instance of mach_node is allocated for each node known to mach.
 *  In-kernel interfaces use a pointer to this structure to refer to a node.
 *  External interfaces and protocols refer to node by id (mach_node_id_t).
 */
typedef struct mach_node *mach_node_t;

struct mach_node {
    /* Static node details, provided by the link driver at registration */
    struct mnl_node_info info;

    lck_spin_t      node_lock_data;

    /* Flags and status word */
    uint32_t        link:2;         // See MNL_LINK* defines
    uint32_t        published:1;    // True if node server has send-right
    uint32_t        active:1;       // True if node is up and ready
    uint32_t        suspended:1;    // True if node is active but sleeping
    uint32_t        dead:1;         // True if node is dead
    uint32_t        _reserved:26;   // Fill out the 32b flags field

    /* port/space/set */
    ipc_space_t     proxy_space;    // Kernel special space for proxy rights
    ipc_pset_t      proxy_port_set; // All proxy ports are in this set
    ipc_port_t      bootstrap_port; // Port for which "noded" holds rcv right
    ipc_port_t      control_port;   // For control & ack/nak messages

    /* Misc */
    int             proto_vers;     // Protocol version in use for this node
    mach_node_t     antecedent;     // Pointer to prior encarnation of this node id
};
```

Review Questions

1. Why is there such an overlap between `mach_port` MIG messages and the `kernelrpc_mach_port_..._traps`? Why would the former not suffice on its own?
2. Write out the pseudocode (or, for more of a challenge, D script or C program, assuming kernel read/write primitives as with the QiLin toolkit), to resolve a mach port name in a task to its kernel address, as shown in Output 10-6.
3. Looking at Output 10-6, what observation can you make on the relation between the `ie_object` reported to user mode (as in the output of /tmp/mp), and the actual kernel object address (in the output of xnoop ports)
4. Why did Apple choose to remove the per-processor cache for `struct kmsg` allocation in Darwin 18? What is now used instead? (q.v. Chapter 12)
5. As noted, there is an upper bound on port right descriptors sent in messages. This cap (in `ipc_kmsg_copyin_body`) is far too high as is. Why would any bound imposed at a message level be insufficient to thwart spraying attacks?
6. Given the answer to the above, what other check could Apple impose in XNU to better mitigate port and OOL-memory spraying attacks?
7. (Advanced) Implement the primitive for controlled memory allocations from user mode using Mach OOL descriptors and arbitrary kernel memory read/write described in this chapter.

References

1. Brandon Azad - "voucher_swap: Exploiting MIG reference counting in iOS 12" - https://googleprojectzero.blogspot.com/2019/01/voucherswap-exploiting-mig-reference.html
2. Black et al. - "Fast Local IPC" - http://usenix.org/legacy/publications/library/proceedings/sd96/full_papers/black.ps

11

Mapped out: Mach Memory Management

Memory provides the very fabric of the reality in which processes, and even the kernel itself, operate. The sockets, files and other objects processes "see" in user space - and even the processes and threads themselves - are merely constructs in virtual memory.

Chapter 8 of the previous volume examined virtual memory from the user-mode and process perspective. The kernel view, however, is quite different, and comprised of two different layers. First, we take a look at the `vm_map` layer, which manages the virtual memory mappings of a given task in an architecture independent manner. This layer is rich with `vm_map` structures corresponding to virtual address spaces, laid out through `vm_map_entrys` and backed by `vm_objects`, which maintain the underlying `vm_pages`. `memory_object` pagers handle the fetching and syncing of these pages from their backing store.

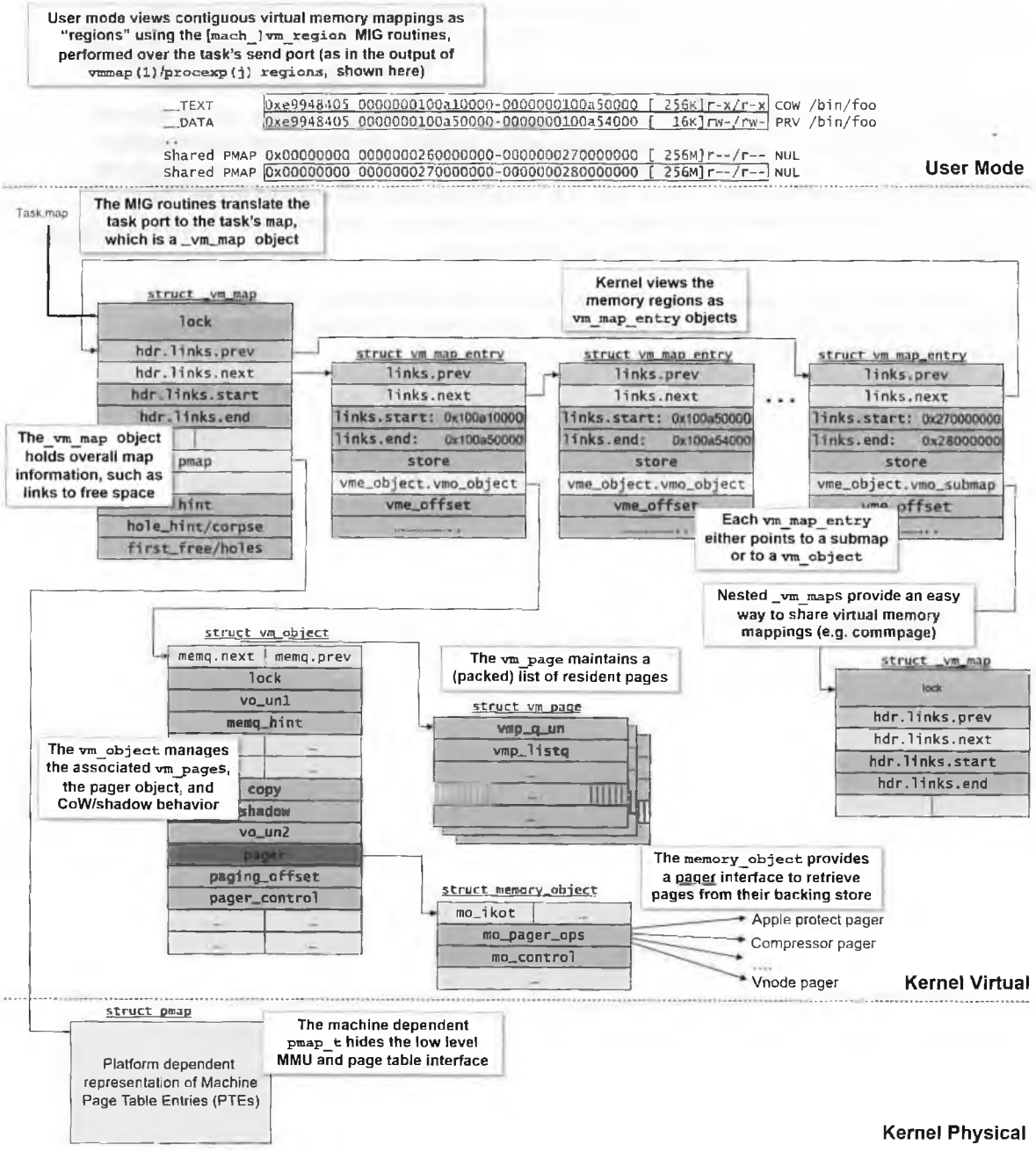
We then turn to the `pmap` layer, which handles the physical memory backing the virtual pages. This layer is naturally machine-dependent, yet remains surprisingly uniform across architecture despite MMU and page table idiosyncrasies.

A Bird's Eye View

Although XNU exposes the BSD memory management system calls (I/8) and, internally, provides BSD MALLOC zones to other BSD layer components, actual memory management is performed entirely by Mach abstractions. And, although it has evolved considerably over time, the fundamental design tenets of modern day XNU aren't far off from those of the original Mach kernel. To this day, a good reference is Rashid, Tevanian et al.'s article[1]. Though dated, it clearly describes the design rationale behind Mach's Virtual Memory subsystem.

Memory management can conceptually be grasped as two distinct layers. The top layer - `vm_map` - handles virtual memory, and it is the one exposed to applications (via the `mach_vm` and `vm_map` MIG subsystems), and to the kernel itself. The bottom layer - `pmap` handles the physical memory backing the virtual, dealing with aspects such as managing paging, and the backing store - responsible for getting and storing memory when it's not directly in RAM.

Figure 11-1: The high level view of Mach's virtual memory architecture



The vm_map Layer

A virtual address space can be enormous - spanning up to a hypothetical 16 exabytes in a 64-bit architecture. In practice, the actual addressable limit is on order of 1-256 terabytes, but even that is above and beyond the ability of RAM and backing stores to satisfy. Further, the address space is often sparse, in that it contains contiguous chunks, separated by wide, unallocated gaps. An effective representation, therefore, is in the form of linked lists, wherein each list element represents a contiguous block of the address space, and points to the next, in increasing addressing order. This is exactly what the `struct _vm_map` aims to achieve.

The `struct _vm_map`

The `_vm_map` represents the address space mappings of a task. The `_vm_maps` are created through `vm_map_create[_options]()`, which takes the map's address range (`min` and `max`), and a `pmap` pointer. This pointer links the virtual address space to its physical mappings, while maintaining the physical layer opaque (as a `pmap_t`). The `pmap` may be left NULL, usually when temporary maps are created to hold entries.

`vm_create[_options]()` allocates `_vm_maps` from the `maps` zone, but the objects are of differing sizes. The `vm_map` is presently 244 bytes in MacOS 14 (down from the 256 of MacOS 13) , with the `maps` zone element size set to 248 (to preserve address alignment). In *OS variants `maps` is 280 bytes, owing the difference in size to the `lck_rw_t` in its beginning.

Embedded in each `_vm_map` (following the lock) is a `struct vm_map_header`, which chains together `vm_map_entry` structures. Each `vm_map_entry` represents a single mapping, which is visible in user mode as a region. The `vm_map_links` a doubly linked list (`next/prev`) chains the header and the mappings together. This way, the `vm_map_header`'s `vm_map_links` span the entire mapped virtual address space of the process (i.e. point to the first and last `vm_map_entry`), and the each `vm_map_entry` objects links with its previous and next region, is ascending adress order.

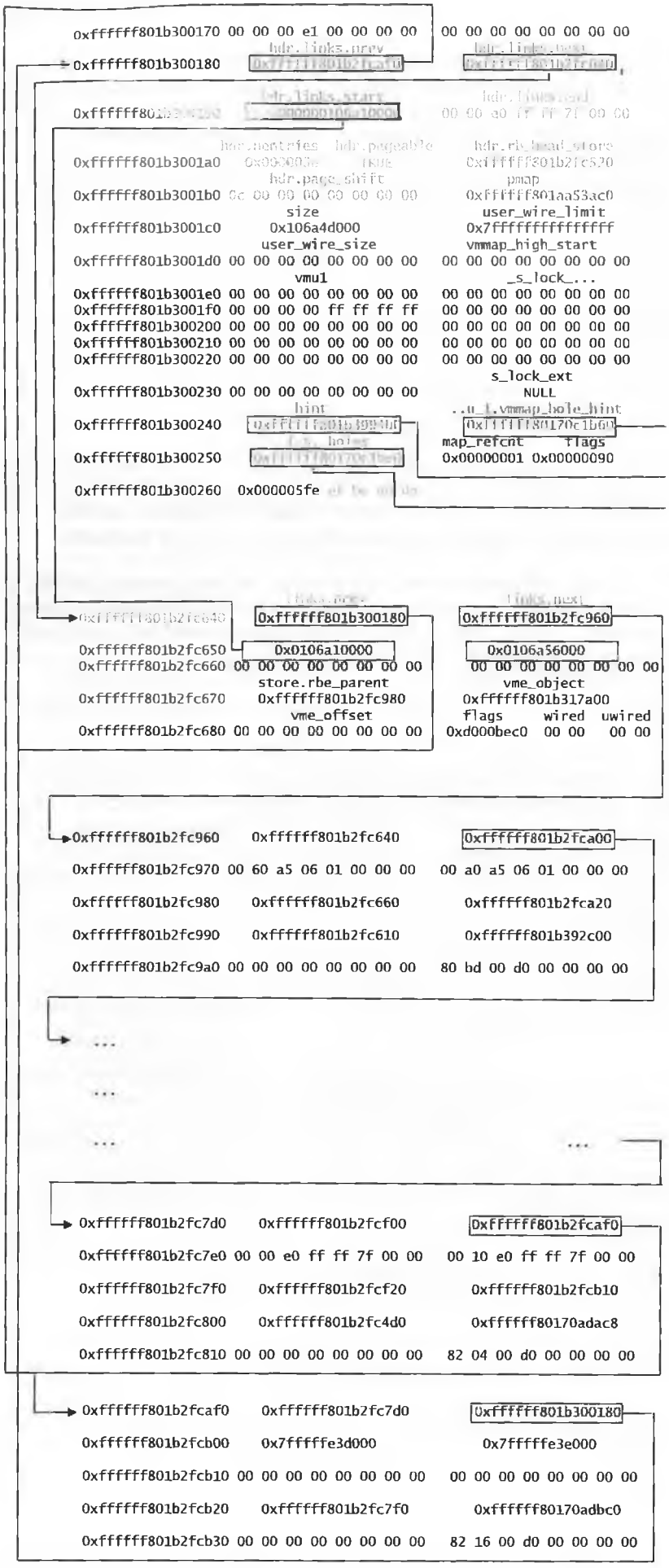
The `vm_map_entry` objects are the same across all Darwin variants, with all conditional `..DEBUG` fields not used. The objects are presently 80 bytes, allocated from a dedicated zone (`vm map entries`). Each `vm_map_entry` may represent a single contiguous mapping, or itself be a submap. In the former case, the `vm_map_entry`'s `is_sub_map` flag is false, and its `vme_object`'s `vmo_object`, is a `vm_object_t`, along with an offset into the object. In the latter case, `is_sub_map` is true, and the `vme_object` union is instead treading as a `vmo_submap` pointer to a sub map. Using submaps makes it easy to share memory between processes, as tasks which use the same submap see the same `vm_entry` objects backing them. Indeed, the `vm_map_entry` contains an additional `is_shared` flag, indicating the entry (i.e. one or more pages) is shared, and `use_pmap`, indicating that the physical memory should be shared as well (rather than be copy on write).

The `_vm_map` structure maintains a `_first_free` field as a hint to the first free space in the map. Over time, though, holes may form - either when allocations request specific addresses, or when freed. Keeping track of holes is useful, since it keeps the `vm_map` as compact as possible when holes are reused. As of Darwin 15, the `struct _vm_map` supports hole optimization, as indicated by a `holelistenabled` flag. Hole entries are `vm_map_links` (that is, just the queue chain specifying the doubly linked list, start and end), which are allocated from the `vm_map_holes_zone`. The entries are then linked into the `vm_map`'s `holes_list`, which overlaps with the `_first_free`, repurposed inside the `f_s` union. A `hole_hint` pointer is kept in another union (`vm_map_u_1`), repurposing the `vmmap_corpse_footprint`. This naturally means that the hole optimization is mutually exclusive with corpses.

Figure 11-2 (spanning the next two pages) shows an example `_vm_map` with some `vm_map_entry` objects, and other substructures, in detail.

---

* - `use_pmap` doesn't make sense for `vm_object` entries, so it its purpose is to indicate use of pmap accounting, or other.


0xffffff801b300170 00 00 00 e1 00 00 00 00 00 00 00 00 00 00 00 00
hdr.links.prev
hdr.links.next
0xffffff801b300180
hdr.links.start
hdr.links.end
hdr.nentries hdr.pageable hdr.rb_head_store
0xffffff801b3001a0 0x00000006 TRUE 0xffffff801b2fc520
hdr.page_shift pmap
0xffffff801b3001b0 0c 00 00 00 00 00 00 00 0xffffff801aa53ac0
size user_wire_limit
0xffffff801b3001c0 0x106a4d000 0x7fffffffffffff
user_wire_size vmmap_high_start
0xffffff801b3001d0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
vmu1 _s_lock_...
0xffffff801b3001e0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0xffffff801b3001f0 00 00 00 00 ff ff ff ff 00 00 00 00 00 00 00 00
0xffffff801b300200 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0xffffff801b300210 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0xffffff801b300220 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
s_lock_ext
0xffffff801b300230 00 00 00 00 00 00 00 00 NULL
hint ..u_l.vmmap_hole_hint
0xffffff801b300240 0xffffff801b3934b0 0xffffff80170c1b60
holes_entry map_refcnt flags
0xffffff801b300250 0xffffff80170c1be0 0x00000001 0x00000090
0xffffff801b300260 0x000005fe
0xffffff801b2fc640 0xffffff801b300180 0xffffff801b2fc960
0xffffff801b2fc650 0x0106a10000 0x0106a56000
0xffffff801b2fc660 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
store.rbe_parent vme_object
0xffffff801b2fc670 0xffffff801b2fc980 0xffffff801b317a00
vme_offset flags wired uwired
0xffffff801b2fc680 00 00 00 00 00 00 00 00 0xd000bec0 00 00 00 00
0xffffff801b2fc960 0xffffff801b2fc640 0xffffff801b2fca00
0xffffff801b2fc970 00 60 a5 06 01 00 00 00 00 a0 a5 06 01 00 00 00
0xffffff801b2fc980 0xffffff801b2fc660 0xffffff801b2fca20
0xffffff801b2fc990 0xffffff801b2fc610 0xffffff801b392c00
0xffffff801b2fc9a0 00 00 00 00 00 00 00 00 80 bd 00 d0 00 00 00 00
...
...
...
...
0xffffff801b2fc7d0 0xffffff801b2fcf00 0xffffff801b2fcaf0
0xffffff801b2fc7e0 00 00 e0 ff ff 7f 00 00 00 10 e0 ff ff 7f 00 00
0xffffff801b2fc7f0 0xffffff801b2fcf20 0xffffff801b2fcb10
0xffffff801b2fc800 0xffffff801b2fc4d0 0xffffff80170adac8
0xffffff801b2fc810 00 00 00 00 00 00 00 00 82 04 00 d0 00 00 00 00
0xffffff801b2fcaf0 0xffffff801b2fc7d0 0xffffff801b300180
0xffffff801b2fcb00 0x7fffffe3d000 0x7fffffe3e000
0xffffff801b2fcb10 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0xffffff801b2fcb20 0xffffff801b2fc7f0 0xffffff80170adbc0
0xffffff801b2fcb30 00 00 00 00 00 00 00 00 82 16 00 d0 00 00 00 00

```
struct _vm_map {

  lck_rw_t                lock;             /* map lock */
  struct vm_map_header    hdr;              /* Map entry header */
  pmap_t                  pmap;             /* Physical map */
  vm_map_size_t           size;             /* virtual size */
  vm_map_size_t           user_wire_limit;  /* rlimit user locked mem */
  vm_map_size_t           user_wire_size;   /* size of user locked mem */
#if __x86_64__
  vm_map_offset_t         vmmap_high_start; // x86_64 only
#endif
  union {
        vm_map_offset_t   vmu1_highest_entry_end;
        vm_map_offset_t   vmu1_lowest_unnestable_start;
  } vmu1;
  decl_lck_mtx_data(,     s_lock)           /* Lock ref, res fields */
  lck_mtx_ext_t           s_lock_ext;
  vm_map_entry_t          hint;             /* hint for quick lookups */
  union {
      struct vm_map_links* vmmap_hole_hint;
      struct vm_map_corpse_footprint_header *vmmap_corpse_footprint;
  } vmmap_u_1;
  union{
     vm_map_entry_t       _first_free;      /* First free space hint */
     struct vm_map_links  _holes;           /* links all holes between entries */
  } f_s;              /* Union for free space data structures being used */
  int                     map_refcnt;       /* Reference count */

#if   TASK_SWAPPER    // FALSE on MacOS
  int                     res_count;        /* Residence count (swap) */
  int                     sw_state;         /* Swap state */
#endif  /* TASK_SWAPPER */

  unsigned int
  /* boolean_t */         flags:13
  /* reserved */          pad:19;
  unsigned int            timestamp;        /* Version number */
}
```

```
0xffffff80170c1b60    0xffffff80170c1980    0xffffff801b40ecc0
0xffffff80170c1b70 00 90 11 08 01 00 00 00  00 60 22 06 00 70 00 00

0xffffff801b3994b0    0xffffff801b399b40    0xffffff801c6cd280
0xffffff801b3994c0    0x7000062a9000        0x7000062a9000

0xffffff80170c1be0    0xffffff80170c1b40    0xffffff80170c19e0
0xffffff80170c1bf0    0x106a73000           0x0106aa5000
```

```
__TEXT   0x68024927 0000000106a10000-0000000106a56000 [ 280K]r-x/rwx COW /sbin/launchd
__DATA   0x68024927 0000000106a10000-0000000106a56000 [ 280K]r-x/rwx COW /sbin/launchd
...
MALLOC metadata         0x68130a27 0000000106a72000-0000000106a73000 [   4K]r--/rwx PRV
MALLOC_LARGE metadata   0x68138927 0000000106aa5000-0000000106aa6000 [   4K]rw-/rwx PRV
...
MALLOC_LARGE (freed)    0x6949ef27 0000000108019000-0000000108119000 [1024K]rw-/rwx PRV
STACK_GUARD             0x00000000 00007000006226000-00007000006227000 [   4K]---/rwx NUL
Stack                   0x68138827 00007000006227000-00007000062a9000 [ 520K]rw-/rwx PRV
STACK_GUARD             0x00000000 00007000062a9000-00007000062aa000 [   4K]---/rwx NUL
...
(0)     0x00000000 00007fffffe3d000-00007fffffe3e000 [   4K]r-x/r-x NUL
```

```
struct vm_map_entry {
        struct vm_map_links     links;          /* links to other entries */
        struct vm_map_store     store;          // RBTree: rbe_[left/right/parent] */
        union vm_map_object     vme_object;     /* object I point to */
        vm_object_offset_t      vme_offset;     /* offset into object */
        /* boolean_t */ is_shared:1,    /* region is shared */
        /* boolean_t */ is_sub_map:1,   /* Is "object" a submap? */
        /* boolean_t */ in_transition:1, /* Entry being changed */
        /* boolean_t */ needs_wakeup:1, /* Waiters on in_transition */
        /* vm_behavior_t */ behavior:2, /* user paging behavior hint */
        /* boolean_t */ needs_copy:1,   /* object need to be copied? */
        /* vm_prot_t */ protection:3,   /* protection code */
        /* vm_prot_t */ max_protection:3, /* maximum protection */
        /* vm_inherit_t */ inheritance:2, /* inheritance */
        /* boolean_t */ use_pmap:1,     /* overloaded.. */
        /* boolean_t */ no_cache:1,     /* should new pages be cached? */
        /* boolean_t */ permanent:1,    /* mapping can not be removed */
        /* boolean_t */ superpage_size:1, /* use superpages of a certain size */
        /* boolean_t */ map_aligned:1,  /* align to map's page size */
        /* boolean_t */ zero_wired_pages:1, /* zero out when deleted ..*/
        /* boolean_t */ used_for_jit:1,
        /* boolean_t */ pmap_cs_associated:1, /* pmap_cs will validate */
        /* boolean_t */ from_reserved_zone:1, /* Allocated from kernel reserved */
        /* iokit accounting: use the virtual size rather than resident size: */
        /* boolean_t */ iokit_acct:1,
        /* boolean_t */ vme_resilient_codesign:1,
        /* boolean_t */ vme_resilient_media:1,
        /* boolean_t */ vme_atomic:1, /* entry cannot be split/coalesced */
        __unused:4;
        unsigned short          wired_count;    /* can be paged if = 0 */
        unsigned short          user_wired_count; /* for vm_wire */
```

vm_objects

The vm_map_entry structures hold a vme_object union, which either points to a nested vm_map structure (in the case of shared or moved mappings), or to a vm_object structure. In the case of submaps, more vm_map_entry structures will be involved, so eventually all virtual memory mappings point to vm_objects.

The vm_object structure (defined in osfmk/vm/vm_object.h and partially explained in osfmk/object/vm_object.c*) provides the link between the memory mapping and the actual mapped pages. In other words, whereas the vm_map_entry provided the address space detail of the mapping, the vm_object provides the metadata about the memory which is mapped at said address range.

The vm_objects thus provide a list of the vm_pages (representing resident memory pages) providing the mapping, and metadata about these pages. The vm_page list is maintained in the memq member, which is a vm_page_queue_head_t, whose other queue elements are vm_page structures. The rest of the vm_object structure holds a lck_rw_t, a memq_hint, several counts, pointers to shadow and copy objects, a link to the backing store's pager (described later), and numerous flags, as well as an all_wanted array of 11 bits, used for synchronization through vm_object_[assert_]wait, .._sleep() and ..wakeup() (in osfmk/vm/vm_object.h).

An important field in the vm_object_t is the copy_strategy. This is one of the MEMORY_OBJECT_COPY_* constants defined in osfmk/mach/memory_object[_types].h. The supported copy strategies (..SYMMETRIC, ..DELAY (asymmetric copy) or ..NONE) are called on when the vm_object is copied using vm_copy_strategically(). A strategy of ..NONE requires a slow copy, by faulting in each source page and vm_page_copy()ing to a new page. In objects mapped for Copy-on-Write, ..SYMMETRIC or ..DELAY dictates the semantics: In the symmetric case, the same vm_object is used across multiple copy operations (i.e. all vm_map_entry objects of the same CoW memory use the same vm_map_object). This holds true until the copy operation occurs, at which point the dirtying task gets its own "shadow" vm_object, whose shadow field points to the original. In the asymmetric case, the vm_object holds only partial data, with the rest of the details held in another vm_object, pointed to by the copy field. The copy object points back to the source object through its shadow field (which also increases the reference count by one). Should a copy on write occur, the copy object will get the changed pages, and this way the source remains unchanged. This will be revisited later in this chapter, when discussing copy operations.

Of the multiple vm_objects used, the kernel_object_store stands out. This object, defined in osfmk/vm/vm_object.c and commonly used througout the kernel through its kernel_object pointer, holds all of the kernel's wired memory, and backs the kernel_map. As we will see later, kernel_memory_allocate() is commonly called with the kernel_map and KMA_KOBJECT, specifying allocations are to be backed by the kernel_object, unless pageable. There are also many cases wherein handled memory objects are checked against the kernel_object, to determine if they are for kernel wired pages or not.

vm_pages

The MMU divides the system RAM into physical pages, and the vm_page structure provide XNU's representation of these pages. The vm_pages array, indexed by physical page number, thus maintains the metadata for every such page. The array is of a dynamically allocated number of vm_page entries, determined in pmap_startup() by checking pmap_free_pages() and then calling pmap_steal_memory(). Once allocated, vm_page_array_beginning.. and ...end_addr mark its boundaries, and a loop iterates over all the pages, calling vm_page_init() for each. In addition to the vm_page_array, some vm_page structures do not correspond to resident pages. These pages are allocated from the vm pages zone.

* A comment in ofmk/vm/vm_object.c starts describing the vm_object detail, but then dozes off with a "ZZZ Continue this comment." which also remains uncontinued some 15 Darwin version later..

At any time, a given physical page may either be in use (i.e. belonging to some `vm_object`) or may be free. When a page is in use, its corresponding `vm_page` structure holds a `vmp_object` pointer to the `vm_object` to which it belongs. It also maintains the `vmp_offset` into the object, and a `vmp_listq`, a `vm_page_queue_chain_t` holding its position in a list of all pages belonging to the same `vm_object`. Another important field is the `vmp_next_m`, a (packed) pointer to a hash bucket in the **Virtual to Physical** table (also known as the "VP table", which is used by `vm_page_lookup()` when resolving a `vm_object` and `vm_offset` to a page.

The `vm_page` holds a plethora of `vmp_*` flags, capturing the page state - e.g. dirty, write protected, code signed tainted/valid, etc. The structure (defined in osfmk/vm/vm_page.h) explains these flags in detail. Depending on the field, locking is managed either through the page queue lock, or the lock on the page's owning `vm_object`, or both (in the case of `vmp_[object/offset]`).

Page queue states are recorded in the four bits of `vmp_q_state`. The state machine in kernel corresponds roughly to the one visible in user mode and depicted in I/9-1, but contains more states, shown in Listing 11-3:

Listing 11-3: The `vm_page` possible `vmp_q_states`

```
#define VM_PAGE_INACTIVE(m)                     (vm_page_inactive_states[m->vmp_q_state])
#define VM_PAGE_PAGEABLE(m)                     (vm_page_pageable_states[m->vmp_q_state])
#define VM_PAGE_NON_SPECULATIVE_PAGEABLE(m)     (vm_page_non_speculative_pageable_states[m->vmp_q_state])
#define VM_PAGE_ACTIVE_OR_INACTIVE(m)           (vm_page_active_or_inactive_states[m->vmp_q_state])

#define VM_PAGE_NOT_ON_Q                0       /* page is not present on any queue, nor is it wired... m
#define VM_PAGE_IS_WIRED                1       /* page is currently wired */
#define VM_PAGE_USED_BY_COMPRESSOR      2       /* page is in use by the compressor to hold compressed da
#define VM_PAGE_ON_FREE_Q               3       /* page is on the main free queue */
#define VM_PAGE_ON_FREE_LOCAL_Q         4       /* page is on one of the per-CPU free queues */
#define VM_PAGE_ON_FREE_LOPAGE_Q        5       /* page is on the lopage pool free list */
#define VM_PAGE_ON_THROTTLED_Q          6       /* page is on the throttled queue... we stash anonymous p
#define VM_PAGE_ON_PAGEOUT_Q            7       /* page is on one of the pageout queues (internal/externa
#define VM_PAGE_ON_SPECULATIVE_Q        8       /* page is on one of the speculative queues */
#define VM_PAGE_ON_ACTIVE_LOCAL_Q       9       /* page has recently been created and is being held in on
#define VM_PAGE_ON_ACTIVE_Q             10      /* page is in global active queue */
#define VM_PAGE_ON_INACTIVE_INTERNAL_Q  11      /* page is on the inactive internal queue a.k.a.  anonymo
#define VM_PAGE_ON_INACTIVE_EXTERNAL_Q  12      /* page in on the inactive external queue a.k.a.  file ba
#define VM_PAGE_ON_INACTIVE_CLEANED_Q   13      /* page has been cleaned to a backing file and is ready t
#define VM_PAGE_ON_SECLUDED_Q           14      /* page is on secluded queue */
#define VM_PAGE_Q_STATE_LAST_VALID_VALUE  14 /* we currently use 4 bits for the state... don't let thi
```

The `vm_page` objects are initialized by `vm_page_init()` (osfmk/vm/vm_resident.c). This routine prepopulates all the fields from a `vm_page_template` (itself set up during `vm_page_bootstrap()`), such that new pages have all their fields set to 0, their flags `FALSE` (with the notable exception of `vmp_busy`), and are not on any queue.

When the system needs a new page, `vm_page_grab[_options]()` is called, and returns a `vm_page_t` which can be used. The routine first attempts to find a page on the per-cpu free list. If none can be found, it acquires the `vm_page_queue_free_lock`, but prior to getting a page from the global queue, makes an attempt to check the per-cpu free list. If it's still empty, the routine moves a few pages to the per-cpu page, and returns one of them.

When `CONFIG_SECLUDED_MEMORY` is `#defined`, `vm_page_grab_options()` can recognize the `VM_PAGE_GRAB_SECLUDED`, which is set when the `vm_object`'s `can_grab_secluded` is `TRUE`. This also requires that `task_can_use_secluded_mem()`, as defined in osfmk/kern/task.c. Secluded pages are handled in their own queue, `vm_page_queue_secluded`. Secluded memory exists for IOKit, Apps and the FileCache.

Not all pages, however, need to be physical. Guard pages, are one example of **fictitious pages**, whose purpose lies solely in their memory protections. Since guard pages allow no access, there is no sense in backing them anyway. Fictitious pages are grabbed by `vm_page_grab_fictitious[_common]()`, and their physical page number is set to -1.

If pages were grabbed from the global queue, it means there are now fewer free pages in the system. If `vm_page_free_count` falls below `vm_page_free_min`, a call to `VM_CHECK_MEMORYSTATUS` indicates a memory pressure condition, which either triggers jetsam (if `CONFIG_JETSAM`), or `vm_pressure_response()`, in MacOS.

The `vm_pages` structures are specifically optimized to fit into 64-bytes or less, to optimize the CPU cache lines of 64-bit architectures. This requires the queue links to be packed on those architectures. Beginning with the `memq` links from the `vm_object`, each `vm_page` link is condensed from 64-bit to 32 using macros which resolve to `vm_page_[un]pack_ptr()` inlined calls. The packing routine checks if the pointer falls within the `vm_page_array`. If it does, it replaces it with its 32-bit offset from the `vm_page_array_beginning_addr`, and flags its most significant bit (0x80000000) indicating a packed pointer. Otherwise, it takes the offset of the pointer from the beginning of kernel memory (`VM_MIN_KERNEL_AND_KEXT_ADDRESS`), and then right shifts it by `VM_PACKED_POINTER_SHIFT` (6), so the offset fits in 31 bits. This way, the most significant bit serves as an indication for the unpacking routine, on which memory address to use as a reference base. Walking the queue naturally requires performing these unpacking operations at every step, for which the `vm_page_queue_[first/last/iterate]` macros (from osfmk/vm/vm_page.h) are used.

User mode interface

The older `vm_map` and newer `mach_vm` MIG subsystems enable manipulating some of the `vm_map` and `vm_map_entry` fields from user mode. `[mach_]vm_protect` alters `vm_map_entry` objects' `protection` and `max_protection`. `[mach_]vm_behavior` alters the `behavior` field. `vm_map_exec_lockdown` (#3831, as of Darwin 18) sets the `vm_map`'s `map_disallow_new_exec`, disallowing new mappings with `VM_PROT_EXECUTE`. Calling BSD's memory syscalls (`minherit(2)`, `madvise(2)`, `mmap(2)`, etc.) likewise affects the resulting `vm_map_entry` objects. As an example, `mmap(2)`'s undocumented `MAP_JIT` flag indirectly (via `vm_map_enter`'s `vmkf_map_jit`) sets `used_for_jit` in the created `vm_map_entry`.

The `[mach_]vm_region[_recurse][_64]` APIs*, or the `proc_info`s flavors of `PROC_PIDREGION[PATH]INFO` can be used from user mode to query the `vm_map_entry` of a task or process. Every "region" returned by these APIs is essentially a `vm_map_entry`, reduced to its user-mode exported properties. Both `vmmap(1)` and `procexp(j) regions` are highly useful to display the memory regions of a task, and the latter is preferred since it can (presently) display the regions of *any* task, when used on *OS or on MacOS when SIP is disabled.

Another interesting set of MIG calls are the `mach_vm`'s `_mach_make_memory_entry` (#4817) and `vm_map`'s `mach_make_memory_entry[_64]`. These enable the creation of a `vm_object` according to `MAP_MEM_*` flags in osfmk/mach/memory_object_types.h, which is visible in user mode as a `mem_entry_name_port_t` (in kernel, a port right of type `IKOT_NAMED_ENTRY`). This object, essentially an unmapped collection of pages, can be sent over Mach IPC, and passed to `[mach_]vm_map` as the *object* parameter (with an optional *offset* into it), so that it can be mapped, and thus shared. There are also two routines in `mach_host`, `mach_memory_object_memory_entry[_64]` (#203, 215), for use by pagers, and Darwin 18 has a new <mach/memory_entry.defs> subsystem (#4900) offering purgeable_control and access tracking.

The `mach_host` subsystem's `host_virtual_physical_table_info` (#209) MIG routine could at one time return the VP table information to user mode, but has long since been disabled unless `MACH_VM_DEBUG`. This routine was just one of several interfaces offered to user mode: `vm_mapped_pages_info` (#3819) could provide the array of resident pages (but is now also only if `MACH_VM_DEBUG`). Another `vm_map` MIG call, `vm_region_object_create` (#3820), at one time supported creating entire `vm_maps` in this way, but its user-mode interface has been removed. MIG routines still supported are `[mach/vm_map]_page_query`, which provide the same functionality as BSD's `mincore(2)`, and `mach_vm_page_info` (#4819), which provides the `vm_page_info_basic` (from <mach/vm_region.h>), with the id of the owning `vm_object`, similar to those shown by `procexp` *`pid`* `regions`.

---

* - As before, a funny observation is that `vm_region_recurse_64` is marked as `TEMPORARYTEMPORARY...` promising it will go away after `vm_region_recurse` migrates to 64-bits. 15 Darwin versions later, it's still there..

Experiment: Inspecting `vm_objects` in memory

Using the DTrace kernel memory reader or `xnoop`, you can dump any `vm object` type. The output below is from Figure 7-2, displaying the first `vm_map_entry`'s `vm_object`.

Output 11-4-a: `vm_objects` in memory

```
#
# Verifying that the address is indeed a vm object...
#
bash-3.2# xnoop zone 0xffffff801b317a00
0xffffff801b317a00:vm objects (Element size: 256 bytes)
bash-3.2# xnoop dump 0xffffff801b317a00,256 smart
0xffffff801b317a00 e8 c5 6c 02 e8 c5 6c 02   00 00 00 21 00 00 00 00    ..l...l....!....
0xffffff801b317a10 00 00 00 00 00 00 00 00   00 50 05 00 00 00 00 00    .........P......
0xffffff801b317a20 00 00 00 00 00 00 00 00   03 00 00 00 00 00 00 00    ................
0xffffff801b317a30 00 00 00 00 00 00 00 00   00 00 00 00 00 00 00 00    ................
                   vm objects (copy)
0xffffff801b317a40    0xffffff801b286e00     00 00 00 00 00 00 00 00    .n(.............
0xffffff801b317a50 00 00 00 00 00 00 00 00   00 00 00 00 00 00 00 00    ................
0xffffff801b317a60 00 00 00 00 00 00 00 00   04 00 00 00 00 00 00 00    ................
0xffffff801b317a70 00 00 00 00 00 00 39 01   00 00 00 00 00 00 00 00    ......9.........
0xffffff801b317a80 00 00 00 00 00 00 00 00   00 00 00 00 00 00 00 00    ................
0xffffff801b317a90 00 00 00 00 00 00 00 00   00 00 00 00 00 00 00 00    ................
0xffffff801b317aa0 ff ff ff ff ff ff ff ff   80 00 0c 00 00 00 00 00    ................
                                               cached_list.next
0xffffff801b317ab0 00 00 00 00 00 00 00 00      0xffffff801b317ab8
                    cached_list.prev
0xffffff801b317ac0    0xffffff801b317ab8     00 00 00 00 00 00 00 00    .z1.............
0xffffff801b317ad0 00 00 00 00 00 00 00 00   00 00 00 00 00 00 00 00    ................
0xffffff801b317ae0 00 00 00 00 00 00 00 00   00 00 00 00 00 00 00 00    ................
0xffffff801b317af0 00 00 00 00 00 00 00 00      0xdeadbeefdeadbeef
```

Note the eight bytes right at the beginning of the object. These are clearly the same `uint32_t` value repeated twice. The most significant bit of the value is not set, which means it falls outside the `vm_page` array. Shifting the value by six and adding to `VM_MIN_KERNEL_AND_KEXT_ADDRESS` yields:

```
0x026cc5e8 << 6 = 0x9b317a00 + VM_MIN_KERNEL_AND_KEXT_ADDRESS =
    0xFFFFFF8000000000 - 0x80000000 = 0xffffff7f80000000 =
                        ffffff801b317a00
```

Meaning the `memq` points to itself, with a single element. This is pointing back to the `vm_object_t`, which is indeed out of the `vm_page` array, as is indicated by the topmost bit. In other words, the queue of pages is empty.

As another example, consider the second `vm_map_entry`'s `vm_object`:

Output 11-4-b: `vm_objects` in memory

```
bash-3.2# xnoop zone 0xffffff801b392c00
0xffffff801b392c00:vm objects (Element size: 256 bytes)
bash-3.2# xnoop dump 0xffffff801b392c00,256
0xffffff801b392c00 9e e8 05 80 a1 e8 05 80   00 00 00 21 00 00 00 00    ...........!....
0xffffff801b392c10 00 00 00 00 00 00 00 00   00 40 00 00 00 00 00 00    .........@......
0xffffff801b392c20    0xffffff80187b0338     01 00 00 00 04 00 00 00    8.{.............
..
```

This time, the two 32-bit values are different (but close to each other), and the most significant bit in each is set, indicating they fall within the `vm_page_array`. The `vm_page_array_beginning_addr` is an exported symbol in MacOS, so we can read this value easily, and work out the math:

Output 11-4-c: The `vm_page` packed pointers

```
bash-3.2# xnoop dump _vm_page_array_beginning_addr,16
0xffffff80144c1908    0xffffff8017302000 0xffffff8018e8aba0
#
# Get the value, top bit shaved off, but multiply times sizeof(vm_page)
#
bash-3.2# perl -e 'printf ("0x%x\n",0xffffff8017302000 + (0x0005e89e * 64))'
0xffffff80175f64f0
```

Which, indeed, falls been the `vm_page_array_beginning_addr` and `..ending_addr`.

`vm_map_enter` and friends

After familiarizing with the `_vm_map`, we can now turn to the most important aspect of its lifecycle - memory allocation. From user mode, memory allocation is commonly performed by calling `[mach_]vm_allocate()` (MIG routines #4800 or #3801). The two APIs exist side by side due to 32-bit legacy, but the `mach_vm_allocate()` is the newer and preferred way to go. The `host_priv` subsystem also offers `vm_allocate_cpm()` (#406), which is used to allocate contiguous physical memory. The `_kernelrpc_mach_vm_allocate_trap` (#10) can also be used. In kernel mode, `vm_allocate_kernel()` is often called instead.

There are also several kernel-only routines which are used quite often for specific allocation types. These include `vm_map_enter_mem_object[_helper]()` and its variants, `vm_map_enter_mem_object[_prefault/control]()`. All of these allow specifying an `IKOT_NAMED_ENTRY` or `IKOT_MEMORY_OBJECT` port* whose `ip_kobject` is a copy object (`vm_map_copy_t`, described later), a submap or `vm_object_t` to provide the mapped pages for the allocation. The `_prefault` variant allows specifying a list of pages to pre-fault as an optimization, and the `_control` variant provides a `memory_object_control` (described under "pagers", later) from which the `vm_object_t` can be deduced.

All roads, however, sooner or later lead to `vm_map_enter`**. This is a complex routine (defined in osfmk/vm/vm_map.c) of many arguments, shown in Table 11-5:

Table 11-5: The many arguments of vm_map_enter

Argument	Provides
`vm_map_t map`	The target map to allocate in (kernel's or task's)
`vm_map_offset_t *address`	An in/out parameter specifying the requested address (if NULL, allocated anywhere)
`vm_map_size_t size`	Size of allocation requested
`vm_map_offset_t mask`	Bitmask (for aligned allocations). Usually 0
`int flags`	`VM_FLAGS_*` from osfmk/mach/vm_statistics.h
`vm_map_kernel_flags_t vmk_flags`	Kernel private allocation flags from osfmk/mach/vm_statistics.h
`vm_tag_t alias`	Specify memory tag for allocation (also through high bits of *flags*)
`vm_object_t object`	Backing memory object; may be left NULL
`vm_object_offset_t offset`	Offset into memory object, if applicable
`boolean_t needs_copy`	Copy-on-Write mapping
`vm_prot_t cur_protection`	Initial protection flags (`VM_PROT_*`)
`vm_prot_t max_protection`	Maximum settable protection flags
`vm_inherit_t inheritance)`	Inheritance, per `mach_vm_inherit` (`VM_INHERIT_[NONE/SHARE/COPY/DONATE]`)

Putting all these arguments into a more human readable form: The `vm_map_enter` routine, then, will create an allocation in the specified *map*, at the *address* specified, subject to the *mask* (or will determine the address if left NULL). The allocation of *size* bytes will optionally be backed by the *object* at the sepcified *offset*, and will be allocated according to the *flags* and *vmk_flags*, possibly as a Copy-On-Write (COW) mapping, if *needs_copy*, and with *inheritance* (as per `mach_vm_inherit()` or POSIX's `minherit()`). The memory protection of this allocation shall be initially set to *cur_protection*, but will not exceed *max_protection*.

Of all header files to put the flags, they ended up in osfmk/mach/vm_statistics.h (also exported to user mode as <mach/vm_statistics.h>), with some documentation in that header. The following flags are presently defined:

---

* - another "temporary" workaround until named entry and memory object handling were to be unified, but JMM never got to implement..

** - ARM64 also has a `vm_map_enter_fourk()` variant, using the 4k pager (which is discussed later). This is similar enough to the standard `vm_map_enter()` that we can choose to overlook it

Table 11-6: The `VM_FLAGS_*` accepted by `vm_map_enter()`

	VM_MAP_FLAGS_*	Purpose
0x00000000	VM_FLAGS_FIXED	Request allocation at fixed address (as `in` parameter), if unset, or at any address (as `out` parameter), if set
0x00000001	VM_FLAGS_ANYWHERE	
0x00000002	VM_FLAGS_PURGABLE	Allocation is purgeable memory
0x00000004	VM_FLAGS_4GB_CHUNK	Darwin 18: Region chunked up into 4GB sized pieces
0x00000008	VM_FLAGS_RANDOM_ADDR	Darwin 16: choose random address, not next available hole
0x00000010	VM_FLAGS_NO_CACHE	Place on speculative, rather than active queue
0x00000020	VM_FLAGS_RESILIENT_CODESIGN	Darwin 15: Codesigning/Media is resilient
0x00000040	VM_FLAGS_RESILIENT_MEDIA	
0x00004000	VM_FLAGS_OVERWRITE	Delete any existing mappings first (if `VM_MAP_FIXED`)
0x00010000	VM_FLAGS_SUPERPAGE_NONE	No superpage use
0x00020000	VM_FLAGS_SUPERPAGE_SIZE_ANY	Any superpage preferred
0x00040000	VM_FLAGS_SUPERPAGE_SIZE_NONE	2MB superpages preferred (Intel)
0x00100000	VM_FLAGS_RETURN_DATA_ADDR	Darwin 13: Return address of data, rather than of page
0x00800000	VM_FLAGS_RETURN_4K_DATA_ADDR	Darwin 15: Return 4K aligned address of data (ARM64)
0xFF000000	VM_FLAGS_ALIAS_MASK	Not flags, but rather the memory tag used for the allocation

As noted in I/8, an undocumented feature of XNU's `mmap(2)` implementation is that the `fd` argument, when used in `MAP_ANON` cases, has been repurposed so as to pass memory tags. In practice, the `fd` can pass other `VM_FLAGS_*` as well, specifically `VM_FLAGS_SUPERPAGE_*`, `..._PURGEABLE` and `..._4GB_CHUNK`. Two particularly intriguing flags are the `VM_FLAGS_RESILIENT[_MEDIA/CODESIGN]`. These are interesting in that they do not seem to be set anywhere in the kernel proper (unless set from user mode Mach APIs or POSIX's `mmap(2)` with the non-POSIX undocumented `MAP_RESILIENT_CODESIGN` flag.)

The osfmk/mach/vm_statistics.h header also contains the definitions for the kernel-only flags, which are naturally `KERNEL_PRIVATE` and thus not in the user mode header. These are defined as a bit structure, and are well documented in the macro which ensures they are all unset:

Listing 11-7: The kernel-only flags of `vm_map_enter()` (as of XNU 4903)

```
#define VM_MAP_KERNEL_FLAGS_NONE (vm_map_kernel_flags_t) {                      \
  .vmkf_atomic_entry = 0, /* keep entry atomic (no coalescing)                  \
  .vmkf_permanent = 0,    /* mapping can NEVER be unmapped                      \
  .vmkf_guard_after = 0,  /* guard page after the mapping                       \
  .vmkf_guard_before = 0, /* guard page before the mapping                      \
  .vmkf_submap = 0,       /* mapping a VM submap                                \
  .vmkf_already = 0,      /* OK if same mapping already exists                  \
  .vmkf_beyond_max = 0,   /* map beyond the map's max offset                    \
  .vmkf_no_pmap_check = 0, /* do not check that pmap is empty                   \
  .vmkf_map_jit = 0,      /* mark entry as JIT region                           \
  .vmkf_iokit_acct = 0,   /* IOKit accounting                                   \
  .vmkf_keep_map_locked = 0, /* keep map locked when returning from vm_map_enter() \
  .vmkf_fourk = 0,        /* use fourk pager                                    \
  .vmkf_overwrite_immutable = 0,  /* can overwrite immutable mappings           \
  .vmkf_remap_prot_copy = 0, /* vm_remap for VM_PROT_COPY                       \
  .vmkf_cs_enforcement_override = 0, /* override CS_ENFORCEMENT                 \
  .vmkf_cs_enforcement = 0,  /* new value for CS_ENFORCEMENT                    \
  .__vmkf_unused = 0                                                            \
}
```

Allocating memory (highlights)

The flow of `vm_map_enter()` is quite long, and even more complicated considering it is usually called from `vm_map_enter_mem_object_[helper/control]()`, and their wrappers, which add their own handling. Let's focus on the simple case, of just `vm_map_enter()` itself.

The routing has many special cases to consider, such as invalid flag combinations, or contradictions between kernel flags and user specified ones. The first special case considered is for superpages, which on the x86_64 architectures are 2MB pages. Next, special consideration is given in *OS for cases where the requested `cur_protection` specifies `VM_PROT_EXECUTE`. When `VM_PROT_WRITE` is also requested, unless the entry is for JIT (as specified by `vmkf_map_jit`), the `VM_PROT_EXECUTE` bit is silently dropped. Darwin 18 extends this to MacOS as well, wherein the mapping simply fails (due to `#if VM_PROTECT_WX_FAIL`). JIT mappings require the caller to hold the `dynamic-codesigning` entitlement, and (in MacOS 14) `com.apple.security.cs.allow-jit`. In *OS, only one JIT mapping is allowed per map. Also, as of Darwin 18, the `_vm_map` holds a `map_disallow_new_exec` flag. Set by `vm_map_exec_lockdown()` (`mach_vm` MIG subsystem routine #3831), this prevents any additional mappings of executable memory.

Most allocations specify `VM_MAP_ANYWHERE`, and so locating a new address for the allocation is simple. Unless a random address was requested (or forced, as is the case on *OS for JIT entries), a search for space begins at the map's `min_offset`. If the map does not allow reuse (`disable_vmentry_reuse` is `TRUE`), the `VM_MAP_HIGHEST_ENTRY` macro (from osfmk/vm/vm_map_store.h) retrieves the highest available address. Otherwise, if the map maintains a hole list, it will be traversed to find a hole which fits the requested size. This code has been factored out of `vm_map_enter` into the `vm_map_find_space` routine (also in osfmk/vm/vm_map.c), which is called from `kernel_memory_allocate()`, and several other routines on the `kernel_map`. Additionally, when a `vm_map_entry` is freed, it will either form a hole or coalesce with an existing hole. This is handled by the `vm_map_store`, which is an `RB_HEAD`, in the process of rebalancing the tree.

Those allocations specifying `VM_MAP_FIXED` require extra handling. There are more cases in which the allocations may fail (if the address is invalid, or out of bounds), or overlap with existing mappings. A temporary `zap_old_map` is created to hold any existing `vm_map_entry` mappings which overlap, and as they are removed prior to the allocation, `vm_map_delete()` saves the entries. Should any part of the process fail, any saved entries in the `zap_old_map` can (hopefully) be restored prior to returning, and the temporary maps destroyed.

Once a suitable address has been located, the allocation can be attempted. An optimization attempts to coalesce the previous entry with the current one. This requires over 25 conditions to be met (matching the previous entry's attributes with those of the requested allocation), but - if met - `vm_object_coalesce()` extends the previous entry's `vm_object`, and the entry's size and range are adjusted accordingly, and the operation succeeds.

Suffice it, however, for one of conditions to not hold, and a new entry will have to be created. This is handled by `vm_map_entry_insert()`. Following the insertion, if the entry is backed by a pager (discussed later), the pager is requested (through `memory_object_map()`) to map the pages. If the map's `wiring_required` is set, a call to `vm_map_wire_kernel` wires (locks) the resident pages. The mapping operation is complete.

Memory operations (in particular, copying, discussed later) require atomicity, as race conditions can lead to controlled memory corruption and other TOCTOU vulnerabilities. Maverick Darwin security researcher Ian Beer demonstrated a novel way to exploit such issues using specically crafted OOL descriptors in Mach messages, resulting in effectively sharing the memory. A detailed discussion can be found in the Google Project Zero Blog[2], and his Objective-By-The-Sea 2.0 presentation[3].

The vm_map_copy object

The vm_map_copy is an object representing a pending copy operation of some memory region "in transit" from one map to another. That is, it specifies the source object being copied or moved, before its destination map can be determined. Copy objects are commonly created when vm_map_copyin() takes the corresponding memory to be copied from the source map, and are placed into the destination map (once determined) by vm_map_copyout() (to fill an unused region) or vm_map_copy_overwrite[_aligned] (to overwrite an existing region). As discussed in Chapter 10, vm_map_copy objects are commonly used to represent Mach message OOL memory descriptors, during the time the message is sent by the sender but before it is received by the recipient.

The vm_map_copy objects are allocated from their own zone, vm map copies, when created by a call to vm_map_copy_allocate(), and freed through a call to vm_map_discard(). The object definition is in osfmk/vm/vm_map.h:

Listing 11-8: The vm_map_copy_t (from XNU 4903's osfmk/vm/vm_map.h)

```
struct vm_map_copy {
        int                     type;
#define VM_MAP_COPY_ENTRY_LIST          1
#define VM_MAP_COPY_OBJECT              2
#define VM_MAP_COPY_KERNEL_BUFFER       3
        vm_object_offset_t      offset;
        vm_map_size_t           size;
        union {
                struct vm_map_header    hdr;      /* ENTRY_LIST */
                vm_object_t             object;   /* OBJECT */
                uint8_t                 kdata[0]; /* KERNEL_BUFFER */
        } c_u;
};
```

VM_MAP_COPY_ENTRY_LIST

As we have seen, the vm_map_header represents a contiguous virtual memory region. This is the same header found in the _vm_map, which makes it easy to use with existing map handling logic. The vm_map_copy_to_entry returns the hdr's links, and vm_map_copy_[first/last]_entry get the first and last links.

When vm_map_copy_discard()ing a VM_MAP_COPY_ENTRY_LIST, the hdr links list is walked, unlinking each entry in turn, and calling vm_[map/object]_deallocate (for submap or regular entries), before doing away with the entire vm_map_copy object, using vm_map_copy_entry_dispose().

VM_MAP_COPY_OBJECT

A common use of VM_MAP_COPY_OBJECTs is to void an existing vm_map_copy_t, prior to disposing of it. In those cases, the vm_map_copy's type is set to 2, and the object pointer is set to VM_OBJECT_NULL. This is performed by vm_map_copy_copy(), which copies the contents of one vm_map_copy object into another. There is also a routine to perform vm_map_copyin_object(), but it is unused.

VM_MAP_COPY_KERNEL_BUFFER

The third case is that of the copied data being in the kdata. The kdata is a zero-sized array so as not to change the over all structure size (80 bytes), but the size holds its true size, which is up to MSG_OOL_SIZE_SMALL_MAX (2 pages), or kalloc_max_prerounded (largest kalloc zone, which is also two pages). This means, that the vm_map_copy_t is of variable size, and therefore cannot be allocated from the vm map copies zone. The objects are created by the vm_map_copyin_kernel_buffer() routine, which ensures their size does not exceed the above maximum, and then kalloc()s the copy object (per its size, plus the header size up to the union), calling copyinmap() from the src_map and src_addr to handle the actual memory copy. The vm_map_copy_discard() is a simple kfree() of the kalloc()ed size.

Copying memory

There are several memory copy scenarios, but the simplest one to explore are the `[mach_]vm_copy()` MIG routines (in osfmk/vm/vm_user.c), because both the source and target map are the same. The implementation is a straightforward application of `vm_map_copyin()` from the map, which creates (as an *out* parameter) the copy object. If successful, a call to `vm_map_copy_overwrite()` on the same map with the copy object (this time as an *in* parameter). This also consumes the copy object, otherwise (if unsuccessful) the copy object is explicitly released with a call to `vm_map_discard()`. The `[mach_]vm_read_overwrite()` MIG routines are implemented in the similar manner, with the only difference that the target map is the thread's current map.

Zooming in on `vm_map_copyin()`, leads to `vm_map_copyin_common()`, which in turn wraps `vm_map_copyin_internal()`. All these functions need no knowledge of the target, and therefore take as argument only the *src_map* and *address/len*, along with some flags. `vm_map_copyin_internal()` is responsible for obtaining the source memory, which is performed (after validation of the address/len and map) in one of two ways:

- Small enough copies that can fit in a `VM_MAP_COPY_KERNEL_BUFFER` call on `vm_map_copyin_kernel_buffer()`, which allocates a `vm_map_copy` object of that type (from the nearest `kalloc.*` zone, and performs a physical copy of the memory into it, returning the `vm_map_copy` object.
- All others require the allocation of a `VM_MAP_COPY_ENTRY_LIST` object from the `vm map copies` zone (per a call to `vm_map_copy_allocate()`). The *src_map* needs to be traversed from the *address*, over however many entries make up the *len* bytes. Each entry may be a submap, or backed by a `vm_object`. Since submap entries have to be recursed until their `vm_objects` are found, eventually all handling is at the `vm_object` level. A `new_entry` is created, and the `src_object` is copied to the new entry's object - either slowly (if wired), delayed (shared memory) or strategically (in most other cases). There are also optimizations, such as moving the object when the source memory is to be destroyed anyway. The `new_entry` is then added (with `vm_map_copy_entry_link()`) to the `vm_map_copy` object, which is then returned to the caller.

At this point, the `vm_map_copy` object returned by `..copyin` is passed (along with the *dst_map* and *dst_addr* to `vm_map_copyout()` (There is no need to pass the size, since that is a member of the *copy* object). This falls through to `vm_map_copyout_internal()`, which again has to handle multiple cases:

- Copy objects of type `VM_MAP_COPY_ENTRY_LIST` are copied in a manner similar to how they were copied in. The `vm_map_copy` object's entries are linked into the target map, using either `vm_map_copy_insert()` (which consumes the object), or `vm_map_copy_remap()` (which copies the entries but does not free the object.
- Copy objects of type `VM_MAP_COPY_OBJECT` are just placed directly into the target map, using `vm_map_enter()` on the `dst_map` at `dst_addr`, with the copy object. This, however, is a rare occurrence since it assumes the object was created by `vm_map_copyin_object()`, which is no longer used anywhere in the kernel. The `vm_map_copy_overwrite[_nested]` variants (which copy over previously allocated memory) assert against this type.
- Copy objects of type `VM_MAP_COPY_KERNEL_BUFFER` can immediately be copied to the destination map, by a call to `vm_map_copyout_kernel_buffer`. This routine calls `vm_map_enter()` to perform the destination memory allocation, and then `copyout()` to perform a physical memory copy from the buffer inside the `vm_map_copy` object to the target map (temporarily calling `vm_map_switch()` to assume the target map's identity for the operation.

Other copy scenarios the reader is encouraged to look into are `[mach_]vm_read()`, which is similar, but returns the copy object to the caller, rather than performing a copy. In this case, the MIG generated code is responsible for handling the object and performing the final copy into the caller's address space. Likewise, `mach_vm_read_list()`, which handles arrays of `vm_copy_ts`, iterating over them using `vm_map_copyout()`.

Pagers

Most systems inevitably use more virtual memory than available RAM. For this reason, it is often said that the physical memory acts as a window into the virtual, as virtual memory is paged in to and out of physical pages from and to a **backing store**. The backing store is usually a more persistent form of storage, such as a (named) file or (anonymous) swap, a device, or even another form of memory, such as encrypted or compressed RAM.

True to its micro-kernel origins, Mach supports the concept of **external memory managers** (EMMs), commonly referred to as **pagers***. A pager is an object whose purpose is to handle paging operations between virtual memory and its backing store. Using a pager completely hides the implementation of these operations, allowing its caller - the vm pageout daemon - to remain blissfully unaware of paging operation details. The actual operation may be performed inline, through a dedicated thread, implemented by a kext, and in theory even page to a remote backing store. Conversely, the pager does not involve itself with the paging policy - and merely implements the low level operations as directed by the pageout daemon. In some ways, the present implementation of memory management has devolved since its inception. The original design allowed the memory manager to be a truly external (i.e. outside of kernel space) entity, with which communication could be carried over Mach ports and messages.

The original pager implementation - the default pager, handling anonymous (swap) memory - was used from Mach's early days till XNU-3248, after which it was removed in favor of the compressor. The pager mechanism is easily extensible to new backing store types, and Apple has already made use of that fact several times, adding its own pagers, and ending up with seven pagers** as of Darwin 18.

Table 11-9: Pager types supported in Darwin 18

Pager type	Implemented in	Backing Store
Vnode	osfmk/vm/bsd_vm.c	Vnodes (`mmap(2)`ed data from files)
Device	osfmk/vm/device_vm.c	Devices
Apple Protect	osfmk/vm/vm_apple_protect.c	Encrypted memory
swapfile	osfmk/vm/vm_swapfile_pager.c	Swap file vnodes (see below)
Compressor	osfmk/vm/vm_compressor_pager.c	Compressed RAM (Darwin 13)
4K	osfmk/vm/vm_fourk_pager.c	4K page emulation on 16K pages (Darwin 15)
shared	osfmk/vm/vm_shared_region_pager.c	DYLD Shared cache (Darwin 18)

The Pager object

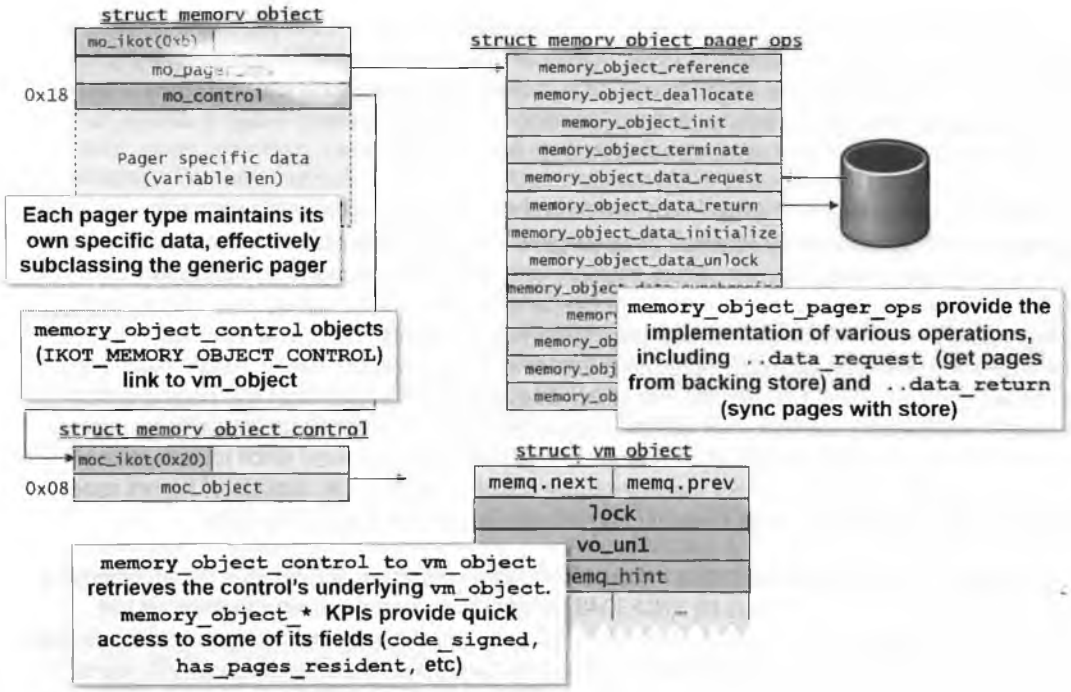
All pagers extend the base `struct memory_object`. This is simple 24-byte structure, whose first field `mo_ikot` intentionally overlaps with the `ipc_object_t`'s `io_bits`, and holds the object type - always an `IKOT_MEMORY_OBJECT` (0xb). Following that are the `mo_pager_ops` - a(n aligned) pointer to the set of function pointers for pager lifecycle callbacks, which will be discussed shortly. The pager objects also include an `mo_control` field, which is a pointer to a `struct memory_object_control`.

The `struct memory_object_control` is an even smaller structure (of 16 bytes), allocated from the `mem_obj_control` zone, and holding only the four bytes of `moc_ikot` - always an `IKOT_MEMORY_OBJECT_CONTROL` (0x20), and a pointer to the corresponding `vm_object`. Helper functions of `memory_object[_control]_to_vm_object()` provide easy access to the `vm_object`, and further helpers (all in osfmk/mach/memory_object.h) build on them to access attributes in the `vm_object`, e.g. `memory_object[_is]_signed()`, `..._mark_[un]used]`, `..[eligible_for_secluded|io_tracking]`, etc.

The various pager types also have their own specific data, which follows the 24-byte `memory_object` structure. Pagers are free to extend the "base class" in any way seen fit, and indeed the various pagers have different sizes as a result. Some pagers (namely, the vnode, device and compressor) are allocated from dedicated zones, while the rest are `kalloc()`ed.

* - BSD also adopted Mach pagers, but those of 2.0 and far back in 4.4BSD. The BSD implementation thus differs from that of XNU.
**- Another pager, the default freezer, was added in Darwin 12 to support MemoryStatus, but removed by 16

Figure 11-10: Mach Pagers



Note, that there are often multiple instances of the same pager types, corresponding to the `vm_objects` or vnode objects they are meant to assist with. The `vm_objects` are associated with their pagers and control through `vm_object_memory_object_associate()`, commonly called through `memory_object_create_named()`.

Pager Lifecycle callbacks

Pagers follow a specific lifecycle model consisting of a set of callbacks, which the pager object provides through its `mo_pager_ops` field. These correspond to specific pager events, though not all of these need be supported. Some pagers thus leave those as NULL, or call `panic()` in their callbacks - this is the case with read-only pagers (i.e. those which never write back), such as the swapfile or apple_protect pagers. Table 11-11 shows these lifecycle calls:

Table 11-11: The pager lifecycle callbacks

Callback	Called when
`.._referece`	Pager object is refrenced
`.._deallocate`	Release reference, possibly freeing pager
`.._init`	Pager is used for first time, or reused
`.._terminate`	Remove memory object associated with pager
`.._data_request`	Page-in request (get from backing store)
`.._data_return`	Return data to VM manager
`.._data_initialize`	Provide initial data for memory object
`.._data_unlock`	Request unlocking for data access
`.._synchronize`	Perform page sync with backing store. No longer supported.
`.._map`	Memory object associated with pager is being mapped
`.._last_unmap`	Memory object associated with pager no longer mapped anywhere
`.._data_reclaim`	

Wrapper functions in osfmk/vm/memory_object.c are responsible for dereferencing the pager callback methods, and implement the memory_object.defs subsystem (2200), which (though no longer used) is a vestige of the microkernel architecture of Mach, wherein pagers were external.

A notable omission from the `mo_pager_ops` is the constructor - a function to create a new pager object. Pagers by convention do provide a *`pager`*`_create()` function, but each pager requires a different number of arguments when created (or none, in the case of the fourk pager). Also outside the official lifecycle are the pager-specific *`pager`*`_bootstrap()` functions, used to set up any locks, lock groups or zones to support the pager. These bootstrap functions are all called from `vnode_pager_bootstrap()`, itself invoked from `bsd_init()` after the network initialization.

The vnode pager

The vnode pager is one of the most common pager types, responsible for paging data in and out of memory mapped vnodes. Every `VREG` vnode has a corresponding pager object, allocated from the `vnode pager structures` zone: When the vnode is created, `vnode_create_internal()` calls `ubc_info_init_withsize()`, which in turn calls `vnode_pager_setup()`, and links the returned pager to the vnode's UBC info's `ui_pager`. `ubc_info_init_withsize()` then calls `memory_object_create_named()` to create the pager control, which it link to the Vnode's UBC `ui_control`.

The vnode pager object (defined in osfmk/vm/bsd_vm.c) is a small one, spanning 40 bytes - i.e. merely 16 bytes over the common header, allowing for a 32-bit reference count and a pointer to corresponding vnode object. The vnode pager provides the `vnode_pager_ops`, all of which are standard. The two important ones are the `vnode_pager_data_request()` and `..data_return()` callbacks, which call `vnode_pager_cluster_read()` and `..write()`, respectively. The `vnode_pager_cluster_read()` routine calls `vnode_pagein()`, and likewise `..write()` calls `vnode_pageout()` (both in bsd/vm/vnode_pager.c). These, in turn call `VNOP_PAGE[IN/OUT]`, which are handled by the filesystem on which the vnode resides.

The swapfile pager

The swapfile pager, despite its name, is not the one to actually handle swapping to the swap file. Rather, it is specifically designed for one particular use case - satisfying user mode read requests for swap files, i.e. the vnodes in /var/vm, so as to prevent malicious attempts at reading swapped data, which might lead to information disclosure. As explained by a comment in osfmk/vm/vm_swapfile_pager.c, there is no way to prevent memory mapping of vnodes by a privileged process, so the approach taken is to intercept the `_data_request` operations, and `memset` the contents of the returned pages to zero (with a newline at the end of each page), "to keep line counters happy". The memory thus looks like Listing 11-12:

Listing 11-12: The Darwin swap files, as seen from a user mode `read(2)` or `mmap(2)`

```
00000000  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
*
00000ff0  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 0a  |................|
00001000  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
*
00001ff0  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 0a  |................|
00002000  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
*
00002ff0  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 0a  |................|
..
```

The `mmap(2)` syscall (in bsd/kern/kern_mman.c) calls `swapfile_pager_setup()` if `vnode_isswap()` detects that the vnode to be `mmap()`ed for read is a swap node (write mappings are outright disallowed). `mmap(2)`-ed swap vnode access is also restricted - both `vn_read()` and `vn_rdwr_64()` explicitly calls `vnode_isswap()`, and if the vnode read is indeed a swap file, the reading is done by `vn_read_swapfile()` (which fakes the same NULLs and newline at end of page) rather than the usual `VNOP_READ()` path. As of Darwin 19, this also extends to /private/var/vm/kernelcore (`kIOCoreDumpPath`).

The compressor pager

The compressor pager enjoys special status, as it replaced what was commonly called the "default memory manager" or "default pager". The default pager was traditionally used to manage the system swap file - i.e., as the backing store of anonymous memory which the vnode pager, by definition, cannot handle. These `vm_objects`, marked `internal`, would be swapped to /var/vm/swapfile[0..].

Compressing memory, rather than simply swapping it, might not make immediate sense. After all, compressed RAM still takes up physical pages, whereas swapping releases them entirely. Still, RAM tends to compress rather well, and handling its compression is significantly faster than the I/O incured by swap. The winning argument, however, is that NAND storage does not play well at all with swap. Thus, Apple decided to introduce RAM compression in Darwin 13, primarily for iOS 7, but also for MacOS 10.9. The two pagers, default and compressor, lived alongside one another for a few versions, until the former was retired in Darwin 16.

So long as RAM is sufficient, the compressor can operate well. In times of memory pressure, however, swapping to a swapfile may still be an option, depending on the compressor's operating mode. The choice of operating mode is dictated by the `vm_compressor_mode`, also readable in user-mode through the `vm.compressor_mode` sysctl(2) MIB. The value is set in `vm_fault_init()` according to the `vm_compressor` boot-arg, or (if not specified), the device tree, or the default value specified in osfmk/vm/vm_compressor.c, through compilation directives.

On MacOS, the `vm.compressor_mode` is set to `VM_PAGER_COMPRESSOR_WITH_SWAP (4)`, allowing the failsafe to a swapfile. In *OS variants the mode is set to `..._NO_SWAP` (2). Thus, when `compressor_needs_to_swap()` returns TRUE, the swap file handling logic (in osfmk/vm/vm_compressor_backing_store.c and bsd/vm/vm_compressor_backing_file.c) is still used on MacOS. On *OS, memory compaction and throttling occurs instead.

Internals*

Each `vm_object` holds a reference to its own pager, so `vm_compressor_pager_init()` creates the `compressor_pager` zone to hold up to 10,000 such objects, each of which is 64 bytes. It then calls `vm_compressor_init()` (from osfmk/vm/vm_compressor.c to initialize the subsystem. There is an additional `compressor_object` which (like the `kernel_object`) is a `vm_object_t` holding all the pages belonging to the compressor.

As part of its private data, the `compressor_pager` uses a mutex lock, and compressor slots. The slots are used to locate the compressed pages, and can be obtained using `compressor_pager_slot_lookup()`, which can also perform the allocations, if necessary. Two slots are embedded right into the pager data (`cpgr_eslots[2]`). An additional number of direct slots are pointed to from `cpgr_dslots`, and more indirect slots are double dereferenced from `cpgr_islots`. The private data tracks both `cpgr_num_slots[_occupied]`, and which slot is used for a particular page is determined by the offset into the memory object, preferring the embedded, then the direct, and lastly the indirect.

The slots themselves are 32-bit integers, but further subdivided as a `c_slot_mapping`: The most significant 22 bits indicate the number of a compresssor segment (`c_seg`), and the lower 10 bits an index into the segment. The segments are maintained in an array of `c_segu` union pointers. The unions are interpreted either as 32 bit `c_segno`, or as a pointer to a segment object (`c_segment_t`).

Each `c_segment` object contains metadata about compressed pages, and holds them back to back in a `c_store`. The `c_store` is a union, representing the two possible locations of the backing store: in memory (as a `c_buffer`) or (if the `C_SEG_IS_ONDISK` macro indicates it is swapped out), a `c_swap_handle`. Segments are allocated from their dedicated `compressor_segment` zone, but the `c_segment` array, along with a `c_buffers` pointer array, are allocated from the `compressor_map`, a submap of the `kernel_map`.

---

* - An overview of the compressor pager's inner workings can be found in the Digital Investigation journal[4], which analyzes Darwin 13's compressor from a forensics perspective, although the implementation has evolved since the one discussed there.

The compressed pages are manipulated through `vm_compressor_pager_[get|put]()` operations, which operate directly on physical page numbers, either populating their contents (decompressing) from the compressed slot, or compressing their contents to it. The `..get()` operation is called from `vm_compressor_pager_get()`, itself called directly from `vm_fault_page()` when the corresponding `vm_object` is marked `internal` (i.e. on a page fault of a page belonging to the compressor). The `vm_compressor_pager_put()` is called from the `vm_pageout_iothread_internal_continue()` (continuation), when the pageout code swaps an anonymous page.

The default compression algorithm used is based on one by Wilson & Kaplan[5]. The interface is declared in osfmk/vm/WKdm_new.h, with a machine-optimized assembly implementation (in ./osfmk/*arch*/WKdm*.s). The algorithm was supported as far as back as Darwin 10, wherein it was introduced to deal with the hibernation image (supporting code in iokit/Kernel/IOHibernateIO.cpp), through C wrappers (in libkern/kxld/WKdmCompress.c). As of Darwin 16, vm_compression_algorithms.[ch] supports `LZ4`, either instead of or alongside WKdm. The algorithm can be selected through the `vm_compressor_codec` boot argument.

Since faulting code handles the actual decompression, the `.._pager_data_request()` callback simply needs to find the slot by a call to `compressor_pager_slot_lookup()` (and there is no need for a `..data_return()`). The compressor pager also provides a `vm_compressor_pager_state_get()` function (outside of its normal `mo_pager_ops`), through which it notifies the VM subsystem if a given memory object can be retrieved by it. The function returns one of the states defines in osfmk/vm/vm_external.h (`VM_EXTERNAL_STATE_EXISTS`, `..UNKNOWN` or `..ABSENT`).

Lifecycle

The `vm_compressor_init()` routine is in charge of initializing the global compressor. It first runs a few quick sanity checks (ensuring that `C_SLOT_[PACK/UNPACK]_PTR` macros work properly), then sets various system defaults, such as its threshold divisors, and the `compressor_pool_max_size`. If the `vm_compression` boot-arg is set, it uses it to limit the compressor pool size. It then suballocates the `compressor_map` from the `kernel_map` (to hold the `c_segments` and `c_buffers`), starting greedily but retrying up to three times, and halving the pool size on every attempt.

Still in `vm_compressor_init()`, a call to `vm_compression_algorithm_init()` selects the default algorithm to use. Two threads are then started:

- The `vm_compressor_swap_trigger_thread()`: named `VM_cswap_trigger`, is responsible for calling `vm_compressor_compact_and_swap`, i.e. triggering compressor compaction and (on MacOS) possibly swapping to disk.
- The `vm_pageout_iothread_internal()` thread, named `VM_compressor` and started by `vm_pageout_internal_start()`, is in charge of placing internal `vm_object` owned pages (i.e. anonymous memory) onto the swapout queue. Normally, one thread suffices. In MacOS systems with over four CPUs, an additional thread is started. The `vmcomp_threads` boot argument can override the number of threads.

In MacOS, Since `VM_CONFIG_SWAP_IS_PRESENT` is `TRUE`, `vm_compressor_init()` calls `vm_compressor_swap_init()`, which starts three additional threads:

- The `vm_swapout_thread()`: named `VM_swapout`, and responsible for performing the actual swap to the system swap files through calls to `vm_swap_put()`, which performs the actual disk I/O operation using `vm_swapfile_io()`. The corresponding `vm_swap_get()` operations are performed from `c_seg_swapin()` during page decompression. This thread's id is saved in `vm_swapout_thread_id`, for the purposes of throttling.
- The `vm_swapfile_create_thread()`: creates additional swapfile vnodes through `vm_swap_create_file()` and `vm_swapfile_open()` when necessary. During heavy memory utilization these are the 1GB /var/vm/swapfile# which pop up. Manually removing them has no effect, since the kernel holds the vnode reference.

- The vm_[illegible]_thread(): responsible for defragmenting the swapfile, and reclaiming swap space, i.e. calling vm_swapfile_close() and removing the vnodes. The vnodes can be removed in any order, so the files in /var/vm/swapfile# may end up not sequential. The GC thread is created with TASK_POLICY_PASSIVE_IO.

The threads can easily be seen in the output of procexp 0 threads | grep VM. With the compressor subsystem initialized, any pages set to the throttled state may be made active, by a call to vm_page_reactivate_all_throttled() (from osfmk/vm/vm_resident.c).

Although its overall purpose is to save memory, the compressor does take up RAM and CPU, and therefore it is imperative to keep it operating smoothly and in check. The GC thread's initial TASK_POLICY_IO setting of THROTTLE_LEVEL_COMPRESSOR_TIER2. The VM_swapout thread (the one doing most of the I/O work) TASK_POLICY_IO is initially set to ..TIER2 as well. As the compressor does its work, vm_swapout_thread_throttle_adjust() is called to adjust the thread's I/O policy, according to a state machine, thereby deciding when to start the process of memory compaction, as well as when to throttle, unthrottle, or catch up.

Four threshold divisors to govern its lifecycle transitions. In the MacOS case, their values are set according to whether or not total system memory exceeds 3GB. In the *OS case, they are hard-coded.

Table 11-13: Compressor lifecycle settings

vm.compressor... sysctl(2) MIB	Purpose	MacOS	*OS
..minorcompact_threshold_divisor	Minor compaction	11/20	20
..majorcompact_threshold_divisor	Major compaction	13/25	30
..catchup_threshold_divisor	Catchup	35, 50	60
..unthrottle_threshold_divisor	Unthrottling	20/35	40

Note, that in order to avoid expensive floating point operations, the dividend is multiplied by 10, 9 or 11 through VM_PAGE_COMPRESSOR...THRESHOLD macros (in osfmk/vm/vm_compressor.h), themselves used by decision macros (COMPRESSOR_NEEDS_TO_SWAP, HARD_THROTTLE_LIMIT_REACHED, etc, ibid.). This means that the values are effectively 9-11% of the stated.

The Device Pager

The device pager is implemented in osfmk/vm/device_vm.c. Pager objects are allocated from the device node pager structures zone, with an element size of 72 bytes, consisting of a lock, ref_count, device_handle, size, is_mapped and flags. The flags defined in osfmk/device/device_port.h are several DEVICE_PAGER_* flags, controlling WIMG attributes (for memory caching, as described later), and backing memory attributes, specifically DEVICE_PAGER_CONTIGUOUS and ..NOPHYSCACHE.

Output 11-14: A device pager instance, in memory

```
                                                    _device_pager_ops
0xffffff801af6bd80 0b 00 00 00 00 00 00 00      0xffffff8014666d58
                      mem_obj_control
0xffffff801af6bd90     0xffffff801af6d690      00 00 00 00 00 00 00 00
0xffffff801af6bda0 00 00 00 00 ff ff ff ff     03 00 00 00 00 00 00 00
                           kalloc.80
0xffffff801af6bdb0     0xffffff801aabbfa0      00 00 02 00 00 00 00 00
0xffffff801af6bdc0 07 01 00 00 01 00 00 00
mes-Mac# ./xnoop dump 0xffffff801aabbfa0,80 smart
                   device node pager structures
0xffffff801aabbfa0     0xffffff801af6bd80      01 00 00 00 00 00 00 00
                           kalloc.192
0xffffff801aabbfb0     0xffffff801b01bc40      00 00 00 00 00 00 00 00    @...............
0xffffff801aabbfc0 00 00 00 00 00 00 00 00     00 00 00 00 00 00 00 00
0xffffff801aabbfd0 00 00 00 00 00 00 00 00     00 00 00 00 00 00 00 00
0xffffff801aabbfe0 00 00 00 00 00 00 00 00      0xdeadbeefdeadbeef
```

This pager is primarily used with IOMemoryDescriptors (described in Chapter 13). The ..data_return() and ..data_request() callbacks are both implemented over device_data_action (in iokit/Kernel/IOMemoryDescriptor.cpp), which interprets the device_handle port object as an IOMemoryDescriptorReserved. In the opposite direction, IOGeneralMemoryDescriptor::doMap and ..::redirect() both call IOMemoryDescriptor::populateDevicePager(), which then calls device_pager_populate_object().

The 4K Pager (*OS)

The 4K ("fourk") pager is a recent addition to ARM64 variants. Apple moved to an architecture page size of 16K sometime around the A9 and later processors. While a welcome change, this also caused problems with code relying on 4K pages, since this was now below the minimal size that could be allocated or mapped.

The 4K pager therefore provides mappings which are 4K aligned, but not necessarily 16K aligned (i.e. ending at an address of 0x4000, 0x8000 or 0xc000). The bulk of the work is in satisfying `..data_request`, which establishes a physical mapping for the source page, and another mapping for the destination, from which the client can read the data. The pager maintains `FOURK_PAGER_SLOTS` (i.e. 4) backing slots for each of the 4k mappings inside the 16k mapping, with each backing slot tracking the `vm_object` of that page and a `vm_mapping_offset` into it. The physical mappings have to be established directly using the `pmap` layer (specifically, `pmap_enter`), as discussed later in this chapter.

The shared region pager (Darwin 18)

Darwin's shared region support has existed for over a decade, in order to provide the commpage and the dyld shared cache. It is only in Darwin 18, however, that Apple added a specific pager for the dyld shared cache.

As explained in I/7, `dyld` makes use of a shared cache to prelink all of the dylibs and commonly used bundles on disk. Due to its size (over 1GB) it is imperative that this be shared between processes. `dyld` therefore calls the `__shared_region_map_and_slide_np` syscall (#438, from bsd/vm/vm_unix.c), to provide the kernel with the cache's file descriptor. Internally, `_shared_region_map_and_slide` (ibid.) obtains the underlying vnode, makes sure it meets security and code signature requirements, gets the file's memory object (= vnode pager's) control handle from `ubc_getobject()`, and then calls `vm_shared_region_map_file()`, with the number of requested mappings (usually, three), the handle, and the slide information. Most important is the shared cache's `slide_info`, which may be one of several versions - v1 (legacy), v2 (Darwin 16/17), v3 or v4 (both in Darwin 18).

`vm_shared_region_map_file()` (from osfmk/vm/vm_shared_region.c converts the `file_control` to its `vm_object`, and establishes the mappings (by calling `vm_map_enter_mem_object`). It then turns to sliding the region, through `vm_shared_region_slide()`. Over the years, this could only be slid once. Darwin 18 changes this with a call to `vm_region_slide_mapping()`.

`vm_shared_region_slide_mapping()` locates the map entry to slide, and calls `shared_region_pager_setup()`. This creates a new instance of a pager object by calling `shared_region_pager_create()`, which `kalloc()`s it in the nearest zone.

The pager's `..data_request()` calls `vm_shared_region_slide_page()` to perform the sliding. Here, too, are four possible versions of the implementation.

One cardinal issue that has plagued Apple's shared region - the `dyld_shared_cache` for over a decade is the lack of randomization in its placement. Although randomized on boot, the randomization occurs only once, and so all processes end up sharing not only the cache, but its exact address in memory. This is detrimental to system security, as it makes cross process exploitation (for example, by finding ROP/JOP gadgets in a target) trivial.

The Apple Protect pager

The Apple Protect pager is one of the more interesting pagers in XNU, as it provides a novel and (at the time of writing) powerful functionality yet unmatched by any other operating system - memory encryption. Apple chooses to apply this to text segments, if XNU is compiled with `CONFIG_CODE_DECRYPTION`, as is the case by default across all variants.

Like other pagers, Apple Protect is self contained, in osfmk/vm/vm_apple_protect.c. It only handles page-in requests, because text pages are mapped read-only and never dirtied. The pager object differs from the others in that it has a `struct pager_crypt_info *`, which decouples the decryption logic from the pager itself.

Listing 11-15: The `struct pager_crypt_info` from osfmk/kern/page_decrypt.h

```
typedef struct apple_protect_pager {
        /* mandatory generic header */
        struct memory_object ap_pgr_hdr;

        /* pager-specific data */
        queue_chain_t           pager_queue;    /* next & prev pagers */
        unsigned int            ref_count;      /* reference count */
        boolean_t               is_ready;       /* is this pager ready ? */
        boolean_t               is_mapped;      /* is this mem_obj mapped ? */
        vm_object_t             backing_object; /* VM obj w/ encrypted data */
        vm_object_offset_t      backing_offset;
        vm_object_offset_t      crypto_backing_offset; /* for key... */
        vm_object_offset_t      crypto_start;
        vm_object_offset_t      crypto_end;
        struct pager_crypt_info *crypt_info;
} *apple_protect_pager_t;

// The pager_crypt_info is defined in osfmk/kern/page_decrypt.h:
struct pager_crypt_info {
   /* Decrypt one page */
   int     (*page_decrypt)(const void *src_vaddr, void *dst_vaddr,
                           unsigned long long src_offset, void *crypt_ops);
   /* Pager using this crypter terminates - crypt module not needed anymore */
   void    (*crypt_end)(void *crypt_ops);
   /* Private data for the crypter */
   void    *crypt_ops;
   volatile int    crypt_refcnt;
};
```

The two methods provided by the `crypt_info` are thus called from the Apple protect pager's lifecycle: `page_decrypt` is called on `..data_request`, and `crypt_end()` is called in the process of deallocating the `apple_protect_pager`, or when an old pager is reused instead of a newly constructed one.

Listing 11-16: The `apple_protect_pager_data_request()` from osfmk/vm/vm_apple_protect.c

```
   /*
        * Decrypt the encrypted contents of the source page
        * into the destination page.
        */
       for (offset_in_page = 0;
            offset_in_page < PAGE_SIZE;
            offset_in_page += 4096) {
           ...
          ret = pager->crypt_info->page_decrypt(
                      (const void *)(src_vaddr + offset_in_page),
                      (void *)(dst_vaddr + offset_in_page),
                      ((pager->crypto_backing_offset -
                        pager->crypto_start) + /* XXX ? */
                       offset +
                       cur_offset +
                       offset_in_page),
                             pager->crypt_info->crypt_ops);
                      ...
              }
```

Let's consider the two case studies of this pager: MacOS' DSMOS and *OS's FairPlay.

MacOS: Dont Steal Mac OS X.kext

Ever since MacOS's move to the x86 architecture, it has drawn the attention of "Hackintosh" enthusiasts, who correctly figured that if the binaries are compiled for x86 architecture, there should be no real obstacle to running them on generic, non-Apple hardware. This led to the creation of the OSX86 and other projects. Unlike OpenDarwin, which aims to port the Apple open sources alone, Hackintoshes violate the MacOS EULA.

The EULA, however, is a legal obstacle, and not a technical one. Apple therefore sought to put another hurdle, by encrypting some of its most important binaries, particularly those dealing with UI. As explained in I/6, this was done by using the `SG_PROTECTED_VERSION_1` flag in the `LC_SEGMENT[_64]` of the Mach-O binaries, and examples can be found to this day in Finder, Metadata.framework's mds, and others. Dumping the binaries' text section will reveal nonsensical, encrypted contents.

MacOS XNU registered a special kernel extension - with the actual name of "Dont Steal Mac OS X.kext" to enforce this encryption. XNU exports the special `dsmos_page_transform_hook` (from osfmk/kern/page_decrypt.c), which allows DSMOS (as the kext is known internally) to register a hook function into `dsmos_hook`. The hook function is then called by `dsmos_page_transform`, which is set up in the Apple Protect pager's `.page_decrypt` by `unprotect_dsmos_segment()` when the `SG_PROTECTED_VERSION_1` flag is detected in load_segment(). Without the hook, decryption cannot be performed, and the Mach-O cannot be loaded. This, however, is commonly not a problem, as the kernel extension is commonly loaded before the UI (with `dsmos_page_transform_hook` announcing "DSMOS has arrived"), and the boot process can adequately delay until that happens (with `dsmos_page_transform()` `printf()`ing out "Waiting for DSMOS...").

For lack of any hardware backed cryptography, however, the best that Apple could do over time is to hide the encryption key as deep as it could, hoping intrepid reversers couldn't find it. The logical place to bury the key was deep inside the SMC BIOS (which is why DSMOS depends on the AppleSMC.kext). That, too, would fail in the face of the determined reverser, which is why Apple had naught but to try and appeal to their more compassionate side. In the past, the "decryption key" included a poem (which could be found as plaintext just shy of the end of the 32-bit address space):

Listing 11-17-a: The original (32-bit architecture) DSMOS decryption "key"

```
Your karma check for today:
There once was was a user that whined
his existing OS was so blind,
he'd do better to pirate
an OS that ran great
but found his hardware declined.
Please don't steal Mac OS!
Really, that's way uncool.
(C) Apple Computer, Inc.
```

Later, this changed to a short Haiku, of which a hash is taken:

Listing 11-17-b: The more modern (64-bit architecture) DSMOS decryption "key"

```
ourhardworkbythesewordsguardedpleasedontsteal(c)AppleComputerInc
```

More than the appealing hard-coded strings, however, it turns out that encrypting the binaries in this way allows Apple another avenue in their efforts against the blasphemous Hackintosh heathens: The DMCA. These strings are copyrighted, and Apple has invoked this in the past in a lawsuit and won in at least one case, but lost an attempt to seal it[6]. With the new MacBooks of 2018 and later integrating the T2 chip and supplying hardware enforced cryptographic functions as the *OS variants do (see next), it is likely a matter of time before Apple enforces binary encryption in this stronger manner, far less prone to modders' good will.

*OS: Fairplay encryption

Apple controls the platform in *OS architectures from the board up, which is why it can enforce much stronger encryption than MacOS does with the optimistic DSMOS. Both mechanisms naturally make use of the Apple Protect pager, but this is where similarities end.

*OS variants uses a special kext, `IOTextEncryptionFamily.kext`. This kext registers two `IOPersonalities` - `AppleFairplayTextCrypter` and `AppleNullTextCrypter`. When the kext's `IOTextCrypter::start` is called, `text_crypter_create_hook_set()` installs its function, `IOTextCrypter::createSession(pager_crypt_info*, char const*, void*);` as the implementation of `text_crypter_create`.

Decryption occurs when `parse_machfile` encounters `LC_ENCRYPTION_INFO[64]`, or (in Darwin 16 and later) when user mode calls the `mremap_encrypted()` system call (#489). Looking at the former case, a call to `set_code_unprotect()` follows. The load command's cryptid is checked. A cryptid of 0 returns immediately (implying no encryption), but a cryptid of 1 is identified as the `com.apple.unfree` encryption. A `crypt_file_data` structure is constructed, to provide the details of the filename and architecture, and then passed into `text_crypter_create` along with a `crypt_info` to populate. Once the `crypt_info` is thus initialized, a call to `vm_map_apple_protected` (from osfmk/vm/vm_map.c) takes the `crypt_info` and calls `apple_protect_pager_setup()` to initialize the pager, and then `vm_map_enter_mem_object()` to remap the encrypted pages, transparently decrypted through `AppleFairplayTextCrypter::decryptPage`.

Listing 11-18: `set_code_unprotect` (from bsd/kern/mach_loader.c)

```
#if CONFIG_CODE_DECRYPTION
static load_return_t set_code_unprotect( ...)
        ...
        /* set up decrypter first */
        crypt_file_data_t crypt_data = {
                .filename = vpath,
                .cputype = cputype,
                .cpusubtype = cpusubtype};
        kr=text_crypter_create(&crypt_info, cryptname, (void*)&crypt_data);
 ....
remap_now:
        /* now remap using the decrypter */
        MACHO_PRINTF(("+++ set_code_unprotect: vm[0x%llx:0x%llx]\n",
                      (uint64_t) map_offset,
                      (uint64_t) (map_offset+map_size)));

        kr = vm_map_apple_protected(map,
                                    map_offset,
                                    map_offset+map_size,
                                    crypto_backing_offset,
                                    &crypt_info);
..
}
```

This, however, is not where the story ends. The `IOTextEncryptionFamily.kext` calls on `FairPlayIOKit.kext`, which handles the decryption using Apple's powerful FairPlay infrastructure. This involves an upcall (via `host_get_special_port`) to the `HOST_UNFREED_PORT` (#17), which is entrusted to `/usr/sbin/fairplayd.##` (where ## is related to the CPU model)*. In this way, Apple reinvents FairPlay - a DRM technology originally developed for iTunes music tracks. Those who didn't want DRM in their media, eventually got it in their applications.

Both forms of encryption - DSMOS and FairPlay - can still be readily defeated by directly accessing the task `vm_map` and dumping its memory, as can be done from inside the process (by dylib injection) or from outside it (by obtaining a `SEND` right to the task port). Tools such as `procexp` *`pid`* `binary` will dump the decrypted Mach-O back to a file.

* - It's worth noting that FairPlay components are the only ones which Apple obfuscates heavily in *OS.

Page Lists (UPLs)

Pagers operate on Universal Page Lists (UPLs). A UPL represents a set of pages which are to be populated from or commited to the backing store. UPLs control the attributes of these pages (WIMG, permissions, etc), and are also used to populate or sync the page contents.

Listing 11-19: The UPL structure (from XNU 4903's osfmk/vm/vm_pageout.h)

```
struct upl {
        decl_lck_mtx_data(,     Lock)   /* Synchronization */
        int             ref_count;
        int             ext_ref_count;
        int             flags;
        vm_object_offset_t offset;
        upl_size_t      size;       /* size in bytes of the address space */
        vm_offset_t     kaddr;      /* secondary mapping in kernel */
        vm_object_t     map_object;
        ppnum_t         highest_page;
        void*           vector_upl;
        upl_t           associated_upl;
        struct upl_io_completion *upl_iodone;   // Callback on completion
#if CONFIG_IOSCHED // TRUE across Darwin flavors
        int             upl_priority;
        uint64_t        *upl_reprio_info;
        void            *decmp_io_upl;
#endif
#if CONFIG_IOSCHED || UPL_DEBUG  // (TRUE || FALSE)
        thread_t        upl_creator;
        queue_chain_t   uplq;       /* List of outstanding upls on an obj */
#endif
#if     UPL_DEBUG
        /* debugging fields, not found outside development builds */
        ...
#endif
}
```

UPLs can be of several types:

- External UPLs: are tied to a `map_object`, at a certain `offset`. This is the default type of UPLs (i.e. the `UPL_CREATE_EXTERNAL` flag is 0)
- Internal UPLs: flagged by `UPL_INTERNAL`, contain the page information in an adjacent buffer (right after the `struct upl` ends), as an array of `upl_page_info` structures (representing up to `MAX_UPL_SIZE_BYTES`, presently 64MB, divided by `PAGE_SIZE`).
- I/O UPLs: simply referred to as `iopls`, these are created by `memory..`, `kernel..` or `vm_object_iopl_request()`. These can be external or internal, but are "lite" UPLs, in the sense that they are followed by a `wpl_array_t` "lite list" of pages.
- Vector UPLs: are not `struct upls`, but `struct _vector_upls`, whose UPLs are maintained in a `upl_elems` array (of `MAX_VECTOR_UPL_ELEMENTS` = 8 elements), along with a corresponding array of `upl_iostates`, recording each UPL's `offset` and `size`. These UPLs are used exclusively by the VFS layer's `cluster_[read|write]_direct`.

Creating UPLs

The only supported KPI to create a UPL is `ubc_create_upl()`. In the kernel proper, however, there are quite a few ways to create a UPL. Commonly, UPLs are created through the use of `ubc_create_upl_external` (from the KPI), or `...kernel`, as they back a vnode's pages. Other layers have their own interfaces: VM map handling can call `vm_map_create_upl()`. VM object handling can call `vm_object_[upl/iopl]_request`. Pagers can call `memory_object_[upl/iopl]_request`. There are also `[vm/memory]_object_super_upl_request` variants. Another use case is when created through IOKit's `IOMemoryDescriptor` object (q.v. Chapter 13). There is also `vector_upl_create()`, which is used by VFS's `cluster_[write/read]_direct`.

As with the funnelling of `vm_map_enter`, here, too, `upl_create()` is called. The upls are `kalloc()`ed in the zone nearest to their structure size, and (if `UPL_INTERNAL`), the additional `upl_page_info` array. Two `CONFIG_*` settings in XNU's build can affect the structure and handling of UPLs. The first, `UPL_DEBUG`, adds tracking metadata to UPLs, including full stack traces. This greatly increases the size of the UPL, and is `FALSE` outside development kernels. The second is `CONFIG_IOSCHED`, a `TRUE` setting introduced in Darwin 14, which allows limited I/O tracking, and prioritization (expediting) of UPLs.

The `upl_create()` routine also accepts `UPL_CREATE_*` flags, which in turn lead to `UPL_*` flags in the UPL object itself. Because it is seldom called directly, a more common way of affecting the UPL's flags is through control flags (also unfortunately `UPL_*` prefixed, and thus confusing) in osfmk/mach/memory_object_types.h.

Listing 11-20: The UPL flags (from XNU 4903's osfmk/vm/vm_pageout.h)

```
#define UPL_PAGE_LIST_MAPPED    0x1     /* Mapped by vm_map_enter_upl */
#define UPL_KERNEL_MAPPED       0x2     /* Mapped by kernel (unused?) */
#define UPL_CLEAR_DIRTY         0x4     /* from UPL_CLEAN_IN_PLACE */
#define UPL_COMPOSITE_LIST      0x8     /* unused */
#define UPL_INTERNAL            0x10    /* Internal (Followed by upl_page_infos) */
#define UPL_PAGE_SYNC_DONE      0x20    /* from UPL_COPYOUT_FROM */
#define UPL_DEVICE_MEMORY       0x40    /* Device I/O page       */
#define UPL_PAGEOUT             0x80    /* from UPL_FOR_PAGEOUT */
#define UPL_LITE                0x100   /* from UPL_CREATE_LITE */
#define UPL_IO_WIRE             0x200   /* from UPL_SET_IO_WIRE */
#define UPL_ACCESS_BLOCKED      0x400   /* from  UPL_BLOCK_ACCESS - commit or abort first */
#define UPL_SHADOWED            0x1000  /* UPL's map_object has a shadow */
#define UPL_KERNEL_OBJECT       0x2000  /* UPL's map_object is kernel_object */
#define UPL_VECTOR              0x4000  /* vector_upl_create() */
#define UPL_SET_DIRTY           0x8000  /* from UPL_REQUEST_SET_DIRTY */
#define UPL_HAS_BUSY            0x10000 /* at least one page in UPL is marked vmp_busy */
#define UPL_TRACKED_BY_OBJECT   0x20000 /* from UPL_CREATE_IO_TRACKING, if CONFIG_IOSCHED */
#define UPL_EXPEDITE_SUPPORTED  0x40000 /* from UPL_CREATE_EXPEDITE_SUP, if CONFIG_IOSCHED */
#define UPL_DECMP_REQ           0x80000 /* Pages reserved for use by decmps */
#define UPL_DECMP_REAL_IO       0x100000/* Actual pages used by decmpfs */
```

Handling UPLs

XNU provides the osfmk/mach/upl.defs MIG subsystem (#2050), another vestige of its microkernel days, wherein UPLs were handled by external memory managers and could be passed as `IKOT_UPL` port types. The `vm_map` subsystem was even supposed to provide `vm_map_get_upl()`, and `vm_upl_[map/unmap]`. The `vm_map` routines, however, are presently `skip`ped, and the `upl` subsystem is not included in the user mode headers (i.e. there is no <mach/upl.defs> export), and so the subsystem remains unused from user space).

Internally, however, the `upl_*` routines are still called, albeit mostly directly from the various pager implementations. Other cases are calls made through the UBC (q.v. Chapter 7), which exports (as KPIs) the routines shown in Table 11-21:

Table 11-21: The `ubc_upl_*` KPIs (from /bsd/sys/ubc.h)

UBC UPL KPI	Purpose
`ubc_create_upl`	Create a new UPL for vnode, populating its `vm_object`. Removed in Darwin 17
`ubc_create_upl_external`	As above, but meant for external (kext) callers
`ubc_create_upl_kernel`	As above, but meant for in kernel use
`ubc_upl_abort[_range]`	Abort (discard pages in) entire UPL or subrange
`ubc_upl_commit[_range]`	Commit (flush pages to backing store) entire UPL or subrange
`ubc_upl_{un/}map`	Map/unmap associated page list into/from kernel virtual address space
`ubc_upl_maxbufsize`	Return the maximum bufsize `ubc_create_upl()` allows (`MAX_UPL_SIZE_BYTES`))
`ubc_upl_pageinfo`	Retrieve internal page lists of UPLs marked `UPL_INTERNAL`
`ubc_upl_range_needed`	Specify page range needed

UPLs generally end their life when they are either committed or aborted. Committing means flushing the pages back to their backing store (through the `vm_pageout` daemon code), and possibly freeing them. Aborting does not sync, and deactivates the pages. To avoid holding the `vm_page_queue_lock`, both routines fill a `dw_array` of `struct vm_delayed_page_work` for every page in the UPL, then call `vm_page_do_delayed_work()` (in osfmk/vm/vm_resident.c) with the array, indicating the `dw_mask` bits for the operations each page requires. The `vm_page_do_delayed_work()` routine then calls the corresponding `vm_page_*` routines, as indicated by the `DW_*` constant names. These are shown in Listing 11-22:

Listing 11-22: The `DW_*` delayed work operations, from XNU 4903's osfmk/vm/vm_page.h

```
#define DW_vm_page_unwire               0x01
#define DW_vm_page_wire                 0x02
#define DW_vm_page_free                 0x04
#define DW_vm_page_activate             0x08
#define DW_vm_page_deactivate_internal  0x10
#define DW_vm_page_speculate            0x20
#define DW_vm_page_lru                  0x40
#define DW_vm_pageout_throttle_up       0x80
#define DW_PAGE_WAKEUP                  0x100
#define DW_clear_busy                   0x200
#define DW_clear_reference              0x400
#define DW_set_reference                0x800
#define DW_move_page                    0x1000
#define DW_VM_PAGE_QUEUES_REMOVE        0x2000
#define DW_enqueue_cleaned              0x4000
#define DW_vm_phantom_cache_update      0x8000

struct vm_page_delayed_work {
        vm_page_t       dw_m;
        int             dw_mask;
};
```

When the UPL is finally empty, it can be deallocated, and destroyed (if its reference count is zero). Before `upl_destroy()`ing it, (which `kfree()`s it), `upl_deallocate()` optionally calls `upl_callout_iodone()` on UPLs in which `upl_iodone` is not NULL. This structure pointer provides a callback and error code, settable by a call to `upl_set_iodone[_error]()`. Presently the only user of this asynchronous mechanism is the `vm_swapout_thread()`, which uses it with an array of `vm_swapout_ctx` context structures. On every swap operation, one such context structure is submitted, through `vm_swapfile_io()`. The callback in all cases is `vm_swapout_iodone()`, which marks the context as done and wakes up the `vm_swap_output_thread()`, to then call `vm_swapout_complete_soc()` and call the `vm_swap[out/_put]_finish()` handlers.

The pmap Layer

The vm_map layer provides a set of constructs to manage virtual memory, but sooner or later this virtual memory must be backed by physical pages. This is where the **pmap** layer comes in. It is through the pmap objects that XNU can manage the physical page mappings of a virtual memory space.

Unlike the vm_map layer, pmap has an architecture-specific component to it. The physical mappings of virtual pages has to be coordinated by the Memory Management Unit (MMU), and different architectures have different MMUs and page table structures. The implementations are, therefore, in osfmk/[x86_64/arm]/pmap.c (and, for the Intel architecture, with some functions in osfmk/i386/pmap_common.c). The beauty of the abstraction, however, is that the interface - though also seemingly architecture specific - is largely the same in its machine dependent part, (in osfmk/[i386/arm]/pmap.h), and entirely identical in osfmk/vm/pmap.h. The pmap object is left opaque (as a pmap_t), and pages are represented by their Physical Page Numbers (a ppnum_t, i.e. a uint32_t). Physical addresses are as per the underlying CPU's bitness (pmap_paddr_t, nowadays uint64_t).

Table 11-23: The pmap layer interfaces

function	Purpose
pmap_create(*ledger_t, size,64bit*);	Create a new pmap
pmap_enter[_options](*pmap_t, v,pn, prot,fault,flags,wired[,options, arg]*);	Establish a mapping between virtual address *v* and physical page *pn* with specified *prot* in target *pmap_t*, optionally specifying *fault_type* being serviced and whether page is to be *wired*
pmap_[un]lock_phys_page(*pn*);	Un/Lock page in physical memory (prevent swapping)
pmap_[copy/zero]_[part_]page(...);	Physical bcopyphys or bzero() of page, or part thereof
pmap_remove(*pmap_t, start,end*);	Remove virtual address mappings between *start* and *end*
pmap_disconnect[_options] (*pn,[options,arg]*);	Disconnect page *pn*'s mappings (setting its protections to NONE), and return its reference flags, if set.
pmap_[reference/destroy](*pmap_t*);	Increment/Decrement pmap reference count (possibly freeing it)
pmap_page_protect[_options](*pn,prot,[opts,arg]*);	Set protection bits of page *pn* to *prot* (across all mappings)
pmap_map_block(*pmap_t, sva,size, prot, attr,flags*);	Map a block of *size* starting at *sva* with *prot* and *attr*
pmap_protect(*pmap_t, sva,eva, prot*);	Apply VM_PROT_* flags in *prot* virtual addresses from *sva* to *eva*
pmap_find_phys(*pmap_t,va*);	Find physical page address for virtual address *va* in *pmap*
pmap_switch(*pmap_t*);	Switch to a new address space (during context switch)
pmap_clear_[modify/reference](*pn*);	Clear modified/referenced bit of page
pmap_is_[modified/referenced](*pn*);	Retrieve modified/referenced bit of page
phys_attribute_set(*pn,bits*); **phys_attribute_clear**(*pn,bits,opts,arg*)	Set/clear PTE level *bits* (PHYS_*/PP_ATTR*) of page *pn*

The pmaps are commonly constructed with a call to pmap_create. An exception to the rule is the kernel_pmap, which represents the physical addresses backing the kernel space. The kernel_pmap is created in pmap_bootstrap() (called from [i386/arm]_vm_init). This is a global, also accessible through pmap_kernel(void). Allocations in the kernel_pmap can be made with pmap_enter, although pmap_map (which wraps it) is often used instead.

The actual operations of the pmap layer require working with the MMU's page tables, which are extremely processor-dependent and subject to ever idiosyncratic nuances as MMUs evolve to meet the needs and demands of enhanced memory protections. The pmap clients, however, (notably, the vm_map layer and IOKit), can remain blissfully ignorant of the underlying implementation. Another extenuating circumstance is that, for the most part, the kernel rarely requires physical page numbers, as both Intel and ARM architectures work in an all-virtual space. Outside kernel hacking (primarily, run-time patching), usage of pmap APIs is quite rare. The following experiment, however, builds on the sample character device driver started in Chapter 6, and demonstrates a clever use of at least one pmap KPI.

Experiment: Recreating /dev/kmem

Older releases of UN*X provided a /dev/kmem as a virtual character device providing direct access to kernel memory from user mode. This obviously insecure device was deprecated and is no longer provided by default, although in XNU it may be reenabled by passing kmem=1 as a boot argument to the kernel, if CONFIG_DEV_KMEM is set during compile time.

This book's precursor implemented /dev/kmem through a kernel extension[7], and such an implementation certainly makes sense - both as a case study, and for its great usefulness. With minor modifications, the implementation can be made usable in present day MacOS as well. We've already seen the core - character device - portion of it in a previous experiment (see Chapter 6). It doesn't take much to take the character device I/O request and translating its "offset" into a memory address in kernel space.

A big challenge, however, is returning **valid** addresses. An incorrect pointer dereference in kernel mode will result in an immediate panic. A correct implementation would therefore need to make sure the user supplied address ("offset" into the device) is a valid one. One way of ensuring this would be to take the code of XNU's existing kmem implementation. Perusing bsd/dev/mem.c reveals the sanity check is performed by `kernacc(start,len)`, which in turn calls `kvtophys()` on every address in the range, page size apart.

Listing 11-24: Physical memory sanity check performed by `kernacc()` (bsd/dev/mem.c)

```
boolean_t
kernacc(
    off_t       start,
    size_t      len
)
{
        off_t base;
        off_t end;

        base = trunc_page(start);
        end = start + len;

        while (base < end) {
          if(kvtophys((vm_offset_t)base) == 0ULL)
                        return(FALSE);
                base += page_size;
        }

        return (TRUE);
}
```

If you look at the implementation of `kvtophys()`, you'll find that it calls `pmap_find_phys()` (q.v. Table 11-23), which is a KPI, albeit an Unsupported one. If either of these return a non-zero/NULL value, the address can be considered valid, and can be provided to user memory using a `uiomove` operation.

It should now be straighforward to adapt the code of the simple character device (from Chapter 6), replacing the buffer data returned from the clipbboard-style device so that it returns any valid address in kernel space - thereby recreating /dev/kmem. One caveat, introduced in Darwin 18, is a check made in `uiomove` to panic if a length more than the respective zone element size is read. This, however, can be circumvented easily. As a further exercise, you might want to adapt your driver to also provide /dev/mem - granting direct access to physical memory from user space. You can find a full solution for both memory devices on the book's companion website.

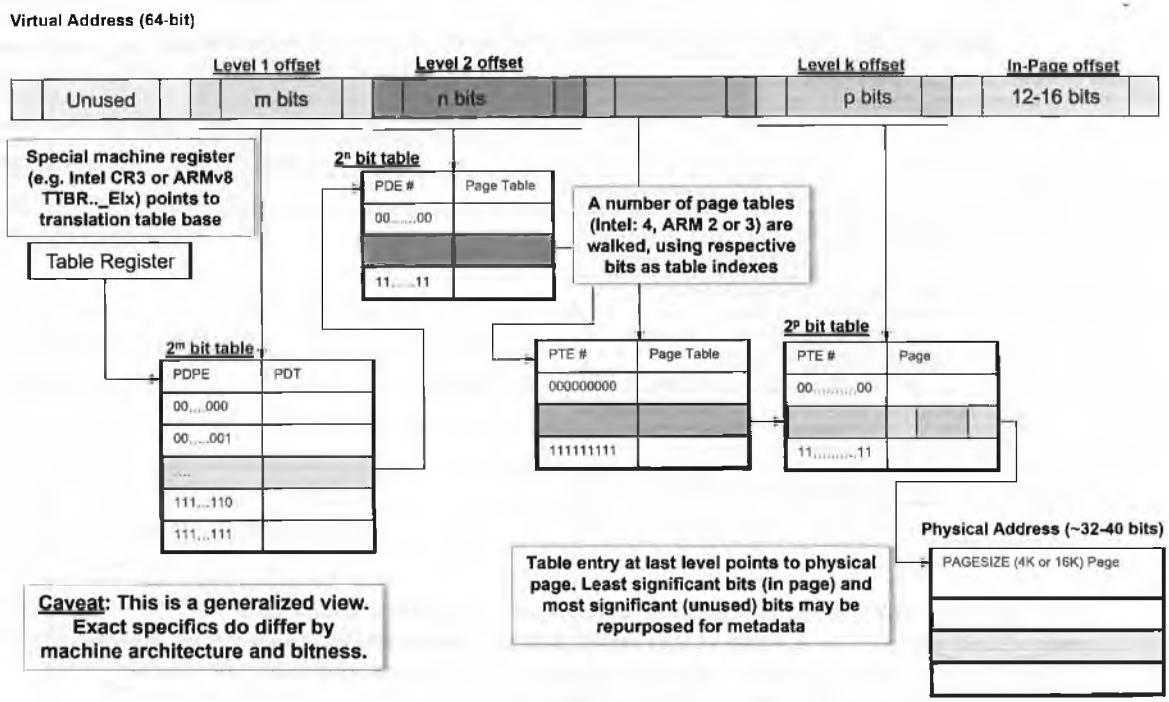
Page Tables

A key concept in memory management are the **page tables**, which are used to maintain the mappings between virtual memory addresses and physical ones. Once virtual memory is enabled, all access to physical memory must be performed via the MMU, and even the kernel cannot directly access physical memory. The kernel can, however, create modify or remove memory mappings by editing the page tables.

In Intel, the special `CR3` register holds the base of the page tables for a given process. In ARM architectures, the **Translation Table Base Register**s (TTBRs) are used instead. ARM64 provides a different TTBR for every exception level, so `TTBR_EL0` is employed by user mode, and `TTBR_EL1` by the kernel.

Tables are multi-level, and traversing them eventually results in a Page Table entry (PTE), which provides the virtual to physical mapping, as well as page-specific attributes, or flags. Both ARM and Intel architectures use `pmap_pte(pmap,va)` to retrieve the machine specific PTE, but this is only used internally and not exported as a pmap KPI. The page table and PTE format understandably varies between architectures, but Figure 11-25 nonetheless tries to depict a high-level, and generalized view:

Figure 11-25: A high level generalized view of MMU page tables



WIMG

Before moving onto the specific architecture details, an important abstraction to consider is that of **WIMG**. The acronym dates back to the PowerPC days, wherein it stood for four memory caching attributes: **W**rite-through, Cache-**I**nhibition, Memory **C**oherence and **G**uarded writes. The `vm_map` layer `#defines` `VM_MEM` constants which map to the four attributes:

Table 11-26: The WIMG architecture-independent attributes

	VM_MEM constant	Attribute
G	VM_MEM_GUARDED	Writes must be ordered by processor (no speculative)
M	VM_MEM_COHERENT	Hardware memory coherence
I	VM_MEM_NOT_CACHEABLE	Neither reads nor writes may be cached
W	VM_MEM_WRITE_THROUGH	If set, writes must not be cached
-	VM_MEM_[INNER|RT]	ARM: Inner shareable domains
-	VM_MEM_EARLY_ACK	ARM: Early acknowledgment from interconnect. (Unused)

The `VM_MEM_*` bitmask consists of the four least significant bits, though in reverse order (GMIW), and as demonstrated by the table the ARM architecture extends the WIMG bits by two more (0x10 and 0x20, respectively). The `vm_map` layer also translates `MAP_MEM_*` to same-suffixed WIMG constants using `vm_prot_to_wimg()`, an inlined function defined in osfmk/vm/vm_map.h. `VM_WIMG_*` macros for bit combinations of WIMG are defined in the architecture-dependent osfmk/*[i386|arm]*/pmap.h, passed as *flags* to some of the `pmap_*` functions, and map to the lower level page table entry bits specific to that architecture. These constants are also available for use with IOKit's `IOMemoryDescriptor`, as `kIO[Map]..` constants. Table 11-27 shows the combinations presently supported.

Table 11-27: WIMG attribute combinations used by XNU

VM_WIMG_* #define	VM_MEM_* bitwise OR
VM_WIMG_USE_DEFAULT	0x80
VM_WIMG_MASK	0xFF
VM_WIMG_COPYBACK	(..._COHERENT)
VM_WIMG_IO VM_WIMG_POSTED (Intel)	(.._COHERENT | .._NOT_CACHEABLE | .._GUARDED)
VM_WIMG_POSTED (ARM)	(.._COHERENT | ..NOT_CACHEABLE | .._GUARDED | .._EARLY_ACK)
VM_WIMG_WTHRU	(..._COHERENT | ..._WRITE_THROUGH | ..._GUARDED)
VM_WIMG_WCOMB	(..._COHERENT | ..._NOT_CACHEABLE)
VM_WIMG_RT (ARM)	(VM_WIMG_IO | ..._RT)
VM_WIMG_INNERWBACK (ARM)	(.._COHERENT | ..._INNER)

I/O Mappings

A very specific type of physical memory mappings is **I/O mappings** - memory mapped between the kernel and some device, enabling the two to communicate directly. This functionality is provide through XNU's machine layer `ml_io_map()`. This is a thin wrapper, implemented (in osfmk/[i386/arm[64]]/machine_routines.c) over `io_map`, passing it the physical address and size, along with `VM_WIMG_IO`. ARM architectures also provide an `ml_io_map_wcomb` wrapper, which passes the `VM_WING_WCOMB` flag instead.

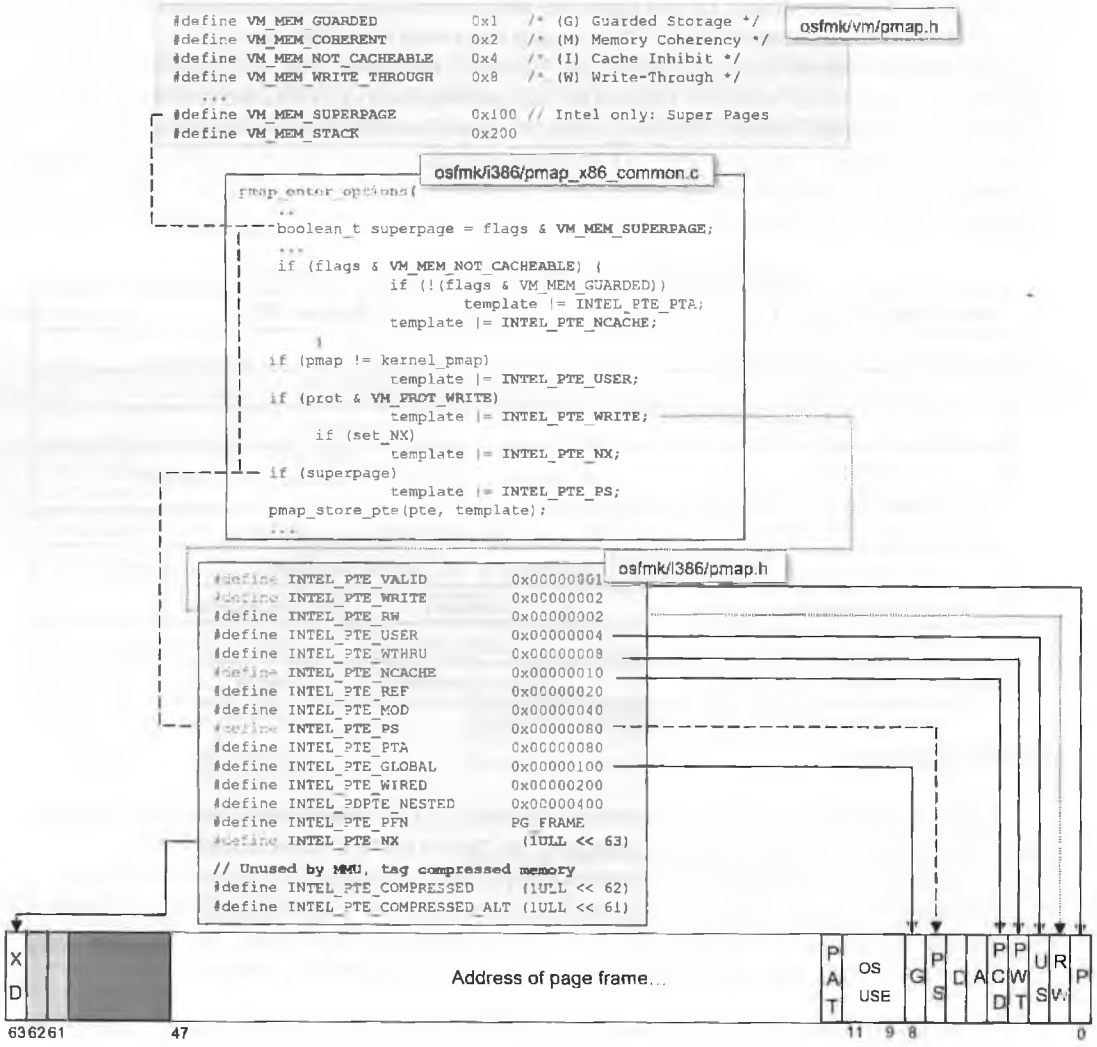
`io_map` then calls either `pmap_map_bd[_with_options]` or `pmap_map()` to establish the physical mapping and return the virtual address. These functions allocate the physical memory from the `kernel_pmap`. The `..bd..` variants are a "backdoor" which is used for early allocations, that is when the `kernel_map` (i.e. the kernel's `vm_map`) has yet to be initialized. Because the memory is added to the `kernel_map`, `kmem_free` is used to free the memory mapping.

Examples of I/O mappings can be seen in primarily in ARM's Platform Expert, mapping serial device memory (`serial_init`). ARM64's osfmk/arm64/cpu.c also uses `ml_io_map` for mapping CoreSight registers, which are commented as debug only, but still present in release kernelcaches. Another interesting example is in the mapping of AMCC memory in `rorgn_stash_range()` (osfmk/arm64/machine_routines.c).

Intel PTEs

Intel architectures use a four level page table, with nine bits per table index, effectively further extending the 36-bit "Physical Address Extension" (PAE) model into a 48-bit model. This leaves the 16 most significant bits of every address unused. Remember, however, that pages in the table entries must be aligned on a page boundary - effectively leaving the 12 least significant bits unused as well. These extra bits are repurposed at the MMU level to support memory attributes and protection features, and the Intel pmap translates the architecture independent `VM_MEM*` flags to those bits inline, as shown in Figure 11-28:

Figure 11-28: The Intel PTE attribute bits



The `pmap`'s `pm_cr3` holds the Intel CR3 value, which traditionally pointed to the page tables of both user space and kernel space, as the latter's page tables were shared across all processes. Midway into Darwin 17, following the Spectre and Meltdown vulnerabilities, the Intel `struct pmap` (in osfmk/i386/pmap.h) had to be modified so as to mitigate those issues, and so later versions of XNU 4570 and onwards add two fields, `pm_ucr3` and `pm_upml4`. Three `vm_object_t`s hold the PDE, PDPT, and PML4 pages. There is also a pmap `ledger` for physical page accounting. The rest of the structure is mostly taken up by a `pmap_statistics` structure, and two vectors, `pmap_pcid_cpus` and `pmap_pcid_coherency_vector`. The first is an array of `PMAP_PCID_MAX_CPUS` (=64) 16-bit PCID (Process Context IDentifier) entries. The second is a corresponding array of 8-bit entries. PCID handling is well detailed in osfmk/x86_64/pmap_pcid.c. The overall size of the MacOS `pmap` to 368 bytes in Darwin 18.

The ARM PTEs

As is with the Intel case, ARM page table entries do not use all 64-bits. Here, too, the lower bits (0-$\log_2$(pagesize)) and the high bits (48-63) are defined as page attribute bits. A detailed discussion of ARM memory attributes can be found in the ARM documentation[8], but osfmk/arm64/proc_reg.h contains ASCII diagrams and `ARM_PTE_*` `#defines` of the bits. While well detailed, note that Apple redefines bits (particularly in the A12, wherein some of the Page Based Hardware Attribute (PBHA) bits get repurposed for PPL, discussed in Volume III and in detailed articles by the author and by @S1guza[s1g]) and those definitions get stripped due to `#ifdefs` which are pruned from the open source. Figure 11-29 shows the mappings of `VM_WIMG*` constants to the machine PTE attribute bits:

Figure 11-29: The ARM64 PTE attribute bits (for a level 3 mapping)

osfmk/arm/pmap.h

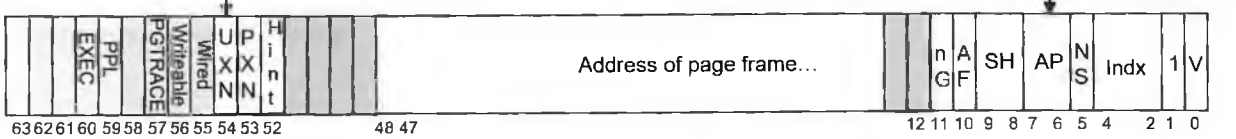
```
#define VM_MEM_INNER            0x10
#define VM_MEM_RT               0x10 // intermediate alias VM_MEM_INNER; ...
#define VM_MEM_EARLY_ACK        0x20
#define VM_WIMG_DEFAULT         (VM_MEM_COHERENT)
#define VM_WIMG_COPYBACK        (VM_MEM_COHERENT)
#define VM_WIMG_INNERWBACK      (VM_MEM_COHERENT | VM_MEM_INNER)
#define VM_WIMG_IO              (VM_MEM_COHERENT | VM_MEM_NOT_CACHEABLE | VM_MEM_GUARDED)
#define VM_WIMG_POSTED          (VM_MEM_COHERENT | VM_MEM_NOT_CACHEABLE | VM_MEM_GUARDED | VM_MEM_EARLY_ACK)
#define VM_WIMG_WTHRU           (VM_MEM_WRITE_THROUGH | VM_MEM_COHERENT | VM_MEM_GUARDED)
#define VM_WIMG_WCOMB           (VM_MEM_NOT_CACHEABLE | VM_MEM_COHERENT)
#define VM_WIMG_RT              (VM_WIMG_IO | VM_MEM_RT)
```

osfmk/arm/pmap.c

```
wimg_to_pte(unsigned int wimg){
    pt_entry_t pte;

    switch (wimg & (VM_WIMG_MASK)) {
        case VM_WIMG_IO:
        case VM_WIMG_RT:
            pte = ARM_PTE_ATTRINDX(CACHE_ATTRINDX_DISABLE);
            pte |= ARM_PTE_NX | ARM_PTE_PNX;
            break;
        case VM_WIMG_POSTED:
            pte = ARM_PTE_ATTRINDX(CACHE_ATTRINDX_POSTED);
            pte |= ARM_PTE_NX | ARM_PTE_PNX;
            break;
        ...
        default:
            pte = ARM_PTE_ATTRINDX(CACHE_ATTRINDX_DEFAULT);
            pte |= ARM_PTE_SH(SH_OUTER_MEMORY);
    ...
    }
```

osfmk/arm64/proc_reg.h

```
#define ARM_PTE_AP(x)           ((x) << 6)              /* access protections */
#define ARM_PTE_ATTRINDX(x)     ((x) << 2)              /* memory attributes index */
#define ARM_PTE_SHSH(x)         ((x) << 8)              /* access shared */
#define ARM_PTE_AF              0x0000000000000400ULL /* value for access */
#define ARM_PTE_NG              0x0000000000000800ULL /* value for a global mapping */
#define ARM_PTE_NS              0x0000000000000020ULL /* value for a secure mapping */
#define ARM_PTE_HINT            0x0010000000000000ULL /* value for contiguous entries hint */
#define ARM_PTE_PNX             0x0020000000000000ULL /* value for privilege no execute bit */
#define ARM_PTE_NX              0x0040000000000000ULL /* value for no execute bit */
#define ARM_PTE_WIRED           0x0080000000000000ULL /* value for software wired bit */
#define ARM_PTE_WRITEABLE       0x0100000000000000ULL /* value for software writeable bit */
#define ARM_PTE_PTRACE          0x0200000000000000ULL /* value for software trace bit (CONFIG_PTRACE)*/
// These are undocumented since A12 is stripped, and the exact likely differs.
#define ARM_PTE_PPL_PROT        0x0900000000000000ULL /* A12: Page protected by PPL*/
#define ARM_PTE_EXECUTABLE      0x1000000000000000ULL /* A12: Page marked as executable by PPL */
```



ARMv8 uses the `TTBR_EL[0/1]` registers to hold the address space. Additionally, a 16 bit Address Space Identifier (ASID) is used to distinguish pages with the same virtual address, but in different (task-specific) address spaces. The arm `pmap` shares the `lock`, `stats` and `ledger` and `nx_enabled` indicator with its Intel counterpart, but the rest of the structure is quite different, offering support for nested pmaps, a link to the global chain of `pmaps`, and other fields. The structure presently occupies 256 bytes, and its definition can be found in osfmk/arm/pmap.h.

Review Questions

1. Provide an example of when a shared submap's `use_pmap` would be deliberately `FALSE`
2. What is the purpose of the `vm_page_array_zone` (created by `vm_page_module_init()`)?
3. What is the use case behind the oddly named `vm_map_copy_copy()`
4. What is the benefit of Apple's PPL, compared with that of KTRR?

References

1. Draves et all - "Virtual Memory" - http://rsim.cs.illinois.edu/arch/qual_papers/systems/virtualmem.pdf or http://www.cs.cornell.edu/courses/cs614/2003sp/papers/RTY88.pdf
2. Ian Beer (Project Zero) - "Splitting Atoms in XNU" - https://googleprojectzero.blogspot.com/2019/04/splitting-atoms-in-xnu.html
3. Ian Beer - "vm_map'ing out XNU virtual memory" - https://objectivebythesea.com/v2/talks/OBTS_v2_Beer.pdf
4. Richard & Case (Digital Investigation) - http://old.dfrws.org/2014/proceedings/DFRWS2014-1.pdf
5. Wilson & Kaplan - WKdm - https://www.usenix.org/legacy/publications/library/proceedings/usenix01/cfp/wilson/wilson_html/acc.html
6. https://www.rcfp.org/sites/default/files/docs/20120105_202426_apple_sealing.pdf
7. Amit Singh - /dev/kmem implementation - http://www.osxbook.com/book/bonus/chapter8/kma/
8. ARM Manuals - Memory Management - https://static.docs.arm.com/100940/0100/armv8_a_address%20translation_100940_0100_en.pdf

12

Ceci n'est pas une "heap"*: Kernel Memory Management

Thankfully, kernel KPI users seldom have to deal with the physical layer, and most calls can remain in the virtual layer. We detail how the kernel manages its own `vm_map` - the `kernel_map` - through `kmem_alloc*` and `kalloc*`. We next present the very important abstractions of kernel zones. Special care is given to the nooks and crannies of the zone allocator, due to their significance over the years in exploitation.

Before concluding, we reintroduce memory pressure - also presented in I/8, but described here from the kernel's perspective. The gentle `memorystatus` of MacOS and the ruthless `jetsam` of *OS are both detailed. This continues with a discussion of XNU's proprietary purgeable memory, which helps deal with memory pressure automatically. Lastly, we conclude with a rough map of the kernel's address space, and consider the kernel slide.

* - Kernel memory is often incorrectly refered to as the the "kernel heap". This term (apparently used within Apple as well) is technically wrong - the heap refers to data strcture used in user-mode in backing `malloc(3)`, as opposed to the automatic, thread-local memory of the stack (although the pedantic will correctly argue that user mode "heaps" no longer use the heap structure as well...). Though the kernel does make use of stack memory (for its threads and user-mode threads when in-kernel), the heap data structure is not used in any way in maintaining kernel memory, which is merely segmented into zones.

Kernel Memory Allocation

XNU makes use of multiple memory allocation facilities in the kernel. We have already touched on those of BSD earlier in Chapter 6 ("BSD Memory Zones"), and we will discuss those of IOKit in Chapter 13. Let us now consider then, those of Mach.

The `kernel_map`

The `kernel_map` is the `vm_map` representing the kernel's addressable virtual memory space. It is set up in `kmem_init()` (osfmk/vm/vm_kern.c, called from `vm_mem_bootstrap()`). The map is then `vm_map_create()`d to span `VM_MIN_KERNEL_AND_KEXT_ADDRESS` to `VM_MAX_KERNEL_ADDRESS`. On MacOS, the range spanned is 0xffffff7f80000000-0xfffffffffffe000, and in *OS it is 0xffffffe000000000-0xffffffff3fffffff. These ranges allow 513GB and "merely" 79GB (respectively), although the map naturally contains wide holes.

All subsequent kernel memory allocations inevitably end up as allocations inside the `kernel_map`. This is either by allocating directly in it (using a `kmem_alloc*()` variant), or carving out a sub map (`kmem_suballoc()`. With few notable exceptions (namely, `kmem_alloc_pageable[_external]()`), all memory in the `kernel_map` is wired - i.e. resident in physical memory.

`kmem_alloc()` and friends

This is where the `kmem_alloc*()` family of functions comes into play. The various functions in the family (with the exception of `kmem_alloc_pages()`) all funnel to `kernel_memory_allocate()`, although with different parameter and flag settings. `kmem_alloc_pages()` is different, in that it allocates virtual pages using `vm_page_alloc()`, but does not commit any of them.

`kernel_memory_allocate()`

The `kernel_memory_allocate()` function is thus a common handler for nearly all memory allocations. It proceeds as follows:

- Round allocation size to page multiple. The allocation size is also checked to not be zero, or not exceed sane boundaries.
- If guard pages were requested (indicated by `KMA_GUARD_[FIRST|LAST]` flags) they are allocated. This is done by a call to `vm_page_grab_guard()`, which sets page protections but only maps them fictitiously.
- If actual wired memory was requested (indicated by the absence of `KMA_VAONLY` and/or `KMA_PAGEABLE`, it is allocated using `vm_page_grab[lo]()`. Low memory is only grabbed if the `KMA_LOMEM` flag was specified.
- If the `KMA_KOBJECT` flag was specified, the `kernel_object` is referenced. If `KMA_COMPRESOR` was specified, the `compressor_object` is referenced. Otherwise, a new memory object is created using a call to `vm_object_allocate()`.
- `vm_map_find_space()` is called to find a suitable map address. This populates both the `map_addr` and the `entry`. The `vm_map_entry_t` is then linked to the object and offset.
- Any `KMA_GUARD_FIRST` guard pages are `vm_page_insert()`ed to the map, followed by wired pages (with `vm_page_insert_wired()`). For the wired page, `PMAP_ENTER_OPTIONS` is called, trying a fast path (`PMAP_OPTIONS_NOWAIT`). Should that fail - a slow path with `PMAP_ENTER` is tried, and must succeed. Any `KMA_GUARD_LAST` pages are then appended to the map, again through `vm_page_insert()`.
- For allocations using the `kernel_object` or `compressor_object`, a call to `vm_map_simplify()` coalesces neighboring regions. Otherwise, the memory object is deallocated.

kmem_suballoc()

The kmem_suballoc() function creates and maps a submap inside a parent map (usually the kernel_map). Important suballocations include the kalloc_map (for kalloc.* zones), ipc_kernel[_copy]_map, the g_kext_map (for kext mappings), the zone_map (and zone tag maps, if #VM_MAX_TAG_ZONES), IOKit pageable spaces, and BSD's bufferhdr_map, mb_map, and the bsd_pageable_map.

The flow of kmem_suballoc() is a simple application of vm_map_enter() to reserve the memory in the parent map, followed by vm_map_create() for the submap, and vm_map_submap() to link the two together.

kmem_realloc()

It's quite rare, but there are certain cases wherein a reallocation of existing memory might be required. Examples of this are the ensureCapacity() methods of libkern's OSSerialize and OSData objects, and vm_purgeable_token_add(). For these cases, kmem_realloc() is used.

There are two fine points with kmem_realloc(). The first is, that it does not free the original address range - meaning that it must be explicitly kmem_free()ed. The second, is that it does not bzero() the new memory allocated (if larger than the original allocation), leading to a potential kernel memory disclosure.

kalloc()

The simplest KPI for memory allocation provided by the kernel is that of kalloc variants. The simplest of these macros, defined in osfmk/kern/kalloc.h, is the kernel-mode equivalent of user-mode's malloc(3). Its variants allow for additional options, like specifying a memory tag or controlling whether the operation can block or not. Table 12-1 shows these macros:

Table 12-1: The various kalloc() macros, defined in osfmk/kern/kalloc.h

Macro variant	Provides
kalloc(*size*)	Basic allocation of *size* bytes
kalloc_tag(*size*,itag)	As above, but with memory tag
kalloc_tag_bt(*size*,itag)	As above, but with tag determined from caller
kalloc_noblock*	All the above, guaranteed to return immediately (but may fail)
kallocp*	All the above, taking *size* by reference and updating actual size allocated

The macros all expand to a call to kalloc_canblock(). Similar to the BSD MALLOC macro, they first set up a vm_allocation_site, and then call the underlying function, which takes the size argument by reference, a boolean as to whether or not it may block, and the site by reference. The reason the size is passed by reference is because kalloc_canblock() reports the actual size of the element allocated, which may be larger than the size requested. The need for this is rare, but kallocp* variants exists to allow passing the *size* argument by reference if necessary. libkern's kern_os_malloc() is one example of this feature's usefulness, as are libkern's calls to kallocp_container(), which ensure the returned memory is entirely bzero()ed up to the size allocated, even if more than initially requested.

kalloc()ed memory is automatically tagged as VM_KERN_MEMORY_KALLOC (#13), though a different tag may be specified with the _tag variants of the macro. As of XNU-3248, it's also possible to request an allocation to be tagged according to the backtrace leading up to it. This is done by tagging the allocation site with a special VM_TAG_BT value (0x800), as is done by the kalloc[/p_/noblock]_tag_bt variants. When the tag is retrieved, vm_tag_bt() (in osfmk/vm/vm_resident.c) walks the current thread's kernel stack, attempting to locate the most recent return address and then calls OSKextGetAllocationSiteForCaller. The function searches for the address in the kernel extension accounting data (described in Chapter 3), and retrieves the associated tag from the allocation site.

kalloc.### zones

Behind the scenes, `kalloc_canblock` tries not to block, by providing the allocation from one of several `kalloc.###` "zones". These zones are pre-allocated regions of memory, spanning an integer number of pages, and "carved" into element slots. Each zone uses a fixed element size, as indicated by the zone name (e.g. "kalloc.16" uses 16-byte elements). The zones are set up early during kernel initialization, by a call to `kalloc_init` (osfmk/kern/kalloc.c) from `vm_mem_bootstrap()`. The `kalloc` operation finds the first zone capable of containing the allocation, unless the allocation size exceeds that of the largest zone (presently, 8192 for MacOS, and 32,768 elsewhere), in which case `kmem_alloc_flags()` is used instead. XNU originally provided kalloc zones with sizes at exact powers of two, starting at 1 (2^0) bytes, but over time Apple has established 16 bytes as the smallest possible allocation, and added additional zones (48,80,96,160,192,224,288,368,400,576,768,1152,1280,1664,6144). If a zone can be found to satisfy the allocation, `kalloc_canblock()` calls `zalloc_canblock_tag()`, to perform the allocation from the zone and tag it according to the `vm_allocation_site`.

The Kalloc DLUT

With so many zones, it's important to keep the operation of finding the right one as quick and as efficient as possible. The **Direct LookUp Table** ("DLUT") was introduced in XNU-2050 as a means to quickly locate the right zone for an element.

Listing 12-2: The Direct LookUp Table of `kalloc` (from osfmk/kern/kalloc.c)

```
/*
 * Many kalloc() allocations are for small structures containing a few
 * pointers and longs - the k_zone_dlut[] direct lookup table, indexed by
 * size normalized to the minimum alignment, finds the right zone index
 * for them in one dereference.
 */

#define INDEX_ZDLUT(size)       \
                (((size) + KALLOC_MINALIGN - 1) / KALLOC_MINALIGN)
#define N_K_ZDLUT       (2048 / KALLOC_MINALIGN)
                /* covers sizes [0 .. 2048 - KALLOC_MINALIGN] */
#define MAX_SIZE_ZDLUT  ((N_K_ZDLUT - 1) * KALLOC_MINALIGN)

static int8_t k_zone_dlut[N_K_ZDLUT];   /* table of indices into k_zone[] */

/*
 * If there's no hit in the DLUT, then start searching from k_zindex_start.
 */
static int k_zindex_start;

static zone_t k_zone[MAX_K_ZONE];

...

/*
 * Given an allocation size, return the kalloc zone it belongs to.
 * Direct LookUp Table variant.
 */
static __inline zone_t
get_zone_dlut(vm_size_t size)
{
        long dindex = INDEX_ZDLUT(size);
        int zindex = (int)k_zone_dlut[dindex];
        return (k_zone[zindex]);
}
```

The slow path

As its name implies, there are circumstances in which `kalloc_canblock` does block. If the allocation cannot fit in one of the many `kalloc.*` zones (and `canblock` is TRUE), a slow path is taken instead. A call to `kmem_alloc_flags()` attempts to allocate the requested memory - first from the `kalloc_map`, and then - as a fallback - the `kernel_map`. The `kalloc_map` is a dedicated `vm_map` for large allocations, which is `kmem_suballoc()`ed by `kalloc_init()` to span virtual memory between `kalloc_map_min` and `kalloc_map_max`. The difference between the two (`kalloc_map_size`) is normally set to the 1/32 of the kernel's `sane_size`, or 128MB for 32-bit kernels.

What makes the slow path so slow is that `kmem_alloc_flags()` calls `kernel_memory_allocate()`, which needs to eventually call `vm_page_grab()` in order to obtain physical memory pages, to back the allocation. This operation will block if there are no pages immediately available, and can only be satisfied after page fault handling, which may take thousands of cycles, if not more. The current thread will surely block, making this path unsuitable and unsafe in atomic contexts. `kmem_alloc_flags()` is described later in this chapter.

OSMalloc*

Yet another memory allocation facility offered by the kernel is **`OSMalloc`**. The function prototypes are declared in libkern/libkern/OSMalloc.h (making them technically part of the Libkern KPI), but their implementation is in osfmk/kern/kalloc.c.

The main advantage of using `OSMalloc` is its support of **memory tags**. The function takes an additional `OSMallocTag` argument, which is a pointer to a structure defined as follows:

Listing 12-3: The `OSMalloc` tag, from libkern/libkern/OSMalloc.h

```
typedef struct _OSMallocTag_ {
    queue_chain_t   OSMT_link;
    uint32_t        OSMT_refcnt;
    uint32_t        OSMT_state; // OSMT_VALID or OSMT_RELEASED
    uint32_t        OSMT_attr;
    char            OSMT_name[OSMT_MAX_NAME];  // 64
} * OSMallocTag;

#define OSMT_DEFAULT    0x00
#define OSMT_PAGEABLE   0x01

extern OSMallocTag OSMalloc_Tagalloc(
    const char * name,
    uint32_t     flags);
..

void *
OSMalloc(
        uint32_t                        size,
        OSMallocTag                     tag)
```

Using a tag offers not only the ability to provide a meaningful name, but to also count the number of times it is used. When a tag is created with `OSMalloc_Tagalloc()`, subsequent allocations using `OSMalloc` increase its reference count. Further, flagging a tag with `OSMT_PAGEABLE` causes the `OSMalloc()` to call `kmem_alloc_pageable_external()`, rather than `kallog_tag_bt()`. Note, however, that the `OSMalloc` tags does not map to that `kalloc_tag*`, for which only `VM_KERN_MEMORY_KALLOC` is used. Unfortunately, there is no way to obtain the `OSMallocTag` of `OSMalloc()`ed memory, nor is there a way (outside of inspecting kernel memory) to obtain the list of tags.

The Zone Allocator

The zones used by `kalloc` are only several of many more, used by the underlying **Zone Allocator**. Zone allocators are very popular across operating system kernels, though they are sometimes known by other names, for example, Linux's Slabs. A **zone** is defined as one or more sets of preallocated, virtually contiguous memory pages. Each zone has a predefined element size, and its pages can be used for obtaining elements of that stated size, but no other: `zalloc()` and its variants accept only the `zone`, but no size arguments. Preallocating pages and allocating elements in them allows compaction of memory space, especially in cases where the element size is much smaller than a page size - as the alternative would have been to consume a page per element, which would have been very wasteful. When elements are no longer needed, they are `zfree()`d back into their zone, and may be reused.

The kernel allocates a large chunk of its virtual memory for zones in `zone_map`, through a call to `kmem_suballoc()` made in `zone_init()` (osfmk/kern/zalloc.c). Just how large the `zone_map` is varies by pointer size and physical memory. `vm_mem_bootstrap` (which calls `zone_init()`) takes the `zsize` boot argument (if specified, in GB) as a base, or defaults to one quarter of physical memory, but no less than `CONFIG_ZONE_MAP_MIN` (specified in config/MASTER). For 64-bit architectures, whichever value used will further be increased by 50%, but clamped to no more than half of RAM (In other words, the `zone_map` is usually 3/8 of available RAM). For 32-bit architectures, it is not increased and further clamped by 1.5GB. Note, however, that the allocation is virtual, not physical - i.e. not all pages of the `zone_map` are guaranteed to be resident (until actually used).

Individual zones can then be allocated inside the `zone_map`. Each zone is initially allocated as one set of pages, using `zinit()` (also from osfmk/kern/zalloc.c). The function takes four arguments - the *size* of the element, the *max*imum size the zone can grow to, the *alloc_size* used for the initial allocation or during expansion, and a human readable name for the zone. The `alloc_size` is chosen so as to divide as cleanly as possible by the number of elements. The name is required so that `mach_zone_info` users (notably, the `zprint(1)` utility) can distinguish between the many dozens of zones*.

The `zinit()` call returns a `zone_t`, which is a pointer to a `struct zone`. This is a relatively simple structure, used to maintain the zone's `free_elements` list, `pages` metadata, its `lock`, `zone_name`, and a bitmap of its associated attributes. These are normally set by default during zone creation (i.e., in `zinit()`), but remain modifiable (only before the zone is used) through flags specified to `zone_change()`.

Table 12-4:

Attribute	Default	Meaning
`Z_EXHAUST (1)`	false	When zone is full no more elements can be allocated.
`Z_COLLECT (2)`	true	Zone applicable for garbage collection.
`Z_EXPAND (3)`	true	Zone may grow with subsequent allocations.
`Z_FOREIGN (4)`	false	May collect foreign elements, outsize zone map.
`Z_CALLERACCT (5)`	true	Allocations accounted to the caller.
`Z_NOENCRYPT (6)`	false	Mark zone memory as not requiring encryption.
`Z_NOCALLOUT (7)`	false	No asynchronous allocations.
`Z_ALIGNMENT_REQUIRED (8)`	false	Mark as alignment required (for `CONFIG_KASAN`)
`Z_GZALLOC_EXEMPT (9)`	false	Mark as untracked by the Guard Zone Allocator (`CONFIG_GZALLOC`)
Darwin 17		
`Z_KASAN_QUARANTINE (10)`	true	`CONFIG_KASAN`: Quarantine `zfree()`d elements
`Z_TAGS_ENABLED (11)`	false	`VM_MAX_TAG_ZONES`: Enable element tags (also requires `-zt` boot-arg)
Darwin 18		
`Z_CACHING_ENABLED (12)`	no	Enables zone to be used with new Zone Cache mechanism

* - The human readable name as a fourth argument to a function both serves to uniquely symbolicate `zinit()`, as well as all of its callers - a useful method used by `jtool2 --analyze` when operating on kernelcaches.

Most zones are named after their element name (i.e. the corresponding `struct`), or very similarly (e.g. "tasks" containing `struct task`). Table 12-5 shows some of the important zones. Those initialized by `kmeminit()` are all Mach zones corresponding to BSD layer zones.

Table 12-5: Some of the zones defined as of XNU 4570

Zone Name	Allocated by	Used for
`kalloc.###`	`kalloc_init()` (osfmk/kern/zalloc.c)	Miscellaneous `kalloc` allocations
`tasks`	`task_init()` osfmk/kern/task.c)	Mach Task objects (q.v. Chapter 10)
`thread`	`thread_init()` osfmk/kern/thread.c)	Mach thread shuttles (q.v. Chapter 10)
`ipc spaces`	`ipc_init()` (osfmk/ipc/ipc_init.c)	Task `ipc_space` objects
`ipc ports`		`ipc_port` objects
`proc`	`kmeminit()` (bsd/kern/kern_malloc.c)	BSD processes (q.v. Chapter 6)
`uthread`		BSD threads (q.v. Chapter 6)
`mount`		Mounted filesystems (q.v. Chapter 7)
`fileglob`		Kernel file representation(q.v. Chapter 7)
`fileproc`		Process files (q.v. Chapter 7)
`file desc`		Process file descriptors (q.v. Chapter 7)
`vnodes`		Vnodes (q.v. Chapter 7)

Once a zone is `zinit()`ed, elements can easily be obtained from it by a call to `zalloc()`. The function takes a single argument - the zone from which to allcoate, and thus ensures the size is as was predetermined by `zinit()`. `zalloc()` calls `zalloc_internal()`, which also controls whether the allocation `canblock`, needs to wait for new pages (`nopagewait`), a `reqsize` indicating how much of the allocation will actually be used, and a memory `tag`. Multiple `zalloc_*` variants exist to wrap these arguments, although their use is uncommon.

When a zone nears exhaustion or is empty, it may be expanded or replenished. Zones marked `async_prio_refill` (presently, `Reserved.VM.map.entries` and `VM.map.holes`) are replenished asynchronously by the `zone_replenish_thread()` when they fall below `prio_refill_watermark`. All other zones marked `expandable` may be expanded in `zalloc_internal()`. Both operations involve `kernel_memory_allocate()`ing a new chunk of the specified zone's `alloc_size` (or, if memory is low, at least one element, rounded up to a page size), and then "cramming" it into the zone, using the `zcram()` routine, which also updates the metadata for the new pages accordingly. There is also a `zfill()` function, which `kernel_memory_allocate()`s a chunk large enough to fill a requested number of elements. The `os_reason_init()` routine (in bsd/kern/sys_reason.c) uses this to ensure the `os_reason_zone` has enough memory, even during jetsam events (which require specifying an `os_reason` object).

The `zprint(1)` command is a highly useful utility to glean information about the state of the zones. `zprint(1)` uses the `mach_[zone|memory]_info` (`mach_host` subsystem #220 and #227) and `task_zone_info` (#3428 in the `task` subsystem) MIG routines. The former MIG calls produce the zone listing, and although part of `mach_host` and not `host_priv` they nonetheless requires `root` privileges. There is also a `mach_zone_info_for_zone()` MIG routine (`mach_host` #231).

`zprint(1)` is part of the system_cmds project, but is not part of the *OS binpack because Apple already provides a signed (and entitled!) binary on *OS, intended to be used as part of the `sysdiagnose(1)` process. As of later *OS variants, however, Apple restricts `mach_memory_info()` through a `#if CONFIG_DEBUGGER_FOR_ZONE_INFO`, which refuses this functionality unless `PE_i_can_has_debugger(NULL)` (=`debug_enabled`) is true, rendering the command useless.

Using `zprint(1)` is a great way to find the `sizeof()` of some common structures, especially those which keep changing in between Darwin versions.

Zone Management

It used to actually take a zone in order to manage zones. Prior to setting up the kernel zones, a special "zone of zones" was created by a call to `zone_bootstrap()`, prior even to `zone_init()`, which sets up the `zone_map`. This practice also supported "fake zones", which were memory regions (e.g. kernel stacks) that `mach_zone_info()` would report to user mode.

As of Darwin 16, the `zones` zone has been removed, and `zone_bootstrap()` instead sets up a `zone_array`, which is statically allocated to accommodate up to `MAX_ZONES` `struct zone` entries. The value was initially 256, but has grown (around Darwin 17.2) to 320, where it remains at this time. The `zone_empty_bitmap` tracks which zones are empty (i.e., it is initially set to all '1's), allowing destroyed zones to be reused. The number of used entries (= '0' bits) in the bitmap tracked with the `num_zones_in_use` variable, and the number of overall zones created with `num_zones`. The zone names themselves are stored (NULL-terminated and concatenated to one another) on a dedicated page, allocated by `kmem_alloc_kobject` and pointed to by `zone_names_start`. An additional pointer, `zone_names_next`, points to the end of the last zone name in the page.

Zone memory is taken from the `zone_map`. This is a `vm_map` which spans from `zone_map_min_address` to `zone_map_max_address`. At the beginning of the `zone_map` is a special area called the **zone metadata region**, through which individual pages can be tracked to their allocating zones. The metadata spans from `zone_metadata_region_min` (usually equal to `zone_map_min_address`) to `zone_metadata_region_max`.

Table 12-6 shows all the zone-related variables, including those described above, and others we will encounter soon. MacOS XNU exports these symbols, but the *OS kernels do not - although all are readily identified by `joker`.

Table 12-6: Zone related variables (all defined in osfmk/kern/zalloc.c)

Variable	Holds
`zone_map`	A `vm_map_t` holding the actual virtual memory used by the all zones
`all_zones_lock`	A `simple_lock` guarding access to the `zone_array`, `num_zones[_in_use]` and the `zone_empty_bitmap`
`zone_map_[min/max]_address`	`vm_offset_t`s specifying beginning and end of zone map
`zone_metadata_region_[min/max]`	`vm_offset_t`s specifying beginning and end of metadata
`zone_metadata_region_lck`	A `mtx_lck_t` protecting the zone metadata region contents
`zone_array`	An array of up to `MAX_ZONES` (presently, 320) `struct zone` entries
`num_zones`	Tracks zones created in `zone_array` (incremented by `zinit`)
`num_zones_in_use`	Tracks non-empty zones (as `num_zones`, decremented by `zdestroy`)
`zone_empty_bitmap`	Bitmap of `MAX_ZONES` (in practice, `num_zones`) bits, tracking use

As an example (from MacOS 14.3), consider the following:

```
bash-3.2# xnoop dump _zone_map_min_address,8
0xffffff801449c950    0xffffff801a038000
bash-3.2# xnoop dump _zone_map_max_address,8
0xffffff801449c958    0xffffff804a038000
bash-3.2# xnoop dump _sane_size,8
0xffffff80145e7408 00 00 00 80 00 00 00 00
```

Working the math and taking the difference between the `zone_map_max_address` and the `zone_map_min_address` we'd get 0x30000000. This is in line with what would be expected from the source, since this value is a three eights of `sane_size` (as set up by `vm_mem_bootstrap()` for `__LP64__` kernels.

The zone_metadata_region

Maintaining metadata for zones is a daunting challenge. On the one hand, it must be done as efficiently as possible. On the other, zones are frequent target for exploitation in controlled kernel memory overwrites, and therefore efficiency should not come at the cost of security. Apple has continuously been modifying zone management, with the latest redesign in Darwin 16 and above.

As of Darwin 16, per page zone metadata has been moved into its very own `zone_metadata_region`. Bound between `zone_metadata_region_min` and `..max`, this is a large array of `struct zone_page_metadata`. Each of these is a fixed size element, so it follows that a formula can be used, given a memory address, to find its zone metadata. First, the page index needs to be found, for which the `PAGE_INDEX_FOR_ELEMENT` macro is used. Listing 12-7 shows this macro:

Listing 12-7: The `PAGE_INDEX` macros in osfmk/kern/zalloc.c

```
/* Macro to get page index (within zone_map) of page containing element */
#define PAGE_INDEX_FOR_ELEMENT(element)                                 \
  (((vm_offset_t)trunc_page(element) - zone_map_min_address) / PAGE_SIZE)

/* Macro to get page for given page index in zone_map */
#define PAGE_FOR_PAGE_INDEX(index)                                      \
  (zone_map_min_address + (PAGE_SIZE * (index)))
```

Recall, that the zone map includes its own pages - and all other zones, whose pages are contiguous in virtual memory. A page's index can therefore be found by taking the rounded page address of the element (bitmasked with the inverse of `PAGE_MASK`, as is performed by the `trunc_page()` macro) and subtracting it from `zone_map_min_address`, then dividing that difference by the `PAGE_SIZE`. Of course, this operation is fully reversible, so the inverse macro, `PAGE_FOR_PAGE_INDEX`, is used in those cases.

Index at hand, finding the metadata is as straightforward as looking at the entry at that index in the array starting at `zone_metadata_region_min` whose elements are `struct zone_page_metadata`. Indeed, this is what the `PAGE_METADATA_FOR_PAGE_INDEX` macro does. This, too, has an inverse operation, and both are shown in Listing 12-8:

Listing 12-8:

```
/* Macro to get metadata structure given a page index in zone_map */
#define PAGE_METADATA_FOR_PAGE_INDEX(index)                             \
  (zone_metadata_region_min + ((index) * sizeof(struct zone_page_metadata)))

/* Macro to get index (within zone_map) for given metadata */
#define PAGE_INDEX_FOR_METADATA(page_meta)                              \
 (((vm_offset_t)page_meta - zone_metadata_region_min) / sizeof(struct zone_page_metadata))
```

Now let's continue our example with an arbitrary address - say, that of the `kernel_task`:

```
bash-3.2# jtool -S /System/Library/Kernels/kernel | grep kernel_task$
ffffff8000c9c218 S _kernel_task
# Apply slide:
bash-3.2# xnoop sdump 0xffffff8000c9c218,8
0xffffff801449c218    0xffffff801aa4d280
```

The `kernel_task` export is, once slid, at 0xffffff801449c218 - which is outside the zone map. This is as it should be, because the `_kernel_task` export is in the `__DATA.__common`. But the `kernel_task` is a `task_t` - i.e. a pointer, and the `struct task` it points to is at `0xffffff801aa4d280` - well within the zone map. To find the page index of this address, we need to manually apply the `PAGE_INDEX_FOR_ELEMENT` macro calculation:

```
PAGE_INDEX_FOR_ELEMENT(0xffffff801aa4d280) =
      (0xffffff801aa4d000 - 0xffffff801a038000) / 0x1000 = 0xa15
```

Getting the metadata for a zone element at a given address is therefore a two step operation, although it can be combined into a larger macro - which is exactly what `PAGE_METADATA_FOR_ELEMENT` achieves.

Continuing our example of locating the page holding the `kernel_task` structure, which was in page 0xa15. Where is the metadata for this zone? First, we find the metadata region, which conveniently overlaps with the beginning of the zone map*:

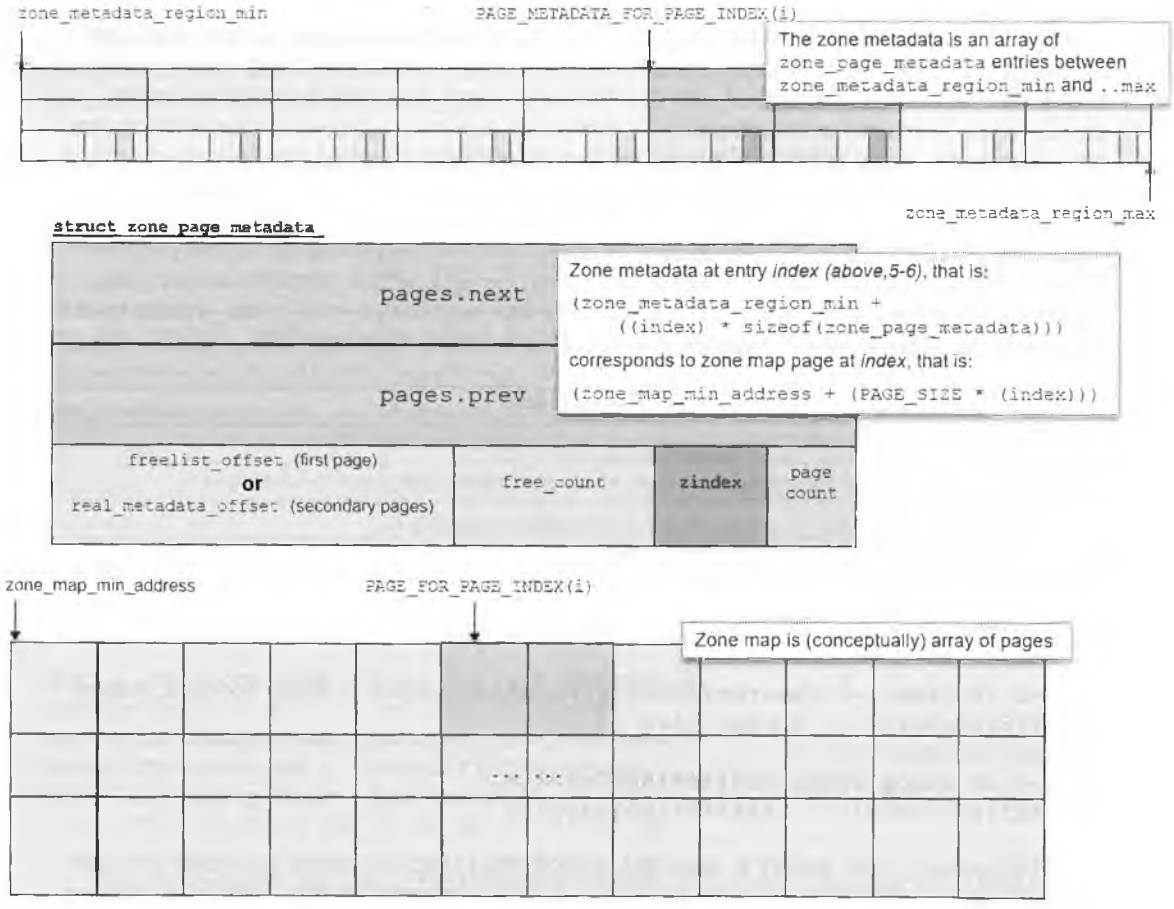
```
bash-3.2# xnoop dump _zone_metadata_region_min,8
0xffffff801449c960    0xffffff801a038000
bash-3.2# xnoop dump _zone_metadata_region_max,8
0xffffff801449c968    0xffffff801a4b8000
```

The `sizeof(struct zone_page_metadata)` is 24 (= 0x18). This means that the metadata entry for page 0xa15 can be found by:

```
PAGE_METADATA_FOR_PAGE_INDEX(0x1a5) =
= 0xffffff801a038000 + (0x1a5 * 0x18) = 0xffffff801a03a778
```

Figure 12-9 shows a graphic example. In it, pages #5 and #6 of the zone map (counting from 0) are highlighted, as is their metadata. Finding the metadata and the page, given its index, is a direct application of the formulae shown in this section.

Figure 12-9: Zone metadata and the zone map



* - Actually, not that conveniently.. Attempting to read from the very beginning of the zone metadata region will fail, and may panic the kernel! In fact, pages #5 and #6 shown in the illustration and chosen for simplicity, are also similarly unreadable, with the metadata becoming safe to read only later on. The reason why is left to ponder as a review question.

The zone metadata

Thus far, we've established pages belonging to zones (and only those pages) have corresponding metadata elements in the `zone_metadata_region`. When a zone claims pages (during its creation or expansion), the metadata of these pages is made resident (by `kernel_memory_populate()`). But what exactly is maintained in the metadata? Listing 12-10 shows the structure definition (from osfmk/kern/zalloc.c), as did Figure 12-9 (in the previous page).

Listing 12-10: The `struct zone_page_metadata` (from osfmk/kern/zalloc.c)

```
struct zone_page_metadata {
  queue_chain_t   pages; /* linkage pointer for metadata lists */

  /* Union maintaining start of element free list and real metadata (for multipage allocations) */
 union {
  /*
  * The start of the freelist can be maintained as a 32-bit offset instead of a pointer because
  * the free elements would be at max ZONE_MAX_ALLOC_SIZE bytes away from the metadata. Offset
  * from start of the allocation chunk to free element list head.
  */
  uint32_t   freelist_offset;
  /*
  * This field is used to lookup the real metadata for multipage allocations, where we mark the
  * metadata for all pages except the first as "fake" metadata using MULTIPAGE_METADATA_MAGIC.
  * Offset from this fake metadata to real metadata of allocation chunk (-ve offset).
  */
  uint32_t   real_metadata_offset;
  };

  /*
  * For the first page in the allocation chunk, this represents the total number of
  * free elements in the chunk.
  */
  uint16_t    free_count;
  unsigned    zindex     : ZINDEX_BITS;    /* Zone index within the zone_array */
  unsigned    page_count : PAGECOUNT_BITS; /* Count of pages within the allocation chunk */
};
```

Prior to Darwin 17 the number of bits in zindex was fixed to 8 - which caused a problem with zone indices greater than 254. After increasing `MAX_ZONES` past 256 (in Darwin 17), `zindex` was allowed to "borrow" two bits from `page_count` (i.e. ZINDEX_BITS is now 10), which is fine considering that zones chunks have a small number of pages.

Most of the fields are self explanatory, but the union demands explanation: In the first page of a zone chunk it maintains an offset to the list of free elements in that zone. Using an offset instead of a pointer is advantageous in that it saves four bytes per element. For other pages within the same chunk, there's no benefit in saving any freelist reference - as it can be walked from the first page anyway. There is, however, a need to quickly find the real metadata. Once again, this is best served by an offset.

Element Free Lists

Even if we assume for a minute that zones start up empty, and elements are added linearly, one after the other, soon enough (as elements are freed) it is inevitable that "holes" form in zones over the freed elements. To maintain efficiency, these free elements need to be tracked so that they can be reallocated. The way to do that is to maintain element **free lists**. The zone metadata maintains a 32-bit `freelist_offset`, from the start of the allocation chunk to the freelist's head. For subsequent pages inside the same allocation chunk, this field is repurposed (via a union) to point to the `real_metadata_offset`

When cramming a zone, `zcram()` introduces basic freelist randomization: `random_free_to_zone()` is called to free the chunk's elements and splice the free list by progressively adding elements from the beginning or the end of the page.

Garbage Collection

As we've established, freeing zone elements results in holes. If such holes are large enough so as to encompass an entire page, the zone can be compacted and the page freed for re-use, possibly by another zone. There is thus a need to collect "garbage" pages periodically, or on low memory conditions.

Garbage collection is performed by calling `consider_zone_gc()`, with a boolean to `consider_jetsams`. The "consideration" is allowing for a special case reclaiming early boot memory (from `kmapoff_kaddr`, as discussed in @@BOOT) and otherwise checking that `zone_gc_allowed` is set to TRUE, which it always is. The calls to `consider_zone_gc()` are made from the vm_pageout thread's `vm_pageout_garbage_collect`, and from `vm_page_find_contiguous()`, on failure to find pages.

The actual garbage collection is performed by `zone_gc()`, which may first call `kill_process_in_largest_zone()` if `consider_jetsams` was true, as discussed under @@JETSAM. `zone_gc()` then acquires the `zone_gc_lock` and iterates over all zones marked as `collectable` in the `zone_array`, calling `drop_free_elements()` if they have pages in their `all_free` queue. Cacheable zones (as of Darwin 18) also have their depot drained, as discussed later. The garbage collection occurs under the `zone_gc_lock`, which ensures only one concurrent garbage collection can take place. Throughout the process the thread's `options` flags `TH_OPT_ZONE_GC`, marking the thread as garbage collecting (and also avoid a potential deadlock with the zone replenish thread).

`drop_free_elements()` locks the zone it operates on, in order to "snatch" its `all_free` queue, replacing it with an empty queue. The (now detached) queue is iterated over once to determine its size and element count, and then the zone is locked again briefly so it can be adjusted accordingly. The detached queue is then iterated over again, this time dequeueing each page chunk in turn, and then calling `kmem_free()` to free the page from the `zone_map`.

Iterating over all the zones' free lists in this manner can be a very long operation, so after every such free operation a call to `thread_yield_to_preemption()` is made (allows possible preemption (for a pending `AST_PREEMPT`, as discussed in Chapter 10). Since `drop_free_elements()` may be called from `zdestroy`, the preemption check is made only as part of a `zone_gc()`, as determined by the aforementioned `TH_OPT_ZONE_GC` option.

GC and UAF

Garbage collection, however, also proves to be an instrumental step in exploitation, as part of the "Feng Shui" required to channel the Qi of exploitation. Rather than thinking about it as "garbage collection" in this context, it helps to consider "memory recycling" - The memory freed following garbage collection can be reused by the system for entirely different purposes. Memory which previously represented a given object in some zone may be repurposed and later used by another object (of the same or of a different type) in some other (or the same) zone.

If a Use-After-Free condition can be triggered, a user mode attacker can cause the reference count of an object to drop to zero, while still nonetheless holding a reference to the object in user mode. If the attacker further has the ability to control write operations to the object's memory after it is repurposed (for example, by spraying fake content in a Mach message OOL descriptor), the object (usually, a port or IOUserClient) can be entirely controlled. Specific examples of these attacks can be found throughout Volume III.

What made this type of exploitation far easier was that garbage collection could be trigged from used mode, by calling `mach_zone_force_gc` (MIG message #221 of the `mach_host` subsystem). The call was synchronous, so it was guaranteed any reclamation of pages would be complete when it returned. Apple eventually figured out this is a security concern, and removed the call (outside of `DEBUG/DEVELOPMENT`) as of Darwin 17. Garbage collection can still be triggered, however, by rapidly allocating many pages (for example, by creating a large amount of kernel objects), or sending (but not receiving) many Mach messages. Destroying the objects or receiving the messages results in freeing the resepective zone elements, eventually coalescing the free pages. Although the operation is nowadays performed asynchronously, the interested caller can simply delay execution sufficiently for collection to reliably complete.

Battling zone corruption

In a perfect world, zone metadata, free lists and allocations could be trusted, since they are in kernel memory. In the cruel, far-from-perfect world of XNU, however, nefarious hackers and jailbreakers find new vulnerabilities leading to kernel memory corruption. Apple has reworked the metadata several times, before settling (for now) on the Darwin 16 and later approach described earlier.

A common exploitation method (exploited, for example, by the TaiG jailbreak, as discussed in III/18) involves freeing an element (using `zfree()` or, as of Darwin 18, the quicker `zfree_direct()`) into the wrong zone. Either variant requires both the zone pointer and the element to be freed, but it is only from Darwin 16 that such `zfree()` operations (but not `zfree_direct()`s, used by the zone cache) are reliably intercepted, thanks to the new zone metadata layout.

During the free operation the element may or may not be "poisoned", by `memset()`ing with `ZP_POISON` (`0xdeadbeefdeadbeef`). The `zfree()` variants both call element poisoning code (refactored in Darwin 18 into the `zfree_poison_element()` routine). Doing so for every element, however, is quite costly, so concessions have to be made:

- Zones whose element sizes is equal to or less than `zp_tiny_zone_limit` always get poisoned. This value is set by `zp_init()` to the CPU's cache line size (`cpu_info.cache_line_size`). The `-no-zp` boot argument sets this value to zero, thereby disabling poisoning altogether.
- For larger zones, `zp_factor` and `zp_scale` govern the frequency of poisoning. These are initially set to `ZP_DEFAULT_[SAMPLING/SCALE]_FACTOR` (16 and 4, respectively), and the default factor is further permuted in one out of every two cases by +/-1 (as determined by two bits from `early_random()`. The `zp-factor` and `zp-scale` boot arguments (with dashes, not underscores..) can override these values. Whichever way they are set, `sample_counter()` tracks the freed zone's `zp_count`, and possibly poisons according to them, with the `zp_scale` providing a right logical shift for the element size. This means that larger elements are less likely to be poisoned, and the `zp_scale` can control the frequency of poisoning. Setting the `zp_factor` to 0 effectively disables this poisoning, and setting it to 1 (or setting the `-zp` boot argument, which does so as well) poisons every operation.

There are further integity checks for zone pointers in the free list, rolled up into `is_sane_zone_ptr()`. The current criteria mandate alignment to pointer boundary, a kernel address, and pointing to somewhere in the zone map (unless the zone allows foreign elements).

Darwin 19 adds a significant improvement with `zone_require (address, zindex)`. Prior to dereferencing an object pointer, this call ensures that the *address* belongs to the zone at *zindex*. This effectively eliminates a common technique of UaF/GC in which fake objects (mostly `ipc_ports`, but potentially tasks, procs, etc) could be constructed (by parking `mach_msgs` and OOL descriptors in kernel).

The Guard Mode Zone Allocator (MacOS)

MacOS `#define`s the `CONFIG_GZALLOC` setting, which enables the "Guard Mode" zone allocator. When set, this makes `zalloc_internal` first call to `gzalloc_alloc()`, rather than the zone cache or the traditional zone allocation. Guarded zone allocations then behave very similarly to the way `libgmalloc(3)` (Guard Malloc) allocations do in user mode, to detect use of uninitialized data or potential overflows, but in kernel mode. It does so by `memset()`ting a pattern ('g') on free and adjoining guard pages (protected to `PROT_NONE`) to allocations.

Similar to libgmalloc(3), the Guard Mode zone allocator can be configured to detect underflows, rather than overflows, and other aspects of allocation behavior can be tweaked. This is done by passing the following boot arguments:

Table 12-11: The boot arguments processed by the Guard Mode zone allocator

-[no]gzalloc_mode	Enables the allocator on all zones, or disables. Default is disabled
gzalloc_[min|max]	Target only zones with elements between min and max (default: none)
gzalloc_uf_mode	Protect underflows, rather than overflows
gzalloc_fc_size	Free element cache size
gzname	Zone to target, by name
-gzalloc_wp	Set guard page permissions to allow read
-gzalloc_no_dfree_check	Disable double free check (default: check)
-gzalloc_noconsistency	Disable consistency checks (default: check)

The guard zone allocator also adds gzalloc_data_t metadata to every zone. This is a simple structure, containing an array of addresses (gzfc), and an index holding its active size. Using this allocator is far more reliable than the probabilistic poisoning, but does waste significant memory. It therefore only applies to gzalloc_tracked() zones, which presently consist of zones whose element sizes fall between the gzalloc_[min|max], or the particular zone matched by the gzname argument. This is only if the zones in questions aren't marked by gzalloc_exempt (through the Z_GZALLOC_EXEMPT flag of zone_change()). The default value of gzalloc_min is greater than the max, so unless changed (or -gzalloc_mode is explicitly stated), no zones (but the possibly named one) will be tracked.

The Zone Cache (Darwin 18+)

Darwin 18 adds a new layer on top of the zone allocator, called the **zone cache**. The layer draws on academic research, and aims to make zone allocations more efficient and scalable across multiple CPUs. Listing 12-12 (next page) shows the verbose description of the zone caching mechanism, from osfmk/kern/zcache.h. Readers remembering the discussion of the user mode magazine allocator (I/8) will likely be able to find the strong parallels between the two.

Zone caching is contingent on CONFIG_ZCACHE being defined, though that is true across all Darwin 18 flavors. Additionally, it requires either specific zone opt-in by specifying the zcc_enable_for_zone_name= boot-arg, global enablement by -zcache_all, or specific opt-in by calling zone_change() with the Z_CACHING_ENABLED flag (presently set only for ipc_kmsg_zone). When caching is enabled for a zone, zcache_init() is called on it, initializing a per-cpu cache for it and setting the zone's cpu_cache_enabled() field. Zones zinit()ed before the zone cache is ready are tagged through their cpu_cache_enable_when_ready field, so that zone_bootstrap() picks up the marking and zcache_init()s them.

zone_caching_enabled(*zone*) checks the criteria on a per zone basis which is that the zone is marked with cpu_cache_enabled, and that the zone is not tagged or followed by zleaks. If met, zalloc_internal() is diverted to zcache_alloc_from_cpu_cache, and likewise zfree() is diverted to call zcache_free_to_cpu_cache().

The magazines are kept in their own zone (zcc_magazine_zone). The zone cache also uses its own zcache_canary, with an early_random() value set by zcache_bootstrap(). The canary is added at the beginning and end of each element when freed to the CPU cache (or when the magazine is filled), and validated when elements are allocated from the cache (or the magazine is drained). This provides another way to intercept potential use after free. When draining the magazine, after the canary is validated the element is freed through zfree_direct(), as a lightweight version of zfree() which skips the cumbersome checks of zfree()ing to the wrong zone.

Listing 12-12: Zone caches (from XNU 4903's osfmk/kern/zcache.h)

```
/*      Below is a diagram of the caching system. This design is based of the
 * paper "Magazines and Vmem: Extending the Slab Allocator to Many CPUs and
 * Arbitrary Resources" by Jeff Bonwick and Jonathan Adams. It is divided into 3
 * layers: the Per-cpu Layer, the Depot Layer, and the Zone Allocator. The
 * Per-CPU and Depot layers store elements using arrays we call magazines.
 *
 *      Magazines function like a stack (we push and pop elements) and can be
 *  moved around for bulk operations.
 *
 *  ___________       ___________       ___________
 * |  CPU 1    |     |  CPU 2    |     |  CPU 3    |
 * |           |     |           |     |           |
 * |  |#| |_|  |     |  |_| |#|  |     |  |#| |#|  |        Per-CPU Layer
 * |  |#| |_|  |     |  |_| |#|  |     |  |#| |#|  |
 * |___________|     |___________|     |___________|
 *
 *  _____________________________________________
 * |                                             |
 * |          |#| |#| |#| |_| |_| |_|            |        Depot Layer
 * |          |#| |#| |#| |_| |_| |_|            |
 * |_____________________________________________|
 *
 *  _____________________________________________
 * | # | # | # | # | # | # | # | # | # | # | # |  |    Zone Allocator
 * |_____________________________________________|
 *
 *      The top layer is the per-cpu cache and consists of a current and
 * previous magazine for each CPU. The current magazine is the one we always try
 * to allocate from and free to first. Only if we are unable, do we check the
 * previous magazine. If the previous magazine can satisfy the allocate or free,
 * then we switch the two and allocate from the new current magazine. This layer
 * requires no locking, so we can access multiple CPU's caches concurrently.
 * This is the main source of the speedup.
 *
 *      We have two magazines here to prevent thrashing when swapping magazines
 * with the depot layer. If a certain pattern of alloc and free are called we
 * can waste a lot of time swapping magazines to and from the depot layer. We
 * prevent this by dividing the per-cpu cache into two separate magazines.
 *
 *      The middle layer is the magazine depot. This layer consists of a
 * collection of full and empty magazines. These are used to reload the per-cpu
 * caches when needed. This is implemented as an array of magazines which are
 * initially all empty and as we fill up magazines we increment the index to
 * point at the first empty magazine. Since this layer is per-zone, it allows us
 *  to balance the cache between cpus, but does require taking a lock.
 *
 *      When neither the current nor previous magazine for a given CPU can
 * satisfy the free or allocation, we look to the depot layer. If there are
 * magazines in the depot that can satisfy the free or allocation we swap
 * that magazine into the current position. In the example below, to allocate on
 * the given CPU we must lock the depot layer and swap magazine A with magazine
 * B and decrement the depot index.
 *
 *      ___________________________       _____________________________________
 *     |     Per-CPU Cache         |     |            Depot Layer              |
 *     |                           |     |                                     |
 *     |   A___        ____        |     |          B___        ____   ____    |
 *     |  |    |      |    |       |     |  ____   |    |      |    | |    |   |
 *     |  |    |      |    |       |     |  |##|   |    |      |    | |    |   |
 *     |  |    |      |    |       |     |  |##|   |    |      |    | |    |   |
 *     |  |    |      |    |       |     |  |##|   |    |      |    | |    |   |
 *     |  |    |      |    |       |     |  |##|   |    |      |    | |    |   |
 *     |  |____|      |____|       |     |  |##|   |____|      |____| |____|   |
 *     |  Current    Previous      |     |  |__|                                |
 *     |___________________________|     |_____________________________________|
 *
 *      The bottom layer is the Zone Allocator. This is already implemented in
 *  XNU and will remain mostly unchanged. Implementation for this can be found
 * in zalloc.c and zalloc.h. We will only use the zone if all other layers are
 * unable to satisfy the allocation or free. When we do use the zone, we will
 * try to allocate an entire magazine of elements or free an entire magazine of
 * elements at once.
 *
 *      Caching must be enabled explicitly, by calling zone_change() with the
 * Z_CACHING_ENABLED flag, for every zone you want to cache elements for. Zones
 * which are good candidates for this are ones with highly contended zone locks.
 * Some good potential candidates are kalloc.16, kalloc.48, Vm objects, VM map
 * entries, ipc vouchers, and ipc ports.
 * ..
 */
```

Memorystatus (MacOS) and Jetsam (*OS)

The user mode perspective of memory pressure conditions, which occur when the system is low on physical memory, was discussed in III/8, which also introduced MacOS's **memorystatus**, and the *OS Jetsam. Whereas the former is a gentle, opt-in mechanism (thanks to the abundant availability of swap space), the latter is a cruel and harsh overlord, which will not hesitate to kill for the most minor of transgressions. The `memorystatus_do_kill()` routine (in bsd/kern/kern_memorystatus.c) takes a `uint32_t` *cause* argument, which is a `kMemoryStatusKilled*` constant from the following:

Table 12-13: The many causes of untimely death by Jetsam/Memorystatus (from sys/kern_memorystatus.h)

kMemorystatusKilled..	Reason
(--)	Jettisoned
Hiwat	High Water Mark (maximum memory utilization)
Vnodes	Maximum vnode utilization (vnode table full)
VMPageShortage	Overall free page shortage
ProcThrashing	Process thrashing
FCThrashing	File Cache thrashing
PerProcessLimit	Per-process page limit exceeded
DiskSpaceShortage	Low disk space
IdleExit	Idle exit (memory status)
ZoneMapExhaustion	Zone map nearing exhaustion
VMCompressorThrashing	Compressor thrashing (excessive operations due to memory handling)
VMCompressorSpaceShortage	Compressor overall space shortage

The various reasons have corresponding `memorystatus_kill_on_..` functions, and those funnel to `memorystatus_kill_process_sync()`, which proceeds to kill either the PID specified, or the top process according to the jetsam priority bands (discussed in I/9). Execution is swift and merciless: `memorystatus_do_kill()` calls the "no-frills", no saving throw `jetsam_do_kill()`, which uses `exit_with_reason()` to smite the process with a `SIGKILL`. `memorytstatus_do_kill()` then triggers memory compaction, to free as much memory as possible.

The most common reason for riling Jetsam is `kMemoryStatusKilledHiwat`, which is common on *OS when a process commits the deadly sin of gluttony, consuming too much memory. Jetsam can be made to warn (`memorystatus_warn_process()`) through `memorystatus_on_ledger_footprint_exceeded()`. This code path, however, is no longer active as of Darwin 16. Instead, `consider_vm_pressure_events()` defers to `memorystatus_update_vm_pressure()`, which dispatches a knote, which Jetsam-fearing apps can respond to (per `didReceiveMemoryWarning()`, q.v. I/9-32).

Senseless bloodshed must be avoided if possible, so `memorystatus_kill_proc()` also attempts a call to `vm_purgeable_purge_task_owned()`, to see if it can purge some of the task's memory. This is done for all causes, save for vnode or zone map exhaustion.

Purgeable memory

Purgeable memory is a non-standard extensions provided by XNU. Such memory is marked at the `vm_object` level, and may be discarded at the kernel's discretion. As explained in I/8, `libmalloc` supports `malloc_make_[non]purgeable()` calls, and `libcache` (or the higher level `Foundation.framework`'s `NSCache`) make use of this facility. Purgeable memory may also be allocated directly, with the `VM_FLAGS_PURGABLE`* flag when calling `[mach_]vm_allocate()` (or through the `fd` argument of `mmap(2)`, when using `MAP_ANON`).

* - The correct spelling is "purgeable", yet portions of the kernel (primarily in osfmk/mach/vm_purgable.h) are spelled "purgable". This is not only an unfortunate mistake, it can get downright frustrating, especially when both spellings are used in the same routine. The comment in osfmk/vm/vm_purgeable_internal.h expects to eventually "change this on occasion", (perhaps by the simple solution of aliasing through macros?) but the occasion has yet to arrive.

The purgeable facility is accessible through MIG routines in several subsystems:

- `task_purgable_info` (#3438): returning a `struct vm_purgeable_info` on the counts of purgeable memory for this task.
- `[mach_]vm_purgable_control()` (#4818/3830): a `VM_PURGABLE_GET_STATE`, `SET_STATE` or `PURGE_ALL` operation.
- `memory_entry_purgable_control()` (#4900): Starting with Darwin 18, XNU provides a new MIG subsystem, `memory_entry` (#4900), whose first routine is `memory_entry_purgable_control()`. This is the same as the VM-level ones, but at a memory entry granularity.

In kernel mode, purgeable memory is maintained at the `vm_object` level, in the `purgable` field. This can exist in one of three states: `VM_PURGABLE_VOLATILE`, `..NONVOLATILE`, or `..DENY`. The high order bits of the state hold the ordering (`VM_PURGABLE_BEHAVIOR_[FIFO/LIFO]`) and grouping of the page, as documented in osfmk/mach/vm_purgable.h:

Listing 12-14: The purgeable state bits (from osfmk/vm/vm_purgeable.h)

```
/*
 * Purgeable state:
 *
 *  31 15 14 13 12 11 10 8 7 6 5 4 3 2 1 0
 * +-----+--+-----+--+----+-+-+---+---+---+
 * |     |NA|DEBUG|  | GRP| |B|ORD|   |STA|
 * +-----+--+-----+--+----+-+-+---+---+---+
 * " ": unused (i.e. reserved)
 * STA: purgeable state
 *      see: VM_PURGABLE_NONVOLATILE=0 to VM_PURGABLE_DENY=3
 * ORD: order
 *      see:VM_VOLATILE_ORDER_*
 * B: behavior
 *      see: VM_PURGABLE_BEHAVIOR_*
 * GRP: group
 *      see: VM_VOLATILE_GROUP_*
 * DEBUG: debug
 *      see: VM_PURGABLE_DEBUG_*
 * NA: no aging
 *      see: VM_PURGABLE_NO_AGING*
 */
```

Purgable status maintenance is performed through `vm_object_purgable_control()`, which is also what the higher level `..purgable_control()` MIG routines call. The VM subsystem calls look up the corresponding `vm_map_entry`, and through it resolve the `VME_OBJECT`. Possible operations are `VM_PURGABLE_[GET/SET]_STATE`, or `.._PURGE_ALL`.

When setting the state to `VM_PURGABLE_VOLATILE`, the object is disconnected from its physical page, and added to one of the `purgeable_queues`. The facility presently maintains three `purgeable_queues`: `PURGEABLE_Q_TYPE_OBSOLETE` (deprecated, FIFO), `.._FIFO` and `...LIFO`, corresponding to the `VM_PURGABLE_BEHAVIOR_*` bits. The queues (defined in osfmk/vm/vm_purgeable_internal.h) further support sub-groups, allowing objects to be sub-classified. The `..FIFO` and `..LIFO` queues also support tokens:

Listing 12-15: The `purgeabl_q` type (from osfmk/vm/vm_purgeable_internal.h)

```
#define NUM_VOLATILE_GROUPS 8
struct purgeable_q {
        token_idx_t token_q_head;     /* first token */
        token_idx_t token_q_tail;     /* last token  */
        token_idx_t token_q_unripe;   /* first token which is not ripe */
        int32_t new_pages;
        queue_head_t objq[NUM_VOLATILE_GROUPS];
        enum purgeable_q_type type;
};
```

Thus, when `vm_object_purgable_control()` is called with `.._PURGE_ALL`, the routine calls `vm_purgeable_object_purge_all()` (from osfmk/vm/vm_purgeable.c), cycling through all queues, over each of the groups, and calling `vm_object_purge()` to dispose of it. All of this is done under up to three locks - `vm_purgeable_queue_lock`, the purgeable `vm_object`'s own lock, and the `vm_page_queue_lock` (when removing the page). Purging the object is performed by force-moving the physical pages of the object to the free queue (unless busy, paged or wired). The object is then marked as "empty".

Kernel Memory Layout

With all the types of kernel memory figured out at this point, we can draw a rough atlas of kernel memory. Certain areas in kernel memory - especially the `zone_map` - are quite volatile, as processes, threads, kexts and other objects pop in and out of existence. At a higher level, however, i.e. one that considers the `zone_map` and other sub maps opaque, the layout is fairly stable, owing to the kernel's deterministic startup and fixed allocations.

The `kernel_map` regions

The `kernel_map` is just a special case of a `vm_map`, so have `kernel_task`, will travel: Using the `mach_vm_region_*` MIG routines, the `kernel_map`'s individual mappings can be retrieved through `vm_region_*_info` flavors. This can actually be accomplished without holding the task, as well, thanks to the powerful `proc_info` system call (#336). This is the method employed by `procexp(j)` when displaying `regions` for PID 0, and requires no entitlement - only `root` privileges. The output of `procexp(j)` will gives similar results similar to Output 12-16, though over time will get cluttered further with kext allocations and other kernel allocations, which may fill the numerous holes.

Output 12-16: The regions of XNU

```
# Display kernel regions through proc_info, weeding out those with dynamic
# tags, which belong to individual kernel extensions
#
root#Zephyr(-)# procexp 0 regions |
pipe>           grep -v 'Tag
Untagged (0)    0x00000000 ffffff7f80000000-ffffff7f97400000 [ 372M]---/--- NUL
Kext            0x00000000 ffffff7f97400000-ffffff8000000000 [   1G]rw-/rwx NUL   # __PRELINK_TEXT
#
# Kernel hiding here in plain sight... (from __TEXT through __LAST, with KLD section jettisoned.
# In *OS this would be in a 4-16G ---/---- mapping, to make it "harder" to figure out slide
Untagged (0)    0x00000000 ffffff8000000000-ffffff80176c2000 [ 374M]---/--- NUL
Untagged (0)    0x00e6e1c9 ffffff80176c2000-ffffff8017830000 [   1M]rw-/rwx PRV # __LINKEDIT (jettisoned)
Untagged (0)    0x00000000 ffffff8017830000-ffffff8021786000 [ 159M]---/--- NUL
..
PMAP            0x00e50640 ffffff8021931000-ffffff8026a5d000 [  81M]rw-/rwx S/A  # pmap structure from pmap_init()
ZONE            0x00000000 ffffff8026a5d000-ffffff80e6a5d000 [   3G]rw-/rwx NUL  # zone_map_[min-max]_address
OSFMK           0x00e50640 ffffff80e6a5d000-ffffff80e6a5e000 [   4K]rw-/rwx S/A  # zone names
kalloc          0x00000000 ffffff80e6a5e000-ffffff80f6a5e000 [ 256M]rw-/rwx NUL  # kalloc_map
....
IPC             0x00000000 ffffff80f6ad3000-ffffff80f6bd3000 [1024K]rw-/rwx NULL # ipc_kernel_map
IPC             0x00000000 ffffff80f6bd3000-ffffff80f73d3000 [   8M]rw-/rwx NULL # ipc_kernel_copy_map
..
OSKext          0x00c73cc9 ffffff811763d000-ffffff8117642000 [  20K]r--/rwx PRV  # gLoadedKextSummaries
compressor      0x00e50640 ffffff811fcce000-ffffff811fccf000 [   4K]rw-/rwx S/A
compressor      0x00000000 ffffff8123069000-ffffff91232a9000 [  64G]rw-/rwx NUL  # compressor_map
..
```

It's possible to determine the semantics of the memory regions thanks to the memory tags assigned by the kernel and the individual kexts: Calls to `kernel_memory_allocate()`, `kmem_suballoc()` and friends take a **tag** value, and the tags are listed in osfmk/mach/vm_statistics.h as `VM_KERN_MEMORY_*` constants (all `KERNEL_PRIVATE`, so they are not visible in the user mode header). Note, that some tags are used only in the context of `kalloc_tag[_bt]` calls, and will thus not be visible in region information. Table 12-17 highlights the tags and - more importantly - their callers:

Table 12-17: The `VM_KERN_MEMORY_*` tags in osfmk/kern/memory_statistics.h

#	VM_KERN_MEMORY_*	caller
0	_NONE	The kernel's own Mach-O is tagged this way
1	_OSFMK	Miscellaneous in osfmk/vm/*
2	_BSD	Temporary mappings for sysctl(8), Mach-O loading, and execve(2)
3	_IOKIT	`gIOKitPageableMaps`, serializer data, etc
4	_LIBKERN	tags used by libkern `kalloc_tag[_bt]` calls
5	_OSKEXT	OSKext structures (primarily, `gLoadedKextSummaries`, etc.
6	_KEXT	Loaded Kext Mach-Os (`__PRELINK_TEXT`)
7	_IPC	`ipc_kernel_map` and `ipc_kernel_copy_map`
8	_STACK	Thread stack space
9	_CPU	per-CPU data
10	_PMAP	MacOS: `pv_*_tables` (from `pmap_init()`
11	_PTE	Used as `vm_object` tag in `vm_page_wire/vm_page_insert_wired()`

Table 12-17: The VM_KERN_MEMORY_* tags in osfmk/kern/memory_statistics.h (cont.)

#	VM_KERN_MEMORY_*	caller
12	_ZONE	The kernel zone map
13	_KALLOC	The kalloc_map
14	_COMPRESSOR	The compressor_map
15	_COMPRESSED_DATA	Unused
16	_PHANTOM_CACHE	Ghost pages in phantom cache
17	_WAITQ	global_waitqs and (#if CONFIG_WAITQ_STATS) g_waitq_stats
18	_DIAG	kdebug & telemetry buffers etc.
19	_LOG	os_log kernel_firehose_addr
20	_FILE	Kernel UPLs, VFS buffers
21	_MBUF	The mbmap for allocating network mbufs
22	_UBC	Reserved for Unified Buffer Cache, but unused
23	_SECURITY	Apple Protect Pager, mac_wire, etc.
24	_MLOCK	mlock(2)ed and/or mach_vm_wire()d memory
25	_REASON	OSReason kcdata bufs (kalloc_tag[_bt] only)
26	_SKYWALK	Skywalk subsystem memory (arenas, etc)
27	_LTABLE	Darwin 18+: Lockless Link tables
28+	_DYNAMIC	Dynamic tags by specific kexts, first come first served

The Kernel Slide

The Kernel Address Space Layout Randomization (KASLR) was introduced in Darwin 12 in an effort to raise the bar on kernel exploitation. Determining the target address space is an important step in successfully overwriting memory or obtaining code execution. The idea behind KASLR, therefore, is to add a random **slide** value into the kernel base, so as to make what are otherwise fixed virtual memory addresses harder to determine, and thus exploit.

The kernel slide is set by the boot loader (EFI/iBoot), prior to loading the kernel. It can then be determined during kernel boot (in [i386/arm]_vm_init), by taking the fixed kernel address and subtracting it from the virtual base address passed in the Platform Expert's boot_args struct. It is then cached in the vm_kernel_slide global.

The kern.slide sysctl MIB is set to 1 if the kernel is slid, and the kas_info syscall (#439) returns the kernel slide value to user mode. On the *OS SECURE_KERNEL this is naturally unimplemented (ENOTSUP). In MacOS, it requires both root privileges and the agreement of the MACF policies hooking mac_system_check_kas_info(), which is enforced by the Sandbox.kext when SIP is enabled. If SIP is disabled, numerous ways of retrieving the value exist, as simple as a one-line DTrace script dumping the value from PE_state->kslide.

It is absolutely imperative to "unslide" any kernel addresses reported back to user mode through debugging interfaces. There are generally two methods of doing so. The first is simply unsliding - i.e. subtracting the vm_kernel_slide value, so the address returned is the same as can be found in the kernel's Mach-O. This is usually the case during backtraces, as it both serves to hide the slide and make the stack traces easy to symbolicate. The second is permuting the address, so that it remains unique but not easy to associate with its original value. This is the case when returning unique object addresses from zones or elsewhere in the kernel_map - for example the iin_objects of mach_port_space_info.

Two macros are commonly used - VM_KERNEL_UNSLIDE[_OR_PERM]. The latter acts unslides VM_KERNEL_IS_SLID addresses, and applies the permutation to other kernel addresses. The permutation is an arbitrary vm_kernel_addrperm value, read_random()ly during the kernel_bootstrap_thread(), and then added to the result of applying the VM_KERNEL_STRIP_PTR macro on the pointer (Unfortunately, the macro merely returns the original pointer). Darwin 17 and later add (but, as of yet, do not use) a third macro, VM_KERNEL_ADDRHASH, which uses a proper hash (presently, SHA-256), along with a much needed vm_kernel_addrhash_salt, also read_random()ly during startup.

Review Questions

1. Prior to Darwin 16, Apple tested other locations for the zone metadata, including putting it in the beginning and end of every page. What is an advantage and a disadvantage of the present solution?
2. What other, possibly simpler way, to get a zone element's metadata by its index, could you consider in place of the `PAGE_METADATA_FOR_PAGE_INDEX` macro? Why is a macro preferred?
3. Looking back at the footnote in the section discussing the zone metadata region, you will note that attempting to read the very beginning of the metadata region (which is also the very beginning of the zone map) will fail and is prone to panic on *OS through `mach_vm_read()` of the `kernel_task`. Why is that?
4. Following on the previous question, what is the formula to determine the first valid metadata entry in the `zone_metadata_region`, which is also safe to read from kernel memory? What is the minimal page index to which this applies?
5. What other concern would one encounter when trying to sequentially read kernel memory from the `zone_metadata_region`? How could that issue be solved?
6. What are the similarities and differences between the user mode magazine allocator and the new kernel mode zone allocator of Darwin 18?
7. Why is it absolutely vital to empty the `vm_object` after force-freeing its pages during `vm_object_purge()`?
8. Where are the **two(!) obvious(!!)** KASLR memory disclosures in `procexp 0 regions` in MacOS (at least up to 15)? Which one of those is (at least up to iOS 13, maybe later) in *OS as well?
9. Why is the salt an absolute requirement for the `VM_KERNEL_ADDRHASH` scenario?

13

All in the Family: IOKit

XNU's IOKit is a direct descendant of NeXTSTEP's original "DriverKit", and remains true to its origins. Built over libkern, IOKit offers a rich C++ class hierarchy allowing for rapid driver development by tapping into pre-defined families. No other modern kernel provides such a rich, nearly self-contained C++ programming environment and model for drivers, and nearly all of Apple's extensions make heavy use of it - as do exploiters.

Apple provided comprehensive documentation as to IOKit Fundamentals[1] and IOKit Device Driver Design Guidelines[2], and Guide to Accessing Hardware from Applications[3], but unfortunately retired these valuable references later on, leaving only the partial header documentation. These documents are archived in the book's companion website, and are a must read for any IOKit developer or reverse engineer.

The chapter begins with a high level view of IOKit, introducing the powerful features of the kernel mode architecture and its client user mode `IOKit.framework`. The two databases - the IORegistry and the IOCatalogue - are discussed next, detailing them both conceptually, and from an API perspective. The former is shown with its multiple traversal and enumeration APIs, and the latter, through its powerful matching APIs. All the above APIs, exported through the `IOKit.framework`, are part of the `device.defs` MIG subsystem, whose entire set of over five dozen routines is explored in this chapter.

We next turn our attention to `LibKern`, which provides the C++ runtime IOKit is so heavily dependent on. The `OS*` base classes and their structures are detailed, noting their APIs and, in particular, serialization. On top of these classes IOKit utilizes its own set of base classes, which are the ones built-in to the kernel, as well as the many **families**, from which specialized drivers may be derived.

Next, we pick up on where we left in Chapter 3, by revisiting the `Info.plist` of IOKit extensions, and their required `IOKitPersonalities` key. An optional aspect of a driver personality is the `IOCFPlugInTypes` key, which enables a kernel extension to provide a special user-mode plugin handler.

We continue our examination of IOKit drivers with a look at the driver lifecycle, beginning with the driver matching process and following with the driver workloop model, an event driven model not unlike the `CFRunLoop` we explored back in I/10. Penultimately, we take a deep and detailed look into **IOUserClients**, which enable the proprietary and the (some would say, too) rich interface drivers provide to user mode.

Lastly, we discuss the latest, and arguably the most important enhancement made in Darwin 19 - the new DriverKit. Briefly introduced in Chapter 3, DriverKit is revisited with an emphasis on its internals, providing what is likely to become the new standard for extension development - at least by third parties, as Apple is making concerted efforts to push them out of the kernel.

A High level view of IOKit

Although claiming throughout the IOKit Fundamentals documentation[1] that IOKit is its own original invention, it really isn't. Rather, it is a direct descendant of NeXTSTEP's **DriverKit**, which Apple acquired along with the rest of the architecture when the latter absorbed the former. The design, programming interfaces, coding conventions and more aspects remain virtually the same to this day, normal evolution notwithstanding.

Indeed, like the rest of Mach, IOKit remains avant garde to this very day. The only other runtime environment which is anywhere near comparable is the Network Driver Interface Specification (NDIS), though it is nowhere near the capabilities of IOKit, which include:

- **A self-contained programming environment:** IOKit provides its own APIs, some of which are adapters over those of the kernel, thus decoupling from the lower level implementation. In this way, the `IOLock*` APIs wrap those of `lck_mtx_*`, `IOMalloc` and `IOFree` wrap `kalloc_tag_bt` and `kfree`, and more abstractions such as `IO[CreateExit]Thread`, etc. In theory, this would have allowed for kernel extensions whose sole dependency is on IOKit (and possibly libkern) - thus achieve driver compatibility across operating systems (similar to Linux's NDISWrapper). In practice, however, this falls far short of achieving this promise: Not only has noone, to date, made attempts to port IOKit, but most IOKit extensions also use Mach and BSD KPIs directly.
- **A (restricted) C++ programming model:** Most kernels are implemented as a mix of C and low-level assembly, but IOKit (with the help of libkern) brings a restricted subset of C++ into the kernel. Not supported are exceptions, templates, and C++ Runtime Type Information (RTTI) - but other features are, and the programming model benefits greatly.
- **Object Orientation:** A key benefit of C++ is its object orientation. Drivers are objects, and can create more objects, with properties and methods which are accessible from user space. This alone provides a programming interface far richer than the classic UN*X device model, wherein user mode clients are limited to `read()/write()`, and `ioctl(2)` for everything else.
- **Inheritance:** A corollary of object orientation, inheritance allows a subclass to reuse, extend or override properties and methods of its superclass. The direct effect on IOKit drivers is a dramatic decrease in driver code size, along with a (general) improvement in code quality - rather than reimplementing common functions, those can be part of a superclass, which drivers can inherit from. These superclasses are referred to as **families**, and IOKit drivers can pick the family closest to them, implementing only the idiosyncratic features of the specific hardware.
- **Object and Driver Registry:** The drivers and any objects created by them are registered in two in-kernel databases. The first, called the **IOCatalog(ue)**, is used exclusively for keep track of known drivers. The second, called the **IORegistry** maintains a multi-planar, hierarchical searchable database which can be queried by matching dictionaries.

The IOKit.framework

Although IOKit is a kernel subsystem, it also has a user mode component - the public IOKit.framework. Just as the subsystem provides various KPIs, so does the framework provide APIs. The framework source remains open in the IOKitUser project, and a few command line tools (notably, the handy `ioreg(8)` utility) can be found in another project, IOKitTools.

The IOKit.framework provides all the necessary APIs for clients to connect, query and control the IOKit subsystem. The framework hides the underlying calls to kernel mode, which are conveyed using the MIG routines of the `iokit` subsystem, <device/device.defs>. The subsystem, however, is unusual in several ways. Not only does it export a large number of routines (over 80) which we explore throughout this chapter, but the different routines use different port types.

The "port of departure" is the **IO Master Port**. A SEND right to the master port can be obtained in several ways - through host special port #3, or through the `mach_host`'s subsystem `host_get_io_master`. The IOKit.framework wraps the latter in its `IOMasterPort()`. Another option is to use the framework defined `kIOMasterPortDefault` (essentially, NULL), which causes an internal framework lookup. The master port is thus a Mach primitive port, similar in idea to the host, task, thread and bootstrap ports - always readily available for retrieval by any task. It is created by `device_service_create()` (from osfmk/device/device_init.c), called from the `kernel_bootstrap_thread`.

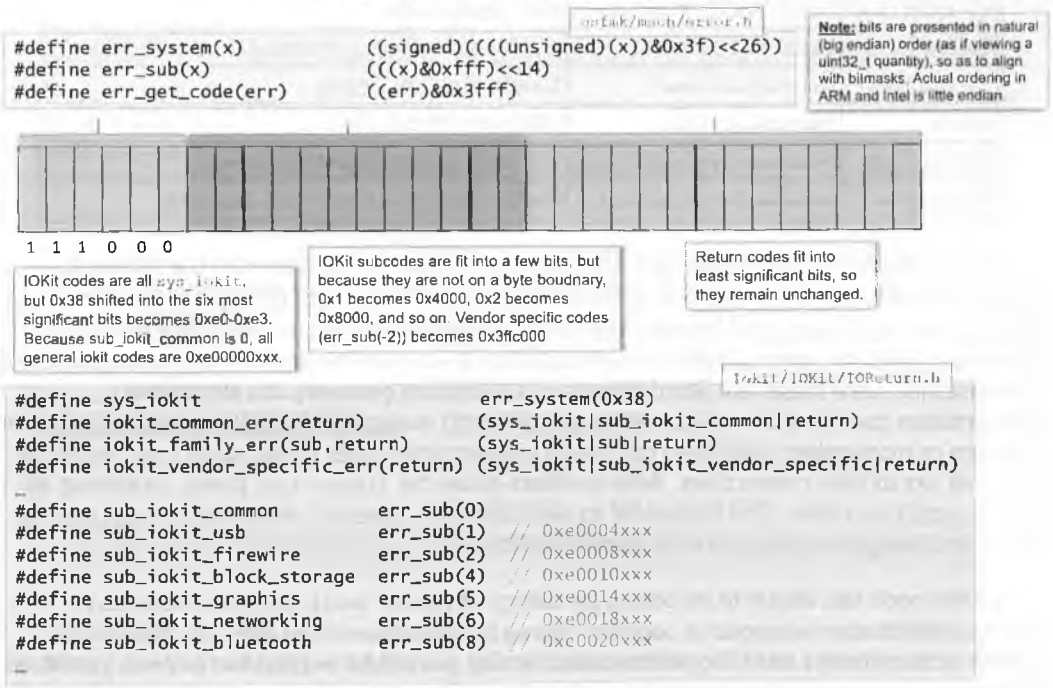
The master port provides an interface to the IORegistry and the IOCatalog. It is through these two dynamic, in-kernel data structures, that the various driver objects can be looked up and contacted. Objects in kernel are represented by an `io_object_t`, and this port type (#30 in Table 10-8, `IKOT_IOKIT_OBJECT`) is used for registry entries, iterators, or unconnected driver handles. When the service handles are connected, the in kernel object providing the services - the `IOUserClient` class - appears as an `io_connect_t` (#29 in Table 10-8, `IKOT_IOKIT_CONNECT`), and through these ports specific driver methods may be invoked. The routine prefix (`io_[registry/catalog]`, `io_object`, `io_connect` or `io_service`) indicates its required port type, and IOKitLib.h further aliases corresponding types over the `io_object_t` (i.e. `io_[registry_entry/iterator/service]_t`).

Another communication channel between user and kernel is via the specialized `iokit_user_client_trap` (#100). This trap can only be used if its first argument is an `io_connect_t` (i.e. a valid `IOUserClient` instance in kernel), and provides a faster way than MIG of calling driver methods. The trap accepts up to six arguments, and IOKit.framework provides `IOConnectTrap[0-6]` to handle variable arguments.

IOKit error codes

MIG routines are expected to return `KERN_SUCCESS` (0) if all goes well, but if anything can go wrong, it often will. IOKit defines an error code convention in iokit/IOKit/IOReturn.h through a set of macros (built on those of osfmk/mach/error.h) which allow to define error classes and specific codes:

Figure 13-1: Constructing IOKit return codes



Additional error codes are defined in iokit/IOKIT/IOMessage.h, as `kIOMessage*` constants. The `IOService::stringFromReturn` method (in iokit/Kernel/IOService.cpp) contains a hard-coded `return_values` table stringifying some of these error codes.

The IORegistry

IOKit maintains an extensive data structure called the **IORegistry**. Bearing only a rough resemblance to the Windows Registry, the IORegistry is a far more advanced searchable repository, which is used to keep track of objects, their properties, and interrelations, represented as a tree (i.e. a direct, acyclical graph). The `IORegistryExplorer.app` (from XCode's Hardware Tools) or the `ioreg(8)` utility (built-in across all Darwin flavors) can be used to inspect the registry from user mode.

All driver objects (instances or descendants of `IOService`) inherit directly from the `IORegistryEntry` class, which means that they all automatically appear in the IORegistry, in the `IOService` plane. The IORegistry, however, is a multiplanar data structure. Additional planes exist, which capture the interrelationships of the objects in them, and each plane presents its own perspective of the registry. Plane views may be changed by using `IORegistryExplorer.app` list box, or specified with the `-p` argument to `ioreg(8)`. Note, that the `IOService` plane shows all *driver objects*, but not all objects of all types. The `IODeviceTree` plane, for example, contains the full device tree, after it has been populated by iBoot and XNU (that is, including data from SysCfg). The `IODeviceTree` registry objects correspond to device tree nodes (and likewise, so do the properties), but as these are not driver created objects, none of them appear in `IOService`.

The list of planes can be retrieved from the `IORegistryPlanes` property of the root object (programmatically, or by using `ioreg -l -w 0 -f | grep IORegistryPlanes`). Certain planes are "out-of-box" (created by IOKit's core), but new planes can be created programmatically using `IORegistryEntry::makePlane(char const *name)` (from IOKit/IORegistryEntry.h). MacOS and iOS have slightly different planes (which have changed over time to accommodate different hardware buses), shown in Table 13-2:

13-2: The IORegistry planes

Plane name	Creator	Contains
`IOService`	(IOService.cpp)	All `IOService` objects
`IOPower`		Power management clients and/or interested parties
`IODeviceTree`	(IODeviceTreeSupport.cpp)	All device tree nodes (passed to XNU from firmware)
`IO80211Plane`	`IO80211Family[V2].kext`	Wi-Fi related kexts
`IOACPIPlane`	`IOACPIFamily.kext`	MacOS: Advanced Configuration & Power Interface
`CoreCapture`	`corecapture.kext`	CoreCapture WiFi logging
`IOUSB`	`IOUSBFamily.kext`	USB bus & devices
`IOAccessory`	`IOAccessoryManager.kext`	iOS: Accessories plugged into iDevice
`IOFireWire`	`IOFireWireFamily.kext`	MacOS: Firewire bus & devices (obsolete)

Here is where it gets complicated: Objects may or may not be visible on the different planes, and their interconnections may be different, resulting in quite different topologies. The official Apple documentation[1] depicts the object nodes appearing as rods, which may or may not intersect with the planes. While this visualization is correct, it is somewhat challenging due to its infeasibility, were these real world objects in a Euclidean geometry. An alternative representation can be of a multi-colored graph, wherein nodes may be colored simultaneously in one more or more colors. With the right tinted glasses, only nodes of the same color would be visible, as would their connections. With no filters at all, the `IOService` plane, containing all objects, would be visible. The equivalent to wearing these "glasses", would be `ioreg(8)`'s `-p` switch, or `IORegistryExplorer`'s plane selection box.

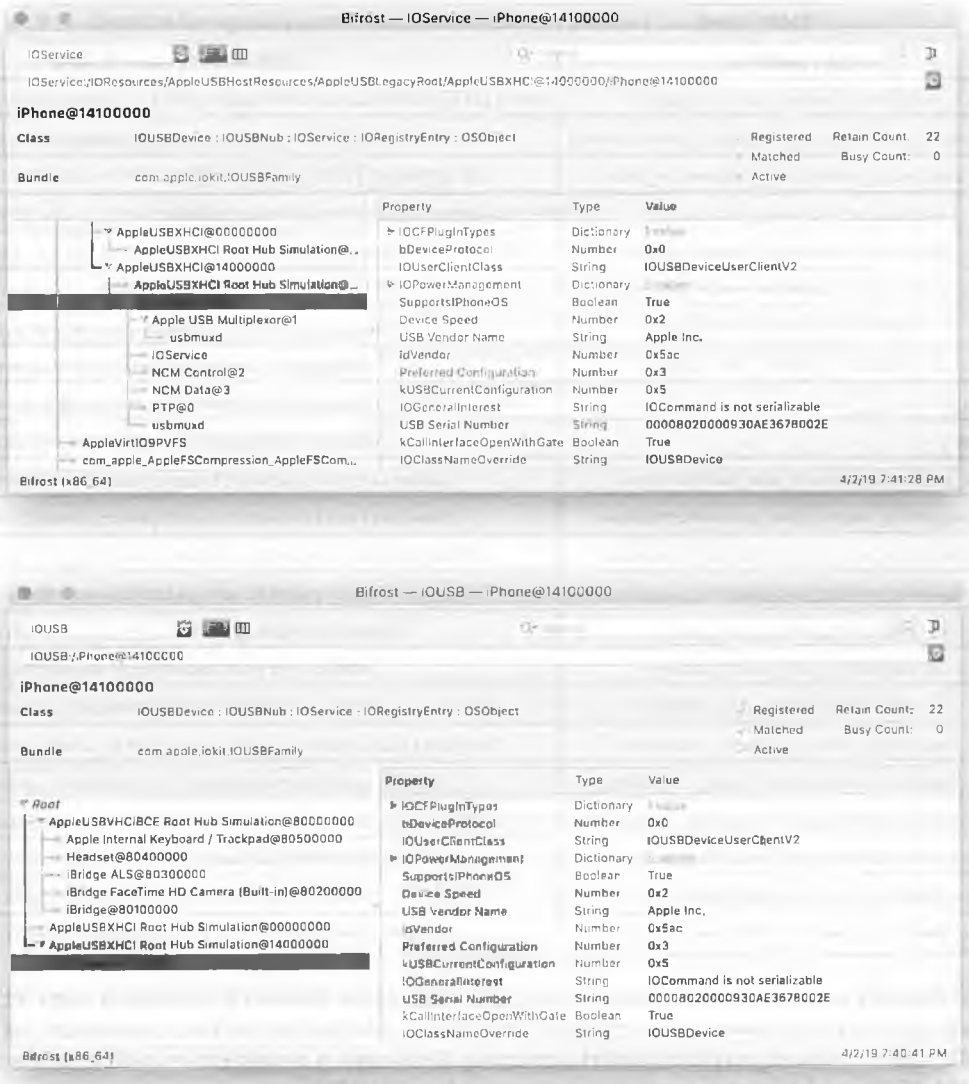
A child node can attach to its parent by calling `virtual bool attachToParent( IORegistryEntry * parent, const IORegistryPlane * plane );`. This, in turn, triggers `attachToChild` in the parent node. Similar `detachFrom[Child/Parent]` exist, as well as `detachAbove()` to disconnect a child from all its parents, or `detachAll`, to completely sever the node from all attachments - parents and children. A node's primary attachments can be retrieved using calls to `copy[Parent/Child]Entry`, and for multigenerational hierarchy, a path between an ancestor and its descendant can be found using `getPath`.

Experiment: Navigating the IORegistry with `IORegistryExplorer`

The `IORegistryExplorer.app`, found in XCode's Additional Tools, is alone well worth the download of this important add-on pack. Although `ioreg(8)` does have its advantages (specifically, availability for iOS and `grep(1)`-friendly CLI), the rich GUI of `IORegistryExplorer` and its capability to quickly shift plane perspectives makes it a valuable tool in navigating through the complex hierarchies of the IORegistry.

Figure 13-3 shows the same object - specifically, a connected iPhone, first through the eyes of the `IOService` plane, and then through `IOUSB`. The latter plane is not as populated, so the path to the iPhone is straightforward, whereas the `IOService` plane path is much deeper. Additionally, the `IOService` plane offers better visibility into subclasses instantiated by the iPhone - for Picture Transfer Protocol, Network Control Mode (tethering), etc - which are not visible through the `IOUSB` perspective.

Figure 13-3: The view of a connected iPhone object through the `IOService` and `IOUSB` planes



Another really useful feature is `IORegistryExplorer`'s responsiveness to IOKit notifications. The mechanism supporting this is described later, but the salient point is that the GUI will automatically refresh, displaying newly detected/connected devices in green, and removed ones in red. This can make it much easier to quickly pinpoint hardware of interest.

User Mode APIs

One of the most powerful aspects of maintaining the IORegistry is the rich set of APIs which are available to interface with it from user mode. These are provided by the IOKit.framework and are well documented by Apple, if only because they are vital for nearly any driver access. The APIs include functions to traverse the IORegistry, locate specific driver objects by various matching criteria, and get/set properties in the objects, i.e. query of affect driver operation from user mode.

The IOKit.framework `IORegistry*` APIs wrap MIG messages, which are a subset of the IOKit subsystem, in <device/device.defs>. The MIG messages are meant to for use with the aforementioned **IOMasterPort**. The .defs file is conditional on `#if IOKIT`, so this should be `#defined` before applying the `mig(1)` preprocessor. Most of the wrappers are thin, calling the underlying MIG directly with the same arguments, with some convenience functions combining the iterator retrieval with an `IOIteratorNext()` so as to operate on its first entry. Table 13-4 shows the relevant MIG subsystem messages, along with their mappings to the framework APIs:

Table 13-4: `io_registry_*` MIG calls and their `IORegistry*` counterparts

#	MIG routine	IOKit.framework API
2805	`io_registry_entry_get_property`	`IORegistryEntrySearchCFProperty(entry,plane,key,allocator,options)`
2855	`io_registry_entry_get_property_recursively`	As previous, with `kIORegistryIterateRecursively` in *options*
2879	`io_registry_entry_get_property_bin`	As previous (`IOKIT_SERVER_VERSION >= 20140421`, Darwin 14)
2806	`io_registry_create_iterator`	`IORegistryCreateIterator(masterPort,plane,options,iterator)`
2807	`io_registry_iterator_enter_entry`	`IORegistryIteratorEnterEntry(iterator)`
2808	`io_registry_iterator_exit_entry`	`IORegistryIteratorExitEntry(iterator)`
2809	`io_registry_entry_from_path`	`IORegistryEntryFromPath(masterPort,path)`
2810	`io_registry_entry_get_name`	`IORegistryEntryGetName(entry,name)`
2811	`io_registry_entry_get_properties`	`IORegistryEntryCreateCFProperties(entry,props,allocator,unused)`
2878	`io_registry_entry_get_properties_bin`	as previous (`IOKIT_SERVER_VERSION >= 20140421`)
2828	`io_registry_entry_set_properties`	`IORegistryEntrySetCFProperties(entry,properties)`
2812	`io_registry_entry_get_property_bytes`	`IORegistryEntryGetProperty(entry,name,buffer,size)`
2813	`io_registry_entry_get_child_iterator`	`IORegistryEntryGetChildIterator(entry,plane,iterator)`
2814	`io_registry_entry_get_parent_iterator`	`IORegistryEntryGetParentIterator(entry,plane,iterator)`
2826	`io_registry_entry_get_path`	`IORegistryEntryGetPath(entry,plane,path)`
2827	`io_registry_get_root_entry`	`IORegistryGetRootEntry(masterPort)`
2829	`io_registry_entry_in_plane`	`IORegistryEntryInPlane(entry,plane)`
2833	`io_registry_entry_create_iterator`	`IORegistryEntryCreateIterator(entry,plane,options,iterator)`
2843	`io_registry_entry_get_name_in_plane`	`IORegistryEntryGetNameInPlane(entry,plane,name)`
2854	`io_registry_entry_get_location_in_plane`	`IORegistryEntryGetLocationInPlane(entry,plane,location)`
2871	`io_registry_entry_get_registry_entry_id`	`IORegistryEntryGetRegistryEntryID(entry,entryID)`
2885	`io_registry_entry_get_path_ool`	`IORegistryEntryCopyPath(entry,plane)`
2886	`io_registry_entry_from_path_ool`	`IORegistryEntryCopyFromPath(masterPort,port)`

IORegistry access was unfettered for decades, until Apple realized it provides a huge attack surface against kernel drivers. This changed with the introduction of the MAC Framework, as specific `iokit_check_*` hooks started appearing. First came a restriction on setting properties (in XNU-1699), and then one on getting properties (in XNU 2782).

Iterators

An important object when dealing with the registry is the `IOIterator`. This is an object which can be created directly from user mode by a call to `IORegistryCreateIterator()` over a plane, or `IORegistryEntryGet[Child/Parent]Iterator()` using an entry. The iterator serves as a cursor to hold state between subseqent IORegistry calls, so that plane traversal can proceed sequentially. The normal operation with an `IOIterator` is forward access using `IOIteratorNext()`, which retrieves the next registry entry (an `io_object_t`), and advances the iterator, until it returns NULL.

A full open source code sample of IORegistry traversal can be found in `ioreg(8)`, or the iodisplayregistry.c file, both in the IOKitUser project. The relevant, abbreviated code is shown in Listing 13-5:

Listing 13-5: Using registry iterators

```
    kern_return_t status     = IOMasterPort(bootstrap_port, &masterPort);
    io_registry_entry_t root = IORegistryGetRootEntry(masterPort);

    traverse(options, plane, 0, root, 0);

...

void traverse(unsigned int options,
              io_name_t plane,
              io_iterator_t services,
              io_registry_entry_t serviceUpNext,
              int depth)
{
    io_registry_entry_t service;
    io_iterator_t children;

    // We loop for every service in the list of services provided.

    while ( (service = serviceUpNext) ) {

        serviceUpNext = IOIteratorNext(services);

        status = IORegistryEntryGetChildIterator(service,
                                                 plane,
                                                 &children);

                ..
        child = IOIteratorNext(children);


        /* Recurse */

        traverse(options, plane, children, child, depth + 1, stackOfBits);

        // Release resources.

        IOObjectRelease(children); children = NULL;
        IOObjectRelease(service);  service  = NULL;
    }

}
```

As with other IOKit.framework APIs, `IOIteratorNext()` is implemented directly over a MIG call. There are also calls reset and check the validity of a given iterator, as shown in Table 13-6:

Table 13-6: The `io_iterator` related MIG calls, and their `IOIterator` counterparts

#	MIG routine	IOKit.framework API
2802	`io_iterator_next`	`IOIteratorNext()`
2803	`io_iterator_reset`	`IOIteratorReset()`
2834	`io_iterator_is_valid`	`IOIteratorIsValid()`

In kernel, the iterator is a port reference, to a port of type `IKOT_IOKIT_OBJECT`. The port's `kdata` is an `IOIterator` kernel object which, despite the name similarity, is not to be confused with the user mode object, but is a wrapper over libkern's `OSCollectionIterator` (as discussed later).

The IOCatalog(ue)

The IORegistry may be the more well known database, but the **IOCatalogue** is just as important - if not more so. The IOCatalogue is used to store the personalities of drivers, by copying the `IOKitPersonalities` dictionary from their kext's Info.plist. The catalogue is a singleton, exported as `gIOCatalogue` (and protected by `gIOCatalogueLock`). Although its contents somewhat intersect with those of the IORegistry, this is a different dynamic database - holding driver entries, rather than `IORegistryEntry` types.

Each driver entry in the `IOCatalogue` is an array of its registered `IOKitPersonalities` dictionaries (from its Info.plist), and multiple driver entries are grouped by their corresponding **nub**s. A "nub", in IOKit parlance, refers to a class representing the attachment point of a driver - usually the underlying bus (PCI, Thunderbolt, AGP, etc), or some other device connector. Drivers declare their nub in the `IOProviderClass` key of their personality. If not defined (or in the case of logical/virtual device drivers with no underlying hardware, such as filesystems), the `IOResources` nub is used.

The `IOCatalogue` is, thus, a dictionary of nub keys (unique `IOProviderClass` values), whose entries are arrays of potentially matching driver personalities, with each personality a dictionary (containing the matching criteria). A visualization of this massively nested structure is shown in Figure 13-8 (next page).

When the system is started no drivers as loaded, and the `IOCatalogue` needs to be bootstrapped, which is performed by calling `IOCatalogue::initialize()` from the `iokit_post_constructor_init()`. This class method uses a hardcoded JSON string containing the `IOPanicPlatform` entry (from iokit/Kernel/IOPlatformExpert.cpp), which matches the IOPlatformExpertDevice nub. If the real mtch cannot be found (to serve as the IORegistry root), this `panic()`s the system.

The IOCatalogue is accessible from user mode through IOKitLib.h's functions. These are conveyed to kernel over IOKit (device.defs) MIG calls (using the American spelling, catalog). These are shown in Table 13-7. The IOKit.framework also provides two other useful wrappers, `OSKextSendKextPersonalitiesToKernel` and `__OSKextRemovePersonalities`, both over `io_catalog_send_data`.

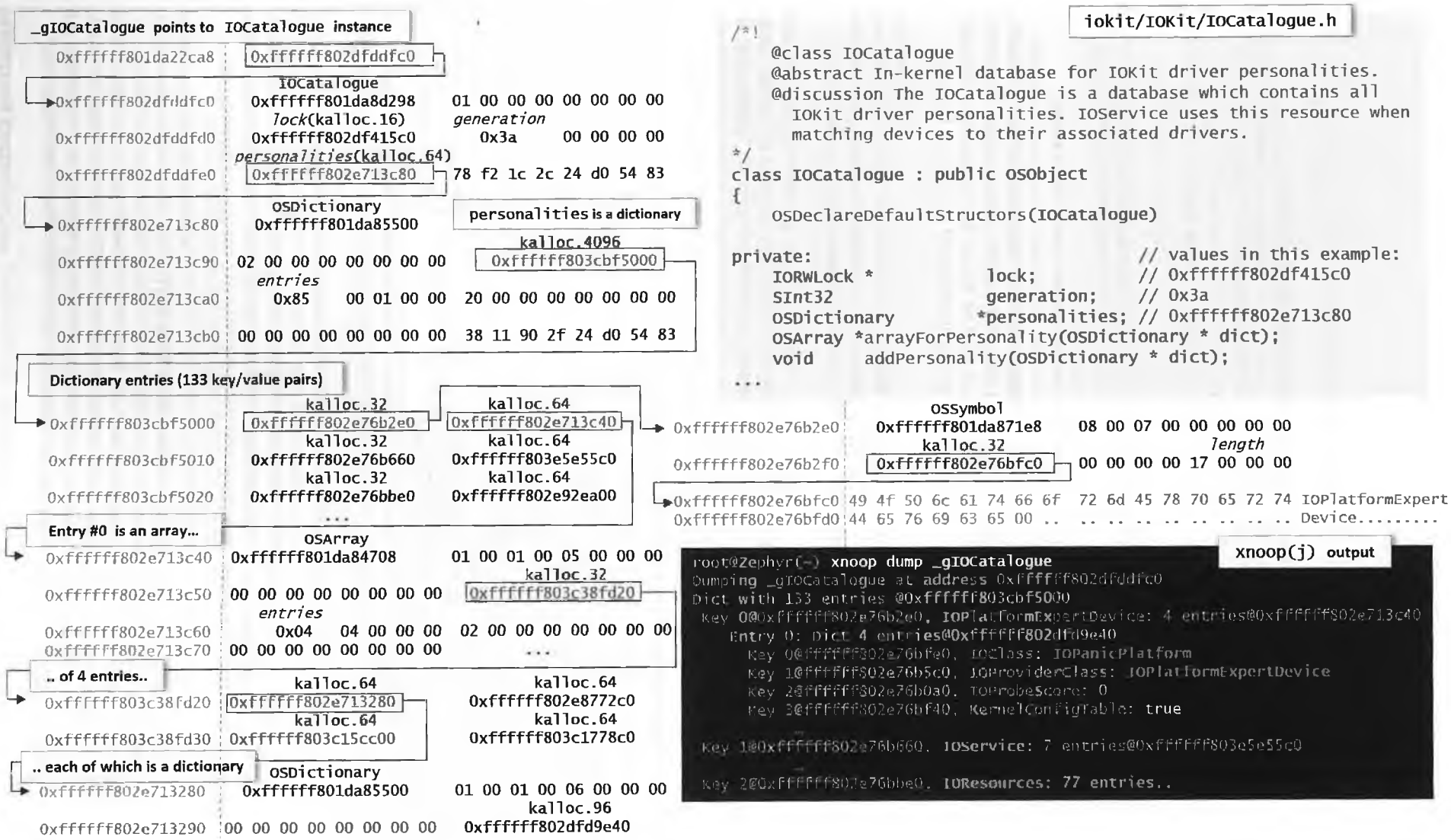
Table 13-7: `io_catalog_*` MIG calls and their `IO*` counterparts

#	device.defs MIG	IOKitLib.h API	Purpose
2836	`io_catalog_send_data`	`IOCatalogueSendData`	Update catalog with new or removed `IOKitPersonality` dictionaries
2837	`io_catalog_terminate`	`IOCatalogueTerminate`	Terminate drivers on module unloading
2838	`io_catalog_get_data`	`IOCatalogueGetData`	Unsupported
2839	`io_catalog_get_gen_count`	`IOCatalogueGetGenCount`	Get catalog generational count
2840	`io_catalog_module_loaded`	`IOCatalogueModuleLoaded`	Inform catalogue module has loaded
2841	`io_catalog_reset`	`IOCatalogueReset`	Reset and recreate catalogue

The catalog calls are predominantly used by the kext_tools binaries - `kextutil`, `kextunload`, and of course `kextd(8)` when kernel extensions are added and removed. These allow the updating of the catalog, starting the internal driver matching process (described later in this chapter) and possibly trigger a rebuild of the prelinked kernel by `kextd(8)`.

As of Darwin 18, `io_catalog_send_data` requires the `com.apple.rootless.kext-secure-management` entitlement, and its body is compiled out of *OS (due to `NO_KEXTD`). Also unsupported on *OS is `io_catalog_terminate`, due to `SECURE_KERNEL`. Finally, `io_catalog_get_data` still exists, but its internal call, `IOCatalogueSerializeData()` (in iokit/Kernel/IOCatalogue.cpp) is no longer supported in any flavor as of XNU-2050, which changed the internal format of the catalogue from an array to its present form. There is, therefore, no API to view the catalog directly from user mode.

Figure 13-8: The gIOCatalogue, visualized



Matching Dictionaries

The IORegistry contains thousands of entries, and while it is certainly possible to traverse the registry by enumerating it (starting with `IORegistryGetRootEntry` and recursing (through `IORegistryEntryGetChildIterator()`), or pinpointing a known entry (using `IORegistryEntryFromPath()`), a more common scenario is searching for specific drivers, i.e. `IOService` objects. The `IOService*` APIs of the IOKit.framework offer this important functionality through the use of **Matching Dictionaries** - which is the main functionality provided by the IOCatalogue.

Matching dictionaries are `CFMutableDictionary` objects, whose keys and values specify registry entry properties, and their desired values (respectively). In this way, a user mode caller can construct criteria as general or as specific as necessary. The dictionaries are created using one of several `IO*Matching()` variants, and then serialized (using `IOCFSerialize()`) into their XML representation, and passed on to the kernel using one of the matching calls:

Table 13-9: The `io_service_get_matching*` related MIG calls, and their `IOService*` counterparts

#	io_service.. routine	IOService.. API	Purpose
2804	.._get_matching_services	..GetMatchingServices()	Perform matching for multiple drivers. Removed as of Darwin 14
2881	.._get_matching_services_bin		As previous (`IOKIT_SERVER_VERSION >= 20140421`, Darwin 14)
2857	.._get_matching_services_ool		As above, for large dictionary sizes
2873	.._get_matching_service	..GetMatchingService()	Perform matching for multiple drivers. Removed as of Darwin 14
2880	.._get_matching_service_bin		As previous (`IOKIT_SERVER_VERSION >= 20140421`, Darwin 14)
2874	.._get_matching_service_ool		As above, for large dictionary sizes
2844	.._match_property_table	..MatchPropertyTable	Check if an `io_service_t` matches supplied properties
2882	.._match_property_table_bin		
2858	.._match_property_table_ool		

In kernel mode, the XML dictionary is deserialized into an `OSDictionary` using `OSUnserializeXML()` (discussed later in this chapter, under "Serialization"). In the case of services sought, the `OSDictionary` is passed to `IOService::getMatchingServices()`, which returns an `IOUserIterator`, which appears in user mode as an `io_object_t` Mach port, and can be used with IORegistry APIs.

In the case of a single service, `IOService::copyMatchingService()` is called, and returns an `IOService` instance, in user mode is another `io_object_t` Mach port, which can then be turned to an `io_connect_t` through `IOServiceOpen()`, as explained later under "IOUserClients".

Matching can also be performed in kernel mode, which is useful in cases when one kernel extension would like to call on the services of some other IOKit kernel extension. A prime example of that is the way Sandbox.kext calls its partner in crime, AppleMobileFileIntegrity.kext. The listing below demonstrates the decompilation of Sandbox 851:

Listing 13-10: Simple matching example, using `IOService` class methods, in kernel mode

```
...
   OSDictionary *amfiDict =
      IOService::nameMatching("AppleMobileFileIntegrity",    // char const *
                                                          )  // OSDictionary *
   if (amfiDict == NULL) {
        panic ("\"%s: failed to allocating matching dict\"@.......entitlements.cpp:298",
               "getAMFI");
        }

         x =  IOService::waitForMatchingService(dict, // OSDictionary*,
                                                -1);  // unsigned long long)
        OSMetaClassBase::safeMetaCast(OSMetaClassBase const*, OSMetaClass const*);

        if (....) {
      panic ("\"%s: failed to lookup AMFI service\n"@.......entitlements.cpp:303",
               "getAMFI");

        }
        ...
```

Experiment: Working With Matching Dictionaries

In order to locate any driver object, a matching dictionary must be constructed. This is such common practice, that IOKit.framework's `IOServiceMatching` API can be used for this, supplying only the name of the driver object in question, which then populates the value of an `IOProviderClass` key, since this is a mandatory key required of all IOKit driver personalities. Other variants are `IOServiceNameMatching` (to match by "IOName"), or `IOBSDNameMatching`, to match the "BSD Name" key to a traditional UN*X device name (e.g. "en0", "disk0", etc). It is also possible to use `registryEntryIDMatching` to create a matching dictionary for an `IOService` instance matching the specified entry ID.

You can find plenty of examples for XML-encoded matching dictionaries in various IOKit kexts' `IOKitPersonalitiesLaunchEvents` specifying `com.apple.iokit.matching`.

In cases where only one match is expected or required, the matching dictionary created is supplied as the second argument to `IOServiceGetMatchingService`. Otherwise, `IOServiceGetMatchingServices` is used instead, and provides an `IOIterator` which can be used to enumerate the matches. The following code provides a working sample:

Listing 13-11: Simple matching example

```
int main (int argc, char **argv) {
  mach_port_t  ioMasterPort;
  kern_return_t  kr = IOMasterPort(MACH_PORT_NULL, &ioMasterPort);

  if (kr != KERN_SUCCESS) {  /* All hope is lost.. fail miserably */ }

  io_iterator_t  ioIter; io_object_t ioObj;

  CFDictionaryRef matchingDict = IOServiceMatching(argv[1]);
  IOServiceGetMatchingServices(ioMasterPort,matchingDict,&ioIter);
  char nameBuf[256]; char pathBuf[1024];

  io_object_t ioServiceMatched = MACH_PORT_NULL;
  while((ioObj = IOIteratorNext(ioIter))) {

   kr = IORegistryEntryGetNameInPlane(ioObj, // io_registry_entry_t ent,
          kIOServicePlane,                   // const io_name_t  plane,
          nameBuf);                          // io_name_t        name);

   kr = IORegistryEntryGetPath(ioObj,        // io_registry_entry_t ent,
          kIOServicePlane,                   // const io_name_t  plane,
          pathBuf);                          // io_string_t      path);

        printf("Name: %s (%s)\n", nameBuf,pathBuf);
  }
}
```

Note the use of `IORegistryEntry*` APIs, to provide the full name and path of the matched object. Thus, catalog entries can freely be "converted" to registry entries. You are encouraged to try this with various inputs - `IOService` will provide a plethora of matches (essentially matching everything).

In cases where multiple matches exist, the caller may also opt to refine the matching dictionary to be as specific as possible. This can be done by adding keys and values to the dictionary, using `CFDictionarySetValue()` calls.

Useful keys are `kIOPropertyMatchKey` (matching on specific property key/values, in the provided nested dictionary), `kIOLocationMatchKey`, `kIOPathMatchKey` and `kIOResourceMatchKey`. These can also be used for wider searches, in cases where the class or object name are not known. The full list of `kIO*Key` identifiers is spread throughout IOKit's headers, with most of the important ones in iokit/IOKit/IOKitKeys.h.

Notifications

One of the most useful APIs provided to user mode are the notification related APIs. These allow an interested client to request notifications when the IORegistry changes: per a matching dictionary supplied and a `kIO...Notification` constant (`Publish`, `FirstPublish`, `Matched`, `FirstMatch` or `Terminated`).

The notifications require a client to create a port (using `IONotificationPortCreate()`, which `mach_port_allocate()`s an ephemeral `RECEIVE` right), and then hand that port to a registry entry (or IOService handle), using one of the calls in Table 13-12:

Table 13-12: The `io_service*` notification related MIG calls, and their `IOService*` counterparts

#	io_service_.. MIG routine	IOKit.framework API	Purpose
2849,68	`.._add_notification[_64]`	`IOServiceAdd[Matching]Notification`	Get notification on device arrival/removal
2859,70	`.._add_notification_ool[_64]`		
2883,84	`.._add_notification_bin[_64]`	--	
2850,69	`.._add_interest_notification[_64]`	`IOServiceAddInterestNotification`	Get notification on state changes in an IOService
2851	`.._acknowledge_notification`	---	`kIOReturnUnsupported` by default

The `..add_interest_notification` expects an interest type, which is one of the codes shown below. When using `kIOGeneralInterest`, the callback is invoked with an additional `messageType`, which is commonly one of the `kIOMessage*` constants in iokit/IOKit/IOMessage.h, which are defined using the `iokit_common_msg` macro, taking the message id and concatenating it with `sys_iokit` and `sub_iokit_common`. The `sys_iokit` value is itself defined as `err_system(0x38)` (Figure 13-1), so that message codes are 0xe000[0-3]xxx. Families can similarly use codes, and drivers may use this for any custom notification, as well.

Table 13-13: The `kIO*` notification types

Interest Type	Notifier	Notification messages
`kIOGeneralInterest`	...	Any notification type
`kIOBusyInterest`	`IOService::_adjustBusy()`	Attach, detach and busy state
`kIOAppPowerStateInterest`	`IOService::tellClients[WithResponse]()`	Power state events
`kIOPriorityPowerStateInterest`	`IOService::tellClientsWithResponse()`	

The main user of the IOKit notification mechanism is `UserEventAgent(8)`, which cooperates with `launchd(8)` to bring the IORegistry's already powerful plug-and-play functionality to the hands of the system administrator, who can use the LaunchDaemon and LaunchAgent property lists to start daemons based on hardware events, and the developer, who can respond to hardware changes in the application.

1. `UserEventAgent`'s `com.apple.iokit.matching` plugin calls `IONotificationPortCreate` and then creates an eponymous `xpc_event_provider` in its entry point (`_init_iokit_matching_module`).
2. The callback for the `xpc_event_provider` is fired when `launchd(8)` parses a LaunchDaemon or LaunchAgent property list `LaunchEvents` dictionary, and finds the `com.apple.iokit.matching` key, as discussed in I/14.
3. The `com.apple.iokit.matching` key's value is a matching dictionary, as shown in the first experiment. The matching dictionary, is passed through the XPC event API to the plugin, which then relays it as the matching dictionary argument to `IOServiceAddMatchingNotification()`.
4. The callback specified by the plugin (invoked when a notification arrives on the port due to the matching dictionary being matched in kernel) calls `xpc_event_provider_token_fire()` to notify `launchd(8)`, which then launches the corresponding daemon.
5. The plugin also calls `IOServiceAddInterestNotification()` specifying `kIOGeneralInterest` to follow the object lifetime in kernel.

Experiment: IOKit matching notifications

IOKit notifications make it easy to create a crude but working "ioregmon" in just a few lines of code. The `IOKitUser` project contains a sample `ionotifyCB.c` to do just that, which can be easily extended. The relevant code is shown here. Trying this with `IOService` will provide plenty of output, but you can get specific .

Listing 13-14-a: A general interest notification handler

```
IONotificationPortRef notifyPort;

void callback(void *refcon, io_iterator_t iterator)
{
   io_string_t path; io_object_t        obj;
   mach_port_t notifyHandle;
   while( (obj = IOIteratorNext( iterator))) {
        io_name_t               name;
        kern_return_t kr = IORegistryEntryGetName( obj, name );
        printf("refcon: 0x%llx, name:%s\n", refcon,name);
        // Can add interest callback, too:
        kr = IOServiceAddInterestNotification(notifyPort,
                        obj, kIOGeneralInterest,
                        generalInterestCallback, (void *) masterPort,
                        &notifyHandle);
        IOObjectRelease(obj);
        }
}

int main(int argc, char **argv) {
  mach_port_t  ioMasterPort;
  kern_return_t  kr = IOMasterPort(MACH_PORT_NULL, &ioMasterPort);
  notifyPort = IONotificationPortCreate(ioMasterPort);

  if (kr != KERN_SUCCESS || !notifyPort) { /* All hope is lost.. AAAAHH */ };

  io_iterator_t notification;
  kr =  IOServiceAddMatchingNotification(notifyPort, // IONotificationPortRef ..
        kIOMatchedNotification,           // const io_name_t notificationType,
        IOServiceMatching(argv[1]),                 // CFDictionaryRef matching,
        callback,                        // IOServiceMatchingCallback callback,
        (void *) 0x12345,                                    // void *refCon,
        &notification);                          // io_iterator_t *notification);

  // Get existing devices and arm notification for new ones.
  // Using different refcon value for existing devices, since port ref is global.
  callback(NULL, notification);

  CFRunLoopAddSource(CFRunLoopGetCurrent(),
                     IONotificationPortGetRunLoopSource(notifyPort),
                     kCFRunLoopDefaultMode);
  CFRunLoopRun()   // wait.
}
```

When registering an interest, the interest notification callback will normally `switch()` on whichever message type is expected. The following example shows the commonly supported types:

Listing 13-14-b: An example interest notification callback routine

```
static void generalInterestNotificationCallback(void *          refCon,
                                        io_service_t    service,
                                        natural_t       messageType,
                                        void *          messageArgument) {
        switch (messageType)
        {
                case  kIOMessageServiceIsTerminated:
                case  kIOMessageServiceIsSuspended:
                case  kIOMessageServiceIsResumed:
                case  kIOMessageServiceIsRequestingClose:
                case  kIOMessageServiceIsAttemptingOpen:
                case  kIOMessageServiceWasClosed:
                        /* lifecycle messages */
                default:
                        /* Unexpected/driver-specific message type .. */
        }
}
```

Interlude: Libkern Base Classes

IOKit is implemented in C++, and so it draws on the libkern for runtime services. IOKit classes form a rich and complex hierarchy, over which properties and methods are inherited and extended. The base classes are those from which everything else derives. Technically, however, the primitive root classes as well as the boxed data belong to libkern.

OSObject

Just as with user mode, wherein everything inherits from `NSObject`, so too in libkern, the `OSObject` class serves as ancestor. The class is simple, offering little more than reference counting and some base methods, most of which are meant to be overridden. There are no direct instances of `OSObject`, and all other objects inherit from it either directly or indirectly.

A corollary of inheritance is that all Libkern (and, by further derivation, IOKit) objects all start with the same `OSObject` 12-byte "header" when in memory. This header is identifiable by its first field - a pointer, followed by an `uint32_t` tracking the reference count of the object. The pointer field directs to 16 bytes inside to the object's vtable. This address will point to the first non NULL vtable entry, holding the object's destructor method - OSObject's (`__ZN8OSObjectD1Ev`), or a one overridden by a subclass. This makes for a good tactic for identifying objects in a memory dump, since all object instances of the same class will have the same value in this field. Following this will be fields added by subclasses, in order of inheritance, and accounting for possible padding for alignment.

OSMetaClass[Base]

Although `OSObject` is, for most intents and purposes, the root class of all other objects, it, too, is a derived class. The true root of all classes is `OSMetaClassBase`. Though not meant for direct use, `OSMetaClassBase` provides an important functionality of the libkern runtime - that of RunTime Type Information (RTTI). The `OSMetaClassBase` is abstract, and though it implements type casting methods, other methods are virtual. The `.h` file also defines `OSMetaClass` as a concrete subclass*.

To provide the actual class metadata, the helper class `OSMetaClass` is used. This is a very simple class, itself deriving from `OSMetaClassBase`, and defining but four private members:

- `className`: An `OSSymbol` pointer (character string) providing the name of the class in question. Referring to classes by name provides powerful features to the runtime, such as introspection, and dynamic type casting.
- `superClassLink`: A pointer to an `OSMetaClass` descendant, the parent of this class. This allows the implemention of inheritance, as a super class's methods may be overridden or invoked from its derived classes.
- `classSize`: Required to allow dynamic object construction and destruction. Instantiation of an object eventually leads to a `kalloc` operation from the nearest sized `kalloc.###` zone.
- `instanceCount`: through which the runtime can keep track of object counts. The `instanceCounts` of all fields are maintained in the IORegistry, and `ioclasscount(8)` (or its source data, the `IOKitDiagnostics` property) can be used to collect this data from user mode.

* - There is also an `OSMetaClassMeta` class which is used as a bootstrap to `OSMetaClass` itself, but this class is not visible outside libkern/c++/OSMetaClass.cpp and thus left out of the discussion to prevent hyper-meta.

APIs

The `OSMetaClass` members are all private, and it therefore falls on public members of `OSMetaClass` and some macros, based off of the static methods) to provide the various functionality.

Table 13-15: The public methods of `OSMetaClass`

Method	Macro	Description
`alloc()`	`OSTypeAlloc(type)`	Factory method/macro. Usually overridden down the hierarchy by specific derived class allocator
`allocClassWithName(name)`	---	Create a new instance of given class name
`type::metaClass`	`OSTypeID()`	Gets metaClass object of a class name (macro) or from static member of class
`getMetaClass`	`OSTypeIDInst(typeinst)`	Gets metaclass of object instance
`safeMetaCast(inst, type)`	`OSDynamicCast(type, inst)`	Perform C++-like `dynamic_cast` if *inst* derives from *type*, or returns NULL
`checkTypeInst`	`OSCheckTypeInst(typeinst, inst)`	`OSMetaClassBase` static method (or macro) checking if *inst* is is derived from *typeinst*.
`release()`	`OSSafeRelease[NULL](inst)`	Release object. Macro is "safe" in that it checks if object is NULL. As of Darwin 15, `OSSafeReleaseNULL` deprecates `OSSafeRelease`, the latter having unsafely led to a UaF enabling Pangu's iOS 9 jailbreak.
`isEqualTo(obj)`	---	Are two objects equal - pointer comparison or otherwise object-specific overriden implementation

IOKit provides a few useful functions for querying `OSObjects`. These functions, accessible from user mode over device.defs MIG subroutines, allow the caller to pass any IOKit handle, which is then internally cast to the primitive `OSObject`, allowing basic RTTI-like functionality. These routines are shown in Table 13-16:

Table 13-16: The `io_object_*` MIG routines

#	io_object_... MIG Routine	Functionality
2800	`io_object_get_class(obj,className)`	Calls `obj->getMetaClass()->getClassName()` and sets *className* accordingly
2801	`io_object_conforms_to (obj,className,conforms)`	Sets the *conforms* boolean specifying whether or not *obj* can be metaCast to *className*
2830	`io_object_get_retain_count (obj,out count)`	Returns the object's `getRetainCount()`
2860	`io_object_get_superclass(obj, className)`	Calls `obj->getMetaClass()->getSuperClass()` and sets *className* accordingly
2861	`io_object_get_bundle_identifier (masterPort, obj, out identifier)`	Returns providing kext's `CFBundleIdentifier`

A good way of testing object handles is to use `IOObjectConformsTo()`. This handy routine checks whether a given IOObject instance is a subclass of another, which is useful before invoking methods.

DefaultStructors

In order for `OSMetaClass` APIs to be able to perform their function, every class must declare and define specific members. These make for boilerplate code, so macros can be used to do so.

The `OSDefineMetaClass[WithInit]` macros generate class global data, member functions, and a special `MetaClass` constructor. The `..WithInit` variant takes the name of a static initializer and invokes it from the `MetaClass` constructor, thus ensuring its invocation before any object of that class is instantiated. Neither macro, however, is intended to be used directly (as emphasized by a **DO NOT USE** warning in the header comment).

The `OSDeclareDefaultStructors` macro (libkern/libkern/c++/OSMetaClass.h) generates the declaration of the default constructors and destructors, and as such is used in the header file for the class. A second macro, `OSDefineDefaultStructors` (ibid.), generates the matching implementations, and therefore goes in the class body. Two other variant macros - `OSDeclare[Abstract/Final]Structors` and `OSDefine[Abstract/Final]Structors` - are used for classes with one or more abstract members, or for final classes (i.e. such that they cannot be further subclassed). Here, too, header documentation marks these macros boldly as **DO NOT USE**, so the `OSDefineMetaClassAnd[/Abstract/Final]Structors` are used instead. These "allowed" macros ensure that the actual structor definition macros are called only after the metaclass itself is defined, using `OSDefineMetaClassWithInit` in their expansion. Listing 13-17 shows the expansion of the macros for a give class name:

Listing 13-17: The expansion of the `OSDefine*` and `OSDeclare*` macros

```
#define OSDeclareDefaultStructors(className)    \
    OSDeclareCommonStructors(className);        \
    public:                                     \
    className ();                               \
    protected:

#define OSDeclareCommonStructors(className)                     \
    private:                                                    \
    static const OSMetaClass * const superClass;                \
    public:                                                     \
    static const OSMetaClass * const metaClass;                 \
        static class MetaClass : public OSMetaClass {           \
        public:                                                 \
            MetaClass();                                        \
            virtual OSObject *alloc() const;                    \
        } gMetaClass;                                           \
        friend class className ::MetaClass;                     \
        virtual const OSMetaClass * getMetaClass() const APPLE_KEXT_OVERRIDE; \
    protected:                                                  \
    className (const OSMetaClass *);                            \
    virtual ~ className () APPLE_KEXT_OVERRIDE

#define OSDefineMetaClassAndStructors(className, superclassName)    \
    OSDefineMetaClassAndStructorsWithInit(className, superclassName, )

#define OSDefineMetaClassAndStructorsWithInit(className, superclassName, init)\
    OSDefineMetaClassWithInit(className, superclassName, init)        \
    OSDefineDefaultStructors(className, superclassName)

#define OSDefineMetaClassWithInit(className, superclassName, init)          \
    /* Class global data */                                                 \
    className ::MetaClass className ::gMetaClass;                           \
    const OSMetaClass * const className ::metaClass =                       \
        & className ::gMetaClass;                                           \
    const OSMetaClass * const className ::superClass =                      \
        & superclassName ::gMetaClass;                                      \
    /* Class member functions */                                            \
    className :: className(const OSMetaClass *meta)                         \
        : superclassName (meta) { }                                         \
    className ::~ className() { }                                           \
    const OSMetaClass * className ::getMetaClass() const                    \
        { return &gMetaClass; }                                             \
    /* The ::MetaClass constructor */                                       \
    className ::MetaClass::MetaClass()                                      \
        : OSMetaClass(#className, className::superClass, sizeof(className))    \
        { init; }
```

The metaclass constructors in a kext are put into a compiler generated function, whose common naming convention is `__GLOBAL__sub_I_`*filename*. The function is invoked, along with any other static initializers, through the kext's `__DATA.__mod_init_func`, when processed by `OSRuntimeCallStructorsInSection` (from `OSRuntimeInitializeCPP`, called during `OSKext::start`, as explained in Chapter 3). Destructors are commonly named `__GLOBAL__D_a` (though the same name is used for all destructors, independent of filename) and similarly invoked through their pointers in `__DATA.__mod_term_func` (from `OSRuntimeFinalizeCPP`). These patterns are best viewed in MacOS kexts, which exist in standalone forms and whose symbols are not stripped.

As an example, consider Output 13-18, using AppleMobileFileIntegrity.kext, though any IOKit kext will display the same pattern:

Output 13-18: Viewing metaclass structures using AppleMobileFileIntegrity.kext as an example

```
morpheus@Chimera (~)$ cd /System/Library/Extensions/A*M*F*I*/Contents/MacOS
#
# The __mod_init_func provides an array of initializers to run on kext loading
#
morpheus@Chimera (.../MacOS)$ jtool2 -d __DATA.__mod_init_func AppleMobileFileIntegrity
Dumping from address 0xe098 (Segment: __DATA.__mod_init_func) to end of section
0xe098: 0x0000000000000cde __GLOBAL__sub_I_AppleMobileFileIntegrityUserClient.cpp
0xe0a0: 0x0000000000001824 __GLOBAL__sub_I_TrustedKeys.cpp
0xe0a8: 0x0000000000002f04 __GLOBAL__sub_I_Entitlements.cpp
0xe0b0: 0x0000000000008155 __GLOBAL__sub_I_AppleMobileFileIntegrity.cpp
#
# As an example, display the User Client MetaClass constructor:
#
morpheus@Chimera (.../MacOS)$ jtool2 -d __DATA.__mod_term_func AppleMobileFileIntegrity
__GLOBAL__sub_I_AppleMobileFileIntegrityUserClient.cpp:
0x0cde   pushq   %rbp
0x0cdf   movq    %rsp, %rbp
0x0ce2   leaq    AppleMobileFileIntegrityUserClient::gMetaClass(%rip), %rdi
0x0ce9   leaq    0x8610(%rip), %rsi ## literal pool for: "AppleMobileFileIntegrityUserClient"
0x0cf0   movq    0xd311(%rip), %rdx
0x0cf7   movl    $0xe8, %ecx
0x0cfc   callq   OSMetaClass::OSMetaClass(char const*, OSMetaClass const*, unsigned int)
0x0d01   leaq    0xdd60(%rip), %rax
0x0d08   movq    %rax, AppleMobileFileIntegrityUserClient::gMetaClass(%rip)
0x0d0f   popq    %rbp
0x0d10   retq
#
# Destructors are in mod_term_func. Note two (identically named) destructors, one per object
#
morpheus@Chimera (.../MacOS)$ jtool2 -d __DATA.__mod_term_func AppleMobileFileIntegrity
Dumping from address 0xe0b8 (Segment: __DATA.__mod_term_func) to end of section:
0xe0b8:            0xd11        __GLOBAL__D_a
0xe0c0:           0x8188        __GLOBAL__D_a
morpheus@Chimera (.../MacOS)$ otool -tV AppleMobileFileIntegrity | c++filt
...
__GLOBAL__D_a:
0x0d11   pushq   %rbp
0x0d12   movq    %rsp, %rbp
0x0d15   leaq    AppleMobileFileIntegrityUserClient::gMetaClass(%rip), %rdi
0x0d1c   popq    %rbp
0x0d1d   jmp     OSMetaClass::~OSMetaClass()
...
```

Members, methods and the Fragile Base Class proble

Constructors and destructors are two special cases of the more general methods, which are implemented as function pointers in the object's vtable. Member variables are listed as regular structure fields. When the class in question is a final (or otherwise leaf) class, this poses no problem - but when the class serves as the base of others, modifications to it may alter its size, member or vtable offsets. This leads to the "Fragile Base Class Problem", as it may break derived classes in various ways.

To combat this syndrome, libkern offers two techniques. The vtable may be padded ahead of time with any number of empty slots, using the `OSMetaClassDeclareReservedUnused`, which takes the class name and a slot number and is placed in the header file. The corresponding `OSMetaClassDefineReservedUnused` is similarly placed in the implementation. if `APPLE_KEXT_VTABLE_PADDING` is `#defined` (as it is on MacOS), the two macros expands (respectively) to a definition and declaration of `_RESERVED`*classNameslot* functions, which call the metaclass's `reservedCalled` method. Unless overloaded, this bubbles up to `OSMetaClass::reservedCalled` - and `panic()`s.

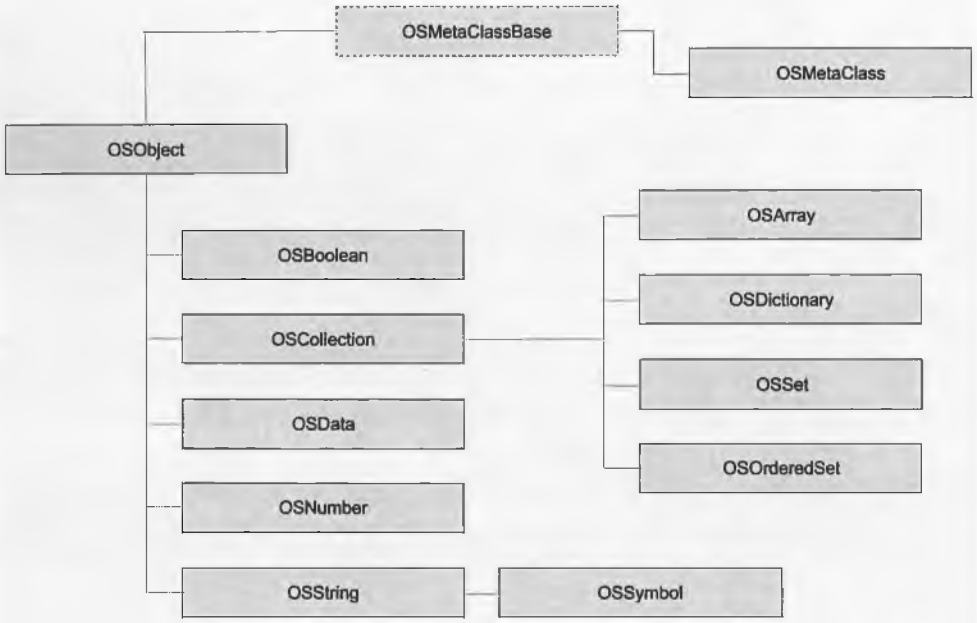
When slots are claimed, `OSMetaClass[Define/Declare]ReservedUsed` macros should replace their `..Unused` counterparts. Apple's documentation suggests "for clarity's sake" to use these macros (which have no code significance, as they evaluate to nothing) just ahead of the function to occupy the slot, though in practice most of their code doesn't do so.

Members cannot be expanded in this way, and so Apple's convention is to defining an opaque `struct ExpansionData`, and define a member as a pointer to this structure. The structure is commonly defined inline as part of the class, and often left empty. If new members are needed , they can be placed into the structure: Since the structure is indirectly included, the total size of the object remains the same. This does require the developer, however, to use a cumbersome type casting of the structure pointer and dereference the required expansion field - for which a macro proves useful.

Object types

Just like CoreFoundation provides the various CFObjects, so too does libkern provides a set of objects for use in kernel mode, shown in Figure 13-19:

Figure 13-19: The libkern `OS*` object hierarchy



Most of these are modeled after their `CF*` counterparts, and IOKit.framework's APIs provide bridging back and forth between them. Being C++ objects, it can be a bit challenging to figure out their in-memory representations. As descendants of `OSObject`, all begin with a pointer to inside their vtable, through which methods can be accessed. Object fields follow next, subject to the usual rules of inheritance (superclass fields before those of subclasses). It certainly doesn't help that the field layout is affected by a conditional `APPLE_KEXT_ALIGN_CONTAINERS` (`#define` in libkern/libkern/c++/OSMetaClass.h) to be true in *OS, false in MacOS), which compacts fields and reorders them in some of the object instances. The macro is set as the inverse of `APPLE_KEXT_VTABLE_PADDING` (which is false on *OS, and true in MacOS), which as described affects the vtable position in the class.

OSStrings and OSSymbols

The `OSString` class is a simple wrapper over C-Strings. Though NULL-terminated, the `OSString` also holds the string length (including the NULL byte), as well the only presently defined flag, `kOSStringNoCopy`, which is used when creating an `OSString` with `initWithCStringNoCopy`. MacOS uses a full 32-bit for both flags and length, whereas *OS compacts both fields into a single 32-bit quantity.

The compaction enables the 32-bit combined flags/length to fit immediately after the `OSObject` header, but at a cost - 18-bits are allocated for the length, and so the maximum string data which can be held is 262,142 bytes (2^{18}-2), standardized across all flavors. The actual string data is held elsewhere (usually in some `kalloc.*` zone), and merely pointed to by the `string` member.

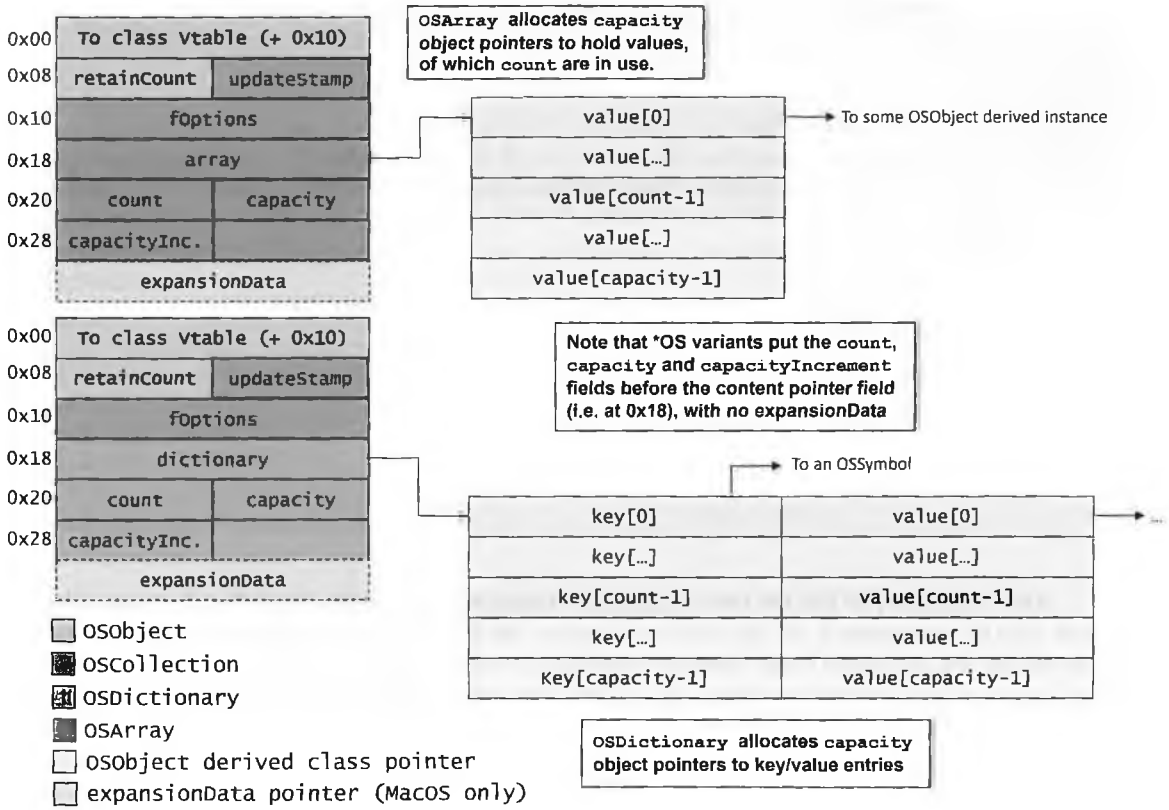
Strings are commonly used in kernel and duplicates can prove wasteful. The `OSSymbol` subclass of `OSString` keeps a private `OSSymbolPool` instance to make sure that, as new `OSSymbol`ss are created, duplicates are suppressed. Other than that the two classes are functionaly equivalent, and take up the same memory size - 24 bytes in *OS, and 32-bytes in MacOS (due to compaction).

OSCollections

The `OSCollection` serves as the parent class of all container objects, though it is meant to be subclassed. It provides basic services such as dynamic capacity adjustment, and enumeration (with the help of the `OSCollectionIterator`). More concrete functions are provided by its subclasses. The `OSSet` provides a simple set, with no duplicates and no ordering. The `OSOrderedSet` adds ordering. The two classes are used primarily by the `IOCatalogue`, but relatively little elsewhere.

Much more popular are the `OSArray` and `OSDictionary` subclasses. The `OSArray` is a container over a dynamic array of object pointers. The `OSDictionary`, like the `CFDictionary`, uses what is effectively an anonymous array of key/value entries. Figure 13-20 shows both the `OSArray` and `OSDictionary`, as they appear in memory. Note, that here, too, object representations in memory differ between the flavors, subject to the `APPLE_KEXT_ALIGN_CONTAINERS` `#define`. Further, the the MacOS objects both have an additional `ExpansionData` pointer, which the *OS flavors do not.

Output 13-20: The `OSArray` and `OSDictionary` objects, on MacOS



Serialization

So long as objects remain in kernel mode, their internal representation is not a problem. Moving objects to and from user mode, however, necessitates some intermediary representation which will both capture the object state in full, while hiding the details of its implementation.

XML Serialization

XML was all the rage back in the NeXTSTEP days, and so it was decided to adopt this promising data format and support it not just in user mode property lists and dictionaries, but also in kernel mode. It is therefore quite common to see use of data items being deserialized from and serialized to an XML representation.

There are two key components involved in handling XML in kernel. The first is the `OSSerialize` class. This class is declared in libkern/libkern/c++/OSSerialize.h, implemented in libkern/c++/OSSerialize.cpp, but is not meant for direct use. Rather, serializable classes override `OSObject::serialize(OSSerialize *)`, and use the `OSSerialize` instance in their argument to add themselves to the serialized representation. The `OSSerialize` instance is created with an initial allocation (by the `::withCapacity` method) using `kmem_alloc()`, and is able to grow as new elements are serialized, by calling on `kmem_realloc()` as the need arises (from its `::ensureCapacity()` method). If not overridden, `::serialize()` merely serializes an "*className* is not serializable" message, which is evident in the many times instances of the `IOCommand` appear this way in the output of `ioreg(8)`.

The second component is the `OSUnserializeXML()` function, implemented in libkern/c++/OSUnserializeXML.cpp. Although a single function, this is arguably one of the most complicated functions in XNU. The function code is autogenerated from OSUnserializeXML.y, a preprocessor file meant to be digested by the `bison(1)` tool. The process of doing so is recorded in the foreboding header comment of both the .y and .cpp files:

Listing 13-21: The ominous header comment of libkern/c++/OSUnserializeXML.cpp

```
/*
 * HISTORY
 *
 * OSUnserializeXML.y created by rsulack on Tue Oct 12 1999
 */

// parser for unserializing OSContainer objects serialized to XML
//
// to build :
//  bison -p OSUnserializeXML OSUnserializeXML.y
//  head -50 OSUnserializeXML.y > OSUnserializeXML.cpp
//  sed -e "s/#include <stdio.h>//" < OSUnserializeXML.tab.c >>OSUnserializeXML.cpp
//
//  when changing code check in both OSUnserializeXML.y and OSUnserializeXML.cpp
//
//
//
//
//               DO NOT EDIT OSUnserializeXML.cpp!
//
//                       this means you!
```

That the history of this file isn't recorded is misleading - over time, Apple had to constantly deal with its maintenance, as `OSUnserializeXML` has been a quite reliable source of memory corruption and disclosure bugs, the last known bug, a memory disclosure in oversized integer data, having been patched as late as iOS 10.2[4]. Past performance may yet imply future results.

Binary Serialization

Apple actually extended the serialization capabilities of XNU, by adding serialization in and out of binary in Darwin 14 (XNU 2782). This was done by extending the `OSSerialize` class to support `::binaryWithCapacity()` as a constructor, and binary serialization methods such as `::binarySerialize()` and `addBinary[Object]()`. These are all implemented in libkern/c++/OSSerializeBinary.cpp, which also holds `OSUnserializeBinary()`.

The main use of binary serialization is in IOUserClient property handling, specifically the `is_io_registry_entry_get_propert[y/ies]_bin` MIG methods (#2879 & #2878, also added in Darwin 14). `OSUnserializeBinary` is called directly from `OSUnserializeXML` if and when its input is detected to be binary, i.e. when beginning with the well known magic of `kOSSerializeBinarySignature` ("\323\0\0") rather than XML. The magic, along with the serialization constants, is listed in libkern/libkern/OSSerializeBinary.h.

Listing 13-22: The `kOSSerialize*` constants (from XNU 4903's libkern/libkern/OSSerializeBinary.h)

```
// Most significant byte serves for encoding of datatype.
// Most significant bit marks end of collection,
// Lower three bytes serve for byte (or, for dictionary, element) counts.
enum
{
  kOSSerializeDictionary   = 0x01000000U,
  kOSSerializeArray        = 0x02000000U,
  kOSSerializeSet          = 0x03000000U,
  kOSSerializeNumber       = 0x04000000U,
  kOSSerializeSymbol       = 0x08000000U,
  kOSSerializeString       = 0x09000000U,
  kOSSerializeData         = 0x0a000000U,
  kOSSerializeBoolean      = 0x0b000000U,
  kOSSerializeObject       = 0x0c000000U,
  kOSSerializeTypeMask     = 0x7F000000U,
  kOSSerializeDataMask     = 0x00FFFFFFU,
  kOSSerializeEndCollecton = 0x80000000U, // Apple typo
};

#define kOSSerializeBinarySignature "\323\0\0"
```

Despite being relatively new and not overly complex code, Apple made some grave mistakes with its introduction. The implementation of `OSUnserializeBinary` provided two of the three prongs of the Trident vulnerabilities, which was the basis of the Pegasus APT until it was discovered (in the wild) and fixed in iOS 9.3.4. The discussion of Trident in III/22 (and specifically, Figure 22-5) provides a detailed walkthrough of both the vulnerability and its exploitation.

The Class Menagerie

IOKit objects all derive from the base classes of libkern we've examined in the previous section, but these are far too primitive to be used by more complex IOKit objects. IOKit therefore introduces a few classes of its own, which greatly extend the basic object orientation and functionality of the libkern classes, offering more IOKit specific base functionality, and are used as the point of departure for drivers.

The libkern inheritance, which results in sizeable vtables, in nothing compared to that of IOKit classes. As further subclasses are added to the hierarcy, the size of the MetaClass and class vtables increases significantly. Thankfully each vtable entry only takes up the `sizeof(void *)`, but with so many entries it is not uncommon to see vtables spanning several kilobytes.

IOKit Built-in classes

Several important base classes are part of the iokit/ branch of the XNU source tree, and are compiled into the kernel proper. These are quite significant, as they serve as primitive objects supporting IOKit's rich functionality and interfaces. Some have already been encountered in this chapter, and others will be shortly. These classes include:

- `IORegistryEntry`: The `IORegistryEntry` class handles all aspects of the object's presence in the IORegistry. Specifically, it provides methods for registration (through `attachToParent`), deregistration (`detachFromParent` or `detachAll`), location (`fromPath`), traversal (`get[Child/Parent]Iterator` and property handling (`[set/get]Property`).
- `IOService`: The base class for drivers, `IOService` inherits from `IORegistryEntry`, so it follows that all descendants of `IOService` also get their own registry entries "for free", whether they want them or not. These entries are all in the `IOService` registry plane, though drivers are free to register in other planes as well, or even create their own planes (via `IORegistryPlane::makePlane`).
- `IO*MemoryDescriptor`: IOKit provides several classes for memory management, all subclasses of the `IOMemoryDescriptor` subclass. In a nutshell, each such descriptor corresponds to a range of memory, which may be either virtual or physical. Operations on a descriptor control the mapping and unmapping of its underlying memory, as well as virtual to physical translations.
- `IO*MemoryCursor`: Cursors are used to convert memory references to physical addresses. The `IOMemoryCursor` is abstract, and its built-in concrete subclasses are `IO[Big/Little/Natural]MemoryCursor`, which account for different machine endianness.
- `IOWorkLoop`: providing a `CFRunLoop`-like primitive, for use in drivers.
- `IO*EventSources` and `IOCommands`: through subclassing, IOKit supports `IOTimerEventSources`, `IO[Filter]InterruptEventSources`, and `IODMAEventSources`, which can be integrated into `IOWorkLoops`.
- `IOCommands` are abstract classes which represent I/O requests generated by the driver and submitted to the underlying device controller. Being abstract, it must be subclassed, and one built-in subclass is `IODMACommand`. A common use case is employing `IOCommandPool` to pre-allocate a pool of `IOCommands` for ease of use and reuse by multiple clients.
- `IOCommandQueues` and `..Gates`: are also subclasses of `IOEventSource`. These classes are used to synchronize operations in a workloop, ensuring single threaded access as we discuss later in this chapter. The `IOCommandGate` is a lightweight version of the `IOCommandQueue`, and is preferred to the latter. Neither is interrupt-safe.

IOKit Families

Although drivers may inherit from the generic `IOService`, Apple provides an even more specialized set of classes which developers can use as their point of departure. These are called **Families**, and each family handles a particular device class. This way, anyone seeking to implement a driver can select the closest family, and inherit from it, rather than from `IOService`.

The families are easy to recognize thanks to the `CFBundleIdentifier` convention of naming them `com.apple.iokit.IO`*`family`*`Family`. Table 13-23 shows the IOKit family in use throughout Darwin 18, most of which are used by all flavors. A large part of these are open source, but some (indicated by an asterisk) remain closed. Apple occasionally adds families (as it did, for example, with `IOSkywalkFamily`), and those generally remain closed.

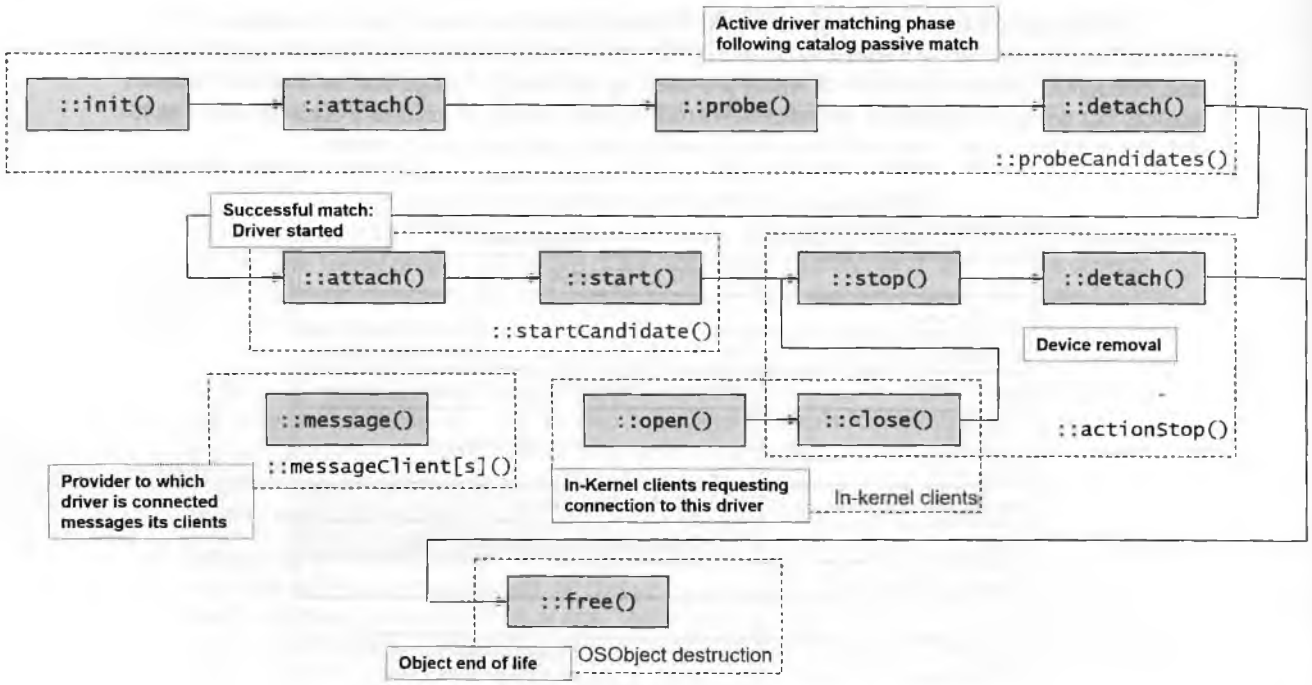
Table 13-23: The known IOKit families, and their purpose

IO....Family Name	Purpose
80211	IEEE 802.11 Wireless Ethernet
Accelerator[2]	
CryptoAccelerator	*OS: Cryptographic hardware acceleration
ACPI	Advanced Configuration and Power Interface
Audio[2]	
BlueTooth	
CEC	*OS: HDMI-CEC support
DART	*OS: Device Address Resoultion Table
[Mobile]Graphics	
HDA	
HDCP	*OS: HDMI Content Protection
HID	Human interface devices
MikeyBus	*OS: TriStar audio bus
Networking	Generic network drivers
NVMe	
PCI	PCI bus
Report	Reporting provided by numerous subsystems
Serial	Serial interfaces
[illegible]	Darwin 17:
SCSIArchitectureModel	
SlowAdaptiveClocking	
Stream	
Storage	
TimeSync	
USB	USB nub
USBMassStorageFamily	
USBHost	USB host
USBDevice	*OS: USB target
80211	802.11 networking (subclasses `IONetworking`)

Driver Life Cycle

IOKit drivers have a clearly defined, event-driven lifecycle, and uses callbacks provided the `IOService` class. All drivers are instances of this class, and they may choose to override the methods according to their specific, device-oriented needs. Figure 13-24 shows this lifecycle, and the rest of this section describes its various stages in more detail.

Figure 13-24: The `IOService` driver lifecycle



Driver Matching

For a driver to even be loaded, it must successfully undergo a process of **driver matching**. Recall, that all IOKit kexts are required to provide an `IOKitPersonalities` dictionary. Each personality must define an `IOProviderClass`, which identifies the type of device it is meant for. There are numerous `IOProviderClass` types - exported by specific nub drivers. Drivers can also claim the special `IOResources` class as their provider, which is used primarily by virtual devices (which handle no hardware, and therefore have no nub to connect to). Claiming `IOResources` mandates specifying an `IOMatchCategory` as well, else two drivers would have the same match and one could block the other.

The common trigger to start the matching process is when some other driver (a nub provider) calls `publishResource()` (for an `IOResources` driver), or directly calls `IOService::registerService()`, indicating it has been loaded and/or some new device has been detected. Another case is when the various `kext_tools` utilities call the `io_catalog_send_data` and `code_io_catalog_module_loaded` MIG routines through the IOKit.framework wrappers (see Table 13-7) following a manual kext loading operation. These calls update the `IOCatalogue` through `::[add/remove/resetAndAdd]Drivers` or `::startMatching()` (respectively). All these result in a call to `IOService::catalogNewDrivers()`, which begins the driver matching process.

The driver matching process is conceptually made up of three stages:

1. The **class matching** stage considers only those drivers in the `gIOCatalogue` whose nub (first level dictionary entry in the catalog) matches the `IOProviderClass`.
2. The **passive matching** stage inspects the registered `IOKitPersonalities` in the `gIOCatalogue` which could be a potential match. A further refinement may be made through the `::matchPropertyTable()` method, which is normally empty (returning true), but may be overridden by a nub to provide matching based on specific properties.

For example, in order to deal with the multitude of PCI devices, the open source `IOPCIFamily` implements `IOPCIDevice::matchPropertyTable()`, overriding the `IOService` method, and calling `IOPCIBridge::matchNubWithPropertyTable()`, which inspects the value of the `IOPCI[/Primary/Secondary/Class]Match` keys in the dictionary, taking only those devices which are most suitable for the driver.

3. The **active matching** takes those candidates found in the previous stage, sorts them by their `IOProbeScore`, and calls the functions up to and including their `probe()` path, to allow the drivers to make a runtime decision if they wish to claim the device or not. If `probe()` chooses to claim the device, it returns an `IOService` instance (or simply a reference to the driver itself, i.e. `this`, as is also returned when `probe()` is left unimplemented). The `IOProbeScore` is passed as an in-out parameter to `probe()`, so that the driver may modify it.

Figure 13-25 presents a high level view of the IOKit driver matching process. This can prove quite lengthy, which is why it can be performed asynchronously. If the `options` argument to `::startMatching()` specifies `kIOServiceAsynchronous` (0x8), a call to `_IOServiceJob:startJob()` starts a `kMatchNubJob` by calling `::pingConfig()`, which creates a new `_IOConfigThread`. The `__IOConfigThread::main()` reduces its priority and calls `doServiceMatch()`, and it is this method which kicks off the matching process. Every driver gets a chance to `::probe()`, and the one with the highest ranking score gets the first attempt at `::start()`ing. On successful `::start` of a candidate, the matching process ends.

Figure 13-25: A high level view of the IOKit active driver matching process

Case Study: VMWare Fusion VMIOPlug

Driver developers aware of the matching proces can take advantage of it. A good example is VMWare Fusion's vmioplug.kext (though you can likely find a similar case in other hypervisor software, such as Parallels or VirtualBox). The kext (in the app's Library/kexts folder) declares the following `IOKitPersonalities` in its Info.plist:

Listing 13-26: VMWare Fusion's vmioplug.kext's registered `IOKitPersonalities`, in SimPLISTic format

```
IOKitPersonalities:
 UsbDevice:
        CFBundleIdentifier: com.vmware.kext.vmioplug.18.7.0
        IOClass: com_vmware_kext_UsbDevice_18_7_0
        IOProviderClass: IOUSBDevice
        idProduct: *
        idVendor: *
        bcdDevice: *
        IOProbeScore: 9005
        IOUSBProbeScore: 4000
 UsbInterface:
        CFBundleIdentifier: com.vmware.kext.vmioplug.18.7.0
        IOClass: com_vmware_kext_UsbDevice_18_7_0
        IOProviderClass: IOUSBInterface
        idProduct: *
        idVendor: *
        bcdDevice: *
        bConfigurationValue: *
        bInterfaceNumber: *
        IOProbeScore: 9005
        IOUSBProbeScore: 6000
 UsbPortArbiter:
        CFBundleIdentifier: com.vmware.kext.vmioplug.18.7.0
        IOClass: com_vmware_kext_UsbPortArbiter_18_7_0
        IOUserClientClass: com_vmware_kext_UsbPortArbiterUserClient_18_7_0
        IOProviderClass: IOResources
        IOResourceMatch: IOKit
        IOMatchCategory: com_vmware_kext_UsbPortArbiter_18_7_0
```

The closed source `IOUSBFamily`, implements property specific matching by implementing `__ZN14IOUSBInterface18matchPropertyTableEP12OSDictionaryPi` (in mangled form), which looks at keys such as `idProduct`, `idVendor` and `bcdDevice`, which the family reports based on USB device identifiers. The family also allows wild card matching (through `__ZN8IOUSBNub15IsWildCardMatchEP12OSDictionaryPKc`)

This enables `vmioplug.kext` to intercept every USB device when it is inserted. The kext declares in its matching dictionary the potential to match any device from the `IOUSBDevice` nub, by declaring a match for `bcdDevice` from any `idVendor` with any `idProduct`. To make sure it is the first in line for matching, the personalities declare a ridiculously high `IOProbeScore`, and another high `IOUSBProbeScore`, used internally by `IOUSB` device matching.

Thus, when any USB device is inserted, the `vmioplug.kext` not only proves eligible at the end of the passive matching phase, but it also gets first dibs on matching, as its scores are the highest. From this point on, the rest is downhill. The kext's code will execute, sending a message to the user mode application, which pops up the dialog asking the user if they wish the device to be connected to the virtual machine, or the host. Should the user decide on the virtual machine, the kext can claim the device. If the user decides otherwise (or times out), the kext loses interest, and the matching process continues with the next candidate in line.

The example of universal device matching shown here is an intentional one, but incorrect matching dictionaries could potentially cause problems. A common case is matching on the `IOProviderClass` of `IOResources`: Since this nub serves as an attachment point for many system-wide resources and virtual devices, specifying a matching dictionary for it without also specifying an `IOMatchCategory` (matching the driver's own `IOClass`) could cause the driver to inadvertently match other resources in the system, instead of the correct drivers for those resources.

Driver activity and the IOWorkLoop

When a device is claimed by the driver, it is reattached, and a call to its `start()` method enables the driver to perform the low level device startup. When in-kernel clients (dis)connect, the driver's `::[open/close]()` method may be called.

The `::open()` method, however, must return, and so commonly most of the driver's work is performed asynchronously, responding to user-mode requests, interrupts, or other events. For this, the driver employs an `IOWorkLoop`: Similar in principle to the `CFRunLoop` pattern of user mode (discussed in I/10), an `IOWorkLoop` enables the driver to respond to events.

The driver need not necessarily spin off threads of its own. In most cases, a driver can share its provider's work loop, if one exists, or the platform's. Calling `IOService`'s `::getWorkLoop()` method will return such a pre-existing workloop, and the driver can then add its own event sources to it.

If a new work loop is necessary, it can be created with one of the factory methods of `IOWorkLoop::workLoop[WithOptions]()`, and will usually be assigned to a specially created thread, spun off in its `::init()` method using `kernel_thread_start()` and tagged with `THREAD_TAG_IOWORKLOOP`.

True to their name, work loop threads loop, calling `runEventSources()` while the loop's `workToDo` flag is set and while its `kLoopTerminate` is unset. The `::threadMain()` uses itself (`__ZN10IOWorkLoop10threadMainEV`) as a continuation (with a goto as backup), with the `IOWorkLoop` instance as the parameter. The threads thus block, until woken up by `::signalWorkAvailable()` (or from `::free()`), from an `IOEventSource`. XNU's drivers create quite a few dedicated workloops. MacOS has on order of 60 of those, and all but a few belong to drivers and are appropriately named., as can be seen in the output of `procexp 0 threads` (q.v. Output 5-34).

IOEventSources and IOCommandGates

The `IOWorkLoops` support `IOEventSources`, in the same way the `CFRunLoops` do `CFRunLoopSources`. Sources can be added/removed to an `IOWorkLoop` using `[add/remove]EventSource()`. There are several available source types, all subclasses of `IOEventSource`:

- **`IODMAEventSources`:** Reserved for use with DMA Controllers.
- **`IOInterruptEventSources`:** meant to handle "secondary" or "indirect" interrupts, that is, interrupts whose handlers run when other "primary" or "direct" interrupts may still fire. Such sources are constructed with the callback routine, the provider handle, and an interrupt index. The source will fire through a call to `::interruptOccurred()`. The `IOFilterInterruptEventSource` is a subclass, suitable for shared interrupt lines.
- **`IOTimerEventSources`:** can be set to run a callback function when some timeout value (set by the `::setTimeoutMS()` method) expires. These are implemented over `timer_calls`.
- **`IOCommandGates`** are mutual exclusion primitive, which is similar in nature to a critical sections: A gate must be closed (by an `IORecursiveLockLock()` on the work loop's `gateLock`) when work is to be performed, and then may be reopened (i.e. an `IORecursiveLockUnlock()`). The `IOCommandGate` also supports condition variables - `::command[Sleep/Wakeup]()` use the work loop's `[sleep/wakeup]Gate()` methods.

A workloop has a built in gate in its `controlG`, which can handle "maintenance" requests. Thus, the work loop's `::addEventSource()` amounts to an `mAddEvent` request, which (under the gate lock) adds the event source to the work loop's `eventChain`, and `::removeEventSource` similarly uses an `mRemoveEvent` request to remove the source.

Messaging

Another aspect of the driver lifecycle is **messaging**, which allows two drivers to communicate by message passing. The sender calls `::messageClients()` to send a message to all of clients on the `gIOGeneralInterest` list, or `::messageClient()` for a particular client. This is effectively the in-kernel equivalent of the general interest notifications in user mode, an example of which was shown in Listing 13-14. Drivers can call the `::registerInterestForNotifier()` method to register for these notification types.

Messaging invokes the driver's `::message()` method, which accepts three arguments - the message `type`, the `provider`, and an opaque `argument`. The codes supported by IOKit were shown in a previous experiment (specifically, Listing 13-14-b). Families and drivers can add their custom codes as well.

Matching Notifications

Figure 13-25 noted the several locations in `IOService::startMatching()` which fire notifications on driver matching for interested parties. Drivers can also employ the powerful facility of matching notifications in kernel. By calling `::addMatchingNotification()` with a `type`, a `matching` dictionary and a `handler` function (which, as of Darwin 18 may be a block), they can ensure the handler gets invoked when `doServiceMatch()` gets a match which satisfies the dictionary criteria, according to the `type`. The available `type` can be one of:

Listing 13-27: Available `types` for `::addMatchingNotification()` and its siblings

```
const OSSymbol *gIOPublishNotification;
const OSSymbol *gIOFirstPublishNotification;
const OSSymbol *gIOMatchedNotification;
const OSSymbol *gIOFirstMatchNotification;
const OSSymbol *gIOTerminatedNotification;
const OSSymbol *gIOWillTerminateNotification;
```

The notifications are asynchronous in nature, but it is simple to use them in a synchronous manner through `IOService::waitForMatchingService()` (as shown in Listing 13-10) or similar functions.

Interrupt Handling

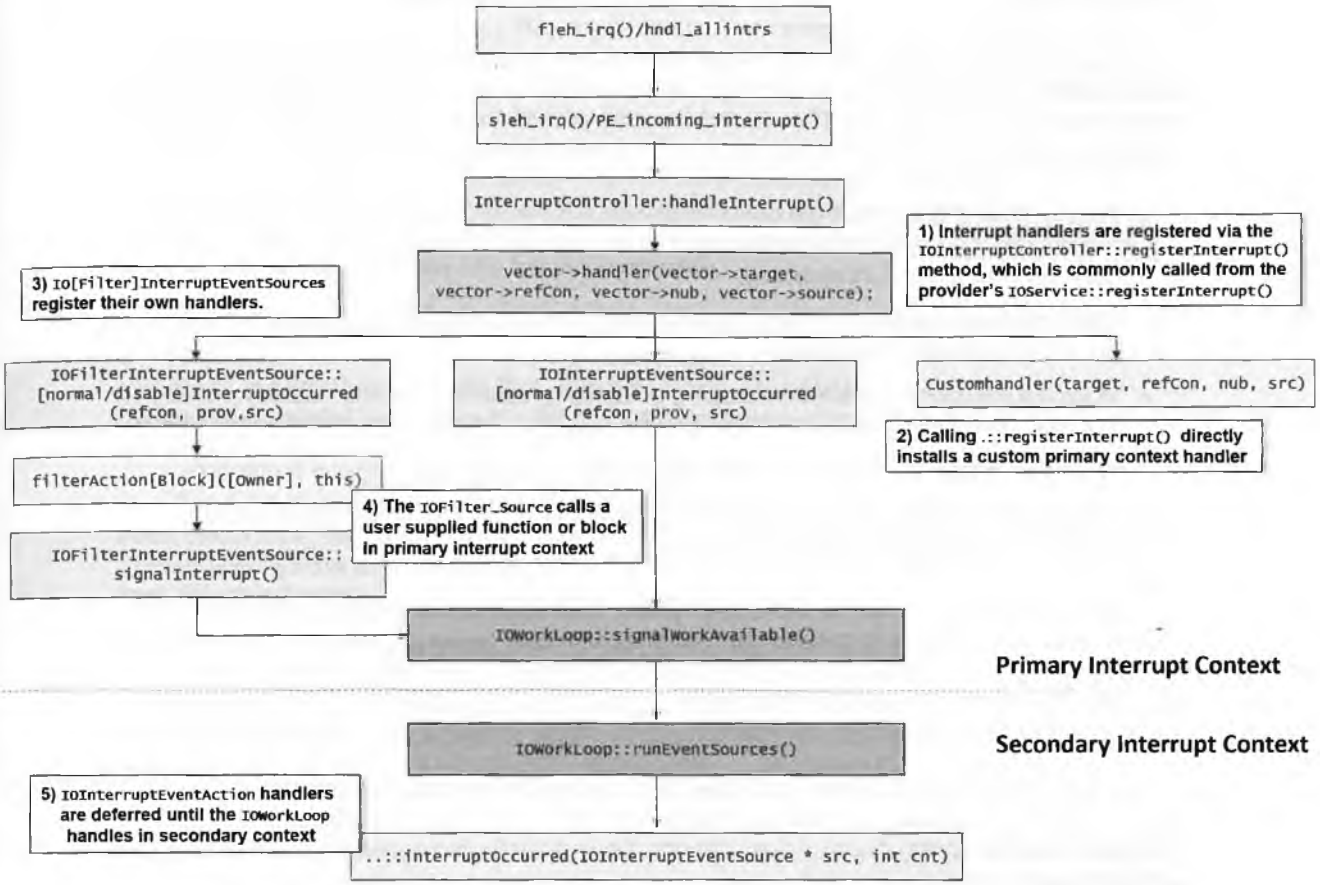
Drivers often interface with hardware, and interrupts are the general mechanism by means of which hardware notifies the CPU of events. IOKit therefore provides drivers with an ability to register their own interrupt handlers. Interrupts were discussed at the machine level in Chapter 4, and now that we have briefly reviewed `IO[Filter]InterruptEventSources`, we can put the two together for a more complete picture of interrupt handling.

Recall that Chapter 4's discussion of interrupts left off with the call by the architecture handler (Intel's `PE_incoming_interrupt()` or ARM's per-CPU data handler) calling the routine installed by `ml_install_interrupt_handler()`, through IOKit's higher level abstraction of an `IOCPUInterruptController`. It is this callback which then dispatches the interrupt handling to IOKit, invoking the handler with four arguments: a `nub`, a `target` and a `refCon` (all held in the installed handler context) and a `source` integer, specifying the interrupt source line.

IOKit recognizes two distinct modes of interrupt handling:

- The **direct interrupt** is handled in **primary interrupt context**, meaning that other interrupts are masked. Code running in this context is part of the same flow which originated with the low level architecture handler. It must not block or sleep in any way, and time is of the essence - once the bare minimum of work has been accomplished, the code must return.
- The **indirect interrupt** is a form of deferred interrupt, handled in **secondary interrupt context**. Other interrupts are enabled at this stage, and the code is generally free of restrictions.

Figure 13-28: The IOKit interrupt handling contexts



IOKit drivers thus have two ways installing their handlers. The first (but not recommended) one is to use `IOService::registerInterrupt`. This takes in an interrupt `source` integer, and uses `::lookupInterrupt()` to find the `IOInterruptController` responsible for it. It first checks its own `_interruptSources` vector, or calls `resolveInterrupt`, which retrieves the `OSArray` held by the `IOInterruptControllers` property. The interrupt controller can then be retrieved from the platform, along with the nub's `IOInterruptsSpecifiers`, and then cached in then cached in the instance's local `_interruptSources`.

Once the `IOInterruptController` has been found, the controller's `::registerInterrupt()` is invoked, prepending the `IOService` instance's own handle to the rest of the arguments - `target`, `handler` and `refCon`. The controller retrieved is commonly a subclass of `IOInterruptController()`. If the interrupt source is already in use, but the device tree specifies the interrupt can be shared, an `IOSharedInterruptController()` is instantiated. The specified `handler` is installed in the controller's internal `vectors` array.

The second (and recommended) way is by instantiating an `IOInterruptEventSource`. This, too, uses `registerInterruptHandler()` method, but the handler installed is the source's own `::normalInterruptOccurred()`, or `::disableInterruptOccurred()`, which automatically disables the interrupt when handled).

Whichever method was chosen to register, when an interrupt occurs, the `IOInterruptController`'s handler - i.e. the `::handleInterrupt()` method (or an overridden implementation) is invoked. This calls the controller's vector handler - as was previously registered.

The actual user supplied handler will run either in direct context (if registered via `IOService::registerInterrupt()`), or in indirect context (when registered via `IOInterruptSource`) when the `IOWorkLoop()` (woken by `::signalWorkAvailable()`) will execute at a later time. There is also the `IOFilterInterruptEventSWource()` subclass, which provides somewhat of an in-between way - the `Action` function pointer (or, as of Darwin 18, block) runs in the primary context, and is meant to be used in order to find if the interrupt is indeed of interest (e.g. when occurring in an interrupt line shared between several devices).

IOUserClients

A major part of a driver's lifecycle is serving requests made by user mode callers. UN*X device drivers traditionally create a device node in order to communicate with user mode. IOKit, on the other hand, provides a different, and far more powerful mechanism of **IOUserClient**s, enabling an IOKit driver to quickly utilize one of several communication channels:

- **Driver properties**: allow the driver to provide key/value pairs which are readable and (at the driver's discretion) settable from user mode, through the driver connection or even right from its `IORegistryEntry`.
- **Notifications**: enable the driver to send asynchronous messages back to user mode, via an a priori specified Mach port.
- **Mapped memory**: enable the mapping of kernel (presumably, device-related) memory into the caller's task, similar to the traditional `mmap(2)` functionality device nodes supply.
- **External traps**: are a legacy mechanism allowing function calls with 0-6 arguments.
- **External methods**: are preferred (as of Darwin 9) for method calls, as they provide better constraints over passed arguments.

These operations provide far more functionality than UNIX device nodes do. Rather than limiting driver functionality to the standard set of system calls (`read(2)/write(2)/ioctl(2)` and `mmap(2)`), IOKit maximizes the object oriented approach. These functions are made available to user mode callers after a successful call to `IOServiceOpen()`. In all cases, the driver may choose to implement the functionality by simply providing the appropriate callbacks.

As with the IORegistry, access to IOUserClient became more restricted over time. The `mpo_iokit_check_open_t` hook was added early on, to defer the authorization of the UserClient creation to the `Sandbox.kext`. Through the rich profile language (discussed in III/8), and particularly the `(allow iokit-open ... (iokit-user-client-class ...)` directives, the minimum necessary drivers for operation can be whitelisted, and all others denied. One way or another, even those which are allowed (specifically, `IOSurface`, which is mandatory for `UIView` support), prove vulnerable and sufficient enough to enable attacking the kernel from a sandboxed context. Once an IOUserClient handle can be obtained, all other operations on it are commonly allowed, unless the driver in question employs specific entitlement checks.

IOUserClient lifecycle

Similar to the MIG routines we've seen thus far, `IOKitLib.h` provides wrappers for several others which are used throughout the IOUserClient lifecycle. These are shown in Table 13-29:

Table 13-29: The other user client related MIG calls, and their `IOService*` counterparts

#	`io_service..` routine	IOKitLib.h	Purpose
2862	`.._open_extended`	`IOServiceOpen()`	Returns an `io_connect_t` to driver's IOUserClient
2816	`.._close`	`IOServiceClose()` `IOCloseConnection()`	Disconnect an `io_connect_t`, possibly deallocating its port.
2831	`.._get_busy_state`	---	Retrieve driver busy state (0 or 1)
2832	`.._wait_quiet`	`IOServiceWaitQuiet()`	Wait until `IOServiceGetBusyState()` is 0
2842	`.._request_probe`	`IOServiceRequestProbe()`	Rescan a bus for device changes
2856	`.._get_state`	`IOServiceGet[Busy]State[AndTime]`	Get count of asynchronous operations in process
2875 2876	`.._[get/set]_authorization_id`	`IOService[Get/Set]AuthorizationID()`	Get/Set authorization (set to process unique ID)

An IOUserClient instance is created through a call to `IOServiceOpen()`, which wraps the `io_service_open_extended()` MIG call (#2862). The underlying MIG call allows setting arbitrary `properties` (which `IOServiceOpen()` passes `NULL` for), but as of Darwin 16 Apple disabled this functionality, returning `kIOReturnUnsupported` if properties are specified, due to this being used as an exploitation vector by Pangu in iOS 9.

The `io_connect_t` used to open an IOUserClient instance can be retrieved by a call to `io_connect_get_service` (MIG #2817, wrapped by `IOConnectGetService()`). Calling `io_connect_add_client` (MIG #2820, wrapped by `IOConnectAddClient()`) is a rarely used routine to inform a driver of a second connection.

The MIG routine implementation (`is_io_service_open_extended`) calls `IOService::newUserClient`. This tries to allocate a new instance of the `gIOUserClientClassKey`, and then calls that class's `initWithTask` method, passing in the `owningTask` pointer, `securityID`, the MIG call's `connect_type`, and the (now NULL) `propertiesDict`.

The driver is expected to override this method, and may reject the call (by returning a value other than `KERN_SUCCESS`), which indeed many drivers do in later versions of Darwin, refusing to even open a service handle to unentitled callers. An example of such a check (intentionally from the closed source eficheck.kext) is shown in Listing 13-30:

Listing 13-30: The `::initWithTask` method of eficheck.kext:

```
com_apple_driver_eficheckuserclient::initWithTask
  (task*, void*, unsigned int, OSDictionary*):
...
if (IOUserClient::copyClientEntitlement(task, "com.apple.private.eficheck"))
0x75e         movq    %rdx, %r13
0x761         movq    %rsi, %rbx
0x764         movq    %rdi, %r14
0x767         leaq    0x70a(%rip), %rsi ## "com.apple.private.eficheck"
0x76e         movq    %rbx, %rdi
0x771         callq   IOUserClient::copyClientEntitlement(task*, char const*)
0x776         testq   %rax, %rax
0x779         je      0x799
{
0x77b         movq    %rax, %r15
0x77e         movq    0x8a3(%rip), %rax
0x785         movq    (%rax), %rax
0x788         movq    (%rax), %r12
0x78b         movq    (%r15), %rax
0x78e         movq    %r15, %rdi
0x791         callq   *0x28(%rax)
0x794         cmpq    %r12, %r15
0x797         je      0x7ab
} else {
                  IOLog ("Rejecting un-entitled IOServiceOpen request");
0x799         leaq    0x6f3(%rip), %rdi ## "Rejecting un-entitled IOServiceOpen r
0x7a0         xorl    %eax, %eax
0x7a2         callq   _IOLog
                  return 0;
0x7a7         xorl    %eax, %eax
0x7a9         jmp     0x7da
}
0x7ab         movq    0x866(%rip), %rax
0x7b2         movq    %r14, %rdi
0x7b5         movq    %rbx, %rsi
0x7b8         movq    %r13, %rdx
0x7bb         movl    -0x2c(%rbp), %ecx
0x7be         movq    -0x38(%rbp), %r8
0x7c2         callq   *0x8f0(%rax)
0x7c8         movq    %rbx, 0xf0(%r14)
0x7cf         movq    $0x0, 0xd8(%r14)
... epilog omitted for brevity ...
```

Once an `IOUserClient` has been successfully opened, the handle returned is an `io_connect_t`. This is just another Mach port type (`IKOT_IOKIT_CONNECT`, #29 in Table 10-8). With it, user mode can get/set properties, get notifications, perform memory mappings of driver-owned memory to user mode, and call driver-supplied methods. Held `IOUserClient` instances are reported in `ioreg(8)`'s output as the `IOUserClientCreator` property, whose value (serialized) is the BSD pid and process name.

Driver Properties

Getting and setting properties is far more intuitive for most of user space needs. Drivers may export their properties via their registry entries by calling the `setPropertyTable` they inherit (indirectly, through `IOService`) from the `IORegistryEntry` class.

Once properties are published, `io_registry_entry_get_properties()` or `io_registry_entry_get_property[_bin/recursively/bytes]()` (from Table 13-4) can be called from user mode (usually through the `IORegistryEntrySearchCFProperty()` or other wrappers) in order to read properties. `io_registry_entry_set_properties()` (#2828) performs the writing. An additional routine, `io_connect_set_properties()` (#2821) is wrapped by `IOConnectSetCFProperties()`, and serves as an alias, by using the `io_connect_t` as an `io_registry_entry_t`.

Most drivers allow read-only access to their properties, but those offering write access do so by implementing the `::setProperties(OSObject*)` method (or, in its mangled form, `..13setPropertiesEP8OSObject`). The driver is free to interpret both read and write operations in any way. Aside from being able to offer user mode greater functionality by modifying driver attributes in real time, this also potentially increases the attack surface. There are quite a few drivers in both MacOS and *OS which allow property set operations, and they can easily be found with a command line such as the following:

```
for i in /System/Library/Extensions/*.kext/Contents/MacOS/*; do
        if jtool2 -S $i |
           grep setProperties | grep 'T '; then
                echo $i;
        fi; done
```

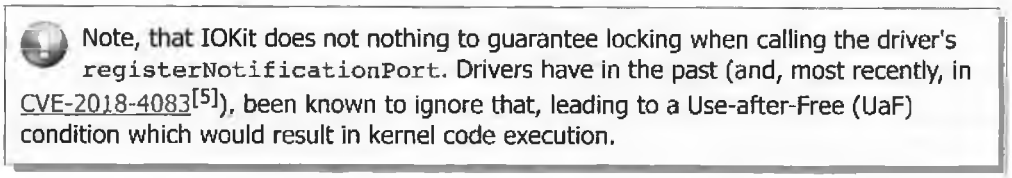
Notifications

Once in possession of an `IOUserClient` handle, a user mode application may provide a Mach port to `IOConnectSetNotificationPort`. This then allows the driver code to use the port to send messages back to user space.

Table 13-31: The `io_connect*` notification related MIG calls, and their `IOConnect*` counterparts

#	io_service_.. MIG routine	IOKit.framework API	Purpose
2818,67	`io_connect_set_notification_port[_64]`	`IOConnectSetNotificationPort`	Set port for driver→client notifications
2852	`io_connect_get_notification_semaphore`	---	`kIOReturnUnsupported` by default

As with other patterns we have explored, `IOConnectSetNotificationPort` is implemented over the `io_connect_set_notification_port` MIG message. The kernel mode server side increments call statistics, locks the user client object in kernel, and then calls on its `::registerNotificationPort()` method. Once again, the default implementation in IOUserClient.cpp merely returns `kIOReturnUnsupported`, but a driver implementing notifications is naturally expected to override this method with a custom implementation. There is also a callback for `releaseNotificationPort` (whose default implementation merely calls `iokit_release_port_send` on the port, if not `MACH_PORT_NULL`). Calls to `::registerNotificationPort` and `::releaseNotificationPort` should be balanced, though there is no mechanism to call the latter from user mode.



Once the user-supplied port is known to the driver, the user client may perform asynchrnous method requests. IOAsyncCallback latter may use the port "at will" to send messages to user mode, using `mach_msg_from_kernel()`. The format of the message and its semantics are entirely up to the driver.

Mapped Memory

Drivers commonly require memory mapping functionality, to enable direct access to kernel (and sometimes, physical) memory from user mode. More often than not, this is for a hardware device which uses DMA or some other low level mapping of device memory into that of the system (common examples being GPU buffers or network adapter packet buffers). UNIX offers this capability through `mmap(2)` over the device node, but IOKit has its own mechanism - `IOConnectMapMemory[64]`. The two functions, along with their inverses (`IOConnectUnmapMemory[64]`) map to lower level MIG routines, though when `LP64` bit is `#define` only the newer variants are used:

Table 13-32:

#	io_connect.. routine	IOKitLib.h
2819	io_connect_map_memory	**IOConnectMapMemory[64]**(*connect*,*memoryType*, *intoTask*, inout *atAddress*, inout *ofSize*,*options*)
2863	io_connect_map_memory_into_task	
2853	io_connect_unmap_memory	**IOConnectUnmapMemory64**(*connect*, *memoryType*, *fromTask*, in *atAddress*)
2864	io_connect_unmap_memory_from_task	

As with the other IOKit.framework wrappers, `IOConnectMapMemory` is well documented in IOkit/IOKitLib.h:

Listing 13-33: ....

```
/*! @function IOConnectMapMemory
    @abstract Map hardware or shared memory into the caller's task.
    @discussion This is a generic method to create a mapping in the callers task.
    The family will interpret the type parameter to determine what sort of mapping
    is being requested. Cache modes and placed mappings may be requested by the calle
    @param connect The connect handle created by IOServiceOpen.
    @param memoryType What is being requested to be mapped, not interpreted by IOKit
     family defined. The family may support physical hardware or shared memory mappi
    @param intoTask The task port for the task in which to create the mapping. This m
     be different to the task which the opened the connection.
    @param atAddress An in/out parameter - if the kIOMapAnywhere option is not set, t
     caller should pass the address where it requests the mapping be created, otherwi
     nothing need to set on input. The address of the mapping created is passed back
     on sucess.
    @param ofSize The size of the mapping created is passed back on success.
    @result A kern_return_t error code. */

kern_return_t
IOConnectMapMemory(
        io_connect_t            connect,
        uint32_t                memoryType,
        task_port_t             intoTask,
        mach_vm_address_t      *atAddress,
        mach_vm_size_t         *ofSize,
        IOOptionBits            options );
```

The caller would therefore call `IOConnectMapMemory` on the handle obtained by `IOServiceOpen()`, and a send right to the target task, usually its own `mach_task_self()`. The documentation all but ignores the `IOOptionBits` for mapping, specifying only `kIOMapAnywhere`, where in fact there are numerous other options, in XNU's iokit/IOKit/IOTypes.h. The various `IOOptionBits` are handled by the `IOMemoryDescriptor` which is internally created to support the mapping operation.

Drivers supporting memory mappings must implement the `::clientMemoryForType`, expecting three arguments - the user supplied `uint32_t memoryType` and `IOOptionBits options`, and an out pointer to an `IOMemoryDescriptor` which is to be returned to the caller. IOKit will translate the descriptor to the `atAddress` and `ofSize` out pointers which the user supplied.

The `IOMemoryDescriptor` abstracts the `MAP_MEM_*` flags of the `vm_map` layer. As explained in Chapter 11, these flags (and their `VM_WING_*` counterparts) control PTE attributes. `IOMemoryDescriptor` provides the `kIOMap*` flags at a higher level, translating them into the `vm_map` layer flags using `::vmProtForCacheMode`, or to device pager flags using `::pagerFlagsForCacheMode()`. `IODefaultCacheBits(pa)` internally calls `pmap_cache_attributes()` on the physical address specified. `IOMapPages` calls `pmap_map_block()`.

Memory mapping can be as dangerous as it is powerful. In the past, numerous vulnerabilities relating to either kernel memory disclosure (due to uninitialized data in mappings) or targeted memory corruptions (due to the user mode task's ability to change the mapping contents at any time) resulted in privilege escalation and worse.

External Traps

IOKitLib.h defines seven different functions in the `IOConnectTrap` family, shown in Listing 13-34:

Listing 13-34: The `IOConnectTrap#` family

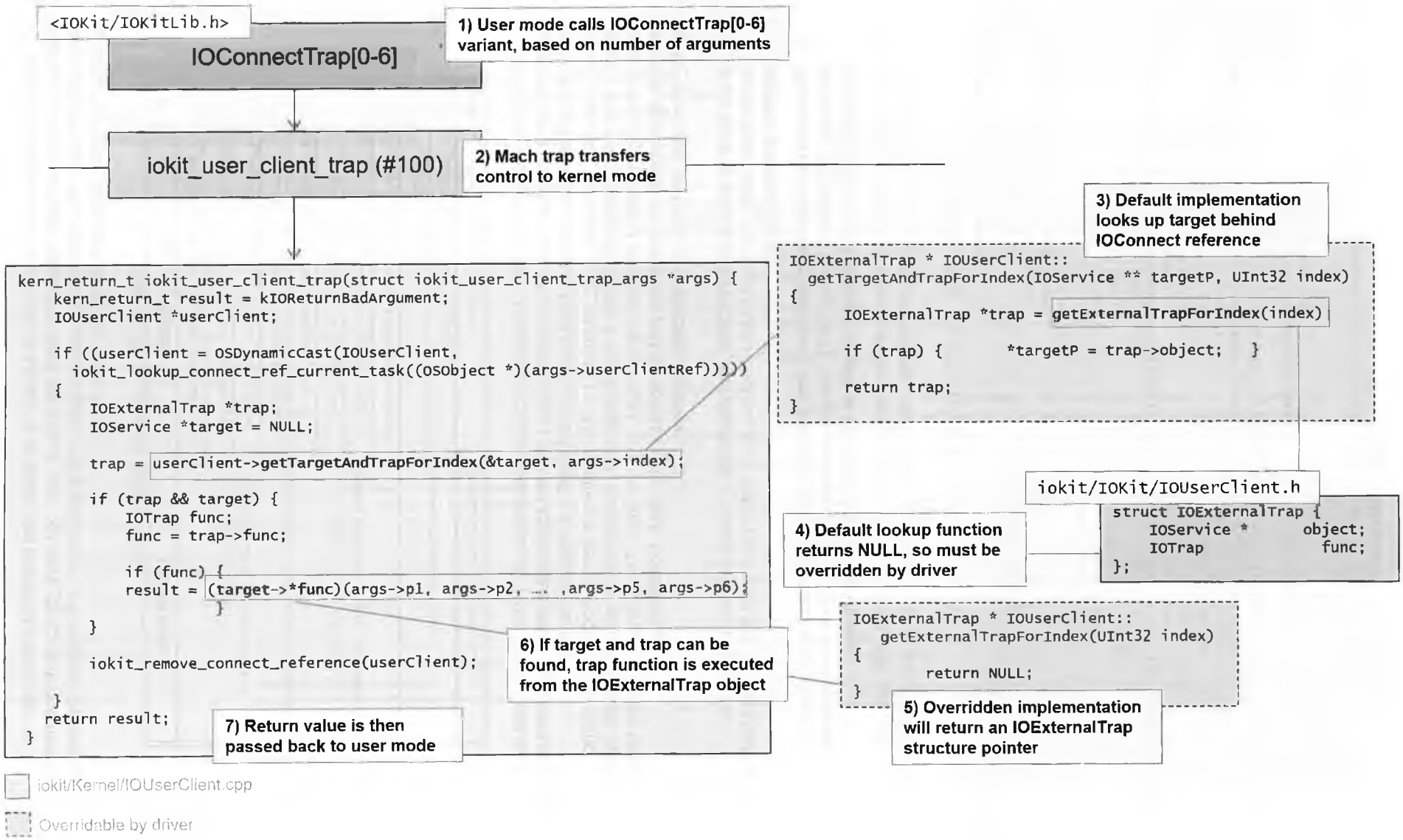
```
kern_return_t
IOConnectTrap0(io_connect_t     connect,
               uint32_t         index);

kern_return_t
IOConnectTrap1(io_connect_t     connect,
               uint32_t         index,
               uintptr_t        p1);

..
kern_return_t
IOConnectTrap6(io_connect_t     connect,
               uint32_t         index,
               uintptr_t        p1,
               uintptr_t        p2,
               uintptr_t        p3,
               uintptr_t        p4,
               uintptr_t        p5,
               uintptr_t        p6);
```

These are all thin wrappers over `iokit_user_client_trap` (Mach trap #100), which always takes the full set of eight arguments (i.e. `connect`, `index`, and `p` 1 through 6), setting unused arguments to zero. The trap transfers execution to kernel mode, wherein a function of the same name (in iokit/Kernel/IOUserClient.cpp) is invoked. The function resolves the `connect` argument to an `IOUserClient` reference, and then calls its `getTargetAndTrapForIndex()` method. If the target object and the trap function can successfully be located, the trap is invoked with its six arguments, and then the reference is removed.

The default implementation of `getExternalTrapForIndex()` is NULL, which causes the trap to return `kIOReturnBadArgument`. Drivers opting to use this mechanism must therefore provide their own implementation to override `getTargetAndTrapForIndex()`, and resolve the trap object and function by some custom manner, usually from a vtable. It is also possible to override `getExternalTrapForIndex()` directly, though this is marked deprecated.

`IOConnect` traps provide the fastest mechanism for invoking driver functions, but are deprecated in favor of external methods. Nonetheless, drivers are still free to use traps. Their ease of use and direct argument mapping make traps popular as kernel execution primitives. Ian Beer's `async_wake` (iOS 11.1.2) exploit, for example, creates a fake IOKit object, and overrides `getExternalTrapForIndex()` with XNU's `csblob_get_cdhash()` (from bsd/kern/kern_cs.c), since it returns a fixed offset (0x40) from its first argument. At this point that offset is controlled, and the first argument is placed there.

Figure 13-35: The IOExternalTrap mechanism



External Methods

External methods are the new (as of Darwin 9, at least) and preferred mechanism of invoking `IOService` calls. Instead of using the `iokit_uer_client_trap`, external methods are invoked through the IOKit (<device/device.defs>) MIG subsystem, and wrapped by IOKitLib.h `IOConnect*` APIs. These are shown in Table 13-36:

Table 13-36: `io_connect_method*` APIs

#	io_.. routine	IOKitLib.h
2822	io_connect_method_scalarI_scalarO	io_connect_method() (when EMULATE_IOCONNECT_64)
2823	io_connect_method_scalarI_structureO	
2824	io_connect_method_scalarI_structureI	
2825	io_connect_method_structureI_structureO	
2845	io_async_method_scalarI_scalarO	io_connect_async_method() (when EMULATE_ASYNC_IOCONNECT_64
2846	io_async_method_scalarI_structureO	
2847	io_async_method_scalarI_structureI	
2848	io_async_method_structureI_structureO	
2865	io_connect_method	IOConnectCallMethod()
2872	io_connect_method_var_output	
2866	io_connect_async_method	IOConnectCallAsyncMethod()

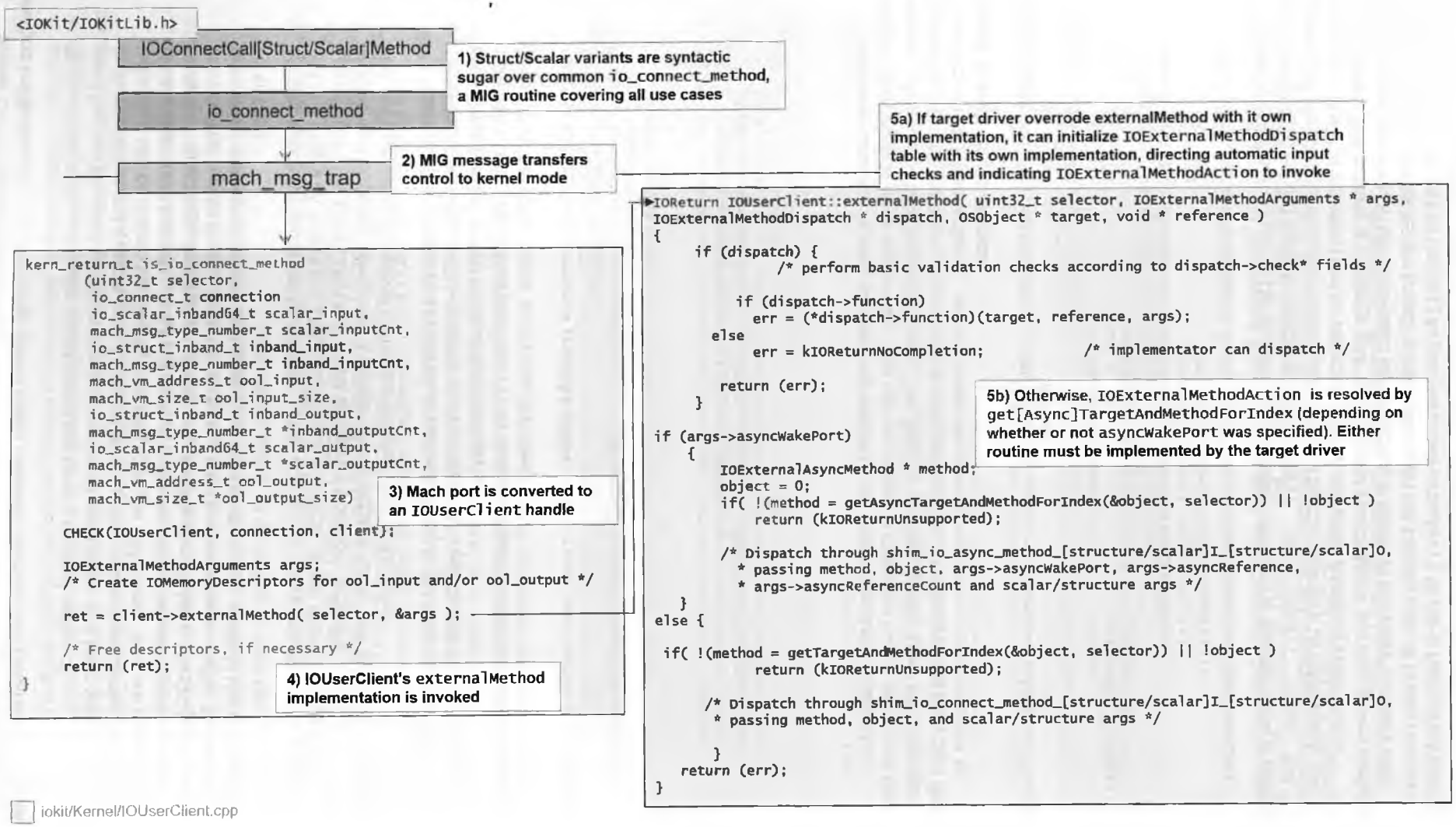
User mode can use variants - `IOConnectCallScalarMethod()` (for scalar, fixed size arguments) or `IOConnectCallStructMethod()` (for structure arguments - but eventually the underlying call is `IOConnectCallMethod()`, which itself is a wrapper for the `is_io_connect_method[_var_output]()` MIG routines in kernel mode (or the async variant). The MIG routines takes over a dozen argument, accounting for all the possible uses - scalar input/output, inband structure input/output, and ool input. The `_async_` variant takes two more arguments, an `wakePort` and an opaque `asyncRef`. Considering `is_io_connect_method()`, it proceeds as follows:

- If ool input or output was indicated, corresponding `IOMemoryDescriptor`s are constructed, using the `::withAddressRange()` constructor over the ool input/output address and size into the `current_task()`.
- The numerous arguments are copied into a single `IOExternalMethodArguments` structure. As shown in Listing 13-37, this structure accounts yet again for all possible input/output type combinations, as well as the asynchronous call model.
- The `externalMethod` method of the `IOUserClient` class is called, with the user requested `selector` (method number) and `IOExternalMethodArgument` structure.

Listing 13-37: The `IOExternalMethodArguments` structure

```
struct IOExternalMethodArguments {
  uint32_t             version;                    // kIOExternalMethodArgumentsCurrentVersion
  uint32_t             selector;                   // method selector
  mach_port_t          asyncWakePort;              // from _async_
  io_user_reference_t *asyncReference;             // from _async_
  uint32_t             asyncReferenceCount;
  const uint64_t *     calarInput;                 // from ScalarInput
  uint32_t             scalarInputCount;           // from ScalarInput
  const void *         structureInput;             // from StructureInput
  uint32_t             structureInputSize;         // from StructureInput
  IOMemoryDescriptor  *structureInputDescriptor;   // Created in kernel for StructureInput
  uint64_t *           scalarOutput;               // to ScalarOutput
  uint32_t             scalarOutputCount;          // to ScalarOutput
  void *               structureOutput;            // to StructureOutput
  uint32_t             structureOutputSize;        // to StructureOutput
  IOMemoryDescriptor  *structureOutputDescriptor;  // to StructureOutput
  uint32_t             structureOutputDescriptorSize; // Created in kernel for StructureOutput
  uint32_t             __reservedA;
  OSObject **          structureVariableOutputData;   // for _var_output_
  uint32_t             __reserved[30];
};

typedef IOReturn (*IOExternalMethodAction)(OSObject * target, void * reference,
                                           IOExternalMethodArguments * arguments);
```

Figure 13-38: The IOExternalMethod mechanism



iokit/Kernel/IOUserClient.cpp

Similar to the case of external traps, here, too, the driver is expected to provide the end method implementation by overriding. Unlike traps, however, there are two possible approaches:

- If the default `IOUserClient::externalMethod()` is called, either of `get[Async]TargetAndMethodForIndex` can be overridden, and their default implementations call `getExternal[Async]MethodForIndex` respectively. Both of the latter may still be overridden directly, though (again, as with traps), they are deprecated. Drivers commonly override `...26getTargetAndMethodForIndexEPP9IOServicej` (note C++ mangling), and these methods are all easily identifiable in MacOS (again, due to mangling) due to their `..PvP25IOExternalMethodArguments` suffix. The driver's overridden method will return an `IOExternalMethodAction`, which is the method to be invoked.
- Another option for the driver to override `IOUserClient::externalMethod()` itself, and in it supply an `IOExternalMethodDispatch` table. It can then call the original `IOService::externalMethod()` via a `super` invocation, providing both the arguments it obtained, along with a pointer to the dispatch table, and the object `target` and `reference` to be passed to the `IOExternalMethodAction`.

  Each `struct IOExternalMethodDispatch` entry in the table contains the `IOExternalMethodAction` pointer, and indicators for Input/Output Scalar/Structure counts/sizes, which will be checked automatically by `IOUserClient::externalMethod()`. An example dispatch table is shown in Listing 13-39, from the open source `IOHIDFamily.kext`:

Listing 13-39: External methods of `IOHIDEventServiceUserClient` (from `IOHIDFamily` sources)

```
const IOExternalMethodDispatch
 IOHIDEventServiceUserClient::sMethods[kIOHIDEventServiceUserClientNumCommands] = {
    { //    kIOHIDEventServiceUserClientOpen
        (IOExternalMethodAction) &IOHIDEventServiceUserClient::_open,
        1, 0, 0, 0
    },
    ...
    { //    kIOHIDEventServiceUserClientSetElementValue
        (IOExternalMethodAction) &IOHIDEventServiceUserClient::_setElementValue,
        3, 0, 0, 0
    },
};
```

Whichever approach is used, the external method is an `IOExternalMethodAction` function, that expects three arguments - the `target` object and opaque `reference`, and the `struct IOExternalMethodArguments`. The function normally breaks apart the individual arguments from the structure, reconstructing serialized objects, if necessary, and then pass them to the actual end implementation, taking discrete arguments. IOHIDFamily.kext can again be tapped to see a simple example:

Listing 13-40: An `IOExternalMethodAction` used in `IOHIDEventServiceUserClient` (from `IOHIDFamily` sources)

```
IOReturn IOHIDEventServiceUserClient::_setElementValue(
                                IOHIDEventServiceUserClient *   target,
                                void *                          reference __unused,
                                IOExternalMethodArguments *     arguments) {
    // User mode calls this via IOConnectCallScalarMethod
    return target->setElementValue((UInt32)arguments->scalarInput[0],
                                   (UInt32)arguments->scalarInput[1],
                                   (UInt32)arguments->scalarInput[2]);
}
```

When using the `IOExternalMethodDispatch` table method, the dispatch table is placed into the kext's (or, in hard-fused kernelcaches, the kernel's) `__DATA_CONST.__const`. The signature of the `IOExternalMethodDispatch` array (function pointers 16 bytes apart, and `uint32_t` values in between) can be detected reliably well when analyzing kexts and/or kernelcaches. This is never a challenge with symbolicated MacOS kexts, but also proves useful in the *OS case, especially in the hard-fused kernelcaches, in which all the kexts' constant data (wherein the dispatch tables are found) is intermixed with the kernel's own. It is also common for drivers overriding `getExternal[Async]MethodForIndex()` to use a hard-coded table of `IOExternalMethod` structures, which is also easy to find to to a similar signature.

IOCFPlugInTypes

Providing a driver interface as rich as `IOUserClients` and methods poses a potential problem, in getting user mode callers to be aware of specific calls offered by this or another driver. Maintaining a list of method selectors, for example, could become problematic as the list grows over time, and must be kept in sync between user and kernel mode.

IOKit addresses this through `IOCFPlugInTypes`. This key, if present in a kext's Info.plist, specifies a dict of UUID keys and path names for string values, of possible .plugin bundles which can be loaded by user mode clients so as to assist with interfacing the driver. The `IOCFPlugInTypes` loads along with other `IOKitPersonalities` properties into the IO Registry, and can then be picked up by a user mode caller.

Plugins are searched for in /System/Library/Extensions and /Library/Extensions, prepended to the plugin path* unless (as of Darwin 15) the path is a fully qualified one (starts with '/'). The process of loading plugins is performed by IOKit.framework's `IOFindPlugIns()`, called from `IOCreatePlugInInterfaceForService` (both in IOKitUser's IOCFPlugIn.c). The returned `IOCFPlugInInterface` object (defined in IOCFPlugin.h) is a COM object (defined in CoreFoundation's CFPlugInCOM.h), so its `IUnknown` interface's `QueryInterface` can be called with the desired interface UUID. This will provide an interface handle, from which specific methods can be invoked. This, however, is not automatic, and the user mode support framework has to explicitly request plugin loading - as is usually performed when the framework representation of the underlying device is created.

Listing 13-41:

```
//
// From CoreFoundation's CFPlugInCOM.h
//
#define IUNKNOWN_C_GUTS \
    __PRAGMA_PUSH_NO_NULLABILITY_COMPLETENESS_WARNINGS \
    void *_reserved; \
    HRESULT (STDMETHODCALLTYPE *QueryInterface)(void *thisPointer, REFIID iid, I
    ULONG (STDMETHODCALLTYPE *AddRef)(void *thisPointer); \
    ULONG (STDMETHODCALLTYPE *Release)(void *thisPointer) \
    __PRAGMA_POP_NO_NULLABILITY_COMPLETENESS_WARNINGS

//
// From IOCFPlugin.h:
//
#define IOCFPLUGINBASE                                                  \
    UInt16      version;                                                \
    UInt16      revision;                                               \
    IOReturn (*Probe)(void *thisPointer, CFDictionaryRef propertyTable, \
                      io_service_t service, SInt32 * order);            \
    IOReturn (*Start)(void *thisPointer, CFDictionaryRef propertyTable, \
                      io_service_t service);                            \
    IOReturn (*Stop)(void *thisPointer)

typedef struct IOCFPlugInInterfaceStruct {
    IUNKNOWN_C_GUTS;
    IOCFPLUGINBASE;
} IOCFPlugInInterface;

kern_return_t
IOCreatePlugInInterfaceForService(io_service_t service,
                CFUUIDRef pluginType, CFUUIDRef interfaceType,
                IOCFPlugInInterface *** theInterface, SInt32 * theScore)
```

Documentation on using `IOCFPlugin` can be found in Apple's (no longer maintained) Guide to Accessing Hardware from Applications[3]. Good examples of the usage patterns can be found in the sources of IOHIDFamily and IOUSBFamily. Both of these families rely on plugins extensively to deal with the ever increasing myriad of device types and functions. One such example is `IOHIDDeviceCreate()`:

---

* - IOKitUser's IOCFPlugIn.c comment erroneously specifies 'appended', not prepended.

Listing 13-42: An example using `IOCFPlugins` (from IOHIDLib's `IOHIDDeviceCreate()`

```
IOHIDDeviceRef IOHIDDeviceCreate(CFAllocatorRef                    allocator,
                                 io_service_t                      service)
{
    IOCFPlugInInterface **          plugInInterface     = NULL;
    IOHIDDeviceDeviceInterface **   deviceInterface     = NULL;
    IOHIDDeviceTimeStampedDeviceInterface **   deviceTimeStampedInterface = NULL;
    IOHIDDeviceRef                  device              = NULL;
    SInt32                          score               = 0;
    HRESULT                         result;

    require_noerr(IOObjectRetain(service), retain_fail);

    /*
     * Create the plug-in instance:
     */
    require_noerr(IOCreatePlugInInterfaceForService(service,
                                                    kIOHIDDeviceTypeID,
                                                    kIOCFPlugInInterfaceID,
                                                    &plugInInterface,
                                                    &score),
                                                    plugin_fail);

    /*
     * Query the generic ("IUNKNOWN") interface for a specific UUID
     */
    require_noerr((*plugInInterface)->QueryInterface(plugInInterface,
                        CFUUIDGetUUIDBytes(kIOHIDDeviceDeviceInterfaceID),
                        (LPVOID)&deviceInterface),
                        query_fail);


    /*
     * Create the plug-in interface for another, well known UUID
     */
    result = (*plugInInterface)->QueryInterface(plugInInterface,
                                CFUUIDGetUUIDBytes(kIOHIDDeviceDeviceInterfaceID2),
                                (LPVOID)&deviceTimeStampedInterface);
    if (result != S_OK) {
        deviceTimeStampedInterface = NULL;
    }

    device = __IOHIDDeviceCreate(allocator, NULL);
    require(device, create_fail);

    device->plugInInterface = plugInInterface;
    device->deviceInterface = deviceInterface;
    device->deviceTimeStampedInterface = deviceTimeStampedInterface;
    device->service         = service;

    return device;
..
}
```

Darwin 19: DriverKit

DriverKit was already introduced in Chapter 3 as one of the main improvements made in Darwin 19, in order to allow "user mode kernel extensions". After exploring IOKit in detail, its design and the rationale behind it can both be readily explained - primarily, the role of the `IOUserServer`.

IOUserServer

Recall, that driver extensions are started in user mode by `launchd(8)`. When IOKit matching is performed and `startCandidate()` is called (Figure 13-25), if the kext's dictionary contains the `gIOUserServerNameKey` then a call is made to `IOUserServer::serviceStarted()`, followed by `OSKext::requestDaemonLaunch()`, which fires a request with a "Dext Daemon Launch" predicate (Table 3-14). When `kextd(8)` receives the request, it sends a `launch_msg` XPC request to `launchd(8)`, similar to Listing 3-22.

The dext then starts up as a user mode process, and starts execution at `DriverKit.framework`'s `DriverExecutableMain()` routine, which serves as the entry point for all DriverKit dexts. This calls `IOUserServerMain()`, and then enters an endless dispatch loop (i.e. calls `dispatch_main()`). As the name implies, this is somewhat of an inverse to the `IOUserClient` model - as this time the kernel is the client, and it is the user mode .dext providing the server functionality in user mode.

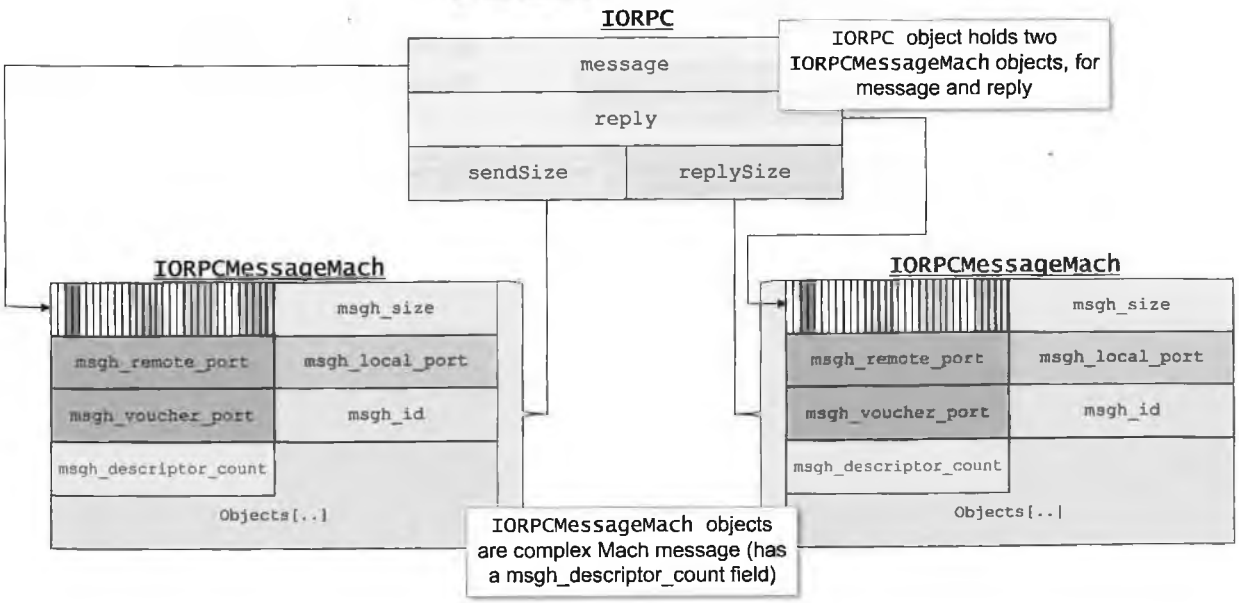
The `IOUserServerMain()` routine calls `io_service_open_extended` to open a kernel `IOUserServer` instance. The external method table of `IOUserServer` presently holds only two selectors. Selector #0x1001 is called to make a `SEND` right in kernel. Next, `IOUserServerMain()` calls `IOLibInit()` (which checks the boot arguments for "dk=1", to enable debugging), and `OSScanLibraries()`, which iterates over all classes and calls selector #0x1000, to perform registration in-kernel.

`IOUserServerMain()` next creates the user mode instance, and a `Root` dispatch queue, and calls `IOService::RegisterService()`. Althogh this is a user mode function (provided, like the rest of them, by DriverKit.framework), this calls `OSMetaClassBase::Invoke(IORPC)`, which moves the request to kernel mode through an `iokit_user_client_trap()` (Mach Trap #100) call.

IORPC

DriverKit introduces a new object, called IORPC. As depicted in Figure 13-43, this object is really nothing more than a coupling of two Mach messages with port descriptors:

Figure 13-43: The `IORPC` object

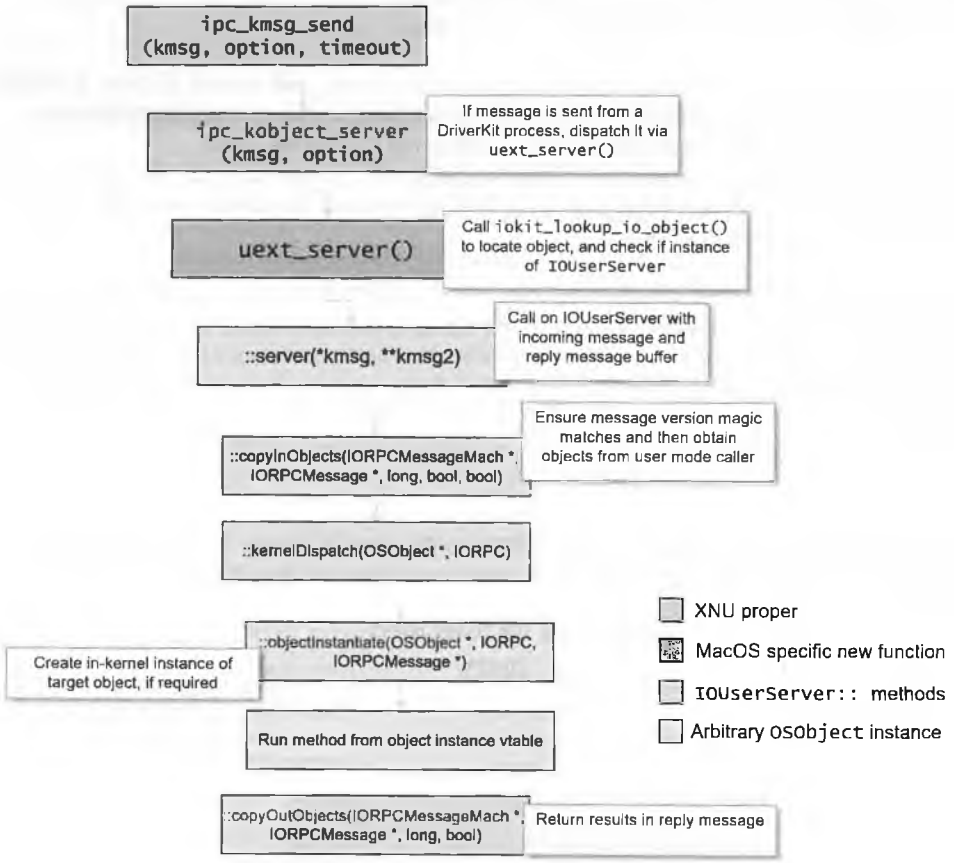


The `IOPRCMessage` serve for invocation of remote methods. Mach messages are "converted" to `IORPCMessage` objects by `IORPCMessageFromMach`. This routine checks its input Mach message descriptor count field looking for a version magic. The `IORPC` is still very much a work in progress, as is evident by the magic values used for versions. The present version is 0x4da2b68[c/d], identified in DriverKit.framework's IORPC.h as `kIORPCVersion190615[Reply]` (respectively). The header also defines several other versions, but message parsing code rejects these versions as obsolete.

The introduction of `IORPC` brought changes across the board to IOKit's objects in kernel. Starting with `OSMetaClassBase`, which has been extended by `::Invoke(IORPC)` and `::Dispatch(IORPC)`. This change ripples through the object hierarchy, allowing them to dispatch requests to the user mode driver kit extension. This way, even `IOInterruptSource` objects can relay the interrupt handling to a user mode driver.

Kernel-user mode relaying of `IORPC` messages is performed using the Mach `io_user_client_trap`. This had to be extended in MacOS 15, and diverts to `IOUserServerUEXTTrap()` for DriverKit callers, to perform in-kernel dispatching of messages. Likewise, the `ipc_kobject_server()` (see Chapter 10) has been modified to call `uext_server` to allow `IOUserServer::server()` to handle kernel messages, as shown in Figure 13-44:

Figure 13-44: The `IORPC` object



Review Questions

1. Compare and contrast the `OSObject` to its `CFObject` counterpart. What are the similarities? What is the fundamental difference in approach?
2. Besides the structural differences between the `IOCatalogue` and the `IORegistry`, what is another important difference between the two?
3. What is the difference between the `io_iterator_t` returned from IORegistry* APIs, and the one returned by `IOService*Matching*` calls?
4. If an `io_connect_t` can be used in `io_registry_entry` APIs, what is the corollary, and what is its potential implication?
5. What is the difference in expansion between the `...Structors`, `..AbstractStructors` and `..FinalStructors` macros?
6. Examine the `OSString` class (libkern/libkern/c++/OSString.h) and analyze the effect of `APPLE_KEXT_ALIGN_CONTAINERS`. What is the benefit of compaction for older versions of *OS? Does this still hold in present day *OS as well?

References

1. Apple Developer - "IOKit Fundamentals" - https://developer.apple.com/library/archive/documentation/DeviceDrivers/Conceptual/IOKitFundamentals/Introduction/Introduction.html
2. Apple Developer - "IOKit Device Driver Design Guidelines" - https://developer.apple.com/library/archive/documentation/DeviceDrivers/Conceptual/WritingDeviceDriver/Introduction/Intro.html#//apple_ref/doc/uid/TP30000694
3. Apple Developer - "Guide to Accessing Hardware from Applications" - https://developer.apple.com/library/archive/documentation/DeviceDrivers/Conceptual/AccessingHardware/AH_Intro/AH_Intro.html#//apple_ref/doc/uid/TP30000376
4. Liang Chen - "Revisiting kernel security enhancements in iOS 10" - http://NewOSXBook.com/files/archives/MOSEC2017-Chen.pdf
5. Ian Beer- "macOS Kernel - Use-After-Free Due to Lack of Locking" (CVE-2018-4083) - https://www.exploit-db.com/exploits/44007/

14

Stacking Up: Kernel Networking

The final chapter of Volume I (added in v1.2) explored the user mode APIs of Darwin networking in great detail. Among the items discussed were Apple's proprietary socket families - `PF_NDRV` and `PF_SYSTEM`, the statistics mechanism, fireawalling at the `ipfw(8)` and `ALF.kext` level, packet filtering with BPF, and the undocumented APIs of Skywalk.

This final chapter of Volume II revisits those very topics, in order, but this time from the kernel implementation perspective. In addition, we explore the structure of Darwin's networking stack - whose origins can be traced to BSD - but with so many Apple proprietary enhancements, has evolved in an entirely different direction.

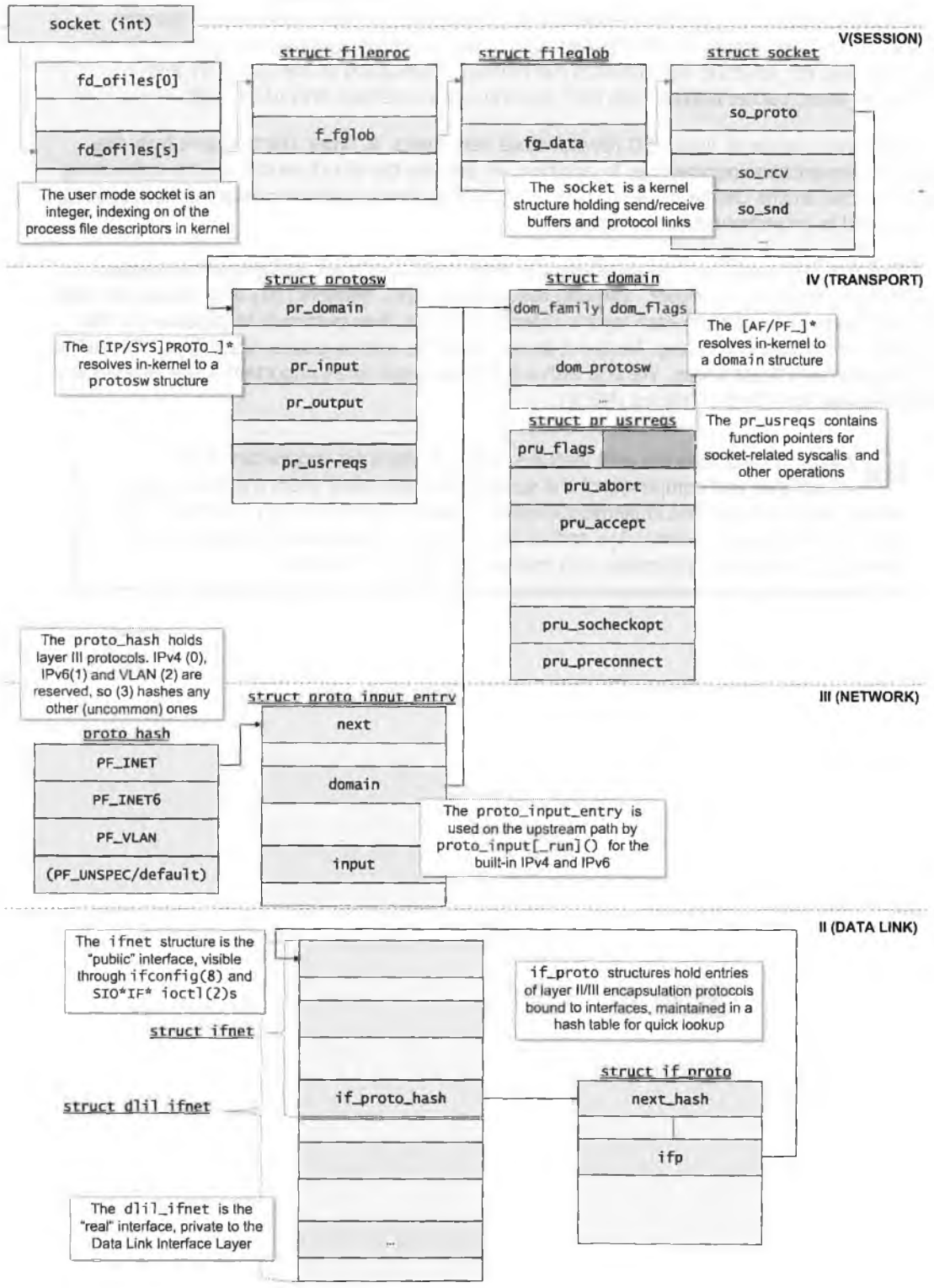
The approach taken here is of walking down the OSI layer stack, starting at the session layer (V) and descending deeper - through the transport (IV), network (III) and, finally, the Data Link (II) layers - Focusing on each layer's objects and KPIs, then putting it all together for the full picture of packet processing. Next, we discuss how the various packet filtering and firewalling facilities plug into these layers. We end with a brief discussion of an important Apple extension - Network Extension Control Policies (NECP).

Darwin's stack is laden with features, most of which are proprietary. The capabilities and complexity of the socket interfaces make them a popular attack vector. Kernel bugs in the implementations of advanced protocols (e.g. MultiPath TCP, in 11.3) and obscure `ioctl(2)`s and `setsockopt(2)`s (e.g. SockPuppet, in 12.2) gave rise to important jailbreaks, and may do so again in the future.

The High Level View

Most network stacks roughly correlate to the Open Systems Interconnection (OSI) model, which abstracts the sending and receiving of data over the network using seven layers, from the physical to the application. The well known sockets fit into the Session (fifth) layer of this model, with the layers above them (Presentation and Application) handled by user mode libraries or the application itself. The Transport (fourth) and Network (third) layers are handled by kernel code, commonly of the kernel proper. The Data Link (second) layer is handled by a combination of kernel and extension code: The kernel contains built-in support for the standard representation of a network interface and the related framing, whereas the extensions are responsible for sending and receiving those frames over the network interface controller (NIC).

Figure 14-1: A high level view of the Darwin network stack



Layer V: Sockets

Like most other UN*X objects, sockets appear in user mode as the same nondescript integer descriptors. In kernel, every corresponding `fileproc`'s `fileglob` structure (depicted in Figure 6-16) points to a `struct socket`.

The `struct socket`

The `struct socket` is defined in bsd/sys/socketvar.h. It is another one of the larger structure types, at 968/976 bytes as of Darwin 18/19. As with other massive structures, it's important to highlight some of its key fields:

- **so_zone**: is the index of the Mach zone the socket was allocated from. Normally, this is the `socket` zone, underlying the `M_SOCKET` BSD_ZONE, but `AF_INET` sockets are allocated from the `socache.zone`.
- **so_options**: holds the options which `[get/set]sockopt(2)` access.
- **so_proto and so_pcb**: are pointers to the socket's associated protocol switch and the protocol control block. The switch provides the protocol handler functions, as determined at time of creation from the combination of the type (stored in `so_type`) and the address family. The protocol functions can use the PCB to store relevant data. Protocols are detailed later in this chapter.
- **so_rcv and so_snd**: point to `struct sockbufs` for the receive and send buffers, accordingly.
- **so_flags and so_flags1**: hold `SOF_*` and `SOF1_*` flags (`#defined` inside the `struct sock` in bsd/sys/socketvar.h). The latter include many Apple specific extension, such as support for NECP, background sockets, QoS and more.
- **so_label and so_peerlabel**: The MACF is aware of sockets, and therefore both the socket itself and its connected peer (if any) can be assigned labels.
- **accept()-related data**: Sockets which may `accept()` connections need to maintain `so_head` (as a back pointer to the accepted socket), and queues for connections in accepted or partially accepted.
- **so_cfil and so_cfil_db**: hold content filter data. This is conditional of `#if CONTENT_FILTER`, but is set across flavors.
- **e_pid, e_upid and e_uuid**: hold the PID, Unique (64-bit) PID and UUID of the effective owner of the socket.
- **last_pid, last_upid and last_uuid**: similary hold the details of the last accessor of the socket.

Socket Creation

XNU offers the POSIX `socket(2)` system call, with its three arguments - `[AF/PF]_*` family, `SOCK_[STREAM/DGRAM/RAW/..]` type, and `..PROTO*` protocol (often left unused to auto-select the default by type). As of Darwin 13, however, it provides the `socket_delegate()` system call, as well. This undocumented call takes another argument - a PID in which to create the socket.

Both call flows merge in `socket_common()`, which first audits (on MacOS) and then checks that the MACF policy agrees with the operation (i.e. `mac_socket_check_create()` returns 0). A delegate call also requires the `PRIV_NET_PRIVILEGED_SOCKET_DELEGATE`. If the security requirements are met, `socket_common()` proceeds to `falloc()` the `fd` file descriptor in the calling process, marking its `f_flag` for `FREAD | FWRITE`, and linking the socket operations structure (`socketops`) to its `f_ops`. The flow then diverges to either `socreate` or `socreate_delegate`, but only artifically, as both merge in `socreate_internal()` (bsd/kern/uipc_socket.c), which handles the socket creation. If the socket is successfully created, the descriptor's `f_data` is linked to the new object, reference counts are adjusted using `procfdtbl_releasefd()` and `fp_drop()` on the `fd`, and the integer is returned to the caller.

socreate_internal()

`socreate_internal()` (from bsd/kern/uipc_socket.c) resolves the family (now considered a "domain"), type and protocol combination to a `struct protosw` pointer, by calling `pffindproto()` with these three arguments. If a protocol was not specified, the default protocol is retrieved by `pffindtype()`, taking only the domain and the type. If that, too, is unsuccessful, an error is returned - `EAFNOSUPPORT` (if `pffinddomain()` fails), `EPROTONOSUPPORT` (if domain is found, but protocol is not supported), or `EPROTOTYPE` (if protocol was found, but mismatches specified type).

A call to `soalloc()` allocates the memory required for the socket structure. As a BSD-level object, the allocation is performed using `MALLOC_ZONE` from the `socket_zone` (`M_SOCKET`), but IPv4 TCP sockets (`AF_INET/SOCK_STREAM`) are directly allocated from the Mach `socache.zone`. Each element in that zone is significantly greater (2,432 in Darwin 18, 2,488 in 19) than the socket, as it accounts also for the IPv4 PCB (`struct inpcb`) and TCPv4 (`struct tcpcb`).

Once the socket is allocated, the `net_api_stats` total for this address family is incremented. The socket's many fields can then be initialized. First, the type is saved (in the `so_type`), and then the pid, unique pid, and UUID of the process creating the socket (directly or by delegation) are saved in respective fields of the structure. The credentials of the caller are saved, with the `SS_PRIV` toggled in the `so_state` for privileged callers. The resolved protocol is saved to the socket's `so_proto` field, and the MACF layer is notified of label association. The socket use count is incremented (effectively, set to 1), and the protocol's `pru_attach` callback is called. Assuming the attachment is successful, the domain's ref count is incremented, the socket's event list is initialized, and socket filters (if any) can be attached. The default traffic class is set, and - if the creator holds `TASK_POLICY_NEW_SOCKETS_BG`, the socket is marked as `TRAFFIC_MGT_SO_BACKGROUND` and associated with the `current_thread()`. Lastly, UNIX domain, system sockets and multipath are marked `SOF_NODEFUNCT`, and the socket is ready.

sockbufs

Each `socket` holds a `so_rcv` and `so_snd`, both `sockbuf` structures (defined inline as part of `struct socket`, in bsd/sys/socketvar.h), containing pointers to the lower-level buffer (`mbuf`) chains and metadata. Figure 14-2 shows these important structures:

Listing 14-2: The `sockbuf` structure

```
struct sockbuf {
        u_int32_t       sb_cc;          /* actual chars in buffer */
        u_int32_t       sb_hiwat;       /* max actual char count */
        u_int32_t       sb_mbcnt;       /* chars of mbufs used */
        u_int32_t       sb_mbmax;       /* max chars of mbufs to use */
        u_int32_t       sb_ctl;         /* non-data chars in buffer */
        u_int32_t       sb_lowat;       /* low water mark */
        struct mbuf     *sb_mb;         /* the mbuf chain */
        struct mbuf     *sb_mbtail;     /* the last mbuf in the chain */
        struct mbuf     *sb_lastrecord; /* first mbuf of last record */
        struct socket   *sb_so;         /* socket back ptr for kexts */
        struct selinfo  sb_sel;         /* process selecting rd/wr */
        struct timeval  sb_timeo;       /* timeout for read/write */
        u_int32_t       sb_flags;       /* flags, see below */
        u_int32_t       sb_idealsize;   /* Ideal size for the sb based */
                                        /* on bandwidth and delay      */
        void    (*sb_upcall)(struct socket *, void *arg, int waitf);
        void    *sb_upcallarg;          /* Arg for above */
        u_int32_t       sb_wantlock;    /* # of SB_LOCK waiters */
        u_int32_t       sb_waiters;     /* # of data/space waiters */
        thread_t        sb_cfil_thread; /* content filter thread */
        u_int32_t       sb_cfil_refs;   /* # of nested calls */
        u_int32_t       sb_preconn_hiwat; /* preconnect hiwat mark */
} so_rcv, so_snd;
#define SB_MAX (8192*1024) /* default for max chars in sockbuf */ #define LOW_SB_MAX (2*9*1024)
/* lower limit on max socket buffer size, 2 max datagrams */
```

As explained in the header comment (in bsd/kern/uipc_socket2.c) explains the usage of socket buffers in detail:

Listing 14-3: The `sockbuf` structure

```
/*
* Socket buffer (struct sockbuf) utility routines.
*
* Each socket contains two socket buffers: one for sending data and
* one for receiving data.  Each buffer contains a queue of mbufs,
* information about the number of mbufs and amount of data in the
* queue, and other fields allowing select() statements and notification
* on data availability to be implemented.
*
* Data stored in a socket buffer is maintained as a list of records.
* Each record is a list of mbufs chained together with the m_next
* field.  Records are chained together with the m_nextpkt field. The upper
* level routine soreceive() expects the following conventions to be
* observed when placing information in the receive buffer:
*
* 1. If the protocol requires each message be preceded by the sender's
*    name, then a record containing that name must be present before
*    any associated data (mbuf's must be of type MT_SONAME).
* 2. If the protocol supports the exchange of ``access rights'' (really
*    just additional data associated with the message), and there are
*    ``rights'' to be received, then a record containing this data
*    should be present (mbuf's must be of type MT_RIGHTS).
* 3. If a name or rights record exists, then it must be followed by
*    a data record, perhaps of zero length.
*
* Before using a new socket structure it is first necessary to reserve
* buffer space to the socket, by calling sbreserve().  This should commit
* some of the available buffer space in the system buffer pool for the
* socket (currently, it does nothing but enforce limits).  The space
* should be released by calling sbrelease() when the socket is destroyed.
*/
```

When incoming packets arrive, their `mbuf`s are connected to the `socket`'s `so_rcv` sockbuff by calling one of the `soappend*()` variants (from bsd/kern/uipc_socket2.c, none are KPI), and shown in Table 14-4:

Table 14-4: The `sbappend*` variants (as of XNU 4903)

sbappend* variant	Purpose
sbappend(*sb, m*)	Append `mbuf` chain *m* to `sockbuf` *sb*
sbappendstream(*sb, m*)	As above, optimized from stream sockets
sbappendrecord(*sb, m0*)	
sbappendchain(*sb, m, space*)	
sbappendaddr(*sb*, struct sockaddr *asa, *m0, control, error_out*)	
sbappendcontrol_internal(*sb, m0,*	
sbappendcontrol(*sb, m0, control, error_out*)	
sbappendmsgstream_rcv(*sb, m, seqnum, unordered*)	
sbappendstream_rcvdemux(*so, m, seqnum, unordered*)	
sbappendmptcpstream_rcv(*sb, m*)	Specific for MPTCP use
sbappendmsg_snd(*sb, m*)	

All of the variants call out to any socket filters (`sflt_data_in()`) and content filters (`cfil_sock_data_in()`), which may claim the data (by returning a non-zero `error` code). Otherwise, the actual appending of the data is performed (unless `SB_NOCOMPRESS`) by `sbcompress()`, which is called with the target `sockbuf`, the `mbuf` chain, and an already appended `mbuf` on the `sockbuf`.

When data is available (or when a TCP window slides) a call to `so[r/w]wakeup` wakes up waiters on the corresponding `so_[rcv/snd]` buffer, if the buffer's `sb_waiters` indicates such waiters exist, and the flags indicate interest (`SB_SEL`, `SB_ASYNC`, `SB_UPCALL`, or `SB_KNOTE`), per `sb_notify()` (all in bsd/net/uipc_socket2.c).

mbufs

Network packets require a standardized representation. Linux has `sk_buff`, Windows has `NET_BUFFER_LIST`, and XNU follows the BSD model, where in **MBufs** are used.

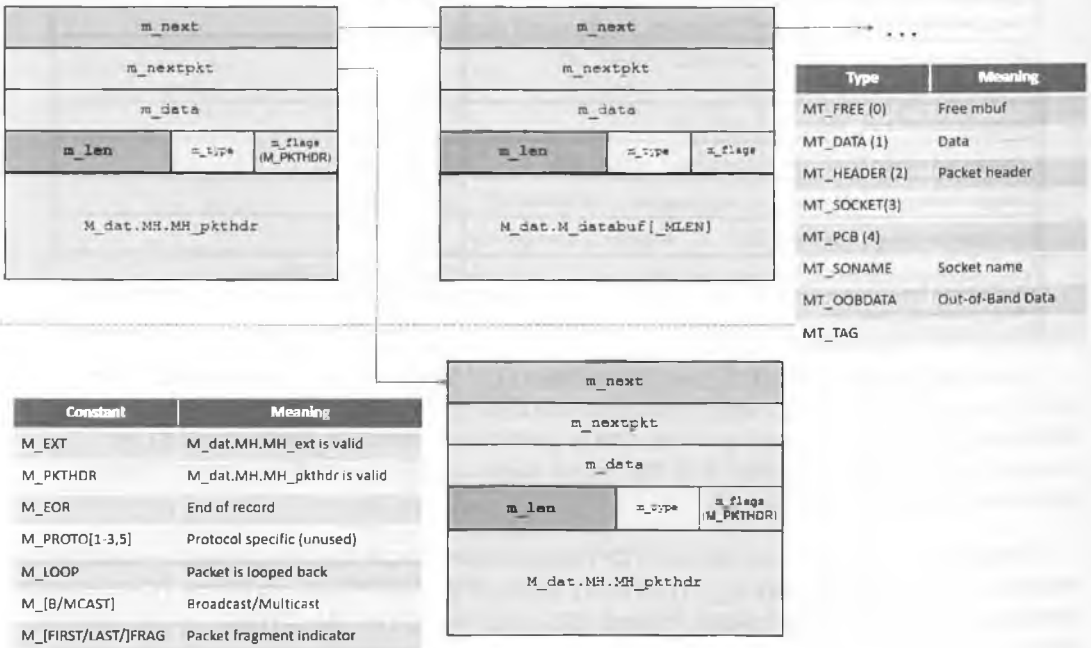
Listing 14-5: The `struct mbuf` definition

```
/*
 * The mbuf object
 */
struct mbuf {
        struct m_hdr m_hdr;
        union {
                struct {
                        struct pkthdr MH_pkthdr;        /* M_PKTHDR set */
                        union {
                                struct m_ext MH_ext;    /* M_EXT set */
                                char    MH_databuf[_MHLEN];
                        } MH_dat;
                } MH;
                char    M_databuf[_MLEN];               /* !M_PKTHDR, !M_EXT */
        } M_dat;
};
```

The `struct mbuf` starts with a fixed `struct m_hdr`, is comprised of a union containing another unions. The `m_hdr`'s `mh_flags` indicate which fields of the union are the relevant ones. When the `M_PKTHDR` is set, the `m_hdr` is followed by a `struct pkthdr`. When the `M_EXT` header is set, the next (or first, if no `M_PKTHDR`), is a `struct m_ext` containing external storage data. Data follows next, but if `M_TAGHDR` is set, the data is preceded by a `struct m_tag`, "tagging" the packet with module (=driver) details.

An `mbuf` will not contain more than one packet, but a packet may span multiple `mbufs`. The first `mbuf` belonging to a packet will be marked `M_PKTHDR`, as its `MH_dat.MH_databuf` will contain the packet header. Additional `mbufs` may be chained through the header's `mh_next` (aliased as `m_next` for easy access). As more packets arrive, their `mbuf` list heads can be chained through `m[h]_nextpkt` (similarly aliased). Figure 14-6 shows a simple example of `mbuf` chains.

Figure 14-6: Visualizing `mbuf` structures



Each mbuf also has a specific type, denoted by `m_hdr.mh_type` (or simply `m_type`). Types are defined as `MT_*` constants in bsd/sys/mbuf.h, `MBUF_TYPE_*` aliases in bsd/sys/kpi_mbuf.h, and string names in the `mbtypes` array (from bsd/kern/uipc_mbuf.c). There are some 16 types, but not all are used in XNU:

Table 14-7: `mbuf` types used in XNU

Type	Payload
`MT_FREE`	A free or recycled `mbuf` for free list
`MT_DATA`	Generic protocol data
`MT_OOBDATA`	Out-Of-Band data, marked by protocol. Similar to `MSG_OOB`
`MT_HEADER`	A packet header
`MT_SONAME`	A socket name (address)
`MT_CONTROL`	Control information
`MT_TAG`	Tagging

Note, that the `mbuf` should remain opaque. The rich KPI of `bsd/sys/kpi_mbuf.` abstracts `mbuf` handling with getters and setters, and over 100 other functions. Nonetheless, the next section delves a bit deeper into their implementation, focusing on lifecycle and handling functions. Fortunately, these KPIs are in the `com.apple.kpi.bsd` bucket, and the header is decently documented.

`mbuf` lifecycle

To obtain an `mbuf`, a client can call `MGET[HDR]` macros in bsd/sys/mbuf.h, shown in Listing 14-8. (next page) The caller needs to specify a `how` parameter, denoting either of `M_WAIT/M_DONTWAIT`, whether or not the allocation is allowed to block. The `mbuf` can be freed through a call to the `MFREE` macro. As explained in the header, `mbufs` are all of a fixed `MSIZE`, set in [x86_64/arm]/machine/param.h to 256 (in all architectures). Should a bigger size be necessary, an `mbuf` **cluster** may be used. Clusers normally hold up to `MCLBYTES` (2,048), but bigger cluster sizes (`MBINGCLBYTES`, (4,096) or even `M16KCLBYTES`) can be used. The `mbuf_get_minclsize()` KPI (from bsd/sys/kpi_mbuf.h) can be used to determine the minimal cluster size.

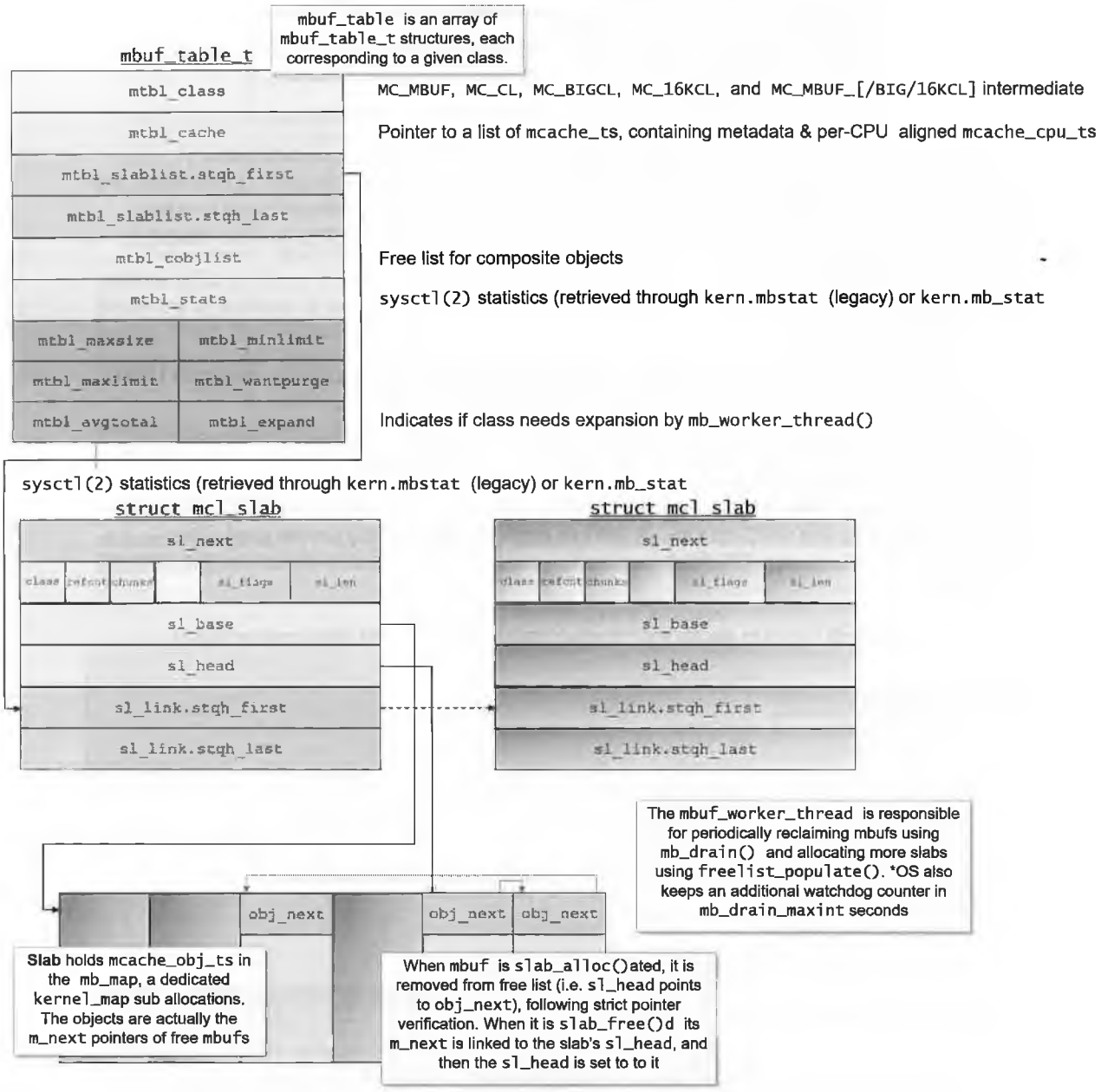
Listing 14-8: Macros to allocate and free `mbufs` and clusters (from XNU 4903's bsd/sys/mbuf.h)

```
/*
 * mbuf allocation/deallocation macros:
 *
 *      MGET(struct mbuf *m, int how, int type)
 * allocates an mbuf and initializes it to contain internal data.
 *
 *      MGETHDR(struct mbuf *m, int how, int type)
 * allocates an mbuf and initializes it to contain a packet header
 * and internal data.
 */
#define MGET(m, how, type)      ((m) = m_get((how), (type)))
#define MGETHDR(m, how, type)   ((m) = m_gethdr((how), (type)))
/*
 * MFREE(struct mbuf *m, struct mbuf *n)
 * Free a single mbuf and associated external storage.
 * Place the successor, if any, in n.
 */
#define MFREE(m, n) ((n) = m_free(m))
/*
 * Mbuf cluster macros.
 * MCLALLOC(caddr_t p, int how) allocates an mbuf cluster.
 * MCLGET adds such clusters to a normal mbuf;
 * the flag M_EXT is set upon success.
 * MCLFREE releases a reference to a cluster allocated by MCLALLOC,
 * freeing the cluster if the reference count has reached 0.
 *
 * Normal mbuf clusters are normally treated as character arrays
 * after allocation, but use the first word of the buffer as a free list
 * pointer while on the free list.
 */
union mcluster {
        union   mcluster *mcl_next;
        char    mcl_buf[MCLBYTES];      // 2048
};
#define MCLALLOC(p, how)        ((p) = m_mclalloc(how))
#define MCLFREE(p)              m_mclfree(p)
#define MCLGET(m, how)          ((m) = m_mclget(m, how))
```

mbufs were traditionally allocated from the BSD M_MBUF zone, backed by the mbuf Mach zone. Clusters - groups of related mbufs - were allocated from M_MCLUST, backed by mbuf.clust. Additional BSD zones (M_RTABLE, M_HTABLE, and M_FTABLE) for special mbuf types, originally shared M_MBUF.

The zones still exist, but none of them are in use. The mbuf allocation model has shifted to a separate allocation model, backed by BSD's mcache mechanism. The mcache is an efficient memory allocation mechanism over the per-CPU caches. If an available mbuf cannot be obtained, allocation falls to a global layer, the **mbuf slab** (as documented by a verbose header comment in bsd/kern/uipc_mbuf.c. The slab allocator can be visualized as shown below:

Figure 14-9: Visualizing the mbuf slab allocator mechanism



The MBuf slab uses a separate **slab allocator** (self contained in bsd/kern/uipc_mbuf.h), which uses a dedicated kernel submap - the mb_map - and an mbuf_worker_thread. The thread, started by mbinit(), blocks mbuf_thread_needs_wakeup, using itself as a continuation. When woken up, it calls mb_drain() to purge all caches, and freelist_populate() on any of the mbuf regions - MC_CL, MC_BIGCL and/or MC_16KCL - which need it, based on the metadata in the mbuf_table.

XNU also supports an "mbuf watchdog", toggled through kern.ipc.mb_watchdog (set by default on *OS). This also triggers drainage up to every kern.ipc.mb_drain_maxint seconds. Additionally, kern.ipc.mb_drain_force MIB can trigger mbuf drainage.

Sockets in kernel mode

There are times when kernel mode code - commonly, some kernel extension - needs to create or manipulate a socket structure. Examples of this can be found in network file system drivers (e.g. acfs.kext, smbfs.kext or webdav_fs.kext), or the IO80211Family[V2].kexts (which require in-kernel `sock_ioctl` functionality).

Apple provides the functions in bsd/kern/kpi_socket.c, which offer a different take on the well known user mode programming paradigms*. The KPI (part of `com.apple.kpi.bsd`, and documented nicely in bsd/sys/kpi_socket.h) is very similar to the API of user mode, albeit prefixed by `sock_..` (e.g. `sock_accept`, etc), with the obvious changing of the socket parameter from an `int` (the user mode file descriptor) to a `socket_t`. In-kernel sockets are thus created by `sock_socket()`, etc. There are a few useful functions with no user mode counterparts as well, shown in Table 14-10:

Table 14-10: Socket KPIs with no user mode counterparts

KPI	Purpose
`sock_connectwait`	As `sock_connect()`, but with timeout value (an NFS hack)
`sock_[get/set]tclassopt`	Get/set socket traffic class (an AFP hack)
`sock_receivembuf`	Variant of `sock_receive` providing direct `mbuf`s
`sock_sendmbuf`	Variant of `sock_send` providing direct `mbuf`s
`sock_[retain/release]`	Increment/decrement `so_retaincnt` and `so_usecount`
`sock_setpriv`	Toggle marking as privileged (`so_state`'s `SS_PRIV`)
`sock_isconnected`	Retrieve connected state (`so_state`'s `SS_ISCONNECTED`)
`sock_isnonblocking`	Retrieve non-blocking state (`so_state`'s `SS_NBIO`)
`sock_gettype`	Retrieve socket's address family (domain), type and protocol
`sock_nointerrupt`	Toggle `SB_NOINTR` on socket's receive/send buffers
`sock_setupcall[s]`	Provide async read/write `sock_upcall` callback
`sock_catchevents`	Provide async `sock_evupcall` event callback
`sock_getlistener`	Get listening socket
`sock_getaddr`	Combined `get[sock/peer]addr()` to retrieve `sockaddr` of socket
`sock_freeaddr`	BSD `FREE(sa, M_SONAME)`
`sock_iskernel`	Returns true if kernel-created socket (`so->last_pid == 0`)

The `so_flags1` `SOF1_IN_KERNEL_SOCKET` is used to mark in-kernel sockets, but is never actually checked in XNU proper. Instead, `sock_iskernel()` checks if the `last_pid` owner of the socket is 0, implying kernel.

To provide non-blocking I/O, the KPI supports "upcall" functions. These are callbacks which are invoked with the `socket_t` in question, a `void *` argument (referred to as a 'cookie'), and a boolean indicating whether the callback can be blocked. Normally, the same callback can be used for both read and write operations, and is either set on creation (i.e. as an argument to `sock_create()` or `sock_accept()`). The callback can also be set later (or changed) on an existing socket can be set by either `sock_setupcall()` or `sock_setupcalls()` (for separate read and write callbacks). A third routine, `sock_catchevents()` sets upcalls for event notifications. The `sock_evupcall` event upcalls are of a slightly different type, accepting the event type as an integer, rather than the boolean.

Setting an upcall through any of the above transparently handles the required locking and unlocking of the socket around calling `.._locked` variants. When using upcalls of either type, however, care must be taken as the upcalls may be called from multiple thread contexts. There is therefore no guarantee of context, nor of serialized execution - any any locking and concurrency constraints are up to the implementation.

* - This capability is not unique to Darwin - Windows has Winsock Kernel (WSK) functions providing similar functionality. Linux does not offer a separate API, instead allowing `sock_create` etc. to be called by modules.

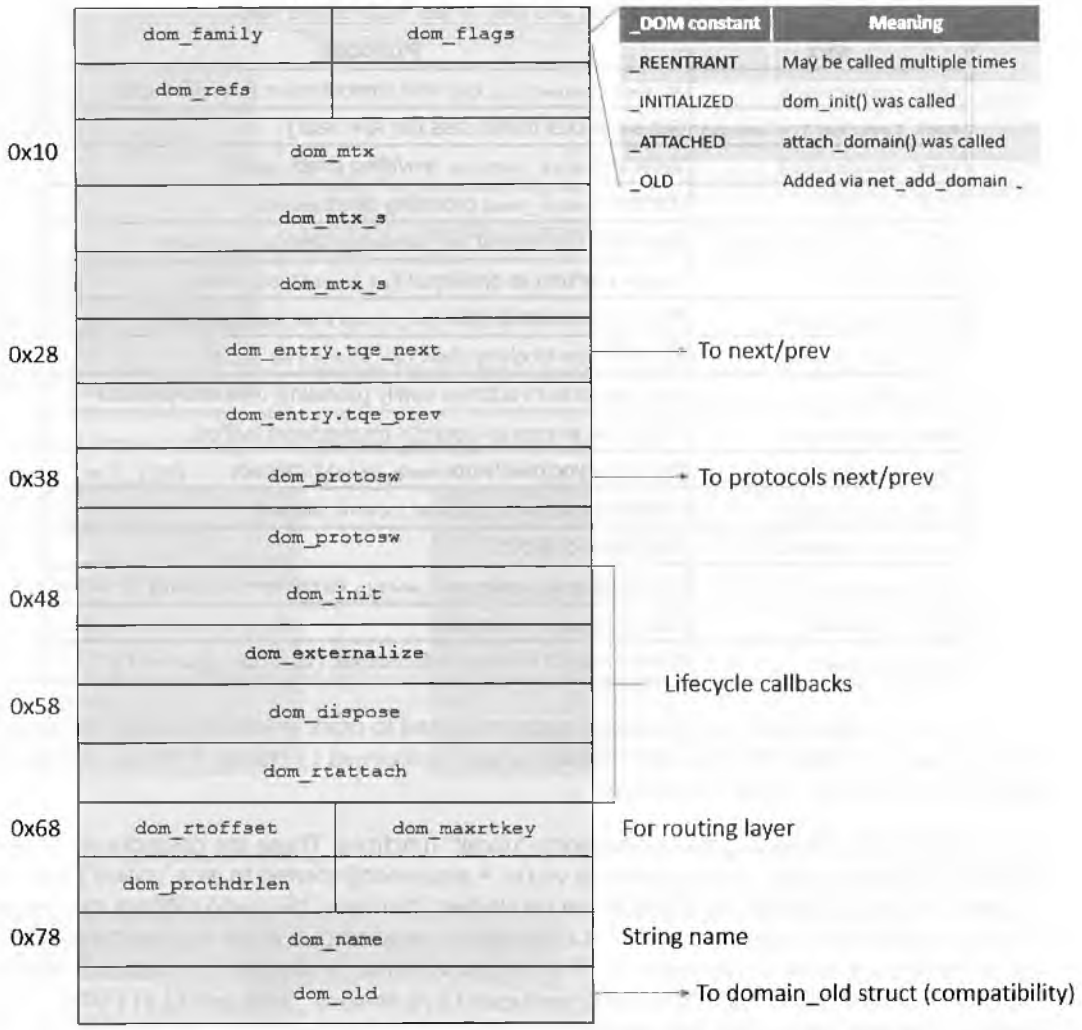
Layer IV: Domains & Protocols

As the previous section has shown, every socket is associated with an address family and a protocol. User mode treats these as two integers - `AF_*` and `*PROTO*` constants - but in kernel mode these resolve to complicated structures and function pointers.

Domains

What user mode calls address (or protocol) families, XNU calls **domains**. A domain, defined in bsd/sys/domain.h, is quite literally a protocol family, as its main element is `dom_protosw`, a `TAILQ_HEAD` of `protosw` structures (described shortly). The structure also keeps additional meda, such as flags, a reference count, and a mutex, as well as a set of callbacks for the domain lifecycle, as shown in Figure 14-11:

Figure 14-11: The `struct domain`, visualized



There are several hard-coded domains (e.g. `inet[6]domain_s`, `localdomain_s` and other `..domain_s` globals) in XNU, spread around across several files bsd/kern, bsd/net and bsd/netinet. The `domaininit()` routine (from bsd/kern/uipc_domain.c) calls `attach_domain()` on each of them. `attach_domain()` takes the domain pointer and `TAILQ_INSERT_TAIL()`s them into the global `domains` list (which `domaininit()` holds the lock for), also raising their `DOM_ATTACHED` (0x1) flag. The `attach_domain()` routine is not KPI, but additional domains may be added by extensions using `net_add_domain[_old]` (with the `.._old` variant exported for backward compatibility), though the only Apple supplied extension known to do that is PPP.kext.

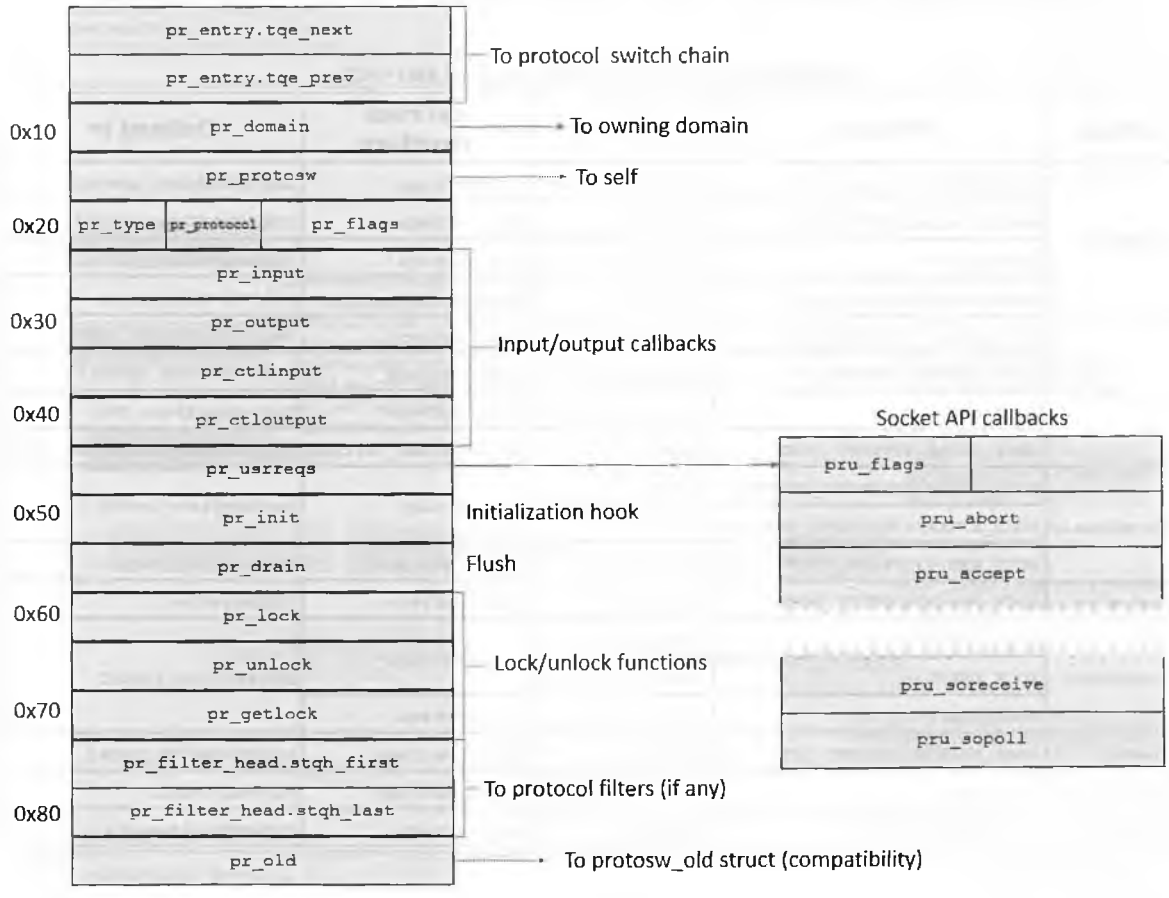
After a domain is attached, a call to `init_domain()` calls the domain's `dom_init` handler (if not NULL), and raises its `DOM_INITIALIZED` (0x2) flag. It follows, therefore, that domains ready for use will have both flags set (0x6). Externally added domains will also have `DOM_OLD` (0x10000000). Another flag, `DOM_REENTRANT` (0x1), marks domains whose protocol input functions may be run concurrently in multiple threads (e.g. `PF_INET[6]`, `PF_MULTIPATH`, `PF_KEY` and `PF_NDRV`), to ensure adequate locking.

The `domains` global list and the hardcoded `*domain_s` structures are actually exported in MacOS, but not meant for direct access. The `pffinddomain()` (resolving a `PF_*` value to its `struct domain` pointer) is deemed private KPI, as are `net_[add/del]_domain_old()` routines. In addition to the `dom_mtx` protecting each domain from concurrent modifications, global domain operations (i.e. traversing or manipulating the domains lists) use either `domain_guard_[deploy/release]()` or `domain_unguard_[deploy/release]()`, with the former pair locking and unlocking the private `domain_proto_mtx()` (thereby guarding the `domains` list), and the latter doing the opposite ("unguarding").

Protocols

Each domain holds a list of 0 (practically, 1) or more protocols in its `dom_protosw` field, a `TAILQ_HEAD`. These structures, which are also linked to the created sockets (in `so_proto`) provide the implementation of the transport layer protocols. Figure 14-12 illustrates the structure:

Figure 14-12: The `struct protosw`, visualized



The `protosw` structure is chained (via the `pr_entry`) to the other protocols in its domain. The domain pointer is also kept for quick reference in `pr_domain`, and a self-referential pointer is also kept at `pr_protosw`. The `pr_type` and `pr_protocol` fields hold the socket and protocol types (e.g. `SOCK_STREAM`/`IPPROTO_TCP`). The `pr_flags` holds various combinations of `PR_*` flags, `#defined` and documented in bsd/sys/protosw.h.

The rest of the `struct protosw`'s fields are all pointers. As with the `struct domain`, there is a `pr_old` pointer at the end of the structure, pointing to a `protosw_old`, for binary compatibility. The other pointers are all function pointers, which include:

- **Protocol lifecycle callbacks**: starting with `pr_init`, called from `init_proto()` after a protocol has been attached to a domain.
- **Protocol user requests:** stored in a separate `pr_usrreqs` structure, this collection of function pointers maps to the various socket system calls which the protocol implementation is expected to handle.

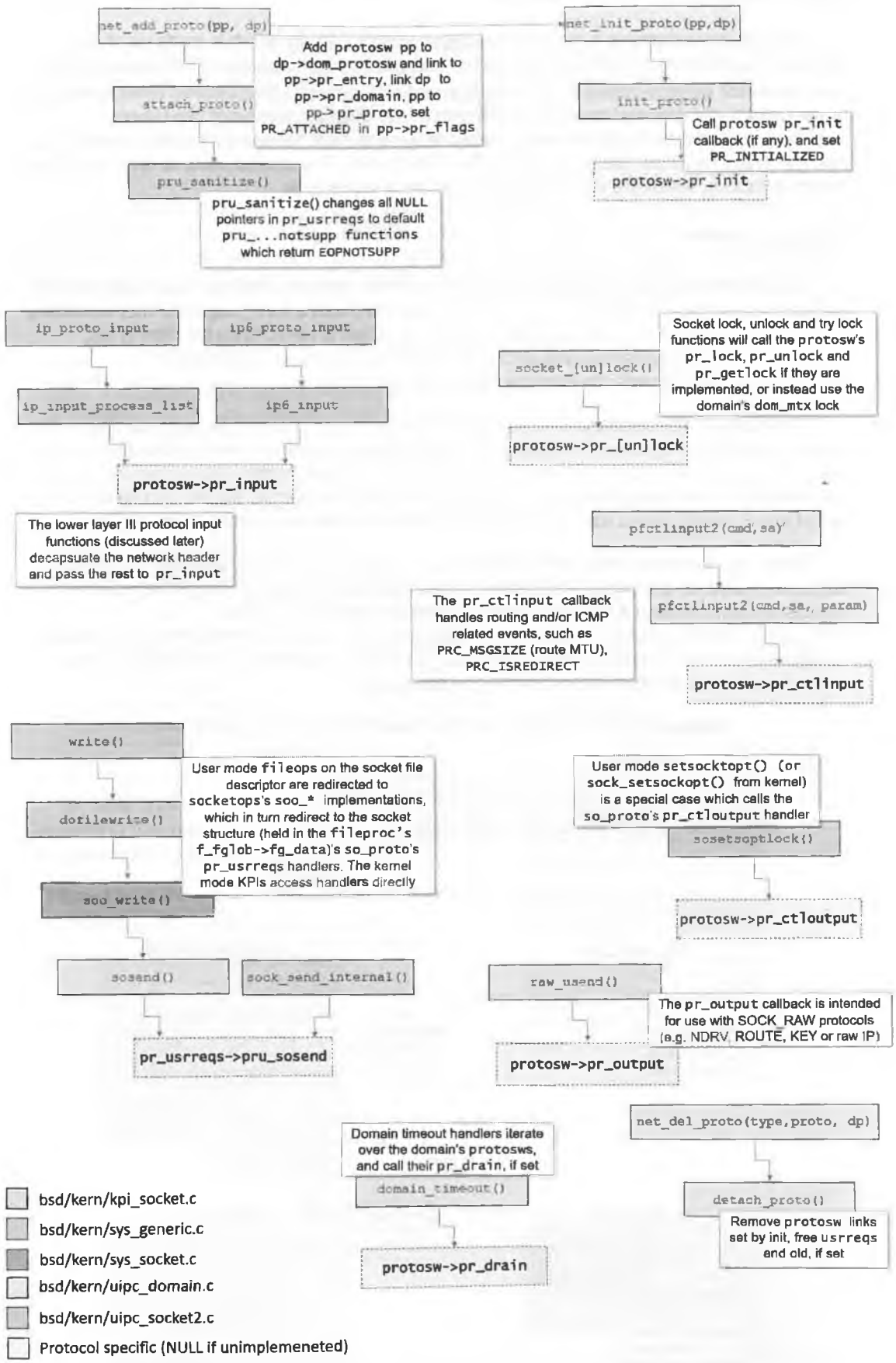
Figure 14-14 shows the `protosw` and the `pr_usrreqs` callbacks, in the context of the `protosw` lifecycle. A protocol is first added to the domain, which involves calling `attach_proto()` to perform the necessary linkage to the domain and any other `protosws` in the chain. Protocols don't necessarily implement all user-mode callbacks, so `pru_sanitize()` takes care of providing stub implementations, which return `ENOTSUPP` to their callers. Addition also requires initialization of the protocol, which results in a call to the protocol's `pr_init` callbck, if set. Deleting a protocol, requires no destructor function, and merely performs `detach_proto()` and an optional `FREE` of the `proto_old` and `pr_usrreqs` structures.

As with the supported address families (= domains), here, too, are built-in protocols which are hard-coded into XNU's source tree. Most of them, however, (particularly the `PF_INET[6]` ones), are initialized from arrays, by calls to `net_add_proto()` and `net_init_proto()` from their domain's initializer. Table 14-13 lists some of these protocols; Others (such as GRE, IPSec, etc) have user request callbacks (i.e. their `pr_usrreqs` pointer is set to `nousrreqs`) and are left out.

Table 14-13: The `usrreqs` instances (as of XNU 4903)

Domain	Protocol	protosw	usrreqs structure	Defined in
inetdomain	SOCK_DGRAM/IPPROTO_UDP	inetsw[0]	udp_usrreqs	bsd/netinet/udp_usrreq.c
	SOCK_STREAM/IPPROTO_TCP	inetsw[1]	tcp_usrreqs	bsd/netinet/tcp_usrreq.c
	SOCK_RAW/*	inetsw[2,3,5,6]	rip_usrreqs	bsd/netinet/raw_ip.c
	SOCK_DGRAM/IPPROTO_ICMP	inetsw[4]	icmp_dgram_usrreqs	bsd/netinet/ip_icmp.c
inet6domain	SOCK_DGRAM/IPPROTO_UDP	inet6sw[0]	udp6_usrreqs	bsd/netinet6/udp6_usrreq.c
	SOCK_STREAM/IPPROTO_TCP	inet6sw[1]	tcp6_usrreqs	bsd/netinet/tcp_usrreq.c
	SOCK_RAW/*	inet6sw[2,3]	rip6_usrreqs	bsd/netinet6/raw_ip6.c
	SOCK_DGRAM/IPPROTO_ICMP	inet6sw[4]	icmp6_dgram_usrreqs	bsd/netinet6/netinet6/raw_ip6.c
systemdomain	SOCK_DGRAM/SYSPROTO_CONTROL	kctlsw[0]	ctl_usrreqs	bsd/kern/kern_control.c
	SOCK_STREAM/SYSPROTO_CONTROL	kctlsw[1]		
	SOCK_RAW/SYSPROTO_EVENT	eventsw[0]	event_usrreqs	bsd/kern/kern_event.c
ndrvdomain	SOCK_RAW/NDRVPROTO_NDRV	ndrvsw[0]	ndrv_usrreqs	bsd/net/ndrv.c
localdomain	SOCK_STREAM/*	localsw[0]	uipc_usrreqs	bsd/kern/uipc_proto.c
	SOCK_DGRAM/*	localsw[1]		
	SOCK_RAW/*	localsw[2]	raw_usrreqs	
mpdomain	SOCK_STREAM/IPPROTO_TCP	mpsw[0]	mptcp_usrreqs	bsd/netinet/mp_proto.c
routedomain	SOCK_RAW/*	routesw[0]	route_usrreqs	bsd/net/rtsock.c
keydomain	SOCK_RAW/PF_KEY_V2	keysw[0]	key_usrreqs	bsd/netkey/keysock.c

Figure 14-14: The `protosw` lifecycle calls



Case Study: PF_SYSTEM sockets

Darwin's PF_SYSTEM is a proprietary address/protocol family which is made up of two protocols - SYSPROTO_EVENT and SYSPROTO_CONTROL. Apple documents both types in the (now archived) Network Kernel Extensions Programming Guide[1]. I/16 showed some examples of using both - the former, for system notifications (in a manner somewhat like Linux's netlink(7)), and the latter, to control networking subsystem functionality (with a specific example given for utun, the User mode TUNnelling facility. This section looks at the kernel-level implementations, which are entirely different for the two protocols.

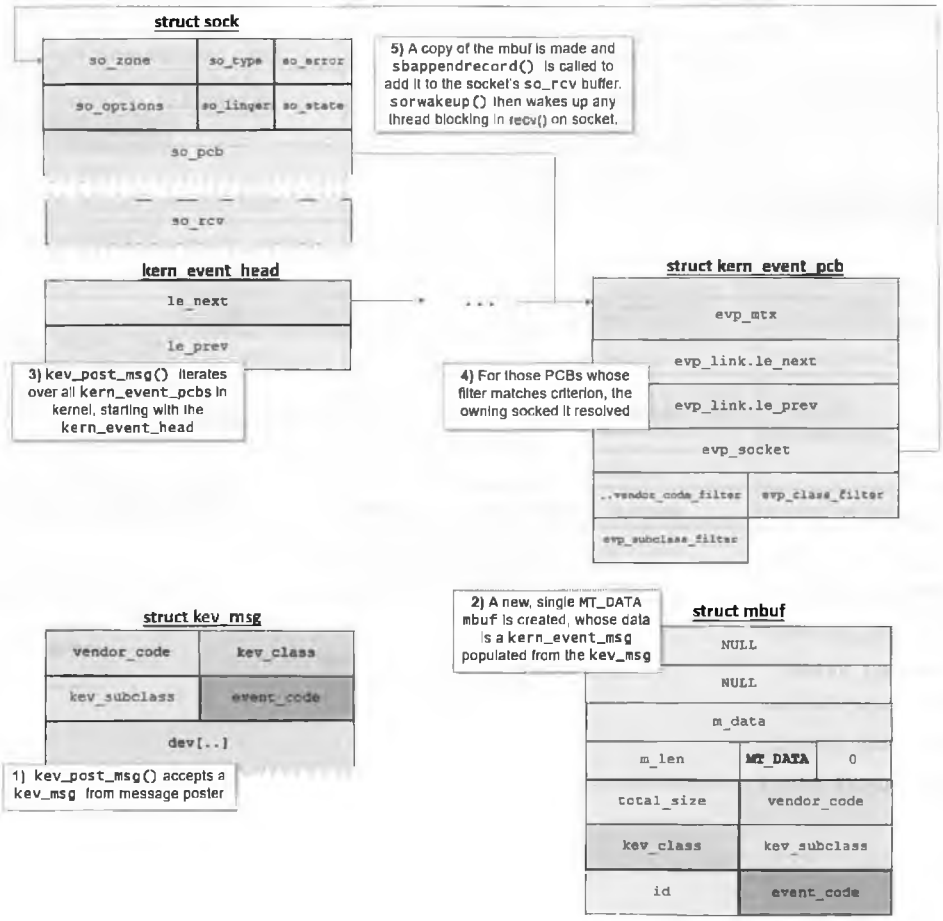
SYSPROTO_EVENT

The SYSPROTO_EVENT sockets are raw PF_SYSTEM sockets which the user mode caller is expected to use the SIOCSKEVFILT ioctl(2) code on, with a kev_request filter consisting of a vendor_code, kev_class and kev_subclass (see I/16-3). Once the filter is set, read(2)/recv(2) operations will block, until matching events are available as kern_event_msg records. An example client is shown in I/16-4.

The internal representation of a PF_SYSTEM/SYSPROTO_EVENT is fairly simple. When the socket is created, the protocol's attachment function (kev_attach()) calls soreserve() to ensure KEV_SNDSPACE and KEV_RECVSPACE. It then zalloc()ates a kern_event_pcb for the socket from the dedicated kerneventpcb zone, ties it back to its socket, and adds it to the global kern_event_head list, wherein the kernel keeps track of all event PCBs.

When the user mode caller sets the filter, kev_control() (in bsd/kern/kern_event.c) copies the values to the socket's PCB. Then, when a kevent is signalled kev_post_msg() iterates the list, checking if the filter settings match (or the filter specified KEV_ANY_[VENDOR/CLASS/SUBCLASS]). Upon a match a copy of the event record is made, sbappendrecord() adds it to the socket's so_rcv buffer, statistics are incremented, and sorwakeup() wakes any process blocking on the socket.

Figure 14-15: The flow from a posted kev_msg to the SYSPROTO_EVENT socket



SYSPROTO_CONTROL

The PF_SYSTEM/SYSPROTO_CONTROL sockets are datagram sockets which the user mode caller is expected to create, and then connect() to, specifying a sockaddr_ctl with an sc_id, obtained by requesting a CTLIOCGINFO ioctl(2) on the unconnected socket, with a ctl_info specifying the control's string name. The socket can then be used with normal recv(2)/send(2) calls, per the control's specific protocol semantics (demonstrated by I/16-7).

The control name specified in the CTLIOCGINFO must belong to kernel code which has a priori called ctl_register(). The call takes a kern_ctl_reg structure (from bsd/sys/kern_control.h), allowing for specifying control settings (id, unit, flags, send and receive sizes), as well as the callback functions implemented for the user mode socket calls, as shown in Figure 14-16:

Figure 14-16: The kern_ctl_reg structure (from XNU 4903's bsd/sys/kern_control.h)

```
struct kern_ctl_reg {
    /* control information */
    char        ctl_name[MAX_KCTL_NAME];
    u_int32_t   ctl_id;
    u_int32_t   ctl_unit;

    /* control settings */
    u_int32_t   ctl_flags;
    u_int32_t   ctl_sendsize;
    u_int32_t   ctl_recvsize;
    /* Dispatch functions */
    ctl_connect_func    ctl_connect;
    ctl_disconnect_func ctl_disconnect;
    ctl_send_func               ctl_send;
    ctl_setopt_func             ctl_setopt;
    ctl_getopt_func             ctl_getopt;
#ifdef KERNEL_PRIVATE
    ctl_rcvd_func               ctl_rcvd;       /* Only valid if CTL_FLAG_REG_E
    ctl_send_list_func          ctl_send_list;  /* Only valid if CTL_FLAG_REG_E
    ctl_bind_func               ctl_bind;
#endif /* KERNEL_PRIVATE */
};
```

As Table I/16-6 has shown, there is no shortage of SYSPROTO_CONTROL providers. Considering the network.statistics provider (from bsd/net/ntstat.c) we focus on its ctl_send_func, nstat_control_send():

Figure 14-17: The ctl_send_func handler of the ntstat control (from XNU 4903's bsd/sys/kern_control.h)

```
static errno_t nstat_control_send(
        kern_ctl_ref    kctl, u_int32_t                 unit,
        void    *uinfo, mbuf_t                  m, __unused int    flags) {

        nstat_control_state     *state = (nstat_control_state*)uinfo;
        struct nstat_msg_hdr    *hdr;
        struct nstat_msg_hdr    storage;
        errno_t                                 result = 0;

        if (mbuf_pkthdr_len(m) < sizeof(*hdr))
        {
                // Is this the right thing to do?
                // is it, indeed? :-)
                mbuf_freem(m);
                return EINVAL;
        }
        if (mbuf_len(m) >= sizeof(*hdr)) { hdr = mbuf_data(m); }
        else {
                mbuf_copydata(m, 0, sizeof(storage), &storage);
                hdr = &storage;
        }
        ...
}
```

The user mode data arrives in the mbuf argument. Accessing it in kernel mode the control uses local storage, which is set to the size of a complete nstat_msg_header. The mbuf packet header len is checked to ensure it is at least that size. If it is, the complete mbuf len is checked. If the mbuf fits in the header (which it might, for a message without payload) it is copied using mbuf_copydata. Otherwise, a pointer to the data is retrieved using the mbuf_data() accessor. From this point, the data can be accessed like any other memory.

Layer III: Network Protocols

Unlike layer IV (transport) protocols, which are plentiful, XNU supports only two layer III (network layer) protocols - IPv4 (`AF_INET`) and IPv6 (`AF_INET6`) - since the rest of the protocol families are all of local scope. Leaving aside the detailed inner workings of their packet processing, we consider only the flows through them.

Incoming packets

Incoming packet callbacks are registered using `proto_register_input()`, which the two protocols call from their respective domain startup functions (i.e. `in_dinit()` and `ip6_init()`). The `proto_register_input()` routine (from bsd/net/kpi_protocol.c) `MALLOC()`s a `proto_input_entry`, and links it to the private `proto_input_add_list`. The function also raises the `DLIL_PROTO_REGISTER` on the `dlil_main_input_thread` global's `input_waiting` field.

The Data Link Interface Layer (DLIL, discussed in the next section) continuously executes `proto_input_run()` from a dedicated input thread. When the routine picks up the `DLIL_PROTO_REGISTER` indication, it acts on it: The `proto_input_add_list` is processed (by `proto_delayed_attach()`), and the list entries put into another private structure, the `proto_hash_table`. The entries in the hash table can then be looked up by a call to the `proto_hash_value()` KPI (from bsd/net/dlil.c), though the name might be a bit misleading, as this is just a simple `switch()` statement returning hard-coded values: 0 for `PF_INET`, 1 for `PF_INET6`, 2 for `PF_VLAN` and 3 in all other cases.

The `struct proto_input_entry` is defined in bsd/net/kpi_protocol.c as follows:

Listing 14-18: The `struct proto_input_entry` (from XNU 4903's bsd/net/kpi_protocol.c)

```
struct proto_input_entry {
  struct proto_input_entry      *next;       // hash table link (effectively, always NULL)
  int                           detach;      // set by proto_unregister_input()
  struct domain                 *domain;     // Owning domain (whose dom_family == protocol)
  int                           hash;        // hash index in proto_hash table
  int                           chain;       // can operate on packet_lists?

  protocol_family_t             protocol;    // PF_* constant
  proto_input_handler           input;       // Input callback
  proto_input_detached_handler  detached;    // Input during detachment (unused)

  mbuf_t                        inject_first; // Packet injected by filters/diverters, if any
  mbuf_t                        inject_last;  // Last packet injected, if any

  struct proto_input_entry      *input_next; // Unused (NULL)
  mbuf_t                        input_first; // Unused (NULL)
  mbuf_t                        input_last;  // Unused (NULL)
};
```

The `proto_input_entry` allows for two callbacks. The first (`input`) is the main handler called from the DLIL when a packet matching the address family (the entry's `protocol`) is received. The two handlers registered here are `ip_proto_input()` and `ip6_proto_input()`, to perform IP-layer tasks, locate the correct `protosw`, and call its `pr_input` handler (see Figure 14-14). There is asecond callback, `detached` (meant for cases wherein the protocol is detached from an interface, so that the protocol may be safely unloaded), but neither family makes use of this.

When the DLIL input thread is woken up and `proto_input_run()` is executed, it iterates over the `proto_hash` table, and processes any `mbuf`s injected into the entry (found in its `inject_first` and `inject_last`) by `proto_inject()`. This is rare, but IP filters, dummynet and IP divert make use of this, as described later in this chapter. Normal packets received from the interface are processed by `proto_input()`. Unlike `proto_input_run()`, `proto_input()` is called from DLIL with an `mbuf` packet list. If the entry's `chain` is set, the list is passed to its `input` handler. Otherwise, it is iterated through, `mbuf_nextpkt()` at a time, as each packet is fed to the `input` handler.

For egress packets, there is no structured way to register protocols. The various layer IV `protosw`s are aware of their lower layer networking protcols, and call the output functions directly. This means there are no set structures or prototypes to conform to. Both IPv4 and IPv6 operate similarly - with an `ip[6]_output()` function to send a single `mbuf`, which calls on an `ip[6]_output_list()` variant, taking the same arguments, but supporting another, `packet_chain`, indicating if the `mbuf` is chained to more packets to process.

Listing 14-19: The `ip[6]_output_list()` definitions

```
/*
 * IP output.  The packet in mbuf chain m contains a skeletal IP
 * header (with len, off, ttl, proto, tos, src, dst).
 * The mbuf chain containing the packet will be freed.
 * The mbuf opt, if present, will not be freed.
 *
 * Route ro MUST be non-NULL; if ro->ro_rt is valid, route lookup would be
 * skipped and ro->ro_rt would be used.  Otherwise the result of route
 * lookup is stored in ro->ro_rt.
 *
 * In the IP forwarding case, the packet will arrive with options already
 * inserted, so must have a NULL opt pointer.
 */
int
ip_output_list(struct mbuf *m0, int packetchain, struct mbuf *opt,
    struct route *ro, int flags, struct ip_moptions *imo,
    struct ip_out_args *ipoa)


/*
 * IP6 output. Each packet in mbuf chain m contains a skeletal IP6
 * header (with pri, len, nxt, hlim, src, dst).
 * This function may modify ver and hlim only.
 * The mbuf chain containing the packet will be freed.
 * The mbuf opt, if present, will not be freed.
 *
 * If ro is non-NULL and has valid ro->ro_rt, route lookup would be
 * skipped and ro->ro_rt would be used.  Otherwise the result of route
 * lookup is stored in ro->ro_rt.
 *
 * type of "mtu": rt_rmx.rmx_mtu is u_int32_t, ifnet.ifr_mtu is int, and
 * nd_ifinfo.linkmtu is u_int32_t.  so we use u_int32_t to hold largest one,
 * which is rt_rmx.rmx_mtu.
 */
int
ip6_output_list(struct mbuf *m0, int packetchain, struct ip6_pktopts *opt,
    struct route_in6 *ro, int flags, struct ip6_moptions *im6o,
    struct ifnet **ifpp, struct ip6_out_args *ip6oa)
```

The flow of either or both these functions is outside the scope of this work, and quite complex, accounting for numerous considerations, such as fragmentation, encapsulation, routing, output interface determination, and miscellaneous protocol specifics. Suffice it to say that, when packets are ready, they are provided to the DLIL using a call to `dlil_output()`.

Layer II: Interfaces - The Data Link Interface Layer

Interfaces are the standard mechanism by means of which clients can contact other hosts. As client bind their sockets to one or another interface, the abstraction takes care of framing the protocol packets according to media requirements, as well as decapsulating packets in incoming frames, and transferring them to their respective protocols. Chapter 16 (in particular, Table 16-10) detailed the many interface types supported by XNU and its networking kernel extensions, and we now take a look at the mechanism internals.

XNU's **Data Link Interface Layer**, commonly referred to as the DLIL, is the code providing the interface abstraction. The code is self contained, in bsd/net/dlil.c, with bsd/net/dlil.h offering the in-kernel API. This is, however, still deemed as private, and extensions should use the supported `ifnet_*` KPIs provided in bsd/net/kpi_interface.h, and exported through com.apple.kpi.bsd.

The `struct ifnet`

The various network interfaces etc are handled in kernel by a `struct ifnet`. This structure, however, is in reality just the visible portion (first field) of a larger `struct dlil_ifnet`, which remains private to the DLIL (i.e. defined solely in bsd/net/dlil.c). The private fields are all protected by a `dl_if_lock`, and include `dl_if_flags` (flags marking `DLIF_[INUSE/REUSE/DEBUG]`, a unique ID, and the `dl_if_inpstorage`, a `dlil_threading_info` structure for thread details and storage space.

Interface lifecycle

The interface lifecycle begins with a call to one of a few `ifnet_allocate()` variants, all of which call `ifnet_allocate_extended()`. The extended function takes an *in* parameter of `ifnet_init_eparams`, and returns the allocated `interface` as an *out* parameter (with a return code specifying success or error). The `ifnet_init_eparams` supplied by the caller are checked for validity, and a unique id is created from the name and the unit number (e.g. "en0", "utun1", etc).

The actual allocation is then performed by `dlil_if_acquire()`, which is in charge of either creating a new interface, or recycling an existing one (given to it as an argument. Before doing anything, `dlil_if_acquire()` first makes sure that the interface creation parameters it was supplied with - the interface name and uniqueid - are indeed unique. If they are, and no existing interface was supplied, it `zalloc()`s a `dlil_ifnet` element from the dedicated `ifnet` zone.

Once an interface is allocated (or recycled), select fields of the structure are initialized: The name, the uniqueid, the `dl_if_flags` marking `DLIF_INUSE`, various locks, and the thread call which will be used. The new interface is linked to the interface list, a `TAILQ` maintained through the `dlil_ifnet_head`.

As the flow returns to `ifnet_allocate_extended()`, the rest of the `struct ifnet` parameters are set according to the caller supplied `ifnet_init_eparams`. Counts and bandwidth are set, and the interface reference count is increased by one, using `ifnet_reference()`.

When the interface is ready for use, its owner calls `ifnet_attach()`, supplying the link layer address for it. This adds the interface to another `TAILQ`, the `ifnet_head`, takes another `ifnet_reference()`. The interface also gets its own `if_proto_hash`, allocated from the `ifnet_proto_hash` zone. Most importantly, `IFT_[ETHER/CELLULAR]` interfaces or those marked with `IFEF_RXPOLL` (opportunistic polling) get their own kernel thread for input, spun by `dlil_create_input_thread()`, and using the `dl_if_inpstorage`. Interfaces also get a `..PRIORITY_KERNEL` thread call, held in the `input_mit_tcall` field.

At last, following many additional settings and sanity checks, the interface can be marked as attach through the `IFRF_ATTACHED` flag in its `if_refflags`. A call to `pf_ifnet_hook()` gives the PF facility (discussed later) a chance to attach to the interface. Additionally, a kevent of `KEV_DL_SUBCLASS/KEV_DL_IF_ATTACHED` is fired using `dlil_post_msg()`.

Case Study: The UTUN interface

The user mode tunneling interfaces (utun) make for a textbook example of handling interface creation. Recall (from I/16-7) that utun interfaces can be created from user mode by a call to `connect(2)` over a `PF_SYSTEM SYSPROTO_CONTROL` socket with the `UTUN_CONTROL_NAME` (i.e. `com.apple.net.utun_control`). The `connect(2)` makes its way (through the `ctl_usrreqs`'s `pru_connect` function pointer to the utun's control `ctl_connect` handler, `utun_ctl_connect()` (in bsd/net/if_utun.c). The code to create the interface on the fly (with the nexus calls omitted) is shown in Listing 14-20:

Listing 14-20: Creating a network interface, from `utun_ctl_connect()`

```
struct ifnet_init_eparams utun_init = {};
..
bzero(&utun_init, sizeof(utun_init));
utun_init.ver = IFNET_INIT_CURRENT_VERSION;
utun_init.len = sizeof (utun_init);
utun_init.name = "utun";                        // interface name
utun_init.unit = pcb->utun_unit - 1;            // unit number appended to name
utun_init.uniqueid = pcb->utun_unique_name;     // %s%d..
utun_init.uniqueid_len = strlen(pcb->utun_unique_name);
utun_init.family = utun_family;
utun_init.subfamily = IFNET_SUBFAMILY_UTUN;
utun_init.type = IFT_OTHER;                     // Data link type, from sys/if_types.h
utun_init.demux = utun_demux;                   // Used to determine PF_* from frame
utun_init.add_proto = utun_add_proto;           // Used for protocol attachment
utun_init.del_proto = utun_del_proto;           // Used for protocol detachment
utun_init.softc = pcb;                          // Driver specific storage
utun_init.ioctl = utun_ioctl;                   // Provides ioctl() support
utun_init.detach = utun_detached;               // Called from ifnet_detach()

// Nexus code omitted

        /*
         * Upon success, this holds an ifnet reference which we will
         * release via ifnet_release() at final detach time.
         */
        result = ifnet_allocate_extended(&utun_init, &pcb->utun_ifp);
        if (result != 0) {
                printf("utun_ctl_connect - ifnet_allocate failed: %d\n", result);
                utun_free_pcb(pcb, true);
                *unitinfo = NULL;
                return result;
        }

        /* Set flags and additional information. */
        ifnet_set_mtu(pcb->utun_ifp, UTUN_DEFAULT_MTU);
        ifnet_set_flags(pcb->utun_ifp, IFF_UP | IFF_MULTICAST | IFF_POINTOPOINT, 0xffff);

        /* The interface must generate its own IPv6 LinkLocal address,
         * if possible following the recommendation of RFC2472 to the 64bit interface ID
         */
        ifnet_set_eflags(pcb->utun_ifp, IFEF_NOAUTOIPV6LL, IFEF_NOAUTOIPV6LL);

        /* Reset the stats in case as the interface may have been recycled */
        struct ifnet_stats_param stats;
        bzero(&stats, sizeof(struct ifnet_stats_param));
        ifnet_set_stat(pcb->utun_ifp, &stats);

        /* Attach the interface */
        result = ifnet_attach(pcb->utun_ifp, NULL);
        if (result != 0) {
                 printf("utun_ctl_connect - ifnet_attach failed: %d\n", result);
                /* Release reference now since attach failed */
                ifnet_release(pcb->utun_ifp);
                utun_free_pcb(pcb, true);
                *unitinfo = NULL;
                return (result);

        }

        /* Attach to bpf */
        bpfattach(pcb->utun_ifp, DLT_NULL, UTUN_HEADER_SIZE(pcb));
}

/* The interfaces resoures allocated, mark it as running */
ifnet_set_flags(pcb->utun_ifp, IFF_RUNNING, IFF_RUNNING);
..
```

Network Data Processing

With all the layers explored, we can now piece together the two most important aspects of the network stack - sending and receiving data. We begin with the (usually) more synchronous of the two, sending. Listing 14-21, although long (over the next two pages), demonstrates a (greatly) abbreviated and annotated flow a DTrace `fbt` probe on `sosend_nocancel`, for a UDP packet (TCP's is considerably longer, due to windowing/buffer considerations).

Sending Data

Data is sent when the caller - user mode `send[/to/msg/file](2)` syscalls, or in kernel `sock_send[mbuf]()` - submits data to the socket. Both paths require the creation of a `uio`, which is then provided to the socket's registered protocol handler (i.e. `so_proto`) `pru_sosend` callback (in the `pr_usrreqs`).

Taking the `AF_INET[6]` cases, both TCP and UDP use `sosend()` (from bsd/kern/uipc_socket.c) as a common sender function. This first calls `sosendcheck()` (ibid.) to verify there is enough space in the socket's `so_snd` buffer, possibly `sbwait()`ing on it to become available (in the `SOCK_STREAM` case). `send()` will then allocate `mbuf`s (or clusters) for the data provided in the `uio`.

Before proceeding, both socket filters (`sflt_data_out()`) and content filters (`cfil_sock_data_out()`) are given a chance to inspect the outgoing packet, unless the `MSG_SKIPCFIL` was explicitly specified. As with the other filtering case we've seen, the code returned from the respective filters may be 0 (allowed), `EJUSTRETURN` (claimed by filter), or some other (discard and free). If the filters agree (or none are set), processing goes to the transport protocol's `pru_send()` callback.

Here, roads diverge. Assuming a datagram protocol such as UDP, sending is quick and unbuffered. In the stream case, acknowledgments and buffering have to be dealt with. The UDP case (`udp[6]_send()`, from bsd/netinet[6]/udp[6]_usrreq.c) is thus quite simple - calling `sotoinpcb()` to retrieve the protocol control block, checking if NECP decrees a flow diversion is in order, and - if not - simply pass the pcb to `udp_output()` (ibid.), or `udp6_output()` (from bsd/netinet6/udp6_output.c). The UDPv6 case also has to handle IPv4 mapped addresses (i.e. IPv4 addresses posing as IPv6 in order to simplify the user-mode address handling), in which case the packet must be subjected to potential flow diversion, before being reevaluated with the UDP's `pru_send`.

The processing performed by `udp[6]_output()` is somewhat more involved, figuring out the route by means of which this packet is to be sent. NECP is consulted to figure out a matching flow, and QoS marking is applied accordingly. The UDP header is prepended, and then the packet is subjected to `ip[6]_output()`.

TCP's `tcp_usr_send()` takes the `mbuf` and calls `sbappendstream()`. A special case here is that of an implied connect, i.e. the three way handshake which must be completed (using `tcp[6]_connect`) if the socket has not been connected. Otherwise, `tcp_output` is called.

`tcp_output()` (from bsd/netinet/tcp_output.c) is significantly more complicated than its UDP[6] counterparts, owing to retransmission, windows and timeout cases. Leaving those aside, flow ends up in `tcp_ip_output()` (ibid.), which dances between the IPv4 and IPv6 cases (as it serves both), and eventually send the packet through `ip[6]_output[_list]()`, from bsd/netinet[6]/ip[6]_output.c.

IPv4/IPv6 packet output

IPv4 and IPv6 are substantially different, but we will focus on similarities. The `ip[6]_output()` variants are single `mbuf` versions, which funnel to `ip[6]_output_list()`. Packets are subjected to NECP, which may affect IPSec encapsulation and tunneling. At this level, any registered IP filters must be called (`ipf_output()`), subject to the usual convention. Following intensive routing logic, as the packet is ready to be send, `pf_af_hook()` is called to allow `PF` to potentially filter the packet.

If the packet is allowed egress, (assuming no fragmentation), the `mbuf` list may be sent. Sending is performed in both cases through `dlil_output()`, though in the IPv6 case it goes through `nd6_output_list()` (from bsd/netinet6/nd6.c, for IPv6 Neighbor Discovery) first.

Listing 14-21: An abbreviated, yet annotated output of a DTrace `fbt` probe on `sendto_nocancel()`

```
-> sendto_nocancel                          (struct proc *p, ffffff80332158c0 ffffff8033215900,...)
##
## Create UIO, Lookup socket from descriptor
##
  -> uio_create                             1 0 8
    -> uio_addiov                             uio_t auio, 70000f628d30 30
    -> file_socket                            (int fd= 7, struct socket **sp)
##
## MACF checks
##
     -> mac_socket_check_send           (kauth_cred_get(), struct socket *so, struct sockaddr *to,..
        ... (lengthy Sandbox checks omitted
##
## Send approved by MACF
##
  -> sosend                              (struct socket *so, struct sockaddr *to, uio_t auio,..)
   -> udp_lock                          struct socket *so, 1 ffffff80200f3d2a
   -> so_update_last_owner_locked       struct socket *so, struct proc *p, 3000000
      -> proc_pidoriginatorpid_uuid     (uuid_t so->[e/last]_uuid 10 ffffff80b837bc0c
        -> mach_voucher_attr_command 0 3 1
   -> inp_update_policy                 (struct inpcb *inpcb, 3, 1..)
      -> proc_uuid_policy_lookup          (uuid_t so->[e/last]_uuid, ffffff80b837bbfc ffffff802fbbafe8
      -> inp_update_necp_policy           (struct inpcb *inpcb, 0 struct sockaddr *to,
##
## Check send possible, notify content filter data requirement
##
   -> sosendcheck                       struct socket *so, struct sockaddr *to, 30
      -> sblock                           struct sockbuf *sb, 1 30
      -> cfil_sock_data_space             struct sockbuf *sb, ffffff802fbbade8 ffffff80203c42d7
   -> cfil_sock_data_space              struct sockbuf *sb, 0 ffffff80203c4201
   -> uiomove64                         ffffff80b3fbbad0 30 uio_t auio, .. )
#
# Socket filter callout, picked up by Application Firewall:
#
   -> sockwall_sflt_data_out      ffffff80305df800 struct socket *so, struct sockaddr *to,
      -> sockwall_lock             ffffff80305df800 struct socket *so, struct sockaddr *to,
         -> sockwall_unlock           0 ffffff802af5f8a0 3000000
#
# Pass to Layer IV: (UDP)
#
   -> udp_send                      (struct socket *so, 0 struct mbuf *m,..)
      -> udp_output                    (struct inpcb *inpcb, struct mbuf *m, struct sockaddr *to, ..)
        -> in_pcbconnect               (struct inpcb *inpcb, struct sockaddr *to, struct proc *p)
          -> in_pcbladdr               (struct inpcb *inpcb, struct sockaddr *to, ffffff80b837baa0
            -> sa_copy                 ffffff802cb87e0c ffffff80b837b7d0 ffffff80b837b74c
           <- in_pcbladdr               6e2 0 0
           -> socket_unlock             struct socket *so, 0 0
             -> udp_unlock              struct socket *so, 0 ffffff801ff1e81b
             -> in_pcblookup_hash       ffffff8020611648 7d0efd11 7b00
               -> inp_restricted_recv ffffff80350892e8 0 4
               -> necp_socket_is_allowed_to_recv_on_interface ffffff80350892e8 0 4
             -> udp_lock                struct socket *so, 0 ffffff801ff6bab5
##
## Get random ephemeral send port for UDP
##
             -> in_pcbbind              struct inpcb *inpcb, 0 struct proc *p)
               -> socket_unlock         (struct socket *so, 0)
                  -> udp_unlock          (struct socket *so, 0)
                     -> read_frandom       ffffff80b837b98e 2 1
                       -> cckprng_generate ffffff7fa052b3c0 0 2
                     -> netns_reserve    ffffff802cb87ed0 ffffff80b837b9b0 4
                         -> udp_lock          struct socket *so, 0 ffffff801ff1e9a1
                         -> in_pcbinshash     struct inpcb *inpcb, 1 3000000
                             -> kalloc_canblock ffffff80b837b8b8 1 ffffff8020470bb8
                             -> netns_reserve ffffff802cb87ed0 ffffff80b837b8b8 4
                             -> netns_set_ifnet ffffff802cb87ed0 0 0
                             -> netns_change_flags ffffff802cb87ed0 0 e0
                             -> necp_socket_find_policy_match struct inpcb *inpcb, 0 0
            -> netns_set_ifnet    ffffff802cb87ed0 struct ifnet *ifp, 21000000
                  -> necp_socket_find_policy_match struct inpcb *inpcb, 0 0
                    -> net_flowhash_mh3_x64_128 ffffff80b837b8c0 70 19
##
##  Pass to Layer III: ip_output_list
##
     -> ip_output_list (struct mbuf *m)
       -> ip_randomid 85 85 ffffff80b3fbbab4
        ##
        ##  Call cckprng_refresh, .._generate, and ccaes_vng_ctr_crypt
        ##  for a random packet id
       -> ifa_remref ffffff802af83a50 0 1000000
       -> lck_mtx_unlock_slow ffffff802af83a50 ffffff802af83a50 1000000
```

Listing 14-21: An abbreviated, yet annotated output of a DTrace `fbt` probe on `sendto_nocancel()` (cont.)

```
## Pass to NECP
##
       -> necp_ip_output_find_policy_match  (struct mbuf *m, 100, ffffff80b837bb48,..)
       -> necp_packet_is_allowed_over_interface (struct mbuf *m, struct ifnet *ifp, 21000000...)
##
## Ready to output with DLIL:
##
       -> ip_output_checksum (struct ifnet *ifp, struct mbuf *m, 14)
       -> dlil_output (struct ifnet *ifp, 2 struct mbuf *m,..)
        ##
        ##  Perform ARP lookup
        ##
           -> arp_lookup_ip (struct ifnet *ifp, struct sockaddr *, ffffff80b837b5a0, ..)
             -> route_to_gwroute (struct sockaddr *, ffffff803048aef0 ffffff80b837b4b0,..)
                ...
             <- route_to_gwroute f24 0 0
             -> in_broadcast 252a8c0 struct ifnet *ifp, 0
                ...
           <- arp_lookup_ip 4e8 0 0
           -> ether_frameout_extended struct ifnet *ifp, ffffff80b837b6d8 struct sockaddr *,
             -> m_prepend_2 struct mbuf *m, e 1
             <- m_prepend_2 bb struct mbuf *m2, 0
             -> lck_mtx_unlock_slow ffffff802e07f098 ffffff802e07f098 14
             <- lck_mtx_unlock_slow c9 ffffff802040cec0 0
           <- ether_frameout_extended 16c 0 0
           -> lck_mtx_unlock_slow ffffff802e07eb00 ffffff802e07eb00 6
           -> mbuf_is_service_class_privileged struct mbuf *m2, 0 ffffff801fe02a60
##
## Actual Output through dlil_output_handler(), which calls the interface's function
##
           -> dlil_output_handler (struct ifnet *ifp, struct mbuf *m2)
             -> ifnet_enqueue (struct ifnet *ifp, struct mbuf *m2)
               -> ifclassq_enqueue ffffff802e07e950 ffffff80b837b5a0 ffffff80b837b5dc
                 -> fq_addq ffffff802e081000 ffffff80b837b4c0 ffffff802e081c58
                   -> fq_if_hash_pkt ffffff802e081000 cb59be7e 0
                     -> lck_mtx_convert_spin ffffff802e07e950 cb59be7e 0
                     <- lck_mtx_convert_spin 43 ffffff802040cec0 0
                     -> fq_alloc 1 cb59be7e 0
                     <- fq_alloc 87 ffffff802b356fd8 0
                   <- fq_if_hash_pkt 215 ffffff802b356fd8 0
                 <- fq_addq 3b2 0 0
               <- ifclassq_enqueue 51 0 0
               -> _global_eventq ffffff80b837b510 8 1000000
               <- _global_eventq f3 ffffff80916e74a0 0
               -> waitq_wakeup64_thread ffffff80916e74a0 ffffff802e07e948 ffffff802c927000
               -> lck_mtx_unlock_slow ffffff802e07e908 ffffff802e07e908 1
             <- ifnet_enqueue 21 0 0
             -> ifnet_datamov_end (struct ifnet *ifp, 0 0) # Darwin 19
             -> lck_mtx_unlock_slow ffffff802e07e710 ffffff802e07e710 0
           <- dlil_output 4dd 0 0
              ...
##
##   Return, and clean up:
##
       <- udp_output  15e8 0 0
       -> sbunlock    struct sockbuf *sb, 0 0
       <- sosend      2b0 0 0
       -> file_drop   7 struct socket *so, 1000000
       -> kfree       uio_t auio, 48 0
        -> zfree/gzalloc_free/zcache_free_to_cpu_cache...
     <- sendto_nocancel 14d 0 0
   <= sendto
```

DLIL output

The DLIL output is, thankfully, much simpler compared to its upper layers. The registered protocol's `pre_output` function (from its v1 or v2 KPI structure) is called, if present. This gives a chance for any last nanosecond rerouting decisions. A call to `mac_ifnet_check_transmit()` allows any interested security policy kext to inspect, and possibly bar the packet from being output.

If the interface has a framer function, it is called to wrap the layer III payload (= packet) in a layer II header and trailer (= frame). With the frame assembled, a call to `dlil_interface_filters_output()` allows any interface filters to perform a last minute block of the frame(s). If still allowed, the frame is subjected to the packet tap (`pktap_output()`). Finally, the frame(s) may be sent using the `ifnet`'s `if_output_dlil()`, which is normally the `dlil_output_handler()`. This just wraps the interface's own `if_output` handler.

Receiving data

Data is received when the network device generates an interrupt, signaling the availability of data. The device driver will process the interrupt (usually, through an `IOInterruptSource`, and then signal the DLIL through `ifnet_input[_extended]()` (from bsd/net/dlil.c, which expect either an `mbuf_queue` from `m_head` to m_tail, or a single packet `mbuf` (i.e. passing in `m_head == m_tail` or `m_tail == NULL`).

DLIL frame reception

Both cases fall through to `ifnet_input_common()`. DLIL may run an input sanity check at this point (`#if INET_INPUT_SANITY_CHECK`, which is indeed set, and the `net.link.generic.system.dlil_input_sanity_check` MIB is set). Assuming checks are ok, the interface's input function - in its `if_input_dlil` field - may be called. This was set (during `ifnet_allocate_extended()`) to DLIL's standard `dlil_input_handler()`.

The `dlil_input_handler()` obtains the interface's `dlil_threading_info` structure (from `if_inp`). If none can be found, it assumes the `dlil_main_input_thread`. Using `dlil_affinity_tag()`, the current thread tag is set to the same tag as in the `dlil_threading_info`, effectively creating an affinity between all threads handling input on the same interface. The `mbuf` queue is then chained to the `dlil_threading_info`'s `rcvq_pkts`, using `_addq_multi()`.

Once more, there is an optional `dlil_input_sanity_check()`, to validate the `mbuf` count. The statistics are updated, and then frame processing is set to proceed - flagging the `dlil_threading_info`'s `input_waiting`, and then waking up the dedicated input thread (created during `ifnet_allocate()`) - directly, or via the `input_mit_tcall` thread call.

The input thread is either looping endlessly in a `dlil_input_thread_func()`, or (if opportunistic polling was indicated using `IFEF_RXPOLL`), in `dlil_rxpoll_input_thread_func()`. Both functions sleep on the `input_waiting` indicator. In the simple case, the thread dequeues all packets in the `dlil_threading_info`'s `rcvq_pkts`, using `_getq_all()`. Assuming the thread was not instruced to terminate (using the `DLIL_INPUT_TERMINATE` flag in `input_waiting`, as occurs during interface detachment), stats are updated, and `dlil_input_packet_list_extended()` is called to further process the incoming `mbufs`.

As with the other `..extended` variants, this falls into a `..common()` handler, which is ready to transfer it to the next layer, using the interface's registered `if_demux` function to determine which protocol family this `mbuf` can be handled by. Irrespective of which, `pktap_input()` is called for the packet tap facility (I/16) to pass frame to BPF and/or sniffers.

CLAT (IPv4 in IPv6) requires special handling and translation, but sooner or later the frame is subjected to interface filters (using `dlil_interface_filters_input`) which may claim the frame (as discussed later in this chapter). If not, the protocol family is looked up using `find_attached_proto()`, or a cached `last_ifproto`, to expedite the process.

With the protocol family located, `dlil_ifproto_input()` relays the frame to the upper layer, using the interface's attached protocol input function - which is either a "v1" or "v2", depending on which KPI was used to register it.

IPv4/IPv6 packet input

The two possible cases for layer III are `ip[6]_proto_input()`, which were registered during their respective protocol initializations (i.e. `ip_init()` or `ip6_init()`). Without getting into the RFC-level handling and idiosyncrasies, both functions are fairly similar, and involve more sanity checks, QoS checks, and calling `PF` hooks to give a chance for both filtering mechanisms to claim the packet (by returning `EJUSTRETURN`), allowing it (by returning 0), or forcing a discard and free (by any other return value).

If the packet survives PF, IPv4 dispatches it to the transport layer through `ip_proto_dispatch_in()`, which subjects it to more scrutiny by IP filters. The IPv6 handler (`ip6_input()`) does so directly (with no dispatch function), unless the interface is the "integrated coprocessor", i.e. the IPv6 Ethernet interface used to communicate with BridgeOS. The layer IV protocol in both cases is performed by first finding the `[ip6]protosw` in the `ip[6]_protox` tables, and then calling the `pr_input` handler. The flow at this point (using another DTrace `fbt` probe) is shown in Listing 14-22:

Listing 14-22: An abbreviated, yet annotated output of a DTrace `fbt` probe on `ip_input()`

```
-> ip_input                              (struct mbuf *packet_list);
  -> lck_mtx_convert_spin                ffffff802e07ed28 ffffff802e07ed28 ffffff80b404380e
  -> route_copyout                       ffffff8026cc3b40 ffffff802e07ed70 30
  -> rtunref                             ffffff802af899f8 ffffff802af89a58 1000000
#
# Layer III: UDP
#
  -> udp_input                           (struct mbuf *packet_list, int iphlen = 14)
    -> in_broadcast                      8252a8c0 struct ifnet *ifp,  ffffff80b404380e
    -> in_pcblookup_hash                 ffffff8020611648 252a8c0 3500
      -> IORWLockRead                    ffffff802aed6930 252a8c0 3500
      -> inp_restricted_recv             (struct inpcb *inp, struct ifnet *ifp)
      -> necp_socket_is_allowed_to_recv_on_interface struct inpcb *inp, struct ifnet *ifp)
      -> inp_restricted_recv             (struct inpcb *inp2, struct ifnet *ifp)
      -> necp_socket_is_allowed_to_recv_on_interface struct inpcb *inp2, struct ifnet *ifp,  0
      -> IORWLockUnlock                  ffffff802aed6930 struct ifnet *ifp,  0
    <- in_pcblookup_hash                 248 struct inpcb *inp2, 0
    -> in_pcb_checkstate                 (struct inpcb *inp2, 2 1..)
    -> IORWLockRead                      ffffff802065ab60 ffffff8026cc3b00 struct inpcb *inp2,
    -> net_flowhash_mh3_x64_128          ffffff8026cc38b8 70 19
    -> IORWLockUnlock                    ffffff802065ab60 36a2537e3f6f8c16 0
    -> ip_savecontrol                    struct inpcb *inp2, ffffff8026cc3a50 ffffff80b404380e
      -> usimple_lock                    ffffff802049d618 ffffff8026cc38dc ffffff80b404380e
      -> sbcreatecontrol_mbuf            ffffff8026cc38e0 10 2
        -> sbcreatecontrol               ffffff8026cc38e0 10 2
      <- sbcreatecontrol_mbuf            b1
    <- ip_savecontrol                    429
    -> m_adj                             (struct mbuf *packet_list, 1c 1)
    -> in_stat_set_activity_bitmap       ffffff802cb589e0 ffa2 0
    -> sbappendaddr                      (struct sockbuf *sb, struct sockaddr *from, ,struct mbuf *packet_
      -> socket_unlock                   (struct socket so, 0)
        -> udp_unlock                    (struct socket so, 0)
#
#       Application Firewall picks up... Note also call to content filter, but none registered
#
      -> sockwall_sflt_data_in           (struct socket so, struct sockaddr *from,
          -> mbuf_pkthdr_rcvif           (struct mbuf *packet_list)
          -> ifnet_flags                 struct ifnet *ifp,  struct socket so, struct sockaddr *from,
          -> sockwall_lock               struct ifnet *ifp,  struct socket so, struct sockaddr *from,
          <- sockwall_unlock             1c ffffff802af5f8a0 0
        <- sockwall_sflt_data_in         2ac
        -> IORWLockRead                  ffffff802aed68d0 ffffff802af5f8a0 1000000
        -> IORWLockUnlock                ffffff802aed68d0 2 1000000
        -> udp_lock                      struct socket so, 0 ffffff80200ef410
        -> cfil_sock_data_in             struct socket so, struct sockaddr *from, ,struct mbuf *packet_lis
        -> sbconcat_mbufs                struct sockbuf *sb struct sockaddr *from, ,struct mbuf *packet_li
          -> cfil_sock_data_space        struct sockbuf *sb struct sockaddr *from, ,struct mbuf *packet_li
        <- sbconcat_mbufs                207
        -> sbappendchain                 struct sockbuf *sb ffffff80b4152600 0
          -> sballoc                     struct sockbuf *sb ffffff80b4152600 0
          -> postevent                   0 struct sockbuf *sb 300
          <- postevent                   419 ffffff80350462d0 0
        <- sbappendchain                 122
      <- sbappendaddr                    234
      -> sowakeup                        struct socket so, struct sockbuf *sb 0
        -> selwakeup                     ffffff80350462f0 struct sockbuf *sb 0
        <- waitq_wakeup64_all            d8
      <- sowakeup                        339
    <- udp_input                         14a1
  <- ip_input
```

The transport layer protocols perform the decapsulation of the data, with the usual checksumming, etc, and eventually the data gets `sbappend()`ed to the socket's `so_rcv` buffers. This allows socket filters(`sflt_data_in()`) and content filters (`cfil_sock_data_in()` to be called, and per the usual semantics either allow (0), claim (`EJUSTRETURN`) or force a drop (otherwise) of the packet, in which case in never gets to be seen by the socket owner.

Firewalling & Filtering mechanisms

Network borne data might be sensitive, unauthorized or malicious. It is therefore important to provide filterting capabilities, to allow security mechanisms to be notified of and inspect connection state and traffic. XNU offers filtering mechanisms at all layers, and now that we've considered the regular flow of packets up and down the stack, we can turn our attention to the numerous filtering mechanisms (the user-mode accessible ones were described in I/16), and how they integrate in that flow.

Table 14-23: A comparison of XNU's available filtering techniques

Mechanism	Layer	Technique	Mode	Parallels in other OSes
Socket filters	Session (V)	API hooks	Kernel	Winsock SPI, Linux socket hooks
Content filters		Packet inspection	User	Proprietary
pf	Transport (IV)	Firewall	User	BSD, Linux: IPTables
IP filters	Network (III)		Kernel	Linux: NetFilter
Interface Filters	Data Link (II)			Linux: BRTables
BPF		Packet inspection	User	BSD, Linux

Socket Filters

Socket filters offer kernel mode code the ability to hook socket lifecycle calls. The interested client sets up a `struct sflt_filter`, populating it with callbacks and a 32-bit handle, and then uses `sflt_register()` (from bsd/sys/sys/kpi_socketfilter.h) to install the filter. Two MacOS kexts using this facility are webcontentfilter.kext and the Application Layer Firewall (ALF.kext). Listing 14-24 shows an example of socket filter registration, from the decompiled code of ALF.kext. Note the use of different `sf_handle` identifiers as different filters are registered for the protocols. Those handles are then used when ALF is unloaded, as `sockwall_sflt_stop()` calls `sflt_unregister()` for each filter handle.

Listing 14-24: Decompiling ALF.kext's socket filter registration

```
#define ALF_ID          "com.apple.nke.applicationfirewall"

int sockwall_sflt_start(void) {

  struct sflt_filter ALFilter;

  bzero (&ALFilter, sizeof(ALFilter));                    /* 0xc0       */
  ALFilter.sf_handle = 0x575647066;                       /* (%rbx)     */
  ALFilter.sf_name = ALF_ID;                              /*  0x8(%rbx) */
  ALfilter.sf_unregistered = sockwall_sflt_unregistered;  /* 0x10(%rbx) */
  ALfilter.sf_attach = sockwall_sflt_attach;              /* 0x18(%rbx) */
  ALfilter.sf_detach = sockwall_sflt_detach;              /* 0x20(%rbx) */
  ALfilter.sf_connect_in = sockwall_sflt_connect_in;      /* 0x50(%rbx) */
  ALfilter.sf_connect_out = sockwall_sflt_connect_out;    /* 0x58(%rbx) */
  ALfilter.sf_bind = sockwall_sflt_bind;                  /* 0x60(%rbx) */
  ALfilter.sf_setoption = sockwall_sflt_setopt;           /* 0x68(%rbx) */

  ALFilter.sf_len = sizeof(ALFilter); /* Unnecessary and wrong, but no effect */

  if (OSBitOrAtomic(0x2, installed) != 2) { /* 38a0 */
      ...
  }
  ALfilter.sf_data_in = sockwall_sflt_data_in;     /* -0xc8(%rbp) */
  ALfilter.sf_data_out = sockwall_sflt_data_out;   /* -0xc8(%rbp) */


  sflt_register_internal(&ALFilter, AF_INET, SOCK_DGRAM, IPPROTO_UDP);

  /* TCP sockets may also listen(), so install another callback */
  ALFilter.sf_listen = sockwall_sflt_listen;
  ALFilter.sf_handle = 0x74637066;
  sflt_register_internal(&ALFilter, AF_INET, SOCK_STREAM, IPPROTO_TCP);
  ..
  ALFilter.sf_handle = 0x74637036;
  sflt_register_internal(&ALFilter, AF_INET6, SOCK_STREAM, IPPROTO_TCP);

  ....
  kev_vendor_code_find(ALF_ID, &myvendorID);
}
```

A socket filter can be registered with `sf_flags` set to `SFLT_GLOBAL` (0x1), to apply to all new sockets, or `SFLT_PROG` (0x2) which means the filter must be opted into (by calling `setsockopt(2)` with `SO_NKE`. To allow extensibility, the `SFLT_EXTENDED` (0x4) indicates the structure has extended fields in the `sf_len` bytes which follow it. Lastly, the `SFLT_EXTENDED_REGISTRY` (0x8) allows retroactively attaching to existing sockets.

Callbacks then get invoked from socket code in bsd/kern/uipc_socket[2].c. An example of this was `sflt_data_in()` being called from the `sbappend*()` variants on `mbuf` reception by a socket, as described earlier. A socket filter callback returns 0 if it agrees with the packet, or non-zero if the packet is to be discarded. The special code `EJUSTRETURN` is reserved for silently "swallowing" the packet, telling the `sbappend*()` variants to `m_freem` to `mbuf`. The `sf_attach` callback may set a `cookie` value, which subsequent callbacks will receive.

Looking through the `struct socket` definition, there is a `struct socket_filter_entry` pointer, called `so_filt`. The entry is a private structure (from bsd/kern/kpi_socketfilter.c), which provides linkage with other filter entries, the `sf_attach` cookie, a ref count, and a pointer to the socket. All installed socket filters are maintained in a `socket_filter_list`, a `TAILQ_HEAD` of `socket_filter_entry` objects:

Listing 14-25: The `socket_filter[_entry]` structure s(from bsd/netinet/kpi_socketfilter.c)

```
struct socket_filter_entry {
        struct socket_filter_entry      *sfe_next_onsocket;
        struct socket_filter_entry      *sfe_next_onfilter;
        struct socket_filter_entry      *sfe_next_oncleanup;

        struct socket_filter            *sfe_filter;
        struct socket                   *sfe_socket;
        void                            *sfe_cookie;

        uint32_t                        sfe_flags;
        int32_t                         sfe_refcount;
};

struct socket_filter {
        TAILQ_ENTRY(socket_filter)      sf_protosw_next;
        TAILQ_ENTRY(socket_filter)      sf_global_next;
        struct socket_filter_entry      *sf_entry_head;

        struct protosw                  *sf_proto;
        struct sflt_filter              sf_filter;
        u_int32_t                       sf_refcount;
};

TAILQ_HEAD(socket_filter_list, socket_filter);
```

An open source example can be found in Apple's "logNKE" code sample. The sample has apparently been removed as of late, but was fortunately salvaged and mirrored on GitHub by @OSXReverser[2].

Content Filters (Darwin 14+)

Content filters bear some similarities to socket filters - `#if CONTENT_FILTER` (which is set by default), the `sbappend*()` variants call `cfil_sock_data_in()` with the `mbuf` right after the call `sflt_data_in()`, and expect the same semantics (0 if ok, `EJUSTRETURN` to drop). The implementation, however, is different: whereas socket filters keep their decision logic in the kernel (or, with the help of a dedicated kernel extension like ALF.kext, enable user mode control), content filters are designed to work with user mode clients. A detailed discussion of using content filters from a user mode client was discussed in I/16, which is also available online[3]

Rather than allow callback hooks the way socket filters do, content filters allow a user mode client to request notifications of socket activity, and perform content inspection. The `CFM_OP_SOCKET_[ATTACHED/DETACHED]` messages provide the former, and the `CFM_OP_DATA_[IN/OUT]` messages, along with the copies of the socket `mbuf` provided to the user-mode clients, allow the latter.

Content filters are presently limited to TCP sockets, and are attached to sockets when they become active - i.e. by calls from `tcp[6]_usr_connect()` and `..._accept()`, which trigger `CFM_OP_SOCKET_ATTACHED` events. Similarly, `CFP_OP_SOCKET_CLOSED` are triggered by `soclose_locked()` and `sodealloc()`. The `CFM_OP_DATA_[IN/OUT]` are triggered as a result of `cfil_sock_data_[in/out]`, from the `sbappend*()` variants, as the `mbuf()` is readied to be added to the `sockbuf`, and from `sosend[_list()`. If a socket filter is not active on the socket, these functions silently return.

The content filter control socket allows `getsockopt(2)` with two codes. The first, `CFIL_OPT_GET_SOCKET_INFO`, will retrieve the socket information associated with the filter. The second, `CFIL_OPT_NECP_CONTROL_UNIT` retrieves the NECP control unit which provides the mapping of the network flow to the socket filter. The control unit can be set using `setsockopt(2)`. The user mode caller is expected to do so before activating the filter. Packet flows in the kernel undergo NECP matching anyway, so this is beneficial. Once a socket has a content filter attached, its `so_flags` indicate `SOF_CONTENT_FILTER` and the `so_cfil` holds the content filter information structure. The `cfil_info` is a large structure, allocated from the dedicated `cfil_info` zone.

IP Filters

IP filters allow an interested party to intercept all ingress and egress IP-based traffic. Traffic is intercepted on all interfaces, but since the IPv4 and IPv6 stacks are separate flows, the filters must be explicitly added to either or both, using `ipf_addv[4/6]()`. Listing 14-26 shows the definition of `ipf_filter` structures, from bsd/netinet/kpi_ipfilter.h:

Listing 14-26: The `ipf_filter` structure (from bsd/netinet/kpi_ipfilter.h)

```
/*!
        @typedef ipf_filter
        @discussion This structure is used to define an IP filter for
                use with the ipf_addv4 or ipf_addv6 function.
        @field cookie A kext defined cookie that will be passed to all
                filter functions.
        @field name A filter name used for debugging purposes.
        @field ipf_input The filter function to handle inbound packets.
        @field ipf_output The filter function to handle outbound packets.
        @field ipf_detach The filter function to notify of a detach.
*/
struct ipf_filter {
   void            *cookie;
   const char      *name;
   ipf_input_func  ipf_input;  // (void *cookie, mbuf_t *data, int offset, u_int8_t protocol);
   ipf_output_func ipf_output; // (void *cookie, mbuf_t *data, ipf_pktopts_t options);
   ipf_detach_func ipf_detach; // (void *cookie);
};

extern errno_t ipf_addv4(const struct ipf_filter *filter,
    ipfilter_t *filter_ref);

extern errno_t ipf_remove(ipfilter_t filter_ref);
```

The semantics of the return code for IP Filters is the same as for the other filters we have seen. Output filters, though, may sometimes opt to modify the outgoing packets. When that is the case, the `ipf_output_func` can call `ipf_inject_output()`, and return `EJUSTRETURN`. This preserves the reference counts on the underlying packet.

IP Filters remain unused (at least, officially) in both MacOS and *OS - which is why calls do not appear in Listings 14-21 and 14-22. Their use has been detailed in Snare's 2011 presentation[4].

PF

The BSD PF facility (packet filter, not to be confused with BPF) allows user mode to set packet filtering rules using various `DIOC_* ioctl(2)` codes and the `pfctl(8)` command, somewhat resembling Linux's IPTables. Apple also uses the private and a dedicated daemon, `pfd(8)`. The user mode aspects of this facility were described in I/16 (particularly, I/16-32).

In kernel, the rules are enforced when both the input and out paths of IPv4 and IPv6 call out to `pf_af_hook()` at strategic locations during the packet processing path. The `af_hook()` is declared in bsd/net/pfvar.h like so:

Listing 14-27: The `pf_af_hook()` and its arguments (from bsd/net/pfvar.h)

```
int pf_af_hook(struct ifnet *if,     // interface
               struct mbuf **mppn,   // double pointer to the packet mbuf
               struct mbuf **mp,     //
               unsigned int  af,     // Address family (AF_INET or AF_INET6)
               int           input,  //
               struct ip_fw_args *); // from bsd/netinet/ip_flowid.h
```

Because the two address families are different, `pf_af_hook()` splits to either `pf_inet[6]_hook()`, passing them the interface, `mbuf` pointer, `input` indicator, and the `ip_fw_args` pointer. The `ip_fw_args` pointer refers to a structure containing metadata, including the PF rules. Rule evaluation is performed by `pf_test_mbuf()`, which returns `PF_PASS` if the frame is allowed.

There is an additional, lower level hook in the `pf_ifnet_hook()`. This, however, is not open directly for third parties. Rather, it is called from DLIL's `ifnet_attach()`, and allows the PF facility to attach PF rules and filters to the new interface.

Interface Filters

Interface filters offer a way to attach at the data link layer, and intercept all egress and ingress frames, as well as interface operations. Unlike IP-filters, all frames of all protocol types are captured - but only on the specific interface to which the filter is attached. The interface filters are defined in the bsd/net/kpi_interfacefilter.h with a well annotated KPI, consisting of an `iff_filter` object, and five callback functions, very similar to IP filters:

Listing 14-28: The `struct iff_filter` definition, from bsd/net/kpi_interfacefilter.h

```
/*!
   @struct iff_filter
   @discussion This structure is used to define an interface filter for
           use with the iflt_attach function.
   @field iff_cookie A kext defined cookie that will be passed to all
           filter functions.
   @field iff_name A filter name used for debugging purposes.
   @field iff_protocol The protocol of the packets this filter is
           interested in. If you specify zero, packets from all protocols
           will be passed to the filter.
   @field iff_input The filter function to handle inbound packets, may be NULL.
   @field iff_output The filter function to handle outbound packets, may be NULL.
   @field iff_event The filter function to handle interface events, may be null.
   @field iff_ioctl The filter function to handle interface ioctls, may be null.
   @field iff_detached The filter function used to notify the filter that
           it has been detached.
*/

struct iff_filter {
   void                *iff_cookie;
   const char          *iff_name;
   protocol_family_t   iff_protocol;
   iff_input_func      iff_input;    // .. mbuf_t *data, char **frame_ptr);
   iff_output_func     iff_output;   // .. mbuf_t *data);
   iff_event_func      iff_event;    // .. const struct kev_msg *event_msg);
   iff_ioctl_func      iff_ioctl;    // ..unsigned long ioctl_cmd, void *ioctl_arg);
   iff_detached_func   iff_detached; // (void *cookie, ifnet_t interface);
};

extern errno_t iflt_attach(ifnet_t interface, const struct iff_filter *filter,
    interface_filter_t *filter_ref);

extern void iflt_detach(interface_filter_t filter_ref);
```

All callbacks (with the exception of `iff_detached`) take the same three parameters - the filter's `cookie`, the `ifnet_t` to which this callback pertains, and the filter's `iff_protocol`. These values are set when the `struct iff_filter` is created, and attached with a call to `iflt_attach()`.

Fairly few Apple supplied kexts use interface filters: PlugIns of MacOS's IOAVB* provide a few examples, and IOTimeSyncFamily.kext's IOgPTPPlugin.kext (in both MacOS and *OS) is another. Taking it an example:

Listing 14-29: An example of setting an interface filter (from IOgPTPPlugin.kext)

```
IOTimeSyncPortManager::start(IOService*):

   struct iff_filter filt;
000000000000bde4        leaq    -0x70(%rbp), %r14
000000000000bde8        movq    %r12, (%r14)
    filt.iff_cookie = ....;
..
    filt.iff_name =  OSMetaClass::getClassName(); //  0x8(%r14)
    filt.iff_filter = 0;  // 0x10(%r14)
    filt.iff_input = filt.iff_output =  NULL;
000000000000be09        xorl    %eax, %eax
000000000000be0b        movq    %rax, 0x18(%r14)
000000000000be0f        movq    %rax, 0x20(%r14)
    filt.iff_event = IOTimeSyncPortManager::filterEventCallback(void*, __ifnet*, unsigned int, k
000000000000be13        leaq    __ZN21IOTimeSyncPortManager19filterEventCallbackEPvP7__ifnetjPK7
000000000000be1a        movq    %rcx, 0x28(%r14)
    filt.iff_ioctl = NULL;
000000000000be1e        movq    %rax, 0x30(%r14)
    filt.iff_detached = IOTimeSyncPortManager::filterDetachedCallback(void*, __ifnet*);
000000000000be22        leaq    __ZN21IOTimeSyncPortManager22filterDetachedCallbackEPvP7__ifnet(
000000000000be29        movq    %rax, 0x38(%r14)
                ...
    iflt_attach_internal(interface, &filt, &ref); //  0xc8(%r12), %r14,  0xd0(%r12)
}
```

BPF

The Berkeley Packet Filter is a widely popular facility providing an in-kernel automaton. The original intent of this facilty was, as its name implies, to filter packets - but this doesn't stop Linux from integrating it with its seccomp facility to provide enhanced system call filtering.

Darwin's use of BPF, like BSD's, sticks to the original intent - providing user mode callers a character device (/dev/bpf##) through which they can bind to an interface and submit a filter program, made up of the BPF language, a limited set of opcodes with a register and memory inspection capabilities. An example of such a program can be found in I/16-34.

Interfaces normally have to opt-in to allow BPF to bind. Originally, `if_set_bpf_tap()` was used, but this has been deprecated in favor of two simpler functions: `bpf_tap_[in/out]()`. Both internally call `bpf_tap_mbuf()` (from bsd/net/bpf.c), flipping the sixth argument (`outbound`) accordingly.

The other five arguments `bpf_tap_mbuf()` accepts are the `ifnet_t`, a `DLT_*` (data link type) constant indicating the interface type, the `mbuf`, a pointer to the packet header, and the header len. Internally, the routine constructs a `bpf_packet`, which helps the BPF facility manage the packet metadata (e.g. summing up the `mbuf`s' `m_len` in an `mbuf` chain and keeping it in the `bpf_packet`'s `bpfp_total_length`).

When the `bpf_packet` is set up, it is handed to `bpf_tap_imp()`, which performs the interface `DLT_*` type matching, and , if successful, subjects the packet to `bpf_filter()`. This routine (from bsd/net/bpf_filter.c) provides the logic running the very efficient, BPF-opcode driven state machine. The installed filter, if any, has been provided a priori by the user with a `BIOCSETF ioctl(2)` call. Filters are subject to validation (using `bpf_validate()`) before they are allowed to run, which helps mitigate the potential for a malicious program breaking out of the interpreter environment and accessing arbitrary kernel memory. In the past however, there have been several notable BPF-related vulnerabilities which managed to achieve just that.

Interfaces still have to call `bpf_tap_[in/out]()`. Individual interfaces (e.g. utun, gif, vlan, etc) do so explicitly, but when they don't, BPF piggybacks on the pktap facility: The `pktap_[input/output]()` both wrap `pktap_bpf_tap()`, which in turn calls the `bpf_tap_func()` function pointer - either of `bpf_tap_[in/out]()`, based on the packet `outgoing` indication.

Network Extension Control Policies

Volume I/16 introduced **Network Extension Control Policies**, a powerful flow control facilty providing in-kernel session policies which serve as a substrate for more advanced functions, such as packet inspection, QoS, routing and Nexuses. Here, we consider the in-kernel implementation.

The header comment in bsd/net/necp.c provides a verbose explanation of the goals of NECP:

Listing 14-30: The NECP manifesto, from XNU-4903's bsd/net/necp.c

```
/*
* NECP - Network Extension Control Policy database
* ------------------------------------------------
* The goal of this module is to allow clients connecting via a
* kernel control socket to create high-level policy sessions, which
* are ingested into low-level kernel policies that control and tag
* traffic at the application, socket, and IP layers.
*
* ------------------------------------------------
* Sessions
* ------------------------------------------------
* Each session owns a list of session policies, each of which can
* specify any combination of conditions and a single result. Each
* session also has a priority level (such as High, Default, or Low)
* which is requested by the client. Based on the requested level,
* a session order value is assigned to the session, which will be used
* to sort kernel policies generated by the session. The session client
* can specify the sub-order for each policy it creates which will be
* used to further sort the kernel policies.
*
*  Kernel Control Socket --> 1 necp_session --> list of necp_session_policy structs
*
* ------------------------------------------------
* Kernel Policies
* ------------------------------------------------
* Whenever a session send the Apply command, its policies are ingested
* and generate kernel policies. There are two phases of kernel policy
* ingestion.
*
* 1. The session policy is parsed to create kernel policies at the socket
*       and IP layers, when applicable. For example, a policy that requires
*    all traffic from Appl to Pass will generate a socket kernel policy to
*    match Appl and mark packets with ID1, and also an IP policy to match
*    ID1 and let the packet pass. This is handled in necp_apply_policy. The
*    resulting kernel policies are added to the global socket and IP layer
*    policy lists.
*  necp_session_policy --> necp_kernel_socket_policy and necp_kernel_ip_output_policy
*                                  ||                              ||
*                                  \/                              \/
*                        necp_kernel_socket_policies   necp_kernel_ip_output_policies
*
* 2. Once the global lists of kernel policies have been filled out, each
*    list is traversed to create optimized sub-lists ("Maps") which are used during
*    data-path evaluation. IP policies are sent into necp_kernel_ip_output_policies_map,
*    which hashes incoming packets based on marked socket-layer policies, and removes
*    duplicate or overlapping policies. Socket policies are sent into two maps,
*    necp_kernel_socket_policies_map and necp_kernel_socket_policies_app_layer_map.
*    The app layer map is used for policy checks coming in from user space, and is one
*    list with duplicate and overlapping policies removed. The socket map hashes based
*    on app UUID, and removes duplicate and overlapping policies.
*  necp_kernel_socket_policy --> necp_kernel_socket_policies_app_layer_map
*                           |-> necp_kernel_socket_policies_map
*
*  necp_kernel_ip_output_policies --> necp_kernel_ip_output_policies_map
*
* ------------------------------------------------
* Drop All Level
* ------------------------------------------------
* The Drop All Level is a sysctl that controls the level at which policies are allowed
* to override a global drop rule. If the value is 0, no drop rule is applied. If the value
* is 1, all traffic is dropped. If the value is greater than 1, all kernel policies created
* by a session with a priority level better than (numerically less than) the
* Drop All Level will allow matching traffic to not be dropped. The Drop All Level is
* dynamically interpreted into necp_drop_all_order, which specifies the equivalent assigned
* session orders to be dropped.
*/
```

NECP file descriptors

NECP was initially designed to communicate with user mode over file descriptors. The `necp_open()` syscall (#501) (from bsd/net/necp_client.c creates this descriptor. Assuming the client holds the right entitlement and privileges, the syscall `falloc(s)` a file descriptor, and uses its `fg_data` to connect an instance of a `necp_fd_data` structure, `zalloc()`ed from the `necp.clientfd` zone. The descriptor's `fileglob`'s `fg_ops` are set to the `necp_fd_ops`, which also set the `fo_type` to `DTYPE_NPOLICY` (9). This is well documented in the header comment of bsd/net/necp_client.c:

Listing 14-31: NECP Client FD operation documentation (from bsd/net/necp_client.c)

```
/*
* NECP Client Architecture
* ------------------------------------------------
* See <net/necp.c> for a discussion on NECP database architecture.
*
* Each client of NECP provides a set of parameters for a connection or network state
* evaluation, on which NECP policy evaluation is run. This produces a policy result
* which can be accessed by the originating process, along with events for when policies
* results have changed.
*
* ------------------------------------------------
* NECP Client FD
* ------------------------------------------------
* A process opens an NECP file descriptor using necp_open(). This is a very simple
* file descriptor, upon which the process may do the following operations:
*   - necp_client_action(...), to add/remove/query clients
*   - kqueue, to watch for readable events
*   - close(), to close the client session and release all clients
*
* Client objects are allocated structures that hang off of the file descriptor. Each
* client contains:
*   - Client ID, a UUID that references the client across the system
*   - Parameters, a buffer of TLVs that describe the client's connection parameters,
*       such as the remote and local endpoints, interface requirements, etc.
*   - Result, a buffer of TLVs containing the current policy evaluation for the client.
*       This result will be updated whenever a network change occurs that impacts the
*       policy result for that client.
*
*                    +---------------+
*                    |    NECP fd    |
*                    +---------------+
*                            ||
*          ==================================================
*          ||                ||                ||
*   +--------------+  +--------------+  +--------------+
*   |   Client ID  |  |   Client ID  |  |   Client ID  |
*   |     ----     |  |     ----     |  |     ----     |
*   |  Parameters  |  |  Parameters  |  |  Parameters  |
*   |     ----     |  |     ----     |  |     ----     |
*   |    Result    |  |    Result    |  |    Result    |
*   +--------------+  +--------------+  +--------------+
*
* ------------------------------------------------
* Client Actions
* ------------------------------------------------
*   - Add. Input parameters as a buffer of TLVs, and output a client ID. Allocates a
*       new client structure on the file descriptor.
*   - Remove. Input a client ID. Removes a client structure from the file descriptor.
*   - Copy Parameters. Input a client ID, and output parameter TLVs.
*   - Copy Result. Input a client ID, and output result TLVs. Alternatively, input empty
*       client ID and get next unread client result.
*   - Copy List. List all client IDs.
*
* ------------------------------------------------
* Client Policy Evaluation
* ------------------------------------------------
* Policies are evaluated for clients upon client creation, and upon update events,
* which are network/agent/policy changes coalesced by a timer.
*
* The policy evaluation goes through the following steps:
*   1. Parse client parameters.
*   2. Select a scoped interface if applicable. This involves using require/prohibit
*      parameters, along with the local address, to select the most appropriate interface
*      if not explicitly set by the client parameters.
*   3. Run NECP application-level policy evalution
*   4. Set policy result into client result buffer.
*
*/
```

NECP Session FDs

A client has to be able to access the session policies, so in Darwin 16 NECP extends the file descriptor model, so as to export policies to user mode as file descriptors. A client calls `necp_session_open()` syscall (#522) to `necp_create_session()` in-kernel. Similar to `necp_open()`, this syscall also `falloc()`s a file descriptor, sets its `fileglob`'s ops to the `necp_session_fd_ops` (which also sets the descriptor type to `DTYPE_NETPOLICY`, as with the client fd), and the newly created `necp_session` is connected to the `fileglob`'s `fg_data`.

Thus, when the descriptor is accessed from user mode (using `necp_session_action()`, #523), `necp_session_find_from_fd()` routine ensures that the supplied file descriptor's `fileglob`'s `fg_ops->fo_type` indeed matches the `DTYPE`, and then casts the `fileglob`'s `fg_data` back to the `necp_session` structure pointer.

Another way to call `necp_create_session()` is via the `com.apple.net.necp_control PF_SYSTEM/SYSPROTO_CONTROL connect(2)`.

Policy evaluation

You may recall that, as we were going over the Network Data Processing paths in the kernel, NECP made an appearance:

- In the egress path:
  - TCP, UDP and Raw IP all call `necp_socket_is_allowed_to_send_recv[_v[46]` before data flow can ensue, in their respective `..._output()` functions. TCP does so in `tcp_connect_complete()` as well.
  - `ip_output_list()` (bsd/netinet/ip_output.c) calls `necp_ip_output_find_policy_match()`, providing the `mbuf`. If a policy can be matched, the packeted is marked (with `necp_mark_packet_from_ip()`), and the policy result is evaluated. The only acceptable policy evaluation results at this stage are `NECP_KERNEL_POLICY_RESULT_PASS` (allow) or `..IP_TUNNEL`). In both cases, however, there is an additional check for `necp_packet_is_allowed_over_interface()`. This routine takes the `mbuf` pointer and the target `ifnet_t`, looks up a routine rule for the `mbuf` using `necp_get_route_rule_id_from_packet()`, and - if found - checks `necp_route_is_allowed()`.
- In the ingress path:
  - Layer IV Protocols `necp_socket_is_allowed_to_send_recv[_v[46]` in their respective `..._input()` functions.

The policy lookup is performed using the `net_flowhash()` function (in bsd/net/flowhash.h), which is also used elsewhere in the stack (PF, TCP cache, IPv4 PCBs and DLIL). The hash is a MurmurHash3 for 64-bit systems, or JHash for 32-bit ones. Both hashes are taken from public domain sources, and implemented in bsd/net/flowhash.c.

Review Questions

1. Why did Apple move mbuf allocation to the separate submap of `mb_map`, rather than continue to use BSD/Mach zones?
2. Why are there multiple instances of the `SOCK_RAW` protocols in Table 14-13?
3. A comment in xnu-4903's bsd/netinet/tcp_usrreq.c reads: */* xxx - should be const */*. Indeed, the user request pointers of all protocols (or, at the very least, built-in ones) **must** (not just "should") be in const. Why?
4. Why is UDP served by two different `pr_usrreqs`, as opposed to TCP, which is served by only one?
5. What is the advantage of using the `input_mit_tcall` (per-interface thread call) over waking up the dedicated interface input thread directly?
6. What do NECP policies serve as a substrate for (outside the policy lookups mentioned in this chapter)? How else can this powerful infrastructure be used?
7. What is the fundamental issue with using both `necp_fd_data` and `necp_session` as the `fg_data` of NECP file descriptors? (hint: CVE-2018-4425)

References

1. nkepg - Apple Developer - "Network Kernel Extensions Programming Guide" - https://developer.apple.com/library/archive/documentation/Darwin/Conceptual/NKEConceptual/control/control.html
2. GDBInit GitHub - TCPLogNKE - https://github.com/gdbinit/tcplognke/
3. NewOSXBook.com - Volume I, Chapter 16 - Online at http://NewOSXBook.com/bonus/vol1ch16.html
4. Snare - "Defiling Mac OS X" - https://papers.put.as/papers/macosx/2011/Defiling-Mac-OS-X-Ruxcon.pdf

Post Scriptum

This is it. If you got here, that means you've either read through the entire book, or were curious and just skipped to the last page. But for me, this last page also marks where my exploration of Darwin ends. At this point, all three volumes are out, and order is restored as you can peruse the trilogy in the way it was meant to be - First Volume I (User Mode), then this work (Kernel), and finally Volume III (Security).

As with the other books, the bounty for any factual inaccuracies (not typos, not version-related changes) remains, at 0.01 BTC. Presently, this is about $85 (an improvement over the $30 low of earlier this year, but far from the $200 high of all times). Even though other issues aren't rewarded, you will have my thanks if you let me know of anything - preferably via the companion website forum (http://NewOSXBook.com/forum), where others can follow. I will still be recording errata in http://NewOSXBook.com/ChangeLog.html.

As long time readers know, I have continuosly kept updating the books while working on the trilogy: Volume III over time grew by six chapters (owing to six jailbreaks); Volume I got another jumbo chapter on Networking (v1.2) and an appendix of software images (v1.3). Both grew to well over 500 pages. The end of Volume II also marks the end of my continuous updates of the book.

It wasn't an easy decision to make, but I ended up making it. To be perfectly honest, I found myself exhausted by the process - hitting the glass ceiling while investigating Darwin, then Apple raising it on me ever so much with the updates/modifications of a new Darwin version, then hitting it again, every single year. Volume II was delayed long enough that I ended up covering Darwin 19, but for Darwin 20 someone else will have to pick up where I'm leaving off.

True to its namesake, Darwin will keep evolving. Apple will surely add, remove and modify key functionality over the next years (not to mention fix the flaws pointed out in this trilogy, both flat out and by insinuation). I'm willing to put good money that maybe as soon as Darwin 20 MacOS will once again transition - this time, to ARM64 (and just *maybe* lose that 10.x prefix!). If that happens, the two systems will be closer than ever (and `jtool2` will be vindicated for not supporting Intel assembly ;-). The core, either way, will remain mostly the same. Amit Singh's seminal "Mac OS X Internals" remained relevant long after it was published (2006), with some bits and pieces still relevant to this very day. I *truly* hope that this trilogy remains just as definitive and relevant as its precursor did, and - dare I hope - even longer.

Unless... Apple decides to totally scrap Darwin in favor of a new operating system?

And then...? Someone will rise to the challenge and write an Internals book about it!

So long, and thanks for all the fish :-)

Apple's growing family of operating systems is all driven by the same core - the kernel known as XNU. Although mostly open source, the codebase spans over 2,500 files and 1,500,000 lines of code, some of which date back over 30 years ago!

*OS Internals::Kernel Mode is the second volume of the series. Delving deeper than ever before into the most abstruse, complex and yet critical aspects of the kernel, many of them never before documented. With detailed illustrations and articulate explanations, taking on all facets of the kernel in Darwin 19 (MacOS 10.15/iOS 13) and earlier, this volume covers:

- The mechanics of kernel extensions - structure, loading and operation
- Low-level synchronization primitives
- Exception, trap and interrupt handling - in both Intel (MacOS) and ARM (*OS) architectures
- Lifecycle - from bootloader (iBoot/EFI) to shutdown ... or panic
- The BSD layer - processes, threads, file descriptors, and more
- The Virtual Filesystem Switch (VFS) and Apple's proprietary extensions
- The APFS filesystem - adding vital details to Apple's official, yet incomplete specification
- The Mach layers - Scheduling, IPC and memory management, including the zone allocator
- IOKit internals, including MacOS 10.15's new DriverKit
- The Networking stack, from sockets down to the Data Link Interface Layer

There is no knowledge, that is not power - and ultimate knowledge of the kernel yields control over the very fabric of reality for apps, daemons and other processes on the operating system powering millions of Macs, and billions of iDevices in the world today.

Formerly known as "Mac OS X and iOS Internals" - this brand new edition is more than a revision- It is a complete rewrite, expanding the tome into a trilogy. The approach this time is radically different: Hands-on, reverse engineering, and documenting the hitherto undocumented through detailed illustrations, listings and annotations. Not just a collection of details - but also the methodology used to research and uncover them.

As before, whether you get the book or not - the tools, code samples and other bonus materials - remain free at http://NewOSXBook.com!

Jonathan Levin is a longtime trainer and consultant specializing in the system and kernel levels of the "Big Three" - Windows, Linux and Mac OS X, as well as their mobile derivatives. He is the founder and CTO of Technologeeks.com (@Technologeeks) a partnership of experts offering training and consulting on system/kernel programming, debugging and more. He is also the author of "Android Internals" - A two volume series providing the definitive reference to the world's most popular mobile operating system